

Fundamentos Matemáticos para a Ciência da Computação

3ª Edição



Fundamentos Matemáticos para a Ciência da Computação

3ª Edição

Judith L Gersting

Indiana University — Purdue University at Indianapolis
University of Hawaii at Hilo

Tradução

Lúcio Leão Fialho

Pós-graduação em Engenharia de Sistemas e Computação pela UFRJ

Manoel Martins Filho

(Participação nos Capítulos 1 e 2)
Mestre em Engenharia de Sistemas e Computação
pela COPPE — UFRJ



EDITORA

Título do original em inglês
Mathematical Structures for Computer Science

First published in the United States by W. H. Freeman and Company,
New York, New York and Oxford Copyright, 1993. All rights
reserved.

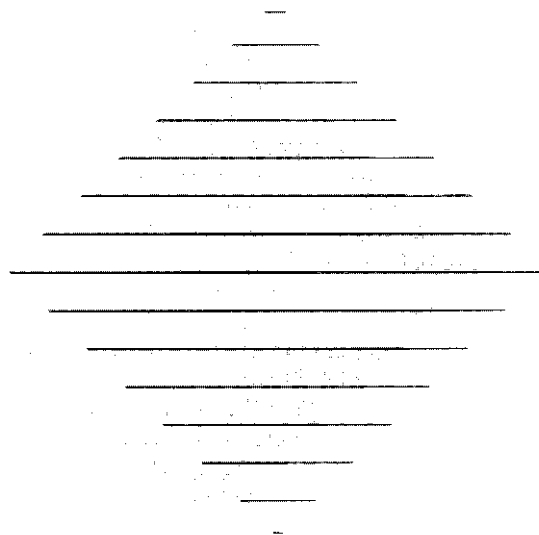
Primeira edição nos EUA por W. H. Freeman and Company, New
York, New York and Oxford Copyright, 1993. Reservados todos os
direitos.

Copyright © 1982, 1987, 1993 by W. H. Freeman and Company

Direitos exclusivos para a língua portuguesa Copyright © 1995 by
LTC - Livros Técnicos e Científicos Editora S.A.
Travessa do Ouvidor, 11
Rio de Janeiro, RJ — CEP 20040-040

Reservados todos os direitos. É proibida a duplicação ou reprodução
deste volume, no todo ou em parte, sob quaisquer formas ou por
quaisquer meios (eletrônico, mecânico, gravação, fotocópia ou
outros), sem permissão expressa da Editora.

Para Adam e Jason
dois fundamentos discretos adoráveis



Prefácio

A matemática discreta continua sendo um elemento importante no ensino da Ciência da Computação. Desejamos oferecer aos estudantes os instrumentos para que desenvolvam um vocabulário preciso, recursos para notação, abstrações úteis e raciocínio formal — e queremos fazê-lo logo, desde o início de sua formação. Explicações claras e exatas são mais importantes do que nunca. Portanto, foi mantido o estilo simples que caracterizou as versões anteriores. Em termos de conteúdo, as "mudanças mínimas" previstas para esta terceira edição acabaram evoluindo para o que resultou nesta revisão maior. Assim, a primeira metade da segunda edição foi bastante ampliada, enquanto na segunda metade houve uma boa redução. A ampliação da primeira metade do livro se dá de duas maneiras: pela introdução de novos tópicos e por um maior detalhamento na apresentação dos tópicos já existentes, como por exemplo os referentes à lógica formal. Mesmo o estudante sem base em cálculo vai achar o material acessível, especialmente com a ajuda dos exercícios práticos e os inúmeros exemplos e exercícios novos. Por outro lado, a fim de manter o livro compatível com um curso de um semestre, alguma coisa tinha que ser eliminada. Atualmente, a maioria dos alunos de graduação tem acesso, através de cursos de teoria de computação, à maior parte do material que figurava no trecho final da segunda edição; de modo que, por enquanto, uma introdução é o suficiente.

As mudanças com relação à segunda edição são mostradas a seguir com mais detalhes.

Capítulo 1: A Seção 1.1 da edição anterior, sobre sentenças e quantificadores, foi desmembrada em duas seções, 1.1 e 1.2. Incluímos exemplos de uso de tautologias para simplificar expressões condicionais de programas. Fizemos uma nova introdução aos predicados e quantificadores e uma caracterização mais formal do que são "wff" e "sentenças". Da mesma forma, a antiga Seção 1.2 sobre lógica proposicional e lógica predicada foi dividida em duas seções, 1.3 e 1.4, nas quais o leitor irá encontrar uma explicação intuitiva e adequada dos axiomas, que muito esclarece sobre o raciocínio envolvido nas demonstrações formais. Na Seção 1.5, o material sobre programação em lógica é totalmente novo e a seção sobre a verificação de programas foi ampliada.

Capítulo 2: A Seção 2.2 inclui uma discussão sobre indução completa, enquanto na edição anterior ela era apresentada apenas como exercício. A Seção 2.3 contém uma ampla discussão sobre definições recursivas — de seqüências, conjuntos, operações — e algoritmos recursivos, bem como um tratamento mais completo das técnicas para resolver relações de recorrência. A Seção 2.4 apresenta uma discussão explícita de algoritmos dividir-para-conquistar e de técnicas para resolver relações de recorrência. Também fornece uma descrição mais completa de invariantes de laços.

Capítulo 3: A Seção 3.1 examina o tipo de dados set das linguagens Pascal e SETL, e implementações possíveis para um tipo de dados abstrato "set" para conjuntos. A noção de "herança" da programação orientada para objeto se relaciona com a idéia de subconjuntos. A Seção 3.2 ilustra o uso de árvores de decisão como mecanismos de contagem. A Seção 3.3 é uma nova seção que trata do Princípio da Inclusão e Exclusão e do Princípio da Casa do Pombo. A Seção 3.4 mostra como contar o número de permutações de n objetos não necessariamente distintos e o número de combinações de r dentre n objetos com repetições. A Seção 3.5 usa o triângulo de Pascal (antes apresentado apenas como exercício) como uma introdução ao teorema binomial; também inclui uma demonstração completa do teorema binomial, que antes era tratado apenas como exercício e problema prático.

Capítulo 4: A Seção 4.1 fornece uma explicação melhor que a da segunda edição sobre como testar as várias propriedades de relações. São discutidos os fechos reflexivo, simétrico e transitivo. A Seção 4.2 é inteiramente nova (e opcional) e fornece uma introdução a bancos de dados, apresentando bancos de dados relacionais como conjuntos de relações matemáticas. As pesquisas são formuladas em termos das operações select, project e join, ou em termos da teoria de conjuntos ou da lógica de predicados. Diagramas PERT, caminhos críticos e ordenação topológica também são abordados pela Seção 4.2. A Seção 4.3, sobre funções, agora inclui uma breve referência a linguagens de programação e suas funções, e mostra como contar o número de funções, funções injetivas e funções sobrejetivas de um conjunto finito a outro. A contagem do número de permutações sem pontos fixos em um conjunto finito também é abordada. O tópico sobre ordem de grandeza de funções foi antecipado e recebeu novos exemplos e uma interpretação gráfica. A Seção 4.4, sobre matrizes, passou a incluir matrizes booleanas e multiplicação de matrizes booleanas.

Capítulo 5: A Seção 5.1 contém um novo material sobre isomorfismo de grafos, grafos planares e coloração de grafos (uma observação interessante: o material sobre o problema das quatro cores figurava na primeira edição, mas tinha sido excluído da segunda). A Seção 5.3 é nova. Ela começa abordando as relações entre grafos direcionados, matrizes booleanas e relações binárias, e depois fala da relação entre fecho transitivo e a alcançabilidade. E fornecido um algoritmo para calcular a matriz R de alcançabilidade através do cálculo de sucessivas potências da matriz de adjacências, além do algoritmo de Warshall e a análise de ambos os algoritmos. A Seção 5.4 também é nova e abrange árvores de decisão, árvores binárias de busca e códigos de Huffman.

Capítulo 6: A Seção 6.2 contém exercícios sobre o algoritmo de caminho mínimo de Bellman-Ford, o algoritmo de Kruskal para obtenção da árvore geradora mínima e o algoritmo de Floyd para obtenção do caminho mínimo entre cada par de vértices de um grafo. A Seção 6.3 introduz uma análise das buscas em profundidade e em largura e inclui também testes de alcançabilidade, ordenação topológica e identificação de componentes conexas. A nova Seção 6.4 (opcional) trata da determinação das articulações e componentes biconexas em grafos simples conexos.

Capítulo 7: O texto sobre álgebras booleanas, antes dividido em dois capítulos (5 e 6), ficou reunido em um único.

Capítulo 8: Este capítulo reúne parte do material que antes figurava nos capítulos 7 a 12. Eliminou-se muito do texto sobre teoria de grupos, da segunda edição, e o capítulo sobre teoria da codificação. Os dois capítulos sobre máquinas de estado finito foram bem reduzidos, assim como os dois capítulos sobre máquinas de Turing e linguagens formais.

Há uma relação de requisitos bem complexa entre os vários tópicos do livro. Essas dependências são apresentadas na tabela a seguir, permitindo a maior flexibilidade possível na escolha e na ordenação dos tópicos lidos.

SEÇÃO	SEÇÕES QUE SÃO PRÉ-REQUISITOS IMEDIATOS	SEÇÕES QUE SÃO PÓS-REQUISITOS IMEDIATOS
1.1	nenhuma	1.2
1.2	1.1	1.3, 1.5 Prova de correção, 2.1
1.3	1.2	1.4
1.4	1.3	1.5
1.5 Programação em lógica	1.4	Nenhuma
1.5 Prova de correção	1.2	2.4 Mais sobre prova de correção
2.1	1.2	2.2
2.2	2.1	2.3, 2.4 Mais sobre prova de correção, 3.1
2.3	2.2	2.4 Análise de algoritmos, 4.3
2.4 Análise de algoritmos	2.3	5.4 Árvores de decisão
2.4 Mais sobre prova de correção	1.5, 2.2	Nenhuma
3.1	2.2	3.2 Princípios da Multiplicação e Adição, 4.1
3.2 Princípios da Multiplicação e Adição	3.1	3.2 Árvores de decisão, 3.3 e 3.4

SEÇÃO	SEÇÕES QUE SÃO PRÉ-REQUISITOS IMEDIATOS	SEÇÕES QUE SÃO PÓS-REQUISITOS IMEDIATOS
3.2 Árvores de decisão	3.2 Princípios de Multiplicação e Adição	5.4 Árvores de decisão
3.3	3.2	4.3 Funções de contagem e permutações sem pontos fixos
3.4	3.2	3.5
3.5	3.4	Nenhuma
4.1	3.1	4.2 Relações e bancos de dados, 4.2 Ordenação topológica, 4.3
4.2 Relações e bancos de dados	4.1	Nenhuma
4.2 Ordenação topológica	4.1	6.3
4.3 (Exceto funções de contagem e permutações sem pontos fixos)	2.3, 4.1	4.3 Funções de contagem e permutações sem pontos fixos, 4.4, 5.1, 7.1
4.3 Funções de contagem e permutações sem pontos fixos	3.3, 4.3 (Exceto funções de contagem e permutações sem pontos fixos)	Nenhuma
4.4	4.3	5.2, 8.1
5.1	4.3	5.2, 5.4 Árvores de decisão, 5.4 Códigos de Huffman, 8.2
5.2	4.4, 5.1	5.3, 6.1, 6.2
5.3	5.2	6.3, 8.2
5.4 Árvores de decisão	2.4 Análise de algoritmos, 3.2 Árvores de decisão, 5.1	Nenhuma
5.4 Códigos de Huffman	5.1	Nenhuma
6.1	5.2	8.3
6.2	5.2	Nenhuma
6.3	4.2 Ordenação topológica, 5.3	6.4
6.4	6.3	Nenhuma
7.1	4.3	7.2, 8.1
7.2	7.1	7.3
7.3	7.2	Nenhuma
8.1	4.4, 7.1	Nenhuma
8.2	5.1, 5.3, Algoritmo de Warshall	8.3
8.3	6.1, 8.2	8.4
8.4	8.3	Nenhuma

Diversos recursos pedagógicos foram incluídos nesta edição:

Objetivos do capítulo

Uso de algoritmos em pseudocódigo semelhante à linguagem Pascal

Diversos novos exercícios, exemplos e problemas práticos

Autotestes para cada capítulo

Exercícios computacionais para cada capítulo

Apêndices sobre a notação de somatórios e a função logarítmica

As respostas de todos os exercícios práticos aparecem ao final do livro, bem como as respostas dos exercícios selecionados, que são destacados pelo símbolo ★.

O pseudocódigo usado no livro varia desde descrições bem informais até quase códigos Pascal, variação que deve ser levada em conta ao tentar codificar os algoritmos. Além disso, uma advertência com relação aos exercícios computacionais. A dificuldade deles varia muito de um exercício para outro. Alguns requerem maiores especificações ou decisões com relação às estruturas de dados adequadas. Não é especificada qual-

quer linguagem. O professor deve considerar a experiência em programação da turma antes de passar esses exercícios.

Os revisores desta edição forneceram muitos comentários de grande ajuda, que foram bastante apreciados. Agradecimentos a Hubert B. Enderton, University of California em Los Angeles; John G. Geske, Michigan State University; Maggie Johnson, Stanford University; Keith R. Pierce, University of Minnesota em Duluth; Mark Ramras, Northeastern University; Thomas W. Parsons, Hofstra University; Gregory J. E. Rawlins, Indiana University Bloomington; George Springer, Indiana University Bloomington.

Novamente, minha família deu grande apoio a meu trabalho que, desta vez, se estendeu por muitas milhas e muitos meses.

Judith L. Gersting

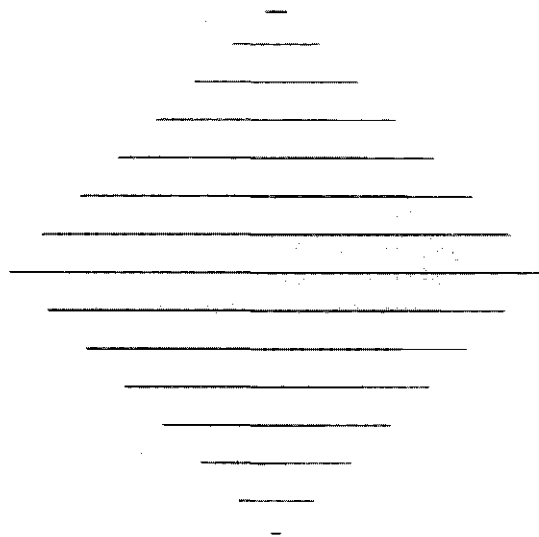
NOTA AOS ESTUDANTES

Este livro contém diversos recursos que o ajudarão a entender o texto, dentre os quais os mais importantes são os diversos problemas práticos. Esses problemas em geral não são difíceis, devendo ser resolvidos à medida que se evolui no livro. As respostas são apresentadas ao final do livro. O aprendizado será muito mais simples se você der a devida atenção a esses problemas no decorrer de sua leitura.

Judith L. Gersting

Sumário Resumido

Prefácio,	VII
Nota aos Estudantes,	X
1 Lógica Formal,	1
2 Demonstrações, Recursão e Análise de Algoritmo,	49
3 Conjuntos e Combinatória,	97
4 Relações, Funções e Matrizes,	150
5 Gratos e Árvores,	216
6 Grafos e Algoritmos,	278
7 Álgebra Booleana e Lógica de Computadores,	315
8 Modelagem Aritmética, Computação e Linguagens,	361
Apêndice A — Notação de Somatório,	432
Apêndice B — A Função Logarítmica,	435
Respostas dos Exercícios Práticos,	438
Respostas dos Exercícios Seleccionados,	465
Respostas dos Autotestes,	509
Índice Alfabético,	513



Sumário

1 Lógica Formal, 1

Objetivos do Capítulo, 1

1.1 Sentenças, Representação Simbólica e Tautologias, 2

Sentenças e conectivos lógicos; tabelas-verdade; fórmulas bem formadas; tautologias; uso de tautologias para simplificar condições lógicas em um programa; algoritmo para testar se algumas fórmulas bem formadas são tautologias.

Exercícios 1.1, 10

1.2 Quantificadores, Predicados e Validade, 12

Predicados; quantificadores universal e existencial; fórmulas bem formadas; interpretações para as fórmulas predicativas; representação de sentenças em português como fórmulas predicativas; negações de sentenças quantificadas; fórmulas válidas; comparação de wffs proposicionais e predicativas.

Exercícios 1.2, 17

1.3 Lógica Proposicional, 20

Sistemas formais; axiomas e regra de inferência para a lógica proposicional; o método dedutivo; exemplos de demonstrações na lógica proposicional; argumentos válidos.

Exercícios 1.3, 26

1.4 Lógica de Predicados, 27

Axiomas e regras de inferência para a lógica de predicados; justificativa da restrição e da generalização; exemplos de demonstrações na lógica de predicados; argumentos válidos.

Exercícios 1.4, 31

1.5 Programação Lógica e Prova de Correção, 33

Introdução à linguagem Prolog; fatos e regras Prolog; pesquisas; cláusulas de Horn e resolução; regras recursivas; sistemas especialistas; verificação de programas através de prova de correção e de testes de programas; asserções de programas; o axioma da atribuição; regra de inferência para comandos condicionais; exemplos de provas de correção.

Exercícios 1.5, 44

Revisão do Capítulo 1, 46

No Computador, 47

2 Demonstrações, Recursão e Análise de Algoritmo, 49

Objetivos do Capítulo, 49

2.1 Técnicas de Demonstração, 50

A natureza dos teoremas matemáticos e demonstrações; raciocínio indutivo e dedutivo; uso de contra-exemplos; técnicas para demonstração de teoremas, incluindo demonstração direta, demonstração por contraposição, demonstração por exaustão e demonstração por contradição.

Exercícios 2.1, 55

2.2 Indução, 56

O princípio da indução matemática; demonstrações por indução envolvendo séries, desigualdades, divisibilidade e diferentes passos básicos; princípio da indução completa; a relação entre a indução e o princípio da boa ordenação; exemplos de indução completa.

Exercícios 2.2, 64

2.3 Recursão e Relação de Recorrência, 67

Definições recursivas e suas relações com demonstrações por indução; definições recursivas para seqüências, conjuntos (especialmente conjuntos de cadeias), operações e algoritmos; comparação entre algoritmos iterativos e recursivos; um algoritmo recursivo de ordenação por seleção; o algoritmo de busca binária; introdução às relações de recorrência; método de solução para relações de recorrência lineares de primeira ordem com coeficientes constantes.

Exercícios 2.3, 78

2.4 Análise de Algoritmos e Mais Sobre Prova de Correção, 83

Introdução à análise de algoritmos; análise dos algoritmos de busca seqüencial e busca binária; método de solução para uma relação de recorrência do tipo dividir-para-conquistar; revisão de prova de correção; regra de inferência para laços; uso da indução para verificar invariantes de laços.

Exercícios 2.4, 90

Revisão do Capítulo 2, 94

No Computador, 95

3 Conjuntos e Combinatória, 97

Objetivos do Capítulo, 97

3.1 Conjuntos, 98

A notação da teoria dos conjuntos; subconjuntos e o conjunto das partes; operações binárias e unárias em um conjunto; operações de união, interseção, complemento e produto cartesiano; identidades de conjuntos e duas formas de prová-las; conjuntos nas linguagens de programação tais como Pascal e SETL; o tipo de dado abstrato para conjunto e possíveis implementações; herança na programação orientada a objeto vista sob o enfoque de subconjuntos; demonstração da denumerabilidade de alguns conjuntos e o uso do método de diagonalização de Cantor para demonstrar a não enumerabilidade.

Exercícios 3.1, 113

3.2 Contagem, 120

Princípios fundamentais da contagem, incluindo o Princípio da Multiplicação e o Princípio da Adição; problemas cujas soluções combinam estes princípios; problemas com diversas abordagens para solução; uso da árvore de decisão como ferramenta para realizar a contagem.

Exercícios 3.2, 125

3.3 Princípio da Inclusão e Exclusão e o Princípio da Casa do Pombo, 128

Demonstrações do Princípio da Inclusão e Exclusão para a união de dois ou três conjuntos; demonstração por indução da extensão da união de qualquer número finito de conjuntos; o Princípio da Casa do Pombo e exemplos de seu uso.

Exercícios 3.3, 131

3.4 Permutações e Combinações, 133

Fórmulas para contagem do número de permutações e combinações de r dentre n objetos distintos; uso das fórmulas juntamente com os Princípios da Multiplicação e Adição; discussão de soluções errôneas que contam alguns elementos mais de uma vez; fórmula para permutações distintas de n objetos que não sejam todos distintos; fórmulas para permutações e combinações de r dentre n objetos distintos onde são permitidas repetições.

Exercícios 3.4, 138

3.5 O Polinômio Binomial, 142

Apresentação do triângulo de Pascal; demonstração da fórmula de Pascal, tanto algébrica quanto combinatória; o teorema binomial e sua demonstração, tanto indutiva quanto combinatória; fórmula para encontrar uma parcela em especial na expansão binomial; uso do teorema binomial na demonstração de identidades.

Exercícios 3.5, 146

Revisão do Capítulo 3, 147

No Computador, 148

4 Relações, Funções e Matrizes, 150

Objetivos do Capítulo, 150

4.1 Relações, 152

Relações binárias descritas por conjuntos ordenados em pares e verbalmente; relações n -árias; operações em relações binárias; as propriedades reflexiva, simétrica, transitiva e antissimétrica de relações binárias; os fechos reflexivo, simétrico e transitivo de uma relação binária; definição de uma terminologia sobre ordenações parciais; grafos de conjuntos finitos parcialmente ordenados; definição de relação de equivalência e classes de equivalência; demonstração de que uma relação de equivalência determina uma partição, e de que uma partição determina uma relação de equivalência; exemplos de classes de equivalências tratadas como objetos; uso de classes de equivalências resultantes da congruência módulo n na aritmética inteira ou em um computador.

Exercícios 4.1, 162

4.2 Relações e Bancos de Dados e Ordenação Topológica, 168

Diagramas Entidade-Relacionamento como uma representação de um negócio; banco de dados relacionais como outro modelo cujas relações podem ser obtidas a partir de diagramas E-R; perspectiva de relações em um banco de dados como relações matemáticas; chaves primárias e chaves estrangeiras; operações select, project e join em relações; formulação de pesquisas na álgebra relacionai, SQL ou no cálculo relacionai; descrição breve de regras de integridade; diagramas PERT como representação de ordenações parciais para o término de uma tarefa; caminho crítico em um diagrama PERT; ordenação topológica para estender ordenações parciais a ordenações totais.

Exercícios 4.2, 178

4.3 Funções, 181

Definição e exemplos de funções; por que todas as três partes da definição são necessárias para descrever a função; linguagens de programação e funções, incluindo uma breve abordagem de linguagens funcionais; propriedades de funções — injetivas; sobrejetivas; bijetivas; fórmulas para contagem do número

de funções, funções injetivas e funções sobrejetivas de um conjunto finito em outro; conjuntos equivalentes e cardinalidade de conjuntos; composição de funções; funções de permutação, incluindo a notação de vetores e cíclica; composição de ciclos; fórmula para contar o número de funções de permutação sem pontos fixos; funções inversas; ordem de grandeza definida como uma relação de equivalência em funções; definição de $f = O(g)$ para duas funções f e g ; ilustração gráfica de ordem de grandeza.
Exercícios 4.3, 198

4.4 Matrizes, 204

Terminologia de matrizes; definição das operações de multiplicação escalar, adição, subtração e multiplicação; propriedades de matrizes sob várias operações; matriz nula e matriz identidade; análise do algoritmo padrão para multiplicação de matrizes; matrizes booleanas; operações e e ou booleanas; multiplicação booleana de matrizes.
Exercícios 4.4, 210

Revisão do Capítulo 4, 213

No Computador, 214

5 Grafos e Árvores, 216

Objetivos do Capítulo, 216

5.1 Terminologia e Aplicações de Grafos, 217

Terminologia de grafos; grafos isomorfos; grafos simples completos e grafos bipartites completos; grafos planares; prova da fórmula de Euler e outras relações em grafos conexos simples planares; exemplo da aplicação do teorema de Kuratowski; discussão do problema das quatro cores e do número cromático de um grafo; demonstração do problema das cinco cores; terminologia de árvores e grafos direcionados; algumas aplicações representadas por grafos.
Exercícios 5.1, 235

5.2 Representações Computacionais de Grafos, 243

Representação de um grafo através de sua definição; representação através de matrizes de adjacências; representação através de lista de adjacências; implementação de listas de adjacências através de vetores e ponteiros; representação de árvores binárias através de seus filhos à esquerda e à direita.
Exercícios 5.2, 248

5.3 Grafos Direcionados e Relações Binárias, e Algoritmo de Warshall, 253

Discussão da equivalência entre um grafo direcionado sem arestas paralelas, uma matriz booleana de adjacências de um grafo e uma relação binária de adjacência entre os vértices de um grafo; propriedades de uma relação binária refletida no grafo e na matriz de adjacências associados; operações matriciais para calcular a soma e o produto de relações binárias; demonstração de que as potências de matrizes booleanas indicam a presença de caminhos de diversos comprimentos; uma fórmula para calcular a matriz de alcançabilidade R para um grafo a partir de potências sucessivas da matriz de adjacências; relação entre alcançabilidade e fecho transitivo; análise do algoritmo para cálculo de R ; algoritmo de Warshall para cálculo de R e sua análise.
Exercícios 5.3, 260

5.4 Árvores de Decisão e Códigos de Huffman, 263

Definição de árvore de decisão; representação das buscas sequencial e binária através de árvore de decisão; limite inferior para o pior caso da busca estabelecida por uma árvore de decisão; busca em uma árvore binária; limite inferior para o pior caso para a ordenação estabelecida por uma árvore de decisão; algoritmos para obtenção de códigos de Huffman; justificativa da optimalidade dos códigos de Huffman.
Exercícios 5.4, 273

Revisão do Capítulo 5, 275

No Computador, 276

6 Grafos e Algoritmos, 278

Objetivos do Capítulo, 278

6.1 Caminho Euleriano e Ciclo Hamiltoniano, 279

Definição de um caminho euleriano; solução do problema do caminho euleriano; análise do algoritmo CaminhoEuleriano; discussão sobre o problema do ciclo hamiltoniano (e o problema do caixeiro viajante) e como eles diferem do problema do caminho euleriano.

Exercícios 6.1, 283

6.2 Caminho Mínimo e Árvore Geradora Mínima, 285

Algoritmo de Dijkstra para resolver o problema do caminho mínimo; análise do algoritmo CaminhoMínimo; discussão sobre o algoritmo de Prim para obtenção da árvore geradora mínima.

Exercícios 6.2, 290

6.3 Algoritmos de Busca, 294

Algoritmos para busca em profundidade e em largura; análise dos algoritmos de busca; árvores associadas às buscas em profundidade e em largura; aplicações das buscas em profundidade e em largura — alcançabilidade, identificação de componentes conexas; uso da busca em profundidade na ordenação topológica; algoritmos de busca em inordem, preordem e posordem.

Exercícios 6.3, 304

6.4 Articulações e Redes de Computadores, 307

Discussão sobre articulações e componentes biconexas em um grafo para redes de computadores e de comunicações; algoritmo para detecção de articulações em um grafo simples conexo.

Exercícios 6.4, 311

Revisão do Capítulo 6, 313

No Computador, 314

7 Álgebra Booleana e Lógica de Computadores, 315

Objetivos do Capítulo, 315

7.1 A Estrutura da Álgebra Booleana, 316

Semelhanças entre a lógica proposicional e a teoria dos conjuntos; estruturas matemáticas como modelos ou abstrações que incorporam propriedades comuns encontradas em diferentes contextos; definição de uma álgebra booleana; outras propriedades de uma álgebra booleana; discussão de estruturas isomorfas, em particular, estruturas com operações binárias; enunciado do teorema que afirma que todas as álgebras booleanas são isomorfas a uma álgebra de um conjunto de partes.

Exercícios 7.1, 324

7.2 Redes Lógicas, 328

Elementos lógicos básicos de portas E, portas OU e portas NÃO; representação de expressões booleanas como redes combinatórias e vice-versa; procedimento para encontrar uma expressão booleana na forma canônica de soma de produtos dada sua função verdade; minimização de expressões booleanas (e redes) usando as propriedades da álgebra booleana; arrays lógicos programáveis; um circuito somador binário; portas NE e NOU; construção de uma função verdade para representar um dispositivo de controle lógico.

Exercícios 7.2, 341

7.3 Minimização, 346

Uso de mapas de Karnaugh para minimizar expressões booleanas na forma de soma de produtos e suas redes lógicas associadas; uso de condições "não-importa"; uso do procedimento de Quine-

McCluskey para minimizar expressões booleanas na forma de soma de produtos e suas redes associadas.

Exercícios 7.3, 356

Revisão do Capítulo 7, 359

No Computador, 360

8 Modelagem Aritmética, Computação e Linguagens, 361

Objetivos do Capítulo, 361

8.1 Estruturas Algébricas, 362

Discussão sobre as propriedades associativa, comutativa, do elemento neutro e do elemento inverso; definição de grupo, monóide e semigrupo; exemplos simples da aritmética de números e matrizes; mais exemplos de polinômios, aritmética modular, transformações e permutações, cadeias; teoremas elementares de grupos — unicidade do elemento neutro e dos inversos, leis de cancelamento, solução de equações lineares; definição de subgrupo; grupo alternante; teorema de Lagrange; definição e exemplos de isomorfismo de grupos; teorema de Cayley.

Exercícios 8.1, 377

8.2 Máquinas de Estado Finito, 382

Definição de máquina de estado finito; tabelas de estado e grafos de estados; uma máquina de estado finito para somar números binários; máquinas de estado finito para aceitar conjuntos; conjuntos regulares; teorema de Kleene; estados inalcançáveis; um algoritmo para minimização de máquinas.

Exercícios 8.2, 394

8.3 Máquinas de Turing, 402

Discussão das limitações das máquinas de estado finito; definição de máquina de Turing e alguns exemplos; máquinas de Turing usadas para reconhecimento de conjuntos; máquinas de Turing usadas para calcular funções; discussão sobre a tese de Church-Turing, sua justificativa e suas consequências; a natureza de um problema de decisão; problemas de decisão clássicos; enunciado do problema da parada; uma variação trivial do problema da parada; demonstração da insolubilidade do problema da parada; a máquina de Turing como um meio para medição da complexidade computacional; definição do conjunto P e do conjunto NP; uma breve discussão sobre problemas NP-completos.

Exercícios 8.3, 415

8.4 Linguagens Formais, 418

Definição de gramáticas do tipo 0, geração de palavras e linguagens formais; descrição de uma linguagem gerada por uma dada gramática; gramáticas formais para gerar partes da língua portuguesa e de linguagens de programação; a hierarquia de gramáticas de Chomsky; gramáticas equivalentes para a mesma linguagem; a relação entre a hierarquia de linguagens formais e autômatos; parsing em linguagens livre de contexto.

Exercícios 8.4, 427

Revisão do Capítulo 8, 429

No Computador, 430

Apêndice A Notação de Somatório, 432

Apêndice B A Função Logarítmica, 435

Respostas dos Exercícios Práticos, 438

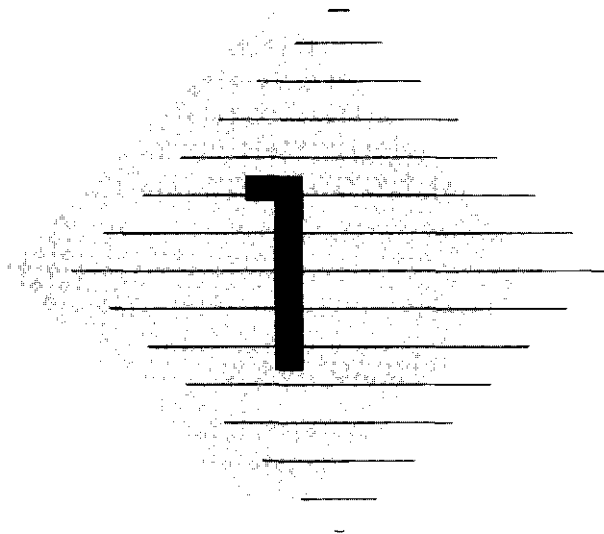
Respostas dos Exercícios Seleccionados, 465

Respostas dos Autotestes, 509

Índice Alfabético, 513

Fundamentos Matemáticos para a Ciência da Computação

3ª Edição



Lógica Formal

Objetivos do Capítulo

Após estudar este capítulo, o leitor deve ser capaz de:

- Reconhecer e trabalhar com os símbolos formais que são usados nas lógicas proposicional e de predicados
- Achar o valor-verdade de uma expressão na lógica proposicional
- Achar o valor-verdade de alguma interpretação de uma expressão na lógica de predicados
- Usar a lógica de predicados para representar sentenças da língua portuguesa
- Construir demonstrações formais nas lógicas proposicional e de predicados, e usá-las para determinar a validade de um argumento da língua portuguesa
- Entender como a linguagem de programação Prolog é constituída em função da lógica de predicados
- Provar matematicamente a correção de programas que usam comandos de atribuição e comandos condicionais

A exemplo de qualquer outra ciência, a Ciência da Computação depende da Matemática para obter um vocabulário preciso, uma notação poderosa, abstrações úteis e um raciocínio rigoroso. O objetivo deste livro é melhorar nosso entendimento da linguagem, das ferramentas e dos processos de raciocínio da Matemática que são usados na Ciência da Computação.

Este capítulo introduz a Lógica Formal, que delineia o método organizado e cuidadoso de pensar que caracteriza qualquer investigação científica ou qualquer outra atividade de raciocínio. Além disso, a

lógica formal tem aplicações diretas na Ciência da Computação. A última seção deste capítulo explora uma linguagem baseada na lógica e no uso da Lógica Formal objetivando verificar a correção de programas de computadores. Ainda, a lógica de circuitos (a lógica que rege os circuitos de computadores) é um análogo direto da lógica de sentenças deste capítulo. Estudaremos este tipo de lógica no Cap. 7.

Seção 1.1 Sentenças, Representação Simbólica e Tautologias

Geralmente nos expressamos, em português, através de interrogações e exclamações, mas, para comunicar fatos ou informações, usamos sentenças. Tecnicamente, uma **sentença** (ou **proposição**) é uma frase que pode ser apenas verdadeira ou falsa.

EXEMPLO 1 Considere o seguinte:

- Dez é menor do que sete.
- Como vai você?
- Ela é muito talentosa.
- Existem formas de vida em outros planetas do universo.

A frase (a) é uma sentença porque é falsa. Como o item (b) é uma pergunta, não pode ser considerado nem verdadeiro nem falso. Não tem valor-verdade e, portanto, não é uma sentença. Na frase (c) a palavra *ela* é uma variável e a frase não é verdadeira nem falsa, pois *ela* não está especificada; portanto, (c) não é uma sentença. A frase (d) é uma sentença porque é verdadeira ou falsa; independentemente de sermos capazes de decidir qual dos dois.

Conectivos e Valores-Verdade

Para enriquecermos nossas conversas não nos limitamos ao uso de simples sentenças. Ao contrário, as combinamos com o uso de conectivos a fim de criarmos sentenças compostas, cujo valor-verdade depende dos valores-verdade de cada sentença que o compõe e dos conectivos usados. Um conectivo comum é a palavra *e*. (Palavras como *mas* e *também* expressam diferentes significados, mas têm o mesmo efeito sobre os valores-verdade.) Se combinarmos as duas sentenças verdadeiras "Elefantes são grandes" e "Bolas são redondas" devemos considerar a sentença resultante "Elefantes são grandes e bolas são redondas" como verdadeira. Na lógica, usamos o símbolo $A \wedge B$ para denotar o conectivo lógico *e* e letras maiúsculas para denotar as sentenças (ao que chamaremos de símbolos proposicionais). Valores-verdade são atribuídos aos símbolos proposicionais. Concordamos, então, que se A e B são verdadeiras, $A \wedge B$ (leia-se " A e B ") deve ser considerada verdadeira.

- PRÁTICA 1**
- Se A é verdadeira e B é falsa, que valor você atribuiria a $A \wedge B$?
 - Se A é falsa e B é verdadeira, que valor você atribuiria a $A \wedge B$?
 - Se A e B são ambas falsas, que valor você atribuiria a $A \wedge B$?

(As respostas aos exercícios práticos estão no fim do livro.)

A expressão $A \wedge B$ é chamada a **conjunção** de A e B ; e A e B são chamados os **fatores** da expressão. Podemos resumir os efeitos das conjunções através do uso da **tabela-verdade** apresentada na Tabela 1.1. Em cada linha da tabela-verdade os valores-verdade são atribuídos aos símbolos proposicionais e o valor-verdade resultante da composição da expressão é, então, mostrado.

A	B	$A \wedge B$	A	B	$A \wedge B$
V	V	V	V	V	V
V	F	F	V	F	F
F	V	F	F	V	F
F	F	F	F	F	F

Tabela 1.1

Outro conectivo é a palavra *ou*, denotada pelo símbolo $\vee$. A expressão $A \vee B$ (leia-se "A ou B") é chamada **disjunção** de A e B e A e B são chamados de **parcelas** da expressão. Se A e B forem ambos verdadeiros, $A \vee B$ deverá ser considerada verdadeira, nos dando a primeira linha da tabela-verdade para a disjunção (veja a Tabela 1.2).

PRÁTICA 2 Use o que entendeu da palavra *ou* para completar a tabela-verdade para a disjunção, isto é, a Tabela 1.2 •

As sentenças podem ainda ser combinadas na forma "se sentença 1, então sentença 2". Se A denota a sentença 1 e B denota a sentença 2, a sentença composta deve ser denotada por $A \rightarrow B$ (leia-se "A implica B"). O conectivo lógico aqui é a **implicação** e indica que a verdade de A implica ou leva à verdade de B. Existem outras maneiras de expressar $A \rightarrow B$ na linguagem cotidiana, tal como "A é condição suficiente para B", "A somente se B", "B é consequência de A". Na expressão $A \rightarrow B$, A constitui a sentença **antecedente** e B a sentença **consequente**.

EXEMPLO 2 A sentença "Fogo é uma condição necessária para fumaça" pode ser reformulada como "Se há fumaça, então há fogo". O antecedente é "há fumaça", e o consequente é "há fogo". •

PRÁTICA 3 Indique o antecedente e o consequente em cada uma das seguintes sentenças. (Dica: reescreva as frases na forma se-então.) •

- Se a chuva continuar, o rio vai transbordar.
- Uma condição suficiente para a falha de uma rede é que a chave geral pare de funcionar.
- Os abacates só estão maduros quando estão escuros e macios.
- Uma boa dieta é uma condição necessária para um gato saudável.

A tabela-verdade para a implicação é menos óbvia do que para a conjunção e disjunção. Para compreendermos sua definição, vamos supor que seu colega de quarto diga "Se eu me formar nesta primavera, vou tirar férias na Flórida." Se ele, de fato, se formar na primavera e tirar suas férias na Flórida, a sentença foi verdadeira. Se A e B forem ambas verdadeiras, consideraremos a implicação $A \rightarrow B$ verdadeira. Se o seu colega se formar e não tirar as férias na Flórida, seu comentário consistiu em uma sentença falsa. Quando A é verdadeira e B é falsa, consideramos $A \rightarrow B$ falsa. Suponhamos agora que seu colega não se formou. Independentemente de ele tirar ou não férias na Flórida, não poderemos acusá-lo de ter formulado uma sentença falsa e lhe daremos o benefício da dúvida. Por convenção, aceitamos $A \rightarrow B$ como verdadeira se A for falsa, independentemente do valor-verdade de B.

PRÁTICA 4 Resuma a explanação acima, escrevendo a tabela-verdade de $A \rightarrow B$. •

A	B	$A \rightarrow B$	$B \rightarrow A$	$(A \rightarrow B) \wedge (B \rightarrow A)$
V	V	V	V	V
V	F	F	V	F
F	V	V	F	F
F	F	V	V	V

Tabela 1.3

Suponhamos que $A \rightarrow B$ seja verdadeira. Então, de acordo com a tabela-verdade da implicação, B pode ser verdadeira ou A pode ser falsa. Portanto, a despeito do fato de A ser verdadeira implicar que B também o seja, B ser verdadeira não implica que A o seja. A frase "S é uma condição suficiente para A" para denotar $A \rightarrow B$ apenas significa que, se A for verdadeira, B também o será.

O conectivo de **equivalência** é denotado pelo símbolo $\leftrightarrow$. A expressão $A \leftrightarrow B$ é a abreviação de $(A \rightarrow B) \wedge (B \rightarrow A)$. Podemos escrever a tabela-verdade para a equivalência construindo, parte por parte, a tabela para $(A \rightarrow B) \wedge (B \rightarrow A)$, como na Tabela 1.3. Perceba, nesta tabela, que $A \leftrightarrow B$ é verdadeira exatamente quando A e B têm o mesmo valor-verdade. A expressão $A \leftrightarrow B$ é normalmente lida como "A se, e somente se, B".

Os conectivos que vimos até agora são chamados de **conectivos binários** pois eles unem duas expressões a fim de produzir uma terceira. Vamos agora considerar um **conectivo unário**, isto é, um conectivo que atua em uma única expressão para produzir uma outra. A **negação** é um conectivo unário. A negação de A, A' é lida como "não A", "A é falsa" ou "A não é verdade". Isto não quer dizer que A' sempre tenha um valor-verdade falso, mas que o valor-verdade de A' é o contrário do de A. (Alguns textos denotam A' como $\neg A$.)

4 Lógica Formal

PRÁTICA 5 Escreva a tabela-verdade para A' . (Serão necessárias apenas duas linhas.) •

EXEMPLO 3 Se A é a sentença "Vai chover amanhã", a sentença A' é "Não é verdade que vai chover amanhã", que pode ser reescrita como "Não vai chover amanhã". •

Achar a negativa de uma sentença composta pode exigir algum esforço. Se P for a sentença "Peter é alto e magro", então a sentença P' será "É falso que Peter seja alto e magro", que pode ser reformulada como "Peter não é alto ou não é magro". Perceba que esta sentença *não* é a mesma que "Peter é baixo e gordo". Se P for a sentença "O rio é raso ou poluído", então P' é a sentença "É falso que o rio seja raso ou poluído", que pode ser reescrita como "O rio nem é raso nem é poluído" ou ainda "O rio é profundo e despoluído". No entanto, P' *não* é equivalente a "O rio não é raso ou não é poluído".

PRÁTICA 6 Qual das frases a seguir representa A' se A é a sentença "Julie adora manteiga mas detesta nata"?

- Julie detesta manteiga e nata.
 - Julie não gosta de manteiga ou nata.
 - Julie não gosta de manteiga mas adora nata.
 - Julie detesta manteiga ou adora nata.
-

Podemos encadear sentenças, seus conectivos e os parênteses (ou colchetes) para obtermos novas expressões, tal como em

$$(A \rightarrow B) \wedge (B \rightarrow A)$$

Naturalmente, como nas linguagens de programação, regem algumas *regras de sintaxe* (regras sob as quais as cadeias são válidas); por exemplo,

$$A)) \wedge \wedge \rightarrow BC$$

não deve ser considerada uma cadeia válida. Expressões que formam cadeias válidas são chamadas de **fórmulas bem-formuladas** ou **wffs** (de *well-formed formulas*). A fim de reduzir o número de parênteses necessários em uma wff, estipulamos uma ordem na qual os conectivos são aplicados. Esta "ordem de precedência" é:

- Conectivos dentro de parênteses, dos mais internos para os mais externos
- '
- $\wedge, \vee$
- $\rightarrow$
- $\leftrightarrow$

Isto significa que a expressão $A \vee B'$ significa $A \vee (B')$ e não $(A \vee B)'$. Analogamente, $A \vee B \rightarrow C$ significa $(A \vee B) \rightarrow C$ e não $A \vee (B \rightarrow C)$.

Wffs compostas de letras representativas de sentenças e conectivos têm valores-verdade que dependem dos valores-verdade atribuídos aos símbolos proposicionais. Escrevemos as tabelas-verdade para qualquer wff montando as partes que as compõem, como fizemos para $(A \rightarrow B) \wedge (B \rightarrow A)$.

EXEMPLO 4 A tabela-verdade para a wff $A \vee B' \rightarrow (A \vee B)'$ é dada na Tabela 1.4. •

A	B	B'	$A \vee B'$	$A \vee B$	$(A \vee B)'$	$A \vee B' \rightarrow (A \vee B)'$
V	V	F	V	V	F	F
V	F	V	V	V	F	F
F	V	F	F	V	F	V
F	F	V	V	F	V	V

Tabela 1.4

Se estivermos montando uma tabela-verdade para uma wff que contenha n símbolos proposicionais diferentes, quantas linhas terá a tabela? Das tabelas feitas até agora, sabemos que uma wff com apenas um símbolo proposicional tem duas linhas em sua tabela-verdade e uma wff com dois símbolos proposicionais tem quatro linhas. O número de linhas é igual ao número de combinações verdadeiro-falso possíveis entre as letras

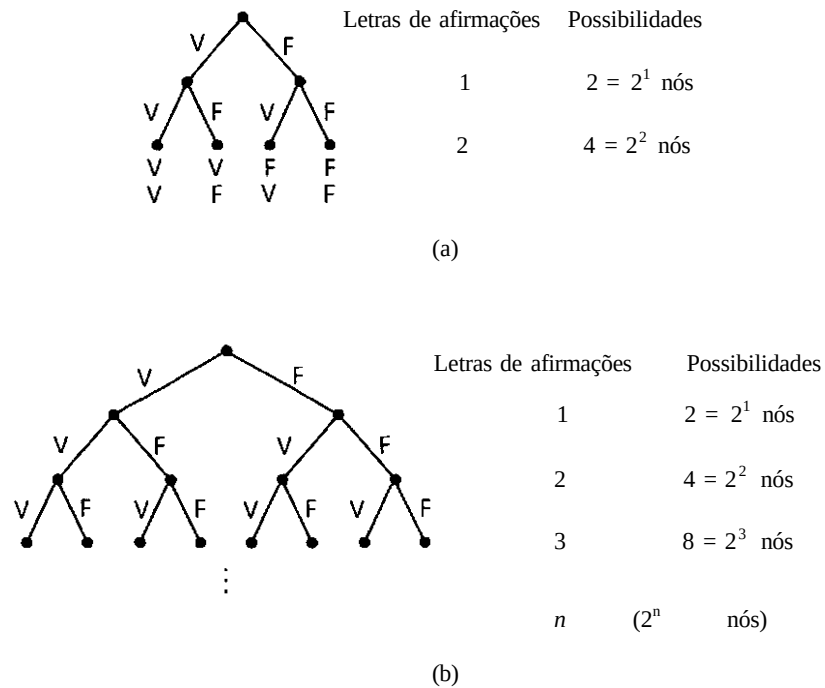


Figura 1.1

das combinações. O primeiro símbolo proposicional tem dois valores possíveis, V e F. Para cada um desses valores, o segundo símbolo proposicional também tem dois valores possíveis. A Fig. 1.1a ilustra isto na forma de uma "árvore" de dois níveis com quatro folhas, que mostram as quatro combinações possíveis de V e F para dois símbolos proposicionais. Para n símbolos proposicionais estendemos a árvore a n níveis, como na Fig. 1.1.b. O número total de folhas será igual, portanto, a 2^n . O número total de linhas em uma tabela-verdade para n símbolos proposicionais será também 2^n .

Essa estrutura de árvore nos mostra como enumerar todas as combinações V-F dentre os n símbolos proposicionais quando precisamos montar uma tabela-verdade. Se lermos cada nível da árvore de baixo para cima, veremos que os valores V-F para o símbolo proposicional n (a que compõe a última coluna da tabela-verdade) desdobra cada valor da linha $n - 1$ em dois valores e os da linha $n - 2$ em quatro valores e assim por diante. Portanto, uma tabela-verdade para três símbolos proposicionais começaria como mostrado na Tabela 1.5. Os valores para a sentença C variam, para os valores da sentença B em grupos de dois e para os valores da sentença A em grupos de quatro, resultando em uma versão horizontal de uma árvore. (Lendo as linhas de cima para baixo e usando 1 para os V e 0 para os F veremos que estamos apenas contando a partir de zero em binário.)

A	B	C
V	V	V
V	V	F
V	F	V
V	F	F
F	V	V
F	V	F
F	F	V
F	F	F

Tabela 1.5

PRÁTICA 7 Construa as tabelas-verdade para as seguintes wffs.

- $(A \rightarrow B) \leftrightarrow (B \rightarrow A)$ (lembre-se que $C \leftrightarrow D$ só é verdadeira quando C e D têm o mesmo valor-verdade)
- $(A \vee A') \rightarrow (B \wedge B')$
- $[(A \wedge B') \rightarrow C']$
- $(A \rightarrow B) \leftrightarrow (B' \rightarrow A')$

Tautologias

Uma wff como a (d) da Prática 7, cujos valores-verdade são sempre verdadeiros, é chamada uma **tautologia**. Uma wff como a do item (b), cujos valores-verdade são sempre falsos, é chamada uma **contradição**. Quando uma wff composta na forma $P \leftrightarrow Q$ é uma tautologia, como na Prática 7(d), os valores-verdade de P e Q conferem para todas as colunas da tabela-verdade. Neste caso, P e Q são chamadas de **wffs equivalentes** e denotadas por $P \leftrightarrow Q$. Desta forma, $P \leftrightarrow Q$ indica um fato, a saber: a wff particular $P \leftrightarrow Q$ é uma tautologia.

Vamos listar algumas equivalências básicas, provar uma ou duas delas através da construção das tabelas-verdade e deixar as demais como exercícios. Representaremos qualquer contradição por 0 e qualquer tautologia por 1.

Algumas Equivalências Tautológicas

1a. $A \vee B \leftrightarrow B \vee A$	1b. $A \wedge B \leftrightarrow B \wedge A$	(propriedades comutativas)
2a. $(A \vee B) \vee C \leftrightarrow A \vee (B \vee C)$	2b. $(A \wedge B) \wedge C \leftrightarrow A \wedge (B \wedge C)$	(propriedades associativas)
3a. $A \vee (B \wedge C) \leftrightarrow (A \vee B) \wedge (A \vee C)$	3b. $A \wedge (B \vee C) \leftrightarrow (A \wedge B) \vee (A \wedge C)$	(propriedades distributivas)
4a. $A \vee 0 \leftrightarrow A$	4b. $A \wedge 1 \leftrightarrow A$	(propriedades de identidade)
5a. $A \vee A' \leftrightarrow 1$	5b. $A \wedge A' \leftrightarrow 0$	(propriedades complementativas)

(Perceba que 2a nos leva a escrever $A \vee B \vee C$ sem a necessidade de parênteses; analogamente, 2b nos leva a escrever $A \wedge B \wedge C$.)

EXEMPLO 5

A tabela-verdade da Tabela 1.6 verifica a equivalência 1a, a propriedade comutativa da disjunção, e a da Tabela 1.7 verifica a 4b, a propriedade da identidade para a conjunção. Perceba que foram necessárias apenas duas linhas para a Tabela 1.7 porque 1 (uma tautologia) não pode assumir valores falsos.

A	B	$A \vee B$	$B \vee A$	$A \vee B \leftrightarrow B \vee A$
V	V	V	V	V
V	F	V	V	V
F	V	V	V	V
F	F	F	F	V

Tabela 1.6

A	1	$A \wedge 1$	$A \wedge 1 \leftrightarrow A$
V	V	V	V
F	V	F	V

Tabela 1.7

PRÁTICA 8

Verifique a equivalência 5a.

As equivalências em nossa lista são grupadas em cinco pares. Em cada par, uma equivalência pode ser obtida da outra substituindo-se os $\wedge$ por $\vee$, $\vee$ por $\wedge$, 0 por 1 e 1 por 0. Cada equivalência em um par é chamada a **dual** da outra. Esta lista de equivalência aparece de uma forma mais geral no Cap. 7.

Duas outras equivalências muito úteis são as **leis de De Morgan**, assim chamadas devido ao matemático inglês do século XIX Augustus De Morgan, que primeiro as formulou. Essas leis são:

$$(A \vee B)' \leftrightarrow A' \wedge B' \quad \text{e} \quad (A \wedge B)' \leftrightarrow A' \vee B'$$

Cada qual é a dual da outra. As leis de De Morgan nos ajudam a representar a negação de sentenças compostas, como na Prática 6.

Suponha que P e Q são equivalentes e que P aparece como uma componente de uma wff R grande. O que acontece se substituirmos Q por P ? Por mais que os valores-verdade mudem, não haverá qualquer alteração. Denotemos por R_Q a wff obtida pela substituição de P por Q em R . Se construirmos as tabelas-verdade para R e para R_Q , a cada linha os valores de Q e P coincidem; logo, a cada linha, os valores de R e R_Q também coincidem. Portanto, R_Q é equivalente à wff original R .

EXEMPLO 6

Seja $R (A \rightarrow B) \rightarrow B$ e seja $P A \rightarrow B$. Da Prática 7(d), sabemos que P é equivalente a $Q = B' \rightarrow A'$. Substituindo P por Q , obtemos $R_{Q'} = (B' \rightarrow A') \rightarrow B$. As tabelas-verdade para R e R_Q podem ser vistas nas Tabelas 1.8 e 1.9. Os valores-verdade de $A \rightarrow B$ e $B' \rightarrow A'$ coincidem para todas as linhas, portanto os valores-verdade de R e R_Q coincidem para todas as linhas. Logo R e R_Q são equivalentes.

A	B	$A \rightarrow B$	$(A \rightarrow B) \rightarrow B$
V	V	V	V
V	F	F	V
F	V	V	V
F	F	V	F

Tabela 1.8

A	B	A'	B'	$B' \rightarrow A'$	$(B' \rightarrow A') \rightarrow B$
V	V	F	F	V	V
V	F	F	V	F	V
F	V	V	F	V	V
F	F	V	V	V	F

Tabela 1.9

Equivalências Tautológicas		
Propriedades Comutativas:	$A \vee B \Leftrightarrow B \vee A$	$A \wedge B \Leftrightarrow B \wedge A$
Propriedades Associativas:	$(A \vee B) \vee C \Leftrightarrow A \vee (B \vee C)$	$(A \wedge B) \wedge C \Leftrightarrow A \wedge (B \wedge C)$
Propriedades Distributivas:	$A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$	$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$
Propriedades de Identidade:	$A \vee 0 \Leftrightarrow A$	$A \wedge 1 \Leftrightarrow A$
Propriedades Complementativas:	$A \vee A' \Leftrightarrow 1$	$A \wedge A' \Leftrightarrow 0$
Leis de De Morgan:	$(A \vee B)' \Leftrightarrow A' \wedge B'$	$(A \wedge B)' \Leftrightarrow A' \vee B'$
Propriedades Idempotentes:	$A \vee A \Leftrightarrow A$	$A \wedge A \Leftrightarrow A$
Dupla Negativa:	$(A')' \Leftrightarrow A$	
Reescrevendo a Implicação:	$(A \rightarrow B) \Leftrightarrow A' \vee B$	
Contraposição:	$(A \rightarrow B) \Leftrightarrow (B' \rightarrow A')$	
Prova Condicional	$A \rightarrow (B \rightarrow C) \Leftrightarrow (A \wedge B) \rightarrow C$	

Apesar de existirem infinitas tautologias, o quadro anterior resume as equivalências mais úteis (que expressam certas tautologias), incluindo aquelas que já havíamos listado anteriormente. Essas equivalências podem ser usadas, como vimos, para reescrever wffs. Como antes, 0 é qualquer contradição e 1 é qualquer tautologia.

Conectivos Lógicos e Programação

Os conectivos lógicos E, OU e NÃO (ou, mais comumente seus equivalentes em inglês AND, OR e NOT) são oferecidos pela maioria das linguagens de programação. Esses conectivos, de acordo com as tabelas-verdade que definimos, agem sobre combinações de expressões verdadeiras e falsas a fim de produzir um valor-verdade final. Desses valores provém a capacidade de tomada de decisão fundamental ao controle do fluxo de programas de computadores. Desta forma, em um desvio condicional de um programa, se o valor-verdade de uma determinada expressão for verdadeiro, o programa irá executar um trecho de seu código; se o valor for falso, o programa executa, em seguida, outro trecho de seu código. Se a expressão condicional for substituída por uma expressão mais simples equivalente, o valor-verdade da expressão e, portanto, o controle do fluxo do programa não serão afetados, mas o novo código torna-se mais simples de ser entendido e poderá ser executado mais rapidamente.

EXEMPLO 7

Vejamos o seguinte comando na linguagem de programação Pascal:

```
if (fluxoext > fluxoint)
    and not ((fluxoext > fluxoint) and (pressão < 1000)) then
    UmProcedimento(lista de parâmetros)
else
    OutroProcedimento(lista de parâmetros);
```

A expressão condicional aqui tem a seguinte forma

$$A \wedge (A \wedge B)'$$

onde A é $\text{fluxoext} > \text{fluxoint}$ e B é $\text{pressão} < 1000$. Esta expressão pode ser simplificada substituindo-se algumas subexpressões por suas expressões equivalentes.

$$\begin{aligned}
A \wedge (A \wedge B)' &\Leftrightarrow A \wedge (A' \vee B') && \text{(lei de De Morgan)} \\
&\Leftrightarrow (A \wedge A') \vee (A \wedge B') && \text{(propriedade distributiva)} \\
&\Leftrightarrow 0 \vee (A \wedge B') && \text{(propriedade complementativa)} \\
&\Leftrightarrow (A \wedge B') \vee 0 && \text{(propriedade comutativa)} \\
&\Leftrightarrow A \wedge B' && \text{(propriedade de identidade)}
\end{aligned}$$

O comando pode então ser reescrito como

```

if (fluxoext > fluxoint) and not (pressão < 1000) then
    UmProcedimento(lista de parâmetros)
else
    OutroProcedimento(lista de parâmetros);

```

Um Algoritmo

Se tivermos uma wff da forma $P \rightarrow Q$, onde P e Q são wffs compostas, podemos usar um procedimento mais rápido para construir uma tabela-verdade para determinar quando $P \rightarrow Q$ é uma tautologia. Assumimos que $P \rightarrow Q$ não é uma tautologia — isto é, que possa assumir valores falsos — e vejamos se isto nos leva a alguma situação impossível. $P \rightarrow Q$ é falsa apenas quando P é verdadeira e Q é falsa. Se atribuirmos verdadeira a P e falsa a Q , determinamos valores-verdade para os wffs reunindo P e Q . Continuamos atribuindo os valores-verdade até que todas as ocorrências de símbolos proposicionais tenham um valor-verdade. Se tivermos atribuído a alguma letra ambos os valores verdadeiro e falso, temos uma situação impossível, logo $P \rightarrow Q$ é uma tautologia. Caso contrário, temos uma maneira de tornar $P \rightarrow Q$ falso, então não é uma tautologia.

O que descrevemos foi um conjunto de instruções, um procedimento, para se encarregar da determinação de se $P \rightarrow Q$ é ou não uma tautologia. Este procedimento pode ser executado através da execução mecânica dessas instruções; em um período finito de tempo teremos a resposta. Em termos da ciência da computação, o procedimento é um *algoritmo*.

Definição: Algoritmo

Um **algoritmo** é um conjunto de instruções que pode ser executado mecanicamente em uma quantidade finita de tempo e que resolve algum problema.

Os algoritmos constituem o coração da Ciência da Computação, e temos mais a dizer sobre eles através deste livro. Você provavelmente já sabe que a principal tarefa ao se escrever um programa para computador consiste em arquitetar um algoritmo (procedimento) para produzir a solução.

Os algoritmos são normalmente descritos em uma forma intermediária entre uma descrição puramente verbal na forma de parágrafos (tal como a que demos para decidir se $P \rightarrow Q$ é ou não uma tautologia) e um programa de computador (que, se executado, iria realizar os passos do algoritmo) escrito em uma linguagem de programação. Esta forma intermediária para descrição de algoritmos é chamada de **pseudocódigo**. Um algoritmo escrito em pseudocódigo não deve ser difícil de entender, mesmo que você não saiba nada a respeito de programação de computadores. (Os algoritmos apresentados neste livro vão desde pseudocódigos descritivos até códigos executáveis em Pascal.) Apresentamos a seguir um pseudocódigo na forma do de Pascal para determinar se $P \rightarrow Q$ é ou não uma tautologia.

ALGORITMO *TestaTautologia*

```

procedure TestaTautologia (P,Q: wffs compostas);
{ determina se a wff  $P \rightarrow Q$  é ou não uma tautologia }

```

```

begin

```

```

  j atribui valores-verdade a  $P$  e  $Q$  }
   $P := \text{true};$ 
   $Q := \text{false};$ 

```

```

repeat

```

```

    for cada wff que já tenha um valor-verdade atribuído, atribua os valores-verdade determinados para
    suas componentes

```

```

until todas as ocorrências de símbolos proposicionais têm valores-verdade;

```



```

if alguma letra tem dois valores-verdade then
  write ('Verdadeiro') {  $P \rightarrow Q$  é uma tautologia }
else
  write ('Falso');           {  $P \rightarrow Q$  não é uma tautologia }

end;

```

Este algoritmo inclui um laço de repetição, dentro do qual as atribuições dos valores-verdade são realizadas para componentes cada vez menores dos P e Q originais, até que a condição de *saída do laço* — todas as ocorrências de símbolos proposicionais tenham valores-verdade atribuídos — seja satisfeita. Em seguida o algoritmo procede com o comando condicional, onde é decidido se $P \rightarrow Q$ constitui ou não uma tautologia.

EXEMPLO 8

Considere uma wff da forma $(A \rightarrow B) \rightarrow (B' \rightarrow A')$. Aqui P é $A \rightarrow B$ e Q é $B' \rightarrow A'$. Seguindo o algoritmo *TestaTautologia*, atribuímos valores

$A \rightarrow B$ verdadeira e $B' \rightarrow A'$ falsa

o que significa que $(A \rightarrow B) \rightarrow (B' \rightarrow A')$ não é uma tautologia. Caminhando no laço, a atribuição de falso à wff composta $B' \rightarrow A'$ requer as atribuições futuras:

B' verdadeira e A' falsa

ou

B falsa e A verdadeira

A combinação de A verdadeira e $A \rightarrow B$ verdadeira requer a atribuição

B verdadeira

Neste ponto todas as ocorrências de símbolos proposicionais têm valores-verdade como mostrado a seguir:

$$\underbrace{A \rightarrow B}_{V} \rightarrow \underbrace{B' \rightarrow A'}_F$$

Isto termina o laço. No passo final do algoritmo, B tem atribuições tanto de V como de F . Esta situação impossível indica que nossa suposição estava errada e, portanto, $(A \rightarrow B) \rightarrow (B' \rightarrow A')$ é uma tautologia. Podemos verificar isso construindo a tabela-verdade. •

O algoritmo *TestaTautologia* determina se wffs de determinadas formas são ou não tautologias. No entanto, o processo de construção da tabela-verdade e o exame dos valores-verdade em sua última coluna constituem um algoritmo que determina se uma wff arbitrária é ou não uma tautologia. Este segundo algoritmo é, portanto, mais poderoso, uma vez que resolve problemas mais gerais.

Revisão da Seção 11

Técnicas

- Construção de tabelas-verdade para wffs compostas
- Reconhecimento de tautologias e contradições

Principais Conceitos

As wffs são representações simbólicas de sentenças.

Os valores-verdade de wffs compostas dependem dos valores-verdade de seus componentes e do tipo dos conectivos usados.

Exercícios 11

As respostas dos itens com estrelas ao lado são dadas ao final do livro.

- ★1. Quais das frases a seguir são sentenças?
- A lua é feita de queijo verde.
 - Dois é um número primo.
 - As taxas do ano que vem serão maiores.
 - $x - 4 = 0$
 - Ele é um homem alto.
 - O jogo terminará logo?
 - As taxas do ano que vem serão menores.
2. Dados os valores-verdade A verdadeiro, B falso e C verdadeiro, qual o valor-verdade de cada uma das seguintes wffs?
- $A \wedge (B \vee C)$
 - $(A \wedge B) \vee C$
 - $(A \wedge B)' \vee C$
 - $A' \vee (B' \wedge C)'$
3. Qual os valores-verdade das seguintes sentenças?
- 8 é par ou 6 é ímpar.
 - 8 é par e 6 é ímpar.
 - 8 é ímpar ou 6 é ímpar.
 - 8 é ímpar e 6 é ímpar.
 - Se 8 é ímpar, então 6 é ímpar.
 - Se 8 é par, então 6 é ímpar.
 - Se 8 é ímpar, então 6 é par.
 - Se 8 é ímpar e 6 é par, então $8 < 6$.
- ★4. Indique o antecedente e o conseqüente de cada uma das seguintes sentenças:
- O crescimento sadio das plantas é conseqüência de quantidade suficiente de água.
 - O crescimento da oferta de computadores é uma condição necessária para o desenvolvimento científico.
 - Haverá novos erros apenas se o programa for alterado.
 - A economia de combustível implica um bom isolamento, ou todas as janelas são janelas para tempestades.
5. Diversas negativas são dadas para cada uma das seguintes afirmações. Quais são as certas?
- A resposta é 2 ou 3.
 - Nem 2 nem 3 são a resposta.
 - A resposta não é 2 ou não é 3.
 - A resposta não é 2 e não é 3.
 - Pepinos são verdes e têm sementes.
 - Pepinos não são verdes e não têm sementes.
 - Pepinos não são verdes ou não têm sementes.
 - Pepinos são verdes e não têm sementes.
 - $2 < 7$ e 3 é ímpar.
 - $2 > 7$ e 3 é par.
 - $2 \geq 7$ e 3 é par.
 - $2 \geq 7$ ou 3 é ímpar.
 - $2 \geq 7$ ou 3 é par.
6. Sejam A , B e C as seguintes sentenças:
- A : Rosas são vermelhas.
 B : Violetas são azuis.
 C : Açúcar é doce.

Traduza as seguintes sentenças compostas para notação simbólica.

- Rosas são vermelhas e violetas são azuis.
- Rosas são vermelhas e, ou bem violetas são azuis ou bem açúcar é doce.
- Sempre que violetas são azuis, as rosas são vermelhas e o açúcar é doce.
- Rosas são vermelhas apenas se as violetas não forem azuis e se o açúcar for azedo.
- Rosas são vermelhas e, se o açúcar for azedo, então as violetas não são azuis ou o açúcar é doce.

7. Com A , B e C como os definidos no Exercício 6, traduza as seguintes wffs para o português:

- $B \vee C'$
- $B' \vee (A \rightarrow C)$
- $(C \wedge A') \leftrightarrow B$
- $C \wedge (A' \leftrightarrow B)$
- $(B \wedge C')' \rightarrow A$
- $A \vee (B \wedge C')$
- $(A \vee B) \wedge C'$

- ★8. Com o uso de letras para denotar as sentenças componentes, traduza as seguintes sentenças compostas para notação simbólica:
- Se os preços subirem, as construções ficarão mais caras, mas se as construções não forem caras, elas serão muitas.
 - Tanto ir para cama como nadar é condição suficiente para trocar de roupa; no entanto, trocar de roupa não significa que se vai nadar.
 - Ou vai chover ou vai nevar, mas não ambos.
 - Se Janet vencer ou perder, ela estará cansada.
 - Ou Janet irá vencer ou, se perder, ficará cansada.

9. Construa as tabelas-verdade para as seguintes wffs. Indique as tautologias e as contradições.

- | | |
|--|---|
| ★a. $(A \rightarrow B) \leftrightarrow A' \vee B$ | ★b. $(A \wedge B) \vee C \rightarrow A \wedge (B \vee C)$ |
| c. $A \wedge (A' \vee B')$ | d. $A \wedge B \rightarrow A'$ |
| e. $(A \rightarrow B) \rightarrow [(A \vee C) \rightarrow (B \vee C)]$ | f. $A \rightarrow (B \rightarrow A)$ |
| g. $A \wedge B \leftrightarrow B' \vee A'$ | h. $(A \vee B') \wedge (A \wedge B)'$ |
| i. $[(A \vee B) \wedge C'] \rightarrow A' \vee C$ | |

- ★10. Um chip da memória de um microcomputador tem 2^4 elementos de dois estados (LIGA-DESLIGA). Qual o número total de configurações LIGA-DESLIGA?

11. Considere o seguinte fragmento de um programa Pascal:

```

for contador := 1 to 5 do
  begin
    read(a);
    if ((a < 5.0)and(2*a < 10.7)) or (sqrt(5.0*a) > 5.1) then writeln(a);
  end;

```

Os valores de entrada para a são 1.0, 5.1, 2.4, 7.2 e 5.3. Quais são os valores de saída?

12. Verifique as equivalências na lista da página 8 através da construção de suas tabelas-verdade. (Já verificamos as 1a, 4b e 5a).

13. Verifique, através da construção das tabelas-verdade que as wffs a seguir são tautologias:

- | | |
|---|---|
| ★a. $A \vee A'$ | b. $(A')' \leftrightarrow A$ |
| ★c. $A \wedge B \rightarrow B$ | d. $A \rightarrow A \vee B$ |
| e. $(A \vee B)' \leftrightarrow A' \wedge B'$ | f. $(A \wedge B)' \leftrightarrow A' \vee B'$ |

14. Suponha que A , B e C representam condições que serão verdadeiras e falsas quando um programa é executado. Suponha ainda que desejamos que este programa realize alguma tarefa somente quando A ou B forem verdadeiras (mas não ambas) e C for falsa. Usando A , B e C e os conectivos E, OU e NOT, escreva uma sentença que será verdadeira apenas nestas condições.

15. Reescreva o programa Pascal a seguir com uma expressão condicional simplificada:

```

if not ((Valor1 < Valor2) or odd(Numero))
or {not(Valor1 < Valor2) and odd(Numero)} then
  comando 1
else
  comando2;

```

16. a. Verifique que $A \rightarrow B$ é equivalente a $A' \vee B$.

b. Usando a parte (a) e outras equivalências, escreva a negação da sentença "Se Sam passar em seu curso de Física, então ele se formará."

17. Use o algoritmo *TestaTautologia* para provar que as expressões a seguir são tautologias:

- | | |
|--|---|
| ★a. $[B' \wedge (A \rightarrow B)] \rightarrow A'$ | b. $[(A \rightarrow B) \wedge A] \rightarrow B$ |
| c. $(A \vee B) \wedge A' \rightarrow B$ | d. $(A \wedge B) \wedge B' \rightarrow A$ |

18. Em cada caso, construa as wffs compostas P e Q a fim de que a sentença dada seja uma tautologia.

- $P \wedge Q$
- $P \rightarrow P'$
- $P \wedge (Q \rightarrow P')$

19. A tabela-verdade de $A \vee B$ mostra que o valor de $A \vee B$ é verdadeiro se A for verdadeira, se B for verdadeira ou ainda se ambos o forem. O uso da palavra *ou* quando o resultado é verdadeiro, se ambos os componentes forem verdadeiros, é chamado de *ou inclusivo*. É este *ou inclusivo* que é entendido na frase "Teremos chuva ou garoa amanhã." Outro uso da palavra *ou* na língua portuguesa é o *ou exclusivo*, algumas vezes chamado de XOU (ou **XOR**, em inglês) no qual o resultado é falso quando ambos os componentes são verdadeiros. O *ou exclusivo* é entendido na frase "No cruzamento, devemos seguir para o norte ou para o sul". O *ou exclusivo* é simbolizado por $A \oplus B$.
- Escreva a tabela-verdade para o *ou exclusivo*.
 - Mostre que $A \oplus B \leftrightarrow (A \leftrightarrow B)'$ é uma tautologia.
20. Toda wff é equivalente a uma sentença que use apenas os conectivos da conjunção e negação. Para garantirmos isto devemos achar wffs equivalentes para $A \vee B$ e $A \rightarrow B$ que usem apenas $\wedge$ e $'$. Estas novas wffs poderão substituir, respectivamente, quaisquer ocorrências de $A \vee B$ e $A \rightarrow B$. (O conectivo $\leftrightarrow$ já foi definido em termos dos outros conectivos, portanto já sabemos que pode ser substituído em uma wff.)
- Mostre que $A \vee B$ é equivalente a $(A' \wedge B')'$.
 - Mostre que $A \rightarrow B$ é equivalente a $(A \wedge B')'$.
21. Mostre que qualquer wff composta é equivalente a uma wff que só contenha os conectivos de:
- $\vee$ e $'$
 - $\rightarrow$ e $'$.
- (Dica: Veja o exercício 20.)
22. Prove que há algumas wffs compostas que não são equivalentes a qualquer wff que use apenas os conectivos $\rightarrow$ e $\vee$.
- ★23. O conectivo binário $\downarrow$ é definido pela tabela-verdade a seguir. Mostre que qualquer wff composta é equivalente a uma wff que use apenas o conectivo $\downarrow$. (Dica: Use o Exercício 20 e ache wffs equivalentes para $A \wedge B$ e A' em termos de $\downarrow$.)

A	B	$A \downarrow B$
V	V	F
V	F	V
F	V	V
F	F	V

Exercício 23

A	B	$A \downarrow B$
V	V	F
V	F	F
F	V	F
F	F	V

Exercício 24

24. O conectivo binário $\downarrow$ é definido pela tabela-verdade dada. Mostre que qualquer wff composta é equivalente a uma wff que use apenas o conectivo $\downarrow$. (Dica: Veja o Exercício 23).
25. Em um determinado país, todos os habitantes são ou um contador de verdade que sempre fala a verdade ou mentirosos que sempre mentem. Viajando neste país, você encontra dois habitantes, Percival e Llewellyn. Percival diz "Se eu for um contador de verdades, Llewellyn também é um contador de verdades". Percival é um mentiroso ou um contador de verdade? E Llewellyn?

Seção 12 Quantificadores, Predicados e Validade

Quantificadores e Predicados

O que podemos expressar através das wffs que vimos na Seção 1.1 é muito limitado. Por exemplo, consideraríamos a sentença "Para todo x , $x > 0$ " como uma sentença verdadeira sobre inteiros positivos, no entanto ela não pode ser simbolizada adequadamente através de símbolos proposicionais, parênteses e conectivos lógicos. Ela contém dois novos elementos, um *quantificador* e um *predicado*. Os quantificadores são frases como "para todo", "para cada" ou "para algum", que indicam de alguma forma *quantos* objetos têm uma determinada propriedade. O **quantificador universal** é simbolizado por um A de cabeça-para-baixo, $\forall$, e é lido "para todo", "para todos", "para cada" ou para "para qualquer". Portanto, a sentença acima pode ser simbolizada como

$$(\forall x)(x > 0)$$

Um quantificador e sua variável são sempre colocados entre parênteses. O segundo par de parênteses indica que o quantificador age sobre a expressão interna, que no caso é " $x > 0$ ".

A frase " $x > 0$ " descreve a propriedade da variável x , que é ser positiva. Uma propriedade também é chamada de **predicado**; usamos a notação $P(x)$ para representar algum predicado não especificado ou propriedade que x possa ter. Portanto, a sentença original é um exemplo da forma mais geral

$$(\forall x)P(x)$$

O valor-verdade da expressão $(\forall x)(x > 0)$ depende do domínio dos objetos sob os quais estamos "interpretando" esta expressão. Isto é, a coleção de objetos dos quais x pode ser escolhido. Esta coleção de objetos é chamada de *domínio de interpretação*. Já havíamos visto que se o domínio de interpretação consistisse em inteiros positivos, a expressão teria o valor-verdade verdadeiro pois qualquer valor possível para x teria a propriedade necessária de ser maior que zero. Se o domínio de interpretação consistisse de todos os inteiros, a expressão teria o valor-verdade falso, pois nem todo x teria a propriedade necessária. Impomos a condição de que o domínio de interpretação tenha pelo menos um elemento a fim de que não nos preocupemos com o caso trivial.

Uma interpretação para a expressão $(\forall x)P(x)$ poderia consistir não apenas na coleção de objetos dos quais x pode tirar seus valores mas também da propriedade que $P(x)$ representa em seu domínio. Portanto, uma interpretação para $(\forall x)P(x)$ poderia ser a seguinte: O domínio consiste em todos os livros em sua biblioteca local, e $P(x)$ é a propriedade de que x deve ter capa vermelha. $(\forall x)P(x)$, nesta interpretação, diz que todo livro em nossa biblioteca local tem capa vermelha. O valor-verdade desta expressão nesta interpretação é, sem dúvida, falso.

PRÁTICA 9 Qual o valor-verdade da expressão $(\forall x)P(x)$ em cada uma das seguintes interpretações?

- $P(x)$ é a propriedade de que x seja amarelo e o domínio de interpretação é o conjunto de todos os canários-da-terra.
- $P(x)$ é a propriedade de que x seja amarelo e o domínio de interpretação é o conjunto de todos os pássaros.
- $P(x)$ é a propriedade de que x seja uma ave e o domínio de interpretação é o conjunto de todos os pássaros.
- $P(x)$ é a propriedade de que x seja positivo ou negativo e o domínio de interpretação consiste em todos os inteiros.

O **quantificador existencial** é simbolizado por um E espelhado, $\exists$, e é lido como "existe um", "para pelo menos um" ou "para algum". Portanto, a expressão

$$(\exists x)(x > 0)$$

deve ser lida como "existe um x tal que x é maior que zero."

Novamente, o valor-verdade desta expressão depende da interpretação. Se o domínio de interpretação contiver um número positivo, a expressão terá valor-verdade verdadeiro; caso contrário, ela terá valor falso. O valor-verdade $(\exists x)P(x)$, se o domínio for todos os livros de nossa biblioteca local e $P(x)$ for a propriedade de x ter a capa vermelha, será verdadeiro, desde que tenhamos pelo menos um livro na biblioteca com a capa vermelha.

PRÁTICA 10

- Construa uma interpretação (i.e., dê o domínio e o significado de $P(x)$) na qual $(\forall x)P(x)$ tenha o valor verdadeiro.
- Construa uma interpretação na qual $(\forall x)P(x)$ tenha o valor falso.
- É possível achar uma interpretação na qual tanto $(\forall x)P(x)$ seja verdadeiro e $(\exists x)P(x)$ seja falso?
- É possível achar uma interpretação na qual tanto $(\forall x)P(x)$ quanto $(\exists x)P(x)$ sejam verdadeiros?

Os predicados que vimos até agora, envolvendo propriedades de uma única variável, são **predicados unários**. Eles podem ser **binários**, quando envolvem propriedades de duas variáveis; **ternários**, quando envolvem propriedades de três variáveis; ou, de forma mais geral, **n-ários**, quando envolvem propriedades de n variáveis. Quantificadores universais podem ser incluídos a expressões com esses predicados de ordens altas.

EXEMPLO 9

A expressão $(\forall x)(\exists y)Q(x, y)$ é lida como "para todo x existe um y tal que $Q(x, y)$ ". Na interpretação onde o domínio consiste em inteiros e $Q(x, y)$ é a propriedade de $x < y$, isto apenas indica que, para qualquer inteiro, existe um inteiro ainda maior. O valor-verdade desta expressão é verdadeiro. Na mesma interpretação, a expressão $(\exists y)(\forall x)Q(x, y)$ indica que existe um inteiro y que é maior que qualquer inteiro x . Seu valor-verdade é falso.

O Exemplo 9 mostra que a ordem na qual os quantificadores aparecem é importante.

Em expressões como $(\forall x)P(x)$ ou $(\exists x)P(x)$, x é uma *variável muda*; isto é, os valores-verdade das expressões permanecem os mesmos em uma dada interpretação, mesmo que escrevamos a expressão, digamos

como $(\forall y)P(y)$ ou $(\exists z)P(z)$, respectivamente. Analogamente, o valor-verdade de $(\forall x)(\exists y)Q(x, y)$ é o mesmo de $(\forall z)(\exists w)Q(z, w)$ para qualquer interpretação. No entanto, $(\forall x)(\exists x)Q(x, x)$ indica algo bem diferente. Na interpretação do Exemplo 9, por exemplo, $(\forall x)(\exists x)Q(x, x)$ indicaria que para todo inteiro x , existe um inteiro x tal que $x < x$. Esta sentença é falsa, a despeito de $(\forall x)(\exists y)Q(x, y)$ ser verdadeiro para a interpretação. Não podemos misturar variáveis diferentes sem mudar a natureza da expressão obtida.

Também são permitidas constantes nas expressões. Um símbolo de constante (a, b, c , etc.) é interpretado como um objeto específico no domínio. Esta especificação é parte da interpretação. Por exemplo, a expressão $(\forall x)Q(x, a)$ é falsa na interpretação na qual o domínio consista em inteiros, $Q(x, y)$ é a propriedade $x < y$, e a contém o valor 7; não é verdade que todo inteiro seja menor que 7.

Agora temos tudo o que é necessário em uma interpretação.

Definição: Interpretação

Uma **interpretação** de uma expressão envolvendo predicados consiste no seguinte:

- um conjunto de objetos chamados o **domínio** da interpretação, que deve conter pelo menos um elemento;
- a atribuição de uma propriedade dos objetos do domínio para cada predicado na expressão; e
- a atribuição de um objeto particular no domínio a cada símbolo constante na expressão.

As expressões podem ser obtidas da combinação de predicados, quantificadores, símbolos de agrupamento (parênteses ou colchetes) e dos conectivos lógicos da Seção 1.1. Como antes, uma expressão precisa obedecer regras sintáticas a fim de ser considerada uma fórmula bem-formulada ou wff. A expressão $P(x)(\forall x) \wedge \exists y$ não é uma fórmula bem-formulada. Exemplos de fórmulas bem-formuladas são:

$$P(x) \vee Q(y) \quad (1)$$

$$(\forall x)[P(x) \rightarrow Q(x)] \quad (2)$$

$$(\forall x)(\exists y)[P(x, y) \wedge Q(x, y)] \rightarrow R(x) \quad (3)$$

e

$$(\exists x)S(x) \vee (\forall y)T(y) \quad (4)$$

Os símbolos de agrupamento ajudam a identificar o escopo de um quantificador, a seção da wff a qual os quantificadores se aplicam. (Isto é análogo ao escopo de um identificador de um programa como a seção do programa na qual o identificador tem significado.) Não há quantificadores na wff (1). Na (2), o escopo do quantificador $(\forall x)$ é $P(x) \rightarrow Q(x)$. Na (3), o escopo do $(\exists y)$ é $P(x, y) \wedge Q(x, y)$, enquanto que o escopo de $(\forall x)$ é toda a expressão nos parênteses que o segue. Na (4), o escopo de $(\exists x)$ é $S(x)$ e o escopo de $(\forall y)$ é $T(y)$.

Sabemos que o valor-verdade de uma wff é determinado em relação a uma dada interpretação. Em alguns casos, como veremos em breve, uma wff pode não ter valor-verdade para uma particular interpretação. De forma que, para considerarmos o valor-verdade de uma wff, precisamos considerar o escopo de seus quantificadores.

EXEMPLO 10 Considere a wff

$$(\forall x)(\exists y)[P(x, y) \wedge Q(x, y)]$$

Aqui o escopo de $(\exists y)$ é todo o $P(x, y) \wedge Q(x, y)$. O escopo de $(\forall x)$ é $(\exists y)[P(x, y) \wedge Q(x, y)]$; os parênteses e os colchetes podem ser eliminados quando o escopo estiver claro. Na interpretação onde o domínio consista em inteiros positivos, $P(x, y)$ seja a propriedade " $x \leq y$ " e $Q(x, y)$ seja a propriedade " x divide y ", a wff é verdadeira. Para qualquer x que seja um inteiro positivo, existe um inteiro positivo y , por exemplo, $y = 2x$, tal que $x \leq y$ e x divide y .

Suponhamos agora que a wff é

$$(\forall x)[(\exists y)P(x, y) \wedge Q(x, y)]$$

Agora o escopo de $(\exists y)$ é apenas $P(x, y)$. Se usarmos a mesma interpretação de antes, não haverá valor-verdade determinado para esta wff. Dado qualquer x podemos escolher um y (tal como $y = 2x$) de forma que $(\exists y)P(x, y)$ seja verdadeira, mas em $Q(x, y)$, y está livre para tomar qualquer valor no domínio de interpretação, exceto os que escolhemos para y satisfazer $(\exists y)P(x, y)$. De fato, lembremos que y em $(\exists y)P(x, y)$ é uma variável muda. Para valores de y que sejam múltiplos de x , $Q(x, y)$ é verdadeiro e, portanto, toda a expressão também o é; para outros valores de y , $Q(x, y)$ é falso e, portanto, toda a expressão também o é. •

Na segunda wff do Exemplo 10, a variável y ocorre três vezes:

$$(\forall x)[(\exists y)P(x, y) \wedge Q(x, y)]$$

$\uparrow \quad \uparrow \quad \uparrow$

A ocorrência de uma variável em uma wff é chamada de uma **ocorrência ligada** em duas condições:

1. Se tratar de uma variável identificando o que o quantificador quantifica, ou
2. estar fora do escopo de um quantificador envolvendo esta variável.

Portanto, a primeira ocorrência de y no exemplo acima é uma ocorrência ligada do primeiro tipo e a segunda ocorrência é uma ocorrência ligada do segundo tipo. A terceira ocorrência é uma **ocorrência livre** — que não é ligada. Ela não é uma variável que seja parte direta de um quantificador, nem é uma variável dentro do escopo de um quantificador que a envolva. Uma variável que tenha pelo menos uma ocorrência ligada em uma wff é uma **variável ligada** na wff, e uma variável com pelo menos uma ocorrência livre é uma **variável livre** na wff. (Perceba que pode ocorrer a situação um tanto o quanto estranha de uma variável ser ao mesmo tempo ligada e livre na mesma expressão.)

Como no Exemplo 10, uma wff com variáveis livres frequentemente — mas nem sempre — não terão valores-verdade para uma dada interpretação. De fato, a wff será verdadeira para alguns valores das variáveis livres e falsa para outros. Portanto, as wffs com predicados diferem das wffs da Seção 1.1, que eram compostas apenas por símbolos proposicionais e conectivos lógicos, e que sempre tinham valores-verdade. No entanto, uma wff com predicados e sem variáveis livres tem valor-verdade em qualquer interpretação, conquanto este valor possa alterar-se de uma interpretação para outra.

PRÁTICA 11 Qual o valor-verdade da wff

$$(\exists x)(A(x) \wedge (\forall y)[B(x, y) \rightarrow C(y)])$$

na interpretação onde o domínio consiste em todos os inteiros, $A(x)$ é " $x > 0$ ", $B(x, y)$ é " $x > y$ " e $C(y)$ é " $y \leq 0$ "? Construa outra interpretação com o mesmo domínio, no qual a sentença tenha o valor-verdade oposto. •

Muitas sentenças em português podem ser expressas como wffs contendo predicados e quantificadores. Por exemplo, a sentença "todo papagaio é feio" está, na verdade, dizendo que qualquer coisa que seja um papagaio é feia. Fazendo $P(x)$ denotar " x é um papagaio" e $U(x)$ denotar " x é feio" vemos que a sentença pode ser simbolizada como

$$(\forall x)[P(x) \rightarrow U(x)]$$

Outras variantes com o mesmo significado na língua portuguesa são "Qualquer papagaio é feio" e "Cada papagaio é feio".

Da mesma forma, "Existe um papagaio feio" é denotado como

$$(\exists x)[P(x) \wedge U(x)]$$

As variantes aqui são "Alguns papagaios são feios" e "Existem papagaios feios".

Ao representar essas sentenças da língua portuguesa como wffs usamos $(\forall x)$ para a implicação e $(\exists x)$ para a conjunção. As duas outras combinações possíveis quase nunca expressam o que se deseja dizer. A wff $(\forall x)[P(x) \wedge U(x)]$ indica que todos os elementos no domínio — entendido aqui como todo o mundo — são um papagaio feio; a wff $(\exists x)[P(x) \rightarrow U(x)]$ é verdadeira na medida em que haja algum elemento no domínio, chamado x , que não seja um papagaio, pois neste caso, $P(x)$ assume falso e a implicação é verdadeira.

PRÁTICA 12 Usando os símbolos predicados $S(x)$, $I(x)$ e $M(x)$, escreva wffs que expressem o pedido. (O domínio é a coleção de todas as pessoas.)

- a. Todos os estudantes são inteligentes.
 - b. Alguns estudantes inteligentes gostam de música.
 - c. Todos que gostam de música são estudantes estúpidos.
-

Negar sentenças com quantificadores, assim como negar sentenças compostas, requer um certo cuidado. A negação da sentença "Tudo é bonito" é "Não é verdade que tudo é bonito" ou "Algo não é bonito". Simbolicamente,

$$[(\forall x)A(x)]' \leftrightarrow (\exists x)[A(x)]'$$

é válido. Perceba que "Tudo não é bonito", ou $(\forall x)[A(x)]'$, diz algo *mais forte* que a negação da sentença original.

A negação de "Algo é bonito" é "Nada é bonito" ou "Tudo não é bonito". Simbolicamente,

$$[(\exists x)A(x)]' \leftrightarrow (\forall x)[A(x)]'$$

é válido. Em inglês, a sentença "Algo não é bonito" pode ser erroneamente interpretada como "Nem tudo é bonito" ou "Há algo que não seja bonito". No entanto, esta interpretação errada, simbolizada por $(\exists x)[A(x)]'$ *não é tão forte* quanto a negação da sentença original.

Validade

A fim de distinguirmos as wffs que contêm apenas símbolos proposicionais e conectivos lógicos (descritas na Seção 1.1) das que contêm predicados e variáveis, chamaremos as primeiras de **wffs proposicionais** e as últimas de **wffs predicativas**. Como vimos antes, uma wff proposicional sempre tem valor-verdade, enquanto que uma wff predicativa pode não ter valor-verdade.

O valor-verdade de uma wff proposicional depende dos valores-verdade atribuídos aos símbolos proposicionais. O valor-verdade (ou falta dele) de uma wff predicativa depende da interpretação. Escolher uma interpretação para uma wff predicativa é análogo a escolher valores-verdade para wffs proposicionais, exceto por haver um número infinito de interpretações possíveis para as wffs predicativas e apenas 2ⁿ linhas possíveis para wffs proposicionais com n símbolos proposicionais.

Uma tautologia é uma wff proposicional que é verdadeira em todas as linhas da tabela-verdade. O análogo à tautologia para as wffs predicativas é a *validade* — uma wff predicativa é **válida** se for verdadeira para qualquer interpretação possível. A validade de uma wff deve ser obtida apenas da forma da wff, uma vez que deve ser independente de qualquer interpretação em particular.

Sabemos que ao construirmos a tabela-verdade para uma wff proposicional e examinarmos todas as atribuições aos símbolos proposicionais, temos um algoritmo para determinar um "tautologismo". No entanto, como obviamente não podemos testar todas as interpretações possíveis, como podemos determinar a validade de uma wff predicativa? Como veremos, não existe algoritmo para determinar uma validade. (Isto não significa simplesmente que ainda não foi encontrado um algoritmo — o que estamos dizendo é que está *provado* que não existe um algoritmo deste tipo.) Precisamos usar o raciocínio para determinar quando a forma de uma wff a torna verdadeira para qualquer interpretação. Naturalmente, podemos provar que uma wff é inválida ao encontramos uma única interpretação na qual a wff tenha valor-verdade falso ou não tenha valor-verdade.

A tabela a seguir compara as wffs proposicionais e predicativas:

Wffs Proposicionais

1. Verdadeira ou falsa, de acordo com os valores atribuídos aos símbolos proposicionais
2. Tautologia — verdadeira para todas as atribuições de valores-verdade
3. Algoritmo (tabela-verdade) para determinar se uma wff é ou não uma tautologia

Wffs Predicativas

1. Verdadeira, falsa ou sem valor-verdade, dependendo da interpretação
2. Wff válida — verdadeira para todas as interpretações
3. Não há algoritmo para determinar se uma wff é ou não válida

Agora vamos pôr mãos à obra para determinar a validade.

EXEMPLO 11

- a. A wff $(\forall x)P(x) \rightarrow (\exists x)P(x)$ é válida. Em qualquer interpretação, se qualquer elemento do domínio tiver uma certa propriedade, então existirá um elemento do domínio que tenha esta propriedade. (Usamos aqui o fato de que o domínio de qualquer interpretação tem que conter pelo menos um elemento.) Portanto, sempre que o antecedente for verdadeiro, o conseqüente também o será, e a implicação é, portanto, verdadeira.
- b. A wff $(\forall x)P(x) \rightarrow P(a)$ é válida porque, em qualquer interpretação, a é um membro particular do domínio e, portanto, goza da propriedade que é compartilhada por todos os elementos do domínio.
- c. A wff

$$(\forall x)[P(x) \wedge Q(x)] \leftrightarrow (\forall x)P(x) \wedge (\forall x)Q(x)$$

é válida. Se tanto P como Q forem verdadeiras para todos os elementos do domínio, então P será verdadeira para todos os elementos e Q será verdadeira para todos os elementos e vice-versa.

- d A wff $P(x) \rightarrow [Q(x) \rightarrow P(x)]$ é válida, apesar de conter uma variável livre. Para comprovarmos isto, consideremos qualquer interpretação, e seja x qualquer membro do domínio. Então x tem ou não tem a propriedade P . Se x não a tiver, então $P(x)$ será falsa; como $P(x)$ é o antecedente da implicação, esta será verdadeira. Se, por outro lado, x tiver a propriedade P , então $P(x)$ é verdadeira e, a despeito do valor-verdade de $Q(x)$, a implicação $Q(x) \rightarrow P(x)$ será verdadeira, e a implicação principal também será verdadeira. •

EXEMPLO 12 A wff $(\exists x)P(x) \rightarrow (\forall x)P(x)$ não é válida. Por exemplo, na interpretação onde o domínio consista em inteiros e $P(x)$ signifique que x é par, é verdade que existe um inteiro par, mas é falso que todo inteiro seja par. O antecedente da implicação é verdadeiro e o conseqüente é falso, e portanto o valor da implicação é falso. •

Naturalmente não somos obrigados a usar um contexto matemático para construir uma interpretação na qual uma wff seja falsa, mas freqüentemente é mais simples fazê-lo pelo fato de as relações entre os objetos serem mais claras.

PRÁTICA 13 A wff

$$(\forall x)[P(x) \vee Q(x)] \rightarrow (\forall x)P(x) \vee (\forall x)Q(x)$$

é válida ou inválida? Justifique. •

Revisão da Seção 12

Técnicas

- Determinação do valor-verdade de uma wff predicativa em uma dada interpretação
- Tradução de sentenças na língua portuguesa para wffs e vice-versa
- Reconhecimento de uma wff válida e a justificação
- Reconhecimento de wffs não-válidas e a construção de uma interpretação na qual ela seja falsa ou não tenha valor-verdade

Idéias Principais

O valor-verdade de wffs predicativas depende da interpretação considerada.

Wffs válidas são wffs predicativas que são verdadeiras para qualquer interpretação e, portanto, a validade é uma propriedade inerente à forma da wff propriamente dita.

Exercícios 12

- Qual o valor-verdade de cada uma das wffs a seguir na interpretação onde o domínio consiste em inteiros, $O(x)$ é " x é ímpar", $L(x)$ é " $x < 10$ " e $G(x)$ é " $x > 9$ "?
 - $(\exists x)O(x)$
 - $(\forall x)[L(x) \rightarrow O(x)]$
 - $(\exists x)[L(x) \wedge G(x)]$
 - $(\forall x)[L(x) \vee G(x)]$
- Qual o valor-verdade de cada uma das wffs na interpretação onde o domínio consiste nos números inteiros?

★a. $(\forall x)(\exists y)(x + y = x)$	★b. $(\exists y)(\forall x)(x + y = x)$
★c. $(\forall x)(\exists y)(x + y = 0)$	★d. $(\exists y)(\forall x)(x + y = 0)$
e. $(\forall x)(\forall y)(x < y \vee y < x)$	f. $(\forall x)[x < 0 \rightarrow (\exists y)(y > 0 \wedge x + y = 0)]$
g. $(\exists x)(\exists y)(x^2 = y)$	h. $(\forall x)(x^2 > 0)$
- Indique o valor-verdade de cada uma das wffs a seguir na interpretação onde o domínio consiste nos estados do Brasil, $Q(x, y)$ é " x é ao norte de y ", $P(x)$ é " x começa com a letra P" e a é "Paraná".
 - $(\forall x)P(x)$
 - $(\forall x)(\forall y)(\forall z)[Q(x, y) \wedge Q(y, z) \rightarrow Q(x, z)]$
 - $(\exists y)(\exists x)Q(y, x)$
 - $(\forall x)(\exists y)[P(y) \wedge Q(x, y)]$
 - $(\exists y)Q(a, y)$

4. Para as wffs a seguir, ache uma interpretação na qual sejam verdadeiras e outra na qual sejam falsas.

- ★a. $(\forall x)([A(x) \vee B(x)] \wedge [A(x) \wedge B(x)])'$ b. $(\forall x)(\forall y)[P(x, y) \rightarrow P(y, x)]$
 c. $(\forall x)[P(x) \rightarrow (\exists y)Q(x, y)]$ d. $(\exists x)[A(x) \wedge (\forall y)B(x, y)]$
 e. $[(\forall x)A(x) \rightarrow (\forall x)B(x)] \rightarrow (\forall x)[A(x) \rightarrow B(x)]$

5. Identifique o escopo de cada um dos quantificadores nas wffs a seguir e indique quaisquer variáveis livres.

- a. $(\forall x)[P(x) \rightarrow Q(y)]$ b. $(\exists x)[A(x) \wedge (\forall y)B(y)]$
 c. $(\exists x)(\forall y)[P(x, y) \wedge Q(x, y)]$ d. $(\exists x)(\exists y)[A(x, y) \wedge B(y, z) \rightarrow A(a, z)]$

6. Com o uso de símbolos predicados mostrados e os quantificadores apropriados, escreva cada sentença na língua portuguesa como uma wff predicativa. (O domínio é todo o mundo.)

$D(x)$ é "x é um dia."
 $S(x)$ é "x é ensolarado."
 $R(x)$ é "x é chuvoso."

S é "segunda-feira."
 T é "terça-feira."

- ★a. Todos os dias são ensolarados.
 ★b. Alguns dias não são chuvosos.
 ★c. Todo dia que é ensolarado não é chuvoso.
 d. Alguns dias são ensolarados e chuvosos.
 e. Nenhum dia é ensolarado e chuvoso.
 f. Sempre é dia ensolarado se, e somente se, é um dia chuvoso.
 g. Nenhum dia é ensolarado.
 h. Segunda-feira foi ensolarada, portanto todo dia será ensolarado.
 i. Tanto segunda-feira quanto terça-feira foram chuvosos.
 j. Se algum dia for chuvoso, então todos os dias serão ensolarados.

7. Com o uso de símbolos predicados mostrados e os quantificadores apropriados, escreva cada sentença da língua portuguesa como uma wff predicativa. (O domínio é todo o mundo.)

$J(x)$ é "x é um juiz."
 $A(x)$ é "x é um advogado."
 $M(x)$ é "x é uma mulher."

$Q(x)$ é "x é um químico."
 $A(x, y)$ é "x admira y."

- a. Existem algumas mulheres advogadas que são químicas.
 b. Nenhuma mulher é advogada e química.
 ★c. Alguns advogados só admiram juizes.
 d. Todos os juizes admiram apenas juizes.
 e. Apenas juizes admiram juizes.
 f. Todas as mulheres advogadas admiram algum juiz.
 g. Algumas mulheres não admiram advogados.

8. Usando os símbolos predicados mostrados e os quantificadores apropriados, escreva as sentenças na língua portuguesa como wffs predicativas. (O domínio é todo o mundo.)

$C(x)$ é "x é uma Corvette."
 $F(x)$ é "x é uma Ferrari."

$P(x)$ é "x é um Porsche."
 $L(x, y)$ é "x é mais lento que y."

- ★a. Nada é, ao mesmo tempo, uma Corvette e uma Ferrari.
 ★b. Alguns Porsches são apenas mais lentos que as Ferraris.
 c. Apenas Corvettes são mais lentas que Porsches.
 d. Todas as Ferraris são mais lentas que alguma Corvette.
 e. Nenhum Porsche é mais lento que a Corvette.
 f. Se existir uma Corvette que seja mais lenta que uma Ferrari, então todas as Corvettes serão mais lentas que todas as Ferraris.

9. Usando os símbolos predicados mostrados e os quantificadores apropriados, escreva as sentenças na língua portuguesa como wffs predicativas. (O domínio é todo o mundo.)

$A(x)$ é "x é uma abelha."
 $F(x)$ é "x é uma flor."
 $G(x)$ é "x gosta de y."

- | | |
|--|---|
| a. Todas as abelhas gostam de todas as flores. | b. Algumas abelhas gostam de todas as flores. |
| c. Todas as abelhas gostam de algumas flores | d. Toda abelha só odeia flores. |
| e. Apenas abelhas gostam de flores. | f. Toda abelha só gosta de flores. |
| g. Nenhuma abelha gosta só de flores. | h. Algumas abelhas gostam de algumas flores. |
| i. Algumas abelhas gostam apenas de flores. | j. Toda abelha odeia algumas flores. |
| k. Toda abelha odeia todas as flores. | 1. Nenhuma abelha odeia todas as flores. |

10. Se

$B(x)$ for "x é bonito."
 $E(x)$ for "x é elegante."
 $G(x, y)$ for "x gosta de y."
 $H(x)$ for "x é um homem."
 $M(x)$ for "x é uma mulher."
 j for "John."
 k for "Kathy."

dê as traduções para a língua portuguesa das wffs a seguir:

- ★a. $E(j) \wedge G(k, j)$
 ★b. $(\forall x)[H(x) \rightarrow E(x)]$
 c. $(\forall x)(M(x) \rightarrow (\forall y)[G(x, y) \rightarrow H(y) \wedge E(y)])$
 d. $(\exists x)[H(x) \wedge E(x) \wedge G(x, k)]$
 e. $(\exists x)(M(x) \wedge B(x) \wedge (\forall y)[G(x, y) \rightarrow E(y) \wedge H(y)])$
 f. $(\forall x)[M(x) \wedge B(x) \rightarrow G(j, x)]$

11. Diversas formas de negação são apresentadas para cada uma das sentenças a seguir. Qual é a correta?

- a. Algumas pessoas gostam de Matemática.
 1. Algumas pessoas não gostam de Matemática.
 2. Todo o mundo não gosta de Matemática.
 3. Todo o mundo gosta de Matemática.
 b. Todo o mundo gosta de sorvete.
 1. Ninguém gosta de sorvete.
 2. Todo o mundo não gosta de sorvete.
 3. Alguém não gosta de sorvete.
 c. Todo o mundo é alto e magro.
 1. Alguém é baixo e gordo.
 2. Ninguém é alto e magro.
 3. Alguém é baixo ou gordo.
 d. Alguns retratos estão velhos ou apagados.
 1. Nenhum retrato está velho ou apagado.
 2. Alguns retratos não estão velhos ou apagados.
 3. Todos os retratos não estão velhos ou não estão apagados.

12. Explique por que as wffs são válidas.

- a. $(\forall x)(\forall y)A(x, y) \leftrightarrow (\forall y)(\forall x)A(x, y)$
 b. $(\exists x)(\exists y)A(x, y) \leftrightarrow (\exists y)(\exists x)A(x, y)$
 c. $(\exists x)(\forall y)P(x, y) \rightarrow (\forall y)(\exists x)P(x, y)$
 d. $A(a) \rightarrow (\exists x)A(x)$
 e. $(\forall x)[A(x) \rightarrow B(x)] \rightarrow [(\forall x)A(x) \rightarrow (\forall x)B(x)]$

13. Forneça interpretações que provem que as wffs a seguir não são válidas:

- ★a. $(\exists x)A(x) \wedge (\exists x)B(x) \rightarrow (\exists x)[A(x) \wedge B(x)]$
 b. $(\forall x)(\exists y)P(x, y) \rightarrow (\exists x)(\forall y)P(x, y)$
 c. $(\forall x)[P(x) \rightarrow Q(x)] \rightarrow [(\exists x)P(x) \rightarrow (\forall x)Q(x)]$
 d. $(\forall x)[A(x)]' \leftrightarrow [(\forall x)A(x)]'$

14. Determine quais wffs são válidas ou inválidas. Justifique sua resposta.

- a. $(\exists x)A(x) \leftrightarrow ((\forall x)[A(x)]')'$
 b. $(\forall x)P(x) \vee (\exists x)Q(x) \rightarrow (\forall x)[P(x) \vee Q(x)]$
 c. $(\forall x)A(x) \leftrightarrow ((\exists x)[A(x)]')'$
 d. $(\forall x)[P(x) \vee Q(x)] \rightarrow (\forall x)P(x) \vee (\exists y)Q(y)$

Seção 1.3 Lógica Proposicional

Sistemas Formais

Resultados matemáticos são normalmente chamados teoremas. Os sistemas da lógica formal manipulam fórmulas como as das Seções 1.1 e 1.2 e atribuem um significado preciso à palavra teorema. Nestes sistemas, certas wffs são aceitas como **axiomas-wffs** que não precisam ser provadas. Um axioma deve, portanto, ser uma wff cuja "verdade" seja evidente. Então, pelo menos, um axioma deve ser uma tautologia ou, se envolve predicados, uma wff válida. Além dos axiomas, sistemas formais contêm regras de inferência. Uma **regra de inferência** é uma convenção que permite a uma nova wff de uma certa forma ser inferida, ou deduzida, de uma a duas outras wffs de uma certa forma. Uma seqüência de wffs na qual cada wff seja ou um axioma ou o resultado da aplicação de uma das regras de inferência às wffs anteriores na seqüência é chamada de **seqüência de prova**. Um **teorema** é o último componente desta seqüência; a seqüência é a **prova** do teorema.

O esboço a seguir é uma prova típica de um teorema

- wff 1 (um axioma)
- wff2 (um axioma)
- wff3 (inferida da wff 1 e da wff2 por uma regra de inferência)
- wff4 (um axioma)
- wff5 (inferida da wff4 por uma regra de inferência)
- wff6 (inferida da wff3 e wff5 por uma regra de inferência)

A última wff na seqüência, wff6, é o teorema, e a seqüência toda constitui sua prova. (Naturalmente as outras wffs também podem ser teoremas — bastaria termos parado a seqüência nelas.)

Outra condição que incluímos em nosso sistema, além da que os axiomas devam ser tautologias ou wffs válidas é que haja o mínimo de axiomas e de regras quanto for possível. Convém minimizar o número de sentenças que devemos aceitar sem prova, mesmo quando essas sentenças parecem óbvias.

A escolha do conjunto de axiomas e das regras de inferência deve ser cuidadosa. Se escolhermos poucos axiomas, poucas regras de inferência ou regras de inferências fracas, não seremos capazes de provar algumas wffs que são "verdadeiras" e que, portanto, podem ser teoremas. Por outro lado, se escolhermos muitos axiomas, muitas regras de inferência ou regras de inferência muito fortes, seremos capazes de chegar a praticamente qualquer wff e provar que é um teorema, inclusive wffs que não são "verdadeiras" e, portanto, não podem ser teoremas. Como conseguimos exatamente o necessário para os teoremas?

Para termos idéia da palavra intuitiva *verdadeiro*, consideramos dois sistemas lógicos formais — um para as wffs proposicionais e outro para as wffs predicativas. O sistema formal que usa as wffs proposicionais é chamado **lógica proposicional**, **lógica de sentenças** ou **cálculo proposicional**. O sistema formal que usa wffs predicadas é chamado **lógica predicada** ou **cálculo predicado**. (A palavra *cálculo* é usada aqui no sentido mais geral de "avaliação" ou "raciocínio", e não no sentido de "diferenciação" ou "integração".) Estudaremos lógica proposicional a seguir e lógica de predicados na Seção 1.4.

Lógica Proposicional

Na lógica proposicional, as wffs são formadas de símbolos proposicionais, conectivos lógicos e símbolos de agrupamento. Neste sistema, uma wff "verdadeira" significa uma tautologia. Desejamos, portanto, os axiomas e as regras de inferência que nos permitirão provar todas as tautologias, e apenas as tautologias, como teoremas.

Tomemos as seguintes wffs como axiomas, onde P , Q e R representam wffs proposicionais:

1. $P \rightarrow (Q \rightarrow P)$
2. $[P \rightarrow (Q \rightarrow R)] \rightarrow [(P \rightarrow Q) \rightarrow (P \rightarrow R)]$
3. $(Q' \rightarrow P') \rightarrow (P \rightarrow Q)$

Podemos mostrar que cada uma delas é uma tautologia, propriedade que desejamos para nossos axiomas. Não está claro por que esses axiomas em particular devem ser escolhidos ou como devem ser usados; por enquanto, podemos ver que eles dizem algo sobre o processo de raciocínio. O axioma 1 diz que, dado P , qualquer coisa implica P . O axioma 2 diz que dada uma certa implicação que segue de P , então se o antecedente desta implicação também segue de P , o conseqüente também o fará. (Este é um tipo de regra de "transitividade", um pouco parecida com: Se $a > b > c$, então se $a > b$, segue que $a > c$.) O axioma 3 que se "não Q " implica "não P ", então P implica Q , pois, do contrário, "não Q " valeria, e isto implicaria "não P ".

Como P , Q e R podem ser wffs compostas, cada axioma dado acima é, na verdade, um padrão, ou esquema, para um número finito de wffs. Portanto,

$$(A \rightarrow B) \rightarrow [(C \wedge D) \rightarrow (A \rightarrow B)]$$

é um axioma, pois se encaixa no esquema do axioma 1, onde P é a wff $A \rightarrow B$ e Q é a wff $C \wedge D$. Mas ao contrário de um pequeno número de axiomas, isto não indica que temos um número infinito de axiomas? Sim, mas existem apenas três *formas* para os axiomas.

Na lógica proposicional, existe apenas uma regra de inferência: Das wffs P e $P \rightarrow Q$, podemos inferir a wff Q . (Esta regra de inferência é conhecida pelo seu nome latino de **modus ponens**, que significa "método de afirmação".)

A wff $A \rightarrow A$, onde A é um símbolo proposicional, é uma tautologia. Portanto, usando nossos axiomas e a regra de inferência, esperamos ser capazes de provar que ela é um teorema. O exemplo a seguir nos dá a seqüência de prova desta wff. Apesar de a wff ser bem simples, a prova é difícil; em particular, não fica absolutamente claro por onde começar. Não se aflija quanto a isto, pois logo teremos uma técnica que nos ajudará a contornar esta dificuldade. O objetivo do Exemplo 13 é tão-somente ilustrar o uso dos vários axiomas e da regra de inferência em uma seqüência de prova. A justificativa dada para cada passo elucida seu uso, mas elas não constituem parte da seqüência de prova propriamente dita.

EXEMPLO 13 A wff $A \rightarrow A$ é um teorema. Uma seqüência de prova é a seguinte:

- | | |
|--|---|
| 1. $A \rightarrow [(A \rightarrow A) \rightarrow A] \rightarrow$ | (Axioma 2 com $P = A$, $Q = A \rightarrow A$, $R = A$) |
| $[A \rightarrow (A \rightarrow A)] \rightarrow (A \rightarrow A)]$ | |
| 2. $A \rightarrow [(A \rightarrow A) \rightarrow A]$ | (Axioma 1 com $P = A$, $Q = A \rightarrow A$) |
| 3. $[A \rightarrow (A \rightarrow A)] \rightarrow (A \rightarrow A)$ | (de 1 e 2 pelo modus ponens) |
| 4. $A \rightarrow (A \rightarrow A)$ | (Axioma 1 com $P = A$, $Q = A$) |
| 5. $A \rightarrow A$ | (de 3 e 4 pelo modus ponens) |

Uma seqüência de prova semelhante poderia mostrar que $P \rightarrow P$ é um teorema para qualquer wff P ; portanto, o que temos é um esquema de teorema. Em geral, o que faremos é provar esquemas de teoremas ao invés de teoremas individuais.

Os axiomas que escolhemos envolvem apenas implicação e negação. Para wffs que contenham os conectivos de disjunção e conjunção, usamos as equivalências

$$A \vee B \Leftrightarrow A' \rightarrow B \quad e \quad A \wedge B \Leftrightarrow (A \rightarrow B')$$

e nos contentamos em provar as wffs equivalentes que resultam. De fato, poderíamos ter definido a disjunção e conjunção em termos da implicação e negação. Então todas as nossas wffs teriam envolvido apenas os conectivos de implicação e negação.

Apesar de não provarmos isto aqui, este sistema de axiomas e uma regra de inferência fazem exatamente o que desejamos — toda tautologia é um teorema (i.e., tem uma prova), e vice-versa. Esta propriedade é descrita dizendo que nosso sistema formal é **completo** (tudo o que deveria ser um teorema, o é) e **correto** (nada que não deveria ser um teorema não o é).

Permitimos abreviações nas seqüências de provas através do uso de teoremas já provados. Uma vez que T já tenha sido provada por um teorema, então T pode servir como passo em outra seqüência de prova. Isto porque T tem sua própria seqüência de prova, que poderia ser substituída na seqüência de prova que estamos construindo.

Deduções

Freqüentemente desejamos provar, como teoremas, wffs da forma $P \rightarrow Q$, onde P e Q são wffs compostas. P é chamada de **hipótese** do teorema e Q , de **tese**. $P \rightarrow Q$ é também uma implicação, onde, de acordo com a terminologia usada, P é o antecedente e Q o conseqüente. No entanto, nem toda implicação é um teorema. Para que o seja, o antecedente e o conseqüente precisam estar relacionados estruturalmente, de forma que a implicação como um todo seja uma tautologia. A implicação $A \rightarrow B$ para os símbolos proposicionais A e B , por exemplo, não é uma tautologia e portanto não é um teorema. Portanto A , neste caso, é um antecedente mas não uma hipótese; B é um conseqüente, mas não uma tese. No entanto, a implicação $A \wedge B \rightarrow B \wedge A$ é uma tautologia e, portanto, é um teorema. Neste caso, $A \wedge B$ é tanto um antecedente como uma hipótese e $B \wedge A$ é tanto um conseqüente quanto uma conclusão.

Se $P \rightarrow Q$ é um teorema, ele deve ser uma tautologia e sempre que P for verdadeira, Q também o deve ser. Intuitivamente, imaginamos ser possível deduzir Q a partir de P . Formalmente, definimos uma **dedução**

de Q a partir de P como uma seqüência de wffs terminando em Q onde cada wff é um axioma ou é a wff P ou ainda é derivada das wffs anteriores através das regras de inferência. De fato, esta é a prova de um teorema, onde aceitamos P como um axioma. Podemos mostrar que $P \rightarrow Q$ constitui um teorema se, e somente se, Q for dedutível a partir de P . Nossa técnica para demonstrar teoremas da forma $P \rightarrow Q$ é, portanto, incluir a hipótese se como uma das wffs na seqüência e concluir a seqüência com Q .

Para mostrar a potencialidade do método de dedução, refaremos a prova do teorema do Exemplo 13, usando uma wff P arbitrária.

EXEMPLO 14 Usando a lógica proposicional, prove o teorema $P \rightarrow P$.

1. P (hipótese)

Como P não é apenas a hipótese, mas também a tese, a prova está completa. Uma prova confusa de cinco linhas se resumiu a uma prova de uma única linha! •

Naturalmente, mesmo com o método dedutivo, a maioria das demonstrações não será tão trivial como no Exemplo 14, mas sempre temos um início ao incluirmos a hipótese.

EXEMPLO 15 Usando a lógica proposicional, prove o teorema

$$[P \rightarrow (P \rightarrow Q)] \rightarrow (P \rightarrow Q)$$

A hipótese é $P \rightarrow (P \rightarrow Q)$, portanto, citá-la será o primeiro passo da demonstração.

1. $P \rightarrow (P \rightarrow Q)$ (hipótese)

Agora, consideremos o que precisamos fazer para sair deste ponto e chegar à tese desejada, $P \rightarrow Q$. Se introduzíssemos P como um passo da demonstração, o modus ponens e o passo 1 nos permitiria concluir $P \rightarrow Q$. No entanto, neste ponto ainda não temos razão para este passo. Por isso procuraremos um axioma que nos permita usar o passo 1 e o modus ponens. O Axioma 2 tem uma hipótese que se assemelha ao passo 1, desde que façamos, no Axioma 2, $P = P$, $Q = P$ e $R = Q$. O segundo passo da demonstração é, portanto

2. $[P \rightarrow (P \rightarrow Q)] \rightarrow [(P \rightarrow P) \rightarrow (P \rightarrow Q)]$ (Axioma 2)

e, então

3. $(P \rightarrow P) \rightarrow (P \rightarrow Q)$ (1,2, modus ponens)

Agora, se pudéssemos introduzir $P \rightarrow P$ como um passo da demonstração, o modus ponens e o passo 3 nos dariam a conclusão $P \rightarrow Q$. Mas $P \rightarrow P$ é o teorema provado no Exemplo 14 e, portanto, pode ser inserido aqui. A prova completa é a seguinte:

1. $P \rightarrow (P \rightarrow Q)$	(hipótese)
2. $[P \rightarrow (P \rightarrow Q)] \rightarrow [(P \rightarrow P) \rightarrow (P \rightarrow Q)]$	(Axioma 2)
3. $(P \rightarrow P) \rightarrow (P \rightarrow Q)$	(1,2, modus ponens)
4. $P \rightarrow P$	(Exemplo 14)
5. $P \rightarrow Q$	(3, 4, modus ponens) •

PRÁTICA 14 Usando a lógica proposicional, demonstre o teorema

$$P' \rightarrow (P \rightarrow Q)$$

(Dica: Use os Axiomas 1 e 3.) •

O método de dedução pode ser estendido a fim de ser tornado ainda mais poderoso. Se a hipótese do teorema for uma série de conjunções $P_1 \wedge P_2 \wedge \dots \wedge P_n$, simplesmente incluímos cada fator da hipótese como uma wff na seqüência da demonstração e deduzimos a tese dela. Por último, se a tese for, ela própria, uma implicação $R \rightarrow S$, podemos incluir R na seqüência da demonstração, tornando-a parte da hipótese, e apenas deduzir S . (Isto é coerente com nosso entendimento da implicação, mas veja o Exercício 18 ao fim desta seção para uma justificativa formal.)

Para resumir, o método de dedução permite as seguintes abordagens:

1. Para provar o teorema $P \rightarrow Q$, deduzimos Q a partir de P .
2. Para provar o teorema $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow Q$, deduzimos Q a partir de $P_1, P_2, \dots, P_n$.
3. Para provar o teorema $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow (R \rightarrow S)$, deduzimos S de $P_1, P_2, \dots, P_n, R$.

EXEMPLO 16 Uma prova para o teorema

$$(P' \rightarrow Q') \wedge (P \rightarrow S) \rightarrow (Q \rightarrow S)$$

usando a lógica proposicional é

- | | |
|--|----------------------|
| 1. $P' \rightarrow Q'$ | (hipótese) |
| 2. $P \rightarrow S$ | (hipótese) |
| 3. Q | (hipótese) |
| 4. $(P' \rightarrow Q') \rightarrow (Q \rightarrow P)$ | (Axioma 3) |
| 5. $Q \rightarrow P$ | (1, 4, modus ponens) |
| 6. P | (3, 5, modus ponens) |
| 7. S | (2, 6, modus ponens) |

Um teorema da forma $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow Q$ indica que Q pode ser deduzido de $P_1, P_2, \dots, P_n$. Portanto, por exemplo, em uma seqüência de demonstração contendo as wffs $P' \rightarrow Q'$ e $P \rightarrow S$, podemos inserir a sentença $Q \rightarrow S$ e citar o teorema do Exemplo 16 como justificativa.

PRÁTICA 15 Encontre uma demonstração mais curta para o teorema do Exemplo 15.

PRÁTICA 16 Use a lógica proposicional para provar o teorema

$$(P \rightarrow Q) \wedge (Q \rightarrow R) \rightarrow (P \rightarrow R)$$

Argumentos Válidos

Em português, um argumento (a acusação de um advogado, uma propaganda ou um discurso de um político) é normalmente apresentado como uma série de sentenças que podem ser simbolizadas por $P_1, P_2, \dots, P_n$ seguidas de uma conclusão Q . O argumento é um **argumento válido** se a conclusão puder ser deduzida, do ponto de vista da lógica, da conjunção $P_1 \wedge P_2 \wedge \dots \wedge P_n$ — em outras palavras, se $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow Q$ for um teorema.

EXEMPLO 17 O argumento a seguir é válido? "Meu cliente é canhoto, mas o diário não desapareceu, então meu cliente não é canhoto; portanto, o diário desapareceu." Existem apenas duas sentenças envolvidas aqui, que simbolizaremos como:

C : Meu cliente é canhoto.

D : O diário desapareceu.

O argumento é, portanto,

$$[C \wedge (D' \rightarrow C')] \rightarrow D$$

A validade do argumento é estabelecida pela seguinte prova:

- | | |
|--|----------------------|
| 1. C | (hipótese) |
| 2. $D' \rightarrow C'$ | (hipótese) |
| 3. $(D' \rightarrow C') \rightarrow (C \rightarrow D)$ | (Axioma 3) |
| 4. $C \rightarrow D$ | (2, 3, modus ponens) |
| 5. D | (1, 4, modus ponens) |

Perceba que a validade do argumento é uma função apenas de sua forma lógica, e não tem nada a ver com o fato de seus componentes serem ou não verdadeiros. Continuamos a não ter qualquer idéia sobre se o diário está realmente desaparecido ou não. Além disso, o argumento "Skoozes são rosa, mas se Gingoos não gosta-

rem de perskees, os skoozes não serão rosa; portanto, Ginguos não gostam de perkees", que têm a mesma forma, é também válido, apesar de não fazer qualquer sentido. •

Como qualquer tautologia também é um teorema na lógica proposicional, podemos inserir uma tautologia em qualquer passo de uma demonstração. Isto também constitui uma simplificação válida para resumir diversos passos de uma demonstração em um único passo, desde que esteja bem claro que não foram violadas quaisquer regras. Por exemplo, ao invés de

1. $P \wedge Q$ (hipótese)
2. $P \wedge Q \rightarrow Q$ (tautologia)
3. Q (1,2, modus ponens)

poderíamos escrever

1. $P \wedge Q$ (hipótese)
2. Q (1, tautologia $A \wedge B \rightarrow B$, modus ponens)

Tautologias da forma $P \wedge Q \rightarrow R$ nos dizem que R pode ser deduzida das hipóteses P e Q , de forma que a demonstração possa incluir passos como

1. P (hipótese)
2. Q (hipótese)
3. R (tautologia $P \wedge Q \rightarrow R$)

EXEMPLO 18 Considere o argumento "Se a taxa para importação diminuir, o comércio interno aumentará. Ou a taxa federal de desconto diminuirá ou o comércio interno não irá aumentar. A taxa para importação vai diminuir. Portanto a taxa federal de desconto vai diminuir". Usando

I : A taxa para importação vai diminuir.
 M : O mercado interno vai aumentar.
 F : A taxa federal de desconto vai diminuir.

o argumento é

$$[(I \rightarrow M) \wedge (F \vee M') \wedge I] \rightarrow F$$

Uma demonstração para estabelecer a validade é:

1. $I \rightarrow M$ (hipótese)
2. $F \vee M'$ (hipótese)
3. I (hipótese)
4. M (1, 3, modus ponens)
5. F (tautologia $(F \vee M') \wedge M \rightarrow F$) •

A tautologia dada como justificativa do passo 5 mostra que F pode ser deduzida de $F \vee M'$ e M . Uma tabela-verdade separada pode ser usada para provar que $(F \vee M') \wedge M \rightarrow F$ é, de fato, uma tautologia, mas sua verdade é intuitivamente clara — ela diz que se F ou não M forem verdadeiros, e M for verdadeiro (portanto não M não é verdadeiro), então a outra afirmativa, F , precisa ser verdadeira. •

A tautologia $A \wedge B \rightarrow A \wedge B$ é particularmente útil em demonstrações, na medida em que justifica a dedução de $A \wedge B$ das hipóteses A e B que apareceram antes na demonstração. Isto é mostrado no passo 6 do próximo exemplo.

EXEMPLO 19 Mostre que o seguinte argumento é válido "Se usamos a linguagem assembly, então o programa será executado mais rapidamente. Se usamos a linguagem assembly, o programa terá mais linhas de código. Portanto, se usamos a linguagem assembly, então o programa será executado mais rapidamente e terá mais linhas de código." Usando

A : Usamos a linguagem assembly.
 R : O programa será executado mais rapidamente.
 L : O programa terá mais linhas de código.

o argumento é $(A \rightarrow R) \wedge (A \rightarrow L) \rightarrow [A \rightarrow (R \wedge L)]$ e a prova é

1. $A \rightarrow R$ (hipótese)
2. $A \rightarrow L$ (hipótese)
3. A (hipótese)
4. R (1, 3, modus ponens)
5. L (2, 3, modus ponens)
6. $R \wedge L$ (4, 5, $A \wedge B$ pode ser deduzido de A e B)

PRÁTICA 17

Mostre que o argumento a seguir é válido usando as letras P, M e C : "Se o produto for confiável, a parcela do mercado irá aumentar. Ou o produto é confiável ou os custos irão subir. A parcela de mercado não irá aumentar. Portanto os custos irão subir."

Nossa versão da lógica proposicional inclui apenas uma regra de inferência, a modus ponens, que permite que Q seja inserida como uma wff na demonstração a partir da verificação anterior de P e $P \rightarrow Q$. Esta regra de inferência permite que se criem seqüências de demonstração da seguinte forma:

1. $P \rightarrow Q$ (hipótese)
2. Q' (hipótese)
3. $(P \rightarrow Q) \wedge Q'$ (1, 2 $A \wedge B$ pode ser deduzida de A e B)
4. P' (3, tautologia $(A \rightarrow B) \wedge B' \rightarrow A'$, modus ponens)

Algumas versões da lógica proposicional incluem uma outra regra de inferência que reproduz este processo. A **modus tollens** ("método de negação") diz: das wffs $P \rightarrow Q$ e Q' , podemos inferir a wff P' . O sistema formal pode ser estendido para incluir outras regras de inferência. O quadro a seguir ilustra algumas regras de inferências e os nomes usualmente atribuídos a elas. Nós delineamos a lógica proposicional com o uso apenas do modus ponens porque qualquer coisa deduzida com o uso das demais regras de inferência listadas no quadro pode ser deduzida em nosso sistema usando as tautologias apropriadas e o modus ponens, da mesma forma que a dedução acima poderia ser substituída por uma aplicação do modus tollens.

Regras de Inferência para Lógica Proposicional		
De:	Podemos inferir	Nome
P e $P \rightarrow Q$	Q	Modus ponens (nossa única regra de inferência)
$P \rightarrow Q$ e Q'	P'	Modus tollens
$P \vee Q$ e Q'	P	Silogismo disjuntivo
$P \rightarrow Q$ e $Q \rightarrow R$	$P \rightarrow R$	Silogismo hipotético
P e Q	P	Simplificação conjuntiva
P	$P \vee Q$	Ampliação disjuntiva

Uma Preliminar

Podemos ter tido o seguinte pensamento: Se teoremas são como tautologias, então para mostrar que uma wff é um teorema, por que, para mostrar que uma wff é um teorema, não mostramos apenas que é uma tautologia? A determinação de se uma wff é uma tautologia pode ser feita através da construção da tabela-verdade ou do algoritmo *TestaTautologia* da Seção 1.1. Isto seria uma tarefa mais mecânica do que construir uma demonstração. Então por que falamos de provas?

Se fôssemos nos restringir à lógica proposicional, não teríamos a necessidade do conceito de provas formais. No entanto, sabemos que nem todas as sentenças podem ser simbolizadas dentro da lógica proposicional, e que não há procedimento mecânico (como a construção da tabela-verdade) para determinar a validade de wffs predicativas. Uma técnica de prova formal precisa ser usada; e o que fizemos aqui para lógica proposicional pode servir como base para o que será feito na lógica de predicados.

Revisão da Seção 1.3

Técnicas

- Prova de teoremas na lógica proposicional
- Uso da lógica proposicional para provar a validade de um argumento na língua portuguesa

Idéias Principais

A prova de sistemas formais consiste em seqüências de hipóteses, axiomas ou wffs inferidas das wffs anteriores da seqüência.

Todo teorema na lógica proposicional é uma tautologia, e toda tautologia é um teorema.

Em argumentos válidos, a tese pode ser deduzida da conjunção das demais sentenças.

Exercícios 1.3

★1. Justifique cada um dos passos na demonstração de $(P \rightarrow Q) \wedge [P \rightarrow (Q \rightarrow R)] \rightarrow (P \rightarrow R)$ a seguir:

1. $P \rightarrow Q$
2. $P \rightarrow (Q \rightarrow R)$
3. $[P \rightarrow (Q \rightarrow R)] \rightarrow [(P \rightarrow Q) \rightarrow (P \rightarrow R)]$
4. $(P \rightarrow Q) \rightarrow (P \rightarrow R)$
5. $P \rightarrow R$

2. Justifique cada passo na demonstração de $[(P \rightarrow Q) \rightarrow P] \rightarrow [(P \rightarrow Q) \rightarrow Q]$ a seguir:

1. $(P \rightarrow Q) \rightarrow (P \rightarrow Q)$
2. $[(P \rightarrow Q) \rightarrow (P \rightarrow Q)]$
 $[(P \rightarrow Q) \rightarrow P] \rightarrow [(P \rightarrow Q) \rightarrow Q]$
3. $[(P \rightarrow Q) \rightarrow P] \rightarrow [(P \rightarrow Q) \rightarrow Q]$

Nos Exercícios de 3 a 12, demonstre que cada wff é um teorema da lógica proposicional. Você pode usar qualquer teorema demonstrado anteriormente, incluindo os exercícios anteriores.

3. $[(Q' \rightarrow P') \wedge P \wedge (Q \rightarrow R)] \rightarrow R$
- ★4. $P' \wedge (P \vee Q) \rightarrow Q$
5. $[(P \rightarrow (Q \rightarrow R)) \wedge (P \vee S') \wedge Q] \rightarrow (S \rightarrow R)$
6. $P \rightarrow (P \vee Q)$
- ★7. $(P')' \rightarrow P$
8. $P \rightarrow (P')'$
9. $(P \rightarrow Q) \rightarrow (Q' \rightarrow P')$
- ★10. $P \vee Q \rightarrow Q \vee P$
11. $P' \wedge (Q \rightarrow P) \rightarrow Q'$
12. $[P \rightarrow (Q \rightarrow R)] \rightarrow (Q \rightarrow (P \rightarrow R))$

Usando a lógica proposicional, prove que os argumentos dos Exercícios 13 a 17 são válidos. Use os símbolos proposicionais indicados.

13. Se o programa é eficiente, ele executará rapidamente: Ou o programa é eficiente ou ele tem um erro. No entanto, o programa não executa rapidamente. Portanto o programa tem um erro. (E, R, B)
14. A colheita é boa, mas não há água suficiente. Se tivesse bastante água ou não tivesse bastante sol, então haveria água suficiente. Portanto, a colheita é boa e há bastante sol. (C, A, H, S)
- ★15. Rússia tinha um poder superior, e ou a França não era forte ou Napoleão cometeu um erro. Napoleão não cometeu um erro, mas se o exército não tivesse falhado, a França seria forte. Portanto, o exército falhou e a Rússia tinha um poder superior. (R, F, N, E)
16. Não é verdade que se as taxas de eletricidade subirem, o consumo diminuirá, nem é verdade que novas usinas de energia serão construídas ou as contas não serão atrasadas. Portanto o consumo não diminuirá e as contas serão atrasadas. (T, C, U, Co)

17. Se José pegou as jóias ou a sra. Krasov mentiu, então ocorreu um crime. O sr. Krasov não estava na cidade. Se ocorreu um crime, então o sr. Krasov estava na cidade. Portanto José não pegou as jóias, (J, M, C, E)
- ★18. a. Use uma tabela-verdade para verificar que $A \rightarrow (B \rightarrow C) \Leftrightarrow (A \wedge B) \rightarrow C$ é uma tautologia
 b. Prove que $A \rightarrow (B \rightarrow C) \Leftrightarrow (A \wedge B) \rightarrow C$ usando uma série de equivalências.
 c. Explique como esta equivalência justifica a extensão do método de dedução que diz para provar $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow (R \rightarrow S)$, basta deduzir S de $P_1, P_2, \dots, P_n$ e R .

Seção 14 Lógica de Predicados

Na lógica de predicados, as wffs são formadas de predicados, quantificadores, conectivos lógicos e símbolos de grupamento. Neste sistema, uma wff "verdadeira" significa uma wff válida — uma wff que seja válida em qualquer interpretação possível. Desejamos obter os axiomas e as regras de inferência que nos permitam provar todas as wffs válidas, e apenas as válidas, como teoremas.

Axiomas e Regras de Inferência

Wffs predicativas que têm a mesma forma lógica que tautologias são válidas. Por exemplo, no Exemplo 11 mostramos que a wff

$$P(x) \rightarrow [Q(x) \rightarrow P(x)]$$

é válida. Ela tem a forma do axioma $P \rightarrow (Q \rightarrow P)$ no cálculo proposicional, apesar de não ser uma wff proposicional. Se permitirmos as formas dos axiomas e a regra de inferência da lógica proposicional, onde as wffs são wffs predicativas, continuaremos a obter como teoremas todas as wffs predicativas que tenham a forma de tautologias. No entanto, existem algumas wffs predicativas que não têm formas tautológicas, mas ainda assim são válidas devido à sua estrutura e ao significado dos quantificadores universal e existencial (veja o Exemplo 11). Para os termos como teoremas, nosso sistema formal usará novos axiomas e uma nova regra de inferência que lança mão do significado dos quantificadores. Os axiomas para a lógica de predicados são mostrados a seguir, onde P, Q e R representam wffs predicativas:

Axiomas para a Lógica de Predicados

1. $P \rightarrow (Q \rightarrow P)$
2. $[P \rightarrow (Q \rightarrow R)] \rightarrow [(P \rightarrow Q) \rightarrow (P \rightarrow R)]$
3. $(Q' \rightarrow P') \rightarrow (P \rightarrow Q)$
4. $(\forall x)[P(x) \rightarrow Q(x)] \rightarrow [(\forall x)P(x) \rightarrow (\forall x)[Q(x)]]$
5. $(\forall x) P(x) \rightarrow P(x)$ ou $(\forall x)P(x) \rightarrow P(a)$ onde a é uma constante
6. $(\exists x)P(x) \rightarrow P(t)$ onde t é uma constante ou nome de uma variável ainda não usada no decorrer da demonstração.
7. $P(x) \rightarrow (\exists x)P(x)$ ou $P(a) \rightarrow (\exists x)P(x)$ onde a é uma constante e x não ocorre em $P(a)$
8. $[(\exists x)P(x)]' \leftrightarrow (\forall x)[P(x)]'$

Aqui a notação $P(x)$ indica que x ocorre na wff, mas outras variáveis também podem ocorrer. Portanto,

$$(\forall x)(\exists y)S(x, y) \rightarrow (\exists y)S(a, y)$$

onde a é uma constante, é uma instância do Axioma 5 (tome $P(x)$ como sendo $(\exists y)S(x, y)$).

Como na lógica proposicional, é óbvio que estes axiomas particulares poderiam ser usados, mas podemos ver que eles são intuitivamente válidos. O Axioma 4 diz que se todos os elementos do domínio que tiverem a propriedade P também tiverem a propriedade Q , e se todos os elementos do domínio, de fato, tiverem a propriedade P , então todos os elementos do domínio têm a propriedade Q . O Axioma 5 diz que se uma propriedade for verdadeira para todos os elementos do domínio, ela será verdadeira para um x arbitrário ou uma constante a . O Axioma 6 diz que se existe um objeto para o qual a propriedade P é verdadeira, podemos dar um nome a este objeto; no entanto, este nome deve ser arbitrário mas diferente de qualquer outro que já tenhamos usado na seqüência da demonstração. (Esta necessidade faz com que queiramos usar o Axioma 6 o quanto antes possível na demonstração a fim de que os outros axiomas não tenham essas restrições.) O Axioma 7 diz que se P for verdadeira para um valor particular, então há algum membro do domínio para o qual ela é verdadeira. O Axioma 8 confirma nossa compreensão intuitiva do significado dos quantificadores universal e

existencial; se for falso que algum elemento do domínio tem a propriedade P , então todo elemento do domínio não terá a propriedade P e vice-versa.

Os Axiomas 5, 6 e 7 podem ser usados em demonstrações, junto com o modus ponens, para remover quantificadores universais, quantificadores existenciais e incluir quantificadores existenciais, respectivamente. Uma nova regra de inferência nos permite inserir quantificadores universais, mas apenas sob devidas circunstâncias. As regras de inferência para a lógica de predicados são:

Regras de Inferência para a Lógica de Predicados

1. **Modus ponens:** Q pode ser inferida de P e $P \rightarrow Q$.
2. **Generalização:** $(\forall x)Q$ pode ser inferida de Q desde que (a) Q não tenha sido deduzida de qualquer hipótese na qual x seja uma variável livre e (b) Q não tenha sido deduzida pelo uso do Axioma 6 de uma wff da forma $(\exists y)Q(y)$ na qual x seja uma variável livre.

A necessidade das restrições (a) e (b) serão discutidas rapidamente. Antes, porém, vejamos como usar esses axiomas e as regras de inferências em demonstrações.

EXEMPLO 20 Com o uso da lógica de predicados, prove o teorema

$$(\forall x)[P(x) \wedge Q(x)] \rightarrow (\forall x)P(x) \wedge (\forall x)Q(x)$$

No Exemplo 1 l(c) vimos que esta wff é válida, portanto, como todas as wffs válidas são teoremas, somos capazes de achar uma demonstração. Como de costume, a hipótese é nosso ponto de partida.

1. $(\forall x)[P(x) \wedge Q(x)]$ (hipótese)

A tese $(\forall x)P(x) \wedge (\forall x)Q(x)$ será alcançada se $(\forall x)P(x)$ e $(\forall x)Q(x)$ forem passos anteriores da seqüência. A estratégia de ataque geral deveria ser tirar o quantificador universal que aparece no passo 1, o qual fornece o acesso a $P(x)$ e $Q(x)$ e então inserir o quantificador universal separadamente para cada uma das duas wffs, usando a generalização. Uma demonstração é:

1. $(\forall x)[P(x) \wedge Q(x)]$ (hipótese)
2. $P(x) \wedge Q(x)$ (1, Axioma 5, modus ponens)
3. $P(x)$ (2, tautologia $A \wedge B \rightarrow A$, modus ponens)
4. $Q(x)$ (2, tautologia $A \wedge B \rightarrow B$, modus ponens)
5. $(\forall x)P(x)$ (3, generalização)
6. $(\forall x)Q(x)$ (4, generalização)
7. $(\forall x)P(x) \wedge (\forall x)Q(x)$ (5, 6, $A \wedge B$ pode ser deduzida de A e B)

Como mostrado no passo 5, a generalização foi aplicada a $P(x)$, que foi deduzida de $(\forall x)[P(x) \wedge Q(x)]$. Como x não é livre em $(\forall x)[P(x) \wedge Q(x)]$, a condição (a) da generalização é satisfeita. A condição (b) não se aplica. O passo 6 também é uma aplicação válida da generalização. •

PRÁTICA 18 Use a lógica de predicados para provar o teorema

$$(\forall x)P(x) \rightarrow (\exists x)P(x)$$

A lógica de predicados, assim como a lógica proposicional, pode-se mostrar, é completa é correta — toda wff válida é um teorema e todo teorema é uma wff válida. Este sistema de axiomas e regras de inferências permite que exatamente as wffs corretas sejam provadas, mas outro conjunto de axiomas e de regras de inferência também poderia ser usado. Na verdade, mesmo neste sistema, os Axiomas 6 e 7 não são realmente necessários; poderíamos simplesmente ter definido o quantificador existencial em função do quantificador universal (a essência do Axioma 8) e então ter provado os Axiomas 6 e 7. No entanto, a inclusão desses axiomas simplifica nosso trabalho.

As restrições usadas na generalização, no entanto, são necessárias. Sem a restrição (a), a seqüência

1. $P(x)$ (hipótese)
2. $(\forall x)P(x)$ (1, generalização incorreta)

poderia ser uma prova da wff $P(x) \rightarrow (\forall x)P(x)$, que não é uma wff válida. O elemento x do domínio pode ter a propriedade P , mas isto não significa que cada elemento do domínio tenha a propriedade P . Sem a restrição (b) a seqüência

1. $(\forall x)(\exists y)Q(x, y)$ (hipótese)
2. $(\exists y)Q(x, y)$ (1, Axioma 5, modus ponens)
3. $Q(x, y)$ (2, Axioma 6, modus ponens)
4. $(\forall x)Q(x, t)$ (3, generalização incorreta)

seria uma prova da wff $(\forall x)(\exists y)Q(x, y) \rightarrow (\forall x)Q(x, t)$, que também não é uma wff válida. Por exemplo, a interpretação onde o domínio consiste em inteiros e $Q(x, y)$ significa que $x + y = 0$, então é verdade que para cada inteiro x existe um inteiro y (o simétrico de x) tal que $x + y = 0$. No entanto, se t é um elemento fixo particular do domínio, então não é verdade que a soma do mesmo inteiro t a qualquer x resultará em zero.

Pode ser mostrado que se $P \leftrightarrow Q$ for válida, então Q pode ser substituída por P dentro de uma expressão em uma dedução ou seqüência de demonstração. O uso desta expressão doravante simplifica as provas. (Mencionamos uma idéia desta prova antes como uma forma de eliminar os conectivos de disjunção e conjunção.)

EXEMPLO 21 A wff

$$(\forall x)[P(x) \vee Q(x)] \rightarrow (\exists x)P(x) \vee (\forall x)Q(x)$$

é um teorema da lógica de predicados?

Aqui precisamos, antes de mais nada, refletir se a wff parece ou não válida. Se nos parecer válida, tentaremos achar uma prova para ela; se não, tentaremos achar uma interpretação na qual ela seja falsa. Esta wff diz que se todo elemento do domínio tiver a propriedade P ou a propriedade Q , então pelo menos um elemento do domínio tem a propriedade P ou todos elementos do domínio têm a propriedade Q . Isto parece bem razoável, portanto procuraremos uma prova. Os dois primeiros passos na seqüência da demonstração incluem a hipótese e a reescreve em uma forma mais útil para se trabalhar.

1. $(\forall x)[P(x) \vee Q(x)]$ (hipótese)
2. $(\forall x)[(P(x))' \rightarrow Q(x)]$ (substituição de $A \vee B \leftrightarrow A' \rightarrow B$ em 1)

A tese consiste em wffs separadas para $P(x)$ e $Q(x)$, cada qual com seus próprios quantificadores; o Axioma 4 nos permite quebrar a linha 2 em duas wffs diferentes.

3. $(\forall x)[P(x)]' \rightarrow (\forall x)Q(x)$ (2, Axioma 4, modus ponens)

O lado esquerdo do passo 3 sugere que ele pode ser útil para usar o Axioma 8 em seguida.

4. $(\forall x)[P(x)]' \leftrightarrow [(\exists x)P(x)]'$ (Axioma 8)
5. $[(\exists x)P(x)]' \rightarrow (\forall x)Q(x)$ (substituição de 4 em 3)

O passo 5 está intimamente relacionado ao que desejamos. A seqüência da demonstração é:

1. $(\forall x)[P(x) \vee Q(x)]$ (hipótese)
2. $(\forall x)[(P(x))' \rightarrow Q(x)]$ (substituição de $A \vee B \leftrightarrow A' \rightarrow B$ em 1)
3. $(\forall x)[P(x)]' \rightarrow (\forall x)Q(x)$ (2, Axioma 4, modus ponens)
4. $(\forall x)[P(x)]' \leftrightarrow [(\exists x)P(x)]'$ (Axioma 8)
5. $[(\exists x)P(x)]' \rightarrow (\forall x)Q(x)$ (Substituição de 4 em 3)
6. $(\exists x)P(x) \vee (\forall x)Q(x)$ (5, tautologia $(A' \rightarrow B) \rightarrow A \vee B$, modus ponens)

O método de dedução nos permite incluir hipóteses "temporárias" ao longo da demonstração, como podemos ver no próximo exemplo.

EXEMPLO 22 A wff

$$[P(x) \rightarrow (\forall y)Q(x, y)] \rightarrow (\forall y)[P(x) \rightarrow Q(x, y)]$$

é um teorema. Na demonstração a seguir, $P(x)$ é introduzida no passo 2 como uma hipótese temporária que nos permite deduzir $Q(x, y)$ no passo 4. O passo 5 apenas atesta a dependência de $Q(x, y)$ da hipótese temporária; e, naturalmente, toda a wff do passo 5, $P(x) \rightarrow Q(x, y)$ é dependente da hipótese do passo 1. Como y não

é uma variável livre no passo 1, a condição (a) da regra de generalização não é violada no passo 6, e a condição (b) não se aplica.

- | | |
|--|-----------------------------|
| 1. $P(x) \rightarrow (\forall y)Q(x, y)$ | (hipótese) |
| 2. $P(x)$ | (hipótese temporária) |
| 3. $(\forall y)Q(x, y)$ | (1, 2, modus ponens) |
| 4. $Q(x, y)$ | (3, Axioma 5, modus ponens) |
| 5. $P(x) \rightarrow Q(x, y)$ | (4 deduzido do 2) |
| 6. $(\forall y)[P(x) \rightarrow Q(x, y)]$ | (5, generalização) |

PRÁTICA 19 Usando a lógica de predicados, prove o teorema

$$(\forall y)[P(x) \rightarrow Q(x, y)] \rightarrow [P(x) \rightarrow (\forall y)Q(x, y)]$$

O Exemplo 22 e a Prática 19 mostraram que a wff

$$(\forall y)[P(x) \rightarrow Q(x, y)] \leftrightarrow [P(x) \rightarrow (\forall y)Q(x, y)]$$

é válida. Isto significa que o quantificador universal pode ser "negligenciado" para as subwffs que não contêm a variável quantificada. Vale ainda um resultado semelhante para o quantificador existencial. Como um resultado particular, existem duas ou mais formas de se representar sentenças da língua portuguesa como wffs predicativas, como nos Exercícios 7 a 9 da Seção 1.2.

Argumentos Válidos

Para provar a validade de um argumento que contenha sentenças quantificadas, procedemos quase como antes. Convertamos o argumento em forma simbólica, e mostramos que a tese pode ser obtida da hipótese. No entanto, desta vez, trabalharemos com a lógica de predicados, ao invés da lógica proposicional. Um argumento $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow Q$ é válido quando for um teorema, que também o torna uma wff válida.

EXEMPLO 23 Mostre que o argumento a seguir é válido: "Todo microcomputador tem uma porta serial. Alguns microcomputadores têm porta paralela. Portanto alguns computadores têm ambas as portas serial e paralela". Usando

$M(x)$: x é um microcomputador.
 $S(x)$: x tem porta serial.
 $P(x)$: x tem porta paralela.

o argumento é

$$(\forall x)[M(x) \rightarrow S(x)] \wedge (\exists x)[M(x) \wedge P(x)] \rightarrow (\exists x)[M(x) \wedge S(x) \wedge P(x)]$$

Perceba que se tentarmos simbolizar este argumento na lógica proposicional, obteremos $A \wedge B \rightarrow C$, que não é um argumento válido. A lógica proposicional é, simplesmente, não expressiva o suficiente para conter as relações entre as partes deste argumento que o tornam válido.

Uma demonstração é

- | | |
|--|--|
| 1. $(\forall x)[M(x) \rightarrow S(x)]$ | (hipótese) |
| 2. $(\exists x)[M(x) \wedge P(x)]$ | (hipótese) |
| 3. $M(a) \wedge P(a)$ | (2, Axioma 6, modus ponens) |
| 4. $M(a) \rightarrow S(a)$ | (1, Axioma 5, modus ponens) |
| 5. $M(a)$ | (3, tautologia $A \wedge B \rightarrow B$, modus ponens) |
| 6. $S(a)$ | (4, 5, modus ponens) |
| 7. $M(a) \wedge P(a) \wedge S(a)$ | (3, 6, $A \wedge B$ pode ser deduzida de A e B) |
| 8. $M(a) \wedge S(a) \wedge P(a)$ | (substituição de $A \wedge B \leftrightarrow B \wedge A$ em 7) |
| 9. $(\exists x)[M(x) \wedge S(x) \wedge P(x)]$ | (8, Axioma 7, modus ponens) |

Mais uma vez, é a *forma* do argumento que vale e não seu conteúdo. Perceba que na demonstração usamos o Axioma 6 no passo 3 para inserir a nova constante a , e então usamos o Axioma 5 no passo 4 com a mesma constante. Estes dois passos não podem ser trocados pois o Axioma 6 pode introduzir novos nomes apenas se eles ainda não tiverem sido usados na demonstração. Eis por que o Axioma 6 deve ser usado o mais cedo possível na demonstração.

PRÁTICA 20

Mostre que o argumento a seguir é válido: "Todas as músicas de rock são barulhentas. Existem algumas músicas de rock, logo existem algumas músicas barulhentas." Use os predicados $R(x)$ e $B(x)$.

Revisão da Seção 14

Técnicas

- Demonstração de teoremas na lógica de predicados
- Uso da lógica de predicados para provar a validade de um argumento na língua portuguesa

Idéia Principal

Todo teorema da lógica de predicados é uma wff válida, e toda wff válida é um teorema.

Exercícios 14

1. Justifique cada um dos passos na demonstração a seguir

$$(\exists x)[P(x) \rightarrow Q(x)] \rightarrow [(\forall x)P(x) \rightarrow (\exists x)Q(x)]$$

1. $(\exists x)[P(x) \rightarrow Q(x)]$
2. $P(a) \rightarrow Q(a)$
3. $(\forall x)P(x)$
4. $P(a)$
5. $Q(a)$
6. $(\exists x)Q(x)$

- ★2. Considere a wff

$$(\exists x)P(x) \wedge (\exists x)Q(x) \rightarrow (\exists x)[P(x) \wedge Q(x)]$$

- a. Encontre uma interpretação que demonstre que esta wff não é válida.
- b. Encontre falha na seguinte "demonstração" desta wff.

1. $(\exists x)P(x)$ (hipótese)
2. $P(a)$ (1, Axioma 6, modus ponens)
3. $(\exists x)Q(x)$ (hipótese)
4. $Q(a)$ (3, Axioma 6, modus ponens)
5. $P(a) \wedge Q(a)$ ($A \wedge B$ pode ser deduzida de A e B)
6. $(\exists x)[P(x) \wedge Q(x)]$ (5, Axioma 7, modus ponens)

3. Considere a wff $(\forall y)(\exists x)Q(x, y) \rightarrow (\exists x)(\forall y)Q(x, y)$.

- a. Encontre uma interpretação que demonstre que esta wff não é válida.
- b. Encontre a falha na seguinte "demonstração" desta wff.

1. $(\forall y)(\exists x)Q(x, y)$ (hipótese)
2. $(\exists x)Q(x, y)$ (1, Axioma 5, modus ponens)
3. $Q(a, y)$ (2, Axioma 6, modus ponens)
4. $(\forall y)Q(a, y)$ (3, generalização)
5. $(\exists x)(\forall y)Q(x, y)$ (4, Axioma 7, modus ponens)

Nos Exercícios 4 a 7 demonstre que cada wff é um teorema da lógica de predicados.

- ★4. $(\forall x)P(x) \rightarrow (\forall x)[P(x) \vee Q(x)]$

5. $(\forall x)P(x) \wedge (\exists x)Q(x) \rightarrow (\exists x)[P(x) \wedge Q(x)]$

6. $(\exists x)(\exists y)P(x, y) \rightarrow (\exists y)(\exists x)P(x, y)$

7. $(\forall x)(\forall y)Q(x, y) \rightarrow (\forall y)(\forall x)Q(x, y)$

Nos Exercícios 8 a 15, prove que as wffs são teoremas da lógica de predicados ou apresente uma interpretação para provar que não são válidas.

- ★8. $(\exists x)[A(x) \wedge B(x)] \rightarrow (\exists x)A(x) \wedge (\exists x)B(x)$
- 9. $(\exists x)[R(x) \vee S(x)] \rightarrow (\exists x)R(x) \vee (\exists x)S(x)$
- 10. $[(\forall x)P(x) \rightarrow (\forall x)Q(x)] \rightarrow (\forall x)[P(x) \rightarrow Q(x)]$
- ★11. $(\exists x)(\forall y)Q(x, y) \rightarrow (\forall y)(\exists x)Q(x, y)$
- 12. $(\forall x)P(x) \vee (\exists x)Q(x) \rightarrow (\forall x)[P(x) \vee Q(x)]$
- 13. $(\forall x)[A(x) \rightarrow B(x)] \rightarrow [(\exists x)A(x) \rightarrow (\exists x)B(x)]$
- 14. $(\forall y)[Q(x, y) \rightarrow P(x)] \rightarrow [(\exists y)Q(x, y) \rightarrow P(x)]$
- ★15. $[P(x) \rightarrow (\exists y)Q(x, y)] \rightarrow (\exists y)[P(x) \rightarrow Q(x, y)]$

Usando a lógica de predicados, prove que os argumentos dos Exercícios 16 a 20 são válidos. Use os símbolos predicados mostrados.

- 16. Há um astrônomo que não é míope. Todo mundo que usa óculos é míope. Portanto, todo mundo ou usa óculos ou usa lentes de contato. Portanto, algum astrônomo usa lentes de contato. $(A(x), M(x), O(x), L(x))$
- ★17. Todo membro da mesa vem da indústria ou do governo. Todos do governo que são advogados são a favor da moção. John não é da indústria, mas ele não é advogado. Portanto, se John for um membro da mesa, ele é a favor da moção. $(M(x), I(x), G(x), A(x), F(x), j)$
- 18. Existem algumas estrelas de cinema que são mais ricas que as outras. Todo mundo que é mais rico que os outros também paga mais impostos que os outros. Portanto, existe uma estrela de cinema que paga mais impostos que os outros. $(E(x), R(x, y), I(x, y))$
- 19. Todo estudante da Ciência da Computação trabalha mais que alguém e todo mundo que trabalha mais que alguém também dorme menos que esta pessoa. Maria é uma estudante da Ciência da Computação. Portanto Maria dorme menos que outra pessoa. $(E(x), T(x, y), D(x, y), m)$
- 20. Todo embaixador fala apenas com diplomatas e algum embaixador fala com alguém, portanto existe um diplomata. $(E(x), F(x, y), D(x))$
- 21. Prove que

$$[(\forall x)P(x)]' \leftrightarrow (\exists x)[P(x)]'$$

é válida. {Dica: ao invés de uma seqüência de demonstração, use o Axioma 8 e substitua as expressões equivalentes.}

- 22. A equivalência do Exercício 21 diz que se for falso que todo elemento do domínio tem a propriedade P , então algum elemento do domínio também deixa de ter a propriedade P e vice-versa. O elemento que não tem a propriedade P é chamado de **contra-exemplo** da sentença que todo elemento tem a propriedade P . Portanto, um contra-exemplo da sentença

$$(\forall x)(x \text{ é ímpar})$$

no domínio dos inteiros é o número 10, um inteiro par. (Naturalmente, existem diversos outros contra-exemplos para esta sentença). Encontre contra-exemplos no domínio dos inteiros para as seguintes sentenças. (Um inteiro $x > 1$ é **primo** se seus únicos divisores forem 1 e x .)

- a. $(\forall x)(x \text{ é negativo})$
- b. $(\forall x)(x \text{ é a soma dos inteiros pares})$
- c. $(\forall x)(x \text{ é primo} \rightarrow x \text{ é ímpar})$
- d. $(\forall x)(x \text{ é primo} \rightarrow (-1)^x = -1)$
- e. $(\forall x)(x \text{ é primo} \rightarrow 2^x - 1 \text{ é primo})$

Seção 15 Programação Lógica e Prova de Correção

A lógica de predicados, outrora objeto de interesse apenas de matemáticos e filósofos, tem diversas aplicações de importância na Ciência da Computação. Duas delas serão discutidas nesta seção.

Programação Lógica

Na lógica de predicados, usamos regras de inferência para chegarmos a teses a partir das hipóteses. Se uma tese tiver sido demonstrada como consequência de determinada hipótese, então, em uma interpretação na qual a hipótese seja verdadeira, a tese também será verdadeira. A linguagem de programação Prolog, que significa *programming in logic*, também ajuda a chegar a teses a partir das hipóteses. A linguagem inclui predicados, conectivos lógicos e regras de inferência. Ela permite a descrição de uma interpretação, ou melhor, de hipóteses verdadeiras em uma interpretação.

As linguagens de programação com as quais você provavelmente já tem familiaridade, tal como Pascal, são conhecidas como **linguagens procedurais**. A maior parte dos programas escritos em linguagens procedurais destinam-se a resolver o problema à mão. O programador, portanto, diz ao computador como resolver o problema. Prolog, no entanto, é uma **linguagem declarativa** (também chamada de **linguagem descritiva**). Um programa Prolog consiste em declarações ou descrições sobre uma interpretação, isto é, quais as hipóteses que são verdadeiras em uma interpretação. O conjunto de declarações é também chamado de base de dados do Prolog. Para determinar se uma dada tese, posta na forma de uma pergunta pelo usuário, é ou não verdadeira para a interpretação, Prolog usa sua base de dados e aplica suas regras de inferências (sem a necessidade de qualquer instrução por parte do programador).

Itens em uma base de dados do Prolog podem ter duas formas, conhecidas em Prolog como *fatos* e *regras* . (Porém as regras do Prolog são apenas outro tipo de fatos, e não devem ser confundidas com as regras de inferência.)

Os **fatos do Prolog** permitem definir predicados. Por exemplo, suponhamos que desejemos criar um programa Prolog que descreva as cadeias alimentares em uma determinada região ecológica. Devemos começar com um predicado binário *come*. Então descreveremos este predicado fornecendo os pares de elementos no domínio que tornam *come* verdadeiro. Portanto, teríamos os fatos

```
come(urso, peixe)
come(urso, raposa)
come(veado, mato)
```

em nossa base de dados. (Os detalhes exatos dos comandos Prolog variam de implementação para implementação, portanto daremos aqui apenas o espírito da linguagem através do uso de um pseudocódigo semelhante ao Prolog.) Neste exemplo, "urso", "peixe", "raposa", "veado" e "mato" são constantes porque representam elementos específicos do domínio. Como o domínio propriamente dito não é especificado, exceto na declaração dos predicados, neste ponto podemos fazer inferir que o domínio consiste em "urso", "peixe", "raposa", "veado" e "mato". É saudável que o usuário mantenha um entendimento e faça um uso consistente dos predicados em um programa Prolog. Portanto,

```
come(urso, peixe)
```

pode ser usado tanto para representar o fato de que ursos comem peixes ou de que peixes comem ursos! Arbitramos a convenção de que *come(x, y)* significa "x come y".

Podemos incluir descrições de dois predicados unários, *animal* e *planta* para a base de dados incluindo os fatos

```
animal(urso)
animal(peixe)
animal(raposa)
animal(veado)
planta(mato)
```

De posse deste programa Prolog (base de dados), podemos fazer algumas perguntas simples.

EXEMPLO 24 A pergunta

```
is(animal(urso))
```

simplesmente pergunta se o fato *animal(urso)* está na base de dados. Como este fato está na base de dados, o Prolog responderá à pergunta com yes. Outros diálogos com o Prolog poderiam incluir

```
is(come(veado,mato))
```

```
yes
```

```
is(come(urso, coelho))
```

```
no
```

Perguntas podem incluir variáveis, como mostrado no próximo exemplo.

EXEMPLO 25 A pergunta

```
which(x: come(urso,x))
```

produz

```
peixe
```

```
raposa
```

como resposta. O Prolog respondeu à pergunta procurando em sua base de dados por todos os fatos que se ajustassem ao padrão *come(urso, x)*, onde *x* é uma variável. A resposta "peixe" é dada antes porque as regras são percorridas da primeira para a última.

As perguntas podem conter os conectivos lógicos **and**, **or** e **not**.

PRÁTICA 21 Dada a base de dados

```
come(urso, peixe)
```

```
come(urso, raposa)
```

```
come(veado, mato)
```

```
animal(urso)
```

```
animal(peixe)
```

```
animal(raposa)
```

```
animal(veado)
```

```
planta(mato)
```

qual será a resposta do Prolog para a pergunta

```
which(x: come(x, y) and planta(y))
```

O segundo tipo de item em uma base de dados Prolog é uma **regra Prolog**. Uma regra é uma descrição de um predicado através de uma implicação. Por exemplo, poderíamos usar uma regra para definir um predicado para *presa*:

```
presa(x) if come(y, x) and animal(x)
```

Isto indica que *x* é uma presa se for animal que é comido. Se incluirmos esta regra a nossa base de dados, então a resposta à pergunta

```
which(x: presa(x))
```

teremos a resposta

```
peixe
```

```
raposa
```

Classes de Horn e Resolução

Como os fatos e as regras do Prolog se relacionam com o formalismo da lógica de predicados? Podemos descrever os fatos em nossa base de dados pelas wffs

$C(u, p)$
 $C(u, r)$
 $C(v, m)$
 $A(u)$
 $A(p)$
 $A(r)$
 $A(v)$
 $P(m)$

e a regra pela wff

$$C(y, x) \wedge A(x) \rightarrow Pr(x)$$

Quantificadores universais são partes explícitas da regra que aparecem em um programa Prolog, mas o Prolog trata a regra como sendo quantificada universalmente,

$$(\forall y)(\forall x)[C(y, x) \wedge A(x) \rightarrow Pr(x)]$$

e usa repetidamente nosso Axioma 5 da lógica de predicados para tirar o quantificador universal e permitir que as variáveis assumam, a seu tempo, cada valor no domínio.

Tanto os fatos quanto as regras são exemplos de **Cláusulas de Horn**. Uma cláusula de Horn é uma wff composta por predicados ou as negações dos predicados (com variáveis ou constantes como argumentos) unidos por disjunções, onde no máximo um predicado não é negado. Portanto o fato

$$C(d, g)$$

é um exemplo de cláusula de Horn porque contém um único predicado não-negado. A wff

$$[C(y, x)]' \vee [A(x)]' \vee Pr(x)$$

é um exemplo de uma cláusula de Horn porque consiste em três predicados unidos pela disjunção onde $Pr(x)$ não está negado. Pela lei de De Morgan, ele é equivalente a

$$[C(y, x) \wedge A(x)]' \vee Pr(x)$$

que, por sua vez, é equivalente a

$$C(y, x) \wedge A(x) \rightarrow Pr(x)$$

e, portanto, representa uma regra em nosso programa Prolog.

A regra de inferência usada pelo Prolog é chamada **resolução**. Duas cláusulas de Horn em uma base de dados Prolog são resolvidas em uma nova cláusula de Horn se uma delas contiver um predicado não-negado que corresponda a um predicado negado na outra cláusula. A nova cláusula elimina o termo de correspondência e fica, então, disponível para uso em respostas às perguntas. Por exemplo,

$$\begin{array}{l}
 A(a) \\
 [A(a)]' \vee B(b)
 \end{array}$$

é resolvida para $B(b)$. Isto mostra que o Prolog considera

$$(A(a) \wedge [A(a) \rightarrow B(b)]) \rightarrow B(b)$$

como um teorema, o que é uma simples aplicação do modus ponens. Portanto a regra de inferência do Prolog inclui o modus ponens como um caso especial.

Na aplicação da regra de resolução, as variáveis são consideradas como "correspondentes" a qualquer símbolo constante. (Isto é uma aplicação repetida do Axioma 5.) Em qualquer nova cláusula resultante, as variáveis são substituídas por suas constantes associadas de uma maneira consistente. Portanto, em resposta à pergunta "qual x é uma presa", o Prolog procura, na base de dados, por uma regra com o predicado desejado $Pr(x)$ como o conseqüente. Ele encontra

$$[C(y, x)]' \vee [A(x)]' \vee Pr(x)$$

Ele então prossegue pela base de dados na busca de outras cláusulas que possam ser resolvidas com esta cláusula. A primeira dessas cláusulas é o fato $E(b, fí)$. Estas duas cláusulas se resolvem em

$$[A(p)]' \vee Pr(p)$$

(Perceba que a constante p substituiu x em todos os lugares.) Usando esta nova cláusula, ela poderá ser resolvida com o fato $A(p)$ para concluir $Pr(p)$. Tendo alcançado todas as conclusões possíveis da resolução do fato $C(u, p)$, Prolog refaz o processo para procurar por outra cláusula a resolver com a aplicação da cláusula da regra; desta vez ele encontrará $C(u, r)$.

Como um exemplo mais complexo de resolução, suponhamos que incluímos a regra

$$caçado(x) \text{ if } presa(x)$$

à base de dados. Esta regra, na forma simbólica, é

$$[Pr(x)]' \vee H(x)$$

Que é resolvida com a regra de definição de *presa*,

$$[C(x, y)]' \vee [A(x)]' \vee Pr(x)$$

para chegar à nova regra

$$[C(y, x)]' \vee [A(x)]' \vee H(x)$$

A pergunta

$$\textbf{which}(x:caçado(x))$$

usará esta nova regra para concluir

peixe
raposa

EXEMPLO 26 Suponha que um programa Prolog contenha as seguintes entradas:

```
come(urso, peixe)
come(peixe, peixinho)
come(peixinho, alga)
come(quati, peixe)
come(urso, quati)
come(urso, raposa)
come(raposa, coelho)
come(coelho, mato)
come(urso, veado)
come(veado, mato)
come(gato-selvagem, veado)
```

```
animal(urso)
animal(peixe)
animal(peixinho)
animal(quati)
animal(raposa)
animal(coelho)
animal(veado)
animal(gato-selvagem)
```

```
planta(mato)
planta(alga)
```

```
presa(x) if come(y, x) and animal(x)
```

Então realiza-se o seguinte diálogo com o Prolog:

```
is(animal(coelho))

yes

is(come(gato-selvagem, mato))

no

which(x:come(x, peixe))

urso
quati

which(x, y:come(x, y) and planta(y))

peixinho alga
coelho mato
veado mato

which(x:presa(x))

peixe
peixinho
peixe
quati
raposa
coelho
veado
veado
```

Perceba que peixe é listado duas vezes como satisfazendo a última pergunta por que o peixe é comido pelo urso (fato 1) e pelo quati (fato 3). Analogamente, o veado é comido pelo urso e pelo gato-selvagem. •

PRÁTICA 22

- Formule uma regra Prolog que defina o predicado *predador*.
- Incluindo esta regra à base de dados do Exemplo 26, qual seria a resposta à pergunta

```
which(x:predador(x))
```

•

Recursão

As regras do Prolog são implicações. Seus antecedentes podem depender de fatos, como em

```
presa(x) if come (y, x) and animal{x}
```

ou em outras regras como em

```
caçado(x) if presa(x)
```

O antecedente de uma regra pode também depender da mesma regra, caso no qual a regra é definida em função dela mesma. Uma definição em que o item sendo definido é, ele próprio, parte da definição é chamada de definição recursiva.

Como um exemplo, vamos supor que desejamos usar a base de dados ecológica do Exemplo 26 para estudar a cadeia alimentar. Podemos definir uma relação binária *na-cadeia-alimentar*(x, y) que significa "y está na cadeia alimentar de x". Isto, por sua vez, pode significar duas coisas:

- x come y diretamente

ou

- x come algum animal que come algum animal que come algum animal ... que come y.

O caso 2 pode ser reescrito como:

2'. x come z e y está na cadeia alimentar de z .

O caso 1 simplesmente verifica fatos existentes, mas sem (2'), *na-cadeia-alimentar* não significa nada além de *come*. Por outro lado, (2') sem (1) nos coloca na busca infinita de algum animal que coma algum animal que coma algum animal... sem nos dizer quando parar. Definições recursivas sempre precisam de um ponto de parada que consista em informações específicas.

A regra Prolog para *na-cadeia-alimentar* incorpora (1) e (2') é:

na-cadeia-alimentar (x, y) if *come*(x, y)

na-cadeia-alimentar (x, y) if *come*(x, z) **and** *na-cadeia-alimentar*(z, y)

Esta é uma **regra recursiva**, pois define o predicado de *na-cadeia-alimentar* em termos de *na-cadeia-alimentar*.

EXEMPLO 27 Após a inclusão da regra *na-cadeia-alimentar* à base de dados do Exemplo 26, a seguinte pergunta é feita:

which(y : *na-cadeia-alimentar*(urso, y))

A resposta é a seguinte (os números foram usados para fins de referência):

1. peixe
2. quati
3. raposa
4. veado
5. peixinho
6. alga
7. peixe
8. peixinho
9. alga
10. coelho
11. mato
12. mato

O Prolog simplesmente aplica o caso de

na-cadeia-alimentar(urso, y) if *come*(urso, y)

primeiro, obtendo as respostas 1 a 4 diretamente dos fatos *come*(urso, peixe), *come*(urso, quati) e assim por diante. Passemos ao caso recursivo

na-cadeia-alimentar(urso, y) if *come*(urso, z) **and** *na-cadeia-alimentar*(z, y)

uma correspondência de *come*(urso, z) ocorre com z igual a "peixe". O Prolog então procura todas as soluções para relação *na-cadeia-alimentar*(peixe, y). Usando primeiro o caso simples de *na-cadeia-alimentar*, uma correspondência ocorre com o fato *come*(peixe, peixinho). Este gera a resposta 5, peixinho. Não há outros fatos da forma *come*(peixe, y), portanto a próxima tentativa é o caso recursivo:

na-cadeia-alimentar(peixe, y) if *come*(peixe, z) **and** *na-cadeia-alimentar*(z, y)

Uma correspondência para *come*(peixe, z) ocorre com z igual a "peixinho". O Prolog então procura por todas as soluções para a relação *na-cadeia-alimentar*(peixinho, y). Usando o caso simples de *na-cadeia-alimentar*, uma correspondência pode ocorrer com o fato *come*(peixinho, alga). Isto resulta na resposta 6, alga. Como não há outros fatos na forma *come*(peixinho, y), a próxima coisa a fazer é o caso recursivo

na-cadeia-alimentar(peixinho, y) if *come*(peixinho, z) **and** *na-cadeia-alimentar*(z, y)

Uma correspondência para *come*(peixinho, z) ocorre com z igual a "alga". O Prolog então procura por todas as soluções para a relação *na-cadeia-alimentar*(alga, y). Uma busca em toda base de dados não revela fatos da forma *come*(alga, y) (ou *come*(alga, z)), portanto nem o caso simples nem o caso recursivo de *na-cadeia-alimentar*(alga, y) pode ser seguido deste ponto em diante.

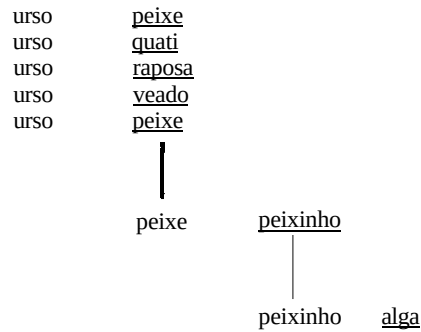


Figura 1.2

A Fig. 1.2 mostra a situação neste ponto. O Prolog encontrou um "beco sem saída" com *na-cadeia-alimentar(alga, y)* e fará um retrocesso no caminho de busca. Como não há outros fatos da forma *come(peixinho, z)*, a busca por soluções de *na-cadeia-alimentar(peixinho, y)* termina. Então, por não haver mais outros fatos na forma *come(peixe, z)*, a busca por soluções de *na-cadeia-alimentar(peixe, y)* também termina. Voltando no caminho de busca, existe outra correspondência para *come(urso, z)* com *z* igual a "quati" que gera outro caminho de busca.

No Exemplo 27, uma vez que o Prolog tenha começado a investigar *na-cadeia-alimentar(peixe, y)*, todas as respostas a perguntas que possam ser obtidas pela exploração deste caminho (respostas 5 e 6) serão geradas antes das demais (respostas 7 a 12). A busca que visa priorizar a exploração até pontos distantes dos caminhos e depois voltar por eles, para então percorrer outros caminhos, é chamada de estratégia de **busca em profundidade**.

PRÁTICA 23

Faça o acompanhamento da execução do programa Prolog do Exemplo 27 e explique por que ocorrem as respostas 7 a 12.

Uma regra recursiva é necessária quando o predicado sendo descrito é passado de um objeto para o outro. O predicado *na-cadeia-alimentar* tem esta propriedade:

$$na-cadeia-alimentar(x, y) \wedge na-cadeia-alimentar(y, z) \rightarrow na-cadeia-alimentar(x, z)$$

Sistemas Especialistas

Muitas aplicações interessantes vêm sendo desenvolvidas, em Prolog e linguagens semelhantes para programação lógica, que reúnem uma base de dados de fatos e regras, e então a usam para gerar conclusões. Tais programas são conhecidos como **sistemas especialistas**, **sistemas baseados no conhecimento** ou **sistemas baseados em regras**. A base de dados em um sistema especialista tenta retratar o conhecimento ("acumulado com a experiência") de um especialista humano em alguma área particular do conhecimento, incluindo os fatos sabidos pelo sujeito e seus métodos de obtenção de conclusões destes fatos. O sistema especialista não apenas simula a ação de um humano, mas pode ser questionado a fim de indicar por que tomou certas decisões ao invés de outras.

Alguns sistemas especialistas vêm sendo construídos a fim de simular diagnósticos de médicos especialistas a partir de sintomas de pacientes, decisões de um gerente de fábrica sobre o controle de válvulas em uma fábrica química baseado nas leituras dos sensores, decisões de um comprador de roupas para uma boutique baseado em pesquisas de mercado, a escolha feita por um consultor ao fazer a especificação da configuração de um sistema de computadores baseado nas necessidades do consumidor, e diversas outras. O desafio de fazer os sistemas especialistas reside em extrair todos os fatos pertinentes e regras do especialista humano.

Prova de Correção

As páginas anteriores mostraram como a lógica de predicados pode ser usada como base para toda uma linguagem de programação. A lógica de predicados é também útil na verificação formal da correção de programas escritos em linguagens de programação mais tradicionais (procedurais).

A **verificação do programa** tenta garantir que um programa de computador está correto. "Correção" tem uma definição mais limitada do que tem no uso do dia-a-dia. Um programa está **correto** se o seu compor-

tamento está de acordo com as especificações. No entanto, isto não necessariamente indica que o programa resolve o problema para o qual foi projetado para resolver; as especificações do programa podem não estar de acordo com ou não prever todos os aspectos das necessidades do cliente. **Validação do programa**, que não discutiremos, tenta garantir que o programa atinja as necessidades originais do cliente. Em um projeto de desenvolvimento de um grande programa, "program V & V" foi considerado tão importante que um grupo de pessoas separadas dos programadores pode ser designado para realizar esta tarefa de validação.

A verificação de programas pode ser abordada informalmente, através de testes do programa ou formalmente, através da *prova de correção*. Os **testes de programa** buscam mostrar que, para valores de entrada particulares produzem valores de saída aceitáveis. A tarefa de testar programas é checar a performance do programa em um largo e representativo conjunto de valores de entrada; em geral, testar para todos os valores de entrada não é possível. A **prova de correção** usa as técnicas da lógica formal para provar que, dada quaisquer variáveis de entradas que satisfaçam determinados predicados ou propriedades, as variáveis de saída produzidas pela execução do programa satisfazem outras propriedades especificadas.

Para distinguir entre a prova de correção e o teste de um programa, considere o seguinte programa para calcular o tamanho c da hipotenusa de um triângulo retângulo, dados os valores positivos a e b de seus catetos. Para provar que o programa está correto devemos estabelecer que sempre que a e b satisfazem os predicados $a > 0$ e $b > 0$, então após a execução do programa, o predicado $a^2 + b^2 = c^2$ é satisfeito. Testar este programa exigiria que tomássemos diversos valores específicos para a e b , computássemos o valor de c e verificássemos se $a^2 + b^2$ é igual a c^2 para cada caso.

Descrevendo a prova de correção mais formalmente, denotemos por X um conjunto arbitrário de valores de entrada de algum programa ou trecho de programa P . O conjunto correspondente de valores-saída Y é produzido a partir de X por quaisquer transformações que P realize nos dados. Denotaremos essas transformações por $Y = P(X)$. Um predicado Q descreve as condições que os valores de entrada devem satisfazer. Por exemplo, se um programa se destina a achar a raiz quadrada de um número positivo, temos um valor de entrada, x , e $Q(x)$ deve ser " $x > 0$ ". Um predicado R que descreve as condições que os valores de saída devem satisfazer. Essas condições sempre envolverão também os valores de entrada; portanto, em nosso caso da raiz quadrada, se y for o único valor de saída, então desejamos que $R(x, y)$ seja " $y^2 = x$ ". O programa P será correto se

$$(\forall x)(Q(x) \rightarrow R[X, P(X)]) \quad (1)$$

Vamos abreviar (1) por

$$\{Q\}P\{R\} \quad (2)$$

Esta notação sugere que Q e R são comentários de programa não-executáveis, mas tenhamos em mente que (2) *representa uma implicação* que precisa ser válida para todo X . A condição Q é chamada de **pré-condição** para o programa P , a condição R é a **pós-condição**.

Ao invés de simplesmente termos um predicado inicial e um predicado final, um programa ou trecho de programa é dividido em comandos individuais s_i com os predicados inseridos antes e depois. Esses predicados são chamados de **asserção** por fazerem exigências sobre o que deve ser verdadeiro a respeito das variáveis de programas nos diversos pontos do mesmo. Portanto, há uma série de asserções, $Q, R_1, R_2, \dots, R_n$. P está provavelmente correto se as seguintes implicações forem válidas:

$$\begin{aligned} &\{Q\}s_0\{R_1\} \\ &\{R_1\}s_1\{R_2\} \\ &\{R_2\}s_2\{R_3\} \\ &\vdots \\ &\{R_{n-1}\}s_{n-1}\{R\} \end{aligned}$$

As asserções intermediárias são normalmente obtidas trabalhando-se da asserção de saída R para trás. Quando as asserções intermediárias tiverem sido decididas, as implicações podem ser provadas usando-se um sistema de lógica formal de axiomas e regras de inferência.

O Axioma da Atribuição

Suponha que o comando s_i seja um comando de atribuição da forma $x := e$, isto é, a variável x recebe o valor de e , onde e é alguma expressão. É aceito como axioma que a implicação

$$\{R_i\}s_i\{R_{i+1}\}$$

é válida se R_i for o predicado R_{i+1} com e substituído em todos os pontos de x . Em forma abreviada, o **axioma da atribuição** é

Axioma da Atribuição

$$\{R_x\}x := e(R)$$

onde $\{R_x\}$ significa “substitui e por x em R ”.

EXEMPLO 28 Se o comando do programa é $x := x - 1$ e a asserção após este comando é $x > 0$, então a asserção antes deste comando deve ser $x - 1 > 0$. Escrevendo a pré-condição e a pós-condição como comentários de programa, o trecho de programa

```
{x - 1 > 0}
x := x - 1
{x > 0}
```

está correto por causa do axioma da atribuição.

Isto realmente funciona? Para todo x , se $x - 1 > 0$ antes do comando ser executado (perceba que isto significa que $x > 1$), então após o comando o valor de x é reduzido a 1, neste caso $x > 0$.

PRÁTICA 24 De acordo com o axioma da atribuição, qual a pré-condição do seguinte trecho de programa? Explique por que funciona. •

```
{pré-condição}
x := x - 2;
{x = y}
```

EXEMPLO 29 Verifique a correção do seguinte trecho de programa para trocar os valores de x e y :

```
temp := x;
x := y;
y := temp;
```

No início deste trecho de programa, x e y têm certos valores. Portanto, devemos expressar a pré-condição como $x = a$ e $y = b$. A pós-condição desejada é, então, $x = b$ e $y = a$. Usando o axioma da atribuição, podemos trabalhar de trás para frente, da pós-condição para a pré-condição para encontrar as asserções anteriores:

```
{y = b, x = a}
temp := x;
{y = b, temp = a}
x := y;
{x = b, temp = a}
y := temp;
{x = b, y = a}
```

A primeira asserção está de acordo com a pré-condição; o axioma da atribuição, aplicado repetidamente, assegura, então, que o trecho de programa está correto. •

PRÁTICA 25 Verifique a correção do seguinte trecho de programa com a pré-condição e pós-condição mostradas:

```
{x = 3}
v := 4;
z := x + y;
{z = 7}
```

Algumas vezes a pré-condição de um trecho de programa é trivialmente verdadeira, como mostrado no próximo exemplo.

EXEMPLO 30 Verifique a correção do seguinte trecho de programa para computar $y = x - 4$.

```
y := x;
y := y - 4;
```

Neste caso a pós-condição desejada é $y = x - 4$. Usando o axioma do fim para o início, a partir da pós-condição, obtemos

```

{ x - 4 = x - 4 }
  y := x;
{ y - 4 = x - 4 }
  y := y - 4;
{ y = x - 4 }

```

A pré-condição é sempre verdadeira; por isso, pelo axioma da atribuição, cada asserção seguinte, incluindo a pós-condição, é verdadeira. •

A Regra Condicional

Um **comando condicional** é um comando de programa da forma

```

if condição  $B$  then
   $P_1$ 
else
   $P_2$ 

```

Quando este comando é executado, uma condição B , que é ou verdadeira ou falsa, é avaliada. Se B for verdadeira, o trecho P_1 do programa é executado, mas se B for falsa, o trecho P_2 do programa é que é executado.

Lembremos que determinar a correção de qualquer comando s_i , do programa envolve provar que a implicação

$$\{Q\} s_i \{R\}$$

é verdadeira, onde Q e R são a pré-condição e a pós-condição, respectivamente, para o comando. Se s_i é um comando condicional, então uma *regra condicional de inferência* pode ser usada para provar esta implicação.

Regra Condicional de Inferência

A implicação

$$\{Q\} s_i \{R\}$$

onde s_i é o comando condicional

```

if condição  $B$  then
   $P_1$ 
else
   $P_2$ 

```

pode ser inferida de

$$\{Q \wedge B\} P_1 \{R\}$$

e

$$\{Q \wedge B'\} P_2 \{R\}$$

EXEMPLO 31 Verifique a correção do trecho de programa abaixo com a pré-condição e pós-condição mostradas:

```

{ n = 5 }
  if n >= 10 then
    y := 100
  else
    y := n + 1;
{ y = 6 }

```

Aqui a pré-condição é $n = 5$, e a condição B a ser avaliada é $n \geq 10$. A fim de aplicar a regra condicional, devemos provar que

$$\{n = 5 \wedge n \geq 10\} y := 100 \{y = 6\}$$

é válida. Esta implicação é verdadeira porque seu antecedente, $n = 5$ e $n \geq 10$, é falso. Precisamos ainda mostrar que

$$\{n = 5 \text{ e } n < 10\} \ y := n + 1 \ \{y = 6\}$$

é válida. Trabalhando de trás para frente, temos

$$\begin{aligned} &\{n + 1 = 6 \text{ ou } n = 5\} \\ &y := n + 1; \end{aligned}$$

Portanto

$$\{n = 5\} y := n + 1 \ \{y = 6\}$$

é verdadeira pelo axioma da atribuição e portanto

$$\{n = 5 \text{ e } n < 10\} \ y := n + 1 \ \{y = 6\}$$

é verdadeira. A regra condicional nos permite concluir que o trecho do programa está correto. •

PRÁTICA 26 Verifique a correção do trecho de programa abaixo com a pré-condição e a pós-condição mostradas:

```
{x = 4}
  if x < 5 then
    y := x - 1
  else
    y := 7;
{y = 3}
```

•

EXEMPLO 32 Verifique a correção do seguinte trecho de programa para computar $\max(x, y)$, o maior dentre dois valores x e y .

```
if x >= y then
  max := x
else
  max := y;
```

A pós-condição desejada reflete a definição de um máximo ($x \geq y$ e $\max = x$) ou ($x < y$ e $\max = y$). As duas implicações a serem provadas são

$$\{x \geq y\} \ \max := x \ \{(x \geq y \text{ e } \max = x) \text{ ou } (x < y \text{ e } \max = y)\}$$

e

$$\{x < y\} \ \max := y \ \{(x \geq y \text{ e } \max = x) \text{ ou } (x < y \text{ e } \max = y)\}$$

Cada qual é verdadeira pelo axioma da atribuição. (No primeiro caso, por exemplo, o axioma da atribuição nos diz que algo da forma $E \vee F \rightarrow C$ é verdadeiro, portanto vale que $E \rightarrow C$ e E é equivalente a $x \geq y$.) A regra condicional assegura, então, que o trecho do programa está correto. •

No próximo capítulo veremos como verificar a correção de um comando de laço, onde a execução de uma seção de código pode ser repetida diversas vezes.

Como vimos, a prova de correção envolve uma porção de trabalho detalhado. É uma ferramenta difícil de ser aplicada a programas grandes que já existam. É muito mais simples provar a correção enquanto o programa estiver sendo desenvolvido. Além disso, a lista de asserções do início ao fim especifica a conduta desejada do programa, e pode ser usada desde o início em sua concepção. Além disso, as asserções servem como uma documentação de valor após o término do programa.

Revisão da Seção 15

Técnicas

- Formulação de fatos e regras na forma do Prolog
- Formulação de perguntas na forma do Prolog
- Determinação das respostas a uma pergunta usando uma base de dados Prolog
- Verificação da correção de um trecho de programa que inclua comandos de atribuição
- Verificação da correção de um trecho de programa que inclua comandos condicionais

Idéias Principais

Uma linguagem declarativa incorpora predicados, conectivos lógicos e regras de inferência para chegar a teses a partir das hipóteses sobre uma determinada interpretação.

Os elementos deste tipo de linguagem são baseados na lógica de predicados ao invés das instruções que realizam um algoritmo.

Um sistema de lógica formado de axiomas e regras de inferência pode ser usado para provar a correção de trechos de programas.

Exercícios 15

(Nota: Nos Exercícios 14, 15, 17 e 20, o * denota multiplicação.)

Os Exercícios de 1 a 6 referem-se à base de dados do Exemplo 26; encontre os resultados das perguntas em cada caso.

1. `is(come(urso, peixinho))`
2. `is(come(raposa, coelho))`
- ★3. `which(x:come(quate, x))`
4. `which(x:come(x, mato))`
5. `which(x:come(urso, x) and come(x, coelho))`
6. `which(x:presa(x) and not(come(raposa, x)))`
- ★7. Formule uma regra Prolog que defina "herbívoros" a fim de incluí-la à base de dados do Exemplo 26.
8. Se a regra do Exercício 7 for incluída na base de dados do Exemplo 26, qual será a resposta à pergunta
`which(x:herbívoros(x))`
9. Declare uma base de dados Prolog que forneça informações sobre estados e suas capitais. Algumas cidades são grandes, outras pequenas. Alguns estados estão ao sul, outros ao norte.
 - a. Escreva uma pergunta para achar todas as cidades capitais.
 - b. Escreva uma pergunta para achar todos os estados cujas capitais sejam cidades pequenas.
 - c. Escreva uma pergunta para achar todos os estados ao norte cujas capitais sejam grandes cidades.
 - d. Formule uma regra para definir as cidades cosmopolitas como as cidades grandes que sejam capitais dos estados do sul.
 - e. Escreva uma pergunta para achar todas as cidades cosmopolitas.
- ★10. Suponha que exista uma base de dados Prolog que forneça informações sobre autores e os livros que escreveram. Os livros serão classificados como ficção, biografia ou referência.
 - a. Escreva uma pergunta para verificar se Chico Buarque escreveu *Estorvo*.
 - b. Escreva uma pergunta para achar todos os livros escritos por Paulo Coelho.
 - c. Formule uma regra para definir autores de livros não-ficção.
 - d. Escreva uma pergunta para achar todos os autores de livros não-ficção.

11. Suponha que exista uma base de dados Prolog que dê informações sobre uma família. Os predicados *homem*, *mulher* e *pais-de* (que indica se se trata do pai ou da mãe de um elemento) foram incluídos.
- Formule uma regra para definir *pai-de*.
 - Formule uma regra para definir *filha-de*.
 - Formule uma regra recursiva para definir *ancestral-de*.
12. Suponha que exista uma base de dados Prolog que forneça informações sobre as partes que compõem um motor de automóvel. Os predicados *grande*, *pequena*, *parte-de* são fornecidos.
- Escreva uma pergunta que encontre todos os itens pequenos que são partes de outros itens.
 - Escreva uma pergunta que encontre todos os itens grandes que têm subitens pequenos.
 - Formule uma regra recursiva para definir *componente-de*.
- ★13. De acordo com o axioma da atribuição, qual a pré-condição do seguinte trecho de programa? Explique por que ele funciona.

```
{pré-condição}
  x := x + 1;
{x = y - 1}
```

14. De acordo com o axioma da atribuição, qual a pré-condição do trecho de programa a seguir? Explique por que ele funciona.

```
{pré-condição}
  x := 2 * x;
{x > y}
```

15. Verifique a correção do seguinte trecho de programa com a pré-condição e pós-condição mostradas.

```
{ x=1 }
  y := x + 3;
  y := 2 * y;
{ y = 8 }
```

16. Verifique a correção do seguinte trecho de programa com a pré-condição e pós-condição mostradas.

```
{ x > 0 }
  y := x + 2;
  z := y + 1;
{ z > 3 }
```

- ★17. Verifique a correção do seguinte trecho de programa com a pré-condição e pós-condição mostradas:

```
v = x(x - 1);

y := x - 1;
y := x * y;
```

18. Verifique a correção do seguinte trecho de programa com a pré-condição e pós-condição mostradas.

```
v := 2x + 1;

y := x;
y := y + y;
y := y + 1;
```

- ★19. Verifique a correção do seguinte trecho de programa com a pré-condição e pós-condição mostradas.

```
{ y = 0 }
if y < 5 then
  y := y + 1
else
  v := 5;
{ y = 1 }
```

20. Verifique a correção do seguinte trecho de programa com a pré-condição e pós-condição mostradas.

```
{x = 7}
  if x <= 0 then
    v := x
  else
    y := 2*x;
{y = 14}
```

21. Verifique a correção do trecho de programa a seguir para computar $\min(x, y)$, o menor dentre dois valores x e y :

```
if x <= y then
  min := x
else
  min := y;
```

22. Verifique a correção do seguinte trecho de programa para computar $|x|$, o valor absoluto de x :

```
if x >= 0 then
  abs := x
else
  abs := -x;
```

Revisão do Capítulo

Terminologia

algoritmo	linguagem procedural	tabela-verdade
antecedente	lógica de sentenças	tautologia
argumento válido	lógica de predicados	teorema
asserção	lógica proposicional	tese
axioma	modus ponens	teste do programa
axioma da atribuição	modus tollens	validação do programa
busca em profundidade	negação	variável ligada
cláusula de Horn	ocorrência ligada (de uma variável)	variável livre
comando	ocorrência livre (de uma variável)	verificação do programa
comando condicional	parcela	wff predicativa
conectivo binário	predicado	wff proposicional
conectivo unário	predicado binário	wff válida
conjunção	predicado n-ário	wffs equivalentes
consequente	predicado ternário	
contradição	predicado unário	
cálculo predicado	programa correto	
cálculo proposicional	proposição	
dedução	prova	
definição recursiva	prova de correção	
disjunção	prova de seqüência	
domínio	pré-condição	
dual de uma equivalência	pseudocódigo	
equivalência	pós-condição	
escopo	quantificador existencial	
esquema de axioma	quantificador universal	
fato Prolog	regra Prolog	
fator	regra condicional de inferência	
fórmula bem-formulada (wff)	regra de inferência	
generalização	regra recursiva	
hipótese	resolução	
implicação	sistema especialista	
interpretação	sistema formal completo	
lei de De Morgan	sistema formal correto	
linguagem declarativa	sistemas baseados em regra	
linguagem descritiva	sistemas baseados no conhecimento	

Auto-Testes

Responda às seguintes respostas com verdadeiro ou falso sem consultar o capítulo.

Seção 1.1

1. Uma contradição é qualquer wff proposicional que não é uma tautologia.
2. A disjunção de qualquer wff proposicional com uma tautologia tem o valor-verdade verdadeiro.
3. O Algoritmo *TestaTautologia* determina se qualquer wff proposicional é ou não uma tautologia.
4. Wffs proposicionais equivalentes têm a mesma tabela-verdade para todas as atribuições de valores-verdade a suas componentes.
5. Uma das leis de De Morgan diz que a negação da disjunção é a disjunção das negações (das parcelas).

Seção 1.2

6. Uma wff predicativa que comece com um quantificador universal é universalmente verdadeira, isto é, é verdadeira em todas as interpretações.
7. Na wff predicativa $(\forall x)P(x, y)$, y é uma variável livre.
8. Algumas wffs predicativas podem não ter valor-verdade em algumas interpretações.
9. O domínio de uma interpretação consiste em valores para os quais a wff predicativa nessa interpretação é verdadeira.
10. Uma wff predicativa não tem interpretação na qual seja falsa.

s

Seção 1.3

11. O modus ponens permite que qualquer wff proposicional seja derivada dos axiomas.
12. A lógica proposicional é completa porque toda tautologia é um teorema.
13. Um argumento válido é um argumento no qual a conclusão é sempre verdadeira.
14. O método de dedução permite o uso de hipóteses como passos adicionais na seqüência de demonstração da tese.
15. Todo axioma da lógica proposicional é uma tautologia, mas nem toda tautologia é um axioma.

Seção 1.4

16. Os axiomas da lógica de predicados permitem que quantificadores existenciais e universais sejam incluídos ou removidos durante a seqüência de demonstração.
17. Um dos axiomas da lógica de predicados diz que "para todo, não" é o mesmo que "não é verdade que exista um".
18. Todo teorema da lógica proposicional é também um teorema da lógica de predicados.
19. Uma wff predicativa que não é válida não pode ser um teorema na lógica de predicados.
20. A generalização deve ser usada o mais cedo possível em uma seqüência de demonstração.

Seção 1.5

21. Uma regra Prolog descreve um predicado.
22. O modus ponens é um caso especial da resolução do Prolog.
23. Um programa provavelmente correto sempre dá as respostas certas a um dado problema.
24. Se uma asserção após uma atribuição é $y > 4$, então a pré-condição precisa ser $y \geq 4$.
25. A prova de correção envolve o desenvolvimento cuidadoso de conjuntos de dados para testes.

No Computador

Para os Exercícios 1 a 5, escreva um programa que produza a saída desejada para uma dada entrada.

1. *Entrada:* Valores-verdade para os dois símbolos proposicionais A e B
Saída: Valores-verdade correspondentes (devidamente rotulados, naturalmente) para

$$A \wedge B, A \vee B, A \rightarrow B, A \leftrightarrow B, A'$$

2. *Entrada:* Valores-verdade para os dois símbolos proposicionais A e B
Saída: Valores-verdade correspondentes para as wffs

$$A \rightarrow B' \quad \text{e} \quad B' \wedge [A \vee (A \wedge B)]$$

3. *Entrada:* Valores-verdade para os dois símbolos proposicionais A , B e C .
Saída: Valores-verdade correspondentes para as wffs

$$A \vee (B \wedge C') \rightarrow B' \quad \text{e} \quad A \vee C' \leftrightarrow (A \vee C)'$$

4. *Entrada:* Valores-verdade para os três símbolos proposicionais A , B e C e uma representação de uma wff proposicional simples. Símbolos especiais podem ser usados como conectivos lógicos, e pode também ser usada a **notação pós-fixa**; por exemplo,

$$A \ B \ \wedge \ C \ \vee \quad \text{para} \quad (A \wedge B) \vee C$$

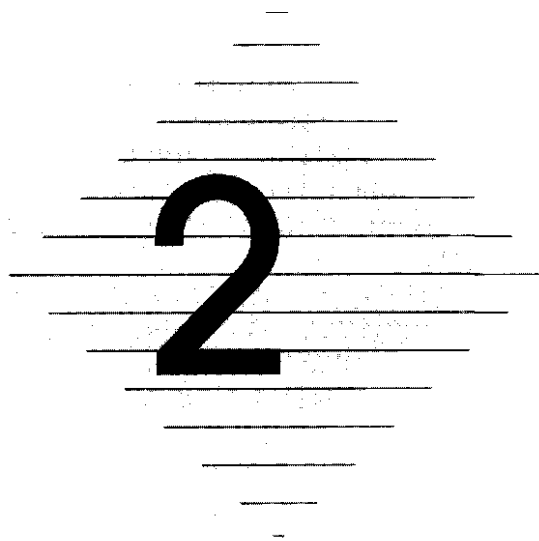
ou

$$A' \ B \ \wedge \quad \text{para} \quad (A' \wedge B)$$

Saída: Valores-verdade correspondentes para as wffs

5. *Entrada:* Uma representação de uma wff simples, como no exercício anterior
Saída: A decisão de se a wff é ou não uma tautologia
6. Se você tiver uma versão disponível do Prolog, entre a base de dados do Exemplo 26 e realize as perguntas indicadas lá. Além disso, inclua a regra recursiva *na-cadeia-alimentar* e realize a pergunta

which(y:na-cadeia-alimentar(urso, y))



Demonstrações, Recursão e Análise de Algoritmo

Objetivos do Capítulo

Após estudar este capítulo, você estará apto a:

- Realizar demonstrações de conjecturas, usando técnicas de demonstração direta, demonstração por contraposição e demonstração por contradição
- Reconhecer quando uma demonstração por indução é apropriada, e realizar uma demonstração deste tipo, usando a indução fraca ou a indução forte
- Compreender definições recursivas para certas seqüências, coleção de objetos e operações sobre objetos
- Escrever definições recursivas de seqüências, coleções de objetos e operações sobre objetos
- Entender como algoritmos recursivos são executados
- Escrever algoritmos recursivos para gerar seqüências definidas recursivamente
- Encontrar o termo geral para certas relações de recorrência
- Trabalhar com demonstrações matemáticas da correção de programas que usam comandos de repetição.

A demonstração de teoremas "de verdade" normalmente não é feita de maneira tão formal como nos sistemas lógicos do Cap. 1. É útil dispor de um arsenal de técnicas para desenvolver uma demonstração. Provas diretas, provas por contraposição e provas por contradição serão examinadas na primeira seção deste capítulo. A Seção 2.2 aborda

a indução matemática, uma técnica de demonstração muito utilizada na ciência da computação.

A Seção 2.3 discute a recursão, que está intimamente relacionada com a indução matemática e é importante para expressar muitas definições e até mesmo algoritmos. Algumas seqüências definidas recursivamente podem também ser definidas por uma fórmula; encontrar esta fórmula envolve a solução de relações de recorrência, e a indução é usada para verificar que a fórmula é correta.

A Seção 2.4 explora a utilização de relações de recorrência para determinar a quantidade de trabalho necessária para executar um algoritmo. Ela ainda se vale da lógica formal (bem como da indução) para verificar a correção de programas de computadores.

Seção 2.1 Técnicas de Demonstração

Teoremas

Resultados matemáticos geralmente são expressos como teoremas da forma "se P , então Q " ou $P \rightarrow Q$, onde P e Q podem representar sentenças compostas. Em um teorema desta forma, tentamos deduzir Q de P , usando axiomas e regras de inferência lógica. Se for possível usar somente axiomas da lógica pura (verdadeira em todas as interpretações), então o teorema também é verdadeiro para todas as interpretações. Nesse caso, o teorema é verdadeiro por causa da sua forma e estrutura, e não por causa do seu conteúdo ou pelo significado das partes que o compõem.

Entretanto, nós geralmente desejamos demonstrar teoremas cujo significado é importante, porque estamos trabalhando um tema em particular — algoritmos gráficos, álgebra booleana, teoria de compiladores, ou o que quer que seja. Neste caso, podem-se usar como axiomas sentenças que, embora não universalmente válidas, são fatos sobre o assunto em questão, como definições e teoremas previamente demonstrados. Tentaremos deduzir Q de P , utilizando uma seqüência lógica de passos que começam em P e terminam em Q ; cada passo da seqüência é ou P , um axioma lógico, um axioma específico, ou um passo que pode ser logicamente inferido de passos anteriores da seqüência.

Pode não ser fácil reconhecer qual assunto específico será útil para preparar uma seqüência de passos que levem logicamente de P até Q . Infelizmente não existe fórmula para construir demonstrações e nem algoritmos gerais ou programas para demonstrarem teoremas. A experiência é sempre útil, não apenas porque melhoramos com a prática, mas também porque a forma de demonstração utilizada para um teorema pode, às vezes, ser modificada para ser usada na demonstração de um teorema semelhante. Com o estudo contínuo do assunto, o seu elenco de resultados que podem ser usados na demonstração de teoremas aumenta. Um dos objetivos deste livro é ajudá-lo a acumular alguns destes resultados teóricos relacionados à ciência da computação.

Teoremas são geralmente expressos e demonstrados de um modo menos formal do que nas lógicas proposicional e de predicados vistas no Cap. 1. Por exemplo, um teorema pode conter um resultado sobre todos os objetos do domínio de interpretação, isto é, o assunto em questão. Neste caso, a expressão formal do teorema poderá começar com pelo menos um quantificador universal; por exemplo, poderia ser algo como $(\forall x) [P(x) \rightarrow Q(x)]$. Este resultado poderia ser informalmente escrito como $P(x) \rightarrow Q(x)$. Se podemos provar que $P(x) \rightarrow Q(x)$, onde x é tratado como um elemento arbitrário do domínio, nós teremos então provado que $(\forall x) [P(x) \rightarrow Q(x)]$. (Este resultado faz uso da regra da generalização da lógica de predicados, discutida na Seção 1.4.)

Consideremos um outro exemplo: podemos saber que todos os objetos em um certo domínio têm algumas propriedades; ou seja, algo da forma $(\forall x)P(x)$ pode ser considerado como um axioma específico. Uma demonstração informal pode ser obtida com a seguinte colocação: "Seja x um elemento do domínio. Então x tem a propriedade P ". (Formalmente estamos fazendo uso do Axioma 5 da lógica predicativa, $(\forall x)P(x) \rightarrow P(x)$, junto com a hipótese $(\forall x)P(x)$ e o *modus ponens* para concluir $P(x)$. No entanto, toda esta justificativa formal é quase sempre omitida.)

Analogamente, demonstrações não são, em geral, escritas com justificativas formais para cada passo. Em vez disso, os passos importantes e a idéia que está por trás deles são delineados por uma narrativa. Tal narrativa, entretanto, pode ser traduzida para uma demonstração formal, se necessário. De fato, o valor de uma demonstração formal decorre do fato de ela funcionar como uma espécie de seguro — se uma demonstração

na forma de uma narrativa *não pode* ser traduzida para uma prova formal, ela deve ser vista com uma boa dose de desconfiança.

Antes de discutirmos a demonstração de teoremas, vamos especular um pouco. Infelizmente, o leitor deste livro está diante de um resultado estático referente a um processo dinâmico, e não pode compartilhar da aventura de desenvolver novas idéias. O livro pode afirmar "Prove o seguinte teorema", e o leitor irá saber se o teorema é verdadeiro; mais adiante, ele será colocado, talvez, em sua forma mais bem preparada. O pesquisador, por sua vez, não irá adquirir repentinamente uma visão sobre qual deve ser a redação perfeita para um teorema, juntamente com a certeza absoluta de que ele é verdadeiro. Tudo que ele deve fazer é trabalhar para encontrar a demonstração. Antes que se chegue ao final da demonstração de um teorema, uma combinação de raciocínios indutivos e dedutivos é utilizada.

Suponha que você é um pesquisador tentando formular e demonstrar um teorema, e que examinou um elenco de casos nos quais se P é verdadeiro, então Q também é verdadeiro. (Por exemplo, você pode ter examinado sete ou oito inteiros divisíveis por 6, e constatou que estes inteiros também são divisíveis por 3.) Com base nesta experiência, você pode conjecturar: Se P , então Q (se um inteiro é divisível por 6, então ele também é divisível por 3). Quantos mais casos você encontrar nos quais Q resulta de P , mais confiante você estará em sua conjectura. Este passo ilustra o **raciocínio indutivo**, construindo uma conclusão baseada em experiência.

Não importa o quanto a conjectura pareça confiável, você não ficará satisfeito até que tenha aplicado a ela o **raciocínio dedutivo**. Neste processo, você tenta verificar se sua conjectura é verdadeira ou falsa. Você pode elaborar a prova de que $P \rightarrow Q$ (construindo um teorema), ou então encontrar um exemplo que contrarie sua conjectura. (Nós usamos o raciocínio dedutivo na lógica predicativa quando provamos que uma wff é um teorema, ou encontramos uma interpretação na qual a wff é falsa.)

Freqüentemente é difícil decidir qual das duas abordagens você deverá seguir — demonstrar ou negar a conjectura! Suponha que você decida tentar negá-la. Você irá procurar um exemplo no qual P é verdadeiro, mas Q é falso — você procurará por um **contra-exemplo** para a sua conjectura. Um único contra-exemplo é suficiente para negar a conjectura. Então, você pode refutar a sua conjectura simplesmente encontrando um inteiro divisível por 6 mas não por 3. Se nossa conjectura for verdadeira, tal inteiro não existe. É claro que o fato de procurarmos por um contra-exemplo sem achá-lo *não* constitui prova de que a conjectura é verdadeira.

EXEMPLO 1

Considere a sentença "Todo inteiro menor que 10 é maior que 5", ou, expresso em uma implicação "Se um inteiro é menor que 10, então ele é maior que 5". Um contra-exemplo para esta implicação é o inteiro 4. Quatro é menor do que 10, porém não é maior do que 5. É claro que existem outros contra-exemplos, porém um é suficiente para negar a afirmação. •

PRÁTICA 1

Forneça contra-exemplos para as seguintes sentenças:

- Todos os animais que vivem nos oceanos são peixes.
- As entradas para um programa de computador são sempre fornecidas através do teclado.

Métodos de Abordagem

Suponha que você decida provar sua conjectura $P \rightarrow Q$. Ainda que um simples contra-exemplo seja suficiente para refutar a conjectura, em geral *muitos* exemplos não provam a suposição — eles simplesmente fortalecem sua inclinação a procurar uma demonstração. A única exceção desta situação ocorre quando você está fazendo uma asserção sobre uma coleção finita. Neste caso, a asserção pode ser provada verdadeira desde que se mostre ser verdadeira para cada um dos elementos da coleção. Por exemplo, a asserção "Se um inteiro entre 1 e 20 é divisível por 6, então ele também é divisível por 3" pode ser provada, mostrando-se simplesmente para os inteiros divisíveis por 6 entre 1 e 20.

Demonstração Direta

No caso geral, como podemos demonstrar que $P \rightarrow Q$ é verdadeira? A abordagem óbvia é a **demonstração direta** — assume-se a hipótese P como verdadeira e deduz-se a tese Q .

EXEMPLO 2

Nós iremos dar uma demonstração direta para o exemplo dado como teorema. "Se um inteiro é divisível por 6, então ele também é divisível por 3." O teorema faz uma afirmação sobre um inteiro arbitrário, sua forma é:

$$(\forall x) (x \text{ divisível por } 6 \rightarrow x \text{ divisível por } 3)$$

onde o domínio de interpretação é entendido como sendo os inteiros. Vamos então representar por x um inteiro arbitrário e provar

$$x \text{ divisível por } 6 \rightarrow x \text{ divisível por } 3$$

Para desenvolver a demonstração, assumimos que a hipótese de que x é divisível por 6 é verdadeira, e então deduzimos que a tese x é divisível por 3 também é verdadeira. Nós temos que utilizar a definição de divisibilidade — a é divisível por b , se a é igual ao produto de um inteiro por b — e também outras propriedades aritméticas.

Hipótese: x é divisível por 6

$$\begin{aligned} x &= k \cdot 6 \text{ para algum inteiro } k && \text{(definição de divisibilidade)} \\ 6 &= 2 \cdot 3 && \text{(fato numérico)} \\ x &= k(2 \cdot 3) && \text{(substituição)} \\ x &= (k \cdot 2)3 && \text{(associatividade do produto)} \\ k \cdot 2 &\text{ é um inteiro} && \text{(fato conhecido dos inteiros)} \end{aligned}$$

Conclusão: x é divisível por 3 (definição de divisibilidade) •

Note que um dos nossos primeiros passos no Exemplo 2 foi identificar claramente a hipótese e a tese, não só o que elas são em palavras, mas o que elas realmente significam, pela aplicação de definições apropriadas. Se não entendemos claramente o que nós temos (a hipótese) e o que nós desejamos (a tese), não podemos esperar construir um elo de ligação entre uma e outra. Esta é a razão da importância de se conhecer definições.

PRÁTICA 2 Forneça uma demonstração direta para o teorema "Se um inteiro é divisível por 6, então duas vezes o inteiro é divisível por 4". Mostre cada passo para se ir da hipótese à tese. •

EXEMPLO 3 Uma demonstração direta de que o produto de dois pares é par é: Seja $x = 2m$ e $y = 2n$, com m e n inteiros. Então $xy = (2m)(2n) = 2(2mn)$, onde $2mn$ é um inteiro. Então xy é da forma $2k$, onde k é um inteiro, logo xy é par.

Esta prova é menos formal do que a do Exemplo 2; ela não indica a hipótese explicitamente e faz uso implícito da definição de um inteiro par. Entretanto, a prova seria perfeitamente aceitável na maioria das circunstâncias. •

Contraposição

Se você tiver tentado assiduamente, mas falhado, produzir uma demonstração direta de sua conjectura $P \rightarrow Q$, e ainda sente que a conjectura é verdadeira, você deve tentar algumas variantes da técnica de prova direta. Se você pode demonstrar o teorema $Q' \rightarrow P'$, pode concluir que $P \rightarrow Q$ pelo uso da tautologia $(Q' \rightarrow P') \rightarrow (P \rightarrow Q)$. $Q' \rightarrow P'$ é a **contrapositividade** de $P \rightarrow Q$. A técnica para demonstrar que $P \rightarrow Q$ construindo uma prova direta de $Q' \rightarrow P'$ é chamada de **demonstração por contraposição**. Já vimos demonstrações diretas, de forma que a única idéia nova aqui é a aplicação da contrapositividade.

EXEMPLO 4 A contrapositividade do teorema "Se um inteiro é divisível por 6, então ele também é divisível por 3" é "Se um inteiro não é divisível por 3, então ele também não é divisível por 6". A forma mais fácil de provar este teorema é a demonstração direta dada no Exemplo 2, porém a demonstração por contraposição também é possível.

Hipótese: x não é divisível por 3

$$x \neq k \cdot 3 \text{ para algum inteiro } k \text{ (Esta é a negação de divisibilidade por 3.)}$$

$$x \neq (2k_1)3 \text{ para algum inteiro } k_1 \text{ (} 2k_1 \text{ pode ser um inteiro } k, \text{ definido acima.)}$$

$$x \neq k_1 (2 \cdot 3) \text{ para algum inteiro } k_1 \text{ (propriedade da multiplicação)}$$

$$x \neq k_1 \cdot 6 \text{ para algum inteiro } k_1 \text{ (fato numérico)}$$

Conclusão: x não é divisível por 6 (negação da divisibilidade por 6). •

PRÁTICA 3 Escreva a contraposição para cada sentença da Prática 3 do Cap. 1.

EXEMPLO 5 Demonstre que se o quadrado de um número é ímpar, então o número também é ímpar.

O teorema é $n^2 \text{ ímpar} \rightarrow n \text{ ímpar}$. Nós faremos a demonstração por contraposição, ou seja, demonstraremos que $n \text{ par} \rightarrow n^2 \text{ par}$. Seja n par. Então $n = 2r$ e $n^2 = 4r^2$ é par pelo Exemplo 3.

A Prática 7 do Cap. 1 mostrou que as wffs $A \rightarrow B$ e $B \rightarrow A$ não são equivalentes. $B \rightarrow A$ é a **recíproca** de $A \rightarrow B$. Se uma implicação é verdadeira, a sua recíproca pode ser verdadeira ou falsa. Portanto, não podemos demonstrar $P \rightarrow Q$ a partir do resultado $Q \rightarrow P$.

EXEMPLO 6 A implicação "Se $a > 5$ então $a > 2$ " é verdadeira, no entanto a sua recíproca "Se $a > 2$ então $a > 5$ " é falsa.

PRÁTICA 4 Escreva a recíproca de cada sentença da Prática 3 do Cap. 1.

Teoremas são, por diversas vezes, enunciados na forma: P se, e somente se, Q , significando P se Q e P somente se Q , ou $Q \rightarrow P$ e $P \rightarrow Q$. Para demonstrar um teorema como esse, deve-se demonstrar tanto uma implicação como a sua recíproca. Lembre-se de que qualquer teorema do tipo "se e somente se" requer uma demonstração em ambas as direções.

EXEMPLO 7 Demonstre que o produto xy é ímpar se, e somente se, x e y são inteiros ímpares.

Provaremos inicialmente que se x e y são ímpares, então xy também o é. Faremos uma demonstração direta. Suponha que x e y são ímpares. Então $x = 2n + 1$ e $y = 2m + 1$, onde m e n são inteiros. Então $xy = (2n + 1)(2m + 1) = 4nm + 2m + 2n + 1 = 2(2nm + m + n) + 1$. Assim xy é da forma $2k + 1$ onde k é um inteiro, logo xy é ímpar.

A seguir mostraremos que se xy é ímpar, então x e y devem ser ímpares, ou

$$xy \text{ ímpar} \rightarrow x \text{ ímpar e } y \text{ ímpar}$$

Faremos aqui uma demonstração por contraposição, demonstraremos que

$$(x \text{ ímpar e } y \text{ ímpar})' \rightarrow (xy \text{ ímpar})'$$

Pela Lei de De Morgan $(A \wedge B)' \Leftrightarrow A' \vee B'$, veremos que esta conjectura pode ser escrita como

$$x \text{ par ou } y \text{ par} \rightarrow xy \text{ par} \quad (1)$$

A hipótese "x par ou y par" pode ser dividida em três partes. Consideremos uma por vez.

1. x par, y ímpar. Temos $x = 2m$, $y = 2n + 1$ e $xy = (2m)(2n + 1) = 2(2mn + m)$, que é par.
2. x ímpar, y par: O procedimento é semelhante ao caso 1.
3. x par, y par: Neste caso xy é par pelo Exemplo 3.

Isto completa a demonstração de (1) e do teorema.

Parte da demonstração do Exemplo 7 utiliza a técnica conhecida como **demonstração por exaustão** que algumas vezes é muito útil. Ela envolve a identificação de todos os casos possíveis com as informações dadas e, em seguida, a prova de cada um desses casos separadamente.

Contradição

Além da demonstração direta e da demonstração por contraposição, podemos usar a técnica de **demonstração por contradição** (algumas vezes chamada *demonstração indireta*; entretanto este termo significa, na verdade, qualquer argumento que não seja uma demonstração direta). Como fizemos no Cap. 1, associaremos 0 valor 0 (zero) a qualquer contradição, isto é, qualquer wff que tem valor-verdade sempre falso. ($A \wedge A'$, por exemplo, é uma wff deste tipo.) Mais uma vez, suponha que estamos tentando demonstrar que $P \rightarrow Q$. Por construção da tabela-verdade, veremos que

$$(P \wedge Q' \rightarrow 0) \rightarrow (P \rightarrow Q)$$

é uma tautologia, então para demonstrar que o teorema $P \rightarrow Q$, é suficiente demonstrar que $P \wedge Q' \rightarrow 0$. Portanto, em uma prova por contradição, você assume que tanto a hipótese como a negação da tese são verdadeiras, e então tenta obter algumas contradições a partir dessas suposições.

EXEMPLO 8

Vamos usar a prova por contradição para a sentença "Se um número somado a ele próprio resulta no próprio número, então o número é 0 (zero)". Representemos por x um número qualquer. A hipótese é $x + x = x$ e a conclusão é $x = 0$. Para construirmos uma demonstração por contradição, assumamos que $x + x = x$ e que $x \neq 0$. Então $2x = x$ e $x \neq 0$. Como $x \neq 0$, podemos dividir ambos os lados da equação $2x = x$ por x , o que nos leva à contradição $2 = 1$. Logo $(x + x = x) \rightarrow (x = 0)$.

EXEMPLO 9

Uma prova por contradição bem conhecida mostra que $\sqrt{2}$ não é um número racional. Lembrando que um **número racional** é um número que pode ser escrito na forma p/q onde p e q são inteiros, $q \neq 0$ e p e q não têm fatores comuns (além de ± 1).

Vamos assumir que $\sqrt{2}$ é racional. Então $\sqrt{2} = p/q$, e $2 = p^2/q^2$, ou seja $2q^2 = p^2$. Então 2 divide p^2 , logo 2 deve dividir p . Isto significa que 2 é um fator de p , logo 4 é um fator de p^2 , e a equação $2q^2 = p^2$ pode ser escrita como $2q^2 = 4x$, ou $q^2 = 2x$. Obtemos desta equação que 2 divide q^2 , logo 2 divide q . Então 2 é fator de q e fator de p , o que contradiz a hipótese de que p e q não têm fatores comuns. Logo $\sqrt{2}$ não é racional.

PRÁTICA 5

Prove por contradição que o produto de dois inteiros pares é par. (Nós fizemos a prova direta deste resultado no Exemplo 7.)

A demonstração por contradição pode ser uma técnica útil, mas é fácil imaginar que fizemos uma demonstração por contradição sem tê-la feito. Por exemplo, suponha que assumimos $P \wedge Q'$, e que deduzimos Q sem usar a hipótese Q' . Então chegamos a $Q \wedge Q'$ como uma contradição. O que de fato realizamos neste caso foi uma demonstração direta de $P \rightarrow Q$, e devemos reescrever a demonstração desta forma. Voltando ao Exemplo 8 nós poderíamos assumir que $x + x = x$ e $x \neq 0$, como antes. Poderíamos argumentar então que de $x + x = x$ nós obtemos $2x = x$ e depois, subtraindo x de ambos os lados obter $x = 0$. Nós temos então, $x = 0$ e $x \neq 0$, o que é uma contradição. Entretanto, no argumento utilizado, em momento algum, fizemos uso da hipótese $x \neq 0$; nós provamos diretamente que $x + x = x$ implica $x = 0$.

Outro engano comum na demonstração por contradição, ocorre quando assumimos $P \wedge Q'$ e estamos aptos a deduzir F sem usar a hipótese P . Então nós assumimos $P \wedge P'$ como uma contradição. O que realmente elaborou-se aqui foi uma prova direta de $Q' \rightarrow P'$, desenvolveu-se, portanto, uma demonstração por contraposição e não uma demonstração por contradição.

Ainda não discutimos um método de demonstração especialmente útil na ciência da computação — a indução matemática. Ele será objeto da próxima seção.

Revisão da Seção 2.1**Técnicas**

- Procura de contra-exemplos
- Construção de demonstrações diretas, demonstrações por contraposição, e demonstrações por contradição

Idéias Principais

O raciocínio indutivo é usado para formular uma conjectura baseada em experiência.

O raciocínio dedutivo é usado tanto para refutar uma conjectura encontrando um contra-exemplo, como para prová-la.

Na demonstração de uma conjectura, fatos lógicos e fatos sobre assuntos particulares podem ser usados.

Se não podemos demonstrar diretamente uma conjectura, podemos tentar demonstrá-la por contraposição ou por contradição.

Exercícios 2.1

As definições a seguir podem ser úteis na resolução de alguns dos exercícios. Um **quadrado perfeito** é um inteiro n tal que $n = k^2$ para algum inteiro k . Um **número primo** é um inteiro $n > 1$ tal que n não é divisível por nenhum inteiro além de 1 e n . Para dois números x e y , $x < y$ significa $y - x > 0$.

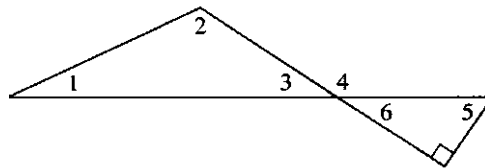
- ★1. Escreva a recíproca e contraposição para cada sentença do Exercício 4 da Seção 1.1.
- 2. Encontre contra-exemplos para cada uma das seguintes afirmações:
 - a. Toda figura geométrica com quatro ângulos retos é um quadrado.
 - b. Se um número real não é positivo, então ele deve ser negativo.
 - c. Todas as pessoas ruivas têm olhos verdes ou são altas.
 - d. Todas as pessoas ruivas têm olhos verdes e são altas.
- ★3. Prove que se $n = 25, 100$ ou 169 então n é um quadrado perfeito e é a soma de dois quadrados perfeitos.
- 4. Prove que se n é um inteiro par, $4 \leq n \leq 12$, então n é a soma de dois números primos.
- 5. Forneça uma demonstração direta de que a soma de inteiros pares é par.
- 6. Prove por contradição que a soma de inteiros pares é par.
- ★7. Prove que a soma de dois inteiros ímpares é par.
- 8. Prove que a soma de um inteiro par e um inteiro ímpar é ímpar.
- 9. Prove que o produto de quaisquer dois inteiros consecutivos é par.
- 10. Prove que a soma de um inteiro e do seu quadrado é par.
- ★11. Prove que o quadrado de um número par é divisível por 4.
- 12. Prove que para qualquer inteiro n , o número

$$3(n^2 + 2n + 3) - 2n^2$$
 é um quadrado perfeito.
- 13. Prove por contradição que se qualquer número x é positivo, então $x + 1$ também é positivo.
- ★14. Sejam x e y números positivos, prove que $x < y$ se, e somente se, $x^2 < y^2$.
- 15. Prove que se $x^2 + 2x - 3 = 0$, então $x \neq 2$.
- 16. Prove que se x é inteiro par e primo, então $x = 2$.
- ★17. Prove que se dois inteiros são ambos divisíveis por um inteiro n , então a sua soma é divisível por n .
- 18. Prove que se o produto de dois inteiros não é divisível por um inteiro n , então nenhum dos inteiros é divisível por n .
- 19. Prove que a soma de três inteiros consecutivos é divisível por 3.
- ★20. Prove que o quadrado de um inteiro ímpar pode ser escrito como $8k + 1$ para algum inteiro k .
- 21. Prove que a diferença de dois cubos consecutivos é ímpar.
- 22. Prove que a soma de quadrados de dois inteiros ímpares não pode ser um quadrado perfeito. (*Sugestão.* Use o Exercício 20.)
- ★23. Prove que o produto dos quadrados de dois inteiros é um quadrado perfeito.

24. Suponha que você usou os passos do Exemplo 9 para tentar mostrar que $\sqrt{4}$ não é um número racional. Em qual passo a prova não seria válida?
25. Prove que $\sqrt{3}$ não é um número racional.
26. Prove que $\sqrt{5}$ não é um número racional.
27. Prove que $\sqrt[3]{2}$ não é um número racional.
- ★28. Prove ou apresente um contra-exemplo: O produto de quaisquer três inteiros consecutivos é par.
29. Prove ou apresente um contra-exemplo: A soma de quaisquer três inteiros consecutivos é par.
30. Prove ou apresente um contra-exemplo: O produto de um inteiro pelo seu quadrado é par.
- ★31. Prove ou apresente um contra-exemplo: A soma de um inteiro com o seu cubo é par.
32. Prove ou apresente um contra-exemplo: Para um inteiro positivo x , $x + \frac{1}{x} \geq 2$.
33. Prove ou apresente um contra-exemplo: Para todo número primo n , $n + 4$ é primo.
34. Prove ou apresente um contra-exemplo: O produto de dois números irracionais é irracional.
- ★35. Prove ou apresente um contra-exemplo: A soma de dois números racionais é racional.

Para os exercícios 36 a 38, use a figura abaixo e os seguintes fatos da geometria:

- A soma dos ângulos internos de um triângulo é 180° .
- Ângulos opostos pelo vértice (ângulos opostos que são formados quando duas linhas se interceptam) têm a mesma medida.
- Um ângulo raso mede 180° .
- Um ângulo reto mede 90° .



36. Prove que a medida do ângulo 4 é a soma das medidas dos ângulos 1 e 2.
- ★37. Prove que a medida do ângulo 5 mais a medida do ângulo 3 é 90° .
38. Se o ângulo 1 e o ângulo 5 têm a mesma medida, então o ângulo 2 é um ângulo reto.

Seção 2.2 Indução

O Método

Existe uma última técnica de demonstração que se aplica a determinadas situações. Para ilustrar o uso desta técnica, imagine que você está subindo em uma escada sem fim. Como você pode saber se será capaz de alcançar um degrau arbitrariamente alto? Suponha que você faça as seguintes afirmações sobre as suas habilidades de subir escadas:

1. Você pode alcançar o primeiro degrau.
2. Se você alcançar um degrau, você pode sempre passar ao degrau seguinte. (Note que esta asserção é uma implicação.)

Tanto a sentença 1 como a implicação na sentença 2 são verdadeiras; então, pela sentença 1 você pode chegar ao primeiro degrau e pela sentença 2 você pode chegar ao segundo; novamente pela 2 você pode chegar ao terceiro; pela sentença 2 novamente você pode chegar ao quarto degrau, e assim sucessivamente. Você pode, então, subir tão alto quanto você queira. Neste caso, ambas as asserções são necessárias. Se apenas a sentença 1 é verdadeira, você não tem garantias de que poderá ir além do primeiro degrau, e se apenas a segunda sentença é verdadeira, você poderá não chegar ao primeiro degrau a fim de iniciar o processo de subida da escada. Vamos assumir que os degraus da escada são numerados com os números inteiros positivos — 1, 2, 3, e assim por diante. Agora vamos considerar uma propriedade específica que um número pode ter. Ao invés de "alcançarmos um degrau arbitrário" podemos mencionar que um inteiro positivo arbitrário tem essa propriedade. Usaremos a notação simplificada $P(n)$ para denotar que o inteiro positivo n tem a propriedade P . Como podemos usar a técnica de subir escadas para provar que para todos inteiros positivos n nós temos $P(n)$? As duas afirmações de que precisamos para a demonstração são:

- | | |
|--|---|
| 1. $P(1)$ | (1 tem a propriedade P .) |
| 2. Para qualquer inteiro positivo k ,
$P(k) \rightarrow P(k + 1)$ | (Se algum número tem a propriedade P , então o número seguinte também a tem.) |

Se pudermos demonstrar as sentenças 1 e 2, então $P(n)$ vale para qualquer inteiro positivo n , da mesma maneira que nós podemos subir até um degrau arbitrário na escada.

O fundamento deste tipo de argumentação é chamado **princípio de indução matemática**, que pode ser enunciado como

- | | |
|--|--|
| 1. $P(1)$ verdadeira | } $\rightarrow P(n)$ verdadeira para todos os n inteiros positivos |
| 2. $(\forall k) [P(k) \text{ verdadeira} \rightarrow P(k + 1) \text{ verdadeira}]$ | |

O princípio da indução matemática é uma implicação. A tese desta implicação é uma sentença da forma " $P(n)$ é verdadeira para todos os n inteiros positivos". Portanto, sempre que desejamos demonstrar que alguma propriedade é válida para todo inteiro positivo n , uma tentativa é o uso da indução matemática como técnica de demonstração.

Para averiguarmos que a tese desta implicação é verdadeira, mostramos que as duas hipóteses, ou seja, as afirmações 1 e 2 são verdadeiras. Para demonstrar a afirmação 1, precisamos apenas mostrar que a propriedade P vale para o número 1, o que é normalmente uma tarefa trivial. Para demonstrar a afirmação 2, uma implicação que deve valer para todo k , assumimos que $P(k)$ é verdadeira para um inteiro arbitrário k , e baseados nesta hipótese mostramos que $P(k + 1)$ é verdadeira. Você deve convencer a si próprio que assumir a propriedade P como válida para o número k não é a mesma coisa que assumir o que desejamos demonstrar (uma freqüente fonte de confusão, quando nos deparamos pela primeira vez com este tipo de demonstração). Assumi-la como verdadeira é simplesmente o caminho para elaborar a prova de que a implicação $P(k) \rightarrow P(k+1)$ é verdadeira.

Ao desenvolver uma demonstração por indução, estabelecemos inicialmente a veracidade da sentença 1, $P(1)$, que é chamada de **base da indução** ou **passo básico**, para a demonstração indutiva. Estabelecer que a sentença $P(k) \rightarrow P(k + 1)$ é verdadeira constitui o **passo indutivo**. Quando assumimos que $P(k)$ é verdadeira com o intuito de demonstrar o passo indutivo, $P(k)$ é chamado de **suposição indutiva**, ou **hipótese indutiva**.

Todos os métodos de demonstração sobre os quais comentamos neste capítulo são técnicas de raciocínio dedutivo — formas de demonstrar uma conjectura que possivelmente foi formulada por um raciocínio indutivo. A indução matemática também é uma técnica *dedutiva*, e não um método para o raciocínio indutivo (procure não ficar confuso com a terminologia utilizada aqui). Para outras técnicas de demonstração, podemos começar com as hipóteses, e encadear fatos até que mais ou menos tropeçemos em uma solução. De fato, mesmo que nossa conjectura seja ligeiramente incorreta, provavelmente acabamos por verificar a tese correta no decorrer do processo de demonstração. Na indução matemática, entretanto, devemos saber no princípio a forma exata da propriedade $P(n)$ que estamos tentando estabelecer. A indução matemática, portanto, não é uma técnica de demonstração exploratória — ela pode apenas confirmar uma conjectura correta.

Demonstrações Indutivas

Suponha que o Sr. Silva casou-se e teve dois filhos. Vamos chamar estes dois filhos de geração 1. Agora suponha que cada um desses dois filhos teve dois filhos; então na geração 2 temos quatro descendentes. Este processo continua de geração em geração. A árvore genealógica da família Silva é semelhante à Fig. 2.1. (Que é exatamente igual à Fig. 1.1b, onde buscamos os possíveis valores T-F para as n letras de afirmações.)

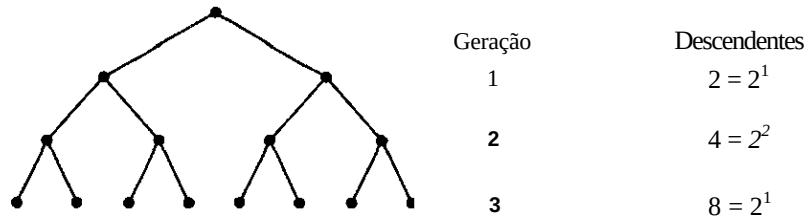


Figura 2.1

Aparentemente a geração n tem 2^n descendentes. De maneira mais formal, se fizermos $P(n)$ denotar o número de descendentes na geração n , então nossa suposição será

$$P(n) = 2^n$$

Podemos usar a indução para *demonstrar* que nosso palpite para $P(n)$ está correto.

A base da indução é estabelecer $P(1)$, que resulta a equação

$$P(1) = 2^1 = 2$$

Que é verdadeira, posto que foi informado que o Sr. Silva tinha dois filhos. Agora vamos supor que nossa premissa é verdadeira para uma geração arbitrária k , ou seja, assumimos que

$$P(k) = 2^k$$

e tentaremos mostrar que

$$P(k+1) = 2^{k+1}$$

Nesta família, cada descendente tem dois filhos; então o número de descendentes na geração $k + 1$ será o dobro do da geração índice k , ou seja, $P(k + 1) = 2 P(k)$. Pela hipótese de indução, $P(k) = 2^k$, logo

$$P(k+1) = 2 P(k) = 2(2^k) = 2^{k+1}$$

então, de fato,

$$P(k+1) = 2^{k+1}$$

Isto completa a nossa demonstração por indução. Agora que sabemos como resolver o problema simples do clã dos Silva, podemos aplicar a técnica de demonstração por indução em problemas menos óbvios.

EXEMPLO 10 Prove que a equação

$$1 + 3 + 5 + \dots + (2n-1) = n^2 \quad (1)$$

é verdadeira para qualquer inteiro positivo n . Neste caso, a propriedade $P(n)$ é que a equação (1) acima é verdadeira. O lado esquerdo desta equação é a soma de todos os inteiros ímpares de 1 até $2n - 1$. Ainda que possamos verificar que a equação é verdadeira para um particular valor de n pela substituição deste valor na equação, nós não podemos substituir todos os possíveis valores inteiros positivos. Por isso, uma demonstração por exemplos não funciona. É mais apropriada uma demonstração por indução matemática.

O passo básico é estabelecer $P(1)$, que é o valor da equação (1) quando n assume o valor 1, ou seja

$$1 = 1^2$$

Isto certamente é verdadeiro. Para a hipótese de indução, vamos assumir $P(k)$ como verdadeira para um inteiro positivo arbitrário k , que é o valor da equação (1) quando n vale k , ou seja

$$1 + 3 + 5 + \dots + (2k-1) = k^2 \quad (2)$$

(Note que $P(k)$ não é a equação $(2k-1) = k^2$ que só é verdadeira quando $k = 1$.) Usando a hipótese de indução, queremos mostrar $P(k+1)$ que é o valor da equação (1) quando n assume o valor $k+1$, ou seja:

$$1 + 3 + 5 + \dots + [2(k+1) - 1] = (k+1)^2 \quad (3)$$

(O pequeno ponto de interrogação sobre o sinal de igualdade serve para nos lembrar que é este fato que desejamos provar, a partir de um outro fato já conhecido.)

A chave para uma demonstração por indução é encontrar uma forma de relacionar o que se deseja mostrar — $P(k+1)$, equação (3) — com o que assumimos como verdadeiro — $P(k)$, equação (2). O lado esquerdo de $P(k+1)$ pode ser reescrito a fim de destacar o penúltimo termo:

$$1 + 3 + 5 + \dots + (2k-1) + [2(k+1) - 1]$$

Esta expressão contém do lado esquerdo a equação (2) como subexpressão. Como assumimos que $P(k)$ é verdadeira, podemos substituir esta expressão pelo lado direito da equação (2). Senão vejamos:

$$\begin{aligned} 1 + 3 + 5 + \dots + [2(k+1) - 1] \\ &= 1 + 3 + 5 + \dots + (2k-1) + [2(k+1) - 1] \\ &= k^2 + [2(k+1) - 1] \\ &= k^2 + [2k + 2 - 1] \\ &= k^2 + 2k + 1 \\ &= (k+1)^2 \end{aligned}$$

Portanto

$$1 + 3 + 5 + \dots + [2(k+1) - 1] = (k+1)^2$$

o que verifica $P(k+1)$ e prova que a equação (1) é verdadeira para qualquer inteiro positivo n . •

EXEMPLO 11 Prove que

$$1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1$$

para qualquer $n > 1$.

Aqui, novamente, a indução é a técnica mais apropriada. $P(1)$ é a equação

$$1 + 2 = 2^{1+1} - 1 \text{ ou } 3 = 2^2 - 1$$

que é verdadeira. Consideremos agora $P(k)$

$$1 + 2 + 2^2 + \dots + 2^k = 2^{k+1} - 1$$

como a hipótese de indução, e busquemos obter $P(k+1)$:

$$1 + 2 + 2^2 + \dots + 2^{k+1} \stackrel{?}{=} 2^{k+1+1} - 1$$

Agora, se reescrevermos a soma do lado esquerdo de $P(k+1)$, obtemos uma forma de usar a hipótese de indução:

$$\begin{aligned} 1 + 2 + 2^2 + \dots + 2^{k+1} \\ &= 1 + 2 + 2^2 + \dots + 2^k + 2^{k+1} \\ &= 2^{k+1} - 1 + 2^{k+1} \quad (\text{da hipótese de indução } P(k)) \\ &= 2(2^{k+1}) - 1 \\ &= 2^{k+1+1} - 1 \end{aligned}$$

Portanto

$$1 + 2 + 2^2 + \dots + 2^{k+1} = 2^{k+1+1} - 1$$

que verifica $P(k+1)$ e completa a demonstração.

PRÁTICA 6 Prove que para qualquer inteiro positivo n ,

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

◆

Nem todas as demonstrações por indução envolvem fórmulas com somas. Outras identidades algébricas sobre inteiros positivos podem ser demonstradas por indução, bem como suposições não-algébricas, como o número de descendentes na geração n da família Silva.

EXEMPLO 12 Demonstre que, para qualquer inteiro positivo n , $2^n > n$.

$P(1)$ é a sentença $2^1 > 1$ que é, sem dúvida, verdadeira. Suponhamos agora $P(k)$, $2^k > k$, como verdadeira, e procuremos concluir a partir daí $P(k+1)$, $2^{k+1} > k+1$. Começando pelo lado esquerdo de $P(k+1)$, temos que $2^{k+1} = 2^k \cdot 2$. Usando a hipótese de indução $2^k > k$ e multiplicando ambos os lados dessa desigualdade por 2, temos $2^k \cdot 2 > k \cdot 2$. Podemos escrever então que:

$$2^{k+1} = 2^k \cdot 2 > k \cdot 2 = k + k \geq k + 1$$

o que resulta em

$$2^{k+1} > k + 1$$

•

EXEMPLO 13 Prove que, para qualquer inteiro positivo n , o número $2^{2n} - 1$ é divisível por 3.

A base da indução é mostrar $P(1)$, ou seja, mostrar que $2^{2(1)} - 1 = 4 - 1 = 3$ é divisível por 3, o que obviamente é verdadeiro.

Assumimos que $2^{2k} - 1$ é divisível por 3, o que equivale a escrever $2^{2k} - 1 = 3m$ para algum inteiro m , ou $2^{2k} = 3m + 1$. Desejamos mostrar que $2^{2(k+1)} - 1$ é divisível por 3.

$$\begin{aligned} 2^{2(k+1)} - 1 &= 2^{2k+2} - 1 \\ &= 2^2 \cdot 2^{2k} - 1 \\ &= 2^2 \cdot (3m + 1) - 1 \quad (\text{pela hipótese de indução}) \\ &= 12m + 4 - 1 \\ &= 12m + 3 \\ &= 3(4m + 1) \text{ onde } 4m + 1 \text{ é um inteiro.} \end{aligned}$$

Logo $2^{2(k+1)} - 1$ é divisível por 3.

•

Em alguns casos, pode ser que, para o primeiro passo do processo de indução, seja mais apropriado começar com valores 0, 2, ou 3, ao invés de 1. O mesmo princípio se aplica, qualquer que seja o degrau da escada que você alcance no primeiro passo.

EXEMPLO 14 Prove que $n^2 > 3n$ para $n \geq 4$.

Neste exemplo devemos aplicar a indução e começar com um passo inicial $P(4)$. (Se testarmos para valores de $n = 1, 2$ e 3, verificaremos que a desigualdade não se verifica.) $P(A)$ é então a desigualdade $4^2 > 3(4)$, ou $16 > 12$, que é verdadeira. A hipótese de indução é $k^2 > 3k$, para $k > 4$, e queremos mostrar que $(k+1)^2 > 3(k+1)$.

$$\begin{aligned} (k+1)^2 &= k^2 + 2k + 1 \\ &> 3k + 2k + 1 \quad (\text{pela hipótese de indução}) \\ &\geq 3k + 8 + 1 \quad (\text{já que } k \geq 4) \\ &> 3k + 3 \\ &= 3(k+1) \end{aligned}$$

PRÁTICA 7 Prove que $2^{n+1} < 3^n$ para todo $n > 1$.

•

Podemos lançar mão de uma prova por indução para aplicações que não sejam tão óbvias quanto às dos exemplos acima. Isto geralmente acontece quando a sentença que desejamos demonstrar contém uma variável que pode assumir valores inteiros arbitrários e não-negativos.

EXEMPLO 15 Uma linguagem de programação pode ser projetada com as seguintes convenções no que se refere à multiplicação. Um fator simples não requer parênteses, mas o produto "o vezes b " deve ser escrito como $(a)b$. Desejamos mostrar que qualquer produto de fatores precisa de um número par de parênteses para ser escrito. A prova é feita por indução, considerando-se como variável o número de fatores. Para um único fator usa-se 0 (zero) parêntese, um número par. Suponhamos que para qualquer produto de k fatores, usamos um número par de parênteses. Consideremos agora o produto P de $k+1$ fatores. P pode ser escrito na forma r vezes s , onde r tem k fatores e s é um fator único. Pela hipótese de indução, r tem um número par de parênteses. Então pode-se escrever r vezes s como $(r)s$, adicionando-se mais dois parênteses ao número par de parênteses em r , e resultando assim um número par de parênteses para P . •

É possível se enganar ao construir uma prova por indução. Quando demonstramos que $P(k+1)$ é verdadeira, sem nos valermos da hipótese $P(k)$, nós elaboramos uma prova direta de $P(k+1)$, onde $k+1$ é arbitrário. A prova deveria ser reescrita para explicitar que é uma prova direta de $P(n)$ para qualquer n , e não uma prova por indução.

Indução Completa

O princípio da indução que tem sido utilizado,

$$\left. \begin{array}{l} 1. P(1) \text{ verdadeira} \\ 2. (\forall k) [P(k) \text{ verdadeira} \rightarrow P(k+1) \text{ verdadeira}] \end{array} \right\} \rightarrow P(n) \text{ verdadeira para todos os } n \text{ inteiros positivos.}$$

é também conhecido como **indução fraca** em oposição ao princípio da **indução forte** ou **indução completa**:

$$\left. \begin{array}{l} 1'. P(1) \text{ verdadeira} \\ 2'. (\forall k) [P(r) \text{ verdadeira para todo } r \\ \quad 1 \leq r \leq k \rightarrow P(k+1) \text{ verdadeira}] \end{array} \right\} \rightarrow P(n) \text{ verdadeira para todos os } n \text{ inteiros positivos.}$$

Estes dois princípios de indução diferem nos itens 2 e 2'. No item 2, devemos ser capazes de provar para um inteiro positivo arbitrário k que $P(k+1)$ é verdadeira, tendo por base apenas a premissa de que $P(k)$ é verdadeira. No item 2', podemos assumir que $P(r)$ é verdadeira para todos os inteiros r entre 1 e um inteiro positivo arbitrário k , para provar que $P(k+1)$ é verdadeira. Este fato nos fornece uma boa "munição", de tal forma que podemos algumas vezes ser capazes de provar a implicação em 2', quando não podemos provar a implicação em 2.

O que nos leva a deduzir $(\forall n)P(n)$ em cada caso? Devemos notar que os dois princípios de indução, ou seja, os dois métodos de demonstração são equivalentes. Em outras palavras, se aceitarmos a indução fraca como um princípio válido, então o princípio da indução forte também será válido e vice-versa. Para provar a equivalência entre esses dois princípios de indução, apresentamos um outro princípio chamado **princípio da boa ordenação**: Toda coleção de números inteiros positivos que contenha pelo menos um elemento tem um mínimo.

Devemos então verificar que as seguintes implicações são verdadeiras:

$$\begin{aligned} \text{indução forte} &\rightarrow \text{indução fraca} \\ \text{indução fraca} &\rightarrow \text{princípio da boa ordenação} \\ \text{princípio da boa ordenação} &\rightarrow \text{indução forte} \end{aligned}$$

Como conseqüência, estes princípios são equivalentes, e aceitar qualquer um dos três como verdadeiro significa aceitar também os outros dois.

Para provar que o princípio da indução forte implica a indução fraca, suponhamos como premissa válida a indução forte. Desejamos então mostrar que a indução fraca é válida, ou seja, que podemos concluir $P(n)$ para todo n a partir das afirmações 1 e 2. Se a afirmação 1 é verdadeira, então a afirmação 1' também o é. Se a afirmação 2 é verdadeira, a 2' também é, porque podemos concluir $P(k+1)$ a partir de $P(r)$ para todo r entre 1 e k , apesar de usarmos apenas a condição $P(k)$. (Mais precisamente, a sentença 2' requer que provemos $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k+1)$, mas $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k)$ e, da afirmação 2, $P(k) \rightarrow P(k+1)$, logo $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k+1)$). Pela indução forte, concluímos $P(n)$ para todo n . A prova de que a indução fraca implica a boa ordenação, e que a boa ordenação implica a indução forte, é deixada como exercício na Seção 3.1.

A fim de ilustrar a diferença entre uma demonstração por indução fraca de uma demonstração por indução forte, vamos examinar um exemplo pitoresco que pode ser demonstrado das duas formas.

EXEMPLO 16 Prove que uma cerca de madeira com n estacas tem $n - 1$ seções para qualquer $n \geq 1$ (veja Fig. 2.2a).

Seja $P(n)$ a sentença de que a cerca com n estacas tem $n - 1$ seções, e provemos que $P(n)$ é verdadeira para todo $n \geq 1$.

Usaremos inicialmente a indução fraca. Pela base da indução, $P(1)$ assegura que uma cerca com apenas uma estaca tem 0 seções, o que é claramente verdadeiro (veja Fig. 2.2b). Assuma que $P(k)$ é verdadeira:

uma cerca com k estacas tem $k - 1$ seções

e tente provar $P(k + 1)$:

(?) uma cerca com $k + 1$ estacas tem k seções

Dada uma cerca com $k + 1$ estacas, como podemos relacioná-la com uma cerca com k estacas a fim de poderemos utilizar a hipótese de indução? Podemos remover de uma cerca a última estaca e a última seção (Fig. 2.2c). A cerca remanescente tem então k estacas, e pela hipótese de indução, $k - 1$ seções. Portanto, a cerca original tinha k seções.

Demonstraremos agora o mesmo resultado utilizando a indução forte. A base da indução é a mesma de antes. Como hipótese de indução, assumimos que:

para todo r , $1 \leq r \leq k$, uma cerca com r estacas tem $r - 1$ seções

e tentaremos provar $P(k + 1)$:

(?) uma cerca com $k + 1$ estacas tem k seções.

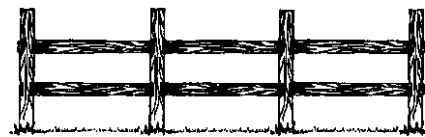
Consideremos uma cerca com $k + 1$ estacas, e vamos separá-la em duas partes, removendo uma seção (Fig. 2.2d). As duas partes da cerca têm r_1 e r_2 estacas, onde $1 \leq r_1 \leq k$, $1 \leq r_2 \leq k$ e $r_1 + r_2 = k + 1$. Pela hipótese de indução, as duas partes têm, respectivamente, $r_1 - 1$ e $r_2 - 1$ seções, logo a cerca original tem:

$(r_1 - 1) + (r_2 - 1) + 1$ seções

(O extra 1 corresponde à seção que foi anteriormente removida.) Através de aritmética simples, conclui-se que

$r_1 + r_2 - 1 = (k + 1) - 1 = k$ seções.

Este resultado prova que uma cerca com $k + 1$ estacas tem k seções, o que verifica $P(k + 1)$ e completa a demonstração por indução forte. •



Cerca com 4 estacas e 3 seções

(a)



Cerca com 1 estaca e 0 seções

(b)



Cerca com a última estaca e a última seção removidas

(c)



Cerca com uma seção removida

(d)

Figura 2.2

O exemplo anterior permitiu o uso das duas formas de demonstração por indução porque é possível remover a última parte da cerca ou ainda separá-la em duas partes a partir de um ponto arbitrário. Para alguns problemas a indução forte é a técnica de demonstração mais indicada, sendo que, para alguns dos quais, o seu uso é absolutamente necessário.

EXEMPLO 17 Prove que para todo $n \geq 2$, n é um número primo ou é um produto de números primos.

Usaremos a indução forte; da mesma forma que na indução fraca, a base da indução não precisa começar com o valor 1, e aqui, por motivos óbvios, começaremos pelo valor 2. $P(2)$ é a sentença de que 2 é um número primo ou o produto de dois primos. Como 2 é primo, $P(2)$ é verdadeira. Suponhamos que para todo r , $2 \leq r \leq k$, $P(r)$ é verdadeira — r é primo ou é o produto de números primos. Tomemos agora o número $k+1$. Se $k+1$ é primo, o resultado se verifica. Se $k+1$ não é primo, então ele é um número composto e pode ser escrito como $k+1 = ab$, onde $1 < a < k+1$ e $1 < b < k+1$, ou $2 \leq a \leq k$ e $2 \leq b \leq k$. A hipótese de indução se aplica a a e a b e, portanto, ou a e b são primos ou são produtos de primos. Logo $k+1$ é o produto de números primos. Isto verifica $P(k+1)$ e completa a prova por indução forte. •

EXEMPLO 18 Prove que qualquer valor postal maior ou igual a oito unidades monetárias pode ser obtido usando-se apenas selos com valores de 3 e 5.

Aqui faremos $P(n)$ denotar que apenas selos nos valores de 3 e 5 são necessários para se obter um valor postal n , e provaremos que $P(n)$ é verdadeira para $n \geq 8$. A base da indução consiste em estabelecer $P(8)$, que resulta da equação

$$8 = 3 + 5$$

Por motivos que oportunamente ficarão mais claros, vamos estabelecer dois resultados adicionais, $P(9)$ e $P(10)$ obtidos das equações

$$9 = 3 + 3 + 3$$

$$10 = 5 + 5$$

Vamos assumir então que $P(r)$ é verdadeira para qualquer r , $8 \leq r \leq k$, e examinar o que ocorre com $P(k+1)$. Podemos assumir que $k+1$ é no mínimo 11, posto que já foi mostrado que $P(r)$ é verdadeira para $r = 8, 9$ e 10 . Se $k+1 \geq 11$, vem $(k+1) - 3 = k - 2 \geq 8$, e pela hipótese de indução, $P(k-2)$ é verdadeira ($P(r)$ é verdadeira para $8 \leq r \leq k$). Portanto $k-2$ pode ser escrita como a soma de 3s e 5s, e adicionando-se o valor 3 obtém-se $k-2+3 = k+1$, o que implica que $k+1$ pode ser escrito como a soma de 3s e 5s. Este resultado comprova que $P(k+1)$ é verdadeira e completa a demonstração.

PRÁTICA 8

- Por que os casos $P(9)$ e $P(10)$ são provados separadamente no Exemplo 18?
- Por que não podemos usar indução fraca na demonstração do Exemplo 18?

•

Revisão da Seção 2.2

Técnicas

- Uso da indução fraca em demonstrações
- Uso da indução forte em demonstrações

Idéias Principais

A indução matemática é uma técnica usada para demonstrar propriedades de números inteiros positivos.

Uma demonstração por indução não precisa começar no valor 1.

A indução pode ser usada para demonstrar resultados sobre quantidades, cujos valores são inteiros não-negativos arbitrários.

A indução fraca e a indução forte podem ser usadas para demonstrarem o mesmo resultado; no entanto, dependendo da situação, uma ou outra abordagem pode ser mais fácil de ser utilizada.

Exercícios 2.2

Nos Exercícios 1 a 16, use a indução matemática para demonstrar que os resultados são válidos para qualquer inteiro positivo n .

$$\star 1. 2 + 6 + 10 + \dots + (4n - 2) = 2n^2$$

$$2. 2 + 4 + 6 + \dots + 2n = n(n+1)$$

$$\star 3. 1 + 5 + 9 + \dots + (4n - 3) = n(2n - 1)$$

$$4. 1 + 3 + 6 + \dots + \frac{n(n+1)}{2} = \frac{n(n+1)(n+2)}{6}$$

$$\star 5. 4 + 10 + 16 + \dots + (6n - 2) = n(3n + 1)$$

$$6. 5 + 10 + 15 + \dots + 5n = \frac{5n(n+1)}{2}$$

$$7. 1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$8. 1^3 + 2^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$\star 9. 1^2 + 3^2 + \dots + (2n - 1)^2 = \frac{n(2n-1)(2n+1)}{3}$$

$$10. 1^4 + 2^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$11. 1 \cdot 3 + 2 \cdot 4 + 3 \cdot 5 + \dots + n(n+2) = \frac{n(n+1)(2n+7)}{6}$$

$$12. 1 + a + a^2 + \dots + a^{n-1} = \frac{a^n - 1}{a - 1} \text{ para } a \neq 0, a \neq 1$$

$$\star 13. \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \frac{1}{3 \cdot 4} + \dots + \frac{1}{n(n+1)} = \frac{n}{n+1}$$

$$14. \frac{1}{1 \cdot 3} + \frac{1}{3 \cdot 5} + \frac{1}{5 \cdot 7} + \dots + \frac{1}{(2n-1)(2n+1)} = \frac{n}{2n+1}$$

$$\star 15. 1^2 - 2^2 + 3^2 - 4^2 + \dots + (-1)^{n+1}n^2 = \frac{(-1)^{n+1}n(n+1)}{2}$$

$$16. 1 \cdot 1! + 2 \cdot 2! + 3 \cdot 3! + \dots + n \cdot n! = (n+1)! - 1 \text{ onde } n! \text{ é o produto dos } n \text{ inteiros positivos de } 1 \text{ até } n.$$

$\star 17.$ Uma **progressão geométrica (seqüência geométrica)** é uma seqüência de termos onde existe um termo inicial a , e cada termo subsequente é obtido pelo produto do anterior por um **valor constante** r . Prove que a fórmula para a soma dos n primeiros termos de uma seqüência geométrica ($n \geq 1$) é:

$$a + ar + ar^2 + \dots + ar^{n-1} = \frac{a - ar^n}{1 - r}$$

18. Uma **progressão aritmética (seqüência aritmética)** é uma seqüência de termos onde existe um termo inicial a e cada termo subsequente é obtido pela soma de um **valor constante** d ao termo anterior. Prove que a fórmula da soma dos n primeiros termos de uma seqüência aritmética ($n \geq 1$) é:

$$a + (a + d) + (a + 2d) + \cdots + [a + (n - 1)d] = \frac{n}{2} [2a + (n - 1)d]$$

- ★19. Prove que $n^2 > n + 1$ para $n \geq 2$.
20. Prove que $n^2 > 5n + 10$ para $n > 6$.
21. Prove que $2^n > n^2$ para $n \geq 5$.
22. Prove que $n! > n^2$ para $n \geq 4$, onde $n!$ é o produto dos inteiros positivos de 1 a n .
- ★23. Prove que $2^n < n!$ para $n \geq 4$.
24. Prove que $n! < n^n$ para $n \geq 2$.
25. Prove que $(1 + x)^n > 1 + x^n$ para $n > 1, x > 0$.
26. Prove que $\left(\frac{a}{b}\right)^{n+1} < \left(\frac{a}{b}\right)^n$ para $n \geq 1$ e $0 < a < b$.
- ★27. Prove que $1 + 2 + \cdots + n < n^2$ para $n > 1$.
28. a. Tente usar a indução para provar que

$$1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^n} < 2 \quad \text{para } n \geq 1$$

O que deu errado?

b. Prove que

$$1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^n} = 2 - \frac{1}{2^n} \quad \text{para } n \geq 1$$

mostrando assim que

$$1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^n} < 2 \quad \text{para } n \geq 1.$$

Para os Exercícios 29 a 40, prove que as sentenças são verdadeiras para todo inteiro positivo.

- ★29. $2^{3n} - 1$ é divisível por 7.
30. $3^{2n} + 7$ é divisível por 8.
31. $7^n - 2^n$ é divisível por 5.
32. $13^n - 6^n$ é divisível por 7.
- ★33. $2^n + (-1)^{n+1}$ é divisível por 3.
34. $2^{5n+1} + 5^{n+2}$ é divisível por 27.
35. $3^{4n+2} + 5^{2n+1}$ é divisível por 14.
36. $7^{2n} + 16n - 1$ é divisível por 64.

★37. $10^n + 3 \cdot 4^{n+2} + 5$ é divisível por 9.

38. $n^3 - n$ é divisível por 3.

39. $n^3 + 2n$ é divisível por 3.

40. $x^n - 1$ é divisível por $x - 1$ para $x \neq 1$.

41. Demonstre o **teorema de DeMoivre**:

$$(\cos \theta + i \operatorname{sen} \theta)^n = \cos n\theta + i \operatorname{sen} n\theta$$

para todo $n \geq 1$. *Dica:* Utilize as fórmulas da trigonometria:

$$\cos(\alpha + \beta) = \cos \alpha \cos \beta - \operatorname{sen} \alpha \operatorname{sen} \beta$$

$$\operatorname{sen}(\alpha + \beta) = \operatorname{sen} \alpha \cos \beta + \cos \alpha \operatorname{sen} \beta$$

42. Prove que

$$\operatorname{sen} \theta + \operatorname{sen} 3\theta + \cdots + \operatorname{sen}(2n-1)\theta = \frac{\operatorname{sen}^2 n\theta}{\operatorname{sen} \theta}$$

para todo $n \geq 1$ e todo θ tal que $\operatorname{sen} \theta \neq 0$.

★43. Use a indução para provar que o produto de quaisquer três inteiros positivos consecutivos é divisível por 3.

44. Suponha que a exponenciação é definida pela equação

$$x^j \cdot x = x^{j+1}$$

para qualquer $j \geq 1$. Use a indução para provar que $x^n \cdot x^m = x^{n+m}$ para $n \geq 1, m \geq 1$.
(*Dica:* Aplique indução em m para um valor arbitrário e fixo de n .)

45. Uma cadeia de Os e Is deve ser processada e convertida para uma cadeia de paridade-par somando-se um bit de paridade no final da cadeia. O bit de paridade é inicialmente 0. Quando um caracter 0 é processado, o bit de paridade permanece inalterado. Quando um caracter 1 é processado, o bit de paridade muda de 0 para 1 ou de 1 para 0. Prove que o número de Is numa cadeia final, ou seja, incluindo o bit de paridade, é sempre par. (*Dica:* considere várias possibilidades.)

★46. O que está errado com a seguinte "demonstração" por indução matemática? Iremos provar que, para todo inteiro positivo n , n é igual a 1 mais n . Suponha que $P(k)$ é verdadeira,

$$k = k + 1$$

Somando 1 a ambos os lados da equação, vem:

$$k + 1 = k + 2$$

logo $P(k+1)$ é verdadeira.

47. O que está errado com a seguinte "demonstração" por indução matemática? Iremos provar que todos os computadores são construídos pelo mesmo fabricante. Em particular, iremos provar que para qualquer conjunto de n computadores, onde n é um inteiro positivo, todos os computadores desse conjunto são construídos pelo mesmo fabricante. Inicialmente provaremos $P(1)$, o que é trivial, pois um conjunto de um único computador tem apenas um fabricante. Agora vamos assumir $P(k)$, ou seja, em qualquer conjunto de k computadores, todos os computadores foram construídos pelo mesmo fabricante. Para provar $P(k+1)$, tomemos qualquer dos $k+1$ computadores. Coloque um dos $k+1$ computadores (chame-o de HAL) fora do conjunto. Por nossa suposição, os k computadores remanescentes têm o mesmo fabricante. Troquemos HAL de posição com um dos k computadores. No novo grupo de k computadores, pela

hipótese de indução, todos têm o mesmo fabricante. Então o fabricante de HAL é o mesmo dos outros computadores, o que prova que os $k+1$ computadores têm o mesmo fabricante.

48. Uma pitoresca tribo nativa tem apenas três palavras na sua língua, *cuco*, *cuca* e *caco*. Novas palavras são compostas pela concatenação destas palavras em qualquer ordem, por exemplo *cucacucocacocuca*. Use a indução completa (para o número de subpalavras na palavra) para provar que qualquer palavra nesta língua tem um número par de c 's.
- ★49. Demonstre que qualquer valor postal maior ou igual a duas unidades monetárias pode ser obtido usando-se somente selos com valor de 2 e 3.
50. Prove que qualquer valor postal maior ou igual a 12 pode ser obtido usando-se somente selos com valores de 4 e 5.
51. Prove que qualquer valor postal maior ou igual a 14 unidades monetárias pode ser obtido usando-se apenas selos de 3 e 8.
- ★52. Prove que qualquer valor postal maior ou igual a 64 unidades monetárias pode ser obtido usando-se somente selos de 5 e 17.
53. Em qualquer grupo de k pessoas, $k \geq 1$, cada um deve apertar a mão de todas as outras pessoas. Encontre uma fórmula que forneça o número de apertos de mão, e demonstre-a como verdadeira, usando indução.

Seção 2.3 Recursão e Relação de Recorrência

Definições Recursivas

Uma definição na qual o item que está sendo definido aparece como parte da definição é chamada **definição indutiva** ou **definição recursiva**. À primeira vista isto pode parecer sem sentido — como algo pode ser definido em termos dele próprio? Este procedimento funciona porque as definições recursivas são compostas de duas partes:

1. uma base, onde alguns casos simples do item que está sendo definido são dados explicitamente, e
2. um passo indutivo ou recursivo, onde outros casos do item que está sendo definido são dados em termos dos casos anteriores.

A parte 1 nos fornece um ponto de partida na medida em que trata alguns casos simples; enquanto a parte 2 nos permite construir novos casos a partir desses casos simples, para então construir outros casos a partir desses novos, e assim por diante. (A analogia com demonstrações por indução matemática justifica o nome "definição indutiva". Em uma demonstração por indução, existe a base da indução, isto é, a demonstração de $P(1)$ — ou a demonstração de P para algum outro valor inicial — e existe a hipótese indutiva, onde a validade de $P(k+1)$ é deduzida da validade de P para valores menores.)

A recursão é uma idéia importante que pode ser utilizada para definir seqüências de objetos, coleções mais gerais de objetos e operações sobre objetos. (O Predicado Prolog *na-cadeia-alimentar* da Seção 1.5 foi definido recursivamente.) Até mesmo algoritmos podem ser definidos recursivamente.

Seqüências Recursivas

Uma **seqüência** S é uma lista de objetos que são enumerados segundo alguma ordem; existe um primeiro objeto, um segundo, e assim por diante. $S(k)$ denota o k -ésimo objeto da seqüência. Uma seqüência é definida recursivamente, explicitando-se seu primeiro valor (ou seus primeiros valores) e, a partir daí, definindo-se outros valores na seqüência em termos dos valores iniciais.

EXEMPLO 19 A seqüência S é definida recursivamente por:

1. $S(1) = 2$
2. $S(2) = 2S(n-1)$ para $n \geq 2$

Pela afirmação 1, $5(1)$, o primeiro objeto em S , é 2. Então pela sentença 2, o segundo objeto em S é $5(2) = 25(1) = 2(2) = 4$. Novamente por 2, $S(3) = 2S(2) = 2(4) = 8$. Dessa forma podemos deduzir que S é a seqüência

$$2, 4, 8, 16, 32, \dots$$

Uma regra semelhante à da sentença 2 no Exemplo 19, na qual se define um valor da seqüência a partir de um ou mais valores anteriores, é chamada **relação de recorrência**.

PRÁTICA 9 A seqüência T é definida recursivamente como se segue:

1. $T(1) = 1$
2. $T(n) = T(n-1) + 3$ para $n \geq 2$

Escreva os primeiros cinco valores da seqüência T .

EXEMPLO 20 A famosa **seqüência de Fibonacci** é definida recursivamente por:

$$\begin{aligned} F(1) &= 1 \\ F(2) &= 2 \\ F(n) &= F(n-2) + F(n-1) \text{ para } n > 2 \end{aligned}$$

Aqui os primeiros dois valores na seqüência são dados, e a relação de recursão define o n -ésimo termo a partir dos dois anteriores.

PRÁTICA 10 Escreva os oito primeiros valores da seqüência de Fibonacci.

A seqüência de Fibonacci tem sido estudada exaustivamente. Por ter sido definida de forma recursiva, podemos usar a indução para provar muitas de suas propriedades.

EXEMPLO 21 Prove que na seqüência de Fibonacci,

$$F(n+4) = 3F(n+2) - F(n) \text{ para todo } n \geq 1.$$

Para a base da indução, devemos mostrar dois casos, $n = 1$ e $n = 2$. Para $n = 1$, temos:

$$F(5) = 3F(3) - F(1)$$

ou (usando valores obtidos na Prática 10)

$$5 = 3(2) - 1$$

que é verdadeiro. Para $n = 2$,

$$F(6) = 3F(4) - F(2)$$

ou

$$8 = 3(3) - 1$$

que também é verdadeiro. Suponhamos agora que para todo r , $1 \leq r \leq k$,

$$F(r+4) = 3F(r+2) - F(r).$$

Agora, mostremos o caso $k+1$, onde $k+1 \geq 3$. Então, desejamos mostrar que:

$$F(k+1+4) \stackrel{?}{=} 3F(k+1+2) - F(k+1)$$

ou

$$F(k+5) \stackrel{?}{=} 3F(k+3) - F(k+1)$$

Da relação de recorrência na sequência de Fibonacci temos:

$$F(k+5) = F(k+3) + F(k+4)$$

e, pela hipótese de indução, para $r = k-1$ e $r = k$, respectivamente, vem:

$$F(k+3) = 3F(k+1) - F(k-1)$$

e

$$F(k+4) = 3F(k+2) - F(k)$$

Logo

$$\begin{aligned} F(k+5) &= 3F(k+1) - F(k-1) + 3F(k+2) - F(k) \\ &= 3[F(k+1) - F(k+2)] - [F(k-1) - F(k)] \\ &= 3F(k+3) - F(k+1) \text{ (usando novamente a relação de recorrência)} \end{aligned}$$

A indução completa foi necessária aqui, porque a relação de recorrência para a sequência de Fibonacci usa mais termos do que o termo imediatamente anterior. •

PRÁTICA 11 Na demonstração do Exemplo 21, por que é necessário provar $n = 2$ como um caso especial? •

Conjuntos Recursivos

Os objetos de uma sequência são ordenados — existe um primeiro objeto, um segundo, e assim por diante. Um *conjunto* de objetos é uma coleção de objetos na qual nenhuma regra de ordenação é imposta. Alguns conjuntos podem ser definidos recursivamente.

EXEMPLO 22 Na Seção 1.1 notamos que certas cadeias de proposições, conectivos lógicos e parênteses tais como $(A \wedge B)' \vee C$, são considerados válidos, enquanto outras cadeias como $\wedge \wedge A''B$ não são válidas. A sintaxe para ordenar estes símbolos constitui a definição do conjunto de fórmulas proposicionais bem-formuladas, que é uma definição recursiva.

1. Qualquer proposição é uma wff.
2. Se P e Q são wffs, então também o serão $(P \wedge Q)$, $(P \vee Q)$, $(P \rightarrow Q)$, (P') e $(P \leftrightarrow Q)$.

Nós normalmente omitimos parênteses quando isso não nos causa confusão; então escrevemos $(P \vee Q)$ como $P \vee Q$, ou (P') como P' . Iniciando com uma proposição e usando a regra 2 repetidamente, podemos construir qualquer wff proposicional. Por exemplo, A, B , e C são wffs pela regra 1. Pela regra 2,

$$(A \wedge B) \text{ e } (C')$$

são também wffs. Novamente pela regra 2,

$$[(A \wedge B) \rightarrow (C')]$$

é uma wff. Aplicando a regra 2 mais uma vez, obtém-se a wff

$$([(A \wedge B) \rightarrow (C')])'$$

Eliminando alguns parênteses, podemos escrever a wff como

$$[(A \wedge B) \rightarrow C']'$$

PRÁTICA 12 Mostre como construir a wff $[(A \vee (B')) \rightarrow C]$ a partir da definição do Exemplo 22. •

PRÁTICA 13 Uma definição recursiva para o conjunto de pessoas que são ancestrais de João pode ser construída a partir da seguinte premissa:

Os pais de João são ancestrais de João

Forneça a base da indução. •

Cadeias de símbolos obtidas a partir de um "alfabeto" finito são objetos normalmente encontrados na ciência da computação. Os computadores armazenam dados em **cadeias binárias**, ou seja, cadeias do alfabeto consistindo em 0s e 1s; os compiladores entendem as instruções de um programa como cadeias de *tokens*, tais como palavras-chave e identificadores. A coleção de todas as cadeias de tamanho finito de um alfabeto, normalmente chamadas de cadeias *sobre* um alfabeto, podem ser definidas recursivamente (veja o próximo exemplo). Muitos conjuntos de cadeias de caracteres com propriedades especiais têm também definições recursivas.

EXEMPLO 23 O conjunto de todas as cadeias de símbolos (de tamanho finito) sobre um alfabeto A é denotado por A^* . A definição recursiva de A^* é:

1. A **cadeia** vazia Λ (a cadeia sem símbolos) pertence a A^* .
2. Qualquer elemento de A pertence a A^* .
3. Se x e y são cadeias em A^* , então a **concatenação** xy também pertence a A^* .

As partes 1 e 2 constituem a base da recursão, e a parte 3 é o passo recursivo desta definição. Perceba que para qualquer cadeia x , $x\Lambda = \Lambda x = x$.

PRÁTICA 14 Se $x = 1.011$ e $y = 001$, escreva as cadeias xy , yx , e $yx\Lambda x$.

PRÁTICA 15 Forneça uma definição recursiva para o conjunto de todas as cadeias binárias que são **palíndromos**, ou seja, cadeias iguais, se lidas do início para o fim ou ao contrário.

EXEMPLO 24 Suponha que em uma certa linguagem de programação, os identificadores podem ser cadeias alfanuméricas de tamanho arbitrário, mas que devem começar com uma letra. Uma definição recursiva para um conjunto de cadeias desta natureza é:

1. Uma letra é um identificador.
2. Se A é um identificador, então a concatenação de A com qualquer letra ou dígito também é um identificador.

Uma notação mais simbólica para descrever conjuntos de cadeias que são definidas recursivamente é chamada **forma de Backus Naur**, ou **BNF**, originalmente desenvolvida para definir a linguagem de programação ALGOL. Na notação BNF, itens que são definidos em termos de outros itens são envolvidos por menor e maior ($< >$), enquanto itens específicos que não são definidos mais adiante não aparecem entre menor e maior. A linha vertical $|$ indica uma escolha com o mesmo significado da palavra *ou*. A definição BNF de um identificador é:

$$\begin{aligned}\langle \text{identificador} \rangle &::= \langle \text{letra} \rangle \mid \langle \text{identificador} \rangle \langle \text{letra} \rangle \mid \langle \text{identificador} \rangle \langle \text{dígito} \rangle \\ \langle \text{letra} \rangle &::= a \mid b \mid c \mid \dots \mid z \\ \langle \text{dígito} \rangle &::= 0 \mid 1 \mid \dots \mid 9\end{aligned}$$

Então o identificador *mel* é obtido da definição por uma sequência de escolhas tais como:

$$\begin{aligned}\langle \text{identificador} \rangle &\text{ pode ser } \langle \text{identificador} \rangle \langle \text{dígito} \rangle \\ &\text{ que pode ser } \langle \text{identificador} \rangle^2 \\ &\text{ que pode ser } \langle \text{identificador} \rangle \langle \text{letra} \rangle^2 \\ &\text{ que pode ser } \langle \text{identificador} \rangle e^2 \\ &\text{ que pode ser } \langle \text{letra} \rangle e^2 \\ &\text{ que pode ser } me^2\end{aligned}$$

Operações Recursivas

Podemos definir recursivamente certas operações sobre objetos, como nos exemplos abaixo.

EXEMPLO 25 Uma definição recursiva para a operação de exponenciação a^n , definida para um número real a diferente de zero e n inteiro não-negativo é:

1. $a^0 = 1$
2. $a^n = (a^{n-1})a$ para $n \geq 1$

•

EXEMPLO 26 Uma definição recursiva para a multiplicação de dois inteiros positivos m e n é:

1. $m(1) = m$
2. $m(n) = m(n-1) + m$ para $n \geq 2$

•

PRÁTICA 16 Seja x uma cadeia sobre algum alfabeto. Forneça uma definição recursiva para a operação x^n (concatenação de x com ele próprio n vezes) para $n \geq 1$.

•

Na Seção 1.1, definimos a operação de disjunção lógica de duas proposições. Isso pode servir como a base da recursão para a definição recursiva da disjunção de n proposições, para $n \geq 2$:

1. $A_1 \vee A_2$ definida como na Seção 1.1
2. $A_1 \vee \cdots \vee A_n = (A_1 \vee \cdots \vee A_{n-1}) \vee A_n$ para $n > 2$

(D)

Usando esta definição, podemos generalizar a propriedade associativa da disjunção (equivalência tautológica 2a) para afirmar que em uma disjunção de n proposições, o grupamento por parênteses é desnecessário, porque todos esses grupamentos são equivalentes à expressão geral da disjunção de n proposições. Simbolicamente, para qualquer n , com $n \geq 3$ e qualquer p , com $1 \leq p \leq n-1$,

$$(A_1 \vee \cdots \vee A_p) \vee (A_{p+1} \vee \cdots \vee A_n) \Leftrightarrow A_1 \vee \cdots \vee A_n$$

Esta equivalência pode ser provada por indução em n . Para $n = 3$,

$$\begin{aligned} A_1 \vee (A_2 \vee A_3) &\Leftrightarrow (A_1 \vee A_2) \vee A_3 && \text{(pela equivalência 2a)} \\ &= A_1 \vee A_2 \vee A_3 && \text{(pela equação (1))} \end{aligned}$$

Suponha que para $n = k$ e $1 \leq p \leq k-1$

$$(A_1 \vee \cdots \vee A_p) \vee (A_{p+1} \vee \cdots \vee A_k) \Leftrightarrow A_1 \vee \cdots \vee A_k$$

Então para $1 \leq p \leq k$

$$\begin{aligned} &(A_1 \vee \cdots \vee A_p) \vee (A_{p+1} \vee \cdots \vee A_{k+1}) \\ &= (A_1 \vee \cdots \vee A_p) \vee [(A_{p+1} \vee \cdots \vee A_k) \vee A_{k+1}] && \text{(pela equação (1))} \\ &\Leftrightarrow [(A_1 \vee \cdots \vee A_p) \vee (A_{p+1} \vee \cdots \vee A_k)] \vee A_{k+1} && \text{(pela equivalência 2a)} \\ &\Leftrightarrow (A_1 \vee \cdots \vee A_k) \vee A_{k+1} && \text{(pela hipótese de indução)} \\ &= A_1 \vee \cdots \vee A_{k+1} && \text{(pela equação (1))} \end{aligned}$$

Algoritmos Recursivos

O Exemplo 19 fornece a definição recursiva de uma sequência S . Suponha que desejamos escrever um programa de computador para obter $S(n)$ para algum inteiro positivo n . Podemos utilizar duas abordagens diferentes. Se desejamos encontrar $S(12)$, por exemplo, podemos começar com $S(1) = 2$ e então computar $S(2)$, $S(3)$, e assim por diante, como fizemos no Exemplo 19, até que finalmente obtemos $S(12)$. Esta abordagem, sem dúvida, envolve iterações através de uma espécie de laço. Uma versão desse *algoritmo iterativo* é mostrada abaixo, implementada como uma função em Pascal.

```

function S(n: integer): integer;
  {cálculo iterativo do valor  $S(n)$  para a seqüência de Exemplo 19}

  var
    i: integer;           {índice do loop}
    ValorCorrente: integer; {valor corrente da função}

  begin
    if n = 1 then
      5 := 2
    else
      begin
        i := 2;
        ValorCorrente := 2;
        while i ≤ n do
          begin
            ValorCorrente := 2 * ValorCorrente;
            i := i + 1;
          end;

          {o ValorCorrente tem agora o valor  $S(n)$ }
          S := ValorCorrente;
        end;
      end;
  end;

```

A base, quando $n = 1$, é obtida na cláusula **then** da instrução **if**. A cláusula **else**, para $n > 1$, realiza uma inicialização e, a seguir, entra no laço **while**, que calcula os demais valores da seqüência, até que o limite superior da mesma é alcançado. Você pode acompanhar a execução passo a passo deste algoritmo para valores pequenos de n , de forma a se convencer de que ele funciona.

A segunda abordagem utilizada para calcular $S(n)$ usa a definição de recursão de 5 diretamente. Uma versão do *algoritmo recursivo*, implementado novamente em Pascal, é apresentada a seguir.

```

function S(n : integer):integer;
  {Calcula recursivamente os valores de  $S(n)$  para a seqüência do Exemplo 19}

  begin
    if n = 1 then
      S := 2
    else
      S := 2 * S(n-1);
    end;

```

O corpo desta função consiste em uma única estrutura **if-then-else**. Para entendermos o seu funcionamento, vamos acompanhar a execução da mesma para calcular $S(3)$. A função é inicialmente chamada para o valor de entrada $n = 3$. Como n é diferente de 1, a execução é direcionada para a cláusula **else**. Neste ponto o procedimento para calcular $S(3)$ é momentaneamente suspenso, até que o valor de $S(2)$ seja conhecido. Qualquer informação relevante para a obtenção de $S(3)$ é armazenada na memória do computador numa *pilha*, para ser posteriormente carregada quando o cálculo puder ser completado. (Uma pilha é uma coleção de dados onde qualquer novo item ao entrar é armazenado no topo, e somente o item do topo da pilha pode ser removido ou acessado. Uma pilha é então uma estrutura do tipo *LIFO* - last in, first out.) A função é novamente chamada para um valor de entrada $n = 2$. Novamente a cláusula **else** é executada e o cálculo de $S(2)$ é momentaneamente suspenso com as informações relevantes armazenadas na pilha, enquanto a função é chamada novamente, agora com $n = 1$ como entrada.

Desta vez o valor de retorno, 2, pode ser calculado diretamente pela cláusula **then** da função. Esta última chamada à função retorna este valor à penúltima chamada, que pode recuperar qualquer informação relevante para o caso $n = 2$ da pilha, calcular $S(2)$ e disponibilizar o resultado à chamada anterior (a primeira chamada). Finalmente, esta chamada inicial de S é capaz de esvaziar a pilha e concluir seus cálculos, retornando o valor de $S(3)$.

Quais são as vantagens e desvantagens de algoritmos iterativo e recursivo para executar uma determinada tarefa? Neste exemplo, a versão recursiva é menor, pois dispensa o gerenciamento do laço. Descrever a

execução de uma versão recursiva é mais complexo do que descrever a versão iterativa, porém todos os passos são realizados automaticamente. Não precisamos saber o que está acontecendo internamente, é necessário apenas termos conhecimento de que uma quantidade grande de chamadas pode demandar muita memória para o armazenamento na pilha das informações necessárias às chamadas anteriores. Se for necessária mais memória do que o que se tem disponível, ocorre um "estouro de pilha". Além de usar muita memória, algoritmos recursivos podem demandar muitos outros procedimentos computacionais e, por isso, podem ser mais lentos do que os não-recursivos (ver Exercício 7, Seção No Computador, no fim deste capítulo).

De alguma forma, a recursão fornece uma maneira natural de se abordar uma grande variedade de problemas, alguns dos quais poderiam ter soluções não-recursivas extremamente complexas. Problemas onde se desejam calcular valores para uma seqüência definida recursivamente são naturalmente escritos em algoritmos recursivos. Muitas linguagens de programação permitem o uso de recursão (infelizmente, nem todas).

PRÁTICA 17 Escreva o corpo da função recursiva para computar $T(n)$ para a seqüência T definida na Prática 9. •

EXEMPLO 27 No Exemplo 26, uma definição recursiva foi dada para multiplicar dois inteiros positivos m e n . Uma versão em Pascal de um algoritmo recursivo para multiplicação baseada nesta definição é dada abaixo:

```
function Produto( $m,n$ :integer):integer;
{calcula recursivamente o produto de  $m$  por  $n$ }

begin
    if  $n = 1$  then
        Produto :=  $m$ 
    else
        Produto := Produto( $m, n - 1$ ) +  $m$ ;
end;
```

Um algoritmo recursivo chama ele próprio para valores "menores" que o valor de entrada. Suponha um problema que pode ser resolvido pela solução de versões menores do mesmo problema, e que essas versões tornam-se, em algum momento, casos triviais que podem ser manipulados facilmente. Então, um algoritmo recursivo pode ser útil, mesmo que o problema original não seja colocado de forma recursiva.

Para nos convenceremos de que um algoritmo recursivo funciona não precisamos começar com um particular valor de entrada, e a partir daí trabalhar com os valores anteriores até alcançarmos o caso trivial e então voltar para calcular o valor inicial. Acompanhamos este procedimento para computar o valor de $S(3)$ apenas para ilustrar o mecanismo computacional de recursividade. Em vez disso, nós poderíamos verificar o caso trivial (da mesma forma que verificamos o passo básico em uma prova por indução) e verificar que se o algoritmo funciona corretamente quando é chamado para valores de entrada menores, então ele resolve o problema para o valor original de entrada (este fato é semelhante a provar $P(k+1)$ a partir da hipótese $P(k)$ em uma prova por indução).

EXEMPLO 28 Uma das tarefas mais comuns em processamento de dados é ordenar uma lista L de n termos em ordem crescente ou decrescente numérica ou alfabeticamente. (A lista pode consistir, por exemplo, em nomes de clientes, e na ordenação "Zé das Couves" deve vir depois de "João da Silva".)

ALGORITMO *SelectionSort*

```
procedure SelectionSort( $L$ : lista;  $j$ : integer);
{ordenação recursiva crescente dos itens de 1 até  $j$  da lista  $L$ }

begin
    if  $j = 1$  then
        ordenação está completa, imprima a lista ordenada
    else
        begin
            encontre o índice  $i$  do maior elemento de  $L$  entre 1 e  $j$ ;
            troque  $L(i)$  com  $L(j)$ ;
            SelectionSort( $L, j - 1$ );
        end;
    end;
```

O algoritmo de **selectionsort** — um algoritmo simples, mas pouco eficiente — é descrito em pseudocódigo no quadro acima. Ele ordena os primeiros j itens de uma lista L em ordem crescente; quando o procedimento é inicialmente chamado j tem o valor n (então a primeira chamada, em última análise, ordena a lista inteira). A parte recursiva do algoritmo reside na cláusula **else**; o algoritmo examina a seção da lista que está sendo considerada e determina i de tal forma que $L(i)$ seja o maior valor. O algoritmo troca então $L(i)$ e $L(j)$, depois disso o máximo valor ocorre na posição j , ou seja, na última posição da seção considerada da lista. $L(j)$ agora está correto e não deve mais ser alterado, este processo é repetido para a lista $L(1)$ a $L(j-1)$. Se esta parte da lista for ordenada corretamente, então a lista inteira também estará corretamente ordenada. Tão logo j assuma o valor 1 (a cláusula **then**) a parte da lista submetida à ordenação consistirá em uma única entrada, a qual certamente estará no lugar correto. Neste ponto a lista inteira estará ordenada. •

EXEMPLO 29 Agora que ordenamos nossa lista, outra tarefa comum é procurar se um particular item ocorre na lista. (Será que João da Silva é um cliente?) Uma técnica eficiente de busca para uma lista ordenada é o **algoritmo de busca binária** recursivo que é descrito em pseudocódigo no quadro a seguir. O algoritmo procura o valor x na seção da lista L entre $L(i)$ e $L(j)$; inicialmente i e j têm os valores 1 e n , respectivamente. A cláusula **then** do primeiro **if** é a base da recursão que determina que x não pode ser encontrado em uma lista vazia, ou seja, uma lista onde o primeiro índice excede o último. Na cláusula **else**, devemos obter o item mediano da seção considerada da lista. (Se a seção contém um número ímpar de itens, existe realmente um item mediano; se a seção contém um número par de itens, é suficiente tomar como mediano o último item da primeira metade da seção em questão.) Comparando x com o valor mediano, ou localizamos x ou determinamos em qual metade da lista devemos continuar a busca. •

ALGORITMO *BuscaBinária*

```

procedure BuscaBinária( $L$ : lista;  $i, j$ : integer;  $x$ : tipoitem);
{procura o item  $x$  em uma lista  $L$  entre  $L(i)$  e  $L(j)$ }

begin
  if  $i > j$  then
    write('não encontrado')
  else
    begin
      encontre o índice  $k$  do item mediano na lista  $L(i) \text{ — } L(j)$ ;
      if  $x = \text{item mediano}$  then
        write('encontrado')
      else
        if  $x < \text{item mediano}$  then
          BuscaBinária ( $L, i, k-1, x$ )
        else
          BuscaBinária ( $L, k+1, j, x$ );
        end;
    end;
end;

```

EXEMPLO 30 Suponha uma lista com os seguintes números:

3, 7, 8, 10, 14, 18, 22, 34

e o valor de busca x igual a 25. A lista inicial é não-vazia, logo podemos localizar e determinar o item mediano que, no caso, é 10. Comparamos então x com o item mediano. Como $x > 10$, a busca é realizada na segunda metade da lista, ou seja

14, 18, 22, 34

Aqui novamente a lista é não-vazia, e o valor mediano é 18. Como $x > 18$ a busca é realizada na segunda metade da lista, ou seja,

22, 34

Para esta lista não-vazia, o valor mediano é 22. Como $x > 22$, a busca continua na segunda metade da lista, ou seja

34

Esta é uma lista com um único elemento, com o valor mediano igual a este elemento. Como $x < 34$ a busca deve começar na "primeira metade" da lista que, no caso, está vazia. Neste ponto o algoritmo é encerrado informando que o valor procurado x não está na lista.

Este procedimento exigiu ao todo quatro comparações, x foi comparado, a cada momento, a 10, 18, 22 e 34. •

PRÁTICA 18

Na busca binária da lista no Exemplo 30, obtenha os valores com os quais x é comparado, caso x assumo o valor 8. •

Resolvendo Relações de Recorrência

Para obtermos o valor $S(n)$ da seqüência S do Exemplo 19, desenvolvemos dois algoritmos, um iterativo e outro recursivo. No entanto, existe ainda uma forma mais fácil para computar $S(n)$. Lembremos que

$$S(1) = 2 \quad (1)$$

$$S(n) = 2S(n-1) \quad \text{para } n \geq 2 \quad (2)$$

Como

$$S(1) = 2 = 2^1$$

$$S(2) = 4 = 2^2$$

$$S(3) = 8 = 2^3$$

$$S(4) = 16 = 2^4$$

e assim por diante, podemos concluir que:

$$S(n) = 2^n \quad (3)$$

Usando a equação (3), podemos escolher um valor para n e computar $S(n)$ sem a necessidade de computarmos inicialmente S para valores menores que n . Uma equação como a (3), onde podemos substituir um valor e obter diretamente o seu resultado, é chamada **solução de forma fechada** para a relação de recorrência (2) sujeita à hipótese básica (base da recorrência) estabelecida em (1). Encontrar uma solução de forma fechada chama-se **resolver** a relação de recorrência. Obviamente, é interessante encontrar uma solução de forma fechada sempre que for possível.

Uma técnica utilizada para resolver relações de recorrência é uma abordagem do tipo "expanda, suponha e verifique", que usa repetidamente a relação de recorrência para expandir a expressão para o n -ésimo termo, até que um padrão geral de comportamento da expressão possa ser deduzido. Finalmente, a suposição é verificada (demonstrada) por indução matemática.

EXEMPLO 31

Consideremos novamente a hipótese básica e a relação de recorrência para a seqüência S do Exemplo 19:

$$S(1) = 2 \quad (4)$$

$$S(n) = 2S(n-1) \quad \text{para } n \geq 2 \quad (5)$$

Vamos supor que ainda não conhecemos a solução de forma fechada, e usar a abordagem de expandir, supor e verificar para encontrá-la. Começando com $S(n)$, expandiremos pelo uso repetido da relação de recorrência. Lembrando que a relação de recorrência é a fórmula que diz que qualquer valor de S pode ser substituído por duas vezes o valor anterior de S . Aplicaremos esta fórmula a S para os valores $n, n-1, n-2$, e assim por diante:

$$\begin{aligned} S(n) &= 2S(n-1) \\ &= 2[2S(n-2)] = 2^2S(n-2) \\ &= 2^2[2S(n-3)] = 2^3S(n-3) \\ &\vdots \\ &\vdots \\ &\vdots \end{aligned}$$

Observando a evolução do padrão, podemos supor que após k expansões sucessivas, a equação terá a forma:

$$S(n) = 2^k S(n-k)$$

Esta expansão dos valores de S em termos dos valores inferiores de S deve parar quando $n - k = 1$, ou seja, quando $k = n - 1$. Neste ponto, teremos

$$\begin{aligned} S(n) &= 2^{n-1} S[n - (n - 1)] \\ &= 2^{n-1} S(1) = 2^{n-1}(2) = 2^n \end{aligned}$$

que é a expressão da solução de forma fechada.

Ainda não terminamos, pois o que fizemos até agora foi supor encontrar um padrão geral. Devemos confirmar agora nossa solução de forma fechada utilizando indução em n . A afirmação que desejamos demonstrar é, portanto, $S(n) = 2^n$ para $n \geq 1$.

Para a hipótese básica, $S(1) = 2^1$. Este resultado é verdadeiro pela equação (4). Suponhamos então que $S(k) = 2^k$. Temos então:

$$\begin{aligned} S(k+1) &= 2 S(k) && \text{(pela equação (5))} \\ &= 2 (2^k) && \text{(pela hipótese indutiva)} \\ &= 2^{k+1} \end{aligned}$$

Este resultado prova que a nossa solução de forma fechada está correta. •

PRÁTICA 19

Encontre uma solução de forma fechada para a relação de recorrência, sujeita à hipótese básica para a sequência T :

1. $T(1) = 1$
2. $T(n) = T(n-1) + 3$ para $n \geq 2$

(Dica: Expanda, suponha e verifique.) •

Os métodos usados na solução de relação de recorrência são semelhantes àqueles usados para resolver equações diferenciais. Em particular, relações de recorrência e equações diferenciais são classificadas em vários tipos, muitos dos quais têm fórmulas de solução que são conhecidas.

Uma relação de recorrência para a sequência $S(n)$ é **linear** se valores anteriores de S que aparecem na definição ocorrem apenas à primeira potência. A relação de recorrência linear mais geral tem a forma:

$$S(n) = f_1(n)S(n-1) + f_2(n)S(n-2) + \dots + f_k(n)S(n-k) + g(n)$$

onde os f_i 's e g podem ser expressões envolvendo n . A relação de recorrência tem **coeficientes constantes** se os f_i 's são todos constantes. Ela é de **primeira ordem** se o n -ésimo termo depende apenas dos termos de ordem $n-1$. Uma relação de recorrência de primeira ordem com coeficientes constantes tem a forma

$$S(n) = cS(n-1) + g(n) \tag{6}$$

Finalmente, uma relação de recorrência é **homogênea** se $g(n) = 0$ para todo n .

Encontraremos a solução de forma fechada da equação (6), a relação de recorrência linear de primeira ordem com coeficientes constantes, sujeita à hipótese básica (base da recorrência) $S(1)$, que é conhecida. Usaremos a abordagem de expandir, supor e verificar. O procedimento a ser implementado aqui é uma generalização do que foi feito no Exemplo 31. Aplicando-se repetidamente a equação (6) e simplificando, obtém-se:

$$\begin{aligned} S(n) &= cS(n-1) + g(n) \\ &= c[cS(n-2) + g(n-1)] + g(n) \\ &= c^2S(n-2) + cg(n-1) + g(n) \\ &= c^2[cS(n-3) + g(n-2)] + cg(n-1) + g(n) \\ &= c^3S(n-3) + c^2g(n-2) + cg(n-1) + g(n) \\ &\vdots \\ &\vdots \\ &\vdots \end{aligned}$$

Após k expansões sucessivas, a fórmula geral parece ser:

$$S(n) = c^k S(n-k) + c^{k-1} g(n-(k-1)) + \dots + cg(n-1) + g(n).$$

Se a seqüência tem o valor base igual a 1, então a expansão termina quando $n - k = 1$ ou $k = n - 1$. Neste ponto teremos:

$$\begin{aligned} S(n) &= c^{n-1}S(1) + c^{n-2}g(2) + \dots + cg(n-1) + g(n) \\ &= c^{n-1}S(1) + c^{n-2}g(2) + \dots + c^1g(n-1) + c^0g(n) \end{aligned} \quad (7)$$

Podemos usar a **notação de somatório** para escrever parte desta expressão de forma mais compacta. A letra grega maiúscula sigma, $\sum$, representa o somatório. A notação $\sum_{i=p}^q (\text{expressão})$ significa que devemos substituir na expressão sucessivos valores de i , o **índice do somatório**, do limite inferior p até o limite superior q , e então somar os resultados obtidos. (Veja Apêndice A mais adiante, para discussões sobre a notação do somatório.)

Então, por exemplo,

$$\sum_{i=1}^7 a(i) = a(1) + a(2) + \dots + a(7)$$

Na notação de somatório a equação (7) será:

$$S(n) = c^{n-1} S(1) + \sum_{i=2}^n c^{n-i} g(i)$$

Podemos usar a indução de forma mais explícita do que a utilizada no Exemplo 31 para verificar que esta fórmula funciona (veja Exercício 67). Entretanto, a solução de forma fechada para a relação de recorrência (6) é

$$S(n) = c^{n-1} S(1) + \sum_{i=2}^n c^{n-i} g(i) \quad (8)$$

EXEMPLO 32 A seqüência $S(n)$ do Exemplo 31,

$$\begin{aligned} S(1) &= 2 \\ S(n) &= 2 S(n-1) \quad \text{para } n \geq 2 \end{aligned}$$

é uma relação de recorrência linear, homogênea, de primeira ordem com coeficientes constantes. Em outras palavras, ela verifica a equação (6) com $c = 2$ e $g(n) = 0$. Da fórmula (8), a solução de forma fechada é

$$S(n) = 2^{n-1}(2) + \sum_{i=2}^n 0 = 2^n$$

o que confirma o resultado previamente obtido no Exemplo 31. •

PRÁTICA 20 Refaça a Prática 19, usando a equação (8). •

Revisão da Seção 2.3

Técnicas

- Geração de valores em uma seqüência definida recursivamente.
- Uso da indução para demonstrar propriedades de uma seqüência definida recursivamente.
- Reconhecimento de objetos de uma coleção definida recursivamente.
- Como fornecer definições recursivas para determinados conjuntos de objetos.
- Como fornecer definições recursivas para certas operações sobre objetos.
- Como escrever algoritmos recursivos para gerar seqüências definidas recursivamente.
- Resolução de relações de recorrência lineares de primeira ordem com coeficientes constantes, usando uma fórmula de solução.
- Resolução de relações de recorrência através da técnica de expansão, suposição e verificação.

Idéias Principais

Podemos usar definições recursivas para seqüências de objetos, conjuntos de objetos e operações sobre objetos, desde que informações básicas sejam conhecidas e novas informações dependam de informações já conhecidas.

Os algoritmos recursivos fornecem uma forma natural de resolver certos problemas, através da utilização da mesma tarefa para solucionar uma versão reduzida do problema.

Certas relações de recorrência têm soluções de forma fechada.

Exercícios 2.3

Para os Exercícios 1 a 10, escreva os cinco primeiros valores das seqüências dadas.

★1. $S(1) = 10$

$$S(n) = S(n-1) + 10 \quad \text{para } n \geq 2$$

2. $A(1) = 2$

$$A(n) = \frac{1}{A(n-1)} \quad \text{para } n \geq 2$$

3. $B(1) = 1$

$$B(n) = B(n-1) + n^2 \quad \text{para } n \geq 2$$

★4. $S(1) = 1$

$$S(n) = S(n-1) + \frac{1}{n} \quad \text{para } n \geq 2$$

5. $T(1) = 1$

$$T(n) = nT(n-1) \quad \text{para } n \geq 2$$

6. $P(1) = 1$

$$P(n) = n^2P(n-1) + (n-1) \quad \text{para } n \geq 2$$

★7. $M(1) = 2$

$$M(2) = 2$$

$$M(n) = 2M(n-1) + M(n-2) \quad \text{para } n > 2$$

8. $D(1) = 3$

$$D(2) = 5$$

$$D(n) = (n-1)D(n-1) + (n-2)D(n-2) \quad \text{para } n \geq 2$$

9. $W(1) = 2$

$$W(2) = 3$$

$$W(n) = W(n-1)W(n-2) \quad \text{para } n > 2$$

10. $T(1) = 1$

$$T(2) = 2$$

$$T(3) = 3$$

$$T(n) = T(n-1) + 2T(n-2) + 3T(n-3) \quad \text{para } n > 3$$

Nos Exercícios 11 e 12, prove as propriedades dadas para os números de Fibonacci diretamente da definição. (Não é necessário usar indução.)

★11. $F(n+1) + F(n-2) = 2F(n) \quad \text{para } n > 2$

12. $[F(n+1)]^2 = [F(n)]^2 + F(n-1)F(n+2) \quad \text{para } n > 1$

Nos Exercícios 13 a 18, demonstre as propriedades dadas para os números de Fibonacci para todo $n \geq 1$.

13. $F(1) + F(2) + \dots + F(n) = F(n+2) - 1$ (Dica: Use a indução fraca.)

14. $F(2) + F(4) + \dots + F(2n) = F(2n+1) - 1$ (Dica: Use a indução fraca.)

★15. $F(n+3) = 2F(n+1) + F(n)$

16. $F(n+6) = 4F(n+3) + F(n)$

17. $F(n) < 2^n$

18. Os valores p e q são definidos como se segue:

$$p = \frac{1 + \sqrt{5}}{2} \text{ e } q = \frac{1 - \sqrt{5}}{2}$$

- Prove que $1 + p = p^2$ e $1 + q = q^2$
- Prove que

$$F(n) = \frac{p^n - q^n}{p - q}$$

- Use o item (b) para provar que

$$F(n) = \frac{\sqrt{5}}{5} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{\sqrt{5}}{5} \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

é uma solução de forma fechada para a sequência de Fibonacci.

- ★19. Escreva uma definição recursiva para uma progressão geométrica com termo inicial a e razão r (veja Exercício 17 na Seção 2.2).
20. Escreva uma definição recursiva para uma progressão aritmética com termo inicial a e razão d (veja Exercício 18, Seção 2.2).
- ★21. Em um experimento, certa colônia de bactérias tem inicialmente uma população de 50.000. Uma leitura é feita a cada duas horas, e no final de cada duas horas de intervalo há três vezes mais bactérias que antes.
- Escreva a definição recursiva para $A(n)$, onde $A(n)$ é o número de bactérias presentes no início do n -ésimo período de tempo.
 - No início de qual intervalo existem 1.350.000 bactérias presentes?
22. Uma quantia de 500 unidades monetárias foi investida em uma conta remunerada a uma taxa de juro composto anual de 10%.
- Escreva a definição recursiva para $P(n)$, a quantia na conta no início do n -ésimo ano.
 - Depois de quantos anos a quantia excederá o valor de 700 unidades monetárias?
23. Uma coleção T de números é definida recursivamente por:
- 2 pertence a T .
 - Se X pertence a T , então $X + 3$ e $2 \cdot X$ também pertencem a T .
- Quais dos seguintes números pertencem a T ?
- 6
 - 7
 - 19
 - 12
24. Uma coleção M de números é definida recursivamente por:
- 2 e 3 pertencem a M .
 - Se X e Y pertencem a M , então $X \cdot Y$ também pertence a M .

Quais dos seguintes números pertencem a M ?

a. 6 b. 9 c. 16 d. 21 e. 26 f. 54 g. 72 h. 218

★25. Uma coleção S de cadeias de caracteres é definida recursivamente por:

1. a e b pertencem a S .

2. Se X pertence a S , então Xb também pertence a S .

Quais das seguintes cadeias pertencem a S ?

a. a b. ab c. aba d. $aaab$ e. $bbbbbb$

26. Uma coleção W de cadeias de símbolos é definida recursivamente por

1. a, b e c pertencem a W .

2. Se X pertence a W , então $a(X)c$ também pertence a W .

Quais das seguintes cadeias pertencem a W ?

a. $a(b)c$ b. $a(a(b)c)c$ c. $a(abc)c$ d. $a(a(a(a)c)c)c$ e. $a(aacc)c$

★27. Forneça uma definição recursiva para o conjunto de todas as wffs predicativas unárias em x .

28. Forneça uma definição para o conjunto de todas as fórmulas bem definidas da aritmética de inteiros que envolvam inteiros e os operadores aritméticos $+$, $-$, $*$ e $/$.

29. Forneça uma definição recursiva para o conjunto de todas as cadeias de parênteses bem balanceados.

30. Forneça uma definição recursiva para o conjunto de todas as cadeias binárias contendo um número ímpar de 0s (zeros).

★31. Forneça uma definição recursiva para x^R , o reverso da cadeia x .

32. Forneça uma definição recursiva para $|x|$, o tamanho da cadeia x .

★33. Use a notação BNF para definir o conjunto dos inteiros positivos.

34. Use a notação BNF para definir o conjunto dos números decimais que consistem em um sinal opcional ($+$ ou $-$), seguido de um ou mais dígitos, o ponto decimal, zero ou mais dígitos.

★35. Forneça uma definição recursiva para o fatorial $n!$, para $n \geq 1$.

36. Forneça uma definição recursiva para a adição de dois inteiros não-negativos m e n .

37. a. Escreva uma definição recursiva para a operação de obtenção do maior dentre n inteiros, $a_1, \dots, a_n$, $n \geq 2$.

b. Escreva uma definição recursiva para a operação de obtenção do menor dentre n inteiros, $a_1, \dots, a_n$, $n \geq 2$.

38. a. Forneça uma definição recursiva para a conjunção de n proposições na lógica proposicional.

b. Escreva uma formalização da propriedade associativa da conjunção (equivalência tautológica 2b da Seção 1.1), e use a indução para prová-la.

★39. Sejam A e $B_1, B_2, \dots, B_n$ afirmações. Prove a extensão finita das equivalências distributivas da lógica proposicional:

$$A \vee (B_1 \wedge B_2 \wedge \dots \wedge B_n) \Leftrightarrow (A \vee B_1) \wedge (A \vee B_2) \wedge \dots \wedge (A \vee B_n)$$

e

$$A \wedge (B_1 \vee B_2 \vee \dots \vee B_n) \Leftrightarrow (A \wedge B_1) \vee (A \wedge B_2) \vee \dots \vee (A \wedge B_n)$$

para $n \geq 2$.

40. Sejam $B_1, B_2, \dots, B_n$ afirmações. Prove a extensão finita da lei de De Morgan:

$$(B_1 \vee B_2 \vee \dots \vee B_n)' \Leftrightarrow B_1' \wedge B_2' \wedge \dots \wedge B_n'$$

e

$$(B_1 \wedge B_2 \wedge \dots \wedge B_n)' \Leftrightarrow B_1' \vee B_2' \vee \dots \vee B_n'$$

para $n \geq 2$.

Nos Exercícios 41 a 46, escreva o corpo da função recursiva para computar $S(n)$ para uma dada sequência S .

41. 1, 3, 9, 27, ...

42. 2, 1, $\frac{1}{2}$, $\frac{1}{4}$, ...

★43. 1, 2, 4, 7, 11, 16, ...

44. 2, 4, 16, 256, ...

45. $a, b, a + b, a + 2b, 2a + 3b, \dots$

★46. $p, p - q, p + q, p - 2q, p + 2q, p - 3q, \dots$

47. Que valor é computado pelo seguinte algoritmo recursivo em Pascal, para um valor de entrada n ?

```
function f(n: integer): integer;
begin
  if n = 1 then

    else
      f := f(n-1) + 1;
end;
```

48. A seguinte função recursiva em Pascal é inicialmente chamada com um valor inicial para i igual a 1. L é uma lista (vetor) com 10 inteiros. Qual o procedimento executado pela função?

```
function g(L: lista; i, x: integer): integer;
begin
  if i > 10 then
    g := 0
  else
    if L[i] = x then
      g := 10
    else
      g := g(L, i+1, x);
end;
```

49. Descreva informalmente um algoritmo recursivo para inverter os elementos de uma lista de itens.

50. Descreva informalmente um algoritmo recursivo para computar a soma dos dígitos de um inteiro positivo.

★51. Simule a execução do algoritmo *SelectionSort* para a lista L abaixo; escreva a lista após cada troca que altere o conteúdo da mesma.

4, 10, -6, 2, 5

52. Simule a execução do algoritmo *SelectionSort* para a lista L abaixo; escreva a lista após cada troca que altere o conteúdo da mesma.

9, 0, 2, 6, 4

53. O algoritmo de busca binária é utilizado na lista a seguir; x tem o valor "Cuiabá". Enumere os elementos com os quais x é comparado.

Brasília, Campinas, Ipu, Nova Iguaçu, Petrolina, São Paulo, Xerém

54. Um algoritmo de busca binária é usado na lista a seguir, tendo como valor de busca $x = \text{"margarina"}$. Indique com quais elementos x é comparado.

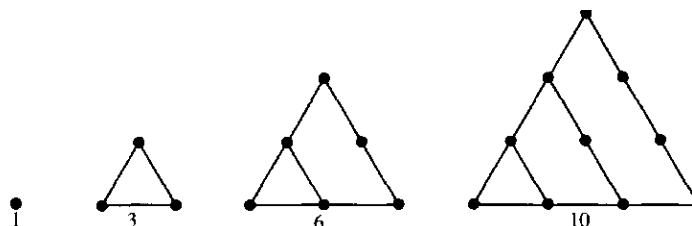
açúcar, chocolate, manteiga, margarina, nata, ovos

Nos Exercícios 55 a 60 resolva as relações de recorrência sujeitas às bases apresentadas.

- ★55. $S(1) = 5$
 $S(n) = S(n-1) + 5$ para $n \geq 2$
56. $F(1) = 2$
 $F(n) = 2F(n-1) + 2^n$ para $n \geq 2$
57. $T(1) = 1$
 $T(n) = 2T(n-1) + 1$ para $n \geq 2$
 (Dica: Veja Exemplo 11.)
- ★58. $A(1) = 1$
 $A(n) = A(n-1) + n$ para $n \geq 2$
 (Dica: Veja Prática 6.)
59. $S(1) = 1$
 $S(n) = S(n-1) + 2n - 1$ para $n \geq 2$
 (Dica: Veja Exemplo 10.)
60. $P(1) = 2$
 $P(n) = 2P(n-1) + n2^n$ para $n \geq 2$
 (Dica: Veja Prática 6.)

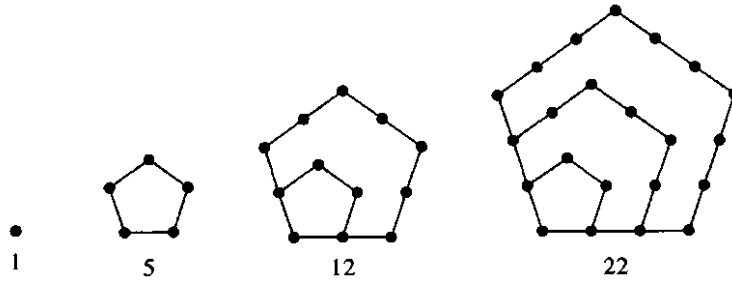
Para os Exercícios 61 e 62, resolva as relações de recorrência sujeitas às bases apresentadas, usando a abordagem de expansão, suposição e verificação.

- ★61. $F(1) = 1$
 $F(n) = nF(n-1)$ para $n \geq 2$
62. $S(1) = 1$
 $S(n) = nS(n-1) + n!$
63. O lixo em um depósito sanitário decompõe-se a uma taxa de 5% ao ano. Se 100 toneladas de lixo são inicialmente depositadas no depósito, quanto permanece não decomposto após 20 anos (ou seja, no início do 21.º ano)? (Dica: Enuncie e resolva uma relação de recorrência.)
64. Um milhão de unidades monetárias é depositado em uma conta, a uma taxa anual de remuneração de 8%. No final de cada ano um adicional de 100 unidades monetárias é depositado na mesma conta. Qual o montante do valor na conta no final de sete anos (ou seja, no início do 8.º ano)? (Dica: Escreva e resolva a relação de recorrência. Consulte também o Exercício 17 da Seção 2.2 para obter a fórmula da soma de uma progressão geométrica.)
- ★65. Membros antigos da Sociedade de Pitágoras definiram **números figurados** como sendo o número de pontos em uma certa configuração geométrica. Os primeiros **números triangulares** são 1, 3, 6, e 10, e são semelhantes ao diagrama da figura abaixo:



Encontre a fórmula para o n -ésimo número triangular. (Dica: Veja a Prática 6.)

66. Os primeiros **números pentagonais** (veja exercício anterior) são 1, 5, 12, 22, e são semelhantes ao diagrama da figura abaixo:



Encontre a fórmula para o n -ésimo número pentagonal. (Dica: Veja Exercício 18 da Seção 2.2 para obter a expressão da soma de uma progressão aritmética.)

67. Use a indução matemática para verificar que a equação (8) desta seção é a solução da relação de recorrência (6) sujeita à condição básica de $S(1)$ ser conhecida.

Seção 2.4 Análise de Algoritmos e Mais Sobre Prova de Correção

Análise de Algoritmos

Geralmente existe mais de um algoritmo para executar a mesma tarefa. Quando comparamos algoritmos, vários critérios podem ser utilizados para julgarmos qual deles é o "melhor". Devemos indagar, por exemplo, qual deles é mais fácil de ser entendido, ou qual é executado de forma mais eficiente. Um modo de julgarmos a eficiência de um algoritmo é estimarmos o número de operações realizadas pelo mesmo. Contamos apenas as operações que são básicas para a tarefa principal, não devemos computar operações cuja contribuição para o processamento seja irrelevante.

Suponhamos, por exemplo, que a tarefa é procurar a ocorrência de um determinado item x em uma lista ordenada de palavras ou números. Uma vez que qualquer algoritmo elaborado deverá comparar os elementos da lista dada com x (elemento a ser procurado), a operação básica a ser contabilizada é, portanto, a comparação.

O **algoritmo de busca seqüencial** compara x com cada valor da lista até que x seja encontrado, ou a lista seja totalmente varrida. Uma descrição em pseudocódigo de um algoritmo de busca seqüencial é apresentado a seguir. O algoritmo tem um tempo de processamento máximo quando x é o último item da lista, ou quando x não é um elemento da lista. Nos dois casos, todos os elementos da lista são comparados a x , totalizando assim n comparações.

ALGORITMO *BuscaSeqüencial*

procedure *BuscaSeqüencial* (L : lista; n : integer; x : tipoitem);
 { procura o item x em uma lista L com n itens }

begin

$i := 1$;

while ($L(i) \neq x$) and ($i < n$) **do**

$i := i + 1$;

if $L(i) = x$ **then**

 imprima ('encontrado')

else

 imprima ('não encontrado');

end;

É claro que x poderia ser o primeiro item da lista, o que acarretaria apenas uma comparação; no entanto, se x estiver no meio da lista serão necessárias aproximadamente $n/2$ comparações. Obviamente, existem muitas possibilidades e seria interessante que tivéssemos alguma forma de estimar o valor médio da quantidade de processamento demandada. Tal estimativa requer informações sobre a lista típica sobre a qual se realiza a busca,

bem como uma relação típica entre o valor procurado e a lista dada. Porém, uma medida para o comportamento médio é geralmente muito difícil de se determinar, não só para este algoritmo específico, mas para a maioria dos algoritmos. Para comparar a eficiência de algoritmos, portanto, nos contentaremos com a análise de seus piores casos.

O estudo da eficiência de algoritmos, isto é, do número de operações executadas, é chamado *análise de algoritmos*. A análise de algoritmos recursivos geralmente envolve a solução de uma relação de recorrência. A título de exemplo, consideremos outro algoritmo de busca em uma lista ordenada, o algoritmo de busca binária da Seção 2.3. Quantas comparações são realizadas por este algoritmo no pior caso? Uma comparação é realizada com o valor mediano, o processo é então repetido com uma das metades da lista. Se a lista original tem n elementos, então a metade da lista tem no pior caso $n/2$ elementos. (No Exemplo 30, por exemplo, quando 10 é o valor mediano, a metade "direita" da lista tem quatro elementos, porém a metade "esquerda" tem apenas três.) Se vamos continuar a cortar a lista pela metade, é conveniente considerar apenas o caso em que obtemos um valor inteiro cada vez que dividimos a lista ao meio; assim devemos assumir que $n = 2^m$ para algum $m \geq 0$. Se $C(n)$ representa o número máximo de comparações realizadas para uma lista com n elementos, então veremos que:

$$C(n) = 1 + C\left(\frac{n}{2}\right)$$

(Este resultado reflete uma comparação com o valor mediano, acrescido de outras comparações que são necessárias para a metade da lista.) A hipótese básica é:

$$C(1) = 1$$

posto que apenas uma comparação é realizada para uma lista com um elemento.

Como isto não é uma relação de recorrência de primeira ordem, para resolvê-la iremos começar com uma abordagem de expansão, suposição e verificação. Além disso, essa solução irá envolver uma função logarítmica; para uma revisão da função logarítmica e suas propriedades, veja o Apêndice B.

EXEMPLO 33 Resolva a relação de recorrência

$$C(n) = 1 + C\left(\frac{n}{2}\right) \quad \text{para } n \geq 2, n = 2^m$$

sujeita à hipótese básica

$$C(1) = 1$$

Expandindo, vem

$$\begin{aligned} C(n) &= 1 + C\left(\frac{n}{2}\right) \\ &= 1 + \left(1 + C\left(\frac{n}{4}\right)\right) \\ &= 1 + 1 + \left(1 + C\left(\frac{n}{8}\right)\right) \\ &\vdots \end{aligned}$$

com o termo geral sendo

$$C(n) = k + C\left(\frac{n}{2^k}\right)$$

O processamento termina quando $2^k = n$ ou $k = \log_2 n$. (Iremos omitir, daqui por diante, a notação na base 2 — assim $\log n$ significa $\log_2 n$.) Então

$$C(n) = \log n + C(1) = 1 + \log n$$

Agora, usaremos indução para mostrar que $C(n) = 1 + \log(n)$ para todo $n \geq 1, n = 2^m$. Esta é uma forma um pouco diferente de indução, porque os únicos valores de interesse são potências de 2. Continuaremos usando

o valor 1 para a base da indução, mas provaremos que, se nossa sentença é verdadeira para um valor k , então ela também será verdadeira para $2k$. A afirmação será então verdadeira para 1, 2, 4, 8, ..., ou seja, para todos inteiros não-negativos potências de 2, exatamente o que desejamos.

$$C(1) = 1 + \log 1 = 1 + 0 \quad \text{verdadeira}$$

Suponhamos então que $C(k) = 1 + \log k$. Então

$$\begin{aligned} C(2k) &= 1 + C(k) && \text{(pela relação de recorrência)} \\ &= 1 + 1 + \log k && \text{(pela hipótese de indução)} \\ &= 1 + \log 2 + \log k; \quad (\log 2 = 1) \\ &= 1 + \log 2k && \text{(propriedade dos logaritmos)} \end{aligned}$$

Este resultado completa a prova indutiva. •

Pelo Exemplo 33, o número máximo de comparações realizadas para uma busca binária em uma lista ordenada com n elementos, e $n = 2^m$ é $1 + \log n$. No Exemplo 30, n vale 8, e quatro comparações ($1 + \log 8$) foram realizadas no pior caso (x não está na lista). Uma busca seqüencial exigiria oito comparações. Como

$$1 + \log n < n \quad \text{para } n = 2^m, \quad n \geq 4$$

a busca binária é quase sempre mais eficiente que a busca seqüencial. Entretanto, um algoritmo de busca seqüencial tem uma grande vantagem — se a lista a ser pesquisada *não estiver ordenada*, o algoritmo de busca seqüencial funcionará mas o de busca binária, não. Se ordenarmos a lista para então usarmos o algoritmo de busca binária, devemos considerar o número de operações envolvidas no procedimento de ordenação. Os Exercícios 7 até 14 no fim desta seção solicitarão que você conte as operações realizadas para ordenar uma lista em diferentes algoritmos.

O algoritmo de busca binária é recursivo, o que significa que o algoritmo chama ele próprio para calcular valores menores que o valor de entrada original. Neste caso, a versão menor do problema é significativamente menor — a entrada tem mais ou menos a metade do tamanho da entrada do problema original. Este resultado decorre claramente da forma da relação de recorrência, onde $C(n)$ depende não de $C(n-1)$ mas de $C(n/2)$. Algoritmos com relações recursivas dessa forma, onde o problema principal é decomposto em pequenos subproblemas, são algumas vezes chamados algoritmos **dividir para conquistar** (*divide and conquer*).

A relação de recorrência da busca binária é um caso particular da forma geral

$$S(n) = cS\left(\frac{n}{2}\right) + g(n) \quad \text{para } n \geq 2, \quad n = 2^m \quad (1)$$

onde c é uma constante e g pode ser uma expressão envolvendo n . Iremos estabelecer uma solução de forma fechada para (1) sujeita à hipótese básica de que $S(1)$ é conhecido. Poderíamos usar novamente a abordagem de expandir, supor e verificar, mas, em vez disso, faremos algumas transformações em (1) para convertê-la em uma relação de recorrência de primeira ordem com coeficientes constantes.

A equação (1) assume que $n = 2^m$ com $n \geq 2$. Daí segue que $m = \log n$ e $m \geq 1$. Substituindo 2^m por n na equação (1), temos

$$S(2^m) = cS(2^{m-1}) + g(2^m) \quad (2)$$

Fazendo $T(m)$ representar $S(2^m)$ na equação (2), obtemos

$$T(m) = cT(m-1) + g(2^m) \quad \text{para } m \geq 1 \quad (3)$$

A equação (3) é uma equação linear de primeira ordem com coeficientes constantes; da equação (8) na Seção 2.3, obtemos a solução

$$T(m) = c^{m-1}T(1) + \sum_{i=2}^m c^{m-i}g(2^i) \quad (4)$$

sujeita à condição básica de que $T(1)$ seja conhecida. Como a equação (3) vale para $m = 1$, temos que

$$T(1) = cT(0) + g(2)$$

Substituindo este valor em (4) vem

$$T(m) = c^m T(0) + \sum_{i=1}^m c^{m-i} g(2^i) \quad (5)$$

Desfazendo a substituição inicial $T(m) = S(2^m)$, a equação (5) resulta em:

$$S(2^m) = c^m S(2^0) + \sum_{i=1}^m c^{m-i} g(2^i)$$

Finalmente, usando $2^m = n$, ou $m = \log n$ vem:

$$S(n) = c^{\log n} S(1) + \sum_{i=1}^{\log n} c^{(\log n)-i} g(2^i) \quad (6)$$

A equação (6) é então uma solução de forma fechada para a relação de recorrência (1).

EXEMPLO 34 A relação de recorrência para o algoritmo de busca binária

$$\begin{aligned} C(1) &= 1 \\ C(n) &= 1 + C\left(\frac{n}{2}\right) \quad \text{para } n \geq 2, n = 2^m \end{aligned}$$

verifica a equação (1) acima, com $c = 1$ e $g(n) = 1$. A solução de acordo com a fórmula (6) é

$$\begin{aligned} C(n) &= 1^{\log n} C(1) + \sum_{i=1}^{\log n} 1^{(\log n)-i} (1) \\ &= 1 + (\log n)(1) = 1 + \log n \end{aligned}$$

que confirma o resultado obtido previamente. •

PRÁTICA 21 Mostre que a solução da relação de recorrência

$$\begin{aligned} S(1) &= 1 \\ S(n) &= 2S\left(\frac{n}{2}\right) + 1 \quad \text{para } n \geq 2, n = 2^m \end{aligned}$$

é $2n - 1$ (Dica: Veja o Exemplo 11 e note que $2^{\log n} = n$). •

Mais adiante neste livro examinaremos o comportamento de outros algoritmos, muitos deles que operam com grafos, e muitos que não são recursivos.

Mais sobre Prova de Correção de Algoritmos

Na Seção 1.5 explicamos o uso de um sistema de lógica formal para provar matematicamente a correção de um programa. Asserções ou predicados envolvendo variáveis do programa são inseridas no início, no fim e pontos intermediários entre as instruções do programa. Provar então a correção de qualquer particular instrução s_i do programa envolve provar que a implicação

$$\{Q\} s_i \{R\} \quad (1)$$

é verdadeira, onde Q e R são assereções conhecidas, respectivamente, como pré-condição e pós-condição para a instrução. O programa é demonstrado como correto se todas as implicações para as instruções no programa são verdadeiras.

No Cap. 1, discutimos um axioma que fornece a condição sob a qual a implicação (1) é verdadeira quando s_i é um comando de atribuição, e uma regra de inferência que fornece as condições para que a implicação (1) seja verdadeira, quando s_i é uma instrução condicional. Agora iremos usar uma regra de inferência que fornece as condições para que a implicação (1) seja verdadeira quando s_i é uma instrução de laço. Nós adiamos as considerações sobre instruções de laço até agora porque usamos a indução matemática na aplicação desta regra de inferência.

Suponha que s_i é uma instrução de laço da forma

while condição B **do**
 S

onde B é uma condição que pode ser verdadeira ou falsa e S é uma instrução do programa. Quando esta instrução é executada, a condição B é computada. Se B é verdadeira, a instrução S é executada e então B é computada novamente. Se B continuar verdadeira, o segmento de programa S é executado e então B é novamente computada, e assim por diante. Se a condição B for falsa, o loop é interrompido.

A forma de implicação (1) que pode ser usada quando s_i é uma instrução de loop impõe (da mesma forma que no axioma da atribuição) uma relação entre Q e R , a pré-condição e a pós-condição. A asserção Q é verdadeira antes de se entrar no laço; um requisito é que Q deve continuar valendo depois que o laço termina (o que significa que deveremos procurar uma asserção Q que *desejamos* seja verdadeira quando o loop terminar). Além disso, B' — a condição para o término do loop — deve ser verdadeira da mesma forma. Então (1) terá a forma

$$\{Q\}s_i\{Q \wedge B'\} \quad (2)$$

EXEMPLO 35 Considere a seguinte função em pseudocódigo, que calcula o produto $x * y$ para dois inteiros não-negativos x e y .

```
function Produto( $x, y$ : inteiros não-negativos):integer;
{computa  $x * y$ }

var
   $i, j$ : integer;

begin
   $i := 0$ ;
   $j := 0$ ;
  while ( $i \neq x$ ) do
    begin
       $j := j + y$ ;
       $i := i + 1$ ;
    end;

    { $j$  agora contém o valor  $x * y$ }
    Produto :=  $j$ ;
  end;
```

Esta função contém um loop; a condição B para o loop continuar executando é $i \neq x$. A condição B' para o loop terminar é $i = x$. Quando o loop é encerrado j deve conter o valor $x * y$. Sabendo-se que $i = x$ quando o loop termina, a asserção $j = i * y$ também deve ser verdadeira. Portanto, quando o laço termina, se a função realiza o que desejamos que realize, a asserção

$$j = i * y \wedge i = x$$

é verdadeira. Isto iguala a forma $Q \wedge B'$, se tomarmos a asserção

$$j = i * y$$

como Q . Objetivando igualar a forma a (2), a asserção $j = i * y$ deverá ser verdadeira antes da instrução de laço. Isto é realmente o caso porque antes do laço, $i = j = 0$.

Parece que para este exemplo nós temos um candidato à asserção Q para a implicação (2), no entanto não temos ainda a regra de inferência que nos permita afirmar quando (2) é uma implicação verdadeira. (Lem-

bre-se de que nós descobrimos Q baseados no que imaginamos ser a operação correta do código da função.)

A asserção Q deve ser verdadeira antes que o laço seja iniciado. Se a implicação (2) for verificada, Q permanecerá verdadeira depois do término do loop. Este fato pode ser garantido se Q permanecer constante após cada iteração do loop, incluindo a iteração final. Q representa um predicado ou relação entre os valores das variáveis do programa. Se esta relação é válida, antes que uma iteração do loop seja executada, e permanece válida depois que a iteração é executada, então a *relação* entre estas variáveis não é afetada pela ação da iteração do loop, ainda que os valores possam ter mudado. Uma relação desse tipo é chamada **invariante do loop**.

A regra de inferência de laço nos permite inferir a veracidade de (2) a partir de uma implicação que diga que Q é uma invariante do loop, isto é, que se Q for verdadeira e a condição B também for verdadeira, caso no qual outra iteração do laço é executada, então Q permanece verdadeira após esta iteração. A regra é apresentada de modo mais formal no quadro a seguir. Para se valer desta regra de inferência, devemos obter uma invariante do loop, Q , que nos seja útil — uma asserção que afirme o que esperamos que deva acontecer — e então demonstrar a implicação.

$$\{Q \wedge B\} S \{Q\}$$

Regra do Loop de Inferência

A implicação

$$\{Q\} s_i \{Q \wedge B'\}$$

onde s_i é uma iteração do loop

while condição B **do**
 S

pode ser inferida de

$$\{Q \wedge B\} S \{Q\}$$

Neste ponto é que a indução entra em cena. Denotamos por $Q(n)$ a sentença de que Q , uma invariante do loop proposta, é verdadeira após n iterações do loop. Por não sabermos necessariamente quantas iterações o loop pode executar (ou seja, por quantos passos a condição B permanece verdadeira), nós desejamos mostrar que $Q(n)$ é verdadeira para todo $n \geq 0$. (O valor $n = 0$ corresponde à asserção quando da entrada no loop, depois de zero iteração do loop.)

EXEMPLO 36 Consideremos novamente a função em pseudocódigo do Exemplo 35. Neste exemplo, supomos que Q é a relação

$$j = i * y$$

Para usar a regra de inferência de loop, nós devemos provar que Q é uma invariante do loop.

Os valores de x e y permanecem inalterados durante toda a execução da função, no entanto i e j têm seus valores alterados no interior do laço. Fazemos i_n e j_n denotar respectivamente os valores de i e j após n iterações do loop. Então $Q(n)$ é a sentença $j_n = i_n * y$. Demonstraremos por indução que $Q(n)$ vale para todo $n \geq 0$.

$Q(0)$ é a sentença

$$j_0 = i_0 * y$$

que, como foi verificado no Exemplo 35, é verdadeira porque, após zero iteração do loop, quando o primeiro comando do loop é alcançado, tanto i quanto j têm o valor 0. (Formalmente, o axioma de transferência pôde ser usado para provar que estas condições em i e j valem neste ponto.)

$$\begin{array}{ll} \text{Assumimos} & Q(k): j_k = i_k * y \\ \text{Mostraremos} & Q(k+1): j_{k+1} = i_{k+1} * y \end{array}$$

Entre os instantes de tempo em que j e i assumem os valores j_k e i_k e os valores j_{k+1} e i_{k+1} , uma iteração do loop é realizada. Nesta iteração, o valor de j é alterado pela adição de y ao seu valor anterior, e o de i pela adição de 1. Assim

$$j_{k+1} = j_k + y \quad (3)$$

$$i_{k+1} = i_k + 1 \quad (4)$$

Então

$$\begin{aligned} j_{k+1} &= j_k + y && \text{(por (3))} \\ &= i_k * y + y && \text{(pela hipótese de indução)} \\ &= (i_k + 1) y \\ &= i_{k+1} * y && \text{(por (4))} \end{aligned}$$

O que conclui a demonstração de que Q é uma invariante do loop.

A regra de inferência de loop nos permite inferir que, após a execução do loop, a condição $Q \wedge B'$ ainda é válida, o que no nosso caso é

$$j = i * y \wedge i = x$$

No entanto, neste ponto a sentença

$$j = x * y$$

é verdadeira, o que é exatamente o que a função pretendia calcular. •

O Exemplo 36 ilustra que invariantes de loops nos fornecem informações mais fortes sobre o programa do que pretendíamos demonstrar; o que desejamos mostrar é o caso especial da invariante do loop após o término do loop. Encontrar a invariante do loop apropriada requer que trabalhem de trás para frente, a partir da conclusão desejada, como no Exemplo 35.

Não demonstramos de fato que o laço neste exemplo chega ao fim de sua execução. O que demonstramos foi sua **correção parcial** — o programa produz o resultado desejado, supondo que sua execução chega ao fim. Como x é um inteiro não-negativo e i é um inteiro que começa em 0 e é incrementado de 1 a cada passo do loop, sabemos que, em algum passo, $i = x$ irá tornar-se uma sentença verdadeira. É claro que este exemplo é irreal: se nós realmente desejássemos calcular $x.y$, o teríamos feito usando uma simples instrução de programa. No entanto, esta mesma técnica também se aplica a laços que tenham um significado mais concreto.

PRÁTICA 22

Mostre que a seguinte função calcula $x+y$ para inteiros x e y não-negativos, provando que $Q : j = x + i$ é uma invariante do loop e avaliando o valor de Q quando o loop termina.

```
function Soma( $x,y$ : inteiros não-negativos): integer;
```

```
var
```

```
   $i, j$ : integer;
```

```
begin
```

```
   $i := 0$ ;
```

```
   $j := x$ ;
```

```
  while ( $i \neq y$ ) do
```

```
    begin
```

```
       $j := j + 1$ ;
```

```
       $i := i + 1$ ;
```

```
    end;
```

```
  {  $j$  agora tem o valor  $x + y$  }
```

```
   $Soma := j$ ;
```

```
end;
```

Revisão da Seção 2.4

Técnicas

- Resolver relações de recorrência de algoritmos que se valham de estratégias do tipo dividir para conquistar através do uso de uma fórmula resolvente.
- Verificação de invariantes de loops.

Idéias Principais

A análise de um algoritmo estima o número de operações básicas que o algoritmo executa.

A análise de algoritmos recursivos conduz às relações de recorrência.

Uma invariante do loop, demonstrada por indução no número de iterações, pode ser usada para demonstrar a correção de um loop de programa.

Exercícios 2.4

- ★1. Descreva a versão recursiva do algoritmo de busca sequencial para realizar uma busca do item x em uma lista L com n itens.
2. Usando a versão recursiva do Exercício 1, escreva uma relação de recorrência para o número de comparações de x com os elementos da lista realizada pelo algoritmo de busca sequencial no pior caso, e em seguida resolva esta relação de recorrência. (Como antes, a resposta deverá ser n .)

Nos Exercícios 3 a 6, resolva a relação de recorrência sujeita à hipótese de indução. (*Dica:* Veja Exemplo 11, e note que $2^{\log n} = n$.)

3. $T(1) = 3$

$$T(n) = T\left(\frac{n}{2}\right) + n \quad \text{para } n \geq 2, n = 2^m$$

4. $P(1) = 1$

$$P(n) = 2P\left(\frac{n}{2}\right) + 3$$

5. $S(1) = 1$

$$S(n) = 2S\left(\frac{n}{2}\right) + n$$

6. $P(1) = 1$

$$P(n) = 2P\left(\frac{n}{2}\right) + n^2$$

Os Exercícios 7 a 9 referem-se ao algoritmo *SelectionSort* da Seção 2.3.

- ★7. Em cada passo do algoritmo *SelectionSort*, o índice do item de valor máximo em uma lista deve ser determinado. Isto requer comparações entre os elementos da lista. Quantas comparações são necessárias no pior caso para determinar o maior elemento de uma lista não ordenada com n elementos? Quantas comparações são necessárias no caso médio?
8. Definindo como operação básica a comparação dos elementos de uma lista e ignorando o processamento necessário para realizar as trocas de elementos, escreva a relação de recorrência para a quantidade de esforço computacional empregado pelo algoritmo *SelectionSort* quando aplicado a uma lista com n elementos. (*Dica:* Use o resultado do Exercício 7.)
9. Resolva a relação de recorrência do Exercício 8.

Os Exercícios 10 a 14 referem-se a um algoritmo para ordenação de dados denominado *MergeSort*, descrito a seguir: Uma lista com um único elemento já está obviamente ordenada e não é necessário qualquer processamento. Para outros casos, divida a lista ao meio, ordene cada uma das partes e então faça um merge das duas listas em uma única lista ordenada.

- ★10. A parte do algoritmo *MergeSort* que executa o merge requer a comparação entre elementos das duas listas ordenadas a fim de escolher qual elemento será incluído na lista ordenada, que será a combinação das duas "metadas já ordenadas". Quando o fim de uma das listas é alcançado, os elementos restantes da outra lista podem ser adicionados à nova lista sem a necessidade de mais comparações. Dadas as duas listas, execute um merge e conte o número de comparações necessárias para executá-lo.
a. 6,8,9 e 1,4,5 b. 1,5,8 e 2,3,4 c. 0,2,3,4,7,10 e 1,8,9
- 11. Em quais circunstâncias ocorre um número máximo de comparações para executar um merge de duas listas ordenadas? Se os tamanhos das listas são respectivamente r e s , qual o número máximo de comparações?
- ★12. Escreva uma relação de recorrência para o número de comparações realizadas entre os elementos das listas pelo algoritmo *MergeSort* no pior caso. Assuma que $n = 2^m$.
- 13. Resolva a relação de recorrência do Exercício 12.
- 14. Estabeleça a comparação entre os algoritmos *SelectionSort* e *MergeSort* para os piores casos com $n = 4, 8, 16$ e 32 (use uma calculadora).

Nos Exercícios 15 a 18 demonstre que o trecho de programa em pseudocódigo está correto, provando que Q é uma invariante do loop, e calculando o seu valor quando o laço é encerrado.

15. Função para calcular x^2 para $x \geq 1$.

function *Quadrado*(x : inteiro positivo): integer;

var

i, j : integer;

begin

$i := 1$;

while ($i \neq x$) **do**

begin

$j := j + 2i + 1$;

$i := i + 1$;

end;

{ j agora tem o valor x^2 }

Quadrado := j ;

end;

$Q: j = i^2$

- ★16. Função para calcular $x!$ para $x \geq 1$.

function *Fatorial*(x : inteiro positivo): integer;

var

i, j : integer;

begin

$i := 2$;

while ($i \neq x + 1$) **do**

begin

$j := j * i$;

$i := i + 1$;

end;

```

    {j agora tem o valor x!}
    Fatorial := j;
end;

```

```

Q:j = (i- 1)!

```

17. Função para calcular x^y para $x, y \geq 1$.

```

function Potência(x,y: inteiro positivo): integer;

```

```

var

```

```

    i,j: integer;

```

```

begin

```

```

    i:= 1;

```

```

    j:=x;

```

```

    while(i  $\neq$  y) do

```

```

        begin

```

```

            j := j*x;

```

```

            i:= i+ 1;

```

```

        end;

```

```

    { j agora tem o valor  $x^y$  }

```

```

    Potência := j;

```

```

end;

```

```

Q:j =  $x^i$ 

```

18. Procedimento para calcular o quociente q e o resto r quando x é dividido por y , $x \geq 0$, $y \geq 1$.

```

procedure Divide(x: inteiro não-negativo; y: inteiro positivo);

```

```

var

```

```

    q, r. inteiros não-negativos;

```

```

begin

```

```

    q:=0;

```

```

    r := x;

```

```

    while (r  $\geq$  y) do

```

```

        begin

```

```

            q:=q+ 1;

```

```

            r := r — y;

```

```

        end;

```

```

    {q e r são agora quociente e resto}

```

```

    write('O quociente é 'q ',e o resto é', r);

```

```

end;

```

```

Q: x = q * y + r;

```

Nos Exercícios 19 a 22 prove que o segmento de programa é correto encontrando e demonstrando a invariante do loop Q apropriada e computando Q ao fim do loop.

19. Função para calcular $x - y$ para $x, y \geq 0$.

```

function Diferença (x, y: inteiros não-negativos): integer;

```

```

var

```

```

    i,j: integer;

```

```

begin

```

```

    i :=0;

```

```

    j:=x;

```

```

    while (i  $\neq$  y) do

```

```

        begin

```

```

            j:=j - 1 ;

```

```

            i:= i + 1;

```

```

        end;

```

```

    {j agora tem o valor  $x - y$ }
    Diferença := j;
end;

```

- ★20. Função para calcular x, y^n para $n \geq 0$.

```

function Computacional (x, y: inteiros; n: inteiro não-negativo): integer;

var
    i, j: integer;

begin
    i := 0;
    j := x;
    while (i  $\neq$  n) do
        begin
            j := j * y;
            i := i + 1;
        end;

    {j agora tem o valor  $x * y^n$ }
    Computacional := j;
end;

```

21. Função para computar $x * n!$ para $n \geq 1$.

```

function OutraFunção (x: integer; n: inteiro positivo): integer;

var
    i, j: integer;

begin
    i := 1;
    j := x;
    while (i  $\neq$  n) do
        begin
            j := j * (i + 1);
            i := i + 1;
        end;

    {j agora tem o valor  $x * n!$ }
    OutraFunção := j;
end;

```

22. Função para calcular o valor do polinômio

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

para um dado valor de x .

```

function Poli ( $a_n, \dots, a_0, x$ : real): real;

var
    i, j: integer;

begin
    i := n;

    while (i  $\neq$  0) do
        begin
            j := j * x +  $a_{i-1}$ ;
            i := i - 1;
        end;

    {j agora tem o valor calculado da polinomial }
    Poli := j;
end;

```

23. Prove a correção da função iterativa da Seção 2.3 para calcular $S(n) = 2^n$.

Revisão do Capítulo 2

Terminologia

algoritmo de busca binária	índice do somatório	raciocínio indutivo
algoritmo de busca seqüencial	indução completa	recíproca
algoritmo selectionsort	indução forte	regra de inferência de loop
algoritmos dividir para conquistar	indução fraca	relação de recorrência
base da indução	invariante de loop	relação de recorrência com coeficientes constantes
cadeia binária	notação de somatório	relação de recorrência de primeira ordem
cadeia vazia	número racional	relação de recorrência
concatenação	palíndromo	homogênea
contra-exemplo	passo indutivo	relação de recorrência linear
contrapositiva	princípio da boa ordenação	seqüência
correção parcial	princípio da indução	seqüência de Fibonacci
definição indutiva	matemática	solução de forma fechada
definição recursiva	prova direta	solução de uma relação de recorrência
Fórmula de Backus Naur (BNF)	prova por contradição	suposição indutiva
hipótese indutiva	prova por contraposição	
	prova por exaustão	
	raciocínio dedutivo	

Avaliação

Sem consultar o capítulo, responda verdadeiro ou falso para as seguintes questões.

Seção 2.1

1. Um teorema nunca poderá ser demonstrado simplesmente, provando-se que é verdadeiro para um número finito de casos.
2. A prova por contradição de $P \rightarrow Q$ é iniciada pela suposição de P e Q' .
3. No enunciado do teorema "o dobro de um número ímpar é par", subentendemos um quantificador existencial.
4. Para demonstrar o "teorema" "Se São Luís é a capital, então Maranhão é o estado", é suficiente demonstrar que "se Maranhão é o estado, então São Luís é a capital".
5. Provar "A se, e somente se, B" requerer a prova de $A \rightarrow B$ e a prova de $B \rightarrow A$.

Seção 2.2

6. A indução é uma técnica de demonstração apropriada para provar resultados envolvendo todos os números inteiros positivos.
7. A base da indução em uma demonstração indutiva requer que se prove que uma propriedade é válida para $n = 1$.
8. Se a condição para que $P(k + 1)$ seja verdadeira depender da veracidade de valores anteriores a $P(k)$, então devemos usar a indução forte.
9. O caminho para uma demonstração por indução fraca é verificar como o fato de P ser verdadeira no valor $k+1$ depende do fato de P ser verdadeira no valor k .
10. A hipótese indutiva na demonstração por indução para a afirmação

$$1^3 + 2^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

é

$$K^3 = \frac{k^2(k+1)^2}{4}$$

Seção 2.3

11. Uma seqüência definida por

$$S(1) = 7$$

$$S(n) = 3S(n-1) + 2 \quad \text{para } n \geq 2$$

contém o número 215.

12. Algoritmos recursivos são sempre nossas primeiras opções porque são mais eficientes que algoritmos iterativos.
13. Aplicando um algoritmo de busca binária à lista

2,5,7, 10, 14,20

onde $x = 8$ é o item a ser procurado, x nunca será comparado com o valor 5.

14. Uma solução de forma fechada para uma relação de recorrência é obtida pela aplicação de indução matemática à relação de recorrência.
15. $S(n) = 2S(n - 1) + 3S(n - 2) + 5n$ é uma relação de recorrência linear de primeira ordem com coeficientes constantes.

Seção 2.4

16. A análise de um algoritmo geralmente avalia a quantidade de trabalho demandada considerando o pior caso, pois é muito difícil se fazer a avaliação para o caso médio.
17. Algoritmos do tipo dividir para conquistar nos leva a relações de recorrência que não são de primeira ordem.
18. Uma invariante de laço permanece verdadeira até que o loop seja encerrado, ocasião em que ele se torna falso.
19. A correção parcial de um laço em um programa significa que o laço comporta-se corretamente para alguns valores de entrada e não para outros.
20. O algoritmo de busca binária é mais eficiente que o algoritmo de busca sequencial em uma lista ordenada com mais de três elementos.

No Computador

Para os Exercícios 1 a 10, escreva um programa de computador que produza o resultado desejado a partir dos dados de entrada.

1. *Entrada:* Número n de termos em uma progressão geométrica (veja Exercício 17, Seção 2.2), o termo inicial a e a razão r .
Saída: Soma dos primeiros n termos usando:
 - a. iteração
 - b. fórmula do Exercício 17, Seção 2.2
2. *Entrada:* Número n de termos de uma progressão aritmética (veja Exercício 18, Seção 2.2), o termo inicial a e a razão d .
Saída: Soma dos primeiros n termos, usando:
 - a. iteração
 - b. fórmula do Exercício 18, Seção 2.2
3. *Entrada:* Número n
Saída: Soma dos n primeiros cubos, usando:
 - a. iteração, usando apenas multiplicação e adição. Mostre, ao final, o número de multiplicações e adições realizadas
 - b. fórmula do Exercício 8, Seção 2.2, usando apenas multiplicação, adição e divisão. Mostre, ao final, o número de adições, multiplicações e divisões realizadas.
4. *Entrada:* Nenhuma
Saída: Tabela mostrando todo inteiro n , $8 \leq n \leq 100$ como soma de 3s e 5s (veja Exemplo 18).
5. *Entrada:* Cadeia Binária
Saída: Mensagem indicando se a cadeia é um palíndromo (veja Prática 15).
Algoritmo: Use recursão.
6. *Entrada:* Cadeia de caracteres x e um inteiro positivo n .
Saída: Concatenação de n cópias de x .
Algoritmo: Use recursão.
(Algumas linguagens de programação fornecem rotinas internas para manipulação de strings, tais como a concatenação.)
7. *Entrada:* Inteiro positivo n
Saída: n -ésimo valor de seqüência de Fibonacci, usando:
 - a. iteração
 - b. recursão

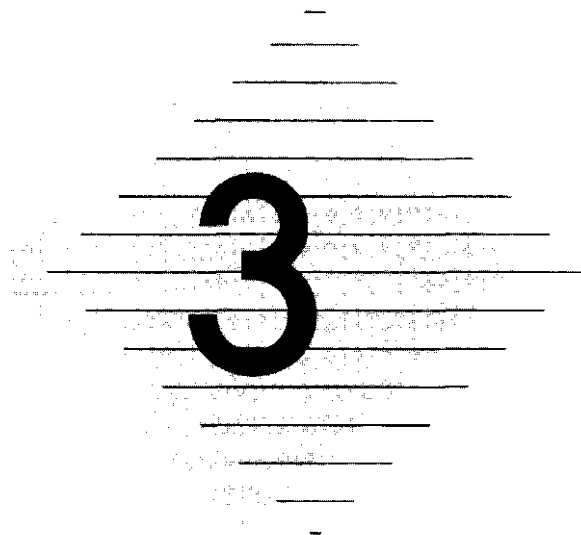
Introduza um contador em cada versão para indicar o número total de operações de adição utilizadas. Execute cada versão para vários valores de n e, em um gráfico simples, marque o número de adições como função de n para ambas as versões.

8. *Entrada'*: Dois inteiros positivos m e n
Saída: $\text{mdc}(m,n)$ **máximo divisor comum** de m e n .
Algoritmo: $\text{mdc}(m,n)$ é definido como o maior inteiro que divide ao mesmo tempo m e n . Suponha que m é menor do que n . Se m divide n então $\text{mdc}(m,n) = m$. Por outro lado, o **algoritmo de Euclides** afirma que o $\text{mdc}(m,n)$ é igual ao maior divisor de m e o resto da divisão de n por m . Por isso, uma relação de recursão é apropriada.
9. *Entrada*: Lista não ordenada de 10 elementos
Saída: Lista ordenada em ordem crescente
Algoritmo: Use o algoritmo de seleção recursiva do Exemplo 28.
10. *Entrada*: Lista ordenada de 10 inteiros e um inteiro x
Saída: Mensagem indicando se x pertence à lista
Algoritmo: Use o algoritmo de busca binária do Exemplo 29.
11. A fórmula $3^n < n!$ é verdadeira para todo $n \geq N$. Escreva um programa para determinar N e depois demonstre o resultado por indução.
12. A fórmula $2^n > n^3$ é verdadeira para todo $n \geq N$. Escreva um programa para determinar N e depois demonstre o resultado por indução.
13. O valor $(1 + \sqrt{5})/2$ conhecido como **razão áurea** é relacionado à seqüência de Fibonacci por:

$$\lim_{n \rightarrow \infty} \frac{F(n+1)}{F(n)} = \frac{1 + \sqrt{5}}{2}$$

Verifique este limite calculando $F(n+1)/F(n)$ para $n = 10, 15, 25, 50$ e 100 , e compare este resultado com a razão áurea.

14. Compare o trabalho realizado pelos algoritmos de busca binária e de busca seqüencial em uma lista ordenada de n entradas, pelo cálculo de $n + 1 + \log n$ para valores de n de 1 até 100. Apresente os resultados de forma gráfica.



Conjuntos e Combinatória

Objetivos do Capítulo

Após estudar este capítulo, o leitor estará apto a:

- Lidar com a notação de teoria de conjuntos
- Encontrar o conjunto das partes de um conjunto finito
- Encontrar a união, interseção, diferença, complemento e produto cartesiano de conjuntos
- Identificar operações binárias e unárias sobre conjuntos
- Usar uma de duas técnicas de demonstração de identidade de conjuntos
- Entender a representação de conjuntos em computadores e como são usados em linguagens de programação e na programação orientada a objeto
- Compreender que nem todos os conjuntos são enumeráveis
- Aplicar o Princípio da Multiplicação e o Princípio da Adição na solução de problemas de contagem
- Usar árvores de decisão para resolver problemas de contagem
- Usar o Princípio da Inclusão e Exclusão na determinação do número de elementos da união de conjuntos
- Usar o Princípio da Casa do Pombo para determinar quando certos eventos devem ocorrer
- Usar as fórmulas de permutação e combinação, com e sem repetições, de r objetos dentre um conjunto de n objetos distintos.

- Encontrar o número de permutações distintas de n objetos que não sejam todos distintos
- Usar o teorema binomial para expandir $(a + b)^n$

A teoria dos conjuntos é uma das balizas-mestras da Matemática. Muitos conceitos na Matemática e na Ciência da Computação podem ser convenientemente expressados na linguagem dos conjuntos. Podem ser realizadas operações em conjuntos a fim de gerar novos conjuntos. Apesar de muitos conjuntos de interesse para a Ciência da Computação serem finitos ou *contáveis*, existem conjuntos com tantos elementos que se torna impossível enumerá-los. A Teoria dos Conjuntos será discutida na Seção 3.1.

Se um conjunto é finito, normalmente será interessante contar o número de elementos que contém. Isto pode não ser uma tarefa trivial. A Seção 3.2 fornece algumas regras básicas para a contagem de elementos em um conjunto resultado de um evento. A contagem dos elementos deste tipo de conjunto pode ser administrada através da quebra do evento em uma seqüência de subeventos ou em eventos disjuntos que não tenham dependência entre si. Alguns princípios de contagem especializados são mostrados na Seção 3.3, enquanto que a Seção 3.4 fornece fórmulas para a contagem do número de formas nas quais os objetos de um conjunto podem ser arranjados ou selecionados.

A Seção 3.5 discute o teorema binomial, um resultado algébrico que também pode ser entendido como consequência das fórmulas de contagem.

Seção 3.1 Conjuntos

As definições são importantes em qualquer ciência, porque contribuem para uma comunicação precisa. No entanto, se procurarmos uma palavra no dicionário, veremos que sua definição é expressa em termos de outras palavras, que são definidas por outras palavras, e assim por diante.

Portanto, precisamos ter um ponto de partida para nossas definições no qual os significados fiquem claros; nosso ponto de partida nessa discussão será a idéia de um *conjunto*, um termo que não definiremos formalmente. Ao invés disso, usaremos apenas a idéia intuitiva de que um conjunto é uma coleção não-ordenada de objetos. Normalmente todos os objetos em um conjunto gozam de uma mesma propriedade (além da de pertencer ao conjunto!); qualquer objeto que contenha a propriedade é um elemento do conjunto e qualquer objeto que não tem a propriedade não é um elemento. (Isto é consistente com nosso uso da palavra *conjunto* na Seção 2.3, onde falamos do conjunto das fórmulas bem-formuladas, o conjunto de todas as cadeias de símbolos de um alfabeto finito, e do conjunto dos identificadores de uma linguagem de programação.)

Notação

Usamos letras maiúsculas para denotarem conjuntos e o símbolo ϵ para denotar que um elemento pertence ao conjunto. Portanto, $a \in A$ significa que a é um elemento, ou membro, do conjunto A e $b \notin A$ significa que o objeto b não é um elemento do conjunto A . Usamos chaves para indicar conjuntos.

EXEMPLO 1

Se $A = \{\text{violeta, mostarda, vermelho}\}$, então mostarda $\in A$ e púrpura $\notin A$.

•

Como um conjunto é uma coleção *não-ordenada* de objetos, a ordem na qual os elementos são escritos não importa; portanto $\{\text{violeta, mostarda, vermelho}\}$, denota o mesmo conjunto que $\{\text{mostarda, vermelho, violeta}\}$. Além disso, cada elemento de um conjunto é listado apenas uma vez; seria redundante listá-los mais do que uma única vez.

Dois conjuntos são **iguais** se contêm os mesmos elementos. (Em uma definição, "se" significa, na verdade, "se, e somente se", portanto dois conjuntos são iguais se, e somente se, eles contêm os mesmos elementos.) Usando a notação da lógica predicada,

$$A = B \text{ significa } (\forall x)[(x \in A \rightarrow x \in B) \wedge (x \in B \rightarrow x \in A)]$$

Ao descrevermos um determinado conjunto, precisamos identificar seus elementos. Para conjuntos finitos, podemos fazê-lo apenas listando os elementos que contêm, como o conjunto A do Exemplo 1. E, embora seja impossível listar todos os elementos de um conjunto infinito, para alguns deles é possível indicar um padrão para a listagem indefinida. Podemos, portanto, escrever $\{2, 4, 6, \dots\}$ para expressar o conjunto S de todos os inteiros positivos pares. (Apesar desta ser uma prática comum, existe sempre o risco do leitor não compreender o padrão que o escritor tinha em mente.) S pode ainda ser definido, definindo explicitamente um de seus elementos e, então, definindo os demais elementos de S em termos dos elementos já conhecidos. Por exemplo:

1. $2 \in S$
2. Se $n \in S$, então $(n + 2) \in S$

Mas a forma mais clara de descrever este conjunto S em particular é descrever a propriedade característica de seus elementos através de palavras e escrever

$$S = \{x \mid x \text{ é inteiro positivo par}\}$$

que é lido como "o conjunto de todos os x tal que x é inteiro positivo par".

As diversas formas pelas quais descreveremos um conjunto são, portanto

1. listando (ou listando parcialmente) os elementos,
2. usando recursão para descrever como gerar o conjunto de elementos, ou
3. descrevendo uma propriedade P que caracterize o conjunto de elementos.

Veremos mais adiante nesta seção que existem conjuntos para os quais a primeira abordagem não funciona; em geral, a segunda é difícil de ser usada. Normalmente acaba restando o terceiro método como melhor opção.

A notação para um conjunto cujos elementos sejam caracterizados como tendo a propriedade P é $\{x \mid P(x)\}$. A propriedade P aqui é um *predicado unário*, termo este que foi apresentado no Cap. 1. Na verdade, a notação da lógica formal do Cap. 1 vem nos ajudar a tornar mais claro o que queremos dizer com a propriedade característica dos elementos de um conjunto:

$$S = \{x \mid P(x)\} \text{ significa } (\forall x)[(x \in S \rightarrow P(x)) \wedge (P(x) \rightarrow x \in S)]$$

Em palavras, todo elemento de S tem a propriedade P e tudo o que tem a propriedade P é um elemento de S .

PRÁTICA 1 Descreva cada um dos seguintes conjuntos, listando seus elementos.

- a. $\{x \mid x \text{ é um inteiro e } 3 < x \leq 7\}$
- b. $\{x \mid x \text{ é um mês com exatamente 30 dias}\}$
- c. $\{x \mid x \text{ é a capital do Brasil}\}$

PRÁTICA 2 Descreva cada um dos seguintes conjuntos, fornecendo-lhes uma propriedade característica.

- a. $\{1, 4, 9, 16\}$
- b. $\{\text{o açougueiro, o padeiro, o fabricante de velas}\}$
- c. $\{2, 3, 5, 7, 11, 13, 17, \dots\}$

É conveniente dar nomes a alguns conjuntos-padrão a fim de referirmo-nos a eles mais facilmente. Usaremos

- $\mathbb{N}$ = conjunto de todos os inteiros não-negativos (perceba que $0 \in \mathbb{N}$)
- $\mathbb{Z}$ = conjunto de todos os inteiros
- $\mathbb{Q}$ = conjunto de todos os números racionais
- $\mathbb{R}$ = conjunto de todos os números reais
- $\mathbb{C}$ = conjunto de todos os números complexos

Algumas vezes também falaremos no conjunto sem qualquer elemento (o **conjunto vazio** ou **conjunto nulo**), denotado por $\emptyset$ ou $\{\}$.

EXEMPLO 2 Se $S = \{x \mid x \in \mathbb{N} \text{ e } x < 0\}$, então $S = \{\}$.

Suponhamos agora que o conjunto A é descrito como

$$A = \{x \mid (\exists y)(y \in \{0, 1, 2\} \text{ e } x = y^3)\}$$

Como y não é uma variável livre, esta descrição continua na forma $A = \{x \mid P(x)\}$. Os elementos de A podem ser encontrados fazendo com que y assuma cada um dos valores 0, 1 e 2, e então tomando a terceira potência desses valores. Logo $A = \{0, 1, 8\}$. Se seguirmos o mesmo processo com o conjunto B ,

$$B = \{x \mid x \in \mathbb{N} \text{ e } (\exists y)(y \in \mathbb{N} \text{ e } x \leq y)\}$$

então, fazendo $y = 0$ temos $x = 0$; fazendo $y = 1$ temos $x = 0$ ou 1 ; fazendo $y = 2$ temos $x = 0, 1$ ou 2 . Em outras palavras, B consiste em todos os inteiros não-negativos que são menores ou iguais a algum inteiro não-negativo, o que quer dizer que $B = \mathbb{N}$. Mas para o conjunto C ,

$$C = \{x \mid x \in \mathbb{N} \text{ e } (\forall y)(y \in \mathbb{N} \rightarrow x \leq y)\}$$

obtemos $C = \{0\}$ porque 0 é o único inteiro não-negativo que é menor ou igual a todo inteiro não-negativo.

PRÁTICA 3 Descreva cada um dos conjuntos definidos abaixo.

- a. $A = \{x \mid x \in \mathbb{N} \text{ e } (\forall y)(y \in \{2, 3, 4, 5\} \rightarrow x \geq y)\}$
 b. $B = \{x \mid (\exists y)(\exists z)(y \in \{1, 2\} \text{ e } z \in \{2, 3\} \text{ e } x = y + z)\}$

Relações entre Conjuntos

Para $A = \{2, 3, 5, 12\}$ e $B = \{2, 3, 4, 5, 9, 12\}$, todo elemento de A é também um elemento de B . Quando isto acontece, dizemos que A é um *subconjunto* de B .

PRÁTICA 4 Complete a definição: A é um **subconjunto** de B se

$$(\forall x)(x \in A \rightarrow \underline{\hspace{2cm}}).$$

Se A é um subconjunto de B , escrevemos, $A \subseteq B$. Se $A \subseteq B$ mas $A \neq B$ (existe pelo menos um elemento de B que não é elemento de A), então A é dito um **subconjunto próprio** de B e denotado por $A \subset B$.

PRÁTICA 5 Use a notação da lógica formal para definir $A \subset B$.

EXEMPLO 3 Seja

$$\begin{aligned} A &= \{1, 7, 9, 15\} \\ B &= \{7, 9\} \\ C &= \{7, 9, 15, 20\} \end{aligned}$$

Então as seguintes sentenças (dentre outras) são todas verdadeiras:

$$\begin{aligned} B &\subseteq C \quad 15 \in C \\ B &\subseteq A \quad \{7, 9\} \subseteq B \\ B &\subset A \quad \{7\} \subset A \\ A &\not\subseteq C \quad \emptyset \subseteq C \end{aligned}$$

Esta última sentença é verdadeira, porque a sentença $(\forall x)(x \in \emptyset \rightarrow x \in C)$ é verdadeira, uma vez que $x \in \emptyset$ é sempre falsa.

PRÁTICA 6

Seja

$$\begin{aligned} A &= \{x \mid x \in \mathbb{N} \text{ e } x \geq 5\} \\ B &= \{10, 12, 16, 20\} \\ C &= \{x \mid (\exists y)(y \in \mathbb{N} \text{ e } x = 2y)\} \end{aligned}$$

Quais das seguintes sentenças são verdadeiras?

- | | |
|--|-------------------------------|
| a. $B \subseteq C$ | b. $B \subset A$ |
| c. $A \subseteq C$ | d. $26 \in C$ |
| e. $\{11, 12, 13\} \subseteq A$ | f. $\{11, 12, 13\} \subset C$ |
| g. $\{12\} \in B$ | h. $\{12\} \subseteq B$ |
| i. $\{x \mid x \in \mathbb{N} \text{ e } x < 20\} \subseteq B$ | j. $5 \subseteq A$ |
| k. $\{\emptyset\} \subseteq B$ | l. $\emptyset \in A$ |

Suponha que $B = \{x \mid P(x)\}$ e que $A \subseteq B$. Como cada elemento de A também é um elemento de B e P é a propriedade que caracteriza todos os elementos de B , então todos os elementos de A também gozam da propriedade $P(x)$. Os elementos de A "herdam" a propriedade P . De fato, para *provar* que $A \subseteq B$, tomamos um x e A arbitrário e mostramos que $P(x)$ é verdadeira. Se A for um subconjunto próprio de B , os elementos de A terão, normalmente, alguma propriedade característica que nem todos os elementos de B têm.

EXEMPLO 4

Seja

$$B = \{x \mid x \text{ é múltiplo de } 4\}$$

e seja

$$A = \{x \mid x \text{ é múltiplo de } 8\}$$

Então temos $A \subseteq B$. Para provar este resultado, seja $x \in A$; perceba que x é um elemento de A completamente arbitrário. Precisamos mostrar que x satisfaz a propriedade característica de B ; isto significa que precisamos mostrar que x é um múltiplo de 4. Como temos $x \in A$, x satisfaz à propriedade característica de A ; isto é, x é um múltiplo de 8 e, como tal, pode ser escrito na forma $x = m \cdot 8$, para algum inteiro m . Esta equação pode ser escrita como $x = m \cdot 2 \cdot 4$ ou $x = k \cdot 4$, onde $k = 2m$, de forma que k é também um inteiro. Isto mostra que x é um múltiplo de 4 e, portanto $x \in B$.

Existem números (como 12) que são múltiplos de 4, mas não são múltiplos de 8; portanto, $A \subset B$. Outra forma de descrever A é

$$A = \{x \mid x = k \cdot 4 \text{ e } k \text{ é um número par}\}$$

Desta forma fica claro que os elementos de A herdam a propriedade característica de B — ser um múltiplo de 4 —, mas há uma restrição adicional que torna A menos geral que B .

PRÁTICA 7

Seja

$$A = \{x \mid x \in \mathbb{R} \text{ e } x^2 - 4x + 3 = 0\}$$

e

$$B = \{x \mid x \in \mathbb{N} \text{ e } 1 \leq x \leq 4\}$$

Prove que $A \subset B$.

Sabemos que A e B são iguais se contêm os mesmos elementos. Podemos reescrever esta igualdade em termos de subconjuntos: $A = B$ se, e somente se, $A \subseteq B$ e $B \subseteq A$. Provar a inclusão em ambas as direções é a forma normal de estabelecer a igualdade entre dois conjuntos.

EXEMPLO 5

Provaremos que $\{x \mid x \in \mathbb{N} \text{ e } x^2 < 15\} = \{x \mid x \in \mathbb{N} \text{ e } 2x < 7\}$.

Seja $A = \{x \mid x \in \mathbb{N} \text{ e } x^2 < 15\}$ e $B = \{x \mid x \in \mathbb{N} \text{ e } 2x < 7\}$. Para mostrar que $A = B$, mostraremos que $A \subseteq B$ e $B \subseteq A$. Para $A \subseteq B$ precisamos escolher um elemento arbitrário de A , isto é, qualquer número que satisfaça sua propriedade característica e mostrar que ele também satisfaz a propriedade característica de B . Seja $x \in A$. Então x é um inteiro não-negativo que satisfaz a desigualdade $x^2 < 15$. Os inteiros não-negativos com quadrado menor que 15 são 0, 1, 2 e 3, portanto esses são os elementos de A . O dobro de cada um desses inteiros é um número menor que 7. Portanto, cada elemento de A é um elemento de B e $A \subseteq B$.

Mostraremos agora que $B \subseteq A$. Qualquer elemento de B é um inteiro não-negativo cujo dobro é menor que 7. Esses números são 0, 1, 2 e 3, que têm, cada qual, um quadrado menor que 15. Portanto, $B \subseteq A$. •

Conjuntos de Conjuntos

Dado um conjunto S , podemos criar um novo conjunto cujos elementos sejam todos os subconjuntos de S . Este novo conjunto é chamado de **conjunto das partes** de S , $\mathcal{P}(S)$. $\mathcal{P}(S)$ conterá, pelo menos o $\emptyset$ e o próprio S , uma vez que $\emptyset \subseteq S$ e $S \subseteq S$ são sempre verdade.

PRÁTICA 8

Para $A = \{1, 2, 3\}$, qual $\mathcal{P}(A)$?

Na Prática 8, A tem três elementos e $\mathcal{P}(A)$ tem oito elementos. Experimente achar $\mathcal{P}(S)$ para outros conjuntos S até que você seja capaz de responder ao problema da prática a seguir.

PRÁTICA 9

Se S tem n elementos, então $\mathcal{P}(S)$ tem _____ elementos. (Sua resposta funciona para $n = 0$ também?) •

Existem diversas formas de mostrar que, para um conjunto S com n elementos, $\mathcal{P}(S)$ tem 2^n elementos. A prova a seguir se vale da indução. Para a base da indução, tomemos $n = 0$. O único conjunto com 0 elementos é o $\emptyset$. O único subconjunto de $\emptyset$ é ele próprio. Portanto, $\mathcal{P}(\emptyset) = \{\emptyset\}$, um conjunto com $1 = 2^0$ elementos. Assumimos como verdadeiro que para qualquer conjunto com k elementos, o seu conjunto das partes tem 2^k elementos.

Agora, tomemos S com $k + 1$ elementos e ponhamos um de seus elementos, digamos x , de lado. O conjunto que resta tem k elementos, cujo conjunto das partes, pela hipótese de indução, tem 2^k elementos. Cada um desses elementos é também um elemento de $\mathcal{P}(S)$. Os únicos elementos de $\mathcal{P}(S)$ que não foram contados ainda são os que incluem o elemento x . Todos os subconjuntos que contêm o elemento x podem ser encontrados tomando-se todos os subconjuntos que não contenham x e incluindo-lhes x . Ao todo, temos 2^k subconjuntos que não contêm x e 2^k subconjuntos que contêm x . Ou $2^k + 2^k = 2 \cdot 2^k = 2^{k+1}$ subconjuntos. Portanto, $\mathcal{P}(S)$ tem 2^{k+1} elementos.

De forma análoga às tabelas-verdade da Seção 1.1 existe outra maneira de se mostrar que $\mathcal{P}(S)$ tem 2^n elementos, para um conjunto S com n elementos. Lá nós tínhamos n símbolos proposicionais e mostramos que existem 2^n combinações verdadeiro-falso dentre esses símbolos. Mas podemos pensar em cada combinação verdadeiro-falso como representando um subconjunto em particular, com V indicando a inclusão e F indicando a não-inclusão de cada elemento de S no subconjunto. (Por exemplo, a linha da tabela-verdade onde todos os símbolos proposicionais valendo F correspondem ao conjunto vazio.) Portanto, o número de combinações verdadeiro-falso envolvendo n símbolos proposicionais é igual ao número de subconjuntos de um conjunto com n elementos; e ambos são 2^n .

Operações Binárias e Unárias

Um conjunto, por si só, não é objeto de muito interesse até que se faça algo com seus elementos. Por exemplo, podemos realizar diversas operações aritméticas sobre os elementos do conjunto $\mathbb{Z}$. Podemos subtrair dois inteiros ou podemos tomar o simétrico de um inteiro. A subtração é uma operação binária em $\mathbb{Z}$. Uma operação binária atua sobre dois números e uma operação unária atua sobre um único número. Para vermos o que é usado em uma operação binária, daremos uma olhada mais detalhada na subtração. Para quaisquer dois inteiros x e y , $x - y$ resultará em uma, e apenas uma resposta, e essa resposta será sempre um inteiro. Finalmente, a subtração é realizada sobre um **par ordenado** de números. Por exemplo, $7 - 5$ não produz o mesmo resultado que $5 - 7$. Um par ordenado é denotado por (x, y) , onde x é o primeiro componente do par e y é o segundo. A ordem é importante em um par ordenado; portanto, os conjuntos $\{1, 2\}$ e $\{2, 1\}$ são iguais mas os pares ordenados $(1, 2)$ e $(2, 1)$ não o são. Você já deve estar familiarizado com os pares ordenados usados como coordenadas para localização de pontos no plano. O ponto $(1, 2)$ é diferente do ponto $(2, 1)$. Dois pares ordenados (x, y) e (u, v) são iguais apenas quando $x = u$ e $y = v$.

PRÁTICA 10

Dado que $(2x - y, x + y) = (7, -1)$, ache x e y . •

PRÁTICA 11 Seja $S = \{3, 4\}$. Liste todos os pares ordenados (x, y) de elementos de S . •

Podemos generalizar essas propriedades da subtração de inteiros a fim de definir uma operação binária $\circ$ no conjunto S . O símbolo $\circ$ representa uma operação qualquer; em qualquer discussão específica, ele será substituído pelo símbolo da operação apropriada, tal como o símbolo da subtração.

Definição: Operação Binária

$\circ$ é uma **operação binária** sobre um conjunto S se para qualquer par ordenado (x, y) de elementos de S , $x \circ y$ existe, é único e é um elemento de S .

O fato do valor $x \circ y$ existir e ser único é descrito ao dizer-se que a operação binária $\circ$ é **bem-definida**. A propriedade de que $x \circ y$ deve sempre pertencer a S é descrita ao dizer-se que S é **fechado** sob a operação $\circ$.

EXEMPLO 6 A adição, subtração e multiplicação são todas operações binárias em $\mathbb{Z}$. Por exemplo, quando realizamos a adição no par ordenado de inteiros (x, y) , $x + y$ existe, é único e é inteiro. •

EXEMPLO 7 As operações lógicas de conjunção, disjunção, implicação e equivalência são operações binárias no conjunto das wffs proposicionais. Se P e Q são wffs proposicionais, então $P \wedge Q$, $P \vee Q$, $P \rightarrow Q$ e $P \leftrightarrow Q$ são wffs proposicionais únicas. •

Uma $\circ$ candidata a uma operação pode não ser uma operação binária em um conjunto S por três motivos: (1) Existem elementos $x, y \in S$ para os quais $x \circ y$ não existe; (2) existem elementos $x, y \in S$ para os quais $x \circ y$ fornece mais de um resultado; ou (3) existem elementos $x, y \in S$ para os quais $x \circ y$ não pertence a S .

EXEMPLO 8 A divisão não é uma operação binária em $\mathbb{Z}$ porque $x \div 0$ não existe. •

EXEMPLO 9 Seja $x \circ y$ definida em $\mathbb{N}$ por

$$x \circ y = \begin{cases} 1 & \text{se } x \geq 5 \\ 0 & \text{se } x < 5 \end{cases}$$

Então, pela primeira parte da definição de $\circ$, $5 \circ 1 = 1$, mas, pela segunda parte, $5 \circ 1 = 0$. Portanto, $\circ$ não é bem-definida em $\mathbb{N}$. •

EXEMPLO 10 A subtração não é uma operação binária em $\mathbb{N}$ porque $\mathbb{N}$ não é fechado sob a subtração. (Por exemplo, $1 - 1 \notin \mathbb{N}$.) •

Para que $\#$ seja uma **operação unária** no conjunto S , é preciso que para qualquer $x \in S$, $x^\#$ exista, seja único e seja um elemento de S . Caso qualquer dessas condições não seja satisfeita, não teremos uma operação unária.

EXEMPLO 11 Seja $x^\#$ definida por $x^\# = -x$, de tal forma que $x^\#$ é o negativo de x . Então $\#$ é uma operação unária em $\mathbb{Z}$, mas não em $\mathbb{N}$ porque $\mathbb{N}$ não é fechado sob $\#$. •

EXEMPLO 12 O conectivo lógico da negação é uma operação unária no conjunto das wffs proposicionais. Se P for uma wff proposicional, então P' será uma wff proposicional. •

Com esses exemplos fica claro que para $\circ$ (ou $\#$) ser ou não uma operação binária (ou unária) não depende apenas de sua definição, mas também do conjunto em questão.

PRÁTICA 12 Quais das seguintes operações não são binárias nem unárias nos conjuntos dados. Por que não?

- $x \circ y = x \div y$; $S =$ conjunto de todos os inteiros positivos
- $x \circ y = x \div y$; $S =$ conjunto de todos os números racionais positivos
- $x \circ y = x^y$; $S = \mathbb{R}$

d. $x \circ y = \text{máximo entre } x \text{ e } y$; $S = \mathbb{N}$

e. $x^{\#} = \sqrt{x}$; $S = \text{conjunto de todos os números reais positivos}$

f. $x^{\#} = \text{solução da equação } (x^{\#})^2 = x$; $S = \mathbb{C}$

Até agora todas nossas operações binárias foram descritas através de uma descrição ou uma equação. Suponha que S é um conjunto finito, $S = \{x_1, x_2, \dots, x_n\}$. Então uma operação arbitrária $\circ$ em S pode ser definida por uma matriz, ou tabela, onde cada elemento i, j (linha i , coluna j) denota $x_i \circ x_j$.

EXEMPLO 13 Seja $S = \{2, 5, 9\}$ e seja $\circ$ definida pela matriz

$\circ$	2	5	9
2	2	2	9
5	5	9	2
9	5	5	9

Portanto, $2 \circ 5 = 2$ e $9 \circ 2 = 5$, $\circ$ é uma operação binária em S .

Operações em Conjuntos

A maioria das operações que vimos opera em números, mas também podemos realizar operações em conjuntos. Dado um conjunto arbitrário S , podemos definir algumas operações binárias e unárias no conjunto $\mathcal{P}(S)$. S , neste caso, é chamado o **conjunto universo** ou **universo de discurso**. O conjunto universo define o contexto dos objetos em questão. Se $S = \mathbb{Z}$, por exemplo, então todos seus subconjuntos conterão apenas inteiros.

Uma operação binária em $\mathcal{P}(S)$ precisa atuar em quaisquer dois subconjuntos de S para produzir um único subconjunto de S . Existem pelo menos duas maneiras naturais pelas quais isso pode acontecer.

EXEMPLO 14 Seja S o conjunto de todos os estudantes da Universidade Selicon. Os elementos de $\mathcal{P}(S)$ são, portanto, conjuntos de estudantes. Seja A o conjunto de graduandos em Ciência da Computação e seja B o conjunto de graduandos em Administração. Tanto A quanto B pertencem a $\mathcal{P}(S)$. Um novo conjunto de estudantes pode ser definido como consistindo em todos os estudantes que sejam graduandos em Ciência da Computação ou Administração (ou ambos); este conjunto é chamado a *união* de A e B . Outro conjunto pode ser definido como sendo composto de todos que sejam graduandos em Ciência da Computação e também em Administração. Este conjunto (que pode ser vazio) é chamado a *interseção* de A e B .

Definição: União de Conjuntos

Seja $A, B \in \mathcal{P}(S)$. A **união** de A e B , denotada por $A \cup B$, é $\{x \mid x \in A \text{ ou } x \in B\}$.

PRÁTICA 13 Seja $A, B \in \mathcal{P}(S)$. Complete a seguinte definição: A **interseção** de A e B , denotada por $A \cap B$ é $\{x \mid x \text{ _____}\}$.

EXEMPLO 15 Seja $A = \{1, 3, 5, 7, 9\}$ e $B = \{3, 5, 6, 10, 11\}$. Podemos considerar A e B como elementos de $\mathcal{P}(\mathbb{N})$. Então, $A \cup B = \{1, 3, 5, 6, 7, 9, 10, 11\}$ e $A \cap B = \{3, 5\}$. Tanto $A \cup B$ quanto $A \cap B$ são elementos de $\mathcal{P}(\mathbb{N})$. ♦

Podemos usar *diagramas de Venn* (cujo nome é dado em função do matemático inglês do século 19, John Venn) para visualizar as operações binárias de união e interseção. As áreas sombreadas nas Figs. 3.1 e 3.2 ilustram os conjuntos que resultam da aplicação das operações binárias nos conjuntos dados.

Definiremos uma operação unária em $\mathcal{P}(S)$.

Definição: Complemento de um Conjunto

Para um conjunto $A \in \mathcal{P}(S)$, o **complemento** de A , A' é $\{x \mid x \in S \text{ e } x \notin A\}$.

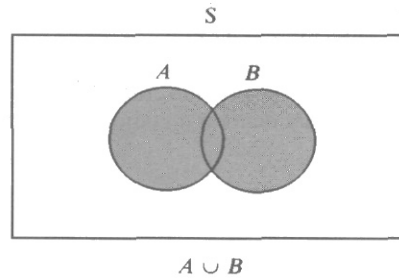


Figura 3.1

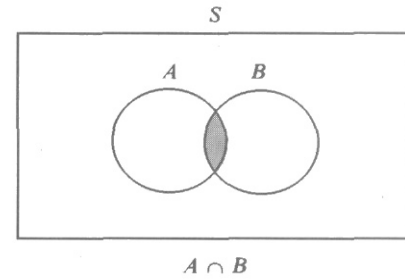


Figura 3.2

PRÁTICA 14 Ilustre A' em um diagrama de Venn.

Outra operação binária entre os conjuntos A e B é a **diferença de conjuntos**: $A - B = \{x \mid x \in A \text{ e } x \notin B\}$. Esta operação pode ser escrita como $A - B = \{x \mid x \in A \text{ e } x \in B'\}$ e, finalmente como $A - B = A \cap B'$.

PRÁTICA 15 Ilustre $A - B$ em um diagrama de Venn.

Dois conjuntos A e B tais que $A \cap B = \emptyset$ são ditos **disjuntos**. Portanto, $A - B$ e $B - A$, por exemplo, são conjuntos disjuntos.

EXEMPLO 16 Sejam

$$\begin{aligned} A &= \{x \mid x \text{ é um inteiro não-negativo par}\} \\ B &= \{x \mid (\exists y)(y \in \mathbb{N} \text{ e } x = 2y + 1)\} \\ C &= \{x \mid (\exists y)(y \in \mathbb{N} \text{ e } x = 4y)\} \end{aligned}$$

subconjuntos de $\mathbb{N}$. Como B representa o conjunto dos números inteiros não-negativos ímpares, A e B são disjuntos. Além disso, todo inteiro ou é par ou é ímpar, portanto, $A \cup B = \mathbb{N}$. Esses dois fatos nos dizem, ainda, que $A' = B$. Todo múltiplo de 4 é um número par, portanto, C é um subconjunto de A , donde $A \cup C = A$. C é, na verdade, um subconjunto próprio de A e $A - C = \{x \mid (\exists y)(y \in \mathbb{N} \text{ e } x = 4y + 2)\}$.

PRÁTICA 16 Sejam

$$\begin{aligned} A &= \{1, 2, 3, 5, 10\} \\ B &= \{2, 4, 7, 8, 9\} \\ C &= \{5, 8, 10\} \end{aligned}$$

subconjuntos de $S = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$. Encontre

- $A \cup B$
- $A - C$
- $B' \cap (A \cup C)$

Produto Cartesiano

Existe uma última operação que definiremos com base nos elementos de $\mathcal{P}(S)$.

Definição: Produto Cartesiano

Sejam A e B subconjuntos de S . O **produto cartesiano (produto cruzado)** de A e B , denotado por $A \times B$ é definido por

$$A \times B = \{(x, y) \mid x \in A \text{ e } y \in B\}$$

Portanto, o produto Cartesiano de dois conjuntos A e B é o conjunto de todos os pares ordenados cujas primeiras coordenadas pertençam a A e as segundas pertençam a B . O produto cruzado não é uma operação

binária em $\mathcal{P}(S)$. Na verdade, ele opera em um par ordenado de membros de $\mathcal{P}(S)$ e fornece um resultado único e o conjunto resultante não é, em geral, um subconjunto de S ; isto é, não é um elemento de $\mathcal{P}(S)$. A condição de fechamento para uma operação binária não é, portanto, satisfeita.

Como estaremos, freqüentemente, interessados no produto cruzado de um conjunto com ele próprio, abreviaremos $A \times A$ por A^2 ; em geral, usaremos A^n para denotar o conjunto de todas as n -uplas $(x_1, x_2, \dots, x_n)$ de elementos de A .

PRÁTICA 17 Sejam $A = \{1, 2\}$ e $B = \{3, 4\}$.

- Encontre $A \times B$.
- Encontre $B \times A$.
- Encontre A^2 .
- Encontre A^3 .

Identidades de Conjuntos

Existem diversas igualdades envolvendo as operações de união, interseção, diferença e complemento que são verdadeiras para todos os subconjuntos de um dado conjunto S . Como elas são independentes dos subconjuntos usados, essas igualdades são chamadas de identidades. Algumas dessas identidades básicas são mostradas a seguir. Os nomes e as formas dessas identidades são muito parecidos com as equivalências tautológicas da Seção 1.1 (verifique). Veremos no Cap. 7 que esta semelhança não é uma coincidência.

Identidades de Conjuntos Básicas

1a. $A \cup B = B \cup A$	1b. $A \cap B = B \cap A$	(propriedades comutativas)
2a. $(A \cup B) \cup C =$ $A \cup (B \cup C)$	2b. $(A \cap B) \cap C =$ $A \cap (B \cap C)$	(propriedades associativas)
3a. $A \cup (B \cap C) =$ $(A \cup B) \cap (A \cup C)$	3b. $A \cap (B \cup C) =$ $(A \cap B) \cup (A \cap C)$	(propriedades distributivas)
4a. $A \cup \emptyset = A$	4b. $A \cap S = A$	(propriedades de identidade)
5a. $A \cup A' = S$	5b. $A \cap A' = \emptyset$	(propriedades de complemento)

(Perceba que a propriedade 2a nos permite escrever $A \cup B \cup C$ sem a necessidade do uso de parênteses; 2b nos permite escrever $A \cap B \cap C$.)

EXEMPLO 17 Vamos demonstrar a identidade 3a. Poderíamos desenhar diagramas de Venn para cada lado da equação e ver que eles são iguais. No entanto, a identidade 3a é tida como válida para todos os subconjuntos A , B e C , e qualquer figura que desenharmos não pode ser completamente geral. Portanto, se desenharmos A e B como disjuntos, seria um caso especial; mas se os desenhássemos com interseção não-vazia, não estaríamos considerando o caso em que A e B são disjuntos. Para não termos que desenhar uma figura para cada caso, vamos demonstrar a igualdade do conjunto através da inclusão em ambas as direções. Portanto, demonstraremos

$$A \cup (B \cap C) \subseteq (A \cup B) \cap (A \cup C)$$

e também que

$$(A \cup B) \cap (A \cup C) \subseteq A \cup (B \cap C)$$

Para mostrar que $A \cup (B \cap C) \subseteq (A \cup B) \cap (A \cup C)$, tomaremos x como um elemento arbitrário de $A \cup (B \cap C)$. Podemos proceder da seguinte maneira:

$$\begin{aligned} x \in A \cup (B \cap C) &\rightarrow x \in A \text{ ou } x \in (B \cap C) \\ &\rightarrow x \in A \text{ ou } (x \in B \text{ e } x \in C) \\ &\rightarrow (x \in A \text{ ou } x \in B) \text{ e } (x \in A \text{ ou } x \in C) \\ &\rightarrow x \in (A \cup B) \text{ e } x \in (A \cup C) \\ &\rightarrow x \in (A \cup B) \cap (A \cup C) \end{aligned}$$

Para mostrar que $(A \cup B) \cap (A \cup C) \subseteq A \cup (B \cap C)$, invertamos o argumento acima. ♦

PRÁTICA 18 Demonstre a identidade 4a. •

Uma vez que tenhamos provado as identidades dessa lista, podemos usá-las para demonstrar outras identidades de conjuntos, como quando usamos identidades algébricas como $(x - y)^2 = x^2 - 2xy + y^2$ para reescrever expressões algébricas.

EXEMPLO 18 Podemos usar as identidades básicas de conjuntos para provar

$$[A \cup (B \cap C)] \cap ([A' \cup (B \cap C)] \cap (B \cap C)') = \emptyset$$

para A, B e C quaisquer subconjuntos de S. Na demonstração a seguir, os números à direita indicam as identidades usadas em cada passo.

$$\begin{aligned} & [A \cup (B \cap C)] \cap ([A' \cup (B \cap C)] \cap (B \cap C)') \\ &= ([A \cup (B \cap C)] \cap ([A' \cup (B \cap C)] \cap (B \cap C)')) \quad (2b) \\ &= ((B \cap C) \cup A) \cap ((B \cap C) \cup A') \cap (B \cap C)' \quad (1a \text{ duas vezes}) \\ &= [(B \cap C) \cup (A \cap A')] \cap (B \cap C)' \quad (3a) \\ &= [(B \cap C) \cup \emptyset] \cap (B \cap C)' \quad (5b) \\ &= (B \cap C) \cap (B \cap C)' \quad (4a) \\ &= \emptyset \quad (5b) \end{aligned} \quad \bullet$$

O **dual** para cada identidade de conjunto na nossa lista também aparece na lista. Os duais são obtidos substituindo-se $\cup$ por $\cap$ e trocando S por $\emptyset$. O dual da identidade no Exemplo 18 é

$$[A \cap (B \cup C)] \cup ([A' \cap (B \cup C)] \cup (B \cup C)') = S$$

que podemos demonstrar substituindo cada identidade básica de conjuntos usada na demonstração do Exemplo 18 por sua dual. Como este método sempre funciona, sempre que tivermos demonstrado uma identidade de conjuntos usando as identidades básicas, teremos também demonstrado sua dual.

PRÁTICA 19 a. Usando as identidades básicas de conjuntos, estabeleça a identidade

$$[C \cap (A \cup B)] \cup [(A \cup B) \cap C'] = A \cup B$$

(A, B e C são quaisquer subconjuntos de S.)

b. Indique a identidade dual que você sabe ser verdadeira. •

Existem, portanto, duas formas de se demonstrar uma identidade de conjuntos: (1) estabelecer a inclusão dos conjuntos em ambas as direções e (2) verificar a identidade (ou sua dual) através do uso de identidades já demonstradas.

Conjuntos na Programação

Linguagens de Programação e Conjuntos

O conceito de conjuntos é um conceito útil e uma noção geral que figura como tipo de dados-padrão em algumas linguagens de programação, tais como Pascal e Modula-2. Nessas linguagens o conjunto universo S precisa ser especificado e então as variáveis que representam subconjuntos de S, isto é, elementos de $\mathcal{P}(S)$, podem ser definidas. Existe um limite de tamanho para o conjunto universo de forma que os elementos de $\mathcal{P}(S)$ não podem ser arbitrariamente grandes; além disso, o conjunto universo precisa ser enumerável ou contável em uma determinada ordem, como uma sequência.

EXEMPLO 19 A declaração de tipo Pascal a seguir define o conjunto universo *Alfabeto* e como o conjunto de todos os caracteres do teclado, tais como A, X, 7 e %.

```
type
  Alfabeto = set of char;
```

Agora, subconjuntos de *Alfabeto* podem ser definidos como variáveis no programa, através de declarações como

```

var
  Iniciais : Alfabeto;
  Letras  : Alfabeto;

```

e as atribuições a seguir seriam válidas para essas variáveis:

```

Iniciais := ['A' .. 'F'];
Letras  := ['C' .. 'G'];

```

onde os pontos indicam uma seqüência na ordem de enumeração que, neste caso, é a alfabética. (Em Pascal são usados colchetes no lugar de chaves para denotar os conjuntos.)

Após a atribuição acima, *Iniciais* tem o valor {A, B, C, D, E, F} e *Letras* tem o valor {C, D, E, F, G}.

A ordem de enumeração é conveniente para a definição de quais são os elementos de um conjunto, mas como eles são conjuntos, a ordem dos elementos não é importante, e a atribuição

```

Iniciais := ['B', 'A', 'D', 'F', 'E', 'C'];

```

dá o mesmo valor a *Iniciais* que a atribuição anterior. Como um conjunto não é ordenado, não podemos referenciar elementos individuais do conjunto; portanto, não podemos dizer "o terceiro elemento" do conjunto *Iniciais*.

As expressões condicionais podem ser formadas pela comparação das variáveis do tipo conjunto *A* e *B* da seguinte maneira:

Sintaxe de Programação	Semântica (Significado)
$A=B$	$A = B$
$A<>B$	$A \neq B$
$A< = B$	$A \subseteq B$
$A> = B$	$B \subseteq A$

Finalmente, as operações de união, interseção e diferença são oferecidas pelos operadores +, * e —, respectivamente. O conteúdo de uma variável conjunto *A* pode ser construído dinamicamente durante a execução do programa, iniciando-se *A* como um conjunto vazio, e então realizando uniões de *A* com conjuntos com um único elemento, a fim de incluir esses elementos em *A*.

PRÁTICA 20 Após as atribuições do Exemplo 19 a *Iniciais* e *Letras*, encontre os seguintes conjuntos:

- Iniciais* * *Letras*
- Iniciais* — *Letras*
- Iniciais* + *Letras*
- Letras* + ['Q']

Uma linguagem de programação, chamada SETL (de *set language* — linguagem de conjuntos), dá ênfase à manipulação de conjuntos para a realização de tarefas relacionadas à programação. A SETL inclui conjuntos e *n*-uplas como tipos de dados básicos. Além do conjunto usual de operações entre conjuntos, novos conjuntos podem ser formados a partir dos primeiros através do uso de tipos de descrições teóricas de conjuntos que temos empregado nesta seção.

EXEMPLO 20 Dado um conjunto *S* de inteiros, novos conjuntos podem ser criados em SETL por descrições semelhantes às seguintes:

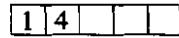
- $\{x \mid x \text{ in } S \mid x < 7\}$
(o conjunto dos elementos de *S* que são menores que 7)
- $\{x * x \mid x \text{ in } S\}$
(o conjunto dos quadrados dos elementos de *S*)
- $\{x \mid x \text{ in } [1..10] \mid \text{exists } y \text{ in } S \mid x = y * y\}$
(o conjunto de quadrados perfeitos dos elementos de *S* entre 1 e 10.)

Se o conjunto não for fornecido como um tipo de dados básico em uma linguagem de programação, então o programador pode implementar as idéias de conjuntos teóricos, implementando um tipo abstrato de dados para "conjuntos". Um **tipo abstrato de dados** é nada mais do que um modelo mental de uma coleção de

itens relacionados, juntamente com as operações apropriadas a serem realizadas entre instâncias dessas coleções. Neste caso, o modelo mental é um conjunto, e as operações apropriadas são as operações entre conjuntos, tais como a união, interseção e complemento. O programador deve usar as facilidades fornecidas pela linguagem para simular os conjuntos e as operações que não forem fornecidas.

Uma forma comum de representação de conjuntos é através de um vetor, sendo cada elemento uma entrada desse vetor. Isto limita o número de elementos do conjunto; isto é, o conjunto universo não pode exceder o tamanho do vetor. Isto também impõe uma ordem artificial aos elementos do conjunto. Operações de conjuntos, tais como a união e interseção, são realizadas através de manipulações do vetor.

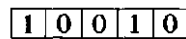
EXEMPLO 21 Suponha que o conjunto universo S seja $S = \{1, 2, 3, 4, 5\}$. Então um vetor de cinco elementos comporta qualquer elemento de $\mathcal{P}(S)$. O conjunto $A = \{1, 4\}$, por exemplo, pode ser representado pelo vetor



Apenas as duas primeiras entradas do vetor são itens válidos. Pode ser mantida uma variável separada que contenha um contador do número de entradas válidas. •

Outra representação para conjuntos impõe uma ordem para o conjunto universo e então usa um vetor de bits, um vetor de Os e Is, para denotar quais elementos do conjunto universo estão presentes ou ausentes ao subconjunto.

EXEMPLO 22 Usando a ordenação natural de S do Exemplo 21, o subconjunto $A = \{1, 4\}$ poderia ser representado pelo vetor de bits



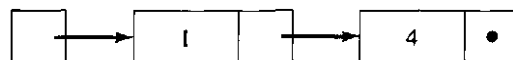
A representação em vetores de bits torna as operações de conjuntos mais simples de serem implementadas. A união é realizada através do OU dos componentes de ambos os vetores, e a interseção é feita através do E. •

PRÁTICA 21 Encontre o vetor de bits que representa o complemento do conjunto do Exemplo 22. •

Uma outra possibilidade é usar listas encadeadas para representar conjuntos. Uma **lista encadeada** é uma lista de dados com "ponteiros" que conectam cada item da lista ao seguinte. Um **ponteiro nulo** (um ponteiro que aponta para lugar algum) marca o fim da lista. Algumas linguagens de programação, tais como Pascal, oferecem um tipo de dado de ponteiro; em outras linguagens, ponteiros precisam ser simulados usando os recursos da linguagem de programação.

A representação através de uma lista encadeada apresenta a vantagem de não impor uma limitação *a priori* no tamanho do conjunto, mas continua a impor uma ordem em seus elementos. As operações entre os conjuntos são realizadas através de varredura, comparação, inserção e deleção nas listas encadeadas.

EXEMPLO 23 O subconjunto $A = \{1, 4\}$ pode ser representado pela lista encadeada mostrada na figura abaixo, onde o ponto representa o ponteiro nulo.



Programação Orientada por Objeto

A programação orientada por objeto baseia-se na idéia de um tipo abstrato de dados um pouco mais elaborado através da possibilidade de **encapsulamento** — a possibilidade de definir um tipo de dado junto com as operações apropriadas para manipulação de suas instâncias, tudo em um pacote com seção de código que todo o resto do programa não consegue acessar. Um tipo objeto de conjunto, SET, poderia ser definido e encapsulado juntamente com os procedimentos para achar o complemento de um conjunto ou a união e interseção de dois conjuntos. Variáveis (objetos) que forem instâncias do tipo objeto SET podem chamar esses procedimentos.

Outra idéia que é importante na programação orientada por objeto é a **herança**. Uma vez o tipo-objeto tenha sido definido, pode-se criar um tipo-filho que "herde" todas as suas propriedades e operações e adicione propriedades ou operações locais especializadas conforme a necessidade. Desta forma, o código que já estava testado no tipo-pai é um **código reutilizável**. Isto reduz a quantidade de código que precisa ser escrito (e tes-

tado). O importante é que o tipo-filho é essencialmente um subconjunto do tipo-pai. Lembremos que se A é um subconjunto próprio de B e $B = \{x \mid P(x)\}$, então todos os elementos de A herdam a propriedade de P além de possuírem alguma propriedade local.

EXEMPLO 24 A definição a seguir define um tipo-objeto chamado *Solido* que tem um certo atributo ou propriedade chamada *Volume*.

```
type
  Solido = object
    Volume : integer;
  end;
```

Agora podemos definir um tipo-filho chamado *Cilindro* que herde a propriedade de volume, mas tem a propriedade local própria, *Raio*.

```
type
  Cilindro = object (Solido)
    Raio : integer;
  end;
```

Qualquer variável declarada como *Cilindro* terá tanto o campo *Volume*, herdado do tipo-*Solido*, quanto o campo *Raio*. Na notação teórica de conjuntos desta seção, poderíamos dizer que

$$\text{Solido} = \{x \mid \text{Volume}(x) \text{ é definido}\}$$

e

$$\text{Cilindro} \subset \text{Solido}$$

onde

$$\text{Cilindro} = \{x \mid \text{Volume}(x) \text{ é definido e Raio}(x) \text{ é definido}\}$$

Conjuntos Contáveis e Incontáveis

Em um conjunto finito S , podemos sempre designar um elemento como o primeiro, s_1 ; outro elemento como o segundo, s_2 , e assim por diante. Se houver k elementos no conjunto, esses elementos podem ser listados na ordem que selecionarmos:

$$s_1, s_2, \dots, s_k$$

Esta lista representa todo o conjunto.

Se o conjunto for infinito, pode ser que ainda sejamos capazes de selecionar um primeiro elemento s_1 , um segundo elemento s_2 e assim por diante, de forma que a lista

$$s_1, s_2, s_3, \dots$$

representa todos os elementos do conjunto. Este tipo de conjunto é dito **denumerável**.^{*} Tanto os conjuntos finitos quanto os denumeráveis são conjuntos **contáveis**, porque podemos contar, ou enumerar, todos os seus elementos. Ser contável não significa que podemos dizer o número total de elementos no conjunto, mas que podemos dizer "Eis o primeiro elemento", "Eis o segundo" e assim por diante, através de todos os elementos do conjunto. Existem, no entanto, conjuntos infinitos que são **não-contáveis**. Em um conjunto não-contável, não há maneira de contar os elementos e obter todo o conjunto no processo. Antes de provarmos que existem conjuntos não-contáveis, vejamos alguns conjuntos denumeráveis (conjuntos infinitos e contáveis).

^{*} Apesar da palavra *denumerable* não constar do vocabulário da língua portuguesa, tomamos a liberdade de lançar mão deste neologismo uma vez que não é uma palavra corriqueira no inglês (*denumerable*) e que contém o mesmo radical de enumerável. (N. do T.)

EXEMPLO 25 O conjunto $\mathbb{N}$ é denumerável.

Para provar que um conjunto é denumerável, precisamos apenas exibir um esquema de contagem. Para o conjunto $\mathbb{N}$ dos inteiros não-negativos, é claro que

$$0, 1, 2, 3, \dots$$

é uma enumeração que inclui, em algum momento, qualquer elemento do conjunto. •

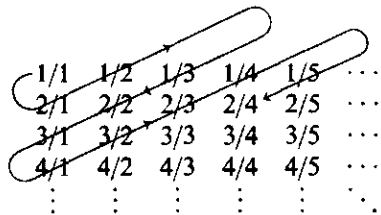
PRÁTICA 22 Prove que o conjunto dos inteiros positivos pares é denumerável. •

EXEMPLO 26 O conjunto $\mathbb{Q}^+$ dos números racionais positivos é denumerável.

Assumimos que cada número racional positivo é escrito como uma fração de inteiros positivos. Podemos escrever todas essas frações como as que têm numerador 1 em uma linha, as que têm numerador 2 na segunda linha, e assim por diante:

$$\begin{array}{cccccc} 1/1 & 1/2 & 1/3 & 1/4 & 1/5 & \dots \\ 2/1 & 2/2 & 2/3 & 2/4 & 2/5 & \dots \\ 3/1 & 3/2 & 3/3 & 3/4 & 3/5 & \dots \\ 4/1 & 4/2 & 4/3 & 4/4 & 4/5 & \dots \end{array}$$

Para mostrar que o conjunto de todas as frações nesta matriz é denumerável, devemos traçar uma seta que passe por toda a matriz, começando no $1/1$; e, em seguida, fornecendo uma enumeração de todo o conjunto. Portanto, a fração $1/3$ é o quarto elemento desta enumeração:



Assim, o conjunto representado pela matriz é denumerável.

Para obter uma enumeração de $\mathbb{Q}^+$, usamos a enumeração do conjunto acima, mas eliminamos as frações que não estejam em sua forma mais simples. Isto evita o problema de listar $1/2$ e $2/4$, por exemplo, que representam o mesmo número racional positivo. A enumeração de $\mathbb{Q}^+$ começa, portanto, com

$$1/1, 2/1, 1/2, 1/3, 3/1, 4/1, \dots$$

Por exemplo, eliminamos $2/2$ que se reduz a $1/1$. •

PRÁTICA 23

Qual a 11.^a fração na enumeração acima? Qual o 11.^o racional positivo? •

Veremos agora que existe um conjunto infinito que não é denumerável. A técnica de demonstração apropriada para mostrar que o conjunto A não tem a propriedade B é assumir que A tem a propriedade B , e buscar uma contradição. A prova abaixo é uma prova muito famosa conhecida como o **método de diagonalização de Cantor** em homenagem a Georg Cantor, matemático alemão do século 19, conhecido como "pai da teoria dos conjuntos".

EXEMPLO 27 Mostraremos que o conjunto de todos os números reais entre 0 e 1 é não-contável. Escreveremos esses números em sua forma decimal, uma vez que qualquer número do conjunto possa ser escrito como

$$0.d_1d_2d_3\dots$$

Um número como 0.24999999 ... pode ser reescrito em sua forma alternativa como 0.2500000 ... A fim de evitar escrever o mesmo elemento duas vezes, optaremos (arbitrariamente) por usar a primeira representação, e não a última. Agora vamos assumir que nosso conjunto é contável. Portanto, existe alguma enumeração do conjunto. Podemos descrever uma enumeração do conjunto como a seguir, d_{ij} é a j -ésima casa decimal do i -ésimo número da enumeração:

$$\begin{array}{cccc} 0.d_{11}d_{12}d_{13}\cdots \\ 0.d_{21}d_{22}d_{23}\cdots \\ 0.d_{31}d_{32}d_{33}\cdots \\ \vdots \quad \vdots \quad \vdots \end{array}$$

Agora construímos um número real $p = 0.p_1p_2p_3\ldots$ da seguinte maneira: p_i é sempre escolhido como 5 se $d_{ii} \neq 5$ e 6 se $d_{ii} = 5$. Portanto, p é um número real entre 0 e 1. Por exemplo, se a enumeração começar por

$$\begin{array}{l} 0.342134\ldots \\ 0.257001\ldots \\ 0.546122\ldots \\ 0.716525\ldots \end{array}$$

então $d_{11} = 3$, $d_{22} = 5$, $d_{33} = 6$ e $d_{44} = 5$, portanto, $p_1 = 5$, $p_2 = 6$, $p_3 = 5$ e $p_4 = 6$. Assim, p começa por 0.5656 ...

Se compararmos p com a enumeração do conjunto, p é diferente do primeiro número na primeira casa decimal, do segundo número na segunda casa decimal, do terceiro na terceira casa decimal, e assim por diante. Portanto, p não é igual a qualquer elemento na enumeração. Além disso, como p não contém Os à direita do ponto decimal, ele não é a representação alternativa de qualquer número na enumeração. Portanto p é um número real entre 0 e 1 diferente de qualquer outro número na enumeração, mas a enumeração é suposta a incluir todos os elementos do conjunto. Temos aqui, portanto, uma contradição e o conjunto de todos os números reais entre 0 e 1 é não-contável. (Por que esta prova é chamada de "método da diagonalização"?) •

Apesar de ser interessante e talvez surpreendente saber que existem conjuntos que não são contáveis, normalmente estaremos interessados nos conjuntos contáveis. Um computador, por via de regra, pode gerir apenas conjuntos finitos. No restante deste capítulo, nós também nos limitaremos aos conjuntos finitos e às várias maneiras de contar seus elementos.

Revisão da Seção 3.1

Técnicas

- Descrever os conjuntos por uma lista de elementos e por uma propriedade característica
- Provar que um conjunto é um subconjunto de outro
- Achar o conjunto das partes de um conjunto
- Verificar se as propriedades para uma operação binária ou unária são satisfeitas
- Formar novos conjuntos tomando a união, interseção, complemento e produto cartesiano dos conjuntos
- Provar identidades de conjuntos mostrando a inclusão de conjuntos em ambas as direções ou usando as identidades básicas
- Representar conjuntos nas formas de vetores, vetores de bits e listas encadeadas
- Demonstrar a denumerabilidade de certos conjuntos
- Usar o método de diagonalização de Cantor para provar que alguns conjuntos são não-contáveis.

Idéias Principais

Conjuntos são coleções não-ordenadas de objetos que podem ser relacionados (conjuntos iguais, subconjuntos etc.) ou combinados (união, interseção etc.).

Conjuntos-padrão têm suas próprias notações.

O conjunto das partes de um conjunto com n elementos tem 2^n elementos.

Existem identidades básicas (em pares duais) e elas podem ser usadas para provarem identidades de conjuntos; uma vez que uma identidade seja provada desta maneira, sua dual também será válida. Os conjuntos podem ser fornecidos como tipos de dados-padrão em linguagens de programação, e um tipo abstrato de dados para representar conjuntos pode ser implementado de diversas formas. A noção da programação orientada a objeto de herança é uma consequência do conceito de subconjunto. Conjuntos contáveis podem ser enumerados e existem conjuntos não-contáveis.

Exercícios 3.1

- ★1. Seja $S = \{2, 5, 17, 27\}$. Quais das sentenças a seguir são verdadeiras?
 a. $5 \in S$ b. $2 + 5 \in S$ c. $\emptyset \in S$
2. Seja $B = \{x \mid x \in \mathbb{Q} \text{ e } -1 < x < 2\}$. Quais das sentenças a seguir são verdadeiras?
 a. $0 \in B$ b. $-1 \in B$ c. $-0.84 \in B$ d. $\sqrt{2} \in B$
3. Quantos conjuntos diferentes são descritos abaixo? Quais são eles?
 $\{2, 3, 4\}$ $\emptyset$
 $\{x \mid x \text{ é a primeira letra de céu, boi ou açude}\}$ $\{x \mid x \text{ é a primeira letra de céu, boi e açude}\}$
 $\{x \mid x \in \mathbb{N} \text{ e } 2 \leq x \leq 4\}$ $\{2, a, 3, b, 4, c\}$
 $\{a, b, c\}$ $\{3, 4, 2\}$
- ★4. Descreva cada um dos conjuntos a seguir, listando seus elementos:
 a. $\{x \mid x \in \mathbb{N} \text{ e } x^2 < 25\}$
 b. $\{x \mid x \in \mathbb{N} \text{ e } x \text{ é par e } 2 < x < 11\}$
 c. $\{x \mid x \text{ é um dos três primeiros presidentes do Brasil}\}$
 d. $\{x \mid x \in \mathbb{R} \text{ e } x^2 = -1\}$
 e. $\{x \mid x \text{ é um dos estados da região Nordeste do Brasil}\}$
 f. $\{x \mid x \in \mathbb{Z} \text{ e } |x| < 4\}$ ($|x|$ denota a função do valor absoluto)
5. Descreva cada um dos conjuntos abaixo, listando seus elementos:
 a. $\{x \mid x \in \mathbb{N} \text{ e } x^2 - 5x + 6 = 0\}$
 b. $\{x \mid x \in \mathbb{R} \text{ e } x^2 = 7\}$
 c. $\{x \mid x \in \mathbb{N} \text{ e } x^2 - 2x - 8 = 0\}$
6. Descreva cada um dos conjuntos abaixo, apresentando uma propriedade característica:
 a. $\{1, 2, 3, 4, 5\}$
 b. $\{1, 3, 5, 6, 9, 11, \dots\}$
 c. $\{\text{Sarney, Collor, Itamar}\}$
 d. $\{0, 1, 10, 11, 100, 101, 110, 111, 1000, \dots\}$
7. Descreva cada um dos conjuntos a seguir:
 a. $\{x \mid x \in \mathbb{N} \text{ e } (\exists q)(q \in \{2, 3\} \text{ e } x = 2q)\}$
 b. $\{x \mid x \in \mathbb{N} \text{ e } (\exists y)(\exists z)(y \in \{0, 1\} \text{ e } z \in \{3, 4\} \text{ e } y < x < z)\}$
 c. $\{x \mid x \in \mathbb{N} \text{ e } (\forall y)(y \text{ par} \rightarrow x \neq y)\}$
- ★8. Dada a descrição de um conjunto A como $A = \{2, 4, 8, \dots\}$, você acha que $16 \in A$?
9. Sejam
 $A = \{x \mid x \in \mathbb{N} \text{ e } 1 < x < 50\}$
 $B = \{x \mid x \in \mathbb{R} \text{ e } 1 < x < 50\}$
 $C = \{x \mid x \in \mathbb{Z} \text{ e } |x| \geq 25\}$
 Quais das sentenças a seguir são verdadeiras?
 a. $A \subseteq B$ b. $17 \in A$ c. $A \subseteq C$
 d. $-40 \in C$ e. $\sqrt{3} \in B$ f. $\{0, 1, 2\} \subseteq A$
 g. $\emptyset \in B$ h. $\{x \mid x \in \mathbb{Z} \text{ e } x^2 > 625\} \subseteq C$
10. Sejam
 $R = \{1, 3, \pi, 4.1, 9, 10\}$ $S = \{\{1\}, 3, 9, 10\}$
 $T = \{1, 3, \pi\}$ $U = \{\{1, 3, \pi\}, 1\}$

Quais das sentenças a seguir são verdadeiras? Justifique as que não forem.

- | | | |
|---------------------|----------------------------------|----------------------------|
| ★a. $S \subseteq R$ | ★b. $1 \in R$ | ★c. $1 \in S$ |
| ★d. $1 \subseteq U$ | ★e. $\{1\} \subseteq T$ | ★f. $\{1\} \subseteq S$ |
| g. $T \subset R$ | h. $\{1\} \in S$ | i. $\emptyset \subseteq S$ |
| j. $T \subseteq U$ | k. $T \in U$ | l. $T \notin R$ |
| m. $T \subseteq R$ | n. $S \subseteq \{1, 3, 9, 10\}$ | |

11. Seja

$$A = \{x \mid x \in \mathbb{R} \text{ e } x^2 - 4x + 3 < 0\}$$

e

$$B = \{x \mid x \in \mathbb{R} \text{ e } 0 < x < 6\}$$

prove que $A \subset B$.

★12. Seja

$$A = \{(x, y) \mid (x, y) \text{ está a três unidades do ponto } (1, 4)\}$$

e

$$B = \{(x, y) \mid (x - 1)^2 + (y - 4)^2 \leq 25\}$$

Prove que $A \subset B$.

13. O programa QUAD encontra e imprime soluções de equações quadráticas da forma $ax^2 + bx + c = 0$. O programa PAR lista todos os inteiros de $-2n$ a $2n$. Seja Q o conjunto dos valores de saída de Q , e E o conjunto dos valores de saída de PAR.

a. Mostre que para $a = 1$, $b = -2$, $c = -24$ e $n = 50$, $Q \subseteq E$.

b. Mostre que para os mesmos valores de a , b e c , mas para n valendo 2, $Q \not\subseteq E$.

14. Seja $A = \{x \mid \cos(x/2) = 0\}$ e $B = \{x \mid \sin x = 0\}$. Prove que $A \subseteq B$.

15. Quais das sentenças a seguir são verdadeiras para todos os conjuntos A , B e C ?

- | | |
|--|--|
| ★a. Se $A \subseteq B$ e $B \subseteq A$, então $A = B$. | ★b. $\{\emptyset\} = \emptyset$ |
| ★c. $\{\emptyset\} = \{0\}$ | ★d. $\emptyset \in \{\emptyset\}$ |
| ★e. $\emptyset \subseteq A$ | ★f. $\emptyset \in A$ |
| g. $\{\emptyset\} = \{\{\emptyset\}\}$ | h. Se $A \subset B$ e $B \subseteq C$, então $A \subset C$ |
| i. Se $A \neq B$ e $B \neq C$ então $A \neq C$. | j. Se $A \in B$ e $B \not\subseteq C$, então $A \notin C$. |

16. Prove que se $A \subseteq B$ e $B \subseteq C$, então $A \subseteq C$.

17. Prove que se $A' \subseteq B'$ então $B \subseteq A$.

18. Encontre $\mathcal{P}(S)$ para $S = \{a, b\}$.

★19. Encontre $\mathcal{P}(S)$ para $S = \{1, 2, 3, 4\}$. Quantos elementos você espera que este conjunto tenha?

20. Encontre $\mathcal{P}(S)$ para $S = \{\emptyset\}$.

21. Encontre $\mathcal{P}(S)$ para $S = \{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}$.

22. Encontre $\mathcal{P}(\mathcal{P}(S))$ para $S = \{a, b\}$.

★23. O que pode ser dito sobre A se $\mathcal{P}(A) = \{\emptyset, \{x\}, \{y\}, \{x, y\}\}$?

24. O que pode ser dito sobre A se $\mathcal{P}(A) = \{\emptyset, \{a\}, \{\{a\}\}\}$?

25. Prove que se $A \subseteq B$, então $\mathcal{P}(A) \subseteq \mathcal{P}(B)$.

★26. Prove que se $\mathcal{P}(A) = \mathcal{P}(B)$, então $A = B$.

27. Resolva para x e y .

a. $(y, x + 2) = (5, 3)$

b. $(2x, y) = (16, 7)$

c. $(2x - y, x + y) = (-2, 5)$

28. a. Lembre-se de que pares ordenados precisam ter a propriedade $(x, y) = (u, v)$ se, e somente se, $x = u$ e $y = v$. Prove que $\{\{x\}, \{x, y\}\} = \{\{u\}, \{u, v\}\}$ se, e somente se, $x = u$ e $y = v$. Portanto, apesar de sabermos que $(x, y) \neq \{x, y\}$, podemos definir o par ordenado (x, y) como o conjunto $\{\{x\}, \{x, y\}\}$.

b. Mostre através de um exemplo que não podemos definir a tripla ordenada (x, y, z) como o conjunto $\{\{x\}, \{x, y\}, \{x, y, z\}\}$.

29. Quais das operações a seguir são binárias ou unárias nos conjuntos dados? Para as que não foram, justifique sua resposta.

★a. $x \circ y = x + 1$; $S = \mathbb{N}$

★b. $x \circ y = x + y - 1$; $S = \mathbb{N}$

★c. $x \circ y = \begin{cases} x-1 & \text{se } x \text{ é ímpar} \\ x & \text{se } x \text{ é par} \end{cases} \quad S = \mathbb{Z}$

d. $x^* = \ln x$; $S = \mathbb{R}$

e. $x^* = x^2$; $S = \mathbb{Z}$

f. $\circ$

	1	2	3
1	1	2	3
2	2	3	4
3	3	4	5

 $S = \{1, 2, 3\}$

1

2

3

g. $x \circ y =$ a fração, x ou y com o menor denominador; $S =$ conjunto de todas as frações

h. $x \circ y =$ a pessoa, x ou y , cujo nome aparece primeiro em ordem alfabética; $S =$ conjunto de 10 pessoas com nomes diferentes

i. $x \circ y = \begin{cases} 1/x & \text{se } x \text{ é positivo} \\ 1/(-x) & \text{se } x \text{ é negativo} \end{cases} \quad S = \mathbb{R}$

j. $x \circ y = xy$ (concatenação); $S =$ conjunto de todas as cadeias de símbolos do conjunto $\{p, q, r\}$ de tamanho finito.

30. Quantas operações binárias diferentes podem ser definidas em um conjunto com n elementos? (Dica: imagine o preenchimento de uma tabela.)

31. Nós temos escrito as operações binárias através da notação **infixa**, onde os símbolos da operação aparecem entre os operandos, como em $A + B$. A avaliação de uma operação aritmética complexa é mais eficiente se a operação for escrita na notação **posfixa**, onde o símbolo da operação aparece após os operandos, como em $A B +$. Muitos compiladores mudam as expressões dos programas da forma infix para a forma posfixa. Uma forma de produzir uma expressão posfixa a partir de uma expressão infix é escrever a operação infix com todos os parênteses possíveis e mover cada operador a fim de que substitua seu "fecha-parênteses" e eliminar todos os "abre-parênteses". (Não há necessidade de parênteses na notação posfixa.) Portanto,

$$A * B + C$$

fica, quando escrita com todos os parênteses,

$$((A * B) + C)$$

e sua notação posfixa é

$$A B * C +$$

Reescreva cada uma das expressões a seguir na notação posfixa:

a. $(A + B) * (C - D)$

b. $A ** B - C * D$ (** denota exponenciação)

c. $A * C + B / (C + D * B)$

- ★32. Avalie expressões posfixas a seguir (veja Exercício 31):

a. $2\ 4\ *\ 5\ +$

b. $5\ 1\ +\ 2\ /\ 1\ -$

c. $3\ 4\ +\ 5\ 1\ -\ *$

33. Sejam

$$A = \{p, q, r, s\}$$

$$B = \{r, t, v\}$$

$$C = \{p, s, t, u\}$$

subconjuntos de $S = \{p, q, r, s, t, u, v, w\}$. Encontre

$$\mathbf{a.} B \cap C$$

$$\mathbf{b.} A \cup C$$

$$\mathbf{c.} C'$$

$$\mathbf{d.} A \cap B \cap C$$

$$\mathbf{e.} B - C$$

$$\mathbf{f.} (A \cup B)'$$

$$\mathbf{g.} A \times B$$

$$\mathbf{h.} (A \cup B) \cap C'$$

34. Sejam

$$A = \{2, 4, 5, 6, 8\}$$

$$B = \{1, 4, 5, 9\}$$

$$C = \{x \mid x \in \mathbb{Z} \text{ e } 2 \leq x < 5\}$$

subconjuntos de $S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. Encontre:

$$\star \mathbf{a.} A \cup B$$

$$\star \mathbf{b.} A \cap B$$

$$\star \mathbf{c.} A \cap C$$

$$\mathbf{d.} B \cup C$$

$$\mathbf{e.} A - B$$

$$\mathbf{f.} A'$$

$$\mathbf{g.} A \cap A'$$

$$\star \mathbf{h.} (A \cap B)'$$

$$\mathbf{i.} C - B$$

$$\mathbf{j.} (C \cap B) \cup A'$$

$$\mathbf{k.} (B - A)' \cap (A - B)$$

$$\mathbf{l.} (C' \cup B)'$$

$$\mathbf{m.} B \times C$$

35.

$$A = \{x \mid x \text{ é o nome de um presidente anterior do Brasil}\}$$

$$B = \{\text{Jânio, Geisel, Figueiredo}\}$$

$$C = \{x \mid x \text{ é nome de um estado}\}$$

Encontre:

$$\mathbf{a.} A \cap B$$

$$\mathbf{b.} A \cap C$$

$$\mathbf{c.} B \cap C$$

36. Sejam

$$A = \{x \mid x \text{ é uma palavra que aparece antes de } \textit{cão} \text{ no dicionário da língua portuguesa}\}$$

$$B = \{x \mid x \text{ é uma palavra que aparece depois de } \textit{bem-te-vi} \text{ no dicionário da língua portuguesa}\}$$

$$C = \{x \mid x \text{ é uma palavra com mais de quatro letras}\}$$

Quais das sentenças a seguir são verdadeiras?

$$\mathbf{a.} B \subseteq C$$

$$\mathbf{b.} A \cup B = \{x \mid x \text{ é uma palavra que aparece no dicionário da língua portuguesa}\}$$

$$\mathbf{c.} \textit{gato} \in B \cap C'$$

$$\mathbf{d.} \textit{bambu} \in A - B$$

★37. Considere os seguintes subconjuntos de $\mathbb{Z}$:

$$A = \{x \mid (\exists y)(y \in \mathbb{Z} \text{ e } y \geq 4 \text{ e } x = 3y)\}$$

$$B = \{x \mid (\exists y)(y \in \mathbb{Z} \text{ e } x = 2y)\}$$

$$C = \{x \mid x \in \mathbb{Z} \text{ e } |x| \leq 10\}$$

Usando as operações de conjuntos, descreva cada um dos seguintes conjuntos em termos de A , B e C :

a. conjunto de todos os inteiros ímpares

$$\mathbf{b.} \{-10, -8, -6, -4, -2, 0, 2, 4, 6, 8, 10\}$$

$$\mathbf{c.} \{x \mid (\exists y)(y \in \mathbb{Z} \text{ e } y \geq 2 \text{ e } x = 6y)\}$$

$$\mathbf{d.} \{-9, -7, -5, -3, -1, 1, 3, 5, 7, 9\}$$

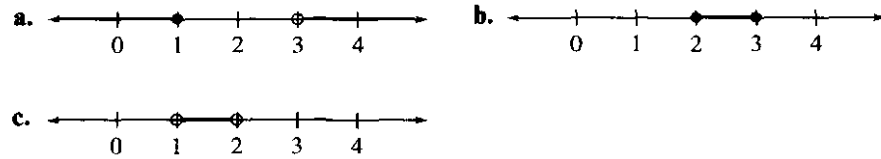
$$\mathbf{e.} \{x \mid (\exists y)(y \in \mathbb{Z} \text{ e } y \geq 5 \text{ e } x = 2y + 1)\} \cup \{x \mid (\exists y)(y \in \mathbb{Z} \text{ e } y \leq -5 \text{ e } x = 2y - 1)\}$$

38. Sejam

$$A = \{x \mid x \in \mathbb{R} \text{ e } 1 < x \leq 3\}$$

$$B = \{x \mid x \in \mathbb{R} \text{ e } 2 \leq x \leq 5\}$$

Usando operações de conjuntos, descreva cada um dos conjuntos mostrados abaixo em termos de A e B



39. Quais das sentenças a seguir são verdadeiras para quaisquer conjuntos A , B e C ?

- | | |
|--|---|
| ★a. $A \cup A = A$ | b. $B \cap B = B$ |
| ★c. $(A \cap B)' = A' \cap B'$ | d. $(A')' = A$ |
| ★e. $A - B = (B - A)'$ | f. $(A - B) \cap (B - A) = \emptyset$ |
| g. If $A \cap B = \emptyset$ então $A \subset B$. | h. $B \times A = A \times B$ |
| i. $\emptyset \times A = \emptyset$ | j. $\emptyset \cap \{\emptyset\} = \emptyset$ |
| k. $(A - B) \cup (B - C) = A - C$ | i. $(A - C) \cap (A - B) = A - (B \cup C)$ |

40. Para cada uma das sentenças a seguir, encontre as condições gerais para os conjuntos A e B para tornar a sentença verdadeira:

- | | | |
|------------------------|----------------------------------|-----------------------------------|
| ★a. $A \cup B = A$ | b. $A \cap B = A$ | c. $A \cup \emptyset = \emptyset$ |
| d. $B - A = \emptyset$ | e. $A \cup B \subseteq A \cap B$ | |

41. Prove que

$$(A \cap B) \subseteq A$$

onde A e B são conjuntos arbitrários.

42. Prove que

$$A \subseteq (A \cup B)$$

onde A e B são conjuntos arbitrários.

★43. Prove que $\mathcal{P}(A) \cap \mathcal{P}(B) = \mathcal{P}(A \cap B)$, onde A e B são conjuntos arbitrários.

44. Prove que $\mathcal{P}(A) \cup \mathcal{P}(B) \subseteq \mathcal{P}(A \cup B)$, onde A e B são conjuntos arbitrários.

45. Prove que se $A \cup B = A - B$, então $B = \emptyset$. (Dica: Faça uma demonstração por contradição.)

★46. Prove que se $(A - B) \cup (B - A) = A \cup B$, então $A \cap B = \emptyset$. (Dica: faça uma demonstração por contradição.)

47. Uma operação binária em conjuntos chamada **diferença simétrica** é definida como $A \oplus B = (A - B) \cup (B - A)$.

a. Desenhe um diagrama de Venn para ilustrar $A \oplus B$.

b. Para $A = \{3, 5, 7, 9\}$ e $B = \{2, 3, 4, 5, 6\}$, ache $A \oplus B$.

c. Prove que $A \oplus B = (A \cup B) - (A \cap B)$ para A e B arbitrários.

d. Para um conjunto A arbitrário, ache $A \oplus A$. Ache $\emptyset \oplus A$.

e. Prove que $A \oplus B = B \oplus A$ para conjuntos arbitrários A e B .

f. Para quaisquer conjuntos A , B e C demonstre que $(A \oplus B) \oplus C = A \oplus (B \oplus C)$.

48. Quais das sentenças a seguir são verdadeiras para quaisquer conjuntos A , B e C ?

- | |
|---|
| a. $A \cup (B \times C) = (A \cup C) \times (A \cup C)$ |
| b. $A \times (B \cap C) = (A \times B) \cap (A \times C)$ |
| c. $A \times \emptyset = \emptyset$ |

- d. $\mathcal{P}(A) \times \mathcal{P}(A) = \mathcal{P}(A^2)$
 e. $A \times (B \times C) = (A \times B) \times C$

49. Verifique as identidades básicas de conjuntos da p. 106 demonstrando a inclusão em ambas as direções. (Já mostramos a 3.^a e 4.^a)
50. A e B são subconjuntos de S . Demonstre as seguintes identidades mostrando a inclusão em ambas as direções:
- a. $(A \cup B)' = A' \cap B'$
 b. $(A \cap B)' = A' \cup B'$ } Leis de De Morgan
- c. $A \cup (B \cap A) = A$ d. $(A \cap B')' \cup B = A' \cup B$
 e. $(A \cap B) \cup (A \cap B') = A$ f. $[A \cap (B \cup C)]' = A' \cup (B' \cap C')$
51. A, B e C são subconjuntos de S . Demonstre as seguintes identidades usando as identidades básicas de conjuntos apresentadas nesta seção:
- ★1. $(A \cup B) \cap (A \cup B') = A$
 2. $[(A \cap C) \cap B] \cup [(A \cap C) \cap B'] \cup (A \cap C)' = S$
 3. $(A \cup C) \cap [(A \cap B) \cup (C' \cap B)] = A \cap B$
- b. Indique o dual de cada uma das identidades acima.
52. A, B e C são subconjuntos de S . Demonstre as seguintes identidades de conjuntos, quer seja mostrando a inclusão em ambas as direções ou usando as identidades previamente demonstradas, incluindo as do Exercício 50.
- ★a. $(A')' = A$
 ★b. $A \cap (B \cup A') = B \cap A$
 c. $(A \cup B) - C = (A - C) \cup (B - C)$
 d. $(A - B) - C = (A - C) - B$
 e. $[(A' \cup B') \cap A']' = A$
53. A operação de união de conjuntos pode ser definida como uma operação n -ária para qualquer inteiro $n \geq 2$.
- a. Forneça uma demonstração semelhante à da união de dois conjuntos para $A_1 \cup A_2 \cup \dots \cup A_n$.
- b. Forneça uma definição recursiva para $A_1 \cup A_2 \cup \dots \cup A_n$.
54. Usando a definição recursiva da união de conjuntos do Exercício 53(b), demonstre a propriedade associativa generalizada da união de conjuntos, que é válida para qualquer n com $n \geq 3$ e qualquer p com $1 \leq p \leq n - 1$,
- $$(A_1 \cup \dots \cup A_p) \cup (A_{p+1} \cup \dots \cup A_n) = A_1 \cup \dots \cup A_n.$$
55. A operação de interseção de conjuntos pode ser definida como uma operação n -ária para qualquer inteiro $n \geq 2$.
- a. Forneça uma definição semelhante à da interseção de dois conjuntos para $A_1 \cap A_2 \cap \dots \cap A_n$.
- b. Forneça uma definição recursiva para $A_1 \cap A_2 \cap \dots \cap A_n$.
56. Usando a definição recursiva da interseção de conjuntos do Exercício 55 (b), demonstre a propriedade associativa generalizada da interseção de conjuntos, que é válida para qualquer n com $n \geq 3$ e qualquer p com $1 \leq p \leq n - 1$,
- $$(A_1 \cap \dots \cap A_p) \cap (A_{p+1} \cap \dots \cap A_n) = A_1 \cap \dots \cap A_n.$$
57. Para $A_1, A_2, \dots, A_n$ e B subconjuntos de S , demonstre as seguintes identidades, onde $n \geq 2$. (Veja Exercícios 53 e 55.)

- ★a. $B \cup (A_1 \cap A_2 \cap \dots \cap A_n) = (B \cup A_1) \cap (B \cup A_2) \cap \dots \cap (B \cup A_n)$
 b. $B \cap (A_1 \cup A_2 \cup \dots \cup A_n) = (B \cap A_1) \cup (B \cap A_2) \cup \dots \cup (B \cap A_n)$

58. Para $A_1, A_2, \dots, A_n$ subconjuntos de S , demonstre as seguintes identidades, onde $n \geq 2$. (Veja Exercícios 50, 53 e 55.)

- a. $(A_1 \cup A_2 \cup \dots \cup A_n)' = A_1' \cap A_2' \cap \dots \cap A_n'$
 b. $(A_1 \cap A_2 \cap \dots \cap A_n)' = A_1' \cup A_2' \cup \dots \cup A_n'$

59. As operações de união e interseção de conjuntos podem ser estendidas a fim de que se apliquem a uma família infinita de conjuntos. Podemos descrever essa família como a coleção de todos os conjuntos A_i , onde i assume quaisquer valores do conjunto fixo I . Neste caso, I é chamado de **conjunto índice** da família. A união da família, $\bigcup_{i \in I} A_i$, é definida por $\bigcup_{i \in I} A_i = \{x \mid x \text{ é elemento de algum } A_i\}$. A interseção da família, $\bigcap_{i \in I} A_i$, é definida por $\bigcap_{i \in I} A_i = \{x \mid x \text{ é elemento de todos os } A_i\}$.
 a. Seja $I = \{1, 2, 3, \dots\}$ e para cada $i \in I$, seja A_i o conjunto dos números reais no intervalo $(-1/i, 1/i)$. Encontre $\bigcup_{i \in I} A_i$ e $\bigcap_{i \in I} A_i$?
 b. Seja $I = \{1, 2, 3, \dots\}$ e para $i \in I$, seja A_i o conjunto dos números reais no intervalo $[-1/i, 1/i]$. Encontre $\bigcup_{i \in I} A_i$ e $\bigcap_{i \in I} A_i$?

60. De acordo com nosso uso da palavra *conjunto*, se A é um subconjunto do conjunto universo S , então qualquer elemento de S ou pertence ou não pertence a A . Em outras palavras, a probabilidade de um elemento x de S pertencer a A é 1 (quando x é um elemento de A) ou 0 (quando x não é um elemento de A). A é um **conjunto fuzzy** se todo $x \in S$ tem a probabilidade p , $0 \leq p \leq 1$, de ser um elemento de A . A probabilidade p associada a x é uma estimativa da possibilidade de que x possa pertencer a A quando a composição de A é desconhecida. Operações de conjunto podem ser realizadas em conjuntos fuzzy da seguinte maneira: Se o elemento x tem a probabilidade p_1 de pertencer a A e a probabilidade p_2 de pertencer a B , então a probabilidade de x ser um elemento de $A \cup B$, $A \cap B$ e A' é, respectivamente, $p_1 + p_2 - p_1 \cdot p_2$, $p_1 \cdot p_2$ e $1 - p_1$.

Seja S um conjunto de possíveis agentes causadores de doenças, $S = \{\text{genética, vírus, nutrição, bactéria, ambiente}\}$. Os conjuntos fuzzy AIDS e mal de ALZHEIMERS são definidos como AIDS = {genética, .2; vírus, .8; nutrição, .1; bactéria, .4; ambiente, .3} e ALZHEIMERS = {genética, .7; vírus, .4; nutrição, .3; bactéria, .3; ambiente .4}.

- a. Encontre o conjunto fuzzy AIDS $\cup$ ALZHEIMERS.
 b. Encontre o conjunto fuzzy AIDS $\cap$ ALZHEIMERS.
 c. Encontre o conjunto fuzzy (AIDS)'.

61. O princípio da boa-ordenação diz que todo conjunto não-vazio de inteiros positivos tem um menor número. Prove que o princípio da indução fraca, isto é,

$$\left. \begin{array}{l} 1. P(1) \text{ verdadeira} \\ 2. (\forall k)[P(k) \text{ verdadeira} \rightarrow P(k+1) \text{ verdadeira}] \end{array} \right\} \rightarrow \text{verdadeira para todo inteiro positivo } P(n)$$

implica o princípio da boa-ordenação. (Dica: admita que o princípio da indução fraca é válido e use uma prova por contradição para mostrar que o princípio da boa-ordenação é válido. Seja T um conjunto não-vazio de inteiros positivos que não tem menor elemento. Seja $P(n)$ a propriedade que todo elemento de T é maior que n .)

★62. Demonstre que o princípio da boa-ordenação (veja Exercício 61) implica o princípio da indução forte. (Dica: Admita que o princípio da boa-ordenação é válido e seja P a propriedade para a qual

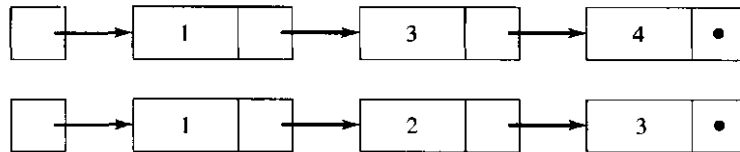
- 1'. $P(1)$ é verdadeira
 2'. $(\forall k)[P(r) \text{ verdadeira para todo } r, 1 \leq r \leq k \rightarrow P(k+1) \text{ verdadeira}]$

Seja T o subconjunto dos inteiros positivos definido por

$$T = \{t \mid P(t) \text{ não é verdadeiro}\}$$

Mostre que T é o conjunto vazio.)

63. Seja o conjunto universo $S = \{1, 2, 3, 4, 5\}$. Usando a representação por vetor de bits, quais são os seguintes conjuntos?
- $A = \{2, 3, 5\}$
 - $B = \{5\}$
 - $\emptyset$
64. Seja o conjunto universo $S = \{1, 2, 3, 4, 5\}$. Usando a representação por vetor de bits, encontre
- O complemento de 10011
 - A união de 11001 e 01011
 - A interseção de 00111 e 10110
- ★65. Sejam os conjuntos A e B com as representações na forma de listas encadeadas mostradas a seguir:



Desenhe a representação na forma de lista encadeada de $A \cup B$.

66. Usando os conjuntos do Exercício 65, desenhe a representação em forma de lista encadeada de $A \cap B$.
- ★67. Prove que o conjunto dos inteiros positivos ímpares é enumerável.
68. Prove que o conjunto $\mathbb{Z}$ de todos os inteiros é enumerável.
69. Prove que o conjunto de todas as strings de tamanho finito de letras a é enumerável.
70. Prove que o conjunto $\mathbb{Z} \times \mathbb{Z}$ é enumerável.
71. Use o método da diagonalização de Cantor para mostrar que o conjunto de todas as seqüências infinitas de inteiros positivos não é contável.
72. Explique por que qualquer subconjunto de um conjunto contável é também contável.
- ★73. Explique por que a união de quaisquer dois conjuntos enumeráveis é também enumerável.
74. Conjuntos podem ter conjuntos como elementos (veja Exercício 10, por exemplo). Seja B o conjunto definido como mostrado a seguir:

$$B = \{S \mid S \text{ é um conjunto e } S \notin S\}$$

Verifique que tanto $B \in B$ quanto $B \notin B$ são verdadeiros. Esta contradição é chamada de **paradoxo de Russell** devido ao famoso matemático e filósofo Bertrand Russell, que primeiro a enunciou em 1901. (A criação cuidadosa de axiomas na teoria dos conjuntos impôs algumas restrições sobre o que se pode chamar de conjunto. Todos os conjuntos ordinários continuam sendo conjuntos, mas conjuntos peculiares que nos induzem a dúvidas, como o B acima, foram evitados.)

Seção 3.2 Contagem

A **combinatória** é o ramo da Matemática que trata da contagem. Tratar a contagem é importante, sempre que temos recursos finitos (Quanto espaço um banco de dados consome? Quantos usuários a configuração de um computador pode suportar?) ou sempre que estamos interessados em eficiência (Quantos cálculos um determinado algoritmo envolve?).

Problemas de contagem normalmente se resumem em determinar quantos elementos existem em um conjunto finito. Esta questão que parece trivial pode ser difícil de ser respondida. Já respondemos algumas questões do tipo "quantos" — quantas linhas existem na tabela-verdade com n símbolos proposicionais, e quantos subconjuntos existem em um conjunto com n elementos? (Na verdade, como já vimos, essas podem ser a mesma pergunta.)

O Princípio da Multiplicação

Resolvemos o problema da tabela-verdade, desenhando uma árvore de possibilidades. Essa árvore sugere um princípio mais geral que pode ser usado para resolver diversos problemas de contagem. Antes de enunciarmos o princípio geral, daremos uma olhada em outro exemplo de árvore.

EXEMPLO 28

A uma criança é permitido escolher um dentre dois confeitos, um vermelho e outro preto, e um entre três chicletes, amarelo, lilás e branco. Quantos conjuntos diferentes de doces a criança pode ter?

Podemos resolver este problema, quebrando a tarefa da escolha dos doces em duas etapas sequenciais: a escolha do confeito e a escolha do chiclete. A árvore da Fig. 3.3 mostra que existem $2 \times 3 = 6$ escolhas possíveis. São elas: $\{V, A\}$, $\{V, L\}$, $\{V, B\}$, $\{P, A\}$, $\{P, L\}$ e $\{P, B\}$.

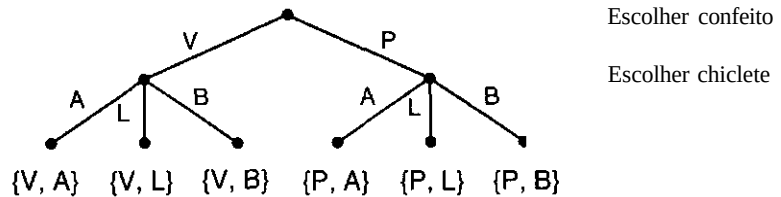


Figura 3.3

Neste problema, a sequência de eventos pode ser invertida; a criança pode escolher o chiclete antes de escolher o confeito, resultando na árvore da Fig. 3.4, mas o número de escolhas possíveis permanece o mesmo ($2 \times 3 = 6$). Pensar como uma sequência de eventos sucessivos nos ajuda a resolver o problema, mas o seqüenciamento não é uma parte do problema, pois o conjunto $\{V, A\}$ é o mesmo que o $\{A, V\}$.

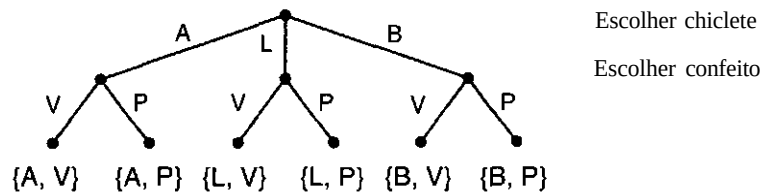


Figura 3.4

O Exemplo 28 mostra que o número de possibilidades para eventos seqüenciados pode ser obtido por meio da multiplicação dos números de possibilidades do primeiro evento pelo número de possibilidades do segundo. Esta idéia é sintetizada no *Princípio da Multiplicação*.

Princípio da Multiplicação

Se existem n_1 possibilidades para um primeiro evento e n_2 possibilidades para um segundo evento, então existem $n_1 \cdot n_2$ possibilidades para a sequência dos dois eventos.

O Princípio da Multiplicação pode ser estendido por indução a fim de se aplicar a sequência de qualquer número finito de eventos. (Veja o Exercício 53 ao fim desta seção.) O Princípio da Multiplicação é útil, quando desejamos contar o número de possibilidades totais de uma tarefa que pode ser quebrada em uma sequência de etapas sucessivas.

EXEMPLO 29

A última parte do número de seu telefone contém quatro dígitos. Quantos números de quatro dígitos existem?

Podemos imaginar um número de quatro dígitos como o total de possibilidades de uma sequência de etapas de escolha do primeiro dígito, depois do segundo, depois do terceiro e, finalmente, do quarto dígito. O primeiro dígito pode ser qualquer dos 10 dígitos entre 0 e 9, portanto há 10 possibilidades para a primeira etapa. Da mesma forma, há 10 possibilidades para as etapas de escolha dos segundo, terceiro e quarto dígitos. Usando o Princípio da Multiplicação, multiplicamos o número de possibilidades de cada etapa da sequência. Portanto, há $10 \cdot 10 \cdot 10 \cdot 10 = 10.000$ números diferentes.

O número de resultados possíveis para eventos sucessivos após o primeiro evento é afetado se o mesmo elemento não puder ser usado novamente, isto é, se não forem permitidas repetições.

EXEMPLO 30 Com relação ao Exemplo 29, quantos números de quatro dígitos sem repetições de dígitos existem?
Novamente, temos a sequência de etapas de seleção dos quatro dígitos, mas não são permitidas repetições. Existem 10 possibilidades para a escolha do primeiro dígito, mas apenas nove para a escolha do segundo, pois não podemos usar o que já foi usado para o primeiro dígito, e assim por diante. Existem, portanto, $10 \cdot 9 \cdot 8 \cdot 7 = 5040$ números diferentes sem repetições de dígitos. •

EXEMPLO 31 a. De quantas maneiras podemos escolher três funcionários de um grupo de 25 pessoas?
b. De quantas maneiras podemos escolher três funcionários de um grupo de 25 pessoas, se uma pessoa puder acumular mais de um cargo?
Em (a) existem três etapas sucessivas sem repetições. A primeira etapa, escolher o primeiro funcionário, tem 25 resultados possíveis. A segunda etapa tem 24 possibilidades, e a terceira 23. O número total de resultados possíveis é $25 \cdot 24 \cdot 23 = 13.800$. Em (b), as mesmas três etapas são realizadas em sequência, mas são permitidas repetições. O número total de repetições é $25 \cdot 25 \cdot 25 = 15.625$. •

PRÁTICA 24 Se um homem tem quatro ternos, oito camisas e cinco gravatas, quantas combinações ele pode compor? •

EXEMPLO 32 Para qualquer conjunto finito S , seja $|S|$ o número de elementos em S . Se A e B são conjuntos finitos, então

$$|A \times B| = |A| \cdot |B|$$

$A \times B$ consiste em todos os pares ordenados com a primeira componente em A e a segunda componente em B . A escolha desses pares ordenados é equivalente a escolher, em sequência, a primeira componente dentre as $|A|$ possibilidades, e então escolher a *segunda*, para a qual existem $|B|$ possibilidades. O resultado segue, então, o Princípio da Multiplicação. •

O Princípio da Adição

Suponha que desejamos escolher uma sobremesa dentre três tortas e quatro bolos. De quantas formas isto pode ser feito? Existem dois eventos, um com três resultados possíveis (escolher uma torta) e outro com quatro resultados possíveis (escolher um bolo). No entanto, não estamos compondo uma sequência de dois eventos, uma vez que desejamos apenas uma sobremesa, que precisa ser escolhida dentre as possibilidades de dois conjuntos disjuntos. O número de possibilidades é o número total de opções que temos, $3 + 4 = 7$. Isto ilustra o *Princípio da Adição*.

Princípio da Adição

Se A e B são eventos disjuntos com n_1 e n_2 possibilidades, respectivamente, **então o número total de possibilidades para o evento A ou B é $n_1 + n_2$.**

O Princípio da Adição pode ser estendido por indução para o caso de qualquer número finito de eventos disjuntos. (Veja o Exercício 54, ao fim desta seção.) O Princípio da Adição é útil sempre que desejamos contar o número total de resultados possíveis para uma tarefa que pode ser quebrada em dois casos disjuntos.

EXEMPLO 33 Um comprador deseja comprar um veículo de uma concessionária. A concessionária tem 23 carros e 14 caminhões em estoque. Quantas possíveis escolhas o comprador pode ter?
O comprador deseja escolher um carro ou caminhão. São eventos disjuntos; escolher um carro tem 23 possibilidades e escolher um caminhão tem 14. Pelo Princípio da Adição, a escolha de um veículo tem $23 + 14 = 37$ possibilidades. Perceba que os requisitos para os eventos A e B são conjuntos disjuntos. Portanto, se um comprador desejar comprar um veículo de uma concessionária que tenha 23 carros, 14 caminhões e 17 veículos vermelhos, não podemos dizer que o comprador tem $23 + 14 + 17$ possibilidades de escolha. •

EXEMPLO 34 Sejam A e B conjuntos finitos disjuntos. Então $|A \cup B| = |A| + |B|$.
A união $|A \cup B|$ pode ser encontrada, no caso de conjuntos disjuntos, pela contagem do número de elementos em A , $|A|$, e do número de elementos em B , $|B|$. Pelo Princípio da Adição, somamos esses dois números. •

EXEMPLO 35 Se A e B são conjuntos finitos, então

$$|A - B| = |A| - |A \cap B|$$

e

$$|A - B| = |A| - |B| \text{ se } B \subseteq A$$

Para provar a primeira igualdade, perceba que

$$\begin{aligned} (A - B) \cup (A \cap B) &= (A \cap B') \cup (A \cap B) \\ &= A \cap (B' \cup B) \\ &= A \cap S \\ &= A \end{aligned}$$

de forma que $A = (A - B) \cup (A \cap B)$. Além disso, $A - B$ e $A \cap B$ são conjuntos disjuntos; portanto, pelo Exemplo 34,

$$|A| = |(A - B) \cup (A \cap B)| = |A - B| + |A \cap B|$$

ou

$$|A - B| = |A| - |A \cap B|$$

A segunda equação é obtida a partir da primeira, aplicando-se que se $B \subseteq A$, então, $A \cap B = B$. •

Freqüentemente, o Princípio da Adição é usado em conjunto com o Princípio da Multiplicação.

EXEMPLO 36 Com referência ao Exemplo 28, suponha que desejamos achar quantas formas diferentes existem para uma criança *escolher* um doce, ao contrário do número de conjuntos de doces que uma criança pode ter. Desta forma, escolher um confeito amarelo, seguido de um chiclete amarelo é diferente de escolher um chiclete amarelo diferente de um confeito amarelo. Podemos considerar dois casos disjuntos — escolher confeitos antes de escolher chicletes ou escolher chicletes antes de escolher confeitos. Cada um desses casos (pelo Princípio da Multiplicação) tem seis possibilidades, portanto (pelo Princípio da Adição) existem $6 + 6 = 12$ formas possíveis de escolher o doce. •

EXEMPLO 37 Quantos números de quatro dígitos começam com 4 ou 5?

Podemos considerar dois casos disjuntos — números que começam por 4 e números que começam por 5. Para a contagem dos números que começam por 4, existe uma forma de escolher o primeiro dígito, e 10 possibilidades para as etapas de escolha de cada um dos outros dígitos. Portanto, pelo Princípio da Multiplicação, existem $1 \cdot 10 \cdot 10 \cdot 10 = 1000$ formas de escolher um número de quatro dígitos começando com 4. O mesmo raciocínio mostra que existem 1000 formas de escolher um número de quatro dígitos começando por 5. Pelo Princípio da Adição, existem $1000 + 1000 = 2000$ resultados possíveis ao todo. •

PRÁTICA 25 Se uma mulher tem sete blusas, cinco saias e nove vestidos, com quantas combinações diferentes ela pode se vestir?

Normalmente, problemas de contagem podem ser resolvidos de mais de uma forma. Apesar da possibilidade de uma segunda solução poder parecer confusa, ela fornece um modo de verificar nosso resultado — se duas abordagens diferentes do mesmo problema produzem o mesmo resultado, isto aumenta a credibilidade de que analisamos o problema corretamente.

EXEMPLO 38 Considere o problema do Exemplo 37 novamente. Podemos evitar usar o Princípio da Adição, pensando sobre o problema em etapas sucessivas, onde a primeira etapa será escolher o primeiro dígito, que tem duas possibilidades de escolha — escolher 4 ou 5. Desta forma, existem $2 \cdot 10 \cdot 10 \cdot 10 = 2000$ possibilidades de escolha. •

EXEMPLO 39 Quantos inteiros de três dígitos (números entre 100 e 999) são pares?

Uma solução faz uso do fato de que números inteiros terminam em 0, 2, 4, 6 ou 8. Considerando esses casos separadamente, o número de inteiros de três dígitos terminando em 0 pode ser encontrado escolhendo seus dígitos em etapas. Existem nove possibilidades, de 1 a 9, para o primeiro dígito; 10 possibilidades, 0 a 9, para o segundo dígito; e uma possibilidade para o terceiro dígito, 0. Pelo Princípio da Multiplicação, existem

90 números de três dígitos terminando em 0. Analogamente, existem 90 números terminando por 2, 4, 6 e 8. Portanto, pelo Princípio da Adição, existem $90 + 90 + 90 + 90 + 90 = 450$ números pares de três dígitos.

Outra solução tira vantagem do fato de que existem apenas cinco escolhas para o terceiro dígito. Pelo Princípio da Multiplicação, existem $9 \cdot 10 \cdot 5 = 450$ números.

Para este problema, existe ainda uma terceira solução. Existem $999 - 100 + 1 = 900$ inteiros de três dígitos. Metade dos quais é ímpar, portanto 450 devem ser pares.

EXEMPLO 40

Suponha que os quatro últimos dígitos de um número de telefone precisam incluir, pelo menos, um dígito repetido. Quantos números deste tipo existem?

Apesar de ser possível resolver este problema com um uso direto do Princípio da Adição, isto é difícil, porque existem diversos casos disjuntos a considerar. Por exemplo, se os primeiros dois dígitos forem iguais, mas os terceiro e quarto forem diferentes, existem $10 \cdot 1 \cdot 9 \cdot 8$ maneiras de escolher o número. Se o primeiro e terceiro dígitos forem iguais, existem $10 \cdot 9 \cdot 1 \cdot 8$ maneiras de escolher o número. Se os dois primeiros forem iguais e os dois últimos também forem iguais, mas diferentes dos dois primeiros, existirão $10 \cdot 1 \cdot 9 \cdot 1$ números. Obviamente, existem diversas outras possibilidades.

Ao contrário da abordagem acima, resolveremos este problema usando o fato de que números com repetições e números sem repetições são conjuntos disjuntos cuja união é o conjunto de todos os números de quatro dígitos. Pelo Exemplo 34, podemos achar quantos números com repetições existem através da subtração da quantidade de números sem repetições (5040, segundo o Exemplo 30) do total de números de quatro dígitos (10.000, de acordo com o Exemplo 29). Portanto, existem 4960 números com repetições.

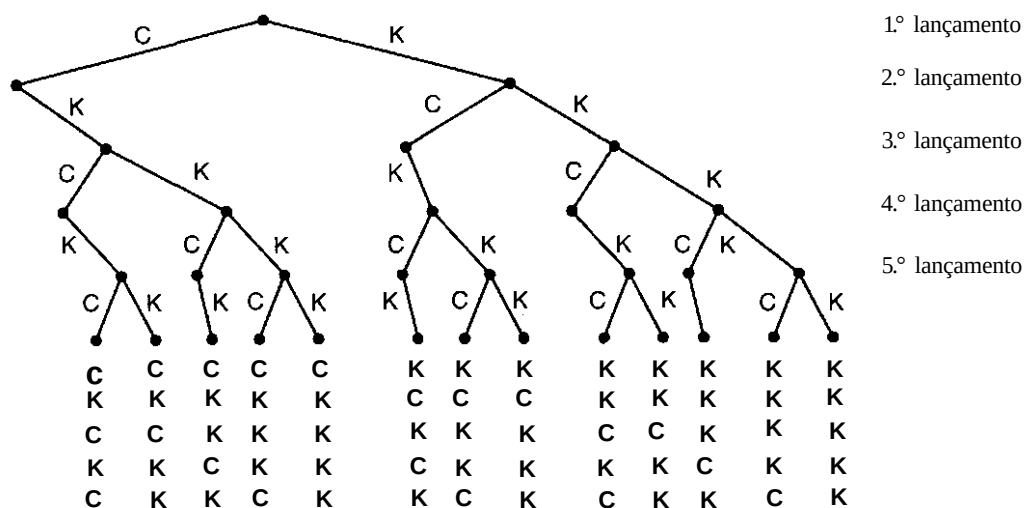
Árvores de Decisão

Árvores como as mostradas nas Figs. 3.3 e 3.4 ilustram o número de possibilidades de um evento baseado em uma série de opções possíveis. Tais árvores são chamadas **árvores de decisão**. Veremos no Cap. 5 como elas são usadas na análise de algoritmos, mas, por enquanto, as usaremos para resolver problemas de contagem adicionais. As árvores das Figs. 3.3 e 3.4 nos levam ao Princípio da Multiplicação, porque o número de possibilidades em cada nível sucessivo da árvore é constante. Na Fig. 3.4, por exemplo, o nível 2 da árvore mostra duas possibilidades para cada um dos três ramos derivados do nível 1. Árvores de decisão menos regulares podem ainda ser usadas para resolver problemas de contagem onde o Princípio da Multiplicação não se aplica.

EXEMPLO 41

Tony está jogando "cara-ou-coroa". Cada lançamento resulta em cara (C) ou coroa (K). De quantas formas ele pode lançar a moeda cinco vezes sem obter duas caras consecutivas?

A Fig. 3.5 mostra a árvore de decisão para este problema. Cada lançamento de moeda tem duas possibilidades: o ramo à esquerda está marcado com um C para cara, e o ramo da direita com um K para coroa. Sempre que um C aparecer em um ramo, o próximo nível pode conter apenas um ramo para a direita (K). Existem 13 possibilidades.

**Figura 3.5**

- PRÁTICA 26** Explique por que o Princípio da Multiplicação não se aplica ao Exemplo 41. •
- PRÁTICA 27** Desenhe a árvore de decisões para o número de cadeias de caracteres com X 's, Y 's e Z 's com tamanho 3 que não contenham um Z seguindo um Y . •

Revisão da Seção 3.2

Técnica

- Uso do Princípio da Multiplicação e o Princípio da Adição e árvores de decisão para a contagem do número de objetos em um conjunto finito.

Idéias Principais

O Princípio da Multiplicação é usado para contar o número de resultados possíveis para uma sequência de eventos, cada qual com um número fixo de possibilidades.

O Princípio da Adição é usado para contar o número de resultados possíveis para eventos disjuntos.

Os Princípio da Multiplicação e da Adição são normalmente usados juntos.

Árvores de decisão podem ser usadas para contar o número de resultados possíveis para uma sequência de eventos onde o número de possibilidades para um dado evento não é constante e depende do resultado do evento anterior.

Exercícios 3.2

- ★1. Uma loja de iogurte congelado permite escolher um sabor (baunilha, morango, limão, cereja ou pêssego), um acompanhamento (raspas de chocolate, jujuba ou castanha de caju) e uma calda (creme batido ou coco ralado). Quantas sobremesas diferentes são possíveis?
- ★2. No Exercício 1, por quantas escolhas de sobremesa podemos optar, se formos alérgicos a chocolate e a morangos?
3. Um jogo de computador é iniciado fazendo-se seleções em cada um dos três menus. O primeiro menu (número de jogadores) tem quatro opções, o segundo menu (nível de dificuldade do jogo) tem oito, e o terceiro menu (velocidade) tem seis. Com quantas configurações o jogo pode ser jogado?
4. Uma prova de múltipla-escolha tem 20 perguntas, cada qual com quatro respostas possíveis, e 10 perguntas adicionais, cada uma com cinco respostas possíveis. Quantas folhas de respostas diferentes são possíveis?
5. Uma senha de usuário em um computador de grande porte consiste em três letras seguidas de dois dígitos. Quantas senhas diferentes são possíveis (considere o alfabeto com 26 letras)?
6. No computador do Exercício 5, quantas senhas serão possíveis se diferenciarmos as letras maiúsculas das minúsculas?
- ★7. Uma conferência telefônica está tendo lugar do centro do Rio de Janeiro até Curitiba, via São Paulo. Existem 45 troncos telefônicos entre o Rio de Janeiro e São Paulo e 13 de São Paulo a Curitiba. Quantas rotas diferentes podem estar sendo usadas?
8. A, B, C e D são nodos (nós) de uma rede de computadores. Existem dois caminhos entre A e C, dois entre B e D, três entre A e B e quatro entre C e D. Por quantos caminhos uma mensagem de A para D pode ser enviada?
9. Quantos números de CPF são possíveis?
- ★'10. Um prédio de apartamentos comprou um novo sistema de fechaduras para seus 175 apartamentos. Essas fechaduras são abertas através de um código numérico de dois dígitos. Será que o síndico fez uma compra inteligente?
- ★11. Um palíndromo é uma cadeia de caracteres que é igual quando lido normalmente ou de trás para frente. Quantas palíndromes de cinco letras são possíveis na língua portuguesa?

12. Quantos números de três dígitos menores que 600 podem ser construídos usando os dígitos 8, 6, 4 e 2?
13. Um conectivo lógico binário pode ser definido fornecendo sua tabela-verdade. Quantos conectivos lógicos binários existem?
- ★14. Um identificador em BASIC precisa ser ou uma letra simples ou uma letra seguida de outra letra ou dígito. Quantos identificadores são possíveis de serem formados?
15. Três cadeiras da câmara dos deputados serão preenchidas, cada qual com alguém de um partido diferente. Existem quatro candidatos concorrendo pelo Partido do Movimento Democrático Brasileiro, três do Partido dos Trabalhadores e dois do Partido Social Democrático. De quantas formas as cadeiras podem ser preenchidas?
16. Um presidente e um vice-presidente precisam ser escolhidos de um comitê de uma organização. Existem 17 voluntários da Divisão Leste e 24 voluntários da Divisão Oeste. Se ambos os funcionários precisarem vir da mesma divisão, de quantas maneiras os funcionários podem ser selecionados?
- ★17. Em um jantar especial, existem cinco aperitivos para serem escolhidos, três saladas, quatro entradas e três bebidas. Quantos jantares diferentes são possíveis?
18. No Exercício 17, quantos jantares diferentes são possíveis, se pudermos ter um aperitivo ou uma salada, mas não ambos?
19. Um novo carro pode ser encomendado com a escolha dentre 10 cores exteriores; sete cores interiores; transmissão automática, câmbio com três marchas ou câmbio com cinco marchas; com ou sem ar condicionado; com ou sem piloto-automático; e com ou sem o pacote opcional que contém as travas elétricas das portas e o desembaçador do vidro traseiro. Quantos carros diferentes podem ser encomendados?
20. No Exercício 19, quantos carros diferentes podem ser encomendados se o pacote opcional só for possível para carros com transmissão automática?
- ★21. Em um estado, as placas dos carros precisam ter dois dígitos (sem zeros à esquerda), seguidos de uma letra mais uma cadeia de dois ou quatro dígitos (podendo conter zeros à esquerda). Quantas placas diferentes são possíveis?
22. Um freguês de uma lanchonete pode pedir um hambúrguer com ou sem mostarda, ketchup, pickles ou cebola; um sanduíche de peixe com ou sem alface, tomate ou molho tártaro; e escolher entre três tipos de bebidas fracas ou dois tipos de milk shakes. Quantos pedidos diferentes são possíveis para fregueses que possam pedir no máximo um hambúrguer, um sanduíche de peixe e uma bebida, mas podem pedir menos também?
- ★23. Qual o valor de *Contador* após a execução do seguinte trecho de programa?

```

Contador := 0;
for i := 1 to 5 do
  for Letra := 'A' to 'C' do
    Contador := Contador + 1;

```

24. Qual o valor da variável *Resultado* após a execução do seguinte trecho de programa?

```

Resultado := 0;
for índice := 20 downto 10 do
  for Interno := 5 to 10 do
    Resultado := Resultado + 2;

```

Os Exercícios de 25 a 30 referem-se ao conjunto dos inteiros de três dígitos (números entre 100 e 999, inclusive).

25. Quantos são divisíveis por 5?
- ★26. Quantos não são divisíveis por 5?
27. Quantos são divisíveis por 4?

28. Quantos são divisíveis por 4 ou 5?
29. Quantos são divisíveis por 4 e 5?
30. Quantos não são divisíveis por 4 nem por 5?

Os Exercícios de 31 a 40 referem-se ao conjunto de cadeias de caracteres de tamanho 8 (cada caracter é ou o dígito 0 ou o dígito 1).

- ★31. Quantas cadeias deste tipo existem?
32. Quantas começam e terminam por 0?
- ★33. Quantas começam ou terminam por 0?
34. Quantas têm 1 como segundo dígito?
35. Quantas começam por 111?
36. Quantas contêm exatamente um 0?
37. Quantas começam por 10 e têm 0 como terceiro dígito?
38. Quantas são palíndromes? (Veja o Exercício 11.)
- ★39. Quantas contêm exatamente sete 1s?
40. Quantas contêm dois ou mais 0s?

Nos Exercícios de 41 a 50, uma mão consiste em uma carta escolhida ao acaso de um baralho de 52 cartas com flores no verso e em uma carta escolhida, também ao acaso, de um baralho de 52 cartas com pássaros no verso.

- ★41. Quantas mãos diferentes são possíveis?
42. Quantas mãos consistem em pares de ases?
43. Quantas mãos contêm todas as cartas de figuras?
- ★44. Quantas mãos consistem em duas cartas do mesmo naipe?
45. Quantas mãos contêm exatamente um rei?
46. Quantas mãos têm o valor total de 5 (considere o ás como 1)?
47. Quantas mãos têm o valor total menor que 5?
48. Quantas mãos não contêm quaisquer cartas de figuras?
- ★49. Quantas mãos contêm pelo menos uma carta de figura?
50. Quantas mãos contêm pelo menos um rei?
- ★51. A votação em determinado debate é feita através de pedaços de papel vermelho, azul e verde que devem ser colocados em um chapéu. Essas tiras de papel são retiradas uma de cada vez, e a primeira cor que receber dois votos ganha. Desenhe uma árvore de decisão para encontrar o número de maneiras que o resultado da votação pode ocorrer.
52. Desenhe uma árvore de decisão (use os times A e B) para encontrar o número de maneiras que as partidas da NBA podem ocorrer, onde o vencedor é o primeiro time a vencer quatro partidas de sete.
53. Use a indução matemática para estender o Princípio da Multiplicação à sequência de m eventos para qualquer inteiro m , $m \geq 2$.

54. Use a indução matemática para estender o Princípio da Adição para m eventos disjuntos para qualquer inteiro m , $m \geq 2$.

Seção 3.3 Princípio da Inclusão e Exclusão e o Princípio da Casa do Pombo

Discutiremos, nesta seção, dois outros princípios da contagem que podem ser usados para resolverem problemas de combinatória.

Princípio da Inclusão e Exclusão

A fim de desenvolvermos o Princípio da Inclusão e Exclusão, precisamos antes perceber que se A e B são quaisquer conjuntos de um conjunto universo S , então $A - B$ e $B - A$ são conjuntos mutuamente disjuntos (veja a Fig. 3.6). Por exemplo, se $x \in A - B$, então $x \notin B$. Portanto, $x \notin B - A$. Além disso, também podemos dizer algo a respeito da união de tais conjuntos.

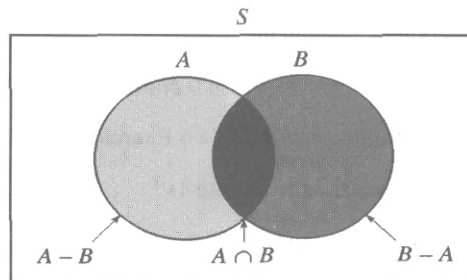


Figura 3.6

PRÁTICA 28 Qual o outro nome do conjunto $(A - B) \cup (B - A) \cup (A \cap B)$?

Pelo Exemplo 34 (estendido a três conjuntos finitos disjuntos)

$$|(A - B) \cup (B - A) \cup (A \cap B)| = |A - B| + |B - A| + |A \cap B|$$

(D)

Do Exemplo 35,

$$|A - B| = |A| - |A \cap B|$$

e

$$|B - A| = |B| - |A \cap B|$$

Usando essas expressões na equação (1), junto com o resultado da Prática 28, obtemos

$$|A \cup B| = |A| - |A \cap B| + |B| - |A \cap B| + |A \cap B|$$

ou

$$|A \cup B| = |A| + |B| - |A \cap B| \quad (2)$$

A equação (2) é a versão para dois conjuntos do Princípio da Inclusão e Exclusão. Este nome decorre do fato de que, quando contamos o número de elementos na união de A e B , precisamos "incluir" (contar) o número de elementos em A e o número de elementos em B , mas devemos "excluir" (subtrair) os elementos que pertencerem a $A \cap B$ para evitar contá-los duas vezes.

PRÁTICA 29 Como a equação (2) se refere ao Exemplo 34 da seção anterior?

EXEMPLO 42 Um entrevistador de opinião pública entrevista 35 pessoas que optam pelo referendun 1, referendun 2 ou por ambos, e conclui que 14 entrevistados escolheram o referendun 1 e 26 o referendun 2. Quantos entrevistados escolheram ambos?

Seja A o conjunto dos entrevistados que escolheram o referendun 1, e B o conjunto dos entrevistados que escolheram o referendun 2; assim, sabemos que

$$|A \cup B| = 35 \quad |A| = 14 \quad |B| = 26$$

Da equação (2),

$$|A \cap B| = |A| + |B| - |A \cup B| = 14 + 26 - 35 = 5$$

portanto, cinco entrevistados escolheram ambos.

A Equação (2) pode ser facilmente estendida para o caso de três conjuntos, como mostrado a seguir:

$$\begin{aligned} |A \cup B \cup C| &= |A \cup (B \cup C)| = |A| + |B \cup C| - |A \cap (B \cup C)| \\ &= |A| + |B| + |C| - |B \cap C| - |(A \cap B) \cup (A \cap C)| \\ &= |A| + |B| + |C| - |B \cap C| - (|A \cap B| + |A \cap C| - |A \cap B \cap C|) \\ &= |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C| \end{aligned}$$

Portanto, a versão do Princípio da Inclusão e Exclusão para três conjuntos é

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C| \quad (3)$$

PRÁTICA 30 Justifique cada uma das igualdades usadas na derivação da equação (3).

EXEMPLO 43 Uma quitanda vende apenas brócolis, cenoura e quiabo. Em determinado dia, a quitanda atendeu 208 pessoas. Se 114 pessoas compraram apenas brócolis, 152 compraram cenouras, 17 compraram quiabos, 64 compraram brócolis e cenouras, 12 compraram cenouras e quiabos e 9 compraram todos os três, quantas pessoas compraram brócolis e quiabos?

Sejam

$$\begin{aligned} A &= \{\text{pessoas que compraram brócolis}\} \\ B &= \{\text{pessoas que compraram cenouras}\} \\ C &= \{\text{pessoas que compraram quiabos}\} \end{aligned}$$

Então $|A \cup B \cup C| = 208$, $|A| = 114$, $|B| = 152$, $|C| = 17$, $|A \cap B| = 64$, $|B \cap C| = 12$ e $|A \cap B \cap C| = 9$. Da equação (3),

$$|A \cap C| = 114 + 152 + 17 - 64 - 12 + 9 - 208 = 8$$

Na equação (2), somamos o número de elementos nos conjuntos simples, e subtraímos o número de elementos da interseção de ambos os conjuntos. Na equação (3), somamos o número de elementos do conjunto simples, subtraímos o número de elementos das interseções dos conjuntos dois a dois e somamos, novamente, o número de elementos da interseção dos três conjuntos. Isto parece sugerir um padrão: Se tivermos n conjuntos, devemos somar o número de elementos dos conjuntos simples, subtrair o número de elementos das interseções dos conjuntos dois a dois, somar o número de elementos das interseções dos conjuntos três a três, subtrair o número de elementos das interseções dos conjuntos quatro a quatro, e assim por diante. Isto nos leva à forma geral do Princípio da Inclusão e Exclusão:

Princípio da Inclusão e Exclusão

Dados os conjuntos finitos $A_1, \dots, A_n$, $n \geq 2$, então,

$$\begin{aligned} |A_1 \cup \dots \cup A_n| &= \sum_{1 \leq i \leq n} |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| \\ &\quad + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| \\ &\quad - \dots + (-1)^{n+1} |A_1 \cap \dots \cap A_n| \end{aligned} \quad (4)$$

Na equação (4), a notação

$$\sum_{1 \leq i < j \leq n} |A_i \cap A_j|$$

por exemplo, indica a soma dos números de elementos de todas as interseções da forma $A_i \cap A_j$, onde i e j podem conter quaisquer valores entre 1 e n , desde que $i < j$. Para $n = 3$, temos $|A_1 \cap A_2|$ ($i = 1, j = 2$), $|A_1 \cap A_3|$ ($i = 1, j = 3$) e $|A_2 \cap A_3|$ ($i = 2, j = 3$). Isto está de acordo com a equação (3), onde $A_1 = A$, $A_2 = B$ e $A_3 = C$.

Para provar a forma geral do Princípio da Inclusão e Exclusão, usamos a indução matemática. O caso básico, $n = 2$ é a equação (2). Admitimos a equação (4) como verdadeira para $n = k$, e mostraremos que ela é válida para $n = k + 1$. Escrevemos

$$\begin{aligned} & |A_1 \cup \dots \cup A_{k+1}| \\ &= |(A_1 \cup \dots \cup A_k) \cup A_{k+1}| \\ &= |A_1 \cup \dots \cup A_k| + |A_{k+1}| \\ &\quad - |(A_1 \cup \dots \cup A_k) \cap A_{k+1}| \quad (\text{pela equação (2)}) \\ &= \sum_{1 \leq i \leq k}^a |A_i| - \sum_{1 \leq i < j \leq k} |A_i \cap A_j| + \sum_{1 \leq i < j < m \leq k} |A_i \cap A_j \cap A_m| \\ &\quad - \dots + (-1)^{k+1} |A_1 \cap \dots \cap A_k| + |A_{k+1}| \\ &\quad - |(A_1 \cap A_{k+1}) \cup \dots \cup (A_k \cap A_{k+1})| \end{aligned}$$

(pela hipótese de indução e a propriedade distributiva)

$$\begin{aligned} &= \sum_{1 \leq i \leq k+1} |A_i| - \sum_{1 \leq i < j \leq k}^b |A_i \cap A_j| + \sum_{1 \leq i < j < m \leq k}^c |A_i \cap A_j \cap A_m| \\ &\quad - \dots + (-1)^{k+1} |A_1 \cap \dots \cap A_k| \\ &\quad - \left(\sum_{1 \leq i \leq k}^b |A_i \cap A_{k+1}| - \sum_{1 \leq i < j \leq k}^c |A_i \cap A_j \cap A_{k+1}| + \dots + \right. \\ &\quad \left. + (-1)^k \sum_{1 \leq i < j < \dots < m \leq k} \underbrace{|(A_i \cap A_{k+1}) \cap (A_j \cap A_{k+1}) \cap \dots \cap (A_m \cap A_{k+1})|}_{k-1 \text{ termo}} \right) \\ &\quad + (-1)^{k+1} |A_1 \cap \dots \cap A_{k+1}| \end{aligned}$$

(combinando os termos 1 acima e usando a hipótese de indução nos k conjuntos $A_1 \cap A_{k+1}, A_2 \cap A_{k+1}, \dots, A_k \cap A_{k+1}$)

$$\begin{aligned} &= \sum_{1 \leq i \leq k+1} |A_i| - \sum_{1 \leq i < j \leq k+1} |A_i \cap A_j| + \sum_{1 \leq i < j < m \leq k+1} |A_i \cap A_j \cap A_m| \\ &\quad - \dots - (-1)^{k+1} |A_1 \cap \dots \cap A_{k+1}| \end{aligned}$$

(combinando os termos do mesmo índice acima)

$$\begin{aligned} &= \sum_{1 \leq i \leq k+1} |A_i| - \sum_{1 \leq i < j \leq k+1} |A_i \cap A_j| + \sum_{1 \leq i < j < m \leq k+1} |A_i \cap A_j \cap A_m| \\ &\quad - \dots + (-1)^{k+2} |A_1 \cap \dots \cap A_{k+1}| \end{aligned}$$

Isto completa a prova da equação (4).

O Princípio da Casa do Pombo

O Princípio da Casa do Pombo recebe este nome estranho devido à seguinte idéia: Se mais do que k pombos pousarem em k casas de pombos, então pelo menos uma casa de pombo ficará com mais de um pombo. Apesar de isto parecer imediatamente óbvio, podemos construir uma prova por contradição. Suponha que mais do que k pombos pousaram em k casas de pombos. Se cada casa contiver no máximo um pombo, teríamos, ao todo, no máximo k pombos, uma contradição. Enunciaremos, agora, o Princípio da Casa do Pombo de uma forma menos pitoresca.

Princípio da Casa do Pombo

Se mais do que k itens são distribuídos entre k caixas, então pelo menos uma caixa conterá mais de um item.

Escolhendo-se apropriadamente os itens e as caixas, podemos resolver uma grande gama de problemas de contagem.

EXEMPLO 44 Quantas pessoas precisam estar no mesmo quarto para se garantir que pelo menos duas pessoas têm o sobrenome iniciado pela mesma letra?

Existem 26 letras no alfabeto (caixas). Se tiverem 27 pessoas, então haverá 27 letras iniciais (itens) que devem ser distribuídas entre as 26 caixas; por isso, pelo menos uma caixa conterá mais de um item. •

PRÁTICA 31 Quantas vezes um único dado precisa ser lançado para termos certeza de que obtivemos algum valor duas vezes? •

EXEMPLO 45 Prove que se 51 inteiros positivos entre 1 e 100 são escolhidos, então pelo menos um deles divide outro.

Sejam os inteiros $n_1, \dots, n_{51}$ diferentes entre si. Cada inteiro $n_i \geq 2$ pode ser escrito como um produto de números primos (veja o Exemplo 17 do Cap. 2); cada número primo, exceto o 2, é ímpar, e o produto de números ímpares é ímpar. Portanto, para cada i , $n_i = 2^{k_i} b_i$, onde $k_i \geq 0$ e b_i é um número ímpar. Além disso, $1 \leq b_i \leq 99$. Mas existem 50 números ímpares entre 1 e 99 inclusive. Pelo Princípio da Casa do Pombo, $b_i = b_j$ para algum $i \neq j$ com $i \neq j$. Portanto, $n_i = 2^{k_i} b_i$ e $n_j = 2^{k_j} b_i$. Se $k_i < k_j$ então n_i divide n_j ; do contrário, n_j divide n_i . •

Revisão da Seção 3.3

Técnicas

- Uso do Princípio da Inclusão e Exclusão para encontrar o número de elementos da união de conjuntos.
- Uso do Princípio da Casa do Pombo para encontrar o número mínimo de elementos que garantem que dois elementos gozam de uma mesma propriedade.

Idéia Principal

O Princípio da Inclusão e Exclusão e o Princípio da Casa do Pombo são mecanismos de contagem adicionais.

Exercícios 3.3

1. Em um grupo de 42 turistas, todos falam inglês ou francês; existem 35 pessoas que falam inglês e 18 pessoas que falam francês. Quantas falam inglês e francês?
- 2. Todos os convidados de uma festa bebem café ou chá; 13 convidados bebem café, 10 bebem chá e 4 bebem café e chá. Quantas pessoas têm neste grupo?
3. O controle de qualidade em uma fábrica introduziu 47 peças com defeitos de pintura, na embalagem ou na parte eletrônica na linha de montagem. Dessas peças, 28 tinham defeito de pintura, 17 tinham defeito na embalagem, 7 tinham defeito na embalagem e na parte eletrônica e 3 tinham defeitos na pintura e na parte eletrônica. Alguma peça tinha os três tipos de defeito?

4. Em um grupo de 24 pessoas que gostam de rock, country e música clássica, 14 gostam de rock, 12 gostam de música clássica, 11 gostam de rock e country, 9 gostam de rock e música clássica, 13 gostam de country e música clássica e 8 gostam de rock, country e música clássica. Quantos gostam de country?
- ★5. Onze produtos diferentes para higiene bucal têm as seguintes estratégias: 10 veiculam que oferecem um hálito puro, oito garantem que protegem a gengiva, sete anunciam que reduzem a placa bacteriana, seis prometem um hálito puro e a redução da placa, cinco dizem prevenir a gengiva e oferecer um hálito puro e cinco dizem prevenir a gengiva e reduzir a placa.
 - a. Quantos produtos veiculam todas as três vantagens?
 - b. Quantos produtos veiculam um hálito puro, mas não veiculam prevenir a formação da placa bacteriana?
6. Dentre 214 clientes de um banco com contas-correntes, caderneta de poupança ou aplicações financeiras, 189 têm contas-correntes, 73 têm cadernetas de poupanças regulares, 114 têm aplicações no mercado financeiro e 69 têm contas-correntes e cadernetas de poupança. Não é possível ter caderneta de poupança e investir no mercado financeiro.
 - a. Quantos clientes têm, ao mesmo tempo, conta-corrente e aplicações no mercado financeiro?
 - b. Quantos clientes têm apenas conta-corrente?
- ★7. Uma pesquisa dentre 150 estudantes revelou que 83 são proprietários de carros, 97 possuem bicicletas, 28 têm motocicletas, 53 são donos de carros e bicicletas, 14 têm carros e motocicletas, sete possuem bicicletas e motocicletas, e dois têm todos os três.
 - a. Quantos estudantes possuem apenas bicicletas?
 - b. Quantos estudantes não têm qualquer dos três?
8. Você está desenvolvendo um novo sabonete e contratou uma empresa de pesquisa de opinião pública para realizar uma pesquisa de mercado para você. A empresa constatou que, em sua pesquisa de 450 consumidores, os fatores a seguir foram considerados relevantes na decisão de compra de um sabonete:

Perfume	425
Fácil produção de espuma	397
Ingredientes naturais	340
Perfume e fácil produção de espuma	284
Perfume e ingredientes naturais	315
Fácil produção de espuma e ingredientes naturais	219
Todos os três fatores	147

Você confiaria nesses resultados? Justifique.

9. Escreva a expressão para $|A \cup B \cup C \cup D|$ a partir da equação (4).
10. Escreva uma expressão para o número de termos da expressão para $|A_1 \cup \dots \cup A_n|$ dada pela equação (4).
11. Quantas cartas precisam ser tiradas de um baralho convencional de 52 cartas para garantirem que tiraremos duas cartas do mesmo naipe?
- ★12. Se 12 cartas são tiradas de um baralho convencional, podemos afirmar que duas têm valores iguais, independentemente do naipe?
13. Quantas pessoas precisam estar em um grupo para se garantir que duas pessoas tenham o mesmo aniversário (não se esqueça de ignorar o ano)?
14. Em um grupo de 25 pessoas, podemos afirmar que existem pelo menos três que nasceram no mesmo mês?
- ★15. Prove que se quatro números são escolhidos do conjunto $\{1, 2, 3, 4, 5, 6\}$, pelo menos um par precisa somar 7. (Dica: Encontre todos os pares de números do conjunto que somem 7.)
16. Quantos números precisam ser escolhidos do conjunto $\{2, 4, 6, 8, 10, 12, 14, 16, 18, 20\}$ afim de se garantir que pelo menos um par soma 22? (Veja a dica para o Exercício 15.)

17. Seja n um inteiro positivo. Mostre que em qualquer conjunto com $n + 1$ elementos, existem pelo menos dois com o mesmo resto, quando dividido por n .

Seção 3.4 Permutações e Combinações

Permutações

Na Seção 3.2, vimos o problema da contagem de todas as possibilidades para os últimos quatro dígitos de um número telefônico sem repetições (Exemplo 30). Neste problema, o número 1259 não é o mesmo que o número 2951, pois a ordem dos dígitos é importante. Um arranjo ordenado de objetos é chamado de **permutação**. A determinação da quantidade de números de quatro dígitos sem dígitos repetidos pode ser considerada a contagem do número de permutações ou arranjos: são 4 objetos distintos escolhidos de um conjunto de 10 objetos distintos (os dígitos). A resposta encontrada pelo Princípio da Multiplicação foi $10 \cdot 9 \cdot 8 \cdot 7$. Em geral, o número de permutações de r objetos distintos escolhidos de n objetos distintos é denotada por $P(n, r)$. Portanto, a solução do problema dos quatro dígitos sem repetição pode ser expressada como $P(10, 4)$.

Uma fórmula para $P(n, r)$ pode ser escrita usando a função fatorial. Para um inteiro positivo n , **fatorial de n** é definido como $n(n - 1)(n - 2) \dots 1$ e denotado por $n!$; além disso, $0!$ é definido como tendo valor 1. Pela definição de $n!$, vemos que

$$n! = n(n - 1)!$$

e que para $r < n$,

$$\begin{aligned} \frac{n!}{(n - r)!} &= \frac{n(n - 1) \cdots (n - r + 1)(n - r)!}{(n - r)!} \\ &= n(n - 1) \cdots (n - r + 1) \end{aligned}$$

Usando a função fatorial,

$$\begin{aligned} P(10, 4) &= 10 \cdot 9 \cdot 8 \cdot 7 \\ &= \frac{10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1}{6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1} = \frac{10!}{6!} = \frac{10!}{(10 - 4)!} \end{aligned}$$

Em geral, $P(n, r)$ é dado pela fórmula

$$P(n, r) = \frac{n!}{(n - r)!} \text{ para } 0 \leq r \leq n$$

EXEMPLO 46 O valor de $P(7, 3)$ é

$$\frac{7!}{(7 - 3)!} = \frac{7!}{4!} = \frac{7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1}{4 \cdot 3 \cdot 2 \cdot 1} = 7 \cdot 6 \cdot 5 = 210$$

EXEMPLO 47 Três casos particulares podem ocorrer ao computarmos $P(n, r)$, que são as duas "condições de fronteira" $P(n, 0)$ e $P(n, n)$ e também $P(n, 1)$. De acordo com a fórmula

$$P(n, 0) = \frac{n!}{(n - 0)!} = \frac{n!}{n!} = 1$$

Isto pode ser interpretado como dizer que existe um único arranjo ordenado de zero objeto — o conjunto vazio.

$$P(n, 1) = \frac{n!}{(n - 1)!} = n$$

Esta fórmula reflete o fato de que existem n arranjos ordenados de um objeto. (Como cada arranjo consiste em apenas um objeto, basta contar quantas formas existem de se escolher um objeto.)

$$P(n, n) = \frac{n!}{(n-n)!} = \frac{n!}{0!} = n!$$

Esta fórmula mostra que existem $n!$ arranjos ordenados de n objetos distintos. (O que apenas reflete o Princípio da Multiplicação — n escolhas para o primeiro objeto, $n - 1$ para o segundo, e assim por diante, com uma única escolha para o n -ésimo objeto.)

Na fórmula de $P(n, r)$, à medida que r cresce, $n - r$, e, portanto, $(n - r)!$, diminui; logo $P(n, r)$ cresce. Desta forma, os valores de $P(n, r)$ cresceram de n a $n!$ para $1 \leq r \leq n$. •

EXEMPLO 48 O número de permutações de três objetos, digamos a , b e c , é dado por $P(3, 3) = 3! = 3 \cdot 2 \cdot 1 = 6$. São elas

$abc, acb, bac, bca, cab, cba$ •

EXEMPLO 49 Quantas palavras de três letras (não necessariamente com sentido) podem ser formadas com as letras da palavra "compilar", se não pudermos repetir letras? Neste caso, desejamos saber o número de permutações de três objetos distintos tomados dentre oito objetos. A resposta é $P(8, 3) = 8!/5! = 336$. •

Perceba que poderíamos ter resolvido o Exemplo 49, usando apenas o Princípio da Multiplicação — existem oito possibilidades para a primeira letra, sete para a segunda e seis para a terceira, de forma que a resposta é $8 \cdot 7 \cdot 6 = 336$. $P(n, r)$ simplesmente nos fornece uma nova maneira de pensar no problema, bem como uma notação mais compacta.

PRÁTICA 32 De quantas maneiras podem ser escolhidos um presidente e um vice-presidente dentre um grupo de 20 pessoas? •

PRÁTICA 33 De quantos modos seis pessoas podem sentar-se em uma sala com seis cadeiras? •

EXEMPLO 50 Uma biblioteca tem quatro livros sobre sistemas operacionais, sete sobre programação e três sobre estrutura de dados. Vamos ver de quantas maneiras esses livros podem ser arrumados em uma prateleira, considerando que todos os livros de cada assunto precisam estar juntos. Podemos pensar neste problema como uma seqüência de subtarefas. Primeiro consideremos a subarefa de arrumar os três assuntos. Existem $3!$ maneiras de fazer isto, isto é, $3!$ maneiras de ordenar os assuntos dos livros na prateleira. As etapas seguintes são arranjar os livros sobre sistemas operacionais ($4!$ maneiras), arrumar os livros sobre programação ($7!$ maneiras) e, então, arrumar os livros sobre estrutura de dados ($3!$ maneiras). Portanto, pelo Princípio da Multiplicação, o número final de arranjos possíveis de todos os livros é $(3!) (4!) (7!) (3!) = 4.354.560$. •

Combinações

Às vezes, desejamos selecionar r objetos de um conjunto de n objetos, mas não desejamos releva a ordem na qual eles são arranjados. Neste caso, estamos contando o número de **combinações** de r objetos distintos escolhidos dentre n objetos distintos, denotadas por $C(n, r)$. Para cada combinação dessas, existem $r!$ maneiras de permutar seus r objetos. Pelo Princípio da Multiplicação, o número de objetos escolhidos dentre n objetos é o produto do número de maneiras de selecionar esses objetos, $C(n, r)$ multiplicado pelo número de maneiras de arranjar esses objetos escolhidos, $r!$. Portanto,

$$C(n, r) \cdot r! = P(n, r)$$

ou

$$C(n, r) = \frac{P(n, r)}{r!} = \frac{n!}{r!(n-r)!} \text{ para } 0 \leq r \leq n$$

Outras notações para $C(n, r)$ são

$${}_nC_r, C_r^n, \text{ e } \binom{n}{r}.$$

EXEMPLO 51 O valor de $C(7, 3)$ é

$$\begin{aligned} \frac{7!}{3!(7-3)!} &= \frac{7!}{3!4!} = \frac{7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1}{3 \cdot 2 \cdot 1 \cdot 4 \cdot 3 \cdot 2 \cdot 1} \\ &= \frac{7 \cdot 6 \cdot 5}{3 \cdot 2 \cdot 1} = 7 \cdot 5 = 35 \end{aligned}$$

Do Exemplo 46, o valor de $P(7, 3)$ é 210 e $C(7, 3) \cdot (3!) = 35(6) = 210 = P(7, 3)$. •

EXEMPLO 52 Os casos especiais para $C(n, r)$ são $C(n, 0)$, $C(n, 1)$ e $C(n, n)$. A fórmula para $C(n, 0)$,

$$C(n, 0) = \frac{n!}{0!(n-0)!} = 1$$

reflete o fato de que existe apenas uma maneira de escolher zero objeto dentre n — o conjunto vazio.

$$C(n, 1) = \frac{n!}{1!(n-1)!} = n$$

Aqui a fórmula mostra que existem n maneiras de escolher 1 dentre n objetos.

$$C(n, n) = \frac{n!}{n!(n-n)!} = 1$$

Aqui vemos que existe uma maneira de se escolher n dentre n objetos e que esta escolha é composta por todos os objetos.

Na fórmula de $C(n, r)$, à medida que r aumenta, $r!$ também aumenta, o que tende a fazer $C(n, r)$ menor, mas $(n-r)!$ diminui, o que tende a tornar $C(n, r)$ maior. Para pequenos valores de r , o aumento de $r!$ não é tão marcante quanto a diminuição de $(n-r)!$ e, desta forma, $C(n, r)$ aumenta de 1 para n e para valores maiores. Em algum ponto, no entanto, o acréscimo de $r!$ supera o decréscimo de $(n-r)!$ e os valores de $C(n, r)$ diminuem até voltar a 1 quando $r = n$. •

EXEMPLO 53 Quantas mãos de pôquer com cinco cartas podem ser sorteadas de um baralho de 52 cartas? Neste caso a ordem não interessa; $C(52, 5) = 52!/(5! 47!) = 2.598.960$. •

Ao contrário dos problemas anteriores, a resposta ao Exercício 53 não pode ser facilmente obtida pela aplicação do Princípio da Multiplicação. Portanto, $C(n, r)$ nos fornece um modo de resolver novos problemas.

PRÁTICA 34 De quantas maneiras podemos escolher um comitê de três pessoas dentre um grupo de 12? •

Lembre-se de que a diferença entre permutações e combinações reside no fato de que os objetos são apenas selecionados ou selecionados e ordenados. Se a ordem for relevante, o problema envolve permutações; se a ordem não importar, o problema envolve combinações. Por exemplo, a Prática 32 é um problema de permutação — duas pessoas serão selecionadas e ordenadas — a primeira será o presidente e a segunda o vice-presidente —, enquanto que a Prática 34 é um problema de combinação — três pessoas serão escolhidas, mas não serão ordenadas. Um bom ponto de partida para os problemas de contagem, então, é determinar se a ordem é importante ou não.

Na solução de problemas de contagem, $C(n, r)$ pode ser usada juntamente com o Princípio da Multiplicação ou o Princípio da Adição.

EXEMPLO 54 Um comitê de oito estudantes deve ser selecionado de uma turma de 19 calouros e 34 veteranos.

- De quantas maneiras podem ser selecionados três calouros e cinco veteranos?
- De quantas maneiras podem ser selecionados comitês com exatamente um calouro?
- De quantas maneiras podem ser selecionados comitês com no máximo um calouro?
- De quantas maneiras podem ser selecionados comitês com pelo menos um calouro?

Como a ordenação das escolhas individuais não é importante, este problema envolve combinações.

Para o item (a), temos uma sequência de duas etapas, selecionar calouros e selecionar veteranos. O Princípio da Multiplicação pode ser usado. (Pensar em uma sequência de subtarefas pode parecer implicar ordenação, mas isto apenas define os níveis da árvore de decisão, a base para o Princípio da Multiplicação. Na verdade, não há ordenação para os estudantes.) Como existem $C(19, 3)$ maneiras de escolher calouros e $C(34, 5)$ formas de escolher veteranos, a resposta é

$$C(19, 3) \cdot C(34, 5) = \frac{19!}{3!16!} \cdot \frac{34!}{5!29!} = (969)(278.256)$$

Para o item (b), temos novamente uma sequência de subtarefas: selecionar um único calouro e então selecionar o resto do comitê dentre os veteranos. Existem $C(19, 1)$ maneiras de selecionar o único calouro e $C(34, 7)$ modos de selecionar os sete outros membros dentre os veteranos. Pelo Princípio da Multiplicação, a resposta é

$$C(19, 1) \cdot C(34, 7) = \frac{19!}{1!(19-1)!} \cdot \frac{34!}{7!(34-7)!} = 19(5.379.616)$$

Para o item (c), obtemos no máximo um calouro escolhendo exatamente um calouro ou escolhendo 0 calouro. Como esses eventos são disjuntos, usaremos o Princípio da Adição. O número de maneiras de escolher exatamente um calouro é a resposta do item (b). O número de escolher 0 calouro é a mesmo do número de maneiras de selecionar todos os oito membros do comitê dentre os 34 veteranos, $C(34, 8)$. Portanto, a resposta é

$$C(19, 1) \cdot C(34, 7) + C(34, 8) = \text{algum número grande}$$

Podemos abordar o item (d) de diversas formas. Uma forma é usar o Princípio da Adição para as seguintes possibilidades disjuntas: exatamente um calouro, exatamente dois calouros, e assim por diante até oito calouros. Podemos computar cada um desses números, e então somá-los. No entanto é mais simples ver de quantas maneiras o comitê pode ser selecionado de um total de 53 pessoas, e então eliminar (subtrair) o número de comitês com 0 calouro (apenas veteranos). Desta forma, a resposta é

$$C(53, 8) - C(34, 8)$$

A função fatorial cresce muito rapidamente. Um número como $100!$ não pode ser computado em diversas calculadoras (ou em muitos computadores, exceto se usarmos aritmética de precisão dupla), mas expressões como

$$\frac{100!}{25!75!}$$

podem, todavia, ser computadas se cancelarmos os fatores comuns em primeiro lugar.

Eliminando Duplicidades

Mencionamos anteriormente que problemas de contagem podem, ser resolvidos amiúde de diferentes maneiras. Infelizmente, também é fácil encontrar assim-chamadas soluções que parecem razoáveis, mas são, na verdade, incorretas. Em geral, são erradas porque contam algum(ns) elemento(s) mais de uma vez (ou às vezes por esquecer de contar algo).

EXEMPLO 55 Considere novamente o item (d) do Exemplo 54. Uma solução inválida para este problema é a seguinte: Imagine uma sequência de duas subtarefas, escolhendo um calouro e então escolhendo o resto do comitê. Existem $C(19, 1)$ maneiras de escolher um calouro. Uma vez que um calouro já tenha sido escolhido, o que garante que pelo menos um calouro compõe o comitê, estamos livres para escolher os demais sete membros do comitê

dentre as 52 pessoas que sobraram, sem quaisquer restrições, o que nos dá $C(52, 7)$ possibilidades. Pelo Princípio da Multiplicação, temos $C(19, 1) \cdot C(52, 7)$. No entanto, este é um número maior do que o obtido na resposta correta.

O problema é o seguinte: suponha que Daniel e Felícia são calouros. Em uma das possibilidades, foi contado o caso no qual Daniel é o calouro escolhido em primeiro lugar, e o resto do comitê foi selecionado de forma que Felícia foi escolhida junto com outros seis membros. Mas também contamos o caso no qual Felícia foi escolhida em primeiro lugar, e Daniel foi selecionado junto com os mesmos seis outros membros do comitê. Este comitê é o mesmo que o anterior, e foi contado duas vezes.

PRÁTICA 35 Um comitê de duas pessoas precisa ser escolhido dentre quatro matemáticos e três físicos, e precisa incluir pelo menos um matemático. Compute os dois valores a seguir

- $C(7, 2) - C(3, 2)$ (a solução correta — todos os comitês menos os sem matemáticos)
- $C(4, 1) \cdot C(6, 1)$ (a solução errada — escolhe um matemático e depois seleciona o outro integrante do comitê)

Perceba que $C(4, 1) \cdot C(6, 1) - C(4, 2)$ nos dá a resposta correta, porque $C(4, 2)$ é o número de comitês com dois matemáticos, e esses comitês foram contados duas vezes em $C(4, 1) \cdot C(6, 1)$.

EXEMPLO 56 a. Quantas permutações distintas podem ser formadas com as letras da palavra FLORIDA?
b. Quantas permutações distintas podem ser formadas com as letras da palavra MISSISSIPI?

O item (a) é um simples problema de encontrar o número de arranjos ordenados de sete objetos distintos, que é $7!$. No entanto, a resposta para o item (b) não é $11!$ porque as 11 letras de MISSISSIPI não são todas distintas. Isto significa que $11!$ conta alguns dos mesmos arranjos mais de uma vez (o mesmo arranjo significa que não podemos ver a diferença entre $MIS_1S_2ISSIPI$ e $MIS_2S_1ISSIPI$.)

Considere qualquer arranjo das letras. Os quatro Ss ocupam certas posições no arranjo. Rearrumar esses Ss resultaria em não alterar o resultado final, de forma que um mesmo arranjo tem $4!$ possibilidades de ser escrito. A fim de evitar contar esse arranjo mais de uma vez, devemos dividir $11!$ por $4!$, que são as formas de trocar os Ss de posição. Analogamente, precisamos dividir por $4!$ a fim de evitar a contagem repetida por parte dos Is e por $2!$ para tratar os dois Ps. O número de permutações distintas é, portanto,

$$\frac{11!}{4!4!2!}$$

Em geral, suponha que existam n objetos dos quais um conjunto de n_1 é igual entre si, outro conjunto de n_2 é também igual entre si, e assim por diante até n_k objetos que são iguais entre si. O número de permutações distintas desses n objetos é

$$\frac{n!}{(n_1!)(n_2!)\cdots(n_k!)}$$

PRÁTICA 36 Quantas permutações distintas são possíveis com as letras da palavra MONOFÁSICOS?

Permutações e Combinações com Repetições

Nossas fórmulas para $P(n, r)$ e $C(n, r)$ assumem que arranjamos ou escolhemos r objetos dentre n objetos disponíveis usando cada objeto apenas uma vez. Portanto, $r \leq n$. Suponha, no entanto, que podemos reutilizar os n objetos tantas vezes quantas desejarmos. Por exemplo, construímos palavras usando as 26 letras do alfabeto; as palavras podem ser tão grandes quanto quisermos, e as letras podem ser repetidas. Ou desejamos sortear cartas de um baralho, repondo-as após cada sorteio; poderemos sortear quantas cartas desejarmos com cartas sendo sorteadas repetidamente. Podemos continuar falando de permutações e combinações de r objetos n a n , mas com a possibilidade de repetições, r pode ser maior que n .

Contar o número de permutações de r objetos n a n objetos distintos com repetições (ou reposição) é simples. Temos n opções para a escolha do primeiro objeto e, uma vez que podemos repetir esse objeto, n opções para a escolha do segundo objeto, n opções para o terceiro e assim por diante. Portanto, o número de permutações de r objetos n a n com a possibilidade de repetições é n^r .

Para determinar o número de combinações de r objetos n a n com a possibilidade de repetições, usamos uma idéia um pouco mais elaborada.

EXEMPLO 57 Um joalheiro, ao projetar um broche, decidiu usar cinco pedras escolhidas entre diamantes, rubis e esmeraldas. De quantas maneiras as pedras podem ser escolhidas?

Como não estamos interessados na ordem em que as pedras serão arranjadas, este é um problema de combinação, e não um problema de permutação. Desejamos obter o número de combinações de cinco objetos três a três, permitindo repetições. O broche pode ser formado de um diamante, três rubis e uma esmeralda, por exemplo, ou cinco diamantes. Podemos representar essas possibilidades representando as pedras escolhidas com asteriscos e a inclusão de separadores entre elas a fim de representar a distribuição entre os três tipos de pedras. Por exemplo, podemos representar a escolha de um diamante, três rubis e uma esmeralda por

* | *** | *

enquanto que a escolha de cinco diamantes, nenhum rubi e nenhuma esmeralda pode ser representada por

***** | |

Estamos, portanto, trabalhando com sete posições (para as cinco pedras e os dois separadores), e as diferentes escolhas são determinadas por quais posições são ocupadas por asteriscos. Estamos contando, portanto, o número de maneiras de escolher cinco itens dentre sete, que é $C(7, 5)$ ou

$$\frac{7!}{5!2!}$$

Em geral, se usarmos o mesmo esquema para representarmos uma combinação de r objetos dentre n objetos distintos com a possibilidade de repetições, existirão $n - 1$ separadores para indicar o número de cópias de cada um dos n objetos. Isto nos dá $r + (n - 1)$ posições a ser preenchidas, e desejamos obter o número de maneiras de selecionar r dessas posições. Portanto, o valor que desejamos é

$$C(r + n - 1, r) = \frac{(r + n - 1)!}{r!(n - 1)!} = \frac{(r + n - 1)!}{r!(n - 1)!}$$

PRÁTICA 37 Seis crianças escolhem um pirulito cada, dentre pirulitos vermelhos, amarelos e verdes. De quantas maneiras essa escolha pode ser feita?

Revisão da Seção 3.4

Técnicas

- Encontrar o número de permutações de r objetos distintos escolhidos dentre n objetos distintos.
- Encontrar o número de combinações de r objetos distintos escolhidos dentre n objetos distintos.
- Usar permutações e combinações em conjunto com o Princípio da Multiplicação e o Princípio da Adição.
- Encontrar o número de permutações distintas de n objetos que não sejam todos distintos.
- Encontrar o número de permutações de r objetos dentre n objetos distintos que possam ser escolhidos repetidas vezes.
- Encontrar o número de combinações de r objetos dentre n objetos distintos que possam ser escolhidos repetidas vezes.

Idéias Principais

Existem fórmulas para contagem de diversas permutações e combinações de objetos. É preciso tomar cuidado ao analisar problemas de contagem a fim de evitar a contagem das mesmas possibilidades mais de uma vez e evitar que se esqueça de contar algumas possibilidades.

Exercícios 3.4

1. Compute o valor das expressões abaixo:

★a. $P(7, 2)$	b. $P(8, 5)$	c. $P(6, 4)$
d. $P(n, 1)$	e. $P(n, n - 1)$	

2. Quantas ordenações para rebatedores é possível em um time de nove jogadores de beisebol?
3. Os 14 times da Confederação Local estão listados no jornal. Quantas listagens diferentes são possíveis?
4. Quantas permutações das letras da palavra COMPUTADOR existem? Quantas delas terminam por uma vogal?
- ★5. Quantas permutações distintas da palavra ERRO existem? (Lembre-se que os Rs não podem ser distinguidos um do outro.)
6. De quantas maneiras seis pessoas podem sentar-se em uma roda com seis cadeiras? (Apenas as posições relativas em um círculo podem ser distinguidas.)
- ★7. De quantas maneiras os primeiro, segundo e terceiro prêmios em um concurso de tortas podem ser atribuídos a 15 concorrentes?
8. a. A designação de títulos de valores é limitada a três letras. Quantas designações existem?
b. Quantas designações existem se as letras não puderem se repetir?
- ★9. De quantas maneiras diferentes podem se sentar 11 homens e 8 mulheres em uma fileira, se todos os homens se sentam juntos e as mulheres também se sentam juntas?
10. De quantas maneiras diferentes podem se sentar 11 homens e oito mulheres em uma fileira sem que duas mulheres se sentem juntas?
11. Compute o valor das seguintes expressões:
★a. $C(10, 7)$ b. $C(9, 2)$ c. $C(8, 6)$ d. $C(n, n - 1)$
12. Compute $C(n, n - 1)$. Explique por que $C(n, n - 1) = C(n, 1)$
- ★13. O controle de qualidade deseja testar 25 chips de microprocessadores dentre os 300 que são produzidos diariamente. De quantas maneiras isto pode ser feito?
14. Um time de futebol leva 18 jogadores na comitiva; 11 jogadores compõem o time titular. De quantas maneiras o time titular pode ser formado?
15. De quantas maneiras pode ser selecionado um júri de cinco homens e sete mulheres dentre um elenco
- ★ de 17 homens e 23 mulheres?
16. De quantas formas uma bibliotecária seleciona quatro novelas e três peças dentre uma coleção de 21 novelas e 11 peças?

Os Exercícios 17 a 20 referem-se à seguinte situação: do pessoal de uma companhia, sete trabalham no projeto, 14 na produção, quatro nos testes, cinco em vendas, dois na contabilidade e três em marketing. Um comitê de seis pessoas deve ser formado para uma reunião com o supervisor.

- ★17. De quantas maneiras podemos formar este comitê, se tiver que haver um membro de cada departamento?
18. De quantas maneiras podemos formar o comitê, se tiver que haver exatamente dois membros do departamento de produção?
19. De quantas maneiras o comitê pode ser formado, se o departamento de contabilidade não for representado e o de marketing tiver exatamente um representante?
- ★20. De quantas maneiras o comitê pode ser formado se a produção tiver que ter pelo menos dois representantes?

Os Exercícios 21 a 26 referem-se a uma mão de cinco cartas tiradas de um baralho de 52 cartas.

- ★21. Quantas mãos consistem em três cartas de espadas e duas de copas?
22. Quantas mãos consistem em cartas apenas de ouros?
- ★23. Quantas mãos consistem em cartas do mesmo naipe?

24. Quantas mãos consistem em cartas apenas com figuras?
25. Quantas mãos contêm uma trinca (três cartas do mesmo tipo)?
26. Quantas mãos contêm um *full house* (uma trinca e um par)?

Nos Exercícios 27 a 30, um conjunto de quatro fichas é escolhido de uma caixa contendo cinco fichas vermelhas e sete fichas pretas.

- ★27. Encontre o número de conjuntos de quatro fichas.
28. Encontre o número de conjuntos nos quais duas fichas são vermelhas e duas são pretas.
29. Encontre o número de conjuntos composto por todas as fichas vermelhas ou todas as fichas pretas.
30. Encontre o número de conjuntos com três ou quatro fichas pretas.

Os Exercícios 31 a 34 referem-se a uma rede de computadores com 60 nós.

31. A rede é projetada para resistir à falha de quaisquer dois nós. De quantas maneiras esse tipo de falha pode ocorrer?
- ★32. De quantas maneiras podem falhar um ou dois nós?
33. Se um nó falhar, de quantas maneiras podemos selecionar sete nós, sem que estes sejam quaisquer dos nós que falharam?
34. Se dois nós falharem, de quantas maneiras podemos selecionar sete nós de forma que eles incluam exatamente um dos nós que falharam?

Nos Exercícios 35 a 38, um comitê do congresso com três integrantes precisa ser selecionado dentre cinco democratas, três republicanos e quatro independentes.

- ★35. De quantas maneiras o comitê pode ser escolhido?
36. De quantas maneiras o comitê pode ser escolhido, se precisar incluir pelo menos um independente?
- ★37. De quantas maneiras podem ser escolhidos comitês que não incluam democratas e republicanos simultaneamente?
38. De quantas maneiras o comitê pode ser escolhido, se precisar ter pelo menos um democrata e um republicano?

Nos Exercícios 39 a 42, uma anfitriã deseja convidar seis pessoas para o jantar de uma lista de 14 amigos.

- ★39. De quantas maneiras ela pode escolher seus convidados?
40. De quantas maneiras ela pode escolher seus convidados, se seis deles são chatos e seis são interessantes e ela deseja ter pelo menos um de cada?
41. De quantas maneiras ela pode escolher seus convidados, se duas de suas amigas não se suportam, e uma não virá se a outra vier?
42. De quantas maneiras ela pode escolher seus convidados, se duas de suas amigas forem muito amigas e uma não for sem a outra?
- ★43. Vinte e cinco pessoas, incluindo Simon e Yuan, são candidatos a um comitê de cinco componentes. Se o comitê precisa incluir Simon e Yuan, de quantas maneiras o comitê pode ser selecionado?
44. Um estudante precisa selecionar cinco dentre 12 cadeiras para cursar no próximo período, mas uma das cadeiras precisa ser ou história americana ou literatura inglesa. De quantas maneiras o estudante pode escolher as cadeiras?

45. Em uma mão com cinco cartas tiradas de um baralho de 52 cartas, quantas formas existem de se ter quatro ases e uma carta de paus?
46. Em uma mão com cinco cartas tiradas de um baralho de 52 cartas, quantas formas existem de se ter três valetes e duas cartas de copas?
- ★47. a. Quantas permutações distintas existem com as letras da palavra HAWAIIAN?
b. Quantas dessas começam por H?
48. a. Quantas permutações distintas é possível conseguir com as letras da palavra APALACHICOLA?
b. Quantas dessas têm os dois Ls juntos?
49. Uma livraria exhibe uma prateleira com cinco, três e quatro cópias, respectivamente, dos três livros mais vendidos. Quantos arranjos diferentes desses livros existem, se os livros do mesmo título não puderem ser distinguidos entre si?
50. O Grupo Unido de Ação Divisiva usa palavras-código que são permutações de cinco letras. Você sabe que existem apenas 10 palavras-código. O que podemos dizer sobre letras repetidas nessas palavras-chave?
- ★51. Cinco pessoas em um jantar repartem um aperitivo. Se as opções são escargot, ovos e nabos, de quantas maneiras as seleções podem ser feitas?
52. Um florista tem rosas, cravos, lírios e bocas-de-leão em estoque. Quantos buquês diferentes de uma dúzia de flores podem ser feitos?
53. Cada um de quatro amigos compra um par de sapatos de corrida dentre uma seleção de uma loja com 14 tipos. De quantas maneiras eles podem ter feito as escolhas?
54. Uma cartela de bingo é distribuída a cada um dos 12 jogadores. De quantas maneiras isto pode ser feito se houver 15 tipos de cartas e puder haver repetições?
- ★55. Seis armazéns estão para receber carregamentos de um dos seguintes materiais: tintas, martelos ou telhas.
a. De quantas maneiras isto pode acontecer?
b. De quantas formas isto pode acontecer, se não tiver havido encomendas de tintas?
c. De quantos modos isto pode acontecer, se houver pelo menos um carregamento de cada item?
56. Em uma festa de aniversário, uma mãe serve um biscoito para cada uma das oito crianças. Há abundância de biscoitos de chocolate, de amendoim e de aveia.
a. De quantas maneiras cada criança pode escolher seu biscoito?
b. De quantas formas cada criança poderá escolher seu biscoito, se pelo menos um tipo de biscoito tiver acabado?
c. De quantos modos cada criança poderá escolher seu biscoito, se ninguém gostar de biscoitos de aveia?
d. De quantas maneiras cada criança poderá escolher um biscoito, se duas crianças se decidirem pelo biscoito de amendoim?
e. De quantas formas cada criança poderá escolher seu biscoito, se só houver dois biscoitos de chocolate?
- ★57. No dia das bruxas, 10 maçãs são distribuídas para sete crianças.
a. De quantas maneiras isto pode ser feito? (*Dica:* Apesar do problema dizer que as maçãs são distribuídas para as crianças, pense como atribuir os nomes das crianças às maçãs; assim um nome de criança pode ir para mais de uma maçã.)
b. De quantas maneiras isto pode ser feito se cada criança for receber pelo menos uma maçã?
58. Oito cofres idênticos são vendidos em um leilão de móveis antigos para três compradores.
a. De quantas maneiras isto pode ser feito? (Veja a dica para o Exercício 57.)
b. De quantas formas isto pode ser feito se o comprador A adquirir apenas um cofre?
- ★59. Quantas soluções inteiras não-negativas distintas existem para a equação

$$x_1 + x_2 + x_3 + x_4 = 10$$

onde a solução

$$\begin{array}{ccccccc}
& & & & 1 & & \\
& & & & 1 & 1 & \\
& & & 1 & 2 & 1 & \\
& & 1 & 3 & 3 & 1 & \\
& 1 & 4 & 6 & 4 & 1 & \\
1 & 5 & 10 & 10 & 5 & 1 & \\
& & & & \vdots & &
\end{array}$$

Se observarmos esta figura, veremos claramente que as arestas externas têm sempre o valor 1. Mas também podemos ver que qualquer elemento no interior do triângulo pode ser obtido pela soma dos dois elementos diretamente acima, na linha anterior (por exemplo, o primeiro 10 na quinta linha está abaixo do 4 e do 6 da quarta linha). Se isto for, de fato, sempre verdadeiro, temos que

$$C(n, k) = C(n-1, k-1) + C(n-1, k) \text{ para } 1 \leq k \leq n-1 \quad (1)$$

A equação (1) é conhecida como **fórmula de Pascal**.

Para demonstrar a fórmula de Pascal, começaremos pelo lado direito:

$$\begin{aligned}
C(n-1, k-1) + C(n-1, k) &= \frac{(n-1)!}{(k-1)![n-1-(k-1)]!} + \frac{(n-1)!}{k!(n-1-k)!} \\
&= \frac{(n-1)!}{(k-1)!(n-k)!} + \frac{(n-1)!}{k!(n-1-k)!} \\
&= \frac{k(n-1)!}{k!(n-k)!} + \frac{(n-1)!(n-k)}{k!(n-k)!}
\end{aligned}$$

(multiplicando o primeiro termo por k/k e o segundo por $(n-k)/(n-k)$)

$$= \frac{k(n-1)! + (n-1)!(n-k)}{k!(n-k)!}$$

(somando as frações)

$$= \frac{(n-1)![k + (n-k)]}{k!(n-k)!}$$

(simplificando o numerador)

$$\begin{aligned}
&= \frac{(n-1)!(n)}{k!(n-k)!} \\
&= \frac{n!}{k!(n-k)!} \\
&= C(n, k)
\end{aligned}$$

Outra forma, menos algébrica, para demonstrar a fórmula de Pascal envolve um argumento de contagem; razão pela qual é chamada de **prova combinatória**. Desejamos calcular $C(n, k)$, o número de maneiras de escolher k objetos dentre n . Existem duas formas de realizar tal escolha: o item 1 é escolhido como um dos k objetos, caso no qual restam $k-1$ escolhas a ser feitas dentre os $n-1$ restantes sem o item 1 e, por outro lado, existem $C(n-1, k-1)$ maneiras de realizar a escolha. Se, por outro lado, o item 1 não for escolhido, deveremos tomar as k escolhas dentre os $n-1$ objetos restantes, para o que existem $C(n-1, k)$ modos. O número total de possibilidades é obtido pelo número de possibilidades desses dois casos disjuntos.

Já que temos a fórmula de Pascal, podemos desenvolver a fórmula para $(a+b)^n$, conhecido como *teorema binomial*.

O Teorema Binomial e sua Demonstração

Na expansão de $(a + b)^2$,

$$a^2 + 2ab + b^2$$

os coeficientes são 1, 2 e 1, que é a linha 2 do triângulo de Pascal.

PRÁTICA 38

Encontre a expansão de $(a + b)^3$ e $(a + b)^4$, e compare seus coeficientes com as linhas 3 e 4 do triângulo de Pascal.

Os coeficientes das expansões de $(a + b)^2$, $(a + b)^3$ e $(a + b)^4$ sugere um resultado geral, isto é, que os coeficientes da expansão de $(a + b)^n$ sejam os valores da linha n do triângulo de Pascal. Isto é conhecido como o teorema binomial.

Teorema Binomial

Para todo inteiro não-negativo n ,

$$(a + b)^n = C(n, 0)a^n b^0 + C(n, 1)a^{n-1}b^1 + C(n, 2)a^{n-2}b^2 + \dots + C(n, k)a^{n-k}b^k + \dots + C(n, n-1)a^1b^{n-1} + C(n, n)a^0b^n \quad (2)$$

Como o teorema binomial é enunciado para todo inteiro não-negativo n , uma prova por indução parece apropriada. Para a base, $n = 0$, a equação (2) se torna

$$(a + b)^0 = C(0, 0)a^0b^0$$

que é

$$1 = 1$$

Uma vez que este resultado é obviamente verdadeiro, a base da indução está verificada.

Como hipótese de indução, assumiremos

$$(a + b)^k = C(k, 0)a^k b^0 + C(k, 1)a^{k-1}b^1 + \dots + C(k, k-1)a^1b^{k-1} + C(k, k)a^0b^k$$

Consideremos agora

$$\begin{aligned} (a + b)^{k+1} &= (a + b)^k(a + b) = (a + b)^k a + (a + b)^k b \\ &= [C(k, 0)a^k b^0 + C(k, 1)a^{k-1}b^1 + \dots + C(k, k-1)a^1b^{k-1} + C(k, k)a^0b^k]a \\ &\quad + [C(k, 0)a^k b^0 + C(k, 1)a^{k-1}b^1 + \dots + C(k, k-1)a^1b^{k-1} + C(k, k)a^0b^k]b \end{aligned}$$

(pela hipótese de indução)

$$\begin{aligned} &= C(k, 0)a^{k+1}b^0 + C(k, 1)a^k b^1 + \dots + C(k, k-1)a^2b^{k-1} \\ &\quad + C(k, k)a^1b^k + C(k, 0)a^k b^1 + C(k, 1)a^{k-1}b^2 \\ &\quad + \dots + C(k, k-1)a^1b^k + C(k, k)a^0b^{k+1} \\ &= C(k, 0)a^{k+1}b^0 + [C(k, 0) + C(k, 1)]a^k b^1 + [C(k, 1) + C(k, 2)]a^{k-1}b^2 \\ &\quad + \dots + [C(k, k-1) + C(k, k)]a^1b^k + C(k, k)a^0b^{k+1} \end{aligned}$$

(agrupando os termos semelhantes)

$$\begin{aligned} &= C(k, 0)a^{k+1}b^0 + C(k+1, 1)a^k b^1 + C(k+1, 2)a^{k-1}b^2 \\ &\quad + \dots + C(k+1, k)a^1b^k + C(k, k)a^0b^{k+1} \end{aligned}$$

(usando a fórmula de Pascal)

$$= C(k+1, 0)a^{k+1}b^0 + C(k+1, 1)a^k b^1 + C(k+1, 2)a^{k-1}b^2 \\ + \dots + C(k+1, k)a^1 b^k + C(k+1, k+1)a^0 b^{k+1}$$

(porque $C(k, 0) = 1 = C(k+1, 0)$ e $C(k, k) = 1 = C(k+1, k+1)$)

Isto conclui a prova indutiva do teorema binomial.

O teorema binomial também pode ter uma prova combinatória. Escrevendo $(a+b)^n$ como $(a+b)(a+b) \dots (a+b)$ (n fatores), sabemos (através da propriedade distributiva) que seu resultado é conseguido ao multiplicar-se cada termo em um fator por um termo de todos os demais fatores. Por exemplo, usando b como o termo de k fatores e a como o termo dos demais $n-k$ fatores, produz a expressão $a^{n-k}b^k$. Usando o fator b de um conjunto diferente de k fatores e a de um conjunto diferente de $n-k$ fatores restantes, também produz $a^{n-k}b^k$. Quantos termos desses existem? Existem $C(n, k)$ diferentes maneiras de selecionar k fatores dos quais usaremos b ; portanto existem $C(n, k)$ termos deste tipo. Após agrupar esses termos, o coeficiente de $a^{n-k}b^k$ será $C(n, k)$. Variando k de 0 a n , o resultado é a soma dos termos no teorema binomial.

Devido a seu uso no teorema binomial, a expressão $C(n, r)$ também é chamada de **coeficiente binomial**.

Aplicando o Teorema Binomial

EXEMPLO 58 Usando o teorema binomial podemos escrever a expressão de $(x-3)^4$ como mostrado a seguir:

$$(x-3)^4 = C(4, 0)x^4(-3)^0 + C(4, 1)x^3(-3)^1 + C(4, 2)x^2(-3)^2 \\ + C(4, 3)x^1(-3)^3 + C(4, 4)x^0(-3)^4 \\ = x^4 + 4x^3(-3) + 6x^2(9) + 4x(-27) + 81 \\ = x^4 - 12x^3 + 54x^2 - 108x + 81$$

PRÁTICA 39 Encontre a expansão de $(x+1)^5$ usando o teorema binomial.

O teorema binomial nos diz que o termo $k+1$ da expansão de $(a+b)^n$ é $C(n, k)a^{n-k}b^k$. Isto nos permite encontrar termos individuais da expansão sem ter que calculá-la por inteiro.

PRÁTICA 40 Qual o quinto termo da expansão de $(x+y)^7$?

Usando determinados valores de a e b no teorema, podemos encontrar certas identidades.

EXEMPLO 59 Seja $a = b = 1$ no teorema binomial. Então

$$(1+1)^n = C(n, 0) + C(n, 1) + \dots + C(n, k) + \dots + C(n, n)$$

ou

$$2^n = C(n, 0) + C(n, 1) + \dots + C(n, k) + \dots + C(n, n) \quad (3)$$

Na verdade, a equação (3) poderia ter sido demonstrada através de argumentos de combinatória. O número $C(n, k)$, o número de maneiras de escolher k itens dentre n , pode ser pensado como o número de subconjuntos de k elementos de um conjunto com n elementos. O lado direito da equação (3) representa o número total de todos os subconjuntos (de todos os tamanhos) de um conjunto de n elementos. Mas já sabemos que o número total de subconjuntos é 2^n .

Revisão da Seção 3.5

Técnica

- Uso do teorema binomial para expandir uma binomial ou encontrar um termo particular de sua expansão.

Idéias Principais

O teorema binomial fornece uma forma de expandir uma expressão binomial sem realizar suas multiplicações.

Os coeficientes de uma binomial elevada a um inteiro não-negativo são combinações de n itens como indicado na linha n do triângulo de Pascal.

Exercícios 3.5

Nos Exercícios 1 a 8, encontre a expansão usando o teorema binomial.

★1. $(a + b)^5$

2. $(x + y)^6$

★3. $(a + 2)^5$

4. $(a - 4)^4$

★5. $(2x + 3y)^3$

6. $(3x - 1)^5$

7. $(2p - 3q)^4$

8. $(3x + \frac{1}{2})^5$

9. Encontre o quarto termo da expansão de $(a + b)^{10}$.

10. Encontre o sétimo termo da expansão de $(x - y)^{12}$.

★11. Encontre o sexto termo da expansão de $(2x - 3)^9$.

12. Encontre o quinto termo da expansão de $(3a + 2b)^7$.

★13. Encontre o último termo da expansão de $(x - 3y)^8$.

14. Encontre o último termo da expansão de $(ab + 3x)^6$.

★15. Encontre o terceiro termo da expansão de $(4x - 2y)^5$.

16. Encontre o quarto termo da expansão de $(3x - \frac{1}{2})^8$.

17. Use o teorema binomial (mais de uma vez) para expandir $(a + b + c)^3$.

18. Faça a expansão de $(1 + 0.1)^5$ a fim de computar $(1.1)^5$.

★19. Qual o coeficiente de x^3y^4 na expansão de $(2x - y + 5)^8$?

20. Qual o coeficiente de $x^5y^2z^2$ na expansão de $(x + y + 2z)^9$?

21. Prove que $C(n + 2, r) = C(n, r) + 2C(n, r - 1) + C(n, r - 2)$ para $2 \leq r \leq n$. (Dica: Use a fórmula de Pascal.)

22. Prove que

$$C(k, k) + C(k + 1, k) + \dots + C(n, k) = C(n + 1, k + 1) \text{ para } 0 < k \leq n$$

(Dica: Use indução para um n fixo e k arbitrário, bem como a fórmula de Pascal.)

23. Use o teorema binomial para provar que

$$C(n, 0) - C(n, 1) + C(n, 2) - \dots + (-1)^n C(n, n) = 0$$

★24. Use o teorema binomial para provar que

$$C(n, 0) + C(n, 1)2 + C(n, 2)2^2 + \dots + C(n, n)2^n = 3^n$$

25. a. Encontre a expansão de $(1+x)^n$.

b. Derive ambos os lados da equação obtida no item (a) em relação a x a fim de obter

$$n(1+x)^{n-1} = C(n, 1) + 2C(n, 2)x + 3C(n, 3)x^2 + \dots + nC(n, n)x^{n-1}$$

c. Prove que

$$C(n, 1) - 2C(n, 2) + 3C(n, 3) - \dots + nC(n, n) = n2^{n-1}$$

d. Prove que

$$C(n, 1) - 2C(n, 2) + 3C(n, 3) - 4C(n, 4) + \dots + (-1)^{n-1} nC(n, n) = 0$$

26. a. Prove que

$$\frac{2^{n+1} - 1}{n+1} = C(n, 0) + \frac{1}{2}C(n, 1) + \frac{1}{3}C(n, 2) + \dots + \frac{1}{n+1}C(n, n)$$

b. Prove que

$$\frac{1}{n+1} = C(n, 0) - \frac{1}{2}C(n, 1) + \frac{1}{3}C(n, 2) - \dots + (-1)^n \frac{1}{n+1}C(n, n)$$

(Dica: Integre ambos os lados da equação do item (a) do Exercício 25.)

Revisão do Cap. 3

Terminologia

árvore de decisão	código reutilizável	permutação
coeficiente binomial	diferença de conjuntos	ponteiro nulo (nil)
combinatória	dual de uma identidade de	princípio da Adição
combinação	conjuntos	princípio da Casa do Pombo
complemento de um conjunto	encapsulamento	princípio da Inclusão e Exclusão
conjunto contável	fatoria de n	princípio da Multiplicação
conjunto das partes	fórmula de Pascal	produto cartesiano (produto
conjunto denumerável	herança	cruzado) de conjuntos
conjunto fechado sob uma	interseção de conjuntos	prova por combinação
operação	lista encadeada	subconjunto
conjunto nulo	método da diagonalização de	subconjunto próprio
conjunto não-enumerável	Cantor	teorema binomial
conjunto universo	operação bem-definida	tipo abstrato de dados
conjunto vazio	operação binária	triângulo de Pascal
conjuntos disjuntos	operação unária	universo de discurso
conjuntos iguais	par ordenado	união de conjuntos

Autotestes

Responda às seguintes perguntas com verdadeiro ou falso.

Seção 3.1

1. O conjunto vazio é um subconjunto próprio de todos os conjuntos

2. Se A e B são conjuntos disjuntos, então $(A - B) \cup (B - A) = A \cup B$.
3. Se um conjunto tem n elementos, então seu conjunto das partes tem 2^n elementos.
4. Se uma operação binária $\circ$ em um conjunto S é bem-definida, então $x \circ y \in S$ para todo x e y em S .
5. O método de diagonalização de Cantor é uma forma de demonstrar que certos conjuntos são denumeráveis.

Seção 3.2

6. De acordo com o Princípio da Multiplicação, o número de possibilidades para uma seqüência de tarefas é o produto do número de possibilidades de cada tarefa separadamente.
7. O Princípio da Adição fornece o número total de ramos em uma árvore de decisão.
8. O Princípio da Adição só pode ser usado se as tarefas em questão tiverem conjuntos disjuntos de possibilidades.
9. O Princípio da Multiplicação diz que o número de elementos em $A \times B$ é igual ao número de elementos em A vezes o número de elementos em B .
10. Qualquer problema que precise de uma árvore de decisões para ser solucionado não pode ser resolvido pelo Princípio da Multiplicação.

Seção 3.3

11. O Princípio da Inclusão e Exclusão só pode ser usado para achar o número de elementos em $A \cup B$ se A e B forem conjuntos disjuntos.
12. O Princípio da Inclusão e Exclusão aplicado a dois conjuntos diz que o número de elementos na união menos o número de elementos na interseção é a soma do número de elementos em cada conjunto.
13. O Princípio da Inclusão e Exclusão aplica-se à união de qualquer número de conjuntos, desde que pelo menos um deles seja finito.
14. O Princípio da Casa do Pombo é uma forma de contar o número de elementos na união de conjuntos disjuntos, ou "casa de pombos".
15. O Princípio da Casa do Pombo garante que se houver oito pessoas em uma sala, pelo menos duas nasceram no mesmo dia de semana.

Seção 3.4

16. Uma permutação é um arranjo ordenado de objetos.
17. O número de combinações de r objetos n a n é menor que o número de permutações de r objetos n a n .
18. Para encontrar o número de maneiras que um subconjunto de r objetos pode ser selecionado dentre n objetos, usamos a fórmula $P(n, r)$.
19. O número de permutações das letras de uma palavra com três conjuntos de letras repetidas é $n!/3$.
20. A fórmula $C(r + n - 1, r)$ calcula o número de combinações de r objetos dentre n objetos onde os objetos podem ser usados repetidamente.

Seção 3.5

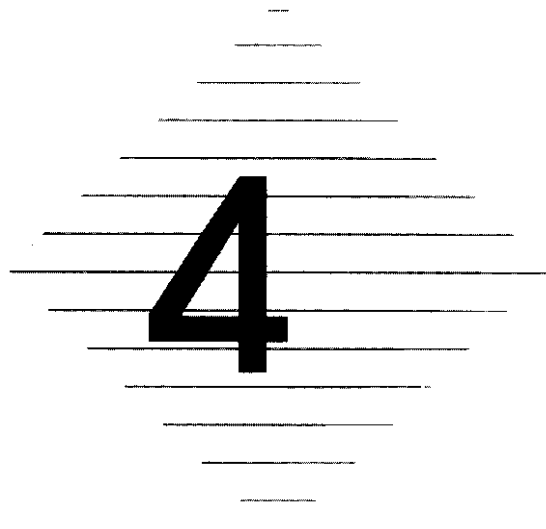
21. O triângulo de Pascal consiste em linhas que representam maneiras de arranjar r dentre n objetos para os vários r .
22. A fórmula de Pascal diz que um número "interior" no triângulo de Pascal é a soma dos dois números imediatamente acima dela no triângulo.
23. Na expansão de um binômio elevado à n -ésima potência, o k -ésimo termo é encontrado na linha k do triângulo de Pascal.
24. Um argumento combinatorial é aquele que é baseado em técnicas de contagem.
25. O coeficiente do sétimo termo da expansão de $(a + b)^{12}$ é dado pela expressão $C(12, 6)$.

No Computador

Para os Exercícios 1 a 7, escreva um programa de computador que produza a saída desejada para a entrada fornecida.

1. *Entrada:* Elementos de um conjunto finito S .
Saída: Elementos de $\mathcal{P}(S)$.
Algoritmo: Use recursão.

2. *Entrada:* Expressão aritmética na notação pós-fixa (veja o Exercício 31 na Seção 3.1)
Saída: Valor da expressão.
3. *Entrada:* Expressão aritmética na notação in-fixa (veja o Exercício 31 na Seção 3.1)
Saída: Forma pós-fixa da expressão.
 Resolva este problema de duas maneiras:
 - a. Assuma que a entrada faz sempre uso de parênteses
 - b. Não assuma que a entrada faz sempre uso de parênteses, mas faça com que o programa aplique a ordem apropriada de precedência de operadores (a ordem de precedência de operadores é: expressões entre parênteses primeiro, em seguida exponenciação, então multiplicação e divisão e, por último, adição e subtração).
4. *Entrada:* Valores para n e r , $0 \leq r \leq n$
Saída: Valor de $P(n, r)$
5. *Entrada:* Valores para n e r , $0 \leq r \leq n$
Saída: Valor de $C(n, r)$
6. *Entrada:* Valor de n
Saída: Todos os valores de $C(n, r)$, $0 \leq r \leq n$
7. *Entrada:* Valores para a , b e n
Saída: Valor de $(a + b)^n$
 - a. Use o teorema binomial para calcular o resultado.
 - b. Calcule $a + b$ e eleve este valor à n -ésima potência, compare sua resposta com a parte (a).
8. *Entrada:* Inteiro não-negativo n e um valor k , $1 \leq k \leq n + 1$
Saída: k -ésimo termo da expansão de $(a + b)^n$
9. Escreva um programa que permita ao usuário entrar um valor para n , $1 \leq n \leq 10$, e então aceite do usuário os valores necessários do lado esquerdo da equação (4) da Seção 3.3 (o Princípio da Inclusão e Exclusão), e calcule o valor de $|A_1 \cup \dots \cup A_n|$.
10. Escreva um programa para gerar um dado número de linhas do triângulo de Pascal. Resolva este problema de duas maneiras:
 - a. Use a definição do triângulo de Pascal (e, eventualmente, use a resposta do Exercício 5 como sub-rotina).
 - b. Use recursão e a fórmula de Pascal.
11. Implemente um tipo de dado abstrato CONJUNTO para subconjuntos de $S = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$, usando vetores de bits. Escreva procedimentos para criar um conjunto vazio, encontrar a união de dois conjuntos, encontrar a interseção de dois conjuntos e encontrar a diferença entre dois conjuntos.



Relações, Funções e Matrizes

Objetivos do Capítulo

Após estudar este capítulo, você estará apto a:

- Identificar pares ordenados de uma relação binária
- Verificar se uma determinada relação é reflexiva, simétrica, transitiva ou anti-simétrica
- Encontrar os fechos reflexivo, simétrico e transitivo de uma relação binária
- Reconhecer e traçar ordenações parciais
- Reconhecer uma relação de equivalência em um conjunto e descrever como ela particiona o conjunto em classes de equivalência
- Entender como as classes de equivalência podem ser objetos de interesse
- Entender o modelo entidade-relacionamento e o modelo relacional de um negócio
- Escrever as relações para um dado modelo relacional e identificar suas chaves primárias
- Realizar as operações select, project e join em um banco de dados relacional
- Criar pesquisas em banco de dados relacionais nas linguagens da álgebra relacional, SQL e cálculo relacional
- Entender por que as regras de integridade impõem limitações nas n -uplas em um banco de dados relacional

- Desenhar um diagrama PERT a partir de uma tabela de tarefas
- Determinar o tempo mínimo para completar o projeto e seu caminho crítico em um diagrama PERT
- Estender uma ordenação parcial em um conjunto finito para uma ordenação total através de uma ordenação topológica
- Determinar se uma relação é ou não uma função
- Verificar se uma função é sobrejetiva
- Verificar se uma função é injetiva
- Calcular o número de funções, de funções sobrejetivas e de funções injetivas de um conjunto finito em outro
- Criar composições de funções
- Manipular a notação cíclica para as funções de permutação
- Calcular o número de permutações sem pontos fixos de um conjunto finito
- Determinar se uma função tem inversa e qual é esta função inversa
- Entender a ordem de grandeza como uma medida relativa da taxa de crescimento da função
- Realizar operações aritméticas em matrizes de dimensões apropriadas
- Realizar operações da aritmética booleana em matrizes booleanas de dimensões apropriadas

Os elementos de um conjunto ou os elementos de conjuntos diferentes freqüentemente apresentam ligações especiais entre si que podem ser descritas como uma *relação*. Estudaremos as relações na Seção 4.1, onde veremos que relações binárias (relações entre pares de elementos) podem ter várias propriedades. Um tipo de relação binária são as chamadas ordenações parciais; os elementos relacionados por uma ordenação parcial podem ser representados através de um grafo. Outro tipo de relação binária é a relação de equivalência; elementos relacionados por uma relação de equivalência podem ser agrupados em classes.

Relações n-árias constituem a base dos bancos de dados relacionais considerados na Seção 4.2. O uso das operações select, project e join de relações em um banco de dados nos permite realizar diversas pesquisas no banco de dados.

Uma ordenação parcial em um conjunto de tarefas, definidas como quais tarefas são pré-requisitos para outras, pode ser representada graficamente em um diagrama PERT. Um diagrama PERT pode ser usado para encontrar o tempo mínimo a fim de completar todo o projeto e determinar quais tarefas estão no caminho crítico. Uma ordenação to-

pológica transforma uma ordenação parcial em uma ordenação total que identifica uma ordem na qual as tarefas possam ser realizadas sequencialmente. Diagramas PERT e ordenações topológicas também serão apresentados na Seção 4.2.

Uma função é um tipo especial de relação binária. Funções, assim como relações, descrevem algumas situações do mundo real. As funções também podem ter propriedades especiais, como será discutido na Seção 4.3.

Na Seção 4.4 consideraremos matrizes e desenvolveremos uma aritmética para a manipulação das mesmas. Mais adiante, usaremos matrizes para representarem relações e grafos.

Seção 4.1 Relações

Relações Binárias

Se ouvirmos que duas pessoas, Henriqueta e Horácio, se relacionam, entenderemos que existe algum laço afetivo entre eles — que (Henriqueta, Horácio) distinguem-se dos demais pares ordenados de pessoas por haver uma relação (são parentes, namorados, amigos etc.) que Henriqueta e Horácio verificam. O análogo matemático é distinguir determinados pares ordenados de objetos dos demais porque seus elementos satisfazem alguma relação que os componentes dos demais pares, em geral, não satisfazem.

EXEMPLO 1 Sejam $S = \{1, 2\}$ e $T = \{2, 3\}$; então temos $S \times T = \{(1, 2), (1, 3), (2, 2), (2, 3)\}$. Se estivermos interessados na relação de igualdade, então $(2, 2)$ será o único par que se distinguirá no produto $S \times T$, isto é, o único par ordenado cujas componentes são iguais. Se estivermos interessados na propriedade do primeiro número ser menor do que o segundo, escolheremos os pares $(1, 2)$, $(1, 3)$ e $(2, 3)$ como os pares ordenados de $S \times T$ que se distinguem dos demais por apresentarem tal propriedade. •

No Exemplo 1, poderíamos selecionar os pares ordenados (x, y) dizendo que $x = y$ ou que $x < y$. Analogamente, a notação $x \rho y$ indica que o par ordenado (x, y) satisfaz à relação ρ . A relação ρ pode ser descrita com palavras ou simplesmente pela enumeração dos pares ordenados que a satisfazem.

EXEMPLO 2 Sejam $S = \{1, 2\}$ e $T = \{2, 3, 4\}$. Uma relação ρ no conjunto $S \times T = \{(1, 2), (1, 3), (1, 4), (2, 2), (2, 3), (2, 4)\}$ pode ser definida por $x \rho y$ se, e somente se, $x = \frac{1}{2}y$, de forma mais abreviada $x \rho y \Leftrightarrow x = \frac{1}{2}y$ para todo $(x, y) \in S \times T$. Opcionalmente, poderíamos ter definido a mesma ρ dizendo que $\{(1, 2), (2, 4)\}$ é o conjunto de pares ordenados que satisfazem ρ . •

Estamos falando de relações binárias, isto é, relações entre dois objetos. No Exemplo 2, uma maneira de definir a relação binária ρ é especificar um subconjunto de $S \times T$. Formalmente, uma relação binária é um subconjunto de $S \times T$.

Definição: Relação Binária

Dados os conjuntos S e T , uma relação binária em $S \times T$ é um subconjunto de $S \times T$.

Agora que sabemos que uma relação binária ρ é um subconjunto, vemos que

$$x \rho y \Leftrightarrow (x, y) \in \rho.$$

Normalmente, uma relação binária será definida através da descrição da relação ao invés de listarmos todos os seus pares ordenados. Isto porque a descrição nos fornece uma propriedade característica dos elementos da relação; isto é, é um predicado binário realizado por certos pares ordenados.

EXEMPLO 3 Sejam $S = \{1, 2\}$ e $T = \{2, 3, 4\}$. Seja ρ dada pela descrição $x \rho y \Leftrightarrow x + y$ for ímpar. Então $(1, 2) \in \rho$, $(1, 4) \in \rho$ e $(2, 3) \in \rho$. •

EXEMPLO 4 Sejam $S = \{1, 2\}$ e $T = \{2, 3, 4\}$. Se ρ for definida em $S \times T$ por $\rho = \{(2, 3), (2, 4)\}$, então $2 \rho 3$ e $2 \rho 4$ são verdadeiras, mas, por exemplo, $1 \rho 4$ não é. Neste caso ρ não tem uma descrição verbal óbvia. •

PRÁTICA 1

Para cada uma das seguintes relações binárias ρ em $\mathbb{N} \times \mathbb{N}$, determine quais dos pares ordenados apresentados pertencem a ρ :

- a. $x \rho y \leftrightarrow x = y + 1$; (2, 2), (2, 3), (3, 3), (3, 2)
- b. $x \rho y \leftrightarrow x$ divide y ; (2, 4), (2, 5), (2, 6)
- c. $x \rho y \leftrightarrow x$ é ímpar; (2, 3), (3, 4), (4, 5), (5, 6)
- d. $x \rho y \leftrightarrow x > y^2$; (1, 2), (2, 1), (5, 2), (6, 4), (4, 3)

Podemos definir relações n-árias generalizando a definição de relações binárias.

Definição: Relação n-ária

Dados os conjuntos $S_1, S_2, \dots, S_n$, uma **relação n-ária** em $S_1, S_2, \dots, S_n$ é um subconjunto de $S_1 \times S_2 \times \dots \times S_n$. Um caso especial de relação n-ária é uma **relação unária** ρ em um conjunto S , que é apenas um subconjunto particular de S . Um elemento $x \in S$ satisfaz ρ se e somente se x pertencer ao subconjunto.

Freqüentemente estaremos interessados em relações binárias ou n-árias onde todos os conjuntos dados são o mesmo conjunto S . Essas relações são chamadas de relações *no conjunto* S , como definimos a seguir.

Definição: Relações em um conjunto S

Uma **relação binária em um conjunto S** é um subconjunto de S^2 (o conjunto de pares ordenados de elementos de S). Analogamente, uma **relação n-ária em um conjunto S** é um subconjunto de S^n (um conjunto de n-uplas ordenadas de elementos de S).

Se p for uma relação binária em $S \times T$, então p consistirá em um conjunto de pares ordenados da forma $\{s, t\}$. Uma determinada primeira componente s e uma determinada segunda componente t podem ser relacionadas diversas vezes na relação. A relação é **um-para-um** (ou **injetiva** ou **biunívoca**) se cada primeira componente s e cada segunda componente t aparecem apenas uma vez na relação. A relação é **um-para-vários** se alguma primeira componente s aparece mais de uma vez; isto é, se um s faz par com mais de um t . Ela é dita **vários-para-um** (ou **unívoca**) se alguma segunda componente t fizer par com mais de um s . Finalmente, ela é dita **vários-para-vários** se pelo menos um s fizer par com mais de um t e pelo menos um t fizer par com mais de um s . A Fig. 4.1 ilustra essas quatro possibilidades. Perceba que nem todos os valores de S e de T precisam ser componentes de pares ordenados de p .

PRÁTICA 2

Identifique cada uma das relações em $S \times T$ apresentadas abaixo como sendo um-para-um, um-para-vários, vários-para-um e vários-para-vários, onde $S = \{2, 5, 7, 9\}$ e $T = \{3, 4, 5\}$.

- a. $\{(5, 3), (7, 5), (9, 3)\}$
- b. $\{(2, 4), (5, 5), (7, 3)\}$
- c. $\{(7, 4), (2, 5), (9, 4), (2, 3)\}$

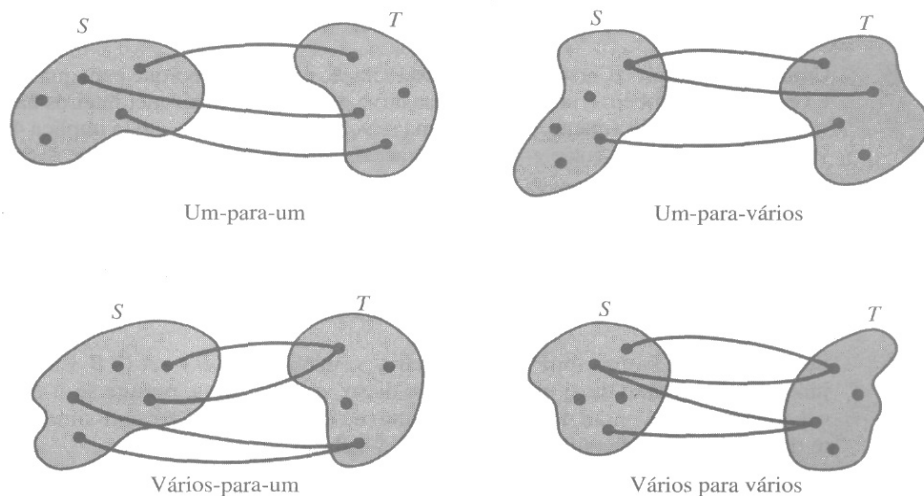


Figura 4.1

Suponha que B é o conjunto de todas as relações binárias em um dado conjunto S . Se ρ e σ pertencerem a B , então elas são subconjuntos de $S \times S$. Como tal, podemos realizar as operações de união, interseção, e complemento de conjuntos que resultam em novos subconjuntos de $S \times S$, isto é, novas relações binárias, que denotaremos por $\rho \cup \sigma$, $\rho \cap \sigma$ e ρ' , respectivamente. Desta forma,

$$\begin{aligned}x(\rho \cup \sigma)y &\leftrightarrow x\rho y \text{ ou } x\sigma y \\x(\rho \cap \sigma)y &\leftrightarrow x\rho y \text{ e } x\sigma y \\x\rho'y &\leftrightarrow \text{não } x\rho y\end{aligned}$$

PRÁTICA 3

Sejam ρ e σ duas relações binárias em $\mathbb{N}$ definidas por $x\rho y \leftrightarrow x = y$ e $x\sigma y \leftrightarrow x < y$. Forneça descrições verbais para (a), (b) e (c); apresente o conjunto definido em (d).

- Qual é a relação $\rho \cup \sigma$?
- Qual é relação ρ' ?
- Qual é a relação σ' ?
- Qual é a relação $\rho \cap \sigma$?

Os fatos que apresentamos a seguir sobre as operações de $\cup$, $\cap$ e $'$ de relações são conseqüências imediatas das identidades de conjuntos encontradas na Seção 3.1. O conjunto S^2 (que é ele próprio um subconjunto de S^2) é entendido aqui como uma relação binária em S .

- | | |
|---|---|
| 1a. $\rho \cup \sigma = \sigma \cup \rho$ | 1b. $\rho \cap \sigma = \sigma \cap \rho$ |
| 2a. $(\rho \cup \sigma) \cup \gamma = \rho \cup (\sigma \cup \gamma)$ | 2b. $(\rho \cap \sigma) \cap \gamma = \rho \cap (\sigma \cap \gamma)$ |
| 3a. $\rho \cup (\sigma \cap \gamma) = (\rho \cup \sigma) \cap (\rho \cup \gamma)$ | 3b. $\rho \cap (\sigma \cup \gamma) = (\rho \cap \sigma) \cup (\rho \cap \gamma)$ |
| 4a. $\rho \cup \emptyset = \rho$ | 4b. $\rho \cap S^2 = \rho$ |
| 5a. $\rho \cup \rho' = S^2$ | 5b. $\rho \cap \rho' = \emptyset$ |

Propriedades das Relações

Uma relação binária em um conjunto S pode ter certas propriedades. Por exemplo, a relação ρ da igualdade em S , $(x, y) \in \rho \leftrightarrow x = y$, tem três propriedades: (1) para qualquer $x \in S$, $x = x$, isto é, $(x, x) \in \rho$; (2) para quaisquer $x, y \in S$, se $x = y$, então $y = x$, isto é, $(x, y) \in \rho \rightarrow (y, x) \in \rho$; e (3) para quaisquer $x, y, z \in S$, se $x = y$ e $y = z$, então $x = z$, isto é, $[(x, y) \in \rho \text{ e } (y, z) \in \rho] \rightarrow (x, z) \in \rho$. Estas três propriedades fazem da igualdade uma relação reflexiva, simétrica e transitiva.

Definição: Relações Reflexivas, Simétricas e Transitivas

Seja ρ uma relação binária em S . Então

ρ reflexiva	significa:	$(\forall x)(x \in S \rightarrow (x, x) \in \rho)$
ρ simétrica	significa:	$(\forall x)(\forall y)(x \in S \wedge y \in S \wedge (x, y) \in \rho \rightarrow (y, x) \in \rho)$
ρ transitiva	significa:	$(\forall x)(\forall y)(\forall z)(x \in S \wedge y \in S \wedge z \in S \wedge (x, y) \in \rho \wedge (y, z) \in \rho \rightarrow (x, z) \in \rho)$

EXEMPLO 5

Considere a relação $\leq$ no conjunto $\mathbb{N}$. Esta relação é reflexiva porque para qualquer inteiro não-negativo x , $x \leq x$ é verdadeira. Também é uma relação transitiva porque para quaisquer inteiros não-negativos x, y e z , se $x \leq y$ e $y \leq z$ então $x \leq z$. No entanto, $\leq$ não é uma relação simétrica; $3 \leq 4$ não implica $4 \leq 3$. De fato, para quaisquer $x, y \in \mathbb{N}$, se $x \leq y$ e $y \leq x$, então $y = x$. Esta característica é descrita dizendo-se que $\leq$ é uma relação anti-simétrica. ♦

Definição: Relação Anti-simétrica

Seja ρ uma relação binária no conjunto S . Então ρ é dita **anti-simétrica** se, e somente se,

$$(\forall x)(\forall y)(x \in S \wedge y \in S \wedge (x, y) \in \rho \wedge (y, x) \in \rho \rightarrow x = y)$$

EXEMPLO 6

Seja $S = \mathcal{P}(\mathbb{N})$. Defina uma relação binária ρ em S por $A\rho B \leftrightarrow A \subseteq B$. Então ρ é reflexiva porque todo conjunto é subconjunto de si próprio. Além disso, ρ é transitiva, porque se A é um subconjunto de B e B é um subconjunto de C , então A é um subconjunto de C . Finalmente, ρ é anti-simétrica porque se A é um subconjunto de B e B é um subconjunto de A , então A e B são iguais. •

Todas as quatro propriedades de relações envolvem o conectivo de implicação. Os usos do quantificador universal indicam que as implicações precisam se verificar para escolhas arbitrárias das variáveis. Lem-

bre-se de que para provar que uma implicação é verdadeira, supomos seu antecedente verdadeiro e provamos que o conseqüente também o é. Para a propriedade reflexiva, o antecedente apenas escolhe um elemento arbitrário em S ; o conseqüente diz que este elemento deve estar relacionado a ele mesmo. Para que uma relação p em um conjunto S seja reflexiva, todo elemento no conjunto precisa estar relacionado a ele próprio, o que indica que certos pares ordenados devem pertencer a p .

No entanto, nas propriedades simétrica, transitiva e anti-simétrica, o antecedente não diz apenas que o elemento pertence a S . Para demonstrar que uma relação é simétrica, por exemplo, precisamos mostrar que se x e y são elementos arbitrários de S e se, além disso, x se relaciona a y então y deve relacionar-se a x . Isto diz que, se certos pares ordenados pertencem a p , alguns outros pares ordenados também devem pertencer a p a fim de que esta seja uma relação simétrica. Em outras palavras, conhecer S é imperativo para a determinação da reflexividade, enquanto que, para as demais propriedades, basta examinarmos os pares ordenados em p .

PRÁTICA 4

Seja $S = \{1, 2, 3\}$

- Se uma relação p em S é reflexiva, quais pares ordenados devem pertencer a p ?
- Se uma relação p em S é simétrica, quais pares ordenados devem pertencer a p ? (Esta pergunta é uma armadilha — veja a resposta ao final do livro.)
- Se uma relação p em S é simétrica e se $(a, b) \in p$, então quais outros pares ordenados devem pertencer a p ?
- Se uma relação p em S é anti-simétrica e se $(a, b) \in p$ e $(b, a) \in p$, o que podemos afirmar? •

As propriedades de simetria e anti-simetria de relações binárias não são exatamente opostas. *Anti-simétrica* não significa "não-simétrica". Uma relação não é simétrica se algum (x, y) pertencer à relação de forma que (y, x) não pertença. Mais formalmente, a "não-simetria" significa

$$\begin{aligned} & ((\forall x)(\forall y)[x \in S \wedge y \in S \wedge (x, y) \in p \rightarrow (y, x) \in p])' \\ & \leftrightarrow (\exists x)(\exists y)[x \in S \wedge y \in S \wedge (x, y) \in p \wedge (y, x) \notin p] \\ & \leftrightarrow (\exists x)(\exists y)[(x \in S \wedge y \in S \wedge (x, y) \in p) \wedge (y, x) \notin p] \\ & \leftrightarrow (\exists x)(\exists y)[(x \in S \wedge y \in S \wedge (x, y) \in p) \wedge (y, x) \notin p] \end{aligned}$$

As relações podem, portanto, ser simétricas e não ser anti-simétricas, ser anti-simétricas e não ser simétricas, ser simétricas e anti-simétricas ou não ser nenhuma das duas.

A relação de igualdade em um conjunto S é tanto simétrica como anti-simétrica. No entanto, a relação de igualdade em S (ou um subconjunto desta relação) é a única relação que contém, ao mesmo tempo, essas duas propriedades. Para ilustrar, suponhamos que p é uma relação simétrica e anti-simétrica em S e seja $(x, y) \in p$. Por simetria, segue que $(y, x) \in p$. Mas pela anti-simetria, $x = y$. Portanto, apenas os elementos iguais podem ser relacionados. A relação $p = \{(1, 2), (2, 1), (1, 3)\}$ no conjunto $S = \{1, 2, 3\}$ não é nem simétrica — $(1, 3)$ pertence à relação, mas $(3, 1)$ não — nem anti-simétrica — $(1, 2)$ e $(2, 1)$ pertencem à relação e $1 \neq 2$.

PRÁTICA 5

Verifique se as relações binárias nos conjuntos abaixo são reflexivas, simétricas, anti-simétricas e transitivas:

- $S = \mathbb{N}$; $x p y \leftrightarrow x + y$ é par
- $S = \mathbb{N}$; $x p y \leftrightarrow x$ divide y
- $S =$ conjunto de todas as linhas do plano; $x p y \leftrightarrow x$ é paralela a y ou x coincide com y
- $S = \mathbb{N}$; $x p y \leftrightarrow x = y^2$
- $S = \{0, 1\}$; $x p y \leftrightarrow x = y^2$
- $S = \{x \mid x \text{ é uma pessoa que mora na Bahia}\}$; $x p y \leftrightarrow x$ é mais velho que y
- $S = \{x \mid x \text{ é um aluno da sua sala}\}$; $x p y \leftrightarrow x$ senta na mesma fileira que y
- $S = \{1, 2, 3\}$; $p = \{(1, 1), (2, 2), (3, 3), (1, 2), (2, 1)\}$ •

EXEMPLO 7

A discussão sobre recursão em Prolog (Seção 1.5) mostrou que podemos usar uma regra recursiva quando o predicado a ser descrito é herdado de um objeto para o próximo. O predicado *na-cadeia-alimentar* descrito naquela seção tem essa propriedade porque

$$na-cadeia-alimentar(x, y) \wedge na-cadeia-alimentar(y, z) \rightarrow na-cadeia-alimentar(x, z)$$

Agora constatamos que isto é apenas a propriedade transitiva. •

Fechos de uma Relação

Se uma relação p em um conjunto S não tem uma certa propriedade, podemos tentar estender p a fim de obter uma relação p^* em S que tenha a propriedade. Por "estender" devemos entender que a nova relação p^* conterá todos os pares ordenados que p contém mais os pares ordenados adicionais necessários para que a propriedade

desejada se verifique. Portanto, $p \subseteq p^*$. Se p^* for o menor desses conjuntos, então p^* é chamado de *fecho* de p com respeito à propriedade em questão.

Definição: Fecho de uma Relação

Uma relação binária p^* em um conjunto S é o **fecho de uma relação p em S com respeito à propriedade P** se

1. p^* tem a propriedade P
2. $p \subseteq p^*$
3. p^* é um subconjunto de qualquer outra relação em S que inclui p e tem a propriedade P .

Podemos considerar o **fecho reflexivo**, o **fecho simétrico** e o **fecho transitivo** de uma relação em um conjunto. Naturalmente, se a relação já realiza uma propriedade, ela é seu próprio fecho com respeito a esta propriedade.

EXEMPLO 8 Seja $S = \{1, 2, 3\}$ e $p = \{(1, 1), (1, 2), (1, 3), (3, 1), (2, 3)\}$. Então p não é reflexiva, não é simétrica e não é transitiva. O fecho de p com respeito à reflexividade é

$$\{(1,1), (1, 2), (1, 3), (3, 1), (2, 3), (2, 2), (3, 3)\}$$

Esta relação é reflexiva e contém p . Além disso, qualquer relação reflexiva em S deve conter os novos pares ordenados que incluímos — $(2, 2)$ e $(3, 3)$ —, de forma que não pode haver relação reflexiva menor do que isto; ou seja, qualquer relação reflexiva contendo p deve conter a relação acima.

O fecho de p com relação à simetria é

$$\{(1, 1), (1, 2), (1, 3), (3, 1), (2, 3), (2, 1), (3, 2)\}$$

Neste caso também está claro que incluímos apenas os pares necessários — $(2, 1)$ e $(3, 2)$ — para que a relação seja simétrica.

Para os fechos reflexivo e simétrico, temos apenas que verificar os pares já em p a fim de encontrar quais pares precisamos incluir (partindo da premissa de que sabemos qual o conjunto S). Os fechos que podem ser encontrados em um único passo são os fechos reflexivo e simétrico. O fecho transitivo demanda uma série de passos para ser encontrado. Verificando os pares ordenados de nosso exemplo p , vemos que precisamos incluir $(3, 2)$ (devido aos pares $(3, 1)$ e $(1, 2)$), $(3, 3)$ (devido aos pares $(3, 1)$ e $(1, 3)$) e $(2, 1)$ (devido a $(2, 3)$ e $(3, 1)$). Isto nos dá a relação

$$\{(1, 1), (1, 2), (1, 3), (3, 1), (2, 3), (3, 2), (3, 3), (2, 1)\}$$

No entanto, esta relação ainda não é transitiva. Pois, devido ao novo par $(2, 1)$ e ao par original $(1, 2)$, devemos incluir o par $(2, 2)$. Isto nos dá a relação

$$\{(1, 1), (1, 2), (1, 3), (3, 1), (2, 3), (3, 2), (3, 3), (2, 1), (2, 2)\}$$

que é transitiva e é também a menor relação transitiva que contém p . •

Como mostramos no Exemplo 8, uma maneira de determinar o fecho transitivo de uma relação é verificar os pares ordenados na relação original, incluir novos pares se necessário, verificar a relação obtida, incluindo novos pares se necessário e assim por diante, até que tenhamos obtido uma relação transitiva. Este é um método de força bruta e veremos um algoritmo menor no Cap. 5, onde entenderemos o fecho transitividade de uma relação binária como a "alcançabilidade" em um grafo direcionado, o que tem diversas aplicações.

PRÁTICA 6 Faz sentido pensarmos no fecho anti-simétrico de uma relação em um conjunto? Justifique. •

PRÁTICA 7 Encontre os fechos reflexivo, simétrico e transitivo da relação $\{(a, a), (b, b), (c, c), (a, c), (a, d), (b, d), (c, a), (d, a)\}$ no conjunto $S = \{a, b, c, d\}$. •

No restante desta seção estaremos interessados em dois tipos de relação binária que são caracterizadas por quais propriedades (reflexividade, simetria, anti-simetria e transitividade) elas satisfazem.

Ordenação Parcial

Definição: Ordenação Parcial

Uma relação binária em um conjunto S que seja reflexiva, anti-simétrica e transitiva é chamada de **ordenação parcial** em S .

Nos exemplos anteriores e da Prática 5, já vimos os seguintes casos de ordenações parciais:

Em $\mathbb{N}$, $x \rho y \leftrightarrow x \leq y$.
 Em $\mathcal{P}(\mathbb{N})$, $A \rho B \leftrightarrow A \subseteq B$.
 Em $\mathbb{N}$, $x \rho y \leftrightarrow x \text{ divide } y$.
 Em $\{0, 1\}$, $x \rho y \leftrightarrow x = y^2$.

Se p é uma ordenação parcial em S , então o par ordenado (S, p) é chamado de um conjunto **parcialmente ordenado** (também conhecido como **poset**). Denotaremos um conjunto arbitrário parcialmente ordenado por $(S, \leq)$. Em qualquer caso particular, $\leq$ tem um significado parecido com "menor ou igual a", "é subconjunto de", "divide" ou coisa parecida.

Seja $(S, \leq)$ um conjunto parcialmente ordenado, e seja $A \subseteq S$. Então $\leq$ é um conjunto de pares ordenados de S , alguns dos quais podem ser pares ordenados de A . Se tomarmos de $\leq$ os pares ordenados de elementos de A , este novo conjunto é chamado de **restrição** de $\leq$ a A e constitui uma ordenação parcial em A . (Você percebe por que as três propriedades necessárias continuam a valer?) Por exemplo, uma vez que sabemos que a relação "x divide y" é uma ordenação parcial em $\mathbb{N}$, automaticamente sabemos que "x divide y" é uma ordenação parcial de $\{1, 2, 3, 6, 12, 18\}$.

Agora é interessante que se introduza alguma terminologia referente aos conjuntos parcialmente ordenados. Seja $(S, \leq)$ um conjunto parcialmente ordenado. Se $x \leq y$, então ou $x = y$ ou $x \neq y$; se $x \neq y$ e $x \leq y$, escrevemos $x < y$ e dizemos que x é um **predecessor** de y ou que y é um **sucessor** de x . Um dado y pode ter diversos predecessores, mas, se $x < y$ e não há z tal que $x < z < y$, então dizemos que x é um **predecessor imediato** de y .

PRÁTICA 8

Considere a relação "x divide y" em $\{1, 2, 3, 6, 12, 18\}$.

- Escreva os pares ordenados (x, y) desta relação.
- Escreva todos os predecessores de 6.
- Escreva todos os predecessores imediatos de 6.

Se S é finito, então podemos descrever um conjunto parcialmente ordenado $(S, \leq)$ visualmente através de um *grafo*. Cada elemento de S é representado por um ponto, chamado de **nó**, **nodo** ou **vértice** do grafo. Se x é um predecessor imediato de y , então o nó para y é desenhado acima do nó para x e os dois nós são ligados por um segmento de linha.

EXEMPLO 9

Considere $\mathcal{P}(\{1, 2\})$ com respeito à relação de inclusão em conjuntos. Este é um conjunto parcialmente ordenado. (Já sabemos que $(\mathcal{P}(\mathbb{N}), \subseteq)$ é um conjunto parcialmente ordenado.) Os elementos de $\mathcal{P}(\{1, 2\})$ são $\emptyset$, $\{1\}$, $\{2\}$ e $\{1, 2\}$. A relação binária $\subseteq$ consiste nos seguintes pares ordenados:

$(\emptyset, \emptyset), (\{1\}, \{1\}), (\{2\}, \{2\}), (\{1, 2\}, \{1, 2\}), (\emptyset, \{1\}), (\emptyset, \{2\}),$
 $(\emptyset, \{1, 2\}), (\{1\}, \{1, 2\}), (\{2\}, \{1, 2\})$

O grafo deste conjunto parcialmente ordenado é mostrado na Fig. 4.2. Perceba que, apesar de $\emptyset$ não ser um predecessor imediato de $\{1, 2\}$, ele é um predecessor (o que é mostrado no grafo pela cadeia de segmentos de linhas que ligam estes dois vértices).

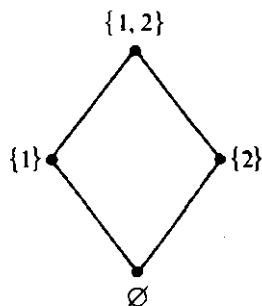


Figura 4.2

PRÁTICA 9

Desenhe o grafo da relação "x divide y" em $j \in \{1, 2, 3, 6, 12, 18\}$.

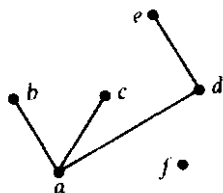


Figura 4.3



Figura 4.4

O grafo de um conjunto parcialmente ordenado (também chamado de **diagrama de Hasse**) contém todas as informações sobre uma ordenação parcial. Podemos reconstruir o conjunto de pares ordenados que formam a ordenação parcial com base apenas no grafo. Portanto, dado o grafo da Fig. 4.3 de uma ordenação parcial $\leq$ em um conjunto $S = \{a, b, c, d, e, f\}$, podemos concluir que $\leq$ é o conjunto

$$\{(a, a), \{b, b\}, \{c, c\}, \{d, d\}, \{e, e\}, \{f, f\}, (a, b), (a, c), (a, d), (a, e), (d, e)\}$$

No Cap. 5 veremos como representar diversas relações binárias além da ordenação parcial de forma gráfica.

Dois elementos de S podem não se relacionar em uma ordenação parcial em S . No Exemplo 9, $\{1\}$ e $\{2\}$ não têm relação entre si, da mesma forma que 2, 3, 12 e 18 na Prática 9. Na Fig. 4.3, f não se relaciona com qualquer outro elemento. Uma ordenação parcial na qual todo elemento do conjunto está relacionado com todos os demais elementos é chamada de **ordenação total** ou **cadeia**. O grafo de uma ordenação total tem a forma do mostrado na Fig. 4.4. A relação $\leq$ em $\mathbb{N}$ é uma ordenação total.

Novamente, seja $(S, \leq)$ um conjunto parcialmente ordenado. Se houver um $y \in S$ com $y \leq x$ para todo $x \in S$, então y é um **elemento mínimo** do conjunto parcialmente ordenado. Um elemento mínimo, quando houver, é único. Se y e z forem ambos elementos mínimos, então $y \leq z$, pois y é elemento mínimo e z é elemento mínimo. Da propriedade anti-simétrica decorre que $y = z$. Um elemento $y \in S$ é **minimal** se não houver outro $x \in S$ com $x \leq y$. No diagrama de Hasse um elemento mínimo se encontra abaixo de todos os outros, enquanto que um elemento minimal é aquele que não tem elementos abaixo dele com os quais se relacione. Definições análogas aplicam-se a elementos máximo e maximal.

PRÁTICA 10

Defina **elemento máximo** e **elemento maximal** em um conjunto parcialmente ordenado $(S, \leq)$.

EXEMPLO 10

No conjunto parcialmente ordenado da Prática 9, 1 é tanto mínimo como minimal. Doze e dezoito são ambos maximais, mas não há elemento máximo.

Um elemento mínimo é sempre minimal e um elemento máximo é sempre maximal, mas as recíprocas não são verdadeiras (veja o Exemplo 10). Em um conjunto totalmente ordenado, no entanto, um elemento minimal é um elemento mínimo e um elemento maximal é um elemento máximo.

PRÁTICA 11

Desenhe o grafo de um conjunto parcialmente ordenado com quatro elementos no qual haja dois elementos minimais, mas não haja elemento mínimo, e dois elementos maximais, mas não haja elemento máximo, e onde todos os elementos se relacionem com outros dois elementos.

Ordenações parciais verificam as propriedades de reflexividade, anti-simetria e transitividade. Outro tipo de relação binária, que veremos a seguir, verifica um outro conjunto de propriedades.

Relações de Equivalência

Definição: Relação de Equivalência

Uma relação binária em um conjunto S que seja reflexiva, simétrica e transitiva é chamada de uma **relação de equivalência** em S .

Já vimos os seguintes exemplos de relações de equivalência:

Em qualquer conjunto S , $x \rho y \leftrightarrow x = y$.

Em $\mathbb{N}$, $x \rho y \leftrightarrow x + y$ é par.

No conjunto de todas as linhas do plano, $x \rho y \leftrightarrow x$ é paralela a y , ou x coincide com y .

Em $\{0, 1\}$, $x \rho y \leftrightarrow x = y^2$.

Em $\{x \mid x \text{ é um aluno de sua classe}\}$, $x \rho y \leftrightarrow x$ senta na mesma coluna que y .

Em $\{1, 2, 3\}$, $\rho = \{(1, 1), (2, 2), (3, 3), (1, 2), (2, 1)\}$.

Podemos ilustrar um aspecto importante de uma relação de equivalência em um conjunto, se observarmos o exemplo $S = \{x \mid x \text{ é um aluno de sua classe}\}$, $x \rho y \leftrightarrow$ "x senta na mesma coluna que y". Vamos indicar todos os alunos de S que se relacionam uns com os outros. Chegaremos à Fig. 4.5. Particionamos o conjunto S em subconjuntos de tal forma que todo aluno da turma pertence a um, e apenas a um, subconjunto.

Definição: Partição de um Conjunto

Uma **partição** de um conjunto S é uma coleção de subconjuntos disjuntos não vazios de S cuja união resulte S .

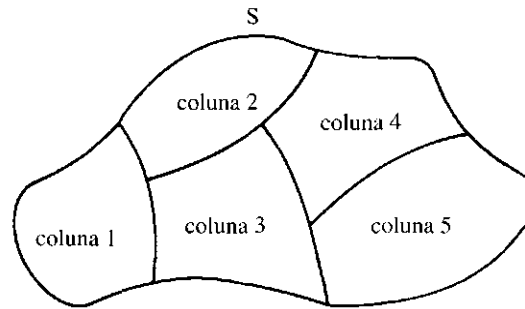


Figura 4.5

Qualquer relação de equivalência, como veremos, particiona o conjunto no qual é definida. Os subconjuntos que formam a partição, normalmente chamados de **blocos** da partição, são formados pelo grupamento dos elementos que se relacionam, como no caso acima.

Sejam p uma relação de equivalência em um conjunto S e $x \in S$, denotamos por $[x]$ o conjunto de todos os elementos de S que se relacionam a x , chamado de **classe de equivalência** de x . Assim

$$[x] = \{y \mid y \in S \wedge x \rho y\}$$

EXEMPLO 11 No caso em que $x \rho y \leftrightarrow$ “ x senta na mesma coluna que y ”, suponha que João, Carlos, José, Júlia e Maria sentem todos na coluna 3. Então $[\text{João}] = \{\text{João, Carlos, José, Júlia, Maria}\}$. Além disso, $[\text{João}] = [\text{José}] = [\text{Júlia}] = [\text{Maria}]$. Pode haver mais de um nome para uma dada classe de equivalência. •

Seja p uma relação de equivalência em S , então as classes de equivalência distintas de S formam uma partição de S . A fim de concordar com a definição de partição, precisamos mostrar que (1) a união das classes distintas resulta em S e (2) as classes distintas são disjuntas. Mostrar que a união das classes resulta em S é fácil, uma vez que é essencialmente uma igualdade de conjuntos; demonstramos a inclusão de conjuntos em ambas as direções. Cada classe de equivalência é um subconjunto de S , portanto, a união das classes também é um subconjunto de S . Para demonstrar a inclusão no outro sentido, seja $x \in S$. Então $x \rho x$ (reflexividade de p); daí $x \in [x]$, e qualquer elemento de S pertence a alguma classe de equivalência e, portanto, à união das classes.

Agora sejam $[x]$ e $[z]$ duas classes de equivalências distintas isto é, $[x] \neq [z]$. Precisamos mostrar que $[x] \cap [z] = \emptyset$. Faremos uma demonstração por contradição. Assumiremos, portanto, que $[x] \cap [z] \neq \emptyset$ e, neste caso, existe um $y \in S$ tal que $y \in [x] \cap [z]$.

$$\begin{array}{ll} y \in [x] \cap [z] & \text{(hipótese)} \\ y \in [x], y \in [z] & \text{(definição de } \cap \text{)} \\ x \rho y, z \rho y & \text{(definição de } [x] \text{ e } [z] \text{)} \\ x \rho y, y \rho z & \text{(simetria de } \rho \text{)} \\ x \rho z & \text{(transitividade de } \rho \text{)} \end{array}$$

O que nos permite mostrar que $[x] = [z]$; demonstraremos a inclusão de conjuntos em ambas as direções. Seja

$$q \in [z] \quad ([z] \neq \emptyset)$$

Então

$$\begin{array}{ll} z \rho q & \text{(definição de } [z] \text{)} \\ x \rho z & \text{(resultado obtido acima)} \\ x \rho q & \text{(transitividade de } \rho \text{)} \\ q \in [x] & \text{(definição de } [x] \text{)} \\ [z] \subseteq [x] & \text{(definição de } \subseteq \text{)} \\ [x] \subseteq [z] & \text{(Prática 12 abaixo)} \\ [x] = [z] & ([z] \subseteq [x] \text{ e } [x] \subseteq [z]) \end{array}$$

Portanto, $[x] = [z]$, o que é uma contradição pois $[x]$ e $[z]$ são distintas. Portanto, nossa hipótese estava errada; $[x] \cap [z] = \emptyset$ e classes de equivalência distintas são disjuntas.

PRÁTICA 12 Com base no argumento acima, demonstre que $[x] \subseteq [z]$. •

Mostramos que uma relação de equivalência em um conjunto determina uma partição. A recíproca também é verdadeira. Dada uma partição de um conjunto S , definimos uma relação ρ por $x \rho y \leftrightarrow$ "está no mesmo subconjunto da partição que y ."

PRÁTICA 13 Mostre que ρ , como definido acima, é uma relação de equivalência em S ; isto é, mostre que ρ é reflexiva, simétrica e transitiva. •

Nós demonstramos o seguinte fato sobre relações de equivalência.

Teorema de Relações de Equivalência e Partições

Uma relação de equivalência em um conjunto S determina uma partição de S , e uma partição de S determina uma relação de equivalência em S .

EXEMPLO 12 A relação de equivalência em $\mathbb{N}$ dada por

$$x \rho y \leftrightarrow x + y \text{ é par}$$

particiona $\mathbb{N}$ em duas classes de equivalência. Se x for um número par, então para qualquer número par y , $x + y$ é par e, portanto, $y \in [x]$. Todos os números pares formam uma classe de equivalência. Se x for um número ímpar e y for qualquer número ímpar, $x + y$ é par e $y \in [x]$. Todos os números ímpares formam a segunda classe de equivalência. A partição pode ser representada como na Fig. 4.6. Perceba novamente que uma classe de equivalência pode ter mais de um nome ou elemento representativo. Neste exemplo, $[2] = [8] = [1048] = \dots$; $[1] = [17] = [947] = \dots$

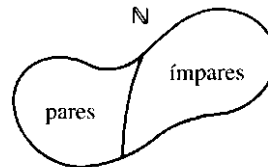


Figura 4.6

PRÁTICA 14 Descreva as classes de equivalência de cada uma das relações de equivalência a seguir:

- No conjunto de todas as linhas no plano, $x \rho y \leftrightarrow x$ é paralela a y ou coincide com y .
- Em $\mathbb{N}$, $x \rho y \leftrightarrow x = y$.
- No conjunto $\{1, 2, 3\}$, $\rho = \{(1, 1), (2, 2), (3, 3), (1, 2), (2, 1)\}$.

◆

O particionamento de um conjunto em classes de equivalência é útil porque frequentemente é conveniente tratar as classes como entidades. Concluiremos esta seção com dois exemplos onde isto ocorre.

EXEMPLO 13 Seja $S = \{a/b \mid a, b \in \mathbb{Z}, b \neq 0\}$. S é, portanto, o conjunto de todas as frações. Duas frações tais como $\frac{1}{2}$ e $\frac{2}{4}$ são ditas equivalentes. Formalmente, a/b é equivalente a c/d , denotado por $a/b \sim c/d$, se, e somente se, $ad = bc$. Mostraremos que a relação binária $\sim$ em S é uma relação de equivalência. Primeiro, $a/b \sim a/b$ porque $ab = ba$. Além disso, se $a/b \sim c/d$, então $ad = bc$ ou $cb = da$ e $c/d \sim a/b$. Portanto, $\sim$ é reflexiva e simétrica. Para mostrar a transitividade de $\sim$, sejam $a/b \sim c/d$ e $c/d \sim e/f$. Então $ad = bc$ e $cf = de$. Multiplicando a primeira equação por f e a segunda por b temos $adf = bcf$ e $bcf = bde$. Portanto, $adf = bde$ ou $af = be$. Portanto, $a/b \sim e/f$ e $\sim$ é transitiva. Alguns exemplos de classes de equivalências de S formadas pela relação $\sim$ são

$$\left[\frac{1}{2} \right] = \left\{ \dots, -\frac{3}{6}, -\frac{2}{4}, -\frac{1}{2}, \frac{1}{2}, \frac{2}{4}, \frac{3}{6}, \dots \right\}$$

$$\left[\frac{3}{10} \right] = \left\{ \dots, -\frac{29}{30}, -\frac{6}{20}, -\frac{3}{10}, \frac{3}{10}, \frac{6}{20}, \frac{9}{30}, \dots \right\}$$

O conjunto $\mathbb{Q}$ de números racionais pode ser entendido como o conjunto de todas as classes de equivalências de S . Um único número racional, tal como $\frac{1}{2}$, tem diversas frações para representá-lo, apesar de preferirmos usar a representação reduzida de frações. Quando somamos dois números racionais, tais como $\left[\frac{1}{2} \right] + \left[\frac{3}{10} \right]$, procuramos por representantes das classes de equivalência que tenham os mesmos denominadores e então os somamos. O resultado é a classe a qual a soma obtida pertence e, normalmente, nos referiremos a ela

através de uma fração reduzida que a represente. Desta forma, para somarmos $[\frac{1}{2}] + [\frac{3}{10}]$, representamos $[\frac{1}{2}]$ por $\frac{5}{10}$ e $[\frac{3}{10}]$ por $\frac{3}{10}$. A soma de $\frac{5}{10}$ e $\frac{3}{10}$ é $\frac{8}{10}$ e $[\frac{8}{10}]$ é mais convenientemente simbolizado por $[\frac{4}{5}]$. Este procedimento nos é tão familiar que já escrevemos $\frac{1}{2} + \frac{3}{10} = \frac{4}{5}$, sem nem nos darmos conta de que classes de frações estão sendo manipuladas através de elementos representativos. •

EXEMPLO 14 Definiremos a relação binária de **congruência módulo 4** no conjunto $\mathbb{Z}$ dos inteiros. Um inteiro x é congruente módulo 4 a y , simbolizado por $x \equiv_4 y$, ou $x \equiv y \pmod{4}$, se $x - y$ é um múltiplo exato de 4. A congruência módulo 4 é uma relação de equivalência em $\mathbb{Z}$. (Você pode provar isto?) Para construir as classes de equivalência, perceba que $[0]$, por exemplo, conterá os números que diferem por 0 de múltiplos de 4, tais como 4, 8, — 12, etc. As classes de equivalência distintas são

$$\begin{aligned} [0] &= \{\dots, -8, -4, 0, 4, 8, \dots\} \\ [1] &= \{\dots, -7, -3, 1, 5, 9, \dots\} \\ [2] &= \{\dots, -6, -2, 2, 6, 10, \dots\} \\ [3] &= \{\dots, -5, -1, 3, 7, 11, \dots\} \end{aligned}$$

Não há razão especial para a escolha do número 4 no Exemplo 14; podemos dar uma definição de **congruência módulo n** para qualquer inteiro positivo n . Esta relação binária será sempre uma relação de equivalência. Esta relação de equivalência e as classes de equivalência resultantes podem ser usadas para aritmética inteira em computadores. Um inteiro é armazenado como uma sequência de bits (0s e 1s) dentro de uma única posição de memória. Cada computador aloca um número fixo de bits em cada posição de memória (este número varia de acordo com a arquitetura do computador, isto é, como sua memória é organizada). Quanto maior o inteiro, maior o número de bits necessários para representá-lo. Portanto, toda máquina tem um limite no tamanho dos inteiros que pode armazenar. Suponha que $n - 1$ é o maior inteiro que pode ser armazenado e que x e y são inteiros tais que $0 \leq x \leq n - 1$ e $0 \leq y \leq n - 1$. Se a soma $x + y$ exceder o limite $n - 1$, ela não poderá ser armazenada. Como uma alternativa, o computador pode realizar a **soma módulo n** e armazenar o resto r da divisão de $x + y$ por n . A equação

$$x + y = qn + r, \quad 0 \leq r < n$$

simboliza esta divisão, onde q é o quociente e r é o resto. Esta equação pode ser escrita como

$$(x + y) \equiv r \pmod{n}$$

que mostra que $(x + y) \equiv r \pmod{n}$ é um múltiplo de n , ou que $(x + y) \equiv r \pmod{n}$. O inteiro r pode ser diferente de $x + y$, mas está na classe de equivalência $[x + y]$ e, como $0 \leq r < n$, está na faixa dos inteiros que podem ser armazenados. (O sistema pode ou não gerar uma mensagem de *estouro de inteiro* se $x + y$ for muito grande para ser armazenado e a soma módulo n precisar ser usada.)

PRÁTICA 15 Quais são as classes de equivalência correspondentes à relação de congruência módulo 5 em $\mathbb{Z}$? •

PRÁTICA 16 Se 4 for o maior inteiro que puder ser armazenado em um (micromicro) computador, qual será o resultado armazenado como resultado de $3 + 4$ se a soma módulo 5 for usada? •

Revisão da Seção 4.1

Técnicas

- Verificar se um par ordenado pertence a uma relação
- Verificar se uma relação binária é reflexiva, simétrica, anti-simétrica e/ou transitiva
- Encontrar o fecho reflexivo, simétrico e transitivo de uma relação
- Esboçar de forma gráfica um conjunto parcialmente ordenado
- Determinar elemento mínimo, minimal, máximo e maximal em uma ordenação parcial
- Encontrar as classes de equivalência associadas a uma relação de equivalência

Idéias Principais

Uma relação binária em um conjunto S é formalmente um subconjunto de $S \times S$; a relação normalmente também tem uma definição verbal.

Operações sobre relações binárias em um conjunto incluem união, interseção e complemento.

Relações binárias podem ter as propriedades reflexiva, simétrica, transitiva e anti-simétrica.

Conjuntos finitos parcialmente ordenados podem ser representados graficamente.

- ★5. Classifique cada relação em $S \times T$, onde $S = T = \mathbb{N}$, como um-para-um, um-para-vários, vários-para-um ou vários-para-vários.
- $\rho = \{(1,2), (1,4), (1,6), (2,3), (4,3)\}$
 - $\rho = \{(9,7), (6,5), (3,6), (8,5)\}$
 - $\rho = \{(12,5), (8,4), (6,3), (7,12)\}$
 - $\rho = \{(2,7), (8,4), (2,5), (7,6), (10,1)\}$
6. Classifique cada uma das relações em 5 como um-para-um, um-para-vários, vários-para-um ou vários-para-vários.
- $S = \mathbb{N}$;
 $x \rho y \leftrightarrow x = y + 1$
 - $S =$ conjunto de todas as mulheres de Nova Friburgo;
 $x \rho y \leftrightarrow x$ é filha de y
 - $S = \mathcal{P}(\{1, 2, 3\})$
 $A \rho B \leftrightarrow |A| = |B|$
 - $S = \mathbb{R}$;
 $x \rho y \leftrightarrow x = 5$
- ★7. Sejam ρ e σ relações binárias em $\mathbb{N}$ definidas por $x \rho y \leftrightarrow$ “ x divide y ”, $x \sigma y \leftrightarrow 5x \leq y$. Determine quais dos pares ordenados satisfazem às relações dadas:
- $\rho \cup \sigma$; (2, 6), (3, 17), (2, 1), (0, 0)
 - $\rho \cap \sigma$; (3, 6), (1, 2), (2, 12)
 - ρ' ; (1, 5), (2, 8), (3, 15)
 - σ' ; (1, 1), (2, 10), (4, 8)
8. Seja $S = \{0, 1, 2, 4, 6\}$. Verifique se as relações binárias em S são reflexivas, simétricas, anti-simétricas e/ou transitivas:
- $\rho = \{(0,0), (1,1), (2,2), (4,4), (6,6), (0,1), (1,2), (2,4), (4,6)\}$
 - $\rho = \{(0,1), (1,0), (2,4), (4,2), (4,6), (6,4)\}$
 - $\rho = \{(0,1), (1,2), (0,2), (2,0), (2,1), (1,0), (0,0), (1,1), (2,2)\}$
 - $\rho = \{(0,0), (1,1), (2,2), (4,4), (6,6), (4,6), (6,4)\}$
 - $\rho = \emptyset$
9. Classifique as relações binárias a seguir nos conjuntos S dados como reflexivas, simétricas, anti-simétricas e transitivas:
- | | |
|---|---|
| ★a. $S = \mathbb{Q}$
$x \rho y \leftrightarrow x \leq y $ | ★b. $S = \mathbb{Z}$
$x \rho y \leftrightarrow x - y$ é um múltiplo de 3 |
| ★c. $S = \mathbb{N}$
$x \rho y \leftrightarrow x \cdot y$ é par | d. $S = \mathbb{N}$
$x \rho y \leftrightarrow x$ é ímpar |
| e. $S =$ conjunto de todos os quadrados no plano
$S_1 \rho S_2 \leftrightarrow$ tamanho do lado de $S_1 =$ tamanho do lado de S_2 | |
| f. $S =$ conjunto de todas as cadeias finitas de caracteres
$x \rho y \leftrightarrow$ número de caracteres em $x =$ número de caracteres em y | |
| g. $S =$ conjunto de todas as pessoas do Brasil
$x \rho y \leftrightarrow x$ é irmão de y | |
| h. $S = \mathcal{P}(\{1, 2, 3, 4, 5, 6, 7, 8, 9\})$
$A \rho B \leftrightarrow A = B $ | |
| i. $S = \mathcal{P}(\{1, 2, 3, 4, 5, 6, 7, 8, 9\})$
$A \rho B \leftrightarrow A \neq B $ | |
| j. $S = \mathbb{N} \times \mathbb{N}$
$(x_1, y_1) \rho (x_2, y_2) \leftrightarrow x_1 \leq x_2$ e $y_1 \geq y_2$ | |
10. Quais das relações binárias do Exercício 9 são relações de equivalência? Para as que o forem, descreva as classes de equivalência associadas.
11. Para cada caso abaixo, apresente um conjunto S e uma relação binária p em S (diferente das apresentadas nos exemplos e nos problemas) que satisfaça às condições pedidas.
- ρ é reflexiva e anti-simétrica, mas não é transitiva.
 - ρ é reflexiva e transitiva, mas não é simétrica.
 - ρ não é reflexiva nem simétrica, mas é transitiva.
 - ρ é reflexiva, mas não é simétrica nem transitiva.

12. Sejam ρ e σ relações binárias em um conjunto S .
- Se ρ e σ forem reflexivas, a união $\rho \cup \sigma$ será reflexiva? E a interseção $\rho \cap \sigma$?
 - Se ρ e σ forem simétricas, a união $\rho \cup \sigma$ será simétrica? E a interseção $\rho \cap \sigma$?
 - Se ρ e σ forem anti-simétricas, a união $\rho \cup \sigma$ será anti-simétrica? E a interseção $\rho \cap \sigma$?
 - Se ρ e σ forem transitivas, a união $\rho \cup \sigma$ será transitiva? E a interseção $\rho \cap \sigma$?

13. Encontre os fechos reflexivos, simétricos e transitivos das relações do Exercício 8.

★14. Descreva em palavras o que o fecho transitivo de cada relação abaixo representa.

- $S =$ conjunto de todos os edifícios da cidade
 $x \rho y \leftrightarrow x$ é um ano mais velho que y
- $S =$ conjunto de todos os homens da Bulgária
 $x \rho y \leftrightarrow x$ é pai de y
- $S =$ conjunto das cidades do Brasil
 $x \rho y \leftrightarrow$ podemos ir de carro da cidade x até a cidade y em um dia

15. Definimos mais duas propriedades de uma relação binária ρ da seguinte maneira:

ρ é **irreflexiva** significa: $(\forall x) (x \in S \wedge (x, x) \notin \rho)$

ρ é **assimétrica** significa: $(\forall x) (\forall y) (x \in S \wedge y \in S \wedge (x, y) \in \rho \rightarrow (y, x) \notin \rho)$

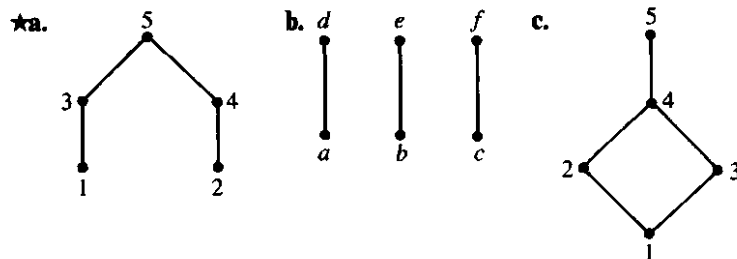
- Apresente um exemplo de relação binária ρ no conjunto $S = \{1, 2, 3\}$ que não seja nem reflexiva nem irreflexiva.
 - Apresente um exemplo de relação binária ρ no conjunto $S = \{1, 2, 3\}$ que não seja simétrica nem assimétrica.
 - Demonstre que se ρ é assimétrica em S , então ρ é irreflexiva.
 - Demonstre que se ρ é uma relação irreflexiva e transitiva em um conjunto S , então ρ é assimétrica.
 - Demonstre que se ρ é uma relação não-vazia, simétrica e transitiva em um conjunto S , então ρ não é irreflexiva.
16. Faz sentido examinarmos o fecho de uma relação com respeito às seguintes propriedades? Justifique.
- propriedade irreflexiva
 - propriedade assimétrica

17. Seja ρ uma relação binária em um conjunto S . Para $A \subseteq S$ e defina

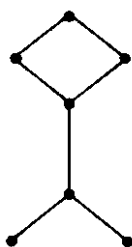
$$\begin{aligned} \#A &= \{x \mid x \in S \wedge (\forall y) (y \in A \rightarrow x \rho y)\} \\ A\# &= \{x \mid x \in S \wedge (\forall y) (y \in A \rightarrow y \rho x)\} \end{aligned}$$

- Demonstre que se ρ é simétrica, então $\#A = A\#$.
 - Demonstre que se $A \subseteq B$, então $\#B \subseteq \#A$ e $B\# \subseteq A\#$.
 - Demonstre que $A \subseteq (\#A)\#$.
 - Demonstre que $A \subseteq \#(A\#)$.
18. Desenhe o grafo das seguintes ordenações parciais:
- $S = \{a, b, c\}$
 $\rho = \{(a, a), (b, b), (c, c), (a, b), (b, c), (a, c)\}$
 - ★ $S = \{a, b, c, d\}$
 $\rho = \{(a, a), (b, b), (c, c), (d, d), (a, b), (a, c)\}$
 - $S = \{\emptyset, \{a\}, \{a, b\}, \{c\}, \{a, c\}, \{b\}\}$
 $A \rho B \leftrightarrow A \subseteq B$
19. Indique os elementos mínimos, minimais, máximos e maximais que aparecem nas ordenações parciais do Exercício 18.
20. Desenhe o grafo da ordenação parcial "x divide y" no conjunto $\{2, 3, 5, 7, 21, 42, 105, 210\}$. Indique os elementos mínimo, minimais, máximo e maximais desta ordenação parcial. Apresente um subconjunto totalmente ordenado com quatro elementos.
21. Desenhe o grafo dos dois conjuntos parcialmente ordenados.
- $S = \{1, 2, 3, 5, 6, 10, 15, 30\}$
 $x \rho y \leftrightarrow x$ divide y
 - $S = \mathcal{P}(\{1, 2, 3\})$
 $A \rho B \leftrightarrow A \subseteq B$
- O que você pode notar a respeito da estrutura desses dois grafos?

22. Para cada grafo de ordenação parcial apresentado abaixo, escreva os pares ordenados que pertencem à relação.



23. Sejam (S, ρ) e (T, σ) dois conjuntos parcialmente ordenados. Uma relação μ em $S \times T$ é definida como $(s_1, t_1) \mu (s_2, t_2) \leftrightarrow s_1 \rho s_2$ e $t_1 \sigma t_2$. Mostre que μ é uma ordenação parcial em $S \times T$.



24. a. Seja $(S, \leq)$ um conjunto parcialmente ordenado e defina uma nova relação binária $\geq$ em S como $x \geq y \leftrightarrow y \leq x$. Mostre que $(S, \geq)$ é um conjunto parcialmente ordenado, chamado de **dual** de $(S, \leq)$.
 b. Se $(S, \leq)$ é um conjunto finito parcialmente ordenado com o grafo mostrado, desenhe o grafo de $(S, \geq)$.
 c. Seja $(S, \leq)$ um conjunto totalmente ordenado e seja $X = \{(x, x) \mid x \in S\}$. Mostre que a diferença de conjuntos $\geq - X$ é igual ao conjunto $\leq'$.

25. Um programa de computador para gerar o dicionário ou o índice de um livro será escrito. Assumiremos um tamanho máximo de n caracteres por palavra. Temos, portanto, um conjunto S com palavras de, no máximo, n caracteres e desejamos gerar uma lista ordenada alfabeticamente com estas palavras. Existe a ordem natural dos caracteres do alfabeto $\leq$ ($a \leq b$, $b \leq c$ etc.), e admitimos que nossas palavras contenham apenas caracteres alfabéticos. Desejamos definir uma ordenação total $\leq$ em S chamada ordenação lexicográfica, que ordene S alfabeticamente. A idéia é comparar duas palavras X e Y caracter a caracter, ignorando os caracteres iguais. Se, em algum momento, o caracter da palavra X precede o caracter correspondente da palavra Y , então X precede Y ; se todos os caracteres de X forem iguais aos caracteres correspondentes de Y , mas os caracteres de X acabaram antes dos de Y , então X precede Y , caso contrário, Y precede X .

Formalmente, sejam $X = (x_1, x_2, \dots, x_j)$ e $Y = (y_1, y_2, \dots, y_k)$ elementos de S com $j \leq k$. Seja β (de branco) um novo símbolo, e preenchamos X com $k - j$ brancos à direita. X agora pode ser escrito como $(X_1, x_2, \dots, x_k)$. Arbitremos ainda que β precede qualquer outro caracter alfabético. Então $X \leq Y$ se

$$x_1 \neq y_1 \text{ e } x_1 \leq y_1$$

ou

$$x_1 = y_1 \text{ e } x_2 = y_2, \dots, x_m = y_m \text{ (} m \leq k \text{)}$$

$$x_{m+1} \neq y_{m+1} \text{ e } x_{m+1} < y_{m+1}$$

Caso contrário, $y \leq x$.

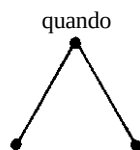
Perceba que, devido à ordenação $\leq$ nos caracteres alfabéticos ser uma ordenação total, se o “outro lado”, $y \leq x$, se verificar então existe $m \leq k$ tal que $x_1 = y_1$ e $x_2 = y_2, \dots, x_m = y_m$, $x_{m+1} \neq y_{m+1}$ e $y_{m+1} \leq x_{m+1}$.

- a. Mostre que $\leq$ em S , conforme definido acima, é uma ordenação total.
 b. Aplique a ordenação total descrita às palavras *roupa*, *rua*, *remédio*, *rato* e *ruga*. Perceba que cada palavra precede a próxima.
26. O Exercício 25 abordou uma ordenação total em um conjunto de palavras com no máximo n caracteres de tamanho que gera, como saída, uma lista linear ordenada alfabeticamente. Suponha que desejamos gerar uma lista com todas as palavras distintas do texto na ordem em que aparecem no mesmo (por exemplo, um compilador precisa gerar uma tabela de símbolos com os nomes das variáveis). Como no Exercício 25, assumimos que as palavras contêm apenas caracteres alfabéticos porque já existe uma relação natural de precedência ($a < b$, $b < c$ etc.). Se forem permitidos caracteres numéricos ou especiais eles precisam ter uma relação de precedência junto aos caracteres alfabéticos (a sequência de ordenação precisa ser definida). Se listarmos as palavras em ordem alfabética, o procedimento para decidir se uma palavra sendo processada é nova é bem simples, mas para colocar a nova palavra no lugar precisamos mover todas as palavras uma linha para baixo. Se, porém, as palavras forem listadas na ordem em que forem processadas, as novas palavras podem ser simplesmente incluídas ao fim da lista sem a neces-

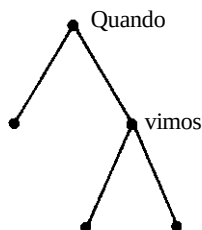
sidade de qualquer rearrumação, mas, para determinar se a palavra sendo processada é nova ou não, é preciso compará-la com todas as outras palavras da lista. Portanto, ambos os tipos de lista apresentam suas desvantagens.

Descreveremos um processo de enumeração que se vale de uma **árvore binária de busca** que permite, para o caso geral, determinar de forma rápida se uma palavra é nova e, se este for o caso, não há a necessidade de realizar uma rearrumação para colocá-la em seu lugar, combinando, portanto, as vantagens dos dois métodos descritos acima. Suponha que desejamos processar a frase "Quando vimos já não era mais possível". A primeira palavra da frase é usada para dar nome ao primeiro vértice de um grafo.

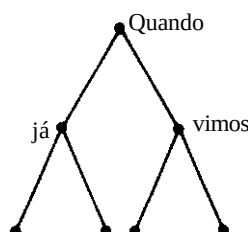
• quando



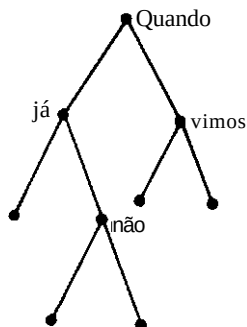
Uma vez que um vértice tenha recebido seu nome, ele recebe duas arestas para baixo, que levam a dois vértices sem nome. Quando a próxima palavra no texto for processada, ela é comparada com o primeiro nó. Se a palavra sendo processada preceder a palavra que dá nome ao vértice, tomamos a aresta da esquerda, do contrário tomamos a aresta da direita. A palavra que se busca se torna o nome do primeiro vértice ainda sem nome que for encontrado. (Se a palavra for igual ao nome de algum vértice, significa que aquela palavra já havia aparecido no texto e passamos ao processamento da palavra seguinte.) Este procedimento se repete para todo o texto. Desta forma,



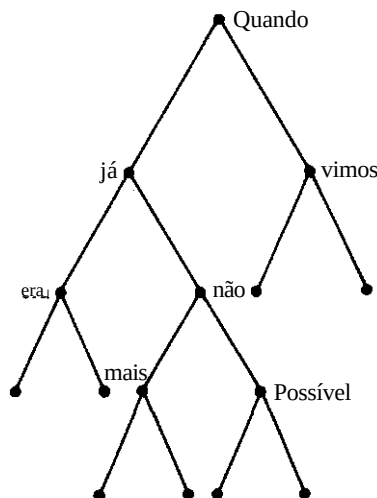
então



então



e, finalmente,



Se varreremos os vértices deste grafo na ordem apropriada (definida sempre como processar os vértices à esquerda antes, depois o próprio vértice e, depois os vértices à direita) obtemos uma lista em ordem alfabética "era, já, mais, não, possível, quando, vimos".

a. Este tipo de grafo é chamado de **árvore**. Se o virarmos de cabeça para baixo, podemos entendê-lo como uma ordenação parcial $\leq$. Qual seria o elemento mínimo? Há um elemento máximo? Quais dos pares a seguir pertenceriam a $\leq$: (não, vimos), (já, era), (já, mais), (era, possível)?

Neste caso, a estrutura de árvore contém mais informação do que a ordenação parcial pois nos diz não só quando uma palavra w_1 precede uma palavra w_2 mas também que w_2 está à esquerda ou à direita de w_1 .

b. Use uma árvore binária de busca para desenhar o grafo de "Suco de laranja faz bem à saúde". E, considerando o grafo (de cabeça para baixo) como uma ordenação parcial, determine os elementos maximais.

17. A ordenação alfabética definida no Exercício 25 pode ser aplicada a palavras de qualquer tamanho finito. Se definirmos A^* como o conjunto de todas as "palavras" (cadeias de caracteres, que não necessari-

amente façam sentido) de tamanho finito formadas pelas letras do alfabeto inglês, então a ordenação alfabética em A^* tem todas as palavras compostas apenas pela letra a precedendo todas as outras palavras. Portanto, todas as palavras da lista infinita

$a, aa, aaa, aaaa, \dots$

precederiam palavras como "b" ou "aaaaaaab". Portanto, esta lista não enumera A^* porque não poderíamos relacionar, seqüencialmente, outras palavras que não as formadas apenas por a . No entanto, o conjunto A^* é enumerável. Demonstramos isto ordenando A^* pelo comprimento das palavras (todas as palavras de tamanho 1 precedem todas as palavras de tamanho 2 etc.) e, depois, ordenando alfabeticamente as palavras de mesmo comprimento.

- ★28. a. Qual o conjunto $[a]$ para a relação de equivalência $\rho = \{(a, a), (b, b), (c, c), (a, c), (c, a)\}$? Ele tem outras representações?
 b. Qual o conjunto $[3]$ para a relação de equivalência $\rho = \{(1, 1), (2, 2), (1, 2), (2, 1), (1, 3), (3, 1), (3, 2), (2, 3), (3, 3), (4, 4), (5, 5), (4, 5), (5, 4)\}$? Qual o conjunto $[4]$?
 c. Qual o conjunto $[1]$ para a relação de equivalência de congruência módulo 2 em $\mathbb{Z}$?
 d. Qual o conjunto $[-3]$ para a relação de equivalência módulo 5 no conjunto $\mathbb{Z}$?
29. a. Dada a partição $\{1, 2\}$ e $\{3, 4\}$ do conjunto $S = \{1, 2, 3, 4\}$, liste os pares ordenados da relação de equivalência correspondente.
 b. Dada a partição $\{a, b, c\}$ e $\{d, e\}$ do conjunto $S = \{a, b, c, d, e\}$, liste os pares ordenados da relação de equivalência correspondente.
30. Seja S o conjunto de todos os livros em uma biblioteca. Seja ρ uma relação binária em S definida por $x \rho y \leftrightarrow$ "a cor da capa de x é a mesma da cor da capa de y ". Mostre que ρ é uma relação de equivalência e descreva as classes de equivalências resultantes.
- ★31. Seja $S = \mathbb{N} \times \mathbb{N}$ e seja ρ uma relação binária em S definida por $(x, y) \rho (z, w) \leftrightarrow y = w$. Mostre que ρ é uma relação de equivalência em S e descreva as classes de equivalência resultantes.
32. Seja $S = \mathbb{N} \times \mathbb{N}$ e seja ρ uma relação binária em S definida por $(x, y) \rho (z, w) \leftrightarrow x + y = z + w$. Mostre que ρ é uma relação de equivalência em S e descreva as classes de equivalência resultantes.
33. Seja $S = \mathbb{N}$ e seja ρ uma relação binária em S definida por $x \rho y \leftrightarrow x^2 - y^2$ é par. Mostre que ρ é uma relação de equivalência em S e descreva as classes de equivalência que define.
34. Seja S o conjunto de todas as wffs proposicionais com n afirmações. Seja ρ uma relação binária em S definida por $P \rho Q \leftrightarrow "P \leftrightarrow Q$ é uma tautologia". Mostre que ρ é uma relação de equivalência em S e descreva as classes de equivalência resultantes. (Usamos a notação $P \Leftrightarrow Q$ para $P \leftrightarrow Q$.)
35. Dadas duas partições π_1 e π_2 de um conjunto S , π_1 é dita um **refinamento** de π_2 se cada bloco de π_1 é um subconjunto de um bloco de π_2 . Mostre que um refinamento é uma ordenação parcial no conjunto de todas as partições de S .
36. Seja P_n a notação para o número total de partições de um conjunto de n elementos, $n \geq 1$.
 a. Encontre P_1 . b. Encontre P_3 . c. Encontre P_4 .
- ★37. Seja $S(n, k)$ o número de maneiras de se particionar um conjunto de n elementos em k blocos.
 a. Encontre $S(3, 2)$ b. Encontre $S(4, 2)$.
38. Demonstre que

$$P_n = \sum_{k=1}^n S(n, k)$$

39. Demonstre que para todo $n \geq 1$, $S(n, k)$ verifica a relação de recorrência
- $$S(n, 1) = 1$$
- $$S(n, n) = 1$$
- $$S(n+1, k+1) = S(n, k) + (k+1) S(n, k+1) \text{ para } 1 \leq k \leq n$$

(Dica: Use uma demonstração combinatória ao invés de uma demonstração por indução. Seja x um elemento fixo porém arbitrário de um conjunto com $n + 1$ elementos e exclua x . Particione o conjunto restante com n elementos. Uma partição do conjunto original pode ser obtida considerando $\{x\}$ um bloco a parte ou incluindo x em um dos blocos já existentes.)

- ★40. Use a fórmula do Exercício 39 para refazer o Exercício 37.
41. Os números $S(n, k)$ são chamados de **números de Stirling**. A relação de recorrência do Exercício 39 é semelhante à da fórmula de Pascal, equação (1) da Seção 3.5. Use esta relação para calcular os valores numéricos das primeiras cinco linhas do **triângulo de Stirling**, que começa com

$$\begin{array}{c} 5(1, 1) \\ 5(2, 1) \ 5(2, 2) \\ 5(3, 1) \ 5(3, 2) \ 5(3, 3) \\ \vdots \end{array}$$

- ★42. Encontre o número de maneiras de distribuir quatro diferentes mármores coloridos entre três containers idênticos de forma que nenhum container fique vazio.
43. Encontre o número de maneiras nas quais cinco tarefas diferentes podem ser atribuídas a três processadores idênticos de forma que cada processador receba pelo menos uma tarefa.
44. Supondo que P_0 tenha o valor 1, demonstre que

$$P_n = \sum_{k=0}^{n-1} C(n-1, k) P_k$$

(Dica: Use uma demonstração combinatória ao invés de uma demonstração indutiva. Tome x fixo, porém arbitrário como um elemento de um conjunto com n elementos. Em cada termo da soma, $n - k$ representa o tamanho do bloco da partição que contém x .)

45. a. Use a fórmula do Exercício 44 para calcular P_1, P_2, P_3 e P_4 e compare suas respostas com as do Exercício 36.
b. Use a fórmula do Exercício 38 e o triângulo de Stirling (Exercício 41) para calcular P_1, P_2, P_3 e P_4

Seção 4.2 Relações e Bancos de Dados e Ordenação Topológica

Bancos de Dados

Um **banco de dados** é um depósito de informações relacionadas a um determinado negócio. Para projetar um banco de dados útil e eficiente é necessário modelar o negócio. Um **modelo conceitual** procura capturar os recursos e mecanismos importantes do negócio. É necessária uma considerável interação com pessoas que tenham familiaridade com os negócios a fim de obter todas as informações necessárias para formular o modelo.

Modelo Entidade-Relacionamento

Uma representação de alto-nível de um negócio é o **modelo entidade-relacionamento**. Neste modelo, importantes objetos, ou **entidades**, do negócio são definidos, juntamente com seus atributos ou propriedades que sejam relevantes, de forma que as relações entre essas diversas entidades sejam anotadas. Esta informação é representada graficamente por um **diagrama entidade-relacionamento** ou **diagrama E-R**. Em um diagrama E-R, os retângulos denotam conjuntos de entidades, elipses denotam atributos, e losangos denotam relações.

EXEMPLO 15 O Clube de Amigos de Animais do Brasil (CAAB) deseja criar um banco de dados. O CAAB adquiriu malas-diretas de seus fornecedores e está interessado em pessoas que possuam animais de estimação e em algumas informações sobre estes animais, tais como o nome, o tipo (cachorro, gato etc.) e a raça.

A Fig. 4.7 mostra um diagrama E-R para o CAAB. Este diagrama diz que pessoas e cachorros são entidades. As pessoas têm os atributos *Nome*, *Endereço*, *Cidade* e *Estado*. Os animais têm os atributos *Nome-Animal*, *Tipo-Animal* e *Raça*. O diagrama também nos diz que as pessoas possuem animais. Se imaginarmos as entidades como conjuntos, os conjuntos Pessoas e os conjuntos Animais, a relação "possui" é uma relação binária em Pessoas X Animais — a relação de uma pessoa possuir um animal é obtida por pares ordenados (pessoa, animal). O "1" e o "N" nas linhas de conexão indicam que esta relação binária é do tipo um-para-vários; isto é, em um particular negócio, uma pessoa pode possuir diversos animais, mas os animais não têm

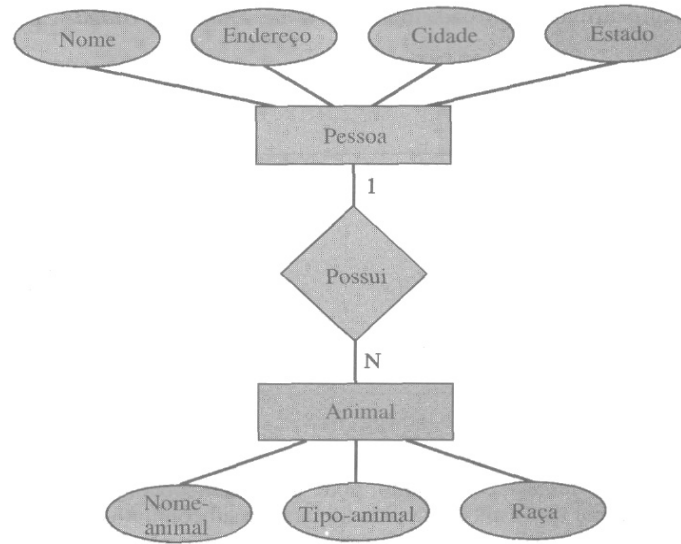


Figura 4.7

mais de um dono. (Animais com mais de um dono resultariam em uma relação vários-para-vários). Além disso, neste exemplo, algumas pessoas podem não possuir animais, e pode haver animais sem donos.

O fato de um animal não possuir mais de um dono é uma das "regras do negócio". Essas regras do negócio são importantes de serem identificadas quando do projeto do banco de dados, pois elas determinam vários recursos do banco de dados, como veremos.

Modelo Relacionai

O **modelo relacionai**, outra representação de um negócio, pode ser desenvolvido a partir do modelo E-R. Tanto o conjunto de entidades como o conjunto de relações de um modelo E-R se tornam relações (no sentido matemático) no modelo relacionai. As relações são descritas por tabelas. Um **banco de dados relacionai** consiste em coleções de tabelas de relações.

Uma tabela com o conjunto de entidades é criada para o conjunto de entidades. Cada linha na tabela contém os valores de n atributos para uma ocorrência particular deste conjunto de entidades (tal como um registro tem os valores de seus n campos). Portanto, a tabela relacionai pode ser entendida como um conjunto de n -uplas (linhas) e uma linha individual é chamada de **tupla**. Assim como no conceito de conjunto, não pode haver tuplas repetidas, e não assumimos qualquer ordenação das tuplas. A ordem dos atributos não é relevante, exceto pelo fato de que deve-se manter a consistência dos mesmos; isto é, cada coluna na tabela deve conter valores para um atributo específico em todas as tuplas.

De maneira mais formal, uma relação de um banco de dados é um subconjunto de $D_1 \times D_2 \times \dots \times D_n$, onde D_i é o domínio de onde o atributo A_i tira seu valor. Isto significa que o uso da palavra *relação* em bancos de dados é consistente com nossa definição de relação n -ária (Ver anteriormente).

EXEMPLO 16 Uma ocorrência da relação Pessoa no banco de dados do CAAB pode conter os seguintes dados:

Pessoa			
Nome	Endereço	Cidade	Estado
Antônio Pedro	Av. Brasil, 23	Porto Seguro	BA
Marco Antônio	R. Cruz das Almas, 42	Campos	RJ
Maria da Silva	Av. Sernambetiba, 456	Rio de Janeiro	RJ
Jaime Luiz	Av. Paulista, 1498	São Paulo	SP
Bruno da Silva	R. Presidente Vargas, 89	Recife	PE
Jânio Rocha	R. Santa Clara, 54	Manaus	AM
Maria Garcia	R. Rep. Chile, 32	Curitiba	PR

Os quatro atributos para cada tupla são *Nome*, *Endereço*, *Cidade* e *Estado*. Uma ocorrência da relação *Animal* pode ser:

Animal		
<i>Nome-animal</i>	<i>Tipo-animal</i>	<i>Raça</i>
Lulu	Cachorro	Pequinês
Cacau	Gato	Siamês
Chica	Cachorro	Collie
Lassie		Collie
Vanda	Peixe	Dourado
Ibraim	Pássaro	Periquito
Tigrão	Gato	Pêlo-curto-brasileiro

Como não há tuplas duplicadas em uma relação, fornecer o valor de todos os n atributos de uma tupla a distingue das demais. No entanto, pode haver um subconjunto minimal de atributos que podem ser usados para identificar de forma única cada tupla. Este subconjunto é chamado de **chave primária** da relação; se este subconjunto consistir em mais de um atributo, então ele é chamado de uma **chave primária composta**. Na tabela que descreve a relação, o nome da chave primária aparece sublinhado na linha dos nomes de atributos.

Outra regra dos negócios da CAAB é que as pessoas devem ter nomes únicos; portanto, *Nome* é suficiente para identificar cada tupla e foi escolhido como chave primária na relação *Pessoa*. Perceba que o exemplo da relação *Pessoa* mostra que *Estado* não serviria como chave primária porque existem duas tuplas com *Estado* = "RJ". No entanto, não é apenas porque *Nome* tem valores únicos neste exemplo que impedimos a inclusão de nomes repetidos. São as regras do negócio que determinam que os nomes são únicos. (Não há regras que digam que os endereços ou cidades são únicos, de forma que nenhum desses atributos pode constituir chaves primárias, a despeito de não haver repetições no exemplo mostrado.)

A premissa de que os nomes são únicos é uma regra simplista. A chave primária em uma relação que envolva pessoas é geralmente um número de identificação, tal como o CPF, que é um atributo único conveniente. Como *Nome-animal* é a chave primária na relação *Animal* do Exemplo 16, podemos supor qual é a regra mais surpreendente da CAAB: os animais têm nomes únicos. Um cenário mais realista exigiria a criação de um atributo único para cada animal, uma espécie de número de CPF, para ser usado como chave primária. Esta chave poderia não ter qualquer outra utilidade na aplicação verdadeira, de forma que o usuário nunca tomasse conhecimento dela; este tipo de chave é chamado de **chave cega**.

Como todo domínio de atributo D_i em um banco de dados relacionai deve conter um valor especial nulo, uma dada tupla pode ter valores nulos para um ou mais de seus atributos. No entanto, nenhum dos atributos que constituem a chave primária pode conter valor nulo (vazio). Esta restrição de **integridade de entidade** apenas confirma que cada tupla deve conter um valor de chave primária a fim de distingui-la e que os atributos da chave primária são necessários para identificarem uma tupla de forma única.

Um atributo em uma relação (chamada de relação "filha") pode ter o mesmo domínio que uma chave primária em outra relação (chamada de relação "pai"). Este tipo de atributo é chamado de **chave estrangeira** (na relação filha) da relação pai. Uma relação para estabelecer os relacionamentos entre entidades se vale de chaves estrangeiras para definir as conexões entre as entidades. Existirá uma chave estrangeira na relação de relacionamento para cada entidade que participe do relacionamento.

EXEMPLO 17 O CAAB identificou a seguinte cópia do relacionamento Possui. O atributo *Nome* de Possui é uma chave estrangeira da relação *Pessoa*, onde *Nome* é uma chave primária; *Nome-animal* de Possui é uma chave estrangeira da relação *Animal*, onde constitui uma chave primária. A primeira tupla estabelece o relacionamento Possui entre Bruno da Silva e Lulu; isto é, indica que Bruno da Silva possui Lulu.

Possui	
<i>Nome</i>	<i>Nome-animal</i>
Bruno da Silva	Lulu
Marco Antônio	Cacau
Jaime Luiz	Chica
Jaime Luiz	Lassie
Maria da Silva	Ibraim
Jânio Rocha	Tigrão

Uma relação de equivalência em um conjunto define classes de equivalência que podem ser tratadas como entidades. Uma relação de equivalência em S sempre define uma partição de S , e vice-versa.

Exercícios 4.1

★1. Indique quais pares ordenados pertencem a cada uma das relações binárias ρ em $\mathbb{N}$ abaixo:

- $x \rho y \leftrightarrow x + y < 7$; (1, 3), (2, 5), (3, 3), (4, 4)
- $x \rho y \leftrightarrow x = y + 2$; (0, 2), (4, 2), (6, 3), (5, 3)
- $x \rho y \leftrightarrow 2x + 3y = 10$; (5, 0), (2, 2), (3, 1), (1, 3)
- $x \rho y \leftrightarrow y$ é um quadrado perfeito; (1, 1), (4, 2), (3, 9), (25, 5)

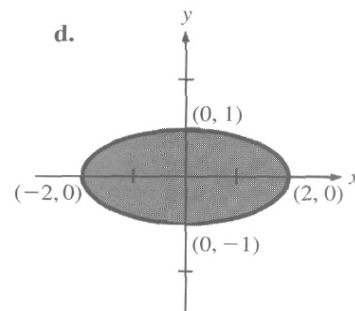
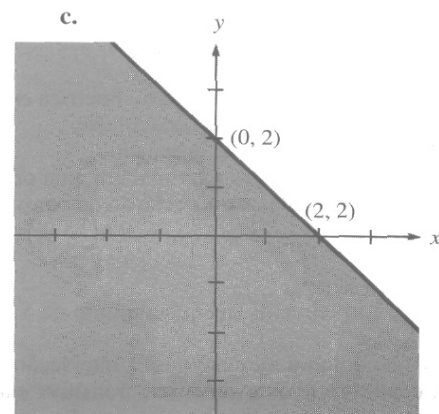
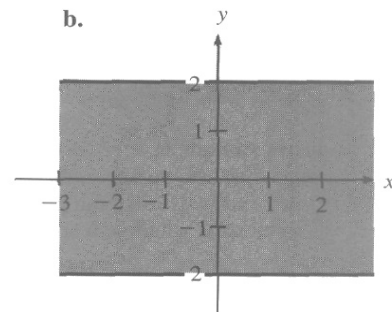
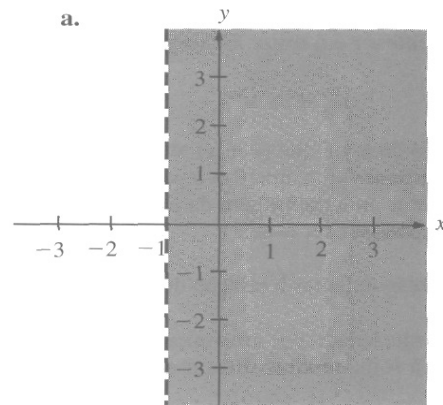
2. Determine quais dos pares dados satisfazem a relação em questão.

- ρ é a relação binária em $\mathbb{Z}$, $x \rho y \leftrightarrow x = -y$;
(1, -1), (2, 2), (-3, 3), (-4, -4)
- ρ é a relação unária em $\mathbb{N}$, $x \in \rho \leftrightarrow x$ é primo;
19, 21, 33, 41
- ρ é a relação ternária em $\mathbb{N}$, $(x, y, z) \in \rho \leftrightarrow x^2 + y^2 = z^2$;
(1, 1, 2), (3, 4, 5), (0, 5, 5), (8, 6, 10)
- ρ é a relação binária em $\mathbb{Q}$, $x \rho y \leftrightarrow x \leq 1/y$;
(1, 2), (-3, -5), (-4, $\frac{1}{2}$), ($\frac{1}{2}$, $\frac{1}{3}$)

3. Para cada uma das relações binárias em $\mathbb{R}$ abaixo, desenhe uma figura que mostre que região do plano ela define:

- $x \rho y \leftrightarrow y \leq 2$
- $x \rho y \leftrightarrow x = y - 1$
- $x \rho y \leftrightarrow x^2 + y^2 \leq 25$
- $x \rho y \leftrightarrow x \geq y$

4. Para cada uma das figuras abaixo, forneça a relação binária em $\mathbb{R}$ que descreve a área sombreada.



As pessoas que não possuem animais não são representadas em Possui nem os animais que não possuem donos. A chave primária é *Nome-animal*. Lembre-se de que uma das regras do negócio era que nenhum animal admite mais de um dono. Se fossem permitidos animais com mais de um dono, a chave primária composta *Nome / Nome-animal* teria que ser usada.

Algumas vezes uma tabela separada de relacionamento não é necessária. Na verdade, não é necessária em um relacionamento um-para-um e, algumas vezes, não é necessária em relacionamentos um-para-vários, tais como em nosso exemplo.

EXEMPLO 18 Como *Nome-animal* na relação Possui é uma chave estrangeira da relação Animal, as duas relações podem ser combinadas (usando uma operação chamada de *outer join over Nome-animal*) a fim de formar a relação Animal-Dono.

Animal-Dono			
<i>Nome</i>	<i>Nome-animal</i>	<i>Tipo-animal</i>	<i>Raça</i>
Bruno da Silva	Lulu	Cachorro	Pequinês
Marco Antônio	Cacau	Gato	Siamês
Jaime Luiz	Chica	Cachorro	Collie
Jaime Luiz	Lassie	Cachorro	Collie
NULO	Vanda	Peixe	Dourado
Maria da Silva	Ibraim	Pássaro	Periquito
Jânio Rocha	Tigrão	Gato	Pêlo-curto-brasileiro

Esta relação Animal-Dono pode substituir ambas as relações Possui e Animal sem perda de informação. A relação Animal-Dono contém uma tupla com um valor nulo para o atributo *Nome*. Isto não viola a integridade da entidade porque *Nome* não é um componente da chave primária, apesar de ser uma chave estrangeira da tabela Pessoa.

Operações com Relações

Podemos realizar duas operações unárias com as relações: *select* e *project*. A operação **select** (seleciona) cria uma nova tabela composta pelas tuplas da tabela original que satisfaçam a uma certa propriedade. A operação **project** (projeta) cria uma nova tabela composta por determinadas colunas da tabela original (eliminando quaisquer tuplas duplicadas).

EXEMPLO 19 A operação

Select Animal-Dono **where** *Tipo-animal* = "Cachorro" **giving** Cachorro-Dono

resulta na relação Cachorro-Dono abaixo:

Cachorro-Dono			
<i>Nome</i>	<i>Nome-animal</i>	<i>Tipo-animal</i>	<i>Raça</i>
Bruno da Silva	Lulu	Cachorro	Pequinês
Jaime Luiz	Chica	Cachorro	Collie
Jaime Luiz	Lassie	Cachorro	Collie

A operação

Project Animal-Dono **over** (*Nome*, *Tipo-animal*) **giving** Preferência

resulta na relação Preferência a seguir:

Preferência	
<i>Nome</i>	<i>Tipo-Animal</i>
Bruno da Silva	Cachorro
Marco Antônio	Gato
Jaime Luiz	Cachorro
Maria da Silva	Pássaro
Jânio Rocha	Gato

(Perceba que é necessário o uso de uma chave composta neste caso porque uma pessoa pode possuir diversos tipos de animais.)

As operações *select* e *project* podem ser entendidas em termos de subconjuntos. A operação *select* cria um subconjunto de linhas que satisfaçam certas propriedades e a operação *project* cria um subconjunto de colunas que representam certos atributos.

PRÁTICA 17 Escreva a tabela da relação que resulta da operação

Project Pessoa **over** (*Nome, Estado*) **giving** Local

Se as relações forem consideradas como conjuntos de n -uplas (linhas), as operações binárias de união, interseção e diferença de conjuntos aplicam-se a duas relações com a mesma estrutura básica. Portanto, em nosso exemplo, duas tabelas que contenham informações sobre proprietários de animais e que tenha a mesma estrutura podem ter sua interseção avaliada a fim de produzirem uma relação com todas as 4-uplas comuns.

Outra operação binária, **join** (junção), pode ser realizada em duas relações com um atributo (coluna) em comum. Esta operação forma, inicialmente, o produto cartesiano de todas as n -uplas (linhas) da primeira relação com todas as k -uplas (linhas) da segunda relação. Ele vê o resultado como um conjunto de $(n + k)$ -uplas e, dentre estas, seleciona o subconjunto daquelas cujo atributo em comum possuía o mesmo valor, escrevendo o resultado como um conjunto de $(n + k - 1)$ -uplas (o argumento em comum é escrito apenas uma vez). A operação *join* não é, portanto, uma operação independente, mas o resultado de um produto cartesiano seguido de um *Select*.

EXEMPLO 20 A operação

Join Pessoa **and** Animal-Dono **over** *Nome* **giving** Listagem

gera a relação Listagem abaixo:

Listagem						
<i>Nome</i>	<i>Endereço</i>	<i>Cidade</i>	<i>Estado</i>	<i>Nome-animal</i>	<i>Tipo-animal</i>	<i>Raça</i>
Marco Antonio	R. Cruz das Almas, 42	Campos	RJ	Cacau	Gato	Siamês
Maria da Silva	Av. Sernambetiba, 456	Rio de Janeiro	RJ	Ibraim	Pássaro	Periquito
Jaime Luiz	Av. Paulista, 1498	São Paulo	SP	Chica	Cachorro	Collie
Jaime Luiz	Av. Paulista, 1498	São Paulo	SP	Lassie	Cachorro	Collie
Bruno da Silva	R. Presidente Vargas, 89	Recife	PE	Lulu	Cachorro	Pequinês
Jânio Rocha	R. Santa Clara, 54	Manaus	AM	Tigrão	Gato	Pêlo-curto

As operações *select*, *project* e *join* podem ser aplicadas a várias combinações a fim de formularem pesquisas que o usuário deseje realizar em um banco de dados. Por exemplo, suponha que a pesquisa seja

Forneça os nomes de todos os gatos cujos donos morem no Rio de Janeiro. (1)

Se as únicas relações existentes forem Pessoa e Animal-Dono, a seguinte sequência de operação produzirá uma relação que responda a esta pergunta:

Select Animal-Dono **where** *Tipo-animal* = "Gato" **giving** Tabela 1

Tabela 1			
Nome	Nome-animal	Tipo-animal	Raça
Marco Antônio	Cacau	Gato	Siamês
Jânio Rocha	Tigrão	Gato	Pêlo-curto brasileiro

Select Pessoa **where** Estado = "RJ" **giving** Tabela 2

Tabela 2			
No me	Endereço	Cidade	Estado
Marco Antônio	Rua Cruz das Almas, 42	Campos	RJ
Maria da Silva	Av. Sernambetiba, 456	Rio de Janeiro	RJ

Join Tabela 2 **and** Tabela 1 **over** Nome **giving** Tabela 3

Tabela 3						
Nome	Endereço	Cidade	Estado	Nome-animal	Tipo-animal	Raça
Marco Antônio	R. Cruz das Almas, 42	Campos	RJ	Cacau	Gato	Siamês

Project Tabela 3 **over** Nome-animal **giving** Tabela-Final

Tabela Final
Nome-animal
Cacau

EXEMPLO 21

A **Álgebra relacional** é uma linguagem teórica de banco de dados relacionais na qual as operações **select**, **project** e **join** podem ser combinadas. O comando da álgebra relacional equivalente à sequência que usamos para achar o nome dos gatos cujos donos moram no estado do Rio de Janeiro seria o seguinte

project (join (select Animal-Dono where Tipo-animal = "Gato" and (select Pessoa where Estado = "RJ") over Nome) over Nome-animal giving Tabela-Final. (2)

SQL é uma linguagem de padrão internacional para banco de dados relacionais; a pesquisa acima seria escrita em SQL com o comando a seguir:

SELECT Nome-animal **FROM** Animal-Dono, Pessoa
WHERE Animal-Dono. Nome = Pessoa. Nome
AND Tipo-animal = "Gato"
AND Estado = "RJ"; (3)

(O comando **SELECT** da SQL é capaz de realizar **selects**, **projects** e **joins** da álgebra relacional, como mostrando no exemplo acima.)

Ao contrário de usar a abordagem da álgebra relacional, em que as operações **select**, **project** e **join** são usadas para processarem uma pesquisa, podemos usar a abordagem do **cálculo relacional**. Nesta abordagem, ao invés de especificarmos as operações a ser realizadas a fim de processar uma pesquisa, fornecemos uma descrição teórica de conjuntos do resultado desejado da pesquisa. A descrição do conjunto pode envolver a

notação da lógica predicada; lembremo-nos de que a lógica proposicional também é chamada de cálculo proposicional, daí o nome *cálculo relacionai*. A álgebra relacionai e o cálculo relacionai são equivalentes em termos do poder de expressão de cada um deles; isto é, qualquer pesquisa que possa ser formulada em uma linguagem pode ser formulada na outra.

EXEMPLO 22 A expressão em cálculo relacionai para a pesquisa dos nomes dos gatos cujos donos moram no estado do Rio de Janeiro é

Range of x é Animal-Dono
 Range of y é Pessoa
 $\{x.Nome-animal \mid x.Tipo-animal = \text{"Gato"} \text{ and } \exists y(y.Nome = x.Nome \text{ and } y.Estado = \text{"RJ"})\}$ (4)

Neste caso, "Range of x é Animal-Dono" especifica a relação da qual a upla x pode ser escolhida e "Range of y é Pessoa" especifica a relação da qual a upla y pode ser escolhida. (O uso do termo *range* (faixa) é infeliz. Na verdade estamos especificando o domínio da mesma forma que especificamos o domínio em uma interpretação na lógica predicada — o conjunto de valores em potencial.) A notação "exists v " remonta o quantificador existencial ($\exists y$).

As expressões (1) a (4) representam a mesma pesquisa expressa em língua inglesa, álgebra relacionai, SQL e cálculo relacionai, respectivamente.

PRÁTICA 18 Usando as relações Pessoas e Animal-Dono, expresse a seguinte pesquisa na forma de álgebra relacionai, SQL e cálculo relacionai:

Forneça os nomes de todas as cidades onde morem donos de cachorros.

Integridade de Bancos de Dados

A todo momento novos dados são incluídos em um banco de dados, dados obsoletos são excluídos e alterações ou atualizações são realizadas sobre os dados já existentes. Em outras palavras, o banco de dados estará sujeito a operações de **inclusão**, **exclusão** e **alteração**. Uma operação de inclusão pode ser realizada através da criação de uma segunda tabela da relação com o novo dado, e realizando a união de conjuntos da tabela existente e a nova tabela. A exclusão pode ser executada criando-se uma segunda tabela com as tuplas a ser excluídas, e realizando, então, uma diferença de conjuntos que subtraia a nova tabela da tabela já existente. Alterações podem ser realizadas por uma exclusão (da tupla a ser alterada), seguida de uma inclusão (do novo conteúdo da tupla).

Estas operações precisam ser realizadas de tal modo que a informação no banco de dados continue correta e consistente, de forma a ser coerente com as regras do negócio. Devemos garantir as três regras de "integridade". A **integridade de dados** exige que os valores de um atributo sejam realmente valores de seu domínio. Em nosso exemplo, o atributo *Estado* da relação Pessoa precisa ser uma abreviação de duas letras válida (ou o valor nulo). A integridade de entidade, como já discutimos, exige que os elementos de chaves primárias não sejam nulos. Está claro que estas restrições de integridade afetam as tuplas que podem ser incluídas em uma relação.

A **integridade referencial** exige que quaisquer valores de chaves estrangeiras de outras relações ou sejam nulas ou tenham valores que sejam encontrados nas respectivas chaves primárias dessas relações. As restrições de integridade referencial afetam as operações de inclusão e de exclusão (e, portanto, as operações de alteração). Por exemplo, não podemos incluir uma tupla a Animal-Dono com um valor não-nulo para *Nome* que não exista na relação Pessoa, porque isto violaria a relação Possui como uma relação binária em Pessoa X Animal. Além disso, se a tupla do Bruno da Silva for excluída da relação Pessoa, então a tupla do Bruno da Silva deve ser excluída da relação Animal-Dono ou o valor do campo *Nome* "Bruno da Silva" deve ser alterado para nulo (uma regra de negócio deve especificar o que deve ocorrer) a fim de que a chave estrangeira *Nome* da relação Animal-Dono não viole a integridade referencial. Isto evita a inconsistência de uma referência a Bruno da Silva na relação Animal-Dono quando Bruno da Silva não existir mais como "Pessoa".

Ordenação Topológica

A ordenação topológica e os diagramas de Hasse podem ser usados para representarem problemas de agendamento de tarefas.

EXEMPLO 23 Ernane e seus irmãos gerenciam uma marcenaria, nas montanhas da Serra do Mar, que fabrica cadeiras de balanço com acentos de almofada. O processo de fabricação pode ser dividido em algumas tarefas, algumas

das quais têm como pré-requisitos outras tarefas. A tabela a seguir mostra as tarefas envolvidas na fabricação das cadeiras de balanço, seus pré-requisitos e o número de horas necessário para realizar cada uma:

Tarefa	Tarefas pré-requeridas	Horas para realização
1. Seleção da madeira	Nenhuma	3,0
2. Entalhamento dos arcos	1	4,0
3. Entalhamento do acento	1	6,0
4. Entalhamento do encosto	1	7,0
5. Entalhamento dos braços	1	3,0
6. Seleção do tecido	Nenhuma	1,0
7. Costura da almofada	6	2,0
8. Montagem do acento e do encosto	3,4	2,0
9. Fixação dos braços	5,8	2,0
10. Fixação dos arcos	2,8	3,0
11. Aplicação de verniz	9, 10	5,0
12. Colocação da almofada	7, 11	0,5

Podemos definir uma ordenação parcial no conjunto de tarefas por

$$x \leq y \leftrightarrow \text{tarefa } x = \text{tarefa } y \text{ ou tarefa } x \text{ é um pré-requisito para a tarefa } y$$

É fácil ver que esta relação é reflexiva, anti-simétrica e transitiva. Além disso,

$$x < y \leftrightarrow \text{tarefa } x \text{ é um pré-requisito para a tarefa } y$$

No diagrama de Hasse para esta ordenação parcial, os vértices são tarefas; incluiremos em cada vértice a informação referente ao tempo necessário para a realização da tarefa. Além disso, como de costume, orientaremos o diagrama de forma que se $x < y$, então x está à esquerda de y ao invés de abaixo de y . Assim, todo o gráfico deve ser percorrido da esquerda para direita ao invés de baixo para cima. Este tipo de diagrama para agendamento de tarefas é comumente chamado de **diagrama PERT (program evaluation and review technique)**, originalmente desenvolvido para nortear a construção de submarinos da marinha, mas útil para o gerenciamento de qualquer projeto complexo que possa ser dividido em subtarefas. O diagrama PERT para a fabricação de cadeiras de balanço é mostrado na Fig. 4.8, com os nomes das tarefas representados por seus números e com setas apontando da(s) tarefa(s) pré-requisito(s) para a(s) tarefa(s) seguinte(s).

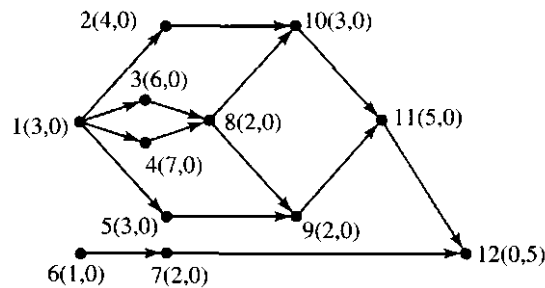


Figura 4.8

Tarefa	Tarefas pré- requeridas	Dias para realização
1. Aterramento do lote	Nenhuma	4
2. Fundação	1	3
3. Construção das paredes	2	7
4. Colocação das telhas	3	6
5. Revestimento externo das paredes	3	4
6. Instalação de encanamento e fiação	4,5	6
7. Colocação de janelas e portas	3	5
8. Revestimento interno das paredes	6	5
9. Pintura interior	7,8	5

Um projeto representado por um diagrama PERT precisa começar pelas tarefas representadas pelos vértices mais à esquerda do diagrama PERT e acabar nas tarefas representadas pelos vértices mais à direita do diagrama. Um limite superior para o tempo necessário para a realização do projeto pode ser obtido pela adição dos tempos de realização de cada tarefa, mas isto não leva em conta o fato de que duas tarefas podem ser realizadas paralelamente, tais como as tarefas 2 e 5 do Exemplo 23. Para obter o tempo mínimo necessário para realizar o projeto, podemos caminhar no diagrama, da esquerda para direita, computando o tempo mínimo para realização do trabalho desde o início do projeto até o término da tarefa representada pelo vértice em questão. Se um vértice x tem diversos vértices como pré-requisitos, todos esses pré-requisitos devem estar prontos antes que comecemos a tarefa x ; portanto devemos adicionar ao tempo necessário para a tarefa x o *maior* tempo de realização dentre os vértices que são seus pré-requisitos.

EXEMPLO 24 Vamos computar o tempo para o término de cada tarefa do Exemplo 23.

Tarefa 1: 3.0
Tarefa 2: $3.0 + 4.0 = 7.0$
Tarefa 3: $3.0 + 6.0 = 9.0$
Tarefa 4: $3.0 + 7.0 = 10.0$
Tarefa 5: $3.0 + 3.0 = 6.0$
Tarefa 6: 1.0
Tarefa 7: $1.0 + 2.0 = 3.0$

Tarefa 8: $\text{máx}(\text{tempo para terminar a tarefa 3, tempo para terminar a tarefa 4})$
 $+ \text{tempo para realizar a tarefa 8}$
 $= \text{máx}(9.0, 10.0) + 2.0 = 10.0 + 2.0 = 12.0$

Tarefa 9: $\text{máx}(\text{tempo para terminar a tarefa 5, tempo para terminar a tarefa 8})$
 $+ \text{tempo para realizar a tarefa 9}$
 $= \text{máx}(6.0, 12.0) + 2.0 = 12.0 + 2.0 = 14.0$

Tarefa 10: $\text{máx}(\text{tempo para terminar a tarefa 2, tempo para terminar a tarefa 8})$
 $+ \text{tempo para realizar a tarefa 10}$
 $= \text{máx}(7.0, 12.0) + 3.0 = 12.0 + 3.0 = 15.0$

Tarefa 11: $\text{máx}(\text{tempo para terminar a tarefa 9, tempo para terminar a tarefa 10})$
 $+ \text{tempo para realizar a tarefa 11}$
 $= \text{máx}(14.0, 15.0) + 5.0 = 15.0 + 5.0 = 20.0$

Tarefa 12: $\text{máx}(\text{tempo para terminar a tarefa 7, tempo para terminar a tarefa 11})$
 $+ \text{tempo para realizar a tarefa 12}$
 $= \text{máx}(3.0, 20.0) + 0.5 = 20.0 + 0.5 = 20.5$

Portanto, o número mínimo de horas necessárias para fabricar uma cadeira de balanço é 20.5. Podemos caminhar de trás para frente, a partir do vértice 12, selecionando a cada ponto de mais de uma possibilidade o vértice que contribui com o valor máximo. Isto nos dá a sequência de vértices

12, 11, 10, 8, 4, 1

ou, revertendo a ordem desta sequência,

1, 4, 8, 10, 11, 12

A soma dos tempos para realizar cada tarefa desta sequência é 20,5. Se qualquer dessas tarefas levar mais do que o tempo a ela designado originalmente para ser realizada, então o projeto todo levará mais do que 20,5 horas para ser realizado. Esta sequência de vértices é um **caminho crítico** ao longo do diagrama PERT — a realização dessas tarefas no tempo previsto é imperativa para que o projeto seja concluído sem atrasos. •

O caminho crítico em um diagrama PERT representa o menor tempo necessário para realização do projeto todo. Se uma tarefa que não faça parte do caminho crítico leva mais do que o tempo previsto para ser realizada, pode ser que o caminho crítico se altere a fim de incluir essa tarefa, transformando-se, assim, no gargalo que dificulta o término do projeto. Em um projeto complexo, o caminho crítico precisa ser recomputado repetidamente a fim de determinar onde se deve alocar recursos a fim de adiantar o máximo possível o andamento do projeto.

PRÁTICA 20

Calcule o tempo mínimo necessário para o término e as tarefas no caminho crítico do projeto de construção da casa da Prática 19. •

Dada uma ordenação parcial ρ em um conjunto finito, sempre há uma ordenação total σ que é uma extensão de ρ , significando que se $x \rho y$, então $x \sigma y$. O processo de **ordenação topológica** encontra uma ordenação total deste tipo a partir de uma dada ordenação parcial. Isto é, na verdade, um processo de ordenação na medida em que os elementos terminam totalmente ordenados; mas, uma vez que os elementos precisam estar antes parcialmente ordenados, este é um método de ordenação muito especializado.

Lembremo-nos de que em um conjunto finito parcialmente ordenado, um elemento é minimal se não tiver predecessores. Em um conjunto finito não vazio parcialmente ordenado, há pelo menos um elemento minimal. Para demonstrarmos isto, tomemos x como um elemento do conjunto. Se x não é minimal, então existe (pelo menos) um elemento y no conjunto para o qual $y \rho x$, $y \neq x$. Se y não for minimal, então existe um z no conjunto com $z \rho y$, $z \neq y$ e assim por diante. Como o conjunto é finito, este processo não pode prosseguir indefinidamente, portanto algum elemento precisa ser minimal. Um elemento minimal em um diagrama de Hasse não possui elementos abaixo dele; um elemento minimal em um diagrama PERT não tem elementos à sua esquerda.

Para construir uma ordenação topológica, tome um elemento minimal x_1 do conjunto finito parcialmente ordenado (S, ρ) . (Se houver mais de uma possibilidade, pegue qualquer um aleatoriamente.) Agora consideremos o conjunto $S - \{x_1\}$; este conjunto ainda é um conjunto finito parcialmente ordenado; escolha, então, aleatoriamente, um elemento minimal x_2 deste conjunto. Considere agora o conjunto $S - \{x_1, x_2\}$, escolha aleatoriamente um elemento minimal x_3 , e prossiga assim por diante, até que todos os elementos tenham sido usados. A ordenação

$$x_1 < x_2 < x_3 < \dots < x_n$$

é uma ordenação total. Para demonstrarmos que ele é uma extensão de ρ , suponha que $x_i \rho x_j$. Então x_j foi escolhido como um elemento minimal após x_i ter sido escolhido e excluído do conjunto; caso contrário, $x_i \rho x_j$ significaria que x_j não era minimal. Portanto, $i < j$ e $x_i < x_j$.

EXEMPLO 25

Uma ordenação topológica da ordenação do Exemplo 23 é

6, 1, 7, 2, 3, 5, 4, 8, 10, 9, 11, 12

Na Fig. 4.8, tanto 6 como 1 são minimais e podem ser escolhidos como primeiros elementos. Se o 6 for escolhido e removido do conjunto então, como mostrado na Fig 4.9, o 1 e o 7 serão minimais. Se o 1 for então escolhido e removido da figura (Fig. 4.10), então 2, 3, 4, 5 e 7 tornam-se minimais, e qualquer deles pode ser o próximo escolhido. O processo continua até que todos os vértices tenham sido escolhidos. Se os irmãos de Emanuele forem para a cidade e ele ficar sozinho para montar as cadeiras de balanço, a ordenação topológica fornece-lhe uma ordem em que ele pode realizar as tarefas sequencialmente.

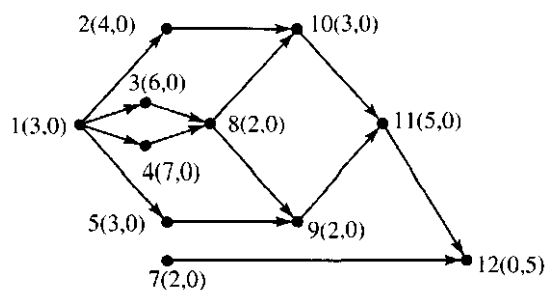


Figura 4.9

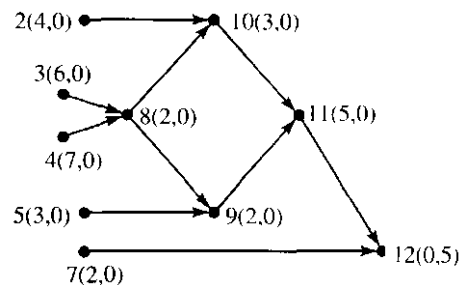


Figura 4.10



PRÁTICA 21 Encontre outra ordenação topológica para a ordenação parcial do Exemplo 23. •

PRÁTICA 22 Encontre uma ordenação topológica para a ordenação parcial da Prática 19. •

Revisão da Seção 4.2

Técnicas

- Operações select, project e join em um banco de dados relacionais
- Formulação de pesquisas em banco de dados relacionais usando álgebra relacionai, SQL e cálculo relacionai
- Construção de diagramas PERT a partir de uma tabela de tarefas
- Obtenção do caminho crítico em um diagrama PERT
- Realização de uma ordenação topológica em um conjunto parcialmente ordenado

Idéias Principais

Um banco de dados relacionais usa tabelas de relações para modelar os objetos e as relações em uma aplicação.

As operações de banco de dados select, project e join são operações em relações (conjuntos de tuplas).

Pesquisas em bancos de dados relacionais podem ser formuladas usando as operações select, project e join, comandos SQL ou notações fornecidas pela teoria de conjuntos e lógica predicada.

Diagramas PERT são grafos de conjunto parcialmente ordenados que representam tarefas e os pré-requisitos para essas tarefas.

Uma ordenação topológica estende uma ordenação topológica em um conjunto finito a uma ordenação total.

Exercícios 4.2

Os Exercícios 1 a 18 são relacionados à mesma aplicação.

- ★ 1. Uma biblioteca mantém um banco de dados de seus livros. As informações sobre o autor incluem o nome do autor, seu país de origem e os títulos dos livros do autor. As informações referentes aos livros incluem o título, o número ISBN, a editora e o assunto. Os autores e os livros são as entidades da aplicação e "escreve" é um relacionamento entre essas entidades. Esboce um diagrama E-R para esta aplicação. Na ausência de qualquer regra de negócio, o que deve ser assumido no que tange à relação binária "escreve" com relação a ser um-para-um, um-para-vários etc?
2. Admita que uma regra de negócio é que os autores são unicamente identificados por seus nomes. Isto muda sua resposta para a última pergunta do Exercício 1 ?
3. Em um modelo relacionai do banco de dados da biblioteca existe uma relação autores, uma relação livros e uma relação escreve. Forneça o cabeçalho da tabela de cada uma das relações, sublinhando a chave primária. Justifique a escolha das chaves primárias. Qual a regra de negócio implícita (além da apresentada no Exercício 2) está por trás da escolha dos atributos do autor?

Nos Exercícios 4 a 14, use as seguintes tabelas de relações e escreva as tabelas de relações que resultam das operações indicadas.

Autor		
<i>Nome</i>	<i>País</i>	<i>Título</i>
Dorothy King	Inglaterra	Jardinagem de Primavera
Jon Nkoma	Quênia	Pássaros da África
Won Lau	China	Pinturas em Porcelana
Bert Kovalsco	EUA	Basquete Moderno
Jimmy Chan	China	Pinturas em Porcelana
Dorothy King	Inglaterra	Anuário de Outono
Jane East	EUA	Jardinagem de Primavera

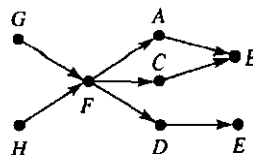
Livro			
<i>Título</i>	<i>ISBN</i>	<i>Editora</i>	<i>Assunto</i>
Jardinagem de Primavera	816-35421-8	Harding	Natureza
Pinturas em Porcelana	364-87547-8	Bellman	Arte
Pássaros da África	115-67813-3	Loraine	Natureza
Jardinagem de Primavera	816-89335-8	Swift-Key	Natureza
Basquete Moderno	778-53705-7	Harding	Arte
Anuário de Outono	414-88506-9	Harding	Natureza

Escreve		
<i>Nome</i>	<i>Título</i>	<i>ISBN</i>
Jimmy Chan	Pinturas em Porcelana	364-87547-8
Dorothy King	Anuário de Outono	414-88506-9
Jane East	Jardinagem de Primavera	816-89335-8
Bert Kovalsko	Basquete Moderno	778-53705-7
Won Lau	Pinturas em Porcelana	364-87547-8
Jon Nkoma	Pássaros da África	115-67813-3
Dorothy King	Jardinagem de Primavera	816-35421-8

- ★4. **Select** Autor **where** País = "EUA" **giving** Tabela 1.
5. **Select** Escreve **where** Nome = "Dorothy King" **giving** Tabela 2.
6. **Select** Livro **where** Editora = "Bellman" **or** Editora = "Swift-Key" **giving** Tabela 3.
7. **Select** Livro **where** Editora = "Harding" **and** Assunto = "Arte" **giving** Tabela 4.
- ★8. **Project** Autor **over** (Nome, Título) **giving** Tabela 5.
9. **Project** Autor **over** (Nome, País) **giving** Tabela 6.
10. **Project** Livro **over** (Editora, Assunto) **giving** Tabela 7.
11. **Project** Livro **over** (Título, ISBN, Assunto) **giving** Tabela 8.
- ★12. **Join** Livro **and** Escreve **over** Título **and** ISBN **giving** Tabela 9.
13. **Join** Autor **and** Escreve **over** Nome **and** Título **giving** Tabela 10.
14. O que daria errado se comandássemos um join de Autor e Livro através de *Título*?

Nos Exercícios 15 a 18, de posse das tabelas de relações apresentadas acima, exprima cada pesquisa na forma de álgebra relacionai, SQL e cálculo relacionai. Forneça também o resultado de cada pesquisa.

- ★15. Forneça o título de todos os livros escritos por autores norte-americanos.
17. Forneça o nome de todos os autores que escreveram livros sobre natureza.
18. Indique as editoras de todos os livros de arte cujos autores moram nos Estados Unidos.
19. Realize uma ordenação topológica no conjunto parcialmente ordenado apresentado na figura a seguir.



Exercício 19

20. As tarefas a seguir são necessárias para a montagem de uma bicicleta. Como fabricante, você deve escrever uma ordem de instruções sequenciais para o comprador seguir. Será que a ordem fornecida abaixo funciona? Forneça outra sequência que possa ser usada.

Tarefa	Tarefas pré-requeridas
1. Apertar os parafusos do quadro	nenhuma
2. Fixar o guidom no quadro	1
3. Fixar a coroa	1
4. Montar o pneu no aro da roda	nenhuma
5. Fixar a roda no quadro	1,4
6. Instalar o mecanismo de freio	2,3,5
7. Colocar os pedais	6
8. Fixar o selim	1
9. Ajustar a altura do selim	7,8

- ★21. Construa um diagrama PERT para a seguinte tabela de tarefas:

Tarefa	Tarefas pré-requeridas	Tempo de realização
A	E	3
B	C, D	5
C	A	2
D	A	6
E	nenhuma	2
F	A, G	4
G	E	4
H	B, F	1

22. Construa um diagrama PERT para a seguinte tabela de tarefas:

Tarefa	Tarefas pré-requeridas	Tempo de realização
1	2	4
2	3	2
3	8	5
4	3	2
5	4,7	2
6	5	1
7	3	3
8	nenhuma	5

23. Calcule o tempo mínimo necessário para o término e os vértices do caminho crítico para o problema do Exercício 21.
24. Calcule o tempo mínimo necessário para o término e os vértices do caminho crítico para o problema do Exercício 22.
- ★25. Encontre uma ordenação topológica para o problema do Exercício 21.
26. Encontre uma ordenação topológica para o problema do Exercício 22.

Seção 4.3 Funções

Nesta seção discutiremos funções, que são casos especiais de relações binárias. Esta abordagem de funções é um tanto o quanto sofisticado; por isso, nossa abordagem será conduzida de forma a construir a teoria de forma gradativa.

A Definição

A palavra *função* é muito comum, mesmo em contextos não técnicos. Um jornal pode ter um artigo sobre o aumento neste ano dos salários iniciais para recém-formados. O artigo poderia dizer algo como "O aumento dos salários varia de acordo com a área" ou "O aumento dos salários é uma função da área". O jornal pode ilustrar este relacionamento de função com um gráfico como o mostrado na Fig. 4.11, que mostra que cada área tem um valor (representado graficamente por uma coluna) que mostra o aumento dos salários associados. Nenhuma área tem mais de um valor associado, mas tanto a Física quanto as Artes têm o mesmo valor, 3%.

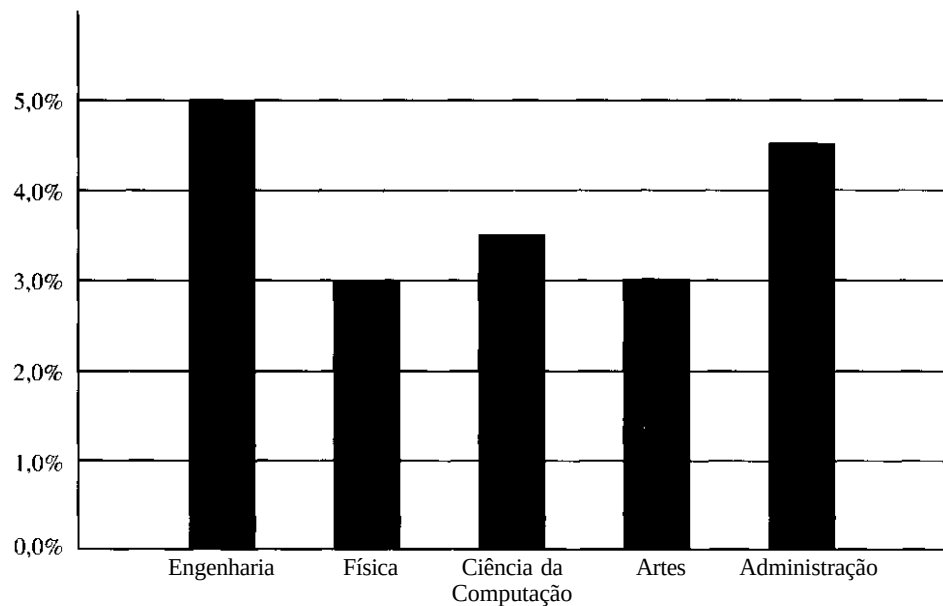


Figura 4.11

Naturalmente, também usamos funções matemáticas na álgebra e no cálculo. A equação $g(x) = x^3$ expressa uma relação funcional entre os valores de x e os valores correspondentes que resultam da substituição, na equação, de x por seus valores. Portanto, um valor 2 para x tem o valor $2^3 = 8$ associado. (Este número é expresso como $g(2) = 8$.) Analogamente, $g(1) = 1^3 = 1$, $g(-1) = (-1)^3 = -1$, e assim por diante. Um único valor de $g(x)$ é associado a cada valor de x . Se traçarmos esta função em um sistema de coordenadas ortogonal, os pontos $(2, 8)$, $(1, 1)$ e $(-1, -1)$ seriam pontos do gráfico. Se fizermos com que x tome quaisquer valores reais, o gráfico resultante será a curva contínua mostrada na Fig. 4.12.

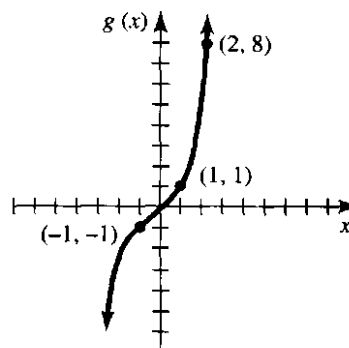


Figura 4.12

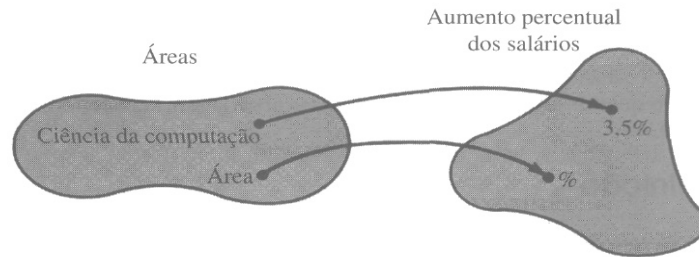


Figura 4.13

A função do exemplo dos aumentos de salários pode ser descrita da seguinte maneira: definimos o que é ilustrado pela Fig. 4.13, isto é, que a função sempre inicia com uma área arbitrária e que um aumento de salários está associado a esta área. A associação propriamente dita é descrita pelos pares ordenados $\{(Engenharia, 5,0\%), (Física, 3,0\%), (Ciência da Computação, 3,5\%), (Artes, 3,0\%), (Administração, 4,5\%)\}$.

Para o exemplos algébrico $g(x) = x^3$, a Fig. 4.14 mostra que a função sempre começa por um número real dado e associa a ele um outro número real. A associação propriamente dita é descrita por $\{(x, g(x)) \mid g(x) = x^3\}$, ou simplesmente $g(x) = x^3$. Este conjunto inclui $(2, 8)$, $(1, 1)$, $(-1, -1)$ mas, como ele é um conjunto infinito, não podemos listar todos seus elementos; por isso temos que descrevê-los.

Pelos exemplos acima, podemos concluir que existem três componentes de uma função: (1) um conjunto de valores iniciais, (2) um conjunto do qual os valores associados são tomados e (3) a associação propriamente dita. O conjunto dos valores iniciais é chamado de *domínio* da função, e o conjunto com os valores associados é chamado de *contradomínio* da função. Portanto, o domínio e o contradomínio representam o elenco dos valores passíveis de serem usados pela função. (Esta terminologia é consistente com o uso da palavra *domínio* quando abordamos wffs predicativas na Seção 1.2, onde o domínio de uma interpretação é um conjunto dos valores que uma variável pode assumir e que as constantes podem representar. De forma semelhante, o domínio D_i de um atributo A_i em uma relação de banco de dados, discutida na Seção 4.2, é um conjunto com os possíveis valores para o atributo.)

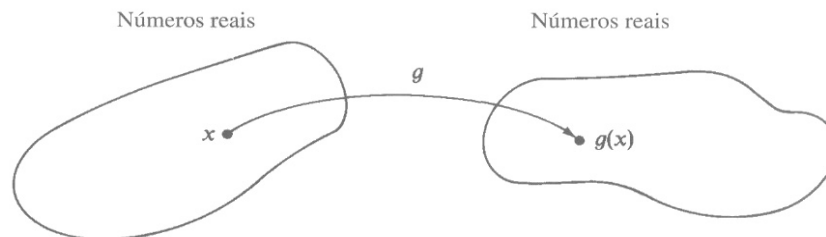


Figura 4.14

A representação de uma função arbitrária/é mostrada na Fig. 4.15. Neste caso,/é uma função de S em T , simbolizada por $f: S \rightarrow T$. A associação propriamente dita é um conjunto de pares ordenados da forma (s, t) onde $s \in S$ e $t \in T$ e t é o valor de T que a função associa ao valor s de S ; $t = f(s)$. Portanto, a associação é um subconjunto de $S \times T$ (uma relação binária em $S \times T$). Mas a propriedade importante nesta relação é que cada elemento de S deve ter um, e apenas um, valor associado de T , isto é, cada s de S aparecerá exatamente uma vez como primeiro componente de um par ordenado (s, t) . (Esta propriedade não impede que um determinado valor de T apareça mais de uma vez.)

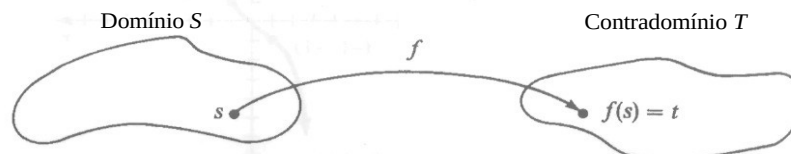


Figura 4.15

Agora estamos prontos para uma definição formal de função.

Definições: Terminologias para Funções

Sejam S e T conjuntos. Uma **função (aplicação)** de S em T , $f: S \rightarrow T$, é um subconjunto de $S \times T$ onde cada elemento de S aparece exatamente uma única vez como primeiro componente de um par ordenado. S é o **domínio** e T é o **contradomínio** da função. Se (s, t) pertence à função, então t é denotado por $f(s)$; t é a **imagem** de s por f , s é a **pré-imagem** de t por f e diz-se que f leva s em t . Para $A \subseteq S$, $f(A)$ denota $\{f(a) \mid a \in A\}$.

Uma função de S em T é um subconjunto de $S \times T$ com certas restrições sobre o que os pares ordenados podem conter. Por isso, nos referimos às funções como um tipo especial de relação binária. Pela definição de funções, relações um-para-vários (vários-para-vários) não pode ser uma função. Além disso, todo elemento de S precisa ser usado como primeiro componente.

Já falamos bastante sobre os valores dos conjuntos S e T , mas, como nosso exemplo de aumentos de salários mostra, estes valores não são necessariamente números, nem a associação precisa ser necessariamente uma equação.

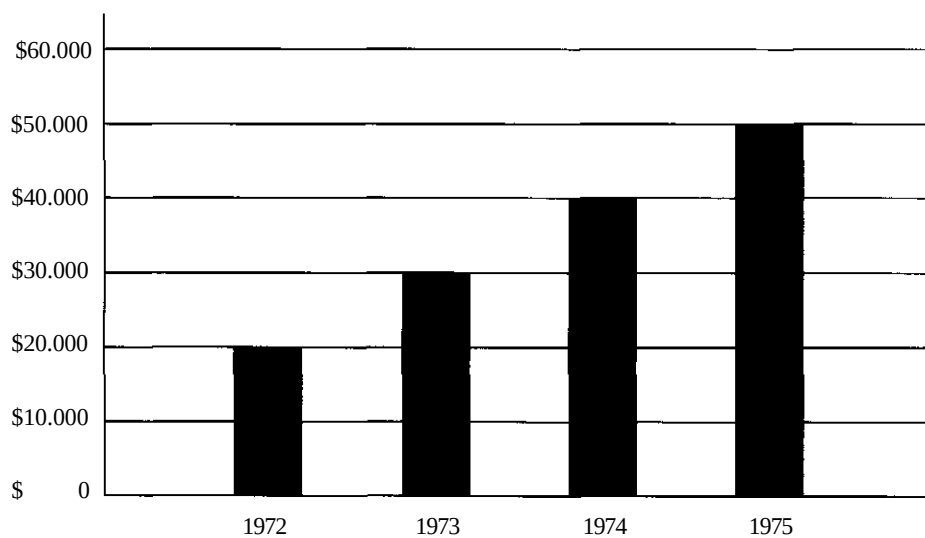
PRÁTICA 23

Quais dos itens a seguir definem funções do domínio no contradomínio indicados? Para as que não forem, justifique sua resposta.

- $f: S \rightarrow T$ onde $S = T = \{1, 2, 3\}$, $f = \{(1, 1), (2, 3), (3, 1), (2, 1)\}$
- $g: \mathbb{Z} \rightarrow \mathbb{Z}$ onde g é definida como $g(x) = |x|$ (o valor absoluto de x)
- $h: \mathbb{N} \rightarrow \mathbb{N}$ onde h é definido por $h(x) = x - 4$
- $f: S \rightarrow T$ onde S é o conjunto de todas as pessoas em sua cidade, T é o conjunto dos números de CPF e f associa cada pessoa a seu número de CPF
- $g: S \rightarrow T$ onde $S = \{1972, 1973, 1974, 1975\}$, $T = \{\$20.000, \$30.000, \$40.000, \$50.000, \$60.000\}$, e g é definido pelo gráfico da Fig. 4.16.
- $h: S \rightarrow T$ onde S é o conjunto de todos os polinômios quadráticos em x com coeficientes inteiros, $T = \mathbb{Z}$ e h é definido por $h(ax^2 + bx + c) = b + c$
- $f: \mathbb{R} \rightarrow \mathbb{R}$ onde f é definida por $f(x) = 4x - 1$
- $g: \mathbb{N} \rightarrow \mathbb{N}$ onde g é definida por

$$g(x) = \begin{cases} x + 3 & \text{se } x \geq 5 \\ x & \text{se } x \leq 5 \end{cases}$$

♦



Lucros da American Earthworm Corp.

Figura 4.16

PRÁTICA 24

- Para $f: \mathbb{Z} \rightarrow \mathbb{Z}$ dada por $f(x) = x^2$ qual a imagem de -4 ?
- Quais as pré-imagens de 9 ?

•

EXEMPLO 26 Quando estudamos definições recursivas na Seção 2.3, falamos de seqüências, onde uma seqüência 5 pode ser escrita como

$$S(1), S(2), S(3), \dots$$

Se mudarmos a notação para

$$f(1), f(2), f(3), \dots$$

veremos que uma seqüência não é nada além de uma lista de valores de uma função/cujo domínio é o conjunto dos inteiros positivos. De fato, os algoritmos que construímos para calcular os valores dessas seqüências eram funções Pascal (*functions*).

Além disso, na Seção 2.3, falamos de operações recursivas tais como a^n , onde a é um número real fixo diferente de zero e $n \geq 0$. Isto também é simplesmente a função $f(n) = a^n$, cujo domínio é $\mathbb{N}$. •

A definição de funções inclui funções de mais de uma variável. Podemos ter uma função $f: S_1 \times S_2 \times \dots \times S_n \rightarrow T$ que associe a cada n -upla ordenada de elementos $(s_1, s_2, \dots, s_n)$, $s_i \in S_i$, um único elemento de T .

EXEMPLO 27 $f: \mathbb{Z} \times \mathbb{N} \times \{1, 2\} \rightarrow \mathbb{R}$ é dada por $f(x, y, z) = x^y + z$. Então $f(-4, 3, 1) = (-4)^3 + 1 = -64 + 1 = -63$. ♦

EXEMPLO 28 Definimos na Seção 3.1 uma operação unária em um conjunto S como sendo uma operação que associa a cada elemento x de S um único elemento $x^\#$, também de S . Isto significa que operações unárias em S são funções com domínio e contradomínio S . Também definimos uma operação binária $\circ$ em um conjunto S como a associação de um único elemento de S , denotado por $x \circ y$ a cada par (x, y) de elementos de S . Portanto, uma operação binária em S é uma função com domínio $S \times S$ e contradomínio S . •

Ressaltamos mais uma vez: valores de domínio e de contradomínio não são necessariamente números.

EXEMPLO 29 Seja S o conjunto de todas as cadeias de caracteres de tamanho finito. Então a associação que relaciona a cada cadeia o número de caracteres que contém é uma função de domínio S e contradomínio $\mathbb{N}$ (permitimos a "cadeia vazia", cujo número de caracteres é zero). •

EXEMPLO 30 Qualquer wff proposicional com n letras de afirmações define uma função com domínio $\{V, F\}^n$ e contradomínio $\{V, F\}$. O domínio consiste em todas as n -uplas de valores V - F ; a cada n -upla está associado um único valor de V ou F . A tabela-verdade da wff nos dá a associação. Por exemplo, se a wff é $A \vee B'$, então a tabela-verdade

A	B	B'	$A \vee B'$
V	V	F	V
V	F	V	V
F	V	F	F
F	F	V	V

nos diz que a imagem da dupla (F, V) por esta função é F . Se chamarmos esta função de w , então $w(F, V) = F$. •

PRÁTICA 25 Seja a função definida pela wff $A \rightarrow (B \wedge C')$ denotada por f . Qual o valor de $f(V, V, F)$? E de $f(F, V, F)$? •

O Exemplo a seguir define duas funções que são freqüentemente úteis na análise de algoritmos.

EXEMPLO 31 A **função chão** $\lfloor x \rfloor$ associa a cada número real x o maior inteiro menor ou igual a x . A **função teto** $\lceil x \rceil$ associa a cada número real x o menor inteiro maior ou igual a x . Portanto, $\lfloor 2, 8 \rfloor = 2$, $\lceil 2, 8 \rceil = 3$, $\lfloor -4, 1 \rfloor = -5$ e $\lceil -4, 1 \rceil = -4$. Tanto a função chão quanto a função teto são funções de $\mathbb{R}$ em $\mathbb{Z}$.

PRÁTICA 26

- Esboce um gráfico para a função $\lfloor x \rfloor$.
- Esboce um gráfico para a função $\lceil x \rceil$.

Partes da Definição

Mencionamos que a definição da função $f: S \rightarrow T$ inclui três partes — o conjunto domínio S , o conjunto contradomínio T e a associação propriamente dita. Será que há necessidade de tudo isso? Por que não podemos apenas escrever uma equação tal como $g(x) = x^3$ para definir uma função?

A resposta mais simples é que nem todas as associações funcionais podem ser descritas por uma equação (veja o Exemplo 29). Mas há mais motivos — vamos verificar o caso em que podemos usar uma equação para descrever a associação, tal como $g: \mathbb{R} \rightarrow \mathbb{R}$ onde $g(x) = x^3$. Tanto na álgebra como no cálculo, é comum dizer "consideremos a função $g(x) = x^3$ ", significando que a equação *é* a função. Tecnicamente, no entanto, a equação apenas define como associar valores.

Por outro lado, a função $f: \mathbb{Z} \rightarrow \mathbb{R}$ dada por $f(x) = x^3$ não é a mesma que g . O domínio é diferente, o que muda o conjunto de pares ordenados. O gráfico de f consiste em pontos discretos (separados) (veja a Fig. 4.17). Muitas das funções nas quais estamos interessados serão deste tipo. Em um computador digital, a informação é processada em uma série de passos distintos (discretos). Mesmo em situações onde um valor varie continuamente com outro, aproximamos através de pequenos intervalos discretos, como a aproximação do gráfico de $g(x)$ (Fig. 4.12) pelo gráfico de $f(x)$ (Fig. 4.17).

Finalmente, vejamos a função $k: \mathbb{R} \rightarrow \mathbb{C}$ dada por $k(x) = x^3$. A equação e o domínio são iguais aos da $g(x)$; o contradomínio é que foi aumentado, o que, no entanto, não afeta os pares ordenados. Será, porém, que $k(x)$ é considerada a mesma função que $g(x)$? A resposta é não, mas para vermos a razão da diferença entre elas teremos que esperar até termos visto a propriedade *sobrejetiva* de funções. Então veremos que g é sobrejetiva, enquanto que k não o é. Por isso, não as consideraremos a mesma função.

Em resumo, uma definição completa de uma função requer que se forneça seu domínio, seu contradomínio e a associação, sendo que esta última pode ser fornecida através de uma descrição verbal, um gráfico, uma equação ou uma coleção de pares ordenados. Suponha que estamos tentando demonstrar que duas funções com os mesmos domínios e contradomínios são iguais. Então precisamos mostrar que suas associações são iguais. Então precisamos mostrar que as associações são as mesmas. Isto pode ser feito mostrando que ambas as funções atuam da mesma forma em um elemento arbitrário do domínio, isto é, o levam ao mesmo elemento do contradomínio.

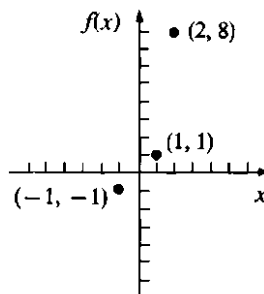


Figura 4.17

PRÁTICA 27 Sejam $S = \{1, 2, 3\}$ e $R = \{1, 4, 9\}$. A função $f: S \rightarrow R$ é definida por $f = \{(1, 1), (2, 4), (3, 9)\}$. A função $g: S \rightarrow R$ é definida pela equação

$$g(n) = \frac{\sum_{k=1}^n (4k - 2)}{2}$$

(Veja o Apêndice A para uma consulta sobre a notação de somatório.) Demonstre que $f = g$. •

Linguagens de Programação e Funções

A maioria das linguagens de programação oferecem algumas funções intrínsecas (já prontas) e também permitem ao programador definir suas próprias funções. Por exemplo, podemos desejar usar uma função que calcule a média de três valores reais. O código Pascal a seguir define uma função deste tipo.

```
function Média (x, y, z: real): real;
{retorna a média dos três valores reais}

begin {function Média}
  Média: = (x + y + z)/3.0;
end; {function Média}
```

O cabeçalho da função contém duas das três componentes essenciais à definição de uma função. A parte do cabeçalho entre parênteses indica que o domínio da função consiste em qualquer tripla de valores reais. Para qualquer tripla dessas, a função deve produzir um único resultado. A parte do cabeçalho após os dois-pontos indica que o contradomínio da função consiste em todos os números reais. A função irá associar um valor real a qualquer tripla de valores reais. Portanto, o cabeçalho da função estabelece o domínio e o contradomínio da função; o corpo da função, que neste caso é composto pelo único comando

Média: $=(x + y + z)/3.0$;

determina a associação.

Uma classe de linguagens de programação, chamada de **linguagens funcionais** entende as tarefas de programação em termos das funções que realizarão transformações em objetos. O programa contém descrições de funções, comumente de natureza recursiva, que fornecem regras computacionais para avaliação de funções. Por exemplo, uma função para computar o tamanho de uma lista de itens, escrita na linguagem de programação funcional ML (de meta/anguage) seria algo parecido com o trecho a seguir (neste caso, uma lista é entendida como um único primeiro elemento ou elemento "cabeça", seguido dos demais elementos — o "corpo" (tail) da lista):

```
fun tamanho (lista) =
  if lista = [ ]
  then 0
  else 1 + tamanho (tail (lista));
```

O domínio desta função é o conjunto das listas. O contradomínio (que é inferido e não é definido explicitamente) é o conjunto $\mathbb{N}$. A associação é definida pelo corpo da função, que é uma regra computacional recursiva. Se a lista for vazia, o valor de retorno da função é 0; caso contrário, a função é aplicada ao corpo da lista para que o tamanho do mesmo seja computado e então acrescido de 1 a fim de remontar o tamanho da lista original.

Ao contrário de uma linguagem de programação procedural como Pascal, não precisamos fornecer os detalhes de como esta regra computacional será tratada a tempo de execução, seguindo, por exemplo, os ponteiros de uma lista encadeada. Nem precisamos especificar o tipo de dados dos elementos da lista; esta função funcionará em listas compostas por elementos de qualquer tipo. Em particular, serviria para implementar a função do Exemplo 29 se a cadeia de caracteres fosse tratada como uma lista.

De forma mais geral, independente do tipo da linguagem de programação usada, qualquer programa pode ser visto como uma única função que associa valores de saídas únicos a cada valor de entrada. (Pode parecer que a restrição de unicidade é violada se o programa se baseia, por exemplo, em sinais de interrupção ou em um gerador de números aleatórios, mas os valores fornecidos destas maneiras devem ser considerados parte da coleção de valores de entrada.)

Propriedades das Funções

Funções Sobrejetivas

Seja $f: S \rightarrow T$ uma função arbitrária com domínio S e contradomínio T (veja a Fig. 4.18). Parte da definição de uma função é que cada elemento de S tem uma imagem por f e que todas as imagens são elementos de T ; o conjunto / destas imagens é chamado de **conjunto imagem** da função/. Assim, $f = \{f(s) \mid s \in S\}$, ou $I = f(S)$. Obviamente $I \subseteq T$; o conjunto imagem / aparece sombreado na Fig. 4.19. Se acontecer de $f = T$, isto é, o conjunto imagem ser igual ao contradomínio, a função é dita uma função *sobrejetiva*.

Definição: Função Sobrejetiva

Uma função $f: S \rightarrow T$ é uma função **sobrejetiva** se a imagem de $f(f(S))$ é igual ao contradomínio de $f(T)$.

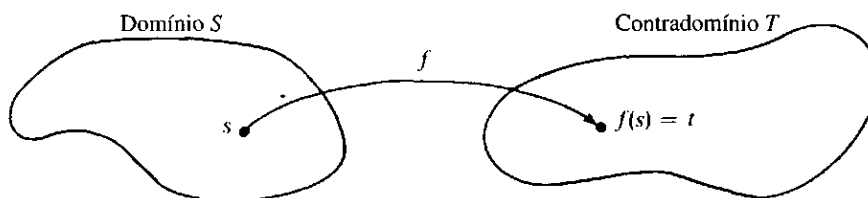


Figura 4.18

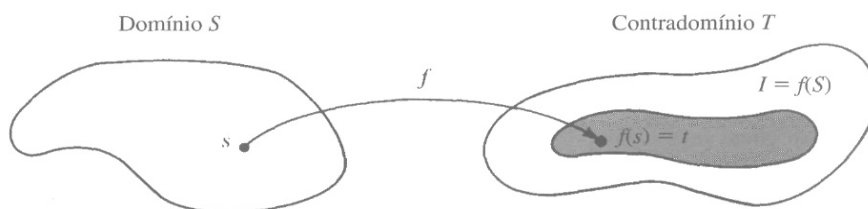


Figura 4.19

Em toda função com imagem I e contradomínio T , $I \subseteq T$. Para mostrar que uma função é sobrejetiva, precisamos mostrar que $T \subseteq I$ e, então, teremos mostrado que $I = T$. Precisamos mostrar, então, que um elemento arbitrário do contradomínio é um elemento da imagem, isto é, que algum elemento do domínio é levado nele. Por outro lado, se pudermos exibir um elemento do contradomínio que não seja imagem de qualquer elemento do domínio, então teremos provado que a função não é sobrejetiva.

EXEMPLO 32 A função $g: \mathbb{R} \rightarrow \mathbb{R}$ definida por $g(x) = x^3$ é uma função sobrejetiva. Para provar isto, seja r um número real arbitrário e seja $x = \sqrt[3]{r}$. Então x é um número real, de forma que x pertence ao domínio de g e $g(x) = (\sqrt[3]{r})^3 = r$. Daí, qualquer número no contradomínio é imagem por g de algum número do domínio. A função $k: \mathbb{R} \rightarrow \mathbb{C}$ dada por $k(x) = x^3$ não é sobrejetiva. Existem diversos números complexos (i , por exemplo) que não podem ser obtidos pela elevação de um número real ao cubo. Portanto g e k não são as mesmas funções. •

EXEMPLO 33 Seja $f: \mathbb{Q} \rightarrow \mathbb{Q}$ definida por $f(x) = 3x + 2$. Para verificarmos se f é sobrejetiva, seja $q \in \mathbb{Q}$. Desejamos um $x \in \mathbb{Q}$ tal que $f(x) = 3x + 2 = q$. Então resolvemos esta relação para x e encontramos $x = (q - 2)/3$ como único valor possível e observamos que ele também pertence a $\mathbb{Q}$. Portanto, q é a imagem de um elemento de $\mathbb{Q}$ por f e f é sobrejetiva. No entanto, a função $h: \mathbb{Z} \rightarrow \mathbb{Q}$ definida por $h(x) = 3x + 2$ não é sobrejetiva porque existem diversos valores de $q \in \mathbb{Q}$, por exemplo 0, para os quais a equação $3x + 2 = q$ não tem solução inteira. •

PRÁTICA 28 Quais das funções encontradas na Prática 23 são sobrejetivas?

PRÁTICA 29 Suponha uma função $f: \{V, F\}^n \rightarrow \{V, F\}$ definida por uma wff proposicional P (veja o Exemplo 30). Sob que duas condições de P , f não será uma função sobrejetiva? •

Funções Injetivas

A definição de funções garante que há apenas uma imagem para cada elemento do domínio. No entanto, um dado elemento da imagem pode ter mais de uma pré-imagem. Em nosso primeiro exemplo de função (aumentos de salários), tanto os salários dos formandos em Física quanto os dos formandos em Artes sofreram um aumento anual de 3%. Esta função não é injetiva (um-para-um).

Definição: Função Injetiva (um-para-um)

Uma função $f: S \rightarrow T$ é **injetiva** ou **um-para-um** se nenhum elemento de T for imagem por f de dois elementos distintos de S .

A idéia de injetividade aqui é a mesma das relações binárias discutidas na Seção 4.1, exceto que todo elemento de S precisa aparecer como uma primeira componente em um par ordenado (consequência da definição de função).

Para demonstrar que uma função é injetiva, assumimos que existem elementos s_1 e s_2 de S tais que $f(s_1) = f(s_2)$ e então devemos mostrar que $s_1 = s_2$. Para provar que uma função não é injetiva, devemos exibir um contra-exemplo, um elemento na imagem que tenha duas pré-imagens no domínio.

EXEMPLO 34 A função $g: \mathbb{R} \rightarrow \mathbb{R}$ definida por $g(x) = x^3$ é injetiva porque se x e y são números reais com $g(x) = g(y)$, então $x^3 = y^3$ e $x = y$. A função $f: \mathbb{R} \rightarrow \mathbb{R}$ definida por $f(x) = x^2$ não é injetiva porque, por exemplo, $f(2) = f(-2) = 4$. No entanto, a função $h: \mathbb{N} \rightarrow \mathbb{N}$ definida como $h(x) = x^2$ é injetiva porque se x e y são inteiros não-negativos com $h(x) = h(y)$, então $x^2 = y^2$; como x e y são ambos não-negativos, $x = y$. •

PRÁTICA 30 Quais das funções da Prática 23 são injetivas? •

EXEMPLO 35 A função chão e a função teto do Exemplo 31 são notoriamente não-injetivas. Isto também fica claro no gráfico dessas funções (Prática 26), que tem diversas seções horizontais, indicando que diferentes valores do domínio $\mathbb{R}$ são levados pela função aos mesmos valores do contradomínio $\mathbb{Z}$. •

Bijeções

Definição: Função Bijetiva

Uma função $f: S \rightarrow T$ é **bijetiva** se for, ao mesmo tempo, injetiva e sobrejetiva.

EXEMPLO 36 A função $g: \mathbb{R} \rightarrow \mathbb{R}$ definida por $g(x) = x^3$ é uma bijeção. A função do item (g) da Prática 23 é uma bijeção. A função $f: \mathbb{R} \rightarrow \mathbb{R}$ definida por $f(x) = x^2$ não é uma bijeção (não é injetiva) e nem a função $k: \mathbb{R} \rightarrow \mathbb{C}$ definida por $k(x) = x^3$ (que não é sobrejetiva). ♦

Contagem de Funções

Suponha que S e T são conjuntos finitos, sigamos que $|S| = m$ e $|T| = n$. O que podemos afirmar sobre o número de funções com as diversas propriedades que levam S em T ? Primeiro, vamos apenas contar o número de funções $f: S \rightarrow T$ sem a restrição de gozar de qualquer propriedade. Podemos usar o Princípio da Multiplicação porque podemos imaginar que desejamos definir uma função atribuindo uma imagem para cada um dos m elementos de seu domínio. Isto nos dá uma seqüência de m escolhas. Cada passo tem n possibilidades, uma vez que todo elemento de S pode ser levado a qualquer elemento de T . Portanto, o número de funções possíveis é

$$\underbrace{n \times n \times n \times \cdots \times n}_{m \text{ fatores}} = n^m$$

Quantas funções injetivas existem de S em T ? Precisamos ter $m \leq n$ ou não poderemos ter qualquer função injetiva. (Todos os elementos de S devem ser levados em elementos diferentes de T e se $m > n$, então existem muitos elementos em S para que seja possível criar uma relação um-para-um. Na verdade, este argumento é o princípio da casa do pombo.) Podemos novamente resolver este problema através de uma seqüência de atribuições de uma imagem para cada elemento em S , mas desta vez não poderemos mais usar qualquer elemento da imagem que já tenha sido usado antes. Pelo Princípio da Multiplicação, temos, como resultado, o produto que começa com os fatores

$$n(n-1)(n-2) \dots$$

e precisa conter um total de m fatores, portanto, o resultado é

$$\begin{aligned} n(n-1)(n-2) \dots [n-(m-1)] &= n(n-1)(n-2) \dots (n-m+1) \\ &= \frac{n!}{(n-m)!} = P(n, m) \end{aligned}$$

E quantas funções sobrejetivas existem de S em T ? Desta vez precisamos ter $m \geq n$ «a fim de que haja suficientes valores do domínio para fornecer pré-imagens para todos os elementos do contradomínio. Nossa estratégia será subtrair o número de funções que não sejam sobrejetivas do número total de funções, que já conhecemos. Para contar o número de funções não-sobrejetivas, usaremos o Princípio da Inclusão e Exclusão.

Enumeremos os elementos do conjunto T como $t_1, \dots, t_n$. Para cada i , $1 \leq i \leq n$, seja A_i das funções de S a T que não levam nenhum elemento do domínio ao elemento t_i . (Estes conjuntos A_i , não serão disjuntos, mas toda função não-sobrejetiva pertencerá a pelo menos um deles.)

$$\begin{aligned} |A_1 \cup \cdots \cup A_n| &= \sum_{1 \leq i \leq n} |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| \\ &\quad + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| - \cdots \\ &\quad + (-1)^{n+1} |A_1 \cap \cdots \cap A_n| \end{aligned} \quad (1)$$

Para qualquer i , $|A_i|$ é o número de funções que não levam nenhum elemento do domínio em t_i , mas não tem outras restrições. Pelo Princípio da Multiplicação, podemos contar o número desse tipo de funções contando para cada um dos m elementos do domínio, suas $n - 1$ imagens possíveis. O resultado será $|A_i| = (n-1)^m$. Portanto o primeiro somatório da equação (1) soma os termos que são todos do mesmo tamanho. Existe uma parcela para cada conjunto individual A_i , dentre esses n conjuntos, de forma que existem $C(n, 1)$ termos deste tipo.

Para quaisquer i, j , $|A_i \cap A_j|$ é o número de funções que não levam nenhum elemento do domínio em t_i , nem em t_j , deixando $n - 2$ possíveis valores de imagens para os m elementos de S . Portanto, $|A_i \cap A_j| = (n - 2)^m$. O segundo somatório soma, uma parcela para cada grupo distinto de dois conjuntos dentre os n , existem, portanto, $C(n, 2)$ desses conjuntos.

Um resultado semelhante vale para todas as parcelas de interseções. Se existirem k conjuntos na interseção, então existem $(n - k)^m$ funções na interseção, e $C(n, k)$ grupos distintos de k elementos para a interseção. A Equação (1) pode ser reescrita como

$$|A_1 \cup \dots \cup A_n| = C(n, 1)(n-1)^m - C(n, 2)(n-2)^m + C(n, 3)(n-3)^m - \dots + (-1)^{n+1}C(n, n)(n-n)^m \quad (2)$$

Agora a expressão do lado esquerdo da Equação (2) representa todas as funções cujas imagens não contenham pelo menos um elemento de T , isto é, todas as funções não-sobrejetivas. Se subtrairmos este número do número total de funções, que sabemos que é n^m , teremos o número de funções sobrejetivas. Portanto, o número de funções sobrejetivas é

$$n^m - C(n, 1)(n-1)^m + C(n, 2)(n-2)^m - C(n, 3)(n-3)^m + \dots + (-1)^{n-1}C(n, n-1)[n - (n-1)]^m + (-1)^nC(n, n)(n-n)^m$$

Realizando a soma do penúltimo termo e ignorando o último termo, que vale zero, a resposta final é

$$n^m - C(n, 1)(n-1)^m + C(n, 2)(n-2)^m - C(n, 3)(n-3)^m + \dots + (-1)^{n-1}C(n, n-1)(1)^m$$

Resumiremos estes resultados.

Teorema Sobre o Número de Funções com Domínios e Contradomínios Finitos

Se $|S| = m$ e $|T| = n$, então

1. O número de funções $f: S \rightarrow T$ é n^m .
2. O número de funções injetivas $f: S \rightarrow T$, assumindo que $m \leq n$, é

$$\frac{n!}{(n-m)!}$$

3. O número de funções sobrejetivas $f: S \rightarrow T$, assumindo que $m \geq n$ é

$$n^m - C(n, 1)(n-1)^m + C(n, 2)(n-2)^m - C(n, 3)(n-3)^m + \dots + (-1)^{n-1}C(n, n-1)(1)^m$$

EXEMPLO 37 Sejam $S = \{A, B, C\}$ e $T = \{a, b\}$. Encontre o número de funções de S em T .

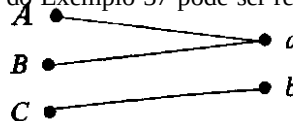
Neste exemplo, $m = 3$ e $n = 2$. Por nosso teorema sobre o número de funções, existem

$$2^3 - C(2, 1)(1)^3 = 8 - 2 \cdot 1 = 6$$

deste tipo.

PRÁTICA 31

Uma das seis funções sobrejetivas do Exemplo 37 pode ser representada como mostrado a seguir:



Desenhe os diagramas para as cinco demais funções.

Conjuntos Equivalentes

Definições: Conjuntos Equivalentes e Cardinalidade

Um conjunto S é **equivalente ao conjunto** T se existir uma bijeção $F: S \rightarrow T$. Dois conjuntos que são equivalentes têm a mesma **cardinalidade**.

Se S é equivalente a T , então todos os elementos de S e T relacionam-se por meio de f através de uma correspondência injetiva. Se S e T são conjuntos finitos, esta correspondência só pode ocorrer se S e T têm o mesmo número de elementos. Em conjuntos infinitos, a idéia de tamanho do conjunto torna-se um pouco mais confusa, pois algumas vezes podemos provar que um determinado conjunto é equivalente ou *assemelha-se* a um conjunto menor. A cardinalidade é a extensão apropriada do conceito de tamanho (número de elementos) de um conjunto para conjuntos infinitos.

PRÁTICA 32 Descreva uma bijeção $f: \mathbb{Z} \rightarrow \mathbb{N}$, mostrando, assim, que $\mathbb{Z}$ é equivalente a $\mathbb{N}$ (que $\mathbb{Z}$ e $\mathbb{N}$ têm a mesma cardinalidade) a despeito de $\mathbb{N} \subset \mathbb{Z}$. ♦

Se encontrarmos uma bijeção entre um conjunto S e $\mathbb{N}$, teremos estabelecido uma correspondência um-para-um entre os elementos de S e os inteiros não-negativos. Podemos nos referir aos elementos de S , de acordo com esta correspondência, escrevendo s_0 para o valor de S associado a 0, s_1 para o valor de S associado a 1, e assim por diante. Desta forma, a lista

$$s_0, s_1, s_2, \dots$$

inclui todos os elementos de S . Uma vez que esta lista constitui uma enumeração de S , S é um conjunto enumerável. Reciprocamente, se S é enumerável, então existe uma lista dos elementos de S que pode ser usada para definir uma bijeção entre S e $\mathbb{N}$. Portanto, um conjunto é enumerável se, e somente se, é equivalente a $\mathbb{N}$.

Para conjuntos finitos, sabemos que S tem n elementos, então $\mathcal{P}(S)$ tem 2^n elementos. Naturalmente, $2^n > n$ e, portanto, não podemos encontrar uma bijeção entre um conjunto com n elementos e um conjunto com 2^n elementos. Portanto, S e $\mathcal{P}(S)$ não são equivalentes. Este resultado também é válido para conjuntos infinitos.

Teorema de Cantor

Para qualquer conjunto S , S e $\mathcal{P}(S)$ não são equivalentes.

Demonstração: Realizaremos uma demonstração por contradição. Admita que S e $\mathcal{P}(S)$ são equivalentes. Seja f a bijeção entre S e $\mathcal{P}(S)$. Para qualquer elemento s de S , $f(s)$ é um elemento de $\mathcal{P}(S)$, de forma que $f(S)$ é um conjunto que contém alguns elementos de S e possivelmente contém o próprio s . Definamos agora um conjunto $X = \{x \in S \mid x \notin f(x)\}$. Como X é um subconjunto de S , ele é um elemento de $\mathcal{P}(S)$ e, portanto, precisa ser igual a $f(y)$ para algum $y \in S$. Então y pode ser ou não um elemento de X . Se $y \in X$, então, pela definição de X , $y \notin f(y)$, mas como $f(y) = X$, $y \in X$. Por outro lado, se $y \notin X$, então, uma vez que $X = f(y)$, $y \notin f(y)$ e, pela definição de X , $y \in X$. Em ambos os casos, chegamos a uma contradição, e nossa hipótese inicial estava errada. Portanto, S e $\mathcal{P}(S)$ não são equivalentes.

A demonstração do teorema de Cantor depende da natureza do conjunto X , que foi cuidadosamente construído para causar a contradição necessária. Neste ponto, esta demonstração é semelhante ao método de diagonalização (Exemplo 27 do Cap. 3) usado para demonstrar a existência de um conjunto não-enumerável de números. Na verdade, a existência de um conjunto incontável pode ser demonstrada diretamente a partir do teorema de Cantor.

EXEMPLO 38 O conjunto $\mathbb{N}$ é, obviamente, um conjunto enumerável. Pelo teorema de Cantor, $\mathcal{P}(\mathbb{N})$ não é equivalente a $\mathbb{N}$ e, portanto, não é um conjunto enumerável, uma vez que é claramente um conjunto infinito. •

Composição de Funções

Suponha que f e g são funções com $f: S \rightarrow T$ e $g: T \rightarrow U$. Então, para qualquer $s \in S$, $f(s)$ é um elemento de T , que também é domínio de g . Portanto, a função g pode ser aplicada a $f(s)$. O resultado é $g(f(s))$, um elemento de U ; veja a Fig. 4.20. Na verdade, tomar um elemento arbitrário s de S , aplicar a função f e então aplicar a função g a $f(s)$ é a mesma coisa que associar um único elemento de U a s . Resumindo, criamos uma função $S \rightarrow U$, chamada de composta de f e g e denotada por $g \circ f$.

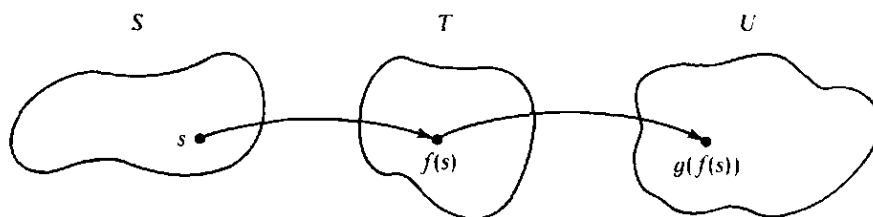


Figura 4.20

Definição: Função Composta

Seja $f: S \rightarrow T$ e $g: T \rightarrow U$. Então a **função composta**, $g \circ f$ é uma função de S em U definida por $(g \circ f)(s) = g(f(s))$.

Perceba que a função $g \circ f$ é aplicada da direita para a esquerda, isto é, f é aplicada primeiro, e então aplica-se a função g .

O diagrama da Fig. 4.21 ilustra a definição da função composta. Os vértices indicam os domínios e contradomínios das três funções. O diagrama diz que começando com um elemento em S , se seguirmos pelo caminho $g \circ f$ ou pelo caminho f e depois pelo caminho g , obteremos o mesmo elemento de U . Diagramas que ilustram que caminhos alternativos geram os mesmos resultados são chamados de **diagramas comutativos**.

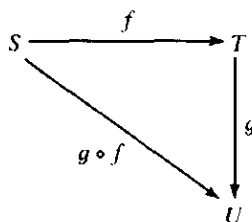


Figura 4.21

PRÁTICA 33

Seja $f: \mathbb{R} \rightarrow \mathbb{R}$ definida por $f(x) = x^2$. Seja $g: \mathbb{R} \rightarrow \mathbb{R}$ definida por $g(x) = \lfloor x \rfloor$.

- Qual o valor de $(g \circ f)(2.3)$?
- Qual o valor de $(f \circ g)(2.3)$?

A Prática 33 nos mostra que a ordem da composição de funções é importante, o que não deve ser surpresa. Se formos escrever um programa de computador para realizar a composição de funções, geralmente precisaremos de um comando de atribuição para computar cada função. Trocar a ordem da composição trocaria a ordem dos dois comandos do programa o que quase sempre altera o resultado do programa.

A composição de funções preserva as propriedades de sobrejeção e injeção. Novamente, sejam $f: S \rightarrow T$ e $g: T \rightarrow U$, mas vamos supor que tanto f quanto g são funções sobrejetivas. Então a função composta $g \circ f$ também é sobrejetiva. Lembre-se de que $g \circ f: S \rightarrow U$, então devemos pegar um elemento arbitrário $u \in U$ e mostrar que ele tem uma pré-imagem por $g \circ f$ em S . Como g é sobrejetiva, existe $t \in T$ tal que $g(t) = u$. E, como f é sobrejetiva, existe $s \in S$ tal que $f(s) = t$. Então $(g \circ f)(s) = g(f(s)) = g(t) = u$ e $g \circ f$ é uma função sobrejetiva.

PRÁTICA 34

Sejam $f: S \rightarrow T$ e $g: T \rightarrow U$ e assumamos que tanto f quanto g são funções injetivas. Demonstre que $g \circ f$ é uma função injetiva. (Dica: Assuma que $(g \circ f)(s_1) = (g \circ f)(s_2)$.)

Nós demonstramos o teorema abaixo.

Teorema sobre Composição de Bijeções

A composição de duas bijeções é uma bijeção.

Funções de Permutação

O teorema de composição de duas bijeções refere-se à função composta *quando ela existe*, isto é, quando os domínios e contradomínios permitem a criação de uma função composta. Naturalmente, isto não é problema

se apenas um conjunto estiver envolvido, isto é, se estivermos lidando apenas com funções de um conjunto A no próprio conjunto A .

Definição: Permutações de um Conjunto

Dado um conjunto A , $S_A = \{f \mid f: A \rightarrow A \text{ e } f \text{ é uma bijeção}\}$. S_A é, portanto, o conjunto de todas as bijeções (e, portanto, sobrejeções) do conjunto A nele mesmo; as funções deste tipo são chamadas de **permutações de A** .

Nosso teorema de composição de bijeções diz que se criarmos uma função composta usando dois elementos de S_A , obtemos um (único) elemento de S_A ; portanto, a composição de funções é uma operação binária no conjunto S_A .

Na Seção 3.4 descrevemos uma permutação de objetos em um conjunto como sendo um arranjo ordenado desses objetos. De forma semelhante, as funções de permutação representam arranjos ordenados de objetos no domínio. Se $A = \{1, 2, 3, 4\}$, uma função de permutação de A é dada por $f = \{(1, 2), (2, 3), (3, 1), (4, 4)\}$. Podemos ainda descrever a função/na forma de uma matriz, listando os elementos do domínio em uma linha e, diretamente abaixo, as imagens desses elementos por/. Portanto,

$$f = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 1 & 4 \end{pmatrix}$$

A linha de baixo é um arranjo ordenado dos objetos da linha de cima.

Notação Cíclica

Uma maneira mais abreviada de descrever a permutação/mostrada acima na forma matricial é usar a *notação cíclica* e escrever $f = (1, 2, 3)$ — que deve ser entendida como f leva cada elemento ao elemento escrito a sua direita, exceto pelo último elemento que é levado no primeiro elemento e todos os elementos que não são listados são levados neles mesmos. Desta forma, todos os ciclos $(1, 2, 3)$, $(2, 3, 1)$, $(3, 1, 2)$ representam f .

PRÁTICA 35

- a. Seja $A = \{1, 2, 3, 4, 5\}$ e $f \in S_A$ dada, na forma matricial, por

$$f = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 4 & 2 & 3 & 5 & 1 \end{pmatrix}$$

Escreva/em sua forma cíclica.

- b. Seja $A = \{1, 2, 3, 4, 5\}$ e seja $g \in S_A$ dada na forma cíclica por $g = (2, 4, 5, 3)$. Escreva g em sua forma matricial.

Se f e g são elementos de S_A para algum conjunto A , então $g \circ f \in S_A$ e o efeito de $g \circ f$ em qualquer elemento de A é determinado pela aplicação da função f seguida da função g . Se f e g são ciclos, então $g \circ f$ também será considerada um ciclo. Se $A = \{1, 2, 3, 4\}$ e $f, g \in S_A$ são dadas por $f = (1, 2, 3)$ e $g = (2, 3)$, então $g \circ f = (2, 3) \circ (1, 2, 3)$. Para o elemento 1 de A , $1 \rightarrow 2$ por f e $2 \rightarrow 3$ por g , de forma que $1 \rightarrow 3$ por $g \circ f$. Analogamente, $2 \rightarrow 3$ por f e $3 \rightarrow 2$ por g , de modo que $2 \rightarrow 2$ por $g \circ f$. Se verificarmos o que acontece com o 3 e o 4, veremos que $g \circ f = (1, 3)$. Mas, se tivéssemos computado $f \circ g$, teríamos obtido $(1, 2)$. (Já sabemos que a ordem é relevante na composição de funções.) Se, no entanto, f e g forem elementos de S_A , e f e g forem **ciclos disjuntos** — ciclos que não têm elementos em comum —, então $f \circ g = g \circ f$.

PRÁTICA 36

Seja $A = \{1, 2, 3, 4, 5\}$ e sejam f e g elementos de S_A . Encontre $g \circ f$ e $f \circ g$ para os seguintes casos:

- $f = (5, 2, 3)$; $g = (3, 4, 1)$. Escreva as respostas em notação cíclica.
- $f = (1, 2, 3, 4)$; $g = (3, 2, 4, 5)$. Escreva as respostas em suas formas matriciais.
- $f = (1, 3)$; $g = (2, 5)$. Escreva as respostas em suas formas matriciais.

Novamente, seja $A = \{1, 2, 3, 4\}$ e consideremos o ciclo $f \in S_A$ dado por $f = (1, 2)$. Se computarmos $f \circ f = (1, 2) \circ (1, 2)$, veremos que cada elemento de A é levado nele mesmo. A permutação que leva cada elemento nele mesmo, isto é, que deixa todos os elementos inalterados, é chamada de **permutação identidade** ou **função identidade** de A , denotada por i_A .

Se A é um conjunto infinito, nem toda permutação de A pode ser escrita como um ciclo. E, mesmo quando A é finito, nem toda permutação de A pode ser escrita como um ciclo; por exemplo, a permutação $g \circ f$ da

Prática 36(b) não pode ser escrita como um ciclo. No entanto, toda permutação em um conjunto finito pode ser escrita como a composição de um ou mais ciclos disjuntos. A permutação

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 4 & 2 & 5 & 1 & 3 \end{pmatrix}$$

da Prática 36(b) é $(1, 4) \circ (3, 5)$ ou $(3, 5) \circ (1, 4)$.

PRÁTICA 37 Escreva

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 2 & 4 & 5 & 1 & 3 & 6 \end{pmatrix}$$

como uma composição de ciclos disjuntos.

Permutações sem Pontos Fixos

Dentre as permutações de A , algumas levam alguns elementos de A neles mesmos. Estes elementos são chamados de pontos fixos da permutação. Por outro lado, a permutação pode misturar os elementos de A de tal forma que nenhum elemento seja levado nele mesmo. Uma permutação em um conjunto que não leve qualquer elemento nele mesmo é chamada de **permutação sem ponto fixo**.

EXEMPLO 39 A permutação/em $A = \{1, 2, 3, 4, 5\}$ é dada, em sua forma matricial, por

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 5 & 4 & 1 & 3 \end{pmatrix}$$

é uma permutação sem pontos fixos. Os elementos de S_A que possuem pontos fixos, se escritos na forma cíclica ou como composição de ciclos, terão pelo menos um elemento de A não-listado. Portanto, $g \in S_A$ definida por $g = (1, 4) \circ (3, 5)$ leva 2 nele mesmo e, por isso, g não é uma permutação sem pontos fixos.

Vamos contar o número de permutações sem pontos fixos no conjunto A com $|A| = n$. Nossa estratégia é semelhante à usada para a contagem de funções sobrejetivas. Usaremos o Princípio da Inclusão e Exclusão para calcular o número de permutações que têm pontos fixos e então subtrair este número do número total de funções de permutações.

Enumeremos os elementos do conjunto A como $a_1, a_2, \dots, a_n$. Para cada i , $1 \leq i \leq n$, seja A_i o conjunto de todas as permutações em que a_i for um ponto fixo. (Estes conjuntos não são disjuntos, mas todas as permutações que têm pontos fixos pertencem a pelo menos um deles.) Pelo Princípio da Inclusão e Exclusão, podemos escrever

$$\begin{aligned} |A_1 \cup \dots \cup A_n| = & \sum_{1 \leq i \leq n} |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| \\ & - \dots + (-1)^{n+1} |A_1 \cap \dots \cap A_n|. \end{aligned} \quad (3)$$

Para qualquer i , $|A_i|$ é o número de permutações nas quais a_i é um ponto fixo. Pelo Princípio da Multiplicação, podemos contar o número de elementos dessas funções contando para cada um dos n elementos do domínio, começando por a_i , suas possíveis imagens. Existe apenas uma opção de escolha para a imagem de a_i , porque ele é um ponto fixo e, como tal, deve ser levado nele mesmo; o próximo elemento pode ser levado em qualquer outro elemento, exceto a_i , portanto, existem $n - 1$ possibilidades; o elemento seguinte pode ser levado em qualquer elemento, exceto as duas imagens já usadas, de forma que existem $n - 2$ possibilidades para sua escolha, e assim por diante. Continuando assim, teremos

$$(1)(n-1)(n-2) \dots (1) = (n-1)!$$

elementos em A_i para cada a_i . Assim, o primeiro somatório da Equação (3) agrupa as parcelas referentes a conjuntos do mesmo tamanho. O número dessas parcelas é igual ao número de maneiras de se escolher um conjunto A_i , dentre n conjuntos, ou seja, $C(n, 1)$.

No segundo somatório, as parcelas contam o número de permutações de n elementos que mantenham dois desses elementos fixos. Existem

$$(1)(1)(n-2) \dots (1) = (n-2)!$$

funções em cada $A_i \cap A_j$ dadas e $C(n, 2)$ maneiras de escolher os dois conjuntos dentre os n . Em geral, se existem k conjuntos na interseção, então k elementos precisam ser mantidos fixos, de forma que existem $(n-k)!$ funções no conjunto interseção, e $C(n, k)$ maneiras de escolher os k conjuntos da interseção. Portanto, a equação (3) torna-se

$$|A_1 \cup \dots \cup A_n| = C(n, 1)(n-1)! - C(n, 2)(n-2)! + C(n, 3)(n-3)! - \dots + (-1)^{n+1} C(n, n)(n-n)!$$

Esta expressão representa todas as permutações que têm pontos fixos. Subtraímos este número total de funções de permutações, que é $n!$:

$$n! - C(n, 1)(n-1)! + C(n, 2)(n-2)! - C(n, 3)(n-3)! + \dots + (-1)^n C(n, n)(n-n)!$$

Simplificando, obtemos

$$\begin{aligned} n! - \frac{n!}{1!(n-1)!}(n-1)! + \frac{n!}{2!(n-2)!}(n-2)! - \frac{n!}{3!(n-3)!}(n-3)! \\ + \dots + (-1)^n \frac{n!}{n!0!}0! \\ = n! - \frac{n!}{1!} + \frac{n!}{2!} - \frac{n!}{3!} + \dots + (-1)^n \frac{n!}{n!} \\ = n! \left[1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + (-1)^n \frac{1}{n!} \right] \end{aligned} \quad (4)$$

EXEMPLO 40 Para $n = 3$, a equação (4) diz que o número de permutações sem pontos fixos é

$$3! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \right) = \frac{3!}{2!} - \frac{3!}{3!} = 3 - 1 = 2$$

Essas duas permutações são, em suas formas matriciais

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \end{pmatrix} \text{ e } \begin{pmatrix} 1 & 2 & 3 \\ 3 & 1 & 2 \end{pmatrix}$$

Funções Inversas

Existe mais uma propriedade importante das funções bijetivas. Seja $f: S \rightarrow T$ uma bijeção. Como f é sobrejetiva, todo $t \in T$ tem pré-imagem em S . Como f é injetiva, esta pré-imagem é única. Podemos, então, associar a cada elemento t de T um único elemento de S tal que $f(s) = t$. Esta associação descreve uma função $g: T \rightarrow S$. A Fig. 4.22 ilustra as relações entre f e g . Os domínios e contradomínios de g e f são tais que podemos formar tanto $g \circ f: S \rightarrow S$, quanto $f \circ g: T \rightarrow T$. Se $s \in S$, então $(g \circ f)(s) = g(f(s)) = g(t) = s$. Portanto, $g \circ f$ leva cada elemento de S nele mesmo. Logo $g \circ f = i_S$, a função identidade em S .

PRÁTICA 38 Mostre que $f \circ g = i_T$.

Acabamos de ver que se f é uma bijeção, $f: S \rightarrow T$, então existe uma função $g: T \rightarrow S$ com $g \circ f = i_S$ e $f \circ g = i_T$. A recíproca também é verdadeira. Para demonstrar a recíproca, suponha $f: S \rightarrow T$ e que exista $g: T \rightarrow S$ tal que $g \circ f = i_S$ e $f \circ g = i_T$. Podemos demonstrar que f é uma bijeção. Para mostrar que f é sobrejetiva, seja $t \in T$. Então $t = i_T(t) = (f \circ g)(t) = f(g(t))$. Como $g: T \rightarrow S$, $g(t) \in S$ e $g(t)$ é a pré-imagem de t por f . Para mostrar que f é injetiva, suponha que $f(s_1) = f(s_2)$. Então $g(f(s_1)) = g(f(s_2))$ e $(g \circ f)(s_1) = (g \circ f)(s_2)$, acarretando $i_S(s_1) = i_S(s_2)$ ou $s_1 = s_2$. Portanto, f é uma bijeção.

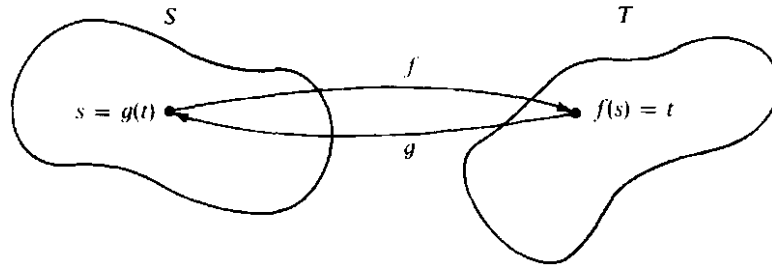


Figura 4.22

Definição: Função Inversa

Seja f uma função, $f: S \rightarrow T$. Se existir uma função $g: T \rightarrow S$ tal que $g \circ f = i_S$ e $f \circ g = i_T$, então g é chamada de **função inversa** de f e denotada por f^{-1} .

Nós já demonstramos o seguinte teorema.

Teorema sobre Bijeções e Funções Inversas

Seja $f: S \rightarrow T$. Então f é uma bijeção se, e somente se, f^{-1} existe.

De fato, fomos um tanto o quanto relaxados ao falar da função inversa de f . O que mostramos é que, se f é uma bijeção, isto equivale dizer que tem uma função inversa. Mas é fácil ver que existe apenas uma função inversa. Quando desejamos demonstrar que algo é único, a técnica corriqueira é assumir que existem duas ocorrências diferentes dele, e então obter uma contradição. Suponha, então, que tem duas funções inversas, f^{-1}_1 e f^{-1}_2 (a existência de qualquer uma das inversas implica que f é uma bijeção). Tanto f^{-1}_1 quanto f^{-1}_2 são funções de T em S ; se não forem a mesma função, então precisam ter comportamentos diferentes em algum lugar. Admita que existe um $t \in T$ tal que $f^{-1}_1(t) \neq f^{-1}_2(t)$. Como f é injetiva, então $f(f^{-1}_1(t)) \neq f(f^{-1}_2(t))$, ou ainda $(f \circ f^{-1}_1)(t) \neq (f \circ f^{-1}_2)(t)$. Mas tanto $f \circ f^{-1}_1$ quanto $f \circ f^{-1}_2$ são i_T então $t \neq t$, o que é uma contradição. Justificamos, portanto, o uso do artigo definido ao dizermos a função inversa de f . Se f for uma bijeção, e, portanto f^{-1} existir, então f^{-1} é a função inversa de f ; portanto f^{-1} é também uma bijeção.

PRÁTICA 39

$f: \mathbb{R} \rightarrow \mathbb{R}$ dada por $f(x) = 3x + 4$ é uma bijeção. Descreva f^{-1} .

Ordem de Grandeza de Funções¹

A *ordem de grandeza* é um modo de comparar a "taxa de crescimento" de diferentes funções. Já sabemos, por exemplo, que se computarmos $f(x) = x$ e $g(x) = x^2$ para valores cada vez maiores de x , os valores de g serão maiores que os valores de f . Esta diferença na taxa de crescimento não pode ser superada simplesmente multiplicando-se os valores de f por uma constante; independentemente do quão grande seja o valor da constante, os valores de g superarão, após algum ponto, os valores de f . Nossa experiência nos mostra que as funções f e g têm taxas de crescimento com comportamentos completamente diversos. A fim de caracterizar esta diferença, definiremos uma relação binária entre as funções.

Seja S o conjunto de todas as funções cujos domínios e contradomínios são os números reais não-negativos. Podemos definir uma relação binária em S por

$$f \rho g \leftrightarrow \text{existem constantes positivas } n_0, c_1 \text{ e } c_2 \text{ tais que, para todo } x \geq n_0, c_1 g(x) \leq f(x) \leq c_2 g(x)$$

EXEMPLO 41

Sejam f e g funções em S onde $f(x) = 3x^2$ e $g(x) = 200x^2 + 140x + 7$. Seja $n_0 = 2$, $c_1 = \frac{1}{100}$ e $c_2 = 1$. Então, para $x \geq 2$,

$$\frac{1}{100} (200x^2 + 140x + 7) \leq 3x^2 \leq (1)(200x^2 + 140x + 7)$$

¹ A autora adotou a nomenclatura "Ordem de Grandeza" para denotar o que mais freqüentemente se chama de ordem de crescimento ou domínio assintótico do crescimento. Como mantivemos a nomenclatura usada no original, é importante que o leitor não confunda o termo aqui usado com seu significado convencional, que consiste no uso de potências de 10 para exprimir um número estimado. (N. do T.)

ou

$$2x^2 + 1.4x + 0.07 \leq 3x^2 \leq 200x^2 + 140x + 7 \quad (5)$$

Portanto/ p g .

PRÁTICA 40

- Verifique a inequação (5) para os seguintes valores de x : 2, 3, 4, 5. (Use uma calculadora).
- No Exemplo 41, n_0 , poderia ter o valor 1 se c_1 e c_2 permanecessem os mesmos?
- Encontre um conjunto diferente para os três valores n_0 , c_1 e c_2 que também sirvam para mostrar que $f p g$ no Exemplo 41.

A relação p é uma relação de equivalência em S . Por exemplo, para demonstrar que $f p f$, podemos tomar $n_0 = c_1 = c_2 = 1$ e termos

$$(1)f(x) \leq f(x) \leq (1)f(x)$$

PRÁTICA 41

- Demonstre que p é simétrica.
- Demonstre que p é transitiva.

Dado que p é uma relação de equivalência, ela particiona S em classes de equivalência. Se f está na mesma classe de equivalência que g , então dizemos que/tem a mesma ordem de grandeza que g , denotado por $f = \Theta(g)$. (A notação $f = O(g)$ é usada amiúde, apesar de, tecnicamente, esta "notação O " ter um significado técnico diferente.) Devido à simetria, podemos também dizer que g tem a mesma ordem de grandeza de f e escrever $g = \Theta(f)$. (A notação $f = \Theta(g)$ constitui um erro indevido do símbolo de igualdade, porque $\Theta(g)$ não é uma função idêntica a f . Trata-se apenas de uma forma abreviada de dizer que $f \in [g]$ segundo a relação de equivalência p definida acima.)

Definição: Ordem de Grandeza

Sejam f e g funções dos reais não-negativos nos reais não-negativos. Então f tem a mesma ordem de grandeza que g , escrito como $f = \Theta(g)$, se existirem constantes positivas n_0 , c_1 e c_2 tais que para $x \geq n_0$, $c_1 g(x) \leq f(x) \leq c_2 g(x)$.

Normalmente, tentamos encontrar o representante mais simples de uma classe de equivalência. Portanto, para as funções f e g do Exemplo 41, poderíamos dizer que $f = \Theta(x^2)$ e $g = \Theta(x^2)$. A ordem de grandeza de um polinômio é sempre o termo de maior grau; os termos de menor grau e os coeficientes podem ser ignorados. Isto não nos surpreende, uma vez que para valores grandes de x , o termo com maior grau é o que mais contribui para o resultado.

PRÁTICA 42

Prove (encontrando as constantes apropriadas que satisfaçam à definição de ordem de grandeza) que $f = \Theta(x^2)$ e $g = \Theta(x^2)$ para as funções f e g do Exemplo 41. ♦

A fim de termos uma compreensão mais intuitiva do significado do que essas classes de equivalência significam, traçaremos alguns gráficos. Seja $h(x) \in S$, onde $h(x) = x^2$. A Fig. 4.23 mostra o gráfico de $h(x)$. Suponha agora que multiplicamos os valores de h pelas duas constantes $c_1 = \frac{1}{2}$ e $c_2 = 2$. As funções $c_1 h(x)$ e $c_2 h(x)$ são mostradas com linhas tracejadas na Fig. 4.24. Essas linhas tracejadas formam uma espécie de envelope em torno dos valores de $h(x)$, que esboçam a mesma forma de $h(x)$. Alterar os valores das constantes altera o tamanho deste envelope, mas não sua forma básica. Se $h_1(x)$ é uma função com $h_1 = \Theta(h)$, então existe uma constante positiva n_0 e algum envelope em torno de h tal que, para todos os valores do domínio à direita de n_0 , os valores de h_1 estão todos dentro deste envelope, como mostrado na Fig. 4.25. Portanto, os valores de h_1 não podem afastar-se muito dos valores de h . As funções h_1 e h têm, grosso modo, a mesma forma — elas têm a mesma ordem de grandeza.

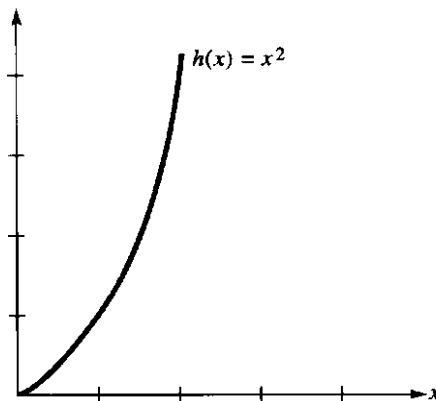


Figura 4.23

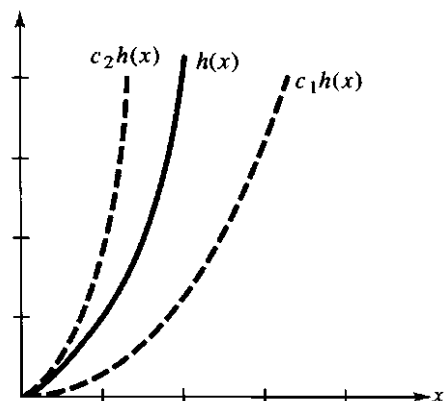


Figura 4.24

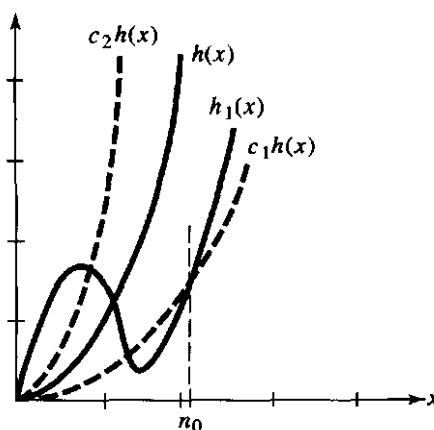


Figura 4.25

EXEMPLO 42

Seja $f(x) = x$ e $h(x) = x^2$. A Fig. 4.26 mostra que, para as constantes $c_1 = \frac{1}{2}$ e $c_2 = 2$, f logo fica abaixo do envelope. Reduzindo a constante c_1 (o que abaixa a aresta inferior do envelope) apenas se adia o problema. Formalmente, podemos fazer uma demonstração por contradição para mostrarmos que f não é $\Theta(x^2)$. Suponha $f = \Theta(x^2)$. Então existem constantes n_0 e c_1 com $c_1 x^2 \leq f(x)$ para $x \geq n_0$. Mas isto implica que $c_1 x^2 \leq x$ ou $c_1 x \leq 1$ ou $x < 1/c_1$ para todo $x \geq n_0$. Como c_1 é fixo, podemos sempre escolher x suficientemente grande para que $x > 1/c_1$, o que é uma contradição. Portanto, $f(x) = x$ não é $\Theta(x^2)$.

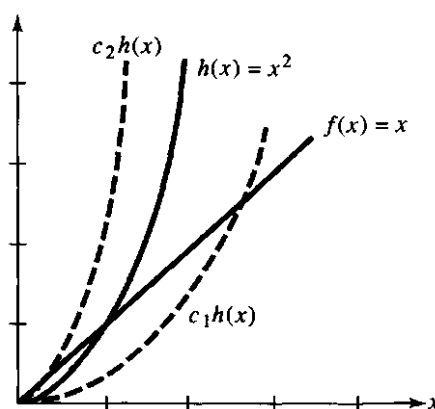


Figura 4.26

A ordem de grandeza das funções é importante na análise de algoritmos. Ao analisarmos um algoritmo, identificamos as tarefas importantes que o algoritmo deve realizar. Em geral, o número dessas tarefas depende do tamanho da entrada. Por exemplo, a busca em uma lista de n elementos ou a ordenação de uma lista de n elementos requerem mais trabalho à medida que n cresce. Tipicamente podemos representar ao tamanho da entrada como um inteiro não-negativo, de forma que as funções que exprimem a quantidade de trabalho são funções cujo domínio é $\mathbb{N}$. Veremos na Seção 2.4 que uma busca seqüencial em n elementos requer n comparações no pior caso, enquanto que uma busca binária requer $1 + \log n$ comparações no pior caso (admitindo que n é uma potência de 2). Ao invés de computar a função exata da quantidade de trabalho necessário, é mais fácil e freqüentemente válido contentarmo-nos com a informação acerca de sua ordem de grandeza. A busca seqüencial é $\Theta(n)$ e a busca binária é $\Theta(\log n)$. As ordens de grandeza, quando usadas em conjunto com a análise de algoritmos, são chamadas de **classes de complexidade**.

Podemos compor uma hierarquia de classes de complexidade. Por exemplo, a classe $\Theta(n)$ é uma ordem de grandeza menor do que $\Theta(n^2)$, porque funções que são $\Theta(n)$ são inevitavelmente, em algum momento, se tornam inferiores a funções $\Theta(n^2)$. Além disso, a $\Theta(\log n)$ é uma ordem de grandeza menor do que $\Theta(n)$ (veja o Exercício 49 ao final desta seção) de modo que a busca binária é uma melhoria, em termos de ordem de grandeza, da busca seqüencial.

Podemos desenvolver uma espécie de aritmética para as classes de complexidade. Por exemplo, se $f_1(n) = n$ e $f_2(n) = n^2$, então a função $(f_1 + f_2)(n) = f_1(n) + f_2(n) = n + n^2 = \Theta(n^2)$. Em geral, se $f_1 = \Theta(g_1)$ e $f_2 = \Theta(g_2)$, então $f_1 + f_2 = \Theta(\max(g_1, g_2))$ (veja o Exercício 45). Quando expressa de forma abreviada, esta aritmética nos leva a equações bastante bizarras, como $\Theta(n) + \Theta(n^2) = \Theta(n^2)$ ou $\Theta(n^2) + \Theta(n^2) = \Theta(n^2)$.

Para verificarmos o efeito da ordem de grandeza na execução de algoritmos, suponhamos que temos os algoritmos A e A' para realizar a mesma tarefa e que suas complexidades têm diferentes ordens de grandeza, digamos que A é $\Theta(n)$ e A' é $\Theta(n^2)$. Ainda que cada passo do algoritmo tome apenas 0,0001 segundo, essa diferença vai afetar o tempo total de execução do programa à medida que n cresça. As duas primeiras linhas da Tabela 4.1 fornecem os tempos totais de processamento de A e A' para diversos tamanhos da entrada. Suponha agora que exista um terceiro algoritmo A'' cuja complexidade não é dada por um polinômio, mas por uma função exponencial, digamos 2^n . Os tempos para a execução de A'' são mostrados na terceira linha da Tabela 4.1.

Perceba que o caso exponencial cresce em uma velocidade incrível! Ainda que assumamos que cada passo computacional tome muito menos tempo do que 0,0001 segundo, a taxa de crescimento relativo entre as funções polinomial e exponencial continuará a seguir este mesmo comportamento. Devido ao crescimento exagerado, os algoritmos que não são de ordem polinomial ou inferior não são geralmente úteis para valores de n . De fato, problemas para os quais não existem algoritmos de complexidade polinomial ou inferior são chamados de **intratáveis**.

Algumas vezes, algoritmos cuja complexidade é maior do que polinomial no pior dos casos podem ser eficientes para entradas de dados "médias" — e úteis. Não obstante, na tentativa de aumentar a eficiência, devemos sempre nos questionar se há um algoritmo diferente cuja complexidade seja de uma ordem de grandeza menor antes de nos preocuparmos com os detalhes de ajuste fino de determinado algoritmo.

Tempo Total de Processamento				
Algoritmo	Ordem	Tamanho da Entrada n		
		10	50	100
A	n	0,001 segundo	0,005 segundo	0,01 segundo
A'	n^2	0,01 segundo	0,25 segundo	1 segundo
A''	2^n	0,1024 segundo	3.570 anos	4 X 10 ¹⁶ séculos

Tabela 4.1

Revisão da Seção 4.3

Técnicas

- Determinar se uma dada relação é uma função
- Determinar se uma função é injetiva e sobrejetiva
- Contar o número de funções, de funções injetivas e de funções sobrejetivas de um conjunto finito em outro
- Encontrar a imagem de um elemento por uma composição de funções
- Escrever permutações de um conjunto de forma matricial ou cíclica
- Contar o número de permutações sem pontos fixos de um conjunto finito
- Determinar se duas funções têm a mesma ordem de grandeza

Idéias Principais

O conceito de função, principalmente funções bijetivas, é extremamente importante.

A composição de funções preserva a bijetividade.

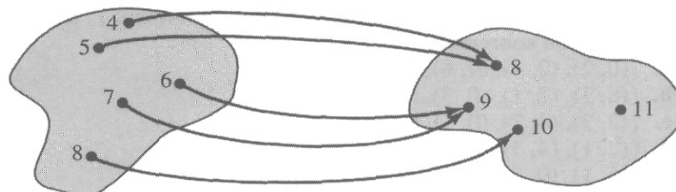
Permutações são bijeções de um conjunto nele próprio.

A função inversa de uma bijeção também é uma bijeção.

As funções podem ser agrupadas em classes de equivalências de acordo com suas ordens de grandeza, que é uma medida de sua taxa de crescimento.

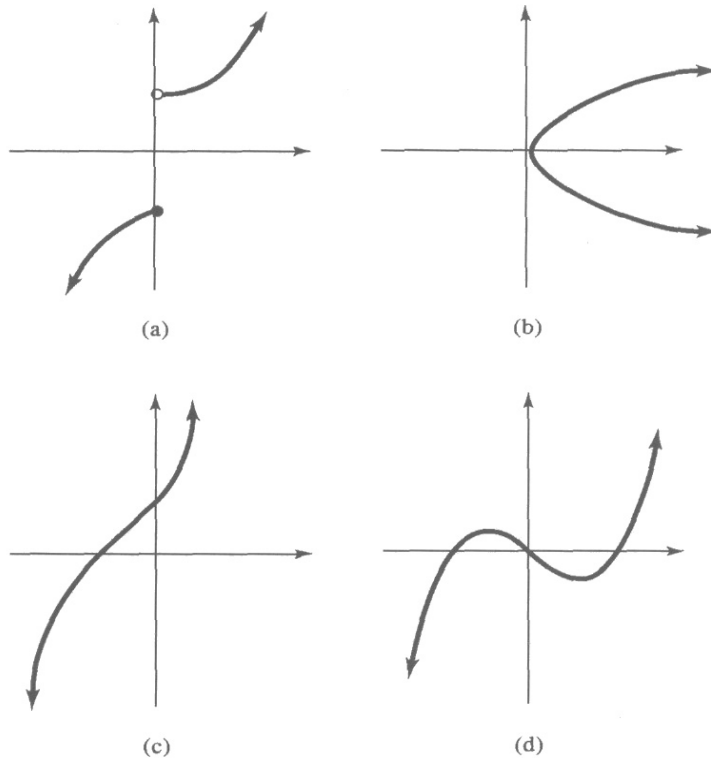
Exercícios 4.3

- ★1. A figura a seguir representa uma função.



Exercício 1

- Qual seu domínio? Qual seu contradomínio? Qual o conjunto imagem?
 - Qual a imagem de 5? E de 8?
 - Quais as pré-imagens de 9?
 - Esta função é sobrejetiva? É injetiva?
2. A figura a seguir ilustra várias relações binárias em $\mathbb{R} \times \mathbb{R}$. Quais são funções? Dentre as que são funções, quais as sobrejetivas? E quais as injetivas?



Exercício 2

3. Usando a notação $f(x) = 2x - 1$ para descrever a associação da função, escreva um conjunto de pares ordenados para os casos do contradomínio ser $\mathbb{R}$ e
- o domínio ser $S = \{0, 1, 2\}$
 - o domínio ser $S = \{1, 2, 4, 5\}$
 - o domínio ser $S = \{\sqrt{7}, 1, 5\}$

- ★4. Se $f: \mathbb{Z} \rightarrow \mathbb{Z}$ é definida por $f(x) = 3x$, encontre $f(A)$ para
- $A = \{1, 3, 5\}$
 - $A = \{x \mid x \in \mathbb{Z} \text{ e } (\exists y) (y \in \mathbb{Z} \text{ e } x = 2y)\}$
5. Seja $S = \{0, 2, 4, 6\}$ e $T = \{1, 3, 5, 7\}$. Determine se cada um dos conjuntos de pares ordenados a seguir é ou não uma função com domínio S e contradomínio T . Em cada caso afirmativo, indique se a função é injetiva e/ou sobrejetiva.
- $\{(0, 2), (2, 4), (4, 6), (6, 0)\}$
 - $\{(6, 3), (2, 1), (0, 3), (4, 5)\}$
 - $\{(2, 3), (4, 7), (0, 1), (6, 5)\}$
 - $\{(2, 1), (4, 5), (6, 3)\}$
 - $\{(6, 1), (0, 3), (4, 1), (0, 7), (2, 5)\}$
6. Apresente a função inversa para cada bijeção do Exercício 5.
7. Quais dos itens a seguir representam funções? Quais funções são injetivas? Quais são sobrejetivas? Apresente a função inversa para as funções bijetivas.
- ★a. $f: \mathbb{Z} \rightarrow \mathbb{N}$, onde f é definida por $f(x) = x^2 + 1$
 - ★b. $g: \mathbb{N} \rightarrow \mathbb{Q}$, onde g é definida por $g(x) = 1/x$
 - ★c. $h: \mathbb{Z} \times \mathbb{N} \rightarrow \mathbb{Q}$, onde h é definida por $h(z, n) = z/(n + 1)$
 - ★d. $f: \{1, 2, 3\} \rightarrow \{p, q, r\}$, onde $f = \{(1, q), (2, r), (3, p)\}$
 - ★e. $g: \mathbb{N} \rightarrow \mathbb{N}$, onde g é dada por $g(x) = 2^x$
 - ★f. $h: \mathbb{R}^2 \rightarrow \mathbb{R}^2$, onde h é definida por $h(x, y) = (y + 1, x + 1)$
 - $g: \mathbb{Z}^2 \rightarrow \mathbb{N}$, onde g é definida por $g(x, y) = x^2 + 2y^2$
 - $h: \mathbb{R} \rightarrow \mathbb{R}$, onde h é definida por $h(x) = 1/\sqrt{x+1}$
 - i. $h: \mathbb{N}^3 \rightarrow \mathbb{N}$, onde h é dada por $h(x, y, z) = x + y - z$
8. Sejam $S = \{a, b, c, d\}$ e $T = \{x, y, z\}$.
- a. Apresente um exemplo de função de S em T que não seja nem sobrejetiva nem injetiva.
 - b. Apresente um exemplo de função de S em T que seja sobrejetiva, mas não seja injetiva.
 - c. É possível encontrar uma função de S em T que seja injetiva?
9. O que podemos dizer sobre x se $\lfloor x \rfloor = \lceil x \rceil$?
10. Demonstre que $\lfloor x \rfloor = -\lceil -x \rceil$.
- ★11. Demonstre que se $2^k < n < 2^{k+1}$, então $k = \lfloor \log n \rfloor$ e $k + 1 = \lceil \log n \rceil$. (Neste caso, $\log n$ significa $\log_2 n$.)
12. Demonstre que $\lfloor \log n \rfloor + 1 = \lceil \log(n + 1) \rceil$. (Neste caso, $\log n$ significa $\log_2 n$.) (Dica: Seja $2^k \leq n \leq 2^{k+1}$.)
13. Sejam S um conjunto e A um subconjunto de S . A **função característica** de A é uma função $c_A: S \rightarrow \{0, 1\}$ com $c_A(x) = 1$ apenas quando $x \in A$.
- a. Seja $S = \{1, 2, 3, 4, 5\}$ e $A = \{1, 3, 5\}$. Apresente os pares ordenados que compõem c_A .
 - b. Demonstre que para qualquer conjunto S e quaisquer subconjuntos A e B de S , $c_{A \cap B}(x) = c_A(x) \cdot c_B(x)$.
 - c. Verifique se é verdade que, para qualquer conjunto S e quaisquer subconjuntos A e B de S , $c_{A \cup B}(x) = c_A(x) + c_B(x)$. Prove ou dê um contra-exemplo.
14. A **função de Ackerman**, que leva $\mathbb{N}^2$ em $\mathbb{N}$ é uma função que cresce muito rapidamente. Ela é dada por
- $$A(0, y) = 1 \text{ para todos os } y \in \mathbb{N}$$
- $$A(1, 0) = 2$$
- $$A(x, 0) = x + 2 \text{ para } x \geq 2$$
- $$A(x + 1, y + 1) = A(A(x, y + 1), y) \text{ para todos } x \in \mathbb{N}, y \in \mathbb{N}.$$
- a. Encontre uma equação $f(x)$ que descreva $A(x, 1)$ para todo $x \geq 1$.
 - b. Encontre uma equação $g(x)$ que descreva $A(x, 2)$ para todo $x \geq 1$.
 - c. Calcule o valor de $A(4, 3)$.
- ★15. Sejam $S = \{2, 4, 6, 8\}$ e $T = \{1, 5, 7\}$.
- a. Encontre o número de funções de S em T .
 - b. Encontre o número de funções sobrejetivas de S em T .

16. Sejam $S = \{P, Q, R\}$ e $T = \{k, l, m, n\}$.
- Encontre o número de funções de S em T .
 - Encontre o número de funções injetivas de S em T .
17. a. Para $|S| = 2, 3$ e 4 , respectivamente, use o teorema sobre o número de funções para mostrar que o número de funções injetivas de S em S é igual ao número de funções sobrejetivas de S em S .
 b. Argumente que para $|S| = n$, $f: S \rightarrow S$ é injetiva se, e somente se, S é sobrejetiva.
 c. Encontre um conjunto infinito S e uma função $f: S \rightarrow S$ tal que f seja injetiva, mas não seja sobrejetiva.
 d. Encontre um conjunto infinito S e uma função $f: S \rightarrow S$ tal que f seja sobrejetiva, mas não seja injetiva.
- ★18. Seja $|S| = n$. Encontre:
- O número de funções de S em S
 - O número de funções injetivas de S em S
 - O número de funções sobrejetivas de S em S (veja o Exercício 17)
 - O número de permutações de S sobre S
 - O número de permutações sem pontos fixos de S em S
 - Ordene os valores obtidos nos itens (a) a (e) do menor para o maior, e explique por que esta ordenação é razoável.
19. a. O projeto de desenvolvimento de um sistema precisa que cinco tarefas sejam atribuídas a Maria, João e Solange. De quantas maneiras esta distribuição pode ser feita de forma que todos os três envolvidos se encarreguem de pelo menos uma tarefa?
 b. De quantas maneiras as tarefas podem ser divididas se Maria precisar encarregar-se do desenvolvimento do plano de teste, que é uma das cinco tarefas, mas ainda puder acumular outra tarefa? (Dica: Considere separadamente os casos em que Maria acumula e não acumula outra tarefa.)
20. Demonstre que $S(m, n)$, o número de maneiras de particionar um conjunto em n blocos, é igual a $1/n!$ vezes o número de funções sobrejetivas de um conjunto com m elementos em um conjunto com n elementos. (Dica: Compare o Exercício 19 acima com o Exercício 43 da Seção 4.1.)
21. Seja f uma função, $f: S \rightarrow T$.
- Mostre que para todos os subconjuntos A e B de S , $f(A \cap B) \subseteq f(A) \cap f(B)$.
 - Mostre que $f(A \cap B) = f(A) \cap f(B)$ para todos os subconjuntos de S se, e somente se, f é injetiva.
22. Pela definição de uma função f de S em T , f é um subconjunto de $S \times T$, onde a imagem de todo $s \in S$ por f é determinada de forma única pela segunda componente do par ordenado (s, t) em f . Consideremos agora uma relação p qualquer em $S \times T$. A relação p é um subconjunto de $S \times T$ no qual alguns elementos de S podem aparecer mais de uma vez. Podemos entender p como uma **função não-determinística** de um subconjunto de S em T . Um $s \in S$ que não apareça como primeiro elemento de qualquer par ordenado representa um elemento fora do domínio de p . Para um elemento $s \in S$ que apareça uma ou mais vezes como primeira componente, p pode escolher para imagem de s qualquer uma das segundas componentes associadas a s pelos pares ordenados.
- Seja $S = \{1, 2, 3\}$, $T = \{a, b, c\}$ e $U = \{m, n, o, p\}$. Seja p uma relação binária em $S \times T$ e σ uma relação binária em $T \times U$ definidas por
- $$p = \{(1, a), (1, b), (2, b), (2, c), (3, c)\}$$
- $$\sigma = \{(a, m), (a, o), (a, p), (b, n), (b, p), (c, o)\}$$
- Imaginando p e σ como funções não-determinísticas de S em T e T em U , respectivamente, podemos formar a composição $\sigma \circ p$, uma função não-determinística de S em U .
- Qual o conjunto de possíveis imagens de 1 por $\sigma \circ p$?
 - Qual o conjunto de possíveis imagens de 2 por $\sigma \circ p$? E de 3?
23. Seja $\mathcal{C}$ uma coleção de conjuntos, e defina uma operação binária ρ em $\mathcal{C}$ da seguinte maneira: Para $S, T \in \mathcal{C}$, $S \rho T \leftrightarrow "S \text{ é equivalente a } T"$ Mostre que ρ é uma relação de equivalência em $\mathcal{C}$.
24. Agrupe os conjuntos a seguir em classes de equivalência, segundo a relação de equivalência do Exercício 23:

$$A = \{2, 4\}$$

$$B = \mathbb{N}$$

$$C = \{x \mid x \in \mathbb{N} \text{ e } (\exists y) (y \in \mathbb{N} \text{ e } x = 2 * y)\}$$

$$D = \{a, b, c, d\}$$

$$E = \mathcal{P}(\{1, 2\})$$

$$F = \mathbb{Q}^+$$

25. Sejam $S = \{1, 2, 3, 4\}$, $T = \{1, 2, 3, 4, 5, 6\}$ e $U = \{6, 7, 8, 9, 10\}$. Sejam ainda $f = \{(1, 2), (2, 4), (3, 3), (4, 6)\}$ uma função de S em T e $g = \{(1, 7), (2, 6), (3, 9), (4, 7), (5, 8), (6, 9)\}$ uma função de T em U . Escreva os pares ordenados da função $g \circ f$.
- ★26. Seja $f: \mathbb{N} \rightarrow \mathbb{N}$ definida por $f(x) = x + 1$. Seja $g: \mathbb{N} \rightarrow \mathbb{N}$ definida por $g(x) = 3x$. Calcule o seguinte:
- $(g \circ f)(5)$
 - $(f \circ g)(5)$
 - $(g \circ f)(x)$
 - $(f \circ g)(x)$
 - $(f \circ f)(x)$
 - $(g \circ g)(x)$
27. a. Seja $f: \mathbb{R} \rightarrow \mathbb{Z}$ definida por $f(x) = \lfloor x \rfloor$. Seja $g: \mathbb{Z} \rightarrow \mathbb{N}$ definida por $g(x) = x^2$. Quanto vale $(g \circ f)(-4, 7)$?
- b. Seja f uma função que leve o conjunto de todos os livros em inteiros de tal forma que f atribua a cada livro o número da última página. Seja $g: \mathbb{Z} \rightarrow \mathbb{Z}$ dada por $g(x) = 2x$. Quanto vale $(g \circ f)$ (este livro)?
- c. Seja f uma função que leve cadeia de caracteres alfabéticos e espaços em branco em cadeias de caracteres apenas com consoantes, de forma que f remove qualquer vogal ou espaço em branco da cadeia original. Seja g uma função que leve cadeias de consoantes em inteiros, associando a cada cadeia de consoantes o número de caracteres que contém. Quanto vale $(g \circ f)$ (abraham lincoln)?
28. As funções a seguir são aplicações de $\mathbb{R}$ em $\mathbb{R}$. Forneça equações que descrevam as funções compostas $g \circ f$ e $f \circ g$ para cada item:
- $f(x) = 6x^3$, $g(x) = 2x$
 - $f(x) = (x - 1)/2$, $g(x) = 4x^2$
 - $f(x) = \lceil x \rceil$, $g(x) = \lfloor x \rfloor$
- ★29. Seja $A = \{1, 2, 3, 4, 5\}$. Escreva cada uma das permutações de A de forma cíclica:
- $f = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 3 & 1 & 5 & 4 & 2 \end{pmatrix}$
 - $f = \{(1, 4), (2, 5), (3, 2), (4, 3), (5, 1)\}$
30. Seja $A = \{a, b, c, d\}$. Escreva cada uma das permutações de A a seguir na forma de matriz:
- $f = \{(a, c), (b, b), (c, d), (d, a)\}$
 - $f = \{c, a, b, d\}$
 - $f = \{d, b, a\}$
 - $f = (a, b) \circ (b, d) \circ (c, a)$
31. Seja A qualquer conjunto e S_A o conjunto de todas as permutações de A . Sejam $f, g, h \in S_A$. Demonstre que as funções $h \circ (g \circ f)$ e $(h \circ g) \circ f$ são iguais, mostrando assim que podemos escrever $h \circ g \circ f$ sem o uso de parênteses.
- ★32. Encontre as composições dos seguintes ciclos que representam permutações em $A = \{1, 2, 3, 4, 5, 6, 7, 8\}$. Escreva sua resposta como uma composição de um ou mais ciclos disjuntos.
- $(1, 3, 4) \circ (5, 1, 2)$
 - $(2, 7, 8) \circ (1, 2, 4, 6, 8)$
 - $(1, 3, 4) \circ (5, 6) \circ (2, 3, 5) \circ (6, 1)$ (Pelo Exercício 31, podemos omitir os parênteses para indicação de agrupamento.)
 - $(2, 7, 1, 3) \circ (2, 8, 7, 5) \circ (4, 2, 1, 8)$
33. A "pilha" é uma estrutura de armazenamento de dados cuja operação é muito parecida com uma pilha de pratos na pia de um restaurante. Todos os locais de armazenamento começam vazios. Um elemento de dado é incluído no topo da pilha através da instrução "push", que "empurra" todos os itens já empilhados uma posição para baixo a fim de abrir espaço para ele. Apenas o elemento no topo da pilha pode ser acessado a qualquer momento, e ele pode ser examinado e removido da pilha através da instrução "pop".

Consideremos cadeias de inteiros que tenham um número par de caracteres de tamanho n e onde a metade dos caracteres são inteiros positivos e a outra metade são zeros. Processaremos essas cadeias através de um armazenamento na forma de pilha como a seguir: à medida que lermos da esquerda para direita, aplicamos a instrução push a qualquer caracter diferente de zero. Caracteres zero causam a instrução pop na pilha e a impressão do elemento recuperado por esta instrução. Portanto, o processamento da cadeia 12030040 resulta na saída 2314, enquanto que o processamento da cadeia 12304000 resulta na saída 3421. (Uma cadeia como 10020340 não pode ser tratada por este procedimento porque não podemos efetuar pop de dois elementos da pilha enquanto esta contiver apenas um.) Tanto 2314 quanto 3421 podem ser expressos como permutações,

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 1 & 4 \end{pmatrix} \text{ e } \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 4 & 2 & 1 \end{pmatrix}$$

no conjunto $A = \{1, 2, 3, 4\}$.

- a. Qual permutação de $A = \{1, 2, 3, 4\}$ é gerada pela aplicação deste procedimento à cadeia 12003400?
 - b. Apresente uma permutação de $A = \{1, 2, 3, 4\}$ que não possa ser gerada a partir de qualquer cadeia onde os dígitos 1, 2, 3 e 4 apareçam nesta ordem, independentemente da posição em que os zeros sejam colocados.
34. Sejam $f: S \rightarrow T$ e $g: T \rightarrow U$ funções.
- a. Prove que se $g \circ f$ é injetiva, então/também g é.
 - b. Prove que se $g \circ f$ é sobrejetiva, então g também o é.
 - c. Apresente um exemplo onde $g \circ f$ seja injetiva sem que g seja injetiva.
 - d. Apresente um exemplo onde $g \circ f$ seja sobrejetiva sem que f seja sobrejetiva.
35. Seja $S = \{a, b, c, d\}$. Quantos elementos tem S_A ? Quantos desses elementos são permutações sem pontos fixos? Escreva todas as permutações sem pontos fixos na forma matricial.
- ★36. Em uma turma de programação com sete alunos, o instrutor deseja que cada aluno modifique o programa de uma aula anterior. Mas nenhum deles pode alterar seu próprio programa. De quantas maneiras o instrutor pode distribuir os programas entre os alunos?
37. a. Procure, em um livro de cálculo, a representação de e^x por séries de Maclaurin.
 b. Use a resposta do item (a) para encontrar uma representação por séries para e^{-1} .
 c. Use uma calculadora para calcular um valor aproximado para e^{-1} .
 d. De que forma as respostas dos itens (b) e (c) podem nos ajudar a achar o número aproximado de permutações sem pontos fixos de n objetos quando n for muito grande, digamos $n \geq 10$? (Dica: Veja equação (4) desta seção.)
 e. Aplique esta abordagem ao Exercício 36 e compare os resultados.
 f. Quantas permutações sem pontos fixos existem, aproximadamente, para 10 objetos?
- ★38. Para cada uma das seguintes bijeções $f: \mathbb{R} \rightarrow \mathbb{R}$, encontre f^{-1} .
- a. $f(x) = 2x$
 - b. $f(x) = x^3$
 - c. $f(x) = (x + 4)/3$
39. a. Seja f uma função $f: S \rightarrow T$. Se existir uma função $g: T \rightarrow S$ tal que $g \circ f = i_S$, então g é chamada de **inversa à esquerda** de f . Mostre que f tem uma inversa à esquerda se, e somente se, f for injetiva.
 b. Seja f uma função $f: S \rightarrow T$. Se existir uma função $g: T \rightarrow S$ tal que $g \circ f = i_T$, então g é chamada de **inversa à direita** de f . Mostre que f tem uma inversa à direita se, e somente se, f for sobrejetiva.
 c. Seja $f: \mathbb{N} \rightarrow \mathbb{N}$ dada por $f(x) = 3x$. Então f é injetiva. Encontre duas inversas à esquerda diferentes para f .
40. Sejam f e g bijeções, $f: S \rightarrow T$ e $g: T \rightarrow U$. Então f^{-1} e g^{-1} existem. Além disso, $g \circ f$ é uma bijeção de S em U . Mostre que $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$.
41. Seja f uma função, $f: S \rightarrow T$.
- a. Defina uma relação binária ρ em S por $x \rho y \leftrightarrow f(x) = f(y)$. Prove que ρ é uma relação de equivalência.
 - b. Para $S = T = \mathbb{Z}$ e $f(x) = 3x^2$, qual a classe de equivalência [4] sob a relação do item (a)?
42. Demonstre, exibindo constantes que satisfaçam à definição de ordem de grandeza, que $f = \Theta(g)$ se $f(x) = x$ e $g(x) = 17x + 1$.

43. Demonstre exibindo constantes que satisfaçam à definição de ordem de grandeza, que $f = \Theta(g)$ se $f(x) = 3x^3 - 7x$ e $g(x) = x^3/2$.
44. Vimos nesta seção que $h_1 = \Theta(h)$ implica que a partir de algum ponto, h_1 está contido em um "envelope" de h . Será que este envelope pode estar totalmente acima ou totalmente abaixo de h ? Explique.
45. Demonstre que se f_1 é uma função que é $\Theta(g_1)$ e f_2 é uma função que é $\Theta(g_2)$, então a função $f_1 + f_2$, definida por $(f_1 + f_2)(x) = f_1(x) + f_2(x)$ é $\Theta(\max(g_1, g_2))$, onde $(\max(g_1, g_2))(x) = \max(g_1(x), g_2(x))$.

Os Exercícios 46 a 50 exigem uma certa familiaridade com algumas idéias do cálculo.

Como alternativa à definição de ordem de grandeza, um teste de limite pode ser usado para demonstrar que $f = \Theta(g)$:

$$f = \Theta(g) \text{ se } \lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = p, \text{ onde } p \text{ é um número real positivo}$$

Portanto, $2x^2 + 7 = \Theta(x^2)$, porque

$$\lim_{x \rightarrow \infty} \frac{2x^2 + 7}{x^2} = \lim_{x \rightarrow \infty} \frac{2 + \frac{7}{x^2}}{1} = \frac{2}{1} = 2$$

- ★46. Use o teste de limite para fazer o Exercício 42 novamente.
47. Use o teste de limite para fazer o Exercício 43 novamente.

Se $\lim_{x \rightarrow \infty} f(x) = \infty$ e $\lim_{x \rightarrow \infty} g(x) = \infty$ e f e g são funções diferenciáveis, então a regra de L'Hospital diz que

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{f'(x)}{g'(x)}$$

É outro teste de limite se

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = 0$$

então a classe de complexidade $\Theta(f)$ tem ordem de grandeza menor do que a classe $\Theta(g)$.

- ★48. Use o segundo teste de limite acima para demonstrar que a classe $\Theta(x)$ tem ordem de grandeza menor do que a classe $\Theta(x^2)$.
49. Use o segundo teste de limite acima para demonstrar que a classe $\Theta(\log x)$ tem ordem de grandeza menor do que a classe $\Theta(x)$.
50. Use ambos os testes de limite para agrupar as funções a seguir em classes de complexidade, e ordene estas classes:

$$x, \sqrt{x}, \log x, x^3, x \log x, 2x^3 + x, e^x, (\log x)^2, \ln x \text{ (i.e., } \log_e x), x^3 + \log x$$

Seção 4.4 Matrizes

Terminologia

Uma **matriz** é uma disposição retangular de dados, normalmente numéricos. Por exemplo, A pode ser uma matriz onde

$$A = \begin{bmatrix} 1 & 0 & 4 \\ 3 & -6 & 8 \end{bmatrix}$$

Neste caso, A tem duas linhas e três colunas. As **dimensões** da matriz são o número de linhas e colunas que a compõem; portanto, A é uma matriz 2×3 .

Os elementos da matriz A são denotados por a_{ij} onde i é o número da linha e j o número da coluna do elemento. Na matriz A acima, $a_{23} = 8$ pois 8 é o elemento da segunda linha, terceira coluna de A .

EXEMPLO 43 Seja

$$\mathbf{B} = \begin{bmatrix} 1 & 5 & 7 & 0 \\ -5 & 1 & 9 & 4 \\ 3 & 0 & -4 & 2 \end{bmatrix}$$

Então B é uma matriz 3×4 com $b_{14} = 0$, $b_{22} = 1$ e $b_{31} = 3$.

PRÁTICA 43 Na matriz

$$\mathbf{A} = \begin{bmatrix} 1 & 4 & -6 & 8 \\ 3 & 0 & 1 & -7 \end{bmatrix}$$

quais são os elementos a_{23} , a_{24} e a_{13} ?

Em uma matriz, a posição de seus elementos é importante. Portanto, para que duas matrizes sejam **iguais**, elas devem ter as mesmas dimensões e os mesmos elementos nos mesmos lugares.

EXEMPLO 44 Sejam

$$\mathbf{X} = \begin{bmatrix} x & 4 \\ 1 & y \\ z & 0 \end{bmatrix} \quad \mathbf{Y} = \begin{bmatrix} 3 & 4 \\ 1 & 6 \\ 2 & w \end{bmatrix}$$

Se $\mathbf{X} = \mathbf{Y}$, então $x = 3$, $y = 6$, $z = 2$ e $w = 0$.

Freqüentemente estaremos interessados em matrizes quadradas, nas quais o número de linhas é igual ao número de colunas. Se A é uma matriz quadrada $n \times n$, então os elementos $a_{11}, a_{22}, \dots, a_{nn}$ formam a **diagonal principal** da matriz. Se os elementos correspondentes ao imaginarmos a matriz dobrada ao longo de sua diagonal principal forem iguais, então a matriz é considerada simétrica. Em uma matriz **simétrica**, $a_{ij} = a_{ji}$.

EXEMPLO 45 A matriz quadrada 3×3

$$\mathbf{A} = \begin{bmatrix} 1 & 5 & 7 \\ 5 & 0 & 2 \\ 7 & 2 & 6 \end{bmatrix}$$

é simétrica. A parte triangular superior (acima da diagonal principal) é uma reflexão da parte triangular inferior. Perceba que, por exemplo, $a_{21} = a_{12} = 5$.

Matrizes são úteis para representarem qualquer tipo de dado que se encaixe em forma tabular.

EXEMPLO 46 Temperaturas médias de cada mês em três diferentes cidades podem ser arrumadas de forma conveniente em uma matriz 3×12 :

Cidade	Mês											
	J	F	M	A	M	J	J	A	S	O	N	D
<i>P</i>	23	26	38	47	58	71	78	77	69	55	39	33
<i>Q</i>	14	21	33	38	44	57	61	59	49	38	25	21
<i>R</i>	35	46	54	67	78	86	91	94	89	75	62	51

Um modo mais geral de representar arranjos de dados são os **arrays**. Os arrays são arranjos «-dimensionais de dados, onde n pode ser qualquer inteiro positivo. Se $n = 1$, então os dados são arrumados ao longo de

uma única linha, que é, portanto, uma lista ou uma seqüência finita de elementos. Se $n = 2$, o array é uma matriz. Se $n = 3$, podemos representar o array como diversas camadas de matrizes bidimensionais. Para $n > 3$ podemos tratar formalmente os elementos do array, mas não conseguimos visualizar o arranjo. A estrutura de dados para arrays é oferecida pela maioria das linguagens de programação de alto nível; geralmente o número de elementos de cada dimensão do array deve ser explicitamente definido no programa. O array X do Exemplo 44 poderia ser declarado como um array 3×2 — um array bidimensional (uma matriz) com três elementos em uma dimensão e 2 elementos na outra (isto é, três linhas e duas colunas).

Operações de Matrizes

Definiremos quatro operações aritméticas de matrizes cujos elementos sejam numéricos. A primeira envolve multiplicação de uma matriz por um **escalar**, um único número real. Simplesmente multiplicamos cada elemento da matriz pelo escalar e obtemos uma matriz com as mesmas dimensões que a matriz original. Esta operação é chamada de **multiplicação por escalar**.

EXEMPLO 47 O resultado da multiplicação da matriz

$$\mathbf{A} = \begin{bmatrix} 1 & 4 & 5 \\ 6 & -3 & 2 \end{bmatrix}$$

pelo escalar $r = 3$ é

$$3\mathbf{A} = \begin{bmatrix} 3 & 12 & 15 \\ 18 & -9 & 6 \end{bmatrix}$$

A **adição** de duas matrizes A e B só é definida quando A e B têm as mesmas dimensões; neste caso, ela consiste apenas na soma dos elementos correspondentes. Formalmente, se A e B são ambas matrizes $n \times m$, então $C = A + B$ é uma matriz $n \times m$ cujos elementos são

EXEMPLO 48 Para

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 6 \\ 1 & 0 & 4 \\ -4 & 5 & 1 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 0 & -2 & 8 \\ 1 & 5 & 2 \\ 2 & 3 & 3 \end{bmatrix}$$

a matriz $A + B$ é

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} 1 & 1 & 14 \\ 3 & 5 & 6 \\ -2 & 8 & 4 \end{bmatrix}$$

PRÁTICA 44 Para $r = 2$,

$$\mathbf{A} = \begin{bmatrix} 1 & 7 \\ -3 & 4 \\ 5 & 6 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 4 & 0 \\ 9 & 2 \\ -1 & 4 \end{bmatrix}$$

encontre $r\mathbf{A} + \mathbf{B}$.

A **subtração** de matrizes é definida por $\mathbf{A} - \mathbf{B} = \mathbf{A} + (-1)\mathbf{B}$.

Em uma **matriz nula** todos os elementos são 0. Se somarmos uma matriz nula $n \times m$, denotada por $\mathbf{0}$ a qualquer matriz $n \times m$ $\mathbf{A}$, o resultado será a própria matriz $\mathbf{A}$. Podemos representar isto pela equação matricial

$$\mathbf{0} + \mathbf{A} = \mathbf{A}$$

Esta equação é verdadeira devido a uma equação semelhante que vale para todos os elementos individualmente, $0 + a_{ij} = a_{ij}$. Outras equações matriciais são verdadeiras devido às equações semelhantes que são verdadeiras para cada um de seus elementos individualmente.

EXEMPLO 49 Se A e B são matrizes $n \times m$ e r e s são escalares, as equações matriciais a seguir são verdadeiras:

$$\begin{aligned} 0 + A &= A \\ A + B &= B + A \\ (A + B) + C &= A + (B + C) \\ r(A + B) &= rA + rB \\ (r + s)A &= rA + sA \\ r(sA) &= (rs)A \end{aligned}$$

Para demonstrar que $A + B = B + A$, por exemplo, é suficiente notar que $a_{ij} + b_{ij} = b_{ij} + a_{ij}$ para cada elemento das matrizes A e B . •

Poderíamos esperar que na **multiplicação** de matrizes, os elementos fossem simplesmente multiplicados individualmente com seus correspondentes. No entanto, esta definição não é tão simples. A definição da multiplicação de matrizes é baseada no uso de matrizes pela Matemática para representar funções chamadas transformações lineares, que levam pontos de espaços «-dimensionais em pontos de espaços m -dimensionais. Apesar de não usarmos a multiplicação de matrizes para este fim, usaremos esta operação da forma convencional.

Para calcular A vezes B , $A \cdot B$, o número de colunas de A deve ser igual ao número de linhas de B . Desta forma, podemos calcular $A \cdot B$ se A for uma matriz $n \times m$ e B for uma matriz $m \times p$. O resultado é uma matriz $n \times p$. Uma entrada na linha i , coluna j de $A \cdot B$ é obtida multiplicando-se os elementos da linha i de A pelos elementos correspondentes da coluna j de B e somando-se os resultados. Formalmente, $A \cdot B = C$, onde

$$c_{ij} = \sum_{k=1}^m a_{ik} b_{kj}$$

EXEMPLO 50 Seja

$$A = \begin{bmatrix} 2 & 4 & 3 \\ 4 & -1 & 2 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 3 \\ 2 & 2 \\ 6 & 5 \end{bmatrix}$$

A é uma matriz 2×3 e B é uma matriz 3×2 , então o produto $A \cdot B$ existe e é uma matriz C de dimensões 2×2 . Para encontrar o elemento c_{11} , multiplicamos os elementos da linha 1 de A aos elementos correspondentes da coluna 1 de B e somamos os resultados:

$$2(5) + 4(2) + 3(6) = 10 + 8 + 18 = 36$$

$$\begin{bmatrix} 2 & 4 & 3 \\ 4 & -1 & 2 \end{bmatrix} \begin{bmatrix} 5 \\ 2 \\ 6 \end{bmatrix} = \begin{bmatrix} 36 & \text{—} \\ \text{—} & \text{—} \end{bmatrix}$$

O Elemento c_{12} é obtido pela multiplicação dos elementos da linha 1 de A com os elementos correspondentes da coluna 2 de B e somamos os resultados:

$$\begin{bmatrix} 2 & 4 & 3 \\ 4 & -1 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 2 \\ 5 \end{bmatrix} = \begin{bmatrix} 36 & 29 \\ \text{—} & \text{—} \end{bmatrix}$$

O produto completo é

$$\begin{bmatrix} 2 & 4 & 3 \\ 4 & -1 & 2 \end{bmatrix} \begin{bmatrix} 5 & 3 \\ 2 & 2 \\ 6 & 5 \end{bmatrix} = \begin{bmatrix} 36 & 29 \\ 30 & 20 \end{bmatrix}$$

•

PRÁTICA 45 Encontre $A \cdot B$ e $B \cdot A$ para

$$A = \begin{bmatrix} 1 & 4 \\ 6 & -2 \end{bmatrix} \quad B = \begin{bmatrix} 3 & 6 \\ 3 & 4 \end{bmatrix}$$

Na Prática 45, vimos que ainda que A e B tenham dimensões tais que tanto $A \cdot B$ quanto $B \cdot A$ estejam definidas, $A \cdot B$ não é necessariamente igual a $B \cdot A$. Existem, no entanto, diversas equações matriciais envolvendo multiplicações que são verdadeiras.

EXEMPLO 51 Se A , B e C são matrizes com as dimensões apropriadas, as seguintes equações matriciais são verdadeiras (a notação $A(B \cdot C)$ é a abreviação de $A \cdot (B \cdot C)$):

$$A(B \cdot C) = (A \cdot B)C$$

$$A(B + C) = A \cdot B + A \cdot C$$

$$(A + B) \cdot C = A \cdot C + B \cdot C$$

Verificar essas equações matriciais para dimensões particulares é simples, ainda que seja um trabalho tedioso.

A matriz $n \times n$ com 1's na diagonal principal e 0's nas demais posições é chamada de **matriz identidade** e denotada por I . Se multiplicarmos I por qualquer matriz $n \times n$ A , obteremos A como resultado. Vale, portanto, a equação

$$I \cdot A = A \cdot I = A$$

PRÁTICA 46 Seja

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

Verifique que $I \cdot A = A \cdot I = A$

É fácil escrever um algoritmo para multiplicação de matrizes a partir da definição. Uma versão em pseudocódigo do algoritmo é mostrada a seguir, onde a notação com colchetes $A[i, j]$ indica o elemento a_{ij} .

Algoritmo *MultiplicaçãoMatrizes*

procedure *MultiplicaçãoMatrizes* (A, B array; **var** C : array);
 {calcula $A \cdot B$ para A matriz $n \times m$ e B matriz $m \times p$, armazena o resultado em C }

begin

for i : = 1 **to** n **do**

for j : = 1 **to** p **do**

begin

$C[i, j]$:= 0;

for k : = 1 **to** m **do**

 {*} $C[i, j]$:= $C[i, j] + A[i, k] * B[k, j]$;

end;

 escreve o produto da matriz C

end;

Os passos computacionais realizados neste algoritmo são multiplicações e adições; uma multiplicação e uma adição são realizadas sempre que a linha {*} é executada. A linha {*} está dentro do loop mais interno de três loops aninhados e é executada $n \cdot p$ vezes. (Apesar de isto ser óbvio, pode ser justificado pelo Princípio da Multiplicação como o número de possibilidades de se escolher os valores dos índices i, j e k .) Se A e B são

ambas matrizes $n \times n$, então são necessárias $\Theta(n^3)$ multiplicações e $\Theta(n^3)$ adições. A quantidade total de trabalho é, portanto, $\Theta(n^3) + \Theta(n^3) = \Theta(n^3)$.

Dada a definição de multiplicação de matrizes, é difícil ver como podemos evitar algoritmos $\Theta(n^3)$ para calcular o produto de duas matrizes $n \times n$, mas uma abordagem mais profunda (que não realizaremos aqui) permite este tipo de melhoria.

Matrizes Booleanas

No próximo capítulo, estaremos interessados em matrizes apenas com 0s e 1s como elementos, chamadas matrizes **booleanas** (em homenagem a George Boole, um matemático inglês do século XIX; Boole também cedeu seu nome à *álgebra booleana*, que será vista mais adiante neste livro). Podemos definir uma operação de *multiplicação de matrizes booleanas* $A \times B$, usando multiplicações e adições booleanas no lugar das multiplicação e adição tradicionais. Essas operações são definidas da seguinte maneira:

$$\begin{aligned} \text{Multiplicação booleana: } & x \wedge y = \text{mín}(x, y) \\ \text{Adição booleana: } & x \vee y = \text{máx}(x, y) \end{aligned}$$

PRÁTICA 47 Preencha as tabelas a seguir com os resultados da multiplicação e adição booleanas:

x	y	$x \wedge y$	x	y	$x \vee y$
1	1		1	1	
1	0		1	0	
0	1		0	1	
0	0		0	0	

Tome as tabelas da Prática 49 e substitua os 1s por V e os 0s por F. Elas tornam-se as tabelas-verdade para a conjunção e disjunção, respectivamente; por isso, essas operações são frequentemente chamadas de **e booleano (e lógico)** e **ou booleano (ou lógico)**. (Mencionamos essas operações no Cap. 3 como uma forma de se realizar a união e a interseção de conjuntos representados como vetores de bits.)

A operação de **multiplicação das matrizes booleanas** $A \times B$ (para matrizes booleanas com as dimensões apropriadas) é então definida como

$$c_{ij} = \bigvee_{k=1}^m (a_{ik} \wedge b_{kj})$$

Podemos ainda definir dois análogos à adição convencional de matrizes (para matrizes booleanas de mesmas dimensões): $A \wedge B$, onde os elementos correspondentes são combinados através da multiplicação booleana e $A \vee B$, onde os elementos correspondentes são combinados através da adição booleana.

EXEMPLO 52 Sejam A e B matrizes booleanas,

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Então

$$A \wedge B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad A \vee B = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

e o produto booleano $A \times B$ é

$$A \times B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

PRÁTICA 48 No Exemplo 52, $A \times B = A \cdot B$? •

PRÁTICA 49 Calcule $B \times A$ no Exemplo 52. •

Revisão da Seção 4.4

Técnicas

- Soma, subtração, multiplicação e realização de multiplicação por escalar de matrizes.
- Realização de e, ou e multiplicação de matrizes booleanas

Idéias Principais

As matrizes são arranjos retangulares de dados que são usados para representar informações de forma tabular.

As matrizes têm sua própria aritmética, com operações de adição, subtração, multiplicação e multiplicação por escalares.

Matrizes booleanas podem ser manipuladas através de operações de e, ou e multiplicações booleanas.

Exercícios 4.4

★ 1. Para a matriz

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 0 \\ -4 & 1 \end{bmatrix}$$

quais os elementos a_{12} e a_{31} ?

2. Encontre x e y se

$$\begin{bmatrix} 1 & 3 \\ x & x+y \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 2 & 6 \end{bmatrix}$$

★ 3. Encontre x , y , z e w se

$$\begin{bmatrix} x+y & 2x-3y \\ z-w & z+2w \end{bmatrix} = \begin{bmatrix} 4 & -7 \\ -6 & 6 \end{bmatrix}$$

4. Se A é uma matriz simétrica, encontre u , v e w :

$$\mathbf{A} = \begin{bmatrix} 2 & w & u \\ 7 & 0 & v \\ 1 & -3 & 4 \end{bmatrix}$$

Nos Exercícios 5 a 18, se $r = 3$, $s = -2$, sabendo que

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ -1 & 0 \\ 3 & 4 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 4 & 1 & 2 \\ 6 & -1 & 5 \\ 1 & 3 & 2 \end{bmatrix}$$

$$\mathbf{C} = \begin{bmatrix} 2 & 4 \\ 6 & -1 \end{bmatrix} \quad \mathbf{D} = \begin{bmatrix} 4 & -6 \\ 1 & 3 \\ 2 & -1 \end{bmatrix}$$

Calcule os itens a seguir (quando possível):

★5. $A + D$

6. $A - D$

7. rB

8. sC

★9. $A + rD$

10. $B - rC$

11. $r(A + D)$

12. $r(sC)$

★13. $B \cdot D$

14. $D \cdot C$

15. $A \cdot C$

16. $C \cdot A$

★17. $C^2 = C \cdot C$

18. $B \cdot A + D$

Nos Exercícios 19 a 22 sejam

$$\mathbf{A} = \begin{bmatrix} 3 & -1 \\ 2 & 5 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 4 & 1 \\ 2 & -1 \end{bmatrix} \quad \mathbf{C} = \begin{bmatrix} 6 & -5 \\ 2 & -2 \end{bmatrix}$$

19. Calcule $A \cdot B$ e $B \cdot A$.

20. Calcule $A(B \cdot C)$ e $(A \cdot B)C$.

★21. Calcule $A(B + C)$ e $A \cdot B + A \cdot C$.

22. Calcule $(A + B)C$ e $A \cdot C = B \cdot C$.

23. Para

$$\mathbf{A} = \begin{bmatrix} 2 & 3 \\ 4 & 1 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} x & 3 \\ y & 2 \end{bmatrix}$$

encontre x e y tais que $A \cdot B = B \cdot A$.

24. a. Demonstre que $P = I$ para qualquer matriz identidade I .

b. Demonstre que $I^n = I$ para qualquer matriz identidade I e qualquer inteiro positivo n .

25. Uma matriz $n \times n$ A é **invertível** se existir uma matriz $n \times n$ B tal que $A \cdot B = B \cdot A = I$. Neste caso, B é chamada a **inversa** de A .

★a. Mostre que para

$$\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 2 & 2 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} -\frac{1}{2} & \frac{3}{4} \\ \frac{1}{2} & -\frac{1}{4} \end{bmatrix}$$

$$\mathbf{A} \cdot \mathbf{B} = \mathbf{B} \cdot \mathbf{A} = \mathbf{I}.$$

★b. Mostre que

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

não é inversível.

c. Mostre que

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

é inversível e sua inversa é

$$\mathbf{B} = \frac{1}{a_{11}a_{22} - a_{12}a_{21}} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}$$

se, e somente se, $a_{11}a_{22} - a_{12}a_{21} \neq 0$.

26. A **transposta** $\mathbf{A}^T$ de uma matriz $\mathbf{A}$ é obtida trocando-se suas linhas e colunas. Portanto, se denotamos o elemento na i -ésima linha, j -ésima coluna de $\mathbf{A}$ por $A(i, j)$, então $A^T(i, j) = A(j, i)$.

a. Encontre $\mathbf{A}^T$ para

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 4 \\ 6 & -2 & 1 \end{bmatrix}$$

b. Demonstre que se $\mathbf{A}$ é uma matriz quadrada, então $\mathbf{A}$ é simétrica se, e somente se, $\mathbf{A}^T = \mathbf{A}$.

c. Prove que $(\mathbf{A}^T)^T = \mathbf{A}$.

d. Prove que $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$.

e. Prove que $(\mathbf{A} \cdot \mathbf{B})^T = \mathbf{B}^T \cdot \mathbf{A}^T$.

★27. Encontre duas matrizes 2×2 $\mathbf{A}$ e $\mathbf{B}$ tais que $\mathbf{A} \cdot \mathbf{B} = \mathbf{0}$ mas $\mathbf{A} \neq \mathbf{0}$ e $\mathbf{B} \neq \mathbf{0}$.

28. Encontre três matrizes 2×2 $\mathbf{A}$, $\mathbf{B}$ e $\mathbf{C}$ tais que $\mathbf{A} \cdot \mathbf{C} = \mathbf{B} \cdot \mathbf{C}$, mas $\mathbf{A} \neq \mathbf{B}$.

29. Se $\mathbf{A}$ e $\mathbf{B}$ são matrizes $n \times n$, é sempre verdade que $(\mathbf{A} + \mathbf{B})^2 = \mathbf{A}^2 + 2(\mathbf{A} \cdot \mathbf{B}) + \mathbf{B}^2$? Isto sempre será verdadeiro?

★30. Para

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

encontre $\mathbf{A} \wedge \mathbf{B}$, $\mathbf{A} \vee \mathbf{B}$, $\mathbf{A} \times \mathbf{B}$ e $\mathbf{B} \times \mathbf{A}$.

31. A equação $\mathbf{A} \vee \mathbf{B} = \mathbf{A} \wedge \mathbf{B}$ pode ser verdadeira? Em caso afirmativo, quando?

32. Demonstre que $\mathbf{A} \vee \mathbf{B} = \mathbf{B} \vee \mathbf{A}$ e $\mathbf{A} \wedge \mathbf{B} = \mathbf{B} \wedge \mathbf{A}$.

★33. Demonstre que se uma matriz quadrada $\mathbf{A}$ de elementos numéricos é simétrica, então $\mathbf{A}^2$ também o é, onde $\mathbf{A}^2 = \mathbf{A} \cdot \mathbf{A}$.

34. Demonstre que $\mathbf{A} \cdot \mathbf{A}^T$ é simétrica para qualquer matriz $\mathbf{A}$ (veja o Exercício 26.)

35. Seja

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Para $n \geq 1$, seja $F(n)$ o n -ésimo valor da sequência de Fibonacci (veja o Exercício 20 do Cap. 2); seja $F(0) = 0$. Prove que para qualquer $n \geq 1$, A^n é dada por

$$\begin{bmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{bmatrix}$$

Revisão do Capítulo 4

Terminologia

álgebra relacionai
alteração em um banco de dados
array
banco de dados
banco de dados relacionai
bijeção
bloco
cadeia
cálculo relacionai
caminho crítico
cardinalidade de um conjunto
chão, função
chave cega
chave estrangeira
chave primária
chave primária composta
ciclos disjuntos
classe de complexidade
classes de equivalência
congruência módulo n
conjunto imagem
conjunto parcialmente ordenado
conjuntos equivalentes
contradomínio
diagonal principal
diagrama comutativo
diagrama de Hasse
diagrama entidade-relacionamento
(diagrama E-R)
diagrama PERT
dimensões de uma matriz
domínio
e booleano (e lógico)
elemento maximal
elemento máximo
elemento minimal
elemento mínimo.

entidades
escalar
exclusão de um banco de dados
fecho de uma relação
fecho reflexivo
fecho simétrico
fecho transitivo
função
função composta
função identidade
função injetiva (um-para-um)
função inversa
função permutação
função sobrejetiva
imagem
inclusão em um banco de dados
integridade de dados
integridade de entidades
integridade referencial
join
junção de banco de dados
linguagem funcional
matriz
matriz booleana
matriz identidade
matriz nula
matriz simétrica
matrizes iguais
modelo conceituai
modelo relacionai
multiplicação booleana de matrizes
multiplicação de matrizes
multiplicação por escalar
nodo (nó)
ordem de grandeza
ordenação parcial
ordenação topológica

ordenação total
ou booleano (ou lógico)
partição
permutação identidade
permutação sem ponto fixo
poset
predecessor em uma ordenação
parcial
predecessor imediato em uma ordenação parcial
pré-imagem
problema intratável
projeto
relação anti-simétrica
relação binária em $S \times T$
relação binária em um conjunto S
relação de equivalência
relação n -ária em $S_1 \times S_2 \times \dots \times S_n$
relação n -ária em um conjunto S
relação reflexiva
relação simétrica
relação transitiva
relação um-para-um (injetiva)
relação um-para-vários
relação unária
relação vários-para-um
relação vários-para-vários
restrições de uma ordenação parcial
select
soma de matrizes
soma módulo n
SQL
subtração de matrizes
sucessor em uma ordenação parcial
Teorema de Cantor
teto, função
tupla
vértice

Autotestes

Responda às seguintes questões com verdadeiro ou falso.

Seção 4.1

1. Em uma relação um-para-um, a primeira componente de pelo menos um elemento deve aparecer em dois diferentes pares ordenados.
2. Se uma relação simétrica binária contém (x, y) , então (y, x) não pertence à relação.
3. Um elemento mínimo de um conjunto parcialmente ordenado precede todos os elementos, exceto ele próprio.
4. Uma relação de equivalência não pode ser também uma ordenação parcial.
5. Uma ordenação parcial em um conjunto determina uma partição deste conjunto.

Seção 4.2

6. Uma relação em um banco de dados relacionai é um conjunto de n -uplas de valores de atributos.
7. Uma chave primária em uma relação é um conjunto mínimo de valores de atributos que identificam de maneira única cada tupla.
8. A operação select pode ser conseguida por uma união seguida de uma interseção.
9. Se uma tarefa não está no caminho crítico de um diagrama PERT, então esta tarefa é facultativa.
10. Se x precede y após uma ordenação topológica de um conjunto finito parcialmente ordenado, então x precede y na ordenação parcial original.

Seção 4.3

11. Uma relação binária em $S \times T$ que não é um-para-vários ou vários-para-vários é uma função de S em T .
12. Para demonstrar que uma função é sobrejetiva, tomamos um elemento arbitrário da imagem e mostramos que ele tem uma pré-imagem.
13. Para mostrar que uma função é injetiva, assumimos $f(s_1) = f(s_2)$ para quaisquer s_1 e s_2 no domínio e mostramos que $s_1 = s_2$.
14. A composição de duas funções permutação em um conjunto é também uma função permutação no conjunto.
15. Se f é $\Theta(g)$, então para valores grandes de x , os valores de $f(x)$ devem estar entre $\frac{1}{2}g(x)$ e $2g(x)$.

Seção 4.4

16. Duas matrizes que não tenham as mesmas dimensões não podem ser somadas.
17. Se A e B são matrizes quadradas, então $A \cdot B = B \cdot A$.
18. Se 0 denota a matriz nula $n \times n$, então $A \cdot 0 = 0 \cdot A$ para toda matriz A $n \times n$.
19. O algoritmo tradicional para multiplicação de matrizes é $\Theta(n^3)$.
20. Se A e B denotam matrizes booleanas quadradas, então $A \cdot B = B \cdot A$, onde $A \cdot B$ representa o produto booleano.

No Computador

Para os Exercícios 1 a 16, escreva um programa de computador que produza a saída desejada para a entrada dada.

1. *Entrada:* Os elementos de um conjunto finito S e uma lista de pares ordenados representando uma relação binária em S .
Saída: Mensagem que indique se a relação é um-para-um, um-para-vários, vários-para-um ou vários-para-vários.
2. *Entrada:* Os elementos em um conjunto finito S e duas listas de pares ordenados representando duas relações binárias em S .
Saída: Os pares ordenados da união e interseção das duas relações e os pares ordenados do complemento de cada relação.
3. *Entrada:* Os elementos de um conjunto finito S e uma lista de pares ordenados representando uma relação binária em S .
Saída: Mensagem que indique quais propriedades — reflexiva, simétrica, transitiva e/ou anti-simétrica — a relação goza.
4. *Entrada:* Os elementos de um conjunto finito S e uma lista de pares ordenados representando uma relação binária em S .
Saída: Fechos reflexivo, simétrico e transitivo da relação.

5. *Entrada:* Os elementos de um conjunto finito S e uma lista de pares ordenados representando uma ordenação parcial em S .
Saída: Uma lista de todos os elementos minimais e maximais
6. *Entrada:* Os elementos de um conjunto finito S e uma lista de pares ordenados representando uma ordenação parcial em S .
Saída: Uma lista com todos os elementos mínimos e máximos.
Perceba que esta tarefa é mais difícil que a do exercício anterior.
7. *Entrada:* Os elementos de um conjunto finito S , uma lista de pares ordenados representando uma relação de equivalência em S e um elemento x de S .
Saída: Os elementos de $[x]$.
8. *Entrada:* Representação matricial de tabelas de relações e dados de entrada apropriados para as operações select, project e join.
Saída: Representação matricial das tabelas de relações resultantes.
9. *Entrada:* Os elementos de um conjunto finito S e uma lista de pares ordenados representando uma ordenação parcial de S .
Saída: Sequência representando a ordenação total que resulta da ordenação topológica.
(Dica: Aproveite parte do código que você escreveu para o Exercício 5.)
10. *Entrada:* Os elementos de um conjunto finito S e do conjunto finito T e uma lista de pares ordenados representando uma relação binária em $S \times T$.
Saída: Uma indicação de se a relação é ou não uma função de S em T e, em caso afirmativo, se é injetiva, sobrejetiva ou ambos.
11. *Entrada:* O número de elementos em dois conjuntos finitos S e T .
Saída: O número de funções de S em T , o número de funções injetivas de S em T (ou uma indicação de que não existe nenhuma, se for o caso) e o número de funções sobrejetivas de S em T (ou uma indicação de que não existe nenhuma, se for o caso).
12. *Entrada:* Duas listas de pares ordenados representando as funções f e g de S em S .
Saída: Lista de pares ordenados que representem a função composta $g \circ f$.
13. *Entrada:* Elementos de um conjunto finito S e duas listas que representem (em forma cíclica) as permutações f e g em S .
Saída: Uma ou mais listas que representem a composição $g \circ f$ em forma cíclica ou na forma de composição de ciclos disjuntos.
14. *Entrada:* O número de elementos de um conjunto finito S .
Saída: O número de permutações sem pontos fixos de S .
15. *Entrada:* n e os elementos de duas matrizes $n \times n$ A e B .
Saída: A soma $A + B$ e os produtos $A \cdot B$ e $B \cdot A$.
16. *Entrada:* Dimensões da matriz A e seus elementos.
Saída: A^T (veja o Exercício 26 da Seção 4.4).
17. O **determinante** de uma matriz $n \times n$ é usado na solução de sistemas de equações lineares, bem como para outros fins. O determinante de A pode ser definido em termos de menores e cofatores. O **menor** do elemento a_{ij} é o determinante da matriz $(n - 1) \times (n - 1)$ obtida através da exclusão da linha i e coluna j da matriz A ; denotemos este menor por M_{ij} . O **cofator** do elemento a_{ij} denotado por C_{ij} é definido por

$$C_{ij} = (-1)^{i+j} M_{ij}$$

O determinante de A é determinado pela soma da multiplicação de todos os elementos em qualquer linha fixa de A por seus respectivos cofatores. Por exemplo, se usarmos a primeira linha, o determinante de A é dado por

$$\sum_{k=1}^n (a_{1k})(C_{1k})$$

Escreva um programa que, dados n e os elementos da matriz A de dimensões $n \times n$, calcule o determinante de A . Formule um algoritmo recursivo.

5

Grafos e Árvores

Objetivos do Capítulo

Após estudar este capítulo o leitor estará apto a:

- Entender e usar diversos termos relacionados a grafos, grafos direcionados e árvores
- Demonstrar que dois grafos dados são isomorfos ou fornecer uma razão que mostre que eles não o são
- Desenhar grafos completos e grafos bipartidos completos
- Usar a fórmula de Euler para um grafo simples, conexo e planar
- Entender a regra dos grafos específicos K_5 e $K_{3,3}$ com relação à planaridade de grafos
- Discutir o problema das quatro cores e seu histórico
- Demonstrar diversas propriedades elementares sobre grafos e árvores
- Perceber o uso de grafos, grafos direcionados e árvores como ferramentas de representação em um grande número de situações
- Usar as representações de grafos e grafos direcionados através de matrizes e listas de adjacências
- Usar representações por vetores e ponteiros de árvores binárias
- Converter matrizes de adjacências, relações de adjacências e grafos direcionados uns nos outros
- Usar a matriz de alcançabilidade de um grafo para determinar se um vértice é alcançável a partir de outro
- Computar a matriz de alcançabilidade de um grafo diretamente ou através do algoritmo de Warshall
- Usar árvores de decisão para representar os passos realizados em um

algoritmo de busca ou ordenação

- Expressar o número de comparações do pior caso de algoritmos de busca e ordenação em uma lista de n elementos
- Construir uma árvore binária de busca e realizar buscas em árvores deste tipo
- Encontrar o código de Huffman para caracteres cujas frequências de ocorrências sejam fornecidas

Um grafo é uma representação gráfica de elementos de dados e das conexões entre alguns destes itens. Uma árvore é um caso particular de grafo onde as conexões entre os elementos não são circulares. Um número surpreendente de situações do mundo real — diagramas de organizações, mapas rodoviários, redes de transporte e de comunicações e outras — pode ser representado como grafos ou árvores. Mais tarde veremos outros usos de grafos para representar redes lógicas, máquinas de estado finito e derivações em linguagens formais. A teoria dos grafos é um tópico extenso; a Seção 5.1 apresenta algumas terminologias referentes a esta teoria. Esta seção discute ainda o que significa para um grafo ser isomorfo, o conceito de planaridade e o problema de interesse histórico da coloração de grafos.

Para representarmos um grafo na memória de um computador, os dados precisam ser arrumados de um modo que preserve toda a informação contida na representação gráfica do grafo. Duas formas de representação de grafos em computadores, matrizes de adjacências e listas de adjacência serão vistas na Seção 5.2.

A Seção 5.3 relaciona grafos direcionados com relações binárias e a alcançabilidade em um grafo com o fecho transitivo de uma relação binária. São fornecidos dois algoritmos referentes à alcançabilidade.

As árvores de decisão são representações gráficas das atividades de certos tipos de algoritmos. Na Seção 5.4 apresentamos as árvores de decisão usadas para encontrar limites inferiores para o comportamento do pior caso de algoritmos de ordenação e busca. Além disso, nessa seção, apresentamos um algoritmo para construção de árvores binárias que permitem compressão de dados.

Seção 5.1 Terminologia e Aplicações de Grafos

Grafos

Uma forma de passar o tempo em uma viagem de avião é olhar os impressos nos bolsos das poltronas. Este material geralmente inclui um mapa com as rotas da companhia aérea que você está usando, tal como a mostrada na Fig. 5.1. Todas essas informações sobre as rotas podem ser expressas de forma verbal; por exemplo, existe uma rota direta entre Chicago e Nashville, mas não entre St. Louis e Nashville. No entanto a forma verbal será muito longa e confusa, e não será capaz de passar as informações tão rapidamente e de forma tão clara quanto o mapa. Existem muitos casos em que "uma figura vale por centenas de palavras".

Em inglês, o termo usado para designar um grafo é *graph* que também é comumente usado de forma coloquial para designar qualquer representação visual de dados (gráficos, em português), tais como a Fig. 5.1; outras formas comuns incluem o gráfico de barras, o gráfico de figuras e o gráfico de setores, que são mostradas na Fig. 5.2. Já vimos também gráficos de funções em sistemas de coordenadas retangulares. Adotaremos uma definição de grafo muito específica (veja abaixo) que não menciona nada sobre uma representação visual. No

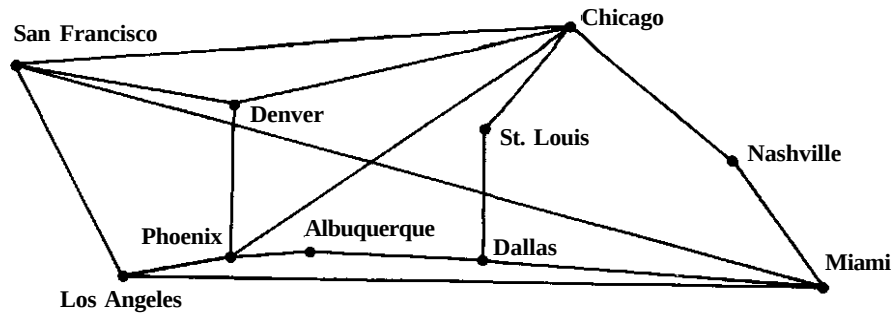


Figura 5.1

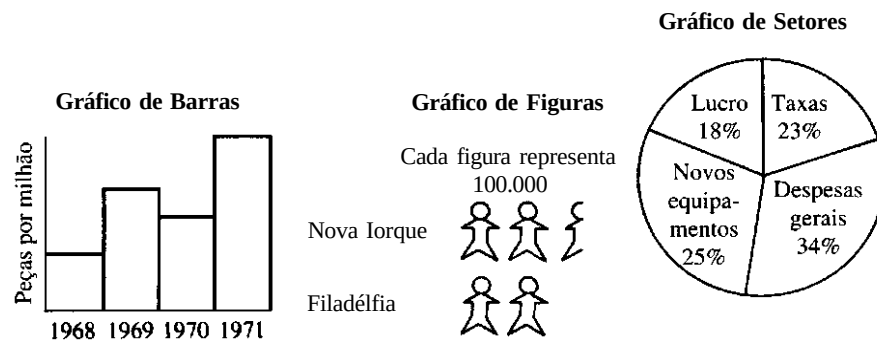


Figura 5.2

entanto, figuras como a mostrada na Fig. 5.1 são imediatamente entendidas como coerentes com nossa definição de grafo, de forma que também chamaremos este tipo de figuras de grafos.

Definição: Grafos, Vértices e Arestas

Um grafo é uma tripla ordenada (N, A, g) onde

N = um conjunto não-vazio de vértices (nós ou nodos)

A = um conjunto de arestas (arcos)

g = uma função que associa cada aresta a a um par não-ordenado x - y de vértices chamados de extremos de a

Nossos grafos terão sempre um número finito de vértices e de arestas.

EXEMPLO 1

O conjunto de vértices do mapa da empresa aérea da Fig. 5.1 é $\{\text{Chicago, Nashville, Miami, Dallas, St. Louis, Albuquerque, Phoenix, Denver, San Francisco, Los Angeles}\}$. Existem 16 arestas; por exemplo, Phoenix-Albuquerque (neste caso, denotamos as arestas pelos seus extremos), Albuquerque-Dallas etc.

EXEMPLO 2

No grafo da Fig. 5.3, temos cinco vértices e seis arestas. A função que associa as arestas aos seus extremos assume os seguintes valores: $g(a_1) = 1-2$, $g(a_2) = 1-2$, $g(a_3) = 2-2$, $g(a_4) = 2-3$, $g(a_5) = 1-3$ e $g(a_6) = 3-4$.

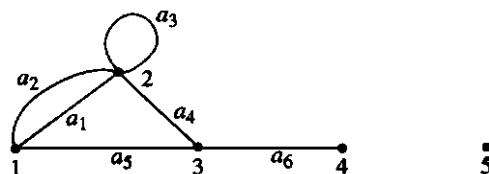


Figura 5.3

PRÁTICA 1

Trace um grafo que tenha os vértices $\{1, 2, 3, 4, 5\}$, as arestas $\{a_1, a_2, a_3, a_4, a_5, a_6\}$ e a função $g(a_1) = 1-2$, $g(a_2) = 1-3$, $g(a_3) = 3-4$, $g(a_4) = 3-4$, $g(a_5) = 4-5$ e $g(a_6) = 5-5$.

Precisaremos de alguma terminologia da teoria de grafos. Dois vértices em um grafo são ditos **adjacentes** se forem os extremos de uma mesma aresta. No grafo da Fig. 5.3, 1 e 3 são vértices adjacentes, mas 1 e 4 não são. O vértice 2 é adjacente a ele mesmo. Um **laço** em um grafo é uma aresta com extremos $n-n$ para algum nó n ; na Fig. 5.3, a aresta a_3 é um laço com extremos 2-2. Um grafo pode não conter laços, caso no qual é chamado de **sem laços**. Duas arestas que tenham os mesmos extremos são chamadas de **arestas paralelas**; as arestas a_1 e a_2 na Fig. 5.3 são paralelas. Um **grafo simples** é um grafo que não tenha arestas paralelas nem laços. Um **vértice isolado** não é adjacente a qualquer outro vértice; na Fig. 5.3, 5 é um vértice isolado. O **grau** de um vértice é o número de arestas que o tem como ponto extremo. Na Fig. 5.3, os vértices 1 e 3 têm grau 3, o vértice 2 tem grau 5, o vértice 4 tem grau 1 e o vértice 5 tem grau 0.

Como a função g , que relaciona cada aresta a seus extremos, é entendida como uma função propriamente dita, cada aresta tem um único par de pontos extremos. Se g for uma função injetiva, então haverá apenas uma aresta associada a cada par de vértices e o grafo não terá arestas paralelas. Um grafo **completo** é aquele no qual todos os vértices distintos são adjacentes. Neste caso, g é quase uma função sobrejetiva — todo par $x-y$ de vértices distintos está no conjunto imagem de g —, mas não há um laço em cada vértice, de forma que pares do tipo $x-x$ não devem ter imagem.

Um **subgrafo** de um grafo consiste em um conjunto de vértices e um conjunto de arestas que são subconjuntos dos conjuntos de vértices e arestas originais, respectivamente, nos quais os extremos de qualquer aresta precisam ser os mesmos que no grafo original. Em outras palavras, é um grafo obtido apagando-se parte do grafo original e deixando o restante sem alterações. A Fig. 5.4 mostra dois subgrafos do grafo da Fig. 5.3. Perceba que o grafo na Fig. 5.4a é simples e também completo.

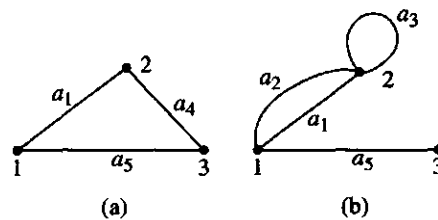


Figura 5.4

Um **caminho** de um vértice n_0 a um vértice n_k é uma seqüência

$$n_0, a_0, n_1, a_1, \dots, n_{k-1}, a_{k-1}, n_k$$

de vértices e arestas onde, para cada i , os extremos da aresta a_i são $n_i - n_{i+1}$. No grafo da Fig. 5.3, um caminho do vértice 2 ao vértice 4 consiste na seqüência 2, a_1 , 1, a_2 , 2, a_4 , 3, a_6 , 4. O **comprimento** de um caminho é o número de arestas que ele contém; se uma aresta for usada mais de uma vez, ela deve ser contada tantas vezes quantas for usada. O comprimento do caminho que acabamos de descrever entre os nós 2 e 4 é 4. Um grafo é dito **conexo** se houver um caminho entre quaisquer dois vértices. Ambos os grafos da Fig. 5.4 são conexos, mas o grafo da Fig. 5.3 não é. Um **ciclo** em um grafo é um caminho de algum vértice n_0 até n_0 de novo de forma que nenhum vértice ocorra mais de uma vez no caminho, n_0 é o único vértice que ocorre mais de uma vez e este ocorre apenas nos extremos do caminho. (Os vértices e as arestas podem repetir-se em um caminho, mas não em um ciclo — exceto pelo vértice n_0 .) No grafo da Fig. 5.3,

$$1, a_1, 2, a_4, 3, a_5, 1$$

é um ciclo. Um grafo sem ciclos é dito **acíclico**.

PRÁTICA 2

Com relação ao grafo obtido na Prática 1,

- Encontre dois vértices que não sejam adjacentes.
- Encontre um vértice que seja adjacente a ele mesmo.
- Encontre um laço.
- Encontre duas arestas paralelas.
- Encontre o grau do vértice 3.
- Encontre um caminho de comprimento 5.

- g. Encontre um ciclo.
- h. Este grafo é completo?
- i. Este grafo é conexo?

Podemos demonstrar alguns fatos (bem triviais) sobre grafos diretamente a partir das definições acima.

EXEMPLO 3 Demonstre que todo gráfico acíclico é simples.

Usaremos uma prova por contraposição. Se um grafo não for simples, ele tem arestas paralelas ou um laço. As arestas paralelas e seus extremos, ou o laço e seu extremo constituem um ciclo e, portanto, o grafo não é acíclico.

Perceba que a recíproca do enunciado do Exercício 3 não é verdadeira: a Fig. 5.4 é um grafo simples que contém um ciclo.

PRÁTICA 3

- a. Demonstre que todo grafo completo é conexo.
- b. Encontre um grafo conexo que não seja completo.

Grafos Isomorfos

Dois grafos podem parecer muito diferentes em suas representações gráficas, mas serem, ainda assim, o mesmo grafo de acordo com nossa definição. Os grafos nas Figs. 5.5 e 5.6 são os mesmos—eles têm os mesmos vértices, as mesmas arestas e a mesma função de associação de arestas e seus extremos. (Na representação de um grafo, as arestas podem interceptar-se em pontos que não sejam vértices do grafo.) O grafo da Fig. 5.7 também é o mesmo grafo. Se mudarmos os rótulos dos vértices e arestas do grafo da Fig. 5.5 segundo a correspondência a seguir, os grafos se tornarão os mesmos:

$$\begin{array}{ll} f_1: 1 \rightarrow a & f_2: a_1 \rightarrow e_2 \\ & 2 \rightarrow c \\ & 3 \rightarrow b \\ & 4 \rightarrow d \end{array}$$

As estruturas que são iguais a menos de um novo rotulamento são chamadas de *isomorfas*. Para mostrar que duas estruturas são isomorfas, precisamos produzir novos rótulos (uma aplicação [função] injetiva e sobrejetiva entre os elementos de ambas as estruturas) e então mostrar que as propriedades importantes das estruturas são "preservadas" (mantidas) pelo novo rotulamento. No caso de grafos, os elementos são vértices e arestas. A "propriedade importante" em um grafo é a lista de quais arestas conectam quais vértices.

As aplicações f_1 e f_2 descritas acima são funções injetivas e sobrejetivas dos vértices e arestas do grafo da Fig. 5.5 nos vértices e arestas do grafo da Fig. 5.7. Além disso, se uma aresta a no grafo da Fig. 5.5 tem extremos $x-y$, então a aresta $f_2(a)$ no grafo da Fig. 5.7 tem extremos $f_1(x) - f_1(y)$ e vice-versa. Por exemplo, a aresta a , na Fig. 5.5 tem extremos $1-3$, enquanto que a aresta correspondente a_2 na Fig. 5.7 tem extremos $a-b$, que são os vértices na Fig. 5.7 que correspondem aos vértices 1 e 3 na Fig. 5.5. Podemos formalizar, então, esta idéia.

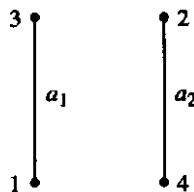


Figura 5.5

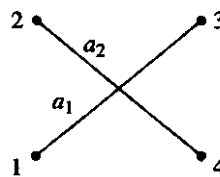


Figura 5.6

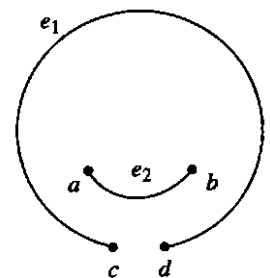


Figura 5.7

Definição: Grafos Isomorfos

Dois grafos (N_1, A_1, g_1) e (N_2, A_2, g_2) são **isomorfos** se existirem bijeções $f_1: N_1 \rightarrow N_2$ e $f_2: A_1 \rightarrow A_2$ tais que para cada aresta $a \in A_1$, $g_1(a) = x-y$ se, e somente se, $g_2[f_2(a)] = f_1(x) - f_1(y)$.

EXEMPLO 4 Os grafos mostrados na Fig. 5.8 são isomorfos. As bijeções que estabelecem o isomorfismo são parcialmente dadas abaixo:

$$\begin{array}{ll} f_1: 1 \rightarrow c & f_2: a_1 \rightarrow e_1 \\ 2 \rightarrow e & a_2 \rightarrow e_4 \\ 3 \rightarrow d & a_3 \rightarrow e_2 \\ 4 \rightarrow b & \vdots \\ 5 \rightarrow a & \end{array}$$

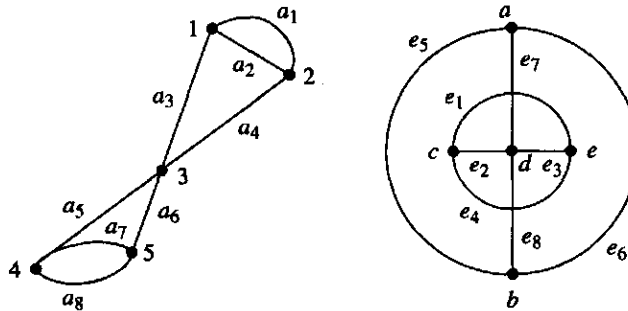


Figura 5.8

Neste caso $g_1(a_3) = 1-3$ e $g_2[f_2(a_3)] = g_2(e_2) = c-d = f_1(1) - f_1(3)$. Isto mostra que a relação entre arestas e seus vértices extremos é preservada pelo novo rotulamento no caso da aresta a_3 . Para provarmos que os grafos são isomorfos, precisamos completar a definição da função f_2 , e então mostrar que a relação entre arestas e seus extremos é preservada para todos os casos possíveis.

PRÁTICA 4 Complete a definição da função f_2 no Exemplo 4.

O isomorfismo de grafos é fácil de estabelecer no caso de examinarmos grafos simples. Se pudermos encontrar uma função/ , apropriada que leve vértices em vértices, então uma função f_2 que leve arestas em arestas é trivial porque existe no máximo uma aresta entre cada par de vértices. Portanto, vale o seguinte teorema:

Teorema sobre Isomorfismo de Grafos Simples

Dois grafos simples (N_1, A_1, g_1) e (N_2, A_2, g_2) são **isomorfos** se houver uma bijeção $f: N_1 \rightarrow N_2$ tal que para quaisquer vértices n_i e n_j de N_1 , n_i e n_j são adjacentes se, e somente se, $f(n_i)$ e $f(n_j)$ são adjacentes. (A função f é chamada de **isomorfismo** do grafo 1 no grafo 2).

PRÁTICA 5 Encontre um isomorfismo do grafo da Fig. 5.9a no grafo da Fig. 5.9b.

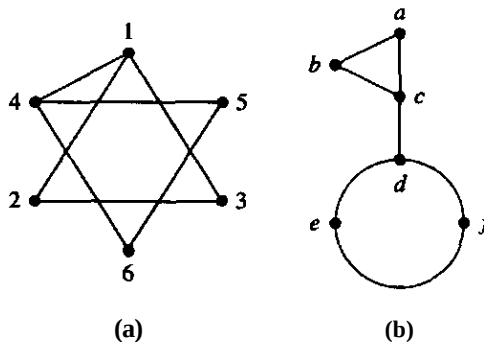


Figura 5.9

Determinar que dois grafos são isomorfos requer que encontremos a bijeção (ou, para grafos não-simples, as bijeções) e então mostremos que a propriedade da adjacência (ou relação entre arestas e seus extremos) é preservada. Para mostrar que dois grafos não são isomorfos, precisamos mostrar que a(s) bijeção(ões) necessária(s) não existe(m). Poderíamos tentar todas as bijeções possíveis (já que há um número finito de vértices e arestas, existe um número finito de bijeções). No entanto, este método se tornaria rapidamente inviável em grafos de qualquer tamanho razoável. Ao invés disso, podemos procurar outras razões pelas quais tais bijeções não possam existir. Apesar de não ser uma tarefa simples em todos os casos, existem certas condições sob as quais se torna fácil ver que dois grafos não são isomorfos (veja o Exercício 15). Essas condições incluem:

1. Um grafo tem mais vértices que o outro.
2. Um grafo tem mais arestas que o outro.
3. Um grafo tem arestas paralelas e o outro não.
4. Um grafo tem um laço e o outro não.
5. Um grafo tem um vértice de grau k e o outro não.
6. Um grafo é conexo e o outro não.
7. Um grafo tem um ciclo e o outro não.

PRÁTICA 6

Demonstre que os dois grafos da Fig. 5.10 não são isomorfos:

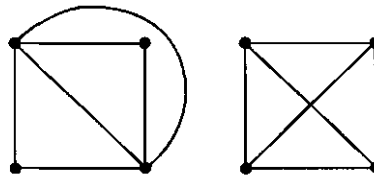
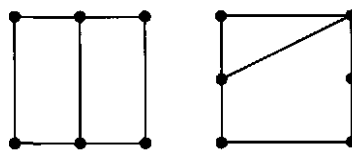


Figura 5.10

EXEMPLO 5

Os dois grafos da Fig. 5.11 não são isomorfos. Perceba que cada grafo tem seis vértices e sete arestas. Nenhum dos dois tem arestas paralelas ou laços. Ambos são conexos. Ambos têm três ciclos, quatro vértices de grau 2 e dois vértices de grau 3. Portanto, nenhum dos testes óbvios de isomorfismo se aplica. No entanto, o grafo na Fig. 5.11b tem um vértice de grau 2 que é adjacente a dois vértices de grau 3; o que não ocorre na Fig. 5.11a, e, portanto, os grafos não são isomorfos.



(a)

(b)

Figura 5.11

Mais uma vez: grafos isomorfos são considerados "os mesmos", exceto por diferenças cosméticas na forma como são desenhados ou rotulados, enquanto que grafos não-isomorfos têm diferenças estruturais.

Outras Propriedades de Grafos

Nesta seção nos dedicaremos aos grafos simples e a algumas propriedades que eles têm.

EXEMPLO 6

A Fig. 5.12 mostra os grafos simples completos com 1, 2, 3 e 4 vértices. Estes grafos são denotados por K_n .

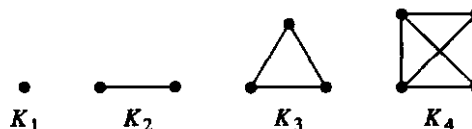


Figura 5.12

PRÁTICA 7

Desenhe K_5 .

•

Considere agora o grafo simples da Fig. 5.13. Ele não é um grafo completo porque nem todo vértice é adjacente a todos os demais vértices. No entanto, os vértices podem ser divididos em dois conjuntos disjuntos, $\{1,2\}$ e $\{3,4,5\}$ de tal forma que quaisquer dois vértices tomados no mesmo conjunto não são adjacentes, mas todos dois vértices escolhidos em conjuntos diferentes são adjacentes. Grafos deste tipo são chamados de *grafos bipartidos completos* ou *grafos bipartites completos*.

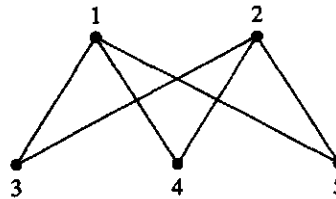


Figura 5.13

Definição: Grafo Bipartido Completo

Um grafo é um **grafo bipartido completo** (ou **grafo bipartite completo**) se seus vértices podem ser particionados em dois conjuntos não-vazios N_1 e N_2 tais que dois vértices x e y sejam adjacentes se, e somente se, $x \in N_1$ e $y \in N_2$. Se $|N_1| = m$ e $|N_2| = n$, este grafo é denotado por $K_{m,n}$.

A Fig. 5.13 mostra, portanto, $K_{2,3}$.

PRÁTICA 8

Desenhe $K_{3,3}$.

•

Planaridade

Um **grafo planar** é um grafo que pode ser desenhado (em uma folha de papel, isto é, em um plano) de forma que suas arestas se interceptem apenas em vértices. O grafo da Fig. 5.5 é notoriamente planar. No entanto, sabemos que ele é isomorfo ao grafo da Fig. 5.6, portanto este também é planar. A palavra-chave na definição de grafo planar é que ele *pode* ser desenhado de uma certa maneira.

PRÁTICA 9

Demonstre que K_4 é um grafo planar.

•

EXEMPLO 7

Considere K_5 , o grafo completo simples com cinco vértices. Podemos construir K_5 sem que suas arestas se interceptem, começando apenas com algumas arestas e incluindo, dentro do possível, novas arestas que não se cruzem com as já incluídas. Começaremos com cinco vértices ligados como mostrado na Fig. 5.14a. (Como todos os vértices de K_n são simétricos, não faz diferença a forma que os rotulamos.) Em seguida, conectamos 1 ao 3 e 1 ao 4, como mostrado na Fig. 5.14b. Agora o vértice 2 precisa ser ligado ao 4 e ao 5. Isto pode ser feito sem violar a planaridade do grafo desenhando essas arestas pelo lado de fora do grafo, como na Fig. 5.14c. A aresta que falta é entre os vértices 3 e 5. Mas não há maneira de traçar esta aresta sem cruzar a aresta 2-4 ou outras arestas do interior do grafo, tais como a 1-4.

Nós tomamos uma decisão relativa a onde desenhar as arestas 1-3 e 1-4; desenhamo-las no interior do grafo. Poderíamos verificar se desenhar essas arestas do lado externo traria alguma diferença, mas veríamos que não causa qualquer diferença (veja a Prática 10). Portanto, parece que K_5 não é um grafo planar. No entanto, nós

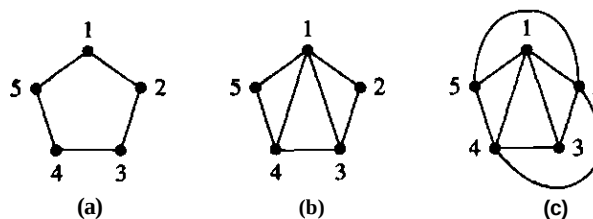


Figura 5.14

ainda desejamos uma demonstração deste fato com uma base mais sólida — até agora isto se parece muito com um argumento do tipo "eu não sei fazer isto, logo isto não pode ser feito". •

PRÁTICA 10 Mostre que se colocássemos as arestas 1-3 e 1-4 como exteriores ao construir K_5 seríamos levados, novamente, a uma situação em que as arestas se interceptassem. •

PRÁTICA 11 Apresente um argumento baseado em construção de que $K_{3,3}$ não é um grafo planar. •

Um fato sobre grafos planares foi descoberto pelo matemático suíço do século dezoito Leonhard Euler (pronuncia-se "óiler"). Um grafo simples, conexo e planar (quando desenhado em sua representação planar, sem interseção de arestas) divide o plano em um número de regiões, incluindo as regiões totalmente fechadas e uma região infinita exterior. Euler observou uma relação entre o número n de vértices, o número a de arestas e o número r de regiões neste tipo de grafos. Esta relação é denominada de **fórmula de Euler**:

$$n - a + r = 2 \quad (1)$$

PRÁTICA 12 Verifique a fórmula de Euler para o grafo conexo, planar e simples da Fig. 5.9b. •

Para demonstrarmos a fórmula de Euler, faremos uma prova por indução em a , o número de arestas. No caso básico, $a = 0$ e o grafo consiste em um único vértice; a única região é a região exterior (Fig. 5.15a). Neste caso, $n = 1$, $a = 0$ e $r = 1$ e a equação (1) se verifica. Assumimos agora que a fórmula se verifica para a representação planar de qualquer grafo conexo, planar e simples com k arestas, e consideremos o grafo com $k + 1$ arestas. Como de costume, devemos relacionar o "caso $k + 1$ " com o "caso k " de forma que possamos fazer uso da hipótese de indução. Consideramos dois casos para o grafo com $k + 1$ arestas.

Caso 1. O grafo tem um vértice de grau 1. Excluimos temporariamente este vértice e a aresta que o conecta (Fig. 5.15b); isto nos deixa com um grafo conexo, planar e simples com k arestas, algum número n de vértices e algum número r de regiões para os quais (pela hipótese de indução)

$$n - k + r = 2$$

No grafo original, existe uma aresta a mais e um vértice a mais, porém o mesmo número de regiões, de modo que a fórmula correspondente é

$$(n + 1) - (k + 1) + r = 2$$

que, pela hipótese de indução, é verdadeira.

Caso 2. O grafo não tem vértices de grau 1. Então apagamos, temporariamente, uma aresta que ajude a definir uma região fechada (Fig. 5.15c). (Se nenhuma aresta participa da delimitação de uma região fechada, então o grafo é uma cadeia e tem um vértice de grau 1.) Isto resulta um grafo conexo, planar e simples com k arestas, algum número n de vértices e algum número r de regiões para os quais (pela hipótese de indução)

$$n - k + r = 2$$

No grafo original, existe uma aresta a mais e uma região a mais, porém o mesmo número de vértices, de maneira

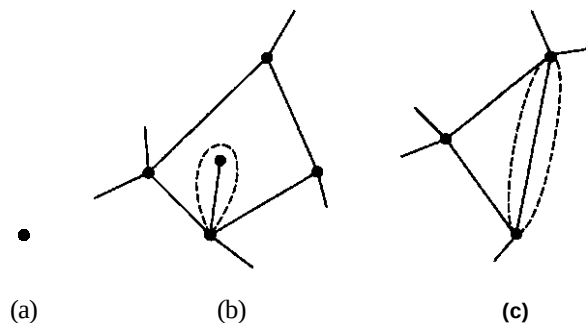


Figura 5.15

que a fórmula correspondente é

$$n - (k + 1) + (r + 1) = 2$$

que, pela hipótese de indução, é verdadeira.

PRÁTICA 13

Explique, na demonstração da fórmula de Euler, por que no caso 2 a aresta apagada precisa fazer parte da definição de uma região fechada. Forneça duas razões. •

Existem duas consequências da fórmula de Euler, se incluirmos mais restrições no grafo. Suponha que precisemos que o grafo não seja apenas simples, conexo e planar, mas tenha, pelo menos, três vértices. Em uma representação planar deste grafo, podemos contar o número de arestas que são adjacentes a (formam fronteira com) cada região, incluindo a região exterior. Arestas que estão totalmente dentro de uma região contribuem com duas arestas para esta região; por exemplo, se traçarmos a fronteira da região interior mostrada na Fig. 5.15b, passamos por seis arestas, incluindo a aresta que leva ao vértice de grau 1 e o caminho de volta. Arestas que separam duas regiões contribuem com uma aresta para cada região. Portanto, se houver a arestas no grafo, o número de arestas de regiões é $2a$.

Não há regiões com apenas uma aresta adjacente, porque não há laços no grafo. Não há regiões com exatamente duas arestas adjacentes porque não há arestas paralelas e o grafo consistindo apenas em uma aresta ligando dois vértices (que tem duas arestas adjacentes à região exterior) foi excluído. Portanto, cada região tem pelo menos três arestas adjacentes, de forma que $3r$ é o número mínimo de arestas de região. Por isso,

$$2a \geq 3r$$

ou, pela equação (1),

$$2a \geq 3(2 - n + a) = 6 - 3n + 3a$$

e, finalmente,

$$a \leq 3n - 6 \quad (2)$$

Se impusermos uma última restrição de que não haja ciclos de comprimento 3, cada região terá pelo menos quatro arestas adjacentes, portanto $4r$ será o número mínimo de arestas de região. Isto nos leva à desigualdade

$$2a \geq 4r$$

que pode ser escrita como

$$a \leq 2n - 4 \quad (3)$$

Esses resultados são resumidos pelo teorema a seguir:

Teorema sobre o Número de Vértices e Arestas

Para um grafo conexo, simples e planar com n vértices e a arestas:

1. Se a representação planar divide o plano em r regiões, então

$$n - a + r = 2 \quad (1)$$

2. Se $n \geq 3$, então

$$a \leq 3n - 6 \quad (2)$$

3. Se $n \geq 3$ e não existem ciclos de comprimento 3, então

$$a \leq 2n - 4 \quad (3)$$

Perceba que a desigualdade (3) determina um limite superior mais rígido sobre o número de arestas que a desigualdade (2), mas impõe uma condição extra ao grafo.

Podemos usar este teorema para demonstrar que certos grafos não são planares.

EXEMPLO 8 K_5 é um grafo conexo simples com cinco vértices (e 10 arestas). Se ele fosse um grafo planar, a desigualdade (2) do nosso teorema deveria ser verificada, mas $10 < 3(5) - 6$. Portanto, como nosso argumento de construção tinha indicado, K_5 não é planar. $K_{3,3}$ é um grafo conexo simples com seis vértices (e nove arestas). Não possui ciclos de comprimento 3, uma vez que isto exigiria que dois vértices em um dos subconjuntos fossem adjacentes. Se fosse um grafo planar, a desigualdade (3) deveria se verificar, mas $9 < 2(6) - 4$. Portanto, $K_{3,3}$ não é planar.

PRÁTICA 14 Mostre que a desigualdade (2) se verifica para $K_{3,3}$, o que ilustra que esta desigualdade é necessária, mas não suficiente para a planaridade de um grafo com $n \geq 3$.

Os grafos não-planares K_5 e $K_{3,3}$ ilustram uma regra básica de todos os grafos não-planares. Para enunciarmos esta regra, precisamos de mais uma definição.

Definição: Grafos Homeomorfos

Dois grafos são homeomorfos se ambos puderem ser obtidos do mesmo grafo por uma sequência de subdivisões elementares, nas quais uma única aresta $x-y$ é substituída por duas novas arestas $x-v$ e $v-y$ que se conectam a um novo vértice v .

EXEMPLO 9 Os grafos das partes (b) e (c) da Fig. 5.16 são homeomorfos porque podem ser obtidos do grafo da Fig. 5.16a através de uma sequência de subdivisões elementares. (No entanto, um não pode ser obtido do outro através de subdivisões elementares.)

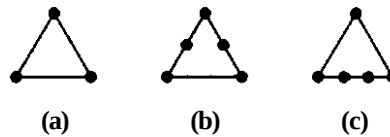


Figura 5.16

EXEMPLO 10 Um grafo planar não pode ser transformado em um grafo não-planar através de subdivisões elementares (veja o Exercício 26). Como um resultado, grafos homeomorfos são ambos planares ou não-planares. O teorema a seguir, atribuído ao matemático polonês Kuratowski, caracteriza grafos não-planares.

Teorema de Kuratowski

Um grafo é não-planar se, e somente se, contém um subgrafo homeomorfo a K_5 ou $K_{3,3}$.

Não vamos demonstrar este teorema, apesar de uma direção ser fácil de se verificar. Se um grafo tem um subgrafo homeomorfo a um grafo não-planar K_5 ou $K_{3,3}$, então, o subgrafo — e portanto todo o grafo — é não-planar.

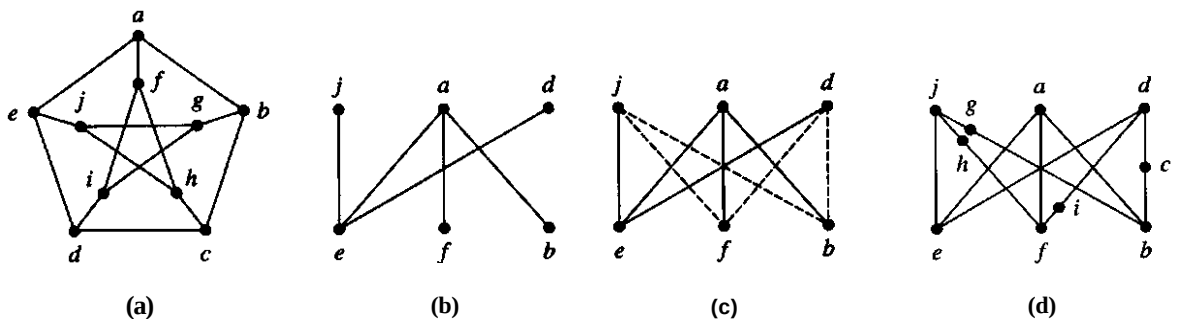


Figura 5.17

A Fig. 5.17a mostra o "grafo de Petersen". Devemos demonstrar que este grafo não é planar, encontrando um subgrafo homeomorfo a $K_{3,3}$. Se olharmos na parte superior do grafo, podemos ver que o vértice a é adjacente aos vértices e, f e b e nenhum destes são adjacentes uns aos outros. Além disso, o vértice e é adjacente aos vértices d e j , bem como ao vértice a e os vértices a, d e j não são adjacentes uns aos outros. Esta informação é transcrita no grafo da Fig. 5.17b, que é um subgrafo de $K_{3,3}$. As arestas necessárias para completar $K_{3,3}$ são mostradas como linhas tracejadas na Fig. 5.17c. Essas arestas não pertencem ao grafo de Petersen; por exemplo, não há a aresta $j-f$. No entanto, existe um caminho no grafo de Petersen que de j a f usa o vértice intermediário h , isto é, $j-h$ e $h-f$. Analogamente, existem caminhos $j-g$ e $g-b$, $d-i$ e $i-f$ e $d-c$ e $c-b$. A inclusão desses caminhos à Fig. 5.17b resulta na Fig. 5.17d, que é um subgrafo do grafo de Petersen e também pode ser obtida através de uma sequência de subdivisões elementares da Fig. 5.17c.

Coloração

Um dos problemas mais famosos dos tempos modernos é o **problema das quatro cores**. Ele é famoso por ser fácil de ser entendido, apesar de sua solução ter fugido aos matemáticos por mais de 100 anos. Suponha que um mapa de vários países desenhado em uma folha de papel precisa ser pintado de forma que dois países vizinhos não possam ter a mesma cor. (Não consideramos países que se encontram apenas em um ponto, e levamos em conta apenas países "conexos".) Qual o número mínimo de cores necessário para pintar qualquer mapa?

EXEMPLO 11 No mapa da Fig. 5.18 está claro que são necessárias três cores e não são necessárias mais do que três cores para colorir o mapa.

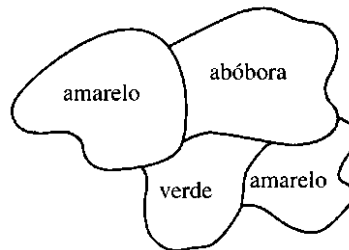


Figura 5.18

PRÁTICA 15 Desenhe um mapa que precise de quatro cores para ser colorido.

É fácil de ver que pelo menos quatro cores são necessárias para resolver o problema geral da coloração de mapas (veja a Prática 15). Como ninguém conseguiu desenhar um mapa que precisasse mais do que quatro cores para ser colorido, foi formulada a conjectura de que quatro cores seriam sempre o suficiente. Esta conjectura tornou-se conhecida como o problema das quatro cores. Ela foi inicialmente proposta para o matemático Augustus De Morgan por um de seus alunos, em 1852. De Morgan escreveu sobre o problema para William Rowan Hamilton, outro matemático famoso sobre quem ouviremos falar mais no capítulo seguinte. O problema não foi discutido com profundidade até 1878, quando Artur Cayley, um dos matemáticos mais conhecidos de seu tempo, admitiu num encontro da London Mathematical Society que era incapaz de resolver o problema. Um ano mais tarde, A. B. Kempe, um advogado, publicou uma "demonstração" do problema das quatro cores que levou 10 anos até que se percebesse que a demonstração apenas garantia que cinco cores eram suficientes.

Após o estabelecimento do teorema das cinco cores, continuaram a aparecer "demonstrações" do problema das quatro cores que eram posteriormente mostradas como incorretas. Muita energia e grandes cabeças dedicaram-se ao problema sem sucesso. Em 1976, dois matemáticos da Universidade de Illinois, Wolfgang Haken e Kenneth Appel, anunciaram uma demonstração do problema das quatro cores. A demonstração deles usou as idéias originalmente introduzidas por Kempe em 1879 e corrigiu a falha de seu argumento.

Kempe definiu um *mapa normal* como um mapa onde não podem haver mais do que três regiões se encontrando em um ponto, e nenhuma região inclui outra. Ele então mostrou que se houver um mapa que requer cinco cores, então deve haver um mapa normal que tenha o menor número de regiões dos mapas que requerem cinco cores para serem coloridos, que é chamado de um mapa minimal cinco-cromático. Ele mostrou que todo mapa contém um país com cinco ou menos países adjacentes. Isto fornece um pequeno conjunto de configurações inevitáveis, isto é, de arranjos de regiões, pelo menos uma das quais deve ocorrer em todo mapa

normal. Kempe assumiu, então, a existência de um mapa cinco-cromático normal (que ocorreria se existisse qualquer mapa que requer cinco cores para sua coloração). No entanto, sua demonstração falhou ao demonstrar que qualquer configuração inevitável deste mapa leva a uma contradição. Haken e Appel, usando um computador, construíram finalmente um conjunto de 1482 configurações inevitáveis, e mostraram que qualquer delas que faça parte de um mapa cinco-cromático normal leva a uma contradição. Portanto, nenhum mapa requer cinco cores.

Teorema das Quatro Cores

Quatro cores são suficientes para colorir qualquer mapa no plano.

O problema das quatro cores é o caso de um problema mais geral na teoria dos grafos. Para transformar o mapa em um grafo, criamos o grafo **dual** do mapa. Colocamos um vértice em cada região do mapa, e uma aresta entre dois vértices que representem países adjacentes. Então o problema de coloração do mapa torna-se o problema de coloração de vértices de um grafo dual de forma que não haja dois vértices adjacentes que tenham a mesma cor.

EXEMPLO 12 A Fig. 5.19 mostra o grafo dual do mapa da Fig. 5.18.

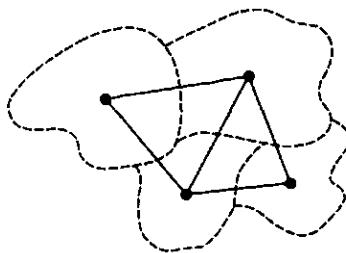


Figura 5.19

Definições: Coloração e Número Cromático

Uma **coloração** (de vértices) de um grafo é a atribuição de uma cor a cada vértice do grafo de tal forma que dois vértices adjacentes não tenham a mesma cor. O **número cromático** do grafo é o menor número de cores necessárias para se obter uma coloração.

O grafo dual de um mapa, da maneira que é construído, será sempre simples, conexo e planar. Além disso, qualquer grafo simples, conexo e planar pode ser entendido como o grafo dual de um mapa. O teorema das quatro cores estabelece, então, que o número cromático de qualquer grafo simples, conexo e planar é no máximo 4.

Ao contrário do problema das quatro cores, uma demonstração do teorema das cinco cores não é difícil, e constitui um caso interessante de prova por indução completa. Antes de ver esta demonstração, provaremos outro pequeno resultado que será necessário. Estas demonstrações de apoio são chamadas de *lemas*; os lemas são procedimentos intermediários que, uma vez demonstrados, podem ser usados quando necessários, sem a necessidade de se interromper o raciocínio principal com detalhes irrelevantes.

Lema

Sem um grafo simples, conexo e planar com três ou mais vértices, existe pelo menos um vértice com grau menor ou igual a 5.

Demonstração: Se este resultado não fosse verdadeiro, então todo vértice em um grafo do tipo acima teria grau maior que 5, isto é, grau 6 ou maior. O número total de extremos de arestas em um grafo seria pelo menos $6n$, onde n é o número de vértices. Mas o número de extremos de arestas é exatamente o dobro do número a de

arestas, portanto

$$6n \leq 2a$$

Pela desigualdade (2) do teorema do número de vértices e arestas, $a \leq 3n - 6$ ou $2a \leq 6n - 12$. Combinando essas desigualdades, obtemos

$$6n \leq 6n - 12$$

que é uma contradição.

Teorema das Cinco Cores

O número cromático de um grafo simples, conexo e planar é no máximo 5.

Esboçaremos uma demonstração de indução matemática no número de vértices do grafo. Como passo básico para o processo de indução, é claro que cinco cores são suficientes para o caso do número de vértices ser menor ou igual a 5. Assumamos agora que qualquer grafo simples, conexo e planar com $\leq k$ vértices pode ser colorido com cinco cores, e consideremos um grafo com as mesmas características com $k+1$ vértices. Podemos assumir que $k+1$ é pelo menos 6, pois os casos de cinco ou menos vértices já foram tratados. Pelo nosso lema, pelo menos um vértice v do grafo deve ter grau menor ou igual a 5; se removermos temporariamente v (e as arestas adjacentes a ele) do grafo, restará um ou mais grafos simples, conexos e planares, cada qual com no máximo k vértices (Figs. 5.20a e 5.20b). Pela hipótese de indução, cada subgrafo tem uma coloração com no máximo cinco cores (usamos o mesmo elenco de cores para colorir cada subgrafo). Voltemos ao grafo original. Se v tem grau menor que 5 ou se, no caso contrário, os cinco vértices adjacentes a v não usam cinco cores diferentes, existe uma quinta cor disponível para ser usada em v . Portanto, assumimos que v é adjacente a cinco vértices v_1, v_2, v_3, v_4 e v_5 , dispostos em sentido horário em torno de v e coloridos, respectivamente, com as cores 1, 2, 3, 4 e 5 (Fig. 5.20c).

Tomemos agora todos os vértices do grafo que estejam coloridos com as cores 1 ou 3. Suponha que não exista caminho, usando estes vértices, entre n_1 e n_3 . Então, ao considerarmos os vértices com cores 1 e 3, existem duas componentes disjuntas do grafo, uma contendo n_1 e outra contendo n_3 . Na seção que contém n_1 , trocamos as cores 1 e 3 de todos os vértices. Isto não viola a coloração (própria) dos subgrafos, ela colore n_1 com a cor 3, e mantém a cor 1 para ser usada em v .

Suponha agora que exista um caminho entre n_1 e n_3 usando apenas os vértices de cores 1 e 3. Neste caso, consideramos todos os vértices do restante do grafo que tenham cores 2 e 4. Será que existe um caminho entre n_2 e n_4 a que use apenas estes vértices? A resposta é não. Isto porque, devido à arrumação dos vértices n_1, n_2, n_3, n_4 e n_5 , este caminho deveria cruzar o caminho que conecta n_1 e n_3 . Como o grafo é planar, estes dois caminhos teriam que se encontrar em um vértice, que teria que ter as cores 1 ou 3 atribuídas em função do caminho $n_1 - n_3$ e as cores 2 ou 4 atribuídas em função do caminho $n_2 - n_4$, uma impossibilidade. Portanto, não há caminho que use apenas os vértices coloridos com 2 ou 4 entre n_2 e n_4 e podemos rearrumar as cores como no caso anterior. Isto conclui a demonstração.

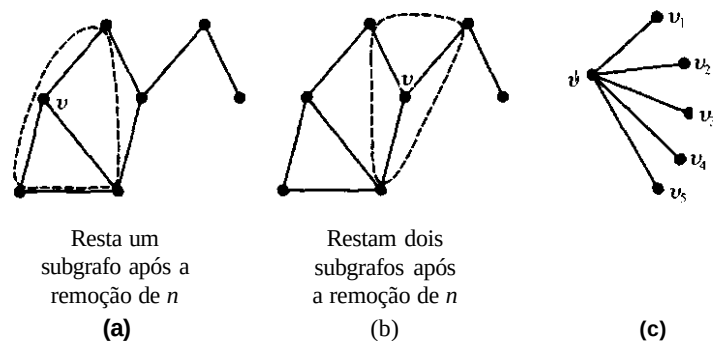


Figura 5.20

PRÁTICA 16

A Fig. 5.21 mostra um grafo parcialmente colorido. Precisamos atribuir uma cor ao vértice n . Use a demonstração do teorema das cinco cores para realizar a coloração do grafo.

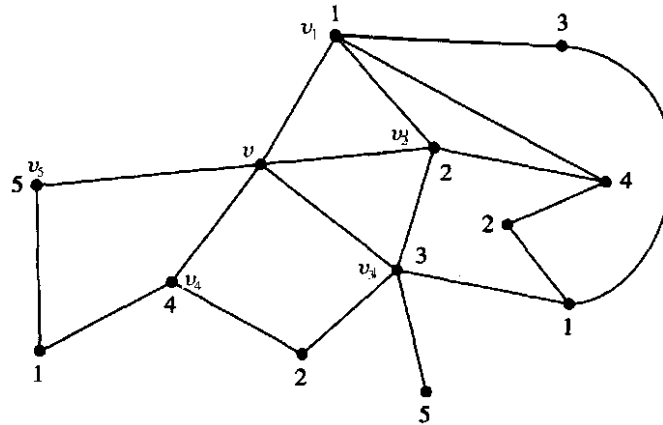


Figura 5.21

O teorema das quatro cores aplica-se apenas a grafos planares. O número cromático de K_5 , por exemplo, que não é planar, é 5 (veja o Exercício 41).

Árvores

Uma **árvore** é um grafo acíclico e conexo com um nó designado como a **raiz** da árvore. As Figs. 5.22a e 5.22b mostram duas árvores. Curiosamente, os cientistas da computação preferem desenhar as árvores com a raiz no topo. Um grafo acíclico e conexo sem a designação de um vértice como raiz é chamado de **árvore não-enraizada**. (Perceba que diversos livros definem árvores como grafos acíclicos e conexos, e então os chamam de "árvores enraizadas" quando têm um vértice raiz designado.)

Uma árvore pode ser construída recursivamente. Um único vértice é uma árvore (este vértice é a raiz). Se $T_1, T_2, \dots, T_t$ são árvores disjuntas com raízes $r_1, r_2, \dots, r_t$, o grafo formado pela ligação de um novo vértice r , por uma única aresta a cada uma dos vértices $r_1, r_2, \dots, r_t$ constitui uma árvore de raiz r . Os vértices $r_1, r_2, \dots, r_t$ são **filhos** de r , e r é **pai** de $r_1, r_2, \dots, r_t$. A Fig. 5.22c mostra o passo final da construção recursiva da árvore da Fig. 5.22b.

Como uma árvore é um grafo conexo, existe um caminho entre a raiz e todos os vértices da árvore; como a árvore é acíclica, este caminho é único. A **profundidade de um vértice** em uma árvore é o comprimento do caminho da raiz até o vértice; em particular, a raiz tem profundidade 0. A **altura (profundidade) da árvore** é a maior profundidade de todos seus vértices; em outras palavras, é o comprimento do maior caminho entre a raiz e um vértice. Um vértice sem filhos é chamado de **folha**; os vértices que não são folhas são chamados de **vértices internos** ou **nós internos**. Uma **floresta** é qualquer grafo acíclico (não necessariamente conexo); portanto, uma floresta é uma coleção de árvores disjuntas. As Figs. 5.22a e 5.22b juntas formam uma floresta.

Árvores binárias, onde cada nó tem no máximo dois filhos, constituem um caso de particular interesse. Em uma árvore binária, cada filho é designado como o **filho à esquerda** ou o **filho à direita** deste nó. Uma **árvore binária completa** é aquela em que todos os nós internos têm dois filhos e todas as folhas têm a mesma profundidade. A Fig. 5.23 mostra uma árvore binária de altura 4 e a Fig. 5.24 mostra uma árvore binária completa de altura 3.

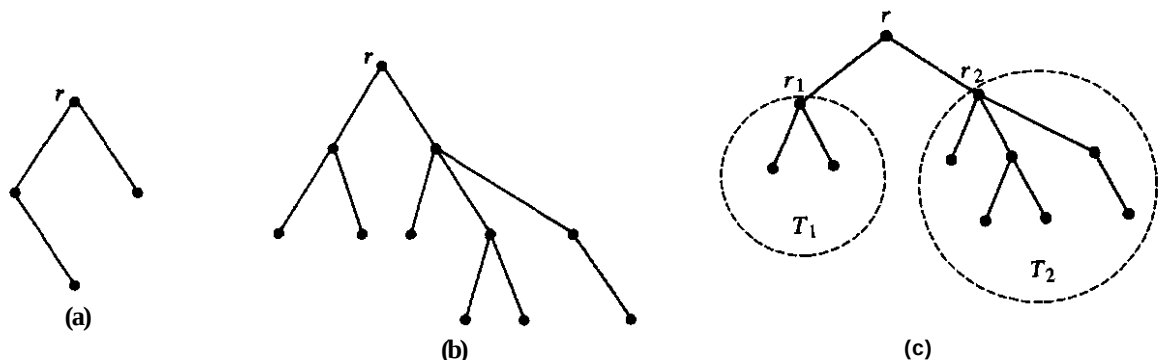
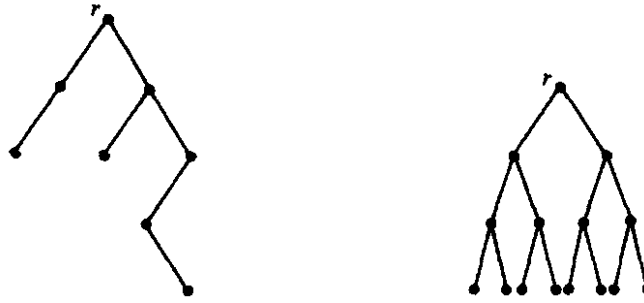


Figura 5.22

**PRÁTICA 17**

Responda as perguntas a seguir sobre a árvore binária mostrada na Figura 5.25. (Admita o nó 1 como a raiz da árvore.)

- Qual a altura?
- Qual o filho à esquerda do nó 2?
- Qual a profundidade do nó 5?

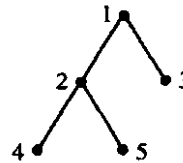


Figura 5.25

O estudo de árvore é bem fértil (sem trocadilhos) no tocante a demonstrações por indução no número de nós ou da altura. O teorema a seguir, por exemplo, que parece ser válido para todas as árvores que pudermos desenhar, pode ser demonstrado por indução.

Teorema no Número de Arestas de uma Árvore

Uma árvore com n nós tem $n - 1$ arestas.

Demonstração: Usaremos indução em n , $n \geq 1$. Para $n = 1$, a árvore consiste em um único nó e nenhuma aresta, de forma que o número de arestas é o número de vértices menos 1. Admita que qualquer árvore com k nós tem $k - 1$ arestas e consideremos uma aresta com $k + 1$ nós. Seja x uma folha da árvore (pelo menos uma folha deve existir, uma vez que a árvore deve ser finita). Então x tem um único pai y . Remova da árvore o nó x e a única aresta a que conecta x a y . O grafo resultante ainda é uma árvore e tem k nós. Portanto, pela hipótese de indução, ele tem $k - 1$ arestas e o grafo original, que contém a aresta a , tinha $(k - 1) + 1 = k$ arestas. O que conclui a demonstração.

Perceba que na prova por indução deste teorema (e também na prova por indução do teorema das cinco cores), devemos realizar a demonstração com muito mais palavras do que as demonstrações por indução anteriores. No Exemplo 11 do Cap. 2, por exemplo, a prova por indução de que

$$1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1$$

consistiu principalmente em manipulação das expressões matemáticas da equação, mas precisamos formular argumentações verbais. Palavras não são apenas aceitas em uma demonstração, elas podem constituir a maior parte da demonstração.

Grafos Direcionados

Podemos desejar que as arestas de um grafo comecem em um vértice e terminem em outro; neste caso, podemos usar um *grafo direcionado*.

Definição: Grafo Direcionado

Um **grafo direcionado (digrafo)** é um tripla ordenada (N, A, g) onde

N = um conjunto de vértices

A = um conjunto de arestas

g = uma função que associe a cada aresta a um par ordenado (x, y) de vértices, onde x é o **ponto inicial** e y é o **ponto final** de a .

Em um grafo direcionado, portanto, existe uma direção associada a cada aresta. O conceito de caminho estende-se de forma intuitiva: Um **caminho** do vértice n_0 até o vértice n_k é uma seqüência $n_0, a_0, n_1, a_1, \dots, n_{k-1}, a_{k-1}, n_k$ onde, para cada i , n_i é o ponto inicial e n_{i+1} é o ponto final de a_i . Se existe um caminho do vértice n_0 até o vértice n_k , então n_k é **alcançável** a partir de n_0 . A definição de um ciclo também se aplica a grafos direcionados.

EXEMPLO 13 No grafo direcionado da Fig. 5.26, existem diversos caminhos do vértice 1 ao vértice 3: $1, a_4, 3$ e $1, a_1, 2, a_2, 2, a_3, 3$ são dois possíveis caminhos. O vértice 3 é certamente alcançável a partir do vértice 1. O vértice 1, no entanto, não é alcançável a partir de qualquer outro vértice. Os ciclos do grafo são o laço a_2 e o caminho $3, a_5, 4, a_6, 3$.

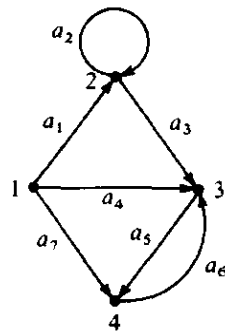


Figura 5.26

Além de impor direções às arestas de um grafo, podemos desejar modificar a definição básica de um grafo de outras maneiras. Comumente desejamos que os vértices de um grafo contêm informações de identificação, como os nomes das cidades no mapa das rotas da companhia aérea. Neste caso temos um **grafo rotulado**. Podemos desejar usar **grafos ponderados**, onde cada aresta tenha um valor numérico, ou *um peso*, associado. Por exemplo, podemos desejar indicar as distâncias das várias rotas no mapa da companhia aérea.

Aplicações

Apesar da idéia de grafos ser muito simples, uma quantidade surpreendente de situações tem relações que as levam a ser representadas na forma de grafos. Não é de surpreender que neste livro haja diversos grafos. Um grafo direcionado referente ao Exercício 23 do Cap. 3 representou uma lista encadeada. Árvores de decisão foram usadas para resolverem problemas de contagem no Cap. 3 e serão usadas novamente na Seção 4 deste capítulo para ajudar a estabelecer limites inferiores para alguns algoritmos. Representações gráficas de conjuntos parcialmente ordenados foram introduzidas e serão usadas novamente na Seção 3 deste capítulo. O diagrama E-R (por exemplo, a Fig. 4.7) é um grafo. Um diagrama PERT (por exemplo, a Fig. 4.8) é um grafo direcionado. O diagrama comutativo ilustrando a composição de funções (veja a Fig. 4.21) é um grafo direcionado. O Cap. 7 introduzirá redes lógicas e as representará como grafos direcionados. Grafos direcionados também serão usados para descrever máquinas de estado finito no Cap. 8, e a derivação de palavras em linguagens formais serão mostradas como árvores também no Cap. 8 (são as árvores de parse geradas por um compilador quando da análise de um programa de computador).

O Exercício 26 da Seção 4.1 descreve a organização de dados em uma estrutura de árvore binária. Através dessas árvores, uma coleção de registros pode ser eficientemente procurada a fim de localizar um registro em particular ou para determinar que um registro não está na coleção. Exemplos deste tipo de busca poderiam ser

uma busca por um título em uma biblioteca, pelo registro de um paciente em um hospital ou por um registro pessoal de crédito em um banco. Veremos a busca em árvores mais adiante neste capítulo.

Sabemos que o mapa de todas as companhias aéreas é um grafo. Uma representação de qualquer rede de transporte (um mapa rodoviário, por exemplo), linhas de comunicação (como em uma rede de computadores), ou rotas de distribuição de produtos ou serviços tal como dutos de transporte de gás natural ou água resulta em um grafo. A estrutura química de uma molécula é representada na forma de um grafo. Uma árvore genealógica de uma família, mesmo que existam casamentos entre membros da mesma família, será tecnicamente tratada como uma árvore. (As informações que podem ser obtidas de árvores genealógicas não são apenas interessantes, mas também podem ser úteis para pesquisas genéticas.) O diagrama organizacional indicando quem está subordinado a quem em uma grande empresa é normalmente uma árvore.

PRÁTICA 18 Desenhe o grafo referente a cada um dos casos a seguir:

- A Fig. 5.27 é o mapa rodoviário de uma parte do estado do Arizona.
- A Fig. 5.28 é uma representação de uma molécula de ozônio com três átomos de oxigênio.

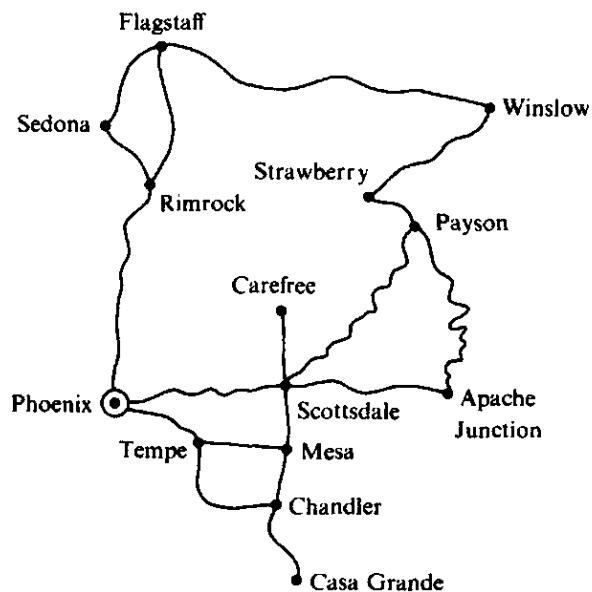


Figura 5.27

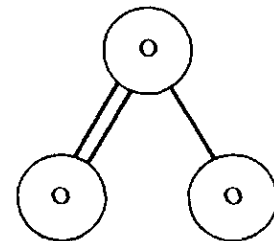


Figura 5.28

EXEMPLO 14 Expressões algébricas envolvendo operações binárias podem ser representadas através de árvores binárias rotuladas. As folhas são rotuladas como operandos, e os vértices internos são rotulados como operações binárias. Para qualquer vértice interno, a operação binária de seu rótulo é realizada sobre as expressões associadas a suas subárvores esquerda e direita. Desta forma, a árvore binária da Fig. 5.29 representa a expressão algébrica $(2 + x) - (y * 3)$.

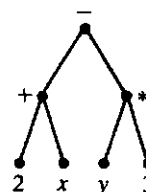


Figura 5.29

PRÁTICA 19 Qual é a árvore que representa a expressão $(2 + 3) * 5$?

EXEMPLO 15 Uma visão de alto nível de um fluxo de informação em um escritório de licenciamento de automóveis é preparado como primeiro passo no desenvolvimento de um novo sistema de licenciamento computado-rizado.

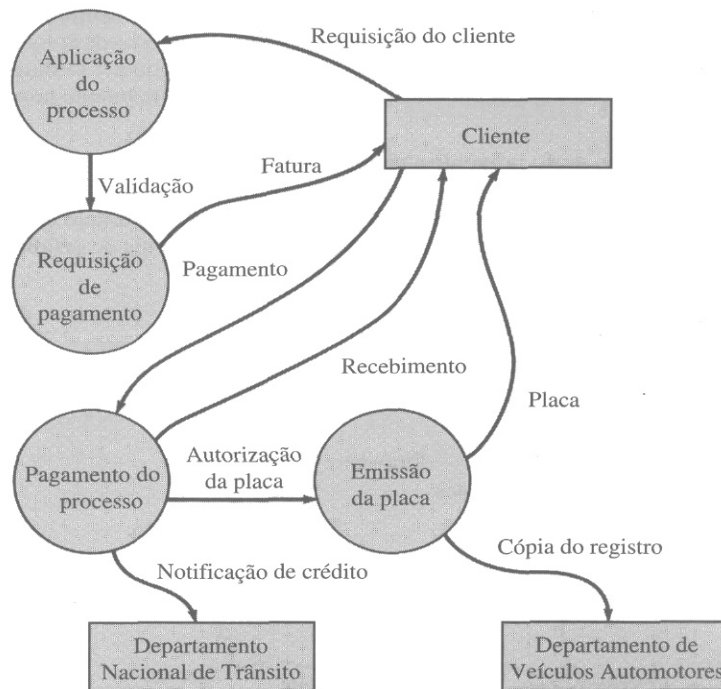


Figura 5.30

A Fig. 5.30 mostra o grafo direcionado resultante, normalmente chamado de **diagrama de fluxo de dados**. No decorrer do desenvolvimento do sistema de licenciamento, "Pagamento do processo" foi identificado como uma tarefa a ser realizada pelo software e foi quebrada em subtarefas, como mostrado na Fig. 5.31. Este **diagrama de estrutura** é uma árvore.

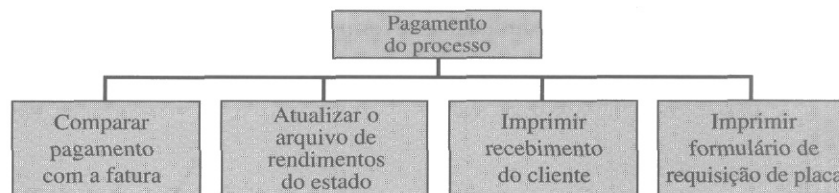


Figura 5.31

EXEMPLO 16 Redes neurais, instrumentos usados na Inteligência Artificial para os quais tarefas como o reconhecimento de padrões são representadas por grafos direcionados ponderados. A Fig. 5.32 mostra uma rede de vários níveis consistindo em unidades de entrada, unidades de saída e um "nível oculto" de unidades. Os pesos das arestas do grafo são ajustados à medida que a rede neuronal "aprende" como reconhecer certos padrões de julgamento.

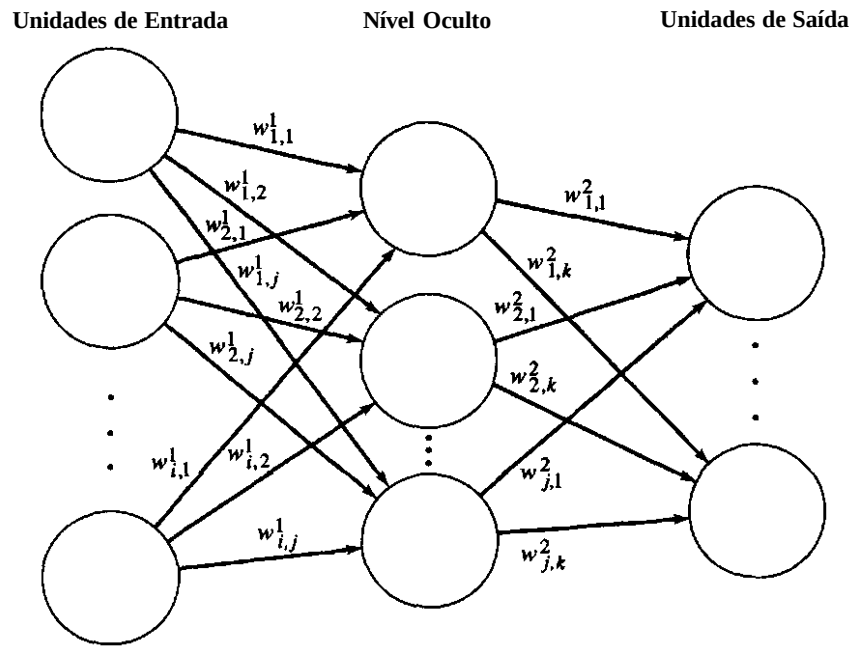


Figura 5.32

Revisão da Seção 5.1

Técnicas

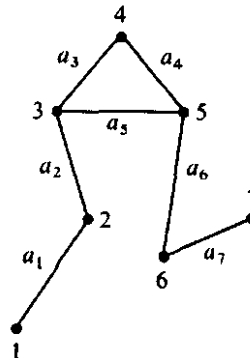
- Usar da terminologia de grafos
- Mostrar que dois grafos são ou não isomorfos
- Encontrar uma representação plana de um grafo simples ou demonstrar que não existe tal representação
- Encontrar o número cromático de um grafo
- Construir árvores para expressões

Idéia Principal

Diversas situações podem ser representadas através de grafos.

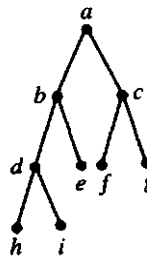
Exercícios 5.1

1. Responda as seguintes perguntas sobre o grafo mostrado a seguir:
 - a. Este grafo é simples?
 - b. Este grafo é completo?



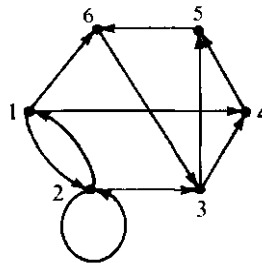
Exercício 1

- c. Este grafo é conexo?
- d. Existem dois caminhos entre os vértices 3 e 6?
- e. Este grafo possui algum ciclo?
- f. O grafo possui algum vértice cuja remoção o tornaria uma árvore?
- g. O grafo possui algum vértice cuja remoção o tornaria desconexo?
- ★2. Esboce uma figura para cada um dos seguintes grafos:
- um grafo simples com três vértices, cada qual com grau 2
 - quatro vértices, com ciclos de tamanho 1, 2, 3 e 4
 - uma árvore com cinco vértices e altura 1
3. Responda as perguntas a seguir sobre a árvore abaixo com o vértice a como raiz.
- Ela é uma árvore binária?
 - Ela é uma árvore binária completa?
 - Qual o pai do vértice e ?
 - Qual é o filho à esquerda do vértice e ?
 - Qual a altura de g ?
 - Qual é a altura da árvore?



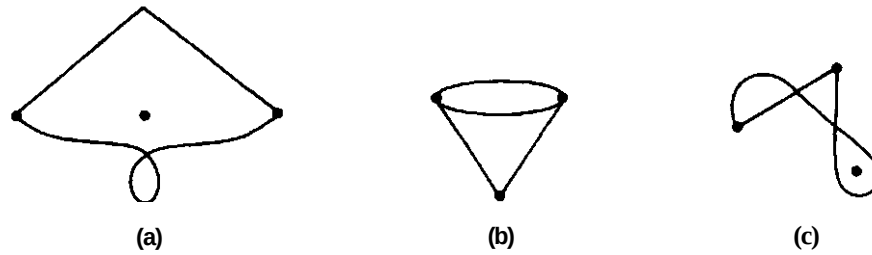
Exercício 3

- ★4. Use o grafo direcionado da figura a seguir para responder as perguntas abaixo
- Quais vértices são alcançáveis a partir do vértice 3?
 - Qual é o comprimento do menor caminho entre 3 e 6?
 - Qual é o caminho do vértice 1 ao vértice 6 com comprimento 8?



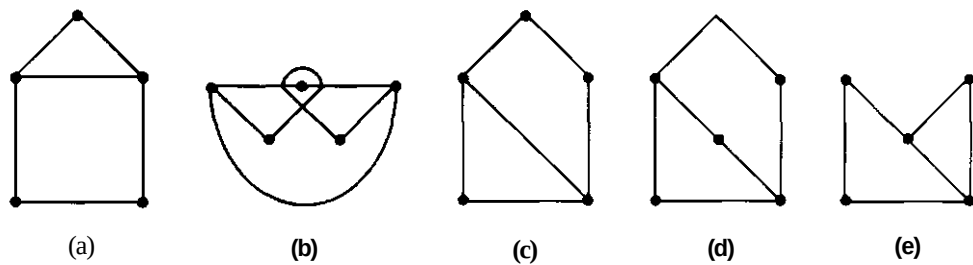
Exercício 4

5. a. Desenhe K_6 . b. Desenhe $K_{3,4}$.
6. Para cada uma das características a seguir, desenhe um grafo ou explique por que um grafo com as características pedidas não existe.
- Quatro vértices de graus 1, 2, 3 e 4, respectivamente.
 - Simplex com quatro vértices de graus 1, 2, 3 e 4, respectivamente.
 - Quatro vértices de graus 2, 3, 3 e 4, respectivamente.
 - Quatro vértices de graus 2, 3, 3 e 3, respectivamente.
- ★7. Qual dos grafos na figura abaixo não é isomorfo aos outros? Por quê?



Exercício 7

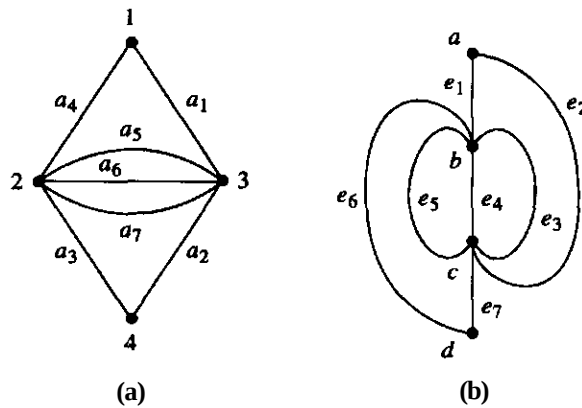
8. Qual dos grafos na figura abaixo não é isomorfo aos outros? Por quê?



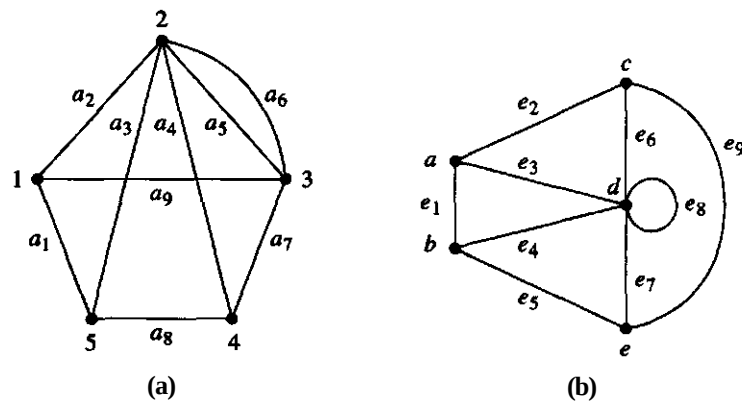
Exercício 8

Para os Exercícios 9 a 14, diga se os dois grafos apresentados são ou não isomorfos. Se forem, apresente a função que estabelece o isomorfismo entre eles; caso contrário, explique por quê.

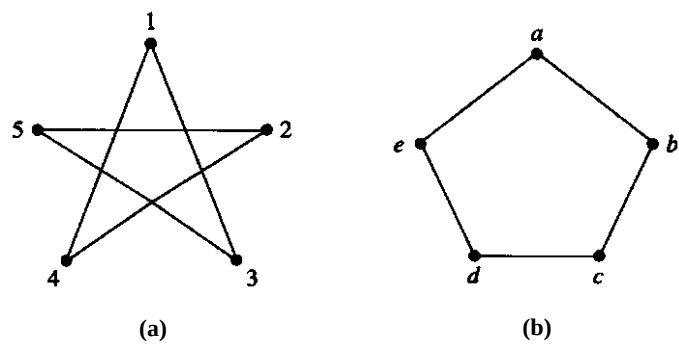
★9.



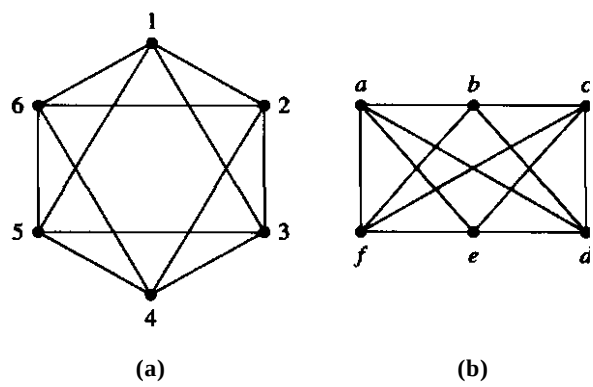
10.



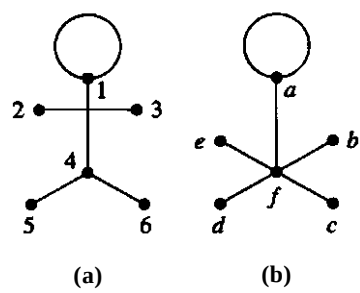
11.



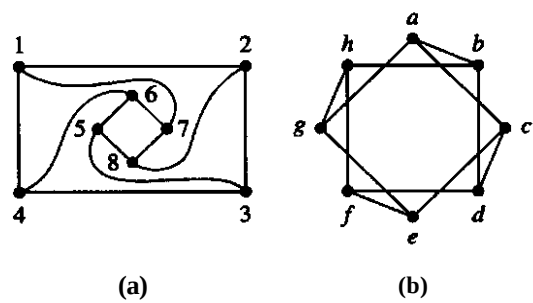
12.



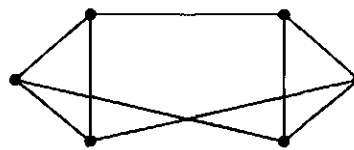
★13.



14.



15. Demonstre que dois grafos não são isomorfos se:
 - a. um deles tem mais vértices que o outro
 - b. um deles tem mais arestas que o outro
 - c. um deles tem arestas paralelas, e o outro não tem
 - d. um deles tem um laço e o outro não tem
 - e. um deles tem um vértice de grau k , e o outro não tem
 - f. um deles é conexo, e o outro não é
 - g. um deles tem um ciclo, e o outro não tem
- ★16. Desenhe todos os grafos não-isomorfos, simples com dois vértices.
17. Desenhe todos os grafos não-isomorfos, simples com três vértices.
18. Desenhe todos os grafos não-isomorfos, simples com quatro vértices.
19. Encontre uma expressão para o número de arestas de K_n e demonstre que a expressão que você encontrou está correta.
20. Verifique a fórmula de Euler para o grafo simples, conexo e planar da Fig. 5.21.
- ★21. Demonstre que $K_{2,3}$ é um grafo planar.
22. Demonstre que o grafo a seguir é um grafo planar.

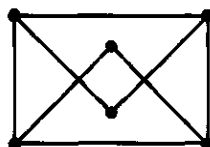


Exercício 1

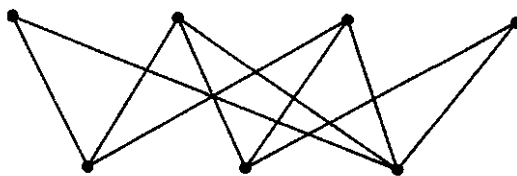
- ★23. Se um grafo simples, conexo e planar tem seis vértices, todos de grau 3, em quantas regiões ele divide o plano?
24. Se todos os vértices de um grafo simples, conexo e planar têm grau 4 e o número de arestas é 12, em quantas regiões ele divide o plano?
25. A fórmula de Euler (equação (1) do teorema sobre o número de vértices e de arestas) aplica-se a grafos que não sejam simples? E as desigualdades (2) e (3) do mesmo teorema?
26. O que está errado no argumento a seguir que visa tornar um grafo não-planar em um grafo planar através de subdivisões?
 Em um grafo não-planar existem duas arestas a_i e a_j que se intersectam em um ponto v que não é um vértice. Faça uma subdivisão elementar de a_i com a inserção de um vértice em v e uma subdivisão elementar de a_j com a inserção de um vértice em v . No grafo resultante, o ponto de interseção é um vértice. Repita este processo com todas as interseções que ocorram fora de vértices; o grafo resultante não será planar.

Nos Exercícios 27 a 30, determine se o grafo é planar (mostrando uma representação planar) ou não-planar (encontrando um subgrafo homeomorfo a K_5 ou $K_{3,3}$).

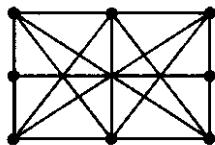
★27.



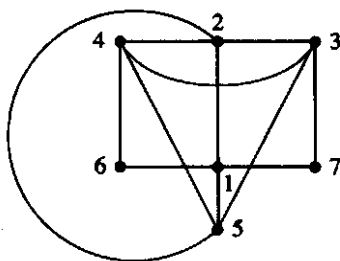
28.



★29.



30.



Os Exercícios 31 a 36 referem-se ao **complemento** de um grafo. Se G é um grafo simples, o **complemento** de G , denotado por G' é o grafo simples com o mesmo conjunto de vértices, onde os vértices $x - y$ são adjacentes em G' se, e somente se, eles não são adjacentes em G .

★31. Desenhe G' para o grafo da Fig. 5.9a.

32. Desenhe K'_4 .

33. Mostre que se dois grafos simples G_1 e G_2 são isomorfos, então seus complementos G'_1 e G'_2 também o são.

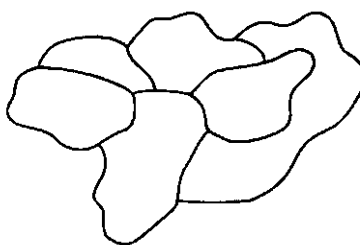
34. Um grafo simples é **autocomplementar** se for isomorfo ao seu complementar. Demonstre que em um grafo autocomplementar com n vértices ($n > 1$), $n = 4k$ ou $n = 4k + 1$ para algum inteiro k . (Dica: Use o resultado do Exercício 19.)

★35. a. Demonstre que em qualquer grafo simples G com pelo menos dois vértices, se G não for conexo, então G' é conexo. (Dica: Se G não é conexo, então G consiste em uma coleção de subgrafos conexos "dis-juntos".)

b. Encontre um grafo G onde tanto G quanto G' são conexos, mostrando, assim, que a recíproca do item (a) é falsa.

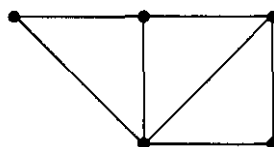
36. Demonstre que se $|N| \geq 11$ em um grafo simples e conexo, então G e G' não podem ser ambos planares.

37. Desenhe o grafo dual do mapa da figura abaixo.



Exercício 37

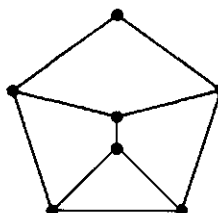
38. Desenhe um mapa para o qual o grafo da figura a seguir sirva como grafo dual.



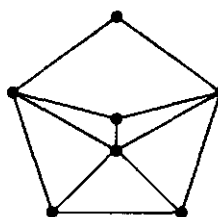
Exercício 38

Nos Exercícios 39 a 42, encontre o número cromático de cada grafo.

★39.



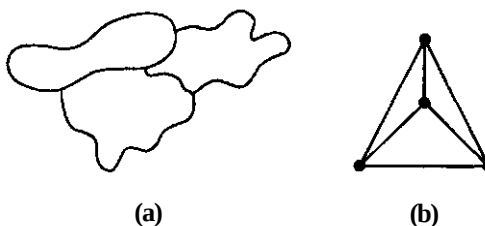
40.



41. K_n

42. $K_{m,n}$

43. O teorema das seis cores pode ser provado sem usar o grafo dual de um mapa. No lugar disto, tornamos as fronteiras das regiões retas, de forma que o problema da coloração do mapa mostrado na parte (a) da figura a seguir seja representado pelo grafo da parte (b) da mesma figura. Primeiro admitimos que nenhum país tenha um buraco no meio. Por isso, o grafo não terá laços, será planar e conexo. Além disso, todo vértice terá ao menos grau 3.



Exercício 43

- ★a. Mostre que podemos considerar o grafo como simples mostrando que se seis cores são o suficiente para colorir um grafo simples, elas serão suficientes para um grafo com arestas paralelas também. (Dica: Imagine países pequenos nos vértices temporariamente.)
- b. Demonstre que, em um grafo simples, conexo e planar com R regiões fechadas, $n - a + R = 1$.
- c. Considere um grafo simples, conexo e planar, e admita que toda região fechada tem pelo menos seis

- arestas adjacentes a ela. Mostre que $2a \leq 3n - 3$.
- d. Considere agora um grafo simples, conexo e planar onde todos os vértices têm grau pelo menos 3. Mostre que este tipo de grafo tem pelo menos uma região fechada com não mais do que cinco arestas adjacentes.
- e. Demonstre que seis cores são suficientes para colorir qualquer mapa plano onde nenhum país tenha um buraco no meio.
- f. Demonstre que seis cores são suficientes para colorir qualquer mapa plano. (*Dica:* Retire algumas fendas temporariamente do mapa.)
44. Cinco lobistas políticos estão visitando sete membros do Congresso (chamados de A a G) no mesmo dia. Os membros do Congresso que os cinco lobistas precisam visitar são:

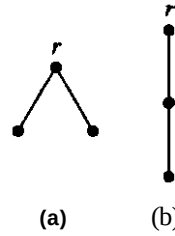
1: A, B, D
 2: B, C, F
 3: A, B, D, G
 4: E, G
 5: D, E, F

Cada membro do Congresso estará disponível por uma hora. Qual o menor número de intervalos de visita que deve ser usado a fim de que não haja conflitos entre os lobistas? (*Dica:* Trate isto como um problema de coloração de grafos.) O que ocorreria se o lobista 3 descobrisse que não precisa visitar o membro B, e o lobista 5 descobrisse que não precisa visitar o membro D?

Nos Exercícios 45 a 48, desenhe a árvore da expressão.

45. $[(x - 2) * 3] + (5 + 4)$
- ★46. $[(2 * x - 3 * y) + 4 * z] + 1$
47. $1 - (2 - [3 - (4 - 5)])$
48. $[(6 \div 2) * 4] + [(1 + x) * (5 + 3)]$
49. Demonstre que em qualquer grafo simples G com n vértices e a arestas, $2a \leq n^2 - n$.
- ★50. Demonstre que um grafo simples e conexo com n vértices tem pelo menos $n - 1$ arestas. (*Dica:* Mostre que isto pode ser tratado como um problema do tipo "Um grafo simples e conexo com m arestas tem, no máximo, $m + 1$ vértices." E então use indução em m .)
51. Demonstre que um grafo simples com n vértices ($n \geq 2$) e mais de $C(n - 1, 2)$ arestas é conexo. (*Dica:* Use os Exercícios 35 e 50.)
52. Demonstre que uma árvore com n vértices, $n \geq 2$, tem pelo menos dois vértices de grau 1.
- ★53. Demonstre que um grafo simples é uma árvore não-enraizada se, e somente se, existir um único caminho entre quaisquer dois vértices.
54. Seja G um grafo simples. Demonstre que G é uma árvore não-enraizada se, e somente se, G for conexo e a remoção de qualquer aresta sua o tornar desconexo.
55. Seja G um grafo simples. Demonstre que G é uma árvore não-enraizada se, e somente se, a inclusão de uma aresta tornar G um grafo com exatamente um ciclo.
- ★56. Demonstre que uma árvore binária tem no máximo 2^d vértices em seu nível d .
57. a. Desenhe uma árvore binária completa de altura 2. Quantos vértices ela tem?
 b. Desenhe uma árvore binária completa de altura 3. Quantos vértices ela tem?
 c. Formule uma conjectura sobre quantos vértices tem uma árvore binária completa de altura h .
 d. Demonstre sua conjectura. (*Dica:* Use o Exercício 56.)
58. Demonstre que uma árvore binária completa com x vértices internos tem $x + 1$ folhas.
59. Demonstre que o número de folhas de uma árvore binária é 1 mais o número de vértices com dois filhos.

Nos Exercícios 60 e 61, duas árvores são isomorfas, se houver uma bijeção $f: N_1 \rightarrow N_2$ onde f leve a raiz de uma árvore na raiz da outra e onde $f(y)$ é um filho de $f(x)$ na segunda árvore, se y for um filho de x na primeira. Desta forma, a figura a seguir:



as duas árvores são grafos isomorfos, mas não árvores isomorfas (em (a) a raiz tem dois filhos, e em (b) não). Essas são as únicas árvores não-isomorfas com três vértices.

- ★60. Mostre que existem quatro árvores não-isomorfas com quatro vértices.
- 61. Mostre que existem nove árvores não-isomorfas com cinco vértices.
- 62. Na Fig. 5.25, admita que o vértice 4 é a raiz, e redesenhe a árvore com a raiz em cima.
- 63. Na Fig. 5.25, admita que o vértice 2 é a raiz, e redesenhe a árvore com a raiz em cima.

Seção 5.2 Representações Computacionais de Grafos

Já dissemos que a maior vantagem dos grafos é sua representação visual das informações. Mas para o armazenamento e manipulação de grafos por um computador, esta informação precisa ser representada de outras maneiras. Uma opção é usar a definição formal de grafos; isto é, podemos armazenar um conjunto de vértices, um conjunto de arestas e uma função que associe a cada aresta um conjunto com seus dois vértices extremos.

EXEMPLO 17 Para representar o grafo da Fig. 5.2 usando esta definição, seria necessário armazenar os conjuntos

$$\{1, 2, 3, 4, 5\}$$

$$\{a_1, a_2, a_3, a_4, a_5, a_6\}$$

e

$$\{[a_1, 1, 2], [a_2, 1, 2], [a_3, 2, 2], [a_4, 2, 3], [a_5, 1, 3], [a_6, 3, 4]\}$$

onde, neste conjunto, a relação funcional foi representada por vetores de três elementos. •

Esta representação poderia ser bem mais eficiente se nos limitássemos a grafos simples e conexos. Neste caso não precisaríamos explicitar os nomes das arestas ou fornecer o conjunto de vértices.

EXEMPLO 18 A representação do grafo da Fig. 5.4a exigiria o armazenamento do conjunto

$$\{[1, 2], [2, 3], [1, 3]\}$$

•

Ambas as representações acima são um tanto desajeitadas. Suponha que desejamos determinar (em um grafo simples e conexo) se dois vértices são ou não adjacentes. Dependendo dos recursos da linguagem de programação que estivermos usando, poderemos testar, por exemplo, se $[1, 3]$ ou $[3, 1]$ pertence ao conjunto. (A linguagem Pascal oferece o operador `in` para verificar a pertinência de um elemento a um conjunto.) No entanto, se desejássemos encontrar todos os vértices adjacentes ao vértice 1, precisaríamos repetir este tipo de pergunta diversas vezes. Na verdade, as representações de grafos com conjuntos nunca é usada. Ao invés dela, usam-se duas estruturas de dados: uma *matriz de adjacências* ou uma *lista de adjacências*.

Matriz de Adjacências

Suponha que um grafo tem n vértices numerados $n_1, n_2, \dots, n_n$. Esta numeração define uma ordenação arbitrária no conjunto de vértices; lembremos que este tipo de conjunto é chamado de um conjunto ordenado. No entanto, isto é feito com o único intuito de identificar os vértices — não há qualquer relevância no fato de um vértice aparecer antes de outro na ordenação. De posse dos vértices ordenados, podemos formar uma matriz $n \times n$ onde o elemento i, j é o número de arestas entre o vértice n_i e n_j . Esta matriz é chamada de **matriz de adjacências** A do grafo com relação à ordenação. Portanto,

$$a_{ij} = p \text{ onde existem } p \text{ arestas entre } n_i \text{ e } n_j$$

EXEMPLO 19 A matriz de adjacências para o grafo da Fig. 5.33 com respeito à ordenação 1, 2, 3, 4 é uma matriz 4×4 . O elemento 1,1 é um 1 devido ao fato de haver um laço no vértice 1. Todos os demais elementos da diagonal são 0. O elemento 2,1 (segunda linha, primeira coluna) é 1 porque existe apenas uma aresta entre os vértices 2 e 1, o que também indica que o elemento 1,2 vale 1. Até este ponto, temos

$$A = \begin{bmatrix} 1 & 1 & - & - \\ 1 & 0 & - & - \\ - & - & 0 & - \\ - & - & - & 0 \end{bmatrix}$$

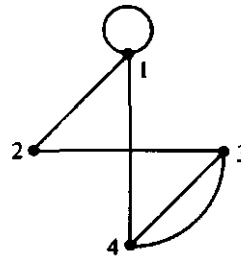


Figura 5.33

PRÁTICA 20 Complete a matriz de adjacências relativa ao grafo da Fig. 5.33.

A matriz de adjacências da Prática 20 é simétrica, o que ocorrerá para qualquer grafo não-direcionado — se há uma aresta p entre os vértices n_i e n_j , então p também é uma aresta entre os vértices n_j e n_i . A simetria da matriz indica que apenas os elementos da diagonal principal e os elementos abaixo dela precisam ser armazenados. Portanto, todas as informações contidas no grafo da Fig. 5.33 estão representadas na porção triangular inferior da matriz, e o grafo pode ser reconstruído a partir desta porção.

$$\begin{bmatrix} 1 & & & \\ 1 & 0 & & \\ 0 & 1 & 0 & \\ 1 & 0 & 2 & 0 \end{bmatrix}$$

Em um grafo direcionado, a matriz de adjacências A reflete a direção das arestas. Em uma matriz direcionada

$$a_{ij} = p \text{ onde } p \text{ é o número de arestas do vértice } n_i \text{ para o vértice } n_j$$

Uma matriz de adjacências de um grafo direcionado não é necessariamente simétrica, uma vez que uma aresta do vértice n_i para o vértice n_j não implica uma aresta do vértice n_j para o vértice n_i .

EXEMPLO 20 Considere o grafo direcionado da Fig. 5.34. Sua matriz de adjacências é

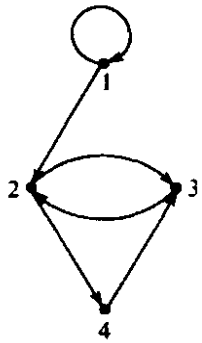


Figura 5.34

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Em um grafo simples ponderado, os elementos da matriz de adjacências podem indicar o peso de cada aresta no lugar de apenas indicar a presença da aresta pelo número 1.

Lista de Adjacências

Diversos grafos, ao contrário de serem completos, têm relativamente poucas arestas. Esses grafos têm matrizes de adjacências ditas **esparsas**; isto é, suas matrizes de adjacências contêm muitos zeros. No caso de o grafo ter n vértices, serão necessários n^2 elementos para representarem sua matriz de adjacências, ainda que a maior parte desses elementos seja zero. Qualquer algoritmo que precise que todas as arestas do grafo sejam verificadas realizará comparações com todos os n^2 elementos da matriz, uma vez que não há outro meio de determinar quais os elementos que não são zero além de examiná-los. Para encontrar todos os vértices adjacentes ao vértice n_i , precisamos varrer toda a i -ésima linha da matriz de adjacências, que tem um total de n elementos.

Um grafo com relativamente poucas arestas pode ser representado mais eficientemente, se armazenarmos apenas os elementos não-nulos de sua matriz de adjacências. Esta representação consiste em uma lista para cada vértice de todos os vértices adjacentes a ele. Usam-se ponteiros para permitir que caminhemos de um elemento da lista para o seguinte. Este tipo de estrutura de dados é chamado de **lista encadeada**. (Já falamos de listas encadeadas no Cap. 3 como uma alternativa de implementação para conjuntos.) Temos um vetor de n ponteiros, um para cada vértice, para obter o início de cada lista. Esta representação na forma de **lista encadeada**, apesar de precisar de mais memória para os ponteiros, pode ser mais eficiente que a matriz de adjacências. Para encontrarmos todos os vértices adjacentes a n_i precisamos varrer a lista referente a n_i , que deve ter menos elementos que os n que teríamos que examinar na matriz de adjacências. No entanto, existem desvantagens; se desejarmos determinar se um vértice n_j em particular é adjacente ao vértice n_i , temos que varrer toda a lista encadeada de n_i , enquanto que na matriz de adjacências poderíamos pesquisar o elemento i, j diretamente.

EXEMPLO 21 A lista de adjacências para o grafo da Fig. 5.33 contém um vetor de quatro elementos de ponteiros, um para cada vértice. O ponteiro de cada vértice aponta para um vértice adjacente, que aponta para outro vértice adjacente, e assim por diante. A estrutura da lista de adjacências é mostrada na Fig. 5.35.

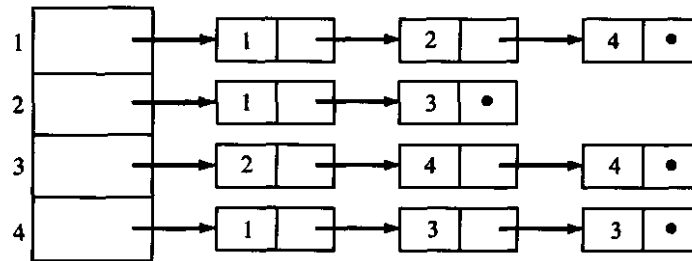


Figura 5.35

Nesta figura, os pontos indicam um **ponteiro nulo**, indicando que não há mais nada a ser apontado e que se alcançou o fim da lista. Tratamos o caso de arestas paralelas listando-as mais de uma vez na lista de adjacências de n_i se há mais de uma aresta entre n_i e o vértice.

PRÁTICA 21 Desenhe a lista de adjacências que representa o grafo mostrado na Fig. 5.36.

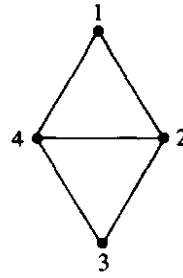


Figura 5.36

Em um grafo não-direcionado, cada aresta é representada duas vezes. Se n_j está na lista de adjacências referente a n_i , então n_i também está na lista de adjacências de n_j . A representação por lista de adjacências de um grafo direcionado inclui n_j na lista de adjacências de n_i , se existe uma aresta do vértice n_i para o vértice n_j ; n_i não precisa estar na lista de adjacências de n_j . Em um grafo rotulado ou em um grafo ponderado, outros dados podem ser armazenados juntamente com o nome do vértice na lista de adjacências.

EXEMPLO 22 A Fig. 5.37 mostra um grafo ponderado. A representação por listas de adjacências para este grafo é mostrada na Fig. 5.37b. Para cada registro na lista, o primeiro item de dado é o vértice, o segundo é o peso da aresta que leva a este vértice e o terceiro é o ponteiro para os dados do próximo vértice. Perceba que o elemento 4 no vetor de ponteiros iniciais é nulo porque não há qualquer aresta que comece no vértice 4.

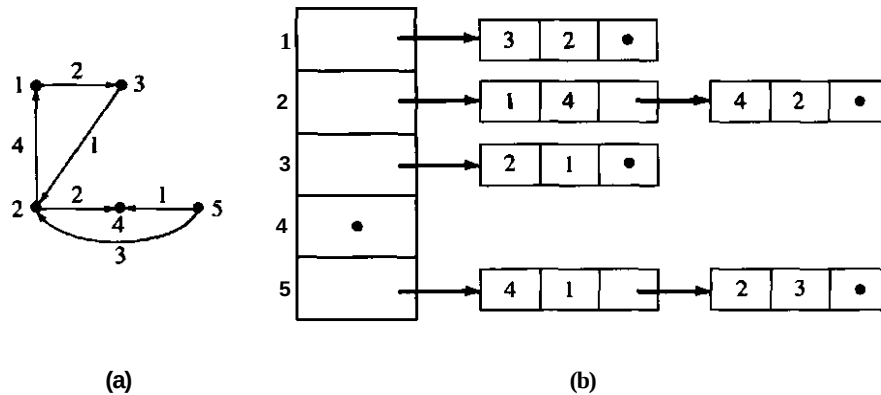


Figura 5.37

Em uma linguagem de programação que não ofereça ponteiros, podemos implementar listas de adjacências através de vetores de diversas colunas (ou um vetor de registros), onde uma coluna contém os vértices e outra coluna contém o índice no vetor do próximo vértice na lista de adjacências — "um pseudoponteiro". A desvantagem deste tipo de abordagem é que a quantidade máxima de espaço de memória que pode vir a ser necessária precisa ser alocada inicialmente para o vetor; não é possível criar mais espaço para o mesmo ao longo da execução do programa.

EXEMPLO 23 A representação por vetor-ponteiro do grafo da Fig. 5.37a é mostrada na Fig. 5.38. Um ponteiro nulo é indicado pelo índice 0 do vetor.

A linha 2 desse vetor, que representa o vértice 2, tem um ponteiro para o índice 7. No índice 7 do vetor, encontramos o vértice 1 com peso 4, representando a aresta de peso 4 entre o vértice 2 para o vértice 1. O ponteiro para o índice 8 indica que a lista de adjacências para o vértice 2 tem mais entradas. No índice 8, vemos que existe uma aresta do vértice 2 para o vértice 4 com peso 2 e que isto encerra a lista de adjacências para o vértice 2.

	Vértice	Peso	Ponteiro
1			6
2			7
3			9
4			0
5			10
6	3	2	0
7	1	4	8
8	4	2	0
9	2	1	0
10	4	1	11
11	2	3	0

Figura 5.38

Representação de Árvore Binária

Como uma árvore também é um grafo, todas as representações que vimos até agora para grafos em geral podem também ser usadas para representar árvores. As árvores binárias, no entanto, apresentam características especiais que não desejamos perder em suas representações, especificamente a identificação dos filhos à esquerda e à direita. O equivalente à representação através de uma matriz de adjacências é um vetor de duas colunas (um vetor de registros) onde os dados de cada vértice são os filhos à esquerda e à direita. O equivalente à representação através de uma lista de adjacências é uma coleção de registros com os campos referentes aos filhos à esquerda e à direita contendo ponteiros para outros registros deste tipo.

EXEMPLO 24 Para a árvore binária mostrada na Fig. 5.39, a representação do vetor de filhos à esquerda e à direita é mostrada na Fig. 5.40a. Zeros, novamente, indicam ponteiros nulos. A representação através de ponteiros é dada na Fig. 5.40b.

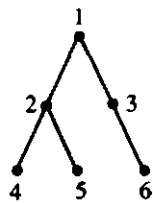
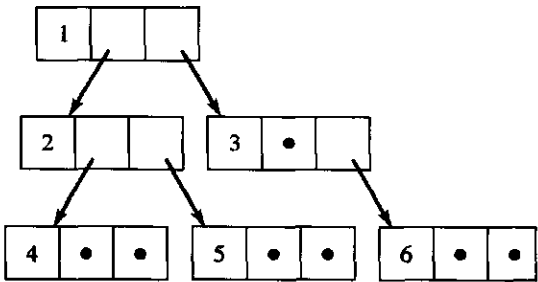


Figura 5.39

	Filho à esquerda	Filho à direita
1	2	3
2	4	5
3	0	6
4	0	0
5	0	0
6	0	0



(b)

Figura 5.40

PRÁTICA 22 Forneça a representação através do vetor de filhos à esquerda e à direita da árvore binária da Fig. 5.41.

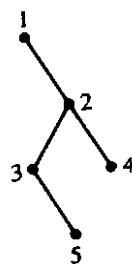


Figura 5.41

Revisão da Seção 5.2

Técnicas

- Construção de matrizes e listas de adjacências para grafos e grafos direcionados
- Construção de representações de árvores binárias na forma de vetores

Idéias Principais

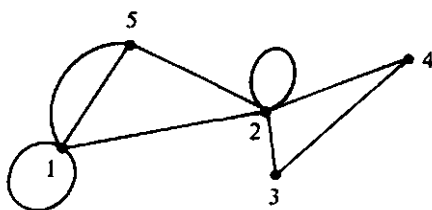
Grafos podem ser representados como matrizes e listas encadeadas.

Árvores binárias podem ser representadas como vetores e como listas encadeadas.

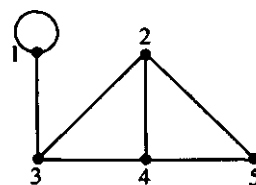
Exercícios 5.2

Nos Exercícios 1 a 6 escreva a matriz de adjacências que representa o grafo nas figuras.

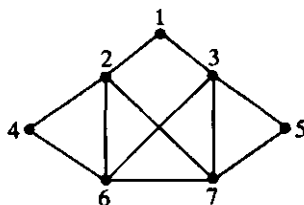
★1.



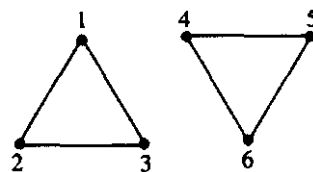
2.



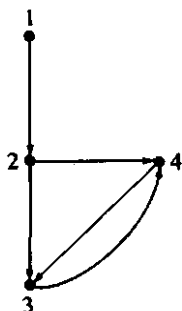
3.



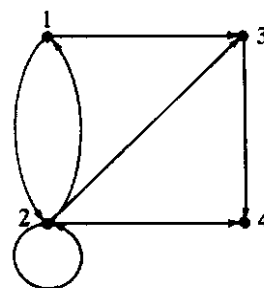
4.



★5.



6.



Nos Exercícios 7 a 10, desenhe o grafo representado pelas matrizes de adjacências apresentadas.

7.
$$\begin{bmatrix} 0 & 2 & 0 \\ 2 & 0 & 2 \\ 0 & 2 & 0 \end{bmatrix}$$

8.
$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 2 & 0 \end{bmatrix}$$

★9.
$$\begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

10.
$$\begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

★11. A matriz de adjacências para um grafo não-direcionado é dada em sua forma triangular inferior por

$$\begin{bmatrix} 2 & & & \\ 1 & 0 & & \\ 0 & 1 & 1 & \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Desenhe o grafo que ela representa.

12. A matriz de adjacências de um grafo direcionado é dada por

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 2 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Desenhe o grafo que ela representa.

- ★13. Descreva o grafo cuja matriz de adjacências é I_n a matriz identidade $n \times n$.
- 14. Descreva a matriz de adjacências para K_n , o grafo simples e completo com n vértices.
- 15. Dada uma matriz de adjacências A para um grafo simples G , descreva a matriz de adjacências para o complemento de G (veja a definição no Exercício 31 da Seção 5.1).
- 16. Dada a matriz de adjacências A para um grafo direcionado G , descreva o grafo representado pela matriz de adjacências A^T (veja o Exercício 26 na Seção 4.4).

Para os Exercícios 17 a 22, desenhe a representação na forma de lista de adjacências para o grafo indicado.

17. Exercício 1

18. Exercício 2

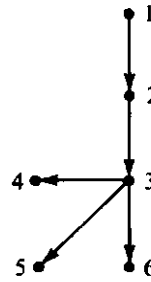
★19. Exercício 3

20. Exercício 4

21. Exercício 5

22. Exercício 6

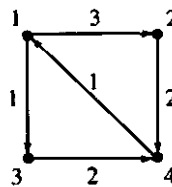
23. Com relação ao grafo a seguir.



Exercício 23

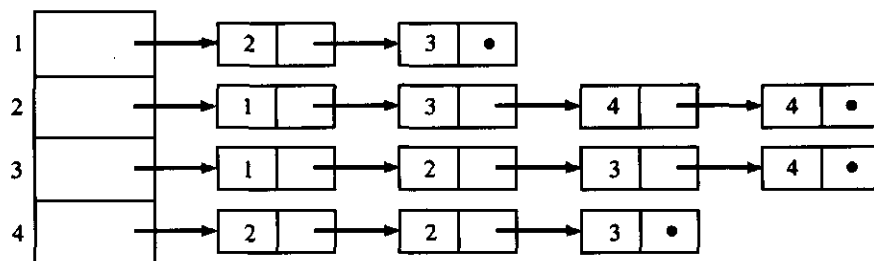
- Deseje sua representação na forma de lista de adjacências.
- Quantas posições de memória são necessárias para o armazenamento da lista de adjacências? (Admita que um ponteiro ocupa uma posição de memória.)
- Quantas posições de memória são necessárias para o armazenamento da matriz de adjacências deste grafo?

★24. Desenhe a representação na forma da lista de adjacências para o grafo ponderado da figura a seguir.



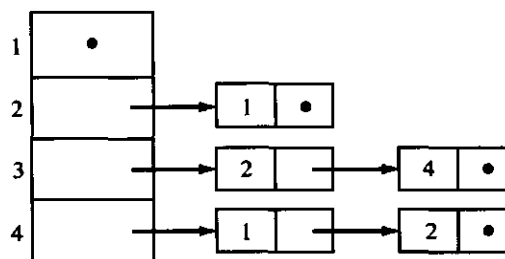
Exercício 24

- ★25. Construa a representação vetor-ponteiro para o grafo direcionado do Exercício 6.
26. Construa a representação vetor-ponteiro para o grafo direcionado do Exercício 24.
27. Desenhe o grafo não-direcionado representado pela lista de adjacências da figura a seguir.



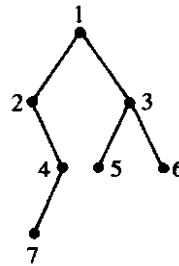
Exercício 27

28. Desenhe o grafo direcionado representado pela lista de adjacências da figura dada.



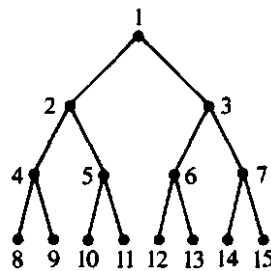
Exercício 28

- ★29. Escreva a representação por vetor de filhos à esquerda e à direita para a árvore binária da figura dada.



Exercício 29

30. Escreva a representação por vetor de filhos à esquerda e à direita para a árvore binária da figura dada.



Exercício 30

- ★31. Desenhe a árvore binária representada pelo vetor de filhos à esquerda e à direita da figura dada (1 é a raiz).

	Filho à esquerda	Filho à direita
1	2	3
2	4	0
3	5	0
4	6	7
5	0	0
6	0	0
7	0	0

Exercício 31

32. Desenhe a árvore binária representada pelo vetor de filhos à esquerda e à direita da figura dada (1 é a raiz).

	Filho à esquerda	Filho à direita
1	2	0
2	3	4
3	0	0
4	5	6
5	0	0
6	0	0

Exercício 32

33. Escreva a representação por vetor de filhos à esquerda e à direita para a árvore binária de busca que é gerada pelo processamento da frase "Todo gaulês divide-se em três partes" (veja o Exercício 26 da Seção 4.1). Além disso, armazene o nome de cada nó da árvore.
34. A figura a seguir representa uma árvore binária na qual o filho à esquerda e o pai de cada vértice são dados. Desenhe a árvore binária (1 é a raiz).

	Filho à esquerda	Pai
1	2	0
2	4	1
3	0	1
4	0	2
5	0	2
6	0	3

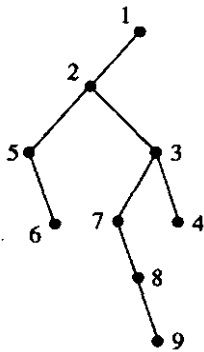
Exercício 34

35. A figura dada representa uma árvore (não necessariamente binária), onde para cada nó, o filho mais à esquerda e o irmão mais próximo à direita são dados. Desenhe a árvore (1 é a raiz).

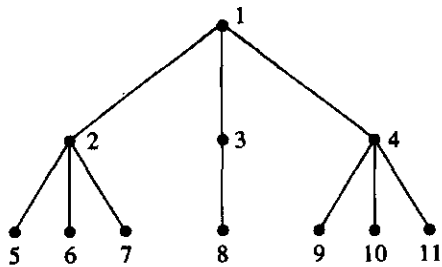
	Filho à esquerda	Filho à direita
1	2	0
2	5	3
3	0	4
4	8	0
5	0	6
6	0	7
7	0	0
8	0	0

Exercício 35

36. a. Escreva a representação na forma do vetor com o filho mais à esquerda e o irmão à direita descrita no Exercício 35 para a árvore da figura dada.



Exercício 37

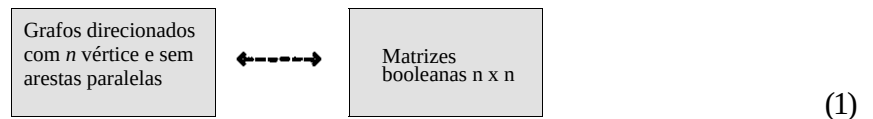


Exercício 36

- b. Agora desenhe a árvore binária que resulta se tratarmos a resposta do item (a) como a representação por vetor de filhos à esquerda e à direita da árvore. Uma árvore arbitrária pode, portanto, ser pensada como tendo uma representação por árvore binária.
37. A árvore binária da figura dada é a representação de uma árvore geral (como no item (b) do Exercício 36.) Desenhe a árvore.

Seção 5.3 Grafos Direcionados e Relações Binárias, e Algoritmo de Warshall

Nesta seção, voltaremos nossa atenção para grafos direcionados (não ponderados) sem arestas paralelas. (Em um grafo direcionado, duas arestas do vértice a para o vértice b são paralelas, mas uma aresta do vértice a para o vértice b e outra do vértice b para o vértice a não são paralelas.) Considere a matriz de adjacências do grafo (admitindo alguma ordenação arbitrária de seus n vértices, que sempre assumiremos ao tratar a matriz de adjacências de um grafo). Ela será uma matriz $n \times n$, não necessariamente simétrica. Além disso, por não haver arestas paralelas no grafo, a matriz de adjacências será uma matriz booleana, isto é, uma matriz cujos elementos sejam apenas 0s e 1s. Reciprocamente, dada uma matriz booleana $n \times n$, podemos reconstruir o grafo direcionado representado por esta matriz, e ele não terá arestas paralelas. Portanto, existe uma correspondência um-para-um, que pode ser representada por



Veremos agora como as relações binárias se encaixam nesta correspondência.

Grafos Direcionados e Relações Binárias

Suponha que G é um grafo direcionado com n vértices e sem arestas paralelas. Seja N o conjunto de vértices. Se (n_i, n_j) é um par ordenado de vértices, então existe ou não uma aresta entre os vértices n_i e n_j . Podemos usar esta propriedade para definir uma relação binária no conjunto N :

$$n_i \rho n_j \leftrightarrow \text{existe uma aresta em } G \text{ de } n_i \text{ para } n_j$$

Esta relação é a relação de adjacência do grafo.

EXEMPLO 25 Para o grafo direcionado da Fig. 5.42, a relação de adjacência é $\{(1, 2), (1, 3), (3, 3), (4, 1), (4, 2), (4, 3)\}$.

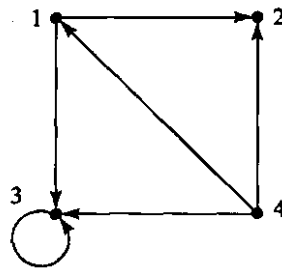


Figura 5.42

Reciprocamente, se ρ é uma relação binária no conjunto N , podemos definir um grafo direcionado G com N sendo o conjunto de vértices, e uma aresta de n_i para n_j se, e somente se, $n_i \rho n_j$. G não terá arestas paralelas.

EXEMPLO 26 Para o conjunto $N = \{1, 2, 3, 4\}$ e a relação binária $\{(1, 4), (2, 3), (2, 4), (4, 1)\}$ em N , obtemos o grafo direcionado mostrado na Fig. 5.43.

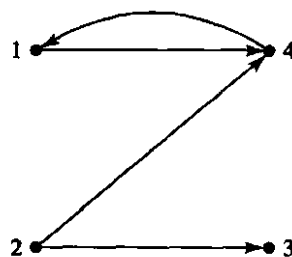
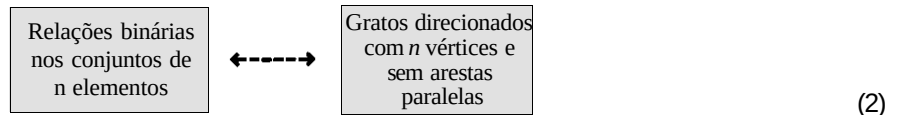
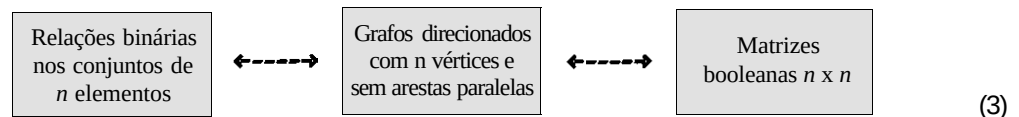


Figura 5.43

Agora temos outra correspondência um-para-um:



Naturalmente, uma relação um-para-um significa a existência de uma bijeção. Como a bijeção é mantida pela composição de funções, se compusermos as bijeções (1) e (2) obteremos uma bijeção que nos fornece uma correspondência um-para-um entre as relações binárias e matrizes. Portanto, temos os três conjuntos equivalentes:



Um elemento de qualquer um dos três conjuntos tem representações correspondentes nos outros dois conjuntos.

PRÁTICA 23 Forneça o conjunto de pares ordenados na relação de adjacência da seguinte relação booleana; além disso, desenhe o grafo direcionado:

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Lembre-mos das propriedades reflexiva, simétrica, anti-simétrica e transitiva de uma relação binária em um conjunto que vimos no Cap. 4. Se uma relação binária em um conjunto N tem uma determinada propriedade, esta propriedade reflete-se no grafo e na matriz booleana correspondentes. De forma análoga, certas características de um grafo direcionado ou de uma matriz booleana implicam certas propriedades da relação de adjacências correspondentes.

EXEMPLO 27 Se ρ é uma relação reflexiva em um conjunto N , então para cada $n_i \in N$, $n_i \rho n_i$. No grafo direcionado correspondente existirá um laço em cada vértice, e a matriz booleana correspondente terá 1 s ao longo de sua diagonal principal.

PRÁTICA 24 Explique por que a relação binária correspondente ao grafo direcionado da Fig. 5.26 não é anti-simétrica.

No Cap. 4 representamos ordenações parciais em um conjunto através do diagrama de Hasse. De que forma esta representação difere da representação de um grafo direcionado? O diagrama de Hasse é uma simplificação da representação através de grafos direcionados. Suponha que G seja uma representação na forma de um grafo direcionado de uma ordenação parcial. Como uma ordenação parcial é reflexiva, G terá um laço em cada vértice; podemos eliminar esses laços em um diagrama de Hasse sem perder qualquer informação, pois sabemos que todo vértice tem um laço; isto é, que todo vértice relaciona-se com ele mesmo. Como uma ordenação parcial é transitiva, se $a \rho b$ e $b \rho c$, então $a \rho c$. Em um grafo direcionado, precisaríamos ter uma aresta de a para b , outra de b para c e ainda outra de a para c . No diagrama de Hasse, podemos eliminar a aresta de a para c sem perda de informações se mantivermos em mente a propriedade transitiva. Finalmente, o diagrama de Hasse não é um grafo direcionado propriamente dito, mas criamos a convenção de que se a for o predecessor imediato de b , então a aparecerá abaixo de b . Portanto, podemos obter um grafo direcionado a partir do diagrama de Hasse, se colocarmos setas apontando para cima em todas as arestas. A propriedade anti-simétrica evita qualquer conflito como o vértice a estar abaixo do vértice b , e o vértice b estar abaixo do vértice a .

No Cap. 4 também vimos operações de conjuntos que podem ser aplicadas a duas relações binárias ρ e σ em um conjunto N , as operações $\rho \cup \sigma$ e $\rho \cap \sigma$. A $\rho \cup \sigma$ é a união dos pares ordenados de ρ e de

σ , enquanto que a operação $\rho \cap \sigma$ é a interseção dos pares ordenados de ρ e de σ . Seja R e S matrizes booleanas para ρ e σ , respectivamente. A matriz booleana $\rho \cap \sigma$ terá 1 na posição i, j se, e somente se, tiver um 1 na posição i, j de R ou se tiver um 1 na posição i, j de S . Cada elemento da matriz booleana para $\rho \cup \sigma$ é o máximo dos dois elementos correspondentes em R e S , de forma que a matriz booleana para $\rho \cup \sigma$ é $R \vee S$ (veja a discussão sobre operações em matrizes booleanas na Seção 4.4). De forma análoga, a matriz booleana para $\rho \cap \sigma$ terá um 1 na posição i, j se, e somente se, tanto a matriz R quanto a matriz S tiverem 1 na posição i, j . Portanto, a matriz booleana para $\rho \cap \sigma$ é $R \wedge S$.

Alcançabilidade

A propriedade de "alcançabilidade" tem uma interpretação interessante em cada uma das três formas equivalentes em (3) — grafos direcionados, relação de adjacências e matriz de adjacências. Já temos, da Seção 5.1, uma definição para este termo em grafos direcionados, que repetimos agora.

Definição: Vértice Alcançável

Em um grafo direcionado, o vértice n_j é alcançável a partir do vértice n_i se existir um caminho de n_i até n_j .

EXEMPLO 28 No grafo direcionado da Fig. 5.43, o vértice 3 não é alcançável a partir do vértice 4 ou do vértice 1. O vértice 1 é alcançável a partir do vértice 2 pelo caminho 2-4-1. •

Em um sistema modelado por um grafo direcionado (um diagrama de fluxo de dados, por exemplo) com um "vértice inicial", qualquer vértice que não é alcançável a partir do vértice inicial nunca poderá afetar o sistema e, portanto, pode ser excluído do grafo. Se o grafo direcionado representar alguma coisa como as rotas de uma companhia aérea ou os trajetos dos dados em uma rede de computadores, não devemos desejar ter nós que não sejam alcançáveis por algum outro nó. Portanto, o recurso de testar a alcançabilidade tem diversas aplicações práticas.

A matriz de adjacências A de um grafo direcionado G com n vértices e sem arestas paralelas terá um 1 na posição i, j se houver uma aresta do vértice n_i para o vértice n_j . Este é um caminho de tamanho 1 de n_i para n_j . A matriz de adjacências nos dá informações relativas a uma forma limitada de alcançabilidade, as que são obtidas através de caminhos de comprimento 1. No entanto, vamos realizar a multiplicação de matrizes $A \times A$. Denotaremos este produto por $A^{(2)}$ a fim de distinguir de A^2 , o resultado de $A \cdot A$ usando a multiplicação ordinária de matrizes. Lembremos, da Seção 4.4, que na definição da multiplicação booleana de matrizes o elemento i, j de $A^{(2)}$ é dado por

$$A^{(2)}[i, j] = \bigvee_{k=1}^n (a_{ik} \wedge a_{kj}) \quad (4)$$

Se um termo como $a_{i2} \wedge a_{2j}$ nesta soma valer 0, então ou $a_{i2} = 0$ ou $a_{2j} = 0$ (ou ambos), e não há caminho de comprimento 1 de n_i a n_2 ou não há caminho de comprimento 1 de n_2 a n_j (ou ambos). Portanto, não existe caminho de comprimento 2 de n_i para n_j passando por n_2 . Se, por outro lado, $a_{i2} \wedge a_{2j}$ não for 0, então tanto $a_{i2} = 1$ e $a_{2j} = 1$. Neste caso, existe um caminho de comprimento 1 de n_i para n_2 e um caminho de comprimento 1 de n_2 para n_j , de forma que há um caminho de comprimento 2 de n_i para n_j passando por n_2 . Um caminho de comprimento 2 de n_i para n_j existirá se, e somente se, existir um caminho de comprimento 2 que passe por algum dos vértices de 1 a n , isto é, se, e somente se, pelo menos uma das parcelas da disjunção (4) for 1 e portanto $A^{(2)}[i, j] = 1$. Portanto, as entradas em $A^{(2)}$ nos indicam a alcançabilidade através de caminhos de comprimento 2.

PRÁTICA 25 Encontre A para o grafo da Fig. 5.43 e calcule $A^{(2)}$. O que o elemento 2,1 indica? •

A matriz $A^{(2)}$ indica a presença ou ausência de caminhos de comprimento 2. Nós devemos conjecturar que este resultado vale para potências e tamanhos de caminhos arbitrários.

Teorema Sobre Matrizes Booleanas de Adjacências e Alcançabilidade

Se A é a matriz booleana de adjacências para um grafo direcionado G com n vértices e sem arestas paralelas, então $A^{(m)}[i, j] = 1$ se, e somente se, houver um caminho de comprimento m do vértice n_i ao vértice n_j .

Demonstração: Realizaremos uma prova por indução em m . Já vimos que o resultado é verdadeiro para $m = 1$ (e $m = 2$). Suponha que $A^{(p)}[i, j] = 1$ se, e somente se, existir um caminho de comprimento p de n_i até n_j . Sabemos que

$$A^{(p+1)}[i, j] = \bigvee_{k=1}^n (A^{(p)}[i, k] \wedge a_{kj})$$

Esta expressão valerá 1 se, e somente se, pelo menos uma parcela valer 1, digamos $A^{(p)}[i, q] \wedge a_{qj} = 1$, ou $A^{(p)}[i, q] = 1$ e $a_{qj} = 1$. Isto será verdade se, e somente se, existir um caminho de comprimento p de n_i até n_q (pela hipótese de indução) e existir um caminho de comprimento 1 de n_q a n_j , que significa que existe um caminho de comprimento $p + 1$ de n_i a n_j .

PRÁTICA 26

Com relação ao grafo da Fig. 5.43, o que você espera que seja o valor do elemento 2,1 de $A^{(4)}$? Calcule $A^{(4)}$ e verifique este valor.

Se o vértice n_j é alcançável a partir do vértice n_i ele o será através de um caminho de algum tamanho. Este caminho será evidenciado por um 1 na posição i, j de A , $A^{(2)}$, $A^{(3)}$ etc, mas não podemos computar um número infinito de multiplicações de matrizes. Felizmente, o tamanho dos caminhos entre n_i e n_j pelos quais devemos procurar é limitado. Se houver n vértices no grafo, então qualquer caminho com $n + 1$ ou mais vértices deve ter pelo menos um vértice repetido. Isto decorre do Princípio da Casa do Pombo — existem n "caixas" (vértices distintos) nas quais estamos pondo mais de n objetos (os vértices de um caminho com n ou mais arestas). A Seção de um caminho que se encontre entre os vértices repetidos é um ciclo. Se $n_i \neq n_j$, então o ciclo pode ser eliminado para formar um caminho mais curto; assim, se existe um caminho entre dois vértices n_i e n_j este caminho tem comprimento máximo $n - 1$. Se $n_i = n_j$, então o ciclo poderá ser o caminho completo de n_i a n_i com comprimento máximo n ; mas, neste caso, poderíamos eliminar o ciclo (se notarmos que todo vértice pode ser considerado alcançável a partir de si próprio), mas mantemo-lo para mostrar que pode existir um caminho não-trivial entre n_i e n_i .

Conseqüentemente, seja $n_i = n_j$ ou $n_i \neq n_j$, nunca devemos procurar por um caminho de n_i até n_j com comprimento maior que n . Portanto, para determinar a alcançabilidade, devemos apenas considerar os elementos i, j em A , $A^{(2)}$, ..., $A^{(n)}$. Alternativamente, podemos definir a **matriz de alcançabilidade** R como

$$R = A \vee A^{(2)} \vee \dots \vee A^{(n)}$$

Então n_j é alcançável a partir de n_i se, e somente se, o elemento i, j em R for positivo.

Vimos agora como a alcançabilidade em um grafo pode ser expressa em termos da matriz de adjacências. Mas como a alcançabilidade pode ser expressa em termos da relação de adjacência que corresponde ao grafo?

Se p é a relação de adjacência para o grafo G , denotaremos por ρ^R a relação binária de alcançabilidade; isto é, $(n_i, n_j) \in \rho^R$ exatamente quando existe um caminho em G de n_i até n_j . Podemos então mostrar que ρ^R é o fecho transitivo de ρ . Lembremos, da definição de fecho de uma relação, que o fecho transitivo de p é uma relação transitiva que contém ρ e é um subconjunto de qualquer relação transitiva contendo ρ .

Para verificarmos que ρ^R é transitiva, seja (n_i, n_j) e (n_i, n_k) pertencentes a ρ^R . Então existe um caminho em G de n_i a n_j e um caminho em G de n_j a n_k . Portanto, existe um caminho em G de n_i a n_k e (n_i, n_k) pertencente a ρ^R . Para verificarmos que ρ^R contém ρ , seja (n_i, n_j) pertencente a ρ . Então existe uma aresta de n_i até n_j em G , o que significa que existe um caminho de comprimento 1 de n_i para n_j e (n_i, n_j) pertencente a ρ^R . Finalmente, suponha que a é qualquer relação transitiva nos vértices de G que contenha ρ e seja (n_i, n_j) pertencente a ρ^R . Isto significa que existe um caminho de n_i a n_j usando, digamos, os vértices $n_i, n_x, n_y, \dots, n_w, n_j$. Então existe uma aresta de cada vértice deste caminho para o seguinte e todos os pares ordenados $(n_i, n_x), (n_x, n_y), \dots, (n_w, n_j)$ pertencem a ρ e, portanto, todos eles pertencem a a . Como a é transitiva, isto implica que (n_i, n_j) pertence a a e ρ^R é um subconjunto de a . Portanto ρ^R é o fecho transitivo de ρ .

Para resumir, com relação às três representações equivalentes a seguir: relação de adjacência p , o grafo direcionado G e a matriz de adjacências A , temos que

$$(n_i, n_j) \text{ pertence ao fecho transitivo de } \rho \iff n_j \text{ alcançável a partir de } n_i \text{ em } G \iff R[i, j] = 1 \text{ onde } R = A \vee A^{(2)} \vee \dots \vee A^{(n)}$$

EXEMPLO 29 Seja G o grafo direcionado da Fig. 5.44; G tem cinco vértices.

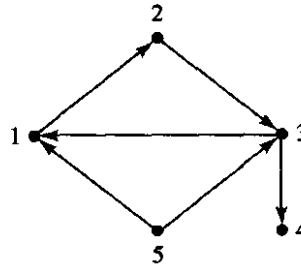


Figura 5.44

A matriz de adjacências A de G é

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

A relação de adjacência ρ é $\rho = \{(1,2), (2,3), (3,1), (3,4), (5,1), (5,3)\}$.

As potências sucessivas de A são

$$\mathbf{A}^{(2)} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix} \quad \mathbf{A}^{(3)} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$\mathbf{A}^{(4)} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix} \quad \mathbf{A}^{(5)} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

Estas matrizes indicam, por exemplo, que existe um caminho de comprimento 2 de 2 a 1 porque $A^{(2)}[2, 1] = 1$ (o caminho é 2-3-1) e existe um caminho de comprimento 4 de 5 a 3 porque $A^{(4)}[5, 3] = 1$ (o caminho é 5-3-1-2-3), mas não há caminho de comprimento 3 de 1 a 3 porque $A^{(3)}[1, 3] = 0$.

A matriz de alcançabilidade R é a soma booleana de A , $A^{(2)}$, $A^{(3)}$, $A^{(4)}$ e $A^{(5)}$:

$$\mathbf{R} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Os valores 1 na matriz R indicam que existem caminhos em G dos vértices 1, 2, 3 e 5 para todos os vértices, exceto para o vértice 5, mas não há caminho do vértice 4 para qualquer outro vértice, o que pode ser confirmado olhando a Fig. 5.44

Demonstramos que as entradas 1 em R indicam os pares ordenados que pertencem ao fecho transitivo de ρ . Este fecho transitivo terá, portanto, os seguintes pares ordenados:

{(1, 1), (1, 2), (1, 3), (1, 4), (2, 1), (2, 2), (2, 3), (2, 4),
(3, 1), (3, 2), (3, 3), (3, 4), (5, 1), (5, 2), (5, 3), (5, 4)}

Começando com p e seguindo o procedimento descrito no Cap. 4 para obtenção do fecho transitivo de uma relação, vemos que, para obter a transitividade, primeiro devemos incluir os pares (1,3), (2, 1), (2, 4), (3, 2), (5, 2) e (5, 4). Considerando o novo conjunto, vemos que precisamos incluir também os pares (1, 1), (1, 4), (2, 2) e (3, 3). O conjunto resultante de pares ordenados é transitivo (e está de acordo com o obtido acima). •

PRÁTICA 27

Calcule R para o grafo direcionado da Fig. 5.43. Quais informações a coluna 2 nos diz? •

No Cap. 4 prometemos um algoritmo melhor para encontrar o fecho transitivo de uma relação. Ei-lo: escreva a relação binária na forma de uma matriz de adjacências e calcule

$$\mathbf{R} = \mathbf{A} \vee \mathbf{A}^{(2)} \vee \dots \vee \mathbf{A}^{(n)}$$

Quanto trabalho é necessário para se executar este algoritmo? A expressão de R indica que precisam ser feitas operações booleanas em matrizes, mas as operações matriciais precisam, por sua vez, de operações de e ou booleanas em seus elementos. Usaremos, portanto, as operações booleanas e e ou para medir o trabalho necessário. Na Seção 4.4, vimos que a multiplicação ordinária de duas matrizes $n \times n$ requer $\Theta(n^3)$ multiplicações e adições; por um argumento análogo, a multiplicação de matrizes booleanas requer $\Theta(n^3)$ operações booleanas de e/ou. O algoritmo acima requer $n - 1$ multiplicações de matrizes booleanas (para encontrar os produtos $\mathbf{A}^{(2)}, \mathbf{A}^{(3)}, \dots, \mathbf{A}^{(n)}$ que entram no cálculo de R). O cálculo desses $n - 1$ produtos requer $(n - 1) \Theta(n^3) = \Theta(n^4)$ operações booleanas. Para calcular $\mathbf{C} \vee \mathbf{D}$, onde C e D são duas matrizes booleanas $n \times n$, precisamos de n^2 operações ou booleanas. Para calcular R , precisamos de $n - 1$ dessas operações matriciais, de forma que $(n - 1) n^2 = \Theta(n^3)$ operações ou booleanas são realizadas. A quantidade total de trabalho necessária é $\Theta(n^4) + \Theta(n^3) = \Theta(n^4)$.

A seguir, veremos um algoritmo mais eficiente para se computar o fecho transitivo de uma relação (ou a matriz de alcançabilidade de um grafo).

Algoritmo de Warshall

Para um grafo G com n vértices, o algoritmo de Warshall calcula uma série de $n + 1$ matrizes $M_0, M_1, M_2, \dots, M_n$. Para cada $k, 0 \leq k \leq n, M_k[i, j] = 1$ se, e somente se, existir um caminho em G de n_i até n_j cujos vértices interiores (i.e., os vértices que não são os extremos do caminho) forem apenas vértices do conjunto $\{n_1, n_2, \dots, n_k\}$.

Vamos examinar as "condições de contorno". Quando $k = 0$, o conjunto $\{n_1, n_2, \dots, n_0\}$ é o conjunto vazio, de forma $M_0[i, j] = 1$ se, e somente se, existir um caminho em G de n_i a n_j cujos pontos interiores pertençam ao conjunto vazio; isto é, que não haja vértices interiores. O caminho de n_i a n_j deve, então, consistir apenas em seus pontos extremos e uma aresta, de forma que n_i e n_j são adjacentes. Portanto, $M_0 = A$. A outra condição de contorno ocorre quando $k = n$. Então o conjunto $\{n_1, n_2, \dots, n_n\}$ consiste em todos os vértices em G , de modo que não há restrições sobre os vértices interiores no caminho e $M_n[i, j] = 1$ se, e somente se, existir um caminho de n_i até n_j o que significa que $M_n = R$.

Portanto, o algoritmo de Warshall começa em $A = M_0$ e computa sucessivamente $M_1, M_2, \dots, M_n = R$. Este cômputo pode ser definido indutivamente. O caso básico é fazer $M_0 = A$. Admita agora que M_k foi computado e considere como calcular M_{k+1} ou, mais especificamente, $M_{k+1}[i, j]$. Temos $M_{k+1}[i, j] = 1$ se, e somente se, existir um caminho de n_i a n_j cujos vértices interiores pertençam todos ao conjunto $\{n_1, n_2, \dots, n_{k+1}\}$. Isto pode acontecer de duas maneiras.

1. Todos os pontos interiores vêm de $\{n_1, n_2, \dots, n_k\}$, caso no qual $M_k[i, j] = 1$. Devemos, portanto, manter todos os elementos 1 de M_k em M_{k+1} .
2. O vértice n_{k+1} é um vértice interior. Podemos admitir que n_{k+1} é um vértice interior apenas uma vez, pois podemos eliminar os ciclos de um caminho. Então precisa existir um caminho de n_i até n_{k+1} cujos pontos interiores pertençam a $\{n_1, n_2, \dots, n_k\}$ e um caminho de n_{k+1} até n_j cujos pontos interiores pertençam a $\{n_1, n_2, \dots, n_k\}$. Isto significa que $M_k[i, k+1] = 1$ e $M_k[k+1, j] = 1$, o que quer dizer que $M_k[i, k+1] \wedge M_k[k+1, j] = 1$; esta condição pode ser verificada porque nossa hipótese é que M_k já foi calculada.

Na versão a seguir do algoritmo de Warshall, o valor inicial da matriz M é A . Cada passo do laço externo computa a matriz seguinte da série $M_1, M_2, \dots, M_n = R$.

ALGORITMO Warshall

```

procedure Warshall (var  $M$ : matriz booleana  $n \times n$ );
{inicialmente,  $M$  = matriz de adjacências de um grafo direcionado  $G$  sem arestas paralelas}

begin
  for  $k := 1$  to  $n$  do
    for  $i := 1$  to  $n$  do
      for  $j := 1$  to  $n$  do
         $M[i, j] := (M[i, j] \vee (M[i, k] \wedge M[k, j]));$ 
  end; {ao término,  $M$  = matriz de alcançabilidade de  $G$ }

```

EXEMPLO 30 As matrizes computadas pelo algoritmo de Warshall para o grafo do Exemplo 29 foram:

$$\begin{aligned}
 \mathbf{M}_0 &= \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix} & \mathbf{M}_1 &= \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix} \\
 \mathbf{M}_2 &= \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix} & \mathbf{M}_3 &= \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix}
 \end{aligned}$$

As matrizes M_4 e M_5 não conterão qualquer novo valor 1 porque não há caminhos de 4 ou 5 para qualquer outro vértice, de modo que expressões da forma $M[4, j]$ e $M[i, 5]$ são 0. Portanto, $M_3 = M_4 = M_5 = R$ como computado no Exemplo 29. Perceba, no entanto, que as matrizes calculadas pelo algoritmo de Warshall, exceto pelas A e R , não são iguais às potências de A usadas em nosso algoritmo anterior para o cálculo de R . •

Cada passada pelo laço externo do algoritmo de Warshall modifica a matriz que existia no fim do passo anterior. O algoritmo de Warshall não precisa de qualquer armazenamento extra para outras matrizes. Este é outro ponto que precisamos verificar. Como estamos modificando a matriz ao longo do processamento, durante qualquer passo do laço externo, algumas entradas pertencerão a M_{k+1} , enquanto que outras continuarão pertencendo a M_k . Especificamente, no passo $k+1$, poderíamos considerar $M[i, k+1] \wedge M[k+1, j]$ após estes valores já terem sido computados para o passo corrente e, portanto, representariam $M_{k+1}[i, k+1]$ e $M_{k+1}[k+1, j]$, ao invés dos valores de $M_k[i, k+1]$ e $M_k[k+1, j]$ que usamos em nossa justificativa para este algoritmo. (No Exemplo 30, por exemplo, no cálculo de $M_4[3, 2]$, usamos $M_3[3, 1]$ e $M_3[1, 2]$ e ambos já tinham sido computados no passo corrente.) Então será que existe algum caso no qual os valores de $M_{k+1}[i, k+1]$ e $M_{k+1}[k+1, j]$ são 1, de forma que incluímos um 1 em $M_{k+1}[i, j]$, quando os valores de $M_k[i, k+1]$ e $M_k[k+1, j]$ são 0? Não — se $M_{k+1}[i, k+1] = 1$, existe um caminho de n_i até n_{k+1} com vértices interiores pertencentes ao conjunto $\{n_1, n_2, \dots, n_{k+1}\}$. No entanto, como n_{k+1} é um ponto extremo, e os ciclos podem ser eliminados, deve existir ainda um caminho com vértices interiores pertencentes ao conjunto $\{n_1, n_2, \dots, n_k\}$ de forma que $M_k[i, k+1] = 1$. Um argumento análogo pode ser usado para $M_{k+1}[k+1, j]$.

PRÁTICA 28 Use o algoritmo de Warshall para calcular R para o grafo da Fig. 5.43. Compare sua resposta com a da Prática 27. •

Qual o trabalho demandado pelo algoritmo de Warshall medido pelo número de operações booleanas e/ou? O único comando de atribuição do algoritmo localiza-se no interior de um laço triplamente aninhado e será executado n^3 vezes. Cada execução do comando de atribuição precisa de uma operação **e** e outra **ou**; portanto, a quantidade total de trabalho é $2n^3 = \Theta(n^3)$. Lembremos de que nosso algoritmo anterior para obter R era $\Theta(n^4)$.

Revisão da Seção 5.3

Técnicas

- Dados os itens a seguir—relação de adjacência, grafo direcionado e matriz de adjacências—encontrar os outros dois
- Computar a matriz de alcançabilidade R de um grafo G (ou, equivalentemente, encontrar o fecho transitivo de uma relação de adjacência em G) através do uso da fórmula $R = A \vee A^{(2)} \vee \dots \vee A^{(n)}$ e através do uso da fórmula de Warshall

Idéias Principais

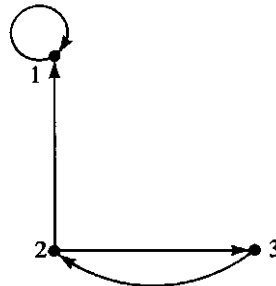
Existe uma correspondência um-para-um entre um grafo direcionado G sem arestas paralelas, a relação de adjacência em G e a matriz de adjacências de G (com respeito a alguma ordenação arbitrária de seus vértices).

A matriz de alcançabilidade de um grafo G também representa o fecho transitivo da relação de adjacência em G .

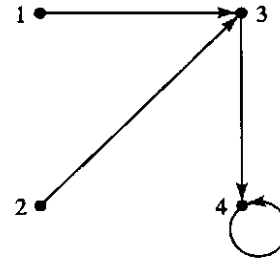
A matriz de alcançabilidade de um grafo pode ser computada com $\Theta(n^4)$ operações **e/ou** booleanas através da soma de potências da matriz de adjacências A ou com $\Theta(n^3)$ operações **e/ou** booleanas através do uso do algoritmo de Warshall.

Exercícios 5.3

- ★1. Encontre a matriz de adjacência e a relação de adjacência para o grafo da figura dada.



Exercício 1



Exercício 2

2. Encontre a matriz de adjacência e a relação de adjacência para o grafo da figura dada.

- ★3. Encontre o grafo direcionado e a relação de adjacência equivalente à matriz a seguir:

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

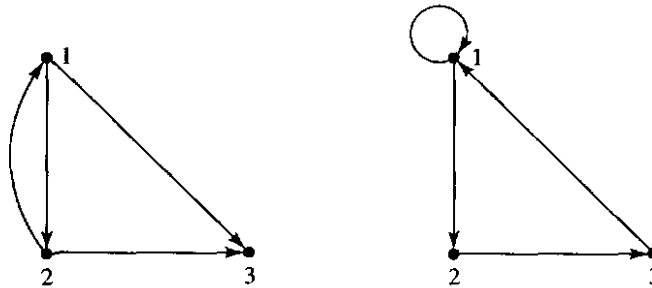
4. Encontre o grafo direcionado e a relação de adjacência equivalente à matriz a seguir:

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

5. Dada a relação de adjacência $\rho = \{(1,4), (1,5), (1,6), (6,2), (6,3), (6,5)\}$ no conjunto $N = \{1, 2, 3, 4, 5, 6\}$, encontre o grafo direcionado e a matriz de adjacência associados.
6. Dada a relação de adjacência $\rho = \{(2,1), (3,2), (3,3), (3,4), (4,5), (6,3), (6,6)\}$ no conjunto $N = \{1, 2, 3, 4, 5, 6\}$, encontre o grafo direcionado e a matriz de adjacência associados.
7. Descreva uma propriedade de um grafo direcionado cuja matriz de adjacência é simétrica.
- ★8. Descreva o grafo direcionado cuja matriz de adjacência tem todos os 1s na linha 1 e na coluna 1 e nas demais posições tem 0s.
9. Descreva o grafo direcionado cuja matriz de adjacência tem 1s nas posições $(i, i+1)$ para $1 \leq i \leq n-1$, um 1 na posição $(n, 1)$ e 0s nas demais posições.
10. Descreva uma propriedade de uma matriz de adjacência de um grafo cuja relação de adjacência seja anti-simétrica.
11. As relações de adjacência ρ e σ têm as matrizes de adjacência R e S associadas. Encontre as matrizes de adjacência associadas às relações $\rho \cup \sigma$ e $\rho \cap \sigma$.

$$R = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \quad S = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

- ★12. Os dois grafos direcionados da figura dada têm relações de adjacência ρ e σ . Desenhe os grafos associados às relações $\rho \cup \sigma$ e $\rho \cap \sigma$.



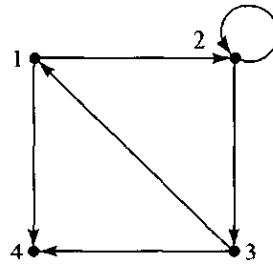
Exercício 12

13. Seja A a matriz

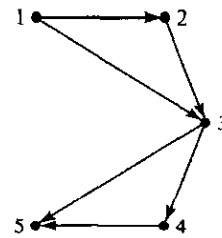
$$A = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

Encontre os produtos A^2 e $A^{(2)}$.

14. A definição de *grafo conexo* pode ser estendida para grafos direcionados. Descreva a matriz de alcançabilidade R para um grafo direcionado conexo.
- ★15. Para o grafo da figura dada, escreva a matriz de alcançabilidade R através de uma inspeção do grafo.



Exercício 15



Exercício 16

16. Para o grafo da figura dada, escreva a matriz de alcançabilidade R através de uma inspeção do grafo.

Nos Exercícios 17 a 22, calcule a matriz de alcançabilidade R aplicando a fórmula $R = A \vee A^{(2)} \vee \dots \vee A^{(n)}$

17. Exercício 1

18. Exercício 2

★19. Exercício 3

20. Exercício 4

21. Exercício 5

22. Exercício 6

Nos Exercícios 23 a 28, calcule a matriz de alcançabilidade R , usando o algoritmo de Warshall.

★23. Exercício 1

24. Exercício 2

25. Exercício 3

26. Exercício 4

★27. Exercício 5

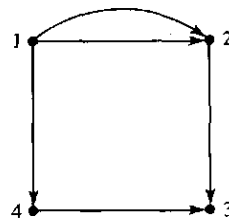
28. Exercício 6

Os Exercícios 29 a 32 usam a multiplicação convencional de matrizes para obterem informações sobre um grafo.

★29. Seja G um grafo direcionado, possivelmente com arestas paralelas, e seja A sua matriz de adjacência. Então A pode não ser uma matriz booleana. Demonstre que a posição i, j da matriz A^2 contém o número de caminhos de comprimento 2 do vértice i ao vértice j .

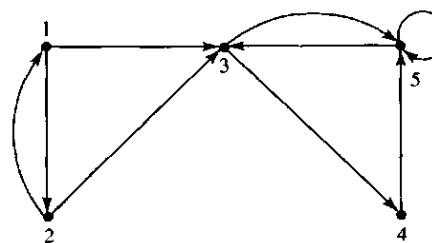
30. Seja A a matriz de adjacência de um grafo direcionado G , possivelmente com arestas paralelas. Demonstre que a entrada i, j da matriz A^n contém o número de caminhos de comprimento n do vértice i ao vértice j .

31. Para o grafo G da figura dada, conte o número de caminhos de comprimento 2 do vértice 1 ao vértice 3. Verifique, calculando A^2 .



Exercício 31

32. Para o grafo G da figura dada, conte o número de caminhos de comprimento 4 do vértice 1 ao vértice 5. Verifique, calculando A^4 .



Exercício 32

33. Seja p uma relação binária definida no conjunto $\{0, \pm 1, \pm 2, \pm 4, \pm 16\}$ por $x p y \leftrightarrow y = x^2$. Desenhe o grafo direcionado associado.

Seção 5.4 Árvores de Decisão e Códigos de Huffman

Podemos usar árvores para representar as ações de certos algoritmos, ainda que estes algoritmos não sejam descritos em termos de árvores. Em outros casos, a melhor maneira de se descrever um algoritmo é explicar seu efeito através de palavras no lugar de uma estrutura de árvore.

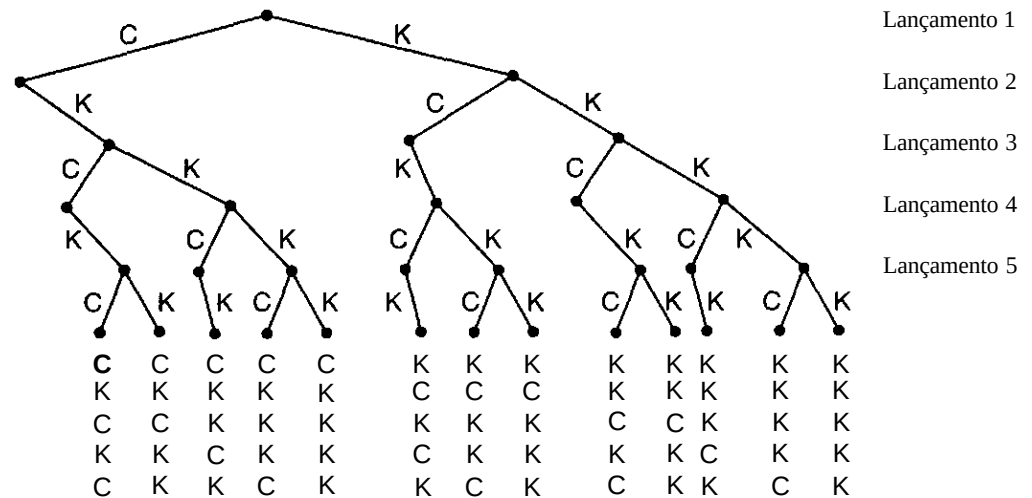


Figura 5.45

Árvores de Decisão

No Cap. 3, usamos árvores de decisão para resolver problemas de contagem. A Fig. 5.45 mostra a árvore usada no Exemplo 41 do Cap. 3 para representar as várias possibilidades de cinco lançamentos de moedas sob a restrição de que duas caras seguidas não possam ocorrer. Cada vértice interno da árvore representa uma ação (um lançamento de moeda) e as arestas para os filhos dos vértices internos representam os resultados dessa ação (cara ou coroa). As folhas da árvore representam os resultados finais, isto é, as diferentes maneiras de se obter cinco lançamentos.

Definição: Árvore de Decisão

Uma **árvore de decisão** é uma árvore na qual os vértices internos representam ações, as arestas representam os resultados de uma ação e as folhas representam os resultados finais.

Busca

Diversos algoritmos funcionam realizando comparações sucessivas entre dois elementos. Um algoritmo de busca, por exemplo, ou encontra um elemento x em uma lista ou determina que x não pertence à lista. Este tipo de algoritmo normalmente funciona realizando comparações sucessivas de x com os elementos da lista. Já vimos dois algoritmos deste tipo: busca sequencial e busca binária. Podemos modelar a atividade desses algoritmos através de árvores de decisão. Os vértices representam as ações de comparação de x com os elementos da lista, onde a comparação de x com o i -ésimo elemento da lista é denotada por $x:L(i)$.

A busca sequencial apenas distingue dois possíveis resultados da comparação de x com $L(i)$. Se $x = L(i)$, o algoritmo termina porque x foi encontrado na lista. Se $x \neq L(i)$, a próxima comparação realizada é $x:L(i+1)$, independente de x ser menor ou maior que $L(i)$. As folhas desta árvore de decisão correspondem aos resultados finais, onde ou x é um dos elementos da lista ou x não está na lista.

EXEMPLO 31 A Fig. 5.46 mostra a árvore de decisão de um algoritmo de busca seqüencial aplicado a uma lista de cinco elementos.

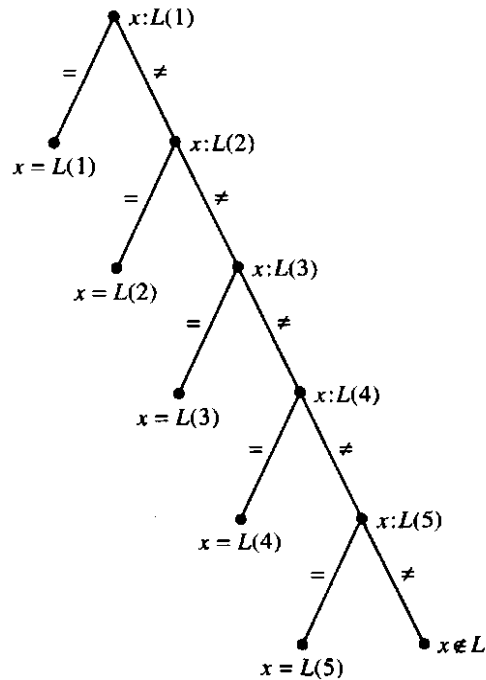


Figura 5.46

Através da árvore de decisão de um dado algoritmo, podemos ver que o número de comparações necessárias para qualquer resultado em particular (folha da árvore) é o número de vértices internos da raiz à folha. Este número é igual ao comprimento do caminho da raiz à folha. O pior caso, isto é, o número máximo de comparações, é o maior comprimento de qualquer caminho deste tipo, que é a altura da árvore. Como toda árvore de decisão para buscas seqüenciais se parece com a árvore da Fig. 5.46, está claro que a altura da árvore de decisão para algoritmos deste tipo, para uma lista de n elementos, é n . Isto confirma o que já sabíamos, isto é, que, no pior caso, uma busca seqüencial em uma lista de n elementos requer n comparações.

A árvore de decisão para o algoritmo de busca binária é mais interessante. A busca binária reconhece três possíveis resultados em cada comparação:

- $x = L(i)$: o algoritmo termina, x foi encontrado
- $x < L(i)$: o algoritmo continua na metade esquerda da lista
- $x > L(i)$: o algoritmo continua na metade direita da lista

Seguiremos o costume, e não escreveremos as folhas que correspondem à "ramificação do meio", isto é a igualdade $x = L(i)$. Se $x < L(i)$, a próxima comparação realizada pelo algoritmo é encontrada no filho à esquerda deste nó; se $x > L(i)$, a próxima comparação realizada pelo algoritmo é representada no filho à direita. Se não houver filhos, o algoritmo termina porque x não está na lista. A árvore que acabamos de descrever é uma árvore binária cujas folhas representam todos os resultados possíveis onde x não está na lista. Existem muito mais folhas referentes a falhas do que na busca seqüencial; isto se deve ao fato de a busca binária indicar porque x não está na lista (por exemplo, $x < L(1)$ ou $L(1) < x < L(2)$).

EXEMPLO 32 A Fig. 5.47 mostra a árvore de decisão para o algoritmo de busca binária aplicado a uma lista de oito elementos.

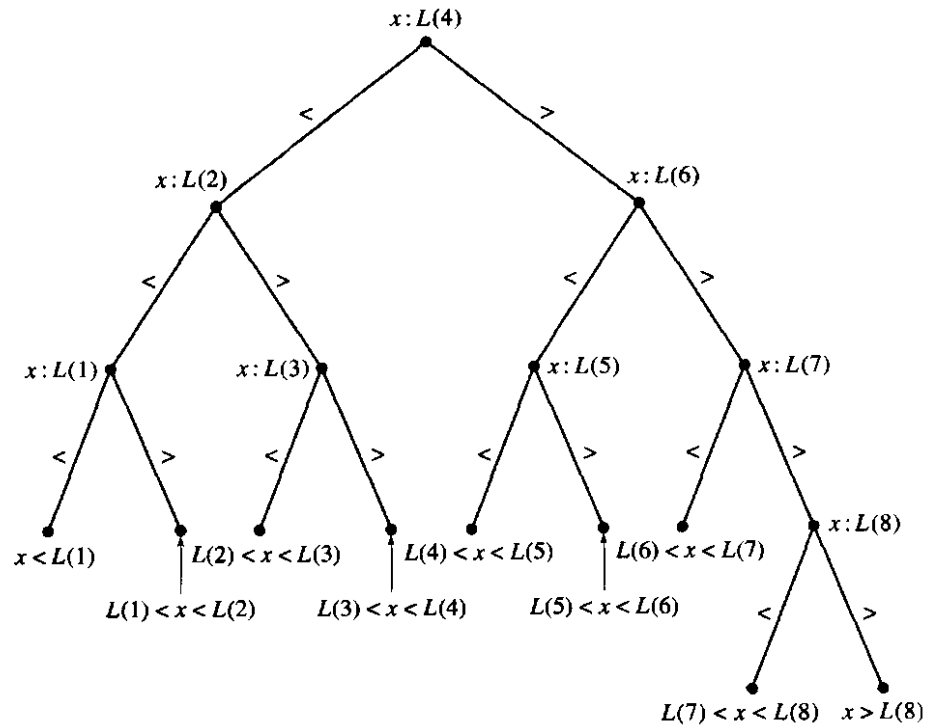


Figura 5.47

O pior caso, isto é, o número máximo de comparações, será novamente a altura da árvore, que é 4 na Fig. 5.47. No Cap. 2, resolvemos uma relação de recorrência para obter o comportamento do pior caso de uma busca binária onde n é uma potência de 2 e encontramos como resultado $1 + \log n$ (lembre-se de que estamos usando logaritmos na base 2). Perceba que $1 + \log 8 = 4$, de forma que a altura da árvore de decisão confirma nosso resultado anterior. A restrição de n ser uma potência de 2 tornou a aritmética da solução da relação de recorrência mais simples. Se n não for uma potência de 2, então a altura da árvore será dada pela expressão $1 + \lfloor \log n \rfloor$.

PRÁTICA 29

- Desenhe a árvore de decisão para o algoritmo de busca binária em uma lista com cinco elementos.
- Encontre a altura da árvore e compare com $1 + \lfloor \log 5 \rfloor$.

Limites Inferiores da Busca

Usamos árvores de decisão para representar as ações de dois algoritmos particulares de busca. Este tipo de árvore pode ser usado para representar as ações de qualquer algoritmo que resolva o problema da busca através de comparações do elemento procurado com os elementos da lista. Os vértices internos de uma árvore deste tipo representam as comparações feitas, e a altura da árvore indica o número de comparações no pior dentre todos os casos. O que podemos dizer sobre uma árvore quando não sabemos as peculiaridades do algoritmo em questão? Podemos dizer que x deve ser comparado com cada elemento da lista pelo menos em algum caso (talvez em mais de um caso, se o algoritmo for um tanto estúpido). Se existir um elemento da lista que não sendo nunca comparado a x , o algoritmo não pode determinar se o elemento é igual a x e, portanto, não pode decidir com certeza se x pertence ou não à lista. As comparações são os vértices internos nas árvores de decisão. Portanto, se m é o número de vértices internos na árvore de decisão T_I para qualquer algoritmo de busca que atue em uma lista de n elementos, então $m \geq n$.

Antes de prosseguirmos com as árvores de decisão, precisamos de mais alguns fatos sobre as árvores binárias em geral. O número de vértices em cada nível em uma árvore binária completa segue uma progressão geométrica: 1 vértice no nível 0, 2^1 vértices no nível 1, 2^2 vértices no nível 2, e assim por diante. Em uma árvore binária completa de altura h , o número total de vértices é, portanto,

$$1 + 2 + 2^2 + 2^3 + \dots + 2^h = 2^{h+1} - 1$$

(veja o Exercício 17 da Seção 2.2). Uma árvore binária completa tem o número máximo de vértices para qualquer árvore binária de uma dada altura. Isto nos dá o fato número 1:

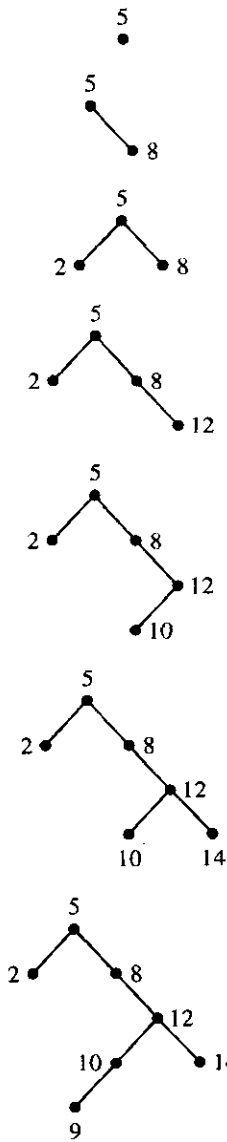


Figura 5.48

1. Qualquer árvore binária de altura h tem no máximo $2^{h+1} - 1$ vértices.

O fato número 2, que será demonstrado logo a seguir, é:

2. Qualquer árvore binária com m vértices tem altura $\geq \lfloor \log m \rfloor$.

Para demonstrarmos o fato 2, usaremos uma demonstração por contradição. Suponha que uma árvore binária tem m vértices e altura $h < \lfloor \log m \rfloor$. Então $h \leq \lfloor \log m \rfloor - 1$. Do fato 1,

$$m \leq 2^{h+1} - 1 \leq 2^{(\lfloor \log m \rfloor - 1) + 1} - 1 = 2^{\lfloor \log m \rfloor} - 1 \leq 2^{\log m} - 1 = m - 1$$

ou

$$m \leq m - 1,$$

uma contradição. Portanto, $h \geq \lfloor \log m \rfloor$.

Voltemos às árvores de decisão. Tiremos temporariamente as folhas da árvore T_1 (com m vértices internos) para criar uma nova árvore T_2 com m vértices, $m \geq n$. Pelo fato 2, T_2 tem altura $h \geq \lfloor \log m \rfloor \geq \lfloor \log n \rfloor$. Portanto, a árvore T_1 tem altura $\geq \lfloor \log n \rfloor + 1$. Como a altura de uma árvore de decisão nos dá o número de comparações no pior caso, podemos enunciar o seguinte teorema:

Teorema Sobre o Limite Inferior de Uma Busca

Qualquer algoritmo que resolva o problema de busca em uma lista de n elementos pela comparação do elemento procurado x com os elementos da lista precisa realizar pelo menos $\lfloor \log n \rfloor + 1$ comparações no pior caso.

Isto nos dá um limite inferior para o número de comparações necessárias no pior caso para qualquer algoritmo de busca que use comparações para resolver o problema. Uma vez que a busca binária não realiza mais trabalho do que o mínimo necessário, a busca binária é dita um **algoritmo ótimo** com respeito ao comportamento de seu pior caso.

Busca em uma Árvore Binária

O algoritmo de busca binária requer que os dados já estejam ordenados. Outro algoritmo de busca, chamado de *busca em uma árvore binária*, pode ser usado em dados que estejam organizados em uma *árvore binária de busca*. Uma árvore binária é construída a partir de itens de dados em uma ordem arbitrária. O primeiro elemento torna-se a raiz da árvore. Os elementos a seguir são inseridos em função de comparações com os elementos já existentes, começando sempre pela raiz. Se o novo elemento for menor que o elemento de um vértice, o próximo vértice testado é o filho à esquerda; do contrário, passa-se ao filho da direita. Quando não houver mais vértices-filho, o novo elemento torna-se um.

EXEMPLO 33

Os itens de dados

5, 8, 2, 12, 10, 14, 9

são organizados em uma árvore binária de busca. A Fig. 5.48 mostra os sucessivos estados da construção da árvore.

Uma **árvore binária de busca**, da forma que é construída, tem a propriedade de que o valor em qualquer vértice é maior que todos os valores de sua subárvore à esquerda (a subárvore cuja raiz é seu filho à esquerda) e menor que todos os valores de sua subárvore à direita. Uma **busca em uma árvore binária** compara o elemento x com sucessivos vértices começando pela raiz. Se x for igual ao elemento do vértice, o algoritmo termina; se x for menor do que o elemento, o filho à esquerda é o próximo a ser testado; se x for maior que o elemento, então passa-se ao filho à direita. Se não houver mais filhos, o algoritmo termina indicando que x não pertence à lista. Portanto, a árvore binária de busca, exceto por suas folhas, é a árvore de decisão de um algoritmo de busca binária. (Este é um caso no qual o algoritmo propriamente dito é descrito em termos de uma árvore.) O número de comparações para o pior caso é igual à altura da árvore mais 1 (para as folhas que faltam). No entanto, uma árvore binária de busca para um determinado conjunto de dados não é única; a árvore depende da ordem na qual os elementos são inseridos nela.

EXEMPLO 34 Os dados do Exemplo 33 entrados na ordem

9, 12, 10, 5, 8, 2, 14

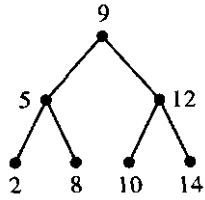


Figura 5.49

produzem a árvore binária da Fig. 5.49

As ações realizadas em uma busca em uma árvore binária certamente remontam o algoritmo "regular" de busca binária; em ambos os casos o procedimento é realizar a comparação e, se não for igual, passar à esquerda ou à direita (na árvore, se for uma busca em uma árvore binária, ou na lista, se for uma busca binária). É possível ordenar os dados para uma busca em árvore binária, de tal forma que a árvore de busca construída seja a mesma da árvore de decisão do algoritmo de busca binária dos mesmos dados quando ordenados. Isto é ilustrado no Exemplo 34 (perceba que a árvore não foi construída a partir dos dados ordenados).

A altura de uma árvore binária de busca para um conjunto de dados pode variar. A altura da árvore na Figs. 5.48 é 4, enquanto que na Fig. 5.49 é 2. Portanto, o número de comparações para o pior caso para a busca de um elemento também pode variar. O processo de construção da árvore pode ser modificado de modo que se tenha uma árvore mais "balanceada", isto é, baixa e larga, no lugar de alta e fina; este tipo de intervenção reduz a altura da árvore e, portanto, o tempo de busca. Sabemos, no entanto, do teorema sobre o limite inferior da busca que uma certa quantidade mínima de trabalho é necessária, independentemente do quanto engenhoso seja o processo de construção da árvore.

PRÁTICA 30

a. Construa a árvore binária de busca para os dados do Exemplo 33 entrados na ordem

12, 9, 14, 5, 10, 8, 2

b. Qual é a altura da árvore?

Ordenação

As árvores de decisão também podem modelar algoritmos que ordenem listas de elementos através de uma sequência de comparações entre dois elementos da lista. Os vértices internos deste tipo de árvore de decisão são rotulados $L(i):L(j)$ a fim de indicarem uma comparação entre os elementos i e j da lista. Para simplificar nossa discussão, vamos assumir que a lista não contenha elementos duplicados. Então o resultado da comparação será $L(i) < L(j)$ ou $L(i) > L(j)$. Se $L(i) < L(j)$, então o algoritmo prossegue pelo filho à direita. Se não houver filhos, o algoritmo termina porque a ordenação foi terminada. A árvore é uma árvore binária e suas folhas representam os resultados finais, isto é, as diversas ordenações possíveis.

EXEMPLO 35

A Fig. 5.50 mostra a árvore de decisão para um algoritmo de ordenação aplicado a uma lista de três elementos. Este algoritmo não é muito astuto, porque ignora a transitividade da relação $<$ e, portanto, realiza algumas operações redundantes. As folhas da árvore indicam os diversos resultados finais, incluindo diversos casos (marcados com um X) que resultam de contradições das comparações realizadas.

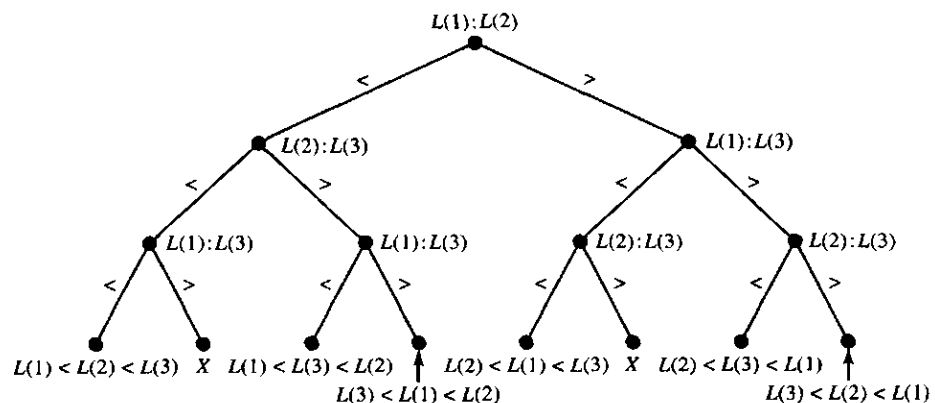


Figura 5.50

PRÁTICA 31

Desenhe a árvore de decisão obtida se alterarmos o algoritmo do Exemplo 35 a fim de evitar comparações desnecessárias.

Um argumento na forma de uma árvore de decisão também pode ser usado para estabelecer um limite inferior, no pior caso, para o número de comparações necessárias para a ordenação de uma lista de n elementos. Da mesma forma que fizemos para o problema de busca, vamos ver o que podemos dizer sobre uma árvore de decisão para a ordenação baseada em comparações, independente do algoritmo que estivermos usando. As folhas dessas árvores representam os resultados finais, isto é, as várias ordenações dos n elementos. Existem $n!$ arrumações deste tipo; assim, se p for o número de folhas da árvore de decisão, então $p \geq n!$. O pior caso será igual à altura da árvore. Mas também é verdade que, se a árvore tem altura h , logicamente $p \leq 2^h$ (Exercício 56 da Seção 5.1). Tomando o logaritmo na base 2 de ambos os lados dessa desigualdade, temos $\log p \leq h$ ou, como h é um inteiro, $h = \lceil \log p \rceil$. Finalmente, obtemos

$$h = \lceil \log p \rceil \geq \lceil \log n! \rceil$$

O que prova o teorema a seguir.

Teorema Sobre o Limite Inferior de uma Ordenação

Qualquer algoritmo que ordena uma lista de n elementos através de comparações de pares de elementos da lista realiza pelo menos $\lceil \log n! \rceil$ comparações em seu pior caso.

Códigos de Huffman

Dados na forma de caracteres, consistem em letras do alfabeto (tanto as maiúsculas quanto as minúsculas), símbolos de pontuação e outros símbolos do teclado, tais como @ e %. Os computadores armazenam cada caracter de forma binária, como uma seqüência de 0s e 1s. O método mais comum é fixar algum comprimento n de forma que 2^n seja tão grande quanto o número de caracteres distintos e então representar cada caracter como uma seqüência particular de n bits. Cada caracter precisará, então, da mesma quantidade de memória para ser armazenado.

Suponha que uma coleção de dados na forma de caracteres que precisa ser armazenada em um arquivo de forma binária seja grande o suficiente para que a quantidade de memória a ser gasta seja relevante. Um esquema de codificação diferente do descrito acima poderia usar um número variável de bits para representar os caracteres e armazenar os que tiverem maior freqüência como seqüências com menos de n bits. A fim de armazenar todos os caracteres distintos, algumas seqüências precisarão ser maiores, mas se essas seqüências maiores forem usadas para os caracteres que ocorrem com menor freqüência, a quantidade total de memória necessária pode ser menor. Estudaremos agora este esquema de **compressão de dados** ou **compactação de dados**, pois são mais bem descritos como uma série de ações tomadas em árvores binárias.

EXEMPLO 36

Como um exemplo trivial, suponha uma coleção de dados que contenha 50.000 ocorrências dos seis caracteres a , c , g , k , p e $?$, os quais ocorrem segundo as seguintes freqüências percentuais:

Caracter	a	c	g	k	p	$?$
Freqüência	48	9	12	4	17	10

Como devemos ser capazes de armazenar seis diferentes caracteres, o esquema de tamanho fixo precisa de três bits para cada caracter ($2^3 = 8 \geq 6$). O espaço total para armazenamento é, então, $50.000 * 3 = 150.000$ bits. Suponhamos agora que usamos o seguinte esquema de codificação:

Caracter	a	c	g	k	p	$?$
Codificação	0	1101	101	1100	111	100

Então o espaço necessário (em número de bits) será

$$50.000 (0,48 * 1 + 0,09 * 4 + 0,12 * 3 + 0,04 * 4 + 0,17 * 3 + 0,10 * 3) = 108.500$$

que é aproximadamente dois terços do que era necessário pelo método anterior.

No método de armazenamento por tamanho fixo com n bits para cada caracter, a cadeia grande de bits com os dados codificados pode ser quebrada a fim de obter os códigos de cada caracter simplesmente tomando n bits de cada vez. Já na codificação com tamanho variável, é preciso que haja uma forma de decidir quando a sequência de um caracter termina e começa a sequência do caracter seguinte.

PRÁTICA 32 Usando o código de tamanho variável do Exemplo 36, decodifique cada uma das seguintes cadeias:

- a. 1111111010100
- b. 1101010101100
- c. 100110001101100

Na Prática 32, as cadeias podem ser quebradas em representações de caracteres de uma única maneira. À medida que um novo bit é considerado, as possibilidades de quais caracteres podem estar sendo representados vão sendo limitadas até que o caracter esteja unicamente identificado pelo último bit de sua representação. Não há a necessidade de adivinhar o que o caracter é, e então retroceder no algoritmo, caso a adivinhação se mostre errada. Este recurso de decodificação única sem inícios falsos e retrocessos ocorre porque este código é um exemplo de **código prefixo**. Em um código prefixo, o código para qualquer caracter nunca é o prefixo do código de qualquer outro caracter. (Na verdade, um código prefixo é um código "antiprefixo"!)

EXEMPLO 37 Considere o código

Caracter	a	b	c
Codificação	01	101	011

que *não* é um código prefixo. Dada a cadeia 01101, ela pode representar ab (01-101) ou ca (011-01). Além disso, ao processar a cadeia 011011 bit a bit, como um computador faria, a codificação poderia começar com um ab (01-101), e então encontraríamos um erro no último bit. Neste caso o processo teria que ser repetido desde o primeiro bit a fim de reconhecermos a sequência cc (011-011).

Em nossa abordagem para os códigos prefixos, devemos construir árvores binárias com os caracteres como folhas. Uma vez que tenhamos construído uma árvore, podemos atribuir um código binário para cada caracter seguindo o caminho desde a raiz até a folha, usando 0s para cada desvio a esquerda e 1s para os desvios à direita. Como as folhas não precedem qualquer outras folhas em caminhos a partir da raiz, o código será um código prefixo. A árvore binária para o código do Exemplo 36 é mostrada na Fig. 5.51.

Suponha que existe uma árvore T de códigos, com as folhas representando os caracteres. Para qualquer folha i , sua profundidade $d(i)$ em T é igual ao número de bits no código que representam o caracter correspondente. Seja $f(i)$ a frequência de percentagem deste caracter nos dados a ser armazenados, e seja S o número total de caracteres a ser armazenados. Então, como no Exemplo 36, o número total de bits necessários é dado pela expressão

$$S * \left[\sum_{\text{todas folhas } i} (d(i)f(i)) \right]$$

Tentaremos construir uma árvore binária T ótima para a qual a expressão

$$E(T) = \sum_{\text{todas folhas } i} (d(i)f(i))$$

seja um mínimo e, portanto, o tamanho do arquivo seja o menor possível.

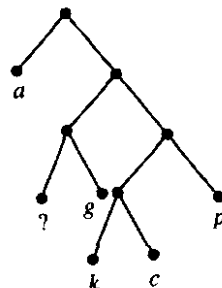


Figura 5.51

Este processo pode ser feito por tentativa e erro, pois há um número finito de caracteres, e portanto um número finito de maneiras de se construir a árvore atribuindo os caracteres às folhas. No entanto, este número finito torna-se rapidamente grande! Ao invés disto, vamos usar um algoritmo conhecido como **codificação de Huffman**.

Algoritmo de Codificação de Huffman

Suponha, então, que temos m caracteres e sabemos a frequência percentual de cada caracter. O algoritmo para construir a árvore funciona gerenciando uma lista L de vértices que são raízes de árvores binárias. Inicialmente, L conterá m raízes, cada qual rotulada com a frequência de um dos caracteres; as raízes serão ordenadas de acordo com a frequência crescente e cada qual não terá filhos.

Uma descrição em pseudocódigo deste algoritmo é a seguinte:

ALGORITMO ÁrvoreDeHuffman

procedure *ÁrvoreDeHuffman* (**var** *L*: lista de vértices);
 {cada vértice em *L* tem uma frequência associada, e *L* é ordenado pela sua frequência crescente; este procedimento constrói a árvore de Huffman}

begin

for $i := 1$ to $m - 1$ do

begin

cria o novo vértice z

sejam x, v os primeiros dois vértices de L {vértices de menores frequências)

$$f(z):=f(x)+f(y);$$
insira o vértice z em L , mantendo a ordenação;

filho à esquerda de z : = vértice x ;

filho à direita de $z :=$ vértice y ;

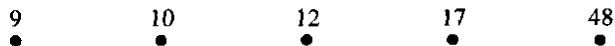
retire os vértices x e y de L ; {não são mais raízes}

end;

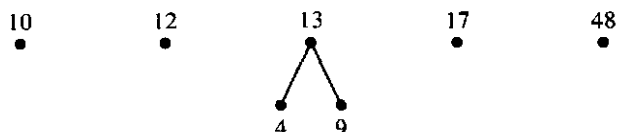
end;

Quando este procedimento termina, L contém apenas um vértice, que é a raiz da árvore binária final. Os códigos podem, então, ser atribuídos a cada folha da árvore, seguindo o caminho da raiz até a folha enquanto se acumulam 0s para os desvios à esquerda e 1s para os desvios à direita. Pela forma que a árvore é construída, todo vértice interno terá exatamente dois filhos. Está claro que este é outro algoritmo que é mais bem descrito em termos de uma árvore, ainda que o problema original (compactação de dados) não tenha nada a ver com árvores.

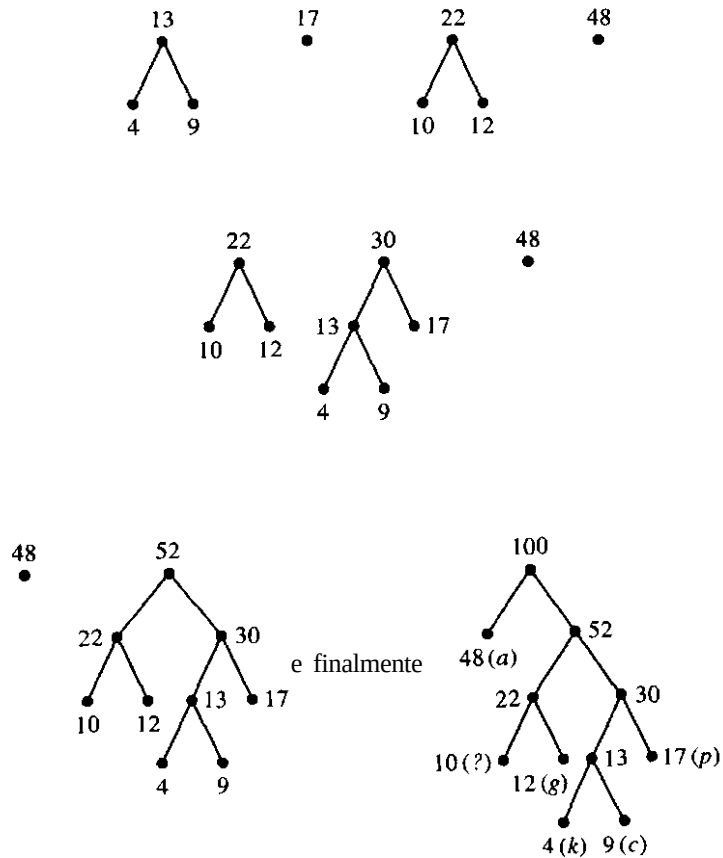
EXEMPLO 38 Usamos o algoritmo *ÁrvoreDeHuffman* para construir a árvore da Fig. 5.51, que é baseada nos dados do Exemplo 36. L contém, inicialmente, seis vértices ordenados pela frequência:



Seguindo o algoritmo, entramos no laço **for** pela primeira vez. Os vértices x e y são os com frequência 4 e 9, respectivamente. Um novo vértice z com frequência $4 + 9 = 13$ é criado e inserido ordenadamente em L , com o vértice x com o seu filho esquerdo, e o vértice y como seu filho direito. A nova lista L fica, então,



Este processo é repetido mais quatro vezes. A lista L resultante após cada passo é mostrada abaixo:



Neste ponto, a árvore está completa e os códigos podem ser atribuídos.

PRÁTICA 33 Construa a árvore de Huffman para os caracteres e frequências dados:

Caracter	w	q	h	e
Frequência	10	12	20	58

PRÁTICA 34 Encontre os códigos de Huffman para os caracteres da Prática 33.

Justificativa

Apesar do algoritmo para construção da árvore de Huffman T ser bastante simples para descrever, precisamos justificar que ele nos fornece o valor mínimo para $E(T)$.

Primeiro, se tivermos uma árvore ótima T para m caracteres, os vértices com menores frequências podem sempre ser entendidos como os vértices à esquerda e à direita de algum outro vértice. Para demonstrar isto, chamemos os dois vértices com as menores frequências de x e y . Se x e y não forem irmãos na árvore, então encontre dois irmãos p e q no menor nível da árvore, e considere o caso no qual x e y não estão neste nível (veja a Fig. 5.52a). Como $f(x)$ é um dos dois menores valores, sabemos que $f(x) \leq f(p)$. Se $f(x) < f(p)$, então a troca de x e p na árvore resultaria em uma nova árvore T' com $E(T') < E(T)$ (veja a Fig. 5.52b) — a folha de maior frequência está agora em um nível mais acima, contradizendo o fato de T ser ótima. Portanto $f(x) = f(p)$ e x e p podem ser trocados na árvore sem alterar $E(T)$. Analogamente, y e q podem ser trocados, resultando na Fig. 5.52c, na qual x e y são irmãos. Se x ou y estão no mesmo nível que p e q , eles certamente podem ser trocados com p ou q sem afetar $E(T)$ (Fig. 5.52d).

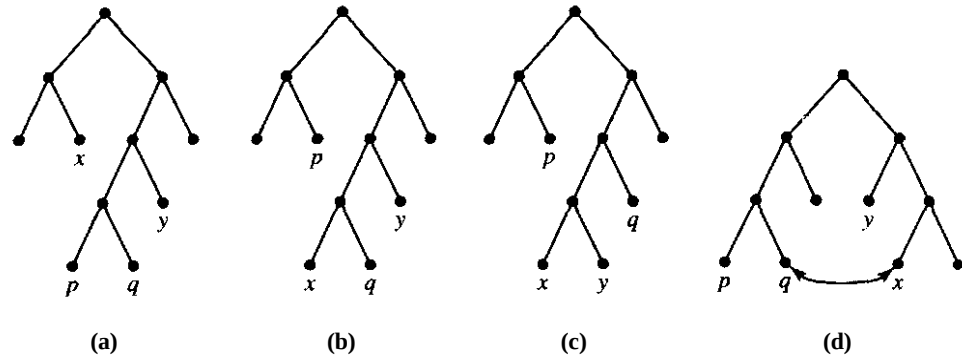


Figura 5.52

Sejam, novamente, $f(x)$ e $f(y)$ as menores frequências, e suponha que temos uma árvore T que é ótima para as outras frequências juntamente com $f(x) + f(y)$ (Fig. 5.53a). Esta soma será a frequência de uma folha; crie uma árvore T' que tenha este vértice como vértice interior com filhos x e y , tendo frequências $f(x)$ e $f(y)$ (Fig. 5.53b). T' será ótima para as frequências $f(x)$, $f(y)$ e as demais. A demonstração deste fato começa com alguma árvore ótima B' para frequências $f(x)$, $f(y)$ e as demais frequências. Sabemos que uma árvore deste tipo existe (uma vez que podemos encontrá-la através de tentativas e erros) e, do parágrafo anterior, podemos admitir que x e y são irmãos em B' (Fig. 5.53c). Criemos agora uma árvore B , tirando os vértices x e y de B' e atribuindo a frequência $f(x) + f(y)$ ao vértice-pai de x e y , agora transformado em uma folha (Fig. 5.53d). Como T é ótima para as outras frequências juntamente com $f(x) + f(y)$, temos

$$E(T) \leq E(B) \quad (1)$$

Mas a diferença entre $E(B)$ e $E(B')$ é uma aresta para x e outra para y , isto é, $E(B') = E(B) + f(x) + f(y)$. Analogamente, temos $E(T') = E(T) + f(x) + f(y)$. Portanto, se somarmos $f(x) + f(y)$ a ambos os lados de (1), obtemos

$$E(T') \leq E(B') \quad (2)$$

Como B' é ótima, não pode ocorrer $E(T') < E(B')$, de forma que $E(T') = E(B')$, e T é ótima.

Finalmente, uma árvore com um único vértice cuja frequência seja a soma de todas as frequências é trivialmente ótima para esta soma. Podemos partir esta soma e criar filhos, de forma que terminemos com a árvore de Huffman. Pelo parágrafo anterior, cada árvore, incluindo a árvore de Huffman final, é ótima.

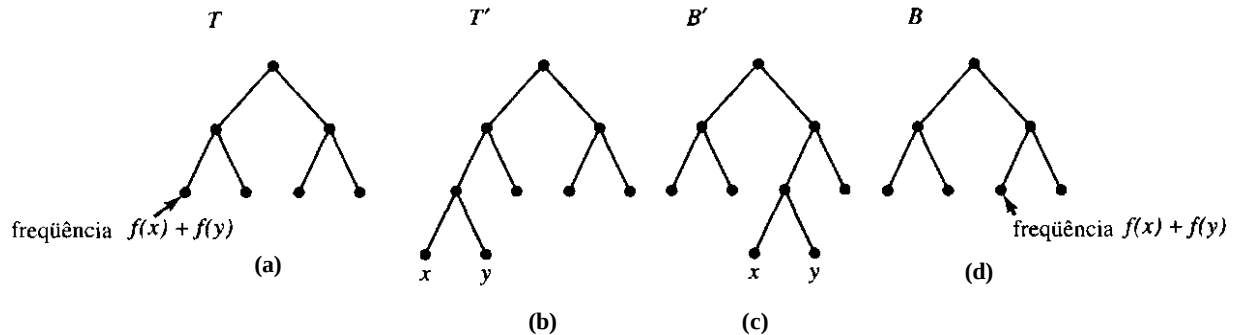


Figura 5.53

EXEMPLO 39 Se aplicarmos o processo que preserva a otimalidade da árvore da Fig. 5.51, poderíamos começar com um único vértice com frequência 100, e "crescer" esta árvore, como mostrado na Fig. 5.54.

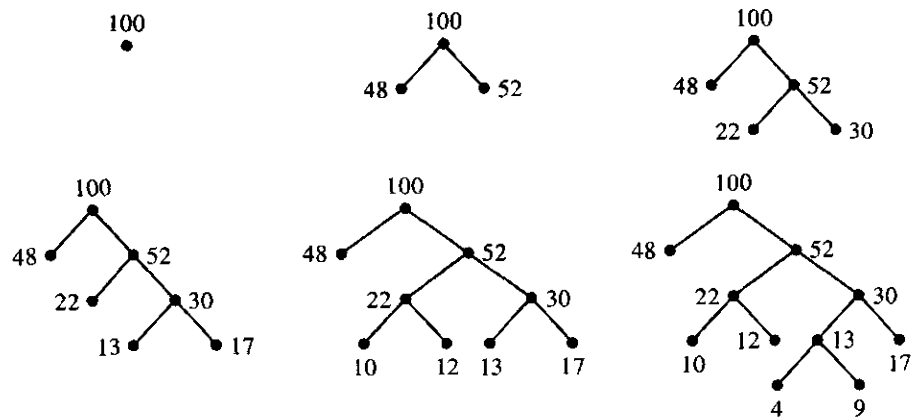


Figura 5.54

Revisão da Seção 5.4

Técnicas

- Desenhar árvores de decisão para algoritmos de busca e de ordenação em listas de n elementos
- Criar uma árvore binária de busca
- Encontrar códigos de Huffman, dado um conjunto de caracteres e suas frequências

Idéias Principais

Árvores de decisão representam seqüências de possíveis ações para certos algoritmos.

A análise de uma árvore de decisão para algoritmos que resolvam certos tipos de problemas pode levar a um limite inferior para a quantidade máxima do trabalho necessário para resolver o problema em seu pior caso.

A tarefa de busca de um elemento x em uma lista de n elementos, se realizada através de comparações de x com os elementos da lista, requer no mínimo $\lfloor \log n \rfloor + 1$ comparações no pior caso.

A tarefa de ordenação de uma lista de n elementos, se realizada através de comparações de pares de elementos da lista, requer pelo menos $\lceil \log n! \rceil$ comparações no pior caso.

Dada a frequência de caracteres em uma coleção de dados, pode-se encontrar uma codificação binária que minimize o número de bits necessários para armazenar os dados e ainda seja um código de simples decodificação.

Exercícios 5.4

- ★1. Desenhe a árvore de decisão para uma busca seqüencial em uma lista de três elementos.
2. Desenhe a árvore de decisão para uma busca seqüencial em uma lista de seis elementos.
3. Desenhe a árvore de decisão para uma busca binária em uma lista de sete elementos. Qual a altura desta árvore?
4. Desenhe a árvore de decisão para uma busca binária em uma lista de quatro elementos. Qual a altura desta árvore?
- ★5. Considere um algoritmo de busca que compare um elemento ao último elemento da lista, depois ao primeiro elemento, depois ao penúltimo, depois ao segundo e assim por diante. Desenhe a árvore de decisão da busca em uma lista ordenada com seis elementos. Qual a altura da árvore? Este parece ser um algoritmo ótimo no pior caso?
6. Considere um algoritmo de busca que compare um elemento ao elemento localizado a um terço do início

da lista; baseado nesta comparação, ele repete a busca na primeira terça-parte ou no restante da lista. Desenhe a árvore de decisão para busca em uma lista ordenada de nove elementos. Qual a altura da árvore? Este algoritmo parece ser ótimo no pior caso?

- ★7. a. Fornecidos os dados

9, 5, 6, 2, 4, 7

construa a árvore binária de busca. Qual a altura da árvore?

- b. Encontre o número médio de comparações realizadas para buscar um elemento que sabemos estar na lista através de uma busca na árvore binária do item (a). (*Dica:* Encontre o número de comparações para buscar cada elemento.)

8. a. Fornecidos os dados

g, d, r, s, b, q, c, m

construa a árvore binária de busca. Qual a altura da árvore?

- b. Encontre o número médio de comparações realizadas para buscar um elemento que sabemos estar na lista, através de uma busca na árvore binária do item (a). (*Dica:* Encontre o número de comparações para buscar cada elemento.)

9. a. Para um conjunto de seis dados, qual o número mínimo no pior caso de comparações que um algoritmo de busca deve realizar?

- b. Dado o conjunto de itens de dados $\{a, d, g, i, k, s\}$, encontre uma ordem para entrada na qual a árvore binária de busca correspondente tenha altura mínima.

10. a. Qual o número mínimo de comparações para o pior caso que um algoritmo de busca realiza em um conjunto de nove elementos?

- b. Dado o conjunto de itens de dados $\{4, 7, 8, 10, 12, 15, 18, 19, 21\}$, encontre uma ordem para entrada na qual a árvore binária de busca correspondente tenha altura mínima.

- ★11. Use o teorema sobre o limite inferior para ordenação a fim de encontrar limites inferiores para o número de comparações necessárias no pior caso para a ordenação de listas com os seguintes tamanhos:

- a. 4 b. 8 c. 16

12. Compare o número de comparações necessárias nos piores casos dos algoritmos selection sort e merge sort com o limite inferior encontrado no Exercício 11 (veja o Exercício 14 da Seção 2.4). Qual a conclusão que você chegou?

Os Exercícios 13 a 18 referem-se ao problema de identificação de uma moeda falsificada (que deve ser mais pesada ou mais leve que as demais) de um conjunto de n moedas. Uma balança de braços é usada para comparar os pesos de um grupo de moedas contra um outro grupo de moedas do conjunto. O resultado deste tipo de comparação é que um grupo A pesa mais, menos ou o mesmo que um grupo B. Uma árvore de decisão representando a sequência de comparações feitas será, portanto, uma *árvore ternária*, onde os vértices internos podem ter três filhos.

13. Uma dentre cinco moedas é falsificada e pesa menos que as outras quatro. O problema é identificar a moeda falsificada.

- a. Qual o número de resultados possíveis (o número de folhas na árvore de decisão)?
b. Encontre um limite inferior para o número de comparações necessárias para resolver este problema no pior caso.
c. Imagine um algoritmo que realize este limite inferior (desenhe sua árvore de decisão).

- ★14. Uma dentre cinco moedas é falsificada e pode ser mais leve ou mais pesada que as demais. O problema é identificar a moeda falsificada e determinar se é mais leve ou mais pesada.

- a. Qual o número de resultados possíveis (o número de folhas na árvore de decisão)?
b. Encontre um limite inferior para o número de comparações necessárias à resolução deste problema no pior caso.
c. Imagine um algoritmo que realize este limite inferior (desenhe sua árvore de decisão).

15. Uma dentre quatro moedas é falsificada e pode ser mais leve ou mais pesada que as demais. O problema é identificar a moeda falsificada, mas não é preciso indicar se é mais leve ou mais pesada.

- a. Qual o número de resultados possíveis (o número de folhas na árvore de decisão)?
 b. Encontre um limite inferior para o número de comparações necessárias para resolver este problema no pior caso.
 c. Imagine um algoritmo que realize este limite inferior (desenhe sua árvore de decisão).
16. Uma dentre quatro moedas é falsificada e pode ser mais leve ou mais pesada que as demais. O problema é identificar a moeda falsificada, e determinar se ela é mais leve ou mais pesada.
 a. Qual o número de resultados possíveis (o número de folhas na árvore de decisão)?
 b. Encontre um limite inferior para o número de comparações necessárias para resolver este problema no pior caso.
 c. Demonstre que não existe um algoritmo que realize este limite. (*Dica:* a primeira comparação pode ser feita com duas ou com quatro moedas. Considere cada caso.)
17. Escreva um algoritmo que resolva o problema do Exercício 16, usando três comparações no pior caso.
18. Uma dentre oito moedas é falsificada e pode ser mais pesada ou mais leve que as demais. O problema é identificar a moeda falsificada e determinar se é mais leve ou mais pesada.
 a. Qual o número de resultados possíveis (o número de folhas na árvore de decisão)?
 b. Encontre um limite inferior para o número de comparações necessárias para resolver este problema no pior caso.
 c. Imagine um algoritmo que realize este limite inferior (desenhe sua árvore de decisão).

★19. Dados os códigos

Caracter	<i>a</i>	<i>e</i>	<i>i</i>	<i>o</i>	<i>u</i>
Codificação	00	01	10	110	111

decodifique as seqüências

- a. 11011011101 b. 1000110111 c. 010101

20. Dados os códigos

Caracter	<i>b</i>	<i>h</i>	<i>q</i>	<i>w</i>	%
Codificação	1000	1001	0	11	101

decodifique as seqüências

- a. 10001001101101 b. 11110 c. 01001111000

★21. a. Construa a árvore de Huffman para os caracteres e as freqüências dados:

Caracter	<i>c</i>	<i>d</i>	<i>g</i>	<i>m</i>	<i>r</i>	<i>z</i>
Freqüência	28	25	6	20	3	18

- b. Encontre os códigos de Huffman para esses caracteres.

22. a. Construa a árvore de Huffman para os caracteres e as freqüências dados:

Caracter	<i>b</i>	<i>n</i>	<i>p</i>	<i>s</i>	<i>w</i>
Freqüência	6	32	21	14	27

- b. Encontre os códigos de Huffman para esses caracteres.

Revisão do Capítulo 5

Terminologia

algoritmo ótimo
 altura de uma árvore
 aresta (arco)
 arestas paralelas
 árvore
 árvore binária

árvore binária completa
 árvore binária de busca
 árvore de decisão
 árvore não-enraizada
 busca em árvore binária
 caminho

ciclo
 codificação de Huffman
 código prefixo
 coloração de um grafo
 compressão de dados
 (compactação de dados)

comprimento de um caminho	grafo simples	ponteiro nulo
diagrama de estrutura	grafos duais	ponto inicial
diagrama de fluxo de dados	grafos homeomorfos	problema das quatro cores
filho à direita	grafos isomorfos	profundidade de um vértice
filho à esquerda	grafos planares	raiz de uma árvore
floresta	grafos simples isomorfos	relação de adjacências
folha	grau de um vértice	subgrafo
fórmula de Euler	isomorfismo	vértice (nó ou nodo)
grafo acíclico	laço	vértice alcançável
grafo bipartido completo	lista de adjacências	vértice interno (nó interno)
grafo completo	lista encadeada	vértice isolado
grafo conexo	matriz de adjacências	vértice-pai
grafo direcionado (dígrafo)	matriz de alcançabilidade	vértices adjacentes
grafo ponderado	matriz esparsa	vértices extremos
grafo rotulado	número cromático de um grafo)	vértices-filho
grafo sem laços		

Autotestes

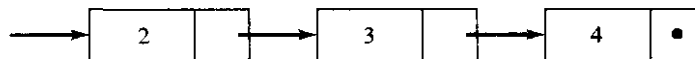
Responda às perguntas a seguir com verdadeiro ou falso.

Seção 5.1

1. Um grafo conexo tem uma aresta entre cada dois vértices.
2. Se G_1 é isomorfo ao grafo G_2 , então um vértice de grau 5 em G_1 será levado em um vértice de grau 5 em G_2 .
3. As arestas de um grafo planar só se encontram em vértices, independentemente de como desenhemos.
4. Um grafo não-planar deve conter K_5 ou $K_{3,3}$ como um subgrafo.
5. A profundidade de qualquer vértice em uma árvore é menor ou igual à altura desta árvore.

Seção 5.2

6. A matriz de adjacências de um grafo direcionado não é simétrica.
7. Se a representação por lista de adjacências de um grafo contém uma parte igual a



significa que o vértice 2 é adjacente ao vértice 3 e que o vértice 3 é adjacente ao vértice 4.

8. A representação por lista de adjacências de um grafo requer menos memória para ser armazenada do que a representação por matriz de adjacências.
9. A determinação de se o vértice 3 é adjacente ao vértice 6 em um grafo com oito vértices e diversas arestas é mais rápida através da representação por matriz de adjacências do que por lista de adjacências.
10. Na representação por vetor de filhos à esquerda e à direita de uma árvore binária, qualquer linha do vetor que corresponda a uma folha terá todas as entradas zero.

Seção 5.3

11. Qualquer relação binária em um conjunto N tem uma matriz de adjacências associada.
12. O fecho transitivo é a relação de adjacências equivalente à alcançabilidade.
13. A matriz de alcançabilidade R de um grafo direcionado G é computada através das potências da matriz de adjacências elevadas a n^2 .
14. O algoritmo de Warshall realiza uma contagem de caminhos entre os vértices de comprimento 1, depois os de comprimento 2, e assim por diante.
15. O algoritmo de Warshall fornece o fecho simétrico no caso de uma relação de adjacência simétrica.

Seção 5.4

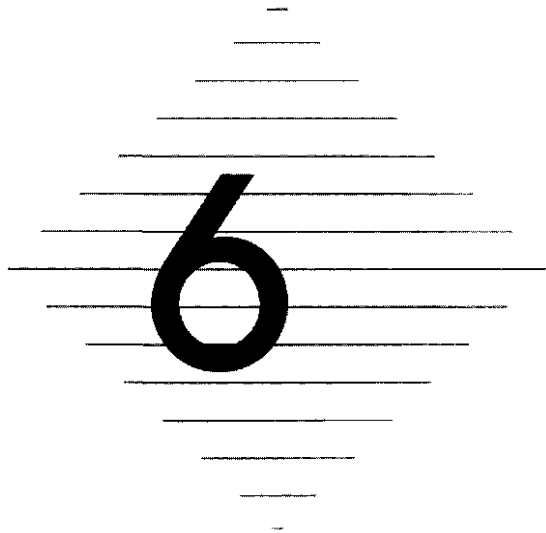
16. A raiz de uma árvore de decisão para um algoritmo de busca binária em uma lista de 11 elementos representaria a comparação do elemento procurado com o quinto elemento da lista.
17. A busca de qualquer elemento x em uma lista com n elementos requer pelo menos $1 + \lfloor \log n \rfloor$ comparações.

18. O valor 14 é procurado através de uma busca em uma árvore binária cuja raiz tem o valor 10; a busca procederá na subárvore à direita.
19. Uma árvore de decisão para a ordenação de n elementos precisa ter uma altura de pelo menos $n!$.
20. Em um código de Huffman, os caracteres que ocorrem mais freqüentemente têm mais Os em sua representação binária.

No Computador

Nos Exercícios 1 a 6, escreva um programa de computador que produza a saída desejada a partir da entrada fornecida.

1. *Entrada:* Lista de adjacências de um grafo
Saída: Matriz de adjacências do mesmo grafo
2. *Entrada:* Matriz de adjacências de um grafo
Saída: Lista de adjacências do mesmo grafo
3. *Entrada:* Lista de adjacências de um grafo e o nome de um vértice n no grafo
Saída: Lista de adjacências do grafo com o vértice n e suas respectivas arestas removidas
4. *Entrada:* Matriz de adjacências A de um grafo direcionado
Saída: Matriz de alcançabilidade R para o grafo computada através da fórmula $R = A \vee A^{(2)} \vee \dots \vee A^{(n)}$
5. *Entrada:* Matriz de adjacências A de um grafo direcionado
Saída: Matriz de alcançabilidade R para o grafo computada através do algoritmo de Warshall
6. *Entrada:* Lista de n caracteres e suas freqüências (inteiras)
Saída: Código de Huffman para os caracteres
(*Dica:* Faça uma lista ordenada dos registros que representam as raízes das árvores binárias. Inicialmente existem n registros deste tipo, cada qual sem filhos; ao final, deve haver apenas um, a raiz da árvore de Huffman.)
7. Escreva um programa que permita ao usuário entrar uma lista de inteiros e construir uma árvore binária de busca com esses inteiros como nós. O usuário poderá, então, entrar um inteiro de cada vez e o programa realizará uma busca e indicará se o inteiro fornecido está ou não na lista.



Grafos e Algoritmos

Objetivos do Capítulo

Após estudar este capítulo, o leitor estará apto a:

- Verificar a existência de um caminho euleriano em um grafo (resolver o "problema do carteiro chinês")
- Entender o problema do ciclo hamiltoniano (o "problema do caixeiro-viajante") e como ele é fundamentalmente diferente do problema do caminho euleriano
- Usar o algoritmo de Dijkstra para encontrar o menor caminho entre dois vértices de um grafo simples, ponderado e conexo
- Usar o algoritmo de Prim para encontrar a árvore geradora mínima de um grafo simples, ponderado e conexo
- Realizar buscas em profundidade em um grafo simples e conexo
- Realizar buscas em largura em um grafo simples e conexo
- Entender como a busca em profundidade pode ser usada para testar a alcançabilidade em um grafo direcionado, realizar uma ordenação topológica em um conjunto parcialmente ordenado representado por um grafo direcionado, e encontrar as componentes conexas de um grafo desconexo
- Realizar buscas em preordem, inordem e posordem em uma árvore
- Encontrar as articulações de um grafo simples e conexo.

Como os grafos têm muitas aplicações, existe um grande interesse em encontrar algoritmos eficientes para resolver certos problemas e realizar determinadas operações com eles. (Por exemplo, dada a ma-

triz de adjacências de um grafo direcionado com n vértices, o algoritmo de Warshall é um algoritmo de ordem $\Theta(n^3)$ que responde à pergunta: O vértice n_i alcança o vértice n_j ? Todos esses algoritmos usam alguma representação conveniente (uma matriz de adjacências, uma lista de adjacências etc.) para os grafos, grafos direcionados ou árvores.

Na Seção 6.1, veremos os algoritmos que respondem a duas perguntas de interesse histórico. Essas perguntas são conhecidas como o *problema do carteiro chinês* e o *problema do caixeiro viajante*. O problema do carteiro chinês é determinar se existe um caminho em determinado grafo que use todas as arestas exatamente uma vez, fornecendo, assim, o meio de um carteiro distribuir suas cartas com eficiência, sem passar pela mesma rua mais de uma vez. O problema do caixeiro viajante é determinar se um dado grafo possui um ciclo que passe por todos os vértices e, se este ciclo existir, determinar qual o ciclo que corresponde à menor distância a ser percorrida. Lembremos que um ciclo é um caminho que termina no mesmo vértice em que foi iniciado e que não usa nenhum outro vértice mais de uma vez; desta forma um ciclo deste tipo fornece uma forma eficiente de um caixeiro-viajante visitar várias cidades em seu território de vendas apenas uma vez e retornar para casa ao final.

A Seção 6.2 fornece soluções computacionais para os problemas de encontrar a distância mínima entre dois vértices em um grafo conexo simples e de minimizar o número de arestas usadas para conectar todos os vértices em um grafo conexo simples. A Seção 6.3 aborda algoritmos para atravessar grafos simples e árvores — "visitando" todos os vértices de alguma forma sistemática. Finalmente, a Seção 6.4 apresenta um algoritmo para identificação de seções relativamente isoladas de um grafo.

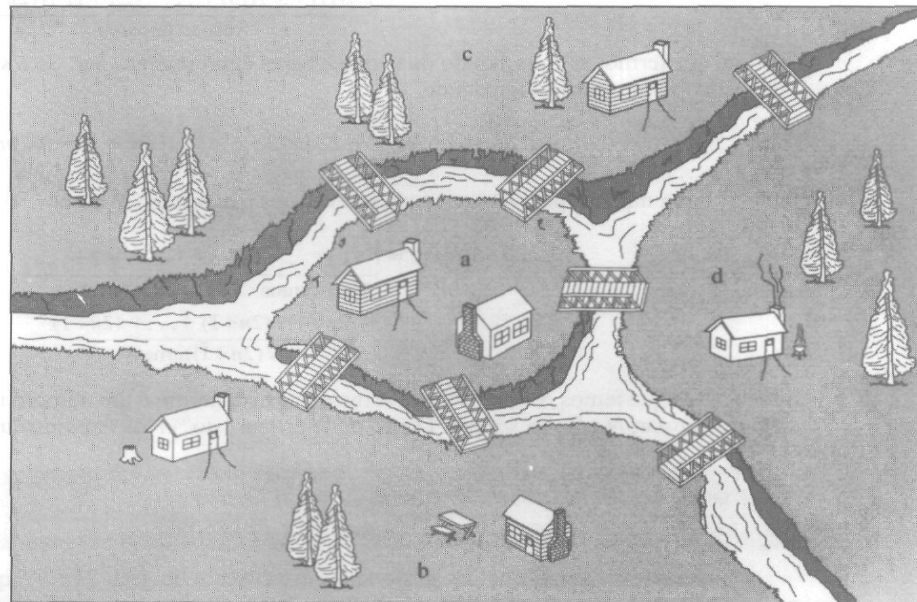


Figura 6.1

Seção 6.1 Caminho Euleriano e Ciclo Hamiltoniano

O Problema do Caminho Euleriano

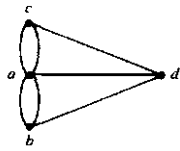


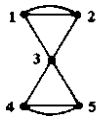
Figura 6.2

O problema do caminho euleriano (o problema do carteiro chinês) foi enunciado muitos anos atrás. O matemático suíço Leonhard Euler (pronuncia-se "óiler") (1707-1783) ficou curioso devido a uma charada popular sobre o lugarejo de Königsberg (uma cidade da antiga Prússia, mais tarde chamada de Kaliningrado, na Rússia). O rio que passa pela cidade forma uma ilha. Diversas pontes atravessavam o rio, como mostra a Fig. 6.1. A charada era determinar se uma pessoa poderia passear pela cidade passando apenas uma vez por cada ponte. É viável responder a esta pergunta, listando todos os caminhos possíveis, de forma que algum dedicado habitante de Königsberg deve ter resolvido este problema em particular. A idéia de Euler foi representar esta situação como um grafo (veja a Fig. 6.2), onde as pontes são arestas e os trechos de terra (chamados de a e d) são os vértices. Ele então resolveu a questão geral de quando um grafo apresenta um caminho euleriano.

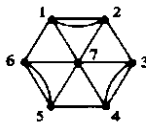
Definição: Caminho Euleriano

Um **Caminho Euleriano** em um grafo G é um caminho que usa cada aresta de G exatamente uma vez.

PRÁTICA 1



(a)



(b)

Figura 6.3

Existem caminhos eulerianos para os grafos da Fig. 6.3? (Use o método de tentativa e erro para responder a esta pergunta. Este é o mesmo jogo infantil de se desenhar uma figura sem tirar o lápis do papel e sem passar o lápis pela mesma linha mais de uma vez.)

Para esta discussão, vamos supor que todos os grafos são conexos, até porque um caminho euleriano não poderia existir em caso contrário. A existência de um caminho euleriano em um determinado grafo depende dos graus de seus vértices. Um vértice é **par** se o seu grau for par, e **ímpar** se o seu grau for ímpar. Ocorre que todo caminho tem um número par de vértices ímpares. Para vermos isto, tome qualquer grafo, e seja N o número de vértices ímpares nele, seja $N(1)$ o número de vértices de grau 1, $N(2)$ o número de vértices de grau 2, e assim por diante. A soma dos graus de todos os vértices do grafo é

$$S = 1 \cdot N(1) + 2 \cdot N(2) + 3 \cdot N(3) + \dots + k \cdot N(k) \quad (1)$$

para algum k . Esta soma é, na verdade, uma conta do número total dos extremos das arestas do grafo. Como o número de extremos de arestas é o dobro do número de arestas, S é um número par. Reorganizando a equação (1) a fim de agrupar os termos ímpares e os termos pares:

$$\begin{aligned} S = & \underbrace{2 \cdot N(2) + 4 \cdot N(4) + \dots + 2m \cdot N(2m)}_{\text{vértices pares}} \\ & + \underbrace{1 \cdot N(1) + 3 \cdot N(3) + \dots + (2n+1) \cdot N(2n+1)}_{\text{vértice ímpares}} \end{aligned}$$

A soma dos termos representando os vértices pares é um número par. Se a subtrairmos de ambos os lados da equação, obtemos a nova igualdade

$$S' = 1 \cdot N(1) + 3 \cdot N(3) + \dots + (2n+1) \cdot N(2n+1) \quad (2)$$

onde S' é um número par. Agora, se reescrevermos a equação (2) como

$$\begin{aligned} S' = & \underbrace{1+1+\dots+1}_{N(1)\text{ termos}} + \underbrace{3+3+\dots+3+\dots}_{N(3)\text{ termos}} \\ & + \underbrace{(2n+1)+(2n+1)+\dots+(2n+1)}_{N(2n+1)\text{ termos}} \end{aligned}$$

onde vemos que temos N termos nesta soma e que cada termo é um número ímpar. Para a soma de N números ímpares ser par, N precisa ser par. (Você pode provar isto?) Nós demonstramos então o teorema a seguir.

Teorema sobre Vértices ímpares em um Grafo

O número de vértices ímpares em qualquer grafo é par.

Suponhamos agora que um grafo tem um vértice n de grau $2k+1$ e que existe um caminho euleriano neste grafo mas que este caminho não começa em n . Então, para cada aresta que usamos para chegar a n , existe outra aresta que usamos para sair de n , até que tenhamos usado k pares de arestas. A próxima vez que

chegarmos a n não haverá mais nenhuma aresta para sair. Portanto, se nosso caminho não começa em n , ele precisa terminar em n . O caminho pode ou não começar em n e, caso não comece, ele termina em n , de forma que o caminho começa ou termina neste vértice ímpar arbitrário. Portanto, se houver mais de dois vértices ímpares em um grafo, não é possível haver um caminho euleriano. Portanto, existem dois casos em que pode haver um caminho euleriano — em um grafo sem vértices ímpares ou em um grafo com dois vértices ímpares.

Considere um grafo sem vértices ímpares. Tome qualquer vértice m e inicie um caminho euleriano. Sempre que entrarmos em um novo vértice, teremos uma outra aresta para sair até que tenhamos voltado ao vértice m . Se tivermos usado todas as arestas do grafo, terminamos nosso caminho. Do contrário, existe algum vértice m' do caminho com arestas não usadas. Então construa um caminho euleriano começando e terminando em m' , da mesma forma que você fez na seção anterior do caminho. Inclua este novo caminho ao caminho original. Se agora você tiver usado todas as arestas do grafo original, o caminho está completo. Do contrário, repita este processo até ter usado todas as arestas.

Se houver exatamente dois vértices ímpares, um caminho euleriano pode ser iniciado em um vértice ímpar e terminado no outro. Se o caminho não tiver usado todas as arestas, pode-se incluir caminhos extras, como no caso anterior.

Agora temos uma solução completa para o problema do caminho euleriano.

Teorema dos Caminhos Eulerianos

Existe um caminho euleriano em um grafo conexo se, e somente se, não houver nenhum ou existirem exatamente dois vértices ímpares. No caso de não haver vértices ímpares, o caminho pode começar em qualquer vértice e terminará neste mesmo vértice; para o caso de haver dois vértices ímpares, o caminho deve começar em um vértice ímpar e terminar no outro.

PRÁTICA 2 Usando o teorema anterior, refaça a Prática 1.

PRÁTICA 3 Existe o caminho de Königsberg?

O teorema dos caminhos de Euler é, na verdade, um algoritmo para determinação de se um caminho de Euler existe ou não em um grafo conexo arbitrário. Para fazer com que se pareça mais com um algoritmo, vamos reescrevê-lo em pseudocódigo. A essência do algoritmo é contar o número de vértices adjacentes a cada vértice e determinar se este é ímpar ou par. Se houver muitos vértices ímpares, não existe um caminho euleriano.

No algoritmo a seguir (algoritmo *CaminhoEuleriano*), a entrada é um grafo conexo representado por sua matriz de adjacências $n \times n$ A . A variável *total* mantém o número de vértices ímpares encontrados no grafo. O grau de qualquer vértice em particular, *grau*, é encontrado pela totalização dos números na linha correspondente a este vértice na matriz de adjacências. A função *odd* retorna o valor "verdadeiro" se, e somente se, seu argumento for um inteiro ímpar.

ALGORITMO *CaminhoEuleriano*

procedure *CaminhoEuleriano* (A : matriz $n \times n$);

{ determina se existe ou não um caminho euleriano em um grafo conexo de matriz de adjacências A }

var

total: integer; {número de vértices ímpares encontrados}

grau: integer; {grau de um vértice}

i, *j*: integer; {índices da matriz}

begin

total := 0;

i := 1;

while (*total* <= 2) **and** (*i* <= *n*) **do**

begin

grau := 0;

for *j* := 1 **to** *n* **do**

{*} *grau* := *grau* + $A[i, j]$;

if *odd*(*grau*) **then**

total := *total* + 1;

```

    i:= i + 1;
end; {while}

if total > 2 then
    write ('Não existe caminho euleriano')
else
    write ('Existe um caminho euleriano');
end;

```

EXEMPLO 1 A matriz de adjacências para o grafo da Fig. 6.3a é mostrada a seguir.

$$\begin{bmatrix} 0 & 2 & 1 & 0 & 0 \\ 2 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 2 & 0 \end{bmatrix}$$

Quando o algoritmo entra pela primeira vez no laço **while**, *total* vale 0 e *i* vale 1. Então *grau* é iniciado com 0. Dentro do laço **for**, os valores da linha 1 da matriz de adjacências são totalizados em *grau*, resultando no valor 3. A função *odd* aplicada a *grau* retorna o valor "verdadeiro", de forma que o valor de *total* é incrementado, passando de 0 para 1. Nem o limite de *total* nem o limite do tamanho da matriz foram ultrapassados, de forma que o laço **while** é executado novamente, desta vez para a linha 2 da matriz. Mais uma vez foi encontrado um valor ímpar para *grau*, o que faz com que o valor de *total* seja alterado para 2. Quando o laço **while** é executado para a linha 3 da matriz, o valor de *grau* é par (4), de forma que o valor de *total* não é alterado e o laço **while** é executado novamente com *i*=4. A linha 4 produz mais um valor ímpar para *grau*, de modo que *total* assume o valor 3. Isto termina o laço **while**, e então a má notícia é impressa: não há caminho euleriano porque o número de vértices ímpares excede 2.

PRÁTICA 4 Escreva a matriz de adjacências para o problema da cidade de Königsberg e acompanhe a execução do algoritmo *CaminhoEuleriano*.

Vamos fazer uma análise do algoritmo *CaminhoEuleriano*. A operação relevante realizada pelo algoritmo é o exame dos elementos da matriz de adjacências, que ocorre na linha marcada com {*}. No pior caso, o laço **while** é executado *n* vezes, uma para cada linha. Dentro do laço **while**, o laço **for** que contém a linha {*} é executado *n* vezes, uma para cada coluna. Portanto, o algoritmo *CaminhoEuleriano* é, no pior caso, um algoritmo $\Theta(n^2)$.

Às custas de mais um pouco de lógica, podemos modificar o algoritmo de forma que não examine a última linha da matriz. Sabemos, do teorema sobre caminhos eulerianos, que o número de vértices ímpares é par. Se, após processarmos a penúltima linha, o número de vértices ímpares for ímpar, saberemos que a última linha representa um vértice ímpar; se este número for par, então a última linha representa um vértice par. Esta modificação resulta em examinarmos $(n - 1)n$ elementos no pior caso, o que ainda é $\Theta(n^2)$.

Se representarmos o grafo *G* por uma lista de adjacências ao invés de uma matriz de adjacências, então a versão correspondente do algoritmo teria que contar o tamanho da lista de adjacências de cada vértice e totalizar quantas delas têm tamanho ímpar. Existem *n* listas de adjacências a serem examinadas, da mesma forma que havia *n* linhas na matriz de adjacências, mas o tamanho de cada lista deve ser menor que *n*, o tamanho de uma linha da matriz. É possível diminuir a ordem aquém de n^2 se o número de arestas do grafo for pequeno, mas o pior caso permanece $\Theta(n^2)$.

Problema do Ciclo Hamiltoniano

Outro matemático famoso, William Rowan Hamilton (1805-1865), propôs um problema em teoria de grafos que se parece muito com o problema de Euler. O problema é determinar se um grafo tem um **ciclo hamiltoniano**, isto é, um ciclo que usa cada vértice do grafo.

PRÁTICA 5 Existem ciclos hamiltonianos para o grafo da Fig. 6.3? (Use um método de tentativa e erro para obter sua resposta.)

O problema do ciclo hamiltoniano pode ser resolvido para um dado grafo através de tentativas e erros da seguinte maneira: comece por algum vértice do grafo e tente qualquer caminho escolhendo suas várias arestas. Se o caminho resultante tiver um vértice repetido, ele não é um ciclo; neste caso, descarte-o e tente um caminho diferente. Se o caminho puder ser completado como um ciclo, verifique então se ele visitou todos os vértices; se não, descarte-o e tente um novo caminho. Continue com este procedimento até que todos os caminhos

possíveis tenham sido testados ou até que se tenha obtido um ciclo hamiltoniano. Este algoritmo envolve algum armazenamento dos caminhos já testados de forma que um mesmo caminho não seja testado mais de uma vez. A abordagem por tentativa e erro é possível — mas é praticamente inviável! Em grafos que não forem pequenos, existirão muitos caminhos a serem testados.

Aqui, então, está a diferença básica entre o problema do caminho euleriano e o problema do ciclo hamiltoniano. Existe um algoritmo eficiente para se determinar se um grafo arbitrário tem ou não um caminho euleriano, mas não se conhece um algoritmo eficiente para se determinar se um ciclo hamiltoniano existe. Na verdade, existem algumas evidências que sugerem que este tipo de algoritmo nunca será encontrado.

No entanto, em certos tipos de grafos, podemos determinar facilmente quando existe um ciclo hamiltoniano. Em particular, um grafo completo com $n > 2$ tem um ciclo hamiltoniano porque, para qualquer vértice no caminho, existe sempre uma aresta para levar a algum vértice ainda não usado e, finalmente, uma aresta para voltar ao vértice inicial.

Suponha que estamos trabalhando com um grafo ponderado. Se houver um ciclo hamiltoniano para o grafo, podemos encontrar o ciclo com menor peso? Este é o problema do caixeiro-viajante. Mais uma vez podemos resolver este problema por tentativas e erros, verificando todos os caminhos possíveis e mantendo um controle dos pesos dos ciclos hamiltonianos encontrados, porém, mais uma vez, este não seria um algoritmo eficiente. (Em particular, o problema do caixeiro viajante foi resolvido para o caso de se visitar as 48 capitais norte-americanas — um total de 10.628 milhas de distância.)

Revisão da Seção 6.1

Técnica

- Determinar se existe um caminho euleriano em um grafo (usando o algoritmo *CaminhoEuleriano*)

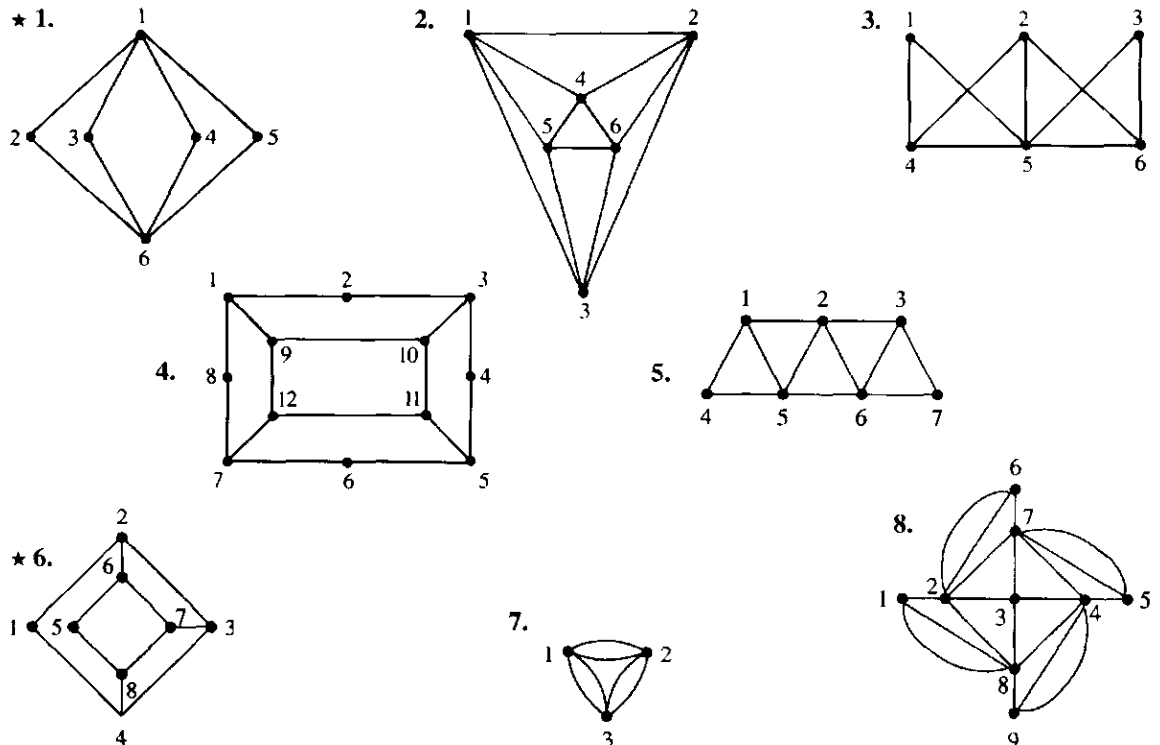
Idéias Principais

Existe um critério simples para determinar se um grafo contém um caminho euleriano, mas não para determinar se um grafo contém um ciclo hamiltoniano.

Um algoritmo que seja $\Theta(n^2)$ no pior caso pode determinar a existência de um caminho euleriano em um grafo conexo com n vértices.

Exercícios 6.1

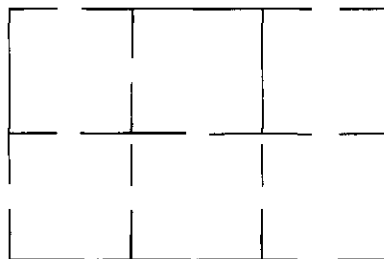
Nos Exercícios 1 a 8, determine se o grafo da figura dada tem um caminho euleriano usando o teorema dos caminhos eulerianos.



- ★9. Escreva a matriz de adjacências para o grafo do Exercício 1. Ao aplicar o algoritmo *CaminhoEuleriano*, qual é o valor de *total* após o segundo passo do laço **while**?
- 10. Escreva a matriz de adjacências para o grafo do Exercício 3. Ao aplicar o algoritmo *CaminhoEuleriano*, qual é o valor de *total* após o quarto passo do laço **while**?
- ★11. Escreva a matriz de adjacências para o grafo do Exercício 5. Ao aplicar o algoritmo *CaminhoEuleriano*, qual o valor de *i* após o fim do laço **while**?
- 12. Escreva a matriz de adjacências para o grafo do Exercício 6. Ao aplicar o algoritmo *CaminhoEuleriano*, qual o valor de *i* após o fim do laço **while**?

Nos Exercícios 13 a 20, determine através de tentativas e erros se os grafos dos exercícios dados apresentam ciclos hamiltonianos.

- ★13. Exercício 1 14. Exercício 2 15. Exercício 3 16. Exercício 4
- 17. Exercício 5 ★18. Exercício 6 19. Exercício 7 20. Exercício 8
- 21. Encontre um exemplo de um grafo desconexo que tenha um caminho euleriano. (*Dica*: Como isto parece contraditório, você deve procurar um caso trivial.)
- ★22. Demonstre que qualquer grafo com um ciclo hamiltoniano é conexo.
- 23. Considere um grafo simples completo com n vértices. A verificação da existência de um ciclo hamiltoniano neste grafo por um método de tentativa e erro pode ser feita gerando todos os caminhos possíveis no grafo com tamanho n .
 - a. Quantos caminhos de tamanho n existem se permitirmos repetições de arestas e vértices?
 - b. Quantos caminhos de comprimento n existem se permitirmos repetições de arestas e vértices, mas uma mesma aresta não puder ser usada duas vezes seguidas?
 - c. Quantos caminhos de comprimento n existem se não permitirmos repetições de vértices nem de arestas?
- 24. É possível caminhar pela casa da figura abaixo de forma que cada porta da casa seja usada exatamente uma vez? Justifique.



Exercício 24

- ★25. Lembremos que K_n denota o grafo simples completo de ordem n .
 - a. Para quais valores de n existe um caminho euleriano em K_n ?
 - b. Para quais valores de n existe um ciclo hamiltoniano em K_n ?
- 26. Lembremos que $K_{m,n}$ denota um grafo bipartido completo cujas partições têm m e n vértices, respectivamente.
 - a. Para quais valores de m e n existe um caminho euleriano em $K_{m,n}$?
 - b. Para quais valores de m e n existe um ciclo hamiltoniano em $K_{m,n}$?
- 27. Considere um grafo conexo com $2n$ vértices ímpares, $n \geq 2$. Pelo teorema dos caminhos eulerianos, não existe caminho euleriano neste grafo.
 - a. Qual o número mínimo de caminhos eulerianos disjuntos, cada qual percorrendo algumas arestas do grafo, necessários para percorrer todas as arestas exatamente uma vez?
 - b. Mostre que o número mínimo é também suficiente.
- ★28. Demonstre que sempre existe um ciclo hamiltoniano em um grafo cujo grau de todos os vértices seja 2.

Seção 6.2 Caminho Mínimo e Árvore Geradora Mínima

O Problema do Caminho Mínimo

Admita que temos um grafo simples, ponderado e conexo, onde os pesos são todos positivos. Então existe um caminho entre quaisquer dois vértices x e y . De fato, pode haver vários desses caminhos. A pergunta é como encontrar um caminho com o menor peso? Como os pesos geralmente representam distâncias, este problema ficou conhecido como o problema do "caminho mínimo". Ele é um problema importante a ser resolvido para redes de computadores ou de comunicações, onde a informação em um vértice precisa ser transmitida a outro vértice da maneira mais eficiente possível, ou para uma rede de transportes, onde os produtos de uma cidade precisam ser levados a outra.

O problema do caixeiro-viajante é um problema de caminho ponderado mínimo com fortes restrições sobre a natureza do caminho, de modo que este caminho pode vir a não existir. No problema de caminho mínimo, não colocamos restrições (além da referente ao peso mínimo) na natureza do caminho; como o grafo é conexo, sabemos que o caminho existe. Por esta razão, podemos esperar que haja um algoritmo eficiente para resolver este problema, ainda que não exista algoritmo eficiente para o problema do caixeiro viajante.

O algoritmo para o caminho mínimo, conhecido como algoritmo de Dijkstra, funciona da seguinte maneira: desejamos encontrar a distância mínima de um dado vértice x a um dado vértice y . Construiremos um conjunto (chama-lo-emos de IN) que contém inicialmente apenas x , mas que cresce à medida que o algoritmo é executado. A qualquer momento, IN contém todos os vértices para os quais já foram determinados os menores caminhos, usando apenas vértices em IN , a partir de x . Para cada vértice z fora de IN , mantemos a menor distância $d[z]$ de x a z usando caminhos onde o único vértice que não está em IN seja z . Também armazenamos o vértice adjacente a z neste caminho, $s[z]$.

Como fazer com que IN cresça, isto é, qual vértice deve ser incluído em IN a seguir? Tomamos o vértice fora de IN com menor distância d . Uma vez que tenhamos acrescentado este vértice, chamemo-lo de p , a IN , devemos recalcular d para todos os vértices restantes fora de IN , pois pode haver um caminho menor a partir de x , passando por p , do que aquele que havia antes de p ser agregado a IN . Se houver um caminho mais curto, precisamos também atualizar $s[z]$ de forma a indicar que p é o vértice adjacente a z pelo novo caminho mínimo. Quando y for incluído em IN , este pára de receber novos elementos. O valor corrente de $d[y]$ é a distância do menor caminho, e seus vértices podem ser obtidos examinando y , $s[y]$, $s[s[y]]$ etc, até que tenhamos voltado a x .

O quadro a seguir (algoritmo *CaminhoMínimo*) apresenta o algoritmo na forma de pseudocódigo. A entrada é a matriz de adjacências de um grafo G simples conexo e com pesos positivos, e os vértices x e y ; a saída do algoritmo é o caminho mínimo entre x e y e a distância deste caminho. Na verdade, usamos uma matriz de adjacências A modificada, onde $A[i,j]$ é o peso da aresta entre i e j se houver uma ou oo se não houver aresta entre os vértices i e j (neste caso, o símbolo ∞ representa um número muito grande, maior que qualquer peso do grafo).

ALGORITMO *CaminhoMínimo*

procedure *CaminhoMínimo*(A : matriz $n \times n$; x, y : vértices);

(Algoritmo de Dijkstra — A é a matriz de adjacências modificada de um grafo simples conexo com pesos positivos; x e y são vértices do grafo; a procedure apresenta o menor caminho de x a y e a distância deste caminho)

var

IN : conjunto de vértices; {menor caminho conhecido a partir de x }
 z, p : vértice; {vértices temporários}
 d : vetor de inteiros; {distância a partir de x usando os vértices de IN }
 s : vetor de vértices; {vértice anterior no caminho mínimo}
 $DistAnterior$: integer; {distância a ser comparada}

begin

(inicia o conjunto IN e os vetores d e s)

$IN := \{x\}$;

$d[x] := 0$;

for todos os vértices z que não pertençam a IN **do**

begin

$d[z] := A[x, z]$;

$s[z] := x$;

end;

```
{processa os vrtices de IN}
while not (y in IN) do
  begin
    {inclui o vrtice de distncia mnima que ainda no est em IN}
    p := vrtice z que no pertenca a IN com d[z] mnima;
    IN:=IN  {p};
    { recalcula d para vrtices que no pertencam a IN, ajusta s se necessrio }
    for todos os vrtices z que no pertencam a IN do
      begin
        DistAnterior := d[z];
        d[z] := min (d[z], d[p] + A[p, z]);
        if not (d[z] = DistAnterior) then
          s[z]:=p;
        end;
      end; {while}

    {Escreve os vrtices do caminho}
    write ('Em ordem inversa, o caminho : ');
    write (y);
    z:= y;
    repeat
      write(s[z]);
      z:=s[z];
    until z = x;

    {Escreve distncia do caminho}
    write('A distncia do caminho  ', d[y]);
  end;
```

EXEMPLO 2 Considere o grafo da Fig. 6.4 e a matriz de adjacncias correspondente mostrada na Fig. 6.5.

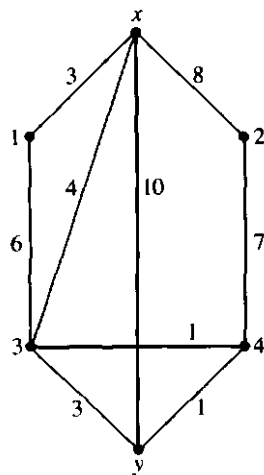


Figura 6.4

	x	1	2	3	4	y
x	∞	3	8	4	∞	10
1	3	∞	∞	6	∞	∞
2	8	∞	∞	∞	7	∞
3	4	6	∞	∞	1	3
4	∞	∞	7	1	∞	1
y	10	∞	∞	3	1	∞

Figura 6.5

Vamos acompanhar a execuo do algoritmo *CaminhoMnimo* neste grafo. Ao final da fase de iniciao, *IN* contm apenas *x*, e *d* encerra todas as distncias de *x* aos demais vrtices:

$IN = \{x\}$

	x	1	2	3	4	y
d	0	3	8	4	∞	10
s	-	x	x	x	x	x

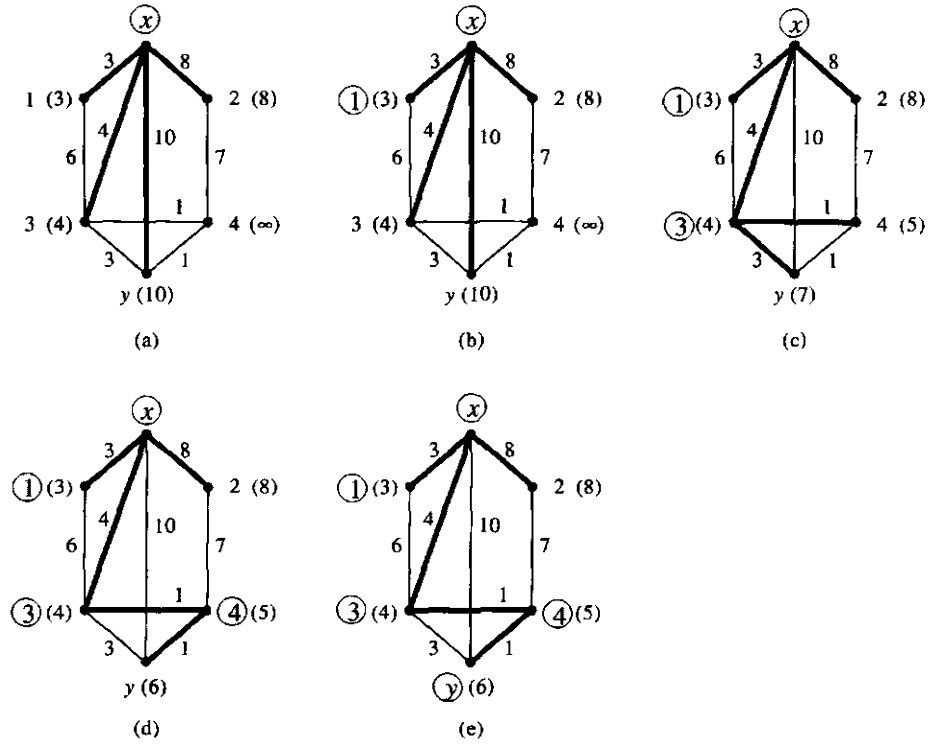


Figura 6.6

Na Fig. 6.6, os vértices circundados são os que pertencem ao conjunto IN , as linhas grossas mostram o caminho mínimo corrente e o valor de d para cada vértice está escrito junto a seus rótulos. A Fig. 6.6a refere-se ao momento após a iniciação.

Entramos, então, no laço **while** e procuramos, entre os valores de d , pelo vértice de menor distância que ainda não pertença a IN , o qual verificamos ser o vértice 1, com $d[1] = 3$. Incluímos o vértice 1 em IN , e no laço **for** recalculamos todos os valores de d para os vértices restantes 2, 3, 4 e y .

$$\begin{aligned}
 p &= 1 \\
 IN &= \{x, 1\} \\
 d[2] &= \min(8, 3 + A[1, 2]) = \min(8, \infty) = 8 \\
 d[3] &= \min(4, 3 + A[1, 3]) = \min(4, 9) = 4 \\
 d[4] &= \min(\infty, 3 + A[1, 4]) = \min(\infty, \infty) = \infty \\
 d[y] &= \min(10, 3 + A[1, y]) = \min(10, \infty) = 10
 \end{aligned}$$

Não há alterações nos valores de d , de forma que não há alterações nos valores de s (não há nenhum caminho a partir de x através do vértice 1 que seja menor do que ir diretamente a partir de x sem passar por 1). A Fig. 6.6b mostra que 1 agora pertence a IN .

O segundo passo do laço **while** produz o seguinte:

$$\begin{aligned}
 p &= 3 \quad (3 \text{ tem o menor valor de } d \text{ dentre } 2, 3, 4 \text{ e } y) \\
 IN &= \{x, 1, 3\} \\
 d[2] &= \min(8, 4 + \infty) = 8 \\
 d[4] &= \min(\infty, 4 + 1) = 5 \quad (\text{foi alterado, portanto atualizamos } s[4]) \\
 d[y] &= \min(10, 4 + 3) = 7 \quad (\text{foi alterado, portanto atualizamos } s[y])
 \end{aligned}$$

	X	1	2	3	4	y
d	0	3	8	4	5	7
s	-	X	X	X	3	3

Caminhos menores de x aos vértices 4 e y foram obtidos passando por 3. A Fig. 6.6c ilustra isto.
No passo seguinte,

$$\begin{aligned} p &= 4 \\ IN &= \{x, 1, 3, 4\} \\ d[2] &= \min(8, 5 + 7) = 8 \\ d[y] &= \min(7, 5 + 1) = 6 \text{ (foi alterado, portanto atualizamos } s[y]) \end{aligned}$$

	x	1	2	3	4	y
d	0	3	8	4	5	6
s	-	x	x	x	3	4

Veja a Fig. 6.6d.
Processando o laço **while** novamente, obtemos

$$\begin{aligned} p &= y \\ IN &= \{x, 1, 3, 4, y\} \\ d[2] &= \min(8, 6 + \infty) = 8 \end{aligned}$$

	x	1	2	3	4	y
d	0	3	8	4	5	6
s	-	x	x	x	3	4

Veja a Fig. 6.6e.
Agora que y pertence a IN , o laço **while** termina. O caminho passa por y , $s[y] = 4$, $s[4] = 3$ e $s[3] = x$. Assim, o caminho usa os vértices x , 3, 4 e y . (O algoritmo nos fornece estes vértices na ordem inversa.) A distância do caminho é $d[y] = 6$. Se olharmos o grafo da Fig. 6.4 e verificarmos todas as possibilidades, constatemos que este é, de fato, o menor caminho de x a y .

O algoritmo *CaminhoMínimo* termina quando y é incluído em IN , ainda que restem outros vértices no grafo que não pertençam a IN (como o vértice 2 no Exemplo 2). Como podemos ter certeza de que não existe um caminho menor usando algum desses vértices excluídos? Se continuássemos o processamento até que todos os vértices tivessem sido incluídos em IN , os valores de d representariam os caminhos mínimos de x a qualquer outro vértice, usando todos os valores em IN , isto é, o caminho mínimo usando quaisquer vértices do grafo. Mas os novos vértices foram trazidos para IN com uso de valores crescentes do vetor d . O caminho mínimo de x a um vértice z que seja incluído em IN depois de y precisa ser pelo menos tão grande quanto o valor de d referente a y quando da inclusão de y em IN . Portanto, não pode haver um caminho menor de x a y passando por z , pois não existe sequer um caminho menor entre x e z .

PRÁTICA 6

Acompanhe o algoritmo *CaminhoMínimo* aplicado ao grafo mostrado na Fig. 6.7. Mostre os valores de p e IN e os valores dos vetores d e s para cada passo do laço **while**. Escreva os vértices do caminho mínimo e sua distância.

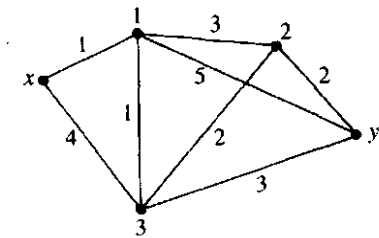


Figura 6.7

Quando procuramos por um novo vértice para ser incluído em IN no algoritmo *CaminhoMínimo*, pode ocorrer de mais de um vértice p apresentar o valor mínimo do vetor d ; neste caso p pode ser selecionado arbitrariamente. Pode ainda haver mais de um caminho mínimo entre os vértices x e y em um grafo.

O algoritmo *CaminhoMínimo* também funciona para grafos direcionados se a matriz de adjacências tiver uma forma apropriada. Também funciona para grafos desconexos; se x e y não estiverem na mesma componente, então $d[y]$ conterá ∞ durante todo o tempo. Após y ser incluído em IN , o algoritmo termina, e este valor ∞ de $d[y]$ indica que não existe caminho entre x e y .

Podemos pensar o algoritmo *CaminhoMínimo* como um algoritmo de "visão curta". Ele não é capaz de ver todo o grafo de uma vez para determinar diversos caminhos mínimos; ele apenas determina, a cada passo, os caminhos mínimos relativos ao conjunto IN . Este tipo de algoritmo é chamado de **algoritmo guloso** — sempre realiza os passos que parecem melhores, baseados apenas em seu conhecimento imediato. Neste caso, o que parece ser o melhor em um determinado momento, se mostra o melhor para a execução do algoritmo como um todo.

Qual a eficiência do algoritmo *CaminhoMínimo*? A maior parte do trabalho parece ter lugar dentro do laço **for** que modifica os vetores d e s . Neste ponto, o algoritmo verifica os n vértices a fim de determinar quais vértices não pertencem a IN e recalcula $d[z]$ para esses vértices, mudando, possivelmente, $s[z]$. Os valores $d[z]$, $d[p]$ e $A[p, z]$ para um dado vértice z estão diretamente disponíveis. Portanto, o laço **for** requer $\Theta(n)$ operações. Além disso, a determinação do vértice p que será incluído em IN também pode ser feita em $\Theta(n)$ operações, verificando todos os n vértices. Com um pouco mais de trabalho para incluir p em IN , cada execução de laço **while** toma $\Theta(n)$ operações. No pior caso, y é o último vértice a ser incluído em IN , e o laço **while** será executado $n - 1$ vezes. Portanto, o número total de operações envolvidas no laço **while** é $\Theta(n(n - 1)) = \Theta(n^2)$. A iniciação e a impressão do resultado tomam $\Theta(n)$ operações, de forma que o algoritmo requer $\Theta(n + n^2) = \Theta(n^2)$ operações em seu pior caso.

O que ocorre se mantivermos IN (ou ainda o complemento de IN) como uma espécie de lista encadeada, de forma que não precisemos examinar todos os vértices do grafo a fim de determinar quais não pertencem a IN ? Certamente, isto tornará o algoritmo mais eficiente. Perceba que o número de vértices que não pertencem a IN é inicialmente $n - 1$ e que este número diminui de 1 a cada passo pelo laço **while**. Dentro do laço **while**, o algoritmo precisa realizar $n - 1$ operações no primeiro passo, $n - 2$ no segundo passo, depois $n - 3$, $n - 4$, e assim por diante. Mas, como uma demonstração por indução poderia mostrar,

$$\begin{aligned}(n-1) + (n-2) + \dots + 1 &= \frac{(n-1)n}{2} \\ &= \Theta(n^2)\end{aligned}$$

Portanto o pior caso continua precisando de $\Theta(n^2)$ operações.

Problema da Árvore Geradora Mínima

Um problema encontrado ao se projetar redes é obter uma forma de conectar todos os vértices de forma eficiente, onde os vértices podem ser computadores, telefones etc. Uma árvore geradora mínima pode fornecer uma solução econômica, que precise de pouco cabo, pouca tubulação, ou pouca quantidade de qualquer que seja o meio de conexão.

Definição: Árvore Geradora

Uma **árvore geradora** de um grafo é uma árvore não-enraizada cujo conjunto de vértices coincide com o conjunto de vértices do grafo e cujas arestas são (algumas das) arestas do grafo.

Uma árvore geradora, portanto, conecta todos os vértices de um grafo sem arestas redundantes (sem ciclos). Existem algoritmos para construção de uma **árvore geradora mínima**, uma árvore geradora que tenha peso mínimo, dado um grafo simples ponderado e conexo.

Um desses algoritmos, o algoritmo de Prim, comporta-se de forma muito parecida com o algoritmo para obtenção do caminho mínimo. Ele usa um conjunto IN , que inicialmente contém um vértice arbitrário. Para cada vértice z que não esteja em IN , avaliamos a menor distância $d[z]$ entre z e qualquer vértice em IN . Incluimos, sucessivamente, vértices a IN , onde o próximo vértice a ser incluído é um vértice que não esteja em IN e cuja distância $d[z]$ seja mínima. A aresta que realiza esta distância mínima é então incluída na árvore geradora. Como pode haver empates entre as distâncias mínimas, a árvore geradora mínima de um grafo não é única. O algoritmo termina quando todos os vértices do grafo pertencem a IN . Não escreveremos o algoritmo (que, como o algoritmo *CaminhoMínimo*, requer $\Theta(n^2)$ operações no pior caso e é um algoritmo guloso), mas vamos ilustrar seu funcionamento com um exemplo.

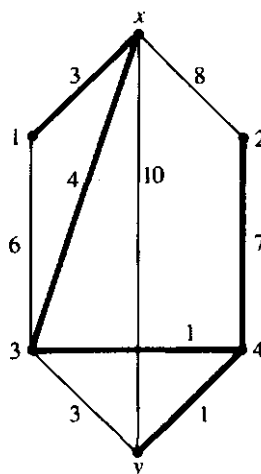


Figura 6.8

EXEMPLO 3 Vamos encontrar uma árvore geradora mínima para o grafo da Fig. 6.4. Fazemos o vértice 1 ser o vértice inicial arbitrário em IN . Em seguida, consideramos todos os vértices adjacentes a qualquer vértice em IN , isto é, todos os vértices adjacentes ao vértice 1, e selecionamos o vértice mais próximo, que é o vértice x . Agora $IN = \{1, x\}$ e consideraremos todos os vértices que não estão em IN e que sejam adjacentes a 1 ou a x . O mais próximo desses vértices é o vértice 3, que está a uma distância de 4 unidades do vértice x . Para $IN = \{1, x, 3\}$, o próximo vértice mais próximo é o vértice 4, a 1 unidade de distância do vértice 3. Os demais vértices incluídos são y e, depois, 2. A Fig. 6.8 mostra a árvore geradora mínima resultante. •

PRÁTICA 7 Encontre uma árvore geradora mínima para o grafo da Fig. 6.7. •

Revisão da Seção 6.2

Técnicas

- Encontrar um caminho mínimo de x a y em um grafo (usando o algoritmo *CaminhoMínimo*)
- Encontrar uma árvore geradora mínima

Idéia Principal

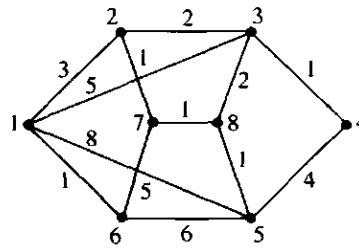
Algoritmos de ordem $\Theta(n^2)$ para o pior caso podem encontrar o caminho mínimo entre dois vértices ou uma árvore geradora mínima em grafos simples ponderados com pesos positivos e conexos com n vértices.

Exercícios 6.2

Para resolver os Exercícios 1 a 4, use o grafo a seguir. Aplique os algoritmos *CaminhoMínimo* (Algoritmo de Dijkstra) para os pares de vértices dados; mostre os valores para p e IN e os valores dos vetores d e s para cada passo do laço **while**. Escreva os vértices que compõem os caminhos mínimos e suas distâncias.

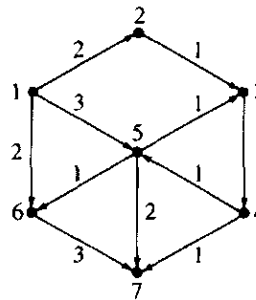
- ★1. Do vértice 2 ao vértice 5
- 2. Do vértice 3 ao vértice 6
- 3. Do vértice 1 ao vértice 5
- 4. Do vértice 4 ao vértice 7

Nos Exercícios 5 e 6, use o grafo direcionado da figura a seguir. Aplique o algoritmo *CaminhoMínimo* (algoritmo de Dijkstra) para os pares de vértices dados; mostre os valores para p e IN e os valores dos vetores d e s para cada passo do laço **while**. Escreva os vértices que compõem os caminhos mínimos e suas distâncias.



Exercícios 1-4

- ★5. Do vértice 1 ao vértice 7
6. Do vértice 3 ao vértice 1



Exercícios 5-6

7. a. Modifique o algoritmo *CaminhoMínimo* a fim de que ele encontre o caminho mínimo de x a todos os outros vértices.
b. Isto altera a ordem de grandeza do algoritmo para seu pior caso?
8. Forneça um exemplo que mostre que o algoritmo *CaminhoMínimo* não funciona com pesos negativos.

Outro algoritmo para obtenção de caminhos mínimos a partir de um único vértice de origem para todos os outros vértices no grafo é o *algoritmo de Bellman-Ford*. Ao contrário do algoritmo de Dijkstra, que mantém um conjunto de vértices para os quais o caminho mínimo já foi determinado, independentemente do comprimento desses caminhos (isto é, o número de arestas no caminho); o algoritmo de *Bellman-Ford* realiza uma busca que visa encontrar sucessivamente caminhos de comprimento 1, depois de comprimento 2, depois de comprimento 3 etc, até um máximo de $n - 1$ como comprimento (se existir um caminho, ele não pode ter comprimento maior do que $n - 1$). Uma descrição em pseudocódigo do algoritmo de *Bellman-Ford* é fornecida a seguir (algoritmo *OutroCaminhoMínimo*); quando usamos este algoritmo, a matriz de adjacências A deve conter $A[i, i] = 0$ para todo i .

ALGORITMO *OutroCaminhoMínimo*,

procedure *OutroCaminhoMínimo*{ A : matriz $n \times n$; x : vértice;
var d : vetor de inteiros;
var s : vetor de vértices};

{ Algoritmo de Bellman-Ford — A é uma matriz de adjacências modificada para um grafo simples, ponderado e conexo; x é um vértice no grafo; quando o procedimento termina, os vértices no caminho mínimo de x a y são $y, s[y], s[s[y]], \dots, x$; a distância deste caminho é $d[y]$. }

var

z, p : vértice; {vértices temporários}
 t : vetor de inteiros; {vetor temporário de distâncias criado a cada iteração}
 i : integer;

```

begin
  {inicia os vetores  $d$  e  $s$  — estabelece os caminhos mínimos de comprimento 1 a partir de  $x$  }
   $d[x] := 0$ ;
  for todos os vértices  $z$  diferentes de  $x$  do
    begin
       $d[z] := A[x, z]$ ;
       $s[z] := x$ ;
    end;

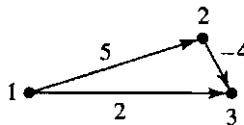
  {encontra os caminhos mínimos de comprimentos 2, 3, etc.}
  for  $i := 2$  to  $n - 1$  do
    begin
       $t := d$ ; {copia o vetor  $d$  corrente em  $t$ }
      {altera  $t$  a fim de guardar os menores caminhos de comprimento  $i$ }
      for todos os vértices  $z$  diferentes de  $x$  do
        begin
          {encontra o caminho mínimo com uma aresta a mais}
           $p :=$  vértice em  $G$  para o qual  $(d[p] + A[p, z])$  é mínimo
           $t[z] := d[p] + A[p, z]$ ;
          if not ( $p = z$ ) then
             $s[z] := p$ ;
          end;

           $d := t$ ; {copia  $t$  de volta para  $d$ }
        end; {laço for}
      end;
    end;

```

Nos Exercícios 9 a 12, use o algoritmo *OutroCaminhoMínimo* (algoritmo de Bellman-Ford) para encontrar o caminho mínimo entre o vértice origem e todos os outros vértices. Mostre os sucessivos valores de d e de s .

- ★9. Grafo dos Exercícios 1-4, vértice origem = 2 (compare sua resposta com a do Exercício 1)
10. Grafo dos Exercícios 1-4, vértice origem = 1 (compare sua resposta com a do Exercício 3)
11. Grafo dos Exercícios 5-6, vértice origem = 1 (compare sua resposta com a do Exercício 5)
12. Grafo a seguir, vértice origem = 1 (compare sua resposta com a do Exercício 8)



Exercício 12

Para computar a distância do caminho mínimo entre quaisquer dois vértices de um grafo, o algoritmo *CaminhoMínimo* poderia ser usado diversas vezes, com cada vértice como vértice origem de cada vez. Um algoritmo diferente, muito semelhante ao algoritmo de Warshall, também pode ser usado para resolver este problema de caminho mínimo entre "todos os pares". Uma descrição deste algoritmo é mostrada a seguir, onde A é a matriz de adjacências do grafo com $A[i, i] = 0$ para todo i .

ALGORITMO *CaminhoMínimoTodosOsPares*

```

procedure CaminhoMínimoTodosOsPares  $ywc$   $A$ : matriz  $n \times n$ ;
  {Algoritmo de Floyd — calcula o caminho mínimo entre quaisquer dois vértices; inicialmente  $A$  é a matriz
  de adjacências; ao fim,  $A$  conterá todas as distâncias dos menores caminhos}

  var
     $i, j, k$ : integer;

```

```

begin
  for k := 1 to n do
    for i := 1 to n do
      for j := 1 to n do
        if  $A[i, k] + A[k, j] < A[i, j]$  then
           $A[i, j] := A[i, k] + A[k, j]$ ;
end;
```

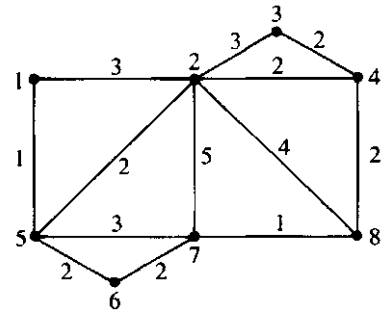
Nos Exercícios 13 e 14 use o algoritmo *CaminhoMínimoTodosOsPares* (algoritmo de Floyd) para encontrar as distâncias de todos os caminhos mínimos. Mostre os sucessivos valores da matriz A para cada passo do laço externo.

★13. Fig. 6.7

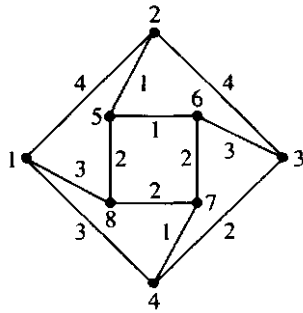
14. Grafo dos Exercícios 1-4

Nos Exercícios 15 a 18, encontre uma árvore geradora mínima para o grafo na figura correspondente.

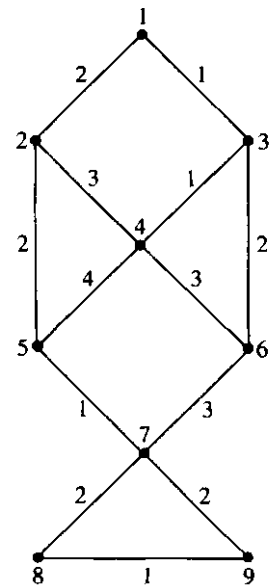
15. Grafo dos Exercícios 1-4



★17.



18.



O algoritmo de Kruskal é outro algoritmo para a obtenção da árvore geradora mínima de um grafo. Enquanto no algoritmo de Prim a árvore "cresce" a partir de um ponto inicial arbitrário, através da inclusão de arestas pequenas adjacentes, o algoritmo de Kruskal inclui arestas na ordem de suas distâncias crescentes, sempre que elas puderem pertencer ao grafo. Os empates são resolvidos arbitrariamente. A única restrição é que uma aresta não seja incluída se a inclusão dela resulta em um ciclo. O algoritmo termina quando todos os vértices tiverem sido incorporados a uma estrutura conexa. Uma descrição (muito informal) em pseudocódigo é mostrada a seguir:

ALGORITMO *OutroAGM*

```

procedure OutroAGM (A: matriz  $n \times n$ ;
                      var T: coleção de arestas);
{Algoritmo de Kruskal para obtenção de uma árvore geradora mínima; T inicialmente está vazia; ao fim, T
 = árvore geradora mínima}

begin
  ordena as arestas de G de forma crescente
  repeat
    if próxima aresta na ordem não cria um ciclo then
      inclui a aresta em T;
    until T é conexo e contém todos os vértices de G;
  end;

```

Nos Exercícios 19 a 22 use o algoritmo *OutroAGM* (algoritmo de Kruskal) para encontrar a árvore geradora mínima.

★19. Grafo dos Exercícios 1 -4

20. Grafo do Exercício 16

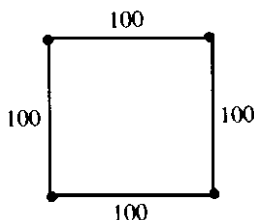
21. Grafo do Exercício 17

22. Grafo do Exercício 18

23. Apresente um exemplo que mostre que a inclusão do vértice mais próximo a *IN* em cada passo, como é feito no algoritmo de Prim para árvore geradora mínima, não garante um caminho mínimo.

★24. Admita que os pesos das arestas representam distâncias. Então a inclusão de novos vértices e arestas a um grafo pode resultar em um novo grafo com árvore geradora mínima com peso menor do que a árvore geradora mínima do grafo original. (A nova árvore geradora pode representar uma rede de custo mínimo para comunicação entre um grupo de cidades obtida através da instalação de uma estação de retransmissão construída fora das cidades.)

a. Encontre uma árvore geradora de peso mínimo para o grafo rotulado da figura dada. Qual é o peso desta árvore?



Exercício 24

b. Inclua um vértice no interior do quadrado. Inclua novas arestas entre o vértice do centro e os vértices do quadrado. Encontre uma árvore geradora para o novo grafo, e calcule seu peso aproximado.

Seção 6.3 Algoritmos de Busca

Apesar de termos considerado neste capítulo caminhos em um grafo sob certas condições — sem passar pela mesma aresta mais de uma vez, sem passar por um vértice mais de uma vez ou minimizando a distância percorrida. Nesta seção, veremos um problema mais simples — apenas desejamos listar todos os vértices de um grafo simples e conexo em uma ordem particular. Isto significa que precisamos visitar cada vértice pelo menos uma vez, mas podemos visitá-lo mais de uma vez se não o listarmos novamente. Podemos também percorrer mais de uma vez as arestas do grafo se necessário, e isto será geralmente necessário se desejarmos visitar todos os vértices de uma árvore. Este processo é chamado de **busca em um grafo** e vamos apresentar diversos algoritmos para isto.

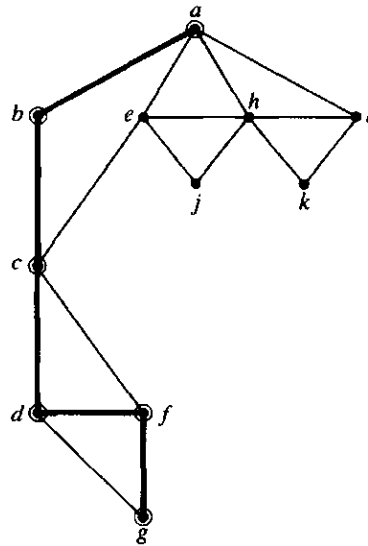


Figura 6.9

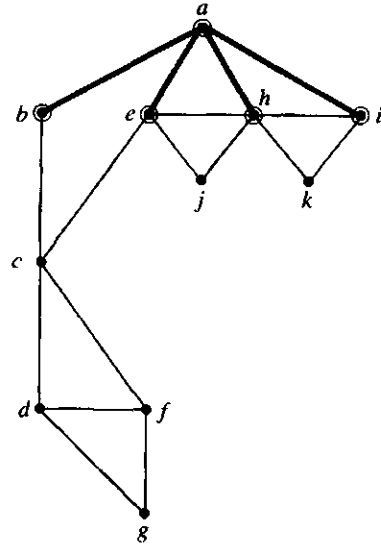


Figura 6.10

Busca em Profundidade e Busca em Largura

Nos dois primeiros algoritmos para busca em grafos, começamos em um vértice arbitrário a do grafo, marcamos-lo como visitado e o escrevemos. No procedimento de **busca em profundidade**, prosseguimos em um caminho para longe de a , visitando e escrevendo os vértices, procedendo para o mais longe possível, até que não haja mais vértices sem terem sido visitados neste caminho. Neste ponto, voltamos pelo caminho, trilhando novos caminhos a cada vértice, sempre que possível; até que chegamos de volta a a . Na **busca em largura**, partimos do vértice a a fim de visitar todos os vértices que lhe são adjacentes; em seguida, partimos desses vértices, e assim por diante. Para ilustrar a diferença entre essas duas abordagens, a Fig. 6.9 mostra os primeiros vértices visitados (marcados com círculos) em um grafo através de uma busca em profundidade, e a Fig. 6.10 mostra os primeiros vértices visitados no mesmo grafo quando percorrido por uma busca em largura.

Busca em Profundidade

A forma mais simples de descrever um algoritmo de busca em profundidade é usar uma recursão, onde o algoritmo chama a si mesmo ao longo de sua execução. No algoritmo a seguir, a entrada é um grafo G simples e conexo e um dado vértice a ; a saída é uma lista de todos os vértices em G percorridos em profundidade a partir de a .

ALGORITMO *Profundidade*

```

procedure Profundidade( $G$ : grafo;  $a$ : vértice);
{escreve os vértices do grafo  $G$  na ordem do percorrimento em profundidade a partir do vértice  $a$  }

begin
  marque  $a$  como visitado;
  escreva( $a$ );
  for cada vértice  $n$  adjacente a  $a$  do
    if  $n$  não foi visitado then
      Profundidade( $G$ ,  $n$ );
end;

```

No passo recursivo, o algoritmo é chamado com um novo vértice especificado como o ponto inicial. Não indicamos como marcar os vértices como visitados ou como encontrar quais vértices n são adjacentes a a .

EXEMPLO 4

Aplicaremos uma busca em profundidade ao grafo da Fig. 6.9, onde a é o vértice inicial. Primeiro, marcamos que a foi visitado (é interessante, ao acompanharmos a execução do algoritmo, circundarmos os vértices visitados), e então escrevemos a . Em seguida, procuramos, dentre os vértices adjacentes a a , um vértice que não

tenha sido visitado. Temos diversas possibilidades (b , e , h e i); selecionemos o vértice b . (Para que todos nós tenhamos a mesma resposta, vamos selecionar o vértice que seja o primeiro em ordem alfabética; na prática, esta escolha dependeria de como os vértices estão armazenados na representação do grafo.) Chamamos então o algoritmo de busca em profundidade começando pelo vértice b .

Isto significa que voltamos ao início do procedimento, dessa vez com o vértice b especificado no lugar do vértice a . Então, primeiro marcamos b como visitado e o escrevemos. Em seguida, procuramos dentre os vértices adjacentes a b por um vértice que ainda não tenha sido marcado. O vértice a é adjacente a b , mas já foi marcado. O vértice c serve e chamamos o procedimento novamente, especificando-o como ponto de partida.

O vértice c é marcado e escrito, e procuramos vértices adjacentes a c que ainda não tenham sido marcados. Pela nossa convenção da ordem alfabética, selecionamos o vértice d . Continuando desta forma, visitamos a seguir o vértice e , então, o vértice g . Quando chegamos ao vértice g , chegamos a um beco sem saída, pois não há vértices adjacentes que ainda não tenham sido visitados. Portanto, o laço **for** da versão do procedimento executada com o vértice g como parâmetro é terminado. (O grafo, neste ponto, se parece com o mostrado na Fig. 6.9.)

Terminamos, portanto, com a execução do algoritmo para o vértice g , mas o vértice g era um dos vértices adjacentes ao vértice f , e continuamos no laço **for** para a versão do procedimento chamado com o vértice f . Mas, como g era o único vértice não visitado adjacente a f , o laço **for**, e portanto o procedimento para o vértice f são terminados. De forma semelhante, voltamos ao vértice d , o procedimento não encontra outros vértices adjacentes ainda não-marcados, e volta à versão do procedimento chamada para o vértice c . Por isso, após processar o vértice d e tudo o que o segue até o beco sem saída, estamos de volta ao laço **for** do procedimento aplicado ao vértice c . Procuramos outros vértices adjacentes a c que não estejam marcados e encontramos um — o vértice e . Assim, aplicamos o algoritmo de busca em profundidade ao vértice e , que nos leva os vértices h , i e k antes de chegarmos a outro beco sem saída. Voltando, temos um novo caminho a ser tentado no vértice h , que nos leva ao vértice f . A lista completa dos vértices, na ordem em que são escritos, é

$a, b, c, d, f, g, e, h, i, k, j$

O Exemplo 4 faz com que o processo de busca em profundidade pareça muito complexo, mas é muito mais fácil de ser realizado do que descrito, como você poderá constatar na Prática 8.

PRÁTICA 8

Escreva os vértices visitados em uma busca por profundidade no grafo da Fig. 6.11. Comece pelo vértice a .

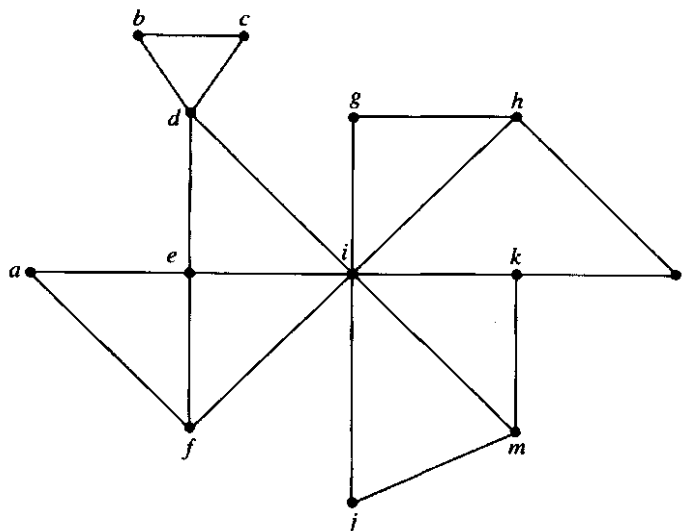


Figura 6.11

Busca em Largura

Em uma busca em largura, os vértices são visitados em torno do vértice corrente de forma semelhante a círculos concêntricos em um lago. Para escrever o algoritmo de uma forma elegante, usaremos uma estrutura de *fila*. Uma *fila* é simplesmente uma estrutura na qual as novas chegadas são incluídas ao fim e as saídas têm lugar na frente. Um exemplo de fila pode ser uma fila de clientes em um supermercado — um novo cliente

entra no fim da fila e os clientes da frente saem dela à medida que o caixa fica livre. A notação *IncluiNaFila(a, Q)* denota a inclusão do elemento *a* ao fim da fila *Q* e a notação *TiraDaFila(Q)* denota a remoção do elemento no início da fila *Q*. Usamos ainda a função *frente(Q)* para retornar o elemento no início da fila *Q* sem retirá-lo da fila. No algoritmo apresentado no quadro a seguir (algoritmo *Largura*), a entrada é um grafo *G* simples e conexo e um vértice *a*; a saída é uma lista de todos os vértices de *G* percorridos em largura a partir de *a*.

ALGORITMO *Largura*

```

procedure Largura(G: grafo; a: vértice);
{escreve os vértices do grafo G na ordem da busca em largura a partir do vértice a}

var
  Q: fila de vértices;

begin
  inicia Q como vazia;
  marque a como visitado;
  escreva(a);
  IncluiNaFila(a, Q);

  while Q não está vazia do
    for cada vértice n adjacente à Frente(Q) do
      if n não foi visitado then
        begin
          marque n como visitado;
          escreva(n);
          IncluiNaFila(n, Q);
        end;
      TiraDaFila(Q);
    end;

```

EXEMPLO 5

Vamos fazer uma busca em largura no grafo da Fig. 6.10 (este é o mesmo grafo no qual realizamos a busca em profundidade do Exemplo 4). Começamos iniciando uma lista vazia *Q*, marcando o vértice *a* como visitado e escrevendo-o; então incluímos *a* na fila. Quando chegamos pela primeira vez ao laço **while**, a fila não está vazia e *a* é o elemento à frente da fila. No laço **for**, procuramos por vértices adjacentes a *a* que ainda não tenham sido visitados, escrevemos estes vértices e os incluímos ao fim da fila. Podemos ter diversos vértices a serem visitados neste ponto; como antes, e apenas por convenção, vamos visitá-los em ordem alfabética. Portanto, a primeira vez que completarmos o laço **for**, teremos visitado e escrito os vértices *b*, *e*, *h* e *i*, nesta ordem, e os teremos incluído ao fim da fila. O grafo neste ponto se parece com o da Fig. 6.10. Removemos, então, o vértice *a* do início da fila, que se torna (do início para o fim)

b, e, h, i

Na próxima iteração do laço **while**, *b* é o elemento do início da fila, e o laço **for** procura vértices adjacentes a *b* que ainda não tenham sido visitados. O único vértice que ainda não foi visitado aqui é *c*, que é escrito e incluído ao fim da fila. Após a remoção de *b*, a fila contém

e, h, i, c

Realizando o laço **while** mais uma vez, *e* está no início da fila. Uma busca nos vértices adjacentes a *e* produz um novo vértice, o vértice *f*. O grafo agora se parece com o mostrado na Fig. 6.12, e após removermos *e* da fila, a fila contém

h, i, c, j

Quando procuramos vértices adjacentes a *h*, temos um novo vértice, o vértice *k*. Quando procuramos por vértices adjacentes a *i*, nenhum novo vértice é incluído na fila. Quando *c* se torna o primeiro elemento da fila, uma busca nos vértices adjacentes a *c* gera dois novos vértices: *d* e *f*. Após incluir esses dois novos vértices na fila (e retirar *c*), a fila contém

j, k, d, f

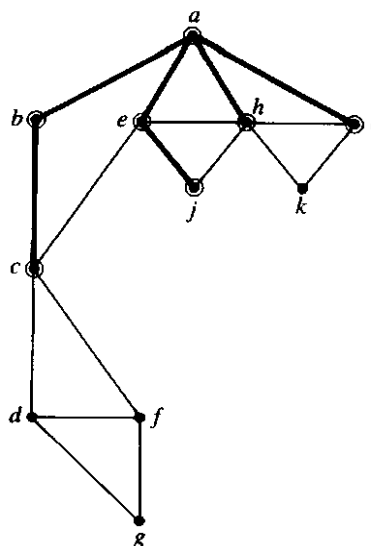


Figura 6.12

A busca por vértices adjacentes a j e depois a k não inclui novos vértices à fila. Quando o início da fila é d , um novo vértice, g , é encontrado e a fila (após remover d) é

f, g

O processamento de f e g não gera novos vértices. Após g ter sido removido da fila, esta fica vazia. O laço **while** — bem como o algoritmo — termina. A lista de vértices escritos durante este processo, isto é, os vértices escritos segundo a ordem de uma busca em largura a partir de a são

$a, b, e, h, i, c, j, k, d, f, g$

•

Assim como no algoritmo de busca em profundidade, o algoritmo de busca em largura não é difícil de ser acompanhado; precisamos apenas manter controle dos vértices que já foram visitados e o conteúdo corrente da fila.

PRÁTICA 9

Escreva os vértices obtidos por uma busca em largura no grafo da Fig. 6.11, começando pelo vértice a .

Análise

Quanto trabalho os algoritmos de busca em profundidade e de busca em largura demandam? Ambos procuram todos os vértices adjacentes a um dado vértice que ainda não tenham sido visitados. Suponha que o grafo contenha n vértices e m arestas. Uma das vantagens de representar um grafo através de sua lista de adjacências ao invés de sua matriz de adjacências é que esta operação em particular é mais eficiente; para encontrar os vértices adjacentes ao vértice i , precisamos percorrer a lista de adjacências de i , que pode ser pequena, ao invés da i -ésima coluna da matriz de adjacências, que contém n entradas. Portanto, admitiremos que estamos usando a representação na forma de lista de adjacências para o grafo.

Na busca em largura, o algoritmo procura toda a lista de adjacências do vértice na frente da fila, marcando, escrevendo e incluindo na fila os vértices encontrados que ainda não tenham sido visitados. Na busca em profundidade, o algoritmo pode ser interrompido diversas vezes, enquanto varre a lista de adjacências de um dado vértice para passar (devido à recursão) ao processamento da lista de adjacências de outros vértices. Ao final, no entanto, toda a lista de adjacências é tratada.

Varrer a lista de adjacências do grafo determina a quantidade de trabalho realizado em cada caso. Existem n listas de adjacências, de forma que a quantidade de trabalho é pelo menos $\Theta(n)$, porque a lista de adjacências para todos os vértices precisa ser verificada, mesmo que esteja vazia. Como existem m arestas, o trabalho para percorrer todas as listas de adjacências é pelo menos $\Theta(m)$. Portanto, tanto o algoritmo de busca em profundidade como o algoritmo de busca em largura são algoritmos $\Theta(\max(n, m))$. Se houver mais arestas que vértices (o caso mais comum) então $\Theta(\max(n, m)) = \Theta(m)$.

Aplicações

Os algoritmos de busca em profundidade e de busca em largura podem ser usados como base para realizar outras tarefas relacionadas com grafos, algumas das quais já resolvemos.

Uma estrutura de árvore não-enraizada que seja um subgrafo do grafo original pode ser associada a cada busca. Quando varremos a lista de adjacências do vértice i , se o vértice j for adjacente ao vértice i e não tiver sido visitado anteriormente, então a aresta $i-j$ é incluída ao subgrafo. Como nenhuma aresta de algum vértice já visitado é usada, não são criados ciclos e o subgrafo é, portanto, uma árvore não-enraizada. Como todos os vértices são visitados, esta árvore é uma árvore geradora para o grafo. Toda árvore tem $n - 1$ arestas, o número mínimo de arestas para conectar n vértices. Estamos assumindo que as arestas não são ponderadas, mas se as considerarmos ponderadas e com peso 1, então a árvore obtida é uma árvore geradora mínima.

As linhas escuras da Fig. 6.9 fazem parte da árvore associada à busca em profundidade do Exemplo 4, e as linhas escuras da Fig. 6.10 e 6.12 fazem parte da árvore associada à busca em largura do Exemplo 5.

PRÁTICA 10

- Complete a árvore associada à busca em profundidade do Exemplo 4.
- Complete a árvore associada à busca em largura do Exemplo 5.

Os algoritmos de busca em profundidade e de busca em largura também se aplicam a grafos direcionados, e com isso fornecem um novo algoritmo para alcançabilidade. Para determinar se o vértice j é alcançável a partir do vértice i , realize uma busca em profundidade (ou em largura), começando pelo vértice i ; quando o algoritmo terminar, verifique se o vértice j foi visitado. A alcançabilidade para "todos os pares", isto é, quais vértices são alcançáveis a partir de quais vértices, pode ser obtida executando-se buscas em profundidade (ou em largura) começando por todos os vértices do grafo. Isto requer um trabalho de ordem $\Theta(n \cdot \max(n, m))$. Se o grafo for bem esparsos, caso no qual temos $\max(n, m) = n$, teremos um algoritmo $\Theta(n^2)$ para alcançabilidade. Lembremos que o algoritmo de Warshall (Seção 5.3) é um algoritmo $\Theta(n^3)$. Esta melhora se deve ao fato de que em grafos esparsos, a maioria das listas de adjacências são pequenas ou vazias, enquanto o algoritmo de Warshall processa as entradas da matriz de adjacências ainda que essas entradas sejam 0s. Mas se o grafo não for esparsos, o número de arestas pode ser $\Theta(n^2)$, caso no qual $\Theta(n \cdot \max(n, m)) = \Theta(n^3)$, a mesma ordem que a do algoritmo de Warshall. Além disso, o algoritmo de Warshall tem a vantagem de ter uma implementação mais simples!

Na Seção 4.2, definimos uma ordenação topológica como forma de estender uma ordenação parcial em um conjunto finito a fim de que se torne ordenação total. Seja o conjunto parcialmente ordenado representado por um grafo direcionado. A ordenação topológica será obtida por uma contagem nos vértices, portanto seja 0 o valor inicial da contagem. Tome um vértice como origem e realize um algoritmo de busca em profundidade a partir deste vértice. Sempre que a busca voltar de um vértice pela última vez, atribua a ele o próximo valor do contador. Quando o algoritmo de busca em profundidade terminar, tome um vértice ainda não visitado como origem para outro algoritmo de busca em profundidade e continue a incrementar o contador. Continue este processo até que não haja mais vértices sem terem sido visitados no grafo. Uma ordenação topológica pode ser obtida pela listagem dos vértices na ordem inversa de seus valores atribuídos. Este processo de ordenação topológica funciona porque atribuímos o valor do contador quando voltamos de um vértice pela última vez. Este valor será maior do que os números atribuídos aos vértices alcançáveis por ele, isto é, todos os vértices dos quais ele é predecessor na ordenação parcial.

EXEMPLO 6

A Fig. 6.13a é um grafo direcionado que representa ordenação parcial. Ao escolhermos d (arbitrariamente) como vértice inicial e realizarmos uma busca em profundidade, visitamos e e f , e neste ponto, devemos voltar. Ao vértice f é atribuído o valor 1, mas ainda não terminamos com o vértice e , pois podemos visitar g . Ao voltarmos de g , atribuímos a ele o valor 2. Neste ponto, voltamos a e pela última vez e atribuímos a ele o valor 3, e então a d o valor 4. Escolha a como o vértice inicial para outra busca em profundidade. Visitamos o vértice c e então voltamos, de forma que c e a recebem os valores 5 e 6, respectivamente. Começando com b como vértice inicial, não há lugar para ir e b recebe o valor 7. Não existem vértices sem serem visitados no grafo e o processo termina. O esquema de numeração é mostrado na Fig. 6.13b.

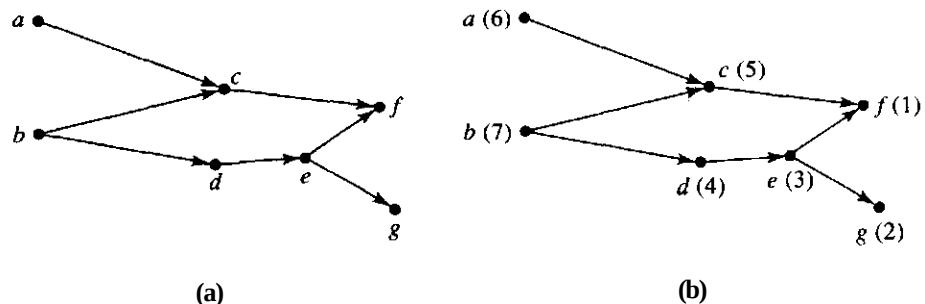


Figura 6.13

Apresentando os valores atribuídos em ordem inversa, obtemos

7	6	5	4	3	2	1
<i>b</i>	<i>a</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>g</i>	<i>f</i>

que é uma ordenação topológica.

PRÁTICA 11

Use o algoritmo de busca em profundidade para realizar uma ordenação topológica no grafo da Fig. 6.14. Indique os valores atribuídos aos vértices no grafo.

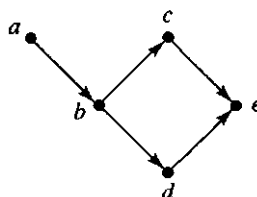


Figura 6.14

Considere agora um grafo G (não-direcionado) que pode não ser um grafo conexo. Uma **componente conexa** de G é um subgrafo de G que é conexo e não é um subgrafo de um subgrafo conexo maior. Na Fig. 6.15 existem três componentes conexas. Naturalmente, se o grafo original for conexo, então ele tem apenas uma componente conexa.

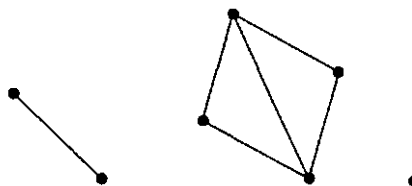


Figura 6.15

Buscas em profundidade e em largura podem ser usadas para determinar as componentes conexas de um grafo. Tomamos um vértice arbitrário como inicial, e realizamos a busca. Quando o algoritmo termina, todos os vértices visitados formam uma componente. Procuramos então um vértice ainda não visitado para servir de início para outra busca que produzirá uma segunda componente. Repetimos este processo, até que não haja mais vértices sem serem visitados no grafo.

Apesar de termos definido a alcançabilidade apenas para grafos direcionados, o conceito também se aplica a grafos não-direcionados e desconexos. Consideremos apenas grafos não-direcionados e desconexos, mas admitamos a convenção de que ainda que não haja laços, cada vértice possa ser alcançado por si próprio. A alcançabilidade torna-se então uma relação de equivalência no conjunto dos vértices do grafo; nossa convenção impõe a propriedade reflexiva, e a simetria e transitividade seguem do fato do grafo ser não-direcionado. Esta relação de equivalência particiona os vértices do grafo em classes de equivalências que consistem nos vértices de cada componente conexa do grafo. O algoritmo de Warshall pode ser aplicado a grafos não-direcionados bem como a grafos direcionados e resulta numa matriz na qual os vértices que compõem cada componente conexa do grafo podem ser determinados, mas isto requer mais esforço computacional do que usar a busca em profundidade. O uso de uma busca em profundidade para identificar as *componentes biconexas* em um grafo será discutido na seção seguinte.

Como uma última observação sobre busca em profundidade, vimos na Seção 1.5 que a linguagem de programação Prolog, quando processa uma pesquisa baseada em uma definição recursiva, segue uma estratégia de busca em profundidade (Exemplo 27).

Busca em Árvores

Como as buscas em profundidade e em largura aplicam-se a qualquer grafo, elas também se aplicam a árvores. No entanto existem três outros algoritmos de busca que são úteis para árvores — preordem, inordem e posordem.

Nesses métodos de busca, é útil usar a definição recursiva de uma árvore, onde a raiz de uma árvore tem ramificações para raízes de subárvores (veja a Fig. 6.16). Assumiremos ainda que uma árvore T tem raiz r ; quaisquer subárvores são rotuladas, da esquerda para a direita, como $T_1, T_2, \dots, T_t$. Como estamos usando uma definição recursiva de uma árvore, será fácil enunciar os algoritmos de busca de forma recursiva.

Os termos *preordem*, *inordem* e *posordem* referem-se à ordem na qual a raiz da árvore é visitada em relação aos vértices de suas subárvores. Na busca **em preordem**, a raiz da árvore é visitada primeiro e então suas subárvores são visitadas da esquerda para direita, cada qual em preordem.

No algoritmo a seguir, a entrada é uma árvore T com subárvores, da esquerda para a direita, $T_1, T_2, \dots, T_t$; a saída é uma lista de vértices em preordem.

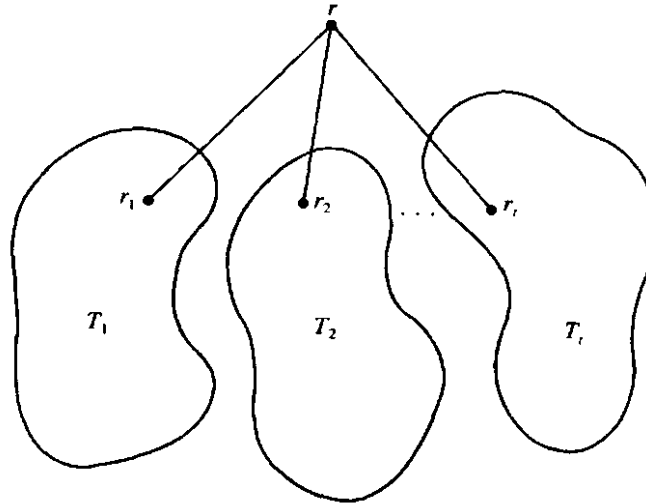


Figura 6.16

ALGORITMO *Preordem*

```

procedure Preordem( $T$ : árvore);
{escreve os vértices de uma árvore de raiz  $r$  em preordem}

begin
  escreva( $r$ );
  for  $i := 1$  to  $t$  do
    Preordem( $T_i$ );
end;

```

Na **busca em inordem**, a subárvore a esquerda é processada por outra busca em inordem, em seguida a raiz é visitada e então as demais subárvores são processadas da esquerda para a direita, cada qual em inordem. Se a árvore for uma árvore binária a raiz é visitada entre o processamento das duas subárvores.

No algoritmo a seguir, a entrada é uma árvore T com, da esquerda para a direita, subárvores $T_1, T_2, \dots, T_t$; a saída é uma lista de vértices em inordem.

ALGORITMO *Inordem*

```

procedure Inordem( $T$ : árvore);
{escreve os vértices de uma árvore com raiz  $r$  em inordem}

begin
  if  $t > 0$  then      {trata caso da subárvore ser uma folha}
    Inordem( $T_1$ );
  Escreva( $r$ );
  for  $i := 2$  to  $t$  do
    Inordem( $T_i$ );
end;

```

Finalmente, na **busca em posordem**, a raiz é visitada por último, após todas as subárvores terem sido processadas da esquerda para a direita em posordem.

No algoritmo a seguir, a entrada é uma árvore T com, da esquerda para a direita, subárvores $T_1, T_2, \dots, T_t$; a saída é uma lista de vértices em posordem.

ALGORITMO *Posordem*

```

procedure Posordem( $T$ : árvore);
{escreve os vértices de uma árvore com raiz  $r$  em posordem}

begin
  for  $i := 1$  to  $t$  do
    Posordem( $T_i$ );
  Escreva( $r$ );
end;

```

EXEMPLO 7

Considere a árvore mostrada na Fig. 6.17. Para uma busca em preordem, primeiro escrevemos a raiz a e então realizamos uma busca em preordem na subárvore à esquerda, com raiz b e então procedemos uma busca em preordem na subárvore à esquerda de b , cuja raiz é d . Escrevemos d e então realizamos uma busca em preordem na subárvore à esquerda de d , cuja raiz é i . Após escrevermos i e tentarmos realizar uma busca em preordem nas subárvores de i , não encontramos subárvores. Portanto terminamos com a busca na subárvore cuja raiz é i , e voltamos para considerar todas as subárvores de d ; que não existem.

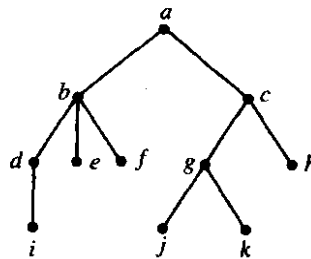


Figura 6.17

Voltando a b , encontramos outras subárvores de b . Processando-as da esquerda para a direita, escrevemos e e então f . Neste ponto, terminamos com as subárvores de b , e voltamos ao vértice a para examinarmos as subárvores mais à direita, encontrando uma subárvore de raiz c . Escrevemos c e passamos a sua subárvore mais à esquerda com raiz g e escrevemos g . Processando as subárvores de g , nós escrevemos j e k ; então voltamos para c , processamos a subárvore que resta e escrevemos h . O vértice c não tem outras subárvores, voltamos para a , que também não tem outras subárvores e, portanto, terminamos. A lista dos vértices visitados em preordem é

$a, b, d, i, e, f, c, g, j, k, h$

Devido à forma pela qual os vértices são rotulados, esta mesma lista pode ser produzida por um algoritmo de busca em profundidade, começando pelo vértice a .

EXEMPLO 8

Para realizar uma busca em inordem da árvore da Fig. 6.17 nós processamos as subárvores à esquerda antes. Isto nos leva ao vértice i , que não tem subárvores. Portanto, escrevemos i e voltamos a d . Já processamos a subárvore à esquerda de d , de forma que podemos escrever d . Uma vez que o vértice d não tem mais subárvores, voltamos ao vértice b . Já processamos a subárvore à esquerda de b , de forma que escrevemos b , e então processamos suas demais subárvores, escrevendo e e f . Voltamos ao vértice a , escrevemo-lo, e então processamos a subárvore à direita de a . Isto nos dá os vértices j, g, k, c e h , nesta ordem, e terminamos. Portanto, a lista em inordem dos vértices é

$i, d, b, e, f, a, j, g, k, c, h$

EXEMPLO 9 A árvore da Fig. 6.17 produz a seguinte lista em posordem dos vértices:

$i, d, e, f, b, j, k, g, h, c, a$

PRÁTICA 12 Realize buscas em preordem, inordem e posordem na árvore da Fig. 6.18.

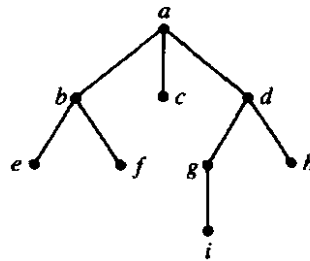


Figura 6.18

EXEMPLO 10

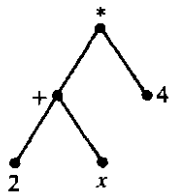


Figura 6.19

O Exemplo 14 do Cap. 5 mostrou como expressões algébricas podem ser representadas como árvores binárias. Se realizarmos uma busca em inordem de uma árvore de expressão, recuperaremos a expressão algébrica original. Para a árvore da expressão da Fig. 6.19, por exemplo, uma busca em inordem nos dá a expressão

$$(2 + x) * 4$$

onde os parênteses são incluídos ao terminarmos o processamento de uma subárvore. Esta forma de se escrever as expressões algébricas, com o símbolo da operação entre os dois operandos é chamada de **notação infixa**. Os parênteses são necessários para indicar a ordem das operações. Sem os parênteses, a expressão seria $2 + x * 4$ que também é uma expressão infixa mas, devido à ordem de precedência da multiplicação sob a adição, não é o desejado.

Uma busca em preordem da Fig. 6.19 nos dá a expressão

$$* + 2 x 4$$

Neste caso, o símbolo da operação precede os operandos. Esta forma de expressão é chamada de **notação prefixa** ou **notação polonesa**. A expressão pode ser traduzida para forma infixa da seguinte maneira:

$$* + 2 x 4 \rightarrow * (2 + x) 4 \rightarrow (2 + x) * 4$$

Uma busca em posordem fornece a expressão

$$2 x + 4 *$$

onde o símbolo de operação segue seus operandos. Esta forma de representar as expressões é chamada de **notação posfixa** ou **notação polonesa reversa**. A expressão pode ser traduzida para a forma infixa da seguinte maneira:

$$2 x + 4 * \rightarrow (2 + x) 4 * \rightarrow (2 + x) * 4$$

As notações prefixa e posfixa não requerem parênteses para evitar ambigüidades. Estas notações fornecem, portanto, uma forma mais eficiente, ainda que menos familiar, de representar expressões algébricas que na notação infixa. Essas expressões podem ser efetuadas sequencialmente, sem a necessidade de "olhar adiante" para localizar expressões em parênteses. Os compiladores normalmente trocam as expressões algébricas dos programas da ordem infixa para a notação posfixa a fim de possibilitar um processamento mais eficiente. •

PRÁTICA 13 Escreva a árvore da expressão para

$$a + (b \cdot c - d)$$

e escreva a expressão em notação prefixa e posfixa. •

Revisão da Seção 6.3

Técnicas

- Realizar uma busca em largura ou em profundidade de um grafo
- Conduzir uma busca em preordem, inordem ou posordem de uma árvore

Idéias Principais

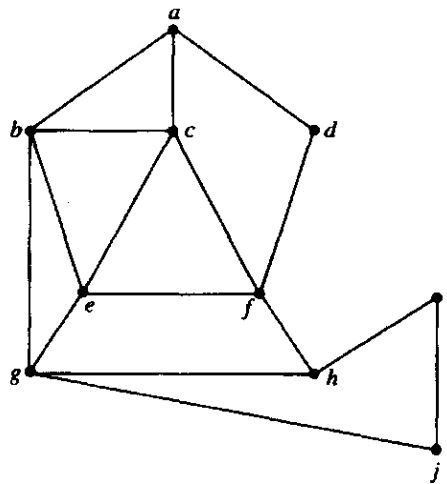
Existem algoritmos para visitar os vértices de um grafo de forma sistemática.

Algoritmos de busca em profundidade e em largura servem como base para outros algoritmos.

Exercícios 6.3

Nos Exercícios 1 a 6, escreva os vértices visitados por uma busca em profundidade no grafo mostrado na figura a seguir, começando pelo vértice indicado.

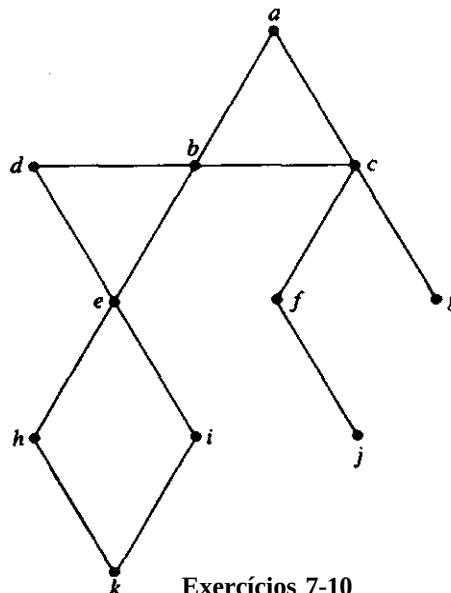
★1. *a* 2. *c* 3. *d* 4. *g* ★5. *e*



Exercícios 1-6

Nos Exercícios 7 a 10, escreva os vértices visitados por uma busca em profundidade no grafo mostrado na figura a seguir, começando pelo vértice indicado.

★7. *a* 8. *e* ★9. *f* 10. *h*



Exercícios 7-10

Nos Exercícios 11 a 16, escreva os vértices visitados por uma busca em largura no grafo dos Exercícios 1 a 6, começando pelo vértice dado.

★11. *a* 12. *c* 13. *d* 14. *g* 15. *e* 16. *h*

Nos Exercícios 17 a 20, escreva os vértices visitados por uma busca em largura no grafo dos Exercícios 7 a 10, começando pelo vértice dado.

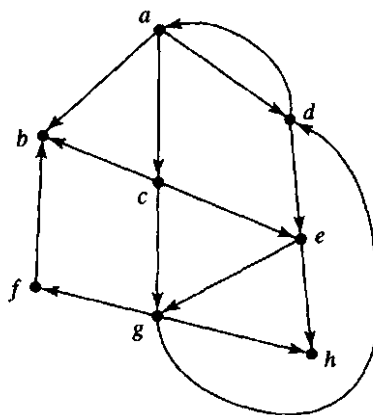
★17. *a* 18. *e* 19. *f* 20. *h*

Nos Exercícios 21 a 23, escreva os vértices visitados por uma busca em profundidade no grafo da figura a seguir, começando pelo vértice especificado.

★21. *a* 22. *g* 23. *f*

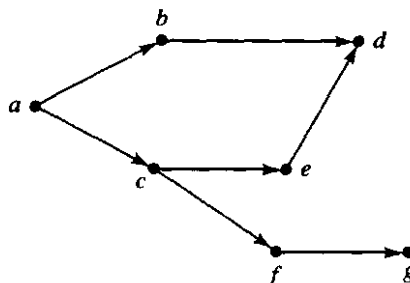
Nos Exercícios 24 a 26 escreva os vértices visitados por uma busca em largura no grafo da figura a seguir, começando pelo vértice especificado.

★24. *a* 25. *g* 26. *f*



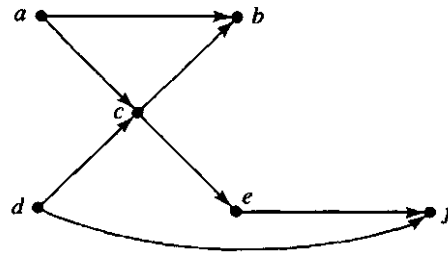
Exercícios 21-26

★27. Use o algoritmo de busca em profundidade para realizar uma ordenação topológica no grafo da figura a seguir. Indique o número atribuído a cada vértice. Além disso, indique o vértice ou os vértices iniciais.



Exercício 27

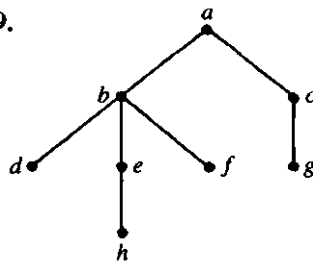
28. Use o algoritmo de busca em profundidade para realizar uma ordenação topológica no grafo da figura a seguir. **Indique** o número atribuído a cada vértice. Além disso, indique o vértice ou os vértices iniciais.



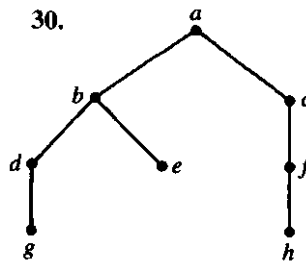
Exercício 28

Nos Exercícios 29 a 34, escreva a lista de vértices resultantes de buscas em preordem, inordem e posordem do grafo dado.

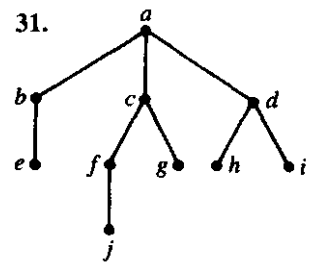
★29.



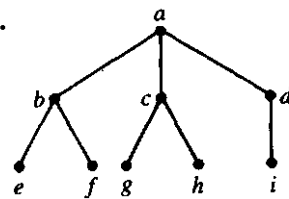
30.



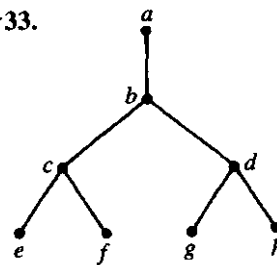
31.



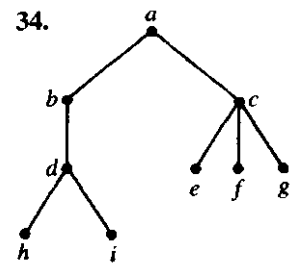
32.



★33.



34.



★35. Escreva a expressão $\frac{3}{4} + (2 - y)$ nas notações prefixa e posfixa.

36. Escreva nas notações prefixa e posfixa: $(x * y + 3/z) * 4$

37. Escreva nas notações infixa e posfixa: $— * + 2 3 * 6 x 7$

38. Escreva nas notações infixa e posfixa: $— + — x y z w$

★39. Escreva nas notações prefixa e infixa: $4 7 x — * z +$

40. Escreva nas notações prefixa e infixa: $x2w + yz*- /$

41. Desenhe uma árvore cuja busca em preordem seja

a, b, c, d, e

e cuja busca e inordem seja

b, a, d, c, e

42. Desenhe uma árvore cuja busca em inordem seja

$f, a, g, b, h, d, i, c, j, e$

e cuja busca em posordem seja

$f, g, a, h, i, d, j, e, c, b$

- ★43. Uma busca em inordem de árvore binária de busca (Seção 5.4) produz uma listagem dos vértices da árvore em ordem crescente. Construa uma árvore binária para a frase "To be or not to be, that is the question", e então realize uma busca em inordem.
- 44. Construa uma árvore binária de busca (veja o Exercício 43) para a frase "As rosas não falam, simplesmente as rosas exalam o perfume que roubam de ti" e então realize uma busca em inordem.
- ★45. Encontre um exemplo de árvore cujas buscas em inordem e em posordem gerem a mesma lista de vértices.
- 46. Encontre duas árvores diferentes que tenham a mesma lista de vértices sob uma mesma busca em preordem.
- 47. Encontre um modo de percorrer uma árvore em *ordem de níveis*, isto é, de forma que os vértices de um mesmo nível sejam listados da esquerda para a direita para profundidades crescentes. (Dica: Já temos um modo de fazer isto.)

Seção 6.4 Articulações e Redes de Computadores

O Enunciado do Problema

Em um grafo que represente uma rede de computadores ou de comunicação, os vértices denotam as entidades que se comunicam (computadores, telefones, etc.) e as arestas denotam o meio de comunicação (cabo coaxial, linhas telefônicas, etc). Grafos deste tipo devem ser conexos a fim de que haja um caminho entre cada par de vértices. Para minimizar o comprimento de cabo ou fio necessário, devemos tomar uma árvore geradora mínima. No entanto, se removemos uma aresta de uma árvore geradora mínima (se uma seção do cabo ou fio apresenta defeito ou é interrompida), o grafo não se mantém conexo. Este é o motivo por que uma rede normalmente contém mais fios do que os da árvore geradora mínima. No entanto, mesmo em um grafo suficientemente rico em arestas, para evitar a quebra da rede no caso de perda de uma aresta, um vértice pode ser o suficiente para causar a falha da rede. Se um vértice deste tipo falhar (e, portanto, for removido logicamente), as arestas para as quais este vértice constitui um extremo são removidas, e isto pode provocar a desconexão do grafo.

Definição: Articulação

Um vértice em um grafo simples e conexo é uma **articulação** se sua remoção (juntamente com suas arestas associadas) torna o grafo resultante desconexo.

EXEMPLO 11 O vértice d do grafo da Fig. 6.20a é uma articulação. A remoção de d resulta no grafo desconexo mostrado na Fig. 6.20b.

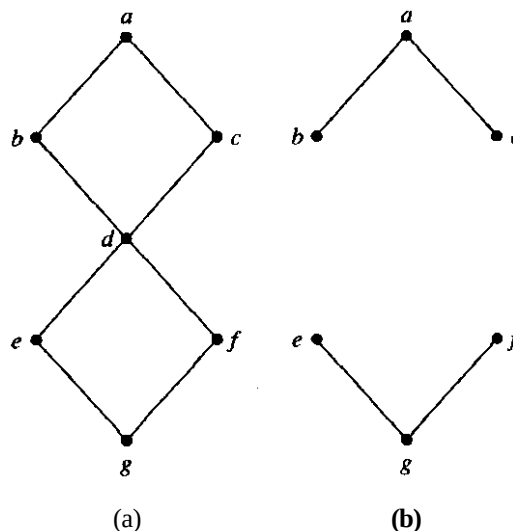


Figura 6.20

Definição: Grafo Biconexo

Um grafo simples e conexo é dito biconexo se não possuir pontos de articulação.

A presença de articulações é claramente indesejada em uma rede. Apesar de ser fácil apontar uma articulação em um grafo pequeno como o da Fig. 6.20a, vamos desenvolver um algoritmo que detecta este tipo de vértice independente do tamanho dos grafos. As articulações dividem o grafo em **componentes biconexas**, subgrafos biconexos que não sejam subgrafos de outros subgrafos biconexos maiores. Na Fig. 6.20, *a-b-d-c* e *d-e-g-f* são componentes biconexas.

A Idéia do Algoritmo

A base deste algoritmo é uma busca em profundidade. Sabemos da seção anterior que uma busca em profundidade determina uma árvore não-enraizada. Uma aresta é incluída na árvore sempre que a busca alcança um vértice que ainda não foi visitado. As arestas do grafo que pertencem a esta árvore são chamadas **arestas de árvore**. As demais arestas do grafo são chamadas de **arestas de retorno** ou frondes.

EXEMPLO 12 Na Fig. 6.20, uma busca em profundidade a partir do vértice *a* visita os vértices *a, b, d, c, e, g e f*. Na Fig. 6.21, as arestas de árvore são as escuras e as de retorno são as mais claras.

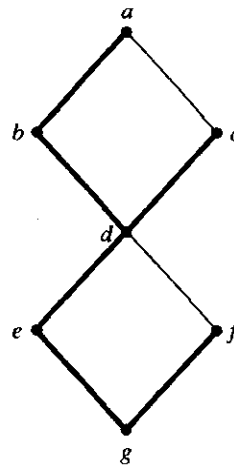


Figura 6.21

A árvore da busca em profundidade passa por todos os vértices. Para determinarmos quais vértices são articulações, devemos examinar suas posições relativas na árvore. Primeiro, vejamos o vértice que é o ponto de partida da busca em profundidade. Se apenas uma aresta de árvore sai deste vértice, então, como a árvore continua, todos os outros vértices do grafo podem ser alcançados a partir do vértice no outro extremo desta aresta de árvore. Portanto, a remoção do vértice inicial não desconectaria o grafo. No entanto, se dois ou mais vértices saem do vértice inicial, então a única maneira de se ir de uma subárvore para outra é passando pelo vértice inicial. Assim, a remoção deste vértice desconectaria o grafo.

Neste caso, na Fig. 6.21, o vértice *a* é o ponto inicial da busca em profundidade, e existe uma única aresta saindo de *a*. A remoção do vértice *a* (e suas duas arestas) não desconecta o grafo. Se, por outro lado, tivéssemos começado a busca em profundidade pelo vértice *d*, a árvore ficaria como mostrada na Fig. 6.22. Teriam duas arestas de árvore saindo do vértice *d*, mostrando que *d* é uma articulação.

Consideremos qualquer vértice *n* que seja uma folha na árvore da busca em profundidade (ligado ao fim de uma aresta de árvore). Esta folha pode ser uma folha do grafo original, isto é, um vértice de grau 1, caso em que fica claro que não é uma articulação. Do contrário, as outras arestas que saem de *n* não foram usadas pela busca em profundidade, de forma que os vértices adjacentes a *n* são alcançáveis através de outros caminhos que não passem por *n*. Como *n* não é necessário para compor o caminho a qualquer outro vértice, sua remoção não desconectaria o grafo. Portanto, as folhas em uma árvore de busca em profundidade não são articulações. Na Fig. 6.21, o vértice *c*, por exemplo, é uma folha na árvore da busca em profundidade; a aresta de *c* a *a* é uma aresta de retorno, de modo que *a* é alcançável através de outro caminho que não passe pelo vértice *c*. O vértice *c* pode ser removido sem desconectar o grafo.

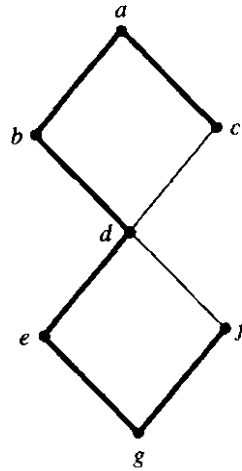


Figura 6.22

Agora consideremos um vértice n que não seja folha nem o vértice inicial da árvore da busca em profundidade. Como n não é uma folha, existem uma ou mais subárvores abaixo de n . Suponha que existe uma única subárvore; seja x um vértice desta subárvore. Se x tem uma aresta de retorno para algum vértice que preceda n na busca em profundidade (um "ancestral" de n), então esta aresta faz parte de um outro caminho que conecta x — e todos os demais vértices na subárvore — ao resto do grafo, sem usar o vértice n . Neste caso, n não é uma articulação. (Veja a Fig. 6.23a, onde a remoção de n e suas arestas não desconecta o grafo.) Se houver mais de uma subárvore abaixo de n , então n não será uma articulação se, e somente se, cada subárvore tiver um "caminho de fuga" que o conecte ao resto do grafo — incluindo as outras subárvores — sem passar por n . (Veja a Fig. 6.23b; perceba que a aresta de retorno de y para z não ajuda, porque não alcança um ancestral de n .)

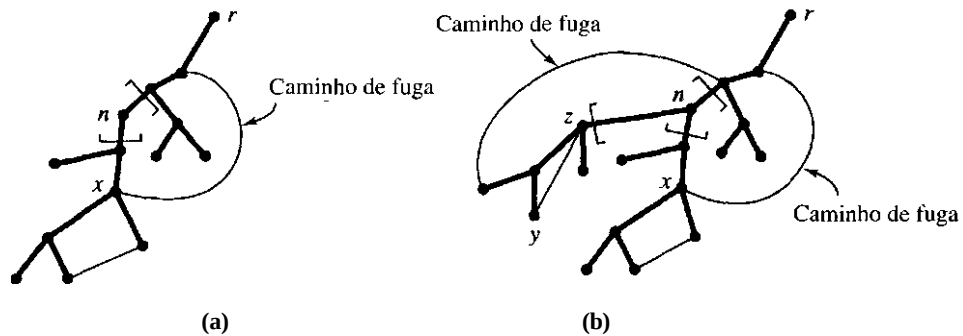


Figura 6.23

O Algoritmo Propriamente Dito

A base do algoritmo, como podemos deduzir da discussão acima, é manter um controle dos destinos das arestas de retorno. Atribuiremos, a cada vértice, um "número de árvore" que corresponda à ordem na qual o vértice foi visitado pela busca em profundidade. Portanto, o vértice inicial em uma busca em profundidade tem o número 1, o próximo vértice visitado recebe o número 2, e assim por diante. Além disso, devemos manter um "número de retorno" para cada vértice x . O número de retorno será o menor número de árvore de um vértice (o vértice de retorno mais longe) alcançável a partir de x usando arestas de retorno de x ou dos descendentes de x na subárvore. Para incorporar a informação sobre as arestas dos descendentes de x , o número de retorno de x é ajustado quando a busca em profundidade volta ao vértice x da parte inferior da árvore. O vértice n é uma articulação se uma subárvore de n não tiver arestas de retorno para ancestrais de n , e isto é detectado quando a busca volta da subárvore para n . Suponha que a busca está voltando de x para n . Se o número de retorno de x neste ponto não for menor que o número de árvore de n , então n é uma articulação.

O algoritmo a seguir (algoritmo *Articulação*) realiza a busca em profundidade e constrói a árvore referente a esta busca. Ele manipula corretamente as folhas e os vértices internos que não o vértice inicial da árvore da busca em profundidade, deixando apenas o caso do vértice inicial como caso especial.

ALGORITMO *Articulação*

```

procedure Articulação(var G: grafo; n: vértice; var NumÁrvore: integer);
{detecta articulações através de uma busca em profundidade a partir de n; NumÁrvore = 0 na primeira chamada}

begin
  marque n como visitado;
  {primeira vez que alcança n — atribua-lhe números de árvore e de retorno}
  NumÁrvore := NumÁrvore + 1;
  NumeroDeÁrvore[n] := NumÁrvore;
  NúmeroDeRetorno[n] := NúmeroDeÁrvore[n];

  for todo vértice x adjacente a n por uma aresta de retorno do
    if x não foi visitado then
      begin
        torne n-x uma aresta de árvore;
        Articulação(G, x, NumÁrvore);

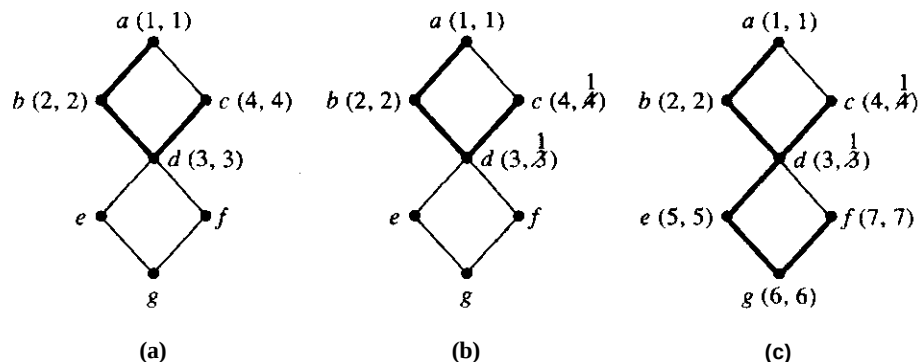
        {busca em profundidade está retornando da subárvore de raiz x para o vértice n.}
        {n é um ponto de articulação — a subárvore de raiz x não tem
        aresta de retorno para algum ancestral de n?}
      {1} if NúmeroDeRetorno[x] >= NúmeroDeÁrvore[n] then
        escreva(n, '... é uma articulação')
      else
        {ajusta o número de retorno de n}
      {2} NúmeroDeRetorno[n] := min(NúmeroDeRetorno[n], NúmeroDeRetorno[x]);
      end;
    else
      {aresta n-x é uma aresta de retorno, ajuste NúmeroDeRetorno[n]}
    {3} NúmeroDeRetorno[n] := min(NúmeroDeRetorno[n], NúmeroDeÁrvore[x]);
  end;

```

EXEMPLO 13

Acompanharemos o algoritmo acima aplicado ao grafo da Fig. 6.20a, onde *a* é o vértice inicial. A árvore começa com as arestas *a-b*, *b-d* e *d-c*. Cada novo vértice é numerado com um *NúmeroDeÁrvore* seqüencial e seu *NúmeroDeRetorno* recebe seu *NúmeroDeÁrvore*. (Veja a Fig. 6.24a, onde os números em parênteses são o *NúmeroDeÁrvore* e o *NúmeroDeRetorno*, respectivamente.) Enquanto processa o vértice *c*, a aresta de retorno para *a* é descoberta e o *NúmeroDeRetorno* de *c* é ajustado para 1, o *NúmeroDeÁrvore* de *a* (linha {3} no algoritmo acima). Isto completa o processamento do vértice *c*, e a busca em profundidade volta para *d*. O *NúmeroDeRetorno* de *c* é menor do que o *NúmeroDeÁrvore* de *d*, de forma que o *NúmeroDeRetorno* de *d* é ajustado para valer o mesmo que o de *c* (linha {2}). A situação neste momento é mostrada na Fig. 6.24b.

A busca em profundidade passa aos vértices *e*, *g* e *f* (Fig. 6.24c). Em/a aresta de retorno para *d* é encontrada e o *NúmeroDeRetorno* de/recebe o *NúmeroDeÁrvore* de *d* (linha {3}). Voltando de/para *g*, o *NúmeroDeRetorno* de *g* é ajustado a fim de se igualar ao *NúmeroDeRetorno* de *f* (linha {2}), e da mesma forma para *e* (linha {2}, novamente). (Veja Fig. 6.24d.) Finalmente, voltando de *e* para *d*, o *NúmeroDeRetorno* de *e* é maior ou igual ao *NúmeroDeÁrvore* de *d*, de forma que *d* é declarado uma articulação (linha {1}). A recursão volta ao vértice *b*, ajustando seu *NúmeroDeRetorno* de *b*, e então ao vértice *a*, onde a linha {1} se aplicaria (Fig. 6.24e). Mas *a* é o ponto inicial da busca e não é um ponto de articulação, pois existe apenas uma aresta de árvore a partir de *a*. •



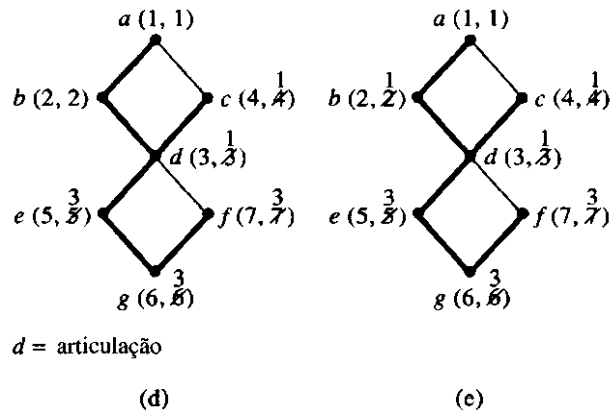


Figura 6.24

PRÁTICA 14 Na Fig. 6.25, a busca em profundidade começa no vértice a . Explique a marcação em cada vértice e porque concluímos que c é uma articulação.

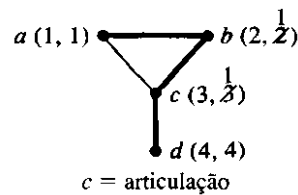


Figura 6.25

Revisão da Seção 6.4

Técnica

- Encontrar articulações em um grafo simples e conexo (usando o *algoritmo Articulação*)

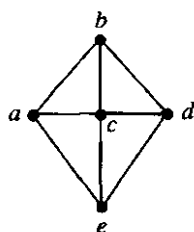
Idéia Principal

As articulações representam pontos de falha em uma rede de computadores ou de comunicações, mas existe um algoritmo para determinar suas existências.

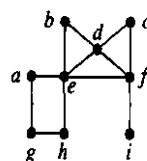
Exercícios 6.4

Nos Exercícios 1 a 6, desenhe as árvores de busca em profundidade, onde a é o vértice inicial da busca. Identifique as arestas de retorno.

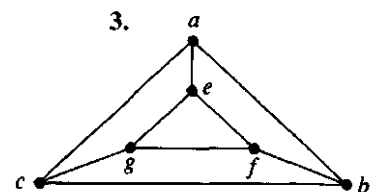
★1.

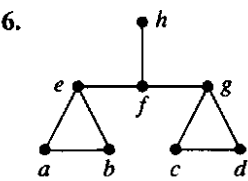
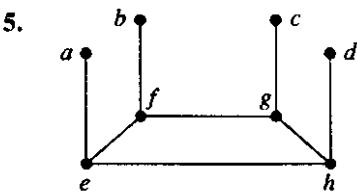
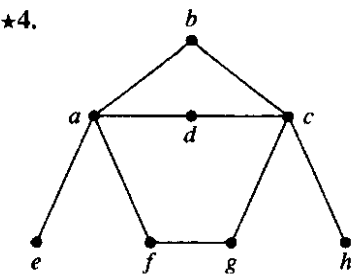


2.

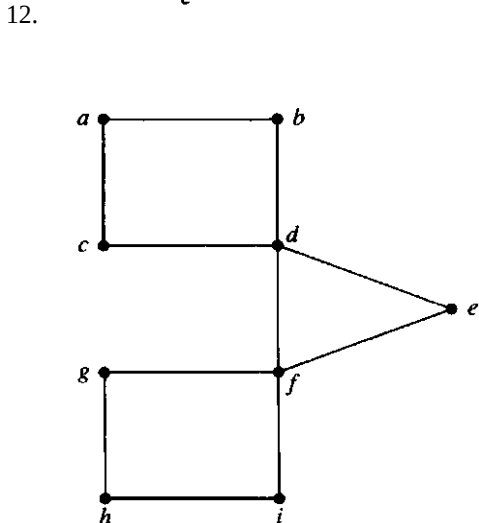
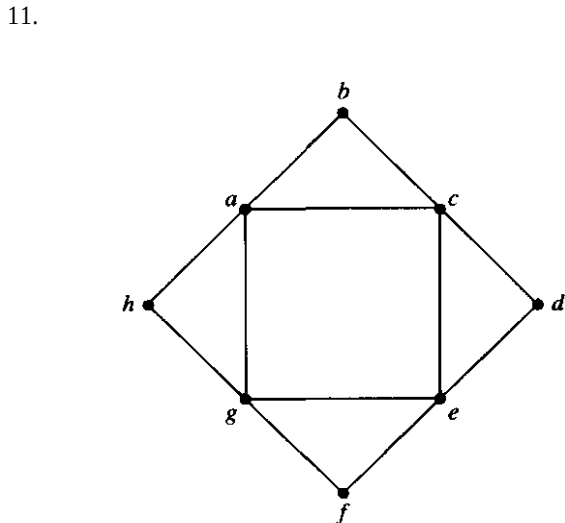
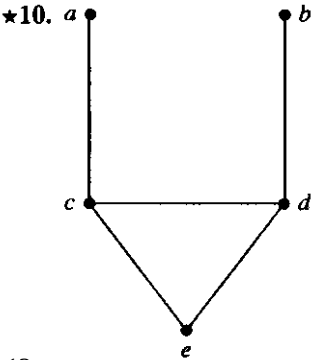
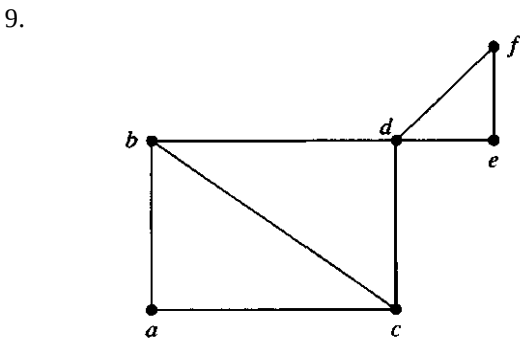
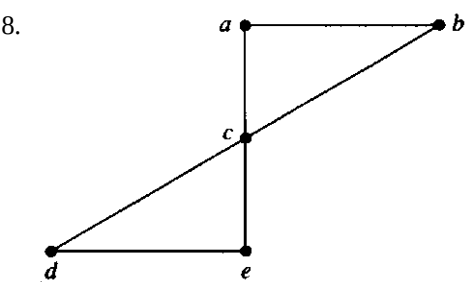
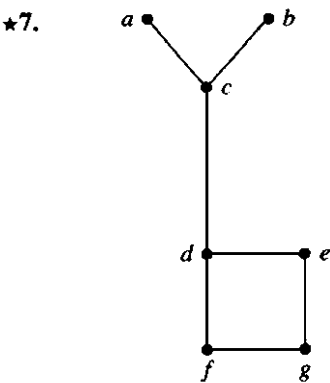


3.





Nos Exercícios 7 a 12, use o algoritmo *Articulação* para encontrar as articulações. Indique o *NúmeroDeÁrvore* e o *NúmeroDeRetorno* para cada vértice, tanto os atribuídos originalmente quanto os que forem sendo alterados. Desenhe as componentes biconexas dos grafos.



Revisão do Capítulo

Terminologia

algoritmo guloso	busca em preordem	grafo biconexo
aresta de árvore	busca em profundidade	notação infixa
aresta de retorno	busca em um grafo	notação polonesa
articulação	caminho euleriano	notação polonesa reversa
árvore geradora	ciclo hamiltoniano	notação posfixa
árvore geradora mínima	componente biconexa	notação prefixa
busca em inordem	componente conexa	vértice ímpar
busca em largura	fila	vértice par
busca em posordem		

Auto-testes

Responda às perguntas a seguir com verdadeiro ou falso.

Seção 6.1

1. Um grafo com quatro vértices ímpares pode ser conexo.
2. Existe um caminho euleriano em qualquer grafo com um número par de vértices ímpares.
3. Existe um algoritmo $\Theta(n^2)$ para determinar a existência de um caminho euleriano em um grafo com n vértices.
4. Um ciclo hamiltoniano usa cada aresta e vértice do grafo exatamente uma vez, exceto pelo vértice inicial e final.
5. Não é conhecido um algoritmo para resolver o problema do ciclo hamiltoniano.

Seção 6.2

6. O algoritmo de Dijkstra para obter o caminho mínimo em um grafo usa um conjunto IN onde inclui, a cada passo, o vértice mais próximo a um vértice de IN .
7. Um algoritmo guloso é um algoritmo que divide o problema recursivamente em tantos subproblemas quanto possível.
8. A árvore geradora mínima de um grafo não é única.
9. O uso de uma representação na forma de lista encadeada para os vértices que não pertencem a IN não melhora a ordem de grandeza do trabalho do algoritmo de Dijkstra no pior caso.
10. A coleção de todas as arestas que não pertencem à árvore geradora mínima de um grafo também forma uma árvore geradora, mas que não é mínima.

Seção 6.3

11. A busca em profundidade visita primeiro os vértices na parte inferior do grafo.
12. Em uma busca em largura começando pelo vértice i , todos os vértices adjacentes a i são visitados em sequência.
13. Uma análise das buscas em profundidade e em largura mostra que elas são algoritmos da mesma ordem de grandeza.
14. A busca em preordem é o equivalente em árvores à busca em largura, usando a raiz como vértice inicial.
15. A busca em posordem de uma árvore de expressão resulta em uma expressão escrita na notação polonesa reversa.

Seção 6.4

16. Se o vértice n é uma articulação em um grafo conexo, então qualquer caminho entre dois vértices no grafo precisa passar por n .
17. Um grafo biconexo é um grafo simples e conexo sem articulações.
18. Quando um vértice n é alcançado pela primeira vez em uma busca em profundidade, quaisquer outras arestas de n aos vértices previamente visitados são arestas de retorno.
19. Um vértice n onde toda subárvore de n na busca em profundidade tem uma aresta de retorno para um predecessor de n não é uma articulação.
20. A raiz de uma busca em profundidade é sempre uma articulação no grafo porque o *NúmeroDeRetorno* de qualquer vértice será maior ou igual ao seu *NúmeroDeÁrvore*.

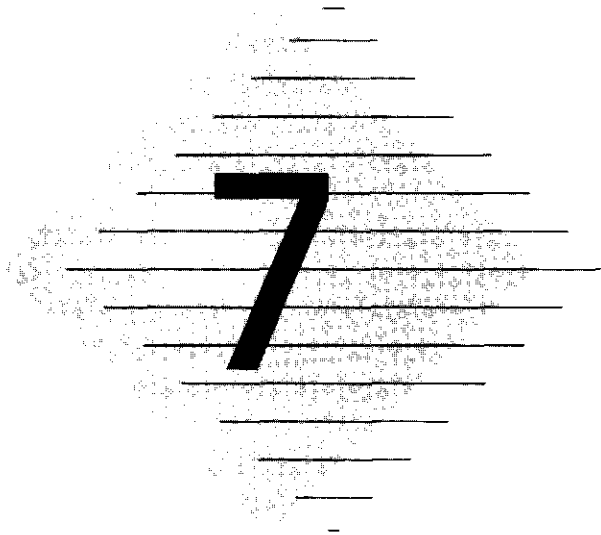
No Computador

Nos Exercícios 1 a 3, escreva um programa de computador que produza a saída desejada para a entrada dada.

1. *Entrada:* Matriz de adjacências de um grafo
Saída: Mensagem indicando se o grafo possui ou não um caminho euleriano
2. *Entrada:* Matriz de adjacências de um grafo simples e ponderado ou grafo direcionado e dois vértices no grafo
Saída: Distância do caminho mínimo entre dois vértices ou uma mensagem de que não existe tal caminho; os vértices no caminho mínimo, se ele existir
{Dica: Você precisará de um modo para denotar quais vértices pertencem a *IN*.)
3. *Entrada:* Matriz de adjacências de um grafo simples, ponderado e conexo
Saída: Arestas (na forma de pares ordenados) da árvore geradora mínima

Nos Exercícios 4 a 6, primeiro escreva uma rotina em que entre as informações referentes a um grafo do usuário e então monte sua representação na forma de uma lista de adjacências; incorpore este procedimento nos programas pedidos.

4. *Entrada:* Informação sobre um grafo (veja instruções acima) e um vértice no grafo
Saída: Vértices em uma busca em profundidade no grafo, começando no vértice dado
5. *Entrada:* Informação sobre um grafo (veja instruções acima) e um vértice do grafo
Saída: Vértices em uma busca em largura no grafo, começando no vértice dado
6. *Entrada:* Informação sobre um grafo (veja instruções acima)
Saída: Articulações do grafo
7. Escreva um programa que permita ao usuário entrar uma lista de inteiros, e então construa uma árvore binária de busca (veja Seção 5.4) com esses inteiros como vértices. O usuário pode então entrar o tipo de busca desejada (inordem, preordem ou posordem) e o programa escreve os vértices da árvore na ordem da busca selecionada.



Álgebra Booleana e Lógica de Computadores

Objetivos do Capítulo

Após estudar este capítulo, você estará apto a:

- Determinar se uma dada estrutura é ou não uma álgebra booleana
- Demonstrar propriedades referentes às álgebras booleanas
- Entender o que significa um isomorfismo preservar os efeitos de uma operação binária ou outras propriedades
- Desenhar uma rede lógica que represente uma expressão booleana
- Escrever uma expressão booleana que represente uma rede lógica
- Escrever uma função-verdade referente a uma expressão booleana ou rede lógica
- Escrever uma expressão booleana na forma canônica de soma de produtos para uma dada função-verdade
- Usar portas NE e NOU no lugar das portas E, OU e NÃO para construir redes lógicas
- Escrever uma função-verdade dada uma descrição de um dispositivo de controle lógico
- Simplificar expressões booleanas e redes lógicas através do mapa de Karnaugh
- Simplificar expressões booleanas e redes lógicas através do método de Quine-McCluskey

Algumas vezes os mesmos procedimentos e propriedades matemáticos podem ser aplicados a diferentes contextos. Uma estrutura matemática é um modelo formal que serve para caracterizar e explicar estas semelhanças, como na Física, em que a lei da gravidade é um modelo formal de por que as maçãs caem, os oceanos têm ondas e os planetas giram em torno do sol.

Na Seção 7.1 definimos a álgebra booleana como um modelo matemático tanto da lógica proposicional quanto da teoria dos conjuntos. A definição requer que todas álgebras booleanas tenham certas propriedades, das quais diversas outras podem ser derivadas. Esta seção discute ainda o que significa para duas ocorrências de álgebra booleana serem isomórficas.

A Seção 7.2 estabelece uma relação entre a estrutura da álgebra booleana e os esquemas dos circuitos eletrônicos dos computadores, calculadoras, dispositivos de controle industrial, sistemas de telefonia e outros. Na verdade, veremos que funções-verdade, expressões formadas de variáveis e operações da álgebra booleana, e esses esquemas de circuitos estão intimamente relacionados. Como um resultado, podemos efetivamente passar de uma formulação para outra preservando as características com respeito aos valores-verdade. Veremos ainda que podemos simplificar os esquemas de circuitos, usando as propriedades das álgebras booleanas. Na Seção 7.3, veremos dois outros procedimentos para simplificar esquemas de circuitos.

Seção 7.1 A Estrutura da Álgebra Booleana

Vamos revisitar as wffs da lógica proposicional e associar a elas um certo tipo de função. Suponha que uma wff proposicional P tenha n símbolos proposicionais. Então cada linha de sua tabela-verdade associa um valor V ou F a uma n -upla de valores V - F . Toda a tabela-verdade define uma função f tal que $f: \{V, F\}^n \rightarrow \{V, F\}$ (veja o Exemplo 30 do Cap. 4). A função associada a uma tautologia é uma aplicação $\{V, F\}^n \rightarrow \{V\}$, enquanto a função associada a uma contradição é uma aplicação $\{V, F\}^n \rightarrow \{F\}$. Mesmo um único símbolo proposicional A pode ser considerado a definição de uma função com n argumentos, se escrevermos a tabela-verdade com n símbolos proposicionais e depois ignorarmos os valores-verdade de todos os demais símbolos proposicionais, exceto os valores de A .

EXEMPLO 1 O símbolo proposicional A define uma função $f: \{V, F\}^2 \rightarrow \{V, F\}$ através da seguinte tabela-verdade:

A	B	A
V	V	V
V	F	V
F	V	F
F	F	F

Neste caso usamos dois símbolos proposicionais na tabela-verdade, mas o valor-verdade resultante é apenas o valor de A . Portanto, $f(V, V) = V$ e $f(V, F) = V$. •

Façamos, para qualquer wff proposicional P com n símbolos proposicionais, o símbolo P denotar não apenas a wff, mas também a função correspondente definida pela tabela-verdade. Se P e Q são wffs equivalentes, então elas têm a mesma tabela-verdade e, portanto, definem a mesma função. Então podemos escrever $P = Q$ ao invés de $P \Leftrightarrow Q$. Isto apenas confirma que uma dada função tem vários nomes, ainda que uma determinada wff defina uma única função.

Isto posto, a pequena lista de equivalências da Seção 1.1 pode ser escrita como mostrado a seguir, onde $\vee$ e $\wedge$ denotam a disjunção e a conjunção, respectivamente, A' denota a negação da sentença A , 0 indica qualquer contradição e 1 indica qualquer tautologia:

1a. $A \vee B = B \vee A$	1b. $A \wedge B = B \wedge A$	(propriedades comutativas)
2a. $(A \vee B) \vee C =$ $A \vee (B \vee C)$	2b. $(A \wedge B) \wedge C =$ $A \wedge (B \wedge C)$	(propriedades associativas)
3a. $A \vee (B \wedge C) =$ $(A \vee B) \wedge (A \vee C)$	3b. $A \wedge (B \vee C) =$ $(A \wedge B) \vee (A \wedge C)$	(propriedades distributivas)
4a. $A \vee 0 = A$	4b. $A \wedge 1 = A$	(propriedades de identidade)
5a. $A \vee A' = 1$	5b. $A \wedge A' = 0$	(propriedades complementativas)

Trocando um pouco as bolas, na Seção 3.1 vimos identidades de conjuntos entre os subconjuntos de um conjunto S . Encontramos a seguinte lista de identidades, onde $\cup$ e $\cap$ denotam a união e a interseção de conjuntos, respectivamente, A' é o complemento de um conjunto A e $\emptyset$ é o conjunto vazio.

1a. $A \cup B = B \cup A$	1b. $A \cap B = B \cap A$	(propriedades comutativas)
2a. $(A \cup B) \cup C =$ $A \cup (B \cup C)$	2b. $(A \cap B) \cap C =$ $A \cap (B \cap C)$	(propriedades associativas)
3a. $A \cup (B \cap C) =$ $(A \cup B) \cap (A \cup C)$	3b. $A \cap (B \cup C) =$ $(A \cap B) \cup (A \cap C)$	(propriedades distributivas)
4a. $A \cup \emptyset = A$	4b. $A \cap S = A$	(propriedades de identidade)
5a. $A \cup A' = S$	5b. $A \cap A' = \emptyset$	(propriedades complementativas)

Estas duas listas de propriedades são semelhantes. A disjunção de sentenças e a união de conjuntos parecem seguir as mesmas regras em seus respectivos contextos. Da mesma forma, a conjunção de sentenças e a interseção de conjuntos. Uma contradição parece corresponder a um conjunto vazio, e uma tautologia a S . O que devemos fazer com esta semelhança?

Modelos ou Abstrações

Encontramos dois exemplos diferentes — a lógica proposicional e a teoria dos conjuntos — que gozam de algumas propriedades em comum. Uma das características do pensamento científico é procurar padrões ou semelhanças entre fenômenos observados. Será que essas manifestações semelhantes são manifestações de algum princípio básico mais geral? Será que este princípio propriamente dito pode ser identificado e estudado? Poderíamos usar procedimentos semelhantes em várias ocorrências deste mesmo princípio?

Os princípios matemáticos são modelos ou abstrações que visam capturar propriedades que podem ser comuns em diferentes ocorrências ou formas. Estes princípios são normalmente expressos como *estruturas matemáticas* — conjuntos abstratos de objetos, juntamente com operações ou relações entre esses objetos que obedecem certas regras. (Isto pode dar uma dica do porquê do título deste livro.)

Podemos comparar uma estrutura matemática a um esqueleto humano. Podemos pensar no esqueleto como a estrutura básica do corpo humano. As pessoas podem ser magras ou gordas, altas ou baixas, pretas ou brancas etc, mas seus esqueletos sempre se parecem. A despeito das aparências externas serem diferentes, a estrutura interna, a forma e a disposição dos ossos são as mesmas. Analogamente, as estruturas matemáticas representam as semelhanças básicas entre situações que pareçam diferentes.

Parece razoável abstrair as propriedades comuns entre as wffs proposicionais e a teoria de conjuntos encontradas acima. Portanto definiremos em breve uma estrutura matemática, chamada *álgebra booleana* que incorpora essas propriedades. Antes, no entanto, observamos que a modelagem ou a abstração não constituem uma idéia totalmente nova para nós:

1. Usamos a lógica de predicados para modelar e definir formalmente uma interpretação como um caso específico da lógica de predicados.
2. Definimos as idéias abstratas de ordenação parcial e relação de equivalência, e consideramos alguns casos específicos que puderam ser modelados como conjuntos parcialmente ordenados nos quais é definida uma relação de equivalência.
3. Vimos que a estrutura de grafo pode modelar um grande número de casos.

A álgebra booleana é apenas um outro modelo ou abstração para o qual já temos dois exemplos. O conceito de álgebra booleana foi inicialmente formulado pelo matemático inglês George Boole, por volta de 1850, a fim de modelar a lógica proposicional.

A Estrutura da Álgebra Booleana

Suponha agora que tentamos caracterizar formalmente as semelhanças entre a lógica proposicional e a teoria de conjuntos. Em ambos os casos, estamos nos referindo a elementos de um conjunto: um conjunto de wffs ou um conjunto de subconjuntos de um conjunto S . Em ambos os casos, temos duas operações binárias e uma operação unária nos elementos do conjunto: disjunção/conjunção/negação ou união/interseção/complemento. Em ambos os casos, existem dois elementos notáveis: 0/1 ou $\emptyset/S$. E, finalmente, existem 10 propriedades que se aplicam em cada caso. Sempre que se apresentam todos esses aspectos, dizemos que temos uma álgebra booleana.

Definição

Uma **álgebra booleana** é um conjunto B no qual são definidas duas operações binárias $+$ e $\cdot$ e uma operação unária $'$ e no qual há dois elementos distintos 0 e 1 tais que valem as seguintes propriedades para todos $x, y, z \in B$:

- | | | |
|---|---|---------------------------------|
| 1a. $x + y = y + x$ | 1b. $x \cdot y = y \cdot x$ | (propriedades comutativas) |
| 2a. $(x + y) + z = x + (y + z)$ | 2b. $(x \cdot y) \cdot z = x \cdot (y \cdot z)$ | (propriedades associativas) |
| 3a. $x + (y \cdot z) = (x + y) \cdot (x + z)$ | 3b. $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$ | (propriedades distributivas) |
| 4a. $x + 0 = x$ | 4b. $x \cdot 1 = x$ | (propriedades de identidade) |
| 5a. $x + x' = 1$ | 5b. $x \cdot x' = 0$ | (propriedades complementativas) |

O que, então, é a estrutura da álgebra booleana? É uma formalização que abstrai, ou modela, os dois casos que estávamos considerando (e talvez outros casos mais). Existe uma sutil diferença filosófica entre a formalização propriamente dita, a *idéia* da estrutura da álgebra booleana e qualquer ocorrência desta formalização, como nos nossos dois exemplos. Não obstante, podemos normalmente usar o termo *álgebra booleana* para descrever tanto as idéias quanto suas ocorrências. Este uso não costuma causar confusão. Normalmente, mentalizamos uma idéia ("cadeira", por exemplo) e quando encontramos um exemplo concreto da idéia, nós o chamamos pelo nome da idéia (este objeto é uma "cadeira").

A formalização nos ajuda na concentração de aspectos importantes comuns a todos os exemplos de álgebras booleanas, e podemos usar tais aspectos — estes fatos da definição de uma álgebra booleana — para provar fatos referentes a álgebras booleanas. Esses novos fatos, uma vez demonstrados genericamente, valem para qualquer caso em particular de uma álgebra booleana. Para usar nossa analogia, se determinarmos que, em um esqueleto humano típico, o fêmur está ligado ao osso do joelho, então não precisaremos verificar este fato para cada pessoa em particular.

Denotamos uma álgebra booleana por $[B, +, \cdot, ', 0, 1]$.

EXEMPLO 2 Seja $B = \{0, 1\}$ (o conjunto dos inteiros 0 e 1) e defina as operações binárias $+$ e $\cdot$ em B por $x + y = \max(x, y)$, $x \cdot y = \min(x, y)$. Então, podemos ilustrar as operações de $+$ e $\cdot$ pelas tabelas abaixo.

$+$	0	1
0	0	1
1	1	1

$\cdot$	0	1
0	0	0
1	0	1

Uma operação unária, pode ser definida através de uma tabela, como mostrado a seguir, ao invés de uma expressão verbal.

$'$	
0	1
1	0

Portanto, $0' = 1$ e $1' = 0$. Então $[B, +, \cdot, ', 0, 1]$ é uma álgebra booleana. Podemos verificar as 10 propriedades testando todos os casos possíveis. Por isso, para a propriedade 2b, a associatividade de $\cdot$, mostramos que

$$\begin{aligned}(0 \cdot 0) \cdot 0 &= 0 \cdot (0 \cdot 0) = 0 \\ (0 \cdot 0) \cdot 1 &= 0 \cdot (0 \cdot 1) = 0\end{aligned}$$

$$\begin{aligned}
(0 \cdot 1) \cdot 0 &= 0 \cdot (1 \cdot 0) = 0 \\
(0 \cdot 1) \cdot 1 &= 0 \cdot (1 \cdot 1) = 0 \\
(1 \cdot 0) \cdot 0 &= 1 \cdot (0 \cdot 0) = 0 \\
(1 \cdot 0) \cdot 1 &= 1 \cdot (0 \cdot 1) = 0 \\
(1 \cdot 1) \cdot 0 &= 1 \cdot (1 \cdot 0) = 0 \\
(1 \cdot 1) \cdot 1 &= 1 \cdot (1 \cdot 1) = 1
\end{aligned}$$

Para a propriedade 4a, mostramos que

$$\begin{aligned}
0 + 0 &= 0 \\
1 + 0 &= 1
\end{aligned}$$

PRÁTICA 1 Verifique a propriedade 4b para a álgebra booleana do Exemplo 2.

Propriedades

Existem diversas outras propriedades que valem para qualquer álgebra booleana. Podemos demonstrar essas propriedades adicionais, usando as propriedades da definição.

EXEMPLO 3 A **propriedade idempotente**, $x + x = x$, vale em qualquer álgebra booleana porque

$$\begin{aligned}
x + x &= (x + x) \cdot 1 & (4b) \\
&= (x + x) \cdot (x + x') & (5a) \\
&= x + (x \cdot x') & (3a) \\
&= x + 0 & (5b) \\
&= x & (4a)
\end{aligned}$$

Apesar da aritmética de inteiros gozar de diversas propriedades de uma álgebra booleana, a propriedade idempotente deve convencer o leitor de que a aritmética não é uma álgebra booleana.

Cada propriedade da definição de uma álgebra booleana tem seu **dual** como parte da definição, onde o dual é obtido trocando $+$ por $\cdot$, e 1 por 0 . Portanto, sempre que uma nova propriedade P referente às álgebras booleanas é demonstrada, cada passo em sua demonstração pode ser substituído por seu dual. O resultado é uma demonstração do dual de P . Portanto, uma vez que tenhamos demonstrado P , sabemos que seu dual também é verdadeiro.

EXEMPLO 4 O dual da propriedade do Exemplo 3, $x \cdot x = x$ é verdadeiro para qualquer álgebra booleana.

PRÁTICA 2 a. Qual o significado da propriedade idempotente do Exemplo 3 no contexto da lógica proposicional?
b. E no contexto da teoria dos conjuntos?

Uma vez que tenhamos demonstrado uma propriedade referente à álgebra booleana, podemos usá-la para demonstrar novas propriedades.

PRÁTICA 3 a. Demonstre que a propriedade $x + 1 = 1$ vale para qualquer álgebra booleana. Forneça uma justificativa para cada passo de sua demonstração.
b. Qual a propriedade dual?

Mais propriedades de álgebras booleanas são mostradas no Exercício 4, ao final desta seção.

Para um elemento x de uma álgebra booleana, o elemento x' é chamado de **complemento** de x . O complemento de x satisfaz

$$x + x' = 1 \text{ e } x \cdot x' = 0$$

Na verdade, x' é o único elemento que satisfaz essas duas propriedades. Para demonstrarmos isto, suponha que x_1 é um elemento de B com

$$x + x_1 = 1 \text{ e } x \cdot x_1 = 0$$

Então

$$\begin{aligned}
x_1 &= x_1 \cdot 1 & (4b) \\
&= x_1 \cdot (x + x') & (x + x' = 1) \\
&= (x_1 \cdot x) + (x_1 \cdot x') & (3b) \\
&= (x \cdot x_1) + (x' \cdot x_1) & (1b) \\
&= 0 + (x' \cdot x_1) & (x \cdot x_1 = 0) \\
&= (x \cdot x') + (x' \cdot x_1) & (5b) \\
&= (x' \cdot x) + (x' \cdot x_1) & (1b) \\
&= x' \cdot (x + x_1) & (3b) \\
&= x' \cdot 1 & (x + x_1 = 1) \\
&= x' & (4b)
\end{aligned}$$

Portanto, $x_1 = x'$ e x' é único. (A unicidade no contexto da lógica proposicional significa que a tabela-verdade é única, mas pode ter diversas wffs associadas a uma tabela-verdade em particular.)

O teorema a seguir resume nossas observações.

Teorema da Unicidade de Complementos

Para qualquer x na álgebra booleana, se existir um elemento x_1 , tal que

$$x + x_1 = 1 \text{ e } x \cdot x_1 = 0$$

então $x_1 = x'$.

PRÁTICA 4 Demonstre que $0' = 1$ e $1' = 0$. (Dica: $1' = 0$ é o dual de $0' = 1$. Para mostrar $0' = 1$, use o teorema da unicidade de complementos.)

Existem diversas maneiras de se definir uma álgebra booleana. Na verdade, em nossa definição de álgebra booleana, poderíamos ter omitido as propriedades associativas, uma vez que elas podem ser entendidas como conseqüências das demais propriedades da definição.

Álgebras Booleanas Isomorfas

O Que é Isomorfismo?

Duas ocorrências de uma estrutura são **isomorfas** se existir uma bijeção (chamada **isomorfismo**) que leva os elementos de uma ocorrência sobre os elementos da outra ocorrência, de forma que as propriedades relevantes são preservadas. (Já vimos o isomorfismo de grafos na Seção 5.1.) Se duas ocorrências de uma estrutura são isomorfas, cada uma é a imagem semelhante à da outra, com um novo rotulamento de seus elementos. Podemos, portanto, usar a idéia de isomorfismo para classificar exemplos de estruturas, associando as que são isomorfas.

EXEMPLO 5 Considere os seguintes conjuntos parcialmente ordenados

$$\begin{aligned}
S_1 &= \{1, 2, 3, 5, 6, 10, 15, 30\}; x \rho y \leftrightarrow x \text{ divide } y \\
S_2 &= \mathcal{P}\{1, 2, 3\}; A \sigma B \leftrightarrow A \subseteq B
\end{aligned}$$

O grafo de cada conjunto parcialmente ordenado pode ser visto na Fig. 7.1. Esses dois grafos são, notoriamente, imagens semelhantes um do outro; apenas em olhar os grafos, um novo rotulamento dos vértices, como mostrado na Fig. 7.2, sugere a associação.

As propriedades relevantes de um conjunto parcialmente ordenado são quais elementos se relacionam, e o grafo exibe esta informação. Por exemplo, a Fig. 7.1a mostra que 1, devido à sua posição na parte inferior do grafo, relaciona-se com todos os elementos em S_1 . Esta propriedade é preservada pelo novo rotulamento da Fig. 7.2? Sim, porque $\emptyset$ é a imagem de 1 pelo novo rotulamento e $\emptyset$ relaciona-se com todos os elementos de S_2 . Analogamente, todas as demais propriedades "relaciona-se com" são preservadas pelo novo rotulamento.

De maneira mais formal, o novo rotulamento é realizado pela bijeção / mostrada a seguir, que leva elementos do conjunto de vértices da Fig. 7.1a sobre os vértices da Fig. 7.1b.

$$\begin{array}{ccccccc}
f(1) & \emptyset & = & f(2) & = \{1\} & f(3) & = \{2\} & f(5) & = \{3\} \\
f(6) & = \{1, 2\} & f(10) & = \{1, 3\} & f(15) & = \{2, 3\} & f(30) & = \{1, 2, 3\}
\end{array}$$

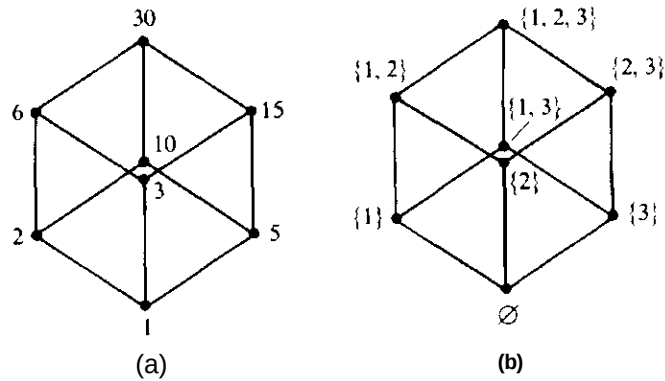


Figura 7.1

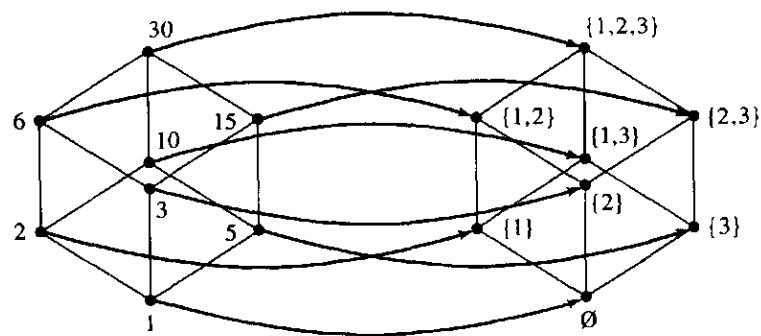


Figura 7.2

A bijeção f é um isomorfismo do conjunto parcialmente ordenado (S_1, ρ) no conjunto parcialmente ordenado (S_2, σ) . Como este isomorfismo existe, os conjuntos parcialmente ordenados (S_1, ρ) e (S_2, σ) são isomorfos. (A função f^{-1} é um isomorfismo de (S_2, σ) em (S_1, ρ)).

No Exemplo 5 foi relativamente fácil encontrar um isomorfismo devido à representação gráfica que capturava as propriedades relevantes (quais elementos se relacionam). Suponha que ao invés de um conjunto parcialmente ordenado, temos uma estrutura (como uma álgebra booleana) onde operações binárias e unárias são definidas em um conjunto. Então as propriedades importantes referem-se a como essas operações atuam. Duas ocorrências dessa estrutura que sejam isomorfas precisam ser semelhantes entre si no sentido de que "a operação seguida da aplicação" seja igual à "aplicação seguida da operação". Neste caso, a aplicação a que nos referimos é a função que visa estabelecer o isomorfismo.

A Fig. 7.3 ilustra essa idéia geral para uma operação binária. Na Fig. 7.3a, a operação binária é realizada em a e b , resultando em c , em seguida c é levada em d . Na Fig. 7.3b, a e b são levados em e e f , nos quais a operação binária é realizada, resultando no mesmo elemento d de antes. Lembre-se de que

$$\text{operação seguida da aplicação} = \text{aplicação seguida da operação}$$

Outra interpretação desta pequena equação pode ser vista no diagrama comutativo da Fig. 7.4.

Isomorfismo Aplicado à Álgebra Booleana

Agora vamos determinar especificamente o que ocorre quando duas ocorrências de álgebras booleanas são isomorfas. Suponha que temos duas álgebras booleanas, $[B, +, \cdot, ', 0, 1]$ e $[b, \&, *, \emptyset, 1]$. Esta notação significa que, por exemplo, se x está em B , x' é o resultado da aplicação, em x , da operação unária definida em B , e se z é um elemento de b , z' é o resultado da aplicação, em z , da operação unária definida em b . Como devemos definir o isomorfismo entre essas duas álgebras booleanas? Primeiro, precisamos de uma bijeção f de B sobre b . Depois disso, f precisa preservar, em b , os efeitos das operações em B . Existem três dessas operações, de forma que usaremos três equações para expressar essa preservação. Para preservar a operação $+$, desejamos ser capazes de operar sobre dois elementos de B e então realizar a aplicação sobre o resultado, a fim de levá-lo a

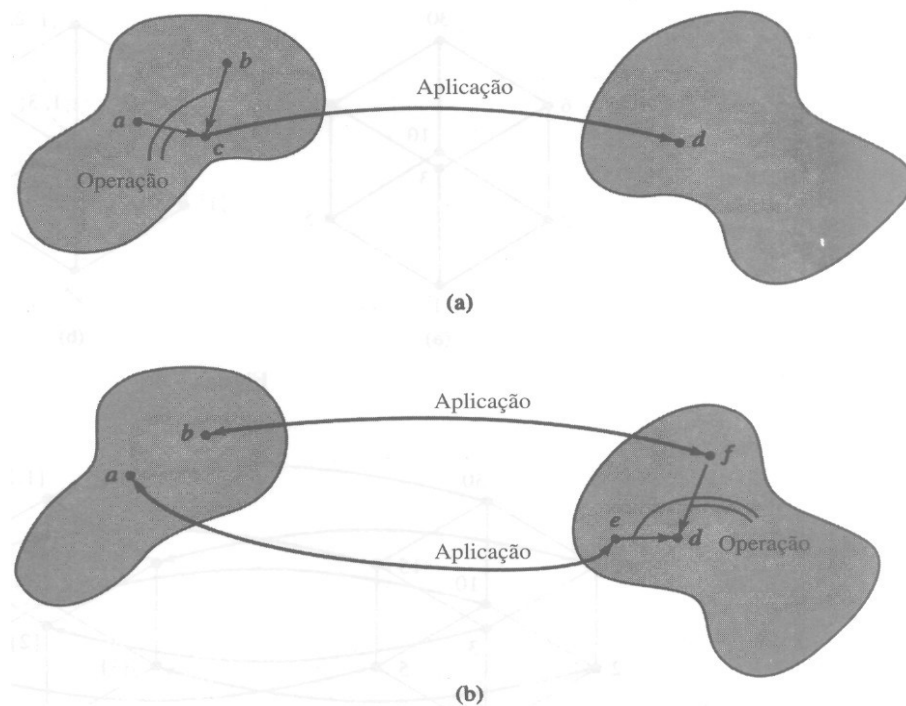


Figura 7.3

b , ou realizar a aplicação em ambos os elementos e então realizar a operação em b . (Veja o diagrama comutativo da Fig. 7.4.) Portanto, para x e y em B , exigimos

$$f(x + y) = f(x) \& f(y)$$

- PRÁTICA 5
- Escreva a equação necessária para que f preserve o efeito da operação binária ..
 - Escreva a equação necessária para que f preserve o efeito da operação unária '.

Aqui temos a definição de um isomorfismo de álgebras booleanas.

Definição: Isomorfismo de Álgebras Booleanas

Sejam $[B, +, \cdot, ', 0, 1]$ e $[b, \&, *, ', \emptyset, 1]$ álgebras booleanas. Uma função $f: B \rightarrow b$ é um **isomorfismo** de $[B, +, \cdot, ', 0, 1]$ em $[b, \&, *, ', \emptyset, 1]$ se

- f é uma bijeção
- $f(x + y) = f(x) \& f(y)$
- $f(x \cdot y) = f(x) * f(y)$
- $f(x') = (f(x))'$

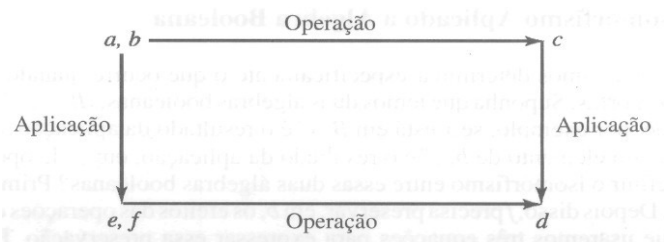


Figura 7.4

PRÁTICA 6 Ilustre as propriedades 2, 3 e 4 da definição através de diagramas comutativos.

Já sabemos (foi uma de nossas inspirações iniciais) que para qualquer conjunto S , $\mathcal{P}(S)$ sob as operações de união, interseção e complemento constitui uma álgebra booleana. Se tomarmos $S = \{1, 2\}$, então os elementos de $\mathcal{P}(S)$ são $\emptyset$, $\{1\}$, $\{2\}$, $\{1, 2\}$. As operações são dadas pelas seguintes tabelas:

$\cup$	$\emptyset$	$\{1, 2\}$	$\{1\}$	$\{2\}$
$\emptyset$	$\emptyset$	$\{1, 2\}$	$\{1\}$	$\{2\}$
$\{1, 2\}$	$\{1, 2\}$	$\{1, 2\}$	$\{1, 2\}$	$\{1, 2\}$
$\{1\}$	$\{1\}$	$\{1, 2\}$	$\{1\}$	$\{1, 2\}$
$\{2\}$	$\{2\}$	$\{1, 2\}$	$\{1, 2\}$	$\{2\}$

$\cap$	$\emptyset$	$\{1, 2\}$	$\{1\}$	$\{2\}$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
$\{1, 2\}$	$\emptyset$	$\{1, 2\}$	$\{1\}$	$\{2\}$
$\{1\}$	$\emptyset$	$\{1\}$	$\{1\}$	$\emptyset$
$\{2\}$	$\emptyset$	$\{2\}$	$\emptyset$	$\{2\}$

$'$	
$\emptyset$	$\{1, 2\}$
$\{1, 2\}$	$\emptyset$
$\{1\}$	$\{2\}$
$\{2\}$	$\{1\}$

Uma álgebra booleana pode ser definida no conjunto $B = \{0, 1, a, a'\}$ onde as tabelas que definem as operações $+$, $\cdot$ e $'$ se parecem com as tabelas a seguir (veja o Exercício 1):

$+$	0	1	a	a'
0	0	1	a	a'
1	1	1	1	1
a	a	1	a	1
a'	a'	1	1	a'

$\cdot$	0	1	a	a'
0	0	0	0	0
1	0	1	a	a'
a	0	a	a	0
a'	0	a'	0	a'

$'$	
0	1
1	0
a	a'
a'	a

Afirmamos que a aplicação $f: B \rightarrow \mathcal{P}(S)$ dada por

$$\begin{aligned} f(0) &= \emptyset \\ f(1) &= \{1, 2\} \\ f(a) &= \{1\} \\ f(a') &= \{2\} \end{aligned}$$

é um isomorfismo. Ela certamente é uma bijeção. Para B , podemos verificar cada uma das equações a seguir

$$\begin{aligned} f(x + y) &= f(x) \cup f(y) \\ f(x \cdot y) &= f(x) \cap f(y) \\ f(x') &= (f(x))' \end{aligned}$$

examinando todos os casos possíveis. Portanto, por exemplo,

$$f(a \cdot 1) = f(a) = \{1\} = \{1\} \cap \{1, 2\} = f(a) \cap f(1)$$

PRÁTICA 7 Verifique as seguintes equações:

- $f(0 + a) = f(0) \cup f(a)$
- $f(a + a') = f(a) \cup f(a')$
- $f(a \cdot a') = f(a) \cap f(a')$
- $f(1') = (f(1))'$

Os demais casos também se verificam. Mesmo sem testar todos os casos, está bem claro que funciona porque ela apenas estabelece novos rótulos para os elementos da tabela de B , de modo que eles remontem a

tabela de $\mathcal{P}(S)$. Em geral, no entanto, pode não ser simples determinar quando uma dada f é um isomorfismo entre duas ocorrências de uma estrutura. É igualmente difícil determinar se duas ocorrências de uma estrutura são isomorfas; precisamos imaginar uma função que realize este isomorfismo ou mostrar que não existe tal função. Um caso em que tal função não existe é quando os conjuntos envolvidos não são do mesmo tamanho; não podemos ter uma álgebra booleana de quatro elementos isomorfa a uma álgebra booleana de oito elementos.

Apenas mostramos que uma álgebra booleana de quatro elementos em particular é isomorfa a $\mathcal{P}(\{1, 2\})$. Isso sugere que qualquer álgebra booleana finita é isomorfa à álgebra booleana de um conjunto de partes. Apesar de enunciarmos isto como um teorema, não o demonstraremos.

Teorema de Álgebras Booleanas Finitas

Seja B qualquer álgebra booleana com n elementos. Então $n = 2^m$ para algum m e B é isomorfa a $\mathcal{P}(\{1, 2, \dots, m\})$.

O teorema acima nos dá duas informações. O número de elementos em uma álgebra booleana precisa ser uma potência de 2. Além disso, vemos que as álgebras booleanas que são conjuntos de partes são — em nossa maneira de pensar, que agrupa os isomorfismos —, na verdade, os únicos tipos de álgebras booleanas que existem. De certa maneira, andamos em círculo. Definimos uma álgebra booleana para representar diversos tipos de situações; agora vemos que (para o caso finito) as situações, exceto pelos rótulos atribuídos aos objetos, são sempre as mesmas!

Revisão da Seção 7.1

Técnicas

- Determinar se algo é uma álgebra booleana
- Demonstrar as propriedades sobre as álgebras booleanas
- Escrever as equações que indicam que uma função/preserva operação de ocorrência de uma estrutura para outra e verificar ou comprovar como falsa tal equação

Idéias Principais

As estruturas matemáticas servem como modelos ou abstrações de propriedades comuns encontradas em diversas situações.

Todas as álgebras booleanas finitas são isomorfas a álgebras booleanas que sejam conjuntos de partes.

Se houver um isomorfismo (uma bijeção que preserve as propriedades) de A para B , onde A e B são ocorrências de uma estrutura, então, exceto pelos rótulos, A e B são iguais.

Exercícios 7.1

*1. Seja $B = \{0, 1, a, a'\}$ e sejam $+$ e $\cdot$ operações binárias em B . A operação unária $'$ é definida pela tabela

$'$	
0	1
1	0
a	a'
a'	a

Suponha que sabemos que $[B, +, \cdot, ', 0, 1]$ é uma álgebra booleana. Usando as propriedades que precisam valer em todas as álgebras booleanas, preencha as tabelas a seguir, a fim de definir as operações binárias $+$ e $\cdot$:

$+$	0	1	a	a'
0				
1				
a				
a'				

$\cdot$	0	1	a	a'
0				
1				
a				
a'				

2. Defina duas operações binárias $+$ e $\cdot$ no conjunto $\mathbb{Z}$ de inteiros como $x + y = \max(x, y)$ e $x \cdot y = \min(x, y)$.
- Mostre que as propriedades comutativa, associativa e distributiva de uma álgebra booleana valem para essas duas operações em $\mathbb{Z}$.
 - Mostre que independentemente de qual elemento de $\mathbb{Z}$ seja escolhido para ser o 0, a propriedade $x + 0 = x$ de uma álgebra booleana não vale.
3. Seja S o conjunto $\{0, 1\}$. Então S^2 é o conjunto de todos os pares ordenados de 0s e 1s; $S^2 = \{(0, 0), (0, 1), (1, 0), (1, 1)\}$. Considere o conjunto B de todas as funções de S^2 em S . Por exemplo, uma função $f(x, y)$ é dada por

$$\begin{aligned} f(0, 0) &= 0 \\ f(0, 1) &= 1 \\ f(1, 0) &= 1 \\ f(1, 1) &= 1 \end{aligned}$$

- Quantos elementos B possui?
- Para f_1 e f_2 elementos de B e (x, y) e S^2 , defina

$$\begin{aligned} (f_1 + f_2)(x, y) &= \max(f_1(x, y), f_2(x, y)) \\ (f_1 \cdot f_2)(x, y) &= \min(f_1(x, y), f_2(x, y)) \\ f'_1(x, y) &= \begin{cases} 1 & \text{se } f_1(x, y) = 0 \\ 0 & \text{se } f_1(x, y) = 1 \end{cases} \end{aligned}$$

Suponha

$$\begin{aligned} f_1(0, 0) &= 1 & f_2(0, 0) &= 1 \\ f_1(0, 1) &= 0 & f_2(0, 1) &= 1 \\ f_1(1, 0) &= 1 & f_2(1, 0) &= 0 \\ f_1(1, 1) &= 0 & f_2(1, 1) &= 0 \end{aligned}$$

Quais são as funções $f_1 + f_2$, $f_1 \cdot f_2$ e f'_1 ?

- Demonstre que $[B, +, \cdot, ', 0, 1]$ é uma álgebra booleana onde as funções 0 e 1 são definidas por

$$\begin{aligned} 0(0, 0) &= 0 & 1(0, 0) &= 1 \\ 0(0, 1) &= 0 & 1(0, 1) &= 1 \\ 0(1, 0) &= 0 & 1(1, 0) &= 1 \\ 0(1, 1) &= 0 & 1(1, 1) &= 1 \end{aligned}$$

4. Demonstre as seguintes propriedades das álgebras booleanas. Justifique cada passo das demonstrações.

- ★ a. $(x')' = x$
- ★ b. $x + (x \cdot y) = x$, $x \cdot (x + y) = x$ (propriedades de absorção)
- c. $(x + y)' = x' \cdot y'$, $(x \cdot y)' = x' + y'$ (leis de De Morgan)
- d. $x \cdot [y + (x \cdot z)] = (x \cdot y) + (x \cdot z)$
 $x + [y \cdot (x + z)] = (x + y) \cdot (x + z)$ } (propriedades modulares)
- e. $(x + y) \cdot (x' + y) = y$
 $(x \cdot y) + (x' \cdot y) = y$
- f. $(x + y) + (y \cdot x') = x + y$
 $(x \cdot y) \cdot (y + x') = x \cdot y$
- ★ g. $x + y' = x + (x' \cdot y + x \cdot y)'$
- h. $[(x \cdot y) \cdot z] + (y \cdot z) = y \cdot z$
- i. $x + (x \cdot y + y) = (x + y) \cdot (y + 1)$
- j. $(x + y') \cdot z = [(x' + z') \cdot (y + z')]'$

5. Uma nova operação binária $\oplus$ em uma álgebra booleana é definida por

$$x \oplus y = x \cdot y' + y \cdot x'$$

Demonstre que

- ★a. $x \oplus y = y \oplus x$
- b. $x \oplus x = 0$
- c. $0 \oplus x = x$
- d. $1 \oplus x = x'$

6. Demonstre as seguintes afirmações referentes a álgebras booleanas:
 - a. Se $x + y = 0$, então $x = 0$ e $y = 0$
 - b. $x = y$ se, e somente se, $x \cdot y' + y \cdot x' = 0$.
7. a. Encontre um exemplo de uma álgebra booleana com elementos x, y e z para a qual $x + y = x + z$ mas $y \neq z$. (Esta é uma evidência de que a aritmética comum de inteiros não é uma álgebra booleana.)
b. Demonstre que em qualquer álgebra booleana, se $x + y = x + z$ e $x' + y = x' + z$, então $y = z$.
8. Demonstre que o elemento 0 em qualquer álgebra booleana é único; demonstre que o elemento 1 em qualquer álgebra booleana é único.
- ★9. Sejam $(S, \leq)$ e $(S', \leq')$ dois conjuntos parcialmente ordenados. $(S, \leq)$ é isomorfo a $(S', \leq')$ se houver uma bijeção $f: S \rightarrow S'$ tal que para x, y em S , $x < y \rightarrow f(x) <' f(y)$ e $f(x) <' f(y) \rightarrow x < y$.
 - a. Mostre que existem exatamente dois conjuntos parcialmente ordenados com dois elementos que não são isomorfos.
 - b. Mostre que existem exatamente cinco conjuntos parcialmente ordenados com três elementos que não são isomorfos.
 - c. Quantos conjuntos parcialmente ordenados com quatro elementos não isomorfos existem?
10. Encontre um exemplo de dois conjuntos parcialmente ordenados $(S, \leq)$ e $(S', \leq')$ e uma bijeção $f: S \rightarrow S'$ onde, para x, y em S , $x <' y \rightarrow f(x) <' f(y)$, mas $f(x) <' f(y) \nrightarrow x < y$.
11. Seja $S = \{0, 1\}$ e seja uma operação binária $\cdot$ definida em S por

$\cdot$	0	1
0	1	0
1	0	1

Seja $T = \{5, 7\}$ e seja uma operação binária $+$ definida em T por

$+$	5	7
5	7	5
7	5	7

Considere $[S, \cdot]$ e $[T, +]$ como estruturas matemáticas.

- a. Se uma função é um isomorfismo de $[S, \cdot]$ em $[T, +]$, quais duas propriedades/precisa satisfazer?
- b. Defina uma função e demonstre que ela é um isomorfismo de $[S, \cdot]$ em $[T, +]$.
- ★12. Seja $\mathbb{R}$ o conjunto dos números reais e $\mathbb{R}^+$ o conjunto dos números reais positivos. A adição é uma operação binária em $\mathbb{R}$, e a multiplicação é uma operação binária em $\mathbb{R}^+$. Considere $[\mathbb{R}, +]$ e $[\mathbb{R}^+, \cdot]$ como estruturas matemáticas.
 - a. Demonstre que a função f definida por $f(x) = 2^x$ é uma bijeção de $\mathbb{R}$ em $\mathbb{R}^+$.
 - b. Escreva a equação que um isomorfismo de $[\mathbb{R}, +]$ em $[\mathbb{R}^+, \cdot]$ precisa satisfazer.
 - c. Demonstre que a função f do item (a) é um isomorfismo de $[\mathbb{R}, +]$ em $[\mathbb{R}^+, \cdot]$.
 - d. Qual a função inversa f^{-1} ?
 - e. Demonstre que f^{-1} é um isomorfismo de $[\mathbb{R}^+, \cdot]$ em $[\mathbb{R}, +]$.
13. Nesta seção definimos um isomorfismo da álgebra booleana de conjunto $B = \{0, 1, a, a'\}$ na álgebra booleana de conjunto $\mathcal{P}(\{1, 2\})$. Como as duas álgebras são essencialmente as mesmas, uma operação em uma pode ser simulada por aplicação, seguida da operação na outra, e então realizando uma aplicação de volta.
 - a. Use a álgebra booleana em $\mathcal{P}(\{1, 2\})$ para simular o cálculo de $1 \cdot a'$ na álgebra booleana em B .
 - b. Use a álgebra booleana em $\mathcal{P}(\{1, 2\})$ para simular o cálculo de $(a)'$ na álgebra booleana em B .

- c. Use a álgebra booleana em B para simular o cálculo de $\{1\} \cup \{2\}$ na álgebra booleana em $\mathcal{P}(\{1, 2\})$.
 d. Use a álgebra booleana em B para simular o cálculo de $\{1\} \cap \{1, 2\}$ na álgebra booleana em $\mathcal{P}(\{1, 2\})$.
14. Considere o conjunto B de todas as funções de $\{0, 1\}^2$ em $\{0, 1\}$. Podemos definir as operações $+$, $\cdot$ e $'$ em B da seguinte maneira:

$$(f_1 + f_2)(x, y) = \max(f_1(x, y), f_2(x, y))$$

$$(f_1 \cdot f_2)(x, y) = \min(f_1(x, y), f_2(x, y))$$

$$f_1'(x, y) = \begin{cases} 1 & \text{se } f_1(x, y) = 0 \\ 0 & \text{se } f_1(x, y) = 1 \end{cases}$$

Então $[B, +, \cdot, ', 0, 1]$ é uma álgebra booleana de 16 elementos (veja o Exercício 3). A tabela a seguir atribui nomes a essas 16 funções.

(x, y)	0	1	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}
$(0, 0)$	0	1	1	1	1	1	1	1	0	0	0	1	0	0	0	0
$(0, 1)$	0	1	0	1	1	0	1	0	1	1	1	0	0	1	0	0
$(1, 0)$	0	1	1	0	1	0	0	1	1	1	0	0	1	0	1	0
$(1, 1)$	0	1	0	0	0	0	1	1	1	0	1	1	1	0	0	1

De acordo com o teorema de álgebras booleanas finitas, esta álgebra booleana é isomorfa a $[\mathcal{P}(\{1, 2, 3, 4\}), \cup, \cap, ', \emptyset, \{1, 2, 3, 4\}]$. Complete a definição a seguir de um isomorfismo de B em $\mathcal{P}(\{1, 2, 3, 4\})$:

$$\begin{aligned} 0 &\rightarrow \emptyset \\ 1 &\rightarrow \{1, 2, 3, 4\} \\ f_4 &\rightarrow \{1\} \\ f_{12} &\rightarrow \{2\} \\ f_{13} &\rightarrow \{3\} \\ f_{14} &\rightarrow \{4\} \end{aligned}$$

- ★15. Suponha que $[B, +, \cdot, ', 0, 1]$ e $[b, \&, *, ', \emptyset, \mathcal{A}]$ são álgebras booleanas isomorfas e que f é um isomorfismo de B em b .

- a. Demonstre que $f(0) = \emptyset$.
 b. Demonstre que $f(1) = \mathcal{A}$.

16. Uma álgebra booleana também pode ser definida como um conjunto parcialmente ordenado com algumas propriedades adicionais. Seja $(B, \leq)$ um conjunto parcialmente ordenado. Para quaisquer $x, y \in B$, definimos **menor limite superior** de x e y como um elemento z tal que $x \leq z$, $y \leq z$ e se houver qualquer elemento z^* com $x \leq z^*$ e $y \leq z^*$, então $z \leq z^*$. O **maior limite inferior** de x e y é um elemento w tal que $w \leq x$, $w \leq y$ e se houver um elemento w^* tal que $w^* \leq x$ e $w^* \leq y$, então $w^* \leq w$. Um **reticulado** é um conjunto parcialmente ordenado em que cada dois elementos x e y têm pelo menos um menor limite superior, denotado por $x + y$, e um maior limite inferior, denotado por $x \cdot y$.

- a. Demonstre que em qualquer reticulado:

- (i) $x \cdot y = x$ se, e somente se, $x \leq y$
 (ii) $x + y = y$ se, e somente se, $x \leq y$

- b. Demonstre que em qualquer reticulado:

- (i) $x + y = y + x$
 (ii) $x \cdot y = y \cdot x$
 (iii) $(x + y) + z = x + (y + z)$
 (iv) $(x \cdot y) \cdot z = x \cdot (y \cdot z)$

- c. Um reticulado L é **complementado** se houver um menor elemento 0 e um maior elemento 1, e para cada $x \in L$ existir x' tal que $x + x' = 1$ e $x \cdot x' = 0$. Demonstre que, em um reticulado complementado L ,

$$x + 0 = x \text{ e } x \cdot 1 = x$$

para todos os $x \in L$.

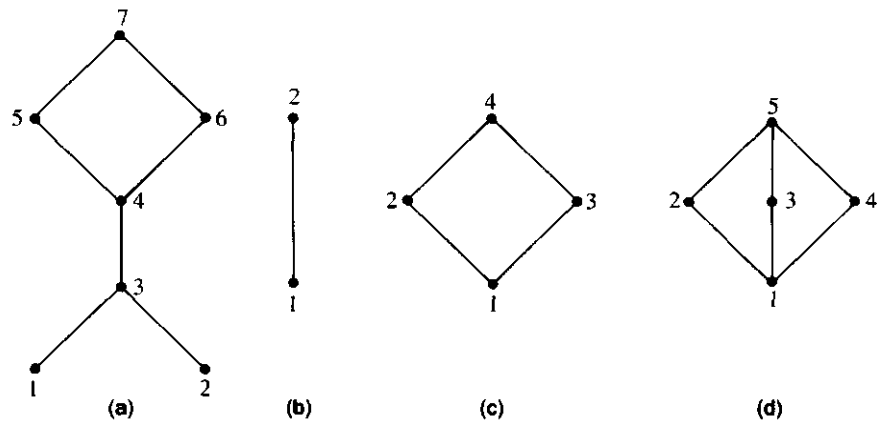
- d. Um reticulado L é **distributivo** se

$$x + (y \cdot z) = (x + y) \cdot (x + z)$$

e

$$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$$

para todos x, y e $z \in L$. Pelos itens (b) e (c), um reticulado complementado e distributivo é uma álgebra booleana. Quais dos diagramas de Hasse referentes a conjuntos parcialmente ordenados da figura a seguir não representam álgebras booleanas? Por quê? (Dica: Em uma álgebra booleana, o complemento de um elemento é único.)



Exercício 16

17. De acordo com o teorema de álgebras booleanas finitas, que não demonstramos, qualquer álgebra booleana finita precisa ter 2^m elementos para algum m . Demonstre a afirmação mais fraca de que nenhuma álgebra booleana pode ter um número ímpar de elementos. (Perceba que na definição de álgebra booleana, 0 e 1 são elementos distintos de B , de forma que B tem pelo menos dois elementos. Arrume os demais elementos de B de forma que cada elemento esteja associado a seu complemento.)

Seção 7.2 Redes Lógicas

Redes Combinatórias

Elementos de Lógica Básica

Em 1938, o matemático americano Claude Shannon percebeu as semelhanças entre a lógica proposicional e a lógica de circuitos, e imaginou que a álgebra booleana poderia ajudar a sistematizar este então novo domínio da eletrônica.

Imaginemos que as voltagens elétricas transmitidas entre os fios possam assumir dois valores, alto ou baixo, que representaremos por 1 e 0, respectivamente. As flutuações de voltagem entre esses dois valores serão ignoradas, de forma que forcamos um fenômeno discreto binário a partir de um fenômeno originalmente análogo. Supomos ainda que chaves mecânicas podem ser usadas a fim de que o sinal 1 faça com que a chave se feche e o sinal 0 faça com que a chave se abra (veja a Fig. 7.5). Então combinamos duas chaves deste tipo, controladas pelas linhas x_1 e x_2 , em paralelo. Os valores $x_1 = 0$ e $x_2 = 0$ fazem com que ambas as chaves este-

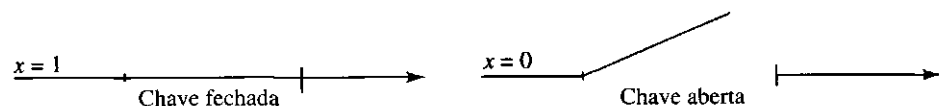


Figura 7.5

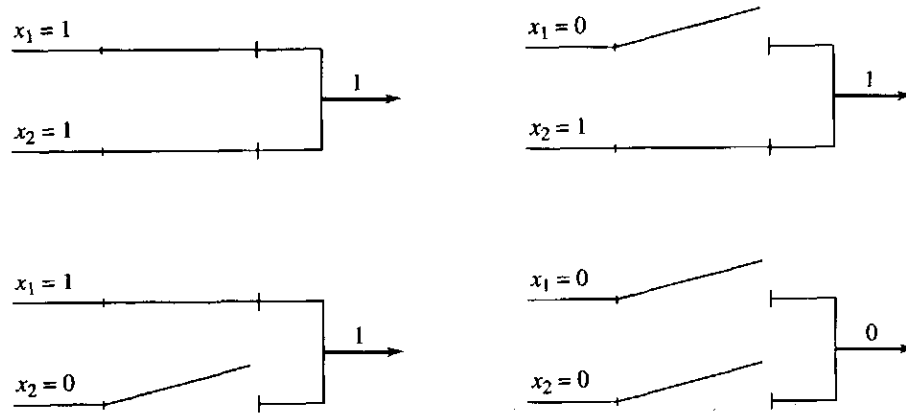


Figura 7.6

x_1	x_2	Fechado
1	1	1
1	0	1
0	1	1
0	0	0

Tabela 7.1

jam abertas e então rompe o circuito, de forma que a voltagem na linha de saída seja 0. Entretanto, se uma das linhas, ou ambas, tiver um valor 1, uma ou ambas as chaves estarão fechadas, e a linha de saída terá valor 1. A Fig. 7.6 apresenta os vários casos.

A Tabela 7.1 resume o comportamento do circuito. Se substituirmos 1 por V e 0 por F, o resultado seria a tabela-verdade do conectivo lógico de disjunção. A disjunção é um exemplo da operação $+$ da álgebra booleana no contexto da lógica proposicional. Portanto, podemos pensar no circuito de forma mais abstrata como um dispositivo eletrônico que realiza a operação booleana $+$. Outros dispositivos realizam as operações booleanas $\cdot$ e $'$. Chaves ligadas em série servem para implementar a operação $\cdot$, por exemplo, onde ambas as chaves devem estar fechadas ($x_1 = 1$ e $x_2 = 1$) a fim de obter uma saída 1. No entanto, vamos ignorar os detalhes da implementação dos dispositivos; é suficiente dizer que a tecnologia avançou de chaves mecânicas em tubos de vácuo e dos posteriores transistores para circuitos integrados. Simplesmente, representaremos esses dispositivos por seus símbolos-padrão.

A **porta OU**, Fig. 7.7a, comporta-se como uma operação booleana $+$. A **porta E**, Fig. 7.7b, comporta-se como uma operação booleana $\cdot$. A Fig. 7.7c mostra um **inversor** (ou **porta NÃO**), que corresponde à operação unária $'$. Devido à propriedade associativa de $+$ e $\cdot$, as portas OU e E podem ter mais do que duas entradas.

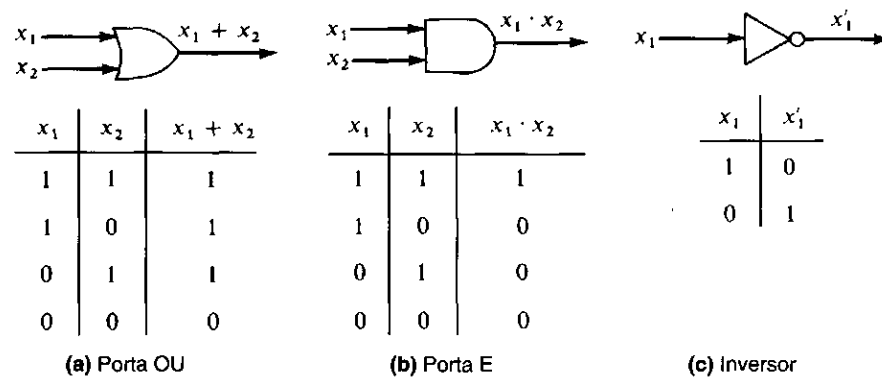


Figura 7.7

Expressões Booleanas

Definição: Expressões Booleanas

Uma expressão booleana com n variáveis, $x_1, x_2, \dots, x_n$ é uma cadeia finita de símbolos formados pela aplicação das seguintes regras:

1. $x_1, x_2, \dots, x_n$ são expressões booleanas.
2. Se P e Q são expressões booleanas, então $(P + Q)$, $(P \cdot Q)$ e (P') também o são.

(A definição de uma expressão booleana é outro exemplo de uma definição recursiva; a regra 1 é o passo básico e a regra 2 é o passo indutivo.) Quando não houver a possibilidade de confusões, podemos omitir os parênteses indicados na regra 2. Além disso, definimos $\cdot$ como tendo precedência sobre $+$ e $'$ como tendo precedência sobre $+$ ou $\cdot$, de forma que $x_1 + x_2 \cdot x_3$, significa $x_1 + (x_2 \cdot x_3)$ e $x_1 + x'_2$ significa $x_1 + (x'_2)$; esta convenção nos permite remover alguns parênteses. E, por último, podemos, geralmente, omitir o símbolo $\cdot$ e simplesmente usar a justaposição dos itens, de forma que $x_1 \cdot x_2$ pode ser escrito como x_1x_2 .

EXEMPLO 6 $x_3, (x_1 + x_2)'x_3, (x_1x_3 + x'_4)x_2$ e $(x'_1x_2)'x_1$ são expressões booleanas. •

Na lógica proposicional, os conectivos lógicos $\vee, \wedge$ e $'$ são exemplos de operações da álgebra booleana. Usando apenas estes conectivos, a definição recursiva de uma wff proposicional (Exemplo 22 do Cap. 2) é um exemplo de definição de expressão booleana.

Funções-Verdade

Definição: Função-Verdade

Uma função-verdade é uma função f tal que $f: \{0, 1\}^n \rightarrow \{0, 1\}$ para algum inteiro $n \geq 1$.

A notação $\{0, 1\}^n$ denota o conjunto de todas as n -uplas de 0s e 1s. Uma função-verdade associa, portanto, um valor 0 ou 1 a cada uma dessas n -uplas.

EXEMPLO 7 A tabela-verdade para a operação booleana $+$ descreve uma função-verdade f com $n = 2$. O domínio de f é $\{(1, 1), (1, 0), (0, 1), (0, 0)\}$ e $f(1, 1) = 1, f(1, 0) = 1, f(0, 1) = 1$ e $f(0, 0) = 0$. Analogamente, a operação booleana $\cdot$ descreve uma função-verdade diferente com $n = 2$ e a operação booleana $'$ descreve uma função-verdade para $n = 1$. •

PRÁTICA 8 a. Se escrevermos uma função-verdade $f: \{0, 1\}^n \rightarrow \{0, 1\}$ de forma tabular (como uma tabela-verdade), quantas linhas terá a tabela?
b. Quantas funções-verdade diferentes existem que levem $\{0, 1\}^2 \rightarrow \{0, 1\}$?
c. Quantas funções-verdade diferentes existem que levem $\{0, 1\}^n \rightarrow \{0, 1\}$? •

Qualquer expressão booleana define uma única função-verdade, da mesma forma que as expressões simples $x_1 + x_2, x_1x_2$ e x'_1 .

EXEMPLO 8 A expressão booleana $x_1x'_2 + x_3$ define a função-verdade dada na Tabela 7.2. •

x_1	x_2	x_3	$x_1x'_2 + x_3$
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	1
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	0

Tabela 7.2 •

Redes e Expressões

Combinando portas E e OU e inversores, podemos construir uma rede lógica que represente determinada expressão booleana e que produza a mesma função-verdade que a função.

EXEMPLO 9 A rede lógica da expressão booleana $x_1x'_2 + x_3$ é mostrada na Fig. 7.8.

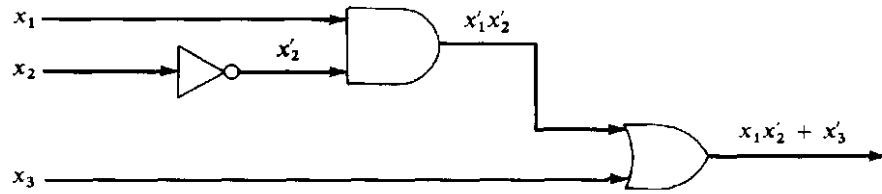


Figura 7.8

PRÁTICA 9 Projete uma rede lógica para as seguintes expressões booleanas:

- a. $x_1 + x_2'$
- b. $x_1(x_2 + x_3)'$

Reciprocamente, se tivermos uma rede lógica, podemos escrever uma expressão booleana com a mesma função-verdade.

EXEMPLO 10 Uma expressão booleana para a rede lógica da Fig. 7.9 é $(x_1x_2 + x_3)'x_3$

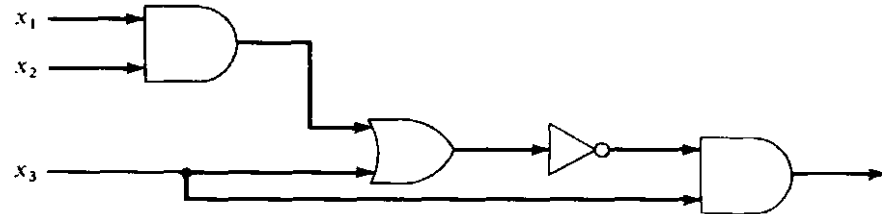


Figura 7.9

PRÁTICA 10 a. Escreva uma expressão booleana para a rede lógica da Fig. 7.10.

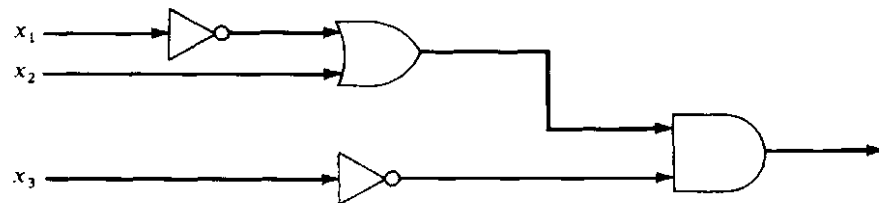


Figura 7.10

b. Escreva a função-verdade (em forma de tabela) para a rede (e a expressão) do item (a).

Redes lógicas construídas com portas E, portas OU e inversores são também chamadas **redes combinatórias**. Elas têm diversos aspectos que devemos perceber. Primeiro, as linhas de entrada ou de saída não se cruzam, exceto ao passarem pelas portas. No entanto, as linhas podem ser divididas a fim de fornecerem entradas para mais de uma porta. Não há laços onde a saída de um componente sirva como entrada para este mesmo componente. Finalmente, a saída de uma rede é uma função instantânea da entrada; não há elementos de atraso que capturem e se lembrem dos sinais de entrada. Perceba ainda que a figura de qualquer rede lógica é, na verdade, um grafo direcionado.

Forma Canônica

Esta é situação até agora (as setas indicam procedimentos que podemos realizar):

função-verdade $\leftarrow$ expressão booleana $\leftrightarrow$ rede lógica

Podemos escrever uma única função-verdade a partir de uma rede ou de uma expressão. Dada uma expressão, podemos encontrar uma rede com a mesma função-verdade, e vice-versa. A última peça do quebra-cabeças é como ir de uma função-verdade arbitrária para uma expressão (e, portanto, para uma rede). Um algoritmo para resolver este problema é explicado no próximo exemplo.

EXEMPLO 11 Suponha que desejamos encontrar uma expressão booleana para a função-verdade f da Tabela 7.3. Existem quatro linhas na tabela (linhas 1, 3, 4 e 8) para as quais f é 1. A forma básica de nossa expressão assumirá a forma de uma soma de quatro parcelas

$$() + () + () + ()$$

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	1
0	0	0	0

Tabela 7.3

de modo que a primeira parcela assume o valor 1 para os valores de entrada da linha 1, a segunda parcela assume o valor 1 para os valores de entrada da linha 3, e assim por diante. Portanto, a expressão toda tem valor 1 para essas entradas e nenhuma outra — precisamente o que desejamos. (Outras entradas fazem com que as parcelas, e portanto toda a soma, tenham valor 0.)

Cada parcela na soma será um produto de termos na forma $\alpha\beta\gamma$, onde α é x_1 ou x'_1 , β é x_2 ou x'_2 e γ é x_3 ou x'_3 . Se o valor de entrada de $x_i, i = 1, 2, 3$ na linha que estivermos olhando for 1, então usamos x_i , do contrário, se o valor de entrada for 0, usamos x'_i . Esses valores farão com que $\alpha\beta\gamma$ seja 1 para a linha em questão e 0 para as demais. Portanto, temos

linha 1: $x_1x_2x_3$
 linha 3: $x_1x'_2x_3$
 linha 4: $x_1x'_2x'_3$
 linha 7: $x'_1x'_2x_3$

A expressão final é

$$(x_1x_2x_3) + (x_1x'_2x_3) + (x_1x'_2x'_3) + (x'_1x'_2x_3)$$

O procedimento descrito no Exemplo 11 sempre nos leva a uma expressão que é uma soma de produtos, chamada **forma canônica de soma de produtos**, ou **forma normal disjuntiva**, para uma dada função-verdade. O único caso que este procedimento não trata é quando a função tem sempre o valor 0. Então usamos uma expressão como

$$x_1x'_1$$

que é também uma soma (um termo) de produtos. Portanto, podemos encontrar uma expressão na forma de soma de produtos para representar qualquer função-verdade. Uma descrição em pseudocódigo do algoritmo é dada a seguir. Para este algoritmo, a entrada é uma tabela-verdade com n variáveis $x_1, x_2, \dots, x_n$; a saída é uma expressão booleana em sua forma normal disjuntiva com a mesma função-verdade.

ALGORITMO Soma — De — Produtos

procedure Soma — De — Produtos(*tabela-verdade*: tabela; *n*: integer);

{a tabela-verdade representa uma função-verdade de n argumentos; o procedimento encontra a expressão canônica na forma de soma de produtos para esta função-verdade}

```

var
  soma; {expressão da soma de produtos}
  produto; {uma única parcela da soma, um produto}
  i, j; {índices de laços}
begin
  soma := vazio;
  j := 1;
  while j <= 2n do
    begin
      if valor-verdade da linha j é 1 then
        begin
          inicia produto;
          for i := 1 to n do
            if xi = 1 then
              inclui xi no produto
            else
              inclui x'i no produto;
          soma := soma + produto;
        end;
      j:=j+1;
    end;
  if soma é vazia then
    soma := x1x'2;
  escreve ('A expressão canônica de soma de produtos para a função-verdade dada é ', soma);
end;

```

Como qualquer expressão tem uma rede correspondente, qualquer função-verdade tem uma representação na forma de uma rede lógica. Além disso, as portas E, OU e NÃO são os únicos componentes necessários para construir a rede. Portanto, podemos constituir a rede de qualquer função-verdade com apenas três tipos de componentes — e bastante fio! Veremos mais adiante que é necessário usar apenas um tipo de componente.

Dada uma função-verdade, a forma canônica de soma de produtos que acabamos de descrever é uma expressão que descreve a função-verdade, mas não é a única. Um método para se obter uma expressão diferente para qualquer função-verdade é dado no Exercício 14, ao fim desta Seção.

EXEMPLO 12 A rede para forma canônica de soma de produtos do Exemplo 11 é mostrada na Fig. 7.11. Desenhamos as entradas para cada porta E separadamente para simplificar, mas, na verdade, as únicas entradas x_1 , x_2 ou x_3 podem ser divididas o quanto for necessário.

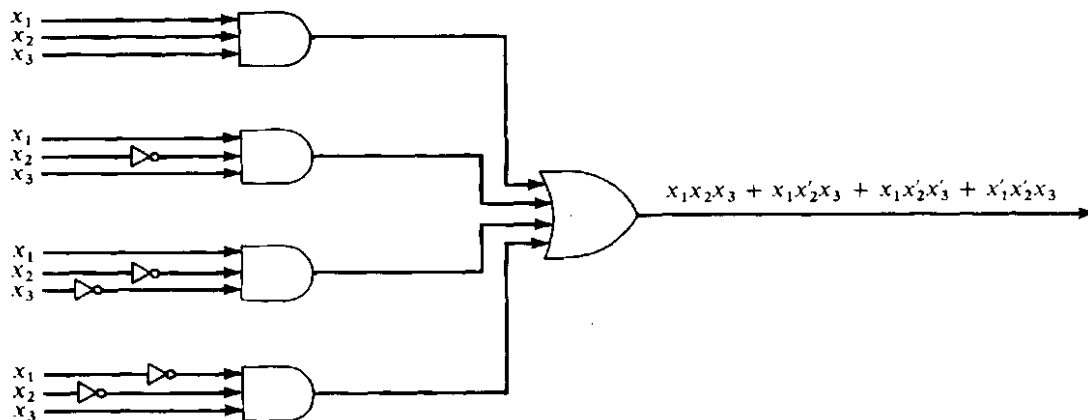


Figura 7.11

- PRÁTICA 11**
- Encontre a forma canônica de soma de produtos para a função-verdade da Tabela 7.4.
 - Desenhe a rede relativa à expressão encontrada no item (a).

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	1
0	0	0	1

Tabela 7.4

Minimização

Como já vimos, uma dada função-verdade pode ser representada por mais de uma expressão booleana e, portanto, por mais de uma rede lógica composta de portas E, OU e inversores.

EXEMPLO 13 A expressão booleana

$$x_1x_3 + x'_2$$

tem a função-verdade da Tabela 7.4. A rede lógica correspondente a esta expressão é mostrada na Fig. 7.12. Compare esta rede com a que você obteve na Prática 11(b)!

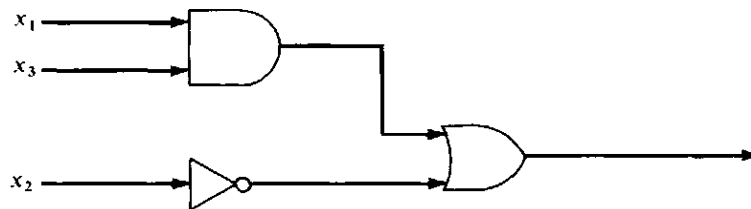


Figura 7.12

Definição: Expressões Booleanas Equivalentes

Duas expressões booleanas são **equivalentes** se têm a mesma função-verdade.

Sabemos que

$$x_1x_2x_3 + x_1x'_2x_3 + x_1x'_2x'_3 + x_1x_2x'_3 + x'_1x'_2x'_3$$

e

$$x_1x_3 + x'_2$$

por exemplo, são expressões booleanas equivalentes.

Naturalmente, a equivalência de expressões booleanas é uma relação de equivalência no conjunto de todas as expressões booleanas de n variáveis. A cada classe de equivalência está associada uma função-verdade distinta. Dada uma função-verdade, o algoritmo *Soma-De-Produtos* produz um elemento particular da classe associada, a saber, a forma canônica de soma de produtos. No entanto, se estivermos tentando projetar a rede lógica para esta função, desejaremos encontrar um elemento da classe que seja o mais simples possível. Desta forma, podemos construir a rede da Fig. 7.12 ao invés da rede obtida na Prática 11 (b).

Como podemos reduzir uma expressão booleana a uma expressão equivalente mais simples? Podemos usar as propriedades de álgebras booleanas, porque elas são equivalentes a expressões booleanas. Se P é uma expressão booleana contendo a subexpressão $x_1 + (x_2x_3)$, por exemplo, e Q é a expressão obtida de P através da substituição de $x_1 + (x_2x_3)$ pela expressão equivalente $(x_1 + x_2)(x_1 + x_3)$, então P e Q são equivalentes.

EXEMPLO 14 Usando as propriedades da álgebra booleana, podemos reduzir

$$x_1x_2x_3 + x_1x'_2x_3 + x_1x'_2x'_3 + x'_1x'_2x_3 + x'_1x'_2x'_3$$

a

$$x_1x_3 + x'_2$$

da seguinte maneira:

$$\begin{aligned} & x_1x_2x_3 + x_1x'_2x_3 + x_1x'_2x'_3 + x'_1x'_2x_3 + x'_1x'_2x'_3 \\ &= x_1x_2x_3 + x_1x'_2x_3 + x_1x'_2x'_3 + x'_1x'_2x_3 + x'_1x'_2x'_3 \quad (\text{propriedade idempotente}) \\ &= x_1x_3x_2 + x_1x_3x'_2 + x_1x'_2x_3 + x_1x'_2x'_3 + x'_1x'_2x_3 + x'_1x'_2x'_3 \quad (1b) \\ &= x_1x_3(x_2 + x'_2) + x_1x'_2(x_3 + x'_3) + x'_1x'_2(x_3 + x'_3) \quad (3b) \\ &= x_1x_3 \cdot 1 + x_1x'_2 \cdot 1 + x'_1x'_2 \cdot 1 \quad (5a) \\ &= x_1x_3 + x_1x'_2 + x'_1x'_2 \quad (4b) \\ &= x_1x_3 + x'_2x_1 + x'_2x'_1 \quad (1b) \\ &= x_1x_3 + x'_2(x_1 + x'_1) \quad (3b) \\ &= x_1x_3 + x'_2 \cdot 1 \quad (5a) \\ &= x_1x_3 + x'_2 \quad (4b) \end{aligned}$$

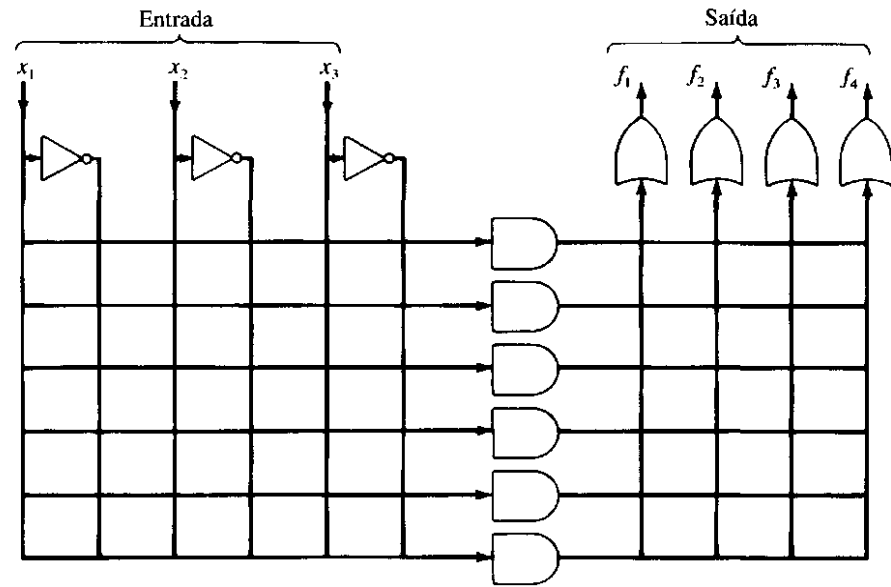
Infelizmente, precisamos de muita engenhosidade ao aplicar as propriedades da álgebra booleana para simplificar uma expressão. Na próxima seção, discutiremos métodos mais sistemáticos para este problema de minimização que requerem menos engenhosidade. Por enquanto, veremos um pouco mais porque desejamos minimizar. Quando redes lógicas eram construídas com portas lógicas separadas, o custo desses componentes era um fator considerável no projeto e era desejável usar o mínimo de componentes possíveis. Agora, no entanto, a maioria de redes é construída com o uso de circuitos integrados, tecnologia que começou a ser desenvolvida no início dos anos 60. Circuito integrado é, ele próprio, uma rede lógica que representa uma ou mais funções-verdade, como se algumas portas lógicas estivessem devidamente interligadas em um pacote. Esses circuitos integrados são então combinados a fim de obter-se o resultado desejado. Como os circuitos integrados são muito pequenos e relativamente baratos, pode parecer que não há mais sentido em minimizar uma rede. No entanto, a minimização é importante porque a confiabilidade de toda a rede é inversamente proporcional ao número de conexões entre os circuitos integrados.

Além disso, os projetistas de circuitos integrados estão altamente interessados no problema da minimização. Os chips quadrados de silício nos quais os circuitos integrados são construídos não podem ter muito mais do que seis milímetros de lado, ainda que possam conter o equivalente a meio milhão de transistores que implementem funções-verdade. Os canais de comunicação necessários para conectar os componentes do chip podem ser tão numerosos que as ligações podem tomar mais espaço do chão do chip que os componentes propriamente ditos. A minimização do número de componentes e da quantidade de ligações necessárias que realizem uma função-verdade torna o chip menos tumultuado e mais fácil de ser projetado. A minimização permite, ainda, que sejam embutidas mais funções em um mesmo chip.

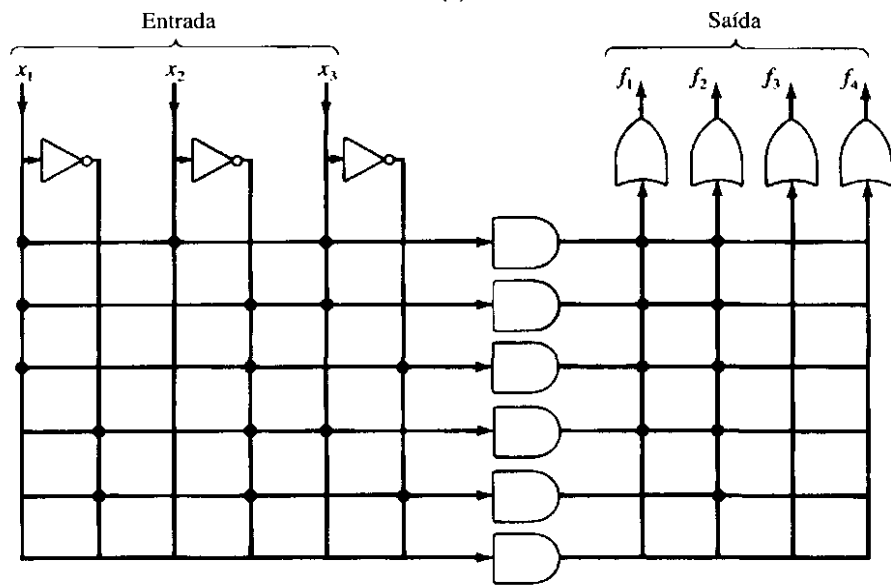
Arrays Lógicos Programáveis

Ao invés de projetar um chip dedicado a implementar algumas funções-verdade em particular, podemos usar um PLA (programmable logic arrays — arrays lógicos programáveis). O PLA é um chip que já contém um array de portas E e um array de portas OU, juntamente com uma grade de fiação e alguns inversores. Uma vez que tenhamos determinado a forma de soma de produtos das funções-verdade desejadas, os elementos correspondentes do PLA são ativados. Apesar de este chip não ser muito eficiente, e ser prático apenas para circuitos lógicos de pequena escala, o PLA pode ser produzido em massa e apenas uma pequena quantidade de tempo (i.e., dinheiro) é necessária para "programá-lo" para as funções desejadas.

EXEMPLO 15 A Fig. 7.13a mostra um PLA para as três entradas x_1 , x_2 e x_3 . Existem quatro linhas de saída, de forma que podemos programar quatro funções neste PLA. Quando o PLA é programado, a linha horizontal que serve de entrada para uma porta E recebe certas entradas e a porta E realiza o produto de tais entradas. A linha vertical que entra em uma porta OU permitirá, quando programada, que a porta OU realize a soma de certos produtos. A Fig. 7.13b mostra a mesma PLA programada a fim de produzir as funções-verdade f , do Exemplo 11



(a)



(b)

Figura 7.13

$(x_1x_2x_3 + x_1x'_2x_3 + x_1x'_2x'_3 + x'_1x'_2x'_3)$ e f_2 da Prática 11 $(x_1x_2x_3 + x_1x'_2x_3 + x_1x'_2x'_3 + x'_1x'_2x'_3)$. Os pequenos círculos representam pontos ativados.

Uma Rede Útil

Podemos projetar uma rede que some números binários, operação básica que um computador precisa saber realizar. As regras para somar dois bits são resumidas na Tabela 7.5.

Podemos expressar a soma como um bit da soma s (o bit da direita da soma verdadeira) junto com um bit de "vai-um" c ; isto nos dá as duas funções-verdade das Tabelas 7.6 e 7.7, respectivamente. A forma canônica de soma de produtos para cada função-verdade é

$$c = x_1x_2$$

x_1	x_2	Soma
1	1	10
1	0	1
0	1	1
0	0	0

Tabela 7.5

x_1	x_2	s
1	1	0
1	0	1
0	1	1
0	0	0

Tabela 7.6

x_1	x_2	c
1	1	1
1	0	0
0	1	0
0	0	0

Tabela 7.7

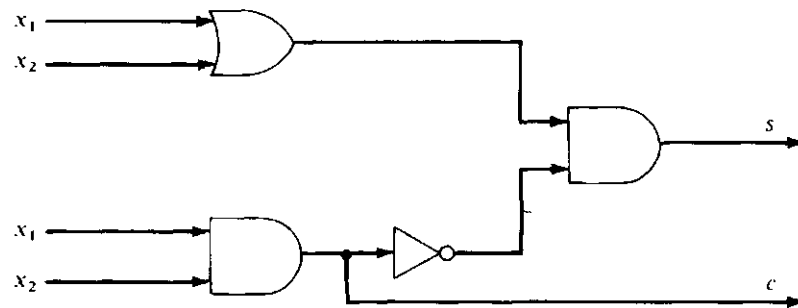
Uma expressão booleana equivalente para s é

$$s = (x_1 + x_2)(x_1 x_2)'$$

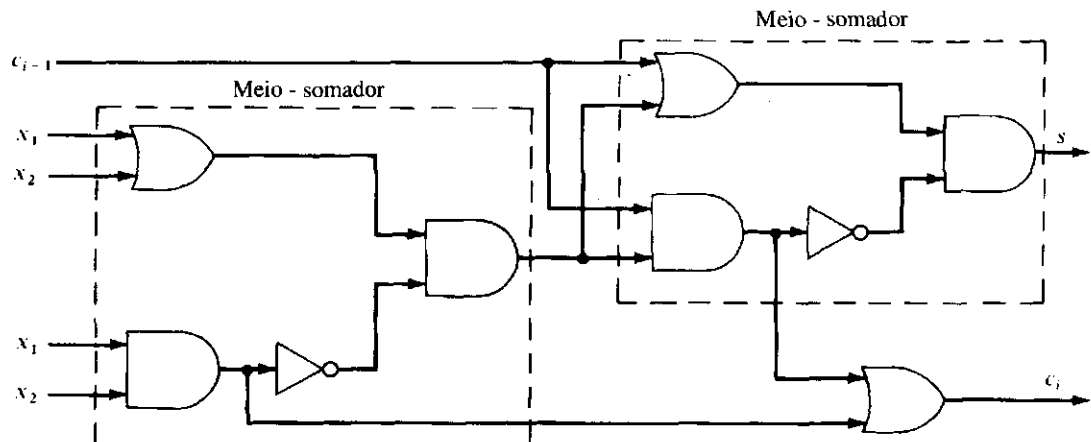
A Fig. 7.14a mostra uma rede com entradas x_1 e x_2 e saídas s e c . Este dispositivo, por motivos que ficarão mais claros um pouco adiante, é chamado **meio-somador**.

Para somar dois números binários, somamos coluna a coluna, a partir dos bits de menor ordem para os bits de maior ordem. A i -ésima coluna (exceto pela primeira coluna) tem como entrada seus dois bits x_1 e x_2 e o bit de "vai-um" da coluna $i - 1$ à sua direita. Por isso, precisamos de um dispositivo que incorpore o bit de "vai-um" como entrada. Isto pode ser conseguido somando-se x_1 e x_2 através de um meio-somador e então somando ao resultado o bit de "vai-um" anterior c_{i-1} , (através de outro meio-somador). Novamente, a saída é um bit de soma e outro de "vai-um" c_i , onde c_i vale 1 se quaisquer dos dois meio-somadores tiverem produzido 1 como bit de "vai-um". O **somador completo** é mostrado na Fig. 7.14b e é composto por dois meio-somadores compostos e uma porta OU adicional.

Para somarmos dois números binários de n bits, os dois bits de baixa ordem, onde não há "vai-um" de entrada, podem ser somados através de um meio-somador. Daí para a frente, o indicador de "vai-um" precisa ser propagado entre $n - 1$ somadores completos. Apesar de termos assumido que as portas lógicas geram suas



(a) Meio - somador



(b) Somador completo

Figura 7.14

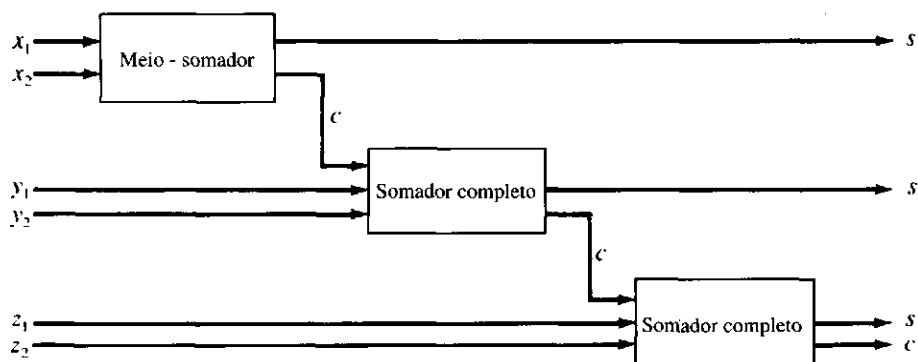


Figura 7.15

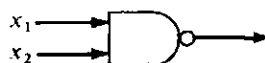
saídas instantaneamente, existe, na verdade, um pequeno atraso que pode ser percebido para n grande. Existem circuitos disponíveis para acelerar o processo de adição nos computadores modernos, embora possamos economizar mais tempo através de uma representação mais inteligente dos números a serem somados.

A Fig. 7.15 mostra os módulos necessários para se somar dois números binários de 3 bits $z_1y_1z_1$ e $z_2y_2z_2$. A soma resultante é obtida lendo-se os bits de saída de cima para baixo (do bit de baixa ordem para o de alta ordem).

PRÁTICA 12 Simule a operação do circuito da Fig. 7.15 para a soma de 101 e 111.

Outros Componentes Lógicos

Na verdade, os componentes básicos usados em circuitos integrados não são as portas E, OU e NÃO, mas as portas NE e NOU. A Fig. 7.16 mostra o símbolo-padrão para a **porta NE** (a porta NÃO-E) e sua função-verdade. A porta NE sozinha é o suficiente para conceber qualquer função-verdade porque as redes que usam apenas as portas NE podem realizar as tarefas de inversores, portas OU e portas E. A Fig. 7.17 mostra essas redes.



x_1	x_2	$(x_1x_2)'$
1	1	0
1	0	1
0	1	1
0	0	1

Figura 7.16

A **porta NOU** (porta NÃO-OU) e sua função-verdade associada são mostradas na Fig. 7.18. Um Exercício ao fim desta seção pedirá que você construa redes usando apenas portas NOU para construir inversores e portas OU e E.

Apesar de podermos construir uma rede para uma função-verdade substituindo as portas E, OU e os inversores na forma canônica ou da forma minimizada pelas redes NE correspondentes, podemos obter melhores resultados, se nos valermos das propriedades específicas da porta NE diretamente.

- PRÁTICA 13**
- Reescreva a rede da Fig. 7.12 com componentes NE, substituindo diretamente as portas E, OU e o inversor, como na Fig. 7.17.
 - Reescreva a expressão booleana $x_1x_3 + x_2'$ da Fig. 7.12, usando as leis de De Morgan e depois construa uma rede usando apenas duas portas NE.

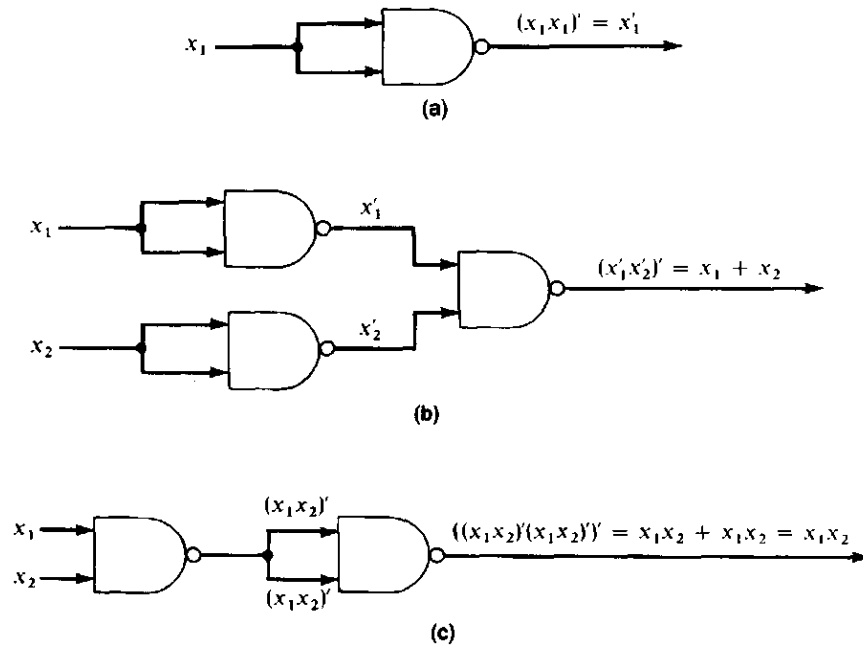


Figura 7.17



x_1	x_2	$(x_1 + x_2)'$
1	1	0
1	0	0
0	1	0
0	0	1

Figura 7.18

Construindo Funções-Verdade

Sabemos como escrever uma expressão booleana e construir uma rede a partir de uma dada função-verdade. Em geral, a função-verdade propriamente dita precisa, antes, ser deduzida a partir da descrição do problema em questão.

EXEMPLO 16

Em uma firma de cosméticos por entrega postal, um dispositivo de controle automático é usado para supervisionar o empacotamento das remessas. A firma vende batons, perfumes, maquilagens e esmalte de unha. Como bonificação, é adicionado um xampu em todas as encomendas que incluam perfume ou que incluam batom, maquilagem e esmalte de unha. Como podemos projetar a rede lógica que controle quando o xampu deve ser incluído?

As entradas da rede representam os quatro itens que podem ser ordenados. Vamos chamá-los assim:

- x_1 = batom
- x_2 = perfume
- x_3 = maquilagem
- x_4 = esmalte de unha

Os valores de x_i serão 1 quando este item fizer parte da encomenda, e 0 em caso contrário. A saída da rede deve ser 1 se o xampu deve ser incluído na encomenda, ou 0, em caso contrário. A tabela-verdade do circuito é

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	1
1	1	1	0	1
1	1	0	1	1
1	1	0	0	1
1	0	1	1	1
0	0	1	0	0
1	0	0	1	0
1	0	0	0	0
0	1	1	1	1
0	1	1	0	1
0	1	0	1	1
0	1	0	0	1
0	0	1	1	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	0

Tabela 7.8

mostrada na Tabela 7.8. A forma canônica de soma de produtos para esta função-verdade é grande, mas a expressão $x_1x_3x_4 + x_2$ também representa a função. A Fig. 7.19 mostra a rede lógica para esta expressão.

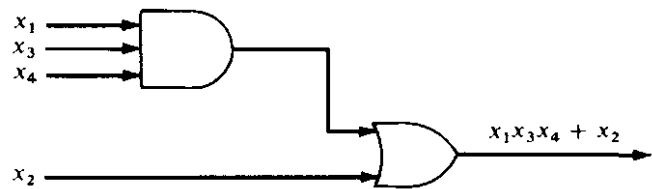


Figura 7.19

PRÁTICA 14 Uma luz de corredor é controlada por dois interruptores, um em cada extremo. Encontre (a) a função-verdade, (b) uma expressão booleana e (c) uma rede lógica que permita que a luz seja ligada e desligada em ambos interruptores.

Em alguns problemas, as funções-verdade correspondentes assumem valores indefinidos porque determinadas combinações de entrada não podem ocorrer (veja o Exercício 23 ao fim desta seção). Sob tais condições não-importa, podemos atribuir qualquer valor à saída.

Em uma linguagem de programação que ofereça os operadores E, OU e NÃO (AND, OR e NOT), projetar a lógica de um programa pode consistir, em parte, em escolher as funções-verdade apropriadas e suas expressões booleanas correspondentes (veja o Exercício 14 da Seção 1.1).

Revisão da Seção 7.2

Técnicas

- Encontrar a função-verdade correspondente a uma dada expressão booleana ou rede lógica
- Construir uma rede lógica com a mesma função-verdade que uma determinada expressão booleana
- Escrever uma expressão booleana com a mesma função-verdade que uma dada rede lógica
- Escrever uma expressão booleana em sua forma canônica de soma de produtos para uma dada função-verdade
- Encontrar uma rede composta apenas de portas NE que tenha a mesma função-verdade que uma dada rede com portas E, OU e inversores
- Encontrar uma função-verdade que satisfaça a descrição de um problema em particular

Idéias Principais

Podemos, efetivamente, converter informações entre as três formas abaixo:

função-verdade ↔ expressão booleana ↔ rede lógica

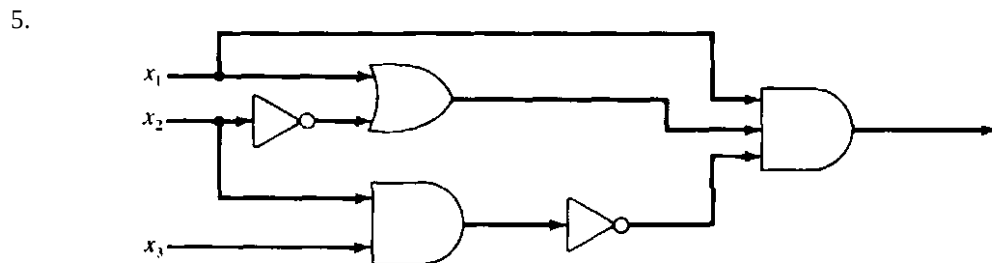
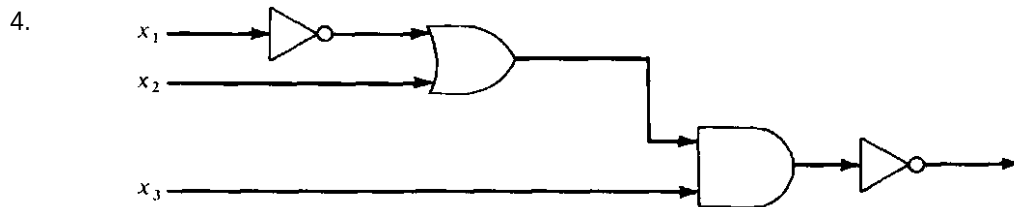
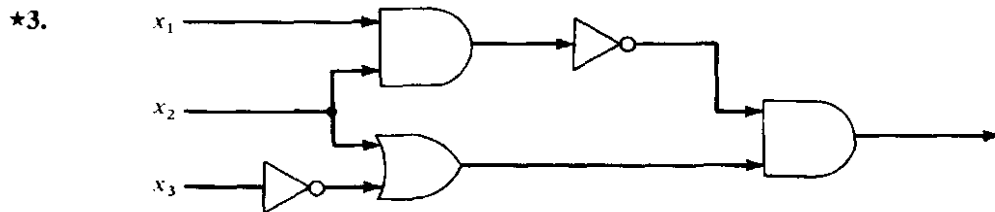
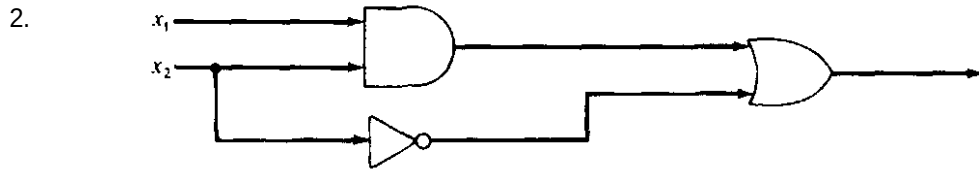
Uma expressão booleana pode, às vezes, ser convertida para uma expressão equivalente mais simples, usando as propriedades da álgebra booleana, produzindo, assim, uma rede mais simples para uma certa função-verdade.

Exercícios 7.2

1. Construa redes lógicas para as expressões booleanas a seguir, usando portas E, portas OU e inversores:

- ★a. $(x'_1 + x_2)x_3$
- b. $(x_1 + x_2)' + x'_1x_3$
- c. $x'_1x_2 + (x_1x_2)'$
- d. $(x_1 + x_2)'x_3 + x'_3$

Nos Exercícios 2 a 5 escreva uma expressão booleana e uma função-verdade para cada uma das redes lógicas mostradas.

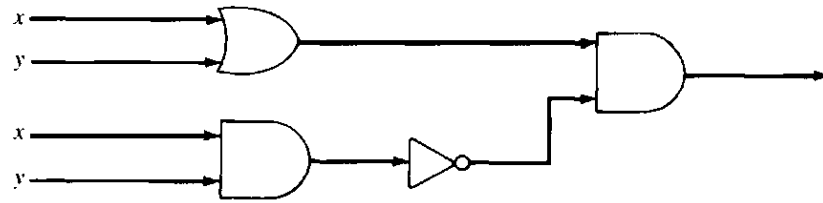


6. a. Escreva a função-verdade referente à operação booleana

$$x \oplus y = xy' + yx'$$

b. Desenhe a rede lógica referente a $x \oplus y$.

c. Mostre que a rede da figura a seguir também representa $x \oplus y$. Explique por que a rede ilustra que $\oplus$ é a operação **OU exclusivo**.



Exercício 6

Nos Exercícios 7 a 10, encontre a forma canônica de soma de produtos para as funções-verdade das tabelas dadas.

★7.

x_1	x_2	$f(x_1, x_2)$
1	1	0
1	0	0
0	1	0
0	0	1

8.

x_1	x_2	$f(x_1, x_2)$
1	1	1
1	0	0
0	1	1
0	0	0

9.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	0
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	0
0	0	0	1

10

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	0
1	1	0	0
1	0	1	1
1	0	0	1
0	1	1	0
0	1	0	1
0	0	1	0
0	0	0	0

★11.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	0
0	0	0	0

- Encontre a forma canônica de soma de produtos referente à função-verdade da tabela acima.
- Desenhe uma rede lógica para a expressão do item (a).
- Use as propriedades da álgebra booleana para reduzir a expressão do item (a) a uma expressão equivalente cuja rede precise apenas de dois componentes lógicos. Desenhe esta rede.

12.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	0
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	1
0	0	1	0
0	0	0	0

- Encontre a forma canônica de soma de produtos referente à função-verdade da tabela acima.
- Desenhe uma rede lógica para a expressão do item (a).

- c. Use as propriedades da álgebra booleana para reduzir a expressão do item (a) a uma expressão equivalente cuja rede só precise de três componentes lógicos. Desenhe esta rede.

13. a. Mostre que as duas expressões booleanas

$$(x_1 + x_2)(x'_1 + x_3)(x_2 + x_3)$$

e

$$(x_1x_3) + (x'_1x_2)$$

são equivalentes escrevendo as tabelas-verdade de ambas.

- b. Escreva a forma canônica de soma de produtos das expressões do item (a).

- c. Use as propriedades da álgebra booleana para reduzir uma das expressões do item (a) a outra.

- ★14. Também existe uma **forma canônica de produto de somas (forma normal conjuntiva)** para qualquer função-verdade. Esta expressão tem a forma

$$() () \dots ()$$

com cada fator sendo uma soma da forma

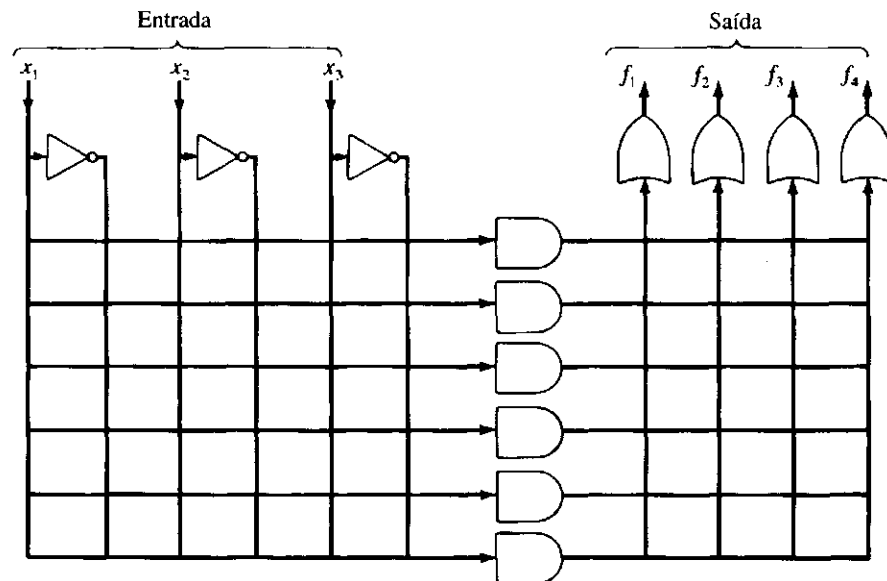
$$\alpha + \beta + \dots + \omega$$

onde $\alpha = x_1$ ou x'_1 , $\beta = x_2$ ou x'_2 e assim por diante. Cada fator é construído como tendo um valor 0 para os valores de entrada referente às linhas da tabela-verdade que tiverem valor 0. Portanto, a expressão toda tem valor 0 para essas entradas e nenhuma outra. Encontre a forma canônica de produto de somas referente à função-verdade dos Exercícios 7 a 10.

15. A figura abaixo mostra um PLA não-programado de três entradas, x_1, x_2 e x_3 . Programe este PLA a fim de gerar as funções-verdade f_1 , e f_3 representadas por

$$f_1: x_1x_2x_3 + x'_1x_2x'_3 + x'_1x'_2x_3$$

$$f_3: x_1x'_2x'_3 + x'_1x'_2x_3 + x'_1x'_2x'_3$$



Exercício 15

16. O **complemento a dois** de um número binário p de n bits é um número binário q de n bits tal que $p + q$ equivale à representação com n bits de zero (um eventual "vai-um" da coluna $n + 1$ é ignorado). Portanto 01110 é o complemento a dois de 10010 porque

$$\begin{array}{r}
 10010 \\
 + 01110 \\
 \hline
 (1)00000
 \end{array}$$

A idéia do complemento a dois pode ser usada para representar inteiros negativos de forma binária. Dado um número binário p , o seu complemento a dois é encontrado varrendo-se p do bit de menor ordem para o de maior ordem (da direita para a esquerda). Enquanto o bit i de p for 0, o dígito i de q também será 0. Quando encontramos o primeiro 1 de p , digamos no bit j , então o bit j de q será 1; mas para os demais bits, $j < i \leq n$, $q_i = p'_i$. Para $p = 10010$, por exemplo, o bit 0 mais à esquerda de p dá origem ao bit 0 em q e o primeiro bit 1 de p dá origem ao bit 1 de q . Os demais bits de q , no entanto, são o inverso dos bits em p (veja a figura a seguir).

$$\begin{array}{r}
 \text{Primeiro 1} \\
 p = 1 \ 0 \ 0 \ 1 \ 0 \\
 q = 0 \ 1 \ 1 \ 1 \ 0 \\
 q_i = p'_i \quad q_j = p_j
 \end{array}$$

Exercício 16

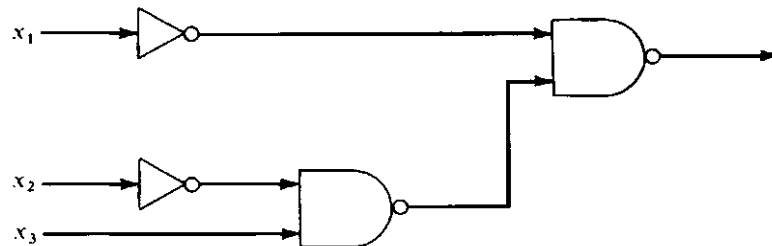
Para cada número binário p dado, encontre q , seu complemento a dois, e calcule $p + q$.

- 1100
- 1001
- 001

- Para qualquer bit x , em um número binário p , seja r_i o bit correspondente em q , o complemento a dois de p (veja o Exercício 16). O valor de r_i depende do valor de x_i e também da posição de x , relativa ao primeiro bit 1 em p . Para o i -ésimo bit, seja c_{i-1} valendo 0 se os bits p_j , $1 \leq j \leq i-1$ são 0 ou valendo 1 em caso contrário. Precisamos computadorizar um valor c_i para passarmos ao próximo bit.
 - Forneça uma função-verdade para r_i com entradas x_i e c_{i-1} . Forneça uma função-verdade para c_i com entradas x_i e c_{i-1} .
 - Escreva expressões booleanas para as funções-verdade do item (a). Simplifique-as tanto quanto possível.
 - Projete um módulo de circuito para obter r_i e c_i a partir das entradas x_i e c_{i-1} .
 - Usando os módulos do item (c), projete um circuito para encontrar o complemento a dois de um número binário de 3 bits zyx . Acompanhe a operação do circuito para o cálculo do complemento a dois de 110.
- Construa uma rede referente à expressão dada, usando apenas componentes NE. Substitua as portas E e OU e os inversores pelas redes compostas de NE apropriadas.

$$x'_3 x_1 + x'_2 x_1 + x'_3$$

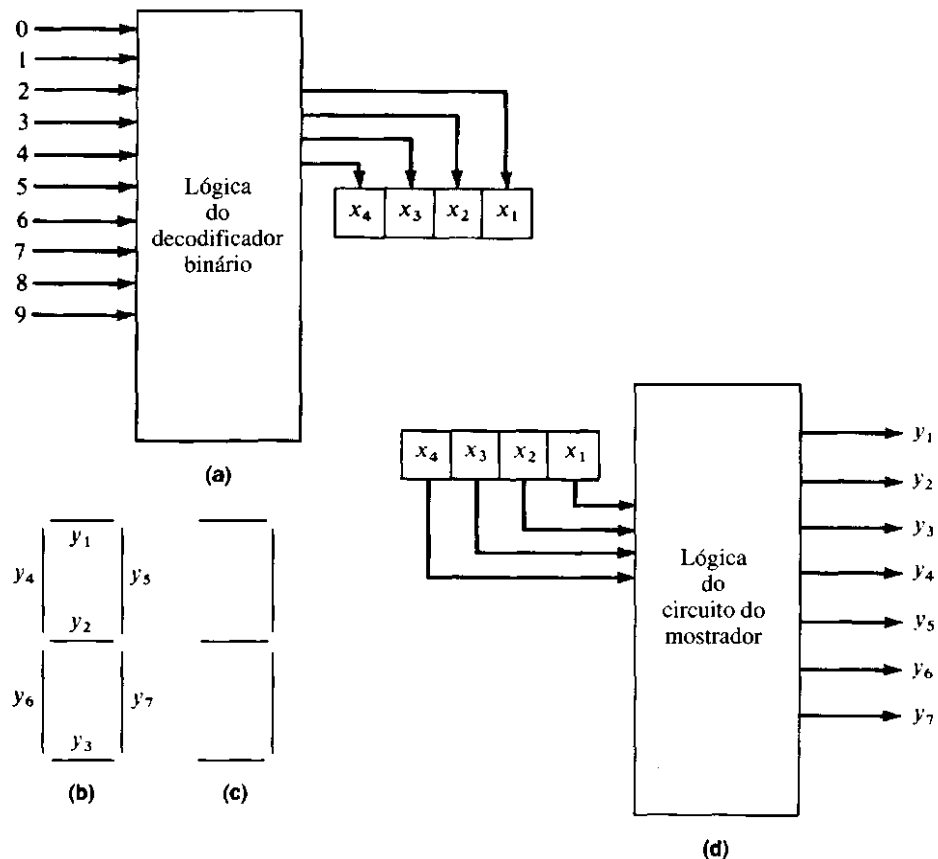
- Use as propriedades de álgebras booleanas para reduzir a expressão do item (a) a uma cuja rede demande apenas três portas NE. Desenhe a rede.
- ★ 19. Substitua a rede da figura abaixo por uma rede equivalente usando apenas uma porta E, uma porta OU e um inversor.



Exercício 19

- Usando apenas portas NOU, construa redes que possam substituir (a) um inversor, (b) uma porta OU e (c) uma porta E.

- ★21. Explique por que os Exercícios 23 e 24 da Seção 1.1 demonstram que as portas NE e NOU são, cada uma, suficientes para realizar qualquer operação lógica.
22. Encontre uma rede equivalente para o meio-somador que use exatamente cinco portas NE.
- ★23. Você acaba de ser contratado pela Mercenary Motors. Seu trabalho é projetar uma rede lógica que determine que um carro só possa ser ligado se sua transmissão automática estiver em ponto-morto ou em posição de estacionamento, e se o cinto de segurança do motorista estiver sendo usado. Encontre uma função-verdade, uma expressão booleana e uma rede lógica. (Há uma condição "não-importa" na função-verdade, uma vez que o carro não pode estar ao mesmo tempo em ponto-morto e em posição de estacionamento.)
24. A Mercenary Motors está ampliando seus negócios para a área de calculadoras. Você precisa projetar o circuito do mostrador de uma nova calculadora. Este projeto envolve duas etapas.
- Qualquer dígito 0, 1,..., 9 entrado na calculadora é logo convertido para binário. A parte (a) da figura a seguir ilustra esta conversão, que envolve quatro redes separadas, uma para cada x_1 a x_4 . Cada rede tem 10 entradas, mas apenas uma entrada pode estar ativa em um dado momento. Escreva uma expressão booleana e então desenhe a rede para x_2 .
 - A representação binária do dígito é então convertida para a forma de exibição através de mostradores de sete segmentos, como mostrado na parte (b) da figura. Para exibir o número 3, por exemplo, y_1, y_2, y_3, y_5 e y_7 , precisam estar ligados, como na parte (c) da figura. Portanto, o segundo passo de sua tarefa pode ser representado pela parte (d), que envolve sete redes separadas, uma para cada y_1 a y_7 , cada qual com quatro entradas x_1 a x_4 . Escreva uma função-verdade, uma expressão booleana e uma rede para y_5 e para y_6 .



Exercício 24

Seção 7.3 Minimização

O Processo de Minimização

Vimos, na última seção, que uma dada função-verdade é associada a uma classe de equivalência de expressões booleanas. Se desejarmos projetar uma rede lógica para a função, o *ideal* é que tenhamos um procedimento que obtenha a expressão booleana mais simples da classe de equivalência. O que chamamos de simples depende da tecnologia que estamos empregando na construção da rede, que tipos de componentes lógicos serão usados etc. De qualquer forma, provavelmente desejaremos minimizar o número total de conexões que precisam ser feitas e o número total de componentes usados. (Ao discutirmos procedimentos de minimização, lembre-se que outros fatores podem influenciar o custo do circuito. Se uma rede for construída apenas uma vez, o tempo gasto na minimização será mais caro do que a construção da rede. Mas se a rede for construída em grande escala, o custo da minimização se justifica.)

Já tivemos alguma experiência na simplificação de expressões booleanas através do uso das propriedades da álgebra booleana. No entanto, não temos procedimentos para usar: apenas temos que adivinhar, atacando o problema individualmente. O que desejamos agora é um procedimento mecânico que possamos usar sem a necessidade de sermos engenhosos. Infelizmente, não desenvolveremos o procedimento ideal. Já sabemos, porém, como obter a forma canônica de soma de produtos da classe de equivalências de expressões para uma certa função-verdade. Nesta seção, discutiremos dois procedimentos para redução de uma forma canônica de soma de produtos a uma forma mínima de soma de produtos. Portanto, podemos minimizar dentro do contexto de forma de soma de produtos, e reduzir, se não minimizar completamente, o número de componentes e conexões necessários.

EXEMPLO 17 A expressão booleana

$$x_1x_2x_3 + x_1'x_2x_3 + x_1'x_2x_3'$$

está na forma de soma de produtos. Uma soma de produtos mínima equivalente é

$$x_2x_3 + x_2x_1'$$

A implementação de uma rede para esta forma requer duas portas E, uma porta OU e um inversor. Usando as leis distributivas da álgebra booleana, esta expressão se reduz a

$$x_2(x_3 + x_1')$$

que requer apenas uma porta E, uma porta OU e um inversor, mas não está mais na forma de soma de produtos. Portanto, uma forma mínima de soma de produtos pode não ser mínima em sentido absoluto. •

Existem duas equivalências extremamente úteis na minimização de uma expressão na forma de soma de produtos. Elas são

$$x_1x_2 + x_1'x_2 = x_2$$

e

$$x_1 + x_1'x_2 = x_1 + x_2$$

PRÁTICA 15 Use as propriedades da álgebra booleana para reduzir as seguintes expressões:

- a. $x_1x_2 + x_1'x_2$ reduzir a x_2
- b. $x_1 + x_1'x_2$ reduzir a $x_1 + x_2$

A equivalência $x_1x_2 + x_1'x_2 = x_2$ significa, por exemplo, que a expressão $x_1'x_2x_3x_4 + x_1x_2'x_3x_4$ se reduz a $x_1'x_2'x_3x_4$. Portanto, quando temos uma soma de dois produtos que diferem apenas por um fator, podemos eliminar este fator. No entanto, a forma canônica de soma de produtos de função-verdade de quatro variáveis, por exemplo, pode ser muito grande, dificultando a localização dos produtos que diferem apenas por um fator. Para nos ajudar nessa busca, podemos usar o *mapa de Karnaugh*, que é uma representação visual da função-verdade que permite que identifiquemos rapidamente as parcelas de uma forma canônica de soma de produtos que só diferem por um fator.

	x_1	x_1'
x_2		
x_2'		

Figura 7.20

O Mapa de Karnaugh

Na forma canônica de soma de produtos de uma função-verdade, estamos interessados nos valores de entrada que produzem saídas 1. O mapa de Karnaugh guarda os 1s da função em uma matriz de tal forma que os produtos referentes a entradas que diferem apenas por um fator fiquem adjacentes. A matriz de uma função de duas variáveis é mostrada na Fig. 7.20. Perceba que a casela correspondente a x_1x_2 , a casela superior esquerda, é adjacente às caselas referentes a $x_1'x_2$ e x_1x_2' , que diferem de x_1x_2 apenas por um fator; no entanto, não é adjacente à casela de $x_1'x_2'$, que difere de x_1x_2 por dois fatores.

EXEMPLO 18

A função-verdade da Tabela 7.9 é representada no mapa de Karnaugh da Fig. 7.21. Em uma rápida olhada, podemos identificar os 1s nas caselas adjacentes, de forma que existem duas parcelas na forma canônica de soma de produtos que diferem por uma única variável; também podemos ver no mapa que a variável que muda é x_1 . Ela pode ser eliminada. Concluimos que a expressão pode ser representada por x_2 . Na verdade, a forma canônica de soma de produtos da função é $x_1x_2 + x_1'x_2$, que, pela nossa regra básica de redução, se reduz a x_2 . No entanto, não foi necessário escrever a forma canônica de soma de produtos — apenas olhamos o mapa.

x_1	x_2	$f(x_1, x_2)$
1	1	1
1	0	0
0	1	1
0	0	0

Tabela 7.9

	x_1	x_1'
x_2	1	1
x_2'		

Figura 7.21

PRÁTICA 16 Desenhe o mapa de Karnaugh e use-o para encontrar uma expressão reduzida para a função da Tabela 7.10.

x_1	x_2	$f(x_1, x_2)$
1	1	0
1	0	0
0	1	1
0	0	1

Tabela 7.10

Mapas para Três e Quatro Variáveis

As formas das matrizes para funções de três e quatro variáveis são mostradas na Fig. 7.22. Nessas matrizes, os quadrados adjacentes também diferem apenas por uma variável. No entanto, na Fig. 7.22a, as caselas mais à esquerda e mais à direita também diferem apenas por uma variável, por isso consideramo-las adjacentes. (Elas seriam adjacentes, se a matriz tivesse montada em um cilindro onde as arestas esquerda e direita estivessem coladas.) Na Fig. 7.22b, não só as caselas à esquerda e à direita são adjacentes, como também o são as caselas de cima e de baixo.

	x_1x_2	x_1x_2'	$x_1'x_2$	$x_1'x_2'$
x_3				
x_3'				

(a)

	x_1x_2	x_1x_2'	$x_1'x_2$	$x_1'x_2'$
x_3x_4				
x_3x_4'				
$x_3'x_4$				
$x_3'x_4'$				

(b)

Figura 7.22

Em mapas de três variáveis, quando duas caselas adjacentes contêm 1, podemos eliminar uma variável; quando quatro caselas adjacentes (em uma única linha ou dispostas em um quadrado) contêm 1, podemos eliminar duas variáveis.

EXEMPLO 19 No mapa da Fig. 7.23, os quadrados que se combinam para formar uma redução são mostrados como um bloco. Esses quatro quadrados adjacentes reduzem-se a x_3 (eliminam as variáveis x_1 e x_2). A redução usa nossa regra básica de redução mais de uma vez:

$$\begin{aligned} x_1x_2x_3 + x_1x_2'x_3 + x_1'x_2x_3 + x_1'x_2'x_3 &= x_1x_3(x_2 + x_2') + x_1'x_3(x_2 + x_2') \\ &= x_1x_3 + x_1'x_3 \\ &= x_3(x_1 + x_1') \\ &= x_3 \end{aligned}$$

	x_1x_2	x_1x_2'	$x_1'x_2$	$x_1'x_2'$
x_3	1	1	1	1
x_3'				

Figura 7.23

Em mapas de quatro variáveis, quando duas caselas adjacentes contêm 1, podemos eliminar uma variável; quando quatro caselas adjacentes contêm 1, podemos eliminar duas variáveis; e quando oito caselas adjacentes contêm 1, podemos eliminar três variáveis.

A Fig. 7.24 ilustra alguns exemplos de duas caselas adjacentes marcadas; a Fig. 7.25, alguns exemplos de quatro caselas adjacentes marcadas, e a Fig. 7.26 mostra exemplos de oito.

	x_1x_2	x_1x_2'	$x_1'x_2$	$x_1'x_2'$
x_3		1	1	
x_3'	1			1

	x_1x_2	x_1x_2'	$x_1'x_2$	$x_1'x_2'$
x_3x_4		1		
x_3x_4'				
$x_3'x_4$	1			1
$x_3'x_4'$		1		

Figura 7.24

	x_1x_2	x_1x_2'	$x_1'x_2$	$x_1'x_2'$
x_3	1	1		
x_3'	1	1		

	x_1x_2	x_1x_2'	$x_1'x_2$	$x_1'x_2'$
x_3	1			1
x_3'	1			1

	x_1x_2	x_1x_2'	$x_1'x_2$	$x_1'x_2'$
x_3x_4		1	1	
x_3x_4'				
$x_3'x_4$				
$x_3'x_4'$		1	1	

	x_1x_2	x_1x_2'	$x_1'x_2$	$x_1'x_2'$
x_3x_4	1			1
x_3x_4'				
$x_3'x_4$				
$x_3'x_4'$	1			1

Figura 7.25

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4	1	1	1	1
$x_3x'_4$				
$x'_3x'_4$				
x'_3x_4	1	1	1	1

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4			1	1
$x_3x'_4$			1	1
$x'_3x'_4$			1	1
x'_3x_4			1	1

Figura 7.26

EXEMPLO 20 No mapa da Fig. 7.27, os quatro cantos reduzem-se a x_2x_4 , e o quadrado interior reduz-se a $x'_2x'_4$.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4	1			1
$x_3x'_4$		1	1	
$x'_3x'_4$		1	1	
x'_3x_4	1			1

Figura 7.27

PRÁTICA 17 Encontre as duas parcelas representadas pelo mapa da Fig. 7.28.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4	1	1		
$x_3x'_4$	1	1		
$x'_3x'_4$				1
x'_3x_4				1

Figura 7.28

Usando o Mapa de Karnaugh

Como fazemos para encontrar uma forma mínima de soma de produtos a partir de um mapa de Karnaugh (ou a partir de uma função-verdade ou uma forma canônica de soma de produtos)? Precisamos usar todas as caselas marcadas do mapa, e desejamos incluir todas as caselas marcadas nos maiores grupos de caselas marcadas possíveis, uma vez que isto reduz a expressão. No entanto, não podemos proceder apenas olhando para o maior bloco de caselas marcadas do mapa.

EXEMPLO 21 No mapa de Karnaugh da Fig. 7.29, se só olharmos para o maior bloco de caselas marcadas, usaremos a coluna de 1s e a reduziremos a $x'_1x'_2$. No entanto, também teremos quatro caselas que ainda não foram consideradas. Cada uma delas pode ser combinada em blocos de duas caselas de uma única maneira (veja a Fig. 7.30), e cada um desses blocos deve ser incluído. Mas quando isto é feito, todas as caselas da coluna de 1s foram usadas e o termo $x'_1x'_2$ é, portanto, redundante. A forma mínima de soma de produtos para este mapa é

$$x'_2x_3x_4 + x'_1x_3x'_4 + x'_2x'_3x'_4 + x'_1x'_3x_4$$

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4		1	1	
$x_3x'_4$			1	1
$x'_3x'_4$		1	1	
x'_3x_4			1	1

Figura 7.29

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4		1	1	
$x_3x'_4$			1	1
$x'_3x'_4$		1	1	
x'_3x_4			1	1

Figura 7.30

Para evitar a redundância ilustrada no Exemplo 21, analisamos o mapa da seguinte maneira. Primeiro, formamos os termos que incluam as caselas marcadas e que não possam ser combinadas a quaisquer outras caselas. Em seguida, procuramos nas demais caselas que podem ser agrupadas apenas em blocos de duas caselas de uma única maneira. Então, dentre as caselas que permanecerem sem uso — isto é, aquelas que ainda não foram atribuídas a um bloco —, procuramos as que só podem ser combinadas em blocos de quatro caselas de uma única maneira; em seguida procuramos quaisquer caselas ainda não usadas que se encaixam apenas em blocos de oito caselas. Em cada passo, se uma casela ainda não usada puder pertencer a mais de um bloco, não fazemos nada com ela. Finalmente, selecionamos blocos que incluam as caselas ainda não usadas (para as quais existem escolhas para que blocos elas pertencem) da forma mais eficiente. Na verdade, se houver muitos 1s no mapa de forma que sejam possíveis diferentes agrupamentos, este procedimento pode vir a não fornecer uma forma mínima (veja o Exemplo 26).

EXEMPLO 22 A Fig. 7.31a mostra a única casela que não pode ser combinada em um bloco maior. Na Fig. 7.31b, formamos o único bloco de duas caselas para a casela $x_1x'_2x'_3$ e o único bloco de duas caselas para a casela $x'_1x'_2x_3$. Todas as caselas marcadas pertencem a blocos. A expressão mínima da soma de produtos é

$$x_1x_2x_3 + x'_2x'_3 + x'_1x'_2$$

Formalmente, os dois últimos termos são obtidos pela expansão de $x'_1x'_2x'_3$ em $x'_1x'_2x'_3 + x'_1x'_2x'_3$ e depois combinando-o com seus vizinhos.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3	1		1	
x'_3		1	1	

(a)

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3	1		1	
x'_3		1	1	

(b)

Figura 7.31

EXEMPLO 23 A Fig. 7.32a mostra os únicos blocos de duas caselas para as caselas $x'_1x'_2x_3x_4$ e $x_1x'_2x'_3x'_4$. Na Fig. 7.32b, as duas caselas não usadas foram combinadas em um único bloco de quatro caselas. A expressão mínima da soma de produtos é

$$x_1x_3 + x'_2x_3x_4 + x_1x'_2x'_4$$

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4	1	1	1	
$x_3x'_4$	1	1		
$x'_3x'_4$		1		
x'_3x_4				

(a)

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4	1	1	1	
$x_3x'_4$	1	1		
$x'_3x'_4$		1		
x'_3x_4				

(b)

Figura 7.32

EXEMPLO 24 A Fig. 7.33a mostra os únicos blocos de duas caselas. Podemos associar a casela restante a dois blocos de duas caselas; esses blocos são mostrados na Fig. 7.33b. Existem duas formas mínimas de soma de produtos,

$$x_1x'_2x'_4 + x'_1x_2x_3 + x'_2x_3x'_4$$

e

$$x_1x'_2x'_4 + x'_1x_2x_3 + x'_1x_3x'_4$$

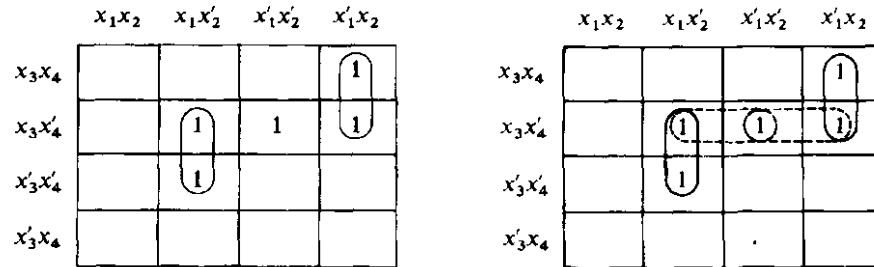


Figura 7.33

EXEMPLO 25

A Fig. 7.34a mostra os únicos blocos de duas caselas e de quatro caselas. As outras duas caselas marcadas podem ser atribuídas a blocos de duas caselas de duas maneiras diferentes, conforme mostrado nas partes (b) e (c). Incluí-las juntas em um único bloco de duas caselas é mais eficiente, uma vez que produz uma soma de produtos com três parcelas, ao invés de quatro. A expressão mínima da soma de produtos é

$$x_1x_3 + x'_1x_2x'_3 + x'_2x'_3x'_4$$

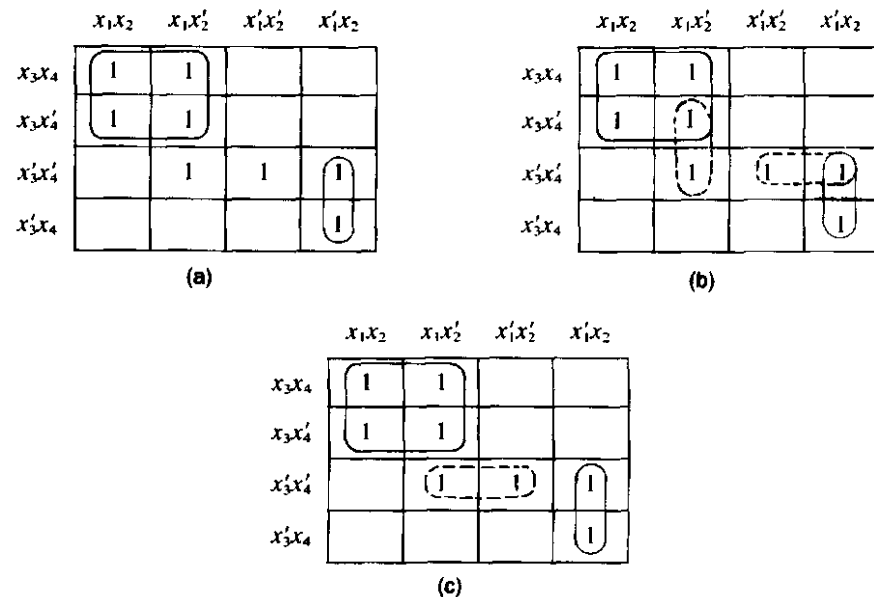


Figura 7.34

EXEMPLO 26 Considere o mapa da Fig. 7.35a. Os únicos dois blocos de quatro caselas determinados pelos quadrados com * foram agrupados. Na Fig. 7.35b, as demais caselas, para as quais existem opções no agrupamento, são combinadas em blocos da maneira mais eficiente possível. A forma de soma de produtos resultante é

$$x_1x_3 + x'_1x'_3 + x_3x_4 + x'_1x_2 + x_1x'_2x'_4$$

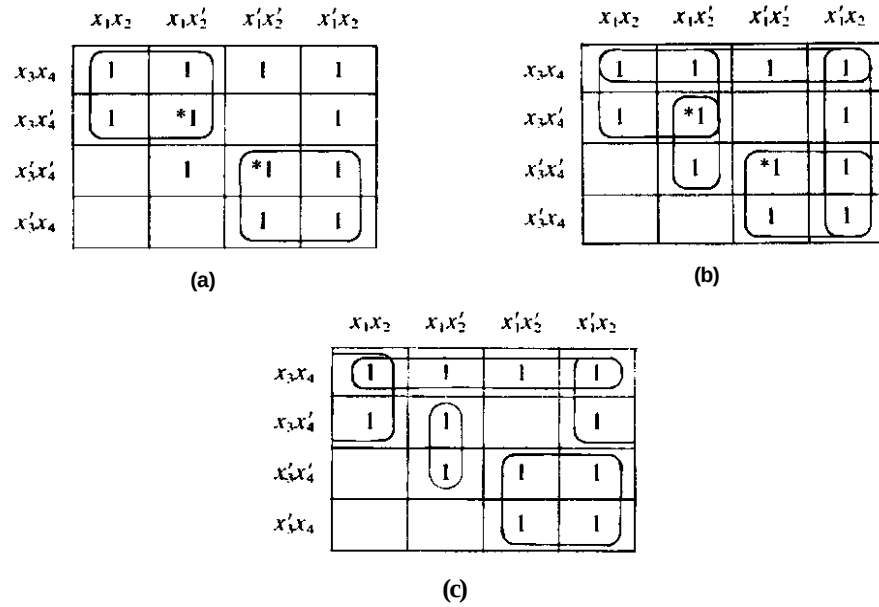


Figura 7.35

Ainda na Fig. 7.35c, a escolha de um bloco de quatro caselas na parte de cima nos leva a uma forma de soma de produtos mais simples,

$$x_2x_3 + x'_1x'_3 + x_3x_4 + x_1x'_2x'_4$$

PRÁTICA 18 Escreva a expressão mínima da soma de produtos para o mapa mostrado na Fig. 7.36.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4		1		
$x_3x'_4$				
$x'_3x'_4$		1	1	
x'_3x_4	1	1	1	

Figura 7.36

Usamos o mapa de Karnaugh para funções de duas, três e quatro variáveis. Se usarmos desenhos tridimensionais ou folhas transparentes sobrepostas, podemos construir mapas de Karnaugh para cinco, seis e até mais variáveis, mas a visualização torna-se muito complicada para compensar o trabalho.

Se o mapa de Karnaugh corresponde a uma função com condições "não-importa", então as caselas com essas condições podem ser consideradas brancas ou como marcadas com 1, de acordo com o que for mais conveniente para o processo de minimização.

O Procedimento de Quine-McCluskey

Lembre-se que o ponto básico para a redução de uma forma canônica de soma de produtos de uma função-verdade é o reconhecimento das parcelas da soma que diferem apenas por um fator. No mapa de Karnaugh, vemos onde essas parcelas ocorrem. Um segundo método para redução, o *procedimento de Quine-McCluskey*, organiza as informações da forma canônica de soma de produtos em uma tabela a fim de simplificar a busca das parcelas que diferem somente por um fator.

O procedimento é um processo de dois passos que se assemelha ao mapa de Karnaugh. Primeiro, encontramos parcelas agrupáveis (como na marcação das caselas no mapa de Karnaugh); em seguida, eliminamos os grupos redundantes e fazemos escolhas para os termos que podem pertencer a diversos grupos.

EXEMPLO 27

Vamos ilustrar o procedimento de Quine-McCluskey pelo uso da função-verdade do Exemplo 21. Não escrevemos a função-verdade lá, mas esta informação pode ser obtida a partir do mapa de Karnaugh. A função-verdade é mostrada na Tabela 7.11. As oito 4-uplas de Os e 1 s que produzem o valor 1 são listadas na Tabela 7.12, que é separada em quatro grupos, de acordo com o número de 1 s. Perceba que os termos da forma canônica de soma de produtos que diferem apenas por um fator estão em grupos adjacentes, o que simplifica a busca por esses termos.

x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	0
1	1	1	0	0
1	1	0	1	0
1	1	0	0	0
1	0	1	1	1
1	0	1	0	0
1	0	0	1	0
1	0	0	0	1
0	1	1	1	0
0	1	1	0	1
0	1	0	1	1
0	1	0	0	0
0	0	1	1	1
0	0	1	0	1
0	0	0	1	1
0	0	0	0	1

Tabela 7.11

Número de 1 s	x_1	x_2	x_3	x_4
Três	1	0	1	1
Dois	0	1	1	0
	0	1	0	1
	0	0	1	1
Um	1	0	0	0
	0	0	1	0
	0	0	0	1
Nenhum	0	0	0	0

Tabela 7.12

Comparamos o primeiro termo, 1011, com cada um dos três termos do segundo grupo, 0110, 0101 e 0011, a fim de encontrar termos que diferem apenas por um fator. Um termo deste tipo é 0011. A combinação 1011 e 0011 reduz-se a $\text{—}011$ quando a variável x_1 , que se alterna, é eliminada. Escrevemos este termo reduzido com um traço na posição x_1 na primeira linha de uma nova tabela (veja a Tabela 7.13b). Também marcamos os dois termos 1011 e 0011 na tabela original (veja a Tabela 7.13a) com um índice 1; este índice 1 é um ponteiro que indica a linha do termo reduzido na Tabela 7.13b que é formada por esses dois elementos (numerar as parcelas equivale a traçar os grupos em um mapa de Karnaugh).

Continuamos este processo com todos os termos. Um termo numerado pode ser usado em outras combinações, da mesma forma que podemos agrupar um termo já agrupado em um mapa de Karnaugh. Quando tivermos terminado, obtemos as Tabelas 7.13, onde os termos na Tabela 7.13b estão novamente agrupados pelo número de 1s.

Número de 1s	x_1	x_2	x_3	x_4	
Três	1	0	1	1	1
Dois	0	1	1	0	2
	0	1	0	1	3
	0	0	1	1	1,4,5
Um	1	0	0	0	6
	0	0	1	0	2,4,7
	0	0	0	1	3,5,8
Nenhum	0	0	0	0	6,7,8

(a)

Número de 1 s	x_1	x_2	x_3	x_4
Dois		0	1	1
Um	0	—	1	0
	0	—	0	1
	0	0	1	—
	0	0	—	1
Nenhum	—	0	0	0
	0	0	—	0
	0	0	0	—

(b)

Tabela 7.13

Repetimos, então, este processo na Tabela 7.13b a fim de obter uma terceira tabela (Tabela 7.14c). Neste ponto, não apenas os grupamentos, mas também os traços nos ajudam a organizar o processo de busca, uma vez que os termos que diferem apenas por uma variável têm traços no mesmo lugar. Novamente, os números cujos termos se combinam na Tabela 7.14b servem como ponteiros para os termos reduzidos da Tabela 7.14c.

Número de 1 s	x_1	x_2	x_3	x_4	
Três	1	0	1	1	1
Dois	0	1	1	0	2
	0	1	0	1	3
	0	0	1	1	1,4,5
Um	1	0	0	0	6
	0	0	1	0	2,4,7
	0	0	0	1	3,5,8
Nenhum	0	0	0	0	6,7,8

(a)

Número de 1 s	x_1	x_2	x_3	x_4	
Dois	—	0	1	1	
Um	0	—	1	0	
	0	—	0	1	
	0	0	1	—	1
	0	0	—	1	1
Nenhum		0	0	0	
	0	0	—	0	1
	0	0	0	—	1

(b)

Número de 1 s	x_1	x_2	x_3	x_4
Nenhum	0	0	—	—

(c)

Tabela 7.14

Quando chegamos às tabelas de redução mostradas na Tabela 7.14, o processo de redução não pode ser mais continuado. Os termos que não foram numerados são irreduzíveis, de forma que representam os grupos de tamanho máximo do mapa de Karnaugh.

No segundo passo do processo, comparamos os termos originais com os termos irreduzíveis. Uma marcação na tabela de comparação (Tabela 7.15) indica que o termo original em uma coluna acaba por levar ao termo irreduzível nesta linha, o que pode ser determinado seguindo-se os ponteiros.

	1011	0110	0101	0011	1000	0010	0001	0000
-011	3			3				
0-10		3				3		
0-01			3				3	
-000					3			3
0 0 - -				3		3	3	3

Tabela 7.15

Se uma coluna na tabela de comparação tem uma marca em apenas uma linha, o termo irreduzível desta linha é o único que cobre o termo original, de forma que ele é um termo essencial e precisa aparecer na forma final de soma de produtos. Portanto, vemos da Tabela 7.15 que os termos —011, 0—10, 0—01 e —000 são essenciais e precisam fazer parte da expressão final. Também percebemos que todas as colunas com marcações na linha 5 também têm marcações em outra linha e, portanto, é coberta por um termo essencial irreduzível da expressão. Portanto, 00-- --é redundante. Como no Exemplo 21, a forma mínima de soma de produtos é

$$x'_2x_3x_4 + x'_1x_3x'_4 + x'_1x'_3x_4 + x'_2x'_3x'_4$$

Em situações onde existem mais de uma forma mínima de soma de produtos, a tabela de comparação terá termos reduzidos não-essenciais e não-redundantes. Neste caso, é preciso fazer uma seleção desses termos reduzidos para abranger todos os termos essenciais não cobertos.

EXEMPLO 28

Usaremos o procedimento de Quine-McCluskey no problema apresentado no Exemplo 24. As tabelas de redução são dadas nas Tabelas 7.16, e a tabela de comparação aparece na Tabela 7.17. Vemos da tabela de comparação que 011 — e 10—0 são termos essenciais reduzidos, e que não há termos redundantes. O único termo original não tratado pelos termos essenciais é 0010, coluna 4, e uma escolha entre a coluna 2 ou coluna 4 trata este termo. Portanto, a forma mínima de soma de produtos é

$$x'_1x_2x_3 + x_1x'_2x'_4 + x'_2x_3x'_4$$

ou

$$x'_1x_2x_3 + x_1x'_2x'_4 + x'_1x_3x'_4$$

Número de 1 s	x_1	x_2	x_3	x_4	
Três	0	1	1	1	¹
Dois	1	0	1	0	^{2,3}
	0	1	1	0	^{1,4}
Nenhum	0	0	1	0	^{2,4}
	1	0	0	0	³

(a)

Número de 1 s	x_1	x_2	x_3	x_4
Dois	0	1	1	—
Um	—	0	1	0
	1	0	—	0
	0	—	1	0

(b)

Tabela 7.16

	0111	1010	0110	0010	1000
0 1 1 -	3		3		
- 0 1 0		3		3	
1 0 - 0		3			3
0 - 1 0			3	3	

Tabela 7.17

PRÁTICA 19 Use o procedimento de Quine-McCluskey para encontrar uma forma mínima de soma de produtos para a função-verdade da Tabela 7.18.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	1
0	0	0	1

Tabela 7.18

O procedimento de Quine-McCluskey aplica-se a funções-verdade com qualquer número de variáveis de entrada, mas, para um número grande de variáveis, o procedimento é demasiado monótono para ser efetuado manualmente. No entanto, ele é bem o tipo de procedimento sistemático que leva a uma solução computadorizada. Ao contrário do mapa de Karnaugh, que se vale da habilidade humana de reconhecer padrões visuais.

Se a função-verdade tiver f poucos valores 0 e um grande número de valores 1, é mais simples implementar o procedimento de Quine-McCluskey para o complemento da função, f' , que terá valores 1 onde f tinha valores 0 e vice-versa. Uma vez que tenhamos obtido uma expressão da mínima soma de produtos para f' , ela pode ser complementada para obter-se uma expressão para f , mas a nova expressão não será uma soma de produtos. (Na verdade, pelas leis de De Morgan, ela terá a forma de um produto de somas equivalente.) Podemos obter a rede para f a partir da rede de soma de produtos para f' , incluindo um inversor ao final.

Toda a questão de minimizar uma rede é simplificar o interior, preservando o comportamento exterior. No Cap. 8, realizaremos o mesmo tipo de simplificação em estruturas de máquinas de estado finito.

Revisão da Seção 7.3

Técnicas

- Minimizar a forma canônica de soma de produtos de uma função-verdade através do uso do mapa de Karnaugh
- Minimizar a forma canônica de soma de produtos de uma função-verdade através do procedimento de Quine-McCluskey

Idéia Principal

Existem algoritmos para redução de uma forma canônica de soma de produtos a uma forma mínima de soma de produtos.

Exercícios 7.3

Nos Exercícios 1 a 5, escreva a forma mínima de soma de produtos dos mapas de Karnaugh das figuras dadas.

★1.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3			1	1
x'_3	1	1		1

2.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3	1			1
x'_3		1		

★3.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3	1	1	1	1
x'_3	1			1

4.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4		1		
$x_3x'_4$		1	1	1
$x'_3x'_4$	1	1	1	
x'_3x_4		1		

5.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4				1
$x_3x'_4$	1	1		
$x'_3x'_4$		1	1	
x'_3x_4			1	

Nos Exercícios 6 e 7, use um mapa de Karnaugh para encontrar a forma mínima de soma de produtos para as funções-verdade dadas.

★6.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	0
0	0	0	0

7.

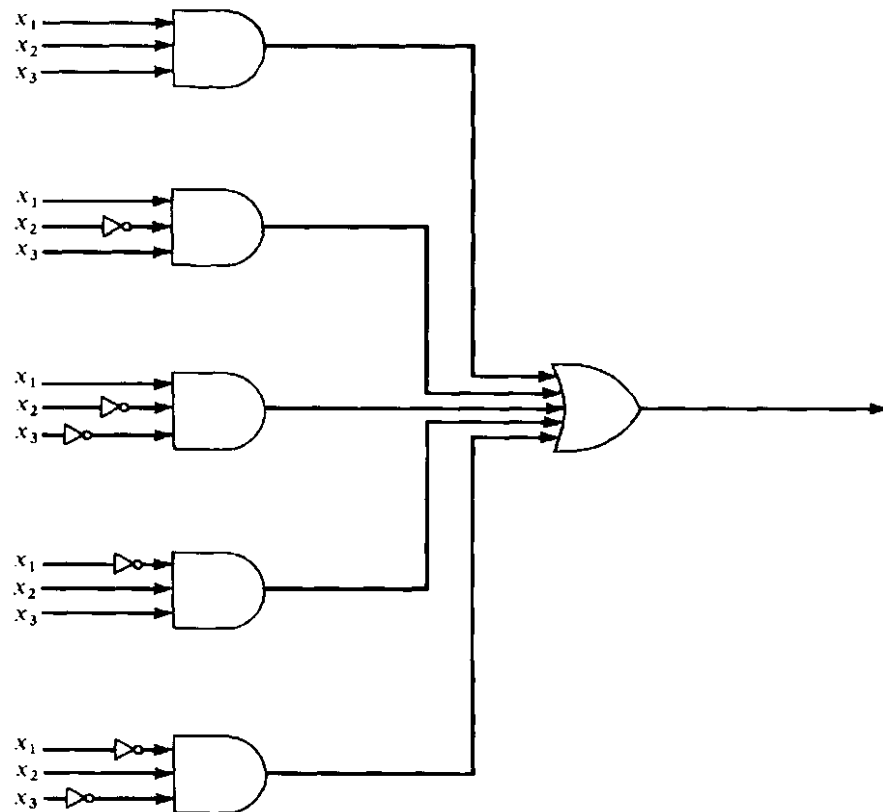
x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
1	1	1	1	1
1	1	1	0	1
1	1	0	1	1
1	1	0	0	1
1	0	1	1	0
1	0	1	0	1
1	0	0	1	0
1	0	0	0	1
0	1	1	1	1
0	1	1	0	1
0	1	0	1	1
0	1	0	0	1
0	0	1	1	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	0

8. Use um mapa de Karnaugh para encontrar a forma mínima de soma de produtos para as seguintes expressões booleanas:

a. $x_1'x_2'x_3x_4 + x_1x_2x_3'x_4 + x_1'x_2'x_3'x_4 + x_1x_2'x_3x_4 + x_1'x_2x_3x_4 + x_1'x_2x_3'x_4 + x_1'x_2'x_3x_4$

b. $x_1'x_2'x_3'x_4 + x_1x_2x_3'x_4 + x_1'x_2'x_3'x_4 + x_1x_2x_3'x_4 + x_1'x_2x_3x_4 + x_1x_2'x_3'x_4$

9. Use um mapa de Karnaugh para encontrar uma expressão mínima da soma de produtos para a rede de três variáveis mostradas na figura a seguir. Desenhe uma nova rede.



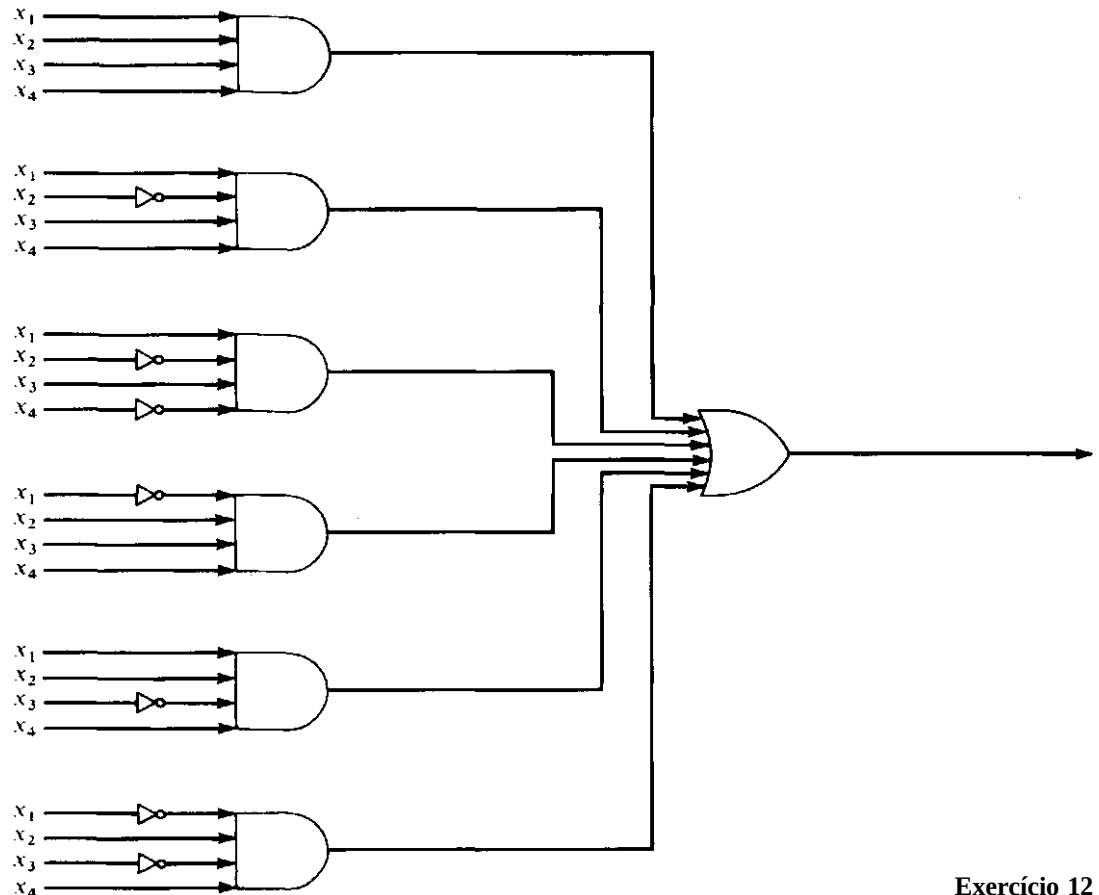
Exercício 9

- ★10. Use um mapa de Karnaugh para encontrar uma forma mínima de soma de produtos para a função-verdade da tabela a seguir. As condições "não-importa" são mostradas como traços.

x_1	x_2	x_3	x_4	$J(x_1, x_2, x_3, x_4)$
1	1	1	1	0
1	1	1	0	1
1	1	0	1	0
1	1	0	0	—
1	0	1	1	0
1	0	1	0	—
1	0	0	1	0
1	0	0	0	0
0	1	1	1	0
0	1	1	0	1
0	1	0	1	0
0	1	0	0	1
0	0	1	1	1
0	0	1	0	0
0	0	0	1	-
0	0	0	0	0

Exercício 10

- ★11. Use o procedimento de Quine-McCluskey para encontrar uma soma de produtos mínima para a função-verdade ilustrada pelo mapa do Exercício 3.
12. Use o procedimento de Quine-McCluskey para encontrar uma soma de produtos mínima para a rede da figura a seguir. Desenhe uma nova rede.



Exercício 12

Nos Exercícios 13 e 14, use o procedimento de Quine-McCluskey para encontrar a soma de produtos mínima para as funções-verdade das tabelas dadas.

13.	x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$	14.	x_1	x_2	x_3	x_4	$f(x_1, x_2, x_3, x_4)$
	1	1	1	1	0		1	1	1	1	1
	1	1	1	0	1		1	1	1	0	0
	1	1	0	1	0		1	1	0	1	1
	1	1	0	0	0		1	1	0	0	0
	1	0	1	1	0		1	0	1	1	1
	1	0	1	0	1		1	0	1	0	0
	1	0	0	1	1		1	0	0	1	1
	1	0	0	0	1		1	0	0	0	1
	0	1	1	1	0		0	1	1	1	1
	0	1	1	0	0		0	1	1	0	0
	0	1	0	1	0		0	1	0	1	1
	0	1	0	0	1		0	1	0	0	1
	0	0	1	1	1		0	0	1	1	1
	0	0	1	0	1		0	0	1	0	0
	0	0	0	1	0		0	0	0	1	1
	0	0	0	0	1		0	0	0	0	1

15. Use o procedimento de Quine-McCluskey para encontrar a soma de produtos mínima das seguintes expressões booleanas:

- ★a. $x_1x_2'x_3x_4' + x_1'x_2'x_3x_4 + x_1'x_2x_3x_4 + x_1'x_2'x_3x_4' + x_1'x_2x_3x_4' + x_1'x_2'x_3x_4$
b. $x_1x_2x_3x_4 + x_1x_2'x_3x_4 + x_1x_2x_3x_4' + x_1x_2'x_3x_4' + x_1'x_2x_3x_4 + x_1'x_2'x_3x_4 + x_1'x_2x_3x_4' + x_1'x_2'x_3x_4' + x_1'x_2x_3x_4 + x_1'x_2'x_3x_4$
c. $x_1x_2x_3x_4 + x_1x_2x_3x_4' + x_1'x_2x_3x_4' + x_1x_2'x_3x_4' + x_1'x_2'x_3x_4' + x_1'x_2x_3x_4' + x_1'x_2'x_3x_4 + x_1'x_2x_3x_4 + x_1x_2x_3x_4$
d. $x_1'x_2x_3x_4x_5' + x_1'x_2x_3x_4'x_5 + x_1x_2x_3x_4x_5 + x_1'x_2'x_3x_4'x_5 + x_1x_2'x_3x_4x_5 + x_1'x_2'x_3x_4'x_5 + x_1x_2x_3x_4'x_5 + x_1'x_2x_3x_4x_5' + x_1'x_2'x_3x_4x_5'$

16. Use o procedimento de Quine-McCluskey para encontrar uma soma de produtos mínima para a função-verdade ilustrada no mapa da Fig. 7.34.

Revisão do Capítulo 7

Terminologia

álgebra booleana	tos (forma normal disjuntiva)	PLA
complemento (de um elemento de álgebra booleana)	função-verdade	porta E
dual (de uma propriedade de álgebra booleana)	instâncias isomorfas de uma estrutura	porta NE
expressão booleana	inversor	porta NOU
expressões booleanas equivalentes	isomorfismo	porta OU
forma canônica de soma de produ-	isomorfismo de álgebras booleanas	propriedade idempotente (de uma álgebra booleana)
	meio-somador	rede combinatória
		somador completo

Autotestes

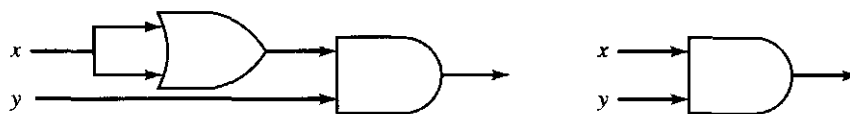
Responda às seguintes perguntas com verdadeiro ou falso.

Seção 7.1

- Em qualquer álgebra booleana, $x + x' = 0$.
- A teoria dos conjuntos é um caso da álgebra booleana na qual $+$ é a união de conjuntos e $\cdot$ é a interseção.
- Em álgebra booleana, $x + (y + x.z) = x + y$.
- O dual da equação da questão anterior é $x.[y.(x + z)] = x.y$.
- Quaisquer duas álgebras booleanas com 16 elementos são isomorfas.

Seção 7.2

6. Pode-se construir uma rede lógica para a expressão booleana $(x + y)'$ usando uma porta E e dois inversores.
7. A forma canônica de soma de produtos de uma função-verdade $f: \{0, 1\}^n \rightarrow \{0, 1\}$ tem n parcelas.
8. Dois números binários de um único bit podem ser somados usando uma rede que consiste em dois meio-somadores.
9. As duas redes lógicas a seguir representam a mesma função-verdade:



10. A maneira mais eficiente de construir uma rede lógica para uma dada função-verdade que use apenas portas NE é construir a rede lógica usando portas E, OU e NÃO, e então substituir cada um desses componentes por seus equivalentes com portas NE.

Seção 7.3

11. Um mapa de Karnaugh é um artifício que ajuda a transformar uma forma canônica de soma de produtos em uma forma reduzida de soma de produtos.
12. Os 1s em um mapa de Karnaugh correspondem aos valores 1 de uma função-verdade.
13. Quando usamos um mapa de Karnaugh para reduzir uma expressão booleana, devemos determinar os maiores blocos possíveis primeiro, porque eles fornecem a melhor redução.
14. No procedimento de Quine-McCluskey, as parcelas que se relacionam precisam ter traços nas mesmas posições.
15. No procedimento de Quine-McCluskey, uma marca em alguma coluna da tabela indica que a parcela para esta coluna é um termo essencial que precisa aparecer na expressão reduzida.

No Computador

1. *Entrada:* n e tabelas definindo duas operações binárias e uma operação unária em um conjunto com n objetos
Saída: Mensagem indicando se a estrutura é ou não uma álgebra booleana
Algoritmo: Testar as 10 propriedades para todos os casos
2. *Entrada:* n , tabelas definindo uma operação binária em cada um dos dois conjuntos de n elementos e uma tabela definindo uma bijeção de um conjunto no outro
Saída: Mensagem indicando se a função é ou não um isomorfismo
Algoritmo: Testar todos os casos possíveis
3. *Entrada:* n e uma tabela representando uma função-verdade com n argumentos
Saída: Expressão canônica de soma de produtos para a função-verdade
4. *Entrada:* n e uma tabela representando uma função-verdade com n argumentos
Saída: Expressão booleana mínima da soma de produtos para a função-verdade
Algoritmo: Use o procedimento de Quine-McCluskey

Modelagem Aritmética, Computação e Linguagens

Objetivos do Capítulo

- Após estudar este capítulo, o leitor estará apto a:
 - Entender como as estruturas algébricas, máquinas de estado finito e máquinas de Turing são modelos para diversas classes de computação, e como as linguagens formais tentam modelar as linguagens naturais
 - Reconhecer certas estruturas bem conhecidas de grupos
 - Demonstrar algumas propriedades acerca dos grupos
 - Entender o que significam grupos isomorfos
 - Acompanhar a operação de uma dada máquina de estado finito para uma cadeia de entrada
 - Construir máquinas de estado finito que reconheçam determinados conjuntos
 - Para uma dada máquina de estado finito, encontrar, se possível, uma máquina equivalente com menos estados
 - Acompanhar a operação de uma máquina de Turing em uma fita de entrada
 - Construir máquinas de Turing para aceitar determinados conjuntos ou realizar certas tarefas computacionais
 - Entender a tese de Church-Turing e o que ela significa para as máquinas de Turing como modelos computacionais
 - Estar atento para a pergunta $P = NP$ com relação à complexidade computacional
 - Dada uma gramática G , construir a derivação das cadeias de $L(G)$
 - Entender a relação entre as diferentes classes de linguagens formais e os diferentes tipos de dispositivos computacionais

Uma estrutura matemática, como vimos no Cap. 7, é um modelo formal que visa a retratar propriedades ou comportamentos encontrados em diferentes contextos. Uma estrutura consiste em um conjunto abstrato de objetos, junto com operações e relações entre esses objetos que obedecem a certas regras. A estrutura de álgebra booleana do Cap. 7 é um modelo das propriedades e comportamentos comuns à lógica proposicional e à teoria dos conjuntos. Enquanto modelo formal é uma entidade abstrata, uma idéia; a lógica proposicional e a teoria de conjuntos são duas ocorrências, ou realizações dessa idéia.

Neste capítulo, estudaremos outras estruturas. Na Seção 8.1, definiremos estruturas algébricas que modelam vários tipos de operações aritméticas, tais como soma de inteiros e multiplicação de números reais

positivos. Essas operações representam uma forma limitada de processamento, mas buscaremos modelos para processamentos mais gerais nas Seções 8.2 e 8.3.

O primeiro modelo deste tipo que veremos, a máquina de estado finito, é um dispositivo útil, mas é, em último caso, muito limitado para modelar o processamento no sentido mais geral. Como modelo que capture a noção de processamento em toda sua abrangência, usaremos as máquinas de Turing. Com o uso de máquinas de Turing como modelo computacional, vemos que algumas tarefas bem definidas não são realizáveis por computadores.

Por último, a Seção 8.4 discute as gramáticas e linguagens formais, que são desenvolvidas como tentativas de modelar linguagens naturais, como o português. Apesar de não resolver este tipo de problema, as gramáticas e linguagens formais servem para modelar diversas construções de linguagens de programação e têm um papel importante na teoria de compiladores.

Seção 8.1 Estruturas Algébricas

Definições e Exemplos

Vamos começar pela análise de uma forma simples da aritmética, a saber, a adição de inteiros. Existe um conjunto de objetos (os inteiros) e uma operação binária nesses objetos (a adição). Lembremos da Seção 3.1 que uma operação binária em um conjunto precisa ser bem definida (fornecer uma única resposta sempre que for aplicada a quaisquer dois elementos do conjunto) e que o conjunto precisa ser fechado em relação à operação (a resposta precisa ser um elemento do conjunto). A notação $[\mathbb{Z}, +]$, denota o conjunto, juntamente com sua operação binária.

Em $[\mathbb{Z}, +]$, uma equação como

$$2 + (3 + 5) = (2 + 3) + 5$$

é verdadeira. Em cada lado da equação os inteiros permanecem na mesma ordem, mas o agrupamento desses inteiros, que indica a ordem na qual as adições são realizadas, muda. A alteração do agrupamento não altera a resposta.

Outro tipo de equação que vale em $[\mathbb{Z}, +]$ é

$$2 + 3 = 3 + 2$$

Mudar a ordem dos inteiros que são somados não altera a resposta.

Equações como

$$\begin{aligned} 2 + 0 &= 2 \\ 0 + 3 &= 3 \\ -125 + 0 &= -125 \end{aligned}$$

também são verdadeiras. Somar zero a qualquer inteiro não altera seu valor.

Finalmente, equações como

$$\begin{aligned} 2 + (-2) &= 0 \\ 5 + (-5) &= 0 \\ -20 + 20 &= 0 \end{aligned}$$

também são verdadeiras; somar o simétrico (o inverso, segundo a adição) de um inteiro a ele próprio, resulta em 0.

Essas equações representam quatro propriedades que ocorrem com tanta frequência que justifica terem nomes.

Definições: Propriedades de Operações Binárias

Seja S um conjunto e seja $\cdot$ uma operação binária em S . (Onde $\cdot$ não denota, necessariamente, a multiplicação, mas apenas qualquer operação binária.)

1. A operação $\cdot$ é dita **associativa** se

$$(\forall x)(\forall y)(\forall z) [x \cdot (y \cdot z) = (x \cdot y) \cdot z]$$

A associatividade nos permite escrever $x \cdot y \cdot z$ sem parênteses, pois o agrupamento dos termos não interfere no resultado.

2. A operação $\cdot$ é dita **comutativa** se

$$(\forall x)(\forall y)(x \cdot y = y \cdot x)$$

3. $[S, \cdot]$ tem um elemento neutro (ou elemento identidade) se

$$(\exists i)(\forall x)(x \cdot i = i \cdot x = x)$$

4. Se $[S, \cdot]$ tem um elemento neutro i , então todo elemento em S tem um **inverso** com relação a $\cdot$ se

$$(\forall x)(\exists x^{-1})(x \cdot x^{-1} = x^{-1} \cdot x = i)$$

Nos enunciados das propriedades acima, os quantificadores universais aplicam-se ao conjunto S ; se a propriedade associativa se verifica, a equação $x \cdot (y \cdot z) = (x \cdot y) \cdot z$ é verdadeira para quaisquer x, y e $z \in S$. Consideração análoga pode ser feita com relação à propriedade comutativa. Os quantificadores existenciais também se aplicam ao conjunto S , de forma que um elemento neutro i , quando existir, deve ser um elemento do conjunto S , assim como um elemento x^{-1} quando existir, deve ser também um elemento de S . Perceba a ordem dos quantificadores: na definição de elemento neutro, o quantificador existencial vem primeiro — deve haver um elemento neutro i que satisfaça a equação $x \cdot i = i \cdot x = x$ para qualquer x em S , da mesma forma que o inteiro 0 em $[\mathbb{Z}, +]$. Na definição do elemento inverso, o quantificador existencial vem depois do quantificador universal — para qualquer x , existe um x^{-1} , e se x for alterado, então x^{-1} pode mudar, tal como o inverso de 2 em $[\mathbb{Z}, +]$ é -2 e o inverso de 5 é -5. Se não houver elemento neutro, então não faz sentido falar de elemento inverso.

Definições: Grupo e Grupo Comutativo

$[S, \cdot]$ é um grupo se S é um conjunto não-vazio e $\cdot$ é uma operação binária em S tal que

1. $\cdot$ é associativa,
2. existe um elemento neutro (em S) e
3. todo elemento de S tem um elemento inverso (em S) em relação a $\cdot$.

Um grupo no qual a operação $\cdot$ é comutativa é chamado de grupo comutativo, ou grupo abeliano.

Mais uma vez o ponto na definição acima é um símbolo que representa uma operação binária genérica. Em qualquer caso específico, a operação binária em particular precisa ser definida. Se a operação é a adição, por exemplo, o símbolo $+$ substitui o símbolo genérico, tal como em $[\mathbb{Z}, +]$. Como uma analogia com a programação, podemos imaginar o símbolo genérico como um parâmetro formal que deve ser substituído por um valor verdadeiro — a operação específica — quando seu valor se torna verdadeiro. Se for claro qual é a operação binária, podemos nos referir ao "grupo 5" ao invés de ao "grupo $[S, \cdot]$ ".

De nossa discussão, deve estar claro que $[\mathbb{Z}, +]$ é um grupo comutativo, com elemento neutro 0. A idéia de grupo não seria útil se não houvessem diversas outras situações nas quais ela se aplicasse.

EXEMPLO 1

Seja $\mathbb{R}^+$ o conjunto dos números reais positivos e seja $\cdot$ a multiplicação de números reais, que é uma operação em $\mathbb{R}^+$. Então $[\mathbb{R}^+, \cdot]$ é um grupo comutativo. A multiplicação é associativa e comutativa. O número real positivo 1 serve como elemento neutro porque

$$x \cdot 1 = 1 \cdot x = x$$

Para qualquer número real positivo x . Todo número real positivo x tem um inverso com respeito à multiplicação, a saber, o número real positivo $1/x$, porque

$$x \cdot \frac{1}{x} = \frac{1}{x} \cdot x = 1$$

•

PRÁTICA 1 O conjunto do Exemplo 1 é limitado aos números reais positivos. Será que $[\mathbb{R}, \cdot]$ é um grupo comutativo? Justifique.

•

EXEMPLO 2 Seja $M_2(\mathbb{Z})$ o conjunto de matrizes 2×2 de elementos inteiros e seja $+$ a adição de matrizes. Então $+$ é uma operação binária em $M_2(\mathbb{Z})$ (perceba que ela é uma operação fechada). Temos um grupo comutativo porque os inteiros são um grupo comutativo, de forma que cada elemento da matriz mantém esta característica. Por exemplo, a soma de matrizes é comutativa porque

$$\begin{aligned} \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} + \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{bmatrix} &= \begin{bmatrix} a_{1,1} + b_{1,1} & a_{1,2} + b_{1,2} \\ a_{2,1} + b_{2,1} & a_{2,2} + b_{2,2} \end{bmatrix} \\ &= \begin{bmatrix} b_{1,1} + a_{1,1} & b_{1,2} + a_{1,2} \\ b_{2,1} + a_{2,1} & b_{2,2} + a_{2,2} \end{bmatrix} \\ &= \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{bmatrix} + \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} \end{aligned}$$

A matriz

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

é o elemento neutro. A matriz

$$\begin{bmatrix} 1 & -4 \\ 2 & 5 \end{bmatrix}$$

é a inversa da matriz

$$\begin{bmatrix} -1 & 4 \\ -2 & -5 \end{bmatrix}$$

•

Uma estrutura chamada **monóide** é obtida se retirarmos a propriedade dos elementos inversos de um grupo; portanto, um monóide tem uma operação associativa e um elemento neutro, mas em um monóide que não seja um grupo, pelo menos um elemento não tem inverso. Um **semigrupo** é obtido se retirarmos a propriedade do elemento neutro e dos elementos inversos de um grupo; portanto, um subgrupo tem uma operação associativa mas, se ele não for um monóide, não há elemento neutro. Diversas formas familiares da aritmética são exemplos de semigrupos, monóides e grupos.

EXEMPLO 3 Considere $[M_2(\mathbb{Z}), \cdot]$ onde $\cdot$ denota a multiplicação de matrizes. Esta estrutura é fechada. Podemos mostrar (Exercício 4) que a multiplicação de matrizes é associativa. A matriz

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

serve como elemento neutro porque

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Portanto, $[M_2(\mathbb{Z}), \cdot]$ é pelo menos um monóide.

•

PRÁTICA 2 Mostre que $[M_2(\mathbb{Z}), \cdot]$ não é um monóide comutativo. •

PRÁTICA 3 Mostre que $[M_2(\mathbb{Z}), \cdot]$ não é um grupo. •

Apesar dos requisitos para que uma estrutura seja um semigrupo serem relativamente modestos, nem toda estrutura aritmética qualifica-se como tal.

PRÁTICA 4 Mostre que $[\mathbb{Z}, -]$ não é um semigrupo, onde $-$ denota a subtração inteira. •

PRÁTICA 5 Seja S o conjunto de números racionais não inteiros, e denotemos por $\cdot$ a multiplicação. $[S, \cdot]$ é um semigrupo? • •

EXEMPLO 4 Cada uma das estruturas a seguir é exemplo de um semigrupo comutativo. Verifique o fechamento, a associatividade e a comutatividade de cada um:

$$[\mathbb{N}, +], [\mathbb{N}, \cdot], [\mathbb{Q}, \cdot], [\mathbb{R}^+, +], [\mathbb{R}, +]$$

EXEMPLO 5 Em qualquer álgebra booleana $[B, +, \cdot, ', 0, 1]$, $[B, +]$ e $[B, \cdot]$ são semigrupos comutativos. Portanto, para qualquer conjunto S , $[\mathcal{P}(S), \cup]$ e $[\mathcal{P}(S), \cap]$ são semigrupos comutativos. ♦

Como as condições necessárias para se caminhar na direção de semigrupos para monóides e para grupos são cada vez mais rígidas, esperamos que alguns exemplos não se enquadrem, mas os que se enquadrarem devem ter comportamentos mais interessantes.

PRÁTICA 6 Quais dos semigrupos a seguir são monóides? Indique seus elementos neutros.

$$[\mathbb{N}, +], [\mathbb{N}, \cdot], [\mathbb{Q}, \cdot], [\mathbb{R}^+, +], [\mathbb{R}, +], [\mathcal{P}(S), \cup], [\mathcal{P}(S), \cap]$$
 •

PRÁTICA 7 Quais dos monóides da Prática 6 são grupos? •

Mais Exemplos

Veremos agora outros exemplos de semigrupos, monóides e grupos.

EXEMPLO 6 Uma expressão na forma

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_0$$

onde $a_i \in \mathbb{R}$, $i = 0, 1, \dots, n$ e $n \in \mathbb{N}$ é um **polinômio em x com coeficientes reais** (ou um **polinômio em x sobre $\mathbb{R}$**). Para cada i , a_i é o coeficiente de x^i . Se i é o maior inteiro diferente de zero para o qual $a_i \neq 0$, o polinômio tem **grau i** ; se não houver este i , o polinômio tem **grau zero**. As parcelas com coeficientes zero geralmente não são escritas. Portanto, $\pi x^4 - \frac{2}{3} x^2 + 5$ é um polinômio de grau 4, e o polinômio constante 6 tem grau zero. O conjunto de todos os polinômios em x sobre $\mathbb{R}$ é denotado por $\mathbb{R}[x]$.

Definimos as operações binárias $+$ e $\cdot$ em $\mathbb{R}[x]$ como as operações convencionais de soma e produto de polinômios. Para os polinômios $f(x)$ e $g(x)$ elementos de $\mathbb{R}[x]$, os produtos $f(x) \cdot g(x)$ e $g(x) \cdot f(x)$ são iguais, porque os coeficientes são números reais e aplicam-se todas as propriedades de somas e produtos de números reais (propriedades como a comutatividade e associatividade). Analogamente, para $f(x)$, $g(x)$ e $h(x)$ pertencentes a $\mathbb{R}[x]$, $(f(x) \cdot g(x)) \cdot h(x) = f(x) \cdot (g(x) \cdot h(x))$. O polinômio constante 1 é o elemento neutro porque $1 \cdot f(x) = f(x)$ e $f(x) \cdot 1 = f(x)$ para todo $f(x) \in \mathbb{R}[x]$. Portanto $[\mathbb{R}[x], \cdot]$ é um monóide comutativo. Ele não é um grupo porque apenas os polinômios constantes não-nulos têm inversos. Por exemplo, não há polinômio $g(x)$ tal que $x \cdot g(x) = 1$, de forma que o polinômio x não tem inverso. No entanto, $[\mathbb{R}[x], +]$ é um grupo comutativo. •

PRÁTICA 8 a. Para $f(x), g(x), h(x) \in \mathbb{R}[x]$, escreva as equações que mostram que $\mathbb{R}[x]$ é comutativo e associativo sob a operação $+$.
b. Qual é o elemento neutro de $[\mathbb{R}[x], +]$?
c. Qual é o inverso de $7x^4 - 2x^3 + 4$ em $[\mathbb{R}[x], +]$? ♦

Os polinômios têm um papel importante na história da teoria dos grupos (o estudo de grupos) porque muitas pesquisas em teoria dos grupos foram inspiradas pelo problema prático de resolver equações na forma $f(x) = 0, f(x) \in \mathbb{R}[x]$. A forma quadrática oferece um algoritmo para encontrar as soluções para qualquer $f(x)$ de grau 2 usando apenas operações algébricas de soma, subtração, multiplicação, divisão e raiz quadrada. Existem algoritmos deste tipo para polinômios de graus 3 e 4. Um dos pontos altos da álgebra abstrata é a prova de que não existe algoritmo que use apenas essas operações para qualquer polinômio de grau 5. (Perceba que esta afirmação é bem mais forte do que apenas dizer que ainda não foi encontrado um algoritmo para esta finalidade: ela indica que não devemos procurá-lo.)

O exemplo a seguir usa a *aritmética modular*. Lembremo-nos da Seção 4.1 (Exemplo 14 e a discussão que o segue) que todo computador tem um limite no tamanho dos inteiros que pode representar. Apesar de desejarmos que um computador seja capaz de simular o comportamento de $[\mathbb{Z}, +]$, o melhor que podemos obter é uma aproximação finita. Esta aproximação é conseguida realizando-se adição módulo n . A "resposta" do cálculo de $x + y$ para $x, y \in \mathbb{Z}$ pode ser o valor verdadeiro de $x + y$ se este valor está dentro do limite que pode ser armazenado ou então um resto obtido pela realização de operações aritméticas modulares, que é equivalente a $x + y$ sob a relação de equivalência da congruência módulo n .

EXEMPLO 7 Seja $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$ e defina a *adição módulo 5*, denotada por $+_5$ em $\mathbb{Z}_5$ por $x +_5 y = r$, onde r é o resto da divisão de $x + y$ por 5. Por exemplo, $1 +_5 2 = 3$ e $3 +_5 4 = 2$. A *multiplicação módulo 5* é definida por $x \cdot_5 y = r$, onde r é o resto da divisão de $x \cdot y$ por 5. Desta forma, $2 \cdot_5 3 = 1$ e $3 \cdot_5 4 = 2$. Então $[\mathbb{Z}_5, +_5]$ é um grupo comutativo e $[\mathbb{Z}_5, \cdot_5]$ é um monóide comutativo. ♦

PRÁTICA 9 a. Complete as tabelas a seguir a fim de definir $+_5$ e $\cdot_5$ em $\mathbb{Z}_5$:

$+_5$	0	1	2	3	4
0					
1			3		
2					
3					2
4					

$\cdot_5$	0	2	3	4
0				
1				
2			1	
3				2
4				

- b. Qual é o elemento neutro de $[\mathbb{Z}_5, +_5]$? E em $[\mathbb{Z}_5, \cdot_5]$?
 c. Qual é o inverso de 2 em $[\mathbb{Z}_5, +_5]$?
 d. Quais elementos em $[\mathbb{Z}_5, \cdot_5]$ têm inversos? ♦

Assim como fizemos em $\mathbb{Z}_5$, podemos definir operações de **adição módulo n** e **multiplicação módulo n** no conjunto $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$ onde n é qualquer inteiro positivo. Novamente, $[\mathbb{Z}_n, +_n]$ será um grupo comutativo e $[\mathbb{Z}_n, \cdot_n]$ será um monóide comutativo.

PRÁTICA 10 a. Forneça a tabela para $\cdot_6$ em $\mathbb{Z}_6$.
 b. Quais elementos em $[\mathbb{Z}_6, \cdot_6]$ têm inversos? ♦

Perceba que ao definirmos uma operação em um conjunto finito através de uma tabela é fácil verificar se esta operação obedece à propriedade comutativa, verificando se a tabela apresenta simetria em relação a sua diagonal principal. Também é fácil identificar o elemento neutro, pois sua linha deve ser igual à parte superior da tabela, e sua coluna deve ser igual à margem esquerda da tabela. É fácil encontrar o inverso de um elemento. Basta procurar em sua linha por um elemento neutro e, em seguida, verificar se ao mudarmos a ordem dos elementos também obtemos o elemento neutro. No entanto, a associatividade não pode ser verificada imediatamente na tabela.

Os dois exemplos a seguir nos fornecem estruturas algébricas onde os elementos são funções.

EXEMPLO 8 Seja A um conjunto e consideremos o conjunto S de todas as *funções f tais* que: $A \rightarrow A$. A operação binária é a composição de funções, denotada por $\circ$. Perceba que S é fechada sob $\circ$ e que a composição de funções é associativa (veja a Prática 11 abaixo). Portanto, $[S, \circ]$ é um semigrupo, chamado de **semigrupo de transformações em A** . Na verdade $[S, \circ]$ é um monóide pois a função identidade i_A , que leva todo elemento de A nele mesmo, é o elemento neutro pois tem a propriedade de que, para qualquer $f \in S$,

$$f \circ i_A = i_A \circ f = f$$

PRÁTICA 11 Demonstre que a composição de funções no conjunto S definida acima é associativa. •

EXEMPLO 9 Seja, novamente, A um conjunto e considere o conjunto S_A de todas as bijeções/tais que $f: A \rightarrow A$ (permutações de A). A bijetividade é preservada pela composição de funções, a composição de funções é associativa, a função identidade i_A é uma permutação e para $f \in S_A$, a função inversa f^{-1} existe e também é uma permutação. Além disso,

$$f \circ f^{-1} = f^{-1} \circ f = i_A$$

Portanto, $[S_A, \circ]$ é um grupo, chamado de **grupo das permutações em A** .

Se $A = \{1, 2, \dots, n\}$ para algum inteiro positivo n , então S_A é chamado **grupo simétrico de grau n** e denotado por S_n . Portanto S_3 , por exemplo, é o conjunto de todas as permutações em $\{1, 2, 3\}$. Existem seis dessas permutações, que são mostradas a seguir (usando a notação cíclica da Seção 4.3):

$$\begin{array}{lll} \alpha_1 = i & \alpha_2 = (1, 2) & \alpha_3 = (1, 3) \\ \alpha_4 = (2, 3) & \alpha_5 = (1, 2, 3) & \alpha_6 = (1, 3, 2) \end{array}$$

Lembre-se de que a notação $(1, 2)$, por exemplo, indica que 1 é levado em 2 e o 2 é levado em 1, e os elementos não mencionados são levados neles mesmos. A composição $(1, 2) \circ (1, 3)$ é realizada da direita para a esquerda, de forma que

$$\begin{array}{l} \text{Por } (1, 3) \text{ Por } (1, 2) \\ 1 \rightarrow 3 \rightarrow 3 \\ 2 \rightarrow 2 \rightarrow 1 \\ 3 \rightarrow 1 \rightarrow 2 \end{array}$$

resultando em $(1, 3, 2)$. Portanto, $\alpha_2 \circ \alpha_3 = (1, 2) \circ (1, 3) = (1, 3, 2) = \alpha_6$.

PRÁTICA 12 a. Complete a tabela do grupo $[S_3, \circ]$.

$\circ$	α_1	α_2	α_3	α_4	α_5	α_6
α_1						
α_2						
α_3						
α_4						
α_5						
α_6						

b. $[S_3, \circ]$ é um grupo comutativo? •

$[S_3, \circ]$ é nosso primeiro exemplo de um grupo não-comutativo ($[M_2(\mathbb{Z}), \cdot]$ era um monóide não-comutativo).

O próximo exemplo é bem simples, mas tem interesse particular porque aparece em diversas áreas da ciência da computação, incluindo a teoria das linguagens formais e de autômatos.

EXEMPLO 10 Seja A um conjunto finito; seus elementos são chamados **símbolos** e A propriamente dito é chamado de **alfabeto**. A^* denota o conjunto de todas as **cadeias**, **strings** ou **palavras** de tamanho finito sobre A . A^* pode ser definido recursivamente (como no Exemplo 23 do Cap. 2), onde $\cdot$ denota a **concatenação** (justaposição) de cadeias:

1. A **cadeia vazia** (a cadeia composta de nenhum símbolo) pertence a A^* .
2. Qualquer elemento simples de A pertence a A^* .
3. Se x e y são cadeias pertencentes a A^* , então $x \cdot y$ também o é.

Portanto, se $A = \{a, b\}$, então $abbaa$, $bbbbba$ e a são cadeias sobre A e $abbaa \cdot a$ fornece a cadeia $abbaaa$. Da definição recursiva, qualquer cadeia sobre A contém apenas um número finito de símbolos. O número de símbolos em uma cadeia é chamado de seu **comprimento**. A cadeia vazia Λ é a única cadeia de comprimento zero.

A cadeia vazia λ não deve ser confundida com o conjunto vazio $\emptyset$, mesmo que A seja $\emptyset$, caso no qual $A^* = \{\lambda\}$. Se A não é vazio, então, qualquer que seja o tamanho de A , A^* é um conjunto denumerável (enumerável e infinito). Se A contém apenas um elemento, digamos $A = \{a\}$, então $\lambda, a, aa, aaa, \dots$ é uma enumeração de A^* . Se A contém mais de um elemento, então podemos impor uma ordenação lexicográfica (alfabética) em seus elementos. Obtemos, então, uma enumeração de A^* considerando primeiro a cadeia vazia, em seguida todas as cadeias de tamanho 1 (existe um número finito delas), ordenadas alfabeticamente, depois todas as cadeias de tamanho 2 (também há um número finito delas), e assim por diante. Perceba ainda que se A não é vazio, podemos encontrar cadeias de tamanhos arbitrários em A^* .

A concatenação é uma operação binária em A^* e é associativa. A cadeia vazia λ é um elemento neutro porque para qualquer cadeia $x \in A$,

$$x \cdot \lambda = \lambda \cdot x = x$$

Portanto, $[A^*, \cdot]$ é um monóide, chamado de **monóide livre gerado por A**. •

- PRÁTICA 13** Para $A = \{a, b\}$, responda
- $[A^*, \cdot]$ é um monóide associativo?
 - $[A^*, \cdot]$ é um grupo?

•

Resultados Básicos Sobre Grupos

Agora vamos mostrar alguns teoremas básicos sobre grupos. Existem centenas de teoremas sobre grupos e diversos livros destinados exclusivamente à teoria dos grupos, de forma que seremos bem superficiais. Os resultados que demonstraremos são conseqüências quase imediatas das definições envolvidas.

Pela definição, um grupo $[G, \cdot]$ (ou um monóide) tem um elemento neutro, e temos tentado ser cuidadosos ao nos referirmos a *um* elemento neutro ao invés de dizermos *o* elemento neutro. No entanto, é válido dizer *o* elemento neutro porque existe apenas um. Para demonstrar que o elemento neutro é único, suponha que i_1 e i_2 são ambos elementos neutros. Então temos

$$i_1 = i_1 \cdot i_2 = i_2$$

- PRÁTICA 14** Justifique os símbolos de igualdade acima. •

Como $i_1 = i_2$, o elemento neutro é único. Portanto, demonstramos o seguinte teorema.

Teorema da Unicidade do Elemento Neutro de um Grupo

Em qualquer grupo (ou monóide) $[G, \cdot]$, o elemento neutro i é único.

Todo elemento x em um grupo $[G, \cdot]$ tem um elemento inverso x^{-1} . Portanto, G contém diversos elementos inversos diferentes, mas para cada x , o inverso é único.

Teorema da Unicidade dos Inversos em um Grupo

Para cada x em um grupo $[G, \cdot]$, x^{-1} é único.

- PRÁTICA 15** Demonstre o teorema anterior. (*Dica:* Admita dois inversos para x , a saber: y e z , e seja i o elemento neutro. Então $y = y \cdot i = y \cdot (x \cdot z) = \dots$) •

Se x e y pertencem a um grupo $[G, \cdot]$, então $x \cdot y$ pertence a G e deve ter um inverso em G . Naturalmente, esperamos que este inverso tenha alguma relação com x^{-1} e y^{-1} que sabemos existir em G . Podemos mostrar que $(x \cdot y)^{-1} = y^{-1} \cdot x^{-1}$; portanto, o inverso de um produto é o produto dos inversos na ordem contrária.¹

Teorema do Inverso de um Produto

Para x e y elementos de um grupo $[G, \cdot]$, $(x \cdot y)^{-1} = y^{-1} \cdot x^{-1}$

¹Neste ponto, a autora refere-se à operação binária associada ao grupo como produto. Trata-se, na verdade, de uma outra nomenclatura que, a exemplo do uso do símbolo $\cdot$ para denotar qualquer operação binária, não deve sugerir que a operação é, necessariamente, a de multiplicação, mas a operação do grupo. (N. do T.)

Demonstração: Mostraremos que $y^{-1} \cdot x^{-1}$ tem as duas propriedades necessárias para $(x \cdot y)^{-1}$. Então, como os inversos são únicos, $(y^{-1} \cdot x^{-1})$ deve ser igual a $(x \cdot y)^{-1}$.

$$\begin{aligned}(x \cdot y) \cdot (y^{-1} \cdot x^{-1}) &= x \cdot (y \cdot y^{-1}) \cdot x^{-1} \\ &= x \cdot i \cdot x^{-1} \\ &= x \cdot x^{-1} \\ &= i\end{aligned}$$

Analogamente, $(y^{-1} \cdot x^{-1}) \cdot (x \cdot y) = i$. Perceba como a associatividade e o significado de i e dos inversos são usados nesta demonstração.

PRÁTICA 16 Escreva 10 como $7 +_{12} 3$ e use o teorema do inverso de um produto para encontrar $(10)^{-1}$ no grupo $[\mathbb{Z}_{12}, +_{12}]$.

Sabemos que diversos sistemas de números tais como $[\mathbb{Z}, +]$ e $[\mathbb{R}, +]$ são grupos. Usamos propriedades de grupos quando realizamos operações algébricas ou aritméticas nesses sistemas. Em $[\mathbb{Z}, +]$, por exemplo, se tomarmos a equação $x + 5 = y + 5$, concluímos que $x = y$. Estamos usando a lei do cancelamento à direita que, como veremos agora, vale para qualquer grupo.

Definição: Leis de Cancelamento

Um conjunto S com uma operação binária $\cdot$ satisfaz à **lei do cancelamento à direita** se para $x, y, z \in S$, $x \cdot z = y \cdot z$ implica $x = y$. E satisfaz à **lei do cancelamento à esquerda** se $z \cdot x = z \cdot y$ implica $x = y$.

Suponha agora que x, y e z são elementos de um grupo $[G, \cdot]$ e que $x \cdot z = y \cdot z$. Para concluirmos que $x = y$, lançamos mão de z^{-1} . Portanto,

$$x \cdot z = y \cdot z$$

implica

$$\begin{aligned}(x \cdot z) \cdot z^{-1} &= (y \cdot z) \cdot z^{-1} \\ x \cdot (z \cdot z^{-1}) &= y \cdot (z \cdot z^{-1}) \\ x \cdot i &= y \cdot i \\ x &= y\end{aligned}$$

Consequentemente, G satisfaz à lei do cancelamento à esquerda.

PRÁTICA 17 Mostre que qualquer grupo $[G, \cdot]$ satisfaz à lei do cancelamento à esquerda.

Demonstramos o seguinte teorema.

Teorema do Cancelamento em Grupos

Qualquer grupo $[G, \cdot]$ satisfaz às leis do cancelamento à esquerda e à direita.

EXEMPLO 11 Sabemos que $[\mathbb{Z}_6, \cdot_6]$ não é um grupo. A equação

$$4 \cdot_6 2 = 1 \cdot_6 2$$

se verifica mas, naturalmente, $4 \neq 1$.

Mais uma vez, trabalhando com $[\mathbb{Z}, +]$, podemos resolver a equação $x + 6 = 13$ pela adição de -6 a ambos os lados, produzindo uma única resposta $x = 13 + (-6) = 7$. Vale também a propriedade que garante ser possível resolver equações lineares em qualquer grupo. Considere a equação $a \cdot x = b$ no grupo $[G, \cdot]$ onde a e b pertencem a G e x deve ser encontrado. Então $x = a^{-1} \cdot b$ é um elemento de G que satisfaz a equação. Sejam x_1 e x_2 soluções para a equação $ax = b$, então $a \cdot x_1 = a \cdot x_2$ e, pelo cancelamento à esquerda, $x_1 = x_2$. Analogamente, a única solução para $x \cdot a = b$ é $x = b \cdot a^{-1}$.

Teorema da Solução de Equações Lineares em um Grupo

Sejam a e b quaisquer elementos de um grupo $[G, .]$. Então as equações lineares $a . x = b$ e $x . a = b$ têm soluções únicas em G .

PRÁTICA 18 Resolva a equação $x +_8 3 = 1$ em $[\mathbb{Z}_8, +_8]$.

O teorema da solução de equações lineares nos diz algo sobre as tabelas de grupos finitos. Será que ao olharmos a linha a da tabela, o elemento b pode aparecer mais de uma vez? Em caso afirmativo, a tabela nos diz que existem dois elementos distintos x_1 e x_2 do grupo tais que $a . x_1 = b$ e $a . x_2 = b$. Mas, pelo teorema da solução de equações lineares, esta ocorrência dupla não pode acontecer. Portanto, um dado elemento de um grupo finito aparece no máximo uma vez em cada linha da tabela do grupo. No entanto, para completar a linha, cada elemento deve aparecer pelo menos uma vez. Um resultado igual vale para as colunas. Portanto, em uma tabela de grupo, cada elemento aparece exatamente uma vez em cada linha e em cada coluna. Apenas esta propriedade, no entanto, não é suficiente para garantir que uma tabela representa um grupo; a operação precisa ainda ser associativa (veja Exercício 24 ao fim desta seção).

PRÁTICA 19 Admita que $\circ$ é uma operação binária em $\{1, a, b, c, d\}$. Complete a tabela a seguir para que defina um grupo cujo elemento neutro é 1.

$\circ$	1	a	b	c	d
1	1				
a			c	d	1
b		c	d		
c		d		a	
d				b	c

Se $[G, .]$ for um grupo onde G é finito com n elementos, então n é a **ordem do grupo**, denotada por $|G|$. Se G é um conjunto infinito, o grupo é de ordem infinita.

PRÁTICA 20 a. Apresente um grupo comutativo de ordem 18.
b. Apresente um grupo não-comutativo de ordem 6.

Mais propriedades de grupos aparecem nos Exercícios ao fim desta seção.

Subgrupos

Já sabemos o que são os grupos e também já sabemos o que são os subconjuntos, portanto não deve ser difícil imaginar o que são os subgrupos. No entanto, veremos um exemplo antes de passarmos à definição. Sabemos que $[\mathbb{Z}, +]$ é um grupo. Seja agora A qualquer subconjunto não-vazio de $\mathbb{Z}$, para quaisquer x e y em A , $x + y$ também pertencem a $\mathbb{Z}$, de forma que $x + y$ existe e é único. O conjunto A "herda" uma operação bem-definida, $+$, de $[\mathbb{Z}, +]$. A propriedade associativa também é herdada, porque para quaisquer $x, y, z \in A$, também é verdade que x, y e $z \in \mathbb{Z}$ e a equação

$$(x + y) + z = x + (y + z)$$

também vale. (Na Seção 3.1, mencionamos a importância da herança na programação orientada a objeto.) Talvez A sob as operações herdadas tenha toda a estrutura de $[\mathbb{Z}, +]$ e seja, ele próprio, um grupo. Isto depende de A .

Suponha que $A = P$, o conjunto dos números pares. P é fechado com relação à adição, P contém 0 (o elemento neutro) e o inverso de todo inteiro par (seu simétrico) é também um inteiro par. $[P, +]$ é, portanto, um grupo. Mas suponha que $A = I$, o conjunto dos números ímpares. $[I, +]$ não é um grupo por diversos motivos. Por exemplo, não é fechado — se somarmos dois inteiros ímpares, obteremos como resultado um inteiro par. (O fechamento depende do conjunto, bem como da operação, portanto não é uma característica herdada.) Uma outra razão é que um subgrupo deve conter um elemento neutro com respeito à adição; 0 é o único inteiro que presta a este fim, e não é ímpar.

Definição: Subgrupo

Seja $[G, .]$ um grupo e $A \subseteq G$. Então $[A, .]$ é um subgrupo de $[G, .]$ se $[A, .]$ também for um subgrupo.

A fim de ser um grupo, $[A, \cdot]$ deve conter um elemento neutro, o qual denotaremos por i_A . Naturalmente, G também tem um elemento neutro, que denotaremos por i_G . É verdade que $i_A = i_G$, mas esta equação não é consequência da unicidade do elemento neutro de um grupo, porque o elemento i_A , até onde sabemos, poderia não ser um elemento neutro de G e, além disso, não podemos afirmar que i_G é um elemento de A . No entanto, $i_A = i_A \cdot i_A$ porque i_A é o elemento neutro de $[A, \cdot]$ e $i_A = i_A \cdot i_G$ porque i_G é o elemento neutro de $[G, \cdot]$. Decorre da lei do cancelamento à esquerda aplicada no escopo do grupo $[G, \cdot]$ que $i_A = i_G$.

Para verificar se $[A, \cdot]$ é um subgrupo de $[G, \cdot]$, podemos admitir as propriedades herdadas referentes à operação bem-definida e à associatividade e verificarmos as três demais propriedades:

Teorema dos Subgrupos

Para $[G, \cdot]$ um grupo com elemento neutro i e $A \subseteq G$, $[A, \cdot]$ é um subgrupo de $[G, \cdot]$ se satisfizer aos três testes a seguir:

1. A é fechado em relação a $\cdot$.
2. $i \in A$.
3. Todo $x \in A$ tem um elemento inverso em A .

PRÁTICA 21 A definição de um grupo exige que o conjunto não seja vazio. No teorema de subgrupos, porque não há a menção explícita de que $A \neq \emptyset$?

EXEMPLO 12 a. $[\mathbb{Z}, +]$ é um subgrupo do grupo $[\mathbb{R}, +]$.
b. $[\{1, 4\}, \cdot_5]$ é um subgrupo do grupo $[\{1, 2, 3, 4\}, \cdot_5]$ (vale o fechamento; $1 \in \{1, 4\}$, $1^{-1} = 1$, $4^{-1} = 4$). ♦

PRÁTICA 22 a. Mostre que $[\{0, 2, 4, 6\}, +_8]$ é um subgrupo do grupo $[\mathbb{Z}_8, +_8]$.
b. Mostre que $[\{1, 2, 4\}, \cdot_7]$ é um subgrupo do grupo $[\{1, 2, 3, 4, 5, 6\}, \cdot_7]$.

Se $[G, \cdot]$ é um grupo de elemento neutro i , então $[\{i\}, \cdot]$ e $[G, \cdot]$ são subgrupos de $[G, \cdot]$. Estes subgrupos um tanto o quanto triviais de $[G, \cdot]$ são chamados de **subgrupos impróprios**. Quaisquer outros subgrupos de $[G, \cdot]$ são chamados de **subgrupos próprios**.

PRÁTICA 23 Encontre todos os subgrupos próprios de S_3 , o grupo simétrico de grau 3. (Você pode encontrá-los procurando na tabela do grupo; veja a Prática 12.)

Um motivo de confusão com relação à terminologia: O conjunto de *todas* as bijeções de um conjunto A nele mesmo sob a composição de funções (tal como S_3) é chamado de *grupo de permutações em A* , e qualquer subgrupo deste conjunto (tal como o da Prática 23) é chamado de um **grupo de permutação**. A diferença é que o grupo de permutações no conjunto A inclui todas as bijeções de A nele mesmo, mas um grupo de permutações no conjunto A pode não incluir todas elas. Grupos de permutações têm interesse especial não apenas por terem sido os primeiros grupos a serem estudados mas também porque são os únicos grupos se considerarmos as estruturas isomorfas como sendo as mesmas. Veremos este resultado em breve.

Existe um subgrupo interessante que sempre poderemos encontrar no grupo simétrico S_n para $n > 1$. Sabemos que todo elemento de S_n pode ser escrito como uma composição de ciclos, mas também é verdade que todo ciclo pode ser escrito como a composição de ciclos de tamanho 2, chamados de **transposições**. Em S_7 , por exemplo, $(5, 1, 7, 2, 3, 6) = (5, 6) \circ (5, 3) \circ (5, 2) \circ (5, 7) \circ (5, 1)$. Podemos verificar isto computando $(5, 6) \circ (5, 3) \circ (5, 2) \circ (5, 7) \circ (5, 1)$ da direita para a esquerda:

$$1 \rightarrow 5 \rightarrow 7 \rightarrow 7 \rightarrow 7 \rightarrow 7$$

Portanto, 1 é levado em 7. Analogamente,

$$7 \rightarrow 7 \rightarrow 5 \rightarrow 2 \rightarrow 2 \rightarrow 2$$

de forma que 7 é levado em 2, e assim por diante, resultando em $(5, 1, 7, 2, 3, 6)$. Também é verdade que $(5, 1, 7, 2, 3, 6) = (1, 5) \circ (1, 6) \circ (1, 3) \circ (1, 2) \circ (2, 4) \circ (1, 7) \circ (4, 2)$.

Para qualquer $n > 1$, a permutação identidade i em S_n pode ser escrita como $i = (a, b) \circ (a, b)$ para quaisquer dois elementos a e b no conjunto $\{1, 2, \dots, n\}$. Esta equação também mostra que o inverso da transposição (a, b) em S_n é (a, b) . Agora apresentamos mais um resultado (sem demonstrá-lo): apesar de haver várias maneiras de se escrever um ciclo como a composição de transposições, para um dado ciclo, o número de trans-

posições será sempre ou par ou ímpar. Consequentemente, classificamos qualquer permutação em S_n , $n > 1$ como **par** ou **ímpar** de acordo com o número de transposições de qualquer representação desta permutação. Por exemplo, em S_7 , (5, 1, 7, 2, 3, 6) é ímpar. Se denotarmos por A_n o conjunto de todas as permutações pares em S_n , então A_n determina um subgrupo de $[S_n, \circ]$. A composição de permutações pares produz uma permutação par e $i \in A_n$. Se $\alpha \in A_n$ e α , escrito como produto de transposições, é $\alpha = \alpha_1 \circ \alpha_2 \circ \dots \circ \alpha_k$, então $\alpha^{-1} = \alpha^{-1} = \alpha_k^{-1} \circ \alpha_{k-1}^{-1} \circ \dots \circ \alpha_1^{-1}$. O inverso de cada transposição é a própria transposição, portanto α^{-1} também é par.

A ordem do grupo $[S_n, \circ]$ (o número de elementos) é $n!$. Qual é a ordem do subgrupo $[A_n, \circ]$? É razoável pensar que metade das permutações em S_n são pares e a outra metade é de permutações ímpares. De fato, isto ocorre. Se denotarmos por I_n o conjunto de permutações ímpares em S_n (que não é fechado sob a composição de funções), então a aplicação $f: A_n \rightarrow I_n$ definida por $f(\alpha) = \alpha \circ (1, 2)$ é uma bijeção.

PRÁTICA 24 Demonstre que $f: A_n \rightarrow I_n$ definida por $f(\alpha) = \alpha \circ (1, 2)$ é injetiva e sobrejetiva. ♦

Como existe uma bijeção entre A_n e I_n , cada conjunto tem o mesmo número de elementos. Mas $A_n \cap I_n = \emptyset$ e $A_n \cup I_n = S_n$, portanto, $|A_n| = |S_n|/2 = n!/2$.

Teorema dos Grupos Alternantes

Para $n \in \mathbb{N}$, $n > 1$, o conjunto A_n das permutações pares determina um subgrupo chamado de grupo alternante de $[S_n, \circ]$ de ordem $n!/2$.

Vimos até agora diversos exemplos de subgrupos de grupos finitos. No Exemplo 12b e na Prática 22, vimos três desses exemplos e a ordem dos grupos e subgrupos foram:

Grupo de ordem 4, subgrupo de ordem 2
Grupo de ordem 8, subgrupo de ordem 4
Grupo de ordem 6, subgrupo de ordem 3

O teorema dos grupos alternantes diz que um particular grupo de ordem $n!$ tem um subgrupo de ordem $n!/2$.

Baseados nesses exemplos, poderíamos inferir que os subgrupos têm sempre a metade dos tamanhos de seus grupos pares! Isto não é necessariamente verdade, mas existe uma relação entre o tamanho de um grupo e o tamanho de um subgrupo. Esta relação é enunciada no teorema de Lagrange, demonstrado pelo grande matemático francês Joseph-Louis Lagrange em 1771 (não apresentaremos a demonstração deste teorema):

Teorema de Lagrange

A ordem de um subgrupo de um grupo finito divide a ordem do grupo.

O teorema de Lagrange nos ajuda a limitar as possibilidades de subgrupos para um grupo finito. Se $|G| = 12$, por exemplo, não precisamos procurar quaisquer subgrupos de ordem 7, uma vez que 7 não é divisor de 12. Além disso, o fato de que 6 divide 12 não implica a existência de subgrupos de G com ordem 6. Na verdade, A_4 é um grupo de ordem $4!/2 = 12$, mas pode ser mostrado que A_4 não possui subgrupos de ordem 6. A recíproca do teorema de Lagrange não é necessariamente verdadeira. Em certos casos, esta recíproca verifica-se — por exemplo, em grupos comutativos finitos (perceba que A_4 não é comutativo).

Finalmente, consideremos subgrupos do grupo $[\mathbb{Z}, +]$. Para qualquer elemento n fixo de $\mathbb{N}$, o conjunto $n\mathbb{Z}$ é definido como o conjunto de todos os múltiplos inteiros de n ; $n\mathbb{Z} = \{nz \mid z \in \mathbb{Z}\}$. Portanto, por exemplo, $3\mathbb{Z} = \{0, \pm 3, \pm 6, \pm 9, \dots\}$

PRÁTICA 25 Mostre que para qualquer $n \in \mathbb{N}$, $[n\mathbb{Z}, +]$ é um subgrupo de $[\mathbb{Z}, +]$. •

Não apenas $[n\mathbb{Z}, +]$ é um subgrupo de $[\mathbb{Z}, +]$ para algum n fixo, como conjuntos da forma $n\mathbb{Z}$ são os únicos subgrupos de $[\mathbb{Z}, +]$. Para ilustrar, seja $[S, +]$ qualquer subgrupo de $[\mathbb{Z}, +]$. Se $S = \{0\}$, então $S = 0\mathbb{Z}$. Se $S \neq \{0\}$, seja m um elemento de S , $m \neq 0$ ou m é positivo ou, se m é negativo, $-m \in S$ e é positivo. O subgrupo S , portanto, contém pelo menos um inteiro positivo. Seja n o menor inteiro positivo em S (que existe, devido ao princípio da boa ordenação). Veremos agora que $S = n\mathbb{Z}$.

Primeiro, uma vez que 0 e $-n$ são elementos de S e S é fechado sob $+$, $n\mathbb{Z} \subseteq S$. Para obter a inclusão na outra direção, seja $s \in S$. Dividimos agora o inteiro s pelo inteiro n a fim de obter um quociente inteiro q e um

resto inteiro r com $0 \leq r < n$. Portanto, $s = nq + r$. Resolvendo para r , $r = s + (-nq)$. Mas $nq \in S$, portanto $-nq \in S$ e $s \in S$, portanto, pelo fecho de S sob $+$, $r \in S$. Se r for positivo, temos uma contradição da definição de n como o menor número positivo em S . Portanto, $r = 0$ e $s = nq + r = nq$. Temos agora que $S \subseteq n\mathbb{Z}$, e portanto $S = n\mathbb{Z}$, o que completa a demonstração do teorema a seguir.

Teorema dos Subgrupos de $[\mathbb{Z}, +]$

Os subgrupos da forma $[n\mathbb{Z}, +]$ para $n \in \mathbb{N}$ são os únicos subgrupos de $[\mathbb{Z}, +]$.

Grupos Isomorfos

Suponha que $[S, \cdot]$ e $[T, +]$ são grupos isomorfos; o que isto significa? Pela discussão sobre isomorfismo da Seção 7.1, estruturas isomorfas são as mesmas exceto pela atribuição de novos rótulos. É necessário que exista uma bijeção entre S e T que realize este novo rotulamento. Esta bijeção precisa também preservar os efeitos da operação binária; isto é, "operar e mapear" deve produzir o mesmo resultado que "mapear e operar". A definição a seguir é mais precisa:

Definição: Isomorfismo de Grupos

Sejam $[S, \cdot]$ e $[T, +]$ grupos. Uma aplicação $f: S \rightarrow T$ é um isomorfismo de $[S, \cdot]$ em $[T, +]$ se,

1. a função f é uma bijeção e
2. para todos $x, y \in S$, $f(x \cdot y) = f(x) + f(y)$.

A propriedade (2) é expressa dizendo-se que f é um homomorfismo.

PRÁTICA 26 Ilustre a propriedade de homomorfismo da definição acima através de um diagrama comutativo. •

Para deixar claro que grupos isomorfos são os mesmos, exceto pela atribuição de novos rótulos, vamos mostrar que sob um isomorfismo o elemento neutro de um grupo é levado ao elemento neutro do outro, e os elementos inversos são levados em elementos inversos. Além disso, se um grupo for comutativo, o outro também o será.

Suponha, então, que f é um isomorfismo do grupo $[S, \cdot]$ no grupo $[T, +]$ e que i_s e i_T são os elementos neutros desses grupos. Sob a função f , i_s é levado a um elemento $f(i_s)$ em T . Seja t qualquer elemento em T . Então, como f é uma função sobrejetiva, $t = f(s)$ para algum $s \in S$. Segue, portanto,

$$\begin{aligned} f(i_s) + t &= f(i_s) + f(s) \\ &= f(i_s \cdot s) && \text{(pois } f \text{ é um homomorfismo)} \\ &= f(s) && \text{(pois } i_s \text{ é o elemento neutro de } S) \\ &= t \end{aligned}$$

Portanto,

$$f(i_s) + t = t$$

Analogamente,

$$t + f(i_s) = t$$

O elemento $f(i_s)$ funciona como um elemento neutro em $[T, +]$ e como o elemento neutro é único, $f(i_s) = i_T$.

PRÁTICA 27 Demonstre que se f é um isomorfismo do grupo $[S, \cdot]$ no grupo $[T, +]$, então para qualquer $s \in S$, $f(s^{-1}) = (f(s))^{-1}$ (inversos são levados em inversos). (Dica: Mostre que $f(s^{-1})$ funciona como o inverso de $f(s)$.) •

PRÁTICA 28 Demonstre que se f é um isomorfismo do grupo comutativo $[S, \cdot]$ no grupo $[T, +]$, então $[T, +]$ também é um grupo comutativo. •

EXEMPLO 13 $[\mathbb{R}^+, \cdot]$ e $[\mathbb{R}, +]$ são ambos grupos. Seja b um número real positivo, $b \neq 1$, e seja f a função de $\mathbb{R}^+$ em $\mathbb{R}$ definida por

$$f(x) = \log_b x$$

Então f é um isomorfismo. Para demonstrar isto, devemos mostrar que f é uma bijeção (é injetiva e sobrejetiva) e que f é um homomorfismo (preserva a operação). Podemos mostrar que f é sobrejetiva: para $r \in \mathbb{R}$, $b^r \in \mathbb{R}^+$ e $f(b^r) = \log_b b^r = r$. Além disso, f é injetiva: se $f(x_1) = f(x_2)$, então $\log_b x_1 = \log_b x_2$. Seja $p = \log_b x_1 = \log_b x_2$. Então $b^p = x_1$ e $b^p = x_2$, portanto $x_1 = x_2$. Finalmente, f é um homomorfismo: para $x_1, x_2 \in \mathbb{R}^+$, $f(x_1 \cdot x_2) = \log_b(x_1 \cdot x_2) = \log_b x_1 + \log_b x_2 = f(x_1) + f(x_2)$. Perceba que $\log_b 1 = 0$, de forma que f leva 1, o elemento neutro de $[\mathbb{R}^+, \cdot]$ em 0, o elemento neutro de $[\mathbb{R}, +]$. Além disso, perceba que

$$\log_b \frac{1}{x} = \log_b 1 - \log_b x = 0 - \log_b x = -\log_b x = -f(x)$$

de forma que f leva o inverso do elemento x em $[\mathbb{R}^+, \cdot]$ no inverso de $f(x)$ em $[\mathbb{R}, +]$. Finalmente, ambos os grupos são comutativos. •

Como os dois grupos do Exemplo 13 são isomorfos, cada um é um espelho do outro e cada qual pode ser usado para simular cálculos no outro. Suponha, por exemplo, que $b = 2$. Então $[\mathbb{R}, +]$ pode ser usado para simular o cálculo de 64.512 em $[\mathbb{R}^+, \cdot]$. Primeiro, passemos de $\mathbb{R}^+$ para $\mathbb{R}$:

$$\begin{aligned} f(64) &= \log_2 64 = 6 \\ f(512) &= \log_2 512 = 9 \end{aligned}$$

Agora em $[\mathbb{R}, +]$, realizamos o cálculo

$$6 + 9 = 15$$

Finalmente, usamos f^{-1} para levar de volta a $\mathbb{R}^+$:

$$f^{-1}(15) = 2^{15} = 32.768$$

(Na era A.C. — antes dos computadores —, os números grandes eram multiplicados através de tabelas de logaritmos comuns, onde $b = 10$, para converter um problema de multiplicação em um problema de adição.) Qualquer um de dois grupos isomorfos pode ser sempre usado para simular operações no outro, como no Exemplo 13.

De acordo com a definição, para que f seja um isomorfismo, ela precisa ser uma bijeção e um homomorfismo.

EXEMPLO 14 Considere as seguintes funções de $\mathbb{Z}$ em $\mathbb{Z}$:

$$\begin{aligned} f(x) &= 0 \\ g(x) &= x + 1 \end{aligned}$$

A função f é um homomorfismo do grupo $[\mathbb{Z}, +]$ no grupo $[\mathbb{Z}, +]$ pois $f(x + y) = 0 = 0 + 0 = f(x) + f(y)$. No entanto, f não é uma bijeção, de forma que não constitui um isomorfismo.

A função g é uma bijeção porque $g(x) = g(y)$ implica $x + 1 = y + 1$, ou $x = y$, de forma que g é injetiva; g também é sobrejetiva porque, para qualquer $z \in \mathbb{Z}$, $z - 1 \in \mathbb{Z}$ e $g(z - 1) = z$. Mas g não é um homomorfismo porque $g(x + y) = (x + y) + 1 \neq (x + 1) + (y + 1) = g(x) + g(y)$. Portanto, g não é um isomorfismo. ♦

EXEMPLO 15 Seja $f: M_2(\mathbb{Z}) \rightarrow M_2(\mathbb{Z})$ dada por

$$f\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = \begin{bmatrix} a & c \\ b & d \end{bmatrix}$$

Para mostrar que f é injetiva, seja

$$f\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = f\left(\begin{bmatrix} e & f \\ g & h \end{bmatrix}\right)$$

Então

$$\begin{bmatrix} a & c \\ b & d \end{bmatrix} \begin{bmatrix} e & g \\ f & h \end{bmatrix}$$

Então $a = e$, $c = g$, $b = f$ e $d = h$ ou

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

Para mostrar que f é sobrejetiva, seja

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \in M_2(\mathbb{Z})$$

Então

$$\begin{bmatrix} a & c \\ b & d \end{bmatrix} \in M_2(\mathbb{Z}) \text{ e } f\left(\begin{bmatrix} a & c \\ b & d \end{bmatrix}\right) = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Além disso, f é um homomorfismo de $[M_2(\mathbb{Z}), +]$ em $[M_2(\mathbb{Z}), +]$ porque

$$\begin{aligned} f\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix} + \begin{bmatrix} e & f \\ g & h \end{bmatrix}\right) &= f\left(\begin{bmatrix} a+e & b+f \\ c+g & d+h \end{bmatrix}\right) = \begin{bmatrix} a+e & c+g \\ b+f & d+h \end{bmatrix} \\ &= \begin{bmatrix} a & c \\ b & d \end{bmatrix} + \begin{bmatrix} e & g \\ f & h \end{bmatrix} = f\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) + f\left(\begin{bmatrix} e & f \\ g & h \end{bmatrix}\right) \end{aligned}$$

A função f é, portanto, um isomorfismo de $[M_2(\mathbb{Z}), +]$ em $[M_2(\mathbb{Z}), +]$.

PRÁTICA 29 Seja $5\mathbb{Z} = \{5z \mid z \in \mathbb{Z}\}$. Então $[5\mathbb{Z}, +]$ é um grupo. Mostre que $f: \mathbb{Z} \rightarrow 5\mathbb{Z}$ dada por $f(x) = 5x$ é um isomorfismo de $[\mathbb{Z}, +]$ em $[5\mathbb{Z}, +]$ ♦

Se f é um isomorfismo de $[S, \cdot]$ em $[T, +]$, então f^{-1} existe e é uma bijeção. Além disso, f^{-1} também é um homomorfismo, desta vez de T em S . Para constatar isso, seja t_1 e t_2 pertencentes a T e considere $f^{-1}(t_1 + t_2)$. Como $t_1, t_2 \in T$ e f é sobrejetiva, $t_1 = f(s_1)$ e $t_2 = f(s_2)$ para algum s_1 e s_2 em S . Portanto,

$$\begin{aligned} f^{-1}(t_1 + t_2) &= f^{-1}(f(s_1) + f(s_2)) \\ &= f^{-1}(f(s_1 \cdot s_2)) \\ &= (f^{-1} \circ f)(s_1 \cdot s_2) \\ &= s_1 \cdot s_2 \\ &= f^{-1}(t_1) \cdot f^{-1}(t_2) \end{aligned}$$

Esta é a razão pela qual podemos nos referir a S e T simplesmente como isomorfos, denotado por $S \cong T$, sem termos que especificar que o isomorfismo é de S a T , ou vice-versa.

Verificar se uma função é ou não um isomorfismo de S em T , como fizemos na Prática 29, não é difícil. Determinar se S e T são isomorfos é mais difícil. Para demonstrar que eles são isomorfos, devemos produzir uma função de isomorfismo. Para demonstrar que eles não são isomorfos, precisamos mostrar que não existe tal função. Como não podemos testar todas as funções, usamos idéias como as seguintes: não há correspondência um-para-um entre S e T , S é comutativo mas T não é, e assim por diante.

Já vimos que os grupos isomorfos são iguais exceto por um novo rotulamento e que cada um dos grupos pode ser usado para simular cálculos no outro. O isomorfismo de grupos é realmente uma relação de equivalência, como a Prática 30 abaixo mostra; portanto grupos isomorfos pertencem à mesma classe de equivalência. Considerar os grupos isomorfos como iguais a menos de um rotulamento é consistente com a idéia de que os elementos em uma classe de equivalência representam diferentes nomes para a mesma coisa.

- PRÁTICA 30**
- a.** Seja $f: S \rightarrow T$ um isomorfismo do grupo $[S, \cdot]$ sobre o grupo $[T, +]$ e $g: T \rightarrow U$ um isomorfismo de $[T, +]$ sobre o grupo $[U, *]$. Mostre que $g \circ f$ é um isomorfismo de S sobre U .
- b.** Seja $\mathcal{S}$ uma coleção de grupos e defina uma relação binária ρ em $\mathcal{S}$ como $S \rho T \leftrightarrow S \cong T$. Mostre que ρ é uma relação de equivalência em $\mathcal{S}$. ♦

Classes de Grupos Isomorfos

Vamos finalizar esta seção com algumas classes de equivalência de grupos sob isomorfismo. Frequentemente tomamos um elemento de uma classe de equivalência e percebemos que ele é o elemento típico desta classe, e que todos os outros elementos da classe se parecem com ele (com nomes diferentes).

Um resultado referente à natureza de grupos muito pequenos é resultado imediato do Exercício 18 ao fim desta seção.

Teorema dos Grupos Pequenos

Todo grupo de ordem 2 é isomorfo ao grupo cuja tabela é

$\cdot$	1	a
1	1	a
a	a	1

Todo grupo de ordem 3 é isomorfo ao grupo cuja tabela é

$\cdot$	1	a	b
1	1	a	b
a	a	b	1
b	b	1	a

Todo grupo de ordem 4 é isomorfo a um dos dois grupos cujas tabelas são

$\cdot$	1	a	b	c
1	1	a	b	c
a	a	1	c	b
b	b	c	1	a
c	c	b	a	1

$\cdot$	1	a	b	c
1	1	a	b	c
a	a	b	c	1
b	b	c	1	a
c	c	1	a	b

Podemos ainda demonstrar que qualquer grupo é essencialmente um grupo de permutações. Suponha que $[G, \cdot]$ é um grupo. Desejamos estabelecer um isomorfismo de G em um grupo de permutações; cada elemento g de G precisa estar associado a uma permutação α_g em algum conjunto. Na verdade, o conjunto será o próprio G ; para qualquer $x \in G$, definimos $\alpha_g(x)$ como $g \cdot x$. Devemos mostrar que $\{\alpha_g \mid g \in G\}$ constitui um grupo de permutações e que este grupo de permutações é isomorfo a G . Primeiro devemos mostrar que para qualquer $g \in G$, α_g é, de fato, uma permutação em G . Da definição $\alpha_g(x) = g \cdot x$, está claro que α_g é uma função do tipo $\alpha_g: G \rightarrow G$, mas é preciso mostrar que α_g é uma bijeção.

- PRÁTICA 31** Mostre que α_g como definida acima é uma permutação em G . •

Consideremos agora $P = \{\alpha_g \mid g \in G\}$ e vamos mostrar que P é um grupo sob a composição de funções. P não é vazio porque G também não o é, e a associatividade sempre vale para a composição de funções. Devemos mostrar que vale o fechamento de P , que P contém um elemento neutro e que todo $\alpha_g \in P$ tem um inverso em P . Para mostrar o fechamento, seja α_g e $\alpha_h \in P$. Para qualquer $x \in G$, $(\alpha_g \circ \alpha_h)(x) = \alpha_g(\alpha_h(x)) = \alpha_g(h \cdot x) = g \cdot (h \cdot x) = (g \cdot h) \cdot x$. Portanto, $\alpha_g \circ \alpha_h = \alpha_{g \cdot h}$ e $\alpha_{g \cdot h} \in P$.

- PRÁTICA 32**
- a.** Denotemos por 1 o elemento neutro G . Mostre que a , é um elemento neutro de P sob a composição de funções.
- b.** Para $\alpha_g \in P$, $\alpha_{g^{-1}} \in P$; mostre que $\alpha_{g^{-1}} = (\alpha_g)^{-1}$.

Sabemos que $[P, \circ]$ é um grupo de permutações, falta apenas demonstrar que a função $f: G \rightarrow P$ dada por $f(g) = \alpha_g$ é um isomorfismo. Está claro que f é sobrejetiva. •

PRÁTICA 33 Mostre que $f: G \rightarrow P$ definida acima é

- a. injetiva
- b. um homomorfismo

Nós demonstramos o seguinte teorema, enunciado e demonstrado originalmente pelo matemático inglês Arthur Cayley nos meados do século dezenove.

Teorema de Cayley

Todo grupo é isomorfo a um grupo de permutações.

Revisão da Seção 8.1

Técnicas

- Verificar se um dado conjunto e uma dada operação têm as propriedades necessárias para formar um semigrupo, um monóide ou um grupo
- Verificar se um dado subconjunto de um grupo é um subgrupo
- Verificar se uma dada função de um grupo em outro é ou não um isomorfismo
- Determinar se dois grupos são ou não isomorfos

Idéias Principais

Diversos sistemas elementares da aritmética são casos particulares de estruturas de semigrupos, monóides ou grupos.

Em qualquer estrutura de grupos, os elementos neutro e inverso são únicos, valem as leis do cancelamento e as equações lineares são solucionáveis; estas e outras propriedades são consequências da definição.

Um subconjunto de um grupo pode também ser um grupo sob a operação herdada.

A ordem de um subgrupo de um grupo finito é um divisor da ordem do grupo.

Os únicos subgrupos do grupo $[\mathbb{Z}, +]$ são da forma $[n\mathbb{Z}, +]$ onde $n\mathbb{Z}$ é o conjunto de todos os múltiplos de um $n \in \mathbb{N}$ fixo.

Se f é um isomorfismo de um grupo em outro, leva o elemento neutro de um no elemento neutro de outro, inversos nos inversos e preserva a comutatividade.

Se S e T são grupos isomorfos, eles são idênticos, exceto por uma nova atribuição de rótulos, e cada um simula cálculos no outro.

O isomorfismo é uma relação de equivalência nos grupos.

Se desconsiderarmos os grupos isomorfos, existe apenas um grupo de ordem 2, um grupo de ordem 3 e dois grupos de ordem 4.

Todo grupo é essencialmente um grupo de permutações.

Exercícios 8.1

1. Cada item abaixo define uma operação binária, denotada por $\cdot$, em um dado conjunto. Quais são associativas? Quais são comutativas?

★a. Em $\mathbb{Z}$: $x \cdot y = \begin{cases} x & \text{se } x \text{ for par} \\ x + 1 & \text{se } x \text{ for ímpar} \end{cases}$

★b. Em $\mathbb{N}$: $x \cdot y = (x + y)^2$

c. Em $\mathbb{R}^+$: $x \cdot y = x^4$

d. Em $\mathbb{Q}$: $x \cdot y = \frac{xy}{2}$

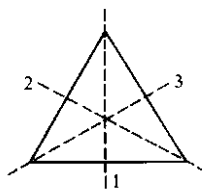
e. Em $\mathbb{R}^+$: $x \cdot y = \frac{1}{x + y}$

- ★2. a. Uma operação binária $\cdot$ é definida no conjunto $\{a, b, c, d\}$ pela tabela da esquerda abaixo. Determine se $\cdot$ é comutativa e associativa.
- b. Seja $S = \{p, q, r, s\}$. Uma operação associativa $\cdot$ é parcialmente definida em S pela tabela da direita abaixo. Complete a tabela para preservar a associatividade. A operação $\cdot$ é comutativa?

$\cdot$	a	b	c	d
a	a	c	d	a
b	b	c	a	d
c	c	a	b	d
d	d	a	a	c

$\cdot$	p	q	r	s
p	p	q	r	s
q	q	r	s	p
r			p	
s	s		q	r

3. Defina operações binárias no conjunto $\mathbb{N}$ que sejam:
- Comutativa, mas não-associativa
 - Associativa, mas não-comutativa
 - Nem associativa nem comutativa
 - Associativa e comutativa
4. Mostre que a multiplicação de matrizes em $M_2(\mathbb{Z})$ é associativa.
5. Determine se as estruturas $[S, \cdot]$ são semigrupos, monóides, grupos ou nenhum deles. Indique os elementos neutros das que forem monóides ou grupos.
- ★a. $S = \mathbb{N}; x \cdot y = \min(x, y)$
- ★b. $S = \mathbb{R}; x \cdot y = (x + y)^2$
- ★c. $S = \{a\sqrt{2} \mid a \in \mathbb{N}\}; \cdot = \text{multiplicação}$
- ★d. $S = \{a + b\sqrt{2} \mid a, b \in \mathbb{Z}\}; \cdot = \text{multiplicação}$
- ★e. $S = \{a + b\sqrt{2} \mid a, b \in \mathbb{Q}, a \text{ e } b \text{ não são ambos } 0\}; \cdot = \text{multiplicação}$
- ★f. $S = \{1, -1, i, -i\}; \cdot = \text{multiplicação (onde } i^2 = -1)$
- ★g. $S = \{1, 2, 4\}; \cdot = \cdot_6$
- h. $S = \{1, 2, 3, 5, 6, 10, 15, 30\}; x \cdot y = \text{mínimo múltiplo comum de } x \text{ e } y$
- i. $S = \mathbb{N} \times \mathbb{N}; (x_1, y_1) \cdot (x_2, y_2) = (x_1, y_2)$
- j. $S = \mathbb{N} \times \mathbb{N}; (x_1, y_1) \cdot (x_2, y_2) = (x_1 + x_2, y_1 y_2)$
- k. $S = \text{conjunto dos inteiros pares}; \cdot = \text{adição}$
- l. $S = \text{conjunto dos inteiros ímpares}; \cdot = \text{adição}$
- m. $S = \text{conjunto de todos os polinômios em } \mathbb{R}[x] \text{ de grau } \leq 3; \cdot = \text{soma de polinômios}$
- n. $S = \text{conjunto de todos os polinômios em } \mathbb{R}[x] \text{ de grau } \leq 3; \cdot = \text{multiplicação de polinômios}$
- o. $S = \left\{ \begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix} \mid z \in \mathbb{Z} \right\}; \cdot = \text{multiplicação de matrizes}$
- p. $S = \{1, 2, 3, 4\}; \cdot = \cdot_5$
- q. $S = \mathbb{R} - \{-1\}; x \cdot y = x + y + xy$
- r. $S = \{f \mid f: \mathbb{N} \rightarrow \mathbb{N}\}; \cdot = \text{função soma, isto é,}$
 $(f + g)(x) = f(x) + g(x)$
6. Seja $A = \{1, 2\}$.
- Descreva os elementos e escreva a tabela do subgrupo de transformações em A .
 - Descreva os elementos e escreva a tabela do grupo de permutações em A .



Exercício 7

7. Dado um triângulo equilátero, podem ser realizadas seis permutações no triângulo que mantêm sua imagem no plano inalterada. Três dessas permutações são rotações em sentido horário de 120° , 240° e 360° em torno do centro do triângulo; essas permutações são denotadas por R_1 , R_2 e R_3 , respectivamente. O triângulo pode ainda ser refletido pelos eixos 1, 2 e 3 (veja a figura ao lado); essas permutações são denotadas por F_1 , F_2 e F_3 , respectivamente. Ao aplicarmos qualquer dessas permutações, os eixos permanecem fixos no plano. A composição de permutações é uma operação binária no conjunto D_3 de todas as seis permutações. Por exemplo $F_3 \circ R_2 = F_2$. O conjunto D_3 sob a composição é um grupo, chamado de **grupo**

de simetrias de um triângulo equilátero. Complete a tabela abaixo para o grupo $[D_3, \circ]$. Qual é o elemento neutro de $[D_3, \circ]$? Qual o inverso de F_1 ? E de R_2 ?

$\circ$	R_1	R_2	R_3	F_1	F_2	F_3
R_1						
R_2						
R_3						
F_1						
F_2						
F_3					F_2	

8. O conjunto S_3 , o grupo simétrico de grau 3, é isomorfo a D_3 , o grupo de simetrias de um triângulo equilátero (veja o Exercício 7). Encontre uma bijeção dos elementos de S_3 nos elementos de D_3 que preserve a operação. (Dica: a rotação R_1 de D_3 pode ser considerada uma permutação em S_3 , levando 1 em 2, 2 em 3 e 3 em 1.)
9. Em cada caso, determine se a estrutura da esquerda é ou não um subgrupo do grupo da direita. Em caso negativo, justifique. (Perceba que S^* denota $S - \{0\}$.)
 - ★a. $[\mathbb{Z}; +_5]; [\mathbb{Z}_5; +_5]$
 - ★b. $[P, +]; [\mathbb{R}[x], +]$ onde P é o conjunto de todos os polinômios em x sobre $\mathbb{R}$ de grau ≥ 3
 - ★c. $[\mathbb{Z}^*, \cdot]; [\mathbb{Q}^*, \cdot]$
 - d. $[A, \circ]; [S, \circ]$, onde S é o conjunto de todas as bijeções em $\mathbb{N}$ e A é o conjunto de todas as bijeções em $\mathbb{N}$ que levam 3 nele mesmo.
 - e. $[\mathbb{Z}, +]; [M_2(\mathbb{Z}), +]$
 - f. $[K, +]; [\mathbb{R}[x], +]$ onde K é o conjunto de todos os polinômios em x sobre $\mathbb{R}$ de grau $\leq k$ para algum k fixo
 - g. $[\{0, 3, 6\}, +_8]; [\mathbb{Z}_8, +_8]$
- ★10. Encontre todos os subgrupos distintos de $[\mathbb{Z}_{12}, +_{12}]$
11. a. Mostre que o subconjunto

$$\begin{aligned}\alpha_1 &= i \\ \alpha_2 &= (1, 2) \circ (3, 4) \\ \alpha_3 &= (1, 4) \circ (2, 3) \\ \alpha_4 &= (1, 3) \circ (2, 4)\end{aligned}$$
 forma um subgrupo do grupo simétrico S_4 .
 b. Mostre que o subconjunto

$$\begin{aligned}\alpha_1 &= i \\ \alpha_2 &= (1, 2, 3, 4) \\ \alpha_3 &= (1, 3) \circ (2, 4) \\ \alpha_4 &= (1, 4, 3, 2) \\ \alpha_5 &= (1, 2) \circ (3, 4) \\ \alpha_6 &= (1, 4) \circ (2, 3) \\ \alpha_7 &= (2, 4) \\ \alpha_8 &= (1, 3)\end{aligned}$$
 forma um subgrupo do grupo simétrico S_4 .
- ★12. Encontre os elementos do grupo alternante A_4 .
13. Em cada caso, determine se a função dada é ou não um homomorfismo do grupo da esquerda no grupo da direita. Algum dos homomorfismos é também um isomorfismo?
 - ★a. $[\mathbb{Z}, +]; [\mathbb{Z}, +]; f(x) = 2$
 - ★b. $[\mathbb{R}, +]; [\mathbb{R}, +]; f(x) = |x|$
 - ★c. $[\mathbb{R}^*, \cdot]; [\mathbb{R}^*, \cdot]$ (onde $\mathbb{R}^*$ denota o conjunto de todos números reais não nulos); $f(x) = |x|$
 - d. $[\mathbb{R}[x], +]; [\mathbb{R}, +]; f(a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0) = a_n + a_{n-1} + \cdots + a_0$
 - e. $[S_3, \circ]; [\mathbb{Z}_2, +_2]; f(\alpha) = \begin{cases} 1 & \text{se } \alpha \text{ for uma permutação par} \\ 0 & \text{se } \alpha \text{ for uma permutação ímpar} \end{cases}$

14. Em cada item, determine se os grupos dados são ou não isomorfos. Em caso afirmativo, apresente uma função de isomorfismo. Em caso negativo, justifique.
- $[\mathbb{Z}, +], [12\mathbb{Z}, +]$ (onde $12\mathbb{Z} = \{12z \mid z \in \mathbb{Z}\}$)
 - $[\mathbb{Z}_5, +_5], [5\mathbb{Z}, +]$
 - $[5\mathbb{Z}, +], [12\mathbb{Z}, +]$
 - $[S_3, \circ], [\mathbb{Z}_6, +_6]$
 - $[\{a_1x + a_0 \mid a_1, a_0 \in \mathbb{R}\}, +], [\mathbb{C}, +]$
 - $[\mathbb{Z}_6, +_6], [S_6, \circ]$
 - $[\mathbb{Z}_2, +_2], [S_2, \circ]$

15. a. Seja $S = \{1, -1\}$. Mostre que $[S, \cdot]$ é um grupo, onde $\cdot$ denota a multiplicação convencional de inteiros.
 b. Seja f a função do grupo $[S, \circ]$ no grupo $[S, \cdot]$ dada por

$$f(\alpha) = \begin{cases} 1 & \text{se } \alpha \text{ for par} \\ -1 & \text{se } \alpha \text{ for ímpar} \end{cases}$$

Mostre que f é um homomorfismo.

- ★16. Seja $M_2^0(\mathbb{Z})$ o conjunto de todas as matrizes 2×2 da forma

$$\begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix}$$

onde $z \in \mathbb{Z}$.

- a. Mostre que $[M_2^0(\mathbb{Z}), \cdot]$ é um grupo, onde $\cdot$ denota a multiplicação de matrizes.
 b. Seja uma função $f: M_2^0(\mathbb{Z}) \rightarrow \mathbb{Z}$ definida por

$$f\left(\begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix}\right) = z$$

Demonstre que f é um isomorfismo de $[M_2^0(\mathbb{Z}), \cdot]$ em $[\mathbb{Z}, +]$

- c. Use $[\mathbb{Z}, +]$ para simular o cálculo de

$$\begin{bmatrix} 1 & 7 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & -3 \\ 0 & 1 \end{bmatrix}$$

em $[M_2^0(\mathbb{Z}), \cdot]$.

- d. Use $[M_2^0(\mathbb{Z}), \cdot]$ para simular o cálculo de $2 + 3$ em $[\mathbb{Z}, +]$.

17. Em qualquer grupo $[G, \cdot]$ mostre que

$$\text{a. } i^{-1} = i \quad \text{b. } (x^{-1})^{-1} = x \text{ para qualquer } x \in G.$$

18. a. Mostre que qualquer grupo de ordem 2 é comutativo, construindo uma tabela de grupo para o conjunto $\{1, a\}$, usando 1 como elemento neutro.
 b. Mostre que qualquer grupo de ordem 3 é comutativo, construindo uma tabela de grupo para o conjunto $\{1, a, b\}$, usando 1 como elemento neutro. (Você pode admitir a associatividade.)
 c. Mostre que qualquer grupo de ordem 4 é comutativo, construindo uma tabela de grupo para o conjunto $\{1, a, b, c\}$, usando 1 como elemento neutro. (Você pode admitir a associatividade.) Existem quatro dessas tabelas, mas três delas são isomorfas, devido a seus elementos receberem apenas rotulamentos diferentes. Encontre estes três grupos isomorfos e indique o novo rotulamento. Portanto, existem dois grupos essencialmente diferentes de ordem 4, e ambos são comutativos.

- ★19. Seja $[S, \cdot]$ um semigrupo. Um elemento $i_L \in S$ é um **elemento neutro à esquerda** se para qualquer $x \in S$, $i_L \cdot x = x$. Um elemento $i_R \in S$ é um **elemento neutro à direita** se para qualquer $x \in S$, $x \cdot i_R = x$.
 a. Mostre que se um semigrupo $[S, \cdot]$ tem ambos os elementos neutros à direita e à esquerda, então ele é um monóide.

- b. Forneça um exemplo de um semigrupo finito com dois elementos neutros à esquerda e nenhum elemento neutro à direita.
- c. Forneça um exemplo de um semigrupo finito com dois elementos neutros à direita e nenhum elemento neutro à esquerda.
- d. Forneça um exemplo de um semigrupo sem elementos neutros à direita nem à esquerda.

20. Seja $[S, \cdot]$ um monóide com elemento neutro i , e seja $x \in S$. Um elemento x_L^{-1} em S é um **inverso à esquerda** de x se $x_L^{-1} \cdot x = i$. Um elemento x_R^{-1} em S é um **inverso à direita** de x se $x \cdot x_R^{-1} = i$.
- a. Mostre que se qualquer elemento em um monóide $[S, \cdot]$ tem tanto um inverso à esquerda quanto um inverso à direita, então $[S, \cdot]$ é um grupo.
 - b. Seja S o conjunto de todas as funções f tais que $f: \mathbb{N} \rightarrow \mathbb{N}$. Então S sob a composição de funções é um monóide. Defina uma função $f \in S$ por $f(x) = 2x$, $x \in \mathbb{N}$. Defina então uma função $g \in S$ por

$$g(x) = \begin{cases} x/2 & \text{se } x \in \mathbb{N}, x \text{ par} \\ 1 & \text{se } x \in \mathbb{N}, x \text{ ímpar} \end{cases}$$

Mostre que g é uma inversa à esquerda de f . Além disso, mostre que g não tem inversa à direita.

21. Para x um elemento do grupo $[G, \cdot]$, podemos definir x^n para qualquer inteiro positivo n como $x^1 = x$, $x^2 = x \cdot x$ e $x^n = x^{n-1} \cdot x$ para $n > 2$. Mostre que em um grupo finito $[G, \cdot]$, para cada $x \in G$ existe um inteiro positivo k tal que $x^k = i$.
22. Seja $[G, \cdot]$ um grupo e sejam $x, y \in G$. Defina uma relação ρ em G por $x \rho y \Leftrightarrow \exists g \in G \text{ tal que } xg^{-1} = y$.
- a. Mostre que ρ é uma relação de equivalência em G .
 - b. Prove que para cada $x \in G$, $[x] = \{x\}$ se, e somente se, G é comutativa.
23. Seja $[S, \cdot]$ um semigrupo com inverso à esquerda i_L (veja o Exercício 19) e com a propriedade de que para qualquer $x \in S$, x tem inversa y tal que $y \cdot x = i_L$. Demonstre que $[S, \cdot]$ é um grupo. (Dica: y também tem inversa à esquerda em S .)
24. Mostre que se $[S, \cdot]$ é um semigrupo no qual as equações lineares $a \cdot x = b$ e $x \cdot a = b$ são solucionáveis para quaisquer $a, b \in S$, então $[S, \cdot]$ é um grupo. (Dica: Use o Exercício 23.)
25. Demonstre que um semigrupo finito que satisfaz às leis do cancelamento à esquerda e à direita é um grupo. (Dica: Use o Exercício 23.)
26. Mostre que um grupo $[G, \cdot]$ é comutativo se, e somente se, $(x \cdot y)^2 = x^2 \cdot y^2$ para cada $x, y \in G$.
27. Mostre que um grupo $[G, \cdot]$ no qual $x \cdot x = i$ para todo $x \in G$ é comutativo.
- ★28. a. Seja $[G, \cdot]$ um grupo e seja $[S, \cdot]$ e $[T, \cdot]$ subgrupos de $[G, \cdot]$. Mostre que $[S \cap T, \cdot]$ é um subgrupo de $[G, \cdot]$.
- b. $[S \cup T, \cdot]$ é um subgrupo de $[G, \cdot]$? Prove ou forneça um contra-exemplo.
29. Seja $[G, \cdot]$ um grupo comutativo com subgrupos $[S, \cdot]$ e $[T, \cdot]$. Seja $ST = \{s \cdot t \mid s \in S, t \in T\}$. Mostre que $[SUT, \cdot]$ é um subgrupo de $[G, \cdot]$.
- ★30. Seja $[G, \cdot]$ um grupo comutativo com elemento neutro i . Para um inteiro positivo fixo k , seja $B_k = \{x \in G, x^k = i\}$. Mostre que $[B_k, \cdot]$ é um subgrupo de $[G, \cdot]$.
31. Para qualquer grupo $[G, \cdot]$, o **centro** do grupo é $A = \{x \in G \mid x \cdot g = g \cdot x \text{ para todo } g \in G\}$.
- a. Demonstre que $[A, \cdot]$ é um subgrupo de $[G, \cdot]$.
 - b. Encontre o centro do grupo de simetrias de um triângulo equilátero, $[D_3, \cdot]$ (veja o Exercício 7).
 - c. Mostre que G é comutativo se, e somente se, $G = A$.
 - d. Sejam x e y elementos de G com $x \cdot y^{-1} \in A$. Mostre que $x \cdot y = y \cdot x$.
32. a. Seja S_A o grupo das permutações em um conjunto A e seja a um elemento fixo de A . Mostre que o conjunto H_a de todas as permutações em S_A que mantêm a fixo forma um subgrupo de S_A .
- b. Se A tem n elementos, quanto vale $|H_a|$?

33. a. Seja $[G, \cdot]$ um grupo de permutações no conjunto $A \subseteq G, A \neq \emptyset$. Mostre que $[A, \cdot]$ é um subgrupo de $[G, \cdot]$ se para cada $x, y \in A, x \cdot y^{-1} \in A$. Este teste para subgrupos às vezes é mais conveniente de ser usado do que o teorema dos subgrupos.
b. Use o teste do item (a) para refazer o Exercício 30 acima.
34. a. Seja $[G, \cdot]$ qualquer grupo de elemento neutro i . Para um $a \in G$ fixo, a^0 denota i e a^{-n} denota $(a^n)^{-1}$. Seja $A = \{a^z \mid z \in \mathbb{Z}\}$. Mostre que $[A, \cdot]$ é um subgrupo de G .
b. O grupo $[G, \cdot]$ é um **grupo cíclico** se para algum $a \in G, A = \{a^z \mid z \in \mathbb{Z}\}$ é todo o grupo G . Neste caso, a é o **gerador** de $[G, \cdot]$. Por exemplo, 1 é um gerador para o grupo $[\mathbb{Z}, +]$; perceba que a operação é a adição. Portanto, $1^0 = 1, 1^1 = 1, 1^2 = 1 + 1 = 2, 1^3 = 1 + 1 + 1 = 3, \dots; 1^{-1} = (1)^{-1} = -1; 1^{-2} = (1^2)^{-1} = -2; 1^{-3} = (1^3)^{-1} = -3, \dots$. Todo inteiro pode ser escrito como uma potência inteira de 1 e $[\mathbb{Z}, +]$ é cíclico com gerador 1. Mostre que o grupo $[\mathbb{Z}_7, +_7]$ é cíclico com gerador 2.
c. Mostre que 5 também é gerador do grupo cíclico $[\mathbb{Z}_7, +_7]$.
d. Mostre que 3 é um gerador do grupo cíclico $[\mathbb{Z}_4, +_4]$.
- ★35. Seja $[G, \cdot]$ um grupo cíclico com gerador a (veja o Exercício 34). Mostre que G é comutativo.
36. Seja $[G, \cdot]$ um grupo comutativo com elemento neutro i . Prove que a função $f: G \rightarrow G$ dada por $f(x) = x^{-1}$ é um isomorfismo.
- ★37. a. Seja $[S, \cdot]$ um semigrupo. Um isomorfismo de S em S é chamado um **automorfismo** em S . Seja $\text{Aut}(S)$ o conjunto de todos os automorfismos em S e mostre que $\text{Aut}(S)$ é um grupo sob a composição de funções.
b. Para o grupo $[\mathbb{Z}_4, +_4]$, encontre o conjunto de automorfismos e mostre sua tabela de grupo sob $\circ$.
38. Seja f um homomorfismo do grupo G sobre um grupo H . Mostre que f é um isomorfismo se, e somente se, o único elemento de G que é levado no elemento neutro de H é o elemento neutro de G .
39. Seja $[G, \cdot]$ um grupo e g um elemento fixo de G . Defina $f: G \rightarrow G$ por $f(x) = g \cdot x \cdot g^{-1}$ para qualquer $x \in G$. Prove que f é um isomorfismo de G em G .

Seção 8.2 Máquinas de Estado Finito

As estruturas algébricas da seção anterior servem de modelo para várias operações aritméticas simples. No entanto, sabemos que a computação deve ir bem além de mera aritmética. Gostaríamos de ter um modelo que capturasse a natureza geral da computação. Talvez possamos começar por tomar versões simplificadas de um computador digital moderno.

Um computador armazena as informações internamente de forma binária. A qualquer momento, o computador contém determinadas informações, de forma que sua memória interna contém algum padrão de dígitos binários, que chamaremos de *estado* do computador nesse momento. Como um computador contém uma quantidade finita de memória, existe um número finito (apesar de grande) de diferentes estados que ele pode assumir. Um relógio interno sincroniza as ações do computador. Em um ciclo do relógio, a entrada pode ser lida, o que pode mudar algumas das posições de memória e, portanto, mudar o estado da máquina para um novo estado. O que o novo estado representa depende da entrada, bem como do estado anterior. Se esses dois fatores forem conhecidos, a alteração é previsível e não aleatória. Como o conteúdo de certas células são disponíveis como saída, o estado da máquina determina sua saída. Desta forma, ao cabo de uma sucessão de ciclos do relógio, a máquina produz uma sequência de saídas em resposta a uma sequência de entradas.

Definição

A máquina de estado finito é um modelo que retrata as características do computador descrito acima. Ao ler a definição, perceba as seguintes propriedades do comportamento de nossa máquina abstrata:

1. As operações da máquina são *sincronizadas* por ciclos discretos do relógio.
2. A máquina procede de uma forma *determinística*; isto é, suas ações em resposta a uma dada sequência de entrada são completamente previsíveis.
3. A máquina responde a *entradas*.
4. Existe um *número finito de estados* que a máquina pode alcançar. A qualquer momento, a máquina está exatamente em um desses estados. Qual o estado no qual ela estará a seguir é uma função do

estado atual e da entrada atual. O estado atual, no entanto, depende dos estados e entradas anteriores, enquanto que o estado anterior depende de *seus* estados e entradas anteriores e assim por diante, até chegarmos de volta à configuração inicial. Portanto, o estado da máquina a qualquer momento serve como uma espécie de memória das entradas anteriores.

5. A máquina é capaz de produzir *saídas*. A natureza da saída é uma função do estado atual da máquina, o que significa que também depende das entradas anteriores.

Definição: Máquina de Estado Finito

$M = [S, I, O, f_s, f_o]$ é uma **máquina de estado finito** se S for um conjunto finito de estados, I for um conjunto finito de símbolos de entrada (o **alfabeto de entrada**), O for o conjunto finito de símbolos de saída (o **alfabeto de saída**) e f_s e f_o forem funções onde $f_s: S \times I \rightarrow S$ e $f_o: S \rightarrow O$. A máquina é sempre iniciada a fim de começar em um estado inicial fixo s_0 .

A função f_s é a função do próximo estado. Ela leva pares (estado, entrada) em estados. Portanto, o estado no ciclo de relógio t_{i+1} , estado (t_{i+1}) , é obtido pela aplicação da função do próximo estado ao estado no ciclo t_i e à entrada do ciclo t_i :

$$\text{estado } (t_{i+1}) = f_s(\text{estado } (t_i), \text{entrada } (r_i))$$

A função f_o é a função de saída. Quando f_o é aplicada a um estado no ciclo t_i , obtemos a saída do ciclo t_i .

$$\text{saída } (t_i) = f_o(\text{estado } (t_i))$$

Perceba que o resultado da função f_o é obtido instantaneamente, mas o efeito da aplicação da função f_s só se faz sentir no ciclo seguinte.

Exemplos de Máquinas de Estado Finito

Para descrever uma máquina de estado finito em particular, precisamos definir os três conjuntos e as duas funções que a definem.

EXEMPLO 16

Uma máquina de estado finito M é descrita da seguinte maneira: $S = \{s_0, s_1, s_2\}$, $I = \{0, 1\}$, $O = \{0, 1\}$. Como as duas funções f_s e f_o atuam em domínios finitos, elas podem ser definidas através de tabelas de estados, como na Tabela 8.1. A máquina M começa no estado s_0 , que tem uma saída 0. Se o primeiro símbolo de entrada for um 0, o próximo estado da máquina é s_1 , que tem uma saída 1.

Estado atual	Próximo Estado		Saída
	Entrada atual 0	1	
s_0	s_1	s_0	0
s_1	s_2	s_1	1
s_2	s_2	s_0	1

Tabela 8.1

Se o próximo símbolo de entrada for 1, a máquina permanece no estado s_1 , com saída 1. Continuando esta análise, vemos que uma sequência de entrada que consista em caracteres 01101 (lidos da esquerda para a direita) produz o seguinte efeito:

Ciclo	t_0	t_1	t_2	t_3	t_4	t_5
Entrada	0	1	1	0	1	--
Estado	s_0	s_1	s_1	s_1	s_2	s_0
Saída	0	1	1	1	1	0

O 0 inicial da cadeia de saída é supérfluo — ele apenas reflete o estado inicial, e não o resultado de qualquer entrada.

De modo semelhante, a seqüência de entrada 1010 produz a saída 00111.

Outra maneira de definir as funções f_s e f_o (na verdade, toda M) é através de um grafo direcionado chamado de **grafo de estados**. Cada estado de M com suas saídas correspondentes é o rótulo do vértice do grafo. A função de próximo estado é dada pelas arestas direcionadas do grafo, com cada aresta mostrando o(s) símbolo(s) de entrada que produz(em) a mudança de estado que ela representa. O grafo de estados de M aparece na Fig. 8.1.

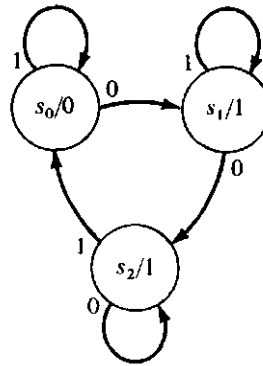


Figura 8.1

PRATICA 34 Qual a seqüência de saída produzida pela máquina M do Exemplo 16 para a entrada 11001 ?

PRATICA 35 Uma máquina M é dada pelo grafo de estados da Fig. 8.2. Forneça a tabela de estados de M .

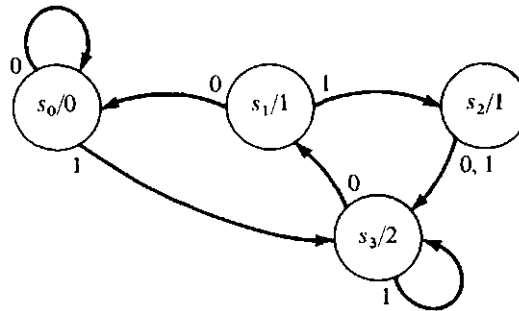


Figura 8.2

PRATICA 36 Uma máquina M é descrita pela tabela de estados mostrada na Tabela 8.2.

- Desenhe o grafo de estados de M .
- Qual a saída correspondente à seqüência de entrada 2110?

Estado Normal	Próximo estado			Saída
	Entrada atual			
	0	1	2	
s_0	s_0	s_1	s_1	0
s_1	s_1	s_0	s_0	1

Tabela 8.2

A máquina do Exemplo 16 não é particularmente interessante. Se as máquinas de estado finito modelam os computadores do mundo real, elas devem ser capazes de realizar alguma coisa. Vamos tentar construir uma máquina de estado finito que some dois números binários. A entrada consiste em uma seqüência de pares de dígitos binários, cada um da forma 00, 01, 10 ou 11. Esses representam os bits do primeiro dos dois números a serem somados, lidos da direita para a esquerda. Portanto, os bits menos significativos dos números a serem

somados são fornecidos primeiro, e a saída fornece os bits menos significativos da resposta em primeiro lugar. Lembremo-nos dos fatos básicos acerca da adição binária:

0	0	1	1
<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>
0	1	1	10

(Perceba que na quarta adição há um vai-um para a próxima coluna.)

Se pensarmos um segundo, veremos que pode haver quatro casos na adição de dois bits em qualquer coluna: (1) a saída deve ser 0, sem vai-um; (2) a saída deve ser 0 mas com um vai-um para a coluna seguinte; (3) a saída deve ser 1 sem vai-um; e (4) a saída deve ser 1 com um vai-um para a próxima coluna. Representaremos estes casos pelos estados s_0 , s_1 , s_2 e s_3 , respectivamente; s_0 , como sempre, é o estado inicial. Já indicamos a saída de cada estado, mas precisamos determinar o próximo estado baseado no estado atual e na entrada. Por exemplo, suponha que estamos no estado s_1 e a entrada é 11. A saída para o estado atual é 0, mas há um vai-um, então na próxima coluna estaremos somando $1 + 1 + 1$, que resulta em uma saída 1 e um vai-um. O próximo estado é s_3 .

PRÁTICA 37 No somador binário que estamos construindo:

- Qual é o próximo estado para o estado atual s_2 e a entrada 11?
- Qual é o próximo estado para o estado atual s_3 e a entrada 10?

Após considerarmos todos os casos, temos o grafo de estados completo da Fig. 8.3. A operação desta máquina ao somar dois números 011 e 101 (começando pelos bits de ordem baixa) pode ser acompanhada como a seguir:

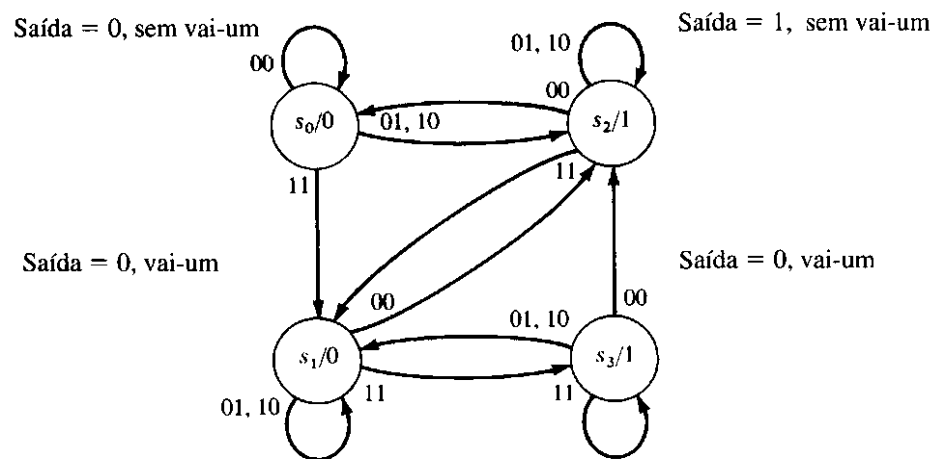


Figura 8.3

Ciclo	t_0	t_1	t_2	t_3	t_4
Entrada	11	10	01	00	
Estado	s_0	s_1	s_1	s_1	s_2
Saída	0	0	0	0	1

A saída é 1000 quando ignoramos o 0 inicial, que não reflete a ação referente a qualquer entrada. Se convertermos esta aritmética para a forma decimal, vemos que calculamos $3 + 5 = 8$. Perceba a simetria desta máquina com relação às entradas 10 e 01, refletindo que a adição binária é comutativa.

PRÁTICA 38 Calcule a soma de 01110110 e 01010101 usando a máquina do somador binário da Fig. 8.3.

Reconhecimento

Já percebemos que uma determinada entrada pode afetar o comportamento de uma máquina de estado finito por mais do que apenas um ciclo do relógio. Devido a memória (limitada) das entradas anteriores representadas pelos estados da máquina, podemos usar estas máquinas para *reconhecer* ou *aceitar* entradas. Podemos construir uma máquina para reconhecer, digamos produzindo uma saída 1, se a entrada fornecida corresponde a uma certa descrição. (Esta operação é essencialmente a realizada por um analisador léxico, ou scanner, em um compilador. Cadeias de entradas são quebradas em subcadeias reconhecíveis, e as subcadeias são então tratadas como unidades no próximo estágio do processo de compilação. O analisador léxico deve ser capaz de reconhecer, por exemplo, uma sequência de símbolos de entrada que constituem uma palavra chave ou uma palavra reservada na linguagem de programação.) Em breve discutiremos com mais detalhes as capacidades de máquinas de estados finitos para fins de reconhecimento. Agora apenas construiremos alguns exemplos.

EXEMPLO 17 A máquina descrita na Fig. 8.4 é uma máquina para verificação de paridade. Quando a entrada recebida no ciclo t_i , contém um número par de 1s, então a saída no tempo t_{i+1} é 1; caso contrário é 0.

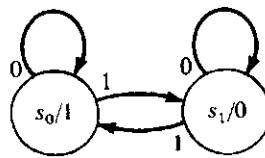


Figura 8.4

EXEMPLO 18 Suponha que desejamos projetar uma máquina que tenha uma saída 1 exatamente quando a cadeia recebida até o presente momento termina por 101. Como um caso especial, uma cadeia que consista apenas em 101 deve ser tratada processando-se diretamente do estado inicial s_0 para os estados s_1 , s_2 e s_3 com saídas 0 para todos esses estados, exceto s_3 , cuja saída é 1. Esta parte do projeto resulta na Fig. 8.5a. Esta figura mostra que desejamos estar no estado s_2 sempre que mais uma entrada 1 deva nos levar ao estado s_3 (com saída 1); portanto, devemos estar em s_2 sempre que as duas últimas entradas tiverem sido 10, independentemente das entradas anteriores. Em particular, uma cadeia 1010 deve nos levar ao estado s_2 ; portanto, a função de próximo estado para o estado s_3 e entrada 0 vale s_2 . Analogamente, podemos usar s_1 para nos lembrar que o último símbolo da entrada que foi recebido foi 1 e que uma entrada 01 nos levaria ao estado s_3 . Em particular, 1011 deve nos levar ao estado s_1 ; portanto a função próximo estado para s_3 com entrada 1 vale s_1 . O restante da função do próximo estado pode ser determinado da mesma forma; a Fig. 8.5b mostra o grafo de estados completo.

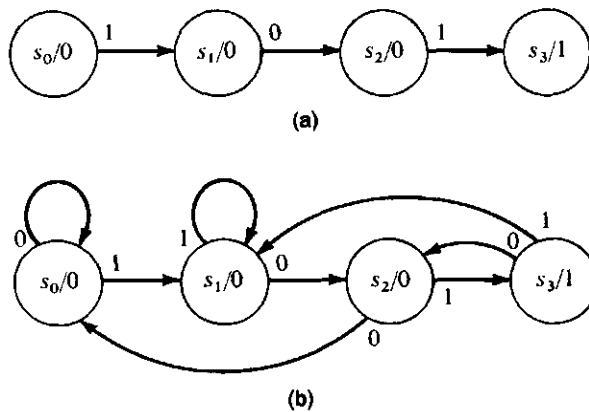


Figura 8.5

Perceba que a máquina está no estado s_2 ao fim da entrada 0110 e também ao fim da entrada 011010 — na verdade, ao fim de qualquer entrada terminando por 10; ainda que s_2 não seja capaz de distinguir entre essas

entradas. Cada estado de M representa uma classe de histórico de entradas indistinguíveis, sendo que s_3 representa o estado de todas as entradas terminadas por 101.

PRÁTICA 39 Desenhe o grafo de estados para uma máquina que produza 1 como saída, exatamente quando a cadeia de entrada recebida termina por 00.

Agora desejamos ver exatamente quais conjuntos as máquinas de estado finito podem aceitar. Lembremos de que a aceitação é possível porque os estados da máquina têm uma memória limitada das entradas anteriores. Apesar de a máquina ser finita, um sinal de entrada particular pode afetar o comportamento da máquina para sempre. No entanto, nem todo sinal de entrada pode fazer isto e algumas classes de entradas precisam que tantas informações sejam guardadas que nenhuma máquina pode detectá-las.

Para evitar escrever saídas, vamos designar estes estados de uma máquina de estado finito cuja saída é 1 por **estados finais** e representá-los no grafo de estados através de um círculo duplo. Podemos, então, dar a seguinte definição formal de reconhecimento, onde I^* denota o conjunto de cadeias de comprimento finito sobre o alfabeto de entrada.

Definição: Reconhecimento por Máquina de Estado Finito

Uma máquina de estado finito M com alfabeto de entrada / reconhece ou aceita um subconjunto S de I^* se M , começando no estado s_0 e processando uma cadeia de entrada a , termina em um estado final se, e somente se, $a \in S$.

PRÁTICA 40 Descreva os conjuntos reconhecidos pelas máquinas da Fig. 8.6.

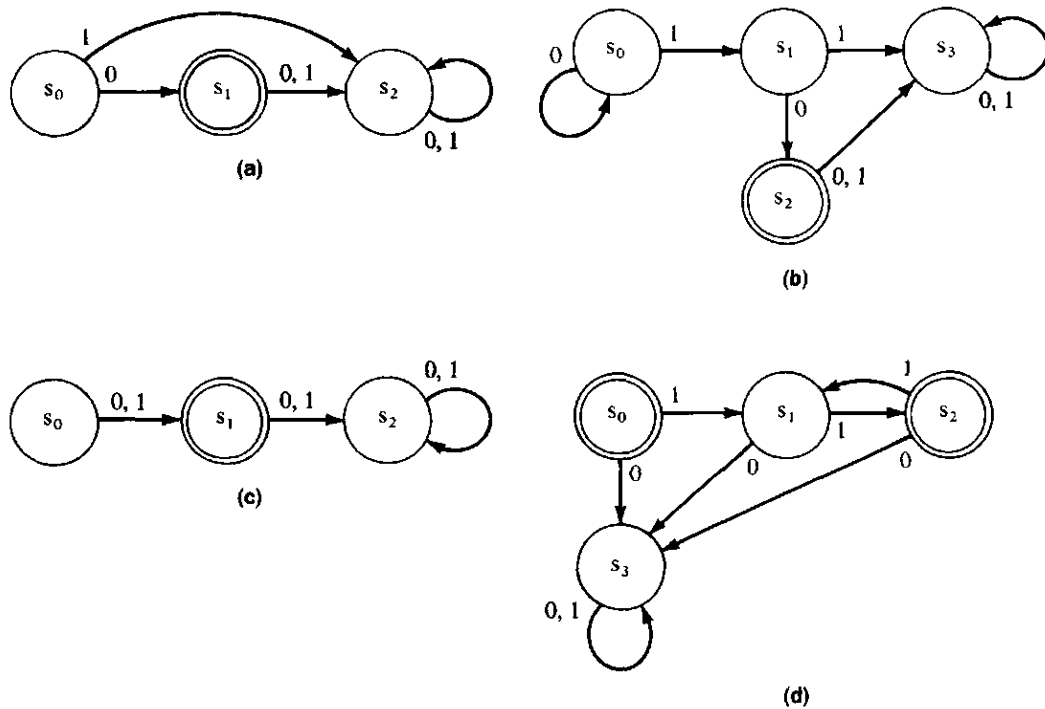


Figura 8.6

Conjuntos Regulares e o Teorema de Kleene

Desejamos uma forma compacta e simbólica para descrever os conjuntos como os que aparecem na resposta da Prática 40. Descreveremos estes conjuntos através de *expressões regulares*; cada expressão regular descreve um conjunto em particular. Primeiro, definiremos o que são as expressões regulares; em seguida veremos como uma expressão regular descreve um conjunto. Assumimos aqui que I é algum conjunto finito de símbolos; mais tarde / será o alfabeto de entrada de uma máquina de estado finito.

Definições: Expressões Regulares sobre I , Conjunto Regular
Expressões regulares sobre I são:

1. O símbolo $\emptyset$ e o símbolo A ,
2. O símbolo i para qualquer $i \in I$ e
3. As expressões (AB) , $(A \vee B)$ e $(A)^*$ se A e B são expressões regulares

(Esta definição de uma expressão regular sobre I é mais um exemplo de definição recursiva.)

Além disso, qualquer conjunto representado por uma expressão regular de acordo com as convenções estabelecidas abaixo é chamado de conjunto regular:

1. $\emptyset$ representa o conjunto vazio,
2. λ representa o conjunto $\{\lambda\}$ contendo a cadeia vazia,
3. i representa $\{i\}$,
4. Para as expressões regulares A e B :
 - a. (AB) representa o conjunto de todos os elementos da forma $\alpha\beta$ onde « α » pertence ao conjunto representado por A e « β » pertence ao conjunto representado por B .
 - b. $(A \vee B)$ representa a união dos conjuntos A e B .
 - c. $(A)^*$ representa o conjunto de todas as concatenações dos elementos do conjunto A .

Percebemos que λ , a cadeia vazia, é um elemento do conjunto representado por A^* . Ao escrevermos expressões regulares, podemos eliminar os parênteses quando isto não causar ambigüidades.

Em nosso texto, seremos um pouco negligentes, e diremos o "conjunto regular" $0^* \vee 10$ ao invés de "o conjunto representado pela expressão regular $0^* \vee 10$ ".

EXEMPLO 19 Eis algumas expressões regulares e uma descrição dos conjuntos que cada uma representa.

- | | |
|---------------------------|--|
| a. $1^*0(01)^*$ | a'. Qualquer número (incluindo nenhum) de 1 s, seguido por um simples 0, seguido por qualquer número (incluindo nenhum) de pares 01. |
| b. $0 \vee 1^*$ | b'. Um único 0 ou qualquer número (incluindo nenhum) de 1s. |
| c. $(0 \vee 1)^*$ | c'. Qualquer cadeia de 0s e 1s, incluindo λ . |
| d. $11((10)^*11)^*(00)^*$ | d'. Uma cadeia não-vazia de pares de 1s intercalados por qualquer número (incluindo nenhum) de pares 10, seguido por pelo menos um 0 |

PRÁTICA 41 Quais cadeias pertencem ao conjunto descrito pelas expressões regulares?

- a. 10100010; $(0^*10)^*$
- b. 011100; $(0 \vee (11)^*)^*$
- c. 000111100; $((011 \vee 11)^*(00)^*)^*$

PRÁTICA 42 Escreva expressões regulares para os conjuntos aceitos pelas máquinas da Prática 40.

Um conjunto regular pode ser descrito por mais de uma expressão regular. Por exemplo, o conjunto de todas as cadeias de 0s e 1 s, que já sabemos do Exemplo 19c que pode ser descrito por $(0 \vee 1)^*$ também é descrita pela expressão regular $[(0 \vee 1^*)^* \vee (01)^*]^*$. Podemos, então, escrever a equação

$$(0 \vee 1)^* = [(0 \vee 1^*)^* \vee (01)^*]^*$$

Apesar de podermos aceitar com facilidade esta equação em particular, pode ser muito difícil decidir em geral quando duas expressões regulares são iguais, isto é, quando elas representam o mesmo conjunto. Um algoritmo eficiente para esta tarefa ainda não foi encontrado.

Apresentamos os conjuntos regulares porque, como veremos adiante, eles são exatamente os conjuntos que as máquinas de estado finito podem reconhecer. Este resultado foi demonstrado pela primeira vez pelo matemático americano Stephen Kleene, em 1956. Enunciamos seu teorema abaixo, mas não apresentamos a demonstração.

Teorema de Kleene

Qualquer conjunto reconhecido por uma máquina de estado finito é regular, e qualquer conjunto regular pode ser reconhecido por uma máquina de estado finito.

O teorema de Kleene estabelece as limitações, bem como as capacidades, de máquinas de estado finito, pois existem certamente diversos conjuntos que não são regulares. Por exemplo, $S = \{0^n 1^n \mid n \geq 0\}$ não é regular onde a^n significa uma cadeia com n cópias de a . (Perceba que 0^*1^* não faz a mesma coisa.) Pelo teorema de Kleene, não há máquina de estado finito capaz de reconhecer S , a despeito de S se parecer um conjunto simples e, certamente, nós humanos sermos capazes de contar uma cadeia de Os seguida de uma cadeia de Is para então verificar se o número de Os é o mesmo do de Is. Esta falha sugere uma deficiência em nosso uso de máquinas de estado finito como modelos para a computação. Investigaremos isto mais adiante na Seção 8.3.

Minimização da Máquina

Apesar de termos tratado as máquinas de estado finito como abstrações, dispositivos eletrônicos (como os elementos lógicos da Seção 7.2 e outros) podem ser usados para construir circuitos que funcionem como máquinas de estado finito. Se desejarmos construir uma máquina física, o número de estados internos é um fator de custo para a construção. A minimização é o processo para encontrar, para uma dada máquina de estado finito M , uma máquina M' com duas propriedades:

1. Se M e M' são ambas iniciadas em seus respectivos estados iniciais e recebem a mesma sequência a de símbolos de entrada, elas devem produzir a mesma sequência de saída.
2. M' tem, se possível, menos estados que M (se não for possível, M já é uma máquina mínima e não pode ser mais reduzida).

Estados Inalcançáveis

Primeiro, vamos observar que podemos remover quaisquer **estados inalcançáveis** de M , os estados que não podem ser alcançados a partir do estado inicial independentemente de que sequência de entrada ocorra.

EXEMPLO 20

Seja M dada pela tabela de estados da Tabela 8.3. Apesar da tabela de estados conter a mesma informação do grafo de estados (Fig. 8.7), o grafo nos mostra de forma imediata que o estado s_2 nunca pode ser alcançado a partir do estado s_0 . Se simplesmente removermos o estado s_2 , teremos o grafo de estados da Fig. 8.8 para uma máquina M com um estado a menos que M e que funciona exatamente como M ; isto é, fornece a mesma saída que M para uma mesma cadeia de entrada.

Estado atual	Próximo Estado		Saída
	Entrada atual 0	1	
s_0	s_1	s_3	0
s_1	s_3	s_0	0
s_2	s_1	s_3	1
s_3	s_0	s_1	1

Tabela 8.3

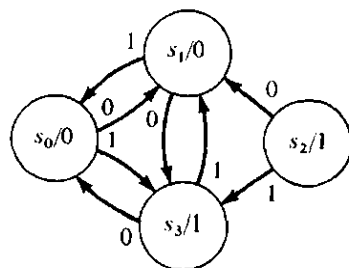


Figura 8.7

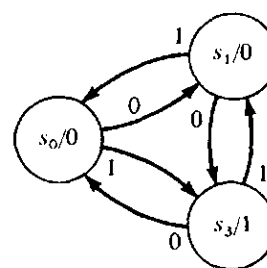


Figura 8.8

PRÁTICA 43 Quais estados são inalcançáveis a partir de s_0 na máquina descrita pela Tabela 8.4? Tente obter sua resposta diretamente da tabela de estados.

Estado atual	Próximo Estado		Saída
	Entrada atual 0	1	
s_0	s_1	s_4	0
s_1	s_4	s_1	1
s_2	s_2	s_2	1
s_3	s_3	s_1	0
s_4	s_0	s_0	1

Tabela 8.4

Como o grafo de estados de uma máquina de estado finito é um grafo direcionado, ele tem uma matriz de adjacências associada. O algoritmo de Warshall (Seção 5.3) pode ser usado para detectar estados inalcançáveis.

Um Procedimento para Minimização

Assumindo que todos os estados inalcançáveis foram removidos de M , podemos continuar a procurar por uma máquina reduzida M' . A chave para se encontrar uma M' reduzida, se existir, é a noção de estados equivalentes.

Definição: Estados Equivalentes

Dois estados s_i e s_j de M são equivalentes se para qualquer $\alpha \in I^*$, $f_o(s_i, \alpha) = f_o(s_j, \alpha)$, onde I^* mais uma vez denota o conjunto das cadeias de comprimento finito sobre o alfabeto de entrada.

Nesta definição de estados equivalentes, a definição de função de saída foi estendida a fim de denotar a sequência de símbolos de saída obtida pela aplicação repetida de f_o , a uma sequência a de símbolos de entrada. Portanto, estados equivalentes de um procedimento da máquina produzem cadeias de saídas idênticas para qualquer cadeia de entrada.

PRÁTICA 44 Demonstre que a equivalência de estados é uma relação de equivalência nos estados de uma máquina

Por ora, vamos adiar o problema de como identificar os estados equivalentes de uma dada máquina. Vamos simplesmente admitir que tenhamos, de alguma forma, encontrado os estados que são equivalentes e tenhamos particionado os estados de M em classes de equivalência. Essas classes têm duas propriedades: (1) Todos os estados na mesma classe têm a mesma saída, e (2) para cada símbolo de entrada, todos os estados na mesma classe são levados pela função próximo estado em estados que estão na mesma classe.

PRÁTICA 45 Mostre que as propriedades (1) e (2) são satisfeitas quando M é particionada em classes de estados equivalentes.

Definimos uma máquina M' cujos estados são as classes de equivalência de M . M' tem os mesmos alfabetos de entrada e saída que M , e seu estado inicial é a classe que contém s_0 o estado inicial de M . A saída de uma classe é o símbolo de saída comum a todos os estados de M em sua classe (propriedade 1). O próximo estado da classe X devido a um símbolo de entrada é a classe que contém todos os estados seguintes a todos os estados da classe X devidos ao mesmo símbolo de entrada (propriedade 2). M' é uma máquina bem-definida e produz as mesmas cadeias de saída ao processar uma dada cadeia de entrada que M . Além disso, o número de estados de M' (classes de equivalência de M) não será maior que o número de estados de M .

O problema de minimização de M reduz-se, portanto, ao problema de encontrar os estados equivalentes de M . Podemos constatar que a abordagem óbvia de tentar satisfazer diretamente a definição de estados equivalentes não funciona. Dados dois estados s_i e s_j de M , não podemos comparar de fato as saídas correspondentes a cada cadeia de entrada possível. Felizmente, o problema não é tão infinito quanto parece; precisamos apenas identificar os estados k -equivalentes.

Definição: Estados k -Equivalentes

Dois estados s_i e s_j de M são k -equivalentes se para qualquer $\alpha \in I^k$ onde α não tem mais do que k símbolos, $f_o(s_i, \alpha) = f_o(s_j, \alpha)$.

Está claro que a k -equivalência é uma relação de equivalência nos estados de M . É possível verificar diretamente se dois estados de M são k -equivalentes, uma vez que podemos produzir o número finito de cadeias de entrada contendo no máximo k símbolos. No entanto ocorre que não precisamos fazer isto. Podemos começar encontrando os estados 0-equivalentes, que são os estados que produzem a mesma saída para cadeias de entrada de tamanho 0, isto é, estados que contêm os mesmos símbolos de saída. Portanto, podemos definir as classes de 0-equivalência diretamente da definição de M .

EXEMPLO 21 Seja M definida pela tabela de estados da Tabela 8.5. (Nesta tabela, escrevemos 0, 1, 2, ... para denotar os estados, no lugar de usar $s_0, s_1, s_2, \dots$.) As classes de 0-equivalência dos estados são

$$\{0, 2, 5\} \text{ e } \{1, 3, 4, 6\}$$

Estado atual	Próximo Estado		Saída
	Entrada atual		
	0	1	
0	2	3	0
1	3	2	1
2	0	4	0
3	1	5	1
4	6	5	1
5	2	0	0
6	4	0	1

Tabela 8.5

Nosso procedimento para encontrar os estados k -equivalentes é recursivo; sabemos como encontrar os estados 0-equivalentes e mostraremos como encontrar os estados k -equivalentes, uma vez que já tenhamos os estados $(k - 1)$ -equivalentes. Suponha, então, que já sabemos quais estados são $(k - 1)$ -equivalentes. Se os estados s_i e s_j são k -equivalentes, eles devem produzir a mesma saída para qualquer cadeia de entrada com no máximo k símbolos, em particular, para qualquer cadeia com no máximo $k - 1$ símbolos. Portanto, s_i e s_j devem ser pelo menos $(k - 1)$ -equivalentes. Mas eles precisam também produzir a mesma saída para cadeias de entrada com comprimento k .

Uma cadeia de entrada arbitrária de comprimento k consiste em um único símbolo de entrada seguido de uma cadeia de entrada de comprimento $k - 1$. Se aplicarmos uma cadeia de entrada de comprimento k aos estados s_i e s_j (que têm os mesmos símbolos de saída), o primeiro símbolo de entrada altera os estados s_i e s_j para s_i' e s_j' , então s_i' e s_j' devem produzir cadeias de saídas idênticas para os demais $k - 1$ símbolos da cadeia de entrada, o que certamente acontecerá se s_i' e s_j' forem $(k - 1)$ -equivalentes. Portanto, *para encontrar estados k -equivalentes, buscamos estados $(k - 1)$ -equivalentes cujos próximos estados devidos a qualquer símbolo de entrada são $(k - 1)$ -equivalentes.*

EXEMPLO 22 Considere novamente a máquina M do Exemplo 21. Sabemos quais são os estados 0-equivalentes. Para encontrar os estados 1-equivalentes, procuramos os estados 0-equivalentes cujos próximos estados sejam também 0-equivalentes. Por exemplo, os estados 3 e 4 são 0-equivalentes; sob o símbolo de entrada 0, eles passam aos estados 1 e 6, respectivamente, que são estados 0-equivalentes e, sob o símbolo de entrada 1, ambos passam ao estado 5 que é, naturalmente, 0-equivalente a ele mesmo. Portanto, os estados 3 e 4 são 1-equivalentes. Mas os estados 0 e 5, que são 0-equivalentes, sob o símbolo de entrada 1, passam aos estados 3 e 0, respectivamente, que não são 0-equivalentes. Portanto os estados 0 e 5 não são 1-equivalentes; a cadeia de entrada 1 produz uma saída 01 a partir do estado 0 e 00 a partir do estado 5. As classes de 1-equivalência de M são

$$\{0, 2\}, \{5\}, \{1, 3, 4, 6\}$$

Para encontrar os estados 2-equivalentes, procuramos por estados 1-equivalentes com estados seguintes também 1-equivalentes. Portanto, os estados 1 e 3, apesar de 1-equivalentes, sob a entrada 1, passam aos esta-

dos 2 e 5, respectivamente, que não são estados 1-equivalentes. Portanto, os estados 1 e 3 não são 2-equivalentes. As classes de 2-equivalência de M são:

$$\{0,2\}, \{5\}, \{1,6\}, \{3,4\}$$

As classes de 3-equivalência de M são as mesmas que as classes de 2-equivalência.

Definição: Refinamento de uma Partição

Dadas duas partições π_1 e π_2 de um conjunto S , π_1 é um refinamento de π_2 se cada bloco de π_1 for um subconjunto de um bloco de π_2 .

No Exemplo 22, cada partição sucessiva dos estados de M em classes de equivalência é um refinamento da partição anterior. Este refinamento sempre acontece; estados k -equivalentes também precisam ser $(k - 1)$ -equivalentes, de forma que os blocos da partição $k - 1$ podem ser mais subdivididos. No entanto, o processo de subdivisão não pode continuar indefinidamente (no pior caso, ele pode prosseguir até que cada bloco da partição contenha apenas um estado); em algum momento os estados $(k - 1)$ -equivalentes e k -equivalentes coincidirão. (No Exemplo 22, os estados 2-equivalentes e 3-equivalentes coincidem.) Uma vez que isto ocorra, todos os estados seguintes dos elementos de um bloco da partição sob qualquer símbolo de entrada estão dentro de um bloco da partição. Portanto, os estados k -equivalentes são também estados $(k + 1)$ -equivalentes e $(k + 2)$ -equivalentes e assim por diante. Portanto, esses estados são equivalentes.

O procedimento total para encontrar estados equivalentes deve começar com os estados 0-equivalentes, então achar os estados 1-equivalentes e assim por diante, até que a partição não seja mais subdividida. Uma pseudodescrição deste algoritmo é dada a seguir.

ALGORITMO *Minimiza*

procedure Minimiza (M : tabela da máquina de estado finito);
{produz uma versão minimizada de M }

var

flag: boolean; {flag para saída do laço quando estados não-equivalentes são encontrados}

begin

encontre os estados 0-equivalentes de M ;

repeat

while existem classes ainda não-testadas **do**

begin

selecione classe de equivalência ainda não-testada;

while ainda existem pares de estados não-testados **do**

begin

selecione um par de estados não-testado na classe atual;

flag := false;

while ainda existem símbolos de entrada não-testados **and not** *flag* **do**

begin

selecione um símbolo de entrada ainda não-testado;

para ambos os estados no par atual, encontre o próximo estado sob o símbolo de entrada atual;

if próximos estados não são equivalentes **then**

flag := true;

end;

if *flag* **then**

marque os estados correntes como classes diferentes;

end;

forme novas classes de equivalência;

end;

until o novo conjunto de classes de equivalência = o conjunto anterior de classes de equivalência;

end;

EXEMPLO 23 Para a máquina M dos Exemplos 21 e 22, a máquina reduzida M' terá estados

$$\begin{aligned} A &= \{0,2\} \\ B &= \{5\} \\ C &= \{1,6\} \\ D &= \{3,4\} \end{aligned}$$

A tabela de estados de M' (Tabela 8.6) é obtida a partir da de M . A máquina M' (que é iniciada no estado A) reproduz a saída de M para qualquer cadeia de entrada, mas tem quatro estados, ao invés de sete.

Estado atual	Próximo Estado		Saída
	Entrada atual		
	0	1	
A	A	D	0
B	A	A	0
C	D	A	1
D	C	B	1

Tabela 8.6

EXEMPLO 24 Vamos minimizar M onde M é dada pela tabela de estados da Tabela 8.7. As classes de 0-equivalência de M são

$$\{0,2,4\}, \{1,3\}$$

As classes de 1-equivalência de M são

$$\{0\}, \{2,4\}, \{1,3\}$$

Estado atual	Próximo Estado		Saída
	Entrada atual 0	1	
0	3	1	1
1	4	1	0
2	3	0	1
3	2	3	0
4	1	0	1

Tabela 8.7

Não é possível qualquer refinamento. Seja

$$\begin{aligned} A &= \{0\} \\ B &= \{2,4\} \\ C &= \{1,3\} \end{aligned}$$

A máquina reduzida é mostrada na Tabela 8.8.

Estado atual	Próximo Estado		Saída
	Entrada atual		
	0	1	
A	C	C	1
B	C	A	1
C	B	C	0

Tabela 8.8

PRÁTICA 46 Minimize as máquinas cujas tabelas de estados são mostradas nas Tabelas 8.9 e 8.10.

Estado atual	Próximo Estado		Saída
	Entrada atual		
	0	1	
0	2	1	1
1	2	0	1
2	4	3	0
3	2	3	1
4	0	1	0

Tabela 8.9

Estado atual	Próximo Estado		Saída
	Entrada atual		
	0	1	
0	1	3	1
1	2	0	0
2	1	3	0
3	2	1	0

Tabela 8.10

Revisão da Seção 8.2

Técnicas

- Avaliar a cadeia de saída de uma dada máquina de estados finitos e uma dada cadeia de entrada
- Desenhar um grafo de estado a partir de uma tabela de estados e vice-versa
- Construir uma máquina de estado finito para aceitar um determinado tipo de entrada
- Encontrar uma expressão regular dada a descrição do conjunto regular
- Determinar se uma dada cadeia pertence ou não a um conjunto regular
- Minimizar máquinas de estado finito.

Idéias Principais

Máquinas de estado finito têm um modo de operação síncrono e determinístico, e capacidade de memória limitada.

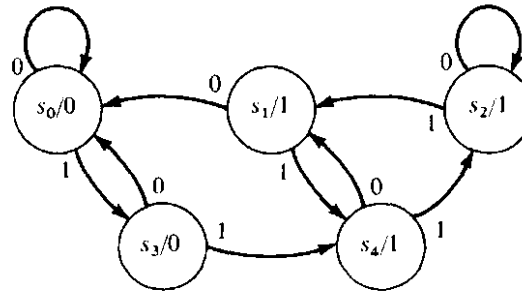
A classe dos conjuntos que as máquinas de estado finito podem reconhecer é a classe de todos os conjuntos regulares; portanto, sua capacidade de reconhecimento é limitada.

Estados inalcançáveis podem ser removidos da máquina.

Após removermos os estados inalcançáveis de uma máquina, podemos encontrar uma versão minimizada da máquina que produza as mesmas cadeias de saída para qualquer entrada.

Exercícios 8.2

- ★1. Para cada seqüência de entrada e máquina dadas, determine a seqüência de saída correspondente (o estado inicial é sempre s_0).
 - a. 011011010

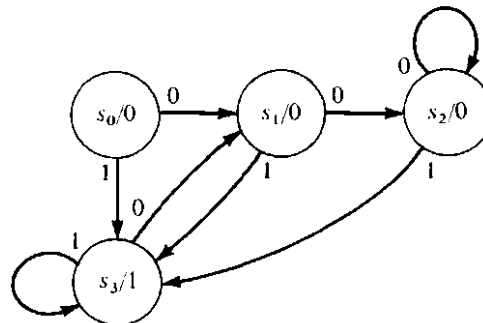


b. *abccaab*

Estado atual	Próximo estado			Saída
	Entrada atual <i>abc</i>			
s_0	s_2	s_0	s_3	a
s_1	s_0	s_2	s_3	b
s_2	s_2	s_0	s_1	a
s_3	s_1	s_2	s_0	c

Tabela 8.2

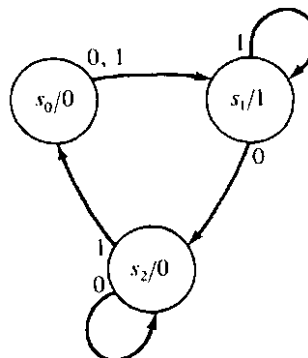
c. 0100110



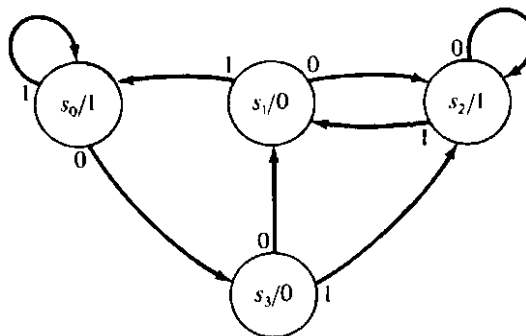
2. a. Para a máquina descrita no Exercício 1(a), encontre todas as seqüências de entrada que geram uma seqüência de saída 0011110.
- b. Para a máquina descrita no Exercício 1(b), encontre todas as seqüências de entrada que geram uma seqüência de saída *abaaca*.
- c. Para a máquina descrita no Exercício 1(c), qual será a saída para uma seqüência de entrada $a_1a_2a_3a_4a_5$ onde $a_i \in \{0, 1\}$, $1 \leq i \leq 5$?

Nos Exercícios 3 a 6, escreva a tabela de estados para a máquina e determine a seqüência de saída para a seqüência de entrada dada.

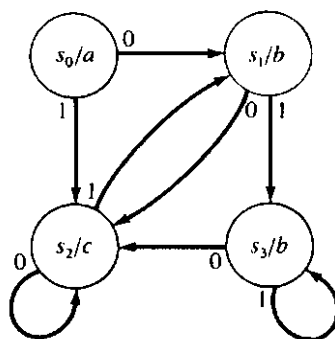
3. 00110



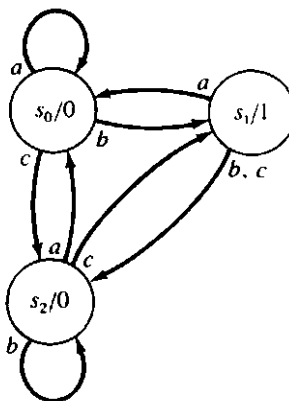
4. 1101100



★5. 01011



6. *acbab*c



Nos Exercícios 7 a 10, desenhe o grafo de estados para a máquina e determine a sequência de saída para a sequência de entrada dada

7. 10001

Estado atual	Próximo Estado		Saída
	Entrada atual 0	1	
s_0	s_0	s_2	1
s_1	s_1	s_0	0
s_2	s_0	s_1	0

8. 0011

Estado atual	Próximo Estado		Saída
	Entrada atual 0	1	
s_0	s_2	s_3	0
s_1	s_0	s_1	1
s_2	s_1	s_3	0
s_3	s_1	s_2	1

★9. *acbbca*

Estado atual	Próximo estado			Saída
	Entrada atual <i>abc</i>			
s_0	s_1	s_1	s_1	0
s_1	s_2	s_2	s_1	0
s_2	s_0	s_2	s_1	1

10. 21021

Estado atual	Próximo estado			Saída
	Entrada atual 0 1 2			
s_0	s_3	s_1	s_2	1
s_1	s_3	s_0	s_1	2
s_2	s_2	s_1	s_1	0
s_3	s_1	s_4	s_0	0
s_4	s_1	s_4	s_2	2

11. Construa uma máquina de estado finito que calcule $x + 1$ onde x é dado na forma binária, com o **bit**-menos significativo primeiro.
12. a. Construa uma máquina de estado finito que calcule o complemento a 2 de p , onde p é um número binário entrado com seus bits menos significativos primeiro. (Veja o Exercício 10, Seção 7.2.)
b. Use a máquina do item (a) para encontrar o complemento a 2 de 1100 e 1011.
- ★13. a. Construa uma *máquina de retardo* que tenha alfabetos de entrada e de saída $\{0, 1\}$ que, para qualquer seqüência de entrada $a_1a_2a_3 \dots$, produz uma seqüência de saída de $0a_1a_2a_3 \dots$.
b. Explique (intuitivamente) porque não podemos construir uma máquina de estado finito que, para qualquer entrada $a_1a_2a_3 \dots$, produza a seqüência de saída $0a_10a_20a_3 \dots$.
- ★14. Você tem uma conta no Banco Nacional do Lucro Certo (BNLC) e um cartão para operar sua conta bancária. Uma vez introduzido seu cartão, a máquina do banco permite processar uma transação apenas se você entrar seu código correto, que é 417. Desenhe um grafo de estados para uma máquina de estado finito desenhada para reconhecer este código. O alfabeto de saída deve ter três símbolos de saída: "bingo" (código correto), "aguarde" (correto também) e "inválido" (código incorreto). O alfabeto de entrada é $\{0, 1, 2, \dots, 9\}$. Para simplificar a notação, podemos nos referir a uma aresta por $I-\{3\}$, por exemplo, que significa que a máquina percorrerá esta aresta para qualquer símbolo de en-

trada que não seja o 3. (No BNLC, você tem apenas uma chance de entrar seu código corretamente.)

15. Construa máquinas de estado finito que aceitem as entradas dadas, produzindo uma saída 1 exatamente quando a entrada recebida satisfizer a descrição. (O alfabeto de entrada e saída em todos os casos é $\{0, 1\}$)
 - a. conjunto de todas as cadeias onde o número de 0s é múltiplo de 3
 - b. conjunto de todas as cadeias contendo pelo menos quatro 1s
 - c. conjunto de todas as cadeias contendo exatamente um 1
 - d. conjunto de todas as cadeias começando por 000
 - e. conjunto de todas as cadeias onde a segunda entrada é 0 e a quarta entrada é 1
 - f. conjunto de todas as cadeias que consistem exclusivamente em qualquer número (inclusive nenhum) de pares 01 ou que consistam exclusivamente em dois 1s seguidos por qualquer número (inclusive nenhum) de 0s
 - g. conjunto de todas as cadeias terminando por 110
 - h. conjunto de todas as cadeias contendo 00
16. Um parágrafo de texto em português deve ser analisado lexicamente e o número de palavras começando por "con" deve ser contado. Projete uma máquina de estado finito que gera como saída 1 sempre que uma palavra deste tipo é encontrada. O alfabeto de saída é $\{0, 1\}$. O alfabeto de entrada é constituído das 26 letras, um número finito de símbolos de pontuação (ponto, vírgula etc.) e um caracter especial β que representa o espaço. Para simplificar a descrição, você pode usar $I-\{m\}$ para denotar, por exemplo, qualquer símbolo de entrada exceto m .
17. a. Em uma certa linguagem de computador (BASIC, por exemplo), qualquer número decimal N precisa ser representado em uma das formas a seguir:

$$sd^* \quad sd^*.d^* \quad d^* \quad d^*.d^* \quad (1)$$

onde s denota o sinal (i.e., $s \in \{+, -\}$), d é um dígito (i.e., $d \in \{0, 1, 2, \dots, 9\}$), e d^* denota uma cadeia de dígitos onde a cadeia pode ser de qualquer tamanho, inclusive zero (a cadeia vazia). Portanto, a seguir, temos exemplos de representação de números decimais:

$$+ \quad 2.74 \quad -.58 \quad 129 \quad +$$

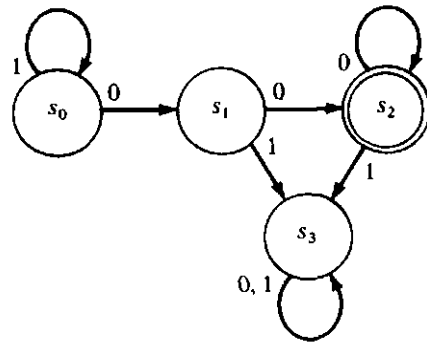
Projete uma máquina de estado finito que aceite números decimais válidos produzindo uma saída 1. Os símbolos de entrada são $+$, $-$, $.$ e os 10 dígitos. Para simplificar a notação, podemos usar d para denotar qualquer símbolo de entrada que seja um dígito.

- b. Modifique a máquina do item (a) para aceitar qualquer seqüência de números decimais, como definido no item (a), separados por vírgulas. Por exemplo, uma máquina deste tipo deve aceitar

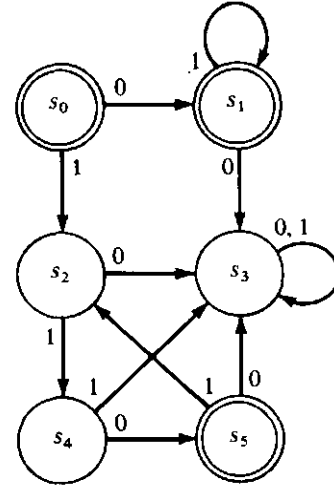
$$+ \quad 2.74, \quad -.58, \quad 129, \quad +$$

O alfabeto de entrada deve ser o mesmo do item (a) com a inclusão do símbolo c para representar a vírgula.

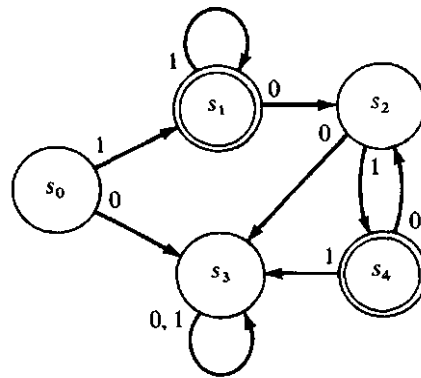
- c. Em Pascal, um número decimal deve ser representado de forma semelhante que em BASIC, exceto que o ponto decimal deve ser precedido por pelo menos um dígito. Escreva uma expressão semelhante à expressão (1) do item (a) para descrever a forma válida para um número decimal. Como você modificaria a máquina do item (a) para reconhecer este tipo de número?
- ★18. Seja M uma máquina de estado finito com n estados. O alfabeto de entrada é $\{0\}$. Mostre que para qualquer seqüência de entradas que seja grande o suficiente, a saída de M será, em algum momento, periódica. Qual o número máximo de entradas antes da parte periódica começar a ocorrer? Qual o tamanho máximo do período?
19. Forneça uma expressão regular para o conjunto aceito por cada uma das máquinas de estado finito da figura abaixo.



(a)



★ (b)



(c)

Exercício 19

20. Forneça uma expressão regular para o conjunto aceito por cada uma das máquinas de estado finito das tabelas abaixo.

Estado atual	Próximo Estado		Saída
	Entrada atual		
	0	1	
s_0	s_3	s_1	0
s_1	s_1	s_2	0
s_2	s_3	s_3	1
s_3	s_3	s_3	0

(a)

Estado atual	Próximo Estado		Saída
	Entrada atual 0 1		
s_0	s_3	s_1	1
s_1	s_1	s_2	1
s_2	s_2	s_2	0
s_3	s_0	s_2	0

(b)

Estado atual	Próximo Estado		Saída
	Entrada atual 0	1	
s_0	s_2	s_1	1
s_1	s_3	s_1	1
s_2	s_3	s_4	0
s_3	s_3	s_3	0
s_4	s_5	s_3	0
s_5	s_2	s_3	1

(c)

21. Forneça uma expressão regular para os seguintes conjuntos:
- conjunto de todas as cadeias de Os e ls começando por 0 e terminando por 1
 - conjunto de todas as cadeias de Os e 1 s que tenham um número ímpar de Os
- ★c. {101, 1001, 10001, 100001,...}
- conjunto de todas as cadeias de Os e ls contendo pelo menos um 0
 - conjunto de todas as cadeias de as e bs onde cada a é seguido por dois bs
 - conjunto de todas as cadeias de 0s e 1s contendo exatamente dois Os
- ★22. As cadeias dadas pertencem aos conjuntos regulares?
- 01110111; $(1^*01)^*(11 \vee 0^*)$
 - 11100111; $[(1^*0)^* \vee 0^*11]^*$
 - 011100101; $01^*10^*(11^*0)^*$
 - 1000011; $(10^* \vee 11)^*(0^*1)^*$
23. Escreva uma expressão regular para o conjunto de todas as cadeias alfanuméricas começando por uma letra. (O conjunto de identificadores válidos de FORTRAN é um subconjunto deste conjunto.)
- ★24. Escreva uma expressão regular para o conjunto de todas as expressões aritméticas indicando a adição ou subtração de dois inteiros positivos.
25. a. Demonstre que se A é um conjunto regular, então o conjunto regular A^R que consiste em todas as cadeias de A revertidas, também é regular,
b. Para qualquer cadeia α , seja α^R a cadeia revertida. Você acha que o conjunto $\{\alpha\alpha^R \mid \alpha \in I^*\}$ é regular?
26. Demonstre que, se A é um conjunto regular cujos símbolos pertencem ao alfabeto I , então $I^* - A$ é um conjunto regular.
- ★27. Identifique quaisquer estados inalcançáveis de cada máquina M na tabela a seguir.

Estado atual	Próximo Estado		Saída
	Entrada atual 0 1		
s_0	s_2	s_0	0
s_1	s_2	s_1	1
s_2	s_2	s_0	1

(a)

Estado atual	Próximo Estado			Saída
	Entrada atual <i>abc</i>			
s_0	s_1	s_0	s_3	0
s_1	s_1	s_3	s_0	1
s_2	s_3	s_2	s_1	0
s_3	s_1	s_1	s_0	0

(b)

★28. Minimize M .

Estado atual	Próximc Estado		Saída
	Entrada atual		
	0	1	
0	3	6	1
1	4	2	0
2	4	1	0
3	2	0	1
4	5	0	1
5	3	5	0
6	4	2	1

29. Minimize M .

Estado atual	Próximo Estado		Saída
	Entrada atual		
	0	1	
0	5	3	1
1	5	2	0
2	1	3	0
3	2	4	1
4	2	0	1
5	1	4	0

30. Minimize M .

Estado atual	Próximo Estado		Saída
	Entrada atual		
	0	1	
0	1	2	0
1	2	3	1
2	3	4	0
3	2	1	1
4	5	4	1
5	6	7	0
6	5	6	1
7	8	1	0
8	7	3	0

31. Minimize M .

Estado atual	Próximo Estado		Saída
	Entrada atual		
	0	1	
0	7	1	1
1	0	3	1
2	5	1	0
3	7	6	1
4	5	6	0
5	2	3	0
6	3	0	1
7	4	0	0

32. Minimize M .

Estado atual	Próximo Estado		Saída
	Entrada atual 0 1		
0	1	3	0
1	2	4	1
2	5	4	0
3	1	2	2
4	2	1	1
5	4	0	2

★33. Minimize M .

Estado atual	Próximo Estado			Saída
	Entrada atual <i>a</i> <i>b</i> <i>c</i>			
0	1	4	0	1
1	4	2	3	0
2	3	4	2	1
3	4	0	1	0
4	1	0	2	0

34. Minimize M .

Estado atual	Próximo Estado		Saída
	Entrada atual 0	1	
0	1	3	0
1	2	0	0
2	0	3	0
3	2	1	0

35. Minimize M .

Estado atual	Próximo Estado		Saída
	Entrada atual 0 1		
0	1	3	1
1	2	0	0
2	4	3	1
3	0	1	1
4	2	4	0

Seção 8.3 Máquinas de Turing

Na seção anterior vimos que, como $S = \{0^n1^n \mid n \geq 0\}$ não é um conjunto regular, o teorema de Kleene nos diz que ele não é reconhecido por qualquer máquina de estado finito. Não demonstramos, na verdade, que S não é um conjunto regular; apenas percebemos que não somos capazes de obter uma expressão regular para ele. Usaremos uma abordagem ligeiramente diferente.

Suponha que S é aceita por uma máquina de estado finito M com m estados. Então todas as cadeias de 5, e apenas cadeias de 5, levam M de seu estado inicial até seu estado final. Agora vamos executar M um determinado número de vezes para entradas $\lambda, 0, 0^2, 0^3, \dots, 0^m$. Ao fim do processamento dessas $m + 1$ cadeias, M

estará em algum estado. Como M tem apenas m estados distintos, deve haver pelo menos duas cadeias dessa lista, digamos 0^v e 0^w , $v \neq w$, que levam M do estado inicial a um mesmo estado. (Isto é, na verdade, uma aplicação do princípio da casa do pombo do Cap. 3, onde os itens são as cadeias de entrada e as casas nas quais devemos colocar os itens são os estados nos quais M está após processar as cadeias.) Como M aceita S , a cadeia de entrada $0^v 1^v$ faz com que M termine em um estado final. Mas como M está no mesmo estado após processar 0^w que depois de processar 0^v , a cadeia $0^w 1^v$, que não pertence a S , levará M ao mesmo estado final. Esta contradição mostra que não existe máquina de estado finito que aceite S .

Provavelmente consideramos nós mesmos como máquinas de estado finito e imaginamos que nossos cérebros, que são compostos por um grande número de células, podem ter apenas um número finito, apesar de bem grande, de configurações ou estados. Temos certeza, no entanto, que se alguém nos apresentar uma cadeia arbitrariamente grande de Os seguida de uma cadeia arbitrariamente grande de Is, podemos determinar se os números de Os e Is são os mesmos. Vamos imaginar alguma técnica que podemos usar.

Para pequenas cadeias de Os e Is, podemos apenas olhar a cadeia e obter a resposta. Portanto podemos dizer sem dificuldade que $000111 \in S$ e que $00011 \notin S$. No entanto, a cadeia

00000000000000011111111111111111

requer que usemos outro procedimento, provavelmente reordenando ou contando os símbolos. Podemos contar o número de Os recebidos e quando recebermos o primeiro 1, escrevemos o número de Os encontrados (ou o memorizamos) para futura comparação, e então começamos a contar os 1s. (Este é o processo que fazemos mentalmente para pequenas cadeias.)

No entanto, fizemos uso de alguma memória extra, pois quando terminamos de contar os 1s precisamos recuperar o número Os encontrados para realizar uma comparação. Mas este tipo de recuperação de informações é o que as máquinas de estado finito não são capazes de fazer; sua única capacidade de lembrar a entrada é ter tido seu estado alterado por um determinado símbolo de entrada. Já vimos que uma máquina de estado finito não é capaz de "lembrar" 0 para n arbitrariamente grande porque excede o número de estados. De fato, se tentarmos resolver este problema em um computador real, encontraremos a mesma dificuldade. Se incrementarmos um contador sempre que lermos um 0, podemos chegar a um estouro (overflow) porque nosso contador pode crescer demais. Para processar $0^n 1^n$ para valores de n arbitrariamente grandes, precisamos de uma quantidade ilimitada de memória auxiliar para armazenar o valor de nosso contador, o que, na prática, não existe.

Outra maneira pela qual os humanos resolvem o problema de aceitar cadeias de S é esperar até que toda a cadeia tenha sido entrada. Então vamos a um extremo da cadeia e cortamos um 0, passamos ao outro extremo e cortamos um 1 e continuamos este procedimento de idas e vindas até esgotarmos os Os ou 1s. A cadeia pertence a S se, e somente se, esgotarmos ambos ao mesmo tempo. Apesar desta abordagem parecer diferente da anterior, ela ainda exige que armazenemos cada um dos símbolos entrados, uma vez que devemos voltar e examiná-los, desde que tenhamos terminado de dar-lhes entrada. Uma máquina de estado finito, naturalmente, não pode ler a entrada mais de uma vez.

Discutimos dois procedimentos computacionais — algoritmos — para decidir se uma dada cadeia de Os e 1s pertence ou não a S . Ambos necessitam de algum tipo de memória adicional, que não é disponível em uma máquina de estado finito. Evidentemente, a máquina de estado finito não constitui um modelo da forma mais geral de um procedimento computacional.

Definição

Para simular procedimentos mais gerais dos que podem ser simulados pelas máquinas de estado finito, usamos uma *máquina de Turing*, proposta pelo matemático inglês Alan M. Turing em 1936. Uma máquina de Turing é essencialmente uma máquina de estado finito com a habilidade de ler suas entradas mais de uma vez e também de apagar ou substituir os valores de suas entradas; ela também tem uma memória auxiliar ilimitada. Portanto, a máquina de Turing supera as deficiências que constatamos nas máquinas de estado finito.

Uma máquina de Turing consiste em uma máquina de estado finito e uma fita dividida em células, cada célula contendo no máximo um símbolo de um alfabeto finito disponível. A qualquer momento, apenas um número finito de células contém valores. Usaremos o símbolo especial b para denotar uma célula sem valor. A unidade de estado finito, através de sua cabeça de leitura/gravação, lê uma célula da fita a cada momento (veja Fig. 8.9). No pulso de relógio seguinte, dependendo do estado corrente da unidade e do símbolo lido, a unidade pode não realizar ação alguma (parar) ou realizar três ações. Estas ações são: (1) imprimir um símbolo do alfabeto na célula lida (pode ser o mesmo símbolo que a célula já contém), (2) passar ao estado seguinte (pode ser o mesmo estado que antes) e (3) mover a cabeça de leitura/gravação uma célula para a esquerda ou para a direita.

Podemos descrever as ações de qualquer máquina de Turing por um conjunto de quintuplas da forma (s, i, i', s', d) , onde s e i indicam o estado atual e o símbolo que está sendo lido da fita, i' denota o símbolo impres-

so, s' denota o novo estado e d denota a direção na qual a cabeça de leitura/gravação deve mover-se (D para a direita e E para a esquerda). Portanto, uma máquina na configuração ilustrada pela Fig. 8.10a, se operar de acordo com as instruções contidas na quintupla $(2, 1, 0, 1, D)$, deve mover-se para a configuração ilustrada pela Fig. 8.10b. O símbolo 1 que estava sendo lido da fita foi alterado para 0, o estado da unidade foi alterado de 2 para 1 e a cabeça foi movida uma célula para a direita.

Definição: Máquina de Turing

Seja S um conjunto finito de estados e I um conjunto finito de símbolos da fita (o alfabeto da fita) incluindo o símbolo especial b . Uma máquina de Turing é um conjunto de quintuplas da forma (s, i, i', s', d) onde $s, s' \in S$; $i, i' \in I$ e $d \in \{D, E\}$ e não há mais de uma quintupla começando pelos mesmos símbolos s e i .

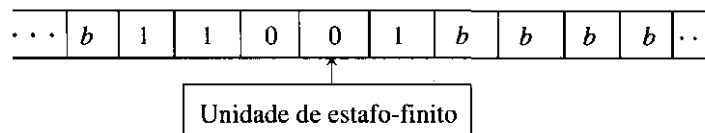


Figura 8.9

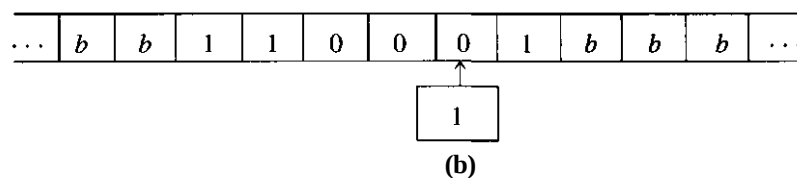
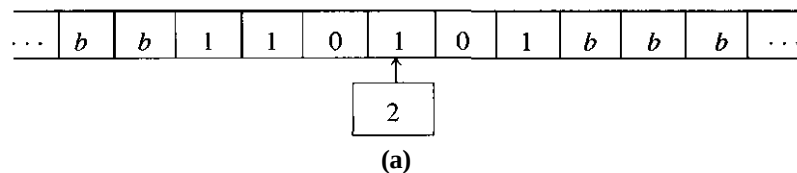


Figura 8.10

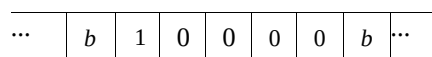
A restrição de que não pode haver mais de uma quintupla começando pelos mesmos símbolos s e i garante que a ação da máquina de Turing é determinística e completamente estabelecida pelos estado e símbolo lido atuais. Se uma máquina de Turing é levada a uma configuração para a qual seu estado atual e o símbolo lido não são os primeiros dois símbolos de qualquer quintupla, a máquina pára.

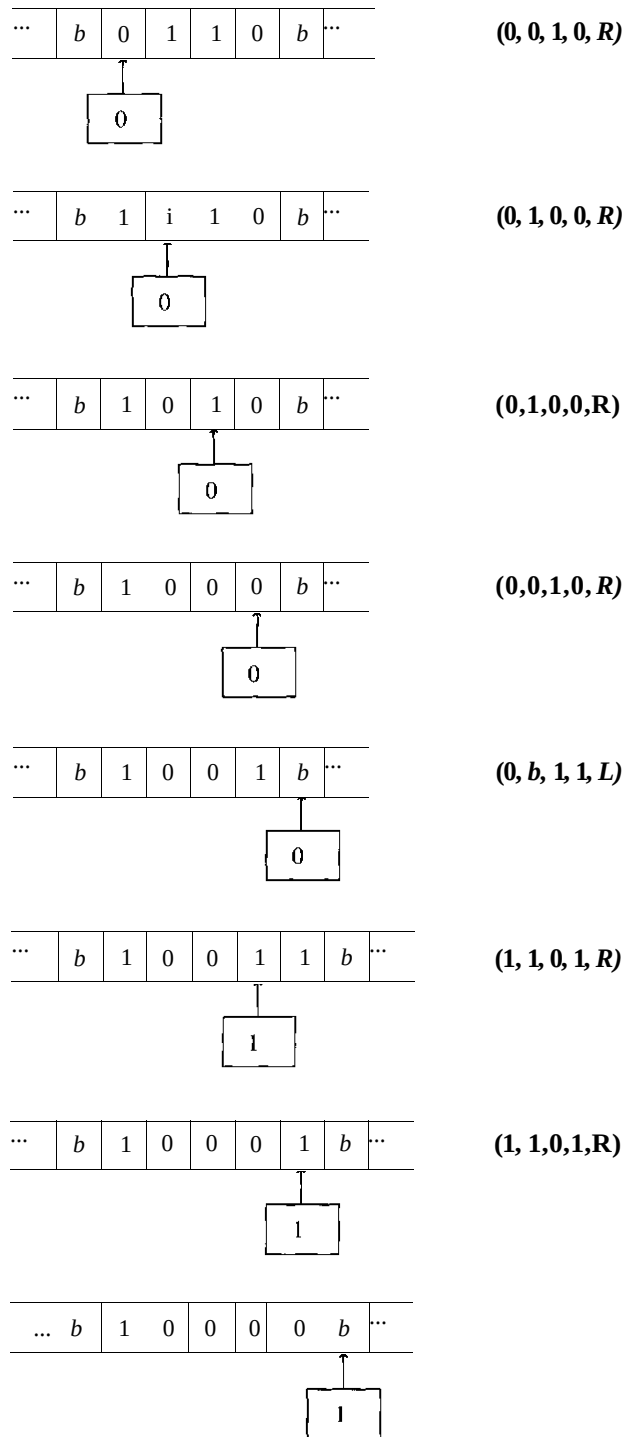
Como no caso de máquinas de estado finito ordinárias, especificamos um estado inicial fixo, denotado por 0, no qual a máquina inicia qualquer processamento. Também assumimos uma configuração inicial para a cabeça de leitura/gravação, a saber, posicionada sobre a célula mais à esquerda da fita que contenha valor. (Se a fita estiver a princípio totalmente vazia, a cabeça de leitura/gravação pode ser, de início, posicionada em qualquer lugar.)

EXEMPLO 25 Uma máquina de Turing é definida pelo conjunto de quintuplas

$(0, 0, 1, 0, D)$
 $(0, 1, 0, 0, D)$
 $(0, b, 1, 1, E)$
 $(1, 0, 0, 1, D)$
 $(1, 1, 0, 1, D)$

A ação desta máquina de Turing, ao processar qualquer fita em particular, é mostrada pela seqüência de configurações na Fig. 8.11, que também mostra a quintupla que se aplica em cada passo. Como não há quintuplas que definam a ação a ser tomada quando se está no estado 1 lendo b , a máquina pára com a fita contendo



**Figura 8.11**

A fita serve como um meio de armazenamento para a máquina de Turing e, em geral, a máquina pode voltar a ler as células da fita. Uma vez que também pode escrever na fita, a porção que contém valores da fita pode ser tão longa quanto se queira, apesar de que sempre teremos apenas um número finito de células com valores a cada momento. Consequentemente, a máquina dispõe de uma quantidade ilimitada, porém finita, de memória. Como as máquinas de Turing superam as deficiências das máquinas de estado finito, elas devem ter capacidades consideravelmente maiores. Na verdade, uma máquina de estado finito é um caso muito especial de máquina de Turing, que sempre imprime o símbolo anterior na célula lida, sempre se move para a direita, e sempre pára no símbolo b .

PRÁTICA 47 Considere a seguinte máquina de Turing:

(0, 0, 0, 1, D)
 (0, 1, 0, 0, D)
 (0, b, b, 0, D)
 (1, 0, 1, 0, D)
 (1, 1, 1, 0, E)

- a. Qual é o conteúdo final da fita, dado seu conteúdo inicial

...	b	1	0	b	...
-----	---	---	---	---	-----

(Não é necessário realizar a tarefa tediosa de desenhar todos os pequenos quadrados; apenas escreva o conteúdo final da fita.)

- b. Descreva o comportamento da máquina quando iniciada com a fita

...	b	0	1	b	...
-----	---	---	---	---	-----

- c. Descreva o comportamento da máquina quando iniciada com a fita

...	b	0	0	b	...
-----	---	---	---	---	-----

Os itens (b) e (c) da Prática 47 mostram dois casos nos quais uma máquina de Turing pode não parar: através de um ciclo sem fim ou movendo-se indefinidamente pela fita.

Máquinas de Turing como Reconhecedores de Conjuntos

As aplicações da máquina de Turing que vimos até agora não tinham qualquer significado prático. Usaremos a máquina de Turing para realizar dois tipos de tarefas. A primeira é usá-la para aceitar conjuntos, da mesma forma que consideramos para as máquinas de estado finito na seção anterior. Podemos ainda fornecer uma descrição muito semelhante, desde que definamos um estado final para a máquina de Turing. Um **estado final** em uma máquina de Turing é um estado que não seja o primeiro elemento de qualquer quintupla. Portanto, ao entrar em um estado final, qualquer que seja o símbolo lido, a máquina de Turing pára.

Definição: Reconhecimento (aceitação) por Máquina de Turing

Uma máquina de Turing T com alfabeto da fita / **reconhece (aceita)** um subconjunto S de I^* se T , começando em alguma configuração inicial em uma fita contendo uma cadeia a de símbolos de entradas, pára em um estado final se, e somente se, $a \in S$.

Perceba que nossa definição de aceitação permite dois comportamentos para T quando aplicada a uma cadeia a de símbolos da fita que não pertença a S . T pode parar em um estado não-final ou T pode vir a não parar.

Podemos agora construir uma máquina de Turing para reconhecer nosso velho amigo $S = \{0^n 1^n \mid n \geq 0\}$. A máquina é baseada em nossa segunda abordagem para este problema de reconhecimento, indo e voltando do início para o fim da entrada e marcando os pares 0—1.

EXEMPLO 26 Desejamos construir uma máquina de Turing que reconheça $S = \{0^n 1^n \mid n \geq 0\}$. Usaremos um símbolo especial a mais, digamos X . Portanto, o alfabeto da fita é $\{0, 1, b, X\}$. O estado 8 é o único estado final. As quintuplas que compõem T são dadas abaixo, junto com uma descrição de suas funções.

(0, b, b, 8, D) Reconhece a fita vazia, que pertence a S .
 (0, 0, X, 1, D) Apaga o 0 mais à esquerda e inicia o movimento para a direita
 (1, 0, 0, 1, D)
 (1, 1, 1, 1, D) Move para a direita, permanecendo no estado 1, até que encontre o
 (1, b, b, 2, E) fim da cadeia inicial; então move para a esquerda no estado 2.

- (2, 1, X, 3, E) Apaga o 1 mais à direita e inicia o movimento para a esquerda.
 (3, 1, 1, 3, E) Move para a esquerda por sobre os 1 s.
 (3, 0, 0, 4, E) Passa ao estado 4 se encontrar mais Os.
 (3, X, X, 7, D) Passa ao estado 7 se não encontrar mais Os na cadeia.
 (4, 0, 0, 4, E) Move para a esquerda por sobre os Os.
 (4, X, X, 5, D) Encontra o extremo esquerdo da cadeia binária e recomeça o processo.
 (5, 0, X, 6, D) Apaga o 0 mais à esquerda
 (6, 0, 0, 6, D)
 (6, 1, 1, 6, D) Move para a direita, permanecendo no estado 6, até que encontre o
 (6, X, X, 2, E) fim da cadeia binária; então move para a esquerda no estado 2.
 (7, X, X, 8, D) Não há mais 1s na cadeia; a máquina aceita a cadeia.

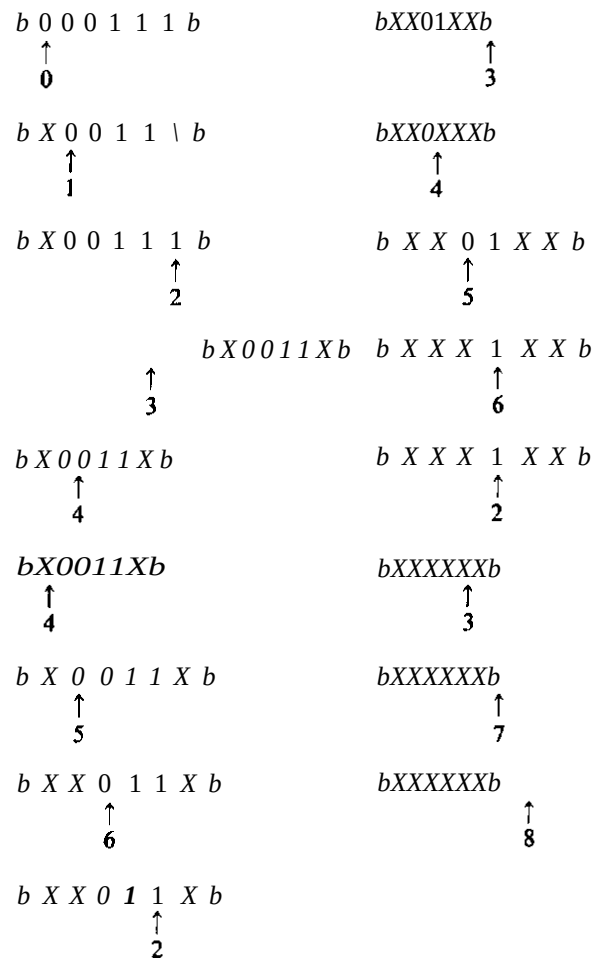


Figura 8.1

Lendo as colunas da Fig. 8.12, podemos ver as configurações-chave no funcionamento da máquina para a fita

...	b	0	0	0	1	1	1	b	...
-----	---	---	---	---	---	---	---	---	-----

•

que, naturalmente, deve ser aceita.

PRÁTICA 48 Para a máquina de Turing do Exemplo 26, descreva a configuração final após o processamento das seguintes fitas de entrada:

a.

...	b	0	0	1	1	1	b	...
-----	---	---	---	---	---	---	---	-----

b.

...	b	0	0	0	1	1	b	...
-----	---	---	---	---	---	---	---	-----

c.

...	b	0	0	0	0	1	1	b	...
-----	---	---	---	---	---	---	---	---	-----

PRÁTICA 49 Projete uma máquina de Turing para reconhecer o conjunto de todas as cadeias de Os e Is terminando por 00. (Este conjunto pode ser descrito pela expressão regular $(0/1)^*00$, de forma que você pode usar uma máquina de Turing que não altera os símbolos da fita e sempre move para a direita.)

PRÁTICA 50 Modifique a máquina de Turing do Exemplo 26 para reconhecer $\{0^n 1^{2n} \mid n \geq 0\}$.

Máquinas de Turing para Computar Funções

A segunda tarefa para qual usaremos as máquinas de Turing é o cômputo de funções. Dada uma determinada máquina de Turing T e uma cadeia α de símbolos da fita, começamos com γ na configuração inicial-padrão em uma fita contendo α . Se T em algum momento pára deixando uma cadeia β na fita, podemos considerar β o valor de uma função avaliada em α . Usando a notação de funções, $T(\alpha) = \beta$. O domínio da função T consiste em todas as cadeias α para as quais γ , em algum momento, pára. Também podemos pensar que T computa as **funções número-teóricas**, funções de um subconjunto de $\mathbb{N}^k$ em $\mathbb{N}$ para qualquer $k \geq 1$. Imaginemos uma cadeia de Is com comprimento $n + 1$ como a representação unária do inteiro não-negativo n ; denotaremos esta codificação de n por $\bar{n}$. (O 1 extra na codificação nos permite distinguir o 0 de uma fita em branco.) Então, uma fita contendo a cadeia $\bar{n}_1 * \bar{n}_2 * \dots * \bar{n}_k$ pode ser entendida como uma representação da k -upla $(n_1, n_2, \dots, n_k)$ de inteiros não-negativos. Se T começar na configuração inicial-padrão com uma fita que cause, em algum momento, a parada da máquina com uma fita final que é a representação de $\bar{m}$ de um inteiro não-negativo m , então T funcionou como uma função de k variáveis T^k , onde $T^k(n_1, n_2, \dots, n_k) = m$. Se T começar em uma configuração inicial-padrão para a qual a máquina não pare ou pare sem uma representação de um inteiro não negativo na fita, a função T^k é indefinida em $(n_1, n_2, \dots, n_k)$.

Existe, portanto, uma sequência infinita $T^1, T^2, \dots, T^k, \dots$ de funções número-teóricas computadas por T associadas a cada máquina de Turing γ . Para cada k , a função T^k é uma **função parcial** em $\mathbb{N}^k$, significando que seu domínio pode ser um subconjunto próprio de $\mathbb{N}^k$. Um caso especial de uma função parcial em $\mathbb{N}^k$ é uma **função total** em $\mathbb{N}^k$, onde a função é definida para todas as k -uplas de inteiros não-negativos.

EXEMPLO 27 Seja uma máquina de Turing T dada pelas quintuplas

(0, 1, 1, 0, D)
(0, b, 1, 1, D)

Se γ é iniciada na configuração inicial-padrão da fita

...	b	1	1	1	b	...
-----	---	---	---	---	---	-----

então γ vai parar com configuração final

b 1 1 1 1 b
 ↑
 1

Portanto, T define uma função de uma variável T^1 que leva $\bar{2}$ em $\bar{3}$. Em geral, T leva $\bar{n}$ em $n + 1$, de forma que $T^1(n) = n + 1$, uma função total de uma variável. ♦

No Exemplo 27, começamos com uma máquina de Turing e observamos que ela computa uma determinada função, mas também podemos começar com uma função número-teórica e tentar achar a máquina de Turing que a compute.

Definição: Função Turing-Computável

Uma função Turing-computável é uma função número-teórica computada por alguma máquina de Turing.

Uma função Turing-computável f pode, na verdade, ser computada por um número infinito de máquinas de Turing. Uma vez tenhamos encontrado uma máquina de Turing que compute f , sempre podemos incluir quintuplas irrelevantes em T , produzindo outras máquinas de Turing que também computem f .

EXEMPLO 28 Desejamos encontrar uma máquina de Turing que compute a função f definida da seguinte maneira:

$$f(n_1, n_2) = \begin{cases} n_2 - 1 & \text{se } n_2 \neq 0 \\ \text{indefinido} & \text{se } n_2 = 0 \end{cases}$$

Portanto, f é uma função parcial de duas variáveis. Vamos considerar a máquina de Turing dada pelas quintuplas a seguir. As ações realizadas pelos diversos grupos de quintuplas são descritas.

- $(0, 1, 1, 0, D)$ } Passa por $\bar{n}_1$ até $\bar{n}_2$.
- $(0, *, *, 1, D)$ }
- $(1, 1, 1, 2, D)$ Conta o primeiro 1 em $\bar{n}_2$.
- $(2, b, b, 3, R)$ $n_2 = 0$; pára.
- $(2, 1, 1, 4, D)$ }
- $(4, 1, 1, 4, D)$ } Encontra o extremo direito de $\bar{n}_2$.
- $(4, b, b, 5, E)$ }
- $(5, 1, b, 6, E)$ Apaga último 1 em $\bar{n}_2$.
- $(6, 1, 1, 6, E)$ }
- $(6, *, b, 7, E)$ } Caminha para a esquerda até $\bar{n}_1$ apagando *
- $(7, 1, b, 7, E)$ Apaga $\bar{n}_1$.
- $(7, b, b, 8, E)$ $\bar{n}_1$ apagado; pára com $\overline{n_2 - 1}$ na fita.

Se T é iniciada com a fita

...	b	1	1	*	1	1	1	1	b	...
-----	-----	---	---	---	---	---	---	---	-----	-----

então T vai parar com a configuração final

$bbbbbb111b$
 $\uparrow$
 8

Esta configuração está de acordo com o requisito $f(1, 3) = 2$. Se T é iniciada com a fita

...	b	1	1	*	1	b	...
-----	-----	---	---	---	---	-----	-----

então T vai parar com a configuração final

$b11*1bb$
 $\uparrow$
 3

Como a fita final não é $\bar{m}$ para qualquer inteiro não-negativo m , a função computada por T é indefinida em $(1, 0)$ — justamente como desejávamos. É fácil ver que esta máquina de Turing computa f e que f é, portanto, uma função Turing-computável. •

PRÁTICA 51 Projete uma máquina de Turing para computar a função

$$f(n) = \begin{cases} n - 2 & \text{se } n \geq 2 \\ 1 & \text{se } n < 2 \end{cases}$$

A Tese de Church-Turing

Falamos, neste capítulo, sobre os modelos "computacionais" ou de "procedimentos computacionais". Apesar de não termos definido o termo, devemos entender por *procedimento computacional* um algoritmo. Já falamos de algoritmos diversas vezes neste livro e fornecemos diversos algoritmos com diferentes propósitos. Lembremos que nossa definição (um tanto quanto intuitiva) de algoritmo é um conjunto de instruções que podem ser executadas mecanicamente em uma quantidade finita de tempo a fim de resolver algum problema. Isto significa que, dada uma entrada apropriada para a tarefa, o algoritmo precisa, em algum momento, terminar (parar) e produzir a resposta correta se existir uma. (Se não existir uma resposta, vamos supor que o algoritmo termina e indica que não há resposta, ou fica indefinidamente procurando a resposta.)

Agora perguntamos: as máquinas de Turing são um modelo para procedimentos computacionais melhor que as máquinas de estado finito? Nos parece provável que qualquer função Turing-computável f seja uma função cujos valores podem ser encontrados por um algoritmo computacional. De fato, se f é computável por uma máquina de Turing T , então o conjunto de quintuplas de T é, ele próprio, o algoritmo; na forma de uma lista de instruções que podem ser executadas mecanicamente, ele satisfaz as várias propriedades de nossa noção de algoritmo. Portanto, nós estamos dispostos a aceitar a proposta ilustrada na Fig. 8.13. A figura mostra "computável por um algoritmo" como uma idéia intuitiva "nebulosa" (na forma de uma nuvem) e "Turing-computável" como uma idéia matematicamente precisa, bem definida. A seta indica que qualquer função Turing-computável é computável por um algoritmo.

Dada a simplicidade da definição de uma máquina de Turing, é pouco natural contemplar a Fig. 8.14, que afirma que qualquer função computável por um algoritmo é Turing-computável. Combinando as Figs. 8.13 e 8.14, obtemos a tese de Church-Turing (Fig. 8.15), assim chamada em função de Turing e outro matemático famoso, Alonzo Church.

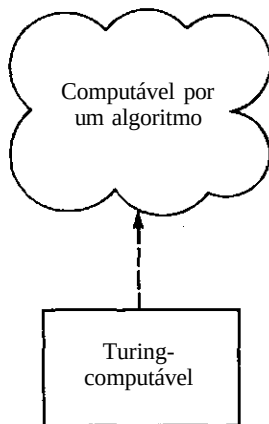


Figura 8.13

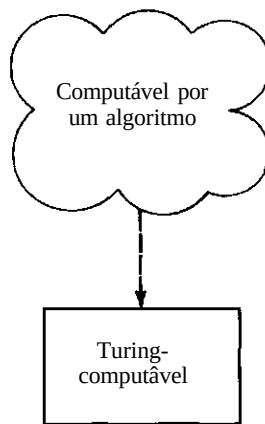


Figura 8.14

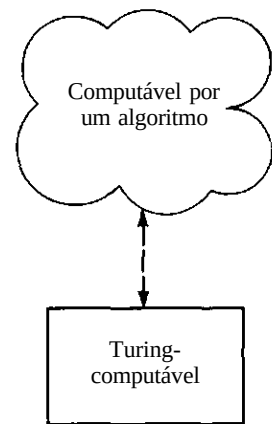


Figura 8.15

Tese de Church-Turing

Uma função número-teórica é computável por um algoritmo se, e somente se, ela for Turing-computável.

Como a tese de Church-Turing estabelece a equivalência entre uma idéia intuitiva e uma idéia matemática, ela não pode ser provada formalmente e deve ser mantida como uma tese, e não como um teorema. Qual é, então, sua justificativa?

Uma parte da evidência é que sempre que um procedimento geralmente suposto como um algoritmo é proposto para computar uma função, alguém é capaz de projetar uma máquina de Turing para computar esta função. (Naturalmente, sempre há o risco de que algum dia isto não ocorra.)

Outra parte desta evidência é que outros matemáticos, muitos deles na mesma época que Turing desenvolveu a máquina de Turing, propuseram outros modelos de procedimentos computacionais. Aparentemente, os modelos não tinham relações uns com os outros. No entanto, como todos esses modelos foram definidos formalmente, assim como a computabilidade de Turing, foi possível estabelecer, com bases matemáticas formais, que, de algum modo, eles eram todos equivalentes. Todos esses modelos, inclusive o de computabilidade por Turing, foram demonstrados equivalentes; isto é, todos eles definiam a mesma classe de funções, o que sugere que a computabilidade por Turing engloba o conceito de todos sobre algoritmo. A Fig. 8.16 ilustra o que foi feito; nesta figura as linhas sólidas representam demonstrações matemáticas e as linhas pontilhadas indicam aplicações da tese de Church-Turing. As datas são referentes a quando os diversos modelos foram propostos.

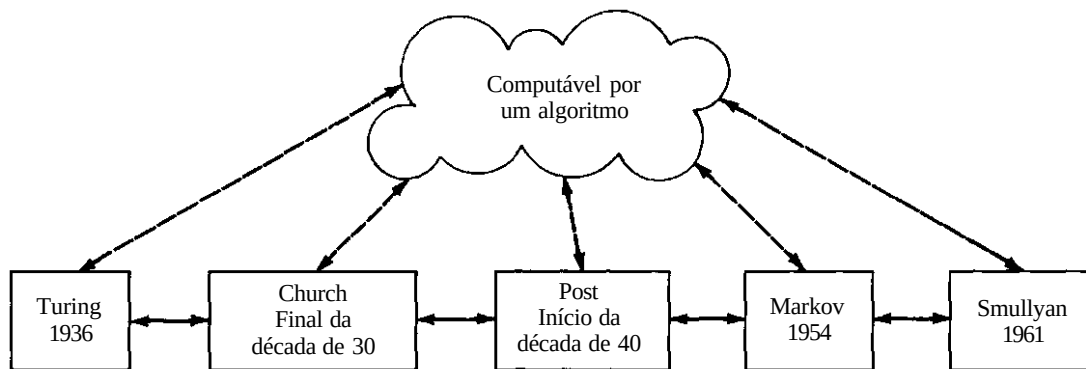


Figura 8.16

Atualmente a tese de Church-Turing é muito aceita como uma ferramenta de trabalho para pesquisadores que lidam com procedimentos computacionais. Se, em um artigo de pesquisa, é definido um método para calcular uma função e o método parece, intuitivamente, ser um algoritmo, então a tese de Church-Turing é invocada e a função é declarada Turing-computável (ou um dos nomes associados a uma das formulações equivalentes à computabilidade por Turing). Esta invocação da tese de Church-Turing significa que o autor é capaz, se exigido, de produzir uma máquina de Turing que compute a função, mas, novamente, a tese de Church-Turing é tão amplamente aceita que ninguém se prende mais a esses detalhes.

Apesar da tese de Church-Turing ser formulada em termos de funções número-teóricas, ela pode ser interpretada de forma mais geral. Qualquer algoritmo que manipule um número finito de símbolos pode ser transformado em uma função número-teórica — através de uma codificação adequada dos símbolos — em inteiros não negativos, da mesma forma que a entrada de um computador real é codificada e armazenada de forma binária. Portanto, pela tese de Church-Turing, sempre podemos dizer que, se existe um algoritmo que realiza uma tarefa de manipulação de símbolos, então também existe uma máquina de Turing para fazê-lo.

Se aceitarmos a tese de Church-Turing, aceitaremos a máquina de Turing como o modelo final de procedimentos computacionais. As capacidades das máquinas de Turing superam as capacidades de qualquer computador verdadeiro, que, afinal, não têm a quantidade ilimitada de memória que uma máquina de Turing tem. Vale observar que Turing propôs este conceito em 1936, bem antes do advento do computador moderno.

Problemas de Decisão

Gastamos um tempo considerável falando do que as máquinas de Turing são capazes de fazer. Pela tese de Church-Turing, elas podem, de fato, realizar um grande papel, apesar de não muito eficientemente. É ainda mais importante, no entanto, considerar o que as máquinas de Turing *não* são capazes de fazer. Como as habilidades de uma máquina de Turing para realizar as tarefas superam as de um computador de verdade, se encontrarmos algo que nenhuma máquina de Turing possa fazer, então um computador verdadeiro também não poderá. De fato, pela tese de Church-Turing, não há algoritmo para realizar este tipo de tarefa, e ela não é computável. O tipo de tarefa que temos em mente aqui é, de forma geral, o de determinar os valores-verdade de cada uma de diversas proposições relacionadas.

Definição: Problema de Decisão

Um **problema de decisão** pergunta se existe um algoritmo que determine se determinadas proposições de alguma classe grande de proposições são ou não verdadeiras.

A solução para um problema de decisão responde se um determinado algoritmo existe ou não. Uma **solução positiva** consiste em demonstrar que existe um algoritmo do tipo desejado e, normalmente, é dada produzindo o algoritmo que funcione. Uma **solução negativa** consiste em demonstrar que não existe um algoritmo do tipo desejado. Perceba que esta afirmação é muito mais forte do que simplesmente dizer que muitas pessoas tentaram mas nenhuma conseguiu formular um algoritmo — isto poderia significar apenas que o algoritmo é muito difícil. É preciso mostrar que é impossível para qualquer pessoa encontrar um algoritmo do tipo desejado. Quando encontramos uma solução negativa para um problema de decisão, o problema é dito **insolúvel** ou **indecidível**. Esta terminologia pode causar confusão porque o problema de decisão propriamente dito — a pergunta de quando existe um algoritmo para realizar uma tarefa — foi resolvido; o que permanecerá para sempre sem uma solução é a tarefa.

Exemplos de Problemas de Decisão

Vamos ver alguns problemas de decisão que foram resolvidos.

EXEMPLO 29 Existe um algoritmo para determinar, dados três inteiros a , b e c se $a^2 = b^2 + c^2$? Naturalmente, este é um problema solúvel. O algoritmo consiste em multiplicar b por ele mesmo, multiplicar c por ele mesmo, somar os resultados e comparar com o resultado da multiplicação de a com ele mesmo. •

Obviamente, o Exemplo 29 é um problema de decisão bem trivial. Historicamente, muitos matemáticos se dedicaram, ainda que indiretamente, a encontrar soluções positivas para problemas de decisão, isto é, produzir algoritmos. Soluções negativas para problemas de decisão só vieram a aparecer no século vinte.

EXEMPLO 30 Um dos primeiros problemas de decisão que foram formulados foi o décimo problema de Hilbert, o décimo problema de uma lista que David Hilbert propôs ao Congresso Internacional de Matemáticos em 1900. O problema consiste em determinar se existe um algoritmo para decidir se qualquer polinômio $P(x_1, x_2, \dots, x_n) = 0$ com coeficientes inteiros tem ou não soluções inteiras. Para equações polinomiais $ax + by + c = 0$, onde a , b e c são inteiros, é sabido que existem soluções inteiras se, e somente se, o máximo divisor comum de a e b também divide c . Portanto, para subclasses particulares de equações polinomiais, existem algoritmos para determinar se existem ou não soluções inteiras, mas o problema de decisão, como enunciado, se aplica a toda a classe de equações polinomiais com coeficientes inteiros. Quando este problema foi proposto e por algum tempo depois, o consenso geral era que certamente existia um algoritmo para este fim, e que o fato de ninguém ainda o ter descoberto só indicava que ele devia ser difícil. Nos meados da década de 30, resultados como o do próximo exemplo começaram a pôr em dúvidas este palpite. Só nos anos 70, no entanto, foi que o problema foi finalmente demonstrado como insolúvel. •

EXEMPLO 31 Nos sistemas da lógica formal, como os que vimos no Cap. 1, certas cadeias de símbolos são identificadas como *axiomas*, e as *regras de inferência* são usadas sempre que uma nova cadeia pode ser obtida a partir das anteriores. Qualquer cadeia que seja a última em uma lista finita de cadeias que consistam em axiomas ou em cadeias obtidas pela aplicação das regras de inferência às regras anteriores na lista é considerada um *teorema*. O problema de decisão para uma teoria formal é determinar se existe um algoritmo para, dada uma cadeia na teoria formal, determinar se ela é ou não um teorema desta teoria.

O trabalho de Church e do famoso lógico do século vinte Kurt Gödel mostrou que qualquer teoria formal que crie axiomas para as propriedades aritméticas (estabelecendo a comutatividade da adição como um axioma, por exemplo) e não seja completamente trivial (nem tudo é um teorema) é indecidível. O trabalho deles pode ser considerado uma boa notícia para os matemáticos, porque significa que a engenhosidade para solucionar os problemas da teoria dos números nunca poderá ser substituída por um procedimento mecânico. •

EXEMPLO 32 Uma determinada máquina de Turing T ao começar com uma fita contendo a cadeia a pode parar em algum momento ou nunca parar. O **problema da parada** para máquinas de Turing é um problema de decisão: Existe algum algoritmo para determinar, dada uma máquina de Turing T e uma cadeia a , se T ao começar com a fita contendo a pára sua execução em algum momento? Turing demonstrou a indecidibilidade do problema da parada no fim da década de 30.

O Problema da Parada

Demonstraremos a insolubilidade do problema da parada após duas observações. Primeiro, já nos deve ter ocorrido que "rodar T em a " pode constituir um algoritmo para determinar se T pára, se aplicada à cadeia a . Se após 25 passos do processamento de T , T tiver parado, então sabemos que T pára com a seqüência a . Mas se após 25.000 passos T não tiver parado, o que podemos concluir? T ainda pode vir a parar em algum momento.

Por quanto tempo devemos esperar? Este assim chamado algoritmo nem sempre nos dá uma resposta a nossa questão.

Uma segunda observação é que o problema da parada requer que um algoritmo seja aplicado a grandes conjuntos de comandos. O problema da parada pergunta se existe um algoritmo para determinar, dado um par (T, α) se T pára ou não quando iniciada em uma fita que contenha α . Primeiro precisamos de um algoritmo, e este algoritmo simples precisa fornecer a resposta correta para todos os pares (T, α) . Na notação da lógica predicada, o problema de parada nos pergunta sobre o valor-verdade do enunciado na forma

$$(\exists \text{ algoritmo}) (\forall (T, \alpha)) (\cdot \cdot \cdot).$$

Considere o seguinte enunciado, que nos parece muito familiar: Dado um par (T, α) , existe um algoritmo para determinar se T pára quando iniciado em uma fita contendo α ? Aqui, o par (T, α) vem antes e, em função dele, escolhemos um algoritmo; para um (T, α) diferente, pode haver um algoritmo diferente. A asserção tornou-se $(\forall (T, \alpha)) (\exists \text{ algoritmo}) (\cdot \cdot \cdot)$. Este problema é solúvel. Suponha que alguém nos dê um (T, α) . Dois algoritmos são: (1) "diga sim" e (2) "diga não". Uma vez que T atuando em α pode ou não parar, um desses dois algoritmos responde corretamente à questão. Esta solução pode parecer trivial ou até trapaceira, mas consideremos novamente o enunciado do problema: Dado um par (T, α) , existe um algoritmo para determinar, e por aí vai. Tal algoritmo *existe*; basta ele dizer sim ou não — não precisamos escolher qual é o correto!

Essa troca de palavras torna o problema de parada insolúvel um problema trivial. Ele também mostra o caráter de um problema de decisão, perguntando se existe um algoritmo para resolver uma grande classe de problemas. Um problema insolúvel tem tanto um lado positivo quanto um lado negativo. Não existe um algoritmo para resolver uma grande classe de problemas, garantindo o emprego dos pensadores criativos que não podem ser substituídos por máquinas de Turing. Mas a classe desses problemas é tão grande que torna o resultado muito geral para ter algum interesse.

Enunciaremos o problema da parada novamente, e demonstraremos sua insolubilidade.

Definição: Problema da Parada

O problema da parada é determinar se existe um algoritmo para decidir, dada quaisquer máquina de Turing T e cadeia a , se T , ao ser iniciada em uma fita contendo a , em algum momento, pára sua execução.

Teorema do Problema da Parada

O problema da parada é insolúvel.

Demonstração: Desejamos demonstrar que algo *não existe*, uma situação feita sob medida para demonstrações por contradição. Portanto, admitamos que o problema da parada é solúvel e que existe um único algoritmo que possa aceitar (T, α) como entrada e, de alguma forma, determinar se T aplicado a a pára. Estamos querendo que este algoritmo resolva uma tarefa de manipulação de símbolos, uma vez que podemos imaginar o conjunto de quintuplas de T codificado como alguma cadeia s_T de símbolos; usamos s_T para denotar a cadeia s_T concatenada à cadeia a . A tarefa se torna, portanto, transformar a cadeia (s_T, α) em uma cadeia representando um sim (a máquina de Turing com descrição s_T pára quando iniciada em uma fita contendo α) ou um não (a máquina de Turing com descrição s_T não pára quando iniciada em uma fita contendo a cadeia α). Pela tese de Church-Turing, como admitimos a existência de um algoritmo que realize esta tarefa, podemos assumir a existência de uma única máquina de Turing X que realize esta tarefa. Portanto, X atua em uma fita contendo (s_T, α) para qualquer T e α e, em algum momento, pára, nos dizendo que T pára quando aplicada a α . Para ser definitivo, suponha que X começando em (s_T, a) pára deixando 1 na fita se e somente se T iniciada com α parar, e X iniciada com (s_T, α) pára deixando 0 na fita se e somente se T começando com a não parar; essas são as duas únicas possibilidades.

Agora incluímos duas quintuplas a X a fim de criar uma nova máquina Y . A máquina Y altera a máquina X de forma que sempre que X pára com 1 em sua fita, Y passa a um estado que move indefinidamente para a direita, de forma que nunca pare. Se X pára deixando 0 em sua fita, Y também faz isto. Finalmente, modificamos Y , obtendo uma máquina Z que se aplique a qualquer entrada β copiando β (veja, por exemplo, o Exercício 13) e então passando ao processamento de Y , de forma que Y atue sobre (β, β) .

Agora, pela maneira que Z foi construída, se Z , aplicada a s_z , parar, é porque Y aplicada a (s_z, s_z) também pára e isto ocorre porque X aplicada a (s_z, s_z) pára deixando 0 na fita; mas, se isto acontecer, significa que Z aplicada a s_z não pára! Portanto,

$$Z \text{ aplicada a } s_z \text{ pára} \rightarrow Z \text{ aplicada a } s_z \text{ não pára} \quad (1)$$

Esta implicação é muito estranha; vamos ver o que ocorre se Z aplicada a s_z não parar. Pela maneira pela qual Z foi construída, se Z aplicada a s_z não parar, Y aplicada a (s_z, s_z) também não pára. Por sua vez, Y aplicada a (s_z, s_z) pára, deixando 1 na fita; mas este resultado implica que Z aplicada a s_z pára! Portanto

$$Z \text{ aplicada a } s_z \text{ não pára} \rightarrow Z \text{ aplicada a } s_z \text{ pára} \quad (2)$$

Juntas, as implicações (1) e (2) fornecem uma contradição hermética, portanto nossa premissa de que o problema da parada é solúvel está errada.

A demonstração da solubilidade do problema da parada depende de duas idéias. Uma é a da codificação da máquina de Turing como uma cadeia e a outra é ter uma visão da máquina e agir segundo sua descrição. Perceba que (1) e (2) sozinhas não são suficientes para demonstrar os resultados. Ambas são necessárias para contradizer a hipótese original da solubilidade do problema da parada.

Já vimos outra demonstração desta natureza, onde a observação que faz com que a demonstração funcione se contradiz. Talvez seja interessante que o leitor reveja a demonstração do teorema de Cantor no Cap. 4.

Complexidade Computacional

Como um modelo computacional, a máquina de Turing nos fornece uma maneira de demonstrar a existência de problemas insolúveis (que não são computáveis). A máquina de Turing não nos ajuda apenas a encontrar os limites da computabilidade, mas também pode nos ajudar a classificar os problemas que são computáveis — que têm algoritmos para suas soluções — pelo trabalho computacional necessário para processar esses algoritmos.

Análise de Algoritmos na Forma de Máquina de Turing

Determinar a quantidade de trabalho necessária para se processar um algoritmo parece análise de algoritmos. Já analisamos diversos algoritmos neste livro, os classificamos como $\Theta(\log n)$, $\Theta(n)$, $\Theta(n^2)$, etc. Pela tese de Church-Turing, qualquer algoritmo pode ser expresso na forma de uma máquina de Turing. Desta forma, a quantidade de trabalho necessária é o número de passos da máquina de Turing (um por ciclo do relógio) que são necessários para que a máquina de Turing pare. (Admitimos que consideramos apenas as tarefas que têm respostas de forma que a máquina de Turing pare com qualquer entrada válida.)

O processamento de máquinas de Turing é bem ineficiente. Ainda que os algoritmos A e A' resolvam o mesmo problema, se expressarmos A como uma descrição de máquina de Turing e A' como pseudocódigo para instruções de uma linguagem de programação de alto nível, a comparação do número de operações que cada algoritmo realiza não tem sentido. Por isso, assumiremos que todos os algoritmos são representados na forma de máquinas de Turing de forma que possamos prontamente comparar a eficiência de dois algoritmos diferentes.

Ao invés de discutirmos quando um algoritmo na forma de máquina de Turing é $\Theta(n)$ ou $\Theta(n^2)$, vamos apenas observar que ele é um algoritmo de tempo polinomial. (Apenas algoritmos triviais podem ser melhores do que algoritmos de tempo polinomial, uma vez que uma máquina de Turing leva pelo menos n passos apenas para examinar sua entrada.) Os problemas para os quais não existem algoritmos de tempo polinomial são chamados de **intratáveis**.

P e NP

Definição: P

P é a coleção de todos os conjuntos reconhecidos (aceitos) por máquinas de Turing em um tempo polinomial.

A consideração do reconhecimento em nossa definição de P não é tão restritiva quanto possa parecer. Como a máquina de Turing pára com qualquer entrada apropriada, ela na verdade decide, parando em um estado final ou não-final, se a cadeia inicial é ou não um elemento do conjunto. Muitos problemas podem ser formulados como problemas de decisão de conjuntos através de uma codificação adequada dos objetos envolvidos no problema.

Por exemplo, considere o problema do ciclo hamiltoniano (Seção 6.1) de determinar se um grafo possui ou não um ciclo que passe por todos seus vértices. Podemos definir algum processo de codificação para representar qualquer grafo como uma cadeia de símbolos. As cadeias que são as representações de grafos são as

entradas apropriadas, e desejamos determinar, dada uma cadeia deste tipo, se ela pertence ou não ao conjunto de cadeias cujos grafos associados têm ciclos hamiltonianos. Se pudermos construir uma máquina de Turing para tomar esta decisão em tempo polinomial, então o problema do ciclo hamiltoniano pertence a P .

Mencionamos, na Seção 6.1, que não há algoritmo eficiente para resolver o problema do ciclo hamiltoniano, mas não há demonstração de que o problema do ciclo hamiltoniano não pertença a P . No entanto, ele pertence a uma classe de problemas chamados **problemas NP** , para a qual uma solução correta pode ser verificada em tempo polinomial. Isto é, dado um caminho que seja um ciclo hamiltoniano, este fato pode ser confirmado em tempo polinomial.

Se uma máquina de Turing pode determinar em tempo polinomial quando uma cadeia arbitrária pertence a um conjunto, certamente pode verificar um elemento do conjunto em tempo polinomial. Portanto, $P \subseteq NP$. No entanto, não se sabe se esta inclusão é própria, isto é, não se sabe se $P = NP$.

O problema do ciclo hamiltoniano pertence a outra classe de problemas chamados de **problemas NP -completos**, o que significa que não só pertencem a NP , como também que, se for encontrado um algoritmo polinomial para um deles, isto é, se qualquer problema desta classe for demonstrado como pertencente a P , então teremos, na verdade, demonstrado que $P = NP$. Um grande número de problemas de diferentes áreas foram identificados como sendo NP -completos desde que esta idéia foi formulada, em 1971, e nenhum algoritmo de tempo polinomial foi encontrado para qualquer um deles. Portanto, suspeita-se hoje que $P \subset NP$ (e, conseqüentemente, todos esses problemas são intratáveis), mas a demonstração disto permanece uma obsessão para a pesquisa da ciência da computação.

Revisão da Seção 8.3

Técnicas

- Descrever a ação de uma dada máquina de Turing em uma dada fita inicial
- Construir uma máquina de Turing para reconhecer um determinado conjunto
- Construir uma máquina de Turing para computar uma dada função número-teórica

Idéias Principais

As máquinas de Turing têm um modo de operação determinístico, a habilidade de ler a entrada mais de uma vez e de reescrever seus valores e uma memória auxiliar ilimitada.

Uma máquina de estado finito é um caso especial de máquina de Turing.

As máquinas de Turing podem ser usadas para aceitar conjuntos e como avaliadoras de funções.

A tese de Church-Turing estabelece a equivalência entre uma função computável por um algoritmo e uma função Turing-computável. Como esta tese expressa uma relação entre uma idéia intuitiva e uma idéia definida formalmente, ela nunca poderá ser demonstrada, contudo ela é amplamente aceita.

Um problema de decisão pergunta se existe um algoritmo que determine se proposições individuais de uma classe grande de proposições são ou não verdadeiras; se não existir nenhum algoritmo, o problema é tido como insolúvel.

Problemas de decisão insolúveis foram encontrados em diversos contextos neste século.

O problema da parada é insolúvel.

$P \subseteq NP$, mas não é sabido se $P \subset NP$.

Exercícios 8.3

★1. Considere a máquina de Turing a seguir:

(0, 0, 0, 0, E)
 (0, 1, 0, $\backslash$, D)
 (0, b , b , 0, E)
 (1, 0, 0, $\backslash$, D)
 (1, 1, 0, 1, D)

- a. Qual é seu comportamento, quando iniciada na fita

...	<i>b</i>	1	0	0	1	1	<i>b</i>	...
-----	----------	---	---	---	---	---	----------	-----

- b. Qual é seu comportamento, quando iniciada na fita

...	<i>b</i>	0	0	1	1	1	<i>b</i>	...
-----	----------	---	---	---	---	---	----------	-----

2. Dada a máquina de Turing

(0, 1, 1,0, D)
 (0,0,0, 1,D)
 (1,1, 1,1, D)
 (1,*b*, 1,2, E)
 (2,1, 1,2,E)
 (2, 0, 0, 2, E)
 (2,*b*, 1,0, D)

- a. Qual seu funcionamento, quando iniciada na fita

...	<i>b</i>	1	0	1	0	<i>b</i>	...
-----	----------	---	---	---	---	----------	-----

- b. Qual seu funcionamento quando iniciada na fita

...	<i>b</i>	1	0	1	<i>b</i>	...
-----	----------	---	---	---	----------	-----

3. Encontre uma máquina de Turing que substitua todos os Os de uma cadeia de Os e 1 s por 1 s e todos os 1 s por Os.
4. Encontre uma máquina de Turing que aceite o conjunto de todas as cadeias de Os e 1s contendo pelo menos um 1.
- ★5. Encontre uma máquina de Turing que aceite o conjunto de todas as cadeias unárias que consistam apenas em um número par de 1s (incluindo a cadeia vazia).
6. Encontre uma máquina de Turing que aceite 0^*10^*1 .
7. Encontre uma máquina de Turing que aceite o conjunto de cadeias não-vazias de parênteses bem-balanceadas. (Perceba que $()(())$ é bem balanceada e $()(())$ não é.)
- ★8. Encontre uma máquina de Turing que aceite $\{0^{2^n}1^n2^{2^n} \mid n \geq 0\}$.
9. Encontre uma máquina de Turing que aceite $\{w * w^R \mid w \in \{0, 1\}^* \text{ e } w^R \text{ é o reverso da cadeia } w\}$.
10. Encontre uma máquina de Turing que reconheça $\{w_1 * w_2 \mid w_1, w_2 \in \{0, 1\}^* \text{ e } w_1 \neq w_2\}$.
11. Encontre uma máquina de Turing que reconheça o conjunto de palíndromos em $\{0, 1\}^*$, isto é, o conjunto de todas as cadeias em $\{0, 1\}^*$ que são iguais, se lidas do início para o fim e do fim para o início, tais como 101.
12. Encontre uma máquina de Turing que converta uma cadeia de Os e 1s representando um número binário não-nulo em uma cadeia com este número de 1s. Como exemplo, a máquina deve, quando iniciada em uma fita contendo

...	<i>b</i>	1	0	0	<i>b</i>	...
-----	----------	---	---	---	----------	-----

parar a execução com uma fita contendo

...	b	1	1	1	1	b	...
-----	-----	---	---	---	---	-----	-----

13. Encontre uma máquina de Turing que, dada uma fita inicial contendo uma cadeia não vazia de 1 s, marque o extremo direito da cadeia com um * e inclua uma cópia da cadeia ao lado do *. Como exemplo, a máquina deve, quando iniciada em uma fita contendo

...	b	1	1	1	b	...
-----	-----	---	---	---	-----	-----

parar com uma fita contendo

...	b	1	1	1	*	1	1	1	b	...
-----	-----	---	---	---	---	---	---	---	-----	-----

- ★14. Que número de funções número-teóricas de três variáveis são computadas pela máquina de Turing dada abaixo?

$(0, 1, b, 0, D)$

$(0, *, b, 1, D)$

$(1, 1, 1, 2, D)$

$(2, *, *, 3, D)$

$(3, 1, 1, 2, E)$

$(2, 1, 1, 4, D)$

$(4, 1, 1, 4, D)$

$(4, *, 1, 5, D)$

$(5, 1, b, 5, D)$

$(5, b, b, 6, D)$

15. Encontre uma máquina de Turing que compute a função

$$f(n) = \begin{cases} n & \text{se } n \text{ for par} \\ n+1 & \text{se } n \text{ for ímpar} \end{cases}$$

- ★16. Encontre uma máquina de Turing que compute a função

$$f(n) = \begin{cases} 1 & \text{se } n = 0 \\ 2 & \text{se } n \neq 0 \end{cases}$$

17. Encontre uma máquina de Turing que compute a função

$$f(n) = 2n$$

18. Encontre uma máquina de Turing que compute a função

$$f(n) = \begin{cases} n/3 & \text{se } n \text{ é múltiplo de } 3 \\ \text{indefinida,} & \text{caso contrário} \end{cases}$$

- ★19. Encontre uma máquina de Turing que compute a função

$$f(n_1, n_2) = n_1 + n_2$$

20. Encontre uma máquina de Turing que compute a função

$$f(n_1, n_2) = \begin{cases} n_1 - n_2 & \text{se } n_1 \geq n_2 \\ 0 & \text{caso contrário} \end{cases}$$

21. Encontre uma máquina de Turing que compute a função

$$f(n_1, n_2) = \max(n_1, n_2)$$

22. Refaça o Exercício 17, desta vez fazendo uso das máquinas T_1 e T_2 dos Exercícios 13 e 19, respectivamente, como "procedimentos". (Formalmente, os estados dessas máquinas devem ser renumerados à medida que as quintuplas são incluídas no "programa principal", mas você pode omitir este detalhe desagradável e simplesmente "chamar T_1 " ou "chamar T_2 ")
23. Descreva verbalmente as ações de uma máquina de Turing que compute a função $f(n_1, n_2) = n_1 \cdot n_2$, isto é, projete o algoritmo, mas não se preocupe em criar todas as quintuplas necessárias. Você pode fazer uso dos resultados dos Exercícios 13 e 19.

Seção 8.4 Linguagens Formais

Suponha que ouvimos a sentença em português "Os carneiros falam muito alto". Apesar de poder nos surpreender com seu significado, ou *semântica*, aceitamos a forma da sentença, ou sua *sintaxe*, como válida na linguagem, o que significa que as várias partes que a compõem (substantivos, verbos etc.) estão encadeadas de maneira correta. Por sua vez, rejeitamos "Alto carneiros os falam" por ser uma combinação ilegal das partes que a compõem, sintaticamente incorreta e estranha à linguagem. Também devemos nos preocupar com a sintaxe nas linguagens de programação, mas nessas, ao invés das linguagens naturais (português, inglês etc.), as combinações de símbolos são especificadas em detalhes. Vejamos uma definição formal de *linguagem*; essa definição será geral o bastante para abranger tanto as linguagens naturais quanto as linguagens de programação.

Definições: Alfabeto, Vocabulário, Palavra, Linguagem

Um **alfabeto**, ou **vocabulário**, V é um conjunto finito não-vazio de símbolos. Uma **palavra** sobre V é uma cadeia de comprimento finito de símbolos de V . O conjunto V^* é o conjunto de todas as palavras sobre V . (Veja o Exemplo 23 no Cap. 2 para uma definição recursiva de V^* .) Uma **linguagem** sobre V é qualquer subconjunto de V^* .

Considerando a língua portuguesa um subconjunto de L , o conjunto de todas as cadeias sobre o alfabeto usual, percebemos que "Os carneiros falam alto" pertence a L , enquanto que "Alto carneiros os falam" não pertence.

Como podemos descrever qualquer linguagem L dada, isto é, especificar exatamente quais palavras pertencem a L ? Se L for finita, podemos apenas enumerar seus elementos, mas se L é infinita, será que podemos encontrar uma descrição finita para L ? Nem sempre — existem mais linguagens do que possíveis descrições finitas. Apesar de só considerarmos as linguagens que podem ser definidas de forma finita, podemos pensar de duas maneiras. Podemos ser capazes de descrever um algoritmo para *determinar* se uma cadeia pertencia a L ; isto é, dada qualquer palavra em V^* , podemos usar nosso algoritmo e receber um sim ou não como resposta que indique se a palavra pertence ou não a L . Ou podemos ser capazes de descrever um procedimento que nos permita apenas *gerar* elementos de L , isto é, produzir, um por vez, uma lista de todos os elementos de L . Vamos estabelecer as linguagens para as quais esta segunda opção é possível e descrever linguagens deste tipo através de seu processo de geração, ou fornecendo uma *gramática* para a linguagem.

Antes de darmos uma definição formal do que vem a ser uma gramática, vamos ver novamente porque "Os carneiros falam alto" parece ser uma sentença aceitável vendo como ela pode ser gerada. Começando da noção de sentença, aceitamos que uma forma legítima de sentença é uma expressão nominal seguida de uma expressão verbal. Simbolicamente,

sentença $\rightarrow$ expressão-nominal expressão-verbal

Uma forma legítima de expressão nominal é um artigo seguido por um substantivo,

expressão-nominal $\rightarrow$ artigo substantivo

e uma forma legítima de expressão verbal é um verbo seguido por um advérbio,

expressão-verbal $\rightarrow$ verbo advérbio

Também devemos concordar com as substituições

artigo $\rightarrow$ os
substantivo $\rightarrow$ carneiros
verbo $\rightarrow$ falam
advérbio $\rightarrow$ alto

Portanto, podemos gerar a sentença "Os carneiros falam alto", realizando, sucessivamente, as substituições:

sentença $\Rightarrow$ expressão-nominal expressão-verbal
 $\Rightarrow$ artigo substantivo expressão-verbal
 $\Rightarrow$ Os substantivo expressão-verbal
 $\Rightarrow$ Os carneiros expressão-verbal
 $\Rightarrow$ Os carneiros verbo advérbio
 $\Rightarrow$ Os carneiros falam advérbio
 $\Rightarrow$ Os carneiros falam alto

Os termos em negrito são os que podem ser futuramente substituídos. Os termos que não estão em negrito param ou terminam o processo de substituição. Essas idéias são incorporadas na definição a seguir.

Definição: Gramática com Estrutura de Frase (Tipo 0)

Uma gramática com estrutura de frase (gramática tipo 0) G é uma 4-upla, $G = (V, V_T, S, P)$ onde

V = vocabulário
 V_T = conjunto não-vazio de V chamado de terminais
 S = elemento de $V - V_T$ chamado de símbolo inicial
 P = conjunto finito de produções da forma $\alpha \rightarrow \beta$ onde α é uma palavra sobre V contendo pelo menos um símbolo não-terminal e β é uma palavra sobre V

EXEMPLO 33 Temos aqui uma gramática muito simples: $G = (V, V_T, S, P)$, onde $V = \{0, 1, S\}$, $V_T = \{0, 1\}$ e $P = \{S \rightarrow 0S, S \rightarrow 1\}$.

As produções de uma gramática nos permitem transformar algumas palavras sobre V em outras; as produções podem ser chamadas de regras de derivação.

Definição: Gerações (Derivações) em uma Linguagem

Seja G uma gramática, $G = (V, V_T, S, P)$ e seja w_1 e w_2 palavras sobre V . Então w_1 **gera diretamente (deriva diretamente)** w_2 , denotado por $w_1 \Rightarrow w_2$, se $\alpha \rightarrow \beta$ é uma produção de G , w_1 contém uma cópia de α e w_2 é obtido de w_1 através da substituição de α por β . Se $w_1, w_2, \dots, w_n$ são palavras sobre V e $w_1 \Rightarrow w_2, w_2 \Rightarrow w_3, \dots, w_{n-1} \Rightarrow w_n$, então w_1 **gera (deriva)** w_n , denotado por $w_1 \xRightarrow{*} w_n$. (Por convenção, $w_1 \xRightarrow{*} w_1$.)

EXEMPLO 34 Na gramática do Exemplo 33, $00S \Rightarrow 000S$ porque a produção $S \rightarrow 0S$ foi usada para substituir o S em $00S$ por $0S$. Além disso, $00S \xRightarrow{*} 00000S$. •

PRÁTICA 52 Mostre que na gramática do Exemplo 33, $0S \xRightarrow{*} 00001$. •

Definição: Linguagem Gerada por uma Gramática

Dada uma gramática G , a **linguagem L gerada por G** , algumas vezes denotada por $L(G)$, é o conjunto

$$L = \{w \in V_T^* \mid S \xRightarrow{*} w\}$$

Em outras palavras, L é o conjunto de todas as cadeias de terminais geradas a partir do símbolo inicial.

Perceba que uma vez que uma cadeia w de terminais tenha sido obtida, não podem mais ser aplicadas produções a w , e w não pode gerar quaisquer outras palavras.

O procedimento a seguir gera uma lista de elementos de L : Comece com o símbolo inicial S e aplique sistematicamente alguma seqüência de produção até que a cadeia w_1 de terminais tenha sido encontrada; então $w_1 \in L$. Volte para S e repita este procedimento usando uma seqüência diferente de produções para gerar outra palavra $w_2 \in L$, e assim por diante. Na verdade, este procedimento não tem seu funcionamento garantido por-

que poderíamos seguir uma seqüência infinita de derivações que nunca nos levaria a uma cadeia de terminais e, portanto, nunca incluiria uma palavra em nossa lista. Ao invés desse procedimento, precisamos efetuar diversas derivações de S simultaneamente, verificando cada uma a cada passo e incluindo as palavras finais na lista de elementos de L sempre que encontrássemos uma. Desta maneira nunca ficaremos esperando indefinidamente, incapazes de encontrar outras palavras em função de uma eventual seqüência infinita de derivações.

PRÁTICA 53 Descreva a linguagem gerada pela gramática G do Exemplo 33. •

As linguagens geradas por gramáticas, da forma que definimos, são chamadas de **linguagens formais**. Se a gramática é definida em primeiro lugar, a linguagem é uma conseqüência da definição. Alternativamente, a linguagem, enquanto conjunto bem-definido de cadeias, pode ser especificada em primeiro lugar e então procuramos uma gramática para ela.

EXEMPLO 35 Seja L o conjunto de todas as cadeias não-vazias que consistem em um número par de 1s. Então L é gerada pela gramática $G = (V, V_T, S, P)$ onde $V = \{1, S\}$, $V_T = \{1\}$ e $P = \{S \rightarrow SS, S \rightarrow 11\}$. Uma linguagem pode ser gerada por mais de uma gramática. L também pode ser gerada pela gramática $G' = (V', V'_T, S', P')$ onde $V' = \{1, S\}$, $V'_T = \{1\}$ e $P' = \{S \rightarrow 1S1, S \rightarrow 11\}$.

PRÁTICA 54 a. Encontre uma gramática que gere a linguagem $\{0^n 10^n \mid n \geq 0\}$.
b. Encontre uma gramática que gere a linguagem $\{0^n 10^n \mid n \geq 1\}$.

Tentar descrever de forma concisa a linguagem definida por uma gramática e definir uma gramática para gerar uma dada linguagem podem ser tarefas bem difíceis. Veremos outro exemplo onde a gramática é um tanto mais complicada do que as que vimos até agora.

EXEMPLO 36 Seja $L = \{a^n b^n c^n \mid n \geq 1\}$. Uma gramática que gera L é $G = (V, V_T, S, P)$ onde $V = \{a, b, c, S, B, C\}$, $V_T = \{a, b, c\}$ e P consiste nas seguintes produções:

1. $S \rightarrow aSBC$
2. $S \rightarrow aBC$
3. $CB \rightarrow BC$
4. $aB \rightarrow ab$
5. $bB \rightarrow bb$
6. $bC \rightarrow bc$
7. $cC \rightarrow cc$

É bem simples ver como gerar qualquer elemento particular de L usando essas produções. Portanto, uma derivação da cadeia $a^2 b^2 c^2$ é

$$\begin{aligned} S &\Rightarrow aSBC \\ &\Rightarrow aaBCBC \\ &\Rightarrow aaBBCC \\ &\Rightarrow aabBCC \\ &\Rightarrow aabbCC \\ &\Rightarrow aabbccC \\ &\Rightarrow aabbcc \end{aligned}$$

Em geral, $L \subseteq L(G)$ onde o esquema da derivação de qualquer $a^n b^n c^n$ é dado a seguir; os números indicam as produções usadas.

$$\begin{aligned} S &\xRightarrow{1} a^{n-1} S(BC)^{n-1} \\ &\xRightarrow{2} a^n (BC)^n \\ &\xRightarrow{3} a^n B^n C^n \\ &\xRightarrow{4} a^n b B^{n-1} C^n \\ &\xRightarrow{5} a^n b^n C^n \\ &\xRightarrow{6} a^n b^n c C^{n-1} \\ &\xRightarrow{7} a^n b^n c^n \end{aligned}$$

Também precisamos mostrar que $L(G) \subseteq L$, o que envolve demonstrar que algumas produções precisam ser usadas antes de outras e que a derivação geral mostrada acima é o único tipo que nos leva à cadeia de terminais. •

Ao tentar inventar uma gramática para gerar o conjunto L do Exemplo 36, poderíamos ter tentado usar as produções da forma $B \rightarrow b$ e $C \rightarrow c$ ao invés das produções 4 a 7. Portanto, teremos, de fato, $L \subseteq L(G)$, mas $L(G)$ conterá palavras como $a^n(bc)^n$.

As linguagens formais foram desenvolvidas na década de 50 pelo lingüístico Noam Chomsky como uma tentativa de modelar as linguagens naturais, tais como o inglês, com vistas a tradução automática. No entanto, uma vez que existe uma linguagem natural e é bem complexa, definir uma gramática formal para gerar uma linguagem natural é bem difícil. Tentativas de fazer isto para o inglês têm conseguido apenas sucessos parciais.

EXEMPLO 37 Podemos descrever uma gramática formal que gere uma classe bem restrita de sentenças em português. Os terminais da gramática são as palavras "os", "um", "legal", "rio", "carneiro", "fala", "corre", "alto", "velozmente", e os "não-terminais" são as palavras **sentença**, **expressão-nominal**, **expressão-verbal**, **artigo**, **substantivo**, **verbo** e **advérbio**. O símbolo inicial é **sentença** e as produções são:

sentença $\rightarrow$ **expressão-nominal** **expressão-verbal**
expressão-nominal $\rightarrow$ **artigo** **pronomes**
expressão-verbal $\rightarrow$ **verbo** **advérbio**
artigo $\rightarrow$ os
artigo $\rightarrow$ um
substantivo $\rightarrow$ rio
substantivo $\rightarrow$ carneiro
verbo $\rightarrow$ fala
verbo $\rightarrow$ corre
advérbio $\rightarrow$ alto
advérbio $\rightarrow$ velozmente

Já sabemos derivar "Os carneiros falam alto" nesta gramática. Eis uma derivação de "Um rio corre velozmente":

sentença $\Rightarrow$ **expressão-nominal** **expressão-verbal**
 $\Rightarrow$ **artigo** **substantivo** **expressão-verbal**
 $\Rightarrow$ um **substantivo** **expressão-verbal**
 $\Rightarrow$ um rio **expressão-verbal**
 $\Rightarrow$ um rio **verbo** **advérbio**
 $\Rightarrow$ um rio corre **advérbio**
 $\Rightarrow$ um rio corre velozmente

Diversas outras sentenças com diversos níveis de coerência, tais como um "carneiro corre alto" também fazem parte da linguagem definida por esta gramática. A dificuldade de especificar uma gramática para o inglês como um todo se torna mais aparente quando consideramos frases como "time flies" que pode ser interpretada como um substantivo seguido de um verbo (tempo voa) ou um verbo seguido de um substantivo (cronometrar moscas)! Esta situação é "ambígua" e ocorre em todas as linguagens naturais de uma maneira ou de outra (veja o Exercício 16 nesta seção). •

As linguagens de programação são menos complexas que as linguagens naturais e algumas linguagens de programação, tais como ALGOL, foram definidas como linguagens formais.

EXEMPLO 38 Uma aplicação de gramática formal para gerar identificadores em uma linguagem de programação pode ser apresentada como o seguir:

identificador $\rightarrow$ **letra**
identificador $\rightarrow$ **identificador** **letra**
identificador $\rightarrow$ **identificador** **dígito**
letra $\rightarrow$ a
letra $\rightarrow$ b

letra $\rightarrow$ z
dígito $\rightarrow$ 0
dígito $\rightarrow$ 1

dígito $\rightarrow$ 9

Neste caso, o conjunto de terminais é $[a, b, \dots, z, 0, 1, \dots, 9]$ e o símbolo inicial é **identificador**. Uma simplificação que permite listar todas essas produções é a chamada forma de **Backus-Naur** (BNF). As produções listadas acima podem ser especificadas em BNF em três linhas (como no Exemplo 24 do Cap. 2):

$$\begin{aligned} \langle \text{identificador} \rangle &::= \langle \text{letra} \rangle \mid \langle \text{identificador} \rangle \langle \text{letra} \rangle \mid \langle \text{identificador} \rangle \langle \text{dígito} \rangle \\ \langle \text{letra} \rangle &::= a \mid b \mid c \mid \dots \mid z \\ \langle \text{dígito} \rangle &::= 0 \mid 1 \mid \dots \mid 9 \end{aligned}$$

Na BNF, os não-terminais são identificados por $\langle \rangle$, as setas de produção se tornam $::=$, e $\mid$ significa ou, identificando várias produções com o mesmo símbolo à esquerda. •

Classes de Gramática

Antes de identificarmos alguns tipos de gramática, vamos dar uma olhada em mais um exemplo.

EXEMPLO 39 Seja L uma cadeia vazia /juntamente com o conjunto de todas as cadeias que consistem em um número ímpar n de Os, $n \geq 3$. A gramática $G = \{V, V_T, S, P\}$ gera L onde $V = \{0, A, B, E, F, W, X, Y, Z, S\}$, $V_T = \{0\}$, e as produções são

$$\begin{array}{lll} S \rightarrow FA & OX \rightarrow XO & OZ \rightarrow ZG \\ S \rightarrow FBA & YO \rightarrow OY & WBZ \rightarrow Et \\ FB \rightarrow FOEBO & FX \rightarrow FOW & F \rightarrow \lambda \\ EB \rightarrow O & YA \rightarrow ZOA & A \rightarrow \lambda \\ EB \rightarrow XBY & WO \rightarrow OW & \end{array}$$

A derivação $S \Rightarrow FA \xRightarrow{*} \lambda\lambda = \lambda$ produz λ . A derivação

$$\begin{aligned} S &\Rightarrow FBA \\ &\Rightarrow FOEBOA \\ &\Rightarrow FOXBYOA \\ &\xRightarrow{*} FXOBOYA \\ &\xRightarrow{*} FOWOBZOYA \\ &\xRightarrow{*} FOOWBZOOA \\ &\Rightarrow FOOEBOOA \\ &\Rightarrow F00000A \\ &\xRightarrow{*} 00000 \end{aligned}$$

produz cinco Os. Perceba como X e Y , e também W e Z vão para frente e para trás nas cadeias de Os, incluindo mais um 0 em cada lado. Esta atividade é muito parecida com o movimento de uma cabeça de leitura/gravação de uma máquina de Turing pela fita aumentando a parte usada da fita. •

A gramática acima permite a produção de cadeias vazias $F \rightarrow \lambda$ e $A \rightarrow \lambda$. Para gerar qualquer linguagem contendo λ , precisamos ser capazes de produzir cadeias vazias. Nas gramáticas que veremos a seguir, nos limitaremos a este tipo de produção. Se houver uma produção de cadeia vazia, ela será da forma $S \rightarrow \lambda$, onde S é o símbolo inicial e não pertence ao lado direito de quaisquer outras produções. Esta restrição nos permite tratar a produção de λ a partir de S como um caso especial e então passar às demais derivações, nenhuma das quais permite qualquer produção de cadeia vazia. Vamos chamar isto de **convenção da produção vazia**. A definição a seguir define três tipos especiais de gramáticas através de restrições nos tipos de produções permitidas.

Definições: Gramáticas Sensível ao Contexto, Livre de Contexto e Regular; Hierarquia de Chomsky

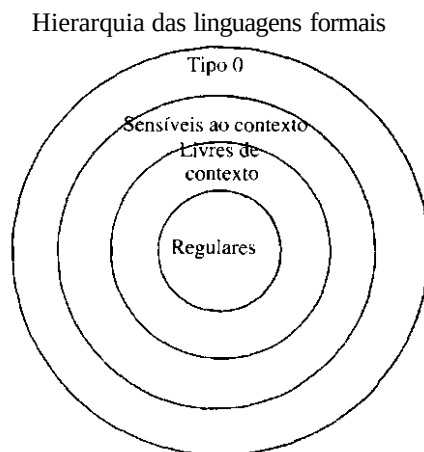
Uma gramática G é **sensível ao contexto (tipo 1)** se obedece a convenção da produção vazia e se, para qualquer produção $\alpha \rightarrow \beta$ (exceto $S \rightarrow \lambda$), a palavra β é pelo menos tão grande quanto a palavra α . Uma gramática G é **livre de contexto (tipo 2)** se obedece a convenção da produção vazia e para qualquer produção $\alpha \rightarrow \beta$, α é um não-terminal único. Uma gramática G é **regular (tipo 3)** se ela obedece a convenção da produção vazia e para qualquer produção $\alpha \rightarrow \beta$ (exceto $S \rightarrow \lambda$), α é um não-terminal único e β é da forma t ou tW , onde t é um símbolo terminal e W é um símbolo não-terminal. Esta hierarquia de gramáticas, do tipo 0 ao tipo 3 também é chamada **hierarquia de Chomsky**.

Em uma gramática livre de contexto, um único símbolo não-terminal à esquerda da produção pode ser substituído, sempre que aparecer, pelo símbolo do lado direito da produção. Em uma gramática sensível ao contexto, um dado símbolo não-terminal pode vir a ser substituído apenas se for parte de uma cadeia em particular, ou contexto — daí os nomes *livre de contexto* e *sensível ao contexto*. Está claro que qualquer gramática regular também é livre de contexto e que qualquer gramática livre de contexto também é sensível de contexto. A gramática do Exemplo 33 é regular (as duas produções têm o único não-terminal S do lado esquerdo, e do lado direito tem 1 — um terminal — ou 05 — um terminal seguido de um não-terminal). Ambas as gramáticas do Exemplo 35 são livres de contexto, mas não são regulares (novamente, apenas o não-terminal S aparece do lado esquerdo das produções, mas os lados direitos consistem em três símbolos ou dois não-terminais ou dois terminais, respectivamente). A gramática do Exemplo 36 é sensível ao contexto mas não é livre de contexto (a produção não transforma quaisquer cadeias em cadeias vazias, mas existem diversos símbolos no lado esquerdo de algumas produções). As gramáticas do Exemplo 37 e do Exemplo 38 são gramáticas livres de contexto, mas não-regulares (por exemplo, as três primeiras produções do Exemplo 38 violam o requisito para uma gramática regular). Finalmente, a gramática do Exemplo 39 é uma gramática do tipo 0, mas não é sensível ao contexto (por exemplo, $EB \rightarrow 0$ é uma contração; além disso, a convenção da produção vazia é violada).

Definição: Tipos de Linguagens

Uma linguagem é **tipo 0 (sensível ao contexto, livre de contexto ou regular)** se puder ser gerada por uma gramática tipo 0 (sensível ao contexto, livre de contexto ou regular).

Devido às relações com quatro tipos de gramáticas, podemos classificar as linguagens como mostrado na Fig. 8.17. Portanto, qualquer linguagem regular é também livre de contexto, porque qualquer gramática regular também é uma gramática livre de contexto, e assim por diante. No entanto, apesar de isto ser verdade, não temos embasamento para afirmar que esses conjuntos estão contidos propriamente uns nos outros. Por exemplo, a linguagem L descrita no Exemplo 39 foi descrita neste exemplo por uma gramática que era tipo 0 mas não-sensível ao contexto, mas isto não implica que L esteja nesta categoria. Diferentes gramáticas podem gerar a mesma linguagem.

**Figura 8.17**

Definição: Gramáticas Equivalentes

Duas gramáticas são **equivalentes** se gerarem a **mesma linguagem**.

EXEMPLO 40 O Exemplo 39 apresentou uma gramática G para gerar a linguagem L . Forneceremos três outras gramáticas equivalentes a G .

$G_1 = (V, V_T, S, P)$ onde $V = \{0, A, B, S\}$, $V_T = \{0\}$ e as produções são

$$\begin{array}{ll} S \rightarrow \lambda & AB \rightarrow 00 \\ S \rightarrow ABA & 0A \rightarrow 000A \\ & A \rightarrow 0 \end{array}$$

G_1 é sensível ao contexto, mas não é livre de contexto.

$G_2 = (V, V_T, S, P)$ onde $V = \{0, A, S\}$, $V_T = \{0\}$ e as produções são

$$\begin{array}{ll} S \rightarrow \lambda & A \rightarrow 00A \\ S \rightarrow 00A & A \rightarrow 0 \end{array}$$

G_2 é livre de contexto, mas não é regular.

$G_3 = (V, V_T, S, P)$ onde $V = \{0, A, B, C, S\}$, $V_T = \{0\}$ e as produções são

$$\begin{array}{ll} S \rightarrow \lambda & B \rightarrow 0 \\ S \rightarrow 0A & B \rightarrow 0C \\ A \rightarrow 0B & C \rightarrow 0B \end{array}$$

G_3 é regular.

Portanto, L é uma linguagem regular. •

PRÁTICA 55 Forneça a derivação de 00000 em G_1, G_2 e G_3 . •

Linguagens Formais e Dispositivos Computacionais

A linguagem L do Exemplo 39 pode ser descrita pela expressão regular $A \vee (000)(00)^*$, de forma que L é um conjunto regular. Do exemplo acima, L também é uma linguagem regular. Não é coincidência que um conjunto regular seja uma linguagem regular. Pode ser mostrado que para qualquer máquina de estado finito, o conjunto que ela aceita é uma linguagem regular. Também podemos mostrar que para qualquer linguagem regular, existe uma máquina de estado finito que reconhece exatamente esta linguagem. (Nas demonstrações desses resultados, as produções de uma gramática regular correspondem às transições de estados de uma máquina de estado finito.) Portanto, os conjuntos aceitos por máquinas de estado finito — os conjuntos regulares — correspondem às linguagens regulares. Portanto a classe de conjuntos aceitos por um dispositivo computacional de capacidade limitada coincide com a classe mais restrita de linguagens.

No outro lado desta questão temos que o dispositivo computacional mais geral é a máquina de Turing e a linguagem mais geral é a de tipo 0. Por isso, os conjuntos reconhecidos por máquinas de Turing são as linguagens de tipo 0.

Existem dispositivos computacionais com capacidades intermediárias entre as máquinas de estado finito e as máquinas de Turing; esses dispositivos aceitam exatamente as linguagens livre de contexto e as linguagens sensíveis ao contexto, respectivamente. O tipo de dispositivo que aceita linguagens livre de contexto é chamado de **autômato de pilha** ou **pda** (do inglês push-down automaton). Um autômato de pilha consiste em uma unidade de estado finito que lê a entrada de uma fita e controla a atividade de uma pilha. Os símbolos do alfabeto podem ser empilhados ou desempilhados do topo da pilha.

A unidade de estado finito, como uma função do símbolo de entrada lido, do estado atual e do símbolo no topo da pilha, tem um número finito de movimentos a realizar. Os movimentos podem ser dos seguintes tipos:

1. passa ao estado seguinte, retira o símbolo do topo da pilha e lê o próximo símbolo da entrada;
2. passa a um novo estado, retira o símbolo do topo da pilha, inclui um número finito de símbolos no topo da pilha e lê o próximo símbolo da entrada; ou
3. ignora o símbolo da entrada sendo lido, manipula a pilha como acima, mas não lê o próximo símbolo da entrada.



Figura 8.18

Um autômato de pilha tem uma escolha de próximos movimentos e ele reconhece o conjunto de todas as entradas para as quais existe alguma sequência de movimentos que faça a pilha ficar vazia. Pode ser mostrado que qualquer conjunto reconhecido por um autômato de pilha é uma linguagem livre de contexto e vice-versa.

O tipo de dispositivo que aceita linguagens sensíveis ao contexto é chamado de **autômato limitado linear**, ou **lba** (do inglês linear bounded automaton). Um lba é uma máquina de Turing cuja cabeça de leitura/gravação está limitada à parte da fita que contém a entrada original; além disso, a cada passo ela tem uma escolha de quais movimentos realizar. Um lba aceita o conjunto de todas as entradas para as quais existe alguma sequência de movimentos que faça a máquina de Turing parar em algum estado. Pode ser mostrado que qualquer conjunto aceito por um lba é uma linguagem sensível ao contexto e vice-versa.

A Fig. 8.18 mostra a relação entre a hierarquia de linguagens e a hierarquia de dispositivos computacionais.

Gramáticas Livres de Contexto

As gramáticas livres de contexto são importantes por três razões. Elas parecem ser as mais simples de se trabalhar, uma vez que só permitem a substituição de um símbolo por vez. Além disso, diversas linguagens de programação são definidas de tal forma que partes de sua sintaxe, se não toda, podem ser descritas através de gramáticas livres de contexto. Finalmente, uma derivação em uma gramática livre de contexto tem uma representação gráfica muito adequada, chamada de **árvore de parse**.

EXEMPLO 41 A gramática do Exemplo 38 é livre de contexto. A palavra $d2q$ pode ser derivada da seguinte maneira: **identificador** $\Rightarrow$ **identificador letra** $\Rightarrow$ **identificador dígito letra** $\Rightarrow$ **letra dígito identificador** $\Rightarrow$ **d dígito letra** $\Rightarrow$ **dl letra** $\Rightarrow$ **dlq**. Podemos representar esta derivação como uma árvore cuja raiz é o símbolo inicial. Quando uma produção é aplicada em um vértice, este vértice é substituído no nível inferior seguinte pela árvore dos símbolos do lado direito da produção usada. Uma árvore para a derivação acima é mostrada na Fig. 8.19.

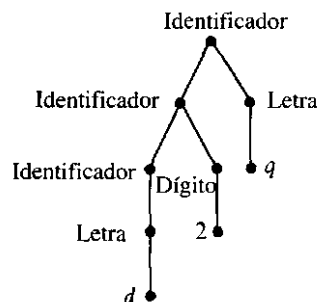


Figura 8.19

PRÁTICA 56 Desenhe uma árvore de parse para a palavra $m34s$ na gramática do Exemplo 38. •

Suponha que uma gramática livre de contexto G descreve uma linguagem de programação. O programador usa as regras de G para gerar cadeias válidas de símbolos, isto é, palavras da linguagem. Neste caso podemos imaginar uma palavra como uma instrução do programa. Portanto, uma palavra consiste em várias subpalavras, por exemplo, identificadores, operadores e palavras-chave da linguagem (como **repeat**, e **until** do Pascal). As instruções do programa alimentam o compilador da linguagem de forma que o programa seja traduzido para a linguagem de máquina do computador. O compilador deve determinar se as instruções do programa são ou não válidas para a linguagem. Esta questão divide-se em duas perguntas: As subpalavras são cadeias válidas e a instrução do programa é uma forma válida de agrupar as subpalavras?

Normalmente, o conjunto de subpalavras válidas de uma linguagem pode ser descrito por uma expressão regular e, portanto, uma máquina de estado finito pode ser usada para detectar as subpalavras; o analisador léxico, ou scanner do compilador, se encarrega desta fase da compilação. Se tudo correr bem, o analisador léxico passa a instrução do programa na forma de uma cadeia de subpalavras legítimas ao analisador sintático. O analisador sintático determina se a cadeia é correta tentando realizar o parsing nela (construir a árvore de parse).

Várias técnicas para realizar o parsing, sobre as quais não entraremos em detalhes, foram desenvolvidas. Uma abordagem óbvia é construir uma árvore começando pelo símbolo inicial, aplicando produções e terminando com a cadeia a ser testada. Este procedimento é chamado de **parsing top-down**. A alternativa é começar com a cadeia, ver quais produções foram usadas para criá-las, aplicar as produções ao contrário e terminar no símbolo inicial. Este processo é chamado de **parsing bottom-up**. O artifício necessário em ambas abordagens é determinar exatamente que regras de produção devem ser usadas.

EXEMPLO 42 Considere a gramática livre de contexto G dada por $G = (V, V_T, S, P)$ onde $V = \{a, b, c, A, B, C, S\}$. $V_T = \{a, b, c\}$ e as produções são

$$\begin{array}{lll} S \rightarrow B & B \rightarrow C & A \rightarrow abc \\ S \rightarrow A & B \rightarrow ab & C \rightarrow c \end{array}$$

Suponha que desejamos testar a cadeia abe . Uma derivação para abe é $S \Rightarrow A \Rightarrow abc$. Se fizermos um parsing top-down, podemos começar por

$$\begin{array}{c} S \\ | \\ B \end{array}$$

Então temos que determinar que isto não funciona e tentar mais alguma coisa. Se tentarmos um parsing bottom-up, devemos começar por

$$\begin{array}{c} BC \\ | \\ abc \end{array}$$

Então temos que determinar que isto não funciona e tentar mais alguma coisa. As técnicas para parsing realizam este processo de forma automática. •

Perceba a diferença entre *gerar* elementos de um conjunto, que o programador faz, e *determinar* se um elemento pertence a um conjunto, o que o compilador faz. Uma vez que pedimos ao compilador para determinar se os elementos pertencem a um conjunto, deve existir um algoritmo de decisão para o conjunto. Isto significa que existem algoritmos de decisão para linguagens livres de contexto, outro ponto a seu favor.

Revisão da Seção 8.4

Técnicas

- Descrever $L(G)$ para uma dada gramática G
- Definir uma gramática a fim de gerar uma dada linguagem L
- Construir árvores de parse em uma gramática livre de contexto

Idéias Principais

Uma gramática G é um mecanismo para gerar sua linguagem $L(G)$.

As linguagens formais foram desenvolvidas como uma tentativa de descrever a sintaxe correta de linguagens naturais; apesar desta tentativa ter sido mal-sucedida devido à grande complexidade das linguagens naturais, ela é muito útil para linguagens de programação de alto-nível.

Classes especiais de gramáticas são definidas pela restrição das produções permitidas.

Os vários tipos de linguagens formais correspondem aos conjuntos aceitos por diversos tipos de autômatos; em particular (1) as linguagens regulares são as linguagens aceitas por máquinas de estado finito, (2) linguagens livres de contexto são aceitas por autômatos push-down, (3) linguagens sensíveis ao contexto são os conjuntos reconhecidos por autômatos limitados lineares e (4) linguagens tipo 0 são os conjuntos reconhecidos por máquinas de Turing.

As derivações em gramáticas livres de contexto podem ser representadas por árvores de parse.

Um compilador de uma linguagem de programação livre de contexto verifica a correção da sintaxe através do processo de parsing.

Exercícios 8.4

★ 1. Descreva $L(G)$ para cada uma das gramáticas G dadas.

a. $G = (V, V_T, S, P)$ onde $V = \{a, A, B, C, S\}$, $V_T = \{a\}$ e P consiste em

$$\begin{array}{ll} S \rightarrow A & B \rightarrow A \\ A \rightarrow BC & aC \rightarrow \lambda \\ A \rightarrow a & \end{array}$$

b. $G = (V, V_T, S, P)$ onde $V = \{0, 1, A, B, S\}$, $V_T = \{0, 1\}$ e P consiste em

$$\begin{array}{lll} S \rightarrow 0A & A \rightarrow 1BB & B \rightarrow 01 \\ S \rightarrow 1A & & B \rightarrow 11 \end{array}$$

c. $G = (V, V_T, S, P)$ onde $V = \{0, 1, A, B, S\}$, $V_T = \{0, 1\}$ e P consiste em

$$\begin{array}{lll} S \rightarrow 0 & A \rightarrow 1B & B \rightarrow 0A \\ S \rightarrow 0A & & B \rightarrow 0 \end{array}$$

d. $G = (V, V_T, S, P)$ onde $V = \{0, 1, A, S\}$, $V_T = \{0, 1\}$ e P consiste em

$$\begin{array}{ll} S \rightarrow 0S & A \rightarrow 1A \\ S \rightarrow 11A & A \rightarrow 1 \end{array}$$

- Quais das gramáticas do Exercício 1 são regulares? Quais são livres de contexto?
- Encontre gramáticas regulares para aceitar cada uma das linguagens do Exercício 1.

★ 3. Descreva $L(G)$ para a gramática $G = (V, V_T, S, P)$ onde $V = \{a, b, A, B, S\}$, $V_T = \{a, b\}$ e P consiste em

$$\begin{array}{ll} S \rightarrow AB & AB \rightarrow AAB \\ A \rightarrow a & AB \rightarrow ABB \\ B \rightarrow b & \end{array}$$

De que tipo de gramática é G ? Encontre uma gramática regular G' que aceite $L(G)$.

- Escreva as produções das seguintes gramáticas em BNF:
 - G_3 do Exemplo 40
 - G do Exercício 1(b)
 - G do Exercício 1(c)

d. G do Exercício 1 (d)

5. Encontre uma gramática que aceite o conjunto de todas as cadeias de parênteses bem-balanceadas.
6. Uma palavra w em V é um palíndromo se $w = w^R$, o reverso de uma cadeia w . Uma linguagem L é uma linguagem palíndroma se L consiste apenas em palíndromos.
 - a. Encontre uma gramática que aceite o conjunto de todos os palíndromos sobre o alfabeto $\{a, b\}$.
 - b. Seja L uma linguagem palíndroma. Demonstre que $L^R = \{w^R \mid w \in L\}$ é uma linguagem palíndroma.
 - c. Seja w um palíndromo. Demonstre que a linguagem descrita pela expressão regular w^* é uma linguagem palíndroma.
- ★7. Encontre uma gramática que aceite a linguagem $L = (0 \vee 1)^*01$.
8. Encontre uma gramática livre de contexto que aceite a linguagem $L = \{0^n 1^n \mid n \geq 0\}$.
9. Encontre uma gramática que aceite a linguagem $L = \{0^{2i} \mid i \geq 0\}$.
10. Encontre uma gramática livre de contexto que aceite a linguagem L onde L consiste nos conjuntos de todas as cadeias não-vazias de Os e Is com um número igual de Os e Is.
11. Encontre uma gramática livre de contexto que aceite a linguagem L onde L consiste no conjunto de todas as cadeias não-vazias de Os e Is cuja quantidade de Os é duas vezes a quantidade de Is.
- ★12. Encontre uma gramática livre de contexto que aceite a linguagem $L = \{ww^R \mid w \in \{0, 1\}^* \text{ e } w^R \text{ é o reverso da cadeia } w\}$.
13. Encontre uma gramática que aceite a linguagem $L = \{ww \mid w \in \{0, 1\}^*\}$.
14. Encontre uma gramática que aceite a linguagem $L = \{a^n \mid n \geq 1\}$. (Pelo Exercício 18 abaixo, L não é uma linguagem livre de contexto, então sua gramática não pode ser tão simples.)
15. Desenhe uma árvore de parse para as seguintes palavras:
 - ★a. 111111 na gramática G do Exemplo 35
 - b. 111111 na gramática G' do Exemplo 35
 - c. 011101 na gramática do Exercício 1(b)
 - d. 00111111 na gramática do Exercício 1(d)
- ★16. Considere a gramática livre de contexto $G = (V, V_T, S, P)$ onde $V = \{0, 1, A, S\}$, $V_T = \{0, 1\}$ e P consiste em

$$\begin{aligned} S &\rightarrow A1A \\ A &\rightarrow 0 \\ A &\rightarrow A1A \end{aligned}$$

Desenhe duas árvores de parse distintas para a palavra 01010 em G . Uma gramática na qual uma palavra tem mais de uma árvore de parse é chamada de **ambígua**.
17. Mostre que para qualquer gramática livre de contexto G existe uma gramática G' na qual, para qualquer produção $\alpha \rightarrow \beta$, β é uma cadeia mais longa que α , $L(G') \subseteq L(G)$ e $L(G) - L(G')$ é um conjunto finito.
18. O que mostraremos agora é o **lema do bombeamento** para linguagens livre de contexto. Seja L qualquer linguagem livre de contexto. Então existe alguma constante k tal que para qualquer palavra w em L com $|w| \geq k$, w pode ser escrito como $av_1w_2w_3w_4w_5$ com $|w_2w_3w_4| \leq k$ e $|w_2w_4| \geq 1$. Além disso, a palavra $w_1w_2^i w_3w_4^i w_5 \in L$ para todo $i \geq 0$.
 - a. Use o lema do bombeamento para mostrar que $L = \{a^n b^n c^n \mid n \geq 1\}$ não é livre de contexto.
 - b. Use o lema do bombeamento para mostrar que $L = \{a^{n^2} \mid n \geq 1\}$ não é livre de contexto.

Revisão do Capítulo 8

alfabeto	problema da parada	subgrupo próprio
grupo alternante	homomorfismo	autômato push-down (pda)
operação binária associativa	elemento neutro	reconhecimento (aceitação) por
forma de Backus-Naur (BNF)	subgrupo impróprio	uma máquina de estado finito
parsing bottom-up	alfabeto de entrada	reconhecimento (aceitação) por
leis de cancelamento	problema intratável	uma máquina de Turing
hierarquia de Chomsky	elemento inverso	expressão regular
coeficiente	isomorfismo	linguagem regular
operação binária comutativa	estados k -equivalentes	conjunto regular
grupo comutativo	linguagem	gramática regular (tipo 3)
concatenação	linguagem gerada por uma gramática G	semigrupo
linguagem livre de contexto	comprimento de uma cadeia	semigrupo de transformações em
gramática livre de contexto (tipo 2)	autômato limitado linear (lba)	um conjunto A
linguagem sensível ao contexto	monóide	símbolo inicial
gramática sensível ao contexto (tipo 1)	multiplicação módulo n	grafo de estados
problema de decisão	solução negativa para um problema de decisão	tabela de estados
grau de um polinômio	problema NP -completo	cadeia
geração (derivação) direta de uma palavra	problema NP	subgrupo
cadeia vazia	função número-teórica	símbolo
gramáticas equivalentes	ordem de um grupo	grupo de simetria de grau n
estados equivalentes	alfabeto de saída	alfabeto da fita
convenção da produção vazia	P	terminal
permutações pares e ímpares	árvore de parse	parsing top-down
estado final	função parcial	função total
máquina de estado finito	refinamento de uma partição	transposição
linguagem formal	grupo de permutação	função Turing-computável
monóide livre gerado por um conjunto A	gramática de estrutura de frase (tipo 0)	máquina de Turing
geração (derivação) de uma palavra	solução positiva para um problema de decisão	linguagem tipo 0
grupo	produção	estado inalcançável
grupo de permutações em um conjunto A		problema de decisão insolúvel (indecidível)
		vocabulário
		palavra

Autotestes

Seção 8.1

1. Uma operação binária é associativa se a ordem dos elementos sendo operados não altera o resultado.
2. A identidade i em um grupo $[G, \cdot]$ tem a propriedade de que $x^{-1} \cdot i = i \cdot x^{-1} = x^{-1}$.
3. Todo grupo também é um monóide.
4. Um grupo de ordem 10 não pode ter um subgrupo de ordem 6.
5. Se $[S, \cdot]$ e $[T, +]$ são dois grupos, então uma função $f: S \rightarrow T$ para a qual $f(x \cdot y) = f(x) + f(y)$ é um isomorfismo.

Seção 8.2

6. O próximo estado de uma máquina de estado finito é determinado por seu estado atual e o símbolo de entrada atual.
7. O conjunto de todas as cadeias terminando em 00 é regular.
8. Uma máquina de estado finito não pode passar a um estado que não tenha saída.
9. De acordo com o teorema de Kleene, um conjunto que não possa ser descrito por uma expressão regular não pode ser reconhecido por uma máquina de estado finito.
10. Em uma máquina de estado finito, estados k -equivalentes são também estados $(k + 1)$ -equivalentes.

Seção 8.3

11. Uma máquina de Turing pára se, e somente se, entrar em um estado final.
12. Uma máquina de Turing que compute a função $f(n) = n + 1$, dada uma entrada n , pára com $(n + 1)$ 1 s em sua fita.
13. A tese de Church afirma que o problema da parada é insolúvel.
14. O problema da parada diz que, dada uma máquina de Turing e sua entrada, não há algoritmo que determine se a máquina de Turing pára quando executada para essa entrada.
15. Um conjunto em P é aceito por uma máquina de Turing em um número de passos não maior do que um polinômio.

Seção 8.4

16. A linguagem gerada por uma gramática G tipo 0 é o conjunto de todas as cadeias de terminais gerados a partir do símbolo inicial através da aplicação das produções de G um número finito de vezes.
17. Começando no símbolo inicial e aplicando as produções de uma gramática G vamos obter, em algum momento, uma cadeia de terminais.
18. Uma linguagem gerada por uma gramática que é sensível ao contexto, mas não é livre de contexto, é sensível ao contexto, mas não é livre de contexto.
19. Qualquer conjunto regular é uma linguagem regular.
20. Uma árvore de parse terá tantas folhas quantos forem os terminais na palavra sendo derivada.

No Computador

Nos Exercícios 1 a 9, escreva um programa de computador que produza a saída desejada a partir da entrada dada.

1. *Entrada:* Duas palavras de um alfabeto A
Saída: A concatenação delas
2. *Entrada:* Um inteiro positivo n e um alfabeto finito A
Saída: Todas as palavras sobre A de comprimento $\leq n$
3. *Entrada:* Um inteiro positivo n
Saída: Tabelas de soma e multiplicação módulo n
4. *Entrada:* Um inteiro positivo n
Saída: Os $n!$ elementos de S_n expressos tanto na forma de vetor como na notação cíclica, a tabela de grupo de $[S_n, \circ]$ e a tabela de grupo para $[A_n, \circ]$.
5. *Entrada:* matriz: $n \times n$, $n \leq 10$, que visa a representar uma operação binária no conjunto finito dos inteiros de 1 a n
Saída: Resposta indicando se o conjunto é um grupo comutativo sob esta operação.
6. *Entrada:* Duas matrizes $n \times n$, $n \leq 10$, que representam dois grupos e uma matriz que represente uma função do primeiro grupo no segundo
Saída: Resposta indicando se a função é ou não um isomorfismo
7. *Entrada:* Um inteiro positivo n , $n \leq 50$, que representa o número de estados de uma máquina de estado finito com alfabeto de entrada = alfabeto de saída = $\{0, 1\}$ e uma matriz $n \times 3$ correspondendo à descrição da tabela de estados da máquina
Saída: Lista dos estados inalcançáveis a partir do símbolo inicial s_0
8. *Entrada:* Inteiro positivo n , $n \leq 50$, representando o número de estados de uma máquina de estado finito com alfabeto de entrada = alfabeto de saída = $\{0, 1\}$ e uma matriz $n \times 3$ representando a descrição da tabela de estados desta máquina.
Saída: matriz $m \times 3$ representando a descrição da tabela de transições de uma versão minimizada de M
9. *Entrada:* Conjunto de terminais em uma gramática e a uma descrição das produções em uma gramática; permitindo ao usuário definir um número máximo de passos para qualquer derivação

Saída: Lista das palavras na linguagem que podem ser derivadas dentro do máximo estabelecido

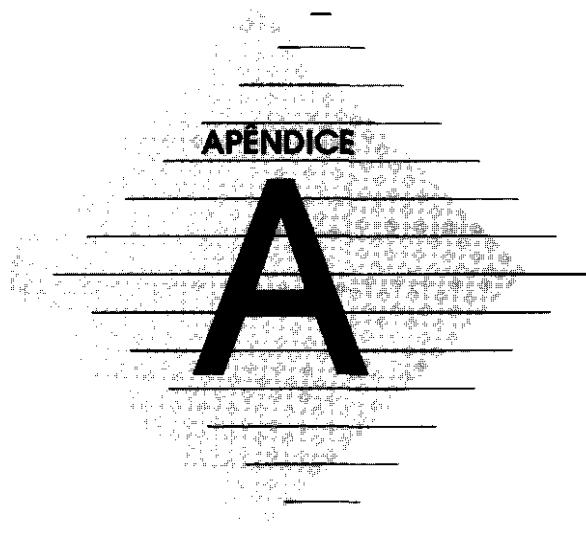
10. Escreva um simulador de máquina de estado finito. Isto é, dados:

um inteiro positivo n , $n \leq 50$, representando o número de estados em uma máquina de estado finito com alfabeto de entrada = alfabeto de saída = $\{0, 1\}$

uma matriz $n \times 3$ representando a descrição da tabela de estados da máquina

seu programa deve pedir a cadeia de entrada e escrever a cadeia de saída correspondente, sempre que o usuário pedir.

11. Escreva um simulador de máquina de Turing. Isto é, dado um conjunto de quintuplas que defina uma máquina de Turing, seu programa deve pedir a fita inicial e escrever uma sequência com as sucessivas configurações da fita. Admita que existem no máximo 100 quintuplas e que o número de células usadas na fita é no máximo 70 e permita ao usuário definir um número máximo de passos para o caso do processamento não parar antes de atingir este número.



Notação de Somatório

A notação de somatório é uma forma abreviada de escrever certas expressões de somas de termos. Como exemplo, considere a soma dos inteiros entre 1 e 5:

$$1 + 2 + 3 + 4 + 5$$

Podemos encarar esta soma da seguinte maneira: suponha que temos uma determinada quantidade i que inicialmente vale 1 e que então recebe os sucessivos valores 2, 3, 4 e 5. A expressão acima é a soma de i em todos seus diferentes valores. A notação de somatório é

$$\sum_{i=1}^5 i$$

A letra grega sigma maiúsculo, Σ , denota o somatório. Neste caso, o número 1 é o *limite inferior do somatório*, e o número 5 é o *limite superior do somatório*. A variável i é chamada de *índice do somatório*. O índice do somatório recebe o valor do limite inferior e então é sucessivamente incrementado até que atinja o valor do limite superior. Todos os valores que o índice do somatório recebe são somados juntos. Portanto,

$$\sum_{i=1}^5 i = 1 + 2 + 3 + 4 + 5 = 15$$

Analogamente,

$$\sum_{i=1}^3 i = 1 + 2 + 3 = 6$$

e

$$\sum_{i=4}^8 i = 4 + 5 + 6 + 7 + 8 = 30$$

Nos exemplos acima, a expressão após o símbolo de somatório é apenas i , o índice do somatório. No entanto, o que aparece após o símbolo do somatório pode ser qualquer expressão, e os sucessivos valores do índice são simplesmente substituídos na expressão. Portanto,

$$\sum_{i=1}^5 i^2 = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55$$

Uma forma geral de simbolizar o somatório é

$$\sum_{i=p}^q a_i$$

Onde o limite inferior, o limite superior e a expressão dentro do somatório não são fornecidos especificamente, mas apenas simbolizados. A notação a_i , é um lembrete de que a expressão deve ser calculada para i assumindo diferentes valores, do limite inferior ao limite superior.

Existem casos especiais a considerar:

$$1. \sum_{i=p}^q 0 = 0$$

Neste caso, a expressão dentro do somatório é a constante 0, que tem o valor 0 independentemente do valor do índice do somatório. A soma de qualquer número de 0s é 0.

$$2. \sum_{i=1}^n 1 = n$$

Novamente, a expressão dentro do somatório é uma constante, e o somatório indica que deve-se somar n cópias de 1, o que resulta em n .

$$3. \sum_{i=1}^0 a_i = 0$$

Neste caso, o limite superior é menor que o limite inferior; a interpretação usual de somatórios não se aplica, mas, por convenção, atribui-se o valor 0 ao somatório.

O índice do somatório é uma *variável muda*, o que significa que ele apenas funciona como uma máscara e que o uso de diferentes variáveis não altera o valor do somatório. Portanto,

$$\sum_{i=1}^3 i = \sum_{j=1}^3 j = 6$$

Pode ser conveniente mudar os limites de um somatório, o que é legítimo desde que o valor final do somatório permaneça o mesmo. Por exemplo,

$$\sum_{i=1}^3 i = \sum_{i=0}^2 (i+1)$$

uma vez que ambos têm o valor

$$1 + 2 + 3 = 6$$

Finalmente, valem as três regras a seguir, como veremos mais adiante:

Regras de Somatório

$$1. \sum_{i=p}^q (a_i + b_i) = \sum_{i=p}^q a_i + \sum_{i=p}^q b_i$$

$$2. \sum_{i=p}^q (a_i - b_i) = \sum_{i=p}^q a_i - \sum_{i=p}^q b_i$$

$$3. \sum_{i=p}^q c a_i = c \sum_{i=p}^q a_i$$

onde c é uma constante

Para demonstrar a regra 1, perceba que

$$\begin{aligned} & a_p + b_p + a_{p+1} + b_{p+1} + \dots + a_q + b_q \\ &= a_p + a_{p+1} + \dots + a_q + b_p + b_{p+1} + \dots + b_q \end{aligned}$$

devido à propriedade comutativa da adição. A prova da regra 2 é semelhante.

Para demonstrar a regra 3, perceba que

$$ca_p + ca_{p+1} + \dots + ca_q = c(a_p + a_{p+1} + \dots + a_q)$$

devido à propriedade distributiva. Esta regra permite que uma constante seja "escrita do lado de fora" de um somatório.

Algumas vezes podemos representar um somatório por uma expressão que não envolva a soma de itens. Por exemplo, de acordo com o Exercício 7 da Seção 2.2,

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} \quad (\text{D})$$

de forma que o valor de $\sum_{i=1}^5 i^2$ pode ser encontrado substituindo-se o limite superior 5 no lado direito de (1), resultando em

$$\frac{5(5+1)(2 \cdot 5+1)}{6} = 55$$

como antes. A Seção 2.2 e seus exercícios fornecem algumas outras expressões de "formas fechadas" para certos somatórios, todas elas demonstradas pela indução matemática.

APÊNDICE

B

A Função Logarítmica

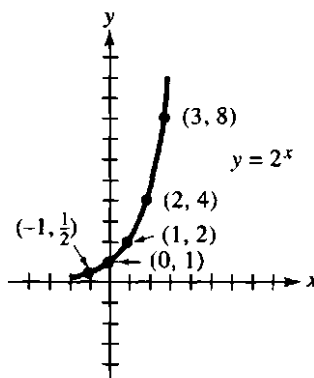
A função logarítmica está intimamente relacionada à *função exponencial*

$$y = b^x$$

onde b , a *base*, é uma constante maior que 1. (Na verdade, b pode ser qualquer número positivo, mas os casos interessantes ocorrem para $b > 1$.) Lembremo-nos das seguintes regras referentes a exponenciação:

1. $b^n b^m = b^{n+m}$ (quando multiplicamos, somamos os expoentes)
2. $b^n / b^{n-m} = b^{n-m}$ (quando dividimos, subtraímos os expoentes)
3. $(b^n)^m = b^{nm}$ (quando elevamos uma potência a um expoente, multiplicamos os expoentes)

Se escolhermos uma base em particular, $b = 2$, por exemplo, podemos traçar $y = 2^x$ para vários valores de x e preencher os demais valores, obtendo o gráfico



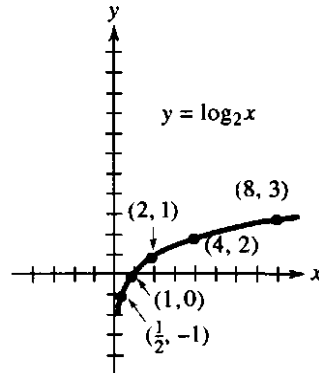
Nesta função, x pode assumir qualquer valor real, e y será sempre positivo. Outra maneira de dizer isto é que o domínio da função é o conjunto $\mathbb{R}$ dos números reais, e a imagem é o conjunto $\mathbb{R}^+$ dos números reais positivos.

Uma função que tem relação com esta (na verdade, é a função inversa) é a *função logarítmica*, definida como

$$y = \log_b x \quad \text{significando } b^y = x$$

Portanto, $\log_2 16 = 4$, por exemplo, porque $2^4 = 16$. Essas duas equações são as equações exponencial e logarítmica do mesmo fato. Analogamente, $\log_2 8 = 3$ e $\log_2 2 = 1$.

O gráfico de $y = \log_2 x$ é mostrado abaixo.



Como a função logarítmica $y = \log_b x$ é a inversa da função exponencial, seu domínio (os valores que x pode assumir) é o conjunto $\mathbb{R}^+$ dos números reais positivos e sua imagem (os valores que y pode assumir) é o conjunto $\mathbb{R}$ dos números reais. A função logarítmica para qualquer base $b > 1$ tem um domínio, uma imagem e uma forma semelhante ao do caso $b = 2$.

Certas propriedades referentes à função logarítmica são válidas devido à sua definição ou devido a propriedades correspondentes da função exponencial. Apresentaremos todas as propriedades da função logarítmica e então as demonstraremos.

Propriedades da Função Logarítmica $y = \log_b x$

1. Se $p < q$ então $\log_b p < \log_b q$ (a função log é estritamente crescente)
2. Se $\log_b p = \log_b q$ então $p = q$ (a função log é injetiva)
3. $\log_b 1 = 0$
4. $\log_b b = 1$
5. $\log_b(b^p) = p$
6. $b^{\log_b p} = p$
7. $\log_b(pq) = \log_b p + \log_b q$ (o log de um produto é a soma dos logs)
8. $\log_b \frac{p}{q} = \log_b p - \log_b q$ (o log de um quociente é a diferença dos logs)
9. $\log_b(p^q) = q(\log_b p)$ (o log de um número elevado a um expoente é igual ao expoente vezes o log do número)
10. $\log_a p = \frac{\log_b p}{\log_b a}$ (fórmula de mudança de base)

Para demonstrar as propriedades da função logarítmica, devemos primeiro observar que muitas das propriedades envolvem os valores $\log_b p$ e $\log_b q$. Vamos chamar esses valores de r e s , respectivamente, de forma que

$$\log_b p = r \text{ e } \log_b q = s$$

o que, por sua vez, significa

$$b^r = p \text{ e } b^s = q$$

1. Como $b > 1$, quanto maior a potência a que b é elevada, maior é o resultado. Portanto, se $p < q$, então $b^r < b^s$, logo $r < s$ e, portanto, $\log_b p < \log_b q$.
2. Se $\log_b p = \log_b q$, então $r = s$, logo $b^r = b^s$ e $p = q$.
3. $\log_b 1 = 0$ porque $b^0 = 1$.
4. $\log_b b = 1$ porque $b^1 = b$.
5. $\log_b(b^p) = p$ porque (traduzindo esta equação para sua forma exponencial) $b^p = b^p$.
6. $b^{\log_b p} = p$ porque (traduzindo esta equação para sua forma exponencial) $\log_b p = \log_b p$.
7. $\log_b(pq) = \log_b p + \log_b q = r + s$ porque é verdade que $b^{r+s} = b^r b^s = pq$, que é a forma exponencial da equação que estamos tentando demonstrar.

8. $\log_b(p/q) = \log_b p - \log_b q = r - s$ porque é verdade que $b^{r-s} = b^r/b^s = p/q$, que é a forma exponencial da equação que estamos tentando demonstrar.
9. $\log_b(p^q) = q(\log_b p) = qr$ porque é verdade que $b^{qr} = (b^r)^q = p^q$, que é a forma exponencial da equação que estamos tentando demonstrar.

$$10. \log_a p = \frac{\log_b p}{\log_b a}$$

Seja $\log_a p = w$. Então $a^w = p$. Agora tome o logaritmo na base b de ambos os lados desta equação:

$$\log_b(a^w) = w(\log_b a) = \log_b p$$

ou

$$w = \frac{\log_b p}{\log_b a}$$

que é o resultado desejado.

As três bases mais úteis de logaritmos são

$b = 10$	(logaritmo comum)
$b = e, \sim 2,7183$	(logaritmo natural ou neperiano)
$b = 2$	(o que usamos neste livro)

Os logaritmos comuns foram muito usados como ferramentas de apoio em cálculos antes do advento das calculadoras e dos computadores. A propriedade 7 da função logarítmica diz que para multiplicar dois números podemos tomar o logaritmo de ambos os números, somá-los e então encontrar o número cujo logaritmo é o valor encontrado. Com o auxílio de uma tabela de logaritmos comuns, um número pode ser traduzido para seu logaritmo, ou um logaritmo traduzido para seu número original. Os logaritmos naturais são úteis em cálculo, e são normalmente escritos como " $\ln p$ " ao invés de " $\log_e p$ ". Os logaritmos na base 2 são muitas vezes denotados por " $\ln p$ " ao invés de " $\log_2 p$ ". Neste livro, todos os logaritmos estão na base 2, de forma que usamos $\log p$ para denotar $\log_2 p$.

Uma última desigualdade envolvendo logaritmos da base 2 (usados na Seção 2.4) é

$$1 + \log n < n \quad \text{para } n \geq 3$$

Para demonstrá-la, perceba que

$$n < 2^{n-1} \quad \text{para } n \geq 3$$

portanto, pela propriedade 1 de logaritmos,

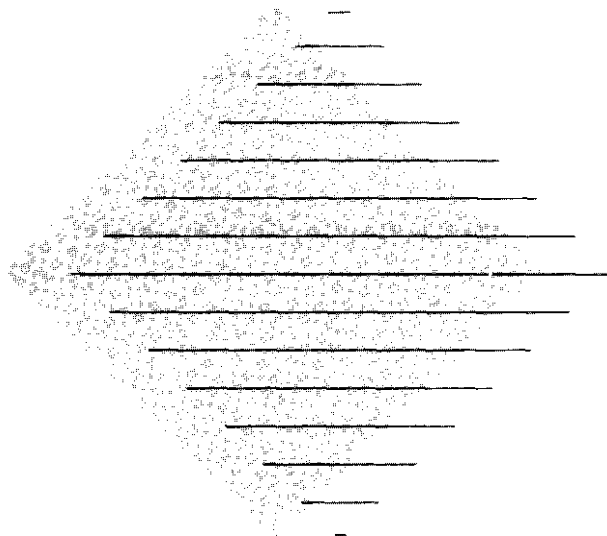
$$\log n < \log 2^{n-1}$$

Pela propriedade 5 de logaritmos, $\log 2^{n-1} = n - 1$. Portanto

$$\log n < n - 1$$

ou

$$1 + \log n < n \quad \text{para } n \geq 3$$



Respostas dos Exercícios Práticos

Nota aos estudantes: Responda todos os itens dos problemas antes de consultar as respostas.

Capítulo 1

1. Falso, falso, falso

2.	A	B	$A \vee B$
	V	V	V
	V	F	V
	F	V	V
	F	F	F

3. a. Antecedente: A chuva continuar
Conseqüente: O rio vai transbordar
b. Antecedente: A chave geral parar de funcionar
Conseqüente: A falha de uma rede
c. Antecedente: Os abacates estarem maduros
Conseqüente: Eles estarem escuros e macios
d. Antecedente: Um gato saudável
Conseqüente: Uma boa dieta

4.	A	B	$A \rightarrow B$
	V	V	V
	V	F	F
	F	V	V
	F	F	V

5.	A	A'
	V	F
	F	V

6. d

7. a.	A	B	$A \rightarrow B$	$B \rightarrow A$	$(A \rightarrow B) \leftrightarrow (B \rightarrow A)$
	V	V	V	V	V
	V	F	F	V	F
	F	V	V	F	F
	F	F	V	V	V

b.	A	B	A'	B'	$A \vee A'$	$B \wedge B'$	$(A \vee A') \rightarrow (B \wedge B')$
	V	V	F	F	V	F	F
	V	F	F	V	V	F	F
	F	V	V	F	V	F	F
	F	F	V	V	V	F	F

c.	A	B	C	B'	$A \wedge B'$	C'	$(A \wedge B') \rightarrow C'$	$[(A \wedge B') \rightarrow C']$
	V	V	V	F	F	F	V	F
	V	V	F	F	F	V	V	F
	V	F	V	V	V	F	F	V
	V	F	F	V	V	V	V	F
	F	V	V	F	F	F	V	F
	F	V	F	F	F	V	V	F
	F	F	V	V	F	F	V	F
	F	F	F	V	F	V	V	F

d.	A	B	A'	B'	$A \rightarrow B$	$B' \rightarrow A'$	$(A \rightarrow B) \leftrightarrow (B' \rightarrow A')$
	V	V	F	F	V	V	V
	V	F	F	V	F	F	V
	F	V	V	F	V	V	V
	F	F	V	V	V	V	V

8	A	1	A'	$A \vee A'$	$A \vee A' \leftrightarrow 1$
	V	V	F	V	V
	F	V	V	V	V

9. a. Verdadeiro (todos os canários-da-terra são amarelos)
 b. Falso
 c. Verdadeiro
 d. Falso (zero não é positivo nem negativo)

10. Por exemplo:

- a. O domínio é o conjunto dos motoristas com carteira de habilitação no Brasil; $P(x)$ é a propriedade de que x tenha 18 anos ou mais.
 b. O domínio é o conjunto de todos os peixes; $P(x)$ é a propriedade de que x tenha cinco pernas.
 c. Não; se todos os elementos do domínio tiverem a propriedade P , então (como o domínio precisa conter pelo menos um elemento), existe um elemento no domínio com a propriedade P .
 d. O domínio são todas as pessoas que moram no Rio de Janeiro; $P(x)$ é a propriedade de x ser homem.

11. Seja $x = 1$; então x é positivo e qualquer inteiro menor que $x \leq 0$, portanto o valor-verdade da sentença é verdadeiro. Para a segunda interpretação, seja $A(x)$ " x é par", $B(x, y)$ " $x < y$ " e $C(y)$ " y é ímpar"; a sentença é falsa, porque não há inteiro par que tenha a propriedade de que todos os inteiros maiores que ele sejam ímpares.

12. a. $(\forall x) [S(x) \rightarrow I(x)]$
 b. $(\exists x) [I(x) \wedge S(x) \wedge M(x)]$
 c. $(\forall x) (M(x) \rightarrow S(x) \wedge [I(x)]')$

13. Inválida. Na interpretação cujo domínio consiste em inteiros, $P(x)$ é " x é ímpar" e $Q(x)$ é " x é par", o antecedente é verdadeiro (todo inteiro é par ou ímpar), mas o conseqüente é falso (não é verdade que todo inteiro é par ou que todo inteiro é ímpar).

14. 1. P' (hipótese)
 2. $P' \rightarrow (Q' \rightarrow P')$ (Axioma 1)
 3. $Q' \rightarrow P'$ (1, 2 *modus ponens*)
 4. $(Q' \rightarrow P') \rightarrow (P \rightarrow Q)$ (Axioma 3)
 5. $P \rightarrow Q$ (3, 4, *modus ponens*)

15. 1. $P \rightarrow (P \rightarrow Q)$ (hipótese)

2. P (hipótese)
 3. $P \rightarrow Q$ (1, 2, *modus ponens*)
 4. Q (2, 3, *modus ponens*)
16. 1. $P \rightarrow Q$ (hipótese)
 2. $Q \rightarrow R$ (hipótese)
 3. P (hipótese)
 4. Q (1, 3, *modus ponens*)
 5. R (2, 4, *modus ponens*)
17. O argumento é $[(P \rightarrow M) \wedge (P \vee C) \wedge M'] \rightarrow C$. A sequência de demonstração é:
 1. $P \rightarrow M$ (hipótese)
 2. $(P \rightarrow M) \rightarrow (M' \rightarrow P')$ (tautologia)
 3. $M' \rightarrow P'$ (1, 2, *modus ponens*)
 4. M' (hipótese)
 5. P' (3, 4, *modus ponens*)
 6. $P \vee C$ (hipótese)
 7. C (tautologia $P' \wedge (P \vee C) \rightarrow C$)
18. 1. $(\forall x) P(x)$ (hipótese)
 2. $P(x)$ (1, Axioma 5, *modus ponens*)
 3. $(\exists x) P(x)$ (2, Axioma 7, *modus ponens*)
19. 1. $(\forall y) [P(x) \rightarrow Q(x, y)]$ (hipótese)
 2. $P(x) \rightarrow Q(x, y)$ (1, Axioma 5, *modus ponens*)
 3. $P(x)$ (hipótese temporária)
 4. $Q(x, y)$ (2, 3, *modus ponens*)
 5. $(\forall y) Q(x, y)$ (4, generalização — perceba que y não é livre em 3 e 1)
 6. $P(x) \rightarrow (\forall y) Q(x, y)$ (5 deduzido de 3)
20. O argumento é $(\forall x)[R(x) \rightarrow B(x)] \wedge (\exists x)R(x) \rightarrow (\exists x)B(x)$
 1. $(\forall x)[R(x) \rightarrow B(x)]$ (hipótese)
 2. $(\exists x)R(x)$ (hipótese)
 3. $R(a)$ (2, Axioma 6, *modus ponens*)
 4. $R(a) \rightarrow B(a)$ (1, Axioma 5, *modus ponens*)
 5. $B(a)$ (3, 4, *modus ponens*)
 6. $(\exists x)B(x)$ (5, Axioma 7, *modus ponens*)
21. veado
22. a. *predador*(x) if come (x, y) **and** *animal* (y)
 b. urso
 peixe
 quati
 urso
 urso
 raposa
 urso
 gato-selvagem
23. As respostas 7-9 resultam de *na-cadeia-alimentar*(*quati*, y); as respostas 10 e 11 resultam de *na-cadeia-alimentar*(*raposa*, y); a resposta 12 resulta de *na-cadeia-alimentar*(*veado*, y).
24. $x-2 = y$ ou $x = y + 2$. Se x é duas unidades maior que y e se subtrairmos 2 de x , então o valor obtido para x será y .
25. Trabalhando de trás para frente, a partir da pós-condição, usando o axioma da atribuição,
 $\{x + 4 = 7\}$
 $y := 4$
 $\{x + y = 7\}$
 $z := x + y;$
 $[z = 1]$

A primeira asserção, $x + 4 = 7$, é equivalente à pré-condição, $x = 3$. O axioma da atribuição, aplicado duas vezes, prova que o segmento do programa está correto.

26. Pelo axioma da atribuição,

$$\{x = 4\}y := x - 1 \quad \{y = 3\}$$

é verdadeiro e, portanto,

$$\{x = 4 \text{ e } x < 5\} y := x - 1 \quad \{y = 3\}$$

é verdadeiro. Além disso,

$$\{x = 4 \text{ e } x \geq 5\} y := 7 \quad \{y = 3\}$$

é verdadeiro porque o antecedente é falso. Pela regra condicional, o trecho de programa está correto.

Capítulo 2

1. Possíveis respostas:

- uma baleia
- entrada para um programa que esteja armazenado em disco

2. *Hipótese:* x é divisível por 6

$$x = k \cdot 6 \quad \text{para algum inteiro } k$$

$$2x = 2(k \cdot 6)$$

$$2x = (2 \cdot k)6$$

$$2x = (k \cdot 2)6$$

$$2x = k(2 \cdot 6)$$

$$2x = k \cdot 12$$

$$2x = k(3 \cdot 4)$$

$$2x = (k \cdot 3)4$$

K : 3 é um inteiro

Conclusão: $2x$ é divisível por 4

- O rio não vai transbordar, então a chuva não vai continuar.
 - Se não houver uma falha na rede, então a chave geral não vai parar de funcionar.
 - Se os abacates não estiverem duros ou macios, então não estão maduros.
 - Se a dieta não for boa, então o gato não é saudável.
- Se o rio vai transbordar, então a chuva vai continuar.
 - Se houver uma falha na rede, então a chave geral vai parar de funcionar.
 - Se os abacates estão escuros e macios, então estão maduros.
 - Se a dieta é boa, então o gato é saudável.

5. Sejam $x = 2m + 1$ e $y = 2n + 1$ e admita que xy é par. Então

$$(2m + 1)(2n + 1) = 2k \quad \text{para algum } k \text{ inteiro}$$

$$4mn + 2m + 2n + 1 = 2k$$

$$1 = 2(k - 2mn - m - n) \quad \text{onde } k - 2mn - m - n \text{ é um inteiro}$$

Isto é uma contradição, uma vez que 1 não é par.

6. $P(1): 1 = \frac{1(1+1)}{2}$, verdadeiro

$$\text{Admita } P(k): 1 + 2 + \dots + k = \frac{k(k+1)}{2}$$

$$\text{Mostre } P(k+1): 1 + 2 + \dots + (k+1) = \frac{(k+1)[(k+1)+1]}{2}$$

$$1 + 2 + \dots + (k+1) = 1 + 2 + \dots + k + (k+1)$$

$$= \frac{k(k+1)}{2} + (k+1) = (k+1) \left(\frac{k}{2} + 1 \right)$$

$$= (k+1) \left(\frac{k+2}{2} \right) = \frac{(k+1)[(k+1)+1]}{2}$$

7. $P(2): 2^{2+1} < 3^2$, ou $8 < 9$, verdadeiro

Admita $P(k)$: $2^{k+1} < 3^k$ e $k > 1$

Mostre $P(k+1)$: $2^{k+2} < 3^{k+1}$

$$\begin{aligned} 2^{k+2} &= 2(2^{k+1}) \\ &< 2(3^k) \\ &< 3(3^k) \text{ (pela hipótese de indução)} \\ &= 3^{k+1} \text{ (já que } 2 < 3) \end{aligned}$$

8. a. Para verificar $P(k+1)$ na implicação 2', subtraímos 3 de $k+1$. A fim de que a hipótese de indução seja verdade, é necessário que $(k+1) - 3 \geq 8$; assim $k+1$ precisa ser ≥ 11 . Portanto, a implicação 2' não pode ser usada para verificar $P(9)$ ou $P(10)$.
b. $P(k+1)$ não pode ser verificada tomando como premissa $P(k)$.

9. 1, 4, 7, 10, 13

10. 1, 1, 2, 3, 5, 8, 13, 21

11. Ao demonstrar para o elemento $k+1$, usamos os elementos $F(k-1)$ e $F(k)$. Se $k+1=2$, então $F(k-1)$ é indefinida. Portanto, no passo indutivo, precisamos ter $k+1 \geq 3$ e o caso $n=2$ precisa ser feito separadamente. Ou seja, o passo indutivo não demonstrar o caso $n=2$, tendo, como premissa, o caso $n=1$.

12. A, B, e C são wffs pela regra 1. Pela regra 2, (B') é uma wff, assim como $[A \vee (B')]$ e $[(A \vee (B')) \rightarrow C]$. Isto pode ser escrito como $(A \vee B') \rightarrow C$.

13. Todo pai de um ancestral de James é um ancestral de James.

14. 1011001, 0011011, 00110111011

15. 1. A, 0 e 1 são palíndromos binários.

2. Se x é um palíndromo binário, então $0x0$ e $1x1$ também o são.

16. 1. $x^1 = x$
2. $x^n = x^{n-1}$ para $n > 1$

17. **if** $n = 1$ **then**

$T := 1$

else

$T := T(n-1) + 3$;

18. 10, 7, 8

$$\begin{aligned} 19. \quad T(n) &= T(n-1) + 3 \\ &= [T(n-2) + 3] + 3 = T(n-2) + 2 * 3 \\ &= [T(n-3) + 3] + 2 * 3 = T(n-3) + 3 * 3 \end{aligned}$$

Em geral, parece que

$$T(n) = T(n-k) + k * 3$$

Quando $n-k=1$, isto é, $k=n-1$,

$$T(n) = T(1) + (n-1) * 3 = 1 + (n-1) * 3.$$

Agora demonstre por indução que $T(n) = 1 + (n-1) * 3$.

$$T(1): T(1) = 1 + (1-1) * 3 = 1, \text{ verdadeiro}$$

Admita que $T(k)$: $T(k) = 1 + (k-1) * 3$

Mostre $T(k+1)$: $T(k+1) \stackrel{?}{=} 1 + k * 3$

$$\begin{aligned} T(k+1) &= T(k) + 3 \\ &= 1 + (k-1) * 3 + 3 \\ &= 1 + k * 3 \end{aligned}$$

20. A relação de recorrência enquadra-se na equação (6) com $c=1$ e $g(n)=3$. Pela equação (8), a solução de forma fechada é

$$\begin{aligned}
 T(n) &= 1^{n-1}(1) + \sum_{i=2}^n 1^{n-i}(3) \\
 &= 1 + \sum_{i=2}^n 3 \\
 &= 1 + (n-1)3
 \end{aligned}$$

21. Ela está na forma da equação (1) com $c = 2$ e $g(n) = 1$. Pela equação (6), a solução é

$$\begin{aligned}
 2^{\log n}(1) + \sum_{i=1}^{\log n} 2^{i(\log n)}(1) &= 2^{\log n} + 2^{(\log n)-1} + 2^{(\log n)-2} + \dots + 2^0 \\
 &= 2^{(\log n)+1} - 1 \\
 &= (2)2^{\log n} - 1 = 2n - 1
 \end{aligned}$$

22. $Q(0): j_0 = x + i_0$ é verdadeiro, pois $j = x$, $i = 0$ antes da entrada no laço

Admita que $Q(k): j_k = x + i_k$

Mostre $Q(k+1): j_{k+1} \stackrel{?}{=} x + i_{k+1}$

$$\begin{aligned}
 j_{k+1} &= j_k + 1 && \text{(pela atribuição } j := j + 1) \\
 &= (x + i_k) + 1 && \text{(pela hipótese de indução)} \\
 &= x + (i_k + 1) \\
 &= x + i_{k+1} && \text{(pela atribuição } i := i + 1)
 \end{aligned}$$

Ao fim do laço, $i = y$ e $j = x + y$.

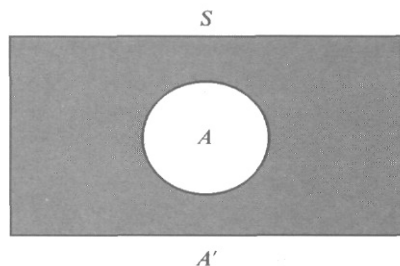
Capítulo 3

1. a. $\{4, 5, 6, 7\}$
b. $\{\text{abril, junho, setembro, novembro}\}$
c. $\{\text{Brasília}\}$
2. a. $\{x \mid x \text{ é um dos quatro primeiros quadrados perfeitos}\}$
b. $\{x \mid x \text{ é um sobrinho do Tio Patinhas}\}$
c. $\{x \mid x \text{ é um número primo}\}$
3. a. $A = \{x \mid x \in \mathbb{N} \text{ e } x \geq 5\}$
b. $B = \{3, 4, 5\}$
4. $x \in B$
5. $A \subset B$ significa $(\forall x)(x \in A \rightarrow x \in B) \wedge (\exists y)[y \in B \wedge (y \in A)']$
6. a, b, d, e, h, i, 1
7. Seja $x \in A$. Então $x \in \mathbb{R}$ e $x^2 - 4x + 3 = 0$, ou $(x-1)(x+3) = 0$, o que nos dá $x = 1$ ou $x = 3$. Em ambos os casos, $x \in \mathbb{N}$ e $1 \leq x \leq 4$, logo $x \in B$. Portanto, $A \subseteq B$. O valor 4 pertence a B , mas A não; logo $A \subset B$.
8. $\mathcal{P}(A) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$.
9. $2''$.
10. Pela definição de igualdade de pares ordenados,
 $2x - y = 1$ e $x + y = -1$
Resolvendo o sistema de equações, temos $x = 2$ e $y = -3$.
11. $(3, 3), (3, 4), (4, 3), (4, 4)$
12. a. Se S não é fechado sob a divisão.

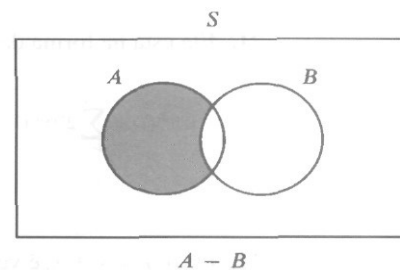
- b. x^* não é único para, digamos, $x = 4$ ($2^2 = 4$ e $(-2)^2 = 4$).
 c. 0° não é definido.

13. $x \in A$ e $x \in B$

14.



15.



16. a. $\{1, 2, 3, 4, 5, 7, 8, 9, 10\}$
 b. $\{1, 2, 3\}$
 c. $\{1, 3, 5, 10\}$

17. a. $A \times B = \{(1, 3), (1, 4), (2, 3), (2, 4)\}$
 b. $B \times A = \{(3, 1), (3, 2), (4, 1), (4, 2)\}$
 c. $A^2 = \{(1, 1), (1, 2), (2, 1), (2, 2)\}$
 d. $A^3 = \{(1, 1, 1), (1, 1, 2), (1, 2, 1), (1, 2, 2), (2, 1, 1), (2, 1, 2), (2, 2, 1), (2, 2, 2)\}$

18. Mostre a inclusão de conjuntos em ambas as direções. Para mostrar $A \cup \emptyset \subseteq A$, tome $x \in A \cup \emptyset$. Então $x \in A$ ou $x \in \emptyset$, mas como $\emptyset$ não tem elementos, $x \in A$. Para mostrar $A \subseteq A \cup \emptyset$, seja $x \in A$. Então $x \in A$ ou $x \in \emptyset$, logo $x \in A \cup \emptyset$.

19. a. $[C \cap (A \cup B)] \cup [(A \cup B) \cap C']$
 $= [(A \cup B) \cap C] \cup [(A \cup B) \cap C']$ (1b)
 $= (A \cup B) \cap (C \cup C')$ (3b)
 $= (A \cup B) \cap S$ (5a)
 $= A \cup B$ (4b)
 b. $[C \cup (A \cap B)] \cap [(A \cap B) \cup C'] = A \cap B$

20. a. $\{C, D, E, F\}$
 b. $\{A, B\}$
 c. $\{A, B, C, D, E, F, G\}$
 d. $\{C, D, E, F, G, Q\}$

21.

0	1	1	0	1
---	---	---	---	---

22. Uma enumeração dos números inteiros pares é 2, 4, 6, 8, 10, 12, ...

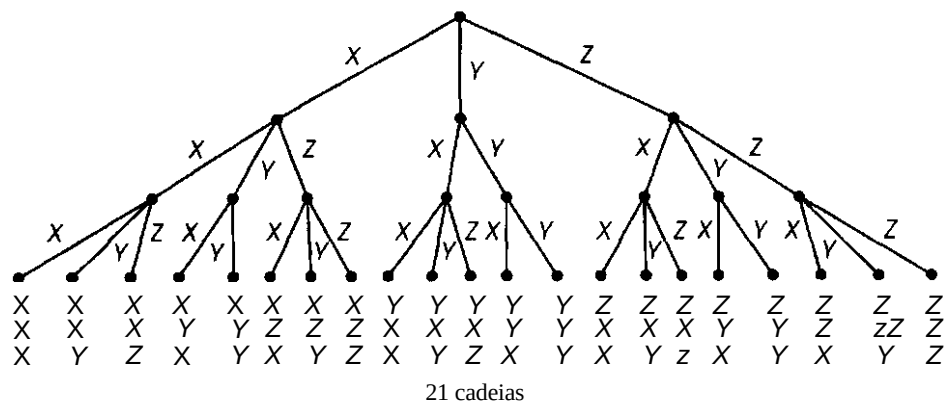
23. $1/5, 5/1$

24. $4(8)(5) = 160$

25. $7(5) + 9 = 44$

26. Apesar do problema consistir em eventos sucessivos — os cinco lançamentos —, o número de possibilidades dos eventos não é constante, variando entre um e dois, dependendo do resultado do evento anterior.

27.

28. $A \cup B$ 29. A equação (2) nos dá o resultado do Exemplo 34 porque se A e B forem disjuntos, então $|A \cap B| = 0$.

30. As razões da igualdade são: a união de conjuntos é associativa; equação (2); equação (2) e a identidade de conjuntos 3b (propriedade distributiva); equação (2); rearrumação dos termos.

31.7

$$32. P(20, 2) = \frac{20!}{18!} = 380$$

33. $6! = 720$

$$34. C(12, 3) = \frac{12!}{3!9!} = 220$$

35.a.18

b. 24

$$36. \frac{9!}{3!2!}$$

37. $C(8, 6)$

$$38. (a + b)^3 = a^3 + 3a^2b + 3ab^2 + b^3$$

Coeficientes: 1 3 3 1, que é a linha $n - 3$ no triângulo de Pascal

$$(a + b)^4 = a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4$$

Coeficientes: 1 4 6 4 1, que é a linha $n - 4$ no triângulo de Pascal

$$39. (x + 1)^5 = C(5, 0)x^5 + C(5, 1)x^4 + C(5, 2)x^3 + C(5, 3)x^2 + C(5, 4)x + C(5, 5) \\ = x^5 + 5x^4 + 10x^3 + 10x^2 + 5x + 1$$

$$40. C(7, 4)x^3y^4$$

Capítulo 4

1. a. $(3, 2) \in \rho$

d. $(2, 1), (5, 2) \in \rho$

b. $(2, 4), (2, 6) \in \rho$

c. $(3, 4), (5, 6) \in \rho$

2. a. Vários-para-um

b. Um-para-um

c. Vários-para-vários

3. a. $x(\rho \cup \sigma)y \leftrightarrow x \leq y$

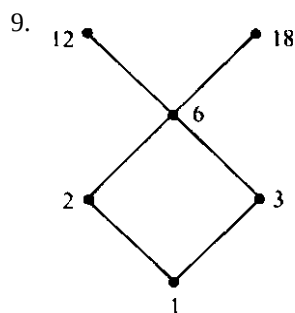
d. $\rho \cap \sigma = \emptyset$

b. $x \rho' y \leftrightarrow x \neq y$

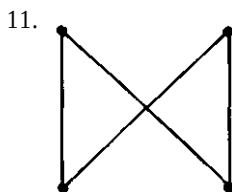
c. $x \sigma' y \leftrightarrow x \geq y$

4. a. $(1, 1), (2, 2), (3, 3)$ b. Saber que uma relação é simétrica não nos dá qualquer informação sobre que pares ordenados pertencem a ρ . Se soubermos que uma relação é simétrica e conhecermos alguns de seus pares ordenados, então alguns outros pares também devem pertencer à relação (veja o item (c)).c. (b, a) d. $a = b$

5. a. Reflexiva, simétrica, transitiva
 b. Reflexiva, anti-simétrica, transitiva
 c. Reflexiva, simétrica, transitiva
 d. Anti-simétrica
 e. Reflexiva, simétrica, anti-simétrica, transitiva
 f. Anti-simétrica (lembre-se da tabela-verdade da implicação), transitiva
 g. Reflexiva, simétrica, transitiva
 h. Reflexiva, simétrica, transitiva
6. Se a relação tem a propriedade anti-simétrica, então ela é seu próprio fecho anti-simétrico. Se a relação não é anti-simétrica, então deve haver dois pares ordenados (x, y) e (y, x) na relação com $x \neq y$. Estendendo a relação através da inclusão de pares ordenados, não muda essa situação; portanto, não faz sentido falarmos de fecho anti-simétrico de uma relação.
7. Fecho reflexivo: $\{(a, a), (b, b), (c, c), (a, c), (a, d), (b, d), (c, a), (d, a), (d, d)\}$
 Fecho simétrico: $\{(a, a), (b, b), (c, c), (a, c), (a, d), (b, d), (c, a), (d, a), (d, b)\}$
 Fecho transitivo: $\{(a, a), (b, b), (c, c), (a, c), (a, d), (b, d), (c, a), (d, a), (d, d), (d, c), (b, a), (b, c)\}$
8. a. $(7, 7), (1, 2), (2, 2), (1, 3), (3, 3), (1, 6), (6, 6), (1, 12), (12, 12), (7, 18), (18, 18), (2, 6), (2, 72), (2, 18), (3, 6), (3, 12), (3, 18), (6, 12), (6, 18)$
 b. 1, 2, 3
 c. 2, 3



10. $y \in S$ é um elemento máximo se $x \leq y$ para todo $x \in S$.
 $y \in S$ é um elemento maximal se não houver $x \in S$ com $y < x$.



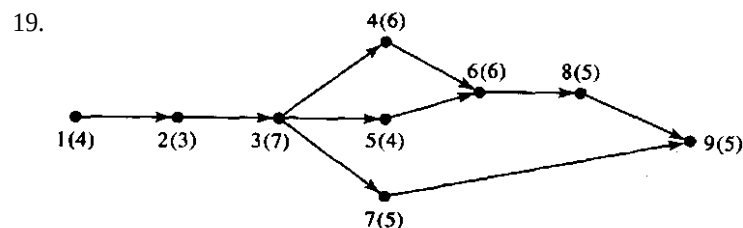
12. Seja $q \in [x]$. Então $x p q$. Como $x p z$, pela simetria, $z p x$. Pela transitividade, $z p x$ junto com $x p q$ nos dá $z p q$. Portanto, $q \in [z]$.
13. Para qualquer $x \in S$, x está no mesmo subconjunto que ele próprio, portanto, $x p x$. Se $x p y$, x está no mesmo subconjunto que y ; portanto, y está no mesmo subconjunto de x , isto é, $y p x$. Se $y p z$, y está no mesmo subconjunto que z ; logo x está no mesmo subconjunto de z , isto é, $x p z$.
14. a. As classes de equivalência são conjuntos que consistem em linhas no plano com a mesma inclinação.
 b. $[n] = \{n\}$; as classes de equivalência são todos os conjuntos unitários com os elementos de $\mathbb{N}$.
 c. $[1] = [2] = \{1, 2\}, [3] = \{3\}$

15. $[0] = \{\dots, -15, -10, -5, 0, 5, 10, 15, \dots\}$
 $[1] = \{\dots, -14, -9, -4, 1, 6, 11, 16, \dots\}$
 $[2] = \{\dots, -13, -8, -3, 2, 7, 12, 17, \dots\}$
 $[3] = \{\dots, -12, -7, -2, 3, 8, 13, 18, \dots\}$
 $[4] = \{\dots, -11, -6, -1, 4, 9, 14, 19, \dots\}$

16. 2

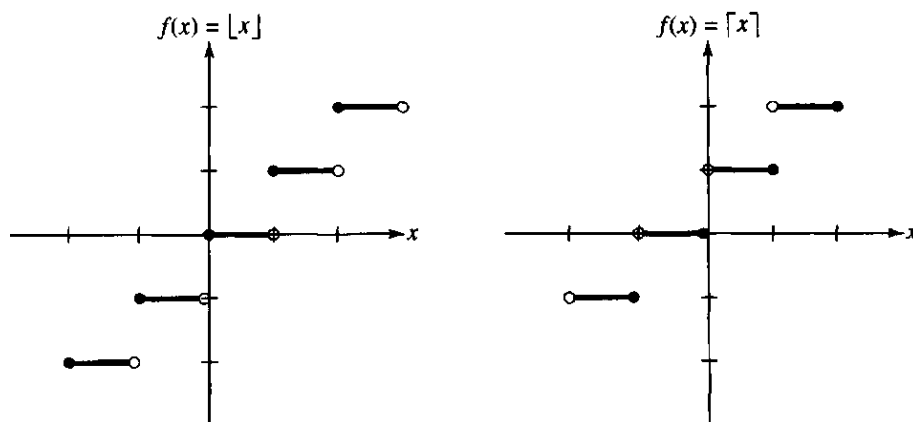
Nome	Local
Antônio Pedro	BA
Marco Antônio	RJ
Maria da Silva	RJ
Jaime Luiz	SP
Bruno da Silva	PE
Jânio Rocha	AM
Maria Garcia	PR

18. a. **project** (**join**(**select** *Dono-animal* **where** *Tipo-animal* = "Cachorro") **and** *Pessoa* **over** *Nome*) **over** *Cidade* **giving** Resultado
b. **SELECT** *Cidade* **FROM** *Pessoa*, *Dono-animal*
WHERE *Pessoa.Nome* = *Dono-animal.Nome*
AND *Dono-animal.Tipo-animal* = "Cachorro"
c. Range of *x* is *Pessoa*
Range of *y* is *Dono-animal*
 $\{x. Cidade \mid \text{exists } y(y.Nome = x.Nome \text{ and } y. Tipo-animal = "Cachorro")\}$



20. O tempo mínimo necessário para o término é 36 dias. O caminho crítico é 1, 2, 3, 4, 6, 8, 9.
21. Por exemplo: 1, 3, 2, 6, 7, 5, 4, 8, 9, 10, 11, 12
22. Por exemplo: 1, 2, 3, 7, 5, 4, 6, 8, 9
23. a. Não é uma função; $2 \in S$ tem dois valores associados
b. Função
c. Não é uma função; para os valores 0, 1, 2, 3 do domínio, os valores de $h(x)$ correspondentes estão fora do contradomínio
d. Não é uma função; nem todo elemento de S tem um número de CPF
e. Função (não é preciso usar todos os valores do contradomínio)
f. Função
g. Função
h. Não é uma função; $5 \in \mathbb{N}$ tem dois valores associados
24. a. 16
b. ± 3
25. V, F

26.



27. f e g têm os mesmos domínio e contradomínio, portanto, devemos mostrar que elas causam o mesmo efeito sobre os elementos do domínio.

$$f(1) = 1 \quad g(1) = \frac{\sum_{k=1}^1 (4k - 2)}{2} = \frac{4 \cdot 1 - 2}{2} = \frac{2}{2} = 1$$

$$f(2) = 4 \quad g(2) = \frac{\sum_{k=1}^2 (4k - 2)}{2} = \frac{(4 \cdot 1 - 2) + (4 \cdot 2 - 2)}{2} = \frac{2 + 6}{2} = 4$$

$$f(3) = 9 \quad g(3) = \frac{\sum_{k=1}^3 (4k - 2)}{2} = \frac{(4 \cdot 1 - 2) + (4 \cdot 2 - 2) + (4 \cdot 3 - 2)}{2} = \frac{2 + 6 + 10}{2} = 9$$

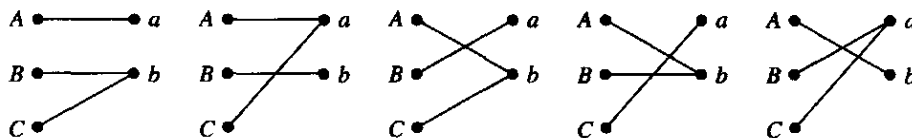
Portanto $f = g$.

28. b, f, g

29. Se P for uma tautologia ou uma contradição.

30. e, g

31.

32. Uma possibilidade: $\{(0, 0), (1, 1), (-1, 2), (2, 3), (-2, 4), (3, 5), (-3, 6), \dots\}$

$$\begin{aligned} 33. \quad (g \circ f)(x) &= g(f(x)) = g((2, 3)^2) = g(5, 29) = 5 \\ (f \circ g)(x) &= f(g(x)) = f(2) = 2^2 = 4 \end{aligned}$$

34. Seja $(g \circ f)(s_1) = (g \circ f)(s_2)$. Então $g(f(s_1)) = g(f(s_2))$ e, como g é injetiva, $f(s_1) = f(s_2)$. Como f é injetiva,

35. a. $(1, 4, 5) = (4, 5, 1) = (5, 1, 4)$

$$\text{b. } \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & 4 & 2 & 5 & 3 \end{pmatrix}$$

$$\begin{aligned} 36. \quad \text{a. } g \circ f &= (1, 3, 5, 2, 4) = (3, 5, 2, 4, 1) = \dots \\ f \circ g &= (1, 5, 2, 3, 4) = (5, 2, 3, 4, 1) = \dots \end{aligned}$$

$$g \circ f = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 4 & 2 & 5 & 1 & 3 \end{pmatrix}$$

b.

$$f \circ g = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 1 & 3 & 5 & 4 \end{pmatrix}$$

$$c. g \circ f = f \circ g = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 3 & 5 & 1 & 4 & 2 \end{pmatrix}$$

$$37. (1, 2, 4) \circ (3, 5) \text{ ou } (3, 5) \circ (1, 2, 4)$$

$$38. \text{ Seja } t \in T. \text{ Então } (f \circ g)(t) = f(g(t)) = f(s) = t.$$

$$39. f^{-1}: \mathbb{R} \rightarrow \mathbb{R}, f^{-1}(x) = (x - 4)/3$$

$$40. a. 10,87 \leq 12 \leq 1087$$

$$22,27 \leq 27 \leq 2227$$

$$37,67 \leq 48 \leq 3767$$

$$57,07 \leq 75 \leq 5707$$

b. Não

$$c. n_0 = 1, c_1 = 1/200, c_2 = 1$$

41. a. Seja $f \rho g$. Então existem constantes positivas, n_0, c_1 e c_2 com $c_1 g(x) \leq f(x) \leq c_2 g(x)$ para $x \geq n_0$. Então, para $x \geq n_0$, vale $(1/c_2)f(x) \leq g(x) \leq (1/c_1)f(x)$, portanto, $g \rho f$.

b. Seja $f \rho g$ e $g \rho h$. Então existem constantes positivas n_0, n_1, c_1, c_2, d_1 e d_2 tais que $c_1 g(x) \leq f(x) \leq c_2 g(x)$ para $x \geq n_0$ e $d_1 h(x) \leq g(x) \leq d_2 h(x)$ para $x \geq n_1$. Então, para $x \geq \max(n_0, n_1)$, $c_1 d_1 h(x) \leq f(x) \leq c_2 d_2 h(x)$, logo $f \rho h$.

$$42. 3x^2 = \Theta(x^2) \text{ usando as constantes } n_0 = 1, c_1 = c_2 = 3.$$

$$200x^2 + 140x + 7 = \Theta(x^2) \text{ usando as constantes } n_0 = 2, c_1 = 1, c_2 = 300.$$

$$43. 1, -7, -6$$

$$44. 2\mathbf{A} + \mathbf{B} = \begin{bmatrix} 6 & 14 \\ 3 & 10 \\ 9 & 16 \end{bmatrix}$$

$$45. \mathbf{A} \cdot \mathbf{B} = \begin{bmatrix} 15 & 22 \\ 12 & 28 \end{bmatrix} \quad \mathbf{B} \cdot \mathbf{A} = \begin{bmatrix} 39 & 0 \\ 27 & 4 \end{bmatrix}$$

$$46. \mathbf{I} \cdot \mathbf{A} = \begin{bmatrix} 1(a_{11}) + 0(a_{21}) & 1(a_{12}) + 0(a_{22}) \\ 0(a_{11}) + 1(a_{21}) & 0(a_{12}) + 1(a_{22}) \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} = \mathbf{A}$$

Similarmente, $\mathbf{A} \cdot \mathbf{I} = \mathbf{A}$.

x	y	$x \wedge y$
1	1	1
1	0	0
0	1	0
0	0	0

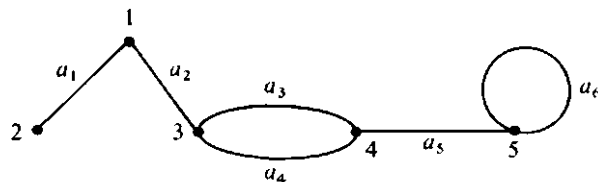
x	y	$x \vee y$
1	1	1
1	0	1
0	1	1
0	0	0

$$48. \text{ Não, } \mathbf{A} \cdot \mathbf{B} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$49. \mathbf{B} \times \mathbf{A} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Capítulo 5

1. Um desenho possível



2. Respostas possíveis:

- a. 2 e 3 f. 2, a, 1, a2, 3, a3, 4, a4, 3, a3, 4
 b. 5 g. 3, a3, 4, a4, 3
 c. a6 h. não
 d. a3 e a4 i. sim
 e. 3

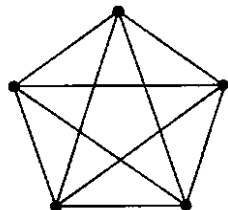
3. a. Em um grafo completo, quaisquer dois vértices distintos são adjacentes, então existe um caminho de comprimento 1 entre quaisquer dois vértices; portanto, o grafo é conexo.
 b. Por exemplo, o grafo da Fig. 5.4b.

4. $f_2: a_4 \rightarrow e_3$
 $a_5 \rightarrow e_8$
 $a_6 \rightarrow e_7$
 $a_7 \rightarrow e_5$ (ou e_6)
 $a_8 \rightarrow e_6$ (ou e_5)

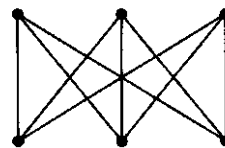
5. $f: 1 \rightarrow d$
 $2 \rightarrow e$
 $3 \rightarrow f$
 $4 \rightarrow c$
 $5 \rightarrow b$
 $6 \rightarrow a$

6. O grafo à esquerda da Fig. 5.10 tem dois vértices de grau 2, mas o grafo da direita não tem; isto é, o grafo da esquerda tem arestas paralelas, mas o da direita não tem.

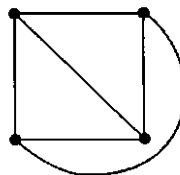
7.

 K_5

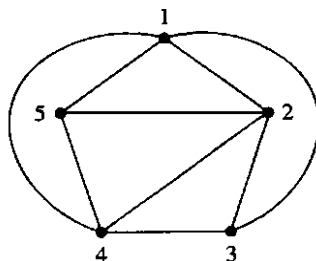
8.

 $K_{3,3}$

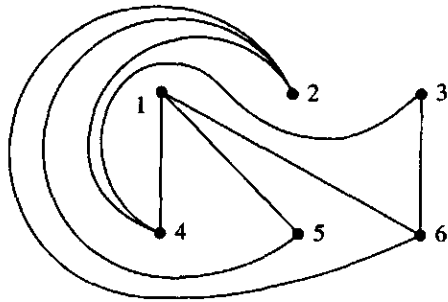
- 9.
- K_4
- pode ser representado como



10. Tornando 1-3 e 1-4 arestas exteriores obtemos o grafo abaixo, no qual ainda é impossível tornar os vértices 3 e 5 adjacentes sem violar a planaridade.

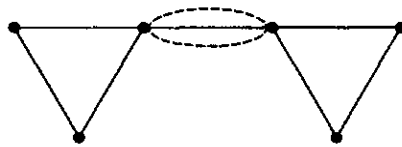


11. Uma tentativa de construir $K_{3,3}$ como um grafo planar nos dá o grafo abaixo; não há maneira de conectar os vértices 3 e 5. Qualquer outra construção nos leva a um problema semelhante.



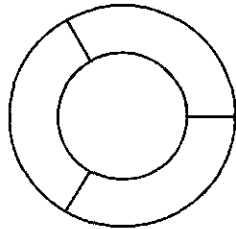
12. $n = 6, a = 7, r = 3$ e $6 - 7 + 3 = 2$

13. Sem essa condição na aresta, podemos obter uma figura como a mostrada abaixo. Neste caso, este grafo pode ser dividido em dois subgrafos conexos e a hipótese de indução não se aplicaria. Além disso, o número de regiões não mudaria.

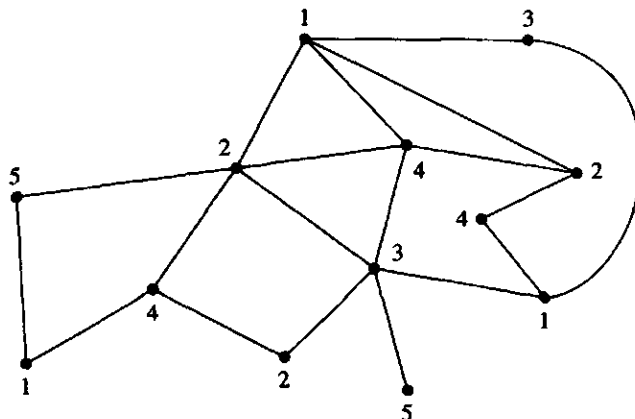


14. Em $K_{3,3}$, $a = 9, n = 6$ e $9 \leq 3(6) - 6$.

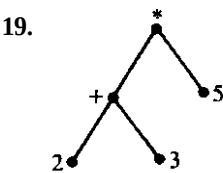
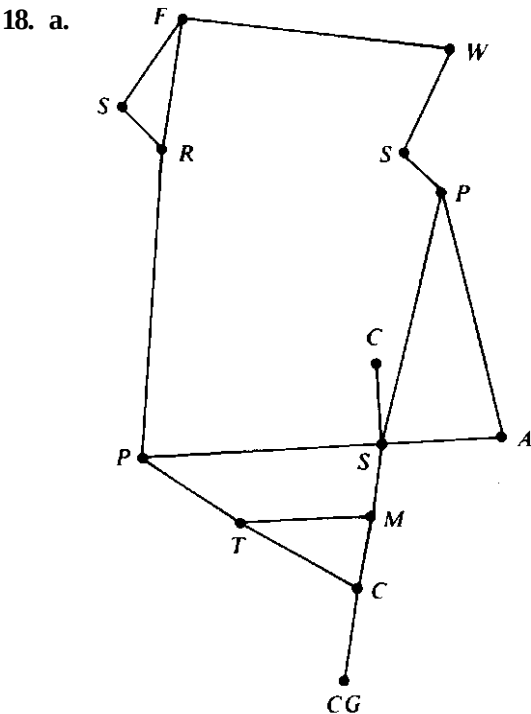
15.



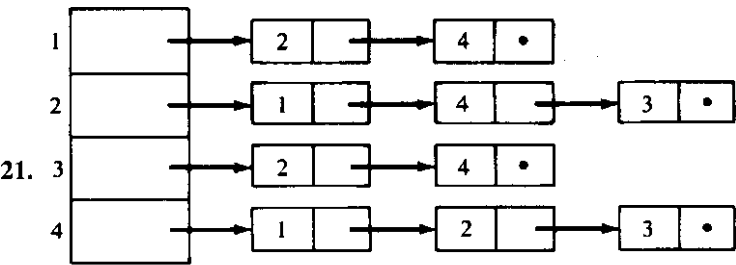
16. Existe um caminho usando apenas as cores 1 e 3 entre n_1 e n_3 . Portanto, não existe caminho entre n_2 e n_4 usando as cores 2 e 4. Na seção contendo n_2 , troque as cores 2 e 4; então n_2 tem cor 4 e n pode ter a cor 2. A coloração resultante é



17. a. 2 b. 4 c. 2



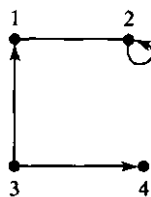
20.
$$A = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 2 \\ 1 & 0 & 2 & 0 \end{bmatrix}$$



Filho à esquerda Filho à direita

1	0	2
2	3	4
3	0	5
4	0	0
5	0	0

23. $\{(2,1), (2,2), (3,1), (3,4)\}$



24. Existem dois vértices distintos, 3 e 4, com $3 \rho 4$ e $4 \rho 3$.

$$25. \mathbf{A} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \quad \mathbf{A}^{(2)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$\mathbf{A}^{(2)}[2, 1] = 1$ porque existe um caminho de 2 a 1 com comprimento 2 ($2 - 4 - 1$).

26. Existe um caminho de comprimento 4 ($2 - 4 - 1 - 4 - 1$) de 2 a 1, portanto $\mathbf{A}^{(4)}[2, 1]$ deve ser 1.

$$\mathbf{A}^{(3)} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \quad \mathbf{A}^{(4)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

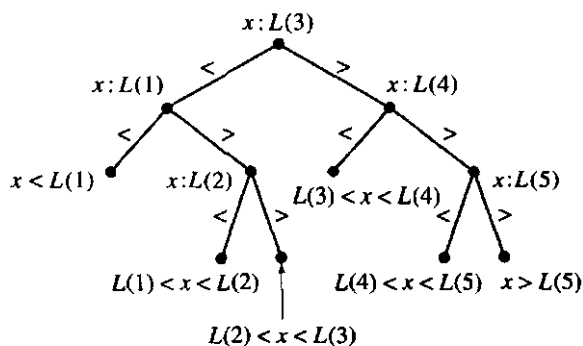
$$27. \mathbf{R} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

A coluna 2 é toda 0, logo 2 não é alcançável a partir de nenhum vértice.

$$28. \mathbf{M}_0 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \quad \mathbf{M}_1 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{M}_2 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

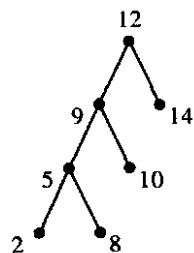
$$\mathbf{M}_3 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{M}_4 = \mathbf{R} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

29. a.



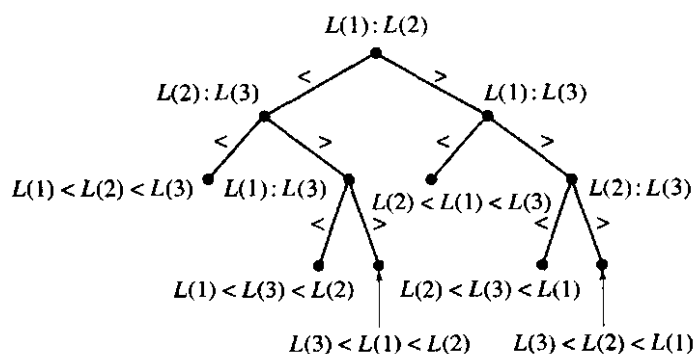
b. Profundidade da árvore = $3 = 1 + \lceil \log 5 \rceil$

30. a.



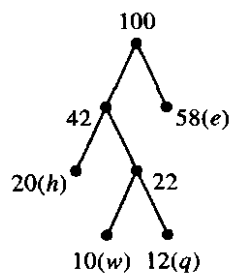
b. $d=3$

31.



32. a. *ppca* ? b. *cagak* c. *?kac* ?

33.



34. w: 010
q: 011
h: 00
e: 1

Capítulo 6

1. a. Não b. Sim
2. a. Não, tem quatro vértices ímpares
b. Sim, não tem vértices ímpares
3. Não

4.

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>A</i>	0	2	2	1
<i>B</i>	2	0	0	1
<i>C</i>	2	0	0	1
<i>D</i>	1	1	1	0

Após processar a linha *C*, $total = 3$, o laço termina e não existe caminho.

5. a. Não b. Sim

6.

$IN = \{x\}$					
	X	1	2	3	y
d	0	1	∞	4	∞
s	—	X	X	X	X

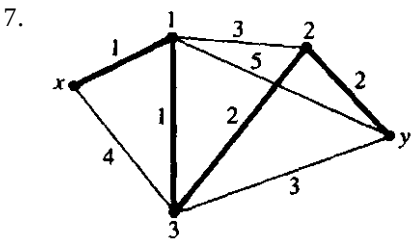
$p=1$ $IN = \{x, 1\}$					
	X	1	2	3	y
d	0	1	4	2	6
s	-	X	1	1	1

$p = 3$ $IN = \{x, 1, 3\}$					
	X	1	2	3	y
d	0	1	4	2	5
s	—	X	1	1	3

$p=2$ $IN = \{x, 1, 3, 2\}$					
	X	1	2	3	y
d	0	1	4	2	5
s	-	X	1	1	3

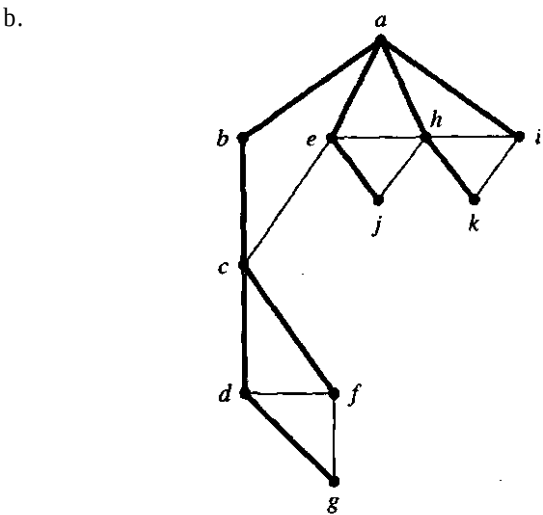
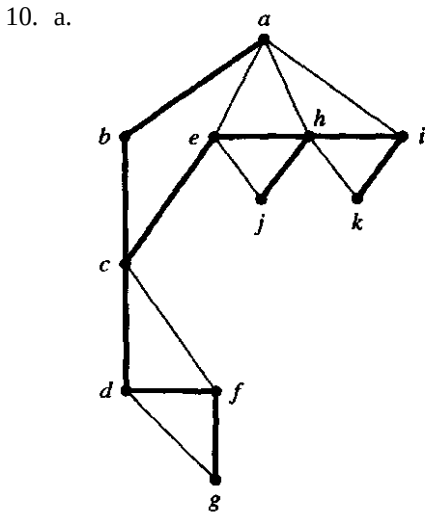
$p = y$ $IN = \{x, 1, 3, 2, y\}$					
	X	1	2	3	y
d	0	1	4	2	5
s	-	X	1	1	3

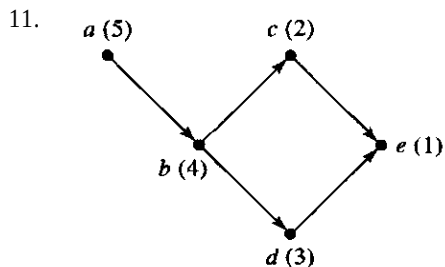
Caminho: $x, 1, 3, y$ Distância: 5



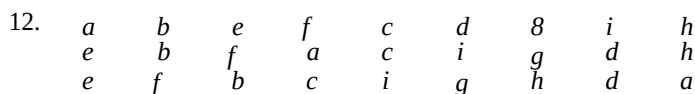
8. $a, e, d, b, c, i, f, g, h, l, k, m, j$

9. $a, e, f, d, i, b, c, g, h, j, k, m, l$

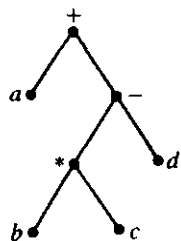




Ordenação topológica: a, b, d, c, e



13. Notação prefixa: $+ a - * b c d$
 Notação posfixa: $a b c * d \text{ ---} +$



14. A busca em profundidade procede do vértice a para o vértice b e então para o vértice c , atribuindo, na sequência, os *números de árvore* e de *retorno*. No vértice c , a aresta de retorno para o vértice a causa a mudança do *número de retorno* de c para o mesmo valor do *número de árvore* de a . A busca evolui para d , que tem *números de árvore* e de *retorno* atribuídos sequencialmente. Em seguida, volta para c . Como $\text{NúmeroDeRetorno}(d) > \text{NúmeroDeÁrvore}(c)$, c é reconhecido como uma articulação. A busca volta ao vértice b e o número de retorno de b é reduzido ao mesmo de c . A busca volta ao vértice, a , mas a não é
- uma articulação porque é a raiz com apenas uma aresta de árvore.

Capítulo 7

1. $0 \cdot 1 = 0$
 $1 \cdot 1 = 1$

2. **a.** $A \vee A = A$
b. $A \cup A = A$

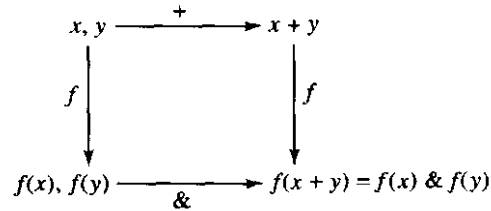
3. $a. x + 1 = x + (x + x')$ (5a, propriedade do complemento)
 $= (x + x) + x'$ (2a, propriedade associativa)
 $= x + x'$ (propriedade idempotente)
 $= 1$ (5a, propriedade do complemento)
b. $x \cdot 0 = 0$

4. $0 + 1 = 1$ (Prática 3a)
 $0 \cdot 1 = 1 - 0$ (1b) $\cdot$
 $= 0$ (Prática 3 b)

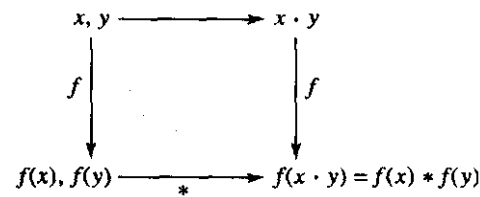
Portanto $1 = 0'$ pelo teorema da unicidade dos complementos.

5. **a.** $f(x \cdot y) = f(x) * f(y)$
b. $f(x') = [f(x)]'$

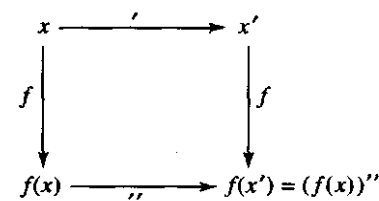
6. Propriedade 2:



Propriedade 3:

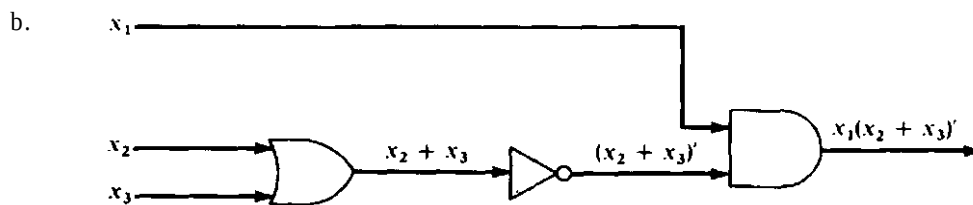
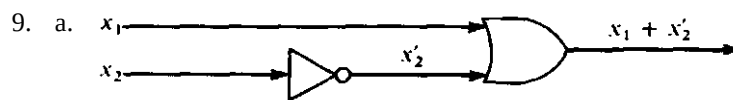


Propriedade 4:



7. **a.** $f(0 + a) = f(a) = \{1\} = \emptyset \cup \{1\} = f(0) \cup f(a)$
b. $f(a + a') = f(1) = \{1, 2\} = \{1\} \cup \{2\} = f(a) \cup f(a')$
c. $f(a \cdot a') = f(0) = \emptyset = \{1\} \cap \{2\} = f(a) \cap f(a')$
d. $f(1') = f(0) = \emptyset = \{1, 2\}' = (f(1))'$

8. a. $2''$ b. $2^4 = 16$ c. $2^{2''}$

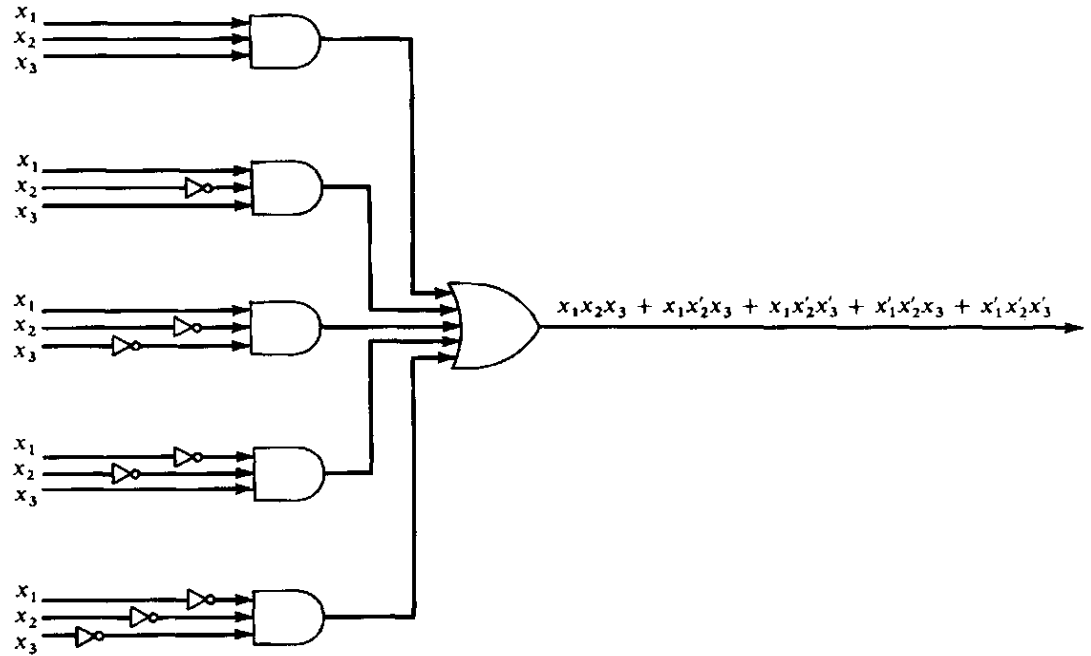


10. a. $(x_1' + x_2)x_3'$

b.

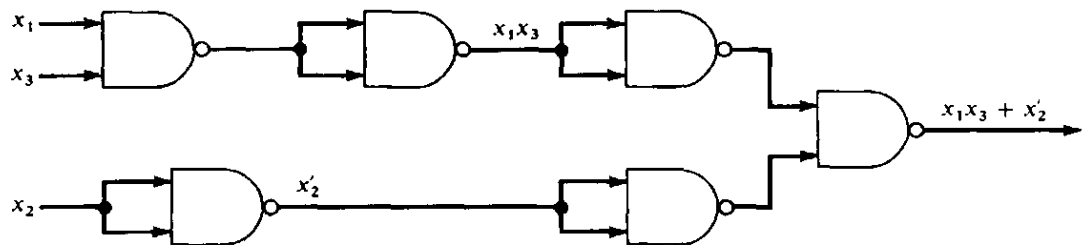
x_1	x_2	x_3	$(x_1 + x_2)x_3'$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	0
0	1	0	1
0	0	1	0
0	0	0	1

11. a. $x_1x_2x_3 + x_1x'_2x_3 + x_1x'_2x'_3 + x'_1x'_2x_3 + x'_1x'_2x'_3$
b.

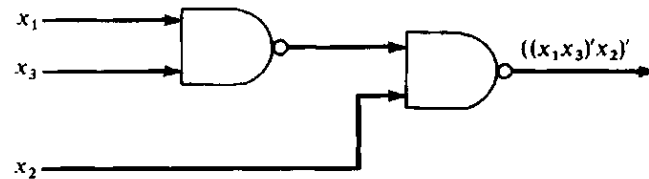


12.
$$\begin{array}{r} 101 \\ 111 \\ \hline (1)100 \end{array}$$

13. a.



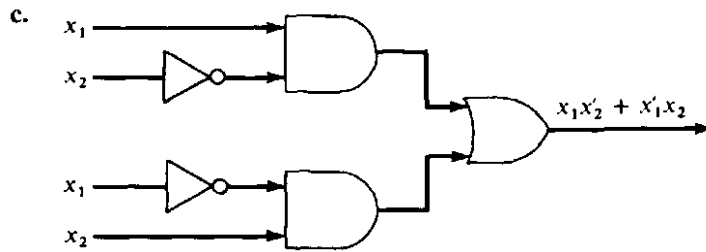
- b. $x_1x_3 + x'_2 = ((x_1x_3)'x'_2)'$



14. a.

x_1	x_2	$f(x_1, x_2)$
1	1	0
1	0	1
0	1	1
0	0	0

- b. Uma possibilidade é a forma canônica de soma de produtos, $x_1x'_2 + x'_1x_2$.



15. a. $x_1x_2 + x'_1x_2 = x_2x_1 + x_2x'_1$
 $= x_2(x_1 + x'_1)$
 $= x_2 \cdot 1$
 $= x_2$

b. $x_1 + x'_1x_2 = x_1 \cdot 1 + x'_1x_2$ (veja a Prática 3)
 $= x_1(1 + x_2) + x'_1x_2$
 $= x_1 + x_1x_2 + x'_1x_2$
 $= x_1 + x_2(x_1 + x'_1)$
 $= x_1 + x_2 \cdot 1 = x_1 + x_2$

16.

	x_1	x'_1
x_2		1
x'_2		1

A expressão reduzida é x'_1 .

17. x_1x_3 (4 caselas) e $x'_1x_2x'_3$ (2 caselas)

18. $x_1x'_2x_4 + x_1x'_3x_4 + x'_2x'_3$ (veja a figura a seguir)

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4		1		
$x_3x'_4$				
$x'_3x'_4$		1	1	
x'_3x_4	1	1	1	

19. A tabela de redução é mostrada na figura a seguir.

Número de 1 s	x_1	x_2	x_3	
Três	1	1	1	¹
Dois	1	1	0	¹²
Um	1	0	0	^{2,3}
	0	0	1	⁴
Nenhum	0	0	0	^{3,4}

Número de 1 s	x_1	x_2	x_3
Dois	1	1	—
Um	1	—	0
Nenhum	—	0	0
	0	0	—

A tabela de comparação é mostrada a seguir

	111	110	100	001	000
11-	✓	✓			
1-0		✓	✓		
-00			✓		✓
00-				✓	✓

Os termos essenciais são $11 - e 00 -$. Tanto $1-0$ quanto -00 podem ser usados como terceira parcela. A soma mínima de produtos é

$$x_1x_2 + x'_1x'_2 + x_1x'_3 \text{ ou } x_1x_2 + x'_1x'_2 + x'_2x'_3$$

Capítulo 8

1. $[\mathbb{R}, \cdot]$ não é um grupo comutativo porque $0 \in \mathbb{R}$ não tem um elemento inverso com relação à multiplicação; não existe número real y tal que $0 \cdot y = y \cdot 0 = 1$.
2. Veja a Prática 45 do Cap. 4.
3. Diversos elementos de $M_2(\mathbb{Z})$ não têm inversos sob a multiplicação de matrizes. Por exemplo, se

$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

tem uma matriz inversa

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

sob a multiplicação, então

$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \cdot \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Pela definição de multiplicação de matrizes, a única matriz que satisfaz esta equação deve ter $2a = 1$, ou $a = \frac{1}{2}$ e, portanto, não é elemento de $M_2(\mathbb{Z})$.

4. A subtração não é associativa; por exemplo, $5 - (3 - 1) = 3$ enquanto que $(5 - 3) - 1 = 1$.
5. Não, S não é fechado sob a multiplicação, por exemplo, $\frac{2}{3} \cdot \frac{3}{2} = 1$.
6. Todos exceto $[\mathbb{R}^+, +]$ (que não tem elemento neutro) são monóides; os elementos neutros são, respectivamente, $0, 1, 1, 0, \emptyset, S$.
7. $[\mathbb{R}, +]$
8. a. $f(x) + g(x) = g(x) + f(x)$
 $[f(x) + g(x)] + h(x) = f(x) + [g(x) + h(x)]$
 b. O polinômio nulo, 0
 c. $-7x^4 - 2x^3 - 4$

9. a.

+5	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

5	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

- b. $0; 1$ c. 3 d. todos exceto 0

10. a.

$\cdot_6$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	2	3	4	5
2	0	2	4	0	2	4
3	0	3	0	3	0	3
4	0	4	2	0	4	2
5	0	5	4	3	2	1

b. 1 e 5

11. Sejam $f, g, h \in S$. Então, para qualquer $x \in A$, $[f \circ g] \circ h(x) = (f \circ g)(h(x)) = f(g[h(x)])$ e $[f \circ (g \circ h)](x) = f[(g \circ h)(x)] = f(g[h(x)])$. Portanto, $(f \circ g) \circ h = f \circ (g \circ h)$.

12. a.

$\circ$	α_1	α_2	α_3	α_4	α_5	α_6
α_1	α_1	α_2	α_3	α_4	α_5	α_6
α_2	α_2	α_1	α_6	α_5	α_4	α_3
α_3	α_3	α_5	α_1	α_6	α_2	α_4
α_4	α_4	α_6	α_5	α_1	α_3	α_2
α_5	α_5	α_3	α_4	α_2	α_6	α_1
α_6	α_6	α_4	α_2	α_3	α_1	α_5

b. Não

13. a. Não b. Não

14. $i_1 = i_1 i_2$ porque i_2 é um elemento neutro
 $i_1 i_2 = i_2$ porque i_1 é um elemento neutro

15. Sejam y e z inversos de x . Seja i o elemento neutro. Então $y = y \cdot i = y \cdot (x \cdot z) = (y \cdot x) \cdot z = i \cdot z = z$.

16. $7^{-1} = 5, 3^{-1} = 9$; logo $10^{-1} = (7 +_{12} 3)^{-1} = 3^{-1} +_{12} 7^{-1} = 9 +_{12} 5 = 2$

17. $z \cdot x = z \cdot y$ implica

$$z^{-1} \cdot (z \cdot x) = z^{-1} \cdot (z \cdot y)$$

$$(z^{-1} \cdot z) \cdot x = (z^{-1} \cdot z) \cdot y$$

$$i \cdot x = i \cdot y$$

$$x = y$$

$$18. x = 1 +_8 (3)^{-1} = 1 +_8 5 = 6$$

19.

$*$	1	a	b	c	d
1	1	a	b	c	d
a	a	b	c	d	1
b	b	c	d	1	a
c	c	d	1	a	b
d	d	1	a	b	c

20. a. $[\mathbb{Z}_{18}, +_{18}]$

- b. $[S_3, \circ]$

21. Requisito 2, $i \in A$, garante que $A \neq \emptyset$.

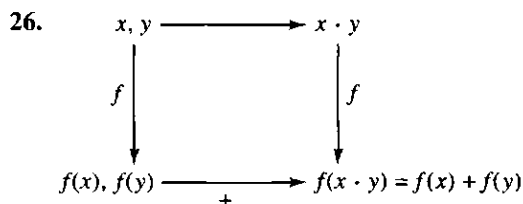
22. a. O fecho vale; $0 \in \{0, 2, 4, 6\}$; $0^{-1} = 0, 2^{-1} = 6, 4^{-1} = 4$ e $6^{-1} = 2$.

- b. O fecho vale; $1 \in \{1, 2, 4\}$; $1^{-1} = 1, 2^{-1} = 4$ e $4^{-1} = 2$.

23. $[\{\alpha_1, \alpha_5, \alpha_6\}, \circ], [\{\alpha_1, \alpha_2\}, \circ], [\{\alpha_1, \alpha_3\}, \circ], [\{\alpha_1, \alpha_4\}, \circ]$

24. Para mostrar que f é injetiva, sejam α e β pertencentes a A_n e suponha $f(\alpha) = f(\beta)$. Então $\alpha^\circ(1, 2) = \beta^\circ(1, 2)$. Pela lei do cancelamento aplicada ao grupo S_n , $\alpha = \beta$. Para mostrar que f é sobrejetiva, seja $\gamma \in O_n$. Então $\gamma^\circ(1, 2) \in A_n$ e $f(\gamma^\circ(1, 2)) = \gamma^\circ(1, 2)^\circ(1, 2) = \gamma$.

25. Para nz_1 e $nz_2 \in n\mathbb{Z}$, $nz_1 + nz_2 = n(z_1 + z_2) \in \mathbb{Z}$, logo o fecho vale; $0 = n \cdot 0 \in n\mathbb{Z}$; para $nz \in n\mathbb{Z}$, $-nz = n(-z) \in n\mathbb{Z}$.



27. $f(s) + f(s^{-1}) = f(s \cdot s^{-1}) = f(i_s) = i_T$. Analogamente, $f(s^{-1}) + f(s) = i_T$. Portanto, $f(s^{-1})$ funciona como a inversa de $f(s)$ em T e, como as inversas são únicas, $f(s^{-1}) = -f(s)$.

28. Sejam t_1 e t_2 elementos de T . Como f é sobrejetiva, $t_1 = f(s_1)$ e $t_2 = f(s_2)$ para algum $s_1, s_2 \in S$. Então
 $t_1 + t_2 = f(s_1) + f(s_2) = f(s_1 \cdot s_2) = f(s_2 \cdot s_1)$ (como $[S, \cdot]$ é comutativo)
 $= f(s_2) + f(s_1) = t_2 + t_1$
 logo $[T, +]$ é comutativo.

29. Está claro que f é sobrejetiva. f também é injetiva: seja $f(x) = f(y)$, então $5x = 5y$ e $x = y$. f é um homomorfismo: para $x, y \in \mathbb{Z}$, $f(x + y) = 5(x + y) = 5x + 5y = f(x) + f(y)$.

30. a. A composição de bijeções é uma bijeção, e para $x, y \in S$, $(g \circ f)(x \cdot y) = g(f(x \cdot y)) = g(f(x) + f(y)) = g(f(x)) * g(f(y)) = (g \circ f)(x) * (g \circ f)(y)$.
 b. $S \simeq S$ pela função identidade. Se f é um isomorfismo de S em T , então f^{-1} é um isomorfismo de T em S . Se $S \simeq T$ e $T \simeq V$, então pelo item (a), $S \simeq V$.

31. Para mostrar que α_g não é uma função sobrejetiva, tome $y \in G$. Então $g^{-1} \cdot y$ pertence a G e $\alpha_g(g^{-1} \cdot y) = g(g^{-1} \cdot y) = (g \cdot g^{-1})y = y$. Para mostrar que α_g é injetiva, seja $\alpha_g(x) = \alpha_g(y)$. Então $g \cdot x = g \cdot y$ e, pela lei do cancelamento, $x = y$.

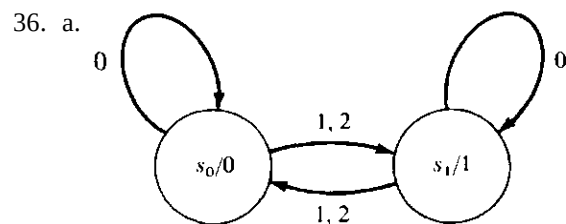
32. a. Para $\alpha_g \in P$, $\alpha_g \circ \alpha_1 = \alpha_{g \cdot 1} = \alpha_g$ e $\alpha_1 \circ \alpha_g = \alpha_{1 \cdot g} = \alpha_g$.
 b. $\alpha_g \circ \alpha_{g^{-1}} = \alpha_{g \cdot g^{-1}} = \alpha_1$, e $\alpha_{g^{-1}} \circ \alpha_g = \alpha_{g^{-1} \cdot g} = \alpha_1$.

33. a. Seja $f(g) = fh$. Então $\alpha_g = \alpha_h$ e, em particular, $\alpha_g(1) = \alpha_h(1)$, ou $g \cdot 1 = h \cdot 1$ e $g = h$.
 b. Para, $g, h \in G$, $f(g \cdot h) = \alpha_{g \cdot h} = \alpha_g \circ \alpha_h = f(g) \circ f(h)$.

34. 000110

35. Estado atual

	Próximo estado		Saída
	Entrada atual		
	0	1	
s_0	s_0	s_3	0
s_1	s_0	s_2	1
s_2	s_3	s_3	1
s_3	s_1	s_3	2

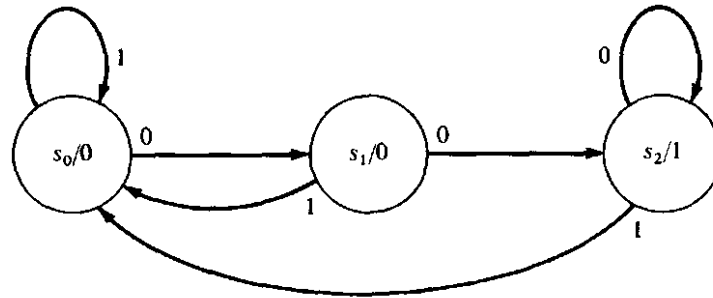


b. 01011

37. a. s_1 b. s_1

38. 11001011

39.



40. a. Conjunto que consiste em um único 0
 b. Conjuntos que consistem em qualquer número (incluindo nenhum) de 0s seguido por 10
 c. Conjuntos que consistem em um único 0 ou um único 1
 d. Conjuntos que consistem em qualquer número (incluindo nenhum) de pares de 1s

41. A cadeia do item (b) não pertence,

42. a. 0 b. 0^*10 c. $0 \vee 1$ d. $(11)^*$

43. s_2, s_3

44. Um estado produz a mesma saída para qualquer que seja sua entrada. Se s_i produz a mesma saída que s_j , então s_j produz a mesma saída que s_i . A transitividade é igualmente fácil.

45. A propriedade 1 é satisfeita porque todos os estados em uma mesma classe têm as mesmas cadeias de saída para qualquer cadeia de entrada, incluindo a cadeia de entrada vazia. Para ver que a propriedade 2 também é satisfeita, admita que s_i e s_j são estados equivalentes que passam, sob o símbolo i , para os estados s'_i e s'_j que não são equivalentes. Então existe uma cadeia de entrada α tal que $f_0(s'_i, \alpha) \neq f_0(s'_j, \alpha)$. Portanto, para a cadeia de entrada $i\alpha$, s_i e s_j produzem cadeias de saídas diferentes, contradizendo a equivalência de s_i e s_j .

46. Os estados equivalentes de M na Tabela 8.9 são $A = \{0, 1, 3\}$, $B = \{2\}$ e $C = \{4\}$. A máquina reduzida é

Estado atual	Próximo estado		Saída
	Entrada atual		
	0	1	
<i>A</i>	<i>B</i>	<i>A</i>	1
<i>B</i>	<i>C</i>	<i>A</i>	0
<i>C</i>	<i>A</i>	<i>A</i>	0

Os estados equivalentes de M na Tabela 8.10 são $\{0\}$, $\{1\}$, $\{2\}$ e $\{3\}$. M já está em sua forma mínima.

47. a. ...b00b...

b. A máquina entra em um ciclo sem fim entre as duas posições não-vazias.

c. A máquina muda as duas posições não-vazias para 0 e 1 e então move sua cabeça para a direita indefinidamente.

48. a. $b \ X \ X \ \overset{1}{\uparrow} \ X \ X \ b$ Para sem aceitar
 7

b. $b \ X \ X \ \overset{X}{\uparrow} \ X \ X \ b$ Para sem aceitar
 2

c. $b \ X \ X \ X \ \overset{0}{\uparrow} \ X \ X \ b$ Para sem aceitar
 2

49. O estado 3 é o único estado final.

(0, 1, 1, 0, D)
 (0, 0, 0, 1, D)
 (1, 0, 0, 2, D)
 (1, 1, 1, 0, D)
 (2, 0, 0, 2, D)
 (2, 1, 1, 0, D)
 (2, b, b, 3, D)

50. Troque (2, 1, X, 3, E) por (2, 1, X, 9, E) e inclua (9, 1, X, 3, E).

51. Uma máquina que funcione, juntamente com uma descrição de suas ações:

(0, 1, 1, 1, D)	lê primeiro 1
(1, b, 1, 6, D)	$n = 0$, muda para 1 e pára
(1, 1, 1, 2, D)	lê segundo 1
(2, b, b, 6, D)	$n = 1$, pára
(2, 1, 1, 3, D)	$n \geq 2$
(3, 1, 1, 3, D)	
(3, b, b, 4, E)	encontra o extremo direito de $\bar{n}$
(4, 1, b, 5, E)	
(5, l, b, 6, E)	apaga dois ls de $\bar{n}$ e pára

52. $0S \Rightarrow 00S \Rightarrow 000S \Rightarrow 0000S \Rightarrow 00001$

53. $E = \{0^n 1 \mid n \geq 0\}$

54. Por exemplo:

a. $G = (V, V_T, S, P)$ onde $V = \{0, 1, S\}$, $V_T = \{0, 1\}$ e $P = \{S \rightarrow 1, S \rightarrow 0S0\}$

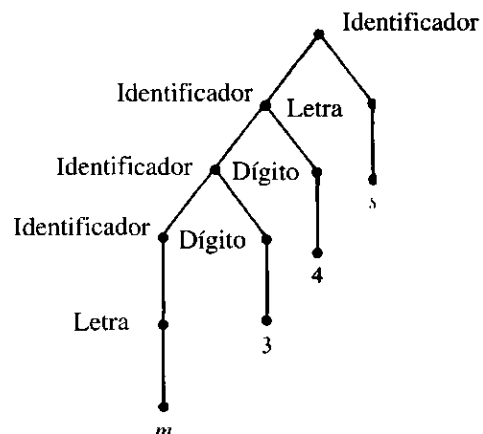
b. $G = (V, V_T, S, P)$ onde $V = \{0, 1, S, M\}$, $V_T = \{0, 1\}$ e $P = \{S \rightarrow 0M0, M \rightarrow 0M0, M \rightarrow 1\}$

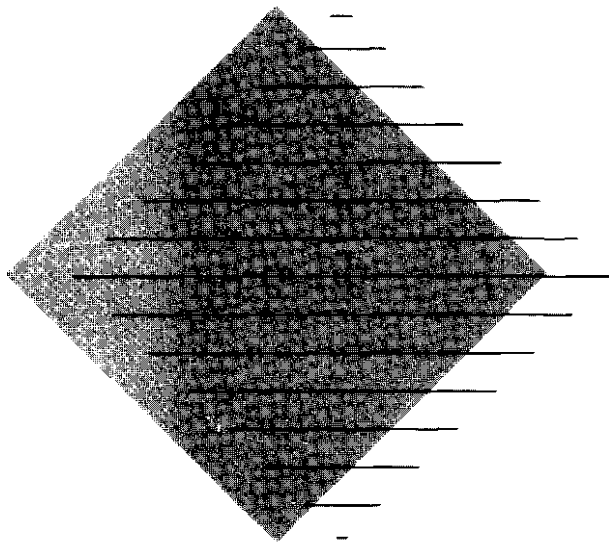
55. Em $G_1 = S \Rightarrow ABA \Rightarrow 00A \Rightarrow 0000A \Rightarrow 00000$

Em $G_2 = S \Rightarrow 00A \Rightarrow 0000A \Rightarrow 00000$

Em $G_3 = S \Rightarrow 0A \Rightarrow 00B \Rightarrow 000C \Rightarrow 0000B \Rightarrow 00000$

56.





Respostas dos Exercícios Seleccionados

Capítulo 1

Exercícios 1.1

1. (a), (c), (e), (f)

4. a. Antecedente: Quantidade suficiente de água
Consequente: Crescimento sadio das plantas
b. Antecedente: Desenvolvimento científico
Consequente: Crescimento da oferta de computadores
c. Antecedente: Haverá novos erros
Consequente: O programa foi alterado
d. Antecedente: Economia de combustível
Consequente: Bom isolamento ou todas as janelas são janelas para tempestade

8. a. A: Os preços subirem; B: As construções serão muitas; C: As construções ficarão mais caras

$$(A \rightarrow B \wedge C) \wedge (C' \rightarrow B)$$

b. A: Ir para a cama; B: Ir nadar; C: Trocar de roupa

$$(A \vee B) \rightarrow C) \wedge (C \rightarrow B)'$$

c. A: Vai chover; B: Vai nevar

$$(A \vee B) \wedge (A \wedge B)'$$

d. A: Janet vencer; B: Janet perder; C: Janet estará cansada

$$(A \vee B) \rightarrow C$$

e. A: Janet vencer; B: Janet perder; C: Janet estará cansada

$$A \vee (B \rightarrow C)$$

9. a

A	B	$A \rightarrow B$	A'	$A' \vee B$	$(A \rightarrow B) \leftrightarrow A' \vee B$
V	V	V	F	V	V
V	F	F	F	F	V
F	V	V	V	V	V
F	F	V	V	V	V

Tautologia

b. A	B	C	$A \wedge B$	$(A \wedge B) \vee C$	$B \vee C$	$A \wedge (B \vee C)$	$(A \wedge B) \vee C \rightarrow A \wedge (B \vee C)$
V	V	V	V	V	V	V	V
V	V	F	V	V	V	V	V
V	F	V	F	V	V	V	V
V	F	F	F	F	F	F	V
F	V	V	F	V	V	F	F
F	V	F	F	F	V	F	V
F	F	V	F	V	V	F	F
F	F	F	F	F	F	F	V

10. $2^{24} = 2^{16}$

a. A	A'	$A \vee A'$
V	F	V
F	V	V

c. A	B	$A \wedge B$	$A \wedge B \rightarrow B$
V	V	V	V
V	F	F	V
F	V	F	V
F	F	F	V

17. a. Atribua

$B' \wedge (A \rightarrow B)$ verdadeiro
 A' falso

Da segunda atribuição, vemos que A é verdadeiro. Da primeira atribuição, B' é verdadeiro (portanto B é falso) e $A \rightarrow B$ é verdadeiro. Se $A \rightarrow B$ é verdadeiro e A é verdadeiro, então B é verdadeiro. Portanto, B é tanto verdadeiro quanto falso e $[B' \wedge (A \rightarrow B)] \rightarrow A'$ é uma tautologia.

23. $A \wedge B$ é equivalente a $(A | B) | (A | B)$

A	B	$A \wedge B$	$A B$	$(A B) (A B)$	$A \wedge B \leftrightarrow (A B) (A B)$
V	V	V	F	V	V
V	F	F	V	F	V
F	V	F	V	F	V
F	F	F	V	F	V

 A' é equivalente a $A | A$

A	A'	$A A$	$A' \leftrightarrow A A$
V	F	F	V
F	V	V	V

Exercícios 1.2

2. a. Verdadeiro (tome $y = 0$)
b. Verdadeiro (tome $y = 0$)
c. Verdadeiro (tome $y = -x$)
d. Falso
4. a. Verdadeiro: O domínio são os inteiros; $A(x)$ é " x é par" e $B(x)$ é " x é ímpar".
Falso: o domínio são os inteiros positivos; $A(x)$ é " $x > 0$ " e $B(x)$ é " $x \geq 1$ ".
6. a. $(\forall x) (D(x) \rightarrow S(x))$
b. $(\exists x) [D(x) \wedge (R(x))']$
c. $(\forall x) [D(x) \wedge S(x) \rightarrow (R(x))']$
7. c. $(\exists x) (L(x) \wedge (\forall y) [A(x, y) \rightarrow J(y)])$ ou $(\exists x) (\forall y) (L(x) \wedge [A(x, y) \rightarrow J(y)])$
8. a. $(\forall x) [C(x) \wedge F(x)]'$
b. $(\exists x) (P(x) \wedge (\forall y) [S(x, y) \rightarrow F(y)])$ ou $(\exists x) (\forall y) (P(x) \wedge [S(x, y) \rightarrow F(y)])$.

10. a. John é bonito e Kathy gosta de John.
b. Todos os homens são bonitos.
13. a. O domínio são os inteiros; $A(x)$ é "x é par" e $B(x)$ é "x é ímpar".

Exercícios 1.3

1. 1. hipótese
2. hipótese
3. Axioma 2
4. 2,3, *modus ponens*
5. 1,4, *modus ponens*
4. Eliminando o conectivo $\vee$, desejamos demonstrar que $P' \wedge (P' \rightarrow Q) \rightarrow Q$.
1. P' (hipótese)
2. $P' \rightarrow Q$ (hipótese)
3. Q (1, 2, *modus ponens*)
- 1. $(P')'$ (hipótese)
2. $(P')' \rightarrow (P' \rightarrow [(P')'])$ (Prática 14)
3. $P' \rightarrow [(P')']$ (1, 2, *modus ponens*)
4. $(P' \rightarrow [(P')']) \rightarrow [(P')' \rightarrow P]$ (Axioma 3)
5. $(P')' \rightarrow P$ (3, 4, *modus ponens*)
6. P (1,5, *modus ponens*)
10. Eliminando o conectivo $\vee$, desejamos demonstrar $(P' \rightarrow Q) \rightarrow (Q' \rightarrow P)$.
1. $P' \rightarrow Q$ (hipótese)
2. $(P' \rightarrow Q) \rightarrow [Q' \rightarrow (P')']$ (Exercício 9)
3. $Q' \rightarrow (P')'$ (1,2, *modus ponens*)
4. Q' (hipótese)
5. $(P')'$ (1,4, *modus ponens*)
6. $(P')' \rightarrow P$ (Exercício 7)
7. P (5, 6, *modus ponens*)
15. O argumento é

$$[R \wedge (F' \vee N)] \wedge N' \wedge (A' \rightarrow F) \rightarrow A \wedge R$$

Uma sequência de demonstração é:

1. $R \wedge (F' \vee N)$ (hipótese)
2. N' (hipótese)
3. $A' \rightarrow F$ (hipótese)
4. R (1, tautologia $A \wedge B \rightarrow A$, *modus ponens*)
5. $F' \vee N$ (1, tautologia $A \wedge B \rightarrow B$, *modus ponens*)
6. $(F' \wedge N) \wedge N'$ ($A \wedge B$ pode ser deduzido de A, B)
7. $(F' \vee N) \wedge N' \rightarrow F'$ (tautologia)
8. F' (6, 7, *modus ponens*)
9. $(A' \rightarrow F) \rightarrow (F' \rightarrow A)$ (tautologia)
10. $F' \rightarrow A$ (3, 9, *modus ponens*)
11. A (8, 10, *modus ponens*)
12. $A \wedge R$ ($A \wedge B$ pode ser deduzido de A, B)

18. a.

A	B	C	$B \rightarrow C$	$A \rightarrow (B \rightarrow C)$	$A \wedge B$	$(A \wedge B) \rightarrow C$	$A \rightarrow (B \rightarrow C) \leftrightarrow (A \wedge B) \rightarrow C$
V	V	V	V	V	V	V	V
V	V	F	F	F	V	F	V
V	F	V	V	V	F	V	V
V	F	F	V	V	F	V	V
F	V	V	V	V	F	V	V
F	V	F	F	V	F	V	V
F	F	V	V	V	F	V	V
F	F	F	V	V	F	V	V

- b. $A \rightarrow (B \rightarrow C) \Leftrightarrow A \rightarrow (B' \vee C) \Leftrightarrow A' \vee (B' \vee C) \Leftrightarrow (A' \vee B') \vee C \Leftrightarrow (A \wedge B)' \vee C \Leftrightarrow (A \wedge B) \rightarrow C$
 c. Pelo item (a) (ou (b)), $(P_1 \wedge P_2 \wedge \dots \wedge P_n) \rightarrow (R \rightarrow S) \Leftrightarrow (P_1 \wedge P_2 \wedge \dots \wedge P_n \wedge R) \rightarrow S$, o que diz para tomarmos $P_1, P_2, \dots, P_n, R$ como hipóteses e deduzirmos S .

Exercícios 1.4

2. a. O domínio é o conjunto dos inteiros, $P(x)$ é "x é par", $Q(x)$ é "x é ímpar".
 b. O passo 4 é um uso ilegal do Axioma 6 porque a constante a já tinha sido usada na seqüência de prova.
4. 1. $(\forall x) P(x)$ (hipótese)
 2. $P(x)$ (1, Axioma 5, *modus ponens*)
 3. $P(x) \rightarrow P(x) \vee Q(x)$ (tautologia)
 4. $P(x) \vee Q(x)$ (2, 3, *modus ponens*)
 5. $(\forall x) [P(x) \vee Q(x)]$ (4, generalização — perceba que $P(x) \vee Q(x)$ foi deduzido a partir de $(\forall x) P(x)$, no qual x não é livre)
8. 1. $(\exists x) [A(x) \wedge B(x)]$ (hipótese)
 2. $A(a) \wedge B(a)$ (1, Axioma 6, *modus ponens*)
 3. $A(a)$ (2, tautologia $A \wedge B \rightarrow A$, *modus ponens*)
 4. $B(a)$ (2, tautologia $A \wedge B \rightarrow B$, *modus ponens*)
 5. $(\exists x) A(x)$ (3, Axioma 7, *modus ponens*)
 6. $(\exists x) B(x)$ (4, Axioma 7, *modus ponens*)
 7. $(\exists x) A(x) \wedge (\exists x) B(x)$ ($A \wedge B$ pode ser deduzido de A, B)
11. 1. $(\exists x)(\forall y) Q(x, y)$ (hipótese)
 2. $(\forall y) Q(a, y)$ (1, Axioma 6, *modus ponens*)
 3. $Q(a, y)$ (2, Axioma 5, *modus ponens*)
 4. $(\exists x) Q(x, y)$ (3, Axioma 7, *modus ponens*)
 5. $(\forall y)(\exists x) Q(x, y)$ (4, generalização — condições (a) e (b) valem).
15. 1. $P(x) \rightarrow (\exists y) Q(x, y)$ (hipótese)
 2. $P(x)$ (hipótese temporária)
 3. $(\exists y) Q(x, y)$ (1, 2, *modus ponens*)
 4. $Q(x, a)$ (3, Axioma 6, *modus ponens*)
 5. $P(x) \rightarrow Q(x, a)$ (4 deduzida a partir de 2)
 6. $(\exists y)(P(x) \rightarrow Q(x, y))$ (5, Axioma 7, *modus ponens*)
17. O argumento é
 $(\forall x)[M(x) \rightarrow I(x) \vee G(x)] \wedge (\forall x)[G(x) \wedge L(x) \rightarrow F(x)] \wedge [I(j)]' \wedge L(j) \rightarrow [M(j) \rightarrow F(j)]$
 Uma seqüência de prova é
 1. $(\forall x)[M(x) \rightarrow I(x) \vee G(x)]$ (hipótese)
 2. $(\forall x)[G(x) \wedge L(x) \rightarrow F(x)]$ (hipótese)
 3. $M(j) \rightarrow I(j) \vee G(j)$ (1, Axioma 5, *modus ponens*)
 4. $G(j) \wedge L(j) \rightarrow F(j)$ (2, Axioma 5, *modus ponens*)
 5. $M(j)$ (hipótese)
 6. $I(j) \vee G(j)$ (3, 5, *modus ponens*)
 7. $[I(j)]'$ (hipótese)
 8. $[I(j)]' \wedge [I(j) \vee G(j)] \rightarrow G(j)$ (tautologia)
 9. $G(j)$ (6, 7, 8, *modus ponens*)
 10. $L(j)$ (hipótese)
 11. $G(j) \wedge L(j)$ ($A \wedge B$ pode ser deduzida de A, B)
 12. $F(j)$ (4, 11, *modus ponens*)

Exercícios 1.5

3. peixe
7. *herbívoro* (x) if *come* (x, y) **and** *planta* (y)
10. a. **is** (*autor-de*(Chico-Buarque, Estorvo))

- b. **which** (x : autor-de(Paulo-Coelho, x))
 c. *autor-não-ficção*(x) if *autor-de*(x , y) **and not** (*ficção*(y))
 d. **which**(x : *autor-não-ficção*(x))

13. $x + 1 = y - 1$ ou $x = y - 2$. Se x vale y menos 2 e o valor de x é incrementado de 1, então o valor final de x é y menos 1.
17. A pós-condição desejada é $y = x(x - 1)$. Fazendo de trás para frente, a partir da pós-condição, usando o axioma da atribuição, temos

$$\begin{aligned} &\{x(x - 1) = x(x - 1)\} \\ &\quad y := x - 1; \\ &\{xy = x(x - 1)\} \\ &\quad y := x * y; \\ &\{y = x(x - 1)\} \end{aligned}$$

Como a pré-condição é sempre verdadeira, cada asserção subsequente também o será, incluindo a pós-condição.

19. As duas implicações a demonstrar são

$$\begin{aligned} &\{y = 0 \text{ e } y < 5\} y := y + 1 \{y = 1\} \\ \text{e} \\ &\{y = 0 \text{ e } y \geq 5\} y := 5 \{y = 1\} \end{aligned}$$

A primeira implicação vale porque

$$\{y = 0\} y := y + 1 \{y = 1\}$$

é verdadeira pelo axioma da atribuição, e a segunda porque o antecedente é falso. O segmento do programa é correto devido à regra condicional.

Capítulo 2

Exercícios 2.1

1. a. Recíproca: o crescimento sadio das plantas implica uma quantidade suficiente de água.
 Contrapositiva: Se não houver crescimento sadio das plantas, então não há água suficiente.
- b. Recíproca: O crescimento da oferta de microcomputadores implica o desenvolvimento tecnológico.
 Contrapositiva: Se não há crescimento na oferta de microcomputadores, então não há avanço tecnológico.
- c. Recíproca: Se houver uma modificação no programa, então serão introduzidos erros.
 Contrapositiva: não modificar o programa implica que não serão introduzidos erros.
- d. Recíproca: Bom isolamento ou todas as janelas são janelas de tempestade implica economia de combustível.
 Contrapositiva: Mau isolamento e alguma janela que não seja janela de tempestade implica economia de combustível.

$$\begin{aligned} 3. \quad 25 &= 5^2 = 9 + 16 = 3^2 + 4^2 \\ 100 &= 10^2 = 36 + 64 + 6^2 + 8^2 \\ 169 &= 13^2 = 25 + 144 = 5^2 + 12^2 \end{aligned}$$

7. Seja $x = 2m + 1$ e $y = 2n + 1$, onde m e n são inteiros. Então

$$\begin{aligned} x + y &= (2m + 1) + (2n + 1) \\ &= 2m + 2n + 2 \\ &= 2(m + n + 1) \end{aligned} \quad \text{onde } m + n + 1 \text{ é um inteiro}$$

portanto, $x + y$ é par.

11. Seja $x = 2m$ onde m é um inteiro. Então $x^2 = (2m)^2 = 4m^2$, onde m^2 é um inteiro, portanto, x^2 é divisível por 4.

14. Se $x < y$, então ao multiplicar ambos os lados da desigualdade pelos números positivos x e y teremos, respectivamente, $x^2 < xy$ e $xy < y^2$, e portanto, $x^2 < xy < y^2$ ou $x^2 < y^2$. Se $x^2 < y^2$, então

$$\begin{aligned} y^2 - x^2 &> 0 && \text{(definição de } < \text{)} \\ (y + x)(y - x) &> 0 && \text{(fatoração)} \\ y + x < 0 \text{ e } y - x < 0 && \text{(um número positivo é o produto de dois números negativos ou dois positivos)} \\ &\text{ou} \\ y + x > 0 \text{ e } y - x > 0 \end{aligned}$$

Mas não pode ser que $y + x < 0$ porque y e x são ambos positivos, portanto $y - x > 0$ e $y > x$.

17. Sejam x e y divisíveis por n . Então $x = k_1n$ e $y = k_2n$, onde k_1 e k_2 são inteiros, e $x + y = k_1n + k_2n = (k_1 + k_2)n$, onde $k_1 + k_2$ é um inteiro. Portanto, $x + y$ é divisível por n .
20. Seja $x = 2n + 1$; então

$$\begin{aligned} x^2 &= (2n + 1)^2 = 4n^2 + 4n + 1 \\ &= 4n(n + 1) + 1 \end{aligned}$$

Mas $n(n + 1)$ é par (Exercício 9), então $n(n + 1) = 2k$ para algum inteiro k . Portanto, $x^2 = 4(2k) + 1 = 8k + 1$.

23. $m^2n^2 = (mn)^2$

28. Demonstração: se x é par, então $x = 2n$ e

$$\begin{aligned} x(x + 1)(x + 2) &= (2n)(2n + 1)(2n + 2) \\ &= 2[(n)(2n + 1)(2n + 2)] \end{aligned}$$

que é par. Se x é ímpar, então $x = 2n + 1$ e

$$\begin{aligned} x(x + 1)(x + 2) &= (2n + 1)(2n + 2)(2n + 3) \\ &= 2[(2n + 1)(n + 1)(2n + 3)] \end{aligned}$$

que é par.

31. Demonstração: Se x é par, então $x = 2n$ e

$$2n + (2n)^3 = 2n + 8n^3 = 2(n + 4n^3)$$

que é par. Se x é ímpar, então $x = 2n + 1$ e

$$\begin{aligned} (2n + 1) + (2n + 1)^3 &= (2n + 1) + (8n^3 + 12n^2 + 6n + 1) \\ &= 8n^3 + 12n^2 + 8n + 2 \\ &= 2(4n^3 + 6n^2 + 4n + 1) \end{aligned}$$

que é par.

35. Sejam x e y números racionais, $x = p/q$, $y = r/s$ com p, q, r, s , inteiros e $q, s \neq 0$. Então $x + y = p/q + r/s = (ps + rq)/qs$, onde $ps + rq$ e qs são inteiros com $qs \neq 0$. Portanto, $x + y$ é racional.

37. O ângulo 6 mais o ângulo 5 mais o ângulo da direita somam 180° pelo primeiro dado. O ângulo da direita é 90° pelo quarto dado. Portanto, o ângulo 6 mais o ângulo 5 somam 90° . O ângulo 6 tem o mesmo tamanho do ângulo 3, pelo segundo dado. Portanto, o ângulo 3 mais o ângulo 5 somam 90° .

Exercícios 2.2

1. $P(1)$: $4 + 1 - 2 = 2(1)^2$ ou $2 = 2$, verdadeiro
 Assuma $P(k)$: $2 + 6 + 10 + \dots + (4k - 2) = 2k^2$
 Mostre $P(k + 1)$: $2 + 6 + 10 + \dots + [4(k + 1) - 2] \stackrel{?}{=} 2(k + 1)^2$

$$2 + 6 + 10 + \dots + [4(k + 1) - 2] \qquad \text{(lado esquerdo de } P(k + 1))$$

$$\begin{aligned}
&= 2 + 6 + 10 + \dots + (4k - 2) + [4(k + 1) - 2] \\
&= 2k^2 + 4(k + 1) - 2 && \text{(usando } P(k)) \\
&= 2k^2 + 4k + 2 \\
&= 2(k^2 + 4k + 1) \\
&= 2(k + 1)^2 && \text{(lado direito de } P(k + 1))
\end{aligned}$$

3. $P(1)$: $1 = 1(2 \cdot 1 - 1)$, verdadeiro

Assuma $P(k)$: $1 + 5 + 9 + \dots + (4k - 3) = k(2k - 1)$

Mostre $P(k + 1)$: $1 + 5 + 9 + \dots + [4(k + 1) - 3] \stackrel{?}{=} (k + 1)[2(k + 1) - 1]$

$$\begin{aligned}
&1 + 5 + 9 + \dots + [4(k + 1) - 3] && \text{(lado esquerdo de } P(k + 1)) \\
&= 1 + 5 + 9 + \dots + (4k - 3) + [4(k + 1) - 3] \\
&= k(2k - 1) + 4(k + 1) - 3 && \text{(usando } P(k)) \\
&= 2k^2 - k + 4k + 1 \\
&= 2k^2 + 3k + 1 \\
&= (k + 1)(2k + 1) \\
&= (k + 1)[2(k + 1) - 1] && \text{(lado direito de } P(k + 1))
\end{aligned}$$

5. $P(1)$: $6 - 2 = 1(3 \cdot 1 + 1)$, verdadeiro

Assuma $P(k)$: $4 + 10 + 16 + \dots + (6k - 2) = k(3k + 1)$

Mostre $P(k + 1)$: $4 + 10 + 16 + \dots + [6(k + 1) - 2] \stackrel{?}{=} (k + 1)[3(k + 1) + 1]$

$$\begin{aligned}
&4 + 10 + 16 + \dots + [6(k + 1) - 2] && \text{(lado esquerdo de } P(k + 1)) \\
&= 4 + 10 + 16 + \dots + (6k - 2) + [6(k + 1) - 2] \\
&= k(3k + 1) + 6(k + 1) - 2 && \text{(usando } P(k)) \\
&= 3k^2 + k + 6k + 4 \\
&= 3k^2 + 7k + 4 \\
&= (k + 1)(3k + 4) \\
&= (k + 1)[3(k + 1) + 1] && \text{(lado direito de } P(k + 1))
\end{aligned}$$

9. $P(1) = 1^2 = \frac{1(2 - 1)(2 + 1)}{3}$, verdadeiro

Assuma $P(k)$: $1^2 + 3^2 + \dots + (2k - 1)^2 = \frac{k(2k - 1)(2k + 1)}{3}$

Mostre $P(k + 1)$: $1^2 + 3^2 + \dots + [2(k + 1) - 1]^2 \stackrel{?}{=} \frac{(k + 1)[2(k + 1) - 1][2(k + 1) + 1]}{3}$

$$\begin{aligned}
&1^2 + 3^2 + \dots + [2(k + 1) - 1]^2 && \text{(lado esquerdo de } P(k + 1)) \\
&= 1^2 + 3^2 + \dots + (2k - 1)^2 + [2(k + 1) - 1]^2 \\
&= \frac{k(2k - 1)(2k + 1)}{3} + [2(k + 1) - 1]^2 && \text{(usando } P(k)) \\
&= \frac{k(2k - 1)(2k + 1)}{3} + (2k + 1)^2 \\
&= (2k + 1) \left[\frac{k(2k - 1)}{3} + 2k + 1 \right] \\
&= (2k + 1) \left(\frac{2k^2 - k + 6k + 3}{3} \right) \\
&= \frac{(2k + 1)(2k^2 + 5k + 3)}{3} \\
&= \frac{(2k + 1)(k + 1)(2k + 3)}{3} \\
&= \frac{(k + 1)[2(k + 1) - 1][2(k + 1) + 1]}{3} && \text{(lado direito de } P(k + 1))
\end{aligned}$$

13. $P(1)$: $\frac{1}{1 \cdot 2} = \frac{1}{1 + 1}$, verdadeiro

$$\text{Assuma } P(k): \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \dots + \frac{1}{k(k+1)} = \frac{k}{k+1}$$

$$\text{Mostre } P(k+1): \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \dots + \frac{1}{(k+1)(k+2)} \stackrel{?}{=} \frac{k+1}{k+2}$$

$$\frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \dots + \frac{1}{(k+1)(k+2)} \quad (\text{lado esquerdo de } P(k+1))$$

$$= \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \dots + \frac{1}{k(k+1)} + \frac{1}{(k+1)(k+2)}$$

$$= \frac{k}{k+1} + \frac{1}{(k+1)(k+2)} \quad (\text{usando } P(k))$$

$$= \frac{k(k+2)+1}{(k+1)(k+2)}$$

$$= \frac{k^2+2k+1}{(k+1)(k+2)}$$

$$= \frac{(k+1)^2}{(k+1)(k+2)}$$

$$= \frac{k+1}{k+2} \quad (\text{lado direito de } P(k+1))$$

$$15. P(1): 1^2 = \frac{(-1)^2(1)(2)}{2}, \text{ verdadeiro}$$

$$\text{Assuma } P(k): 1^2 - 2^2 + \dots + (-1)^{k+1}k^2 = \frac{(-1)^{k+1}(k)(k+1)}{2}$$

$$\text{Mostre } P(k+1): 1^2 - 2^2 + \dots + (-1)^{k+2}(k+1)^2 \stackrel{?}{=} \frac{(-1)^{k+2}(k+1)(k+2)}{2}$$

$$1^2 - 2^2 + \dots + (-1)^{k+2}(k+1)^2 \quad (\text{lado esquerdo de } P(k+1))$$

$$= 1^2 - 2^2 + \dots + (-1)^{k+1}k^2 + (-1)^{k+2}(k+1)^2$$

$$= \frac{(-1)^{k+1}(k)(k+1)}{2} + (-1)^{k+2}(k+1)^2 \quad (\text{usando } P(k))$$

$$= \frac{(-1)^{k+1}(k)(k+1) + 2(-1)^{k+2}(k+1)^2}{2}$$

$$= (-1)^{k+2}(k+1) \frac{[k(-1)^{-1} + 2(k+1)]}{2}$$

$$= (-1)^{k+2}(k+1) \left(\frac{-k+2k+2}{2} \right)$$

$$= \frac{(-1)^{k+2}(k+1)(k+2)}{2} \quad (\text{lado direito de } P(k+1))$$

$$17. P(1): a = \frac{a-ar}{1-r} = \frac{a(1-r)}{1-r}, \text{ verdadeiro}$$

$$\text{Assuma } P(k): a + ar + \dots + ar^{k-1} = \frac{a-ar^k}{1-r}$$

$$\text{Mostre } P(k+1): a + ar + \dots + ar^k \stackrel{?}{=} \frac{a-ar^{k+1}}{1-r}$$

$$a + ar + \dots + ar^k \quad (\text{lado esquerdo de } P(k+1))$$

$$= a + ar + \dots + ar^{k-1} + ar^k$$

$$(\text{usando } P(k))$$

$$\begin{aligned}
 &= \frac{a - ar^k + ar^k(1-r)}{1-r} \\
 &= \frac{a - ar^{k+1}}{1-r} \quad (\text{lado esquerdo de } P(k+1))
 \end{aligned}$$

19. $P(2)$: $2^2 > 2 + 1$, verdadeiro

Assuma $P(k)$: $k^2 > k + 1$

Mostre $P(k+1)$: $(k+1)^2 > k + 2$

$$\begin{aligned}
 (k+1)^2 &= k^2 + 2k + 1 \\
 &> (k+1) + 2k + 1 \quad (\text{usando } P(k)) \\
 &= 3k + 2 > k + 2
 \end{aligned}$$

23. $P(4)$: $2^4 < 4!$ ou $16 < 24$, verdadeiro

Assuma $P(k)$: $2^k < k!$

Mostre $P(k+1)$: $2^{k+1} < (k+1)!$

$$\begin{aligned}
 2^{k+1} &= 2^k \cdot 2 < k!(2) < k!(k+1) \\
 &= (k+1)! \quad (\text{primeira desigualdade devida a } P(k); \text{ segunda devida a } k > 4)
 \end{aligned}$$

27. $P(2)$: $1 + 2 < 2^2$ ou $3 < 4$, verdadeiro

Assuma $P(k)$: $1 + 2 + \dots + k < k^2$

Mostre $P(k+1)$: $1 + 2 + \dots + (k+1) < (k+1)^2$

$$\begin{aligned}
 &1 + 2 + \dots + (k+1) \\
 &= 1 + 2 + \dots + k + (k+1) \\
 &< k^2 + k + 1 \quad (\text{usando } P(k)) \\
 &< k^2 + 2k + 1 = (k+1)^2
 \end{aligned}$$

29. $P(1)$: $2^3 - 1 = 8 - 1 = 7$ e 7 divide 7, verdadeiro

Assuma $P(k)$: 7 divide $(2^{3k} - 1)$ então $2^{3k} - 1 = 7m$ ou $2^{3k} = 7m + 1$ para algum m inteiro.

Mostre $P(k+1)$: $2^{3(k+1)} - 1$ é divisível por 7.

$$\begin{aligned}
 2^{3(k+1)} - 1 &= 2^{3k+3} - 1 = 2^{3k} \cdot 2^3 - 1 \\
 &= (7m + 1)2^3 - 1 \\
 &= 7(2^3m) + 8 - 1 = 7(2^3m + 1) \quad (\text{onde } 2^3m + 1 \text{ é um inteiro})
 \end{aligned}$$

33. $P(1)$: $2 + (-1)^2 = 2 + 1 = 3$ e 3 divide 3, verdadeiro

Assuma $P(k)$: 3 divide $[2^k + (-1)^{k+1}]$ então $2^k + (-1)^{k+1} = 3m$ ou $2^k = 3m - (-1)^{k+1}$ para algum m inteiro.

Mostre $P(k+1)$: $2^{k+1} + (-1)^{k+2}$ é divisível por 3

$$\begin{aligned}
 &2^{k+1} + (-1)^{k+2} \\
 &= 2 \cdot 2^k + (-1)^{k+2} \\
 &= 2[3m - (-1)^{k+1}] + (-1)^{k+2} \quad (\text{usando } P(k)) \\
 &= 3(2m) - 2(-1)^{k+1} + (-1)^{k+2} \\
 &= 3(2m) + (-1)^{k+1}[-2 + (-1)] \\
 &= 3(2m) + (-1)^{k+1}(-3) \\
 &= 3[2m - (-1)^{k+1}] \quad (\text{onde } 2m - (-1)^{k+1} \text{ é um inteiro})
 \end{aligned}$$

37. $P(1)$: $10 + 3 \cdot 4^3 + 5 = 10 + 192 + 5 = 207 = 9 \cdot 23$, verdadeiro

Assuma 9 divide $(10^k + 3 \cdot 4^{k+2} + 5)$ então $10^k + 3 \cdot 4^{k+2} + 5 = 9m$ ou $10^k = 9m - 3 \cdot 4^{k+2} - 5$ para algum m inteiro.

Mostre $P(k+1)$: $10^{k+1} + 3 \cdot 4^{k+3} + 5$ é divisível por 9

$$\begin{aligned}
 &10^{k+1} + 3 \cdot 4^{k+3} + 5 \\
 &= 10 \cdot 10^k + 3 \cdot 4^{k+3} + 5 \\
 &= 10(9m - 3 \cdot 4^{k+2} - 5) + 3 \cdot 4^{k+3} + 5 \quad (\text{usando } P(k)) \\
 &= 9(10m) - 30 \cdot 4^{k+2} - 50 + 3 \cdot 4^{k+2} \cdot 4 + 5 \\
 &= 9(10m) - 45 - 3 \cdot 4^{k+2}(10 - 4) \\
 &= 9(10m - 5) - 18 \cdot 4^{k+2}
 \end{aligned}$$

$$= 9(10m - 5 - 2 \cdot 4^{k+2}) \quad (\text{onde } 10m - 5 - 2 \cdot 4^{k+2} \text{ é um inteiro})$$

41. $P(1)$: $\cos \theta + i \sin \theta = \cos \theta + i \sin \theta$, verdadeiro

Assuma $P(k)$: $(\cos \theta + i \sin \theta)^k = \cos k\theta + i \sin k\theta$

Mostre $P(k+1)$: $(\cos \theta + i \sin \theta)^{k+1} \stackrel{?}{=} \cos(k+1)\theta + i \sin(k+1)\theta$

$$\begin{aligned} (\cos \theta + i \sin \theta)^{k+1} &= (\cos \theta + i \sin \theta)^k (\cos \theta + i \sin \theta) \\ &= (\cos k\theta + i \sin k\theta)(\cos \theta + i \sin \theta) && (\text{usando } P(k)) \\ &= \cos k\theta \cos \theta + i \sin k\theta \cos \theta \\ &\quad + i \cos k\theta \sin \theta + i^2 \sin k\theta \sin \theta \\ &= \cos k\theta \cos \theta - \sin k\theta \sin \theta \\ &\quad + i(\sin k\theta \cos \theta + \cos k\theta \sin \theta) \\ &= \cos(k\theta + \theta) + i \sin(k\theta + \theta) \\ &= \cos(k+1)\theta + i \sin(k+1)\theta \end{aligned}$$

43. A afirmação a ser provada é que $n(n+1)(n+2)$ é divisível por 3 para $n \geq 1$.

$P(1)$: $1(1+1)(1+2) = 6$ é divisível por 3, verdadeiro

Assuma $P(k)$: $k(k+1)(k+2) = 3m$ para algum m inteiro

Mostre $P(k+1)$: $(k+1)(k+2)(k+3)$ é divisível por 3

$$\begin{aligned} (k+1)(k+2)(k+3) &= (k+1)(k+2)k + (k+1)(k+2)^3 \\ &= 3m + (k+1)(k+2)^3 \\ &= 3[m + (k+1)(k+2)^3] \end{aligned}$$

46. $P(1)$ é $1 = 1 + 1$, que não é verdade.

49. $P(2)$ e $P(3)$ são verdadeiras pelas equações $2 = 2$ e $3 = 3$. Assuma agora que $P(r)$ é verdadeira para qualquer r , $2 \leq r \leq k$ e considere $P(k+1)$. Devemos assumir $k+1 \geq 4$, de forma que $(k+1) - 2 \geq 2$ que, pela hipótese de indução, pode ser escrita como uma soma de 2s e 3s. Somando 2 temos $k+1$ como uma soma de 2s e 3s.

52. $P(64)$, $P(65)$, $P(66)$, $P(67)$, $P(68)$ são verdadeiras pelas equações

$$\begin{aligned} 64 &= 6(5) + 2(17) \\ 65 &= 13(5) \\ 66 &= 3(5) + 3(17) \\ 67 &= 10(5) + 17 \\ 68 &= 4(17) \end{aligned}$$

Assuma agora que $P(r)$ é verdadeira para qualquer r , $64 \leq r \leq k$ e considere $P(k+1)$. Devemos assumir que $k+1 \geq 69$, de forma que $(k+1) - 5 \geq 64$ que, pela hipótese de indução, pode ser escrita como uma soma de 5s e 17s. Somando 5 temos $k+1$ como a soma de 5s e 17s.

Exercícios 2.3

1. 10,20,30,40,50

Mostre que

$$F(k+4) \geq 2F(k+2) + F(k+1)$$

onde $k+1 \geq 3$.

$$\begin{aligned} F(k+4) &= F(k+2) + F(k+3) && \text{(relação de recorrência)} \\ &= 2F(k) + F(k-1) + 2F(k+1) + F(k) && \text{(hipótese de indução)} \\ &= 2[F(k) + F(k+1)] + [F(k-1) + F(k)] \\ &= 2F(k+2) + F(k+1) && \text{(relação de recorrência)} \end{aligned}$$

19. 1. $S(1) = a$
2. $S(n) = rS(n-1)$ para $n \geq 2$

21. a. $A(1) = 50.000$
 $A(n) = 3A(n-1)$ para $n \geq 2$
b. 4

25. (a), (b), (c)

27. 1. Qualquer predicado unário em x é uma wff
2. Se P e Q são predicados unários wffs em x , então $(P \wedge Q)$, $(P \vee Q)$, $(P \rightarrow Q)$, (P') , $(P \leftrightarrow Q)$, $(\forall x)P$, e $(\exists x)P$.

31. 1. $\lambda^R = \lambda$
2. Se x é uma cadeia com um único carácter, $x^R = x$.
3. Se $x = yz$, então $x^R = z^R y^R$

33. $\langle \text{dígito positivo} \rangle ::= 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$
 $\langle \text{dígito} \rangle ::= 0 \mid \langle \text{dígito positivo} \rangle$
 $\langle \text{inteiro positivo} \rangle ::= \langle \text{dígito positivo} \rangle \mid \langle \text{inteiro positivo} \rangle \langle \text{dígito} \rangle$

35. 1. $1! = 1$
2. $n! = n(n-1)!$ para $n \geq 2$

39. $A \vee (B_1 \wedge B_2) \Leftrightarrow (A \vee B_1) \wedge (A \vee B_2)$ é a equivalência 3a da Seção 1.1.
Assuma que

$$A \vee (B_1 \wedge \dots \wedge B_k) \Leftrightarrow (A \vee B_1) \wedge \dots \wedge (A \vee B_k)$$

Então

$$\begin{aligned} &A \vee (B_1 \wedge \dots \wedge B_{k+1}) \\ &= A \vee [(B_1 \wedge \dots \wedge B_k) \wedge B_{k+1}] && \text{(pelo Exercício 38)} \\ &\Leftrightarrow [A \vee (B_1 \wedge \dots \wedge B_k)] \wedge (A \vee B_{k+1}) && \text{(pela equivalência 3a)} \\ &\Leftrightarrow [(A \vee B_1) \wedge \dots \wedge (A \vee B_k)] \wedge (A \vee B_{k+1}) && \text{(pela hipótese de indução)} \\ &= (A \vee B_1) \wedge \dots \wedge (A \vee B_{k+1}) && \text{(pelo Exercício 38)} \end{aligned}$$

A prova da outra sentença é semelhante.

43. if $n = 1$ then
 $S := 1$
else
 $S := S(n-1) + (n-1)$

46. if $n = 1$ then
 $S := p$
else
 if *ímpar*(n) then
 $S := S(n-1) + (n-1) * q$
 else
 $S := S(n-1) - (n-1) * q$

51. 4, 10, -6, 2, 5
4, 5, -6, 2, 10

$$4, 2, -6, 5, 10$$

$$-6, 2, \quad 4, 5, 10$$

55. A relação de recorrência satisfaz a equação (6) com $c = 1$ e $g(n) = 5$. Da equação (8), a solução é

$$S(n) = 5 + \sum_{i=2}^n 5 = 5 + (n-1)5 = n(5)$$

58. A relação de recorrência satisfaz a equação (6) com $c = 1$ e $g(n) = n$. Da equação (8), a solução é

$$A(n) = 1 + \sum_{i=2}^n i$$

$$= 1 + 2 + 3 + \dots + n$$

$$= \frac{n(n+1)}{2}$$

$$61. F(n) = nF(n-1)$$

$$= n[(n-1)F(n-2)] = n(n-1)F(n-2)$$

$$= n(n-1)[(n-2)F(n-3)] = n(n-1)(n-2)F(n-3)$$

Em geral, $F(n) = n(n-1)(n-2)\dots [n-(k-1)]F(n-k)$. Quando $n-k=1, k=n-1$ e

$$F(n) = n(n-1)(n-2)\dots (2)F(1)$$

$$= n(n-1)(n-2)\dots (2)(1) = n!$$

Agora a prova por indução que $F(n) = n!$:

$F(1): F(1) = 1! = 1$, verdadeiro

Assuma $F(k): F(k) = k!$

Mostre $F(k+1): F(k+1) = (k+1)!$

$$F(k+1) = (k+1)F(k) = (k+1)k! = (k+1)!$$

65. A relação de recorrência é $P(n) = P(n-1) + n$, com $P(1) = 1$. A equação (8) se aplica e fornece a solução

$$P(n) = 1 + \sum_{i=2}^n i = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Exercícios 2.4

1. Se a lista está vazia, escreva "não encontrado", caso contrário, compare x ao primeiro elemento do segmento corrente da lista: se eles forem iguais, escreva "encontrado", do contrário descarte o primeiro elemento e procure no segmento restante da lista. Inicialmente, o segmento da lista é a lista toda. Mais formalmente,

procedure BuscaSeqüencial(L : lista; i, n : integer; x : tipoitem);
 {procura o elemento x na lista L de i a n }

```
begin
  if  $i > n$  then
    escreva ('não encontrado')
  else
    if  $L(i) = x$  then
      escreva ('encontrado')
    else
      BuscaSeqüencial ( $L, i + 1, n, x$ );
end;
```

4. Ela está na forma da equação (1) com $c = 2$ e $g(n) = 3$. Pela equação (6), a solução é

$$\begin{aligned} & 2^{\log n}(1) + \sum_{i=1}^{\log n} 2^{(\log n)-i}(3) \\ &= 2^{\log n} + 3[2^{(\log n)-1} + 2^{(\log n)-2} + \dots + 2^0] \\ &= 2^{\log n} + 3(2^{\log n} - 1) = n + 3(n - 1) = 4n - 3 \end{aligned}$$

7. $n - 1$ comparações são sempre necessárias; todo elemento após o primeiro precisa ser considerado um máximo em potencial.

10. a. A lista combinada é 1, 4, 5, 6, 8, 9
3 comparações: 6 e 1, 6 e 4, 6 e 5
b. A lista combinada é 1, 2, 3, 4, 5, 8
4 comparações: 1 e 2, 5 e 2, 5 e 3, 5 e 4
c. A lista combinada é 0, 1, 2, 3, 4, 7, 8, 9, 10
8 comparações: 0 e 1, 2 e 1, 2 e 8, 3 e 8, 4 e 8, 7 e 8, 10 e 8, 10 e 9.

12. $C(1) = 0$ (não são necessárias comparações; uma lista de 1 elemento já está ordenada)
 $C(n) = C(n/2) + (n - 1)$ para $n = 2^m, n \geq 2$ ($C(n/2)$ comparações são necessárias para cada metade e $n - 1$ comparações são necessárias para combinar as duas metades.)

16. $Q(0): j_0 = (i_0 - 1)!$, verdadeiro, pois $j = 1, i = 2$ antes de entrar no laço e $1 = 1!$
Assuma $Q(k): j_k = (i_k - 1)!$
Mostre $Q(k + 1): j_{k+1} \stackrel{?}{=} (i_{k+1} - 1)!$

$$j_{k+1} = j_k i_k = (i_k - 1)! i_k = (i_k)! = (i_{k+1} - 1)!$$

Ao fim do laço, $j = (i - 1)!$ e $i = x + 1$, logo $j = x!$

20. $Q: j = x * y^i$
 $Q(0): j_0 = x * y^0$, verdadeiro, pois $j = x, i = 0$ antes de entrar
Assuma $Q(k): j_k = x * y^{i_k}$
 $Q(k + 1): j_{k+1} = x * y^{i_{k+1}}$
 $j_{k+1} = j_k * y = x * y^{i_k} * y = x * y^{i_k + 1} = x * y^{i_{k+1}}$
Ao fim do laço, $j = x * y^i$ e $i = n$, logo $j = x * y^n$.

Capítulo 3

Exercícios 3.1

1. (a)
4. a. $\{0, 1, 2, 3, 4\}$
b. $\{4, 6, 8, 10\}$
c. $\{\text{Deodoro da Fonseca, Floriano Peixoto, Prudente de Moraes}\}$
d. $\emptyset$
e. $\{\text{Maranhão, Piauí, Ceará, Rio Grande do Norte, Paraíba, Pernambuco, Alagoas, Sergipe, Bahia}\}$
f. $\{-3, -2, -1, 0, 1, 2, 3\}$
8. Se $A = \{x \mid x = 2^n \text{ para } n \text{ inteiro positivo}\}$, então $16 \in A$. Mas $A \neq \{x \mid x = 2^n + n(n - 1) \text{ para } n \text{ inteiro positivo}\}$ então $16 \notin A$.
10. a. F; $\{1\} \in S$ mas $\{1\} \notin R$
b. V
c. F; $\{1\} \in S$, e não $1 \in S$
d. F; 1 não é um conjunto; o enunciado correto é $\{1\} \subseteq U$
e. T f. F; $1 \notin S$

12. Seja $(x, y) \in A$. Então (x, y) está a menos de três unidades do ponto $(1, 4)$, portanto, pela fórmula da distância,

$$\sqrt{(x-1)^2 + (y-4)^2} \leq 3 \text{ ou } (x-1)^2 + (y-4)^2 \leq 9$$

que significa

$$(x-1)^2 + (y-4)^2 \leq 25$$

logo $(x, y) \in B$. O ponto $(6, 4)$ satisfaz a desigualdade

$$(x-1)^2 + (y-4)^2 \leq 25$$

portanto, $(6, 4) \in B$, mas $(6, 4)$ está a mais de 3 unidades de $(1, 4)$, e, portanto, não pertence a A .

15. a. V b. F c. F d. V e. V f. F

19. $\mathcal{P}(S) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{4\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}, \{1, 2, 3\}, \{1, 2, 4\}, \{1, 3, 4\}, \{2, 3, 4\}, \{1, 2, 3, 4\}\}$; $2^4 = 16$ elementos.

23. $A = \{x, y\}$

26. Seja $x \in A$. Então $\{x\} \in \mathcal{P}(A)$, logo $\{x\} \in \mathcal{P}(B)$ e $x \in B$. Portanto, $A \subseteq B$. Um argumento semelhante mostra que $B \subseteq A$. Logo $A = B$.

29. a. Operação binária
b. Não; $0 \circ 0 \notin \mathbb{N}$
c. Operação binária

32. a. 13 b. 2 c. 28

34. a. $\{1, 2, 4, 5, 6, 8, 9\}$
b. $\{4, 5\}$
c. $\{2, 4\}$
h. $\{0, 1, 2, 3, 6, 7, 8, 9\}$

37. a. B' b. $B \cap C$ c. $A \cap B$ d. $B' \cap C$
e. $B' \cap C'$ (ou $(B \cup C)'$ ou $B' - C$)

39. a. V
c. F (Sejam $A = \{1, 2, 3\}$, $B = \{1, 3, 5\}$, $S = \{1, 2, 3, 4, 5\}$.
Então $(A \cap B)' = \{2, 4, 5\}$ mas $A' \cap B' = \{4, 5\} \cap \{2, 4\} = \{4\}$).
e. F (Tome A , B e S como em (c), então $A - B = \{2\}$, $(B - A)' = \{1, 2, 3, 4\}$.)

40. a. $B \subseteq A$

43. Seja $C \in \mathcal{P}(A) \cap \mathcal{P}(B)$. Então $C \in \mathcal{P}(A)$ e $C \in \mathcal{P}(B)$, donde $C \subseteq A$ e $C \subseteq B$, portanto $C \subseteq A \cap B$ ou $C \in \mathcal{P}(A \cap B)$. Portanto, $\mathcal{P}(A) \cap \mathcal{P}(B) \subseteq \mathcal{P}(A \cap B)$. O mesmo argumento vale para a recíproca.

46. Suponha $A \cap B \neq \emptyset$. Seja $x \in A \cap B$. Então $x \in A$ e $x \in B$ logo $x \in A \cup B$ mas $x \notin A - B$ e $x \notin B - A$, portanto $x \notin (A - B) \cup (B - A)$, o que contradiz a igualdade de $A \cup B$ e $(A - B) \cup (B - A)$.

51. a. $(1) (A \cup B) \cap (A \cup B') = A \cup (B \cap B') = A \cup \emptyset = A$

52. a. $x \in (A')' \leftrightarrow x \notin A' \leftrightarrow x \notin \{y \mid y \notin A\} \rightarrow x \in A$
b. $A \cap (B \cup A') = (A \cap B) \cup (A \cap A')$ (3b)
 $= (A \cap B) \cup \emptyset$ (5b)
 $= A \cap B$ (4a)
 $= B \cap A$ (1b)

54. A prova é realizada por indução em n . Para $n = 3$,

$$\begin{aligned} A_1 \cup (A_2 \cup A_3) &= (A_1 \cup A_2) \cup A_3 \\ &= A_1 \cup A_2 \cup A_3 \end{aligned}$$

(identidade de conjuntos 2a)
(pela parte dois da resposta do Exercício 53b)

Assuma que para $n = k$ e $1 \leq p \leq k - 1$,

$$(A_1 \cup \dots \cup A_p) \cup (A_{p+1} \cup \dots \cup A_k) = A_1 \cup \dots \cup A_k$$

Então, para $1 \leq p \leq k$,

$$\begin{aligned} &(A_1 \cup \dots \cup A_p) \cup (A_{p+1} \cup \dots \cup A_{k+1}) \\ &= (A_1 \cup \dots \cup A_p) \cup [(A_{p+1} \cup \dots \cup A_k) \cup A_{k+1}] \quad (\text{pelo Exercício 53b}) \\ &= [(A_1 \cup \dots \cup A_p) \cup (A_{p+1} \cup \dots \cup A_k)] \cup A_{k+1} \quad (\text{pela identidade de conjuntos 2a}) \\ &= (A_1 \cup \dots \cup A_k) \cup A_{k+1} \quad (\text{pela hipótese de indução}) \\ &= A_1 \cup \dots \cup A_{k+1} \quad (\text{pelo Exercício 53b}) \end{aligned}$$

57. a. A prova é realizada por indução em n . Para $n = 2$,

$$B \cup (A_1 \cap A_2) = (B \cup A_1) \cap (B \cup A_2) \quad (\text{pela identidade 3a})$$

Assuma que

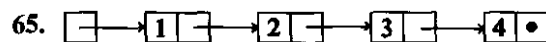
$$B \cup (A_1 \cap \dots \cap A_k) = (B \cup A_1) \cap \dots \cap (B \cup A_k)$$

Então

$$\begin{aligned} &B \cup (A_1 \cap \dots \cap A_{k+1}) \\ &= B \cup [(A_1 \cap \dots \cap A_k) \cap A_{k+1}] \quad (\text{pelo Exercício 55b}) \\ &= [B \cup (A_1 \cap \dots \cap A_k)] \cap (B \cup A_{k+1}) \quad (\text{pela identidade 3a}) \\ &= [(B \cup A_1) \cap \dots \cap (B \cup A_k)] \cap (B \cup A_{k+1}) \quad (\text{pela hipótese de indução}) \\ &= (B \cup A_1) \cap \dots \cap (B \cup A_{k+1}) \quad (\text{pelo Exercício 55b}) \end{aligned}$$

62. Se T é um conjunto não vazio, então, pelo princípio da boa-ordenação, T tem um elemento mínimo t_0 . Então $P(t_0)$ não é verdade, logo pela sentença 1', $t_0 \neq 1$. Além disso, $P(r)$ é verdadeiro para todo r , $1 \leq r \leq t_0 - 1$. Isto contradiz a implicação em 2', então T é o conjunto vazio, e portanto $P(n)$ é verdadeiro para todo n inteiro positivo.

63. a. 01101 b. 00001 c. 00000



67. Uma enumeração do conjunto é
1, 3, 5, 7, 9, 11, ...

73. Sejam A e B conjuntos enumeráveis com enumerações $A = a_1, a_2, a_3, \dots$ e $B = b_1, b_2, b_3, \dots$. Então use a lista $a_1, b_1, a_2, b_2, a_3, b_3, \dots$ e elimine quaisquer duplicatas. Isto será uma enumeração de $A \cup B$, que é, portanto, enumerável.

Exercícios 3.2

1. $5 \cdot 3 \cdot 2 = 30$

2. $4 \cdot 2 \cdot 2 = 16$

7. $45 \cdot 13 = 585$

9. 10^9

11. $26 \cdot 26 \cdot 26 \cdot 1 \cdot 1 = 17.576$

14. $26 + 26 \cdot 10 = 286$

17. $5 \cdot 3 \cdot 4 \cdot 3 = 180$

21. $9 \cdot 10 \cdot 26 \cdot 10 \cdot 10 + 9 \cdot 10 \cdot 26 \cdot 10 \cdot 10 \cdot 10 + 9 \cdot 10 \cdot 26 \cdot 10 \cdot 10 \cdot 10 \cdot 10 = 25.974.000$

23. $5 \cdot 3 = 15$

26. $900 - 180 = 720$

31. $2^8 = 256$

33. $1 \cdot 2^7$ (começam com 0) + $1 \cdot 2^6 \cdot 1$ (começam com 1 e terminam com 0) = $2^7 + 2^6 = 192$

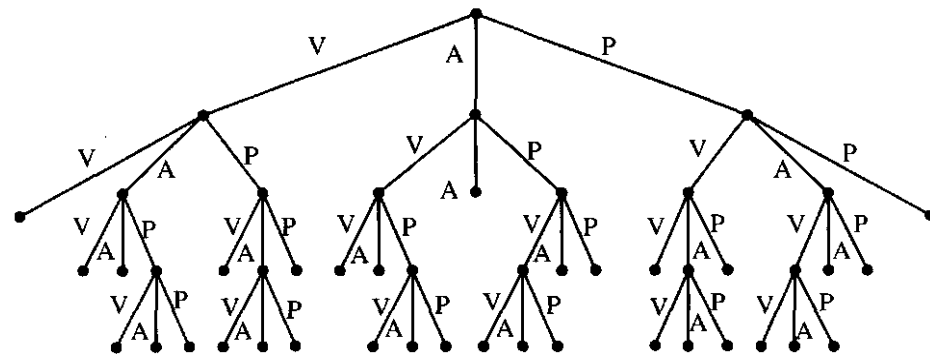
39. 8 (Este é o mesmo problema que o Exercício 36).

41. $52 \cdot 52 = 2704$

44. Existem $4 \cdot 4 = 16$ maneiras de duas cartas serem de um mesmo valor/figura; existem 13 valores/figuras diferentes. Portanto, pelo Princípio da Adição, $16 + 16 + \dots + 16 = 13 \cdot 16 = 208$.

49. $12 \cdot 52$ (carta de figura do baralho de flores e qualquer carta do baralho de pássaros) + $40 \cdot 12$ (carta de número do baralho de flores e carta de figura do baralho de pássaros) = 1104 ou $52 \cdot 52$ (número total de mãos, Exercício 41) — $40 \cdot 40$ (mão sem cartas de figuras, Exercício 48) = 1104.

51.



Exercício 3.3

2. 19

5. Seja A = conjunto dos que veiculam hálito puro, B = conjunto dos que veiculam que protegem a gengiva, C = conjunto dos que veiculam a redução da placa.

a. $|A \cap B \cap C| = 2$ b. $|A - C| = |A| - |A \cap C| = 10 - 6 = 4$

7. Seja A = conjunto de carros, B = conjunto de bicicletas, C = conjunto de motocicletas.

a. $|B - (A \cup C)| = |B| - |B \cap (A \cup C)|$ (pelo Exemplo 35)
 $= |B| - |(B \cap A) \cup (B \cap C)|$
 $= |B| - (|B \cap A| + |B \cap C| - |B \cap A \cap C|)$
 $= 97 - (53 + 7 - 2) = 39$

b. $|A \cup B \cup C| = 97 + 83 + 28 - 53 - 14 - 7 + 2 = 136$

portanto

$$150 - 136 = 14$$

estudantes não possuem qualquer dos três.

12. Não. Existem 13 tipos de cartas, de forma que 12 cartas podem ser diferentes.

15. Existem três pares — 1 e 6, 2 e 5, 3 e 4 — que somam 7. Cada elemento no conjunto pertence a um desses pares. Aplique o Princípio da Casa do Pombo, onde os pares são as casas, e os números são os pombos.

Exercícios 3.4**1. a.** 42

5. No total existem $5!$ permutações e $3!$ arranjos dos 3 Rs para cada permutação distinta. Portanto existem $5!/3! = 5 \cdot 4 = 20$ permutações distintas.

$$7. P(15, 3) = \frac{15!}{12!} 15 \cdot 14 \cdot 13 = 2730.$$

$$9. (2!)(11!)(8!) = 2(39.916.800)(40.320)$$

11. a. 120

$$13. C(300, 25) = \frac{300!}{25!275!}$$

$$15. C(17, 5) \cdot C(23, 7) = (6188) (245.157)$$

$$17. C(7, 1) \cdot C(14, 1) \cdot C(4, 1) \cdot C(5, 1) \cdot C(2, 1) \cdot C(3, 1) = 7 \cdot 14 \cdot 4 \cdot 5 \cdot 2 \cdot 3 = 11.760$$

$$20. (\text{Todos os comitês}) - (\text{nenhum ou 1 de manufatura}) = C(35, 6) - (C(21, 6) + C(14, 1) \cdot C(21, 5))$$

$$21. C(13, 3) \cdot C(13, 2)$$

$$23. C(13, 5) + C(13, 5) + C(13, 5) + C(13, 5) = 4C(13, 5)$$

$$27. C(12, 4)$$

$$32. C(60, 1) + C(60, 2)$$

$$35. C(12, 3)$$

$$37. C(7, 3) (\text{nenhum democrata}) + C(9, 3) (\text{nenhum republicano}) - C(4, 3) (\text{todos os independentes, a fim de não contar duas vezes}) = 115$$

$$39. C(14, 6) = 3003$$

$$43. (\text{Todos os comitês}) - (\text{comitês com nenhum dos dois}) = C(25, 5) - C(23, 5) \\ (\text{Não é } 1 \cdot C(24, 4) + 1 \cdot C(24, 4); \text{ este número é muito grande, porque conta algumas combinações mais de uma vez.})$$

$$47. \text{ a. } \frac{8!}{3!2!}$$

$$\text{ b. } \frac{7!}{3!2!}$$

$$51. C(7, 5)$$

$$55. \text{ a. } C(8, 6)$$

$$\text{ b. } C(7, 6) (\text{escolhe dois dentre seis, com repetições})$$

$$\text{ c. } C(5, 3) (\text{três dos seis carregamentos estão fixos, escolha os outros três dentre três, com repetições}).$$

$$57. \text{ a. } C(16, 10)$$

$$\text{ b. } C(9, 3) (\text{sete dos doces estão fixos, escolha os três restantes dentre sete crianças, com possíveis repetições})$$

$$59. C(13, 10)$$

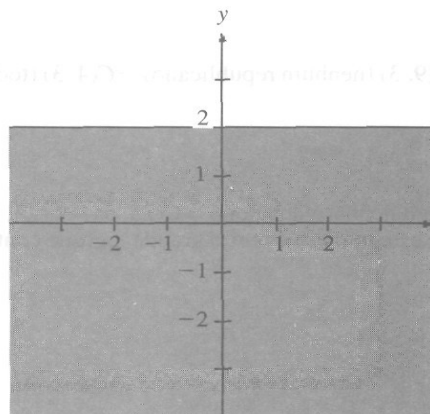
Exercícios 3.5

1. $(a + b)^5 = C(5, 0)a^5 + C(5, 1)a^4b + C(5, 2)a^3b^2 + C(5, 3)a^2b^3 + C(5, 4)ab^4 + C(5, 5)b^5$
 $= a^5 + 5a^4b + 10a^3b^2 + 10a^2b^3 + 5ab^4 + b^5$
3. $(a + 2)^5 = C(5, 0)a^5 + C(5, 1)a^4 \cdot 2 + C(5, 2)a^3 \cdot 2^2 + C(5, 3)a^2 \cdot 2^3 + C(5, 4)a \cdot 2^4 + C(5, 5)2^5$
 $= a^5 + 10a^4 + 40a^3 + 80a^2 + 80a + 32$
5. $(2x + 3y)^3 = C(3, 0)(2x)^3 + C(3, 1)(2x)^2(3y) + C(3, 2)(2x)(3y)^2 + C(3, 3)(3y)^3$
 $= 8x^3 + 36x^2y + 54xy^2 + 27y^3$
11. $C(9, 5)(2x)^4(-3)^5 = -489.888x^4$
13. $C(8, 8)(-3y)^8 = 6561y^8$
15. $C(5, 2)(4x)^3(-2y)^2 = 2560x^3y^2$
19. $C(8, 1)(2x - y)^7 5^1 \rightarrow C(8, 1) \cdot 5[C(7, 4)(2x)^3(-y)^4]$
 $= C(8, 1)C(7, 4)(2)^3 5x^3y^4 = 11.200x^3y^4$
24. $C(n, 0) + C(n, 1)2 + C(n, 2)2^2 + \dots + C(n, n)2^n = (1 + 2)^n = 3^n$

Capítulo 4**Exercícios 4.1**

1. a. (1, 3), (3, 3) b. (4, 2), (5, 3)
 c. (5, 0), (2, 2) d. (1, 1), (3, 9)

3. a.

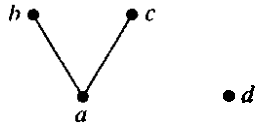


5. a. Vários-para-vários b. Vários-para-um
 c. Um-para-um d. Um-para-vários
7. a. (2, 6), (3, 17), (0, 0)
 b. (2, 12)
 c. Nenhum
 d. (1, 1), (4, 8)
9. a. Reflexiva, transitiva
 b. Reflexiva, simétrica, transitiva
 c. Simétrica

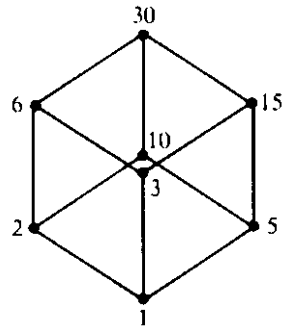
14. a. $x \rho^* y \leftrightarrow x$ é algum número de anos mais velho que y
 b. $x \rho^* y \leftrightarrow x$ é ancestral macho de y
 c. $x \rho^* y \leftrightarrow$ pode-se ir da cidade x à cidade y em um número de dias

15. a. $\rho = \{(1, 1)\}$

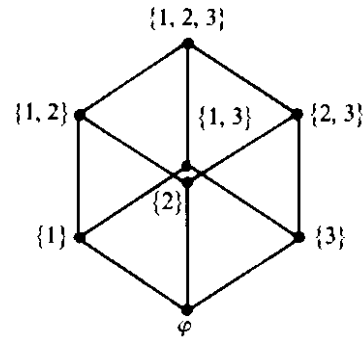
18. b.



21. a.



b.

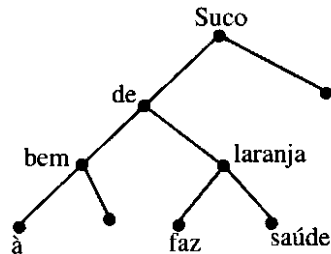


Os dois grafos são idênticos em suas estruturas.

22. a. $\rho = \{(1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (1, 3), (3, 5), (1, 5), (2, 4), (4, 5), (2, 5)\}$

26. a. "Quando"; não; apenas (já, mais)

b.



Elementos maximais; "a", "faz", "saúde"

28. a. $[a] = [a, c] = [c]$
 b. $[3] = \{1, 2, 3\}$; $[4] = \{4, 5\}$
 c. $[1] = \{\dots, -5, -3, -1, 1, 3, 5, \dots\}$
 d. $[-3] = \{\dots, -13, -8, -3, 2, 7, 12, \dots\}$

31. Reflexiva: $(x, y) \rho (x, y)$ porque $y = y$.

Simétrica: Se $(x, y) \rho (z, w)$ então $y = w$ logo $w = y$ e $(z, w) \rho (x, y)$

Transitiva: Se $(x, y) \rho (z, w)$ e $(z, w) \rho (s, t)$ então $y = w$ e $w = t$ logo $y = t$ e $(x, y) \rho (s, t)$.

As classes de equivalência são conjuntos de pares ordenados com as segundas componentes iguais.

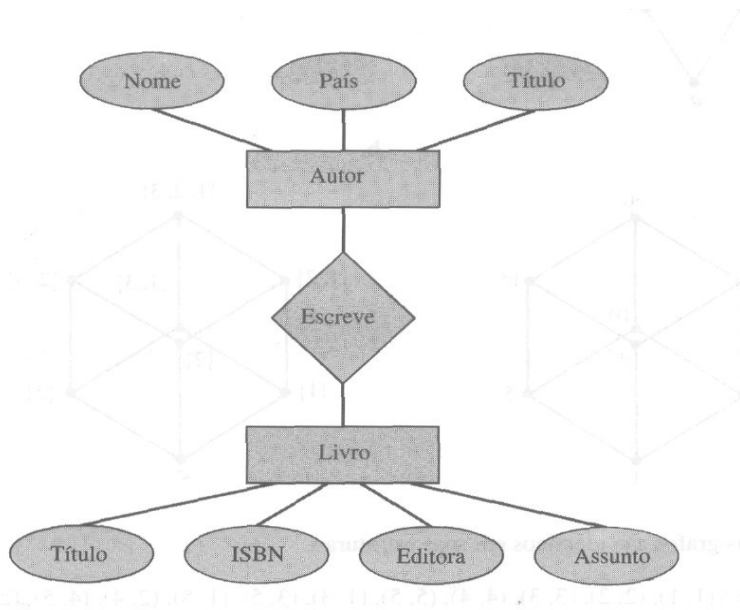
34. Claramente, $P \leftrightarrow P$ é uma tautologia. $P \leftrightarrow Q$ é uma tautologia, então P e Q têm que ter os mesmos valores-verdade em todas as linhas da tabela-verdade, portanto $P \leftrightarrow Q$ é uma tautologia. Se $P \leftrightarrow Q$ e $Q \leftrightarrow R$ são tautologias, então P , Q e R têm sempre que ter os mesmos valores-verdade, e $P \leftrightarrow R$ é uma tautologia. As classes de equivalência são conjuntos que consistem em wffs cujas tabelas-verdade apresentam os mesmos valores-verdade.

37. a. 3 b. 7

40. a. $S(3, 2) = 5(2, 1) + 25(2, 2) = 1 + 2 \cdot 1 = 3$
b. $S(4, 2) = 5(3, 1) + 25(3, 2) = 1 + 2 \cdot 3 = 7$
42. $S(4, 3) = 6$

Exercícios 4.2

1.



A relação "escreve" é vários-para-vários; isto é, um autor pode escrever vários livros e um livro pode ter mais de um autor.

4.

Tabela 1		
Nome	País	Título
Bert Kovalsco	EUA	Basquete Moderno
Jane East	EUA	Jardinagem de Primavera

8.

Tabela 5	
Nome	Título
Dorothy King	Jardinagem de Primavera
Jon Nkoma	Pássaros da África
Won Lau	Pinturas em Porcelana
Bert Kovalsco	Basquete Moderno
Jimmy Chan	Pinturas em Porcelana
Dorothy King	Anuário de Outono
Jane East	Jardinagem de Primavera

12.

Tabela 9				
Título	ISBN	Editora	Assunto	Nome
Jardinagem de Primavera	816-35421-8	Harding	Natureza	Dorothy King
Pinturas em Porcelana	364-87547-8	Bellman	Arte	Jimmy Chan
Pinturas em Porcelana	364-87547-8	Bellman	Arte	Won Lau
Pássaros da África	115-67813-3	Loraine	Natureza	Jon Nkoma
Jardinagem de Primavera	816-89335-8	Swift-Key	Natureza	Jane East
Basquete Moderno	778-53705-7	Harding	Art	Bert Kovalsco
Anuário de Outono	414-88506-9	Harding	Natureza	Dorothy King

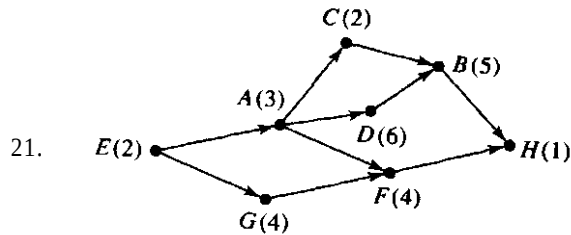
15. a. **project(select Autor where País = "EUA") over Título giving Tabela 11**

b. **SELECT Título FROM Autor**

c. Range of x is Autor $\{x. \text{Título} \mid x. \text{País} = \text{"EUA"}\}$

d. Basquete Moderno

Jardinagem de Primavera



25. Por exemplo: E, A, C, D, G, F, B, H

Exercícios 4.3

1. a. Domínio = $\{4, 5, 6, 7, 8\}$, contradomínio = $\{8, 9, 10, 11\}$, faixa = $\{8, 9, 10\}$

b. 8, 10

c. 6, 7

d. Não, não

4. a. $f(A) \{3, 9, 15\}$

b. $f(A) = \{x \mid x \in \mathbb{Z} \text{ e } (\exists y) (y \in \mathbb{Z} \text{ e } x = 6y)\}$

7. a. Função

b. Não é função; não está definida em $x = 0$

c. Função sobrejetiva

d. Bijeção $f^{-1}: \{p, q, r\} \rightarrow \{1, 2, 3\}$ onde $f^{-1} = \{(q, 1), (r, 2), (p, 3)\}$

e. Função; injetiva

f. Bijeção $h^{-1}: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ onde $h^{-1}(x, y) = (y - 1, x - 1)$

11. Se $2^k < n < 2^{k+1}$, então $\log(2^k) < \log n < (2^{k+1})$ ou $k < \log n < k + 1$ e $\lfloor \log n \rfloor = k, \lceil \log n \rceil = k + 1$.

15. a. 3^4 b. 36

18. a. n^n b. $n!$ c. $n!$ d. $n!$

$$\begin{aligned} \text{e. } n! \left[1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + (-1)^n \frac{1}{n!} \right] &= n! \left[\frac{1}{2!} - \frac{1}{3!} + \dots + (-1)^n \frac{1}{n!} \right] \\ &< n! \left(\frac{1}{2!} \right) = n! \cdot \frac{1}{2} < n! \end{aligned}$$

f. Número de arranjos $< n! < n^n$. O número total de funções sem restrições é o máximo. Apenas algumas dessas funções são injetivas e sobrejetivas, mas isto é a definição de uma permutação. Nem todas as permutações são arranjos, portanto o número de arranjos é ainda menor.

22. a. $\{m, n, o, p\}$

b. $\{n, o, p\}; \{o\}$

26. a. $(g \circ f)(5) = g(f(5)) = g(6) = 18$

b. $(f \circ g)(5) = f(g(5)) = f(15) = 16$

c. $(g \circ f)(x) = g(f(x)) = g(x + 1) = 3(x + 1) = 3x + 3$

d. $(f \circ g)(x) = f(g(x)) = f(3x) = 3x + 1$

e. $(f \circ f)(x) = f(f(x)) = f(x + 1) = (x + 1) + 1 = x + 2$

f. $(g \circ g)(x) = g(g(x)) = g(3x) = 3(3x) = 9x$

29. a. (1, 3, 5, 2)

b. (1, 4, 3, 2, 5)

32. a. (1, 2, 5, 3, 4)
 b. $(1, 7, 8) \circ (2, 4, 6)$
 c. $(1, 5, 2, 4) \circ (3, 6)$
 d. $(2, 3) \circ (4, 8) \circ (5, 7)$

36. A resposta é o número de arranjos de sete itens, que é 1854.

38. a. $f^{-1}(x) = x/2$
 b. $f^{-1}(x) = \sqrt[3]{x}$
 c. $f^{-1}(x) = 3x - 4$

42. Por exemplo, $n_0 = c_2 = 1, c_1 = 1/34$

$$46. \lim_{x \rightarrow \infty} \frac{x}{17x + 1} = \lim_{x \rightarrow \infty} \frac{x}{17x + \frac{1}{x}} = \frac{1}{17}$$

$$48. \lim_{x \rightarrow \infty} \frac{x}{x^2} = \lim_{x \rightarrow \infty} \frac{1}{2x} = 0$$

Exercícios 4.4

1. 2, -4

3. $x=1, y=3, z=-2, w = 4$

$$5. \begin{bmatrix} 6 & -5 \\ 0 & 3 \\ 5 & 3 \end{bmatrix} \qquad 9. \begin{bmatrix} 14 & -17 \\ 2 & 9 \\ 9 & 1 \end{bmatrix}$$

$$13. \begin{bmatrix} 21 & -23 \\ 33 & -44 \\ 11 & 1 \end{bmatrix} \qquad 17. \begin{bmatrix} 28 & 4 \\ 6 & 25 \end{bmatrix}$$

$$21. A(B + C) = AB + AC = \begin{bmatrix} 26 & -9 \\ 40 & -23 \end{bmatrix}$$

25. a. Diretamente da definição de multiplicação de matrizes
 b. Suponha que existe uma matriz B que é a inversa de A. Então

$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

e

$$\begin{aligned} b_{11} + 2b_{21} &= 0 & b_{12} + 2b_{22} &= 0 \\ 2b_{11} + 4b_{21} &= 0 & 2b_{12} + 4b_{22} &= 1 \end{aligned}$$

que é um sistema de equações inconsistente sem solução.

$$27. \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix} \text{ e } \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

$$30. \mathbf{A} \wedge \mathbf{B} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \qquad \mathbf{A} \vee \mathbf{B} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$A \times B = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad B \times A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

33. A entrada i, j de A^2 é $\sum_{k=1}^n a_{ik} a_{kj}$.

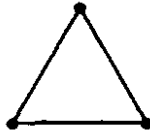
A entrada j, i de A^2 é $\sum_{k=1}^n a_{jk} a_{ki}$.

Mas esses valores são iguais, porque $a_{ik} = a_{kj}$ e $a_{kj} = a_{jk}$ (A é simétrica)

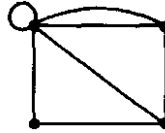
Capítulo 5

Exercícios 5.1

2. a.



b. Por exemplo:



c.



4. a. 4, 5, 6

b. Comprimento 2

c. Por exemplo (rotulando os vértices): 1, 2, 1, 2, 2, 1, 4, 5, 6

7. b. Grafo (b) porque não tem vértice de grau 0.

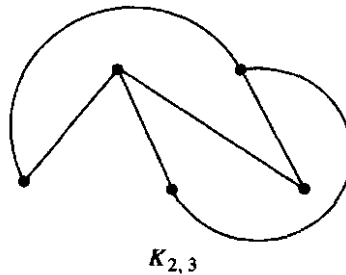
9. $f_1: 1 \rightarrow a$ $f_2: a_1 \rightarrow e_2$
 $2 \rightarrow b$ $a_2 \rightarrow e_7$
 $3 \rightarrow c$ $a_3 \rightarrow e_6$
 $4 \rightarrow d$ $a_4 \rightarrow e_1$
 $a_5 \rightarrow e_3$
 $a_6 \rightarrow e_4$
 $a_7 \rightarrow e_5$

13. Não isomorfo; o grafo em (b) tem um vértice de grau 5, o grafo em (a) não tem.

16.

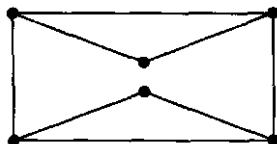


21.

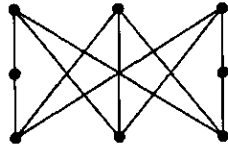


23. 5 (pela fórmula de Euler)

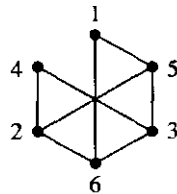
27. Planar



29. Não-planar; o subgrafo abaixo pode ser obtido a partir de $K_{3,3}$ através de subdivisões elementares.

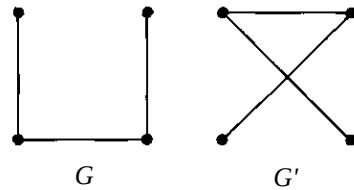


31.



35. a. Se G é desconexo, então G consiste em dois ou mais subgrafos conexos que não tenham caminho entre eles. Seja x e y vértices distintos. Se x e y estão em subgrafos diferentes, então não há nenhuma aresta $x-y$ em G ; portanto, existe uma aresta $x-y$ em G' e existe um caminho de x a y em G' . Se x e y estão no mesmo subgrafo, então pegue um vértice z em um subgrafo diferente. Existe uma aresta $x-z$ e uma aresta $z-y$ em G' ; portanto existe um caminho de x a y em G' .

b.



39. 3

43. a. Um grafo com arestas paralelas deve conter

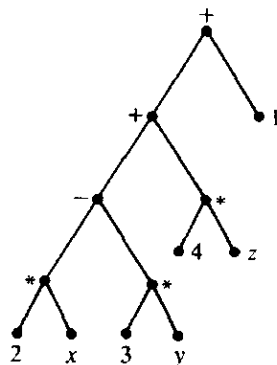


Crie, temporariamente, pequenos países nos vértices das arestas paralelas, obtendo



que é um grafo simples. Qualquer coloração que satisfaça este grafo satisfaz o grafo original (fazendo as regiões se transformarem em pontos).

46.



50. Seja um grafo simples e conexo com n vértices e m arestas. O enunciado original é $m \geq n - 1$, que diz que $n \leq m + 1$, ou que o número de vértices é pelo menos $m + 1$.

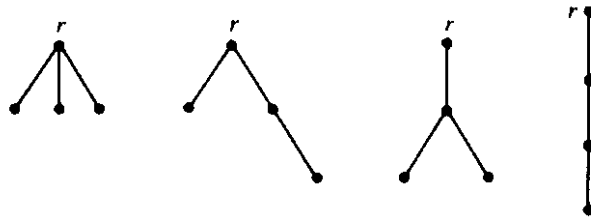
Para cada caso, tome $m = 0$. O único grafo simples e conexo com 0 aresta é um único vértice, onde o número de vértices, 1, é $\leq 0 + 1$. Agora assuma que qualquer grafo simples e conexo com r arestas, $0 \leq r \leq k$ tem pelo menos $r + 1$ vértices. Considere um grafo simples e conexo com $k + 1$ arestas e remova uma aresta. Se o grafo resultante for conexo, ele tem k arestas e, pela hipótese de indução, o número n de vértices satisfaz $n \leq k + 1$. Portanto, no grafo original (com os mesmos vértices), $n \leq k + 1 < (k + 1) + 1$. Se, por outro lado, o grafo resultante não for conexo, ele consiste em dois subgrafos conexos com r_1 e r_2 arestas, $r_1 \leq k$ e $r_2 \leq k$, $r_1 + r_2 = k$. Pela hipótese de indução, um subgrupo tem no máximo $r_1 + 1$ arestas e o outro tem no máximo $r_2 + 1$. O grafo original, portanto, tem no máximo $r_1 + 1 + r_2 + 1 = (k + 1) + 1$ vértices.

53. Considere um grafo simples que seja uma árvore não-enraizada. Uma árvore é um grafo cíclico e conexo, portanto, para quaisquer dois vértices x e y , existe um caminho de x a y . Se o caminho não for único, então os dois caminhos se separariam em algum vértice n_1 e se reencontrariam em algum vértice n_2 , o que formaria um ciclo de n_1 a n_2 e de volta a n_1 , que é uma contradição.

Agora considere um grafo simples que tenha um único caminho entre quaisquer dois vértices. O grafo é claramente conexo. Além disso, não existem ciclos porque a presença de ciclos produziria um caminho não único entre dois vértices do ciclo. O grafo é, portanto, acíclico e conexo e, portanto, é uma árvore não-enraizada.

56. A prova é realizada por indução em d . Para $d = 0$, o único vértice é a raiz, e $2^0 = 1$. Admita que existem pelo menos 2^d vértices na profundidade d e considere a profundidade $d + 1$. Existem no máximo dois filhos para cada vértice da profundidade d , de forma que o número máximo de vértices na profundidade $d + 1$ é $2 \cdot 2^d = 2^{d+1}$.

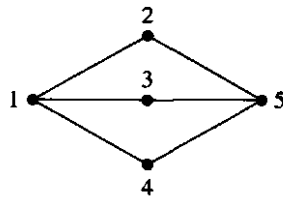
60.



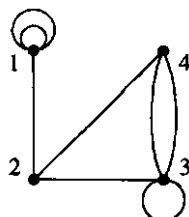
Exercícios 5.2

$$1. \begin{bmatrix} 1 & 1 & 0 & 0 & 2 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 0 & 0 \end{bmatrix} \quad 5. \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

9.

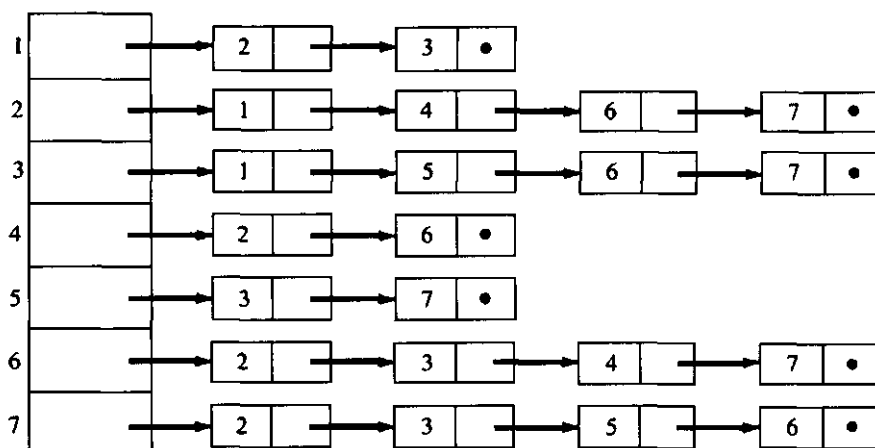


11.

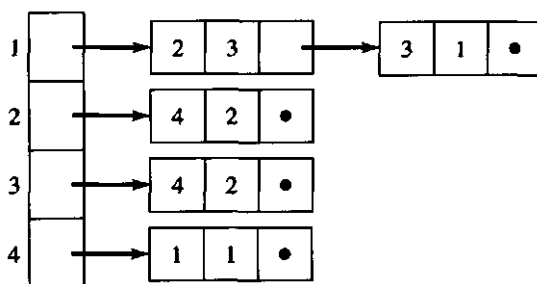


13. O grafo consiste em n vértices desconexos com um laço em cada vértice.

19.



24.



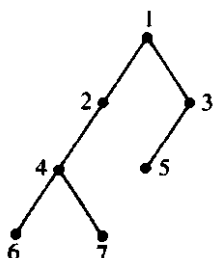
25. *Vértice* *Ponteiro*

1		5
2		7
3		11
4		0
5	2	6
6	3	0
7	1	8
8	2	9
9	3	10
10	4	0
11	4	0

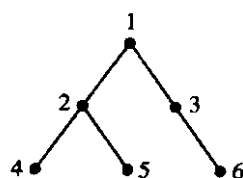
29. *Filho à esquerda* *Filho à direita*

1	2	3
2	0	4
3	5	6
4	7	0
5	0	0
6	0	0
7	0	0

31.



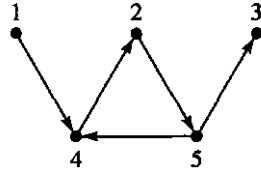
34.



Exercícios 5.3

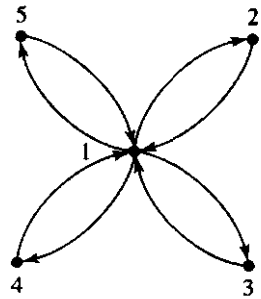
$$1. \mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \rho = \{(1,1), (2,1), (2,3), (3,2)\}$$

3.

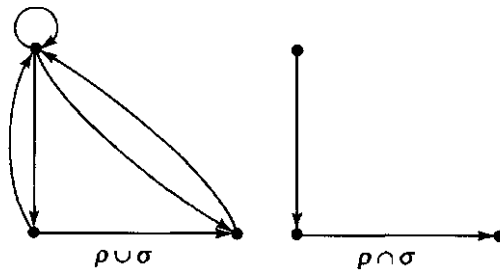


$$\rho = \{(1,4), (2,5), (4,2), (5,3), (5,4)\}$$

8. O grafo é uma "estrela" com o vértice 1 no centro, isto é, 1 é adjacente a todos os vértices e todos os vértices são adjacentes a 1, mas os outros vértices não são adjacentes a mais nenhum vértice.



12.



$$15. \mathbf{R} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$19. \mathbf{R} = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$23. \mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

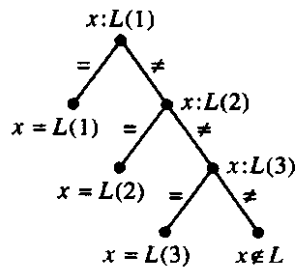
$$27. \mathbf{R} = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

29. $A^2[i, j] = \sum_{k=1}^n a_{ik}a_{kj}$

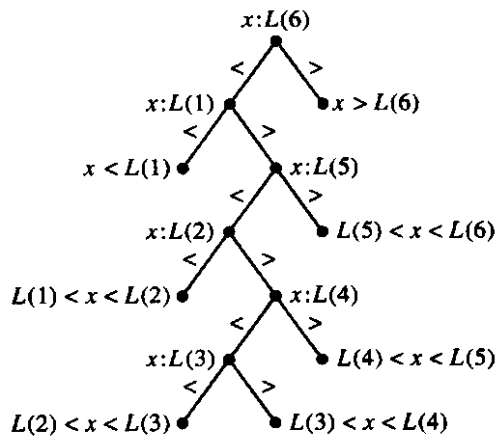
Se uma parcela como $a_{i2}a_{2j}$ da soma é 0, então ou $a_{i2} = 0$ ou $a_{2j} = 0$ (ou ambos), e não existe caminho de comprimento 1 de n_i a n_2 ou não existe caminho de comprimento 1 de n_2 a n_j (ou não existe nenhum dos dois). Portanto, não há caminhos de comprimento 2 de n_i a n_j passando por n_2 . Se $a_{i2}a_{2j} \neq 0$, então $a_{i2} = p$ e $a_{2j} = q$, onde p e q são inteiros positivos. Então existem p caminhos de comprimento 1 de n_i a n_2 e q caminhos de comprimento 1 de n_2 a n_j . Pelo Princípio da Multiplicação, existem pq caminhos possíveis de comprimento 2 de n_i a n_j passando por n_2 . Pelo Princípio da Adição, a soma de todas as parcelas deste tipo fornece o número total de caminhos de comprimento 2 de n_i a n_j .

Exercícios 5.4

1.

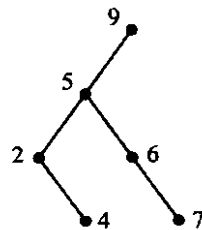


5.



Profundidade = 6; algoritmo não é ótimo

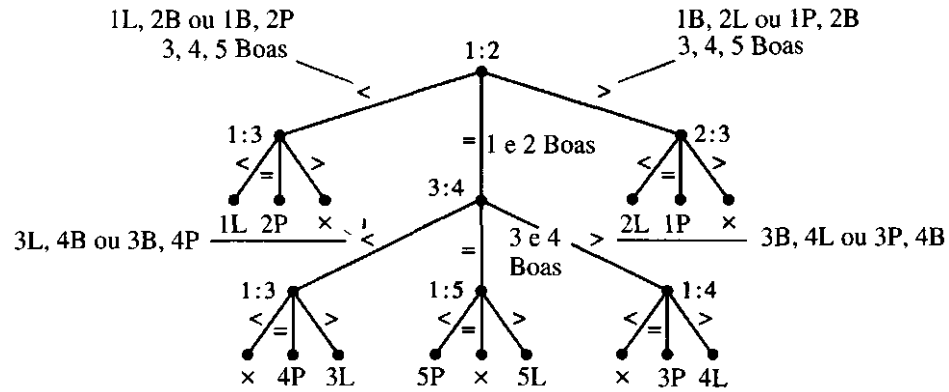
7.



a. Profundidade = 3 b. 2,83

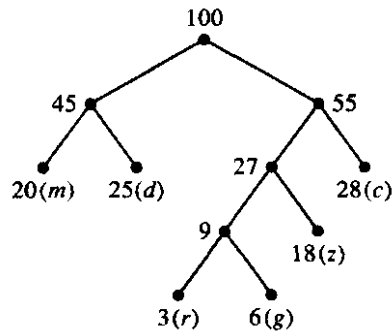
11. a. $\lceil \log 4! \rceil = \lceil \log 24 \rceil = 5$
 b. $\lceil \log 8! \rceil = \lceil \log 40.320 \rceil = 16$
 c. $\lceil \log 16! \rceil = \lceil \log 2,9 \times 10^{13} \rceil = 45$

14. a. 10 (qualquer uma das cinco moedas pode ser leve ou pesada)
 b. 3 (profundidade mínima para uma árvore ternária com 10 folhas)
 c.



19. a. ooue b. iaou c. eee

21. a.



- b. m : 00
 g : 1001
 z : 101
 c : 11
 d : 01
 r : 1000

Capítulo 6

Exercícios 6.1

1. Sim 6. Não

9.
$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

total após a linha 2 é 0

11.
$$\begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$i = 8$

13. Não 18. Sim

22. Quaisquer dois vértices precisam pertencer ao ciclo de Hamilton; portanto existe um caminho entre eles, a saber, o trecho do ciclo que está entre eles.

25. a. $n = 2$ ou $n =$ qualquer número ímpar
 b. $n > 2$
28. Comece em qualquer vértice e saia por uma das arestas deste vértice. Toda vez que se entrar em um novo vértice por uma aresta, existe exatamente uma aresta ainda não usada para sair deste vértice; como a aresta não foi usada, isto levará a um novo vértice ou ao vértice inicial. Ao voltar para o vértice inicial, se houver algum vértice que não tenha sido usado, então existe um caminho ainda não percorrido deste vértice para um vértice usado, o que significa que este vértice usado tem grau ≥ 3 , uma contradição.

Exercícios 6.2

1.

$$IN = \{2\}$$

	1	2	3	4	5	6	7	8
d	3	0	2	∞	∞	∞	1	∞
s	2	-	2	2	2	2	2	2

$$p = 7$$

$$IN = \{2, 7\}$$

	1	2	3	4	5	6	7	8
d	3	0	2	∞	∞	6	1	2
s	2	-	2	2	2	7	2	7

$$p = 3$$

$$IN = \{2, 7, 3\}$$

	1	2	3	4	5	6	7	8
d	3	0	2	3	∞	6	1	2
s	2	-	2	3	2	7	2	7

$$p = 8$$

$$IN = \{2, 7, 3, 8\}$$

	1	2	3	4	5	6	7	8
d	3	0	2	3	3	6	1	2
s	2	-	2	3	8	7	2	7

$$p = 5$$

$$IN = \{2, 7, 3, 8, 5\}$$

	1	2	3	4	5	6	7	8
d	3	0	2	3	3	6	1	2
s	2	-	2	3	8	7	2	7

Caminho: 2, 7, 8, 5 Distância: 3

5.

$$IN = \{1\}$$

	1	2	3	4	5	6	7
d	0	2	∞	∞	3	2	∞
s	-	1	1	1	1	1	1

$$p = 2$$

$$IN = \{1, 2\}$$

	1	2	3	4	5	6	7
d	0	2	3	∞	3	2	∞
s	-	1	2	1	1	1	1

$$p = 6$$

$$IN = \{1, 2, 6\}$$

	1	2	3	4	5	6	7
d	0	2	3	∞	3	2	5
s	-	1	2	1	1	1	6

$$p = 3$$

$$IN = \{1, 2, 6, 3\}$$

	1	2	3	4	5	6	7
d	0	2	3	4	3	2	5
s	-	1	2	3	1	1	6

$$p = 5$$

$$IN = \{1, 2, 6, 3, 5\}$$

	1	2	3	4	5	6	7
d	0	2	3	4	3	2	5
s	-	1	2	3	1	1	6

$$p = 4$$

$$IN = \{1, 2, 6, 3, 5, 4\}$$

	1	2	3	4	5	6	7
d	0	2	3	4	3	2	5
s	-	1	2	3	1	1	6

$$p = l$$

$$IN = \{1, 2, 6, 3, 5, 4, 7\}$$

	1	2	3	4	5	6	7
s	0	2	3	4	3	2	5
	-	1	2	3	1	1	6

Caminho: 1, 6, 7 Distância: 5

9.

	1	2	3	4	5	6	7	8
	3	0	2	∞	∞	∞	1	∞
	2	-	2	2	2	2	2	2

	1	2	3	4	5	6	7	8
d	3	0	2	3	11	4	1	2
s	2	-	2	3	1	1	2	7

	1	2	3	4	5	6	7	8
	3	0	2	3	3	4	1	2
	2	-	2	3	8	1	2	7

Não há mais alterações em d ou s

13. Inicial: A, após $k = x$:

	X	1	2	3	y
x	0	1	∞	4	∞
1	1	0	3	1	5
2	∞	3	0	2	2
3	4	1	2	0	3
y	∞	5	2	3	0

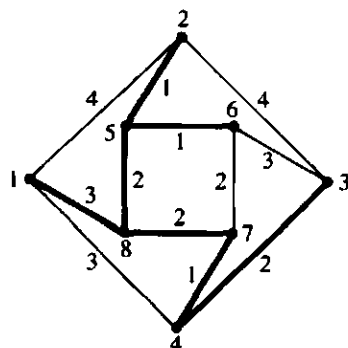
Após $k = 1$ e $k = 2$:

	X	1	2	3	y
	0	1	4	2	6
1	1	0	3	1	5
2	4	3	0	2	2
3	2	1	2	0	3
	6	5	2	3	0

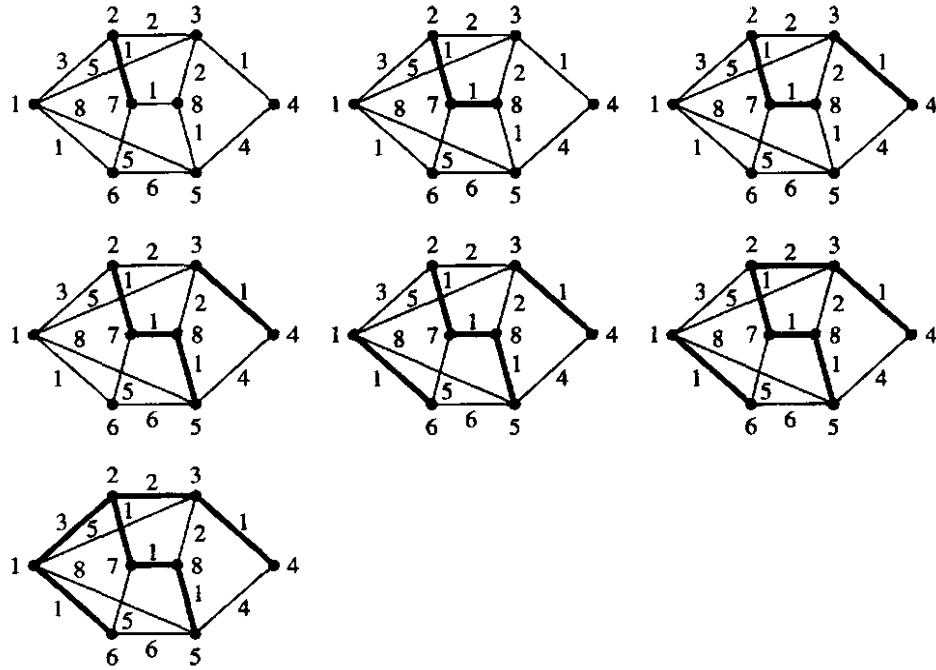
Após $k = 3$ e $k = y$:

	X	1	2	3	y
X	0	1	4	2	5
1	1	0	3	1	4
2	4	3	0	2	2
3	2	1	2	0	3
	5	4	2	3	0

17. $IN = \{1, 8, 5, 6, 2, 7, 4, 3\}$

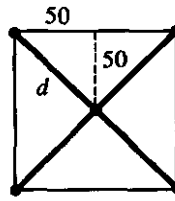
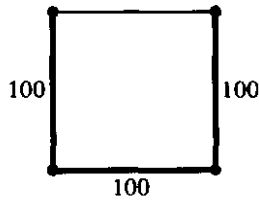


19. O algoritmo de Kruskal desenvolve uma árvore geradora mínima, como mostrado a seguir:



24. a. Peso = 300

b. $d = 50\sqrt{2} = 70,7$
 $4d = 282,8$



Exercícios 6.3

1. a, b, c, e, f, d, h, g, j, i 5. e, b, a, c, f, d, h, g, j, i

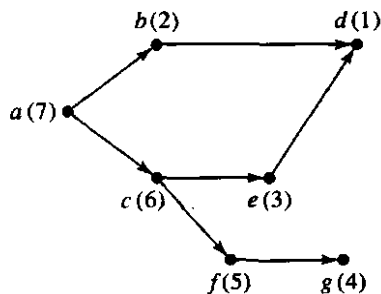
7. a, b, c, f, j, g, d, e, h, k, i 9. f, c, a, b, d, e, h, k, i, g, j

11. a, b, c, d, e, g, f, h, j, i 17. a, b, c, d, e, f, g, h, i, j, k

21. a, b, c, e, g, d, f, h

24. a, b, c, d, e, g, h, f

27. Realize uma busca em profundidade começando no vértice a: a, c, f, g, e, b, d.



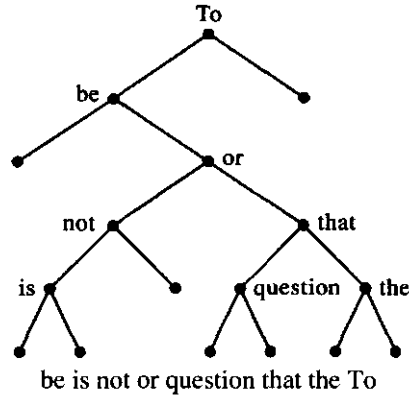
29. Preordem: a, b, d, e, h, f, c, g
 Inordem: d, b, h, e, f, a, g, c
 Posordem: d, h, e, f, b, g, c, a

33. Preordem: a, b, e, f, d, g, h
 Inordem: e, c, f, b, g, d, h, a
 Posordem: e, f, c, g, h, d, b, a

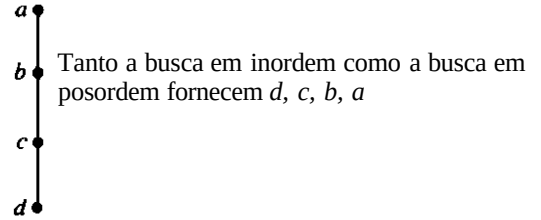
35. Prefixa: $+ / 3 4 - 2 y$
 Posfixa: $3 4 / 2 y - +$

39. Prefixa: $+ * 4 - 7 x z$
 Infixa: $4 * (7 - x) + z$

43.

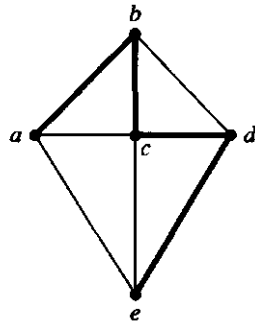


45.



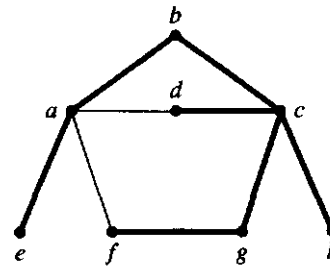
Exercícios 6.4

1.



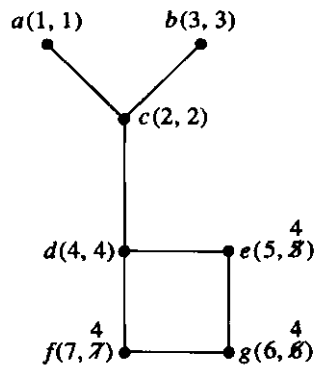
Arestas de retorno: $a-c, a-e, b-d, c-e$

4.



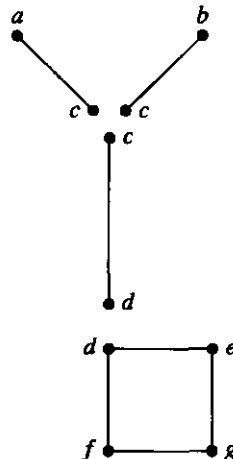
Arestas de retorno: $a-d, a-f$

7.

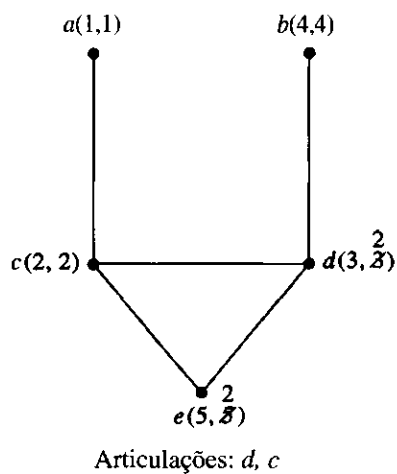


Articulações: c, d

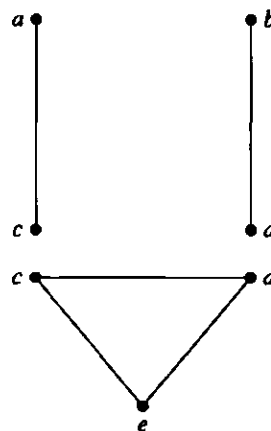
Componentes biconexos



10.



Componentes biconexos



Capítulo 7

Exercícios 7.1

1. +	0	1	a	a'
0	0	1	a	a'
1	1	1	1	1
a	a	1	a	1
a'	a'	1	1	a'

	0	1	a	a'
0	0	0	0	0
1	0	1	a	a'
a	0	a	a	0
a'	0	a'	0	a'

$$4. \quad a. \quad x' + x = x + x' \quad (1a)$$

$$= 1 \quad (5a)$$

e

$$x' \cdot x = x \cdot x' \quad (1b)$$

$$= 0 \quad (5b)$$

Portanto, $x = (x')'$ pelo teorema da unicidade dos complementos.

$$b. \quad x + (x \cdot y) = x \cdot 1 + x \cdot y \quad (4b)$$

$$= x(1+y) \quad (3b)$$

$$= x(y+1) \quad (1a)$$

$$= x \cdot 1 \quad (\text{Prática 3})$$

$$= x \quad (4b)$$

 $x \cdot (x + y) = x$ segue pela dualidade.

$$g. \quad x + (x' \cdot y + x \cdot y)' = x + (y \cdot x' + y \cdot x) \quad (1b)$$

$$= x + [y \cdot (x' + x)] \quad (3b)$$

$$= x + [y \cdot (x + x')] \quad (1a)$$

$$= x + (y \cdot 1) \quad (5a)$$

$$= x + y \quad (4b)$$

$$5. \quad a. \quad x \oplus y = x \cdot y' + y \cdot x' \quad (\text{definição de } \oplus)$$

$$= y \cdot x' + x \cdot y' \quad (1a)$$

$$= y \oplus x \quad (\text{definição de } \oplus)$$

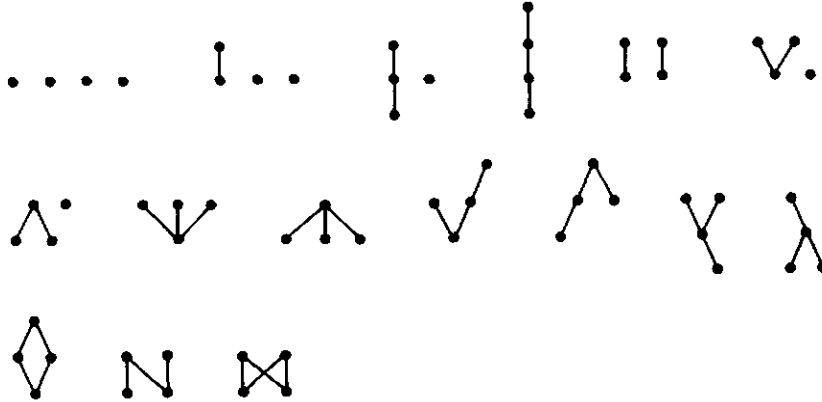
9. a.



b.



c.


 12. a. $f: \mathbb{R} \rightarrow \mathbb{R}^+$
 f é sobrejetiva. Dado $y \in \mathbb{R}^+$, seja $x = \log y$; então $x \in \mathbb{R}$ e $f(x) = 2^x = 2^{\log y} = y$.

 f é injetiva. Se $f(x) = f(w)$, então $2^x = 2^w$ e (tomando o log de ambos os lados) $x = w$.

 b. Para $x, y \in \mathbb{R}$, $g(x + y) = g(x) \cdot g(y)$

 c. f é uma bijeção de $\mathbb{R}$ em $\mathbb{R}^+$ e para $x, y \in \mathbb{R}$, $f(x, y) = 2^{x+y} = 2^x \cdot 2^y = f(x) \cdot f(y)$.

 d. f^{-1} é a função $f^{-1}(y) = \log y$.

 e. f^{-1} é uma bijeção de $\mathbb{R}^+$ em $\mathbb{R}$, e para qualquer $x, y \in \mathbb{R}^+$,
 $f^{-1}(x \cdot y) = \log(x \cdot y) = \log x + \log y = f^{-1}(x) + f^{-1}(y)$.

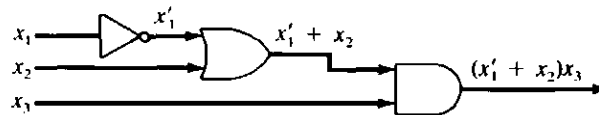
 15. a. Para qualquer $y \in b$, $y = f(x)$ para algum $x \in B$. Então $y \in f(0) = f(x) \in f(0) = f(x + 0) = f(x) = y$, e $f(0) = \emptyset$ porque o elemento nulo de qualquer álgebra é único (veja o Exercício 8).

 b. $f(1) = f(0') = [f(0)]'' = \emptyset'' = 1$

 16. a. (i) Se $x \leq y$, então $x \leq y$ e $x \leq x$, portanto x é um limite inferior de $\{x, y\}$. Se $w^* \leq x$ e $w^* \leq y$, então $w^* \leq x$, logo x é um menor limite superior e $x = x \cdot y$. Se $x = x \cdot y$, então x é um maior limite inferior de $\{x, y\}$, logo $x \leq y$. A demonstração de (ii) é semelhante.

Exercícios 7.2

1. a.

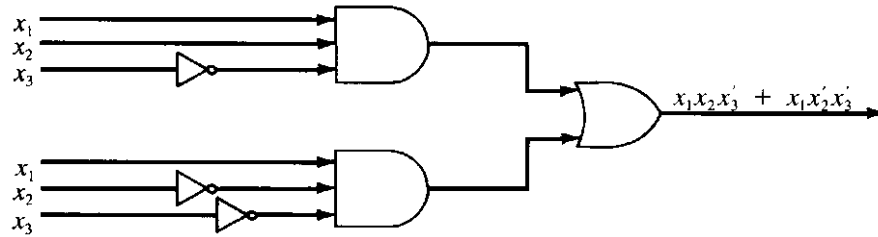

 3. $(x_1 x_2)'(x_2 + x_3)$

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	0
1	1	0	0
1	0	1	0
1	0	0	1
0	1	1	1
0	1	0	1
0	0	1	0
0	0	0	1

 7. $x_1' x_2'$

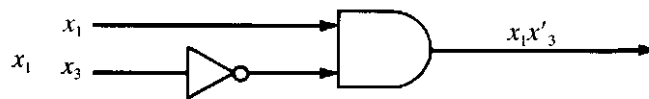
11. a. $x_1x_2x_3' + x_1x_2'x_3'$

b.



c.

$$\begin{aligned} x_1x_2x_3' + x_1x_2'x_3' &= x_1x_3'(x_2 + x_2') \\ &= x_1x_3'(1) \\ &= x_1x_3' \end{aligned}$$



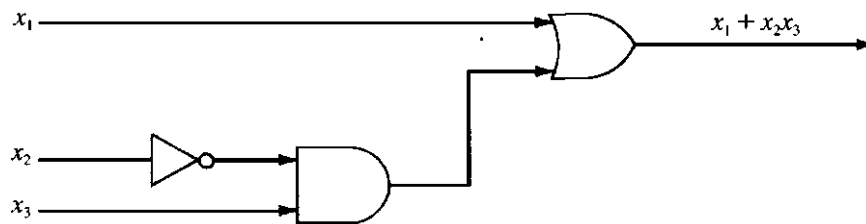
14. a. $(x_1' + x_2')(x_1' + x_2)(x_1 + x_2')$

b. $(x_1' + x_2)(x_1 + x_2)$

c. $(x_1' + x_2' + x_3')(x_1' + x_2 + x_3)(x_1 + x_2' + x_3)(x_1 + x_2 + x_3')$

d. $(x_1' + x_2' + x_3')(x_1' + x_2 + x_3)(x_1 + x_2' + x_3)(x_1 + x_2 + x_3')(x_1 + x_2 + x_3)$

19. A rede é representada por $[x_1'(x_2'x_3)']'$ e $[x_1'(x_2'x_3)']' = x_1 + x_2'x_3$.

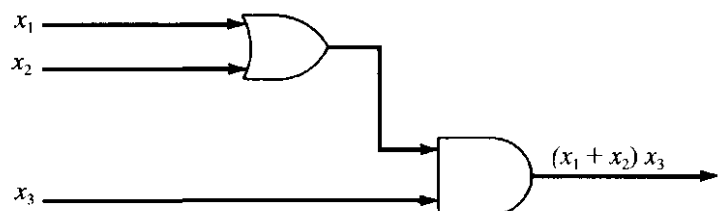


21. A função-verdade de $|$ é a da porta NE, e a função-verdade de $\downarrow$ é a da porta NOU. Na Seção 1.1, vimos que toda sentença composta é a equivalente a uma sentença usando apenas $|$ ou a uma sentença usando apenas $\downarrow$; portanto qualquer função-verdade pode ser concebida usando apenas portas NE ou apenas portas NOU.

23. x_1 = ponto-morto
 x_2 = estacionamento
 x_3 = cinto de segurança

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	—
1	1	0	—
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	0
0	0	0	0

$(x_1 + x_2)x_3$



Exercício 7.3

1.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3			1	1
x'_3	1	1		1

$$x'_1x_3 + x_1x'_3 + x'_1x_2 \quad \text{ou} \quad x'_1x_3 + x_1x'_3 + x_2x'_3$$

3.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3	1	1	1	1
x'_3	1			1

$x_3 + x_2$

6.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3	1			1
x'_3	1			

$x_1x_2 + x_2x_3$

10.

	x_1x_2	$x_1x'_2$	$x'_1x'_2$	x'_1x_2
x_3x_4			1	
$x_3x'_4$	1	—		1
$x'_3x'_4$	—			1
x'_3x_4			1	

$x_2x'_4 + x'_1x'_2x_4$

11.

Número de ls	x_1	x_2	x_3	
Três	1	1	1	1,2,3
Dois	1	0	1	1,4
	0	1	1	2,5,6
	1	1	0	3,7
Um	0	0	1	4,5
	0	1	0	6,7

Número de ls	x_1	x_2	x_3
Um	—	—	1
	—	1	—

Número de ls	x_1	x_2	x_3	
Dois	1	—	1	
	—	1	1	1,2
	1	1	—	2
Um	—	0	1	1
	0	—	1	1
	0	1	—	2
	—	1	0	2

	111	101	011	110	001	010
— — 1	✓	✓	✓		✓	
— 1 —	✓		✓	✓		✓

— 1 — e — — 1 — são essenciais. A forma mínima é $x_3 + x_2$.

15.a

Número de 1s	x_1	x_2	x_3	x_4	
Três	0	1	1	1	¹
Dois	1	0	1	0	
	0	0	1	1	^{1,2}
Um	0	1	0	0	³
	0	0	0	1	^{2,4}
Nenhum	0	0	0	0	^{3,4}

Número de 1s	x_2	x_3	x_4
Dois	0	—	1
Um	0	0	—
Nenhum	0	—	0
	0	0	0

	0111	1010	0011	0100	0001	0000
1010		✓				
0—11	✓		✓			
00—1			✓		✓	
0—00				✓		✓
000—					✓	✓

1010, 0—11, 0—00 são essenciais. Tanto 00—1 quanto 000— podem ser usados como quarto termo. A forma de soma de produtos mínima é

$$x_1x_2x_3x_4' + x_1'x_3x_4 + x_1'x_3'x_4' + x_1'x_2'x_4$$

ou

$$x_1x_2'x_3x_4' + x_1'x_3x_4 + x_1'x_3'x_4' + x_1'x_2'x_3'$$

Capítulo 8

Exercícios 8.1

1. a. Associativa b. Comutativa

2. a. Não comutativa; $a \cdot b \neq b \cdot a$ Não associativa; $a \cdot (b \cdot d) \neq (a \cdot b) \cdot d$

b.	.	P	q	r	s	
	P	P	q	r	s	
	q	q	r	s	p	comutativa
	r	r	s	P	q	
	s	s	P	q	r	

5. a. Semigrupo b. Não é um semigrupo; não associativa

c. Não é um semigrupo; S não é fechada sob. d. Monóide; $i = 1 + 0\sqrt{2}$ e. Grupo; $i = 1 + 0\sqrt{2}$ f. Grupo; $i = 1$ g. Monóide; $i = 1$ 6. a. f_0 = função identidade

$$f_1(1) = 2 \quad f_1(2) = 1$$

$$f_2(1) = 1 \quad f_2(2) = 1$$

$$f_3(1) = 2 \quad f_3(2) = 2$$

$\circ$	f_0	f_1	f_2	f_3
f_0	f_0	f_1	f_2	f_3
f_1	f_1	f_0	f_3	f_2
f_2	f_2	f_2	f_2	f_2
f_3	f_3	f_3	f_3	f_3

b. Os elementos são f_0 e f_1 .

$\circ$	f_0	f_1
f_0	f_0	f_1
f_1	f_1	f_0

7. °	R_1	R_2	R_3	F_1	F_2	F_3
R_1	R_2	R_3	R_1	F_3	F_1	F_2
R_2	R_3	R_1	R_2	F_2	F_3	F_1
R_3	R_1	R_2	R_3	F_1	F_2	F_3
F_1	F_2	F_3	F_1	R_3	R_1	R_2
F_2	F_3	F_1	F_2	R_2	R_3	R_1
F_3	F_1	F_2	F_3	R_1	R_2	R_3

O elemento neutro é R_3 ; o inverso de F_1 , é F_4 ; o inverso de R_2 é R_1 .

9. a. Não, não é a mesma operação.
 b. Não; o polinômio nulo (elemento neutro) não pertence a P .
 c. Não; nem todo elemento de $\mathbb{Z}^*$ tem um inverso em $\mathbb{Z}^*$.

10. $\{0\}, +_{12}, [\mathbb{Z}_{12}, +_{12}], \{0, 2, 4, 6, 8, 10\}, +_{12}, \{0, 4, 8\}, +_{12},$
 $\{0, 3, 6, 9\}, +_{12}, \{0, 6\}, +_{12}$

12. $4!/2 = 24/2 = 12$ elementos

$$\begin{array}{ll} \alpha_1 = i & \alpha_2 = (1, 2) \circ (3, 4) \\ \alpha_3 = (1, 3) \circ (2, 4) & \alpha_4 = (1, 4) \circ (2, 3) \\ \alpha_5 = (1, 3) \circ (1, 2) & \alpha_6 = (1, 2) \circ (1, 3) \\ \alpha_7 = (1, 3) \circ (1, 4) & \alpha_8 = (1, 4) \circ (1, 2) \\ \alpha_9 = (1, 4) \circ (1, 3) & \alpha_{10} = (1, 2) \circ (1, 4) \\ \alpha_{11} = (2, 4) \circ (2, 3) & \alpha_{12} = (2, 3) \circ (2, 4) \end{array}$$

13. a. Não; $f(x + y) = 2, f(x) + f(y) = 2 + 2 = 4$
 b. Não; $f(x + y) = |x + y|, f(x) + f(y) = |x| + |y|$
 c. Sim; $f(x \cdot y) = |x \cdot y|, f(x) \cdot f(y) = |x| \cdot |y|$.

16. Fecho:

$$\begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & w \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & w+z \\ 0 & 1 \end{bmatrix} \in M_2^0(\mathbb{Z})$$

A multiplicação de matrizes é associativa.

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \in M_2^0(\mathbb{Z})$$

O inverso de $\begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix}$ é $\begin{bmatrix} 1 & -z \\ 0 & 1 \end{bmatrix}$, que pertence a $M_2^0(\mathbb{Z})$.

- b. f é uma bijeção e

$$\begin{aligned} f\left(\begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix}\right) \cdot \begin{bmatrix} 1 & w \\ 0 & 1 \end{bmatrix} &= f\left(\begin{bmatrix} 1 & w+z \\ 0 & 1 \end{bmatrix}\right) = w+z = z+w \\ &= f\left(\begin{bmatrix} 1 & z \\ 0 & 1 \end{bmatrix}\right) + f\left(\begin{bmatrix} 1 & w \\ 0 & 1 \end{bmatrix}\right) \end{aligned}$$

$$\begin{aligned} \text{c. } f\left(\begin{bmatrix} 1 & 7 \\ 0 & 1 \end{bmatrix}\right) &= 7 \text{ e } f\left(\begin{bmatrix} 1 & -3 \\ 0 & 1 \end{bmatrix}\right) = -3 \\ 7 + (-3) &= 4 \\ f^{-1}(4) &= \begin{bmatrix} 1 & 4 \\ 0 & 1 \end{bmatrix} \end{aligned}$$

$$\begin{aligned} \text{d. } f^{-1}(2) &= \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \text{ e } f^{-1}(3) = \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix} \\ \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix} &= \begin{bmatrix} 1 & 5 \\ 0 & 1 \end{bmatrix} \\ f\left(\begin{bmatrix} 1 & 5 \\ 0 & 1 \end{bmatrix}\right) &= 5 \end{aligned}$$

19. a. $i_E = i_E \cdot i_D = i_D$, portanto $i_E = i_D$ e ele é o elemento neutro em $[S, \cdot]$.

b. Por exemplo, c. Por exemplo,

$\cdot$	a	b
a	a	b
b	a	b

$\cdot$	a	b
a	a	a
b	b	b

d. Por exemplo, $[\mathbb{R}^+, +]$.

22. a. $x \rho x$ porque $i \cdot x \cdot i^{-1} = x \cdot i^{-1} = x \cdot i = x$. Se $x \rho y$, então para algum $g \in G$, $g \cdot x \cdot g^{-1} = y$, logo $g \cdot x = y \cdot g$ ou $x = g^{-1} \cdot y \cdot g$, que pode ser escrito como $(g^{-1}) \cdot y \cdot (g^{-1})^{-1}$. Portanto, $y \rho x$. Finalmente, se $x \rho y$ e $y \rho z$, então para algum, $g_1, g_2 \in G$, $g_1 \cdot x \cdot g_1^{-1} = y$ e $g_2 \cdot y \cdot g_2^{-1} = z$, logo $g_2 \cdot g_1 \cdot x \cdot g_1^{-1} \cdot g_2^{-1} = z$, ou $(g_2 \cdot g_1) \cdot x \cdot (g_2 \cdot g_1)^{-1} = z$. Portanto, $x \rho z$.

b. Suponha que G é comutativo e $y \in [x]$. Então para algum $g \in G$,

$$y = g \cdot x \cdot g^{-1} = x \cdot g \cdot g^{-1} = x \cdot i = x.$$

Então $[x] = \{x\}$. Reciprocamente, suponha $[x] = \{x\}$ para cada $x \in G$. Sejam $x, y \in G$, e seja $y \cdot x \cdot y^{-1} = z$. Então $x \rho z$, logo $z = x$ e $y \cdot x \cdot y^{-1} = x$ ou $y \cdot x = x \cdot y$, e G é comutativo.

28. a. $S \cap T \subseteq G$. Fecho: Para $x, y \in S \cap T$, $x \cdot y \in S$ por causa do fecho em S , $x \cdot y \in T$ por causa do fecho em T , portanto $x \cdot y \in S \cap T$. Elemento neutro: $i \in S$ e $i \in T$, logo $i \in S \cap T$. Inversos: Para $x \in S \cap T$, $x^{-1} \in S$ e $x^{-1} \in T$, logo $x^{-1} \in S \cap T$.

b. Não. Por exemplo, $[\{0, 4, 8\}, +_{12}]$ e $[\{0, 6\}, +_{12}]$ são subgrupos de $[\mathbb{Z}_{12}, +_{12}]$, mas $[\{0, 4, 6, 8\}, +_{12}]$ não é subgrupo de $[\mathbb{Z}_{12}, +_{12}]$ (não é fechado).

30. Fecho: Sejam $x, y \in B_k$. Então $(x \cdot y)^k = x^k \cdot y^k = i \cdot i = i$, logo $x \cdot y \in B_k$.

Elemento neutro $i^k = i$, logo $i \in B_k$.

Inversos: Para $x \in B_k$, $(x^{-1})^k = (x^k)^{-1} = i^{-1} = i$, logo $x^{-1} \in B_k$.

35. Seja $x = a^{z1}, y = a^{z2} \in G$. Então $x \cdot y = a^{z1} \cdot a^{z2} = a^{z1+z2} = a^{z2+z1} = a^{z2} \cdot a^{z1} = y \cdot x$.

37. a. $[\text{Aut}(S), \circ]$ é fechado porque a composição de isomorfismo é um isomorfismo (Prática 30). A propriedade associativa sempre vale para a composição de funções. A função identidade i_S é um automorfismo em S . Finalmente, se f é um automorfismo em S , então f^{-1} também o é.

b. $i: 0 \rightarrow 0 \quad f: 0 \rightarrow 0$

$$1 \rightarrow 1 \quad 1 \rightarrow 3$$

$$2 \rightarrow 2 \quad 2 \rightarrow 2$$

$$3 \rightarrow 3 \quad 3 \rightarrow 1$$

$\circ$	i	f
i	i	f
f	f	i

Exercícios 8.2

1. a. 0001111110

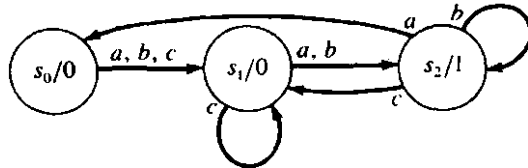
b. aaacaaaa

c. 00100110

5. Estado atual	Próximo estado		Saída
	Entrada atual		
	0	1	
2	s_1	s_2	a
s_1	s_2	s_3	b
s_2	s_2	s_1	c
s_3	s_2	s_3	b

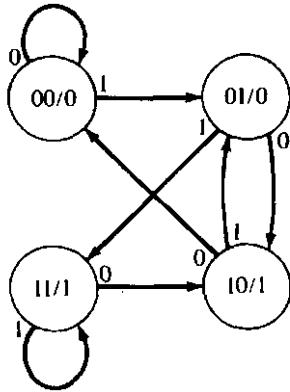
A saída é *abbcbb*.

9.



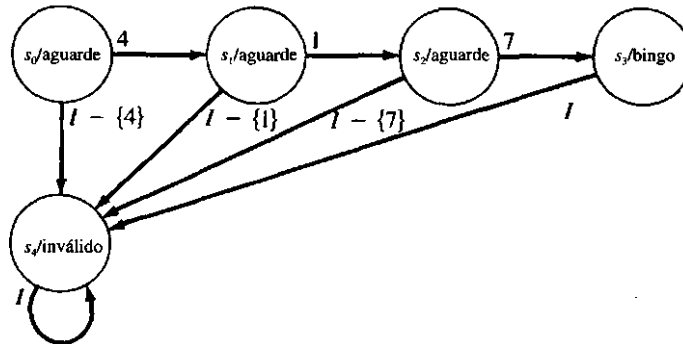
A saída é 0001101.

13. a.



b. A quantidade de tempo necessária para lembrar de uma determinada entrada cresce sem limite e, em algum momento, ultrapassa o número de estados.

14.



18. Uma vez que um estado seja revisitado, o comportamento será periódico, uma vez que não há escolha de caminhos a partir de um estado. O número máximo de entradas que podem ocorrer antes disso acontecer é $n - 1$ (visitando todos os n estados antes de repetir). O tamanho máximo do período é n (saída de todos os n estados com o último estado retornando a s_0).

21. c. 100^*1

22. a. Sim b. Não c. Não d. Sim

24. $dd^*(+ \vee -)dd^*$

27. a. s_1 b. s_2 28. $A = \{0\}, B = \{1, 2, 5\}, C = \{3, 4\}, D = \{6\}$

Estado atual	Próximo estado		Saída
	Entrada atual 0	1	
A	C	D	1
B	C	B	0
C	B	A	1
D	C	B	1

33. $A = \{0, 2\}, B = \{1, 3\}, C = \{4\}$

Estado atual	Próximo estado			Saída
	Entrada atual <i>abc</i>			
<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	1
<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	0
<i>C</i>	<i>B</i>	<i>A</i>	<i>A</i>	0

Exercícios 8.3

1. a. Termina com fita final

...	b	0	0	0	0	0	b	...
-----	---	---	---	---	---	---	---	-----

b. Não altera a fita e move para sempre para a esquerda.

5. Uma resposta: O estado 2 é um estado final.
 (0, b, b, 2, D) fita em branco ou não tem mais ls
 (0, 1, 1, 1, D) leu um número ímpar de ls
 (1, 1, 1, 0, D) leu um número par de 1 s

8. Uma resposta: Estado 9 é um estado final.

(0, b, b, 9, D)	aceita fita em branco
(0, 0, 0, 0, D) }	encontra primeiro 1, marca-o com X
(0, 1, X, 1, D) }	
(1, 1, 1, 1, D) }	
(1, Y, Y, 1, D) }	procura à direita por 2s
(1, 2, Y, 3, D) }	par de 2s, marca com Ys
(3, 2, Y, 4, E) }	
(4, Y, Y, 4, E) }	
(4, X, X, 4, E) }	procura à esquerda por Os
(4, 1, 1, 4, E) }	
(4, Z, Z, 4, E) }	
(4, 0, Z, 5, E) }	par de Os, marca com Zs
(5, 0, Z, 6, D) }	
(6, Z, Z, 6, D) }	
(6, X, X, 6, D) }	anda para a direita até o próximo 1
(6, 1, X, 1, D) }	
(6, Y, Y, 7, D)	não há mais ls
(7, Y, Y, 7, D) }	não há mais 2s
(7, b, b, 8, E) }	
(8, Y, Y, 8, E) }	
(8, X, X, 8, E) }	não há mais Os, termina e aceita
(8, Z, Z, 8, E) }	
(8, b, b, 9, E)	

12. Uma forma usa a estratégia geral a seguir: ponha uma marca à direita da cadeia original X_1 e construa uma nova cadeia X_2 à direita da marca. Varrendo X_1 do bit menor para o bit de maior ordem, para cada novo símbolo em X_1 ponha um bloco deste símbolo ao fim de X_2 . Este bloco deve ser duas vezes maior que o bloco anterior de símbolos incluídos a X_2 . Quando todos os símbolos em X_1 tiverem sido processados, apague X_1 e a marca. Agora, varrendo X_2 da esquerda para a direita, substitua quaisquer Os pelos 1s à direita da cadeia até que não haja mais Os à esquerda em X_2 . (A minha implementação desta abordagem precisou de 23 estados e 85 quintuplas. Você consegue uma solução melhor?)

$$14. f(n_1, n_2, n_3) = \begin{cases} n_2 + 1 & \text{se } n_2 > 0 \\ \text{indefinido} & \text{se } n_2 = 0 \end{cases}$$

16. Uma resposta:

$$\begin{array}{ll} (0, 1, 1, 1, R) & \\ (1, b, 1, 4, R) & n = 0, \text{ some 1 e pare} \\ (1, 1, 1, 2, R) \} & \\ (2, b, 1, 4, R) \} & n = 1, \text{ some mais 1 e pare} \\ (2, 1, 1, 3, R) \} & \\ (3, 1, b, 3, R) \} & n \leq 2, \text{ apague os 1 s extra e pare} \\ (3, b, b, 4, R) \} & \end{array}$$

19. Uma resposta:

$$\begin{array}{ll} (0, 1, b, 1, R) & \text{apague um 1 extra} \\ (1, *, b, 3, R) & n_1 = 0 \\ (1, 1, b, 2, R) \} & \\ (2, 1, 1, 2, R) \} & n_1 > 0, \text{ substitua * com o 1 mais à esquerda de } \bar{n} 1, \text{ pare} \\ (2, *, 1, 3, R) \} & \end{array}$$

Exercícios 8.4

1. a. $L(G) = \{a\}$
 b. $L(G) = \{010101, 010111, 011101, 011111, 110101, 110111, 111101, 111111\}$
 c. $L(G) = 0(10)^*$
 d. $L(G) = 0^*1111^*$
3. $L(G) = aa^*bb^*$. G é sensível ao contexto. Um exemplo de uma gramática regular G' que gere $L(G)$ é $G' = (V, V_T, P, S)$ onde $V = \{a, b, A, B, S\}$, $V_T = \{a, b\}$, e P consiste nas produções

$$\begin{array}{lll} S \rightarrow aA & A \rightarrow aA & B \rightarrow bB \\ S \rightarrow aB & A \rightarrow aB & B \rightarrow b \end{array}$$

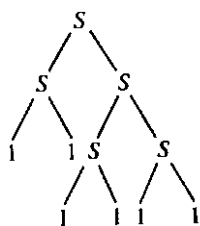
7. Por exemplo, $G = (V, V_T, S, P)$, onde $V = \{0, 1, A, S\}$, $V_T = \{0, 1\}$ e P consiste nas produções

$$\begin{array}{ll} S \rightarrow 01 & A \rightarrow A0 \\ S \rightarrow A01 & A \rightarrow A1 \\ & A \rightarrow 0 \\ & A \rightarrow 1 \end{array}$$

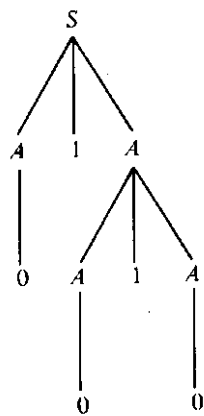
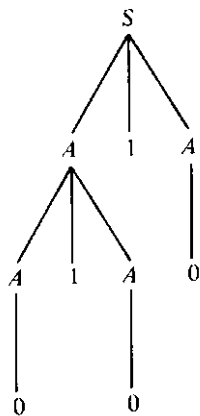
12. Por exemplo, $G = (V, V_T, S, P)$, onde $V = \{0, 1, S, S_1\}$, $V_T = \{0, 1\}$ e P consiste nas produções

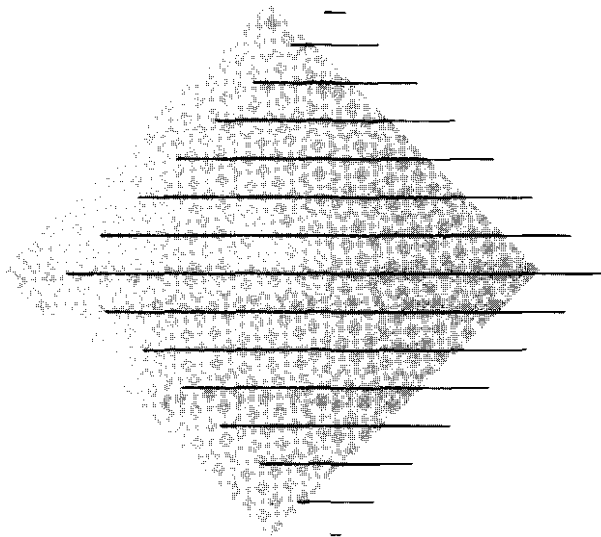
$$\begin{array}{ll} S \rightarrow \lambda & S_1 \rightarrow 0S_10 \\ S \rightarrow S_1 & S_1 \rightarrow 1S_11 \\ & S_1 \rightarrow 00 \\ & S_1 \rightarrow 11 \end{array}$$

15. a.



16.





Respostas dos Autotestes

Capítulo 1

1. F Se uma sentença não é uma tautologia, nem todos os valores de sua tabela-verdade são verdadeiros, mas isto não os torna todos falsos.
2. V
3. F A sentença precisa ser uma implicação.
4. V
5. F A negação de uma disjunção é a conjunção das negações.
6. F Na verdade, $(\forall x) (P(x) \wedge [P(x)]')$ é falso em qualquer interpretação.
7. V
8. V
9. F Não existe wff predicativa definida em todas as interpretações, nem o domínio delas pode ser determinado por valores-verdade.
10. V
11. F Se isso fosse verdade, então toda wff seria um teorema.
12. V
13. F É um argumento no qual hipótese $\rightarrow$ tese é sempre verdadeiro.
14. V
15. V
16. F O quantificador universal só pode se incluído através da generalização.
17. V
18. F Os teoremas da lógica proposicional não são sempre wffs na lógica de predicados.
19. V
20. F Devemos primeiro tentar usar o Axioma 6.
21. V
22. V
23. F Só podemos garantir que a saída satisfaz certas condições uma vez que a entrada satisfaz certas condições.
24. F Não se pode dizer muito sobre a precondição sem conhecer a atribuição, mas a desigualdade estrita não será perdida.
25. F Os testes de programas envolvem conjuntos de dados para testes.

Capítulo 2

1. F Um teorema que só afirme algo sobre um número finito de casos pode ser provado verificando-se todos os casos.
2. V

3. F Subentendemos um quantificador *universal*, pois o enunciado formal do teorema é $(\forall x)(x \text{ ímpar} \rightarrow 2 \cdot x \text{ par})$.
4. F A segunda sentença é a recíproca da primeira, não a contrapositiva.
5. V
6. V
7. F O passo básico não precisa ser $n = 1$.
8. V
9. V
10. F Ela omite os primeiros $k - 1$ termos da série.
11. V
12. F Eles são úteis porque representam formas naturais de pensamentos sobre certos problemas, mas normalmente necessitam de maior espaço de armazenamento e realizam mais operações do que um programa iterativo correspondente.
13. V
14. F A indução pode ser usada para verificar uma solução de forma fechada, mas não para determinar uma expressão deste tipo dada uma relação de recorrência.
15. F Não é linear por causa da presença da parcela $S(n - 2)$.
16. V
17. V
18. F
19. F Significa que a correção só pode ser demonstrada se for dado que o laço termina.
20. V

Capítulo 3

1. F Ele não é um subconjunto próprio dele mesmo.
2. V
3. V
4. F Esta é a propriedade do fechamento.
5. F É um método para demonstrar que determinados conjuntos não são enumeráveis.
6. V
7. F
8. V
9. V
10. F
11. F
12. F O número de elementos da união *mais* o número de elementos da interseção é a soma do número de elementos dos conjuntos.
13. F Todos os conjuntos precisam ser finitos.
14. F
15. V
16. V
17. V
18. F Use $C(n, r)$.
19. F $n!/n_1!n_2!n_3!$
20. V
21. F Combinações, não arranjos.
22. V
23. F Todos os termos se encontram na linha n .
24. V
25. V

Capítulo 4

1. V
2. F (x, x) pode pertencer.
3. V
4. F A relação de igualdade é tanto uma ordenação parcial quanto uma relação de equivalência.
5. F Uma relação de equivalência determina isto.
6. V
7. V
8. F
9. F
10. F A recíproca é verdadeira.
11. F Ela pode não ter uma imagem para cada elemento do domínio.
12. F Todo elemento da faixa tem uma pré-imagem; precisamos começar com um elemento do contradomínio.
13. V
14. V

15. F Outras constantes podem funcionar em situações em que esta não.
16. V
17. F
18. V
19. V
20. F Veja a Prática 49.

Capítulo 5

1. F
2. V
3. F Um grafo planar pode ser desenhado com arestas interceptando-se.
4. F Ela precisa conter um subgrafo homeomorfo para K_5 ou $K_{3,3}$.
5. V
6. F Ela pode ser simétrica.
7. F
8. F Isto depende da "densidade" de arestas no grafo.
9. V
10. V
11. V
12. V
13. F
14. F
15. F
16. V
17. F Este é o pior caso; outros casos podem exigir menos comparações.
18. V
19. F Ela precisa ter pelo menos $n!$ folhas.
20. F

Capítulo 6

1. V
2. F
3. V
4. F Algumas arestas podem não ser usadas.
5. F Não é conhecido qualquer algoritmo eficiente, mas tentativas e erros resolvem o problema.
6. F É assim que funciona o algoritmo de Prim.
7. F
8. V
9. V
10. F Em geral, esta coleção não forma uma árvore.
11. F
12. V
13. V
14. F
15. V
16. F
17. V
18. V
19. V
20. F A raiz é um caso especial.

Capítulo 7

1. F $x + x' = 1$
2. V
3. V
$$\begin{aligned} x + (y + x \cdot z) &= x + (x \cdot z + y) = x \cdot 1 + (x \cdot z + y) \\ &= (x \cdot 1 + x \cdot z) + y = x \cdot (1 + z) + y \\ &= x \cdot (z + 1) + y = x \cdot 1 + y = x + y \end{aligned}$$
4. V
5. V
6. V $(x + y)' = x' \cdot y'$
7. F Ela tem tantas parcelas quantos forem os 1 s da função (ou uma parcela, se todos os valores da função forem 0s).
8. F Só é necessário um meio-somador (veja a Fig. 7.14a).
9. V. Porque $x + x = x$.
10. F Isto geralmente resulta em um número desnecessariamente grande de dispositivos; devemos recomendar a partir da tabela-verdade.

- 11. V
- 12. V
- 13. F Primeiro procure blocos de duas caselas e continue a trabalhar a partir daí.
- 14. V
- 15. F A parcela é essencial se a marca em sua linha for a única em alguma coluna.

Capítulo 8

- 1. F Isto descreve uma operação comutativa.
- 2. V Apesar de esta não ser a definição de identidade.
- 3. V
- 4. V Pelo teorema de Lagrange.
- 5. F /precisa ser uma bijeção.
- 6. V
- 7. V
- 8. F Veja a Fig. 8.6a.
- 9. V
- 10. F
- 11. F Entrar em um estado final faz com que a máquina de Turing pare, mas ela também pode parar em um estado não final, caso não haja instrução para o par estado-entrada corrente.
- 12. F Terão $n + 2$ ls na fita.
- 13. F
- 14. F Esta é a versão com os quantificadores trocados, que é trivialmente verdadeira.
- 15. V
- 16. V
- 17. F Dependendo das produções selecionadas, os símbolos não-terminais podem não vir a ser removidos.
- 18. F Pode existir uma gramática equivalente para a linguagem que é livre de contexto.
- 19. V
- 20. V

Índice Alfabético

A

A», 368
Aceitação por uma máquina de Turing, 406
Adição de matriz, 206
Adição módulo n , 161, 366
Afirmção,
- composta, 2
Alcançabilidade, 255-258, 299
- e matrizes booleanas de adjacências, 255
- matriz de, 256
Alfabeto, 367, 418
- de entrada de uma máquina de estado finito, 383
- de fita, 404
- de saída de uma máquina de estado finito, 383
Álgebra
- booleana, 208, 316-324
- - como reticulado, 327
- - como semigrupo, 364
- - complemento de um elemento na, 319
- - definição de, 318
- - dual de uma propriedade na, 319
- - finita, teorema de, 324
- - isomorfismo para a, 322
- - propriedades na, 319
- —independente, 319
- - teorema da unicidade de complementos, 320
- relacionai, 173
ALGOL, 70, 421
Algoritmo
- articulação, 309
- definição de, 8, 410
- de Euclides, 96
- de Kruskal, 293
- de Prim, 289, 293
- dividir para conquistar, 85
- guloso, 289
- iterativo, 71
- largura, 297
- ótimo, 266
- profundidade, 295
- recursivo, 71-75
- recursivo *versus* iterativo, 72
Veja *também* Análise de algoritmos
Algoritmos
- árvore De Huffman, 270
- busca Binária, 74
- busca Sequencial, 83
- Caminho Euleriano, 281
- Caminho Mínimo, 285
- Caminho Mínimo Todos Os pares, 292
- inordem, 301
- mergesort, 91
- minimiza, 392
- multiplicação Matrizes, 208
- Outro AGM, 294
- posordem, 302
- preordem, 301
- selection Sort, 73
- soma-de-produtos, 332
- Testa Tautologia, 8
- Warshall, 158
Alteração, operação em banco de dados, 174
Altura de uma árvore, 230
Amplificação disjuntiva, 25
Analisador
- léxico, 426
- sintático, 426
Análise de algoritmos, 83-86, 197
- Caminho Euleriano, 282
- Caminho Mínimo, 288
- largura, 298

- profundidade, 298
Antecedente, 3
Anti-simétrica, relação, 154
Aplicação, veja função
Appel, Kenneth, 227
Aresta(s)
- de árvore, 308
- de retorno, 308
- de um grafo, 218
- paralelas, 219
Argumento válido
- na lógica de predicados, 30
- na lógica proposicional, 23
Aritmética modular, 366
Array, 205
- de bits, 109
- lógico programável, 335
Articulação, 307
- algoritmo para, 309
árvore, 230
- altura, 230
- binária, 230
- - completa, 230
- - de busca, 266
- - filho à direita em, 230
- - filho à esquerda em, 230
- - representação de, 247
- busca
- - em inordem, 301
- - em posordem, 302
- - empreordem, 301
- - em profundidade, 307
- - em uma, 300-303
- de decisões, 124, 232, 263-268
- - definição de, 263 .
- de parse, 425
- definição, 230
- folha, 230
- geradora, 289
- - mínima, 289
- algoritmo para, 392
- isomorfa, 243
- não-enraizada, 230
- profundidade, 230
- - de um vértice, 230
- raiz, 230
- teorema do número de arestas, 231
- vértice
- - filho em, 230
- - interno, 230
- - pai, 230
Asserção, 40
Atributos de uma entidade, 168
Autômato
- de pilha, 424
- linear limitado, 425
Automorfismo, 382
Axioma, 20
- da lógica de predicados, 27
- da lógica proposicional, 20
- de atribuição, 40
- esquema de, 21
- lógico, 50
- relacionado ao assunto, 50

B

Backus Naur, forma de, 70, 422
Banco de dados
- e relações, 168-175
- em Prolog, 33

- relacionai, 169
- - chave
- - - cega, 170
- - estrangeira, 170
- - primária composta, 170
- - primária, 170
- - integridade de entidades em, 170
- - integridade em, 174
- - integridade referencial, 174
- - JOIN, operação, 172
- - MODIFY, operação,
- - operação de inclusão, 174
- - PROJECT, operação, 171
- - SELECT, operação, 171
Base, 57
BASIC, 126, 398
Bellman-Ford, algoritmo de, 291
Bijecção, 188
- composição de, 191
- e função inversa, 194
BNF, veja Backus Naur, forma de
Boole, George, 208
Veja *também* Álgebra: booleana; E booleano; Expressões:
booleanas; Matriz: booleana
Busca, 263-267
- binária, 74, 197, 264
- - algoritmo para, 74
- - em árvore, 300-303
- - algoritmos para, 301
- - binária, 166, 233, 266
- —descrição de, 266
- em inordem, 301
- em largura, 295, 296-298
- em posordem, 302
- em preordem, 301
- em profundidade, 39, 295, 307
- limite inferior de uma, 265
- sequencial, 83, 197, 264

C

C, 100
 $C(n,r)$, 134
C, 100
Cadeia, 158
- binária, 70
- comprimento de uma, 367
- concatenação de, 70, 367
- sobre um alfabeto, 70
- vazia, 70, 367, 368
Cálculo
- de predicados, 20
- proposicional, 20
- relacionai, 173
Caminho
- comprimento de um, 219
- crítico, 177
- em um grafo, 219
- - direcionado, 232
- euleriano, 279-282
- - algoritmo para, 281
- - definição de, 280
- - teorema sobre, 281
- mínimo
- - algoritmo do, 285, 291, 292
- - problema do, 285-289
Cantor, Georg, 111
- método da diagonalização de, 111, 190
Cardinalidade, 190
Casa do pombo, princípio da, 131, 188, 403
- enunciado do, 131

514 Índice Alfabético

- Cayley, Arthur, 227, 377
- Teorema de, 377
- Centro de um grupo, 381
- Chave
 - cega, 170
 - estrangeira, 170
 - primária, 170
 - - composta, 170
- Chomsky, Noam, 421
- hierarquia de, 423
- Church, Alonzo, 410, 411, 412
- Church-Turing, tese de, 410
- enunciado da, 410
- Chão. função, 184
- Ciclo(s)
 - disjuntos, 192
 - em um grafo, 219
 - hamiltoniano, 282
- Cinco cores, teorema das, 229
- Classe
 - de complexidade, 97
 - - aritmética de, 197
 - - hierarquia de, 197
 - de equivalência, 159
 - Cláusula de Horn, 35
 - e resolução, 34
- Código
 - de Huffman, 268-272
 - prefixo, 269
 - reutilizável, 110
- Coefficiente(s)
 - binomial, 145
 - constantes em uma relação de recorrência, 76
- Cofator, 215
- Coloração
 - de grafos, 227-230
- Comando condicional, 42
- Combinações, 134-136
 - com repetições, 137
 - eliminando duplicadas, 136
- Veja *também* Redes: combinatórias
- Combinatória, 120
- Compactação de dados, 268
- Complemento
 - a dois, 343
 - de um conjunto, 105
 - de um elemento da álgebra booleana, 319
 - de um grafo, 240
 - propriedade de, 6, 7, 106, 317, 318
 - teorema da unicidade de, 320
- Completeness
 - da lógica de predicados, 28
 - da lógica proposicional, 21
- Complexidade, veja Classe: de complexidade
- Complexidade computacional, 414
- Componente
 - biconexa, 300, 308
 - conexa, 300
- Composição de funções, 190
- Compressão de dados, 268
- Comprimento
 - de um caminho, 219
 - de uma cadeia, 367
- Concatenação de cadeias, 70, 367
- Condição
 - necessária, 3
 - suficiente, 3
- Conectivo lógico, 2-5
 - binário, 3
 - conjunção, 2
 - disjunção, 3
 - E (AND), 7, 34
 - e valores verdade, 2
 - equivalência, 3
 - implicação, 3
 - na programação, 7
 - negação, 3
 - NÃO (NOT), 7, 34
 - ordem de precedência, 4
 - OU (OR), 7, 34
 - unário, 3
 - XOU (XOR), 12
- Congruência módulo n, 161
- Conjunção, 2
- Conjunto(s), 98-112
 - cardinalidade de, 190
 - complemento, 105
 - conjunto das partes de um, 102
 - contável, 110
 - convencional, 99-100
 - das partes, 102
 - denumerável, 110, 190
 - determinação de pertinência, 426
 - diferença simétrica de, 117
 - disjuntos, 105
 - dual de uma identidade de conjuntos, 107
 - enumerável, 110
 - equivalentes, 190
 - definição, 190
 - fecho sob uma operação, 103
 - função característica de, 200
 - fuzzy, 119
 - identidade, ver Identidade: de conjuntos
 - iguais, 99, 101
 - incontável, 110
 - índice, 119
 - interseção, 104
 - notação de, 98
 - nulo, 100
 - não-contável, 110, 165
 - operações em um, 104-106
 - parcialmente ordenado, 157, 232
 - dual de, 165
 - e grafos, 232
 - - isomorfo de, 320
 - - não-isomorfo, 326
 - partição de, 158
 - produto cartesiano de, 105
 - produto cruzado, 106
 - recursivo, 69
 - regular, 387, 388
 - subconjunto de um, 100
 - subconjunto próprio de um, 102
 - união, 104
 - universal, 104
 - vazio, 100
- Consequente, 3
- Contagem de funções, 188
- Contra-exemplo, 32, 51
- Contradição, 6
 - demonstração por, 53
- Contradomínio, 183
- Contraposição, 7
 - demonstração por, 52
- Contrapositividade, 52
- Convenção da produção vazia, 422
- Correção
 - da lógica de predicados, 28
 - da lógica proposicional, 21
 - parcial, 89
- D**
- De Morgan, Augustus, 6, 227
- Definição
 - indutiva, 67
 - recursiva, 67
- DeMoiivre, teorema de, 66
- Demonstração
 - combinatorial, 143, 145
 - condicional, 7
 - direta, 51
 - em sistemas formais, 20
 - indireta, 53
 - por contradição, 53
 - por contraposição, 52
 - por exaustão, 53
 - por indução, 56-63
- Veja *também* Técnicas de demonstração
- Denumerável, conjunto, 110, 190
- Derivação
 - em uma gramática, 419
 - direta, 419
- Determinante de uma matriz, 215
- Dez problemas de Hilbert, 412
- Diagonal principal de uma matriz, 205
- Diagrama
 - comutativo, 191,232
 - - para o isomorfismo, 321
 - de estrutura, 234
 - de fluxo de dados, 234
 - de Hasse, 158,254
 - de Venn, 1004
 - E-R, 168,232
- Dígrafo, 232
- Dijkstra, algoritmo de, 285
- Dimensões de uma matriz, 204
- Disjunção, 3
- Dividir para conquistar, algoritmos, 85
- Domínio
 - de uma função, 183
 - de uma interpretação, 13, 14
- Dual
 - de um conjunto parcialmente ordenado, 165
 - de um grafo, 228
 - de uma equivalência tautológica, 6
 - de uma identidade de conjuntos, 107
 - de uma propriedade de álgebra booleana, 319
- Dupla negativa, 7
- E**
- E booleano, 209
- E lógico, veja E booleano
- E, conectivo, 7, 34
- E, porta, 329, 333
- Elemento
 - inverso, 363
 - máximo, 158
 - mínimo, 158
 - neutro, 363
 - - à direita, 380
 - - à esquerda, 380
- Encapsulamento, 109
- Entidade, 168
 - atributos de, 168
- Entidade-relacionamento
 - diagrama, 168
 - modelo, 168
- Equivalências tautológicas, 6, 7
 - contraposição, 7
 - dual de uma, 6
 - dupla negativa, 7
 - leis de DeMorgan, 6, 7
 - propriedade
 - - associativa, 6, 7
 - - complementativa, 6, 7
 - - comutativa, 6, 7
 - - da identidade, 6, 7
 - - distributiva, 6, 7
 - - idempotente, 7
 - - prova condicional, 7
 - reservando a implicação, 7
- Escalar, 205
- Escopo de um quantificador, 14
- Esquema
 - de axioma, 21
 - de teorema, 21
- Estado(s)
 - equivalentes em uma máquina de estado finito, 390
 - final
 - - de uma máquina de estado finito, 387
 - - de uma máquina de Turing, 406
 - inalcançável, 389
 - k-equivalentes em uma máquina de estado finito, 391
- Estouro inteiro, 161
- Estrutura(s)
 - algébricas, 362-377
 - matemática, 317
- Euler, Leonhard, 224, 279
- fórmula, 224, 225
- Veja *também* Caminho: euleriano
- Exclusão, operação em um banco de dados, 174
- Expressão(ões)
 - booleanas, 329
 - - equivalentes, 334
 - - forma
 - canônica de produto de somas, 343
 - canônica de soma de produtos, 332
 - norma disjuntiva, 332
 - normal conjuntiva, 343
 - regular, 387
- Extremos de uma aresta, 218
- F**
- Fato do Prolog, 33
- Fatorial, 133
- Fecho
 - de um conjunto sob uma operação, 103
 - de uma relação, 155
 - reflexivo, 156
 - simétrico, 156

- transitivo, 156
- Fila, 296
- Filho
- à direita, 230
- à esquerda, 230
- Floresta, 230
- Floyd, algoritmo de, 293
- Folha de uma árvore, 230
- Forma
 - canônica de soma de produtos, 332, 343
 - normal
 - - conjuntiva, 343
 - - disjuntiva, 332
- Fórmula
 - bem formada, 4, 14
 - - equivalente, 6
 - - predicada, 16
 - - proposicional, 16
 - - válida, 16
 - de Euler, 224, 225
- FORTRAN, 400
- Função, 181-198
- bijetiva, 188
- característica, 200
- chão, 184
- como uma relação binária, 183
- composição de, 190
- composta, 191
- contagem de, 188
- contradomínio de uma, 183
- de Ackermann, 200
- de mais de uma variável, 184
- de saída, 383
- definição de, 183
- domínio de uma, 183
- em Pascal, 185
- exponencial, 435
- identidade, 192
- imagem de uma, 183, 186
- injetiva, 187
- inversa, 194
- - a direita de uma, 203
- - a esquerda de uma, 203
- de uma, 194
- logarítmica, 435-437
- logaritmo, 435
- número-teórica, 408
- não-determinística, 201
- ordem de grandeza de uma, 195
- parcial, 408
- permutação, 191-194
- - definição de, 191
- - notação cíclica, 192
- - sem ponto fixo, 193
- pré-imagem sob uma, 183
- propriedades de uma, 186-190
- próximo estado, 383
- saída, 383
- sobrejetiva, 186
- teorema do número de, 189
- teto, 184
- total, 158,408
- Turing-computável, 409
- verdade, 330

G

- Generalização, 28
- restrições sobre, 28
- Geração
 - direta em uma gramática, 419
 - em uma gramática, 419
- Gerador de um grupo cíclico, 382
- Godel, Kurt,412
- Grafo, 216-277
 - acíclico, 219
 - alcançabilidade em um, 299
 - algoritmos para, 278-314
 - de busca, 294, 297
 - aresta, 218
 - - paralelas em um, 219
 - articulação em um, 307
 - árvore geradora
 - de um, 289
 - - mínima de um, 289, 299
 - autocomplementar, 240
 - Bellman-Ford, algoritmo de, 291

- biconexo, 308
- bipartite completo, 223
- busca em um, 294
- - largura em um, 295, 296
- - profundidade em um, 295
- caminho
 - - em um grafo, 219
 - - euleriano
 - em um grafo, 280
 - teorema do, 281
 - - mínimo em um, 285-289
- ciclo
 - - em um grafo, 219
 - - hamiltoniano em um, 283
- cinco cores, teorema das, 229
- coloração de um, 227
- complemento de um, 240
- completo, 219
- componente
 - - biconexa de um, 300, 308
 - - conexa de um, 300
 - - condições para não haver isomorfismo, 222
- conexo, 219
- de estados, 384
- definição de, 218
- Dijkstra, algoritmo de, 285
- direcionado, 231
- - caminho em um, 232
- - definição de, 232
- - e relações binárias, 253-258
- - relação de adjacência de um, 253
- - vértice alcançável em, 232, 255
- direcionado, 231,232
- dual de um, 228
- Floyd, algoritmo de, 292
- grau de um vértice de um, 219
- homeomorfos, 226
- isomorfo, 220-223
- - definição de, 221
- Kruskal, algoritmo de, 293
- Kuratowski, teorema de, 226
- laço em, 219
- número cromático de um, 228
- planar, 223
- planaridade de, 223-227
- ponderados, 232
- Prim, algoritmo de, 289, 293
- quatro cores
 - - problema das, 227
 - - teorema das, 228
- representação
 - - computacional de, 243-248
 - - por vetor-ponteiro, 245
- seis cores, teorema das, 241
- sem laços, 219
- simples, 219
- - isomorfos, 221
- subgrafo, 219
- tamanho de um caminho em um, 219
- teorema
 - - do número de vértices e arestas, 225
 - - dos vértices ímpares, 280
- terminologia, 217-233
- vértice de um, 157, 218
- - adjacentes em um, 219
- - isolado em um, 219
- - Warshall, algoritmo de, 299, 300

Gramática, 418-426

- ambígua, 428
- classes de, 422-424
- com estrutura de frase, 419
- convenção da produção vazia, 422
- de estrutura de frase, 419
- derivação direta em uma, 419
- derivação em uma, 419
- equivalente, 424
- geração direta em, 419
- geração em uma, 419
- hierarquia de Chomsky de, 423
- livre de contexto, 423, 425
- produção em, 419
- regular, 423
- sensível ao contexto, 423
- símbolo inicial em, 419
- terminal em, 419
- tipo
- 0, 419

- 1,423
- - 2, 423
- - 3, 423
- Grau
 - de um polinômio, 365
 - de um vértice, 219
- Grupo
 - - alternante, 372
 - cancelamento em, 369
 - Cayley, teorema de, 377
 - centro de, 381
 - cíclico, 382
 - cíclico, gerador de um, 382
 - classes de grupos isomorfos, 376
 - comutativo, 363
 - de permutações, 371
 - - em um conjunto, 367, 371
 - de polinômios sob a adição, 365
 - de simetrias de um triângulo equilátero, 379
 - definição de, 363
 - homomorfismo de, 373
 - isomorfismo de, 373
 - Lagrange, teorema de, 372
 - ordem de um, 370
 - resolvendo equações lineares em um, 370
 - resultados básicos sobre, 368-370
 - simétrico de grau n, 367
 - subgrupo de um, 370
 - teorema de grupos pequenos, 376
 - teorema do elemento neutro de um, 368

H

- Haken, Wolfgang, 227
- Hamilton, William Rowan, 227
- Herança, 110
- Hierarquia de Chomsky, 423
- Hilbert, David,412
- Hipótese
 - de um teorema, 21
 - indutiva, 57
 - temporária, 30
- Homomorfismo, 373

I

- Identidade
 - de conjuntos, 106
 - - propriedade
 - associativa, 106
 - comutativa, 106
 - distributiva, 106
 - - - do elemento neutro, 106
 - unicidade da, 368
- Imagem, 183
 - de uma função, 186
- Implicação, 3
- InCluiNaFila, na lista, operação de, 297
- Índice de somatório, 77, 432
- Indução, 56-63
 - completa, 61
 - em somatórios, 58
 - forte, 61, 119
 - fraca, 61, 119
 - hipótese indutiva, 57
 - matemática, princípio da, 57
 - passo básico, 57
- Insolubilidade do problema de parada, 413
- Integridade
 - de dados, 174
 - de entidades, 170, 174
 - em um banco de dados, i 74
 - referencial, 174
- Interpretação
 - definição de, 14
 - domínio de, 14
- Interseção de conjuntos, 104
- Invariante do loop, 88
- Inversa
 - à direita
 - - de um elemento, 380
 - - de uma função, 203
 - à esquerda
 - - de um elemento, 381
 - - de uma função, 203
 - de um produto, 368
 - de uma matriz, 211

516 Índice Alfabético

- unicidade de, 368
- Inversor (porta NÃO), 329, 333
- Isomorfismo
 - de álgebras booleanas, 320-324
 - de árvores, 243
 - de classes de grupos, 376
 - de conjuntos parcialmente ordenados, 320
 - de estruturas, 320
 - de grafos, 220-222
 - - simples, 221
 - de grupos, 373

J

JOIN, operação, 172

K

$K_{m,n}$ 223
 K_n , 222
Kempe, A.B., 227
Kleene, Stephen, 388

- teorema de, 388

Kuratowski, 226

- teorema de, 226

L

Lação em um grafo, 219
Lagrange, Joseph-Louis, 372

- teorema de, 372

Lba, veja Autômato: linear limitado
Lei(s)

- de cancelamento, 369
- - à esquerda, 369
- de De Morgan, 6, 7, 35, 118, 355
- - extensão finita das, 81

Lema do bombeamento para linguagens livre de contexto, 428
Limite

- inferior
- - da busca, 265
- - de uma ordenação, 268
- - do somatório, 432
- superior do somatório, 432

Linguagem

- declarativa, 33
- descritiva, 33
- formal, 418-426
 - - aplicações em, 232
 - - definição de, 420
 - - e dispositivos computacionais, 424
- funcional, 186
- gerada por uma gramática, 419
- hierarquia de, 423
- livre de contexto, 423
- - lema do bombeamento para, 428
- palíndroma, 428
- procedural, 33
- regular, 423
- sensível ao contexto, 423
- sobre um alfabeto, 418
- tipo 0, 423

Lista

- de adjacências, 245

-encadeada, 109,232,245
Logaritmo

- base dois, 437
- comum, 437
- natural, 437

Veja também Função: logarítmica
Lógica, 1-48

- conectivos lógicos, 2-5
- de predicados/de primeira ordem, 20, 27
 - - argumentos válidos na, 30
 - - axiomas da, 27
 - - completude da, 28
 - - correteude da, 28
 - - regras de inferência para, 28
- de sentenças, 20
- formal, 1-48
 - proposicional, 20-25
 - - argumentos válidos na, 21
 - - axiomas da, 21
 - - completude da, 21
 - - correteude da, 21

- - definição da, 20
- - regras de inferência da, 21,25
- tabela-verdade, 2
- tautologias, 6
- valores verdade, 2

M

$\{M\} (<<Z>>), +\}$, 364
Maior limite inferior, 327
Mapa de Karnaugh, 347-352
Máquina(s)

- de complemento a dois, 397
- de estado finito, 232, 382-394
 - - alfabeto de entrada de, 383
 - - alfabeto de saída de, 383
 - - de retardo, 397
 - - definição de, 383
 - - estado final de, 387
 - - estado inalcançável em, 389
 - - estados equivalentes em, 390
 - - estados k-equivalentes em, 391
 - - função de saída de, 383
 - - função do próximo estado, 383
 - - grafo de estados para
 - - kleene, teorema de, 388
 - - reconhecimento por, 387
 - - somador binário, 385
 - - tabela de estados para, 383
 - de retardo, 397
 - de Turing, 402-410
 - - alfabeto de fita, 404
 - - como reconhecedor de conjuntos, 406
 - - conjunto aceito por uma, 406
 - - conjunto reconhecido por uma, 406
 - - definição, 404
 - - estado final em uma, 406

Markov, A., 411
Matriz, 204-209

- adição de, 206
- alcançabilidade, 256
- booleana, 208
 - - de adjacências e alcançabilidade, 255
 - - multiplicação de, 209
- cofator de um elemento de uma, 215
- de adjacências, 244
 - - esparsas, 245
- de alcançabilidade, 256
- definição de, 204
- determinante de uma, 215
- diagonal principal de uma, 205
- dimensões de uma, 204
- esparsa de adjacências, 245
- identidade, 208
- igualdade de, 205
- inversa, 211
- inversível, 211
- multiplicação de, 205
- - booleana, 209
- - escalar de, 206
- nula, 206
- operações em uma, 205-208
- quadrada, 205
- simétrica, 205
- subtração, 206
- transposta de, 212

Máximo divisor comum, 96
Meio-somador, 337
Menor limite superior, 327
Método

- da diagonalização de Cantor, 111, 190
- de dedução, 21

Minimização de redes lógicas, 334, 346-355
- mapa de Karnaugh, 347-352
- procedimento de Quine-McCluskey, 352-355
ML, 186
Modelo

- conceituai, 168
- relacionai, 169

Modula-2, 107
Modus

- Ponens, 21,25, 28, 35
- Tollens, 25

Monoid, 364

- inversa
- - à direita em um, 381
- - à esquerda em um, 381

- livre, 364

Multiplicação

- de matrizes, 207
- escalar, 206
- módulo n , 366

N

$<<N>>, 99$
 $\{n <<Z>>, +\}$, 372
Negação

- de sentenças com quantificadores. 16
- de uma sentença composta, 3

Notação

- cíclica, 192
- infixa, 115,303
- O, 196
- polonesa, 303
- - reversa, 303
- pós-fixa, 48, 115,303
- prefixa, 303

Número(s)

- cromático, 228
- deStirling, 168
- figurados, 82
- pentagonais, 83
- primo, 32, 55
- racional, 54
- triangulares, 82

Não-importa, condição, 340

O

Ocorrência

- ligada de uma variável, 15
- livre de uma variável, 15

Operação

- bem-definida, 103
- binária
 - - definição de, 103
- -eunária, 102-104
 - - propriedades de, 363
- de inclusão em um banco de dados, 174
- em conjuntos, 104-106
- recursiva, 70
- unária, 103

Ordem

- de grandeza, 195
- - definição de, 196
- de precedência, 4
- de um grupo, 370

Ordenação

- limite inferior, 268
- merge sort, 91
- parcial, 157
 - - definição de, 157
- - diagrama de Hasse para, 158
- - elemento
 - - maximal em uma, 158
- - máximo de uma, 158
- - - minimal de uma, 158
- - mínimo de uma, 158
- - predecessor em uma, 157
- - predecessor imediato em uma, 157
- - restrição de, 157
- - sucessor em, 157
- selection sort, 73
- topológica, 174-178,299
 - - definição de, 177
- total, 158

OU

- booleano, 209
- exclusivo, 12, 341
- lógico, veja OU booleano

Outerjoin, 171

P

- definição de, 414
- e NP, 414

 $P(n, r)$, 133
Palavra sobre um alfabeto, 367-368, 418
Palíndromo, 70, 416, 428
Par ordenado, 102
Parcela, 3

- da disjunção, 3
- Parênteses bem balanceados. 416
- Parsing
 - bottom-up, 426
 - top-down, 426
- Partição
 - definição de, 158
 - e relação de equivalência, 160
 - refinamento de uma, 167. 392
- Pascal, 7, 8, 33, 107, 186, 398, 426
- formulado, 143, 168
- função em, 185
- triângulo de, 142
- Pascal. Blaise, 142
- Passo
 - básico, 57
 - indutivo, 57
- Pda, *veja* autômato: de pilha
- Permutação, 133
 - com repetições, 137
 - definição de, 133
 - distinta, 137
 - eliminação de duplicatas, 136
 - identidade. 192
 - sem ponto fixo. 193
- PERT, diagrama. 175,232
- Pesquisa em Prolog. 33 (Quecry)
- Pilha, 202
- PLA, *veja* Array: lógico programável
- Planaridade de grafos, 223-227
- Polinômio
 - binomial, 142
 - - demonstração do, 144
 - - enunciado do, 144
 - de grau zero, 365
 - em x sobre $\langle \mathbb{R} \rangle$, 365
 - grau de um. 365
- Ponteiro nulo, 109, 245
- Ponto final, 232
- Ponto inicial, 232
- Pop, instrução, 202
- Porta
 - NE (NAND), 338
 - NOU (NOR), 338
 - OU (OR), 329. 333
- Pós-condição, 40
- Poset, 157
- Post, E.,411
- Pré-condição, 40
- Predecessor, 157
 - imediato, 157
- Predicado, 13
 - binário, 13
 - N -ário, 13
 - ternário, 13
 - unário, 13, 99
- Preimagem, 183
- Princípio
 - da adição, 122-124
 - - enunciado do, 122
 - da boa-ordenação, 61, 372
 - - enunciado do, 119
 - da casa do pombo, 131, 188, 403
 - da inclusão e exclusão, 128, 188, 193
 - - forma geral do, 129
 - da indução matemática, 57
 - da multiplicação, 121, 188, 193,208
 - - enunciado do, 121
- Problema
 - da parada, 412-414
 - - definição do, 413
 - - teorema da insolubilidade do, 413
 - de decisão, 411-414
 - - definição do, 411
 - - solução negativa para, 412
 - - solução positiva para, 412
 - do ciclo hamiltoniano. 414
 - indecidível, 412
 - insolúvel, 412
 - intratável, 197
 - NP, 415
 - - completo, 415
- Procedimento computacional. 410
- Produção, 419
- Produto
 - cartesiano de conjuntos, 105
 - cruzado de conjuntos, 106
- Profundidade

- de um vértice de árvore, 230
- de uma árvore, 230
- Programa correto, 39
- Programação
 - lógica, 33-39
 - orientada a objeto, 109
 - - código reutilizável, 110
 - - encapsulamento, 109
 - - herança, 110
- Progressão
 - aritmética, 65
 - geométrica, 64
- PROJECT, operação, 171
- Prolog, 33, 155, 300
 - banco de dados em, 33
 - cláusula de Horn em, 35
 - fatos em, 33
 - pergunta em, 33
 - recursão em, 37
 - regra em, 35
 - - de inferência em, 35
 - - recursiva em, 38
 - resolução em, 35
- Proposição, 2
- Propriedade
 - associativa, 6, 7, 106, 118, 317, 363
 - - extensão finita da, 71
 - comutativa, 6, 7, 106, 317, 318, 363
 - de absorção, 325
 - de complemento, 6, 7, 106, 317, 318
 - de identidade, 6, 7, 106, 317, 318
 - distributiva, 6, 7, 106,317,318
 - - extensão finita da, 81
 - idempotente, 7, 319
 - modular,
- Prova
 - da correção, 39-44, 86-89
 - - axioma da atribuição, 40
 - - regra condicional na, 42
 - - regra de laço na, 87
- Veja também* Demonstração
- Pseudocódigo, 8
- Push, instrução, 202

Q

Q, 99

- Quadrado perfeito, 55
- Quantificador, 12
 - e predicados, 12-16
 - escopo de um, 14
 - existencial, 13
 - universal, 12
- Quatro cores
 - problemas das, 227
 - teorema das, 228
- Quine-McCluskey, procedimento de, 352-355

R

R, 99

- [$\langle \mathbb{R} \rangle$] +, .], 364
- [$\langle \mathbb{R} \rangle$] [x], +], 365
- Raciocínio
 - dedutivo, 51
 - indutivo, 51
- Raiz de uma árvore, 230
- Recíproca, 53
- Reconhecimento
 - Reconhecimento por uma máquina
 - de estado finito, 387
 - de Turing, 406
- Recursão, 67-75
 - em Prolog, 37
- Redes
 - combinatórias, 328-336
 - - descrição de, 331
 - - minimização de, 334, 346-355
 - lógicas, 323, 328-340
 - - minimização de, 334, 346-355
 - com o mapa de Karnaugh, 347-352
 - com o procedimento de Quine-McCluskey. 352-355
- Reescrevendo implicações, 7
- Refinamento de uma partição, 167
- Regra(s)
 - condicional de inferência, 42
 - de inferência, 20

- para a lógica de predicados, 28
- para a lógica proposicional, 20, 25
- de L'Hospital, 204
- do loop de inferência, 88
- em prolog, 34
- recursiva, 38
- sintáticas, 4
- Relação, 152-161
 - anti-simétrica, 164
 - assimétrica, 164
 - binária
 - - e grafos direcionados, 253-258
 - em $S \times T$, 152
 - - em um conjunto S , 153
 - de adjacências, 253
 - de equivalência, 158-161
 - - definição, 158
 - - e partição, 160
 - de recorrência, 67
 - - coeficientes constantes em uma, 76
 - - de primeira ordem, 76
 - - homogênea, 76
 - - linear, 76
 - - resolução de, 75-78
 - - solução de forma fechada para, 75
- e bancos de dados, 168-174
- fecho
 - de uma, 156
 - - reflexivo de, 156
 - - transitivo de, 156, 258
- irreflexiva, 164
- N -ária, 153
- - em um conjunto S , 153
- operações em, 171-174
- propriedades de, 154
- reflexiva, 154
- transitiva, 154
- um-para-um, 153
- um-para-vários, 153
- unária, 153
- vários-para-um, 153
- vários-para-vários, 153
- Representação de árvore binária por filhos à esquerda e à direita, 247
- Resolução, 35
- Resolvendo relação de recorrência, 75
- Restrição de uma ordenação parcial, 157
- Reticulado, 327
 - complementado, 327
 - distributivo, 328
- Russel, Bertrand, 120
- paradoxo de, 120

S

- S_n , 367
- [S_A , γ], 367
- Scanner, 426
- Seis cores, teorema das, 241
- SELECT
 - comando em SQL, 173
 - operação, 171
- Selection Sort, algoritmo, 73
- Semigrupo, 364
 - automorfismo em, 382
 - de transformação em um conjunto, 366
 - elemento neutro à direita, 380
 - elemento neutro à esquerda de, 380
- Sentença, 2
 - antecedente, 2
 - composta, 2
 - consequente, 3
 - simples, 2
- Seqüência, 67
 - aritmética, 65
 - de Fibonacci, 68
 - de prova, 20
 - - inclusão de tautologias, 24
- geométrica, 64
- recursivas, 67
- Seqüencial, busca, 197, 264
- algoritmo para, 83
- Séries de Maclurin, 203
- SETL, 108
- Shannon, Claude, 328
- Silogismo
- disjuntivo, 25

518 Índice Alfabético

- hipotético, 25
- Símbolo(s), 367
- de constantes, 14
- inicial, 419
- proposicional, 2
- Simplificação conjuntiva, 25
- Sistema(s)
 - baseados cm
 - - conhecimento, 39
 - - regras, 39
 - especialistas, 39
 - formal/lógico
 - - axioma em um, 20
 - - formal, 20
 - - prova em um, 20
 - - regra de inferência em um, 20
 - - sequência de prova em um, 20
 - - teorema em um, 20
- Smullyan. R., 411
- Sociedade de Pitágoras, 82
- Solução
 - de forma fechada, 75
 - negativa de um problema de decisão, 412
 - positiva para um problema de decisão, 412
- Somador
 - binário. 385
- ~ completo, 337
- Somatório
 - índice de, 77, 432
 - limite
 - - inferior de, 432
 - - superior de, 432
 - mudando os limites, 433
 - notação de, 77, 432
 - regras de, 433
- SQL, 173
- Subconjunto, 100
 - próprio, 100
- Subgrafo, 219
- Subgrupo, 370-373
 - $\{\langle \mathbb{Z} \rangle, +\}$, 373
 - definição de, 370
 - impróprio, 371
 - próprio, 371
 - teorema de Lagrange, 372
- Subtração de matrizes, 206
- Sucessor, 157

T

- Tabela
 - de comparação, 354
 - de estados, 383
 - de redução, 354
 - verdade, 2
 - - número de linha em uma, 4
- Tautologia, 6-7
- Técnicas de demonstração, 50-54
 - direta, 51
 - por contradição, 53
 - por contraposição, 52
 - por exaustão, 53
 - por indução, 56-63
- Teorema, 20
 - dedução, 21
 - esquema de, 21
 - hipótese, 21
- Tese de um teorema, 21
- Teste
 - de programa, 40
- Tipo abstrato de dado, 109
- TiraDaFila, operação de, 297
- Token, 70
- Transposição, 371
 - ímpar, 372
 - par, 372
- Transposta de uma matriz, 212
- Triângulo de Stirling, 168
- Tupla, 169
- Turing, Alan M., 403, 411
 - máquina de, veja Máquina: de Turing

U

- Universo de discurso, 104
- União de conjuntos, 104

V

- Validação de um programa, 40
- Validade, 16
- Valor constante, 65
 - de conjuntos, 105

- simétrica, 117
- Valores verdade, 2
- Variável
 - ligada, 15
 - livre, 15
 - muda, 13, 433
 - ocorrência ligada, 15
 - ocorrência livre de, 15
- Venn, John, 104
 - diagrama de, 104
- Verificação do programa, 39
- Vértice
 - adjacente, 219
 - alcançável, 232
 - de um diagrama de Hasse, 157, 158
 - de um grafo, 217-218
 - filho, 230
 - grau de um, 219
 - ímpar, 280
 - interno, 230
 - isolado, 219
 - pai, 230
 - par, 280
 - profundidade de um, 230
- Vocabulário, 418

W

- Warshall, algoritmo de, 258, 299, 300
- Wff(s), 6
 - predicadas, 16
 - proposicionais, 16
 - válida, 16
- Veja *também* Fórmula: bem formada

X

- XOU(XOR), 12

Z

- $\langle \langle \mathbb{Z} \rangle \rangle$, 99
- $[\langle \langle \mathbb{Z} \rangle \rangle, +]$, 362
- $[\langle \langle \mathbb{Z} \rangle \rangle_{n^*} +_n]$, 366